

Database Internals

Bản dịch song ngữ Anh–Việt · A Deep Dive into How Distributed Data Systems Work

Alex Petrov (nguyên tác)

2026

Mục lục

Preface — Lời nói đầu	7
Audience of This Book — Đối tượng của cuốn sách	8
Why Should I Read This Book? — Vì sao tôi nên đọc cuốn sách này?	10
Scope of This Book — Phạm vi của cuốn sách	11
Structure of This Book — Cấu trúc của cuốn sách	11
Conventions Used in This Book — Các quy ước được dùng trong cuốn sách	13
Using Code Examples — Sử dụng các ví dụ mã	17
O'Reilly Online Learning — O'Reilly Online Learning	18
How to Contact Us — Cách liên hệ với chúng tôi	19
Acknowledgments — Lời cảm ơn	19
PART I Storage Engines — PHẦN I Bộ máy lưu trữ	20
Comparing Databases — So sánh các cơ sở dữ liệu	22
TPC-C Benchmark — Đo điểm chuẩn TPC-C	25
Understanding Trade-Offs — Thấu hiểu các đánh đổi	26
CHAPTER 1 Introduction and Overview — Chương 1: Giới thiệu và Tổng quan	29
DBMS Architecture — Kiến trúc DBMS	31
Memory- Versus Disk-Based DBMS — DBMS dựa trên bộ nhớ so với dựa trên đĩa	34
Durability in Memory-Based Stores — Tính bền vững trong các kho dựa trên bộ nhớ	36
Column- Versus Row-Oriented DBMS — DBMS hướng cột so với hướng hàng	38
Row-Oriented Data Layout — Cách bố trí dữ liệu hướng hàng	39
Column-Oriented Data Layout — Cách bố trí dữ liệu hướng cột	40
Distinctions and Optimizations — Các khác biệt và Tối ưu hóa	41
Wide Column Stores — Kho lưu cột rộng	42
Data Files and Index Files — Tập dữ liệu và Tập chỉ mục	46
Data Files — Tập dữ liệu	47
Index Files — Tập chỉ mục	48
Primary Index as an Indirection — Chỉ mục chính như một lớp gián tiếp	51
Buffering, Immutability, and Ordering — Đệm, Tính bất biến và Sắp thứ tự	53
Summary — Tóm tắt	54
Further Reading	55
CHAPTER 2 B-Tree Basics — Chương 2: Kiến thức cơ bản về B-Tree	56
Binary Search Trees — Cây tìm kiếm nhị phân	57
Tree Balancing — Cân bằng cây	59
Trees for Disk-Based Storage — Cây cho lưu trữ trên đĩa	61
Disk-Based Structures — Cấu trúc dựa trên đĩa	64
Hard Disk Drives — Ổ đĩa cứng	64
Solid State Drives — Ổ đĩa thể rắn	65

On-Disk Structures — Cấu trúc trên đĩa	67
Paged Binary Trees — Cây nhị phân phân trang	68
Ubiquitous B-Trees — B-Tree ở khắp mọi nơi	69
B-Tree Hierarchy — Hệ thống phân cấp B-Tree	72
B — B	73
-Trees — -Tree	73
Separator Keys — Khóa phân tách	74
B-Tree Lookup Complexity — Độ phức tạp tra cứu B-Tree	75
B-Tree Lookup Algorithm — Thuật toán tra cứu B-Tree	76
Counting Keys — Đếm khóa	77
B-Tree Node Splits — Tách nút B-Tree	77
B-Tree Node Merges — Gộp nút B-Tree	80
Summary — Tóm tắt	82
Further Reading	83
CHAPTER 3 File Formats — CHƯƠNG 3 Định dạng tệp (File Formats)	84
Motivation — Động lực	85
Binary Encoding — Mã hóa nhị phân	86
Primitive Types — Các kiểu nguyên thủy	87
Strings and Variable-Size Data — Chuỗi và dữ liệu kích thước thay đổi	89
Bit-Packed Data: Booleans, Enums, and Flags — Dữ liệu đóng gói bit: Boolean, Enum và Flag	90
General Principles — Các nguyên lý chung	92
Page Structure — Cấu trúc trang	93
Slotted Pages — Trang có khe (Slotted Pages)	94
Cell Layout — Bố trí ô (Cell Layout)	97
Variable-Size Data — Dữ liệu kích thước thay đổi	99
Combining Cells into Slotted Pages — Kết hợp các ô thành trang có khe	99
Managing Variable-Size Data — Quản lý dữ liệu kích thước thay đổi	101
Versioning — Phiên bản hóa (Versioning)	104
Checksumming — Tổng kiểm (Checksumming)	105
Summary — Tóm tắt	107
Further Reading	107
CHAPTER 4 Implementing B-Trees — CHƯƠNG 4: Hiện thực B-Tree	109
Page Header — Page Header (Header của trang)	109
Magic Numbers — Magic Numbers (Số ma thuật)	110
Sibling Links — Sibling Links (Liên kết anh em)	110
Rightmost Pointers — Rightmost Pointers (Con trỏ cực phải)	111
Node High Keys — Node High Keys (Khóa biên của nút)	113
Overflow Pages — Overflow Pages (Trang tràn)	114
Binary Search — Binary Search (Tìm kiếm nhị phân)	117
Binary Search with Indirection Pointers — Binary Search with Indirection Pointers (Tìm kiếm nhị phân với con trỏ gián tiếp)	118
Propagating Splits and Merges — Propagating Splits and Merges (Lan truyền tách và gộp) .	119
Breadcrumbs — Breadcrumbs (Dấu vết đường đi)	120
Rebalancing — Rebalancing (Tái cân bằng)	121
Right-Only Appends — Right-Only Appends (Ghi thêm chỉ bên phải)	123
Bulk Loading — Bulk Loading (Nạp hàng loạt)	124
Compression — Compression (Nén)	125
Vacuum and Maintenance — Vacuum and Maintenance (Dọn dẹp và bảo trì)	127

Fragmentation Caused by Updates and Deletes — Fragmentation Caused by Updates and Deletes (Phân mảnh do cập nhật và xóa)	128
Page Defragmentation — Page Defragmentation (Chống phân mảnh trang)	130
Summary — Summary (Tóm tắt)	131
Further Reading	132

CHAPTER 5 Transaction Processing and Recovery — CHƯƠNG 5 Xử lý giao dịch và Phục hồi

hồi	133
Buffer Management — Quản lý vùng đệm	135
Bypassing the Kernel Page Cache — Đi vòng qua Page Cache của nhân	137
Caching Semantics — Ngữ nghĩa của việc cache	138
Cache Eviction — Tổng trang khỏi Cache	139
Locking Pages in Cache — Khóa các trang trong Cache	140
Prefetching and Immediate Eviction — Đọc trước và Tổng ra ngay lập tức	141
Page Replacement — Thay thế trang	141
Recovery — Phục hồi	147
PostgreSQL Versus fsync() — PostgreSQL với fsync()	148
Log Semantics — Ngữ nghĩa của nhật ký	149
Operation Versus Data Log — Nhật ký thao tác với nhật ký dữ liệu	151
Steal and Force Policies — Chính sách Steal và Force	152
ARIES — ARIES	153
Concurrency Control — Điều khiển tương tranh	154
Serializability — Tính khả tuần tự	156
Transaction Isolation — Tính cô lập của giao dịch	157
Read and Write Anomalies — Dị thường đọc và ghi	157
Isolation Levels — Các mức cô lập	159
Optimistic Concurrency Control — Điều khiển tương tranh lạc quan	161
Multiversion Concurrency Control — Điều khiển tương tranh đa phiên bản	163
Pessimistic Concurrency Control — Điều khiển tương tranh bi quan	164
Lock-Based Concurrency Control — Điều khiển tương tranh dựa trên khóa	165
Busy-Wait and Queueing Techniques — Các kỹ thuật Busy-Wait và Xếp hàng	172
Latch Upgrading and Pointer Chasing — Nâng cấp chốt và Đuổi theo con trỏ	175
Summary — Tóm tắt	176
Further Reading	177

CHAPTER 6 B-Tree Variants — Chương 6: Các biến thể của B-Tree

Copy-on-Write — Sao-khi-ghi	180
Implementing Copy-on-Write: LMDB — Hiện thực Sao-khi-ghi: LMDB	181
Abstracting Node Updates — Trừu tượng hóa việc cập nhật nút	182
Lazy B-Trees — B-Tree lười	183
WiredTiger — WiredTiger	183
Lazy-Adaptive Tree — Cây thích nghi-lười	185
FD-Trees — FD-Tree	187
Fractional Cascading — Xếp tầng phân đoạn	188
Logarithmic Runs — Các run logarit	190
Bw-Trees — Bw-Tree	191
Update Chains — Các chuỗi cập nhật	192
Taming Concurrency with Compare-and-Swap — Thuần hóa tương tranh bằng So-sánh-và-Hoán-đổi	193
Structural Modification Operations — Các thao tác sửa đổi cấu trúc	194
Consolidation and Garbage Collection — Hợp nhất và Thu gom rác	196

Cache-Oblivious B-Trees — B-Tree không phụ thuộc bộ nhớ đệm	197
van Emde Boas Layout — Bố cục van Emde Boas	198
Summary — Tóm tắt	200
Further Reading	201

CHAPTER 7 Log-Structured Storage — CHƯƠNG 7: Lưu trữ dạng nhật ký (Log-Structured Storage)	202
LSM Trees — LSM Tree	204
LSM Tree Structure — Cấu trúc của LSM Tree	207
Updates and Deletes — Cập nhật và Xóa	213
LSM Tree Lookups — Tra cứu trong LSM Tree	214
Merge-Iteration — Lặp-gộp (Merge-Iteration)	215
Reconciliation — Hòa giải (Reconciliation)	219
Maintenance in LSM Trees — Bảo trì trong LSM Tree	221
Tombstones and Compaction — Tombstone và Nén-gộp	222
Read, Write, and Space Amplification — Khuếch đại đọc, ghi và không gian	226
RUM Conjecture — Giả thuyết RUM (RUM Conjecture)	228
Implementation Details — Chi tiết hiện thực	229
Sorted String Tables — Sorted String Tables	229
SSTable-Attached Secondary Indexes — Chỉ mục phụ gắn với SSTable (SSTable-Attached Secondary Indexes)	231
Bloom Filters — Bộ lọc Bloom (Bloom Filters)	231
Skiplist — Skiplist	235
Disk Access — Truy cập đĩa	238
Compression — Nén dữ liệu (Compression)	239
Unordered LSM Storage — Lưu trữ LSM không có thứ tự (Unordered LSM Storage)	240
Bitcask — Bitcask	241
WiscKey — WiscKey	243
Concurrency in LSM Trees — Tương tranh trong LSM Tree	245
Log Stacking — Chồng tầng nhật ký (Log Stacking)	247
Flash Translation Layer — Lớp dịch flash (Flash Translation Layer)	247
Filesystem Logging — Ghi nhật ký hệ thống tệp (Filesystem Logging)	250
LLAMA and Mindful Stacking — LLAMA và chồng tầng có ý thức (LLAMA and Mindful Stacking)	252
Open-Channel SSDs — Open-Channel SSD	254
Summary — Tóm tắt	255
Further Reading	255
Part I Conclusion — Kết luận Phần I	256
PART II Distributed Systems — PHẦN II Hệ phân tán	259
Basic definitions — Các định nghĩa cơ bản	260

CHAPTER 8 Introduction and Overview — Chương 8: Giới thiệu và Tổng quan	263
Concurrent Execution — Thực thi tương tranh	263
Concurrent and Parallel — Tương tranh và song song	265
Shared State in a Distributed System — Trạng thái dùng chung trong hệ phân tán	266
Fallacies of Distributed Computing — Những ngộ nhận của tính toán phân tán	267
Processing — Xử lý	270
Clocks and Time — Đồng hồ và thời gian	272
State Consistency — Tính nhất quán của trạng thái	273
Local and Remote Execution — Thực thi cục bộ và từ xa	274
Need to Handle Failures — Cần phải xử lý lỗi	275

Network Partitions and Partial Failures — Phân vùng mạng và lỗi cục bộ	276
Cascading Failures — Lỗi dây chuyền	279
Distributed Systems Abstractions — Các trừu tượng hóa của hệ phân tán	282
Links — Các kênh liên kết	283
Two Generals' Problem — Bài toán hai vị tướng	291
FLP Impossibility — Bất khả thi FLP	293
System Synchrony — Tính đồng bộ của hệ thống	295
Failure Models — Các mô hình lỗi	297
Crash Faults — Lỗi sập	297
Omission Faults — Lỗi bỏ sót	298
Arbitrary Faults — Lỗi tùy tiện	299
Handling Failures — Xử lý lỗi	300
Summary — Tóm tắt	300
Further Reading	301
CHAPTER 9 Failure Detection — Chương 9: Phát hiện lỗi	302
Heartbeats and Pings — Nhịp tim và Ping	304
Timeout-Free Failure Detector — Bộ phát hiện lỗi không cần thời gian chờ	306
Outsourced Heartbeats — Nhịp tim thuê ngoài	307
Phi-Accrual Failure Detector — Bộ phát hiện lỗi Phi-accrual	308
Gossip and Failure Detection — Gossip và phát hiện lỗi	310
Reversing Failure Detection Problem Statement — Đảo ngược cách phát biểu bài toán phát hiện lỗi	311
Summary — Tóm tắt	313
Further Reading	314
CHAPTER 10 Leader Election — Chương 10: Bầu chọn Leader	315
Bully Algorithm — Thuật toán Bully	317
Next-In-Line Failover — Dự phòng Next-In-Line	319
Candidate/Ordinary Optimization — Tối ưu hóa Candidate/Ordinary	321
Invitation Algorithm — Thuật toán Invitation	322
Ring Algorithm — Thuật toán Ring	323
Summary — Tóm tắt	325
Further Reading	327
CHAPTER 11 Replication and Consistency — Chương 11: Nhân bản và Nhất quán	328
Achieving Availability — Đạt được tính sẵn sàng	329
Infamous CAP — CAP khét tiếng	330
Use CAP Carefully — Dùng CAP một cách thận trọng	332
Harvest and Yield — Thu hoạch và Năng suất	334
Shared Memory — Bộ nhớ chia sẻ	335
Ordering — Sắp thứ tự	337
Consistency Models — Các mô hình nhất quán	339
Strict Consistency — Nhất quán nghiêm ngặt	341
Linearizability — Tính tuyến tính hóa	341
Reusable Infrastructure for Linearizability — Hạ tầng tái sử dụng cho Tính tuyến tính hóa	348
Sequential Consistency — Nhất quán tuần tự	349
Causal Consistency — Nhất quán nhân quả	353
Session Models — Các mô hình phiên	360
Eventual Consistency — Nhất quán cuối cùng	363
Tunable Consistency — Nhất quán điều chỉnh được	364

Quorums — Quorum	366
Witness Replicas — Replica nhân chứng	367
Strong Eventual Consistency and CRDTs — Nhất quán cuối cùng mạnh và CRDT	369
Summary — Tóm tắt	372
Further Reading	374
CHAPTER 12 Anti-Entropy and Dissemination — Chương 12: Anti-Entropy và Lan truyền	375
Read Repair — Sửa-khi-đọc (Read Repair)	377
Digest Reads — Đọc Tóm tắt (Digest Reads)	379
Hinted Handoff — Hinted Handoff	381
Merkle Trees — Cây Merkle	382
Bitmap Version Vectors — Vector Phiên bản Bitmap (Bitmap Version Vectors)	383
Gossip Dissemination — Lan truyền Gossip (Gossip Dissemination)	386
Gossip Mechanics — Cơ chế Gossip	387
Overlay Networks — Mạng Phủ (Overlay Networks)	388
Hybrid Gossip — Gossip Lai (Hybrid Gossip)	390
Partial Views — Góc nhìn Cục bộ (Partial Views)	392
Summary — Tóm tắt	394
Further Reading	395
CHAPTER 13 Distributed Transactions — Chương 13: Giao dịch phân tán	396
Making Operations Appear Atomic — Làm cho các thao tác có vẻ nguyên tử	398
Two-Phase Commit — Commit hai pha	399
Cohort Failures in 2PC — Lỗi bên tham gia trong 2PC	402
Coordinator Failures in 2PC — Lỗi bên điều phối trong 2PC	403
Three-Phase Commit — Commit ba pha	405
Coordinator Failures in 3PC — Lỗi bên điều phối trong 3PC	407
Distributed Transactions with Calvin — Giao dịch phân tán với Calvin	408
Distributed Transactions with Spanner — Giao dịch phân tán với Spanner	411
Database Partitioning — Phân vùng cơ sở dữ liệu	415
Consistent Hashing — Băm nhất quán	416
Distributed Transactions with Percolator — Giao dịch phân tán với Percolator	417
Coordination Avoidance — Né tránh phối hợp	421
Summary — Tóm tắt	424
Further Reading	425
CHAPTER 14 Consensus — CHƯƠNG 14 Đồng thuận	426
Broadcast — Quảng bá (Broadcast)	428
Atomic Broadcast — Quảng bá nguyên tử (Atomic Broadcast)	429
Virtual Synchrony — Đồng bộ ảo (Virtual Synchrony)	430
Zookeeper Atomic Broadcast (ZAB) — Zookeeper Atomic Broadcast (ZAB)	432
Paxos — Paxos	435
Paxos Algorithm — Thuật toán Paxos	436
Quorums in Paxos — Quorum trong Paxos	439
Failure Scenarios — Các kịch bản lỗi	441
Multi-Paxos — Multi-Paxos	444
Fast Paxos — Fast Paxos	446
Egalitarian Paxos — Egalitarian Paxos	448
Flexible Paxos — Flexible Paxos	453
Generalized Solution to Consensus — Lời giải tổng quát cho Đồng thuận	455
Raft — Raft	459

Leader Role in Raft — Vai trò Leader trong Raft	462
Failure Scenarios — Các kịch bản lỗi	464
Byzantine Consensus — Đồng thuận Byzantine	466
PBFT Algorithm — Thuật toán PBFT	468
Recovery and Checkpointing — Phục hồi và Tạo điểm kiểm tra	471
Summary — Tóm tắt	472
Further Reading	473
Part II Conclusion — Kết luận Phần II	474
Further Reading	476
APPENDIX A Bibliography	477

Preface — Lời nói đầu

Distributed database systems are an integral part of most businesses and the vast majority of software applications. These applications provide logic and a user interface, while database systems take care of data integrity, consistency, and redundancy.

Các hệ cơ sở dữ liệu phân tán là một phần không thể tách rời của hầu hết doanh nghiệp và đại đa số ứng dụng phần mềm. Những ứng dụng này cung cấp logic và giao diện người dùng, trong khi các hệ cơ sở dữ liệu lo việc bảo đảm tính toàn vẹn, nhất quán và dư thừa của dữ liệu.

Back in 2000, if you were to choose a database, you would have just a few options, and most of them would be within the realm of relational databases, so differences between them would be relatively small. Of course, this does not mean that all databases were completely the same, but their functionality and use cases were very similar.

Quay lại năm 2000, nếu bạn phải chọn một cơ sở dữ liệu, bạn chỉ có vài lựa chọn, và phần lớn trong số đó đều nằm trong phạm vi cơ sở dữ liệu quan hệ, nên khác biệt giữa chúng tương đối nhỏ. Tất nhiên, điều này không có nghĩa là mọi cơ sở dữ liệu đều hoàn toàn giống nhau, nhưng chức năng và trường hợp sử dụng của chúng rất tương đồng.

Some of these databases have focused on horizontal scaling (scaling out)—improving performance and increasing capacity by running multiple database instances acting as a single logical unit: Gamma Database Machine Project, Teradata, Greenplum, Parallel DB2, and many others. Today, horizontal scaling remains one of the most important properties that customers expect from databases. This can be explained by the rising popularity of cloud-based services. It is often easier to spin up a new instance and add it to the cluster than scaling vertically (scaling up) by moving the database to a larger, more powerful machine. Migrations can be long and painful, potentially incurring downtime.

Một số trong số các cơ sở dữ liệu này tập trung vào việc mở rộng theo chiều ngang (scaling out) — cải thiện hiệu năng và tăng dung lượng bằng cách chạy nhiều phiên bản (instance) cơ sở dữ liệu hoạt động như một đơn vị logic duy nhất: Gamma Database Machine Project, Teradata, Greenplum, Parallel DB2 và nhiều hệ khác. Ngày nay, mở rộng theo chiều ngang vẫn là một trong những thuộc tính quan trọng nhất mà khách hàng kỳ vọng ở cơ sở dữ liệu. Điều này có thể được lý giải bởi sự phổ biến ngày càng tăng của các dịch vụ trên nền tảng đám mây. Thường thì việc khởi tạo một phiên bản mới và thêm nó vào cụm (cluster) sẽ dễ hơn so với mở rộng theo chiều dọc (scaling up) bằng cách chuyển cơ sở dữ liệu sang một máy lớn hơn, mạnh hơn. Các cuộc di trú (migration) có thể dài và gian nan, tiềm ẩn nguy cơ gây gián đoạn dịch vụ (downtime).

Around 2010, a new class of eventually consistent databases started appearing, and terms such as NoSQL, and later, big data grew in popularity. Over the last 15 years, the open source community, large internet companies, and database vendors have created so many databases and tools that it's easy to get lost trying to understand use cases, details, and specifics.

Khoảng năm 2010, một lớp cơ sở dữ liệu nhất quán cuối cùng (eventually consistent) mới bắt đầu xuất hiện, và các thuật ngữ như NoSQL, và sau đó là big data, trở nên phổ biến. Trong vòng 15 năm qua, cộng đồng mã nguồn mở, các công ty internet lớn và các nhà cung cấp cơ sở dữ liệu đã tạo ra quá nhiều cơ sở dữ liệu và công cụ đến mức người ta dễ bị lạc lối khi cố gắng hiểu các trường hợp sử dụng, chi tiết và đặc thù của chúng.

The Dynamo paper [DECANDIA07], published by the team at Amazon in 2007, had so much impact on the database community that within a short period it inspired many variants and implementations. The most prominent of them were Apache Cassandra, created at Facebook; Project Voldemort, created at LinkedIn; and Riak, created by former Akamai engineers.

Bài báo Dynamo [DECANDIA07], được nhóm tại Amazon công bố năm 2007, có ảnh hưởng lớn đến cộng đồng cơ sở dữ liệu đến mức chỉ trong một thời gian ngắn nó đã truyền cảm hứng cho nhiều biến thể và hiện thực. Nổi bật nhất trong số đó là Apache Cassandra, tạo ra tại Facebook; Project Voldemort, tạo ra tại LinkedIn; và Riak, tạo ra bởi các kỹ sư từng làm việc tại Akamai.

xiii Today, the field is changing again: after the time of key-value stores, NoSQL, and **eventual consistency**, we have started seeing more scalable and performant databases, able to execute complex queries with stronger consistency guarantees.

xiii Ngày nay, lĩnh vực này lại đang thay đổi: sau thời kỳ của các kho lưu khóa-giá trị (key-value store), NoSQL và nhất quán cuối cùng, chúng ta đã bắt đầu thấy nhiều cơ sở dữ liệu có khả năng mở rộng và hiệu năng cao hơn, có thể thực thi các truy vấn phức tạp với những bảo đảm nhất quán mạnh hơn.

Audience of This Book — Đối tượng của cuốn sách

In conversations at technical conferences, I often hear the same question: “How can I learn more about database internals? I don’t even know where to start.” Most of the books on database systems do not go into details of **storage engine** implementation, and cover the access methods, such as B-Trees, on a rather high level. There are very few books that cover more recent concepts, such as different **B-Tree** variants and **log-structured storage**, so I usually recommend reading papers.

Trong các cuộc trò chuyện tại những hội nghị kỹ thuật, tôi thường nghe cùng một câu hỏi: “Làm sao để tôi tìm hiểu thêm về nội tại của cơ sở dữ liệu? Tôi thậm chí không biết bắt đầu từ đâu.” Hầu hết các cuốn sách về hệ cơ sở dữ liệu không đi sâu vào chi tiết hiện thực của bộ máy lưu trữ (storage engine), và chỉ trình bày các phương pháp truy cập, chẳng hạn như B-Tree, ở mức khá tổng quát. Có rất ít cuốn sách đề cập tới những khái niệm gần đây hơn, như các biến thể B-Tree khác nhau và lưu trữ dạng nhật ký (log-structured), nên tôi thường khuyên đọc các bài báo (paper).

Everyone who reads papers knows that it's not that easy: you often lack context, the wording might be ambiguous, there's little or no connection between papers, and they're hard to find. This book contains concise summaries of important database systems concepts and can serve as a guide for those who'd like to dig in deeper, or as a cheat sheet for those already familiar with these concepts.

Ai từng đọc các bài báo đều biết rằng việc đó không hề dễ: bạn thường thiếu ngữ cảnh, cách diễn đạt có thể mơ hồ, mối liên hệ giữa các bài báo ít hoặc không có, và chúng cũng

khó tìm. Cuốn sách này chứa những bản tóm lược súc tích về các khái niệm quan trọng của hệ cơ sở dữ liệu và có thể đóng vai trò như một cẩm nang dẫn đường cho những ai muốn đào sâu hơn, hoặc như một bản ghi nhớ nhanh (cheat sheet) cho những ai đã quen thuộc với các khái niệm này.

Not everyone wants to become a database developer, but this book will help people who build software that uses database systems: software developers, reliability engineers, architects, and engineering managers.

Không phải ai cũng muốn trở thành nhà phát triển cơ sở dữ liệu, nhưng cuốn sách này sẽ giúp ích cho những người xây dựng phần mềm có sử dụng hệ cơ sở dữ liệu: nhà phát triển phần mềm, kỹ sư độ tin cậy (reliability engineer), kiến trúc sư và quản lý kỹ thuật.

If your company depends on any infrastructure component, be it a database, a messaging queue, a container platform, or a task scheduler, you have to **read** the project change-logs and mailing lists to stay in touch with the community and be up-to-date with the most recent happenings in the project. Understanding terminology and knowing what's inside will enable you to yield more information from these sources and use your tools more productively to troubleshoot, identify, and avoid potential risks and bottlenecks. Having an overview and a general understanding of how database systems work will help in case something goes wrong. Using this knowledge, you'll be able to form a hypothesis, validate it, find the root cause, and present it to other project maintainers.

Nếu công ty của bạn phụ thuộc vào bất kỳ thành phần hạ tầng nào, dù là cơ sở dữ liệu, hàng đợi thông điệp (messaging queue), nền tảng container hay bộ lập lịch tác vụ (task scheduler), bạn phải đọc các nhật ký thay đổi (change-log) và danh sách thư (mailing list) của dự án để giữ liên hệ với cộng đồng và cập nhật những diễn biến mới nhất của dự án. Hiểu được thuật ngữ và biết bên trong có gì sẽ cho phép bạn khai thác được nhiều thông tin hơn từ những nguồn này và sử dụng công cụ của mình hiệu quả hơn để khắc phục sự cố, nhận diện và tránh các rủi ro cũng như điểm nghẽn tiềm ẩn. Có cái nhìn tổng quan và hiểu biết chung về cách các hệ cơ sở dữ liệu hoạt động sẽ giúp ích khi có điều gì đó trục trặc. Sử dụng kiến thức này, bạn sẽ có thể đặt ra một giả thuyết, kiểm chứng nó, tìm ra nguyên nhân gốc rễ và trình bày nó với những người bảo trì dự án khác.

This book is also for curious minds: for the people who like learning things without immediate necessity, those who spend their free time hacking on something fun, creating compilers, writing homegrown operating systems, text editors, computer games, learning programming languages, and absorbing new information.

Cuốn sách này cũng dành cho những đầu óc tò mò: cho những người thích học hỏi mà không cần lý do cấp thiết tức thì, những người dành thời gian rảnh để vọc vạch một thứ gì đó thú vị, tạo ra trình biên dịch (compiler), viết hệ điều hành tự chế, trình soạn thảo văn bản, trò chơi máy tính, học các ngôn ngữ lập trình và hấp thụ thông tin mới.

The reader is assumed to have some experience with developing backend systems and working with database systems as a user. Having some prior knowledge of different data structures will help to digest material faster.

Người đọc được giả định là đã có một ít kinh nghiệm phát triển các hệ thống backend và làm việc với hệ cơ sở dữ liệu ở vai trò người dùng. Có sẵn một số kiến thức về các cấu trúc dữ liệu khác nhau sẽ giúp tiêu hóa tài liệu nhanh hơn.

xiv | Preface

xiv | Lời nói đầu

Why Should I Read This Book? — Vì sao tôi nên đọc cuốn sách này?

We often hear people describing database systems in terms of the concepts and algorithms they implement: “This database uses gossip for membership propagation” (see Chapter 12), “They have implemented Dynamo,” or “This is just like what they’ve described in the **Spanner** paper” (see Chapter 13). Or, if you’re discussing the algorithms and data structures, you can hear something like “**ZAB** and **Raft** have a lot in common” (see Chapter 14), “Bw-Trees are like the B-Trees implemented on top of log structured storage” (see Chapter 6), or “They are using sibling pointers like in B **link** - Trees” (see Chapter 5).

Chúng ta thường nghe người ta mô tả các hệ cơ sở dữ liệu qua những khái niệm và thuật toán mà chúng hiện thực: “Cơ sở dữ liệu này dùng gossip để lan truyền thông tin thành viên (membership)” (xem Chương 12), “Họ đã hiện thực Dynamo,” hay “Cái này giống hệt những gì họ mô tả trong bài báo Spanner” (xem Chương 13). Hoặc, nếu bạn đang bàn về thuật toán và cấu trúc dữ liệu, bạn có thể nghe những câu như “ZAB và Raft có rất nhiều điểm chung” (xem Chương 14), “Bw-Tree giống như B-Tree được hiện thực trên nền lưu trữ dạng nhật ký” (xem Chương 6), hay “Họ dùng con trỏ anh em (sibling pointer) như trong Blink-Tree” (xem Chương 5).

We need abstractions to discuss complex concepts, and we can’t have a discussion about terminology every time we start a conversation. Having shortcuts in the form of common language helps us to move our attention to other, higher-level problems.

Chúng ta cần các trừu tượng hóa để bàn về những khái niệm phức tạp, và chúng ta không thể tranh luận về thuật ngữ mỗi khi bắt đầu một cuộc trò chuyện. Có những lối tắt dưới dạng ngôn ngữ chung giúp chúng ta dịch chuyển sự chú ý sang các vấn đề khác ở mức cao hơn.

One of the advantages of learning the fundamental concepts, proofs, and algorithms is that they never grow old. Of course, there will always be new ones, but new algorithms are often created after finding a flaw or room for improvement in a classical one. Knowing the history helps to understand differences and motivation better.

Một trong những lợi thế của việc học các khái niệm, chứng minh và thuật toán nền tảng là chúng không bao giờ lỗi thời. Tất nhiên, sẽ luôn có những cái mới, nhưng các thuật toán mới thường được tạo ra sau khi tìm thấy một khiếm khuyết hoặc khả năng cải tiến trong một thuật toán kinh điển. Biết được lịch sử giúp hiểu rõ hơn các khác biệt và động lực phía sau.

Learning about these things is inspiring. You see the variety of algorithms, see how our industry was solving one problem after the other, and get to appreciate that work. At the same time, learning is rewarding: you can almost feel how multiple puzzle pieces move together in your mind to form a full picture that you will always be able to share with others.

Học về những điều này thật truyền cảm hứng. Bạn thấy sự đa dạng của các thuật toán, thấy ngành của chúng ta đã giải quyết hết vấn đề này đến vấn đề khác như thế nào, và biết trân trọng công sức ấy. Đồng thời, việc học cũng thật xứng đáng: bạn gần như có thể cảm nhận được nhiều mảnh ghép của câu đố di chuyển lại với nhau trong đầu để tạo thành một bức tranh hoàn chỉnh mà bạn sẽ luôn có thể chia sẻ với người khác.

Scope of This Book — Phạm vi của cuốn sách

This is neither a book about relational database management systems nor about NoSQL ones, but about the algorithms and concepts used in all kinds of database systems, with a focus on a storage engine and the components responsible for distribution .

Đây không phải là một cuốn sách về các hệ quản trị cơ sở dữ liệu quan hệ, cũng không phải về các hệ NoSQL, mà là về những thuật toán và khái niệm được dùng trong mọi loại hệ cơ sở dữ liệu, với trọng tâm là bộ máy lưu trữ và các thành phần chịu trách nhiệm phân tán (distribution).

Some concepts, such as query planning, query optimization, scheduling, the relational model, and a few others, are already covered in several great textbooks on database systems. Some of these concepts are usually described from the user's perspective, but this book concentrates on the internals. You can find some pointers to useful literature in the Part II Conclusion and in the chapter summaries. In these books you're likely to find answers to many database-related questions you might have.

Một số khái niệm, như lập kế hoạch truy vấn (query planning), tối ưu truy vấn (query optimization), lập lịch (scheduling), mô hình quan hệ và một vài khái niệm khác, đã được trình bày trong nhiều giáo trình xuất sắc về hệ cơ sở dữ liệu. Một số khái niệm trong đó thường được mô tả từ góc nhìn của người dùng, nhưng cuốn sách này tập trung vào phần nội tại (internals). Bạn có thể tìm thấy vài chỉ dẫn tới tài liệu hữu ích trong phần Kết luận của Phần II và trong các bản tóm tắt chương. Trong những cuốn sách đó, bạn nhiều khả năng sẽ tìm được lời giải đáp cho nhiều câu hỏi liên quan đến cơ sở dữ liệu mà bạn có thể thắc mắc.

Query languages aren't discussed, since there's no single common language among the database systems mentioned in this book.

Các ngôn ngữ truy vấn (query language) không được bàn tới, vì không có một ngôn ngữ chung duy nhất giữa các hệ cơ sở dữ liệu được nhắc đến trong cuốn sách này.

To collect material for this book, I studied over 15 books, more than 300 papers, countless blog posts, source code, and the documentation for several open source

Để thu thập tư liệu cho cuốn sách này, tôi đã nghiên cứu hơn 15 cuốn sách, hơn 300 bài báo, vô số bài viết blog, mã nguồn và tài liệu của một số cơ sở dữ liệu mã nguồn mở.

Preface | xv databases. The rule of thumb for whether or not to include a particular concept in the book was the question: "Do the people in the database industry and research circles talk about this concept?" If the answer was "yes," I added the concept to the long list of things to discuss.

Lời nói đầu | xv Nguyên tắc chung để quyết định có đưa một khái niệm cụ thể vào sách hay không là câu hỏi: "Liệu những người trong ngành cơ sở dữ liệu và giới nghiên cứu có bàn về khái niệm này không?" Nếu câu trả lời là "có," tôi thêm khái niệm đó vào danh sách dài những thứ cần thảo luận.

Structure of This Book — Cấu trúc của cuốn sách

There are some examples of extensible databases with pluggable components (such as [SCHWARZ86]), but they are rather rare. At the same time, there are plenty of examples where databases use pluggable storage. Similarly, we rarely hear database vendors talking about query execution, while they are very eager to discuss the ways their databases preserve consistency.

Có một số ví dụ về cơ sở dữ liệu có thể mở rộng với các thành phần cắm-rút (pluggable component) (chẳng hạn [SCHWARZ86]), nhưng chúng khá hiếm. Đồng thời, có rất nhiều ví dụ trong đó cơ sở dữ liệu dùng lưu trữ cắm-rút (pluggable storage). Tương tự, chúng ta hiếm khi nghe các nhà cung cấp cơ sở dữ liệu nói về thực thi truy vấn, trong khi họ lại rất hào hứng bàn về những cách mà cơ sở dữ liệu của họ bảo toàn tính nhất quán.

The most significant distinctions between database systems are concentrated around two aspects: how they store and how they distribute the data. (Other subsystems can at times also be of importance, but are not covered here.) The book is arranged into parts that discuss the subsystems and components responsible for storage (Part I) and distribution (Part II).

Những khác biệt đáng kể nhất giữa các hệ cơ sở dữ liệu tập trung quanh hai khía cạnh: cách chúng lưu trữ và cách chúng phân tán dữ liệu. (Đôi khi các phân hệ khác cũng có thể quan trọng, nhưng không được đề cập ở đây.) Cuốn sách được sắp xếp thành các phần bàn về những phân hệ và thành phần chịu trách nhiệm cho lưu trữ (Phần I) và phân tán (Phần II).

Part I discusses **node**-local processes and focuses on the storage engine, the central component of the database system and one of the most significant distinctive factors. First, we start with the architecture of a database management system and present several ways to classify database systems based on the primary storage medium and layout.

Phần I bàn về các tiến trình cục bộ trên nút (node-local) và tập trung vào bộ máy lưu trữ, thành phần trung tâm của hệ cơ sở dữ liệu và là một trong những yếu tố phân biệt quan trọng nhất. Trước tiên, chúng ta bắt đầu với kiến trúc của một hệ quản trị cơ sở dữ liệu và trình bày một số cách phân loại các hệ cơ sở dữ liệu dựa trên phương tiện lưu trữ chính và bố cục (layout).

We continue with storage structures and try to understand how disk-based structures are different from in-memory ones, introduce B-Trees, and cover algorithms for efficiently maintaining B-Tree structures on disk, including serialization, **page** layout, and on-disk representations. Later, we discuss multiple variants to illustrate the power of this concept and the diversity of data structures influenced and inspired by B- Trees.

Chúng ta tiếp tục với các cấu trúc lưu trữ và cố gắng hiểu các cấu trúc trên đĩa khác với các cấu trúc trong bộ nhớ (in-memory) như thế nào, giới thiệu B-Tree, và trình bày các thuật toán để duy trì hiệu quả các cấu trúc B-Tree trên đĩa, bao gồm việc tuần tự hóa (serialization), bố cục trang (page layout) và biểu diễn trên đĩa. Sau đó, chúng ta bàn về nhiều biến thể để minh họa sức mạnh của khái niệm này và sự đa dạng của các cấu trúc dữ liệu chịu ảnh hưởng và cảm hứng từ B-Tree.

Last, we discuss several variants of log-structured storage, commonly used for implementing file and storage systems, motivation, and reasons to use them.

Cuối cùng, chúng ta bàn về một số biến thể của lưu trữ dạng nhật ký, thường được dùng để hiện thực các hệ tệp và hệ lưu trữ, cùng động lực và lý do để sử dụng chúng.

Part II is about how to organize multiple nodes into a database cluster. We start with the importance of understanding the theoretical concepts for building fault-tolerant distributed systems, how distributed systems are different from single-node applications, and which problems, constraints, and complications we face in a distributed environment.

Phần II nói về cách tổ chức nhiều nút thành một cụm cơ sở dữ liệu. Chúng ta bắt đầu với tầm quan trọng của việc hiểu các khái niệm lý thuyết để xây dựng những hệ phân tán dung lỗi (fault-tolerant), hệ phân tán khác với ứng dụng đơn nút (single-node) như thế

nào, và những vấn đề, ràng buộc cùng phức tạp mà chúng ta phải đối mặt trong một môi trường phân tán.

After that, we dive deep into distributed algorithms. Here, we start with algorithms for failure detection, helping to improve performance and stability by noticing and reporting failures and avoiding the failed nodes. Since many algorithms discussed later in the book rely on understanding the concept of leadership, we introduce several algorithms for **leader election** and discuss their suitability.

Sau đó, chúng ta đào sâu vào các thuật toán phân tán. Ở đây, chúng ta bắt đầu với các thuật toán phát hiện lỗi (failure detection), giúp cải thiện hiệu năng và độ ổn định bằng cách nhận biết và báo cáo các lỗi cũng như tránh những nút bị lỗi. Vì nhiều thuật toán được bàn ở phần sau của cuốn sách dựa trên việc hiểu khái niệm lãnh đạo (leadership), chúng ta giới thiệu một số thuật toán bầu chọn leader (leader election) và bàn về tính phù hợp của chúng.

xvi | Preface As one of the most difficult things in distributed systems is achieving data consistency, we discuss concepts of replication, followed by consistency models, possible divergence between replicas, and eventual consistency. Since eventually consistent systems sometimes rely on **anti-entropy** for convergence and gossip for data **dissemination**, we discuss several anti-entropy and gossip approaches. Finally, we discuss logical consistency in the context of database transactions, and finish with **consensus** algorithms.

xvi | Lời nói đầu Vì một trong những điều khó nhất trong các hệ phân tán là đạt được tính nhất quán dữ liệu, chúng ta bàn về các khái niệm sao chép (replication), tiếp theo là các mô hình nhất quán, sự phân kỳ có thể xảy ra giữa các bản sao (replica), và nhất quán cuối cùng. Vì các hệ nhất quán cuối cùng đôi khi dựa vào anti-entropy để hội tụ và gossip để lan truyền (dissemination) dữ liệu, chúng ta bàn về một số phương pháp anti-entropy và gossip. Cuối cùng, chúng ta bàn về tính nhất quán logic trong ngữ cảnh giao dịch (transaction) cơ sở dữ liệu, và kết thúc với các thuật toán đồng thuận (consensus).

It would've been impossible to write this book without all the research and publications. You will find many references to papers and publications in the text, in square brackets with monospace font; for example, [DECANDIA07]. You can use these references to learn more about related concepts in more detail.

Sẽ không thể viết được cuốn sách này nếu không có tất cả những nghiên cứu và công bố. Bạn sẽ tìm thấy nhiều tham chiếu tới các bài báo và công bố trong văn bản, đặt trong dấu ngoặc vuông với phông chữ đơn cách (monospace); ví dụ, [DECANDIA07]. Bạn có thể dùng những tham chiếu này để tìm hiểu chi tiết hơn về các khái niệm liên quan.

After each chapter, you will find a summary section that contains material for further study, related to the content of the chapter.

Sau mỗi chương, bạn sẽ tìm thấy một mục tóm tắt chứa tài liệu cho việc nghiên cứu thêm, liên quan đến nội dung của chương.

Conventions Used in This Book — Các quy ước được dùng trong cuốn sách

The following typographical conventions are used in this book:

Cuốn sách này sử dụng các quy ước về kiểu chữ sau đây:

Italic Indicates new terms, URLs, email addresses, filenames, and file extensions.

Chữ nghiêng (Italic) Biểu thị các thuật ngữ mới, URL, địa chỉ email, tên tệp và phần mở rộng tệp.

Constant width

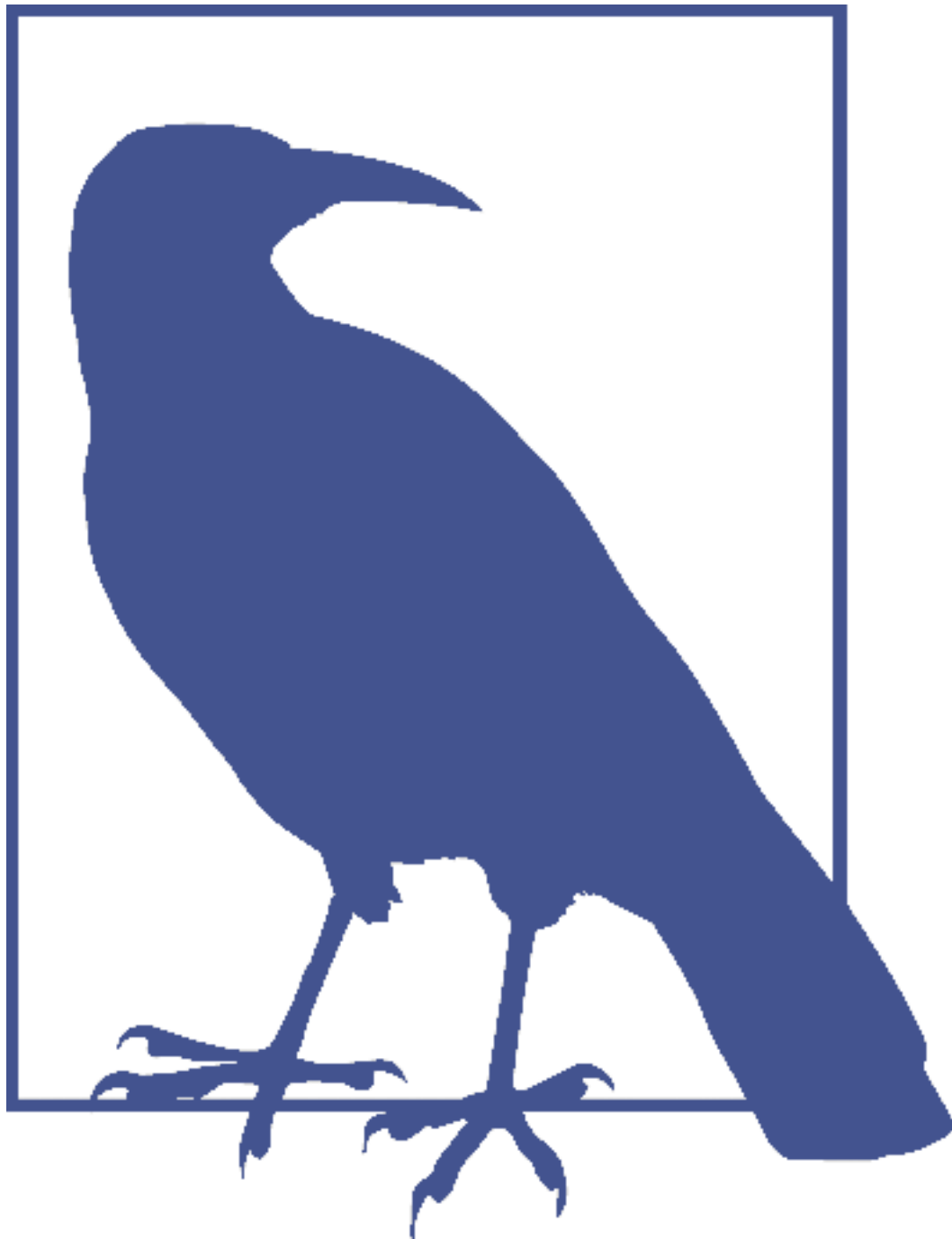
Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Được dùng cho các đoạn liệt kê chương trình, cũng như bên trong các đoạn văn để chỉ tới các thành phần chương trình như tên biến hay tên hàm, cơ sở dữ liệu, kiểu dữ liệu, biến môi trường, câu lệnh và từ khóa.



This element signifies a tip or suggestion.

Phần tử này biểu thị một mẹo hoặc gợi ý.



This element signifies a general note.

Phần tử này biểu thị một ghi chú chung.



This element indicates a warning or caution.

Phần tử này biểu thị một cảnh báo hoặc lưu ý thận trọng.

Preface | xvii

Lời nói đầu | xvii

Using Code Examples — Sử dụng các ví dụ mã

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing a CD-ROM of examples from O'Reilly books does require permission. Answering a question by citing this

book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

Cuốn sách này có mặt để giúp bạn hoàn thành công việc. Nhìn chung, nếu mã ví dụ được cung cấp kèm cuốn sách này, bạn có thể dùng nó trong các chương trình và tài liệu của mình. Bạn không cần liên hệ với chúng tôi để xin phép trừ khi bạn tái tạo một phần đáng kể của mã. Ví dụ, viết một chương trình sử dụng vài đoạn mã từ cuốn sách này không cần xin phép. Việc bán hay phân phối một đĩa CD-ROM chứa các ví dụ từ sách của O'Reilly thì cần xin phép. Trả lời một câu hỏi bằng cách trích dẫn cuốn sách này và dẫn lại mã ví dụ không cần xin phép. Việc đưa một lượng lớn mã ví dụ từ cuốn sách này vào tài liệu của sản phẩm của bạn thì cần xin phép.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “ Database Internals by Alex Petrov (O'Reilly). Copyright 2019 Oleksandr Petrov, 978-1-492-04034-7.”

Chúng tôi trân trọng, nhưng không yêu cầu, việc ghi nguồn (attribution). Một dòng ghi nguồn thường gồm tiêu đề, tác giả, nhà xuất bản và ISBN. Ví dụ: “Database Internals của Alex Petrov (O'Reilly). Bản quyền 2019 Oleksandr Petrov, 978-1-492-04034-7.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com .

Nếu bạn cảm thấy việc sử dụng các ví dụ mã của mình nằm ngoài phạm vi sử dụng hợp lý (fair use) hoặc sự cho phép nêu trên, hãy thoải mái liên hệ với chúng tôi tại permissions@oreilly.com.

O'Reilly Online Learning — O'Reilly Online Learning



For almost 40 years, O'Reilly Media has provided technology and business training, knowledge, and insight to help companies succeed.

Trong gần 40 năm, O'Reilly Media đã cung cấp đào tạo về công nghệ và kinh doanh, kiến thức và sự thấu hiểu nhằm giúp các công ty thành công.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, conferences, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, please visit <http://oreilly.com> .

Mạng lưới độc đáo gồm các chuyên gia và nhà đổi mới của chúng tôi chia sẻ kiến thức và chuyên môn của họ thông qua sách, bài viết, hội nghị và nền tảng học trực tuyến của chúng tôi. Nền tảng học trực tuyến của O'Reilly cho bạn quyền truy cập theo yêu cầu tới các khóa đào tạo trực tiếp, các lộ trình học chuyên sâu, các môi trường lập trình tương tác, cùng một bộ sưu tập khổng lồ gồm văn bản và video từ O'Reilly và hơn 200 nhà xuất bản khác. Để biết thêm thông tin, vui lòng truy cập <http://oreilly.com>.

How to Contact Us — Cách liên hệ với chúng tôi

Please address comments and questions concerning this book to the publisher:

Vui lòng gửi các nhận xét và câu hỏi liên quan đến cuốn sách này tới nhà xuất bản:

O'Reilly Media, Inc. 1005 Gravenstein Highway North Sebastopol, CA 95472 800-998-9938 (in the United States or Canada) 707-829-0515 (international or local) 707-829-0104 (fax)

O'Reilly Media, Inc. 1005 Gravenstein Highway North Sebastopol, CA 95472 800-998-9938 (tại Hoa Kỳ hoặc Canada) 707-829-0515 (quốc tế hoặc nội hạt) 707-829-0104 (fax)

xviii | Preface We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <http://bit.ly/database-internals>.

xviii | Lời nói đầu Chúng tôi có một trang web cho cuốn sách này, nơi chúng tôi liệt kê các đính chính (errata), ví dụ và bất kỳ thông tin bổ sung nào. Bạn có thể truy cập trang này tại <http://bit.ly/database-internals>.

To comment or ask technical questions about this book, please send an email to bookquestions@oreilly.com.

Để bình luận hoặc đặt câu hỏi kỹ thuật về cuốn sách này, vui lòng gửi email tới bookquestions@oreilly.com.

For more information about our books, courses, conferences, and news, see our website at <http://www.oreilly.com>.

Để biết thêm thông tin về sách, khóa học, hội nghị và tin tức của chúng tôi, hãy xem trang web của chúng tôi tại <http://www.oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Tìm chúng tôi trên Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Theo dõi chúng tôi trên Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://www.youtube.com/oreillymedia>

Xem chúng tôi trên YouTube: <http://www.youtube.com/oreillymedia>

Acknowledgments — Lời cảm ơn

This book wouldn't have been possible without the hundreds of people who have worked hard on research papers and books, which have been a source of ideas, inspiration, and served as references for this book.

Cuốn sách này hẳn đã không thể thành hình nếu không có hàng trăm con người đã miệt mài làm việc trên các bài báo nghiên cứu và sách vở, vốn là nguồn ý tưởng, cảm hứng và đóng vai trò là tài liệu tham chiếu cho cuốn sách này.

I'd like to say thank you to all the people who reviewed manuscripts and provided feedback, making sure that the material in this book is correct and the wording is precise: Dmitry Alimov, Peter Alvaro, Carlos Baquero, Jason Brown, Blake Eggleston, Marcus Eriksson, Francisco Fernández Castaño, Heidi Howard, Vaidehi Joshi, Maximilian Karasz, Stas Kelvich, Michael Klishin, Predrag Knežević, Joel Knighton, Eugene Lazin, Nate McCall, Christopher Meiklejohn, Tyler Neely, Maxim Neverov, Marina Petrova, Stefan Podkowinski, Edward Ribiero, Denis Rytsov, Kir Shatrov, Alex Sorokoumov, Massimiliano Tomassi, and Ariel Weisberg.

Tôi muốn nói lời cảm ơn tới tất cả những người đã đọc soát bản thảo và đưa ra phản hồi, bảo đảm rằng tư liệu trong cuốn sách này là chính xác và câu chữ thì chuẩn xác: Dmitry Alimov, Peter Alvaro, Carlos Baquero, Jason Brown, Blake Eggleston, Marcus Eriksson, Francisco Fernández Castaño, Heidi Howard, Vaidehi Joshi, Maximilian Karasz, Stas Kelvich, Michael Klishin, Predrag Knežević, Joel Knighton, Eugene Lazin, Nate McCall, Christopher Meiklejohn, Tyler Neely, Maxim Neverov, Marina Petrova, Stefan Podkowinski, Edward Ribiero, Denis Rytsov, Kir Shatrov, Alex Sorokoumov, Massimiliano Tomassi, và Ariel Weisberg.

Of course, this book wouldn't have been possible without support from my family: my wife Marina and my daughter Alexandra, who have supported me on every step on the way.

Tất nhiên, cuốn sách này hẳn đã không thể thành hình nếu không có sự ủng hộ từ gia đình tôi: vợ tôi Marina và con gái tôi Alexandra, những người đã đồng hành cùng tôi trên mỗi bước đường.

Preface | xix

Lời nói đầu | xix

PART I Storage Engines — PHẦN I Bộ máy lưu trữ

The primary job of any database management system is reliably storing data and making it available for users. We use databases as a primary source of data, helping us to share it between the different parts of our applications. Instead of finding a way to store and retrieve information and inventing a new way to organize data every time we create a new app, we use databases. This way we can concentrate on application logic instead of infrastructure.

Nhiệm vụ chính của bất kỳ hệ quản trị cơ sở dữ liệu (DBMS) nào là lưu trữ dữ liệu một cách đáng tin cậy và làm cho dữ liệu sẵn sàng cho người dùng. Chúng ta sử dụng cơ sở dữ liệu (database) như nguồn dữ liệu chính, giúp chia sẻ dữ liệu giữa các phần khác nhau trong ứng dụng. Thay vì phải tìm cách lưu trữ và truy xuất thông tin cũng như phát minh ra một cách tổ chức dữ liệu mới mỗi lần xây dựng một ứng dụng mới, chúng ta dùng cơ sở dữ liệu. Nhờ vậy, chúng ta có thể tập trung vào logic ứng dụng thay vì hạ tầng.

Since the **term** database management system (**DBMS**) is quite bulky, throughout this book we use more compact terms, database system and database, to refer to the same concept.

Vì thuật ngữ hệ quản trị cơ sở dữ liệu (DBMS) khá cồng kềnh, xuyên suốt cuốn sách này chúng tôi dùng các thuật ngữ gọn hơn là hệ cơ sở dữ liệu (database system) và cơ sở dữ liệu (database) để chỉ cùng một khái niệm.

Databases are modular systems and consist of multiple parts: a transport layer accepting requests, a **query processor** determining the most efficient way to run queries, an **execution engine** carrying out the operations, and a **storage engine** (see “DBMS Architecture” on **page 8**).

Cơ sở dữ liệu là các hệ thống dạng mô-đun và bao gồm nhiều thành phần: một tầng truyền vận (transport layer) tiếp nhận yêu cầu, một bộ xử lý truy vấn (query processor) xác định cách chạy truy vấn hiệu quả nhất, một bộ thực thi (execution engine) tiến hành các thao tác, và một bộ máy lưu trữ (storage engine) (xem mục “Kiến trúc DBMS” ở trang 8).

The storage engine (or database engine) is a software component of a database management system responsible for storing, retrieving, and managing data in memory and on disk, designed to capture a persistent, long-term memory of each **node** [REED78] . While databases can respond to complex queries, storage engines look at the data more granularly and offer a simple data manipulation API, allowing users to create, update, delete, and retrieve records. One way to look at this is that database management systems are applications built on top of storage engines, offering a schema, a query language, indexing, transactions, and many other useful features.

Bộ máy lưu trữ (hay database engine) là một thành phần phần mềm của hệ quản trị cơ sở dữ liệu, chịu trách nhiệm lưu trữ, truy xuất và quản lý dữ liệu trong bộ nhớ và trên đĩa, được thiết kế để nắm giữ bộ nhớ lâu dài, bền bỉ của mỗi nút (node) [REED78]. Trong khi cơ sở dữ liệu có thể đáp ứng các truy vấn phức tạp, bộ máy lưu trữ nhìn dữ liệu ở mức chi tiết hơn và cung cấp một API thao tác dữ liệu đơn giản, cho phép người dùng tạo, cập nhật, xóa và truy xuất bản ghi. Có thể hình dung rằng hệ quản trị cơ sở dữ liệu là các ứng dụng được xây dựng trên nền bộ máy lưu trữ, cung cấp lược đồ (schema), ngôn ngữ truy vấn, lập chỉ mục, giao dịch (transaction) và nhiều tính năng hữu ích khác.

For flexibility, both keys and values can be arbitrary sequences of bytes with no prescribed form. Their sorting and representation semantics are defined in higher-level subsystems. For example, you can use int32 (32-bit integer) as a key in one of the tables, and ascii (ASCII string) in the other; from the storage engine perspective both keys are just serialized entries.

Để linh hoạt, cả khóa (key) lẫn giá trị (value) đều có thể là chuỗi byte tùy ý mà không có định dạng quy định trước. Ngữ nghĩa sắp xếp và biểu diễn của chúng được định nghĩa ở các phân hệ cấp cao hơn. Chẳng hạn, bạn có thể dùng int32 (số nguyên 32-bit) làm khóa trong một bảng và ascii (chuỗi ASCII) trong một bảng khác; từ góc nhìn của bộ máy lưu trữ, cả hai khóa đều chỉ là các mục đã được tuần tự hóa.

Storage engines such as BerkeleyDB , LevelDB and its descendant RocksDB , LMDB and its descendant libmdbx , Sophia , HaloDB , and many others were developed independently from the database management systems they’re now embedded into. Using pluggable storage engines has enabled database developers to bootstrap database systems using existing storage engines, and concentrate on the other subsystems.

Các bộ máy lưu trữ như BerkeleyDB, LevelDB và hậu duệ của nó là RocksDB, LMDB và hậu duệ của nó là libmdbx, Sophia, HaloDB, cùng nhiều cái khác, được phát triển độc lập với các hệ quản trị cơ sở dữ liệu mà chúng nay đã được nhúng vào. Việc dùng các bộ máy lưu trữ có thể cắm-ghép (pluggable) đã giúp các nhà phát triển cơ sở dữ liệu khởi tạo nhanh hệ cơ sở dữ liệu dựa trên các bộ máy lưu trữ sẵn có, và tập trung vào các phân hệ khác.

At the same time, clear separation between database system components opens up an opportunity to switch between different engines, potentially better suited for particular use cases. For example, MySQL, a popular database management system, has several storage engines , including InnoDB, MyISAM, and RocksDB (in the MyRocks distribution). MongoDB allows switching between WiredTiger , In-Memory, and the (now-deprecated) MMAPv1 storage engines.

Đồng thời, việc tách bạch rõ ràng giữa các thành phần của hệ cơ sở dữ liệu mở ra cơ hội chuyển đổi giữa các bộ máy lưu trữ khác nhau, có thể phù hợp hơn cho những trường hợp sử dụng cụ thể. Ví dụ, MySQL, một hệ quản trị cơ sở dữ liệu phổ biến, có nhiều bộ máy lưu trữ, bao gồm InnoDB, MyISAM và RocksDB (trong bản phân phối MyRocks). MongoDB cho phép chuyển đổi giữa các bộ máy lưu trữ WiredTiger, In-Memory và MMAPv1 (nay đã ngừng dùng).

Comparing Databases — So sánh các cơ sở dữ liệu

Your choice of database system may have long-term consequences. If there's a chance that a database is not a good fit because of performance problems, consistency issues, or operational challenges, it is better to find out about it earlier in the development cycle, since it can be nontrivial to migrate to a different system. In some cases, it may require substantial changes in the application code.

Lựa chọn hệ cơ sở dữ liệu của bạn có thể để lại hậu quả lâu dài. Nếu có khả năng một cơ sở dữ liệu không phù hợp do vấn đề hiệu năng, vấn đề về tính nhất quán (consistency), hoặc thách thức vận hành, thì tốt nhất là phát hiện ra điều đó sớm trong chu kỳ phát triển, bởi việc di trú (migrate) sang một hệ khác có thể không hề đơn giản. Trong một số trường hợp, nó có thể đòi hỏi thay đổi đáng kể trong mã ứng dụng.

Every database system has strengths and weaknesses. To reduce the risk of an expensive migration, you can invest some time before you decide on a specific database to build confidence in its ability to meet your application's needs.

Mỗi hệ cơ sở dữ liệu đều có điểm mạnh và điểm yếu. Để giảm rủi ro của một cuộc di trú tốn kém, bạn có thể đầu tư chút thời gian trước khi quyết định chọn một cơ sở dữ liệu cụ thể, nhằm xây dựng niềm tin vào khả năng đáp ứng nhu cầu ứng dụng của nó.

Trying to compare databases based on their components (e.g., which storage engine they use, how the data is shared, replicated, and distributed, etc.), their rank (an arbitrary popularity value assigned by consultancy agencies such as ThoughtWorks or database comparison websites such as DB-Engines or Database of Databases), or implementation language (C++, Java, or Go, etc.) can lead to invalid and premature conclusions. These methods can be used only for a high-level comparison and can be as coarse as choosing between HBase and SQLite, so even a superficial understanding of how each database works and what's inside it can help you land a more weighted conclusion.

Việc cố gắng so sánh các cơ sở dữ liệu dựa trên các thành phần của chúng (ví dụ: chúng dùng bộ máy lưu trữ nào, dữ liệu được chia sẻ, sao chép (replicate) và phân tán ra sao, v.v.), dựa trên thứ hạng (một giá trị mức độ phổ biến tùy tiện do các hãng tư vấn như ThoughtWorks, hoặc các website so sánh cơ sở dữ liệu như DB-Engines hay Database of Databases gán cho), hoặc dựa trên ngôn ngữ hiện thực (C++, Java, hay Go, v.v.) có thể dẫn đến những kết luận sai và vô ích. Các phương pháp này chỉ có thể dùng cho so sánh ở mức tổng quan và có thể thô đến mức như việc chọn giữa HBase và SQLite, nên ngay cả một hiểu biết sơ lược về cách mỗi cơ sở dữ liệu hoạt động và bên trong nó có gì cũng có thể giúp bạn đi đến một kết luận cân nhắc hơn.

Every comparison should start by clearly defining the goal, because even the slightest bias may completely invalidate the entire investigation. If you're searching for a database that would be a good fit for the workloads you have (or are planning to facilitate), the best thing you can do is to simulate these workloads against different database systems, measure the performance metrics that are important for you, and compare results. Some issues, especially when it comes to performance and scalability, start showing only after some time or as the capacity grows. To detect potential

problems, it is best to have long-running tests in an environment that simulates the real-world production setup as closely as possible.

Mọi sự so sánh nên bắt đầu bằng việc định nghĩa rõ ràng mục tiêu, bởi dù chỉ một thiên kiến nhỏ nhất cũng có thể làm vô hiệu hoàn toàn cả cuộc khảo sát. Nếu bạn đang tìm một cơ sở dữ liệu phù hợp với khối lượng công việc (workload) mà bạn có (hoặc đang dự định phục vụ), thì điều tốt nhất bạn có thể làm là mô phỏng các khối lượng công việc này trên những hệ cơ sở dữ liệu khác nhau, đo lường các chỉ số hiệu năng quan trọng đối với bạn và so sánh kết quả. Một số vấn đề, đặc biệt là về hiệu năng và khả năng mở rộng, chỉ bắt đầu lộ ra sau một thời gian hoặc khi dung lượng tăng lên. Để phát hiện các vấn đề tiềm ẩn, tốt nhất là chạy những bài kiểm thử kéo dài trong một môi trường mô phỏng sát nhất có thể với thiết lập sản xuất (production) thực tế.

Simulating real-world workloads not only helps you understand how the database performs, but also helps you learn how to operate, debug, and find out how friendly and helpful its community is. Database choice is always a combination of these factors, and performance often turns out not to be the most important aspect: it's usually much better to use a database that slowly saves the data than one that quickly loses it.

Việc mô phỏng các khối lượng công việc thực tế không chỉ giúp bạn hiểu cơ sở dữ liệu hoạt động ra sao, mà còn giúp bạn học cách vận hành, gỡ lỗi, và tìm hiểu xem cộng đồng của nó thân thiện và hữu ích đến đâu. Lựa chọn cơ sở dữ liệu luôn là sự kết hợp của các yếu tố này, và hiệu năng thường hóa ra không phải là khía cạnh quan trọng nhất: thường thì dùng một cơ sở dữ liệu lưu dữ liệu chậm còn tốt hơn nhiều so với một cơ sở dữ liệu làm mất dữ liệu nhanh.

To compare databases, it's helpful to understand the use case in great detail and define the current and anticipated variables, such as:

Để so sánh các cơ sở dữ liệu, sẽ rất hữu ích nếu hiểu trường hợp sử dụng thật chi tiết và xác định các biến số hiện tại cũng như dự kiến, chẳng hạn như:

- Schema and record sizes
- Number of clients
- Types of queries and access patterns
- Rates of the **read** and write queries
- Expected changes in any of these variables
 - Lượng dữ liệu và kích thước bản ghi
 - Số lượng client
 - Các loại truy vấn và mẫu truy cập (access pattern)
 - Tốc độ của các truy vấn đọc và ghi
 - Những thay đổi dự kiến ở bất kỳ biến số nào trong số này

Knowing these variables can help to answer the following questions:

Biết các biến số này có thể giúp trả lời các câu hỏi sau:

- Does the database support the required queries?
- Is this database able to handle the amount of data we're planning to store?
- How many read and write operations can a single node handle?
- How many nodes should the system have?
- How do we expand the cluster given the expected growth rate?
- What is the maintenance **process**?
 - Cơ sở dữ liệu có hỗ trợ các truy vấn cần thiết không?

- Cơ sở dữ liệu này có khả năng xử lý lượng dữ liệu mà ta dự định lưu trữ không?
- Một nút đơn lẻ có thể xử lý bao nhiêu thao tác đọc và ghi?
- Hệ thống nên có bao nhiêu nút?
- Làm thế nào để mở rộng cụm (cluster) với tốc độ tăng trưởng dự kiến?
- Quy trình bảo trì là gì?

Having these questions answered, you can construct a test cluster and simulate your workloads. Most databases already have stress tools that can be used to reconstruct specific use cases. If there's no standard stress tool to generate realistic randomized workloads in the database ecosystem, it might be a red flag. If something prevents you from using default tools, you can try one of the existing general-purpose tools, or implement one from scratch.

Sau khi trả lời được những câu hỏi này, bạn có thể dựng một cụm kiểm thử và mô phỏng các khối lượng công việc của mình. Hầu hết cơ sở dữ liệu đã có sẵn các công cụ tạo tải (stress tool) có thể dùng để tái dựng các trường hợp sử dụng cụ thể. Nếu không có công cụ tạo tải chuẩn nào để sinh ra các khối lượng công việc ngẫu nhiên sát thực tế trong hệ sinh thái của cơ sở dữ liệu, thì đó có thể là một dấu hiệu cảnh báo. Nếu có điều gì đó ngăn bạn dùng công cụ mặc định, bạn có thể thử một trong các công cụ đa dụng sẵn có, hoặc tự hiện thực một công cụ từ đầu.

If the tests show positive results, it may be helpful to familiarize yourself with the database code. Looking at the code, it is often useful to first understand the parts of the database, how to find the code for different components, and then navigate through those. Having even a rough idea about the database codebase helps you better understand the log records it produces, its configuration parameters, and helps you find issues in the application that uses it and even in the database code itself.

Nếu kết quả kiểm thử cho thấy tín hiệu tích cực, có thể sẽ hữu ích nếu bạn làm quen với mã nguồn của cơ sở dữ liệu. Khi xem mã nguồn, thường thì việc đầu tiên nên làm là hiểu các phần của cơ sở dữ liệu, cách tìm mã cho từng thành phần khác nhau, rồi điều hướng qua chúng. Ngay cả khi chỉ nắm được ý niệm sơ lược về mã nguồn của cơ sở dữ liệu, điều đó cũng giúp bạn hiểu rõ hơn các bản ghi nhật ký (log) mà nó sinh ra, các tham số cấu hình của nó, và giúp bạn tìm ra lỗi trong ứng dụng sử dụng nó, thậm chí trong chính mã nguồn của cơ sở dữ liệu.

It'd be great if we could use databases as black boxes and never have to take a look inside them, but the practice shows that sooner or later a bug, an outage, a performance regression, or some other problem pops up, and it's better to be prepared for it. If you know and understand database internals, you can reduce business risks and improve chances for a quick **recovery**.

Sẽ thật tuyệt nếu chúng ta có thể dùng cơ sở dữ liệu như một hộp đen và không bao giờ phải nhìn vào bên trong, nhưng thực tế cho thấy rằng sớm hay muộn thì một lỗi, một sự cố ngừng hoạt động (outage), một sự suy giảm hiệu năng, hay một vấn đề nào khác cũng sẽ xuất hiện, và tốt hơn hết là hãy chuẩn bị sẵn sàng cho nó. Nếu bạn biết và hiểu nội tại của cơ sở dữ liệu, bạn có thể giảm rủi ro kinh doanh và tăng cơ hội phục hồi nhanh chóng.

One of the popular tools used for benchmarking, performance evaluation, and comparison is Yahoo! Cloud Serving Benchmark (YCSB). YCSB offers a framework and a common set of workloads that can be applied to different data stores. Just like anything generic, this tool should be used with caution, since it's easy to make wrong conclusions. To make a fair comparison and make an educated decision, it is necessary to invest enough time to understand the real-world conditions under which the database has to perform, and tailor benchmarks accordingly.

Một trong những công cụ phổ biến dùng để đo điểm chuẩn (benchmark), đánh giá và so

sánh hiệu năng là Yahoo! Cloud Serving Benchmark (YCSB). YCSB cung cấp một khung làm việc (framework) và một tập hợp các khối lượng công việc chung có thể áp dụng cho các kho dữ liệu khác nhau. Cũng như bất kỳ thứ gì mang tính tổng quát, công cụ này nên được dùng một cách thận trọng, vì rất dễ rút ra những kết luận sai. Để so sánh công bằng và đưa ra quyết định sáng suốt, cần đầu tư đủ thời gian để hiểu các điều kiện thực tế mà cơ sở dữ liệu phải vận hành trong đó, và điều chỉnh các bài đo điểm chuẩn cho phù hợp.

TPC-C Benchmark — Đo điểm chuẩn TPC-C

The **Transaction Processing** Performance Council (TPC) has a set of benchmarks that database vendors use for comparing and advertising performance of their products. TPC-C is an online transaction processing (**OLTP**) benchmark, a mixture of read-only and update transactions that simulate common application workloads.

Hội đồng Hiệu năng Xử lý Giao dịch (Transaction Processing Performance Council - TPC) có một bộ các bài đo điểm chuẩn mà các nhà cung cấp cơ sở dữ liệu dùng để so sánh và quảng bá hiệu năng sản phẩm của họ. TPC-C là một bài đo điểm chuẩn xử lý giao dịch trực tuyến (OLTP), một hỗn hợp giữa các giao dịch chỉ-đọc và các giao dịch cập nhật, nhằm mô phỏng các khối lượng công việc ứng dụng phổ biến.

This benchmark concerns itself with the performance and correctness of executed concurrent transactions. The main performance indicator is throughput : the number of transactions the database system is able to process per minute. Executed transactions are required to preserve **ACID** properties and conform to the set of properties defined by the benchmark itself.

Bài đo điểm chuẩn này quan tâm đến hiệu năng và tính đúng đắn của các giao dịch tương tranh được thực thi. Chỉ số hiệu năng chính là thông lượng (throughput): số lượng giao dịch mà hệ cơ sở dữ liệu có thể xử lý mỗi phút. Các giao dịch được thực thi phải bảo toàn các thuộc tính ACID và tuân thủ tập hợp thuộc tính do chính bài đo điểm chuẩn định nghĩa.

This benchmark does not concentrate on any particular business segment, but provides an abstract set of actions important for most of the applications for which OLTP databases are suitable. It includes several tables and entities such as warehouses, stock (inventory), customers and orders, specifying table layouts, details of transactions that can be performed against these tables, the minimum number of rows per table, and data **durability** constraints.

Bài đo điểm chuẩn này không tập trung vào bất kỳ phân khúc kinh doanh cụ thể nào, mà cung cấp một tập hành động trừu tượng quan trọng đối với hầu hết các ứng dụng phù hợp với cơ sở dữ liệu OLTP. Nó bao gồm vài bảng và thực thể như kho hàng (warehouse), tồn kho (stock/inventory), khách hàng và đơn hàng, đồng thời quy định bố cục bảng, chi tiết của các giao dịch có thể thực hiện trên các bảng này, số hàng tối thiểu mỗi bảng, và các ràng buộc về độ bền dữ liệu (durability).

This doesn't mean that benchmarks can be used only to compare databases. Benchmarks can be useful to define and test details of the service-level agreement, 1 under-

Điều này không có nghĩa là các bài đo điểm chuẩn chỉ có thể dùng để so sánh các cơ sở dữ liệu. Các bài đo điểm chuẩn có thể hữu ích để định nghĩa và kiểm thử chi tiết của thỏa thuận mức dịch vụ,[1] để hiểu

1 The service-level agreement (or SLA) is a commitment by the service provider about the quality of provided services. Among other things, the SLA can include information about latency, throughput, jitter, and the number and frequency of failures.

[1] Thỏa thuận mức dịch vụ (service-level agreement, hay SLA) là một cam kết của nhà cung cấp dịch vụ về chất lượng dịch vụ được cung cấp. Cùng với những thứ khác, SLA có thể bao gồm thông tin về độ trễ (latency), thông lượng, độ rung (jitter), cũng như số lượng và tần suất các lỗi.

standing system requirements, capacity planning, and more. The more knowledge you have about the database before using it, the more time you'll save when running it in production.

các yêu cầu hệ thống, hoạch định dung lượng (capacity planning) và nhiều hơn nữa. Bạn càng có nhiều kiến thức về cơ sở dữ liệu trước khi sử dụng, bạn càng tiết kiệm được nhiều thời gian khi chạy nó trong môi trường sản xuất.

Choosing a database is a long-term decision, and it's best to keep **track** of newly released versions, understand what exactly has changed and why, and have an upgrade strategy. New releases usually contain improvements and fixes for bugs and security issues, but may introduce new bugs, performance regressions, or unexpected behavior, so testing new versions before rolling them out is also critical. Checking how database implementers were handling upgrades previously might give you a good idea about what to expect in the future. Past smooth upgrades do not guarantee that future ones will be as smooth, but complicated upgrades in the past might be a sign that future ones won't be easy, either.

Việc chọn một cơ sở dữ liệu là một quyết định dài hạn, và tốt nhất là theo dõi các phiên bản mới được phát hành, hiểu chính xác điều gì đã thay đổi và tại sao, đồng thời có một chiến lược nâng cấp. Các bản phát hành mới thường chứa các cải tiến cùng bản vá cho lỗi và các vấn đề bảo mật, nhưng cũng có thể đưa vào những lỗi mới, sự suy giảm hiệu năng, hoặc hành vi bất ngờ, nên việc kiểm thử các phiên bản mới trước khi triển khai cũng rất quan trọng. Việc xem xét cách những người hiện thực cơ sở dữ liệu đã xử lý các lần nâng cấp trước đó có thể cho bạn ý niệm tốt về điều cần kỳ vọng trong tương lai. Những lần nâng cấp suôn sẻ trong quá khứ không bảo đảm rằng các lần nâng cấp tương lai cũng suôn sẻ như vậy, nhưng những lần nâng cấp phức tạp trong quá khứ có thể là dấu hiệu cho thấy các lần tương lai cũng sẽ không dễ dàng.

Understanding Trade-Offs — Thấu hiểu các đánh đổi

As users, we can see how databases behave under different conditions, but when working on databases, we have to make choices that influence this behavior directly.

Với tư cách người dùng, chúng ta có thể thấy các cơ sở dữ liệu hành xử ra sao trong những điều kiện khác nhau, nhưng khi làm việc trên chính cơ sở dữ liệu, chúng ta phải đưa ra những lựa chọn ảnh hưởng trực tiếp đến hành vi này.

Designing a storage engine is definitely more complicated than just implementing a textbook data structure: there are many details and edge cases that are hard to get right from the start. We need to design the physical data layout and organize pointers, decide on the serialization format, understand how data is going to be garbage-collected, how the storage engine fits into the semantics of the database system as a whole, figure out how to make it work in a concurrent environment, and, finally, make sure we never lose any data, under any circumstances.

Thiết kế một bộ máy lưu trữ chắc chắn phức tạp hơn nhiều so với việc chỉ hiện thực một cấu trúc dữ liệu sách giáo khoa: có rất nhiều chi tiết và trường hợp biên khó làm cho đúng ngay từ đầu. Chúng ta cần thiết kế bố cục dữ liệu vật lý và tổ chức các con trỏ, quyết định về định dạng tuần tự hóa, hiểu dữ liệu sẽ được thu gom rác (garbage collection) như thế nào, bộ máy lưu trữ khớp vào ngữ nghĩa của toàn bộ hệ cơ sở dữ liệu ra sao, tìm cách

làm cho nó hoạt động trong một môi trường tương tranh, và cuối cùng, bảo đảm chúng ta không bao giờ mất bất kỳ dữ liệu nào, trong bất kỳ hoàn cảnh nào.

Not only there are many things to decide upon, but most of these decisions involve trade-offs. For example, if we save records in the order they were inserted into the database, we can store them quicker, but if we retrieve them in their lexicographical order, we have to re-sort them before returning results to the client. As you will see throughout this book, there are many different approaches to storage engine design, and every implementation has its own upsides and downsides.

Không chỉ có nhiều điều phải quyết định, mà hầu hết các quyết định này đều kéo theo những đánh đổi. Ví dụ, nếu chúng ta lưu các bản ghi theo thứ tự chúng được chèn vào cơ sở dữ liệu, ta có thể lưu chúng nhanh hơn, nhưng nếu ta truy xuất chúng theo thứ tự từ điển, ta phải sắp xếp lại chúng trước khi trả kết quả về cho client. Như bạn sẽ thấy xuyên suốt cuốn sách này, có rất nhiều cách tiếp cận khác nhau để thiết kế bộ máy lưu trữ, và mỗi hiện thực đều có ưu và nhược điểm riêng.

When looking at different storage engines, we discuss their benefits and shortcomings. If there was an absolutely optimal storage engine for every conceivable use case, everyone would just use it. But since it does not exist, we need to choose wisely, based on the workloads and use cases we're trying to facilitate.

Khi xem xét các bộ máy lưu trữ khác nhau, chúng ta sẽ thảo luận về lợi ích và hạn chế của chúng. Nếu tồn tại một bộ máy lưu trữ tối ưu tuyệt đối cho mọi trường hợp sử dụng có thể hình dung ra, thì hẳn ai cũng sẽ chỉ dùng nó. Nhưng vì nó không tồn tại, chúng ta cần lựa chọn một cách khôn ngoan, dựa trên các khối lượng công việc và trường hợp sử dụng mà ta đang cố gắng phục vụ.

There are many storage engines, using all sorts of data structures, implemented in different languages, ranging from low-level ones, such as C, to high-level ones, such as Java. All storage engines face the same challenges and constraints. To draw a parallel with city planning, it is possible to build a city for a specific population and choose to build up or build out. In both cases, the same number of people will fit into the city, but these approaches lead to radically different lifestyles. When building the city up, people live in apartments and population density is likely to lead to more traffic in a smaller area; in a more spread-out city, people are more likely to live in houses, but commuting will require covering larger distances.

Có rất nhiều bộ máy lưu trữ, dùng đủ loại cấu trúc dữ liệu, được hiện thực bằng các ngôn ngữ khác nhau, từ ngôn ngữ cấp thấp như C đến ngôn ngữ cấp cao như Java. Mọi bộ máy lưu trữ đều đối mặt với cùng những thách thức và ràng buộc. Để vẽ một phép loại suy với quy hoạch đô thị, ta có thể xây một thành phố cho một quy mô dân số nhất định và chọn xây lên cao hay xây trải rộng. Trong cả hai trường hợp, cùng một số lượng người sẽ vừa với thành phố, nhưng các cách tiếp cận này dẫn đến những lối sống khác nhau hoàn toàn. Khi xây thành phố lên cao, người ta sống trong căn hộ và mật độ dân số có khả năng dẫn đến nhiều giao thông hơn trong một khu vực nhỏ hơn; còn ở một thành phố trải rộng hơn, người ta có nhiều khả năng sống trong nhà riêng, nhưng việc đi lại sẽ đòi hỏi phải di chuyển quãng đường dài hơn.

Similarly, design decisions made by storage engine developers make them better suited for different things: some are optimized for low read or write latency, some try to maximize density (the amount of stored data per node), and some concentrate on operational simplicity.

Tương tự, các quyết định thiết kế mà những nhà phát triển bộ máy lưu trữ đưa ra khiến chúng phù hợp hơn cho những việc khác nhau: một số được tối ưu cho độ trễ đọc hoặc ghi thấp, một số cố gắng tối đa hóa mật độ (lượng dữ liệu lưu trên mỗi nút), và một số tập

trung vào sự đơn giản trong vận hành.

You can find complete algorithms that can be used for the implementation and other additional references in the chapter summaries. Reading this book should make you well equipped to work productively with these sources and give you a solid understanding of the existing alternatives to concepts described there.

Bạn có thể tìm thấy các thuật toán hoàn chỉnh có thể dùng để hiện thực cùng các tài liệu tham khảo bổ sung khác trong phần tóm tắt của mỗi chương. Việc đọc cuốn sách này sẽ trang bị cho bạn đủ khả năng để làm việc hiệu quả với những nguồn tài liệu đó và cho bạn một hiểu biết vững chắc về các phương án thay thế hiện có cho những khái niệm được mô tả ở đó.

CHAPTER 1 Introduction and Overview — Chương 1: Giới thiệu và Tổng quan

Database management systems can serve different purposes: some are used primarily for temporary hot data, some serve as a long-lived cold storage, some allow complex analytical queries, some only allow accessing values by the key, some are optimized to store time-series data, and some store large blobs efficiently. To understand differences and draw distinctions, we start with a short classification and overview, as this helps us to understand the scope of further discussions.

Hệ quản trị cơ sở dữ liệu (DBMS) có thể phục vụ nhiều mục đích khác nhau: một số chủ yếu dùng cho dữ liệu nóng tạm thời, một số đóng vai trò kho lưu trữ lạnh tồn tại lâu dài, một số cho phép các truy vấn phân tích phức tạp, một số chỉ cho phép truy cập giá trị theo khóa, một số được tối ưu để lưu dữ liệu chuỗi thời gian, và một số lưu các blob lớn một cách hiệu quả. Để hiểu được sự khác biệt và phân biệt rạch ròi, ta bắt đầu bằng một bảng phân loại và tổng quan ngắn gọn, vì điều này giúp ta nắm được phạm vi của các thảo luận tiếp theo.

Terminology can sometimes be ambiguous and hard to understand without a complete context. For example, distinctions between column and wide column stores that have little or nothing to do with each other, or how **clustered** and nonclustered indexes relate to index-organized tables . This chapter aims to disambiguate these terms and find their precise definitions.

Thuật ngữ đôi khi có thể mơ hồ và khó hiểu nếu thiếu ngữ cảnh đầy đủ. Chẳng hạn, sự phân biệt giữa kho lưu cột (column) và kho lưu cột rộng (wide column store) — vốn ít hoặc không liên quan gì đến nhau, hay việc chỉ mục gom cụm (clustered) và không gom cụm (nonclustered) liên hệ thế nào với bảng tổ chức theo chỉ mục (index-organized table). Chương này nhằm làm rõ những thuật ngữ đó và tìm ra định nghĩa chính xác cho chúng.

We start with an overview of database management system architecture (see “**DBMS Architecture**” on **page 8**), and discuss system components and their responsibilities. After that, we discuss the distinctions among the database management systems in terms of a storage medium (see “Memory-Versus Disk-Based DBMS” on page 10), and layout (see “Column- Versus **Row-Oriented** DBMS” on page 12).

Ta bắt đầu với tổng quan về kiến trúc hệ quản trị cơ sở dữ liệu (xem “Kiến trúc DBMS” trang 8), và thảo luận về các thành phần của hệ thống cùng trách nhiệm của chúng. Sau đó, ta bàn về sự khác biệt giữa các hệ quản trị cơ sở dữ liệu xét theo phương tiện lưu trữ (xem “DBMS dựa trên bộ nhớ so với dựa trên đĩa” trang 10), và theo cách bố trí dữ liệu (xem “DBMS hướng cột so với hướng hàng” trang 12).

These two groups do not present a full taxonomy of database management systems and there are many other ways they're classified. For example, some sources group DBMSs into three major categories:

Hai nhóm này không phải là một phân loại đầy đủ cho hệ quản trị cơ sở dữ liệu, và còn nhiều cách phân loại khác. Chẳng hạn, một số nguồn xếp DBMS vào ba nhóm lớn:

Online transaction processing (OLTP) databases These handle a large number of user-facing requests and transactions. Queries are often predefined and short-lived.

Cơ sở dữ liệu xử lý giao dịch trực tuyến (OLTP) Loại này xử lý một lượng lớn các yêu cầu và giao dịch hướng tới người dùng. Các truy vấn thường được định nghĩa sẵn và tồn tại trong thời gian ngắn.

7 Online analytical processing (OLAP) databases These handle complex aggregations. OLAP databases are often used for analytics and data warehousing, and are capable of handling complex, long-running ad hoc queries.

Cơ sở dữ liệu xử lý phân tích trực tuyến (OLAP) Loại này xử lý các phép gộp phức tạp. Cơ sở dữ liệu OLAP thường được dùng cho phân tích và kho dữ liệu, có khả năng xử lý các truy vấn ad hoc phức tạp, chạy lâu dài.

Hybrid transactional and analytical processing (HTAP) These databases combine properties of both OLTP and OLAP stores.

Xử lý lai giữa giao dịch và phân tích (HTAP) Các cơ sở dữ liệu này kết hợp đặc tính của cả kho OLTP và OLAP.

There are many other terms and classifications: key-value stores, relational databases, document-oriented stores, and graph databases. These concepts are not defined here, since the reader is assumed to have a high-level knowledge and understanding of their functionality. Because the concepts we discuss here are widely applicable and are used in most of the mentioned types of stores in some capacity, complete taxonomy is not necessary or important for further discussion.

Còn nhiều thuật ngữ và cách phân loại khác: kho khóa-giá trị (key-value store), cơ sở dữ liệu quan hệ, kho hướng tài liệu (document-oriented store), và cơ sở dữ liệu đồ thị. Các khái niệm này không được định nghĩa ở đây, vì giả định rằng người đọc đã có kiến thức và hiểu biết tổng quát về chức năng của chúng. Bởi các khái niệm bàn đến ở đây có tính ứng dụng rộng và được dùng ở mức độ nào đó trong hầu hết các loại kho đã nêu, nên một phân loại đầy đủ là không cần thiết hay quan trọng đối với các thảo luận tiếp theo.

Since Part I of this book focuses on the storage and indexing structures, we need to understand the high-level data organization approaches, and the relationship between the data and index files (see “Data Files and Index Files” on page 17).

Vì Phần I của cuốn sách này tập trung vào các cấu trúc lưu trữ và lập chỉ mục, ta cần hiểu các cách tiếp cận tổ chức dữ liệu ở mức cao, cũng như mối quan hệ giữa tệp dữ liệu (data file) và tệp chỉ mục (index file) (xem “Tệp dữ liệu và Tệp chỉ mục” trang 17).

Finally, in “**Buffering, Immutability, and Ordering**” on page 21 , we discuss three techniques widely used to develop efficient storage structures and how applying these techniques influences their design and implementation.

Cuối cùng, trong “Đệm, Tính bất biến và Sắp thứ tự” trang 21, ta thảo luận ba kỹ thuật được dùng rộng rãi để phát triển các cấu trúc lưu trữ hiệu quả và việc áp dụng các kỹ thuật này ảnh hưởng ra sao đến thiết kế và hiện thực của chúng.

DBMS Architecture — Kiến trúc DBMS

There's no common blueprint for database management system design. Every database is built slightly differently, and component boundaries are somewhat hard to see and define. Even if these boundaries exist on paper (e.g., in project documentation), in code seemingly independent components may be coupled because of performance optimizations, handling edge cases, or architectural decisions.

Không có một bản thiết kế chung nào cho việc thiết kế hệ quản trị cơ sở dữ liệu. Mỗi cơ sở dữ liệu được xây dựng hơi khác nhau, và ranh giới giữa các thành phần khá khó nhìn ra và khó định nghĩa. Ngay cả khi những ranh giới này tồn tại trên giấy (ví dụ trong tài liệu dự án), thì trong mã nguồn các thành phần tưởng chừng độc lập lại có thể bị ghép chặt với nhau do các tối ưu hiệu năng, xử lý các trường hợp biên, hoặc các quyết định kiến trúc.

Sources that describe database management system architecture (for example, [HELLERSTEIN07], [WEIKUM01], [ELMASRI11], and [MOLINA08]), define components and relationships between them differently. The architecture presented in Figure 1-1 demonstrates some of the common themes in these representations.

Các nguồn mô tả kiến trúc hệ quản trị cơ sở dữ liệu (ví dụ [HELLERSTEIN07], [WEIKUM01], [ELMASRI11], và [MOLINA08]) định nghĩa các thành phần và quan hệ giữa chúng theo những cách khác nhau. Kiến trúc trình bày trong Hình 1-1 minh họa một số chủ đề chung trong các cách biểu diễn đó.

Database management systems use a client/server model, where database system instances (nodes) take the role of servers, and application instances take the role of clients.

Hệ quản trị cơ sở dữ liệu dùng mô hình máy khách/máy chủ (client/server), trong đó các thực thể của hệ cơ sở dữ liệu (các nút - node) đóng vai trò máy chủ, còn các thực thể ứng dụng đóng vai trò máy khách.

Client requests arrive through the **transport subsystem**. Requests come in the form of queries, most often expressed in some query language. The transport subsystem is also responsible for communication with other nodes in the database cluster.

Yêu cầu của máy khách đến qua phân hệ truyền vận (transport subsystem). Các yêu cầu này ở dạng truy vấn, thường được diễn đạt bằng một ngôn ngữ truy vấn nào đó. Phân hệ truyền vận cũng chịu trách nhiệm giao tiếp với các nút khác trong cụm cơ sở dữ liệu.

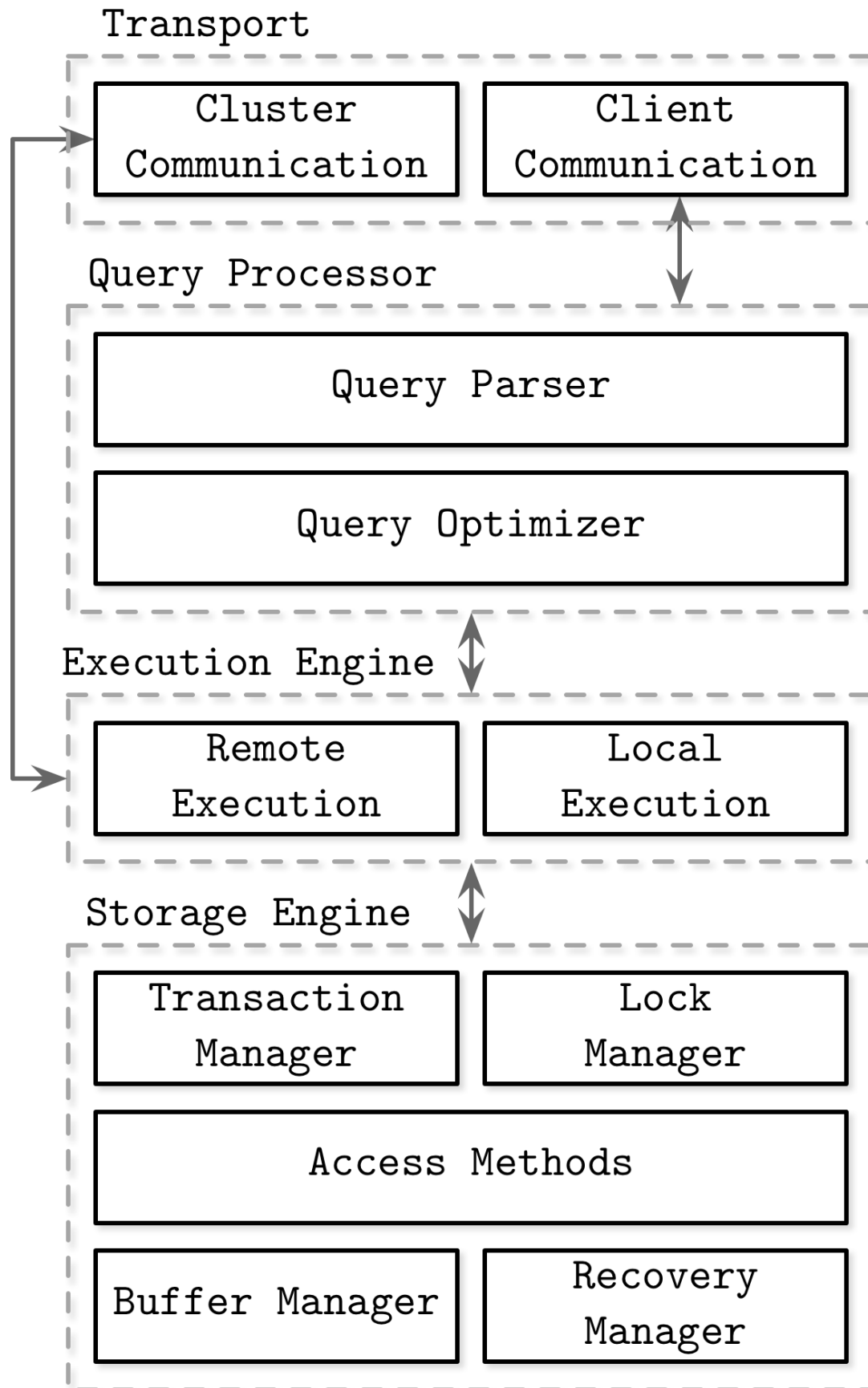


Figure 1-1. Architecture of a database management system

Hình 1-1. Kiến trúc của một hệ quản trị cơ sở dữ liệu

Upon receipt, the transport subsystem hands the query over to a **query processor**, which parses, interprets, and validates it. Later, access control checks are performed, as they can be done fully only after the query is interpreted.

Khi nhận được, phân hệ truyền vận chuyển truy vấn cho bộ xử lý truy vấn (query processor), nơi phân tích cú pháp, diễn giải và kiểm tra hợp lệ truy vấn. Sau đó, các kiểm tra kiểm soát truy cập được thực hiện, vì chúng chỉ có thể hoàn thành trọn vẹn sau khi truy vấn đã được diễn giải.

The parsed query is passed to the **query optimizer**, which first eliminates impossible and redundant parts of the query, and then attempts to find the most efficient way to execute it based on internal statistics (index cardinality, approximate intersection size, etc.) and data placement (which nodes in the cluster hold the data and the costs associated with its transfer). The optimizer handles both relational operations required for query resolution, usually presented as a dependency tree, and optimizations, such as index ordering, cardinality estimation, and choosing access methods.

Truy vấn đã phân tích cú pháp được chuyển cho bộ tối ưu truy vấn (query optimizer), nơi trước tiên loại bỏ các phần bất khả thi và dư thừa của truy vấn, rồi cố gắng tìm ra cách thực thi hiệu quả nhất dựa trên thống kê nội bộ (lực lượng - cardinality của chỉ mục, kích thước giao xấp xỉ, v.v.) và vị trí dữ liệu (những nút nào trong cụm đang giữ dữ liệu và chi phí liên quan đến việc truyền nó). Bộ tối ưu xử lý cả các phép toán quan hệ cần thiết để giải truy vấn, thường được biểu diễn dưới dạng cây phụ thuộc, lẫn các tối ưu hóa như sắp thứ tự chỉ mục, ước lượng lực lượng và lựa chọn phương thức truy cập.

The query is usually presented in the form of an **execution plan** (or query plan): a sequence of operations that have to be carried out for its results to be considered complete. Since the same query can be satisfied using different execution plans that can vary in efficiency, the optimizer picks the best available plan.

Truy vấn thường được biểu diễn dưới dạng một kế hoạch thực thi (execution plan, hay query plan): một chuỗi các thao tác phải được tiến hành để kết quả của nó được coi là hoàn chỉnh. Vì cùng một truy vấn có thể được đáp ứng bằng các kế hoạch thực thi khác nhau với hiệu quả khác nhau, bộ tối ưu chọn kế hoạch tốt nhất hiện có.

The execution plan is handled by the **execution engine**, which collects the results of the execution of local and remote operations. Remote execution can involve writing and reading data to and from other nodes in the cluster, and replication.

Kế hoạch thực thi được xử lý bởi bộ thực thi (execution engine), nơi thu thập kết quả của việc thực hiện các thao tác cục bộ và từ xa. Việc thực thi từ xa có thể liên quan đến ghi và đọc dữ liệu đến và từ các nút khác trong cụm, và sao chép (replication).

Local queries (coming directly from clients or from other nodes) are executed by the **storage engine**. The storage engine has several components with dedicated responsibilities:

Các truy vấn cục bộ (đến trực tiếp từ máy khách hoặc từ các nút khác) được thực thi bởi bộ máy lưu trữ (storage engine). Bộ máy lưu trữ có một số thành phần với trách nhiệm chuyên biệt:

Transaction manager This manager schedules transactions and ensures they cannot leave the database in a logically inconsistent state.

Bộ quản lý giao dịch (Transaction manager) Bộ quản lý này lập lịch cho các giao dịch và bảo đảm chúng không thể đưa cơ sở dữ liệu vào trạng thái không nhất quán về mặt logic.

Lock manager This manager locks on the database objects for the running transactions, ensuring that concurrent operations do not violate physical data integrity.

Bộ quản lý khóa (Lock manager) Bộ quản lý này đặt khóa lên các đối tượng cơ sở dữ liệu cho các giao dịch đang chạy, bảo đảm các thao tác tương tranh không vi phạm tính toàn vẹn vật lý của dữ liệu.

Access methods (storage structures) These manage access and organizing data on disk. Access methods include heap files and storage structures such as B-Trees (see “Ubiquitous B-Trees” on page 33) or LSM Trees (see “LSM Trees” on page 130).

Phương thức truy cập (cấu trúc lưu trữ) Các phương thức này quản lý việc truy cập và tổ chức dữ liệu trên đĩa. Phương thức truy cập bao gồm các tệp heap (heap file) và các cấu trúc lưu trữ như B-Tree (xem “B-Tree phổ biến khắp nơi” trang 33) hay LSM Tree (xem “LSM Tree” trang 130).

Buffer manager This manager caches data pages in memory (see “Buffer Management” on page 81).

Bộ quản lý vùng đệm (Buffer manager) Bộ quản lý này lưu đệm các trang dữ liệu trong bộ nhớ (xem “Quản lý vùng đệm” trang 81).

Recovery manager This manager maintains the operation log and restoring the system state in case of a failure (see “Recovery” on page 88).

Bộ quản lý phục hồi (Recovery manager) Bộ quản lý này duy trì nhật ký thao tác và khôi phục trạng thái hệ thống trong trường hợp xảy ra sự cố (xem “Phục hồi” trang 88).

Together, transaction and lock managers are responsible for **concurrency control** (see “Concurrency Control” on page 93): they guarantee logical and physical data integrity while ensuring that concurrent operations are executed as efficiently as possible.

Cùng với nhau, bộ quản lý giao dịch và bộ quản lý khóa chịu trách nhiệm về điều khiển tương tranh (concurrency control) (xem “Điều khiển tương tranh” trang 93): chúng bảo đảm tính toàn vẹn logic và vật lý của dữ liệu trong khi vẫn bảo đảm các thao tác tương tranh được thực thi hiệu quả nhất có thể.

Memory- Versus Disk-Based DBMS — DBMS dựa trên bộ nhớ so với dựa trên đĩa

Database systems store data in memory and on disk. In-memory database management systems (sometimes called main memory DBMS) store data primarily in memory and use the disk for recovery and logging. Disk-based DBMS hold most of the data on disk and use memory for caching disk contents or as a temporary storage. Both types of systems use the disk to a certain extent, but main memory databases store their contents almost exclusively in RAM.

Các hệ cơ sở dữ liệu lưu dữ liệu trong bộ nhớ và trên đĩa. Hệ quản trị cơ sở dữ liệu trong bộ nhớ (đôi khi gọi là DBMS bộ nhớ chính - main memory DBMS) lưu dữ liệu chủ yếu trong bộ nhớ và dùng đĩa để phục hồi (recovery) và ghi nhật ký. DBMS dựa trên đĩa giữ phần lớn dữ liệu trên đĩa và dùng bộ nhớ để lưu đệm nội dung đĩa hoặc làm kho lưu tạm thời. Cả hai loại hệ thống đều sử dụng đĩa ở một mức độ nhất định, nhưng cơ sở dữ liệu bộ nhớ chính lưu nội dung của chúng gần như hoàn toàn trong RAM.

Accessing memory has been and remains several orders of magnitude faster than accessing disk, ¹ so it is compelling to use memory as the primary storage, and it becomes more economically feasible to do so as memory prices go down. However, RAM prices still remain high compared to persistent storage devices such as SSDs and HDDs.

Việc truy cập bộ nhớ đã và vẫn nhanh hơn nhiều cấp độ so với truy cập đĩa,¹ nên việc dùng bộ nhớ làm kho lưu trữ chính rất hấp dẫn, và điều này ngày càng khả thi về mặt kinh tế khi giá bộ nhớ giảm xuống. Tuy nhiên, giá RAM vẫn còn cao so với các thiết bị lưu trữ bền vững như SSD và HDD.

Main memory database systems are different from their disk-based counterparts not only in terms of a primary storage medium, but also in which data structures, organization, and optimization techniques they use.

Các hệ cơ sở dữ liệu bộ nhớ chính khác với các hệ dựa trên đĩa tương ứng không chỉ ở phương tiện lưu trữ chính, mà còn ở các cấu trúc dữ liệu, cách tổ chức và các kỹ thuật tối ưu mà chúng sử dụng.

Databases using memory as a primary data store do this mainly because of performance, comparatively low access costs, and access granularity. Programming for main memory is also significantly simpler than doing so for the disk. Operating systems abstract memory management and allow us to think in terms of allocating and freeing arbitrarily sized memory chunks. On disk, we have to manage data references, serialization formats, freed memory, and **fragmentation** manually.

Các cơ sở dữ liệu dùng bộ nhớ làm kho dữ liệu chính làm vậy chủ yếu vì hiệu năng, chi phí truy cập tương đối thấp và độ chi tiết truy cập (access granularity). Lập trình cho bộ nhớ chính cũng đơn giản hơn đáng kể so với lập trình cho đĩa. Hệ điều hành trừu tượng hóa việc quản lý bộ nhớ và cho phép ta tư duy theo kiểu cấp phát và giải phóng các khối bộ nhớ có kích thước tùy ý. Trên đĩa, ta phải tự quản lý các tham chiếu dữ liệu, định dạng tuần tự hóa, bộ nhớ đã giải phóng, và sự phân mảnh.

The main limiting factors on the growth of in-memory databases are RAM volatility (in other words, lack of **durability**) and costs. Since RAM contents are not persistent, software errors, crashes, hardware failures, and power outages can result in data loss. There are ways to ensure durability, such as uninterrupted power supplies and battery-backed RAM, but they require additional hardware resources and operational expertise. In practice, it all comes down to the fact that disks are easier to maintain and have significantly lower prices.

Các yếu tố giới hạn chính cho sự phát triển của cơ sở dữ liệu trong bộ nhớ là tính bay hơi của RAM (nói cách khác, thiếu tính bền vững) và chi phí. Vì nội dung RAM không bền vững, các lỗi phần mềm, sự cố sập, hỏng phần cứng và mất điện có thể dẫn đến mất dữ liệu. Có những cách để bảo đảm tính bền vững, chẳng hạn nguồn điện liên tục và RAM có pin dự phòng, nhưng chúng đòi hỏi thêm tài nguyên phần cứng và chuyên môn vận hành. Trên thực tế, tất cả quy về một điều: đĩa dễ bảo trì hơn và có giá thấp hơn đáng kể.

The situation is likely to change as the availability and popularity of Non-Volatile Memory (NVM) [ARULRAJ17] technologies grow. NVM storage reduces or completely eliminates (depending on the exact technology) asymmetry between **read** and write latencies, further improves read and write performance, and allows byte-addressable access.

Tình hình có thể sẽ thay đổi khi tính sẵn có và mức độ phổ biến của các công nghệ Bộ nhớ Không Bay hơi (Non-Volatile Memory - NVM) [ARULRAJ17] tăng lên. Lưu trữ NVM làm giảm hoặc loại bỏ hoàn toàn (tùy theo công nghệ cụ thể) sự bất đối xứng giữa độ trễ đọc và ghi, cải thiện thêm hiệu năng đọc và ghi, và cho phép truy cập theo địa chỉ byte.

Durability in Memory-Based Stores — Tính bền vững trong các kho dựa trên bộ nhớ

In-memory database systems maintain backups on disk to provide durability and prevent loss of the volatile data. Some databases store data exclusively in memory, without any durability guarantees, but we do not discuss them in the scope of this book.

Các hệ cơ sở dữ liệu trong bộ nhớ duy trì các bản sao lưu trên đĩa để cung cấp tính bền vững và ngăn mất dữ liệu bay hơi. Một số cơ sở dữ liệu lưu dữ liệu hoàn toàn trong bộ nhớ, không có bất kỳ bảo đảm bền vững nào, nhưng ta không bàn đến chúng trong phạm vi cuốn sách này.

Before the operation can be considered complete, its results have to be written to a **sequential** log file. We discuss write-ahead logs in more detail in “Recovery” on page 88 . To avoid replaying complete log contents during startup or after a crash, in-memory stores maintain a backup copy . The backup copy is maintained as a sorted

Trước khi một thao tác có thể được coi là hoàn tất, kết quả của nó phải được ghi vào một tệp nhật ký tuần tự. Ta sẽ thảo luận chi tiết hơn về nhật ký ghi-trước (write-ahead log) trong “Phục hồi” trang 88. Để tránh phải phát lại toàn bộ nội dung nhật ký trong lúc khởi động hoặc sau một sự cố sập, các kho trong bộ nhớ duy trì một bản sao lưu (backup copy). Bản sao lưu được duy trì dưới dạng một cấu trúc đã sắp xếp

1 You can find a visualization and comparison of disk, memory access latencies, and many other relevant numbers over the years at https://people.eecs.berkeley.edu/~rcs/research/interactive_latency.html

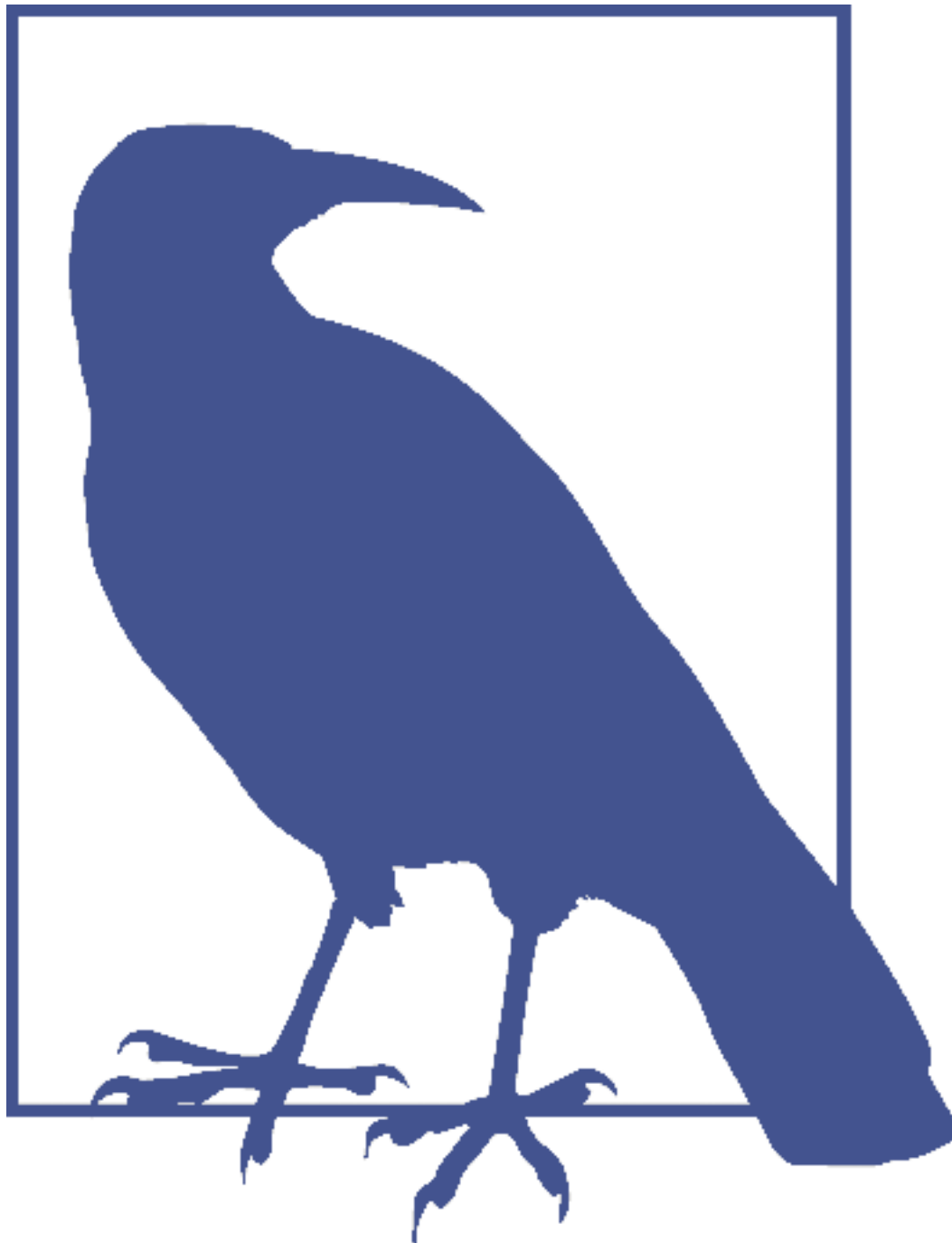
1 Bạn có thể tìm thấy hình ảnh trực quan và so sánh độ trễ truy cập đĩa, bộ nhớ, cùng nhiều con số liên quan khác qua các năm tại https://people.eecs.berkeley.edu/~rcs/research/interactive_latency.html.

disk-based structure, and modifications to this structure are often **asynchronous** (decoupled from client requests) and applied in batches to reduce the number of I/O operations. During recovery, database contents can be restored from the backup and logs.

dựa trên đĩa, và các sửa đổi đối với cấu trúc này thường là bất đồng bộ (tách rời khỏi các yêu cầu của máy khách) và được áp dụng theo lô để giảm số lượng thao tác I/O. Trong quá trình phục hồi, nội dung cơ sở dữ liệu có thể được khôi phục từ bản sao lưu và các nhật ký.

Log records are usually applied to backup in batches. After the batch of log records is processed, backup holds a database snapshot for a specific point in time, and log contents up to this point can be discarded. This **process** is called checkpointing . It reduces recovery times by keeping the disk-resident database most up-to-date with log entries without requiring clients to **block** until the backup is updated.

Các bản ghi nhật ký thường được áp dụng vào bản sao lưu theo lô. Sau khi lô bản ghi nhật ký được xử lý, bản sao lưu giữ một ảnh chụp (snapshot) của cơ sở dữ liệu tại một thời điểm cụ thể, và nội dung nhật ký tính đến thời điểm đó có thể bị loại bỏ. Quá trình này gọi là tạo điểm kiểm tra (checkpointing). Nó giảm thời gian phục hồi bằng cách giữ cho cơ sở dữ liệu nằm trên đĩa luôn cập nhật nhất với các mục nhật ký mà không cần buộc máy khách phải chờ cho đến khi bản sao lưu được cập nhật.



It is unfair to say that the in-memory database is the equivalent of an on-disk database with a huge **page cache** (see “Buffer Management” on page 81). Even though pages are cached in memory, serialization format and data layout incur additional overhead and do not permit the same degree of optimization that in-memory stores can achieve.

Sẽ là không công bằng nếu nói rằng cơ sở dữ liệu trong bộ nhớ tương đương với cơ sở dữ liệu trên đĩa có một page cache khổng lồ (xem “Quản lý vùng đệm” trang 81). Dù các trang được lưu đệm trong bộ nhớ, định dạng tuần tự hóa và cách bố trí dữ liệu vẫn phát sinh thêm chi phí và không cho phép cùng mức độ tối ưu mà các kho trong bộ nhớ có thể đạt được.

Disk-based databases use specialized storage structures, optimized for disk access. In memory, pointers can be followed comparatively quickly, and random memory access is significantly faster than the random disk access. Disk-based storage structures often have a form of wide and short trees (see “Trees for Disk-Based Storage” on page 28), while memory-based implementations can choose from a larger pool of data structures and perform optimizations that would otherwise be impossible or difficult to implement on disk [MOLINA92] . Similarly, handling **variable-size data** on disk requires special attention, while in memory it’s often a matter of referencing the value with a pointer.

Các cơ sở dữ liệu dựa trên đĩa dùng các cấu trúc lưu trữ chuyên biệt, được tối ưu cho việc truy cập đĩa. Trong bộ nhớ, các con trỏ có thể được lần theo tương đối nhanh, và truy cập bộ nhớ ngẫu nhiên nhanh hơn đáng kể so với truy cập đĩa ngẫu nhiên. Các cấu trúc lưu trữ dựa trên đĩa thường có dạng cây rộng và thấp (xem “Cây cho lưu trữ dựa trên đĩa” trang 28), trong khi các hiện thực dựa trên bộ nhớ có thể chọn từ một tập cấu trúc dữ liệu lớn hơn và thực hiện các tối ưu mà nếu không thì sẽ bất khả thi hoặc khó hiện thực trên đĩa [MOLINA92]. Tương tự, việc xử lý dữ liệu kích thước thay đổi trên đĩa đòi hỏi sự chú ý đặc biệt, trong khi trong bộ nhớ nó thường chỉ là chuyện tham chiếu giá trị bằng một con trỏ.

For some use cases, it is reasonable to assume that an entire dataset is going to fit in memory. Some datasets are bounded by their real-world representations, such as student records for schools, customer records for corporations, or inventory in an online store. Each record takes up not more than a few Kb, and their number is limited.

Với một số tình huống sử dụng, có thể hợp lý khi giả định rằng toàn bộ tập dữ liệu sẽ vừa trong bộ nhớ. Một số tập dữ liệu bị giới hạn bởi cách biểu diễn ngoài đời thực của chúng, chẳng hạn hồ sơ học sinh của trường học, hồ sơ khách hàng của doanh nghiệp, hay hàng tồn kho trong một cửa hàng trực tuyến. Mỗi bản ghi chiếm không quá vài Kb, và số lượng của chúng là có giới hạn.

Column- Versus Row-Oriented DBMS — DBMS hướng cột so với hướng hàng

Most database systems store a set of data records , consisting of columns and rows in tables . Field is an intersection of a column and a row: a single value of some type. Fields belonging to the same column usually have the same data type. For example, if we define a table holding user records, all names would be of the same type and belong to the same column. A collection of values that belong logically to the same record (usually identified by the key) constitutes a row.

Hầu hết các hệ cơ sở dữ liệu lưu một tập các bản ghi dữ liệu (data record), gồm các cột và hàng trong các bảng. Trường (field) là giao điểm của một cột và một hàng: một giá trị đơn của một kiểu nào đó. Các trường thuộc cùng một cột thường có cùng kiểu dữ liệu. Ví dụ, nếu ta định nghĩa một bảng giữ hồ sơ người dùng, mọi cái tên sẽ cùng kiểu và thuộc cùng một cột. Một tập hợp các giá trị về mặt logic cùng thuộc một bản ghi (thường được xác định bởi khóa) tạo thành một hàng.

One of the ways to classify databases is by how the data is stored on disk: row- or column-wise. Tables can be partitioned either horizontally (storing values belonging to the same row together), or vertically (storing values belonging to the same column together). Figure 1-2 depicts this distinction: (a) shows the values partitioned column-wise, and (b) shows the values partitioned row-wise.

Một trong những cách phân loại cơ sở dữ liệu là dựa trên cách dữ liệu được lưu trên đĩa: theo hàng hay theo cột. Các bảng có thể được phân hoạch theo chiều ngang (lưu các giá

trị cùng một hàng với nhau), hoặc theo chiều dọc (lưu các giá trị cùng một cột với nhau). Hình 1-2 minh họa sự phân biệt này: (a) cho thấy các giá trị được phân hoạch theo cột, và (b) cho thấy các giá trị được phân hoạch theo hàng.

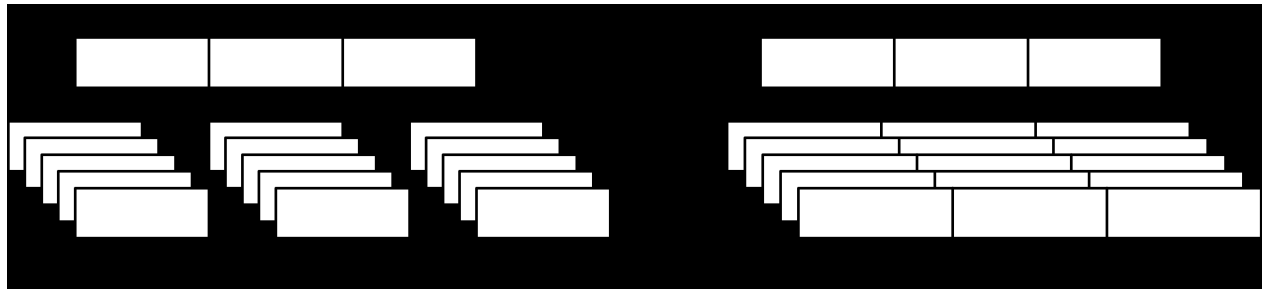


Figure 1-2. Data layout in column- and row-oriented stores

Hình 1-2. Cách bố trí dữ liệu trong kho hướng cột và hướng hàng

Examples of row-oriented database management systems are abundant: MySQL , PostgreSQL , and most of the traditional relational databases. The two pioneer open source **column-oriented** stores are MonetDB and C-Store (C-Store is an open source predecessor to Vertica).

Các ví dụ về hệ quản trị cơ sở dữ liệu hướng hàng rất nhiều: MySQL, PostgreSQL, và hầu hết các cơ sở dữ liệu quan hệ truyền thống. Hai kho hướng cột mã nguồn mở tiên phong là MonetDB và C-Store (C-Store là tiền thân mã nguồn mở của Vertica).

Row-Oriented Data Layout — Cách bố trí dữ liệu hướng hàng

Row-oriented database management systems store data in records or rows . Their layout is quite close to the tabular data representation, where every row has the same set of fields. For example, a row-oriented database can efficiently store user entries, holding names, birth dates, and phone numbers:

Các hệ quản trị cơ sở dữ liệu hướng hàng lưu dữ liệu theo các bản ghi (record) hay hàng (row). Cách bố trí của chúng khá gần với cách biểu diễn dữ liệu dạng bảng, trong đó mỗi hàng có cùng tập trường. Ví dụ, một cơ sở dữ liệu hướng hàng có thể lưu hiệu quả các mục người dùng, gồm tên, ngày sinh và số điện thoại:

ID	Name	Birth Date	Phone Number
10	John	01 Aug 1981	+1 111 222 333
20	Sam	14 Sep 1988	+1 555 888 999
30	Keith	07 Jan 1984	+1 333 444 555

This approach works well for cases where several fields constitute the record (name, birth date, and a phone number) uniquely identified by the key (in this example, a monotonically incremented number). All fields representing a single user record are often read together. When creating records (for example, when the user fills out a registration form), we write them together as well. At the same time, each field can be modified individually.

Cách tiếp cận này hoạt động tốt cho các trường hợp mà nhiều trường tạo thành bản ghi (tên, ngày sinh và số điện thoại) được xác định duy nhất bởi khóa (trong ví dụ này là một số tăng đơn điệu). Mọi trường biểu diễn một bản ghi người dùng đơn lẻ thường được đọc cùng nhau. Khi tạo bản ghi (ví dụ khi người dùng điền vào biểu mẫu đăng ký), ta cũng ghi chúng cùng nhau. Đồng thời, mỗi trường có thể được sửa đổi riêng lẻ.

Since row-oriented stores are most useful in scenarios when we have to access data by row, storing entire rows together improves spatial locality 2 [DENNING68] .

Vì các kho hướng hàng hữu ích nhất trong các tình huống mà ta phải truy cập dữ liệu theo hàng, nên việc lưu toàn bộ các hàng cùng nhau cải thiện tính cục bộ về không gian (spatial locality) 2 [DENNING68].

Because data on a persistent medium such as a disk is typically accessed block-wise (in other words, a minimal unit of disk access is a block), a single block will contain

Bởi vì dữ liệu trên một phương tiện bền vững như đĩa thường được truy cập theo khối (nói cách khác, đơn vị truy cập đĩa tối thiểu là một khối), nên một khối đơn sẽ chứa

2 Spatial locality is one of the Principles of Locality, stating that if a memory location is accessed, its nearby memory locations will be accessed in the near future.

2 Tính cục bộ về không gian là một trong các Nguyên lý về Tính cục bộ (Principles of Locality), phát biểu rằng nếu một vị trí bộ nhớ được truy cập thì các vị trí bộ nhớ lân cận của nó sẽ được truy cập trong tương lai gần.

data for all columns. This is great for cases when we'd like to access an entire user record, but makes queries accessing individual fields of multiple user records (for example, queries fetching only the phone numbers) more expensive, since data for the other fields will be paged in as well.

dữ liệu cho tất cả các cột. Điều này tuyệt vời cho các trường hợp ta muốn truy cập toàn bộ một bản ghi người dùng, nhưng làm cho các truy vấn truy cập các trường riêng lẻ của nhiều bản ghi người dùng (ví dụ các truy vấn chỉ lấy số điện thoại) tốn kém hơn, vì dữ liệu của các trường khác cũng sẽ được nạp trang (paged in) vào theo.

Column-Oriented Data Layout — Cách bố trí dữ liệu hướng cột

Column-oriented database management systems partition data vertically (i.e., by column) instead of storing it in rows. Here, values for the same column are stored contiguously on disk (as opposed to storing rows contiguously as in the previous example). For example, if we store historical stock market prices, price quotes are stored together. Storing values for different columns in separate files or file segments allows efficient queries by column, since they can be read in one pass rather than consuming entire rows and discarding data for columns that weren't queried.

Các hệ quản trị cơ sở dữ liệu hướng cột phân hoạch dữ liệu theo chiều dọc (tức là theo cột) thay vì lưu nó theo các hàng. Ở đây, các giá trị của cùng một cột được lưu liền kề nhau trên đĩa (trái ngược với việc lưu các hàng liền kề như trong ví dụ trước). Ví dụ, nếu ta lưu giá thị trường chứng khoán theo lịch sử, các báo giá được lưu cùng nhau. Việc lưu các giá trị của các cột khác nhau trong các tệp hoặc đoạn tệp riêng biệt cho phép các truy vấn theo cột hiệu quả, vì chúng có thể được đọc trong một lượt thay vì phải nạp toàn bộ hàng rồi loại bỏ dữ liệu của các cột không được truy vấn.

Column-oriented stores are a good fit for analytical workloads that compute aggregates, such as finding trends, computing average values, etc. Processing complex aggregates can be used in cases when logical records have multiple fields, but some of them (in this case, price quotes) have different importance and are often consumed together.

Các kho hướng cột rất phù hợp cho khối lượng công việc phân tích tính toán các phép gộp, chẳng hạn tìm xu hướng, tính giá trị trung bình, v.v. Việc xử lý các phép gộp phức tạp có thể được dùng trong các trường hợp mà các bản ghi logic có nhiều trường, nhưng một số

trong số đó (ở đây là báo giá) có tầm quan trọng khác nhau và thường được tiêu thụ cùng nhau.

From a logical perspective, the data representing stock market price quotes can still be expressed as a table:

Từ góc độ logic, dữ liệu biểu diễn các báo giá thị trường chứng khoán vẫn có thể được biểu diễn dưới dạng một bảng:

ID	Symbol	Date	Price
1	DOW	08 Aug 2018	24,314.65
2	DOW	09 Aug 2018	24,136.16
3	S&P	08 Aug 2018	2,414.45
4	S&P	09 Aug 2018	2,232.32

However, the physical column-based database layout looks entirely different. Values belonging to the same row are stored closely together:

Tuy nhiên, cách bố trí cơ sở dữ liệu vật lý dựa trên cột trông hoàn toàn khác. Các giá trị thuộc cùng một hàng được lưu gần nhau:

Symbol: 1:DOW; 2:DOW; 3:S&P; 4:S&P

Date: 1:08 Aug 2018; 2:09 Aug 2018; 3:08 Aug 2018; 4:09 Aug 2018

Price: 1:24,314.65; 2:24,136.16; 3:2,414.45; 4:2,232.32

To reconstruct data tuples, which might be useful for joins, filtering, and multirow aggregates, we need to preserve some metadata on the column level to identify which data points from other columns it is associated with. If you do this explicitly, each value will have to hold a key, which introduces duplication and increases the amount of stored data. Some column stores use implicit identifiers (virtual IDs) instead and use the position of the value (in other words, its **offset**) to map it back to the related values [ABADI13].

Để tái dựng các bộ dữ liệu (tuple), điều có thể hữu ích cho các phép kết (join), lọc và gộp nhiều hàng, ta cần giữ lại một số siêu dữ liệu ở mức cột để nhận biết những điểm dữ liệu nào từ các cột khác có liên kết với nó. Nếu làm việc này một cách tường minh, mỗi giá trị sẽ phải giữ một khóa, điều này tạo ra sự trùng lặp và làm tăng lượng dữ liệu được lưu. Một số kho hướng cột dùng định danh ngầm định (ID ảo - virtual ID) thay thế và dùng vị trí của giá trị (nói cách khác là độ dời - offset của nó) để ánh xạ ngược lại các giá trị liên quan [ABADI13].

During the last several years, likely due to a rising demand to run complex analytical queries over growing datasets, we've seen many new column-oriented file formats such as Apache Parquet , Apache ORC , RCFile , as well as column-oriented stores, such as Apache Kudu , ClickHouse , and many others [ROY12].

Trong vài năm gần đây, có lẽ do nhu cầu chạy các truy vấn phân tích phức tạp trên các tập dữ liệu ngày càng lớn, ta đã thấy nhiều định dạng tệp hướng cột mới như Apache Parquet, Apache ORC, RCFile, cũng như các kho hướng cột như Apache Kudu, ClickHouse, và nhiều cái khác [ROY12].

Distinctions and Optimizations — Các khác biệt và Tối ưu hóa

It is not sufficient to say that distinctions between row and column stores are only in the way the data is stored. Choosing the data layout is just one of the steps in a series of possible optimizations that columnar stores are targeting.

Nói rằng sự khác biệt giữa kho theo hàng và theo cột chỉ nằm ở cách dữ liệu được lưu là chưa đủ. Việc chọn cách bố trí dữ liệu chỉ là một trong một loạt các bước tối ưu khả dĩ mà các kho dạng cột nhắm tới.

Reading multiple values for the same column in one run significantly improves cache utilization and computational efficiency. On modern CPUs, vectorized instructions can be used to process multiple data points with a single CPU instruction 3 [DREP- PER07] .

Đọc nhiều giá trị của cùng một cột trong một lượt cải thiện đáng kể việc tận dụng cache và hiệu quả tính toán. Trên các CPU hiện đại, các lệnh vector hóa (vectorized instruction) có thể được dùng để xử lý nhiều điểm dữ liệu với một lệnh CPU duy nhất 3 [DREPPER07].

Storing values that have the same data type together (e.g., numbers with other numbers, strings with other strings) offers a better compression ratio. We can use different compression algorithms depending on the data type and pick the most effective compression method for each case.

Việc lưu các giá trị có cùng kiểu dữ liệu cùng nhau (ví dụ số với số, chuỗi với chuỗi) mang lại tỉ lệ nén tốt hơn. Ta có thể dùng các thuật toán nén khác nhau tùy theo kiểu dữ liệu và chọn phương pháp nén hiệu quả nhất cho từng trường hợp.

To decide whether to use a column- or a row-oriented store, you need to understand your access patterns . If the read data is consumed in records (i.e., most or all of the columns are requested) and the workload consists mostly of point queries and range scans, the row-oriented approach is likely to yield better results. If scans span many rows, or compute aggregate over a subset of columns, it is worth considering a column-oriented approach.

Để quyết định nên dùng kho hướng cột hay hướng hàng, bạn cần hiểu các mẫu truy cập (access pattern) của mình. Nếu dữ liệu đọc được tiêu thụ theo bản ghi (tức là hầu hết hoặc tất cả các cột đều được yêu cầu) và khối lượng công việc chủ yếu gồm các truy vấn điểm và quét khoảng, thì cách tiếp cận hướng hàng có khả năng cho kết quả tốt hơn. Nếu các lần quét trải dài qua nhiều hàng, hoặc tính phép gộp trên một tập con các cột, thì đáng cân nhắc cách tiếp cận hướng cột.

Wide Column Stores — Kho lưu cột rộng

Column-oriented databases should not be mixed up with wide column stores , such as BigTable or HBase , where data is represented as a multidimensional map, columns are grouped into column families (usually storing data of the same type), and inside each column family, data is stored row-wise. This layout is best for storing data retrieved by a key or a sequence of keys.

Không nên nhầm lẫn các cơ sở dữ liệu hướng cột với các kho lưu cột rộng (wide column store) như BigTable hay HBase, nơi dữ liệu được biểu diễn dưới dạng một ánh xạ đa chiều, các cột được nhóm thành các họ cột (column family) (thường lưu dữ liệu cùng kiểu), và bên trong mỗi họ cột, dữ liệu được lưu theo hàng. Cách bố trí này phù hợp nhất để lưu dữ liệu được truy xuất theo một khóa hoặc một dãy khóa.

A canonical example from the Bigtable paper [CHANG06] is a Webtable. A Webtable stores snapshots of web page contents, their attributes, and the relations among them at a specific timestamp. Pages are identified by the reversed URL, and all attributes (such as page content and anchors , representing links between pages) are identified by the timestamps at which these snapshots were taken. In a simplified way, it can be represented as a nested map, as Figure 1-3 shows.

Một ví dụ kinh điển từ bài báo Bigtable [CHANG06] là Webtable. Webtable lưu các ảnh chụp nội dung trang web, các thuộc tính của chúng, và các quan hệ giữa chúng tại một

dấu thời gian cụ thể. Các trang được xác định bằng URL đảo ngược, và mọi thuộc tính (như nội dung trang và các neo - anchor, biểu diễn các liên kết giữa các trang) được xác định bằng các dấu thời gian mà tại đó các ảnh chụp được tạo. Theo cách đơn giản hóa, nó có thể được biểu diễn dưới dạng một ánh xạ lồng nhau, như Hình 1-3 minh họa.

3 Vectorized instructions, or Single Instruction Multiple Data (SIMD), describes a class of CPU instructions that perform the same operation on multiple data points.

3 Các lệnh vector hóa, hay Đơn lệnh Đa dữ liệu (Single Instruction Multiple Data - SIMD), mô tả một lớp các lệnh CPU thực hiện cùng một thao tác trên nhiều điểm dữ liệu.

```

{
  "com.cnn.www": {
    contents: {
      t6: html: "<html>..."
      t5: html: "<html>..."
      t3: html: "<html>..."
    }
    anchor: {
      t9: cnnsi.com: "CNN"
      t8: my.look.ca: "CNN.com"
    }
  }
  "com.example.www": {
    contents: {
      t5: html: "<html>..."
    }
    anchor: {}
  }
}

```

Figure 1-3. Conceptual structure of a Wehtable

Hình 1-3. Cấu trúc khái niệm của một Wehtable

Data is stored in a multidimensional sorted map with hierarchical indexes: we can locate the data related to a specific web page by its reversed URL and its contents or anchors by the timestamp. Each row is indexed by its row key. Related columns are grouped together in column families — contents and anchor in this example—which are stored on disk separately. Each column inside a column family is identified by the column key, which is a combination of the column family name and a qualifier (html, cnnsi.com, my.look.ca in this example). Column families store multiple versions of data by timestamp. This layout allows us to quickly locate the higher-level entries (web pages, in this case) and their parameters (versions of content and links to the other pages).

Dữ liệu được lưu trong một ánh xạ đã sắp xếp đa chiều với các chỉ mục phân cấp: ta có thể định vị dữ liệu liên quan đến một trang web cụ thể bằng URL đảo ngược của nó, và nội dung hay các neo của nó bằng dấu thời gian. Mỗi hàng được lập chỉ mục bởi khóa hàng (row key) của nó. Các cột liên quan được nhóm với nhau trong các họ cột — contents và anchor trong ví dụ này — vốn được lưu trên đĩa riêng biệt. Mỗi cột bên trong một họ cột được xác định bởi khóa cột (column key), là sự kết hợp của tên họ cột và một bộ định tính (qualifier) (html, cnnsi.com, my.look.ca trong ví dụ này). Các họ cột lưu nhiều phiên bản dữ liệu theo dấu thời gian. Cách bố trí này cho phép ta định vị nhanh các mục ở mức cao hơn (các trang web trong trường hợp này) và các tham số của chúng (các phiên bản nội dung và liên kết tới các trang khác).

While it is useful to understand the conceptual representation of wide column stores, their physical layout is somewhat different. A schematic representation of the data layout in column families is shown in Figure 1-4: column families are stored separately, but in each column family, the data belonging to the same key is stored together.

Mặc dù việc hiểu cách biểu diễn khái niệm của các kho lưu cột rộng là hữu ích, cách bố trí vật lý của chúng có phần khác biệt. Một biểu diễn sơ đồ về cách bố trí dữ liệu trong các họ cột được trình bày trong Hình 1-4: các họ cột được lưu riêng biệt, nhưng trong mỗi họ cột, dữ liệu thuộc cùng một khóa được lưu cùng nhau.

Column Family: contents

Row Key	Timestamp	Qualifier	Value
com.cnn.www	t3	html	"<html>..."
com.cnn.www	t5	html	"<html>..."
com.cnn.www	t6	html	"<html>..."
com.example.www	t5	html	"<html>..."

Column Family: anchor

Row Key	Timestamp	Qualifier	Value
com.cnn.www	t8	cnnsi.com	"CNN"
com.cnn.www	t5	my.look.ca	"CNN.com"

Figure 1-4. Physical structure of a Webtable

Hình 1-4. Cấu trúc vật lý của một Webtable

Data Files and Index Files — Tập dữ liệu và Tập chỉ mục

The primary goal of a database system is to store data and to allow quick access to it. But how is the data organized? Why do we need a database management system and not just a bunch of files? How does file organization improve efficiency?

Mục tiêu chính của một hệ cơ sở dữ liệu là lưu dữ liệu và cho phép truy cập nhanh tới nó. Nhưng dữ liệu được tổ chức như thế nào? Tại sao ta cần một hệ quản trị cơ sở dữ liệu chứ không chỉ là một mớ tệp? Việc tổ chức tệp cải thiện hiệu quả ra sao?

Database systems do use files for storing the data, but instead of relying on filesystem hierarchies of directories and files for locating records, they compose files using implementation-specific formats. The main reasons to use specialized file organization over flat files are:

Các hệ cơ sở dữ liệu quả thực có dùng tệp để lưu dữ liệu, nhưng thay vì dựa vào các phân cấp thư mục và tệp của hệ thống tệp để định vị bản ghi, chúng tạo ra các tệp bằng các định dạng riêng theo từng hiện thực. Các lý do chính để dùng cách tổ chức tệp chuyên biệt thay vì các tệp phẳng là:

Storage efficiency Files are organized in a way that minimizes storage overhead per stored data record.

Hiệu quả lưu trữ Các tệp được tổ chức theo cách giảm thiểu chi phí lưu trữ phụ trội cho mỗi bản ghi dữ liệu được lưu.

Access efficiency Records can be located in the smallest possible number of steps.

Hiệu quả truy cập Các bản ghi có thể được định vị trong số bước ít nhất có thể.

Update efficiency Record updates are performed in a way that minimizes the number of changes on disk.

Hiệu quả cập nhật Các cập nhật bản ghi được thực hiện theo cách giảm thiểu số lượng thay đổi trên đĩa.

Database systems store data records , consisting of multiple fields, in tables, where each table is usually represented as a separate file. Each record in the table can be looked up using a search key . To locate a record, database systems use indexes : auxiliary data structures that allow it to efficiently locate data records without scanning an entire table on every access. Indexes are built using a subset of fields identifying the record.

Các hệ cơ sở dữ liệu lưu các bản ghi dữ liệu, gồm nhiều trường, trong các bảng, trong đó mỗi bảng thường được biểu diễn như một tệp riêng. Mỗi bản ghi trong bảng có thể được tra cứu bằng một khóa tìm kiếm (search key). Để định vị một bản ghi, các hệ cơ sở dữ liệu dùng các chỉ mục (index): các cấu trúc dữ liệu phụ trợ cho phép định vị hiệu quả các bản ghi dữ liệu mà không cần quét toàn bộ bảng ở mỗi lần truy cập. Các chỉ mục được xây dựng bằng một tập con các trường xác định bản ghi.

A database system usually separates data files and index files : data files store data records, while index files store record metadata and use it to locate records in data files. Index files are typically smaller than the data files. Files are partitioned into pages , which typically have the size of a single or multiple disk blocks. Pages can be organized as sequences of records or as a slotted pages (see “Slotted Pages” on page 52).

Một hệ cơ sở dữ liệu thường tách riêng tệp dữ liệu và tệp chỉ mục: tệp dữ liệu lưu các bản ghi dữ liệu, còn tệp chỉ mục lưu siêu dữ liệu của bản ghi và dùng nó để định vị các bản ghi trong tệp dữ liệu. Tệp chỉ mục thường nhỏ hơn tệp dữ liệu. Các tệp được phân hoạch

thành các trang (page), thường có kích thước bằng một hoặc nhiều khối đĩa. Các trang có thể được tổ chức dưới dạng các dãy bản ghi hoặc dưới dạng các trang có khe (slotted page) (xem “Trang có khe” trang 52).

New records (insertions) and updates to the existing records are represented by key/ value pairs. Most modern storage systems do not delete data from pages explicitly. Instead, they use deletion markers (also called tombstones), which contain deletion metadata, such as a key and a timestamp. Space occupied by the records shadowed by their updates or deletion markers is reclaimed during **garbage collection**, which reads the pages, writes the live (i.e., nonshadowed) records to the new place, and discards the shadowed ones.

Các bản ghi mới (chèn) và các cập nhật vào bản ghi hiện có được biểu diễn bằng các cặp khóa/giá trị. Hầu hết các hệ lưu trữ hiện đại không xóa dữ liệu khỏi trang một cách tường minh. Thay vào đó, chúng dùng các dấu xóa (deletion marker, còn gọi là tombstone), chứa siêu dữ liệu xóa như khóa và một dấu thời gian. Không gian bị chiếm bởi các bản ghi đã bị che khuất bởi bản cập nhật hoặc dấu xóa của chúng được thu hồi trong quá trình thu gom rác (garbage collection), nơi đọc các trang, ghi các bản ghi còn sống (tức là không bị che khuất) sang vị trí mới, và loại bỏ những bản ghi bị che khuất.

Data Files — Tập dữ liệu

Data files (sometimes called primary files) can be implemented as index-organized tables (IOT), heap-organized tables (heap files), or hash-organized tables (hashed files).

Các tập dữ liệu (đôi khi gọi là tập chính - primary file) có thể được hiện thực dưới dạng bảng tổ chức theo chỉ mục (IOT), bảng tổ chức theo heap (tập heap), hoặc bảng tổ chức theo băm (tập băm).

Records in heap files are not required to follow any particular order, and most of the time they are placed in a write order. This way, no additional work or file reorganization is required when new pages are appended. Heap files require additional index structures, pointing to the locations where data records are stored, to make them searchable.

Các bản ghi trong tập heap không bắt buộc tuân theo bất kỳ thứ tự cụ thể nào, và hầu hết thời gian chúng được đặt theo thứ tự ghi. Theo cách này, không cần thêm công việc hay tái tổ chức tập khi các trang mới được nối thêm. Các tập heap đòi hỏi các cấu trúc chỉ mục bổ sung, trỏ tới những vị trí nơi các bản ghi dữ liệu được lưu, để khiến chúng có thể tìm kiếm được.

In hashed files, records are stored in buckets, and the hash value of the key determines which bucket a record belongs to. Records in the bucket can be stored in append order or sorted by key to improve lookup speed.

Trong các tập băm, các bản ghi được lưu trong các xô (bucket), và giá trị băm của khóa quyết định bản ghi thuộc về xô nào. Các bản ghi trong xô có thể được lưu theo thứ tự nối thêm hoặc được sắp xếp theo khóa để cải thiện tốc độ tra cứu.

Index-organized tables (IOTs) store data records in the index itself. Since records are stored in key order, range scans in IOTs can be implemented by sequentially scanning its contents.

Các bảng tổ chức theo chỉ mục (IOT) lưu các bản ghi dữ liệu ngay trong chính chỉ mục. Vì các bản ghi được lưu theo thứ tự khóa, các phép quét khoảng trong IOT có thể được hiện thực bằng cách quét tuần tự nội dung của nó.

Storing data records in the index allows us to reduce the number of disk seeks by at least one, since after traversing the index and locating the searched key, we do not have to address a separate file to find the associated data record.

Việc lưu các bản ghi dữ liệu trong chỉ mục cho phép ta giảm số lần tìm kiếm trên đĩa (disk seek) ít nhất một lần, vì sau khi duyệt chỉ mục và định vị được khóa cần tìm, ta không phải truy đến một tệp riêng để tìm bản ghi dữ liệu liên quan.

When records are stored in a separate file, index files hold data entries , uniquely identifying data records and containing enough information to locate them in the **data file**. For example, we can store file offsets (sometimes called row locators), locations of data records in the data file, or bucket IDs in the case of hash files. In index-organized tables, data entries hold actual data records.

Khi các bản ghi được lưu trong một tệp riêng, các tệp chỉ mục giữ các mục dữ liệu (data entry), xác định duy nhất các bản ghi dữ liệu và chứa đủ thông tin để định vị chúng trong tệp dữ liệu. Ví dụ, ta có thể lưu các độ dời tệp (đôi khi gọi là bộ định vị hàng - row locator), các vị trí của bản ghi dữ liệu trong tệp dữ liệu, hoặc các ID xô trong trường hợp tệp băm. Trong các bảng tổ chức theo chỉ mục, các mục dữ liệu giữ chính các bản ghi dữ liệu thực sự.

Index Files — Tệp chỉ mục

An index is a structure that organizes data records on disk in a way that facilitates efficient retrieval operations. Index files are organized as specialized structures that map keys to locations in data files where the records identified by these keys (in the case of heap files) or primary keys (in the case of index-organized tables) are stored.

Một chỉ mục là một cấu trúc tổ chức các bản ghi dữ liệu trên đĩa theo cách tạo thuận lợi cho các thao tác truy xuất hiệu quả. Các tệp chỉ mục được tổ chức dưới dạng các cấu trúc chuyên biệt ánh xạ các khóa tới các vị trí trong tệp dữ liệu nơi lưu các bản ghi được xác định bởi các khóa này (trong trường hợp tệp heap) hoặc các khóa chính (trong trường hợp bảng tổ chức theo chỉ mục).

An index on a primary (data) file is called the **primary index** . However, in most cases we can also assume that the primary index is built over a primary key or a set of keys identified as primary. All other indexes are called secondary .

Một chỉ mục trên tệp chính (tệp dữ liệu) được gọi là chỉ mục chính (primary index). Tuy nhiên, trong hầu hết trường hợp ta cũng có thể giả định rằng chỉ mục chính được xây dựng trên một khóa chính hoặc một tập khóa được xác định là khóa chính. Mọi chỉ mục khác được gọi là chỉ mục phụ (secondary).

Secondary indexes can point directly to the data record, or simply store its primary key. A pointer to a data record can hold an offset to a **heap file** or an **index-organized table**. Multiple secondary indexes can point to the same record, allowing a single data record to be identified by different fields and located through different indexes. While primary index files hold a unique entry per search key, secondary indexes may hold several entries per search key [MOLINA08] .

Các chỉ mục phụ có thể trở trực tiếp tới bản ghi dữ liệu, hoặc đơn giản là lưu khóa chính của nó. Một con trỏ tới bản ghi dữ liệu có thể giữ một độ dời tới tệp heap hoặc một bảng tổ chức theo chỉ mục. Nhiều chỉ mục phụ có thể trở tới cùng một bản ghi, cho phép một bản ghi dữ liệu đơn lẻ được xác định bởi các trường khác nhau và được định vị qua các chỉ mục khác nhau. Trong khi các tệp chỉ mục chính giữ một mục duy nhất cho mỗi khóa tìm kiếm, các chỉ mục phụ có thể giữ nhiều mục cho mỗi khóa tìm kiếm [MOLINA08].

If the order of data records follows the search key order, this index is called clustered (also known as clustering). Data records in the clustered case are usually stored in the same file or in a clustered file, where the key order is preserved. If the data is stored in a separate file, and its order does not follow the key order, the index is called nonclustered (sometimes called unclustered).

Nếu thứ tự của các bản ghi dữ liệu tuân theo thứ tự khóa tìm kiếm, chỉ mục này được gọi là gom cụm (clustered, còn gọi là clustering). Các bản ghi dữ liệu trong trường hợp gom cụm thường được lưu trong cùng một tệp hoặc trong một tệp gom cụm (clustered file), nơi thứ tự khóa được giữ nguyên. Nếu dữ liệu được lưu trong một tệp riêng, và thứ tự của nó không tuân theo thứ tự khóa, chỉ mục được gọi là không gom cụm (nonclustered, đôi khi gọi là unclustered).

Figure 1-5 shows the difference between the two approaches:

Hình 1-5 cho thấy sự khác biệt giữa hai cách tiếp cận:

- a) Two indexes reference data entries directly from **secondary index** files.
- b) A secondary index goes through the indirection layer of a primary index to locate the data entries.
 - a) Hai chỉ mục tham chiếu trực tiếp các mục dữ liệu từ các tệp chỉ mục phụ.
 - b) Một chỉ mục phụ đi qua lớp gián tiếp của một chỉ mục chính để định vị các mục dữ liệu.

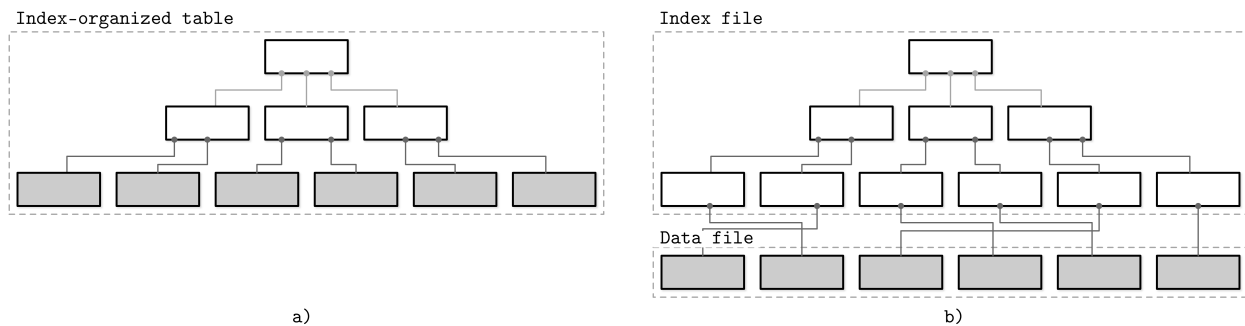
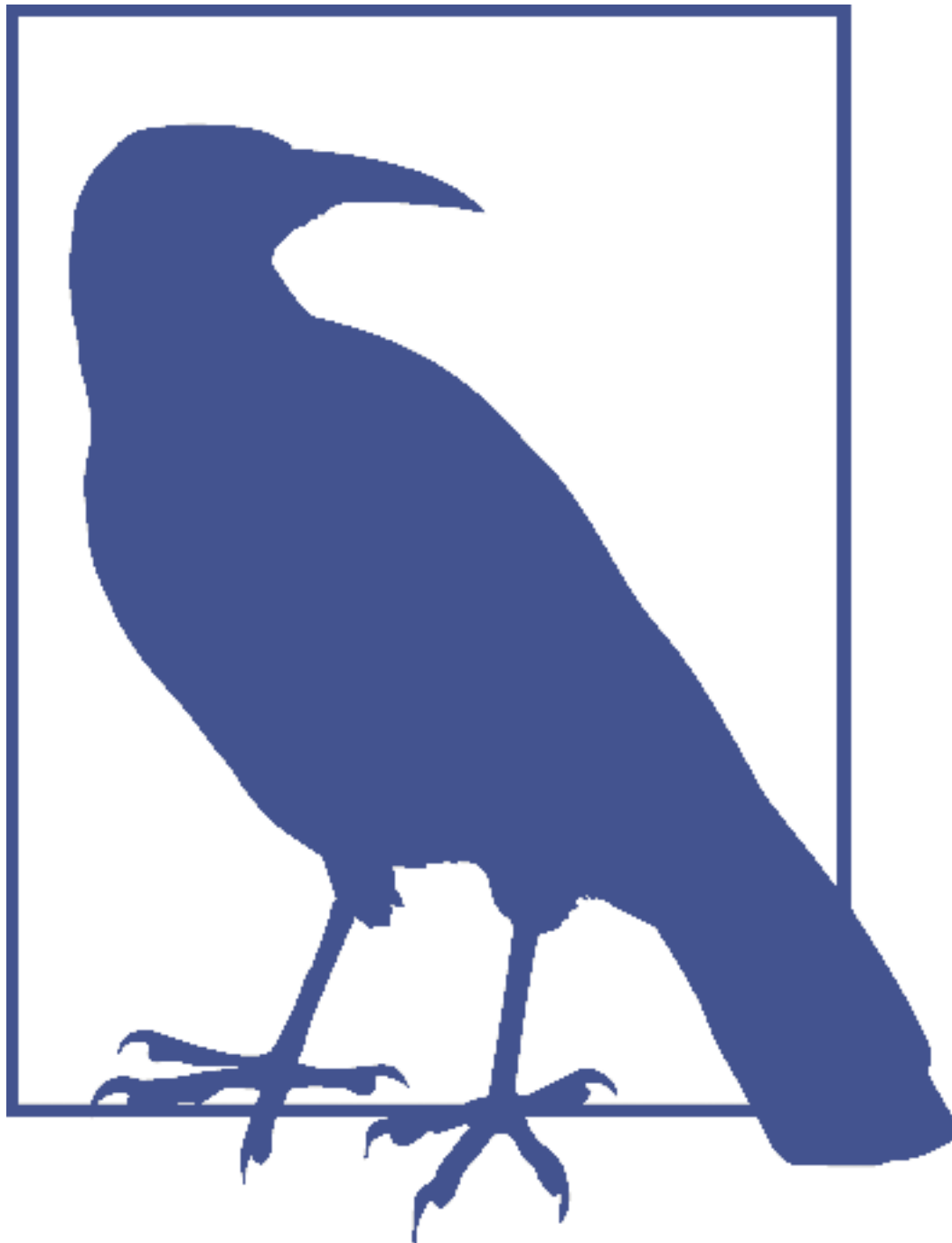


Figure 1-5. Storing data records in an **index file** versus storing offsets to the data file (index segments shown in white; segments holding data records shown in gray)

Hình 1-5. Lưu các bản ghi dữ liệu trong một tệp chỉ mục so với lưu các độ dời tới tệp dữ liệu (các đoạn chỉ mục hiển thị màu trắng; các đoạn giữ bản ghi dữ liệu hiển thị màu xám)



Index-organized tables store information in index order and are clustered by definition. Primary indexes are most often clustered. Secondary indexes are nonclustered by definition, since they're used to facilitate access by keys other than the primary one. Clustered indexes can be both index-organized or have separate index and data files.

Các bảng tổ chức theo chỉ mục lưu thông tin theo thứ tự chỉ mục và theo định nghĩa là gom cụm. Các chỉ mục chính hầu hết là gom cụm. Các chỉ mục phụ theo định nghĩa là không gom cụm, vì chúng được dùng để tạo thuận lợi cho việc truy cập theo các khóa khác khóa chính. Các chỉ mục gom cụm có thể vừa là tổ chức theo chỉ mục, vừa có thể có các tệp chỉ mục và dữ liệu riêng biệt.

Many database systems have an inherent and explicit primary key , a set of columns that uniquely identify the database record. In cases when the primary key is not specified, the storage engine can create an implicit primary key (for example, MySQL InnoDB adds a new auto-increment column and fills in its values automatically).

Nhiều hệ cơ sở dữ liệu có một khóa chính (primary key) cố hữu và tường minh, một tập các cột xác định duy nhất bản ghi cơ sở dữ liệu. Trong các trường hợp không chỉ định khóa chính, bộ máy lưu trữ có thể tạo một khóa chính ngầm định (ví dụ MySQL InnoDB thêm một cột tự tăng mới và tự động điền các giá trị của nó).

This terminology is used in different kinds of database systems: relational database systems (such as MySQL and PostgreSQL), Dynamo-based NoSQL stores (such as Apache Cassandra and Riak), and document stores (such as MongoDB). There can be some project-specific naming, but most often there's a clear mapping to this terminology.

Thuật ngữ này được dùng trong nhiều loại hệ cơ sở dữ liệu khác nhau: các hệ cơ sở dữ liệu quan hệ (như MySQL và PostgreSQL), các kho NoSQL dựa trên Dynamo (như Apache Cassandra và Riak), và các kho tài liệu (như MongoDB). Có thể có cách đặt tên riêng theo từng dự án, nhưng hầu hết thời gian đều có một ánh xạ rõ ràng tới thuật ngữ này.

Primary Index as an Indirection — Chỉ mục chính như một lớp gián tiếp

There are different opinions in the database community on whether data records should be referenced directly (through file offset) or via the primary key index. 4

Có những quan điểm khác nhau trong cộng đồng cơ sở dữ liệu về việc liệu các bản ghi dữ liệu nên được tham chiếu trực tiếp (qua độ dời tệp) hay qua chỉ mục khóa chính.⁴

Both approaches have their pros and cons and are better discussed in the scope of a complete implementation. By referencing data directly, we can reduce the number of disk seeks, but have to pay a cost of updating the pointers whenever the record is updated or relocated during a maintenance process. Using indirection in the form of a primary index allows us to reduce the cost of pointer updates, but has a higher cost on a read path.

Cả hai cách tiếp cận đều có ưu và nhược điểm, và tốt hơn nên bàn trong phạm vi một hiện thực hoàn chỉnh. Bằng cách tham chiếu dữ liệu trực tiếp, ta có thể giảm số lần tìm kiếm trên đĩa, nhưng phải trả giá bằng việc cập nhật các con trỏ mỗi khi bản ghi được cập nhật hoặc bị di dời trong một quá trình bảo trì. Việc dùng cơ chế gián tiếp dưới dạng một chỉ mục chính cho phép ta giảm chi phí cập nhật con trỏ, nhưng có chi phí cao hơn trên đường đọc.

Updating just a couple of indexes might work if the workload mostly consists of reads, but this approach does not work well for write-heavy workloads with multiple indexes. To reduce the costs of pointer updates, instead of payload offsets, some implementations use primary keys for indirection. For example, MySQL InnoDB uses a primary index and performs two lookups: one in the secondary index, and one in a primary index when performing a query [TARIQ11] . This adds an overhead of a primary index lookup instead of following the offset directly from the secondary index.

Việc chỉ cập nhật một vài chỉ mục có thể ổn nếu khối lượng công việc chủ yếu gồm các lần đọc, nhưng cách tiếp cận này không hoạt động tốt cho các khối lượng công việc nặng về ghi với nhiều chỉ mục. Để giảm chi phí cập nhật con trỏ, thay vì dùng độ dời của payload, một số hiện thực dùng khóa chính cho việc gián tiếp. Ví dụ, MySQL InnoDB dùng một chỉ

mục chính và thực hiện hai lần tra cứu: một trong chỉ mục phụ, và một trong chỉ mục chính khi thực hiện một truy vấn [TARIQ11]. Điều này thêm chi phí của một lần tra cứu chỉ mục chính thay vì lần theo độ dời trực tiếp từ chỉ mục phụ.

Figure 1-6 shows how the two approaches are different:

Hình 1-6 cho thấy hai cách tiếp cận khác nhau như thế nào:

- a) Two indexes reference data entries directly from secondary index files.
- b) A secondary index goes through the indirection layer of a primary index to locate the data entries.
 - a) Hai chỉ mục tham chiếu trực tiếp các mục dữ liệu từ các tệp chỉ mục phụ.
 - b) Một chỉ mục phụ đi qua lớp gián tiếp của một chỉ mục chính để định vị các mục dữ liệu.

4 The original post that has stirred up the discussion was controversial and one-sided, but you can refer to the presentation comparing MySQL and PostgreSQL index and storage formats , which references the original source as well.

4 Bài viết gốc khơi mào cuộc thảo luận này gây tranh cãi và phiến diện, nhưng bạn có thể tham khảo bài trình bày so sánh các định dạng chỉ mục và lưu trữ của MySQL và PostgreSQL, vốn cũng dẫn nguồn gốc.

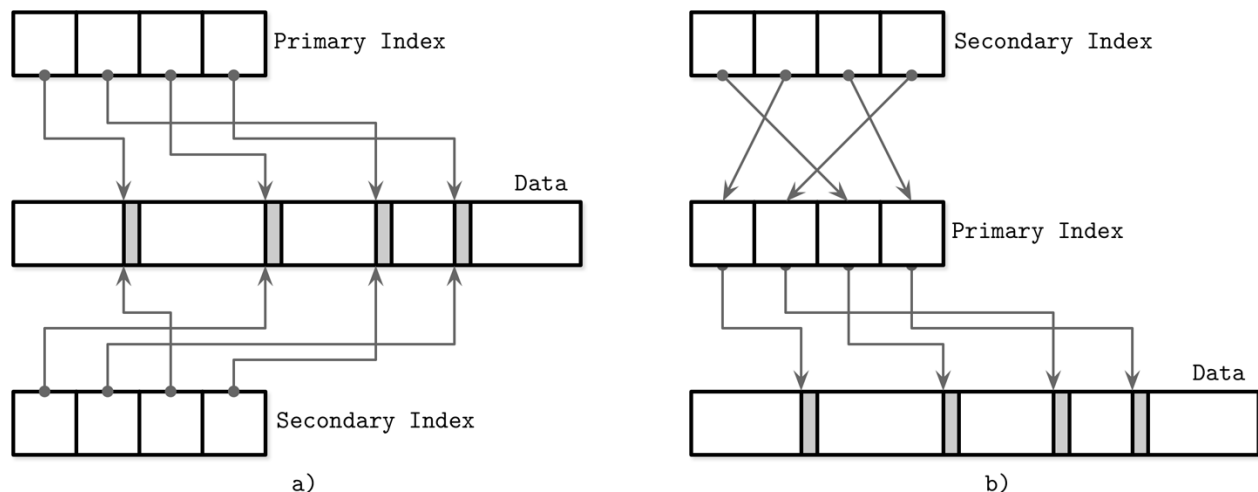


Figure 1-6. Referencing data tuples directly (a) versus using a primary index as indirection (b)

Hình 1-6. Tham chiếu các bộ dữ liệu trực tiếp (a) so với dùng một chỉ mục chính như một lớp gián tiếp (b)

It is also possible to use a hybrid approach and store both data file offsets and primary keys. First, you check if the data offset is still valid and pay the extra cost of going through the primary key index if it has changed, updating the index file after finding a new offset.

Cũng có thể dùng một cách tiếp cận lai và lưu cả độ dời tệp dữ liệu lẫn khóa chính. Đầu tiên, bạn kiểm tra xem độ dời dữ liệu có còn hợp lệ không và trả thêm chi phí đi qua chỉ mục khóa chính nếu nó đã thay đổi, cập nhật tệp chỉ mục sau khi tìm được độ dời mới.

Buffering, Immutability, and Ordering — Đệm, Tính bất biến và Sắp thứ tự

A storage engine is based on some data structure. However, these structures do not describe the semantics of caching, recovery, transactionality, and other things that storage engines add on top of them.

Một bộ máy lưu trữ dựa trên một cấu trúc dữ liệu nào đó. Tuy nhiên, các cấu trúc này không mô tả ngữ nghĩa của việc lưu đệm, phục hồi, tính giao dịch, và những thứ khác mà các bộ máy lưu trữ bổ sung lên trên chúng.

In the next chapters, we will start the discussion with B-Trees (see “Ubiquitous B- Trees” on page 33) and try to understand why there are so many **B-Tree** variants, and why new database storage structures keep emerging.

Trong các chương tiếp theo, ta sẽ bắt đầu thảo luận với B-Tree (xem “B-Tree phổ biến khắp nơi” trang 33) và cố gắng hiểu tại sao lại có quá nhiều biến thể B-Tree, và tại sao các cấu trúc lưu trữ cơ sở dữ liệu mới vẫn liên tục xuất hiện.

Storage structures have three common variables: they use buffering (or avoid using it), use **immutable** (or mutable) files, and store values in order (or out of order). Most of the distinctions and optimizations in storage structures discussed in this book are related to one of these three concepts.

Các cấu trúc lưu trữ có ba biến chung: chúng dùng đệm (buffering) (hoặc tránh dùng nó), dùng tệp bất biến (hoặc khả biến), và lưu các giá trị theo thứ tự (hoặc không theo thứ tự). Hầu hết các khác biệt và tối ưu trong các cấu trúc lưu trữ được bàn trong cuốn sách này đều liên quan đến một trong ba khái niệm này.

Buffering This defines whether or not the storage structure chooses to collect a certain amount of data in memory before putting it on disk. Of course, every on-disk structure has to use buffering to some degree, since the smallest unit of data transfer to and from the disk is a block , and it is desirable to write full blocks. Here, we’re talking about avoidable buffering, something storage engine implementers choose to do. One of the first optimizations we discuss in this book is adding in-memory buffers to B-Tree nodes to amortize I/O costs (see “Lazy B- Trees” on page 114). However, this is not the only way we can apply buffering.

Đệm Khái niệm này xác định liệu cấu trúc lưu trữ có chọn thu thập một lượng dữ liệu nhất định trong bộ nhớ trước khi đưa nó lên đĩa hay không. Tất nhiên, mọi cấu trúc trên đĩa đều phải dùng đệm ở một mức độ nào đó, vì đơn vị truyền dữ liệu nhỏ nhất đến và từ đĩa là một khối, và việc ghi các khối đầy đủ là điều mong muốn. Ở đây, ta đang nói về việc đệm có thể tránh được, điều mà các nhà hiện thực bộ máy lưu trữ chọn làm. Một trong những tối ưu đầu tiên ta bàn trong cuốn sách này là thêm các vùng đệm trong bộ nhớ vào các nút B-Tree để khấu hao chi phí I/O (xem “Lazy B-Tree” trang 114). Tuy nhiên, đây không phải là cách duy nhất ta có thể áp dụng việc đệm.

For example, two-component LSM Trees (see “Two-component **LSM Tree**” on page 132), despite their similarities with B-Trees, use buffering in an entirely different way, and combine buffering with immutability.

Ví dụ, LSM Tree hai thành phần (xem “LSM Tree hai thành phần” trang 132), bất chấp những điểm tương đồng với B-Tree, lại dùng đệm theo một cách hoàn toàn khác, và kết hợp đệm với tính bất biến.

Mutability (or immutability) This defines whether or not the storage structure reads parts of the file, updates them, and writes the updated results at the same location in the file. Immutable structures are append-only : once written, file contents are not modified. Instead, modifications are appended to the end of the file. There are other ways to implement immutability. One of them is **copy-on-write** (see “Copy-on-Write” on page 112), where the modified page, holding the updated version of the record, is written to the new location in the file, instead of its original location. Often the distinction between LSM and B-Trees is drawn as immutable against in-place update storage, but there are structures (for example, “Bw-Trees” on page 120) that are inspired by B-Trees but are immutable.

Tính khả biến (hoặc bất biến) Khái niệm này xác định liệu cấu trúc lưu trữ có đọc các phần của tệp, cập nhật chúng, rồi ghi kết quả đã cập nhật vào cùng vị trí trong tệp hay không. Các cấu trúc bất biến là chỉ-nối-thêm (append-only): một khi đã ghi, nội dung tệp không bị sửa đổi. Thay vào đó, các sửa đổi được nối thêm vào cuối tệp. Có những cách khác để hiện thực tính bất biến. Một trong số đó là sao-khi-ghi (copy-on-write) (xem “Sao khi ghi” trang 112), nơi trang đã sửa đổi, giữ phiên bản đã cập nhật của bản ghi, được ghi vào vị trí mới trong tệp, thay vì vị trí gốc của nó. Thường thì sự phân biệt giữa LSM và B-Tree được vạch ra dưới dạng lưu trữ bất biến đối lập với lưu trữ cập nhật tại chỗ, nhưng có những cấu trúc (ví dụ “Bw-Tree” trang 120) lấy cảm hứng từ B-Tree nhưng lại là bất biến.

Ordering This is defined as whether or not the data records are stored in the key order in the pages on disk. In other words, the keys that sort closely are stored in contiguous segments on disk. Ordering often defines whether or not we can efficiently scan the range of records, not only locate the individual data records. Storing data out of order (most often, in insertion order) opens up for some write-time optimizations. For example, Bitcask (see “Bitcask” on page 153) and WiscKey (see “WiscKey” on page 154) store data records directly in append-only files.

Sắp thứ tự Khái niệm này được định nghĩa là liệu các bản ghi dữ liệu có được lưu theo thứ tự khóa trong các trang trên đĩa hay không. Nói cách khác, các khóa sắp xếp gần nhau được lưu trong các đoạn liên kề trên đĩa. Việc sắp thứ tự thường xác định liệu ta có thể quét hiệu quả một khoảng các bản ghi hay không, chứ không chỉ định vị các bản ghi dữ liệu riêng lẻ. Việc lưu dữ liệu không theo thứ tự (hầu hết thời gian là theo thứ tự chèn) mở ra một số tối ưu tại thời điểm ghi. Ví dụ, Bitcask (xem “Bitcask” trang 153) và WiscKey (xem “WiscKey” trang 154) lưu các bản ghi dữ liệu trực tiếp trong các tệp chỉ-nối-thêm.

Of course, a brief discussion of these three concepts is not enough to show their power, and we’ll continue this discussion throughout the rest of the book.

Tất nhiên, một thảo luận ngắn gọn về ba khái niệm này là chưa đủ để cho thấy sức mạnh của chúng, và ta sẽ tiếp tục thảo luận này xuyên suốt phần còn lại của cuốn sách.

Summary — Tóm tắt

In this chapter, we’ve discussed the architecture of a database management system and covered its primary components.

Trong chương này, ta đã thảo luận về kiến trúc của một hệ quản trị cơ sở dữ liệu và bao quát các thành phần chính của nó.

To highlight the importance of disk-based structures and their difference from in-memory ones, we discussed memory- and disk-based stores. We came to the conclusion that disk-based structures are important for both types of stores, but are used for different purposes.

Để làm nổi bật tầm quan trọng của các cấu trúc dựa trên đĩa và sự khác biệt của chúng so với các cấu trúc trong bộ nhớ, ta đã thảo luận về các kho dựa trên bộ nhớ và dựa trên đĩa. Ta đi đến kết luận rằng các cấu trúc dựa trên đĩa quan trọng đối với cả hai loại kho, nhưng được dùng cho các mục đích khác nhau.

To understand how access patterns influence database system design, we discussed column- and row-oriented database management systems and the primary factors that set them apart from each other. To start a conversation about how the data is stored, we covered data and index files.

Để hiểu cách các mẫu truy cập ảnh hưởng đến thiết kế hệ cơ sở dữ liệu, ta đã thảo luận về các hệ quản trị cơ sở dữ liệu hướng cột và hướng hàng cùng các yếu tố chính phân biệt chúng với nhau. Để bắt đầu cuộc trò chuyện về cách dữ liệu được lưu, ta đã bao quát các tệp dữ liệu và tệp chỉ mục.

Lastly, we introduced three core concepts: buffering, immutability, and ordering. We will use them throughout this book to highlight properties of the storage engines that use them.

Cuối cùng, ta đã giới thiệu ba khái niệm cốt lõi: đệm, tính bất biến và sắp thứ tự. Ta sẽ dùng chúng xuyên suốt cuốn sách này để làm nổi bật các thuộc tính của các bộ máy lưu trữ sử dụng chúng.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Database architecture Hellerstein, Joseph M., Michael Stonebraker, and James Hamilton. 2007. "Architecture of a Database System." *Foundations and Trends in Databases* 1, no. 2 (February): 141-259. <https://doi.org/10.1561/19000000002>.

Column-oriented DBMS Abadi, Daniel, Peter Boncz, Stavros Harizopoulos, Stratos Idreos, and Samuel Madden. 2013. *The Design and Implementation of Modern Column-Oriented Database Systems*. Hanover, MA: Now Publishers Inc.

In-memory DBMS Faerber, Frans, Alfons Kemper, and Per-Åke Alfons. 2017. *Main Memory Database Systems*. Hanover, MA: Now Publishers Inc.

CHAPTER 2 B-Tree Basics — Chương 2: Kiến thức cơ bản về B-Tree

In the previous chapter, we separated storage structures in two groups: mutable and **immutable** ones, and identified **immutability** as one of the core concepts influencing their design and implementation. Most of the mutable storage structures use an in-place update mechanism. During insert, delete, or update operations, data records are updated directly in their locations in the target file.

Trong chương trước, chúng ta đã chia các cấu trúc lưu trữ thành hai nhóm: nhóm có thể thay đổi (mutable) và nhóm bất biến (immutable), và xác định tính bất biến (immutability) là một trong những khái niệm cốt lõi ảnh hưởng đến thiết kế và cách hiện thực của chúng. Hầu hết các cấu trúc lưu trữ có thể thay đổi đều dùng cơ chế cập nhật tại chỗ (in-place update). Trong các thao tác chèn, xóa hoặc cập nhật, bản ghi dữ liệu được cập nhật trực tiếp ngay tại vị trí của chúng trong tệp đích.

Storage engines often allow multiple versions of the same data record to be present in the database; for example, when using multiversion **concurrency control** (see “Multiversion Concurrency Control” on **page 99**) or **slotted page** organization (see “Slotted Pages” on page 52). For the sake of simplicity, for now we assume that each key is associated only with one data record, which has a unique location.

Các bộ máy lưu trữ (storage engine) thường cho phép nhiều phiên bản của cùng một bản ghi dữ liệu cùng tồn tại trong cơ sở dữ liệu; ví dụ, khi sử dụng điều khiển tương tranh đa phiên bản (MVCC) (xem “Multiversion Concurrency Control” ở trang 99) hoặc tổ chức trang có khe (slotted page) (xem “Slotted Pages” ở trang 52). Để cho đơn giản, ở thời điểm này ta giả định rằng mỗi khóa chỉ gắn với một bản ghi dữ liệu duy nhất, có một vị trí duy nhất.

One of the most popular storage structures is a **B-Tree**. Many open source database systems are B-Tree based, and over the years they’ve proven to cover the majority of use cases.

Một trong những cấu trúc lưu trữ phổ biến nhất là B-Tree. Nhiều hệ cơ sở dữ liệu nguồn mở được xây dựng dựa trên B-Tree, và qua nhiều năm chúng đã chứng minh là bao phủ được phần lớn các trường hợp sử dụng.

B-Trees are not a recent invention: they were introduced by Rudolph Bayer and Edward M. McCreight back in 1971 and gained popularity over the years. By 1979, there were already quite a few variants of B-Trees. Douglas Comer collected and systematized some of them [COMER79] .

B-Tree không phải là một phát minh gần đây: chúng được giới thiệu bởi Rudolph Bayer và Edward M. McCreight từ năm 1971 và dần trở nên phổ biến qua các năm. Đến năm 1979,

đã có khá nhiều biến thể của B-Tree. Douglas Comer đã thu thập và hệ thống hóa một số trong số đó [COMER79].

Before we dive into B-Trees, let's first talk about why we should consider alternatives to traditional search trees, such as, for example, binary search trees, 2-3-Trees, and AVL Trees [KNUTH98] . For that, let's recall what binary search trees are.

Trước khi đi sâu vào B-Tree, hãy bàn trước về lý do tại sao ta nên cân nhắc các giải pháp thay thế cho các cây tìm kiếm truyền thống, chẳng hạn như cây tìm kiếm nhị phân (BST), 2-3-Tree, và AVL Tree [KNUTH98]. Để làm điều đó, hãy nhắc lại cây tìm kiếm nhị phân là gì.

25

25

Binary Search Trees — Cây tìm kiếm nhị phân

A **binary search tree** (BST) is a sorted in-memory data structure, used for efficient key-value lookups. BSTs consist of multiple nodes. Each tree **node** is represented by a key, a value associated with this key, and two child pointers (hence the name binary). BSTs start from a single node, called a root node . There can be only one root in the tree. Figure 2-1 shows an example of a binary search tree.

Cây tìm kiếm nhị phân (binary search tree - BST) là một cấu trúc dữ liệu trong bộ nhớ, được sắp xếp, dùng cho việc tra cứu khóa-giá trị hiệu quả. BST gồm nhiều nút. Mỗi nút của cây được biểu diễn bằng một khóa, một giá trị gắn với khóa đó, và hai con trỏ con (do đó có tên gọi nhị phân). BST bắt đầu từ một nút duy nhất, gọi là nút gốc (root node). Trong cây chỉ có thể có một nút gốc. Hình 2-1 minh họa một ví dụ về cây tìm kiếm nhị phân.

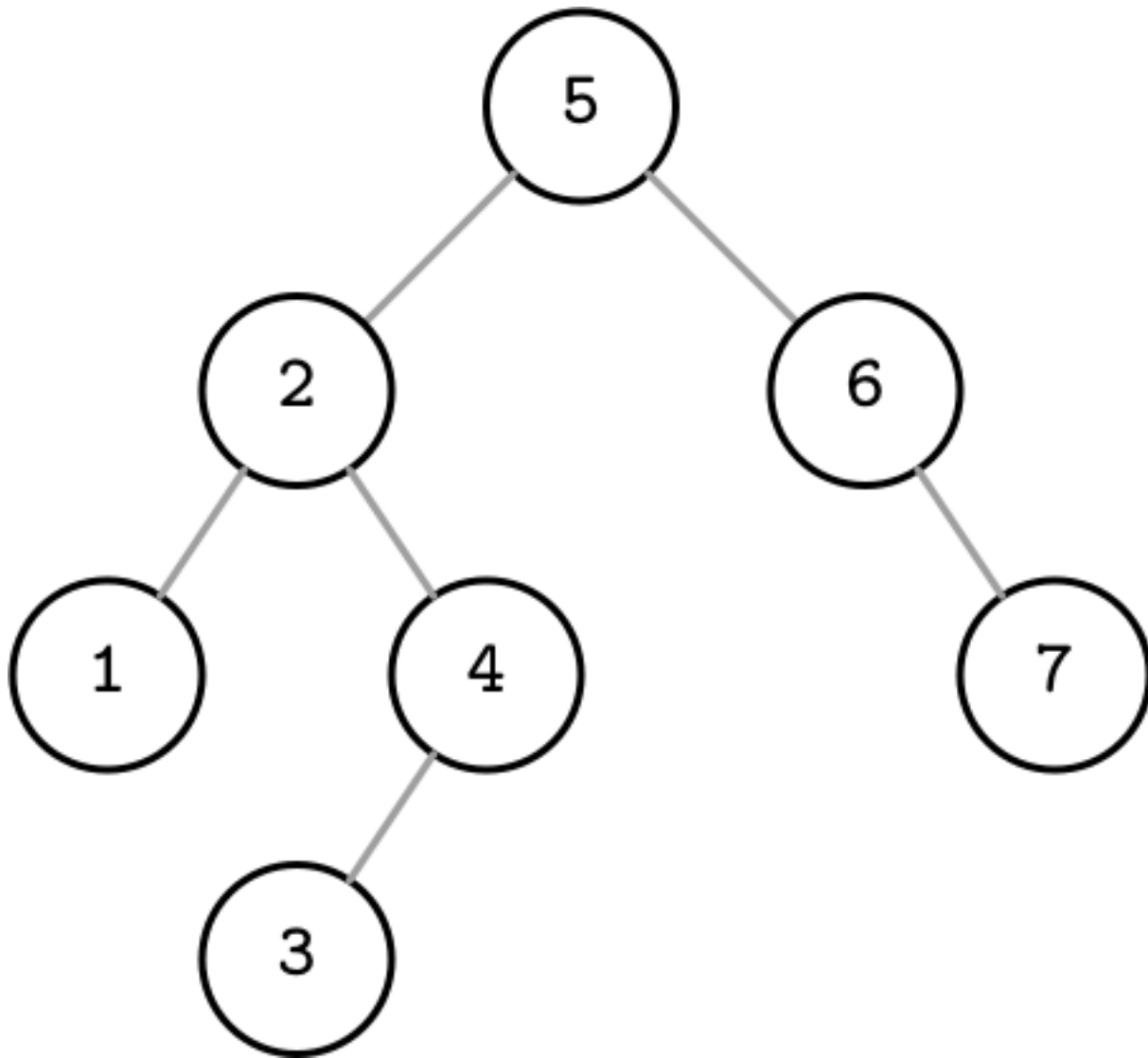


Figure 2-1. Binary search tree

Hình 2-1. Cây tìm kiếm nhị phân

Each node splits the search space into left and right subtrees , as Figure 2-2 shows: a node key is greater than any key stored in its left subtree and less than any key stored in its right subtree [SEGEWICK11] .

Mỗi nút chia không gian tìm kiếm thành cây con trái và cây con phải, như Hình 2-2 minh họa: khóa của một nút lớn hơn bất kỳ khóa nào lưu trong cây con trái của nó và nhỏ hơn bất kỳ khóa nào lưu trong cây con phải của nó [SEGEWICK11].

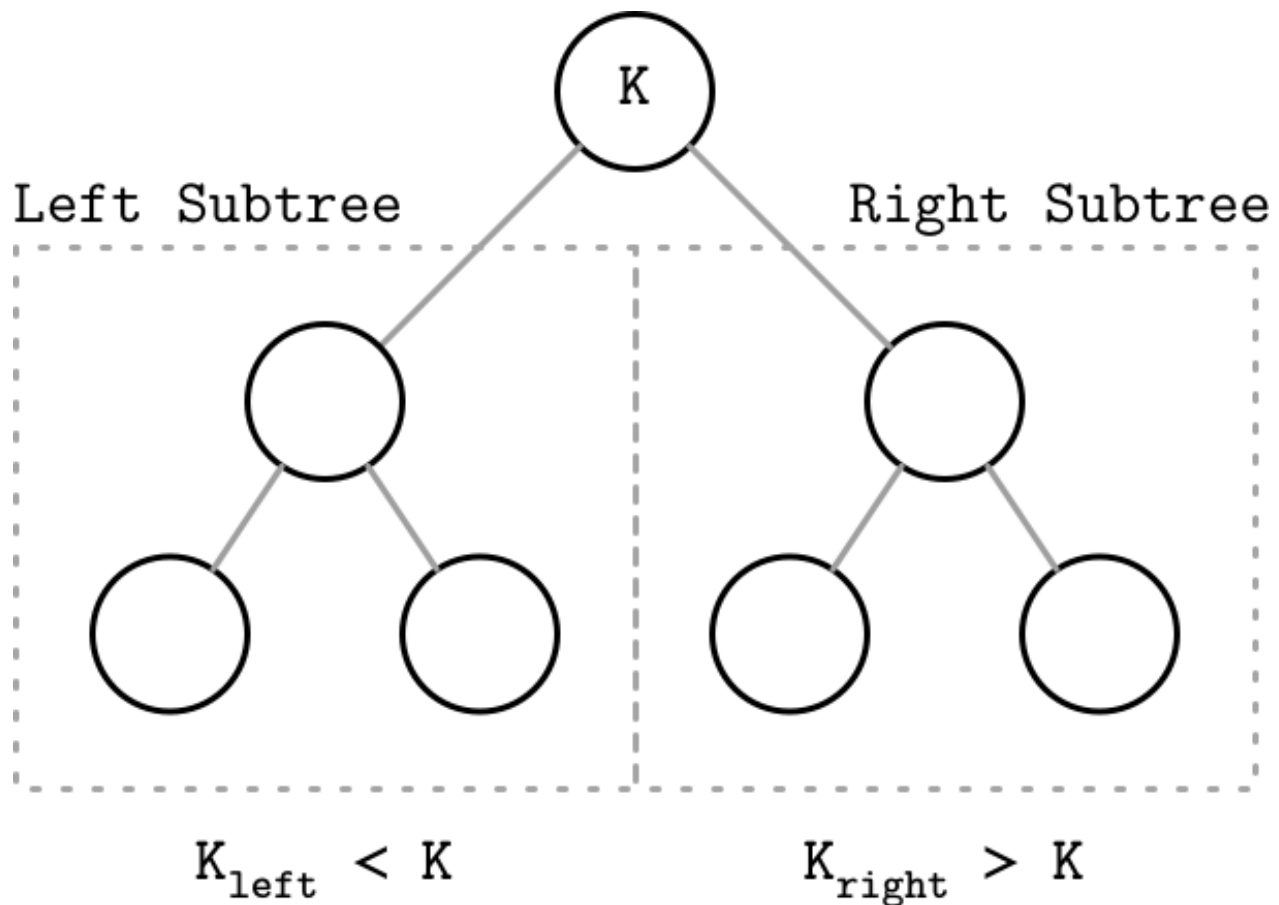


Figure 2-2. Binary tree node invariants Following left pointers from the root of the tree down to the leaf level (the level where nodes have no children) locates the node holding the smallest key within the tree and a value associated with it. Similarly, following right pointers locates the node holding the largest key within the tree and a value associated with it. Values are allowed to be stored in all nodes in the tree. Searches start from the root node, and may terminate before reaching the bottom level of the tree if the searched key was found on a higher level.

Hình 2-2. Bất biến của nút cây nhị phân. Đi theo các con trỏ trái từ gốc của cây xuống tới mức lá (mức mà các nút không còn nút con) sẽ định vị được nút chứa khóa nhỏ nhất trong cây cùng với giá trị gắn với nó. Tương tự, đi theo các con trỏ phải sẽ định vị được nút chứa khóa lớn nhất trong cây cùng giá trị gắn với nó. Giá trị được phép lưu ở tất cả các nút trong cây. Việc tìm kiếm bắt đầu từ nút gốc, và có thể kết thúc trước khi tới mức đáy của cây nếu khóa cần tìm được tìm thấy ở một mức cao hơn.

Tree Balancing — Cân bằng cây

Insert operations do not follow any specific pattern, and element insertion might lead to the situation where the tree is unbalanced (i.e., one of its branches is longer than the other one). The worst-case scenario is shown in Figure 2-3 (b), where we end up with a pathological tree, which looks more like a linked list, and instead of desired logarithmic complexity, we get linear, as illustrated in Figure 2-3 (a).

Các thao tác chèn không tuân theo một khuôn mẫu cụ thể nào, và việc chèn phần tử có thể dẫn tới tình huống cây bị mất cân bằng (tức là một trong các nhánh của nó dài hơn nhánh

khác). Trường hợp xấu nhất được thể hiện ở Hình 2-3 (b), nơi ta kết thúc với một cây bệnh lý, trông giống một danh sách liên kết hơn, và thay vì độ phức tạp logarit như mong muốn, ta nhận được độ phức tạp tuyến tính, như minh họa ở Hình 2-3 (a).

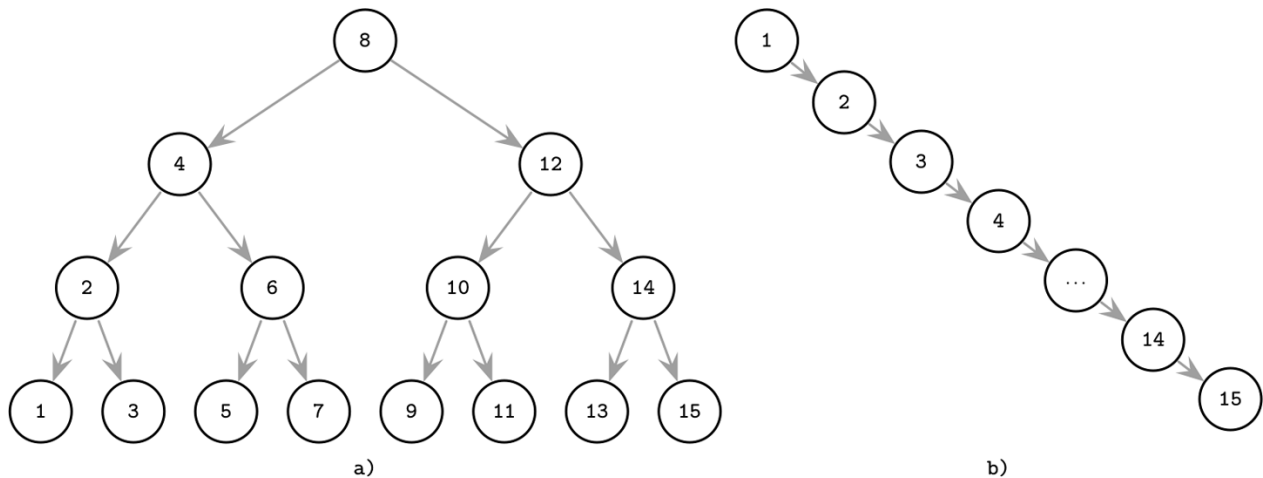


Figure 2-3. Balanced (a) and unbalanced or pathological (b) tree examples

Hình 2-3. Ví dụ về cây cân bằng (a) và cây mất cân bằng hoặc bệnh lý (b)

This example might slightly exaggerate the problem, but it illustrates why the tree needs to be balanced: even though it's somewhat unlikely that all the items end up on one side of the tree, at least some of them certainly will, which will significantly slow down searches.

Ví dụ này có thể hơi phóng đại vấn đề, nhưng nó minh họa lý do tại sao cây cần được cân bằng: dù cho việc tất cả các phần tử đều rơi về một phía của cây là khá khó xảy ra, nhưng chắc chắn ít nhất một số phần tử sẽ như vậy, điều này sẽ làm chậm đáng kể quá trình tìm kiếm.

The balanced tree is defined as one that has a height of $\log N$, where N is the total

Cây cân bằng được định nghĩa là cây có chiều cao $\log N$, trong đó N là tổng

number of items in the tree, and the difference in height between the two subtrees is not greater than one 1 [KNUTH98]. Without balancing, we lose performance benefits of the binary search tree structure, and allow insertions and deletions order to determine tree shape.

số phần tử trong cây, và chênh lệch chiều cao giữa hai cây con không lớn hơn một 1 [KNUTH98]. Nếu không có cân bằng, ta mất đi các lợi ích về hiệu năng của cấu trúc cây tìm kiếm nhị phân, và để cho thứ tự chèn và xóa quyết định hình dạng của cây.

1 This property is imposed by AVL Trees and several other data structures. More generally, binary search trees keep the difference in heights between subtrees within a small constant factor.

1 Thuộc tính này được áp đặt bởi AVL Tree và một số cấu trúc dữ liệu khác. Tổng quát hơn, cây tìm kiếm nhị phân giữ cho chênh lệch chiều cao giữa các cây con nằm trong một hệ số hằng nhỏ.

In the balanced tree, following the left or right node pointer reduces the search space in half on average, so lookup complexity is logarithmic: $O(\log N)$. If the tree is not

Trong cây cân bằng, đi theo con trỏ nút trái hoặc phải sẽ thu hẹp không gian tìm kiếm xuống một nửa trung bình, nên độ phức tạp tra cứu là logarit: $O(\log N)$. Nếu cây không

balanced, worst-case complexity goes up to $O(N)$, since we might end up in the situation where all elements end up on one side of the tree.

cân bằng, độ phức tạp trường hợp xấu nhất tăng lên $O(N)$, vì ta có thể rơi vào tình huống tất cả các phần tử đều nằm về một phía của cây.

Instead of adding new elements to one of the tree branches and making it longer, while the other one remains empty (as shown in Figure 2-3 (b)), the tree is balanced after each operation. Balancing is done by reorganizing nodes in a way that minimizes tree height and keeps the number of nodes on each side within bounds.

Thay vì thêm các phần tử mới vào một trong các nhánh của cây làm nó dài ra, trong khi nhánh kia vẫn rỗng (như thể hiện ở Hình 2-3 (b)), cây được cân bằng lại sau mỗi thao tác. Việc cân bằng được thực hiện bằng cách tổ chức lại các nút theo cách giảm thiểu chiều cao cây và giữ số lượng nút ở mỗi phía nằm trong giới hạn.

One of the ways to keep the tree balanced is to perform a rotation step after nodes are added or removed. If the insert operation leaves a branch unbalanced (two consecutive nodes in the branch have only one child), we can rotate nodes around the middle one. In the example shown in Figure 2-4, during rotation the middle node (3), known as a rotation pivot, is promoted one level higher, and its parent becomes its right child.

Một trong những cách để giữ cây cân bằng là thực hiện một bước xoay (rotation) sau khi các nút được thêm vào hoặc gỡ bỏ. Nếu thao tác chèn để lại một nhánh mất cân bằng (hai nút liên tiếp trong nhánh chỉ có một nút con), ta có thể xoay các nút quanh nút ở giữa. Trong ví dụ thể hiện ở Hình 2-4, trong quá trình xoay nút ở giữa (3), được gọi là điểm xoay (rotation pivot), được nâng lên cao hơn một mức, và nút cha của nó trở thành nút con phải của nó.

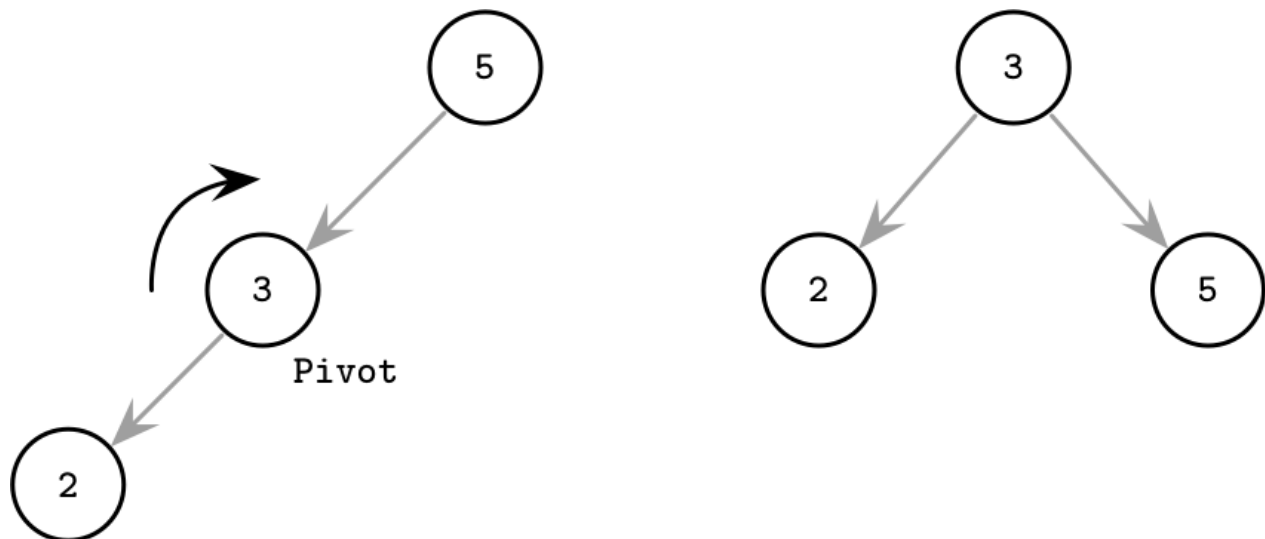


Figure 2-4. Rotation step example

Hình 2-4. Ví dụ về bước xoay

Trees for Disk-Based Storage — Cây cho lưu trữ trên đĩa

As previously mentioned, unbalanced trees have a worst-case complexity of $O(N)$. Balanced trees give us an average $O(\log N)$. At the same time, due to low **fanout**

Như đã đề cập trước đó, cây mất cân bằng có độ phức tạp trường hợp xấu nhất là $O(N)$. Cây cân bằng cho ta trung bình $O(\log N)$. Đồng thời, do độ phân nhánh (fanout) thấp

(fanout is the maximum allowed number of children per node), we have to perform balancing, relocate nodes, and update pointers rather frequently. Increased maintenance costs make BSTs impractical as on-disk data structures [NIEVERGELT74].

(fanout là số lượng nút con tối đa cho phép trên mỗi nút), ta phải thực hiện cân bằng, di dời các nút, và cập nhật các con trỏ khá thường xuyên. Chi phí bảo trì tăng cao khiến BST trở nên không thực tế khi dùng làm cấu trúc dữ liệu trên đĩa [NIEVERGELT74].

If we wanted to maintain a BST on disk, we'd face several problems. One problem is locality: since elements are added in random order, there's no guarantee that a newly created node is written close to its parent, which means that node child pointers may span across several disk pages. We can improve the situation to a certain extent by modifying the tree layout and using paged binary trees (see "Paged Binary Trees" on page 33).

Nếu ta muốn duy trì một BST trên đĩa, ta sẽ gặp phải một số vấn đề. Một vấn đề là tính cục bộ (locality): vì các phần tử được thêm theo thứ tự ngẫu nhiên, không có gì đảm bảo rằng một nút mới tạo được ghi gần với nút cha của nó, điều này có nghĩa là các con trỏ con của nút có thể trải dài qua nhiều trang đĩa. Ta có thể cải thiện tình hình ở một mức độ nhất định bằng cách điều chỉnh bố cục cây và sử dụng cây nhị phân phân trang (paged binary tree) (xem "Paged Binary Trees" ở trang 33).

Another problem, closely related to the cost of following child pointers, is tree height. Since binary trees have a fanout of just two, height is a binary logarithm of the number of the elements in the tree, and we have to perform $O(\log N)$ seeks to locate the

Một vấn đề khác, liên quan mật thiết tới chi phí của việc đi theo con trỏ con, là chiều cao cây. Vì cây nhị phân có fanout chỉ bằng hai, chiều cao là logarit cơ số hai của số phần tử trong cây, và ta phải thực hiện $O(\log N)$ lần tìm kiếm (seek) để định vị

searched element and, subsequently, perform the same number of disk transfers. 2-3-Trees and other low-fanout trees have a similar limitation: while they are useful as in-memory data structures, small node size makes them impractical for external storage [COMER79].

phần tử cần tìm và, kế tiếp, thực hiện cùng số lần chuyển dữ liệu trên đĩa. 2-3-Tree và các cây có fanout thấp khác cũng có hạn chế tương tự: dù chúng hữu ích khi làm cấu trúc dữ liệu trong bộ nhớ, kích thước nút nhỏ khiến chúng không thực tế cho lưu trữ ngoài [COMER79].

A naive on-disk BST implementation would require as many disk seeks as comparisons, since there's no built-in concept of locality. This sets us on a course to look for a data structure that would exhibit this property.

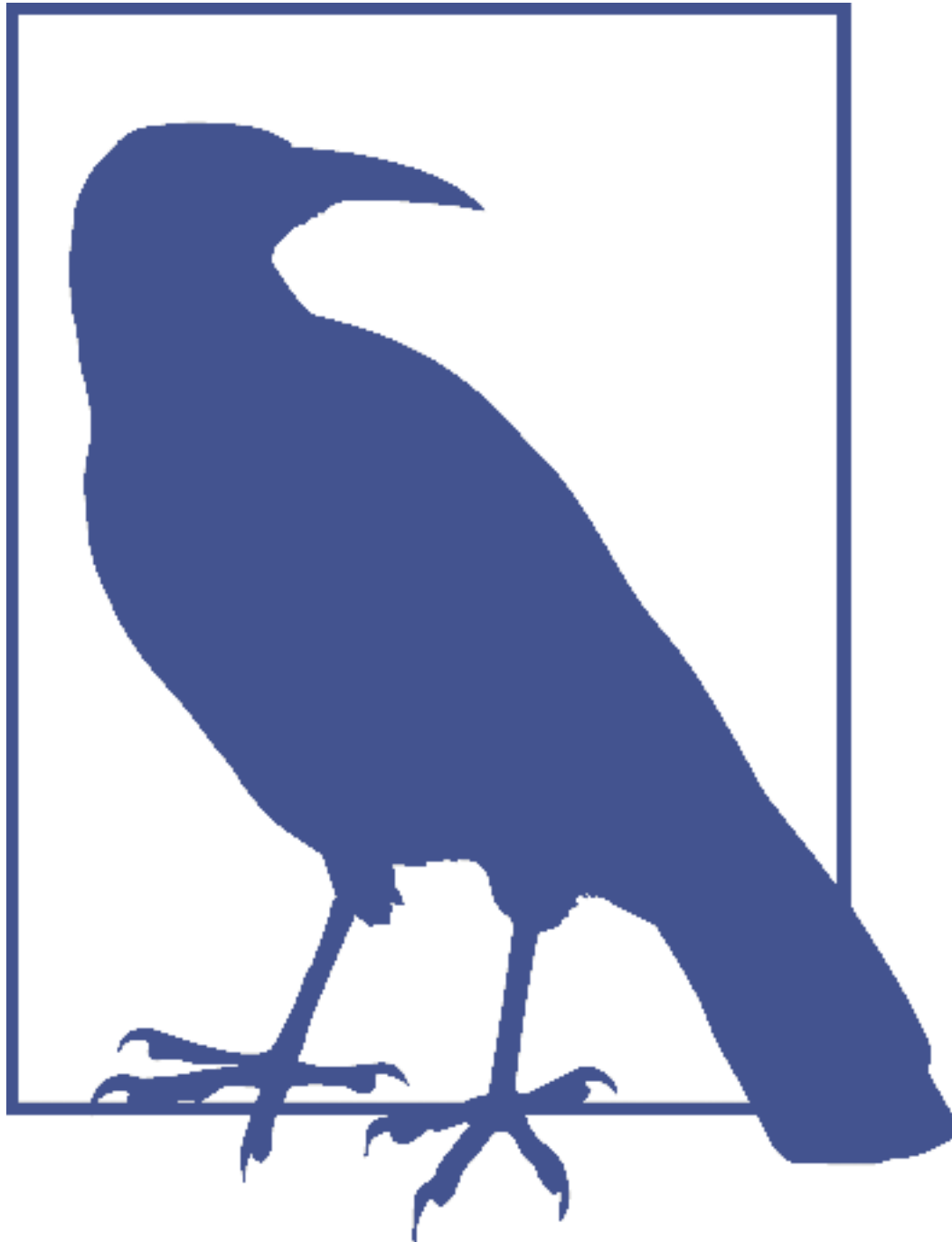
Một cách hiện thực BST trên đĩa ngây thơ sẽ đòi hỏi số lần tìm kiếm (seek) đĩa bằng số lần so sánh, vì không có khái niệm tính cục bộ tích hợp sẵn. Điều này đặt chúng ta vào hành trình tìm kiếm một cấu trúc dữ liệu có được thuộc tính này.

Considering these factors, a version of the tree that would be better suited for disk implementation has to exhibit the following properties:

Cân nhắc các yếu tố này, một phiên bản cây phù hợp hơn cho hiện thực trên đĩa phải thể hiện các thuộc tính sau:

- High fanout to improve locality of the neighboring keys.
- Low height to reduce the number of seeks during traversal.

- Fanout cao để cải thiện tính cục bộ của các khóa lân cận.
- Chiều cao thấp để giảm số lần tìm kiếm trong quá trình duyệt cây.



Fanout and height are inversely correlated: the higher the fanout, the lower the height. If fanout is high, each node can hold more children, reducing the number of nodes and, subsequently, reducing height.

Fanout và chiều cao tỉ lệ nghịch với nhau: fanout càng cao thì chiều cao càng thấp. Nếu fanout cao, mỗi nút có thể chứa nhiều nút con hơn, làm giảm số lượng nút và, kế tiếp, làm giảm chiều cao.

Disk-Based Structures — Cấu trúc dựa trên đĩa

We’ve talked about memory and disk-based storage (see “Memory- Versus Disk- Based **DBMS**” on page 10) in general terms. We can draw the same distinction for specific data structures: some are better suited to be used on disk and some work better in memory.

Chúng ta đã bàn về lưu trữ trong bộ nhớ và lưu trữ trên đĩa (xem “Memory- Versus Disk- Based DBMS” ở trang 10) một cách tổng quát. Ta có thể vạch ra cùng sự phân biệt đó cho các cấu trúc dữ liệu cụ thể: một số phù hợp hơn để dùng trên đĩa và một số hoạt động tốt hơn trong bộ nhớ.

As we have discussed, not every data structure that satisfies space and complexity requirements can be effectively used for on-disk storage. Data structures used in databases have to be adapted to account for persistent medium limitations.

Như đã thảo luận, không phải mọi cấu trúc dữ liệu thỏa mãn yêu cầu về không gian và độ phức tạp đều có thể được dùng hiệu quả cho lưu trữ trên đĩa. Các cấu trúc dữ liệu dùng trong cơ sở dữ liệu phải được điều chỉnh để tính đến các hạn chế của môi trường lưu trữ bền vững.

On-disk data structures are often used when the amounts of data are so large that keeping an entire dataset in memory is impossible or not feasible. Only a fraction of the data can be cached in memory at any time, and the rest has to be stored on disk in a manner that allows efficiently accessing it.

Các cấu trúc dữ liệu trên đĩa thường được dùng khi lượng dữ liệu lớn tới mức việc giữ toàn bộ tập dữ liệu trong bộ nhớ là bất khả thi hoặc không khả thi. Tại bất kỳ thời điểm nào cũng chỉ có một phần nhỏ dữ liệu có thể được lưu vào bộ nhớ đệm (cache), phần còn lại phải được lưu trên đĩa theo cách cho phép truy cập nó một cách hiệu quả.

Hard Disk Drives — Ổ đĩa cứng

Most traditional algorithms were developed when spinning disks were the most widespread persistent storage medium, which significantly influenced their design. Later, new developments in storage media, such as flash drives, inspired new algorithms and modifications to the existing ones, exploiting the capabilities of the new hardware. These days, new types of data structures are emerging, optimized to work with nonvolatile byte-addressable storage (for example, [XIA17] [KANNAN18]).

Hầu hết các thuật toán truyền thống được phát triển khi đĩa quay (spinning disk) là môi trường lưu trữ bền vững phổ biến nhất, điều này ảnh hưởng đáng kể đến thiết kế của chúng. Sau đó, những phát triển mới trong môi trường lưu trữ, chẳng hạn như ổ flash, đã truyền cảm hứng cho các thuật toán mới và các sửa đổi cho các thuật toán hiện có, khai thác khả năng của phần cứng mới. Ngày nay, các loại cấu trúc dữ liệu mới đang xuất hiện, được tối ưu để làm việc với lưu trữ không bay hơi, định địa chỉ theo byte (ví dụ, [XIA17] [KANNAN18]).

On spinning disks, seeks increase costs of random reads because they require disk rotation and mechanical head movements to position the **read/write** head to the desired location. However, once the expensive part is done, reading or writing contiguous bytes (i.e., **sequential** operations) is relatively cheap.

Trên đĩa quay, các lần tìm kiếm (seek) làm tăng chi phí của đọc ngẫu nhiên vì chúng đòi hỏi đĩa quay và đầu đọc cơ học di chuyển để định vị đầu đọc/ghi tới vị trí mong muốn. Tuy

nhiên, một khi phần đầu đã hoàn tất, việc đọc hoặc ghi các byte liên kề (tức là các thao tác tuần tự) lại tương đối dễ.

The smallest transfer unit of a spinning drive is a **sector** , so when some operation is performed, at least an entire sector can be read or written. Sector sizes typically range from 512 bytes to 4 Kb.

Đơn vị chuyển dữ liệu nhỏ nhất của một ổ đĩa quay là một cung từ (sector), nên khi một thao tác được thực hiện, ít nhất một cung từ trọn vẹn được đọc hoặc ghi. Kích thước cung từ thường dao động từ 512 byte đến 4 Kb.

Head positioning is the most expensive part of an operation on the **HDD**. This is one of the reasons we often hear about the positive effects of sequential I/O: reading and writing contiguous memory segments from disk.

Định vị đầu đọc là phần đắt đỏ nhất của một thao tác trên HDD. Đây là một trong những lý do ta thường nghe về các tác động tích cực của I/O tuần tự: đọc và ghi các đoạn bộ nhớ liên kề từ đĩa.

Solid State Drives — Ổ đĩa thể rắn

Solid state drives (SSDs) do not have moving parts: there's no disk that spins, or head that has to be positioned for the read. A typical **SSD** is built of memory cells , connected into strings (typically 32 to 64 cells per string), strings are combined into arrays , arrays are combined into pages , and pages are combined into blocks [LARRIVEE15] .

Ổ đĩa thể rắn (solid state drive - SSD) không có bộ phận chuyển động: không có đĩa quay, cũng không có đầu đọc cần phải được định vị để đọc. Một SSD điển hình được xây dựng từ các ô nhớ (memory cell), được kết nối thành các chuỗi (string) (thường 32 đến 64 ô mỗi chuỗi), các chuỗi được kết hợp thành các mảng (array), các mảng được kết hợp thành các trang (page), và các trang được kết hợp thành các khối (block) [LARRIVEE15].

Depending on the exact technology used, a **cell** can hold one or multiple bits of data. Pages vary in size between devices, but typically their sizes range from 2 to 16 Kb. Blocks typically contain 64 to 512 pages. Blocks are organized into planes and, finally, planes are placed on a die . SSDs can have one or more dies. Figure 2-5 shows this hierarchy.

Tùy thuộc vào công nghệ cụ thể được dùng, một ô có thể chứa một hoặc nhiều bit dữ liệu. Trang có kích thước khác nhau giữa các thiết bị, nhưng thường kích thước của chúng dao động từ 2 đến 16 Kb. Khối thường chứa từ 64 đến 512 trang. Các khối được tổ chức thành các mặt phẳng (plane) và, cuối cùng, các mặt phẳng được đặt lên một đế (die). SSD có thể có một hoặc nhiều đế. Hình 2-5 thể hiện hệ thống phân cấp này.

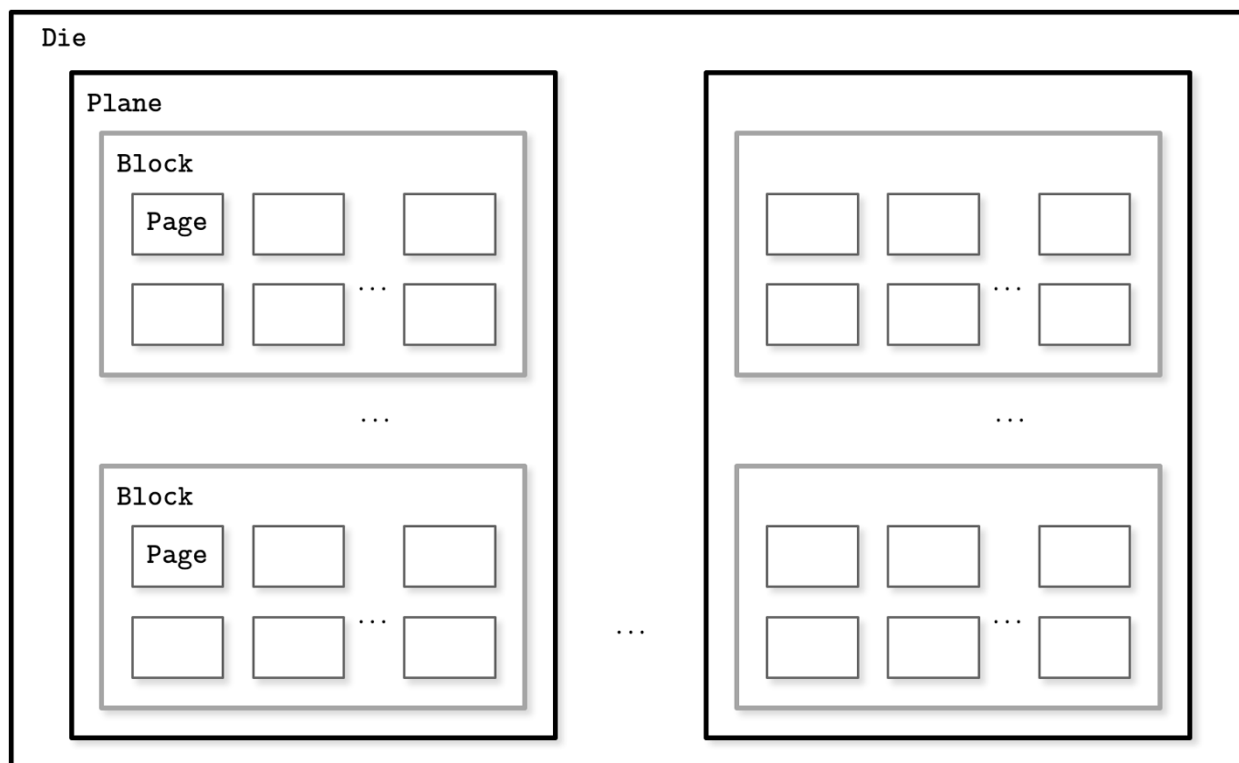


Figure 2-5. SSD organization schematics

Hình 2-5. Sơ đồ tổ chức của SSD

The smallest unit that can be written (programmed) or read is a page. However, we can only make changes to the empty memory cells (i.e., to ones that have been erased before the write). The smallest erase entity is not a page, but a **block** that holds multiple pages, which is why it is often called an erase block. Pages in an empty block have to be written sequentially.

Đơn vị nhỏ nhất có thể được ghi (program) hoặc đọc là một trang. Tuy nhiên, ta chỉ có thể thực hiện thay đổi lên các ô nhớ rỗng (tức là các ô đã bị xóa trước khi ghi). Thực thể xóa nhỏ nhất không phải là một trang, mà là một khối chứa nhiều trang, đó là lý do nó thường được gọi là khối xóa (erase block). Các trang trong một khối rỗng phải được ghi tuần tự.

The part of a flash memory controller responsible for mapping page IDs to their physical locations, tracking empty, written, and discarded pages, is called the Flash Translation Layer (FTL) (see “Flash Translation Layer” on page 157 for more about FTL). It is also responsible for **garbage collection**, during which FTL finds blocks it can safely erase. Some blocks might still contain live pages. In this case, it relocates live pages from these blocks to new locations and remaps page IDs to point there. After this, it erases the now-unused blocks, making them available for writes.

Phần của bộ điều khiển bộ nhớ flash chịu trách nhiệm ánh xạ các ID trang tới vị trí vật lý của chúng, theo dõi các trang rỗng, đã ghi, và đã loại bỏ, được gọi là Tầng Dịch Flash (Flash Translation Layer - FTL) (xem “Flash Translation Layer” ở trang 157 để biết thêm về FTL). Nó cũng chịu trách nhiệm thu gom rác (garbage collection), trong quá trình đó FTL tìm các khối mà nó có thể xóa an toàn. Một số khối có thể vẫn còn chứa các trang còn sống. Trong trường hợp này, nó di dời các trang còn sống từ các khối này tới các vị trí mới và ánh xạ lại các ID trang để trở tới đó. Sau đó, nó xóa các khối hiện không còn dùng, làm cho chúng sẵn sàng cho việc ghi.

Since in both device types (HDDs and SSDs) we are addressing chunks of memory rather than individual bytes (i.e., accessing data block-wise), most operating systems have a block device abstraction [CESATI05]. It hides an internal disk structure and buffers I/O operations internally, so when we're reading a single word from a block device, the whole block containing it is read. This is a constraint we cannot ignore and should always take into account when working with disk-resident data structures.

Vì ở cả hai loại thiết bị (HDD và SSD) ta đang định địa chỉ theo các khối bộ nhớ thay vì từng byte riêng lẻ (tức là truy cập dữ liệu theo khối), hầu hết các hệ điều hành đều có một trừu tượng thiết bị khối (block device) [CESATI05]. Nó che giấu cấu trúc đĩa nội bộ và đệm các thao tác I/O bên trong, nên khi ta đọc một từ (word) đơn lẻ từ một thiết bị khối, toàn bộ khối chứa nó được đọc. Đây là một ràng buộc ta không thể bỏ qua và luôn phải tính đến khi làm việc với các cấu trúc dữ liệu nằm trên đĩa.

In SSDs, we don't have a strong emphasis on random versus sequential I/O, as in HDDs, because the difference in latencies between random and sequential reads is not as large. There is still some difference caused by prefetching, reading contiguous pages, and internal parallelism [GOOSSAERT14].

Ở SSD, ta không nhấn mạnh nhiều về I/O ngẫu nhiên so với tuần tự như ở HDD, bởi chênh lệch độ trễ giữa đọc ngẫu nhiên và đọc tuần tự không quá lớn. Vẫn còn một chút khác biệt gây ra bởi prefetch, đọc các trang liên kề, và song song hóa nội bộ [GOOSSAERT14].

Even though garbage collection is usually a background operation, its effects may negatively impact write performance, especially in cases of random and unaligned write workloads.

Mặc dù thu gom rác thường là một thao tác chạy nền, các tác động của nó có thể ảnh hưởng tiêu cực tới hiệu năng ghi, đặc biệt trong các trường hợp khối lượng công việc ghi ngẫu nhiên và không căn chỉnh (unaligned).

Writing only full blocks, and combining subsequent writes to the same block, can help to reduce the number of required I/O operations. We discuss **buffering** and immutability as ways to achieve that in later chapters.

Chỉ ghi các khối đầy đủ, và kết hợp các lần ghi liên tiếp lên cùng một khối, có thể giúp giảm số lượng thao tác I/O cần thiết. Chúng ta thảo luận về đệm (buffering) và tính bất biến như các cách để đạt được điều đó ở các chương sau.

On-Disk Structures — Cấu trúc trên đĩa

Besides the cost of disk access itself, the main limitation and design condition for building efficient on-disk structures is the fact that the smallest unit of disk operation is a block. To follow a pointer to the specific location within the block, we have to fetch an entire block. Since we already have to do that, we can change the layout of the data structure to take advantage of it.

Bên cạnh chi phí của bản thân việc truy cập đĩa, hạn chế chính và điều kiện thiết kế để xây dựng các cấu trúc trên đĩa hiệu quả là thực tế rằng đơn vị thao tác đĩa nhỏ nhất là một khối. Để đi theo một con trỏ tới một vị trí cụ thể bên trong khối, ta phải nạp toàn bộ khối. Vì ta đã phải làm điều đó, ta có thể thay đổi bố cục của cấu trúc dữ liệu để tận dụng nó.

We've mentioned pointers several times throughout this chapter already, but this word has slightly different semantics for on-disk structures. On disk, most of the time we manage the data layout manually (unless, for example, we're using memory mapped files). This is still similar to regular

pointer operations, but we have to compute the target pointer addresses and follow the pointers explicitly.

Chúng ta đã đề cập tới con trỏ nhiều lần xuyên suốt chương này, nhưng từ này mang nghĩa hơi khác đối với các cấu trúc trên đĩa. Trên đĩa, hầu hết thời gian ta quản lý bố cục dữ liệu một cách thủ công (trừ khi, ví dụ, ta đang dùng các tệp ánh xạ bộ nhớ - memory mapped file). Điều này vẫn tương tự với các thao tác con trỏ thông thường, nhưng ta phải tính toán địa chỉ con trỏ đích và đi theo các con trỏ một cách tường minh.

Most of the time, on-disk offsets are precomputed (in cases when the pointer is written on disk before the part it points to) or cached in memory until they are flushed on the disk. Creating long dependency chains in on-disk structures greatly increases code and structure complexity, so it is preferred to keep the number of pointers and their spans to a minimum.

Hầu hết thời gian, các độ dời (offset) trên đĩa được tính trước (trong các trường hợp khi con trỏ được ghi lên đĩa trước phần mà nó trỏ tới) hoặc được lưu vào bộ nhớ đệm cho tới khi chúng được xả (flush) xuống đĩa. Việc tạo ra các chuỗi phụ thuộc dài trong các cấu trúc trên đĩa làm tăng đáng kể độ phức tạp của mã và của cấu trúc, nên người ta ưu tiên giữ số lượng con trỏ và phạm vi trải của chúng ở mức tối thiểu.

In summary, on-disk structures are designed with their target storage specifics in mind and generally optimize for fewer disk accesses. We can do this by improving locality, optimizing the internal representation of the structure, and reducing the number of out-of-page pointers.

Tóm lại, các cấu trúc trên đĩa được thiết kế với đặc thù của môi trường lưu trữ đích trong đầu và nhìn chung tối ưu cho việc ít truy cập đĩa hơn. Ta có thể làm điều này bằng cách cải thiện tính cục bộ, tối ưu biểu diễn nội bộ của cấu trúc, và giảm số lượng con trỏ nằm ngoài trang (out-of-page pointer).

In “Binary Search Trees” on page 26, we came to the conclusion that high fanout and low height are desired properties for an optimal on-disk data structure. We’ve also just discussed additional space overhead coming from pointers, and maintenance overhead from remapping these pointers as a result of balancing. B-Trees combine these ideas: increase node fanout, and reduce tree height, the number of node pointers, and the frequency of balancing operations.

Trong “Binary Search Trees” ở trang 26, ta đã đi đến kết luận rằng fanout cao và chiều cao thấp là các thuộc tính mong muốn cho một cấu trúc dữ liệu trên đĩa tối ưu. Ta cũng vừa thảo luận về phụ phí không gian bổ sung đến từ các con trỏ, và phụ phí bảo trì từ việc ánh xạ lại các con trỏ này như là kết quả của việc cân bằng. B-Tree kết hợp các ý tưởng này: tăng fanout của nút, và giảm chiều cao cây, số lượng con trỏ nút, và tần suất các thao tác cân bằng.

Paged Binary Trees — Cây nhị phân phân trang

Laying out a binary tree by grouping nodes into pages, as Figure 2-6 shows, improves the situation with locality. To find the next node, it’s only necessary to follow a pointer in an already fetched page. However, there’s still some overhead incurred by the nodes and pointers between them. Laying the structure out on disk and its further maintenance are nontrivial endeavors, especially if keys and values are not presorted and added in random order. Balancing requires page reorganization, which in turn causes pointer updates.

Bố trí một cây nhị phân bằng cách gom các nút vào các trang, như Hình 2-6 minh họa, cải thiện tình hình về tính cục bộ. Để tìm nút kế tiếp, chỉ cần đi theo một con trỏ trong một trang đã được nạp. Tuy nhiên, vẫn còn một số phụ phí phát sinh từ các nút và các con trỏ

giữa chúng. Việc bố trí cấu trúc lên đĩa và việc bảo trì nó về sau là những công việc không tầm thường, đặc biệt nếu khóa và giá trị không được sắp xếp trước và được thêm theo thứ tự ngẫu nhiên. Cân bằng đòi hỏi tổ chức lại trang, điều này lần lượt gây ra các cập nhật con trỏ.

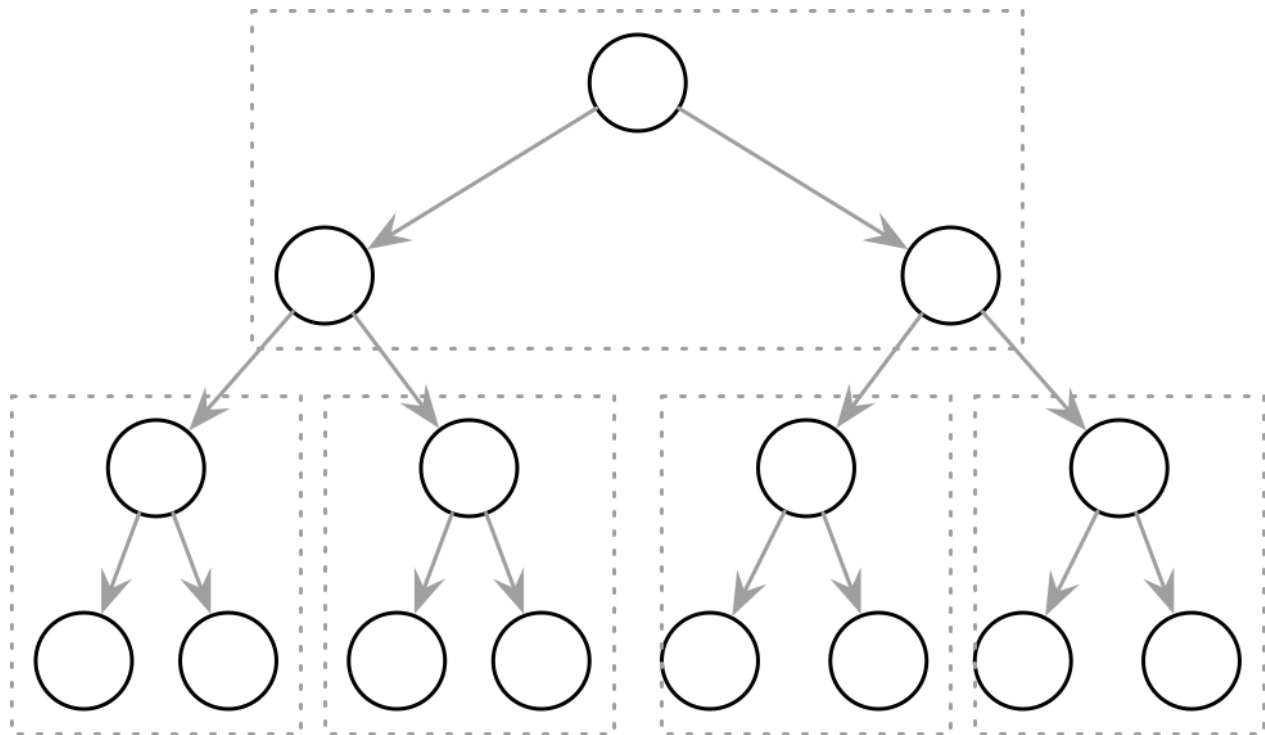


Figure 2-6. Paged binary trees

Hình 2-6. Cây nhị phân phân trang

Ubiquitous B-Trees — B-Tree ở khắp mọi nơi

We are braver than a bee, and a... longer than a tree... —Winnie the Pooh

Chúng tôi gan dạ hơn một con ong, và... dài hơn một cái cây... —Winnie the Pooh

B-Trees can be thought of as a vast catalog room in the library: you first have to pick the correct cabinet, then the correct shelf in that cabinet, then the correct drawer on the shelf, and then browse through the cards in the drawer to find the one you're searching for. Similarly, a B-Tree builds a hierarchy that helps to navigate and locate the searched items quickly.

B-Tree có thể được hình dung như một phòng mục lục rộng lớn trong thư viện: trước tiên bạn phải chọn đúng tủ, rồi đúng kệ trong tủ đó, rồi đúng ngăn kéo trên kệ, rồi lục qua các tấm thẻ trong ngăn kéo để tìm tấm bạn đang tìm kiếm. Tương tự, một B-Tree xây dựng một hệ thống phân cấp giúp điều hướng và định vị nhanh chóng các mục cần tìm.

As we discussed in “Binary Search Trees” on page 26, B-Trees build upon the foundation of balanced search trees and are different in that they have higher fanout (have more child nodes) and smaller height.

Như đã thảo luận trong “Binary Search Trees” ở trang 26, B-Tree được xây dựng trên nền tảng của các cây tìm kiếm cân bằng và khác ở chỗ chúng có fanout cao hơn (có nhiều nút

con hơn) và chiều cao nhỏ hơn.

In most of the literature, binary tree nodes are drawn as circles. Since each node is responsible just for one key and splits the range into two parts, this level of detail is sufficient and intuitive. At the same time, B-Tree nodes are often drawn as rectangles, and pointer blocks are also shown explicitly to highlight the relationship between child nodes and separator keys. Figure 2-7 shows binary tree, 2-3-Tree, and B-Tree nodes side by side, which helps to understand the similarities and differences between them.

Trong hầu hết các tài liệu, các nút cây nhị phân được vẽ dưới dạng vòng tròn. Vì mỗi nút chỉ chịu trách nhiệm cho một khóa và chia khoảng thành hai phần, mức độ chi tiết này là đủ và trực quan. Đồng thời, các nút B-Tree thường được vẽ dưới dạng hình chữ nhật, và các khối con trỏ cũng được hiển thị tường minh để làm nổi bật mối quan hệ giữa các nút con và các khóa phân tách (separator key). Hình 2-7 thể hiện các nút cây nhị phân, 2-3-Tree, và B-Tree cạnh nhau, giúp hiểu được những điểm tương đồng và khác biệt giữa chúng.

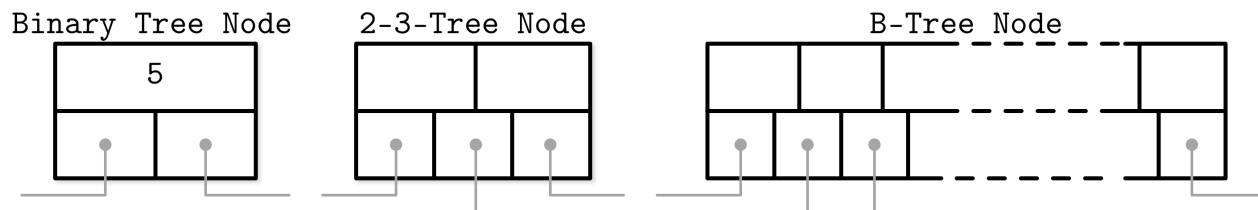


Figure 2-7. Binary tree, 2-3-Tree, and B-Tree nodes side by side

Hình 2-7. Các nút cây nhị phân, 2-3-Tree, và B-Tree cạnh nhau

Nothing prevents us from depicting binary trees in the same way. Both structures have similar pointer-following semantics, and differences start showing in how the balance is maintained. Figure 2-8 shows that and hints at similarities between BSTs and B-Trees: in both cases, keys split the tree into subtrees, and are used for navigating the tree and finding searched keys. You can compare it to Figure 2-1 .

Không có gì ngăn ta vẽ cây nhị phân theo cùng cách như vậy. Cả hai cấu trúc đều có ngữ nghĩa đi theo con trỏ tương tự nhau, và sự khác biệt bắt đầu lộ ra ở cách duy trì sự cân bằng. Hình 2-8 thể hiện điều đó và gợi ý các điểm tương đồng giữa BST và B-Tree: trong cả hai trường hợp, các khóa chia cây thành các cây con, và được dùng để điều hướng cây và tìm các khóa cần tìm. Bạn có thể so sánh nó với Hình 2-1.

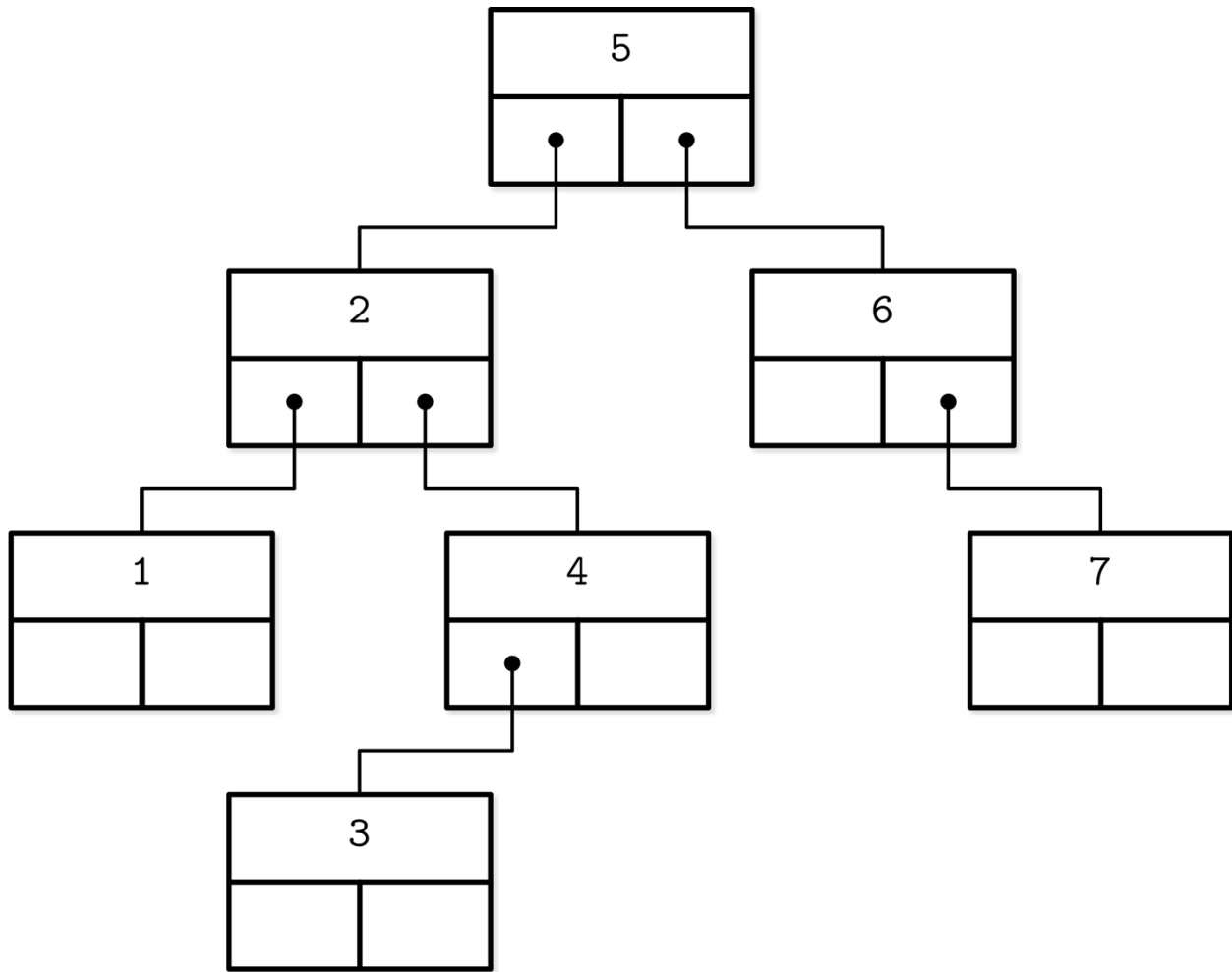


Figure 2-8. Alternative representation of a binary tree

Hình 2-8. Cách biểu diễn thay thế của một cây nhị phân

B-Trees are sorted : keys inside the B-Tree nodes are stored in order. Because of that, to locate a searched key, we can use an algorithm like binary search. This also implies that lookups in B-Trees have logarithmic complexity. For example, finding a searched key among 4 billion (4×10^9) items takes about 32 comparisons (see “B-Tree Lookup Complexity” on page 37 for more on this subject). If we had to make a disk **seek** for each one of these comparisons, it would significantly slow us down, but since B-Tree nodes store dozens or even hundreds of items, we only have to make one disk seek per level jump. We’ll discuss a lookup algorithm in more detail later in this chapter.

B-Tree được sắp xếp: các khóa bên trong các nút B-Tree được lưu theo thứ tự. Bởi vậy, để định vị một khóa cần tìm, ta có thể dùng một thuật toán như tìm kiếm nhị phân. Điều này cũng ngụ ý rằng các tra cứu trong B-Tree có độ phức tạp logarit. Ví dụ, tìm một khóa cần tìm trong số 4 tỷ (4×10^9) mục mất khoảng 32 lần so sánh (xem “B-Tree Lookup Complexity” ở trang 37 để biết thêm về chủ đề này). Nếu ta phải thực hiện một lần tìm kiếm (seek) đĩa cho từng lần so sánh trong số này, nó sẽ làm chậm ta đáng kể, nhưng vì các nút B-Tree lưu hàng chục hoặc thậm chí hàng trăm mục, ta chỉ phải thực hiện một lần tìm kiếm đĩa cho mỗi lần nhảy mức. Chúng ta sẽ thảo luận về thuật toán tra cứu chi tiết hơn ở phần sau của chương này.

Using B-Trees, we can efficiently execute both point and range queries. Point queries, expressed by

the equality (=) predicate in most query languages, locate a single item. On the other hand, range queries, expressed by comparison (< , > , ≤ , and ≥) predicates, are used to query multiple data items in order.

Dùng B-Tree, ta có thể thực thi hiệu quả cả truy vấn điểm (point query) và truy vấn khoảng (range query). Truy vấn điểm, được biểu diễn bằng vị từ bằng (=) trong hầu hết các ngôn ngữ truy vấn, định vị một mục duy nhất. Mặt khác, truy vấn khoảng, được biểu diễn bằng các vị từ so sánh (<, >, ≤, và ≥), được dùng để truy vấn nhiều mục dữ liệu theo thứ tự.

B-Tree Hierarchy — Hệ thống phân cấp B-Tree

B-Trees consist of multiple nodes. Each node holds up to N keys and N + 1 pointers to the child nodes. These nodes are logically grouped into three groups:

B-Tree gồm nhiều nút. Mỗi nút chứa tới N khóa và N + 1 con trỏ tới các nút con. Các nút này được nhóm về mặt logic thành ba nhóm:

Root node This has no parents and is the top of the tree.

Nút gốc Nút này không có cha và là đỉnh của cây.

Leaf nodes These are the bottom layer nodes that have no child nodes.

Nút lá Đây là các nút ở lớp đáy không có nút con nào.

Internal nodes These are all other nodes, connecting root with leaves. There is usually more than one level of internal nodes.

Nút trong Đây là tất cả các nút khác, kết nối gốc với lá. Thường có nhiều hơn một mức nút trong.

This hierarchy is shown in Figure 2-9 .

Hệ thống phân cấp này được thể hiện ở Hình 2-9.

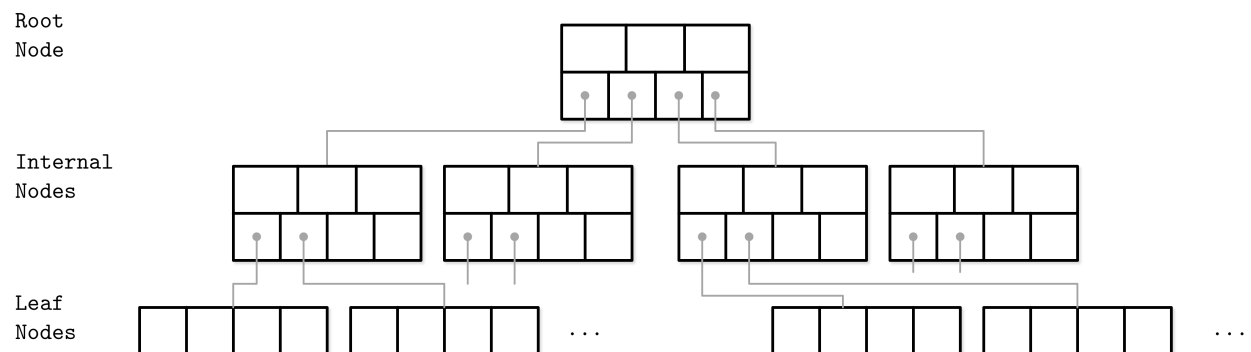


Figure 2-9. B-Tree node hierarchy

Hình 2-9. Hệ thống phân cấp nút B-Tree

Since B-Trees are a page organization technique (i.e., they are used to organize and navigate fixed-size pages), we often use terms node and page interchangeably.

Vì B-Tree là một kỹ thuật tổ chức trang (tức là chúng được dùng để tổ chức và điều hướng các trang có kích thước cố định), ta thường dùng các thuật ngữ nút và trang thay thế cho nhau.

The relation between the node capacity and the number of keys it actually holds is called **occupancy**

Mối quan hệ giữa sức chứa của nút và số khóa nó thực sự chứa được gọi là độ lấp đầy (occupancy).

B-Trees are characterized by their fanout : the number of keys stored in each node. Higher fanout helps to amortize the cost of structural changes required to keep the tree balanced and to reduce the number of seeks by storing keys and pointers to child nodes in a single block or multiple consecutive blocks. Balancing operations (namely, splits and merges) are triggered when the nodes are full or nearly empty.

B-Tree được đặc trưng bởi fanout của chúng: số khóa lưu trong mỗi nút. Fanout cao hơn giúp khấu trừ (amortize) chi phí của các thay đổi cấu trúc cần thiết để giữ cây cân bằng và giảm số lần tìm kiếm bằng cách lưu các khóa và con trỏ tới các nút con trong một khối đơn lẻ hoặc nhiều khối liên kề. Các thao tác cân bằng (cụ thể là tách - split và gộp - merge) được kích hoạt khi các nút đầy hoặc gần như rỗng.

B — B

-Trees — -Tree

We're using the **term** B-Tree as an umbrella for a family of data structures that share all or most of the mentioned properties. A more precise name for the described data structure is B + -Tree. [KNUTH98] refers to trees with a high fanout as multiway trees.

Chúng tôi dùng thuật ngữ B-Tree như một cái ô bao trùm cho một họ các cấu trúc dữ liệu chia sẻ tất cả hoặc hầu hết các thuộc tính đã đề cập. Một cái tên chính xác hơn cho cấu trúc dữ liệu được mô tả là B+-Tree. [KNUTH98] gọi các cây có fanout cao là cây đa hướng (multiway tree).

B-Trees allow storing values on any level: in root, internal, and leaf nodes. B + -Trees store values only in leaf nodes. Internal nodes store only separator keys used to guide the search algorithm to the associated value stored on the leaf level.

B-Tree cho phép lưu giá trị ở bất kỳ mức nào: ở gốc, các nút trong, và các nút lá. B+-Tree chỉ lưu giá trị ở các nút lá. Các nút trong chỉ lưu các khóa phân tách dùng để dẫn dắt thuật toán tìm kiếm tới giá trị liên quan được lưu ở mức lá.

Since values in B + -Trees are stored only on the leaf level, all operations (inserting, updating, removing, and retrieving data records) affect only leaf nodes and propagate to higher levels only during splits and merges.

Vì các giá trị trong B+-Tree chỉ được lưu ở mức lá, tất cả các thao tác (chèn, cập nhật, gỡ bỏ, và truy xuất bản ghi dữ liệu) chỉ ảnh hưởng tới các nút lá và chỉ lan truyền lên các mức cao hơn trong quá trình tách và gộp.

B + -Trees became widespread, and we refer to them as B-Trees, similar to other literature the subject. For example, in [GRAEFE11] B + -Trees are referred to as a default design, and MySQL InnoDB refers to its B + -Tree implementation as B-tree.

B+-Tree đã trở nên phổ biến rộng rãi, và chúng tôi gọi chúng là B-Tree, tương tự như các tài liệu khác về chủ đề này. Ví dụ, trong [GRAEFE11] B+-Tree được gọi là thiết kế mặc định, và MySQL InnoDB gọi hiện thực B+-Tree của nó là B-tree.

Separator Keys — Khóa phân tách

Keys stored in B-Tree nodes are called index entries , separator keys , or divider cells . They split the tree into subtrees (also called branches or subranges), holding corresponding key ranges. Keys are stored in sorted order to allow binary search. A subtree is found by locating a key and following a corresponding pointer from the higher to the lower level.

Các khóa lưu trong các nút B-Tree được gọi là mục chỉ mục (index entry), khóa phân tách (separator key), hoặc ô phân chia (divider cell). Chúng chia cây thành các cây con (còn gọi là nhánh hoặc khoảng con), chứa các khoảng khóa tương ứng. Các khóa được lưu theo thứ tự đã sắp xếp để cho phép tìm kiếm nhị phân. Một cây con được tìm thấy bằng cách định vị một khóa và đi theo con trỏ tương ứng từ mức cao hơn xuống mức thấp hơn.

The first pointer in the node points to the subtree holding items less than the first key, and the last pointer in the node points to the subtree holding items greater than or equal to the last key. Other pointers are reference subtrees between the two keys: K

Con trỏ đầu tiên trong nút trỏ tới cây con chứa các mục nhỏ hơn khóa đầu tiên, và con trỏ cuối cùng trong nút trỏ tới cây con chứa các mục lớn hơn hoặc bằng khóa cuối cùng. Các con trỏ khác tham chiếu các cây con nằm giữa hai khóa: K

$\leq K < K$, where K is a set of keys, and K is a key that belongs to the subtree.

$\leq K < K$, trong đó K là một tập khóa, và K là một khóa thuộc về cây con.

Figure 2-10 shows these invariants.

Hình 2-10 thể hiện các bất biến này.

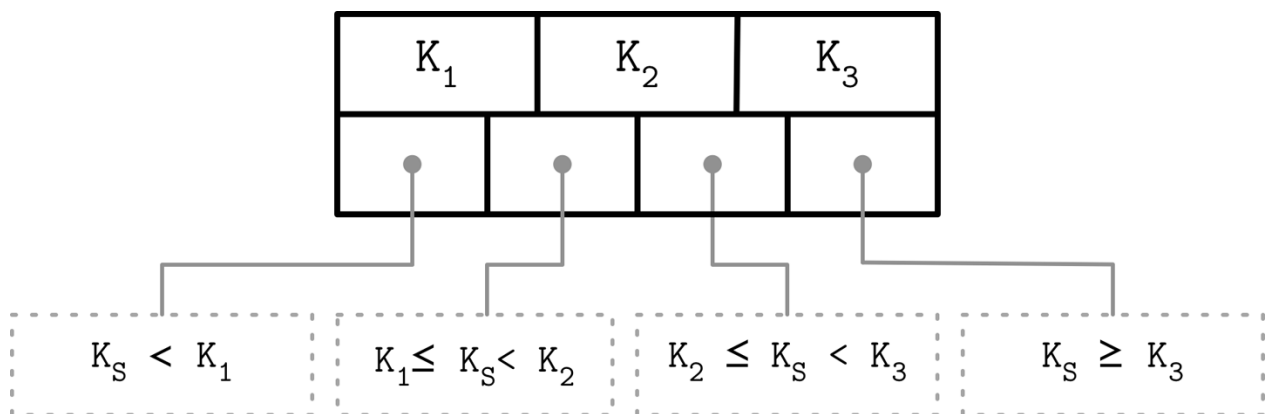


Figure 2-10. How separator keys split a tree into subtrees Some B-Tree variants also have sibling node pointers, most often on the leaf level, to simplify range scans. These pointers help avoid going back to the parent to find the next sibling. Some implementations have pointers in both directions, forming a double-linked list on the leaf level, which makes the reverse iteration possible.

Hình 2-10. Cách các khóa phân tách chia một cây thành các cây con. Một số biến thể B-Tree cũng có các con trỏ nút anh em (sibling), thường nhất là ở mức lá, để đơn giản hóa các quét khoảng (range scan). Các con trỏ này giúp tránh việc phải quay lại nút cha để tìm nút anh

em kế tiếp. Một số hiện thực có con trỏ theo cả hai hướng, tạo thành một danh sách liên kết đôi ở mức lá, giúp cho việc lặp ngược trở nên khả thi.

What sets B-Trees apart is that, rather than being built from top to bottom (as binary search trees), they're constructed the other way around—from bottom to top. The number of leaf nodes grows, which increases the number of internal nodes and tree height.

Điều làm cho B-Tree khác biệt là, thay vì được xây từ trên xuống dưới (như cây tìm kiếm nhị phân), chúng được xây theo chiều ngược lại—from dưới lên trên. Số lượng nút lá tăng lên, điều này làm tăng số lượng nút trong và chiều cao cây.

Since B-Trees reserve extra space inside nodes for future insertions and updates, tree storage utilization can get as low as 50%, but is usually considerably higher. Higher occupancy does not influence B-Tree performance negatively.

Vì B-Tree dành riêng không gian thừa bên trong các nút cho các lần chèn và cập nhật trong tương lai, mức sử dụng lưu trữ của cây có thể thấp tới 50%, nhưng thường cao hơn đáng kể. Độ lấp đầy cao hơn không ảnh hưởng tiêu cực tới hiệu năng B-Tree.

B-Tree Lookup Complexity — Độ phức tạp tra cứu B-Tree

B-Tree lookup complexity can be viewed from two standpoints: the number of block transfers and the number of comparisons done during the lookup.

Độ phức tạp tra cứu B-Tree có thể được xem xét từ hai góc độ: số lần chuyển khối (block transfer) và số lần so sánh thực hiện trong quá trình tra cứu.

In terms of number of transfers, the logarithm base is N (number of keys per node). There are K times more nodes on each new level, and following a child pointer reduces the search space by the factor of N . During lookup, at most $\log M$ (where M is

Xét về số lần chuyển, cơ số logarit là N (số khóa trên mỗi nút). Có gấp K lần số nút ở mỗi mức mới, và đi theo một con trỏ con làm giảm không gian tìm kiếm theo hệ số N . Trong quá trình tra cứu, nhiều nhất là $\log M$ (trong đó M là

a total number of items in the B-Tree) pages are addressed to find a searched key. The number of child pointers that have to be followed on the root-to-leaf pass is also equal to the number of levels, in other words, the height h of the tree.

tổng số mục trong B-Tree) trang được định địa chỉ để tìm một khóa cần tìm. Số con trỏ con phải đi theo trên đường từ gốc tới lá cũng bằng số mức, nói cách khác, là chiều cao h của cây.

From the perspective of number of comparisons, the logarithm base is 2, since searching a key inside each node is done using binary search. Every comparison halves the search space, so complexity is $\log M$.

Từ góc độ số lần so sánh, cơ số logarit là 2, vì việc tìm một khóa bên trong mỗi nút được thực hiện bằng tìm kiếm nhị phân. Mỗi lần so sánh chia đôi không gian tìm kiếm, nên độ phức tạp là $\log M$.

Knowing the distinction between the number of seeks and the number of comparisons helps us gain the intuition about how searches are performed and understand what lookup complexity is, from both perspectives.

Hiểu được sự phân biệt giữa số lần tìm kiếm (seek) và số lần so sánh giúp ta có được trực giác về cách thực hiện việc tìm kiếm và hiểu độ phức tạp tra cứu là gì, từ cả hai góc độ.

In textbooks and articles, 2 B-Tree lookup complexity is generally referenced as $\log M$. Logarithm base is generally not used in complexity analysis, since changing the base simply adds a constant factor, and multiplication by a constant factor does not change complexity. For example, given the nonzero constant factor c , $O(|c| \times n) == O(n)$ [KNUTH97].

Trong sách giáo khoa và bài báo, 2 độ phức tạp tra cứu B-Tree thường được tham chiếu là $\log M$. Cơ sở logarit thường không được dùng trong phân tích độ phức tạp, vì việc thay đổi cơ sở chỉ đơn giản thêm một hệ số hằng, và việc nhân với một hệ số hằng không làm thay đổi độ phức tạp. Ví dụ, cho trước hệ số hằng khác không c , $O(|c| \times n) == O(n)$ [KNUTH97].

2 For example, [KNUTH98].

2 Ví dụ, [KNUTH98].

B-Tree Lookup Algorithm — Thuật toán tra cứu B-Tree

Now that we have covered the structure and internal organization of B-Trees, we can define algorithms for lookups, insertions, and removals. To find an item in a B-Tree, we have to perform a single traversal from root to leaf. The objective of this search is to find a searched key or its predecessor. Finding an exact match is used for point queries, updates, and deletions; finding its predecessor is useful for range scans and inserts.

Bây giờ khi ta đã trình bày cấu trúc và tổ chức nội bộ của B-Tree, ta có thể định nghĩa các thuật toán cho tra cứu, chèn, và gỡ bỏ. Để tìm một mục trong B-Tree, ta phải thực hiện một lần duyệt duy nhất từ gốc tới lá. Mục tiêu của việc tìm kiếm này là tìm một khóa cần tìm hoặc phần tử đứng trước (predecessor) của nó. Tìm một khớp chính xác được dùng cho các truy vấn điểm, cập nhật, và xóa; tìm phần tử đứng trước của nó hữu ích cho các quét khoảng và chèn.

The algorithm starts from the root and performs a binary search, comparing the searched key with the keys stored in the root node until it finds the first **separator key** that is greater than the searched value. This locates a searched subtree. As we've discussed previously, index keys split the tree into subtrees with boundaries between two neighboring keys. As soon as we find the subtree, we follow the pointer that corresponds to it and continue the same search **process** (locate the separator key, follow the pointer) until we reach a target leaf node, where we either find the searched key or conclude it is not present by locating its predecessor.

Thuật toán bắt đầu từ gốc và thực hiện tìm kiếm nhị phân, so sánh khóa cần tìm với các khóa lưu trong nút gốc cho tới khi nó tìm thấy khóa phân tách đầu tiên lớn hơn giá trị cần tìm. Điều này định vị một cây con cần tìm. Như đã thảo luận trước đó, các khóa chỉ mục chia cây thành các cây con với ranh giới giữa hai khóa lân cận. Ngay khi ta tìm thấy cây con, ta đi theo con trỏ tương ứng với nó và tiếp tục cùng quá trình tìm kiếm (định vị khóa phân tách, đi theo con trỏ) cho tới khi ta tới một nút lá đích, nơi ta hoặc tìm thấy khóa cần tìm hoặc kết luận nó không hiện diện bằng cách định vị phần tử đứng trước của nó.

On each level, we get a more detailed view of the tree: we start on the most coarse-grained level (the root of the tree) and descend to the next level where keys represent more precise, detailed ranges, until we finally reach leaves, where the data records are located.

Ở mỗi mức, ta có được một cái nhìn chi tiết hơn về cây: ta bắt đầu ở mức thô nhất (gốc của cây) và đi xuống mức kế tiếp nơi các khóa biểu diễn các khoảng chính xác, chi tiết hơn,

cho tới khi cuối cùng ta tới các lá, nơi đặt các bản ghi dữ liệu.

During the point query, the search is done after finding or failing to find the searched key. During the range scan, iteration starts from the closest found key-value pair and continues by following sibling pointers until the end of the range is reached or the range predicate is exhausted.

Trong truy vấn điểm, việc tìm kiếm hoàn tất sau khi tìm thấy hoặc thất bại trong việc tìm khóa cần tìm. Trong quét khoảng, việc lặp bắt đầu từ cặp khóa-giá trị gần nhất tìm được và tiếp tục bằng cách đi theo các con trỏ anh em cho tới khi đạt tới cuối khoảng hoặc vị trí khoảng đã cạn.

Counting Keys — Đếm khóa

Across the literature, you can find different ways to describe key and child **offset** counts. [BAYER72] mentions the device-dependent natural number k that represents an optimal page size. Pages, in this case, can hold between k and $2k$ keys, but can be partially filled and hold at least $k + 1$ and at most $2k + 1$ pointers to child nodes. The root page can hold between 1 and $2k$ keys. Later, a number l is introduced, and it is said that any nonleaf page can have $l + 1$ keys.

Xuyên suốt các tài liệu, bạn có thể tìm thấy nhiều cách khác nhau để mô tả số lượng khóa và độ dài con. [BAYER72] đề cập tới số tự nhiên phụ thuộc thiết bị k đại diện cho kích thước trang tối ưu. Trang, trong trường hợp này, có thể chứa giữa k và $2k$ khóa, nhưng có thể được lấp đầy một phần và chứa ít nhất $k + 1$ và nhiều nhất $2k + 1$ con trỏ tới các nút con. Trang gốc có thể chứa giữa 1 và $2k$ khóa. Sau đó, một số l được giới thiệu, và nói rằng bất kỳ trang không-phải-lá nào cũng có thể có $l + 1$ khóa.

Other sources, for example [GRAEFE11], describe nodes that can hold up to N separator keys and $N + 1$ pointers, with otherwise similar semantics and invariants.

Các nguồn khác, ví dụ [GRAEFE11], mô tả các nút có thể chứa tới N khóa phân tách và $N + 1$ con trỏ, với ngữ nghĩa và bất biến tương tự về các mặt khác.

Both approaches bring us to the same result, and differences are only used to emphasize the contents of each source. In this book, we stick to N as the number of keys (or key-value pairs, in the case of the leaf nodes) for clarity.

Cả hai cách tiếp cận đều dẫn ta tới cùng một kết quả, và sự khác biệt chỉ được dùng để nhấn mạnh nội dung của mỗi nguồn. Trong cuốn sách này, chúng tôi giữ N là số khóa (hoặc cặp khóa-giá trị, trong trường hợp các nút lá) cho rõ ràng.

B-Tree Node Splits — Tách nút B-Tree

To insert the value into a B-Tree, we first have to locate the target leaf and find the insertion point. For that, we use the algorithm described in the previous section. After the leaf is located, the key and value are appended to it. Updates in B-Trees work by locating a target leaf node using a lookup algorithm and associating a new value with an existing key.

Để chèn giá trị vào một B-Tree, trước tiên ta phải định vị lá đích và tìm điểm chèn. Để làm điều đó, ta dùng thuật toán đã mô tả ở phần trước. Sau khi lá được định vị, khóa và giá trị được nối vào nó. Các cập nhật trong B-Tree hoạt động bằng cách định vị một nút lá đích dùng thuật toán tra cứu và gán một giá trị mới với một khóa hiện có.

If the target node doesn't have enough room available, we say that the node has overflowed [NICHOLS66] and has to be split in two to fit the new data. More precisely, the node is split if the following conditions hold:

Nếu nút đích không có đủ chỗ trống, ta nói rằng nút đã tràn (overflowed) [NICHOLS66] và phải được tách làm hai để vừa với dữ liệu mới. Cụ thể hơn, nút được tách nếu các điều kiện sau đây thỏa mãn:

- For leaf nodes: if the node can hold up to N key-value pairs, and inserting one more key-value pair brings it over its maximum capacity N .
- For nonleaf nodes: if the node can hold up to $N + 1$ pointers, and inserting one more pointer brings it over its maximum capacity $N + 1$.
 - Đối với nút lá: nếu nút có thể chứa tới N cặp khóa-giá trị, và việc chèn thêm một cặp khóa-giá trị nữa đưa nó vượt quá sức chứa tối đa N của nó.
 - Đối với nút không-phải-lá: nếu nút có thể chứa tới $N + 1$ con trỏ, và việc chèn thêm một con trỏ nữa đưa nó vượt quá sức chứa tối đa $N + 1$ của nó.

Splits are done by allocating the new node, transferring half the elements from the splitting node to it, and adding its first key and pointer to the parent node. In this case, we say that the key is promoted. The index at which the split is performed is called the split point (also called the midpoint). All elements after the split point (including split point in the case of nonleaf **node split**) are transferred to the newly created sibling node, and the rest of the elements remain in the splitting node.

Việc tách được thực hiện bằng cách cấp phát nút mới, chuyển một nửa các phần tử từ nút đang bị tách sang nó, và thêm khóa và con trỏ đầu tiên của nó vào nút cha. Trong trường hợp này, ta nói rằng khóa được thăng (promoted). Chỉ số mà tại đó việc tách được thực hiện được gọi là điểm tách (split point) (còn gọi là điểm giữa - midpoint). Tất cả các phần tử sau điểm tách (bao gồm điểm tách trong trường hợp tách nút không-phải-lá) được chuyển sang nút anh em mới tạo, và phần còn lại của các phần tử ở lại trong nút đang bị tách.

If the parent node is full and does not have space available for the promoted key and pointer to the newly created node, it has to be split as well. This operation might propagate recursively all the way to the root.

Nếu nút cha đầy và không có không gian trống cho khóa được thăng và con trỏ tới nút mới tạo, nó cũng phải được tách. Thao tác này có thể lan truyền đệ quy lên tận gốc.

As soon as the tree reaches its capacity (i.e., split propagates all the way up to the root), we have to split the root node. When the root node is split, a new root, holding a split point key, is allocated. The old root (now holding only half the entries) is demoted to the next level along with its newly created sibling, increasing the tree height by one. The tree height changes when the root node is split and the new root is allocated, or when two nodes are merged to form a new root. On the leaf and internal node levels, the tree only grows horizontally.

Ngay khi cây đạt tới sức chứa của nó (tức là việc tách lan truyền lên tận gốc), ta phải tách nút gốc. Khi nút gốc được tách, một gốc mới, chứa khóa điểm tách, được cấp phát. Gốc cũ (giờ chỉ chứa một nửa số mục) bị giáng (demoted) xuống mức kế tiếp cùng với nút anh em mới tạo của nó, làm tăng chiều cao cây lên một. Chiều cao cây thay đổi khi nút gốc được tách và gốc mới được cấp phát, hoặc khi hai nút được gộp để tạo thành một gốc mới. Ở các mức nút lá và nút trong, cây chỉ phát triển theo chiều ngang.

Figure 2-11 shows a fully occupied leaf node during insertion of the new element 11. We draw the line in the middle of the full node, leave half the elements in the node, and move the rest of elements

to the new one. A split point value is placed into the parent node to serve as a separator key.

Hình 2-11 thể hiện một nút lá được lấp đầy hoàn toàn trong quá trình chèn phần tử mới 11. Ta vẽ đường ở giữa nút đầy, để lại một nửa số phần tử trong nút, và chuyển phần còn lại của các phần tử sang nút mới. Một giá trị điểm tách được đặt vào nút cha để đóng vai trò khóa phân tách.

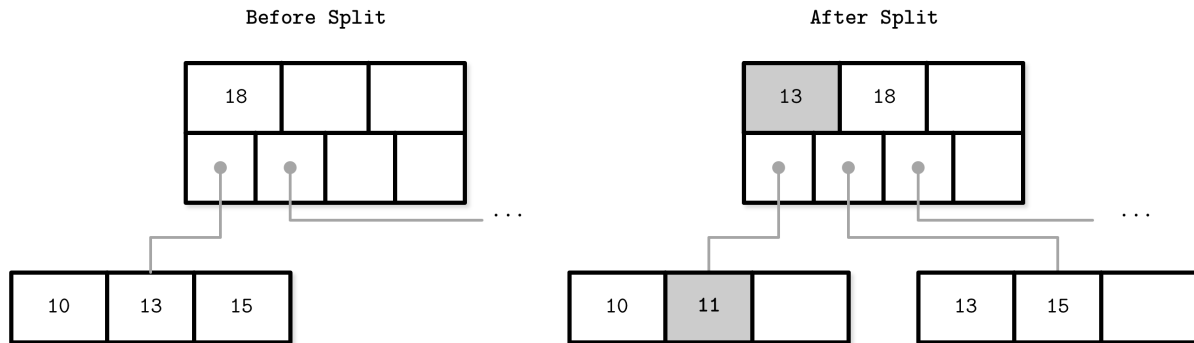


Figure 2-11. Leaf node split during the insertion of 11 . New element and promoted key are shown in gray.

Hình 2-11. Tách nút lá trong quá trình chèn 11. Phần tử mới và khóa được thăng được thể hiện bằng màu xám.

Figure 2-12 shows the split process of a fully occupied nonleaf (i.e., root or internal) node during insertion of the new element 11 . To perform a split, we first create a new node and move elements starting from index $N/2 + 1$ to it. The split point key is promoted to the parent.

Hình 2-12 thể hiện quá trình tách của một nút không-phải-lá (tức là gốc hoặc nút trong) được lấp đầy hoàn toàn trong quá trình chèn phần tử mới 11. Để thực hiện việc tách, trước tiên ta tạo một nút mới và chuyển các phần tử bắt đầu từ chỉ số $N/2 + 1$ sang nó. Khóa điểm tách được thăng lên nút cha.

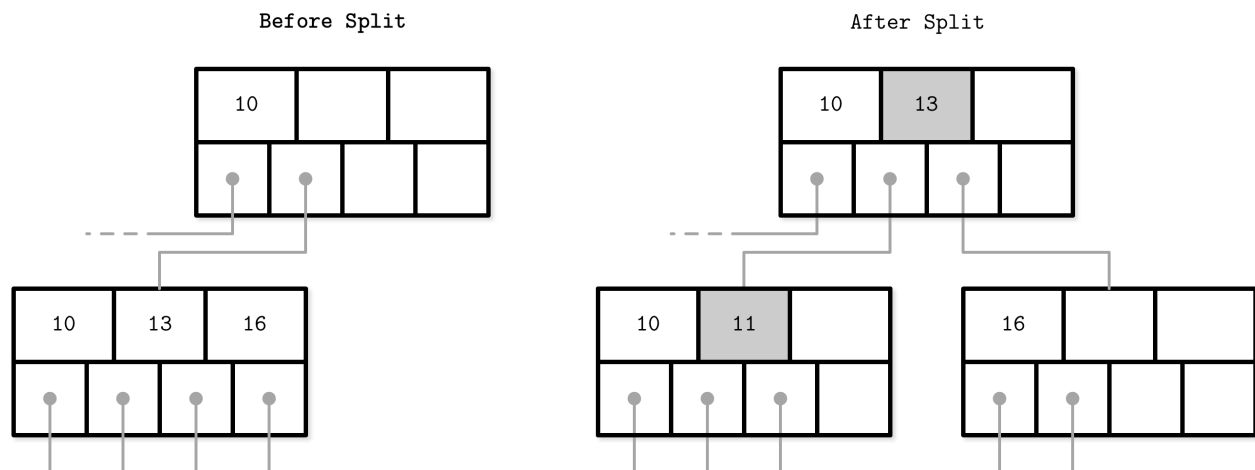


Figure 2-12. Nonleaf node split during the insertion of 11 . New element and promoted key are shown in gray.

Hình 2-12. Tách nút không-phải-lá trong quá trình chèn 11. Phần tử mới và khóa được thăng được thể hiện bằng màu xám.

Since nonleaf node splits are always a manifestation of splits propagating from the levels below, we have an additional pointer (to the newly created node on the next level). If the parent does not have enough space, it has to be split as well.

Vì các lần tách nút không-phải-lá luôn là biểu hiện của các lần tách lan truyền từ các mức bên dưới, ta có thêm một con trỏ (tới nút mới tạo ở mức kế tiếp). Nếu nút cha không có đủ không gian, nó cũng phải được tách.

It doesn't matter whether the leaf or nonleaf node is split (i.e., whether the node holds keys and values or just the keys). In the case of leaf split, keys are moved together with their associated values.

Không quan trọng việc nút lá hay nút không-phải-lá được tách (tức là việc nút chứa khóa và giá trị hay chỉ chứa khóa). Trong trường hợp tách lá, các khóa được chuyển cùng với các giá trị gắn với chúng.

When the split is done, we have two nodes and have to pick the correct one to finish insertion. For that, we can use the separator key invariants. If the inserted key is less than the promoted one, we finish the operation by inserting to the split node. Otherwise, we insert to the newly created one.

Khi việc tách hoàn tất, ta có hai nút và phải chọn đúng nút để hoàn tất việc chèn. Để làm điều đó, ta có thể dùng các bất biến của khóa phân tách. Nếu khóa được chèn nhỏ hơn khóa được thăng, ta hoàn tất thao tác bằng cách chèn vào nút bị tách. Ngược lại, ta chèn vào nút mới tạo.

To summarize, node splits are done in four steps:

Tóm lại, các lần tách nút được thực hiện trong bốn bước:

1. Allocate a new node.
 2. Copy half the elements from the splitting node to the new one.
 3. Place the new element into the corresponding node.
 4. At the parent of the split node, add a separator key and a pointer to the new node.
1. Cấp phát một nút mới.
 2. Sao chép một nửa các phần tử từ nút đang bị tách sang nút mới.
 3. Đặt phần tử mới vào nút tương ứng.
 4. Tại nút cha của nút bị tách, thêm một khóa phân tách và một con trỏ tới nút mới.

B-Tree Node Merges — Gộp nút B-Tree

Deletions are done by first locating the target leaf. When the leaf is located, the key and the value associated with it are removed.

Các lần xóa được thực hiện bằng cách trước tiên định vị lá đích. Khi lá được định vị, khóa và giá trị gắn với nó được gỡ bỏ.

If neighboring nodes have too few values (i.e., their occupancy falls under a threshold), the sibling nodes are merged. This situation is called underflow . [BAYER72] describes two underflow scenarios: if two adjacent nodes have a common parent and their contents fit into a single node, their contents should be merged (concatenated); if their contents do not fit into a single node, keys are redistributed between them to restore balance (see “**Rebalancing**” on page 70). More precisely, two nodes are merged if the following conditions hold:

Nếu các nút lân cận có quá ít giá trị (tức là độ lấp đầy của chúng rơi xuống dưới một ngưỡng), các nút anh em được gộp. Tình huống này được gọi là thiếu hụt (underflow). [BAYER72] mô tả hai kịch bản thiếu hụt: nếu hai nút liền kề có chung một nút cha và nội

dung của chúng vừa vào một nút duy nhất, nội dung của chúng nên được gộp (nối lại); nếu nội dung của chúng không vừa vào một nút duy nhất, các khóa được tái phân phối giữa chúng để khôi phục cân bằng (xem “Rebalancing” ở trang 70). Cụ thể hơn, hai nút được gộp nếu các điều kiện sau đây thỏa mãn:

- For leaf nodes: if a node can hold up to N key-value pairs, and a combined number of key-value pairs in two neighboring nodes is less than or equal to N .
 - For nonleaf nodes: if a node can hold up to $N + 1$ pointers, and a combined number of pointers in two neighboring nodes is less than or equal to $N + 1$.
- Đối với nút lá: nếu một nút có thể chứa tới N cặp khóa-giá trị, và tổng số cặp khóa-giá trị trong hai nút lân cận nhỏ hơn hoặc bằng N .
- Đối với nút không-phải-lá: nếu một nút có thể chứa tới $N + 1$ con trỏ, và tổng số con trỏ trong hai nút lân cận nhỏ hơn hoặc bằng $N + 1$.

Figure 2-13 shows the **merge** during deletion of element 16. To do this, we move elements from one of the siblings to the other one. Generally, elements from the right sibling are moved to the left one, but it can be done the other way around as long as the key order is preserved.

Hình 2-13 thể hiện việc gộp trong quá trình xóa phần tử 16. Để làm điều này, ta chuyển các phần tử từ một trong các nút anh em sang nút kia. Nhìn chung, các phần tử từ nút anh em bên phải được chuyển sang nút bên trái, nhưng nó có thể được thực hiện theo chiều ngược lại miễn là thứ tự khóa được bảo toàn.

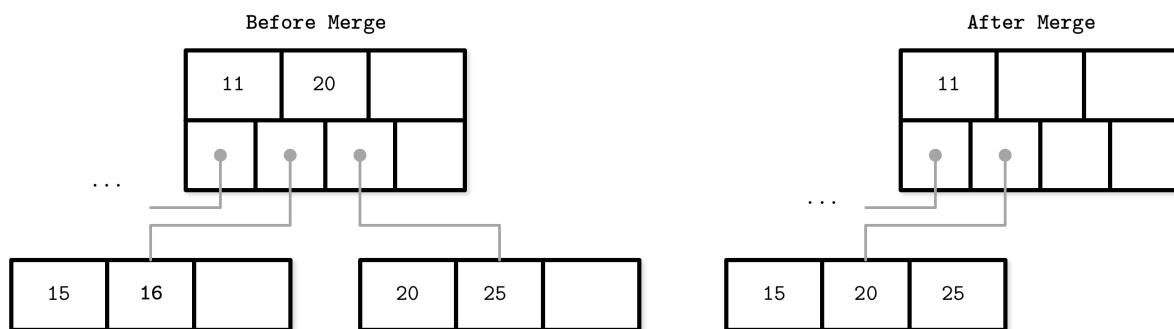


Figure 2-13. Leaf node merge Figure 2-14 shows two sibling nonleaf nodes that have to be merged during deletion of element 10. If we combine their elements, they fit into one node, so we can have one node instead of two. During the merge of nonleaf nodes, we have to pull the corresponding separator key from the parent (i.e., demote it). The number of pointers is reduced by one because the merge is a result of the propagation of the pointer deletion from the lower level, caused by the page removal. Just as with splits, merges can propagate all the way to the root level.

Hình 2-13. Gộp nút lá. Hình 2-14 thể hiện hai nút không-phải-lá anh em phải được gộp trong quá trình xóa phần tử 10. Nếu ta kết hợp các phần tử của chúng, chúng vừa vào một nút, nên ta có thể có một nút thay vì hai. Trong quá trình gộp các nút không-phải-lá, ta phải kéo khóa phân tách tương ứng từ nút cha xuống (tức là giáng nó). Số con trỏ giảm đi một, vì việc gộp này là hệ quả của sự lan truyền việc xóa con trỏ từ mức thấp hơn, do trang bị gỡ bỏ gây ra. Cũng giống như với các lần tách, các lần gộp có thể lan truyền lên tận mức gốc.

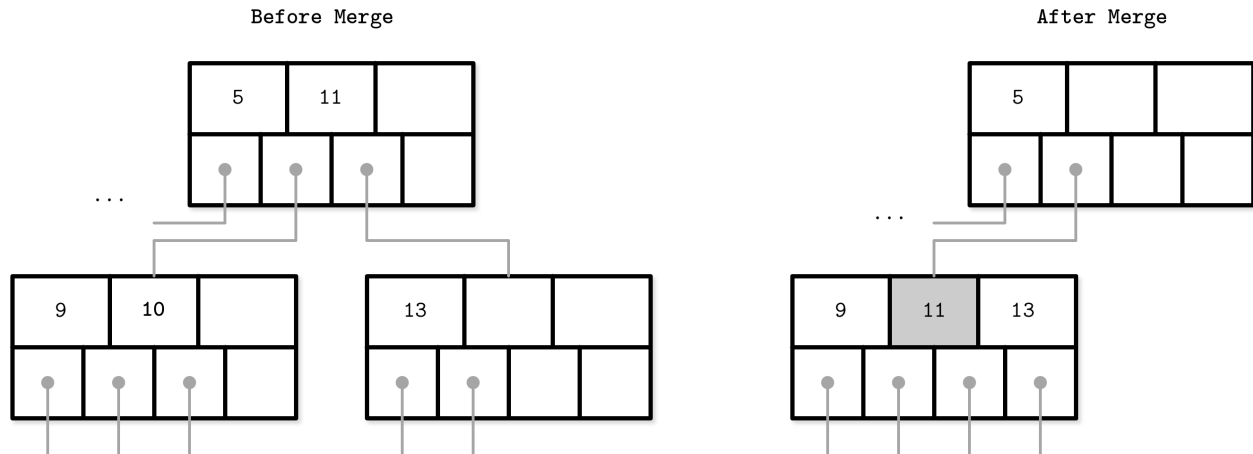


Figure 2-14. Nonleaf node merge

Hình 2-14. Gộp nút không-phải-lá

To summarize, node merges are done in three steps, assuming the element is already removed:

Tóm lại, các lần gộp nút được thực hiện trong ba bước, giả định phần tử đã được gỡ bỏ:

1. Copy all elements from the right node to the left one.
 2. Remove the right node pointer from the parent (or demote it in the case of a nonleaf merge).
 3. Remove the right node.
1. Sao chép tất cả các phần tử từ nút bên phải sang nút bên trái.
 2. Gỡ bỏ con trỏ nút bên phải khỏi nút cha (hoặc giáng nó trong trường hợp gộp nút không-phải-lá).
 3. Gỡ bỏ nút bên phải.

One of the techniques often implemented in B-Trees to reduce the number of splits and merges is rebalancing, which we discuss in “Rebalancing” on page 70 .

Một trong những kỹ thuật thường được hiện thực trong B-Tree để giảm số lần tách và gộp là tái cân bằng (rebalancing), mà ta thảo luận trong “Rebalancing” ở trang 70.

Summary — Tóm tắt

In this chapter, we started with a motivation to create specialized structures for on-disk storage. Binary search trees might have similar complexity characteristics, but still fall short of being suitable for disk because of low fanout and a large number of relocations and pointer updates caused by balancing. B-Trees solve both problems by increasing the number of items stored in each node (high fanout) and less frequent balancing operations.

Trong chương này, chúng ta bắt đầu với động lực tạo ra các cấu trúc chuyên biệt cho lưu trữ trên đĩa. Cây tìm kiếm nhị phân có thể có các đặc điểm độ phức tạp tương tự, nhưng vẫn chưa đủ để phù hợp với đĩa vì fanout thấp và số lượng lớn các lần di dời và cập nhật con trỏ gây ra bởi việc cân bằng. B-Tree giải quyết cả hai vấn đề bằng cách tăng số lượng mục lưu trong mỗi nút (fanout cao) và các thao tác cân bằng ít thường xuyên hơn.

After that, we discussed internal B-Tree structure and outlines of algorithms for lookup, insert, and delete operations. Split and merge operations help to restructure the tree to keep it balanced while

adding and removing elements. We keep the tree depth to a minimum and add items to the existing nodes while there's still some free space in them.

Sau đó, chúng ta thảo luận về cấu trúc nội bộ của B-Tree và phác thảo các thuật toán cho các thao tác tra cứu, chèn, và xóa. Các thao tác tách và gộp giúp tái cấu trúc cây để giữ nó cân bằng trong khi thêm và gỡ bỏ các phần tử. Chúng ta giữ độ sâu của cây ở mức tối thiểu và thêm các mục vào các nút hiện có khi trong chúng vẫn còn một số không gian trống.

We can use this knowledge to create in-memory B-Trees. To create a disk-based implementation, we need to go into details of how to lay out B-Tree nodes on disk and compose on-disk layout using data-encoding formats.

Chúng ta có thể dùng kiến thức này để tạo các B-Tree trong bộ nhớ. Để tạo một hiện thực dựa trên đĩa, ta cần đi vào chi tiết về cách bố trí các nút B-Tree trên đĩa và soạn bố cục trên đĩa dùng các định dạng mã hóa dữ liệu.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Binary search trees Sedgewick, Robert and Kevin Wayne. 2011. Algorithms (4th Ed.) . Boston: Pearson.

Knuth, Donald E. 1997. The Art of Computer Programming, Volume 2 (3rd Ed.): Seminumerical Algorithms . Boston: Addison-Wesley Longman.

Algorithms for splits and merges in B-Trees Elmasri, Ramez and Shamkant Navathe. 2011. Fundamentals of Database Systems (6th Ed.) . Boston: Pearson.

Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. 2010. Database Systems Concepts (6th Ed.) . New York: McGraw-Hill.

CHAPTER 3 File Formats — CHƯƠNG 3 Định dạng tệp (File Formats)

With the basic semantics of B-Trees covered, we are now ready to explore how exactly B-Trees and other structures are implemented on disk. We access the disk in a way that is different from how we access main memory: from an application developer’s perspective, memory accesses are mostly transparent. Because of virtual memory [BHATTACHARJEE17], we do not have to manage offsets manually. Disks are accessed using system calls (see <https://databass.dev/links/54>). We usually have to specify the **offset** inside the target file, and then interpret on-disk representation into a form suitable for main memory.

Sau khi đã nắm được ngữ nghĩa cơ bản của B-Tree, giờ chúng ta đã sẵn sàng khám phá B-Tree và các cấu trúc khác thực sự được hiện thực trên đĩa như thế nào. Chúng ta truy cập đĩa theo cách khác với cách truy cập bộ nhớ chính: dưới góc nhìn của lập trình viên ứng dụng, các truy cập bộ nhớ phần lớn là trong suốt. Nhờ bộ nhớ ảo [BHATTACHARJEE17], chúng ta không phải quản lý các độ dời (offset) một cách thủ công. Đĩa được truy cập bằng các lời gọi hệ thống (xem <https://databass.dev/links/54>). Thông thường chúng ta phải chỉ định độ dời bên trong tệp đích, rồi diễn giải biểu diễn trên đĩa thành dạng phù hợp cho bộ nhớ chính.

This means that efficient on-disk structures have to be designed with this distinction in mind. To do that, we have to come up with a file format that’s easy to construct, modify, and interpret. In this chapter, we’ll discuss general principles and practices that help us to design all sorts of on-disk structures, not only B-Trees.

Điều này có nghĩa là các cấu trúc trên đĩa hiệu quả phải được thiết kế với sự khác biệt này trong tâm trí. Để làm được điều đó, chúng ta phải nghĩ ra một định dạng tệp dễ dựng, dễ sửa đổi và dễ diễn giải. Trong chương này, chúng ta sẽ thảo luận các nguyên lý và thực hành chung giúp thiết kế đủ loại cấu trúc trên đĩa, không chỉ riêng B-Tree.

There are numerous possibilities for **B-Tree** implementations, and here we discuss several useful techniques. Details may vary between implementations, but the general principles remain the same. Understanding the basic mechanics of B-Trees, such as splits and merges, is necessary, but they are insufficient for the actual implementation. There are many things that have to play together for the final result to be useful.

Có vô số khả năng hiện thực B-Tree, và ở đây chúng ta bàn về một vài kỹ thuật hữu ích. Chi tiết có thể khác nhau giữa các bản hiện thực, nhưng các nguyên lý chung vẫn giữ nguyên. Hiểu cơ chế cơ bản của B-Tree, chẳng hạn như tách (split) và gộp (merge), là cần thiết, nhưng vẫn chưa đủ cho việc hiện thực thực tế. Có nhiều thứ phải phối hợp với nhau thì kết quả cuối cùng mới trở nên hữu dụng.

The semantics of pointer management in on-disk structures are somewhat different from in-memory ones. It is useful to think of on-disk B-Trees as a **page** management mechanism: algorithms have to compose and navigate pages. Pages and pointers to them have to be calculated and placed accordingly.

Ngữ nghĩa của việc quản lý con trỏ trong các cấu trúc trên đĩa hơi khác so với các cấu trúc trong bộ nhớ. Sẽ hữu ích nếu coi B-Tree trên đĩa như một cơ chế quản lý trang (page): các thuật toán phải dựng và điều hướng trên các trang. Các trang và con trỏ trỏ tới chúng phải được tính toán và đặt vào vị trí tương ứng.

45 Since most of the complexity in B-Trees comes from mutability, we discuss details of page layouts, splitting, relocations, and other concepts applicable to mutable data structures. Later, when talking about LSM Trees (see “LSM Trees” on page 130), we focus on sorting and maintenance, since that’s where most LSM complexity comes from.

45 Vì phần lớn độ phức tạp trong B-Tree đến từ tính khả biến (mutability), chúng ta sẽ thảo luận chi tiết về cách bố trí trang (page layout), tách trang, di dời (relocation), và các khái niệm khác áp dụng cho các cấu trúc dữ liệu khả biến. Sau này, khi nói về LSM Tree (xem “LSM Trees” ở trang 130), chúng ta sẽ tập trung vào sắp xếp và bảo trì, vì đó là nơi phần lớn độ phức tạp của LSM phát sinh.

Motivation — Động lực

Creating a file format is in many ways similar to how we create data structures in languages with an unmanaged memory model. We allocate a **block** of data and slice it any way we like, using fixed-size primitives and structures. If we want to reference a larger chunk of memory or a structure with variable size, we use pointers.

Việc tạo một định dạng tệp về nhiều mặt giống với cách chúng ta tạo cấu trúc dữ liệu trong các ngôn ngữ có mô hình bộ nhớ không được quản lý (unmanaged memory model). Chúng ta cấp phát một khối dữ liệu và cắt nó theo bất kỳ cách nào ta muốn, sử dụng các kiểu nguyên thủy (primitive) và cấu trúc có kích thước cố định. Nếu muốn tham chiếu tới một mảng bộ nhớ lớn hơn hoặc một cấu trúc có kích thước thay đổi, chúng ta dùng con trỏ.

Languages with an unmanaged memory model allow us to allocate more memory any time we need (within reasonable bounds) without us having to think or worry about whether or not there’s a contiguous memory segment available, whether or not it is fragmented, or what happens after we free it. On disk, we have to take care of **garbage collection** and **fragmentation** ourselves.

Các ngôn ngữ có mô hình bộ nhớ không được quản lý cho phép chúng ta cấp phát thêm bộ nhớ bất cứ khi nào cần (trong giới hạn hợp lý) mà không phải suy nghĩ hay lo lắng về việc liệu có sẵn một đoạn bộ nhớ liền kề hay không, liệu nó có bị phân mảnh (fragmented) hay không, hay điều gì xảy ra sau khi ta giải phóng nó. Trên đĩa, chúng ta phải tự lo việc thu gom rác (garbage collection) và phân mảnh.

Data layout is much less important in memory than on disk. For a disk-resident data structure to be efficient, we need to lay out data on disk in ways that allow quick access to it, and consider the specifics of a persistent storage medium, come up with binary data formats, and find a means to serialize and deserialize data efficiently.

Cách bố trí dữ liệu ít quan trọng hơn nhiều trong bộ nhớ so với trên đĩa. Để một cấu trúc dữ liệu nằm trên đĩa được hiệu quả, chúng ta cần bố trí dữ liệu trên đĩa theo cách cho phép truy cập nhanh, đồng thời cân nhắc các đặc thù của môi trường lưu trữ bền vững,

nghĩ ra các định dạng dữ liệu nhị phân, và tìm cách tuần tự hóa (serialize) cũng như giải tuần tự hóa (deserialize) dữ liệu một cách hiệu quả.

Anyone who has ever used a low-level language such as C without additional libraries knows the constraints. Structures have a predefined size and are allocated and freed explicitly. Manually implementing memory allocation and tracking is even more challenging, since it is only possible to operate with memory segments of predefined size, and it is necessary to **track** which segments are already released and which ones are still in use.

Bất kỳ ai từng dùng một ngôn ngữ cấp thấp như C mà không có thư viện bổ sung đều biết những ràng buộc đó. Các cấu trúc có kích thước định trước và được cấp phát rồi giải phóng một cách tường minh. Việc tự hiện thực cấp phát và theo dõi bộ nhớ thậm chí còn khó khăn hơn, vì chỉ có thể thao tác với các đoạn bộ nhớ có kích thước định trước, và cần phải theo dõi đoạn nào đã được giải phóng và đoạn nào vẫn đang được dùng.

When storing data in main memory, most of the problems with memory layout do not exist, are easier to solve, or can be solved using third-party libraries. For example, handling variable-length fields and oversize data is much more straightforward, since we can use memory allocation and pointers, and do not need to lay them out in any special way. There still are cases when developers design specialized main memory data layouts to take advantage of CPU cache lines, prefetching, and other hardware-related specifics, but this is mainly done for optimization purposes [FOWLER11] .

Khi lưu dữ liệu trong bộ nhớ chính, hầu hết các vấn đề về bố trí bộ nhớ đều không tồn tại, dễ giải quyết hơn, hoặc có thể được giải quyết bằng các thư viện bên thứ ba. Ví dụ, việc xử lý các trường có độ dài thay đổi và dữ liệu quá khổ đơn giản hơn nhiều, vì chúng ta có thể dùng cấp phát bộ nhớ và con trỏ, và không cần bố trí chúng theo bất kỳ cách đặc biệt nào. Vẫn có những trường hợp mà lập trình viên thiết kế các cách bố trí dữ liệu chuyên biệt trong bộ nhớ chính để tận dụng các dòng cache CPU (CPU cache line), nạp trước (prefetching), và các đặc thù phần cứng khác, nhưng điều này chủ yếu được làm vì mục đích tối ưu hóa [FOWLER11].

Even though the operating system and filesystem take over some of the responsibilities, implementing on-disk structures requires attention to more details and has more pitfalls.

Mặc dù hệ điều hành và hệ thống tệp gánh bớt một phần trách nhiệm, việc hiện thực các cấu trúc trên đĩa vẫn đòi hỏi sự chú ý tới nhiều chi tiết hơn và có nhiều cạm bẫy hơn.

Binary Encoding — Mã hóa nhị phân

To store data on disk efficiently, it needs to be encoded using a format that is compact and easy to serialize and deserialize. When talking about binary formats, you hear the word layout quite often. Since we do not have primitives such as malloc and free , but only **read** and write , we have to think of accesses differently and prepare data accordingly.

Để lưu dữ liệu trên đĩa hiệu quả, dữ liệu cần được mã hóa bằng một định dạng gọn nhẹ và dễ tuần tự hóa, giải tuần tự hóa. Khi nói về các định dạng nhị phân, bạn sẽ thường nghe từ “bố trí” (layout). Vì chúng ta không có các nguyên thủy như malloc và free, mà chỉ có read và write, nên chúng ta phải tư duy về truy cập theo cách khác và chuẩn bị dữ liệu cho phù hợp.

Here, we discuss the main principles used to create efficient page layouts. These principles apply to any binary format: you can use similar guidelines to create file and serialization formats or communication protocols.

Ở đây, chúng ta thảo luận các nguyên lý chính dùng để tạo các cách bố trí trang hiệu quả. Các nguyên lý này áp dụng cho mọi định dạng nhị phân: bạn có thể dùng các hướng dẫn tương tự để tạo định dạng tệp và định dạng tuần tự hóa, hoặc các giao thức truyền thông.

Before we can organize records into pages, we need to understand how to represent keys and data records in binary form, how to combine multiple values into more complex structures, and how to implement variable-size types and arrays.

Trước khi có thể tổ chức các bản ghi thành các trang, chúng ta cần hiểu cách biểu diễn khóa và các bản ghi dữ liệu ở dạng nhị phân, cách kết hợp nhiều giá trị thành các cấu trúc phức tạp hơn, và cách hiện thực các kiểu có kích thước thay đổi cùng các mảng.

Primitive Types — Các kiểu nguyên thủy

Keys and values have a type, such as integer, date, or string, and can be represented (serialized to and deserialized from) in their raw binary forms.

Khóa và giá trị có một kiểu (type), chẳng hạn như số nguyên (integer), ngày (date), hoặc chuỗi (string), và có thể được biểu diễn (tuần tự hóa thành và giải tuần tự hóa từ) ở dạng nhị phân thô của chúng.

Most numeric data types are represented as fixed-size values. When working with multibyte numeric values, it is important to use the same byte-order (endianness) for both encoding and decoding. Endianness determines the **sequential** order of bytes:

Hầu hết các kiểu dữ liệu số được biểu diễn dưới dạng các giá trị có kích thước cố định. Khi làm việc với các giá trị số đa byte, điều quan trọng là phải dùng cùng một thứ tự byte (byte-order, hay endianness) cho cả mã hóa và giải mã. Endianness quyết định thứ tự tuần tự của các byte:

Big-endian The order starts from the most-significant byte (MSB), followed by the bytes in decreasing significance order. In other words, MSB has the lowest address.

Big-endian Thứ tự bắt đầu từ byte có trọng số lớn nhất (most-significant byte, MSB), theo sau bởi các byte theo thứ tự trọng số giảm dần. Nói cách khác, MSB có địa chỉ thấp nhất.

Little-endian The order starts from the least-significant byte (LSB), followed by the bytes in increasing significance order.

Little-endian Thứ tự bắt đầu từ byte có trọng số nhỏ nhất (least-significant byte, LSB), theo sau bởi các byte theo thứ tự trọng số tăng dần.

Figure 3-1 illustrates this. The hexadecimal 32-bit integer 0xAABBCCDD, where AA is the MSB, is shown using both big- and little-endian byte order.

Hình 3-1 minh họa điều này. Số nguyên 32-bit ở dạng thập lục phân 0xAABBCCDD, trong đó AA là MSB, được thể hiện bằng cả thứ tự byte big-endian lẫn little-endian.

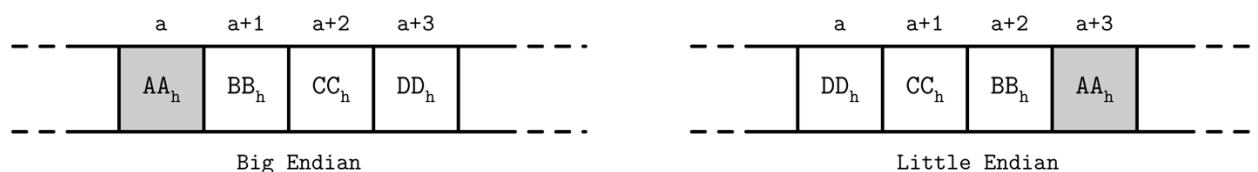


Figure 3-1. Big- and little-endian byte order. The most significant byte is shown in gray. Addresses, denoted by a, grow from left to right.

Hình 3-1. Thứ tự byte big-endian và little-endian. Byte có trọng số lớn nhất được tô màu xám. Các địa chỉ, ký hiệu bằng a, tăng từ trái sang phải.

For example, to reconstruct a 64-bit integer with a corresponding byte order, RocksDB has platform-specific definitions that help to identify target platform byte order . 1 If the target platform endianness does not match value endianness (EncodeFixed64WithEndian looks up kLittleEndian value and compares it with value endianness), it reverses the bytes using EndianTransform , which reads values byte-wise in reverse order and appends them to the result.

Ví dụ, để tái dựng một số nguyên 64-bit với thứ tự byte tương ứng, RocksDB có các định nghĩa đặc thù theo nền tảng giúp xác định thứ tự byte của nền tảng đích. 1 Nếu endianness của nền tảng đích không khớp với endianness của giá trị (EncodeFixed64WithEndian tra cứu giá trị kLittleEndian và so sánh nó với endianness của giá trị), nó sẽ đảo các byte bằng EndianTransform, hàm này đọc các giá trị theo từng byte theo thứ tự ngược lại và nối chúng vào kết quả.

Records consist of primitives like numbers, strings, booleans, and their combinations. However, when transferring data over the network or storing it on disk, we can only use byte sequences. This means that, in order to send or write the record, we have to serialize it (convert it to an interpretable sequence of bytes) and, before we can use it after receiving or reading, we have to deserialize it (translate the sequence of bytes back to the original record).

Các bản ghi gồm các nguyên thủy như số, chuỗi, boolean, và các tổ hợp của chúng. Tuy nhiên, khi truyền dữ liệu qua mạng hoặc lưu nó trên đĩa, chúng ta chỉ có thể dùng các chuỗi byte. Điều này có nghĩa là, để gửi hay ghi bản ghi, chúng ta phải tuần tự hóa nó (chuyển nó thành một chuỗi byte có thể diễn giải được) và, trước khi có thể dùng nó sau khi nhận hoặc đọc, chúng ta phải giải tuần tự hóa nó (chuyển chuỗi byte trở lại thành bản ghi gốc).

In binary data formats, we always start with primitives that serve as building blocks for more complex structures. Different numeric types may vary in size. byte value is 8 bits, short is 2 bytes (16 bits), int is 4 bytes (32 bits), and long is 8 bytes (64 bits).

Trong các định dạng dữ liệu nhị phân, chúng ta luôn bắt đầu với các nguyên thủy đóng vai trò là các khối xây dựng cho các cấu trúc phức tạp hơn. Các kiểu số khác nhau có thể khác nhau về kích thước. Giá trị byte là 8 bit, short là 2 byte (16 bit), int là 4 byte (32 bit), và long là 8 byte (64 bit).

Floating-point numbers (such as float and double) are represented by their sign , fraction , and exponent . The IEEE Standard for Binary Floating-Point Arithmetic (IEEE 754) standard describes widely accepted floating-point number representation. A 32-bit float represents a single-precision value. For example, a floating-point number 0.15652 has a binary representation, as shown in Figure 3-2 . The first 23 bits represent a fraction, the following 8 bits represent an exponent, and 1 bit represents a sign (whether or not the number is negative).

Các số dấu phẩy động (như float và double) được biểu diễn bằng dấu (sign), phần phân số (fraction), và số mũ (exponent). Chuẩn IEEE về Số học Dấu phẩy động Nhị phân (IEEE 754) mô tả cách biểu diễn số dấu phẩy động được chấp nhận rộng rãi. Một float 32-bit biểu diễn một giá trị độ chính xác đơn (single-precision). Ví dụ, số dấu phẩy động 0.15652 có một biểu diễn nhị phân như trong Hình 3-2. 23 bit đầu tiên biểu diễn phần phân số, 8 bit tiếp theo biểu diễn số mũ, và 1 bit biểu diễn dấu (số có âm hay không).

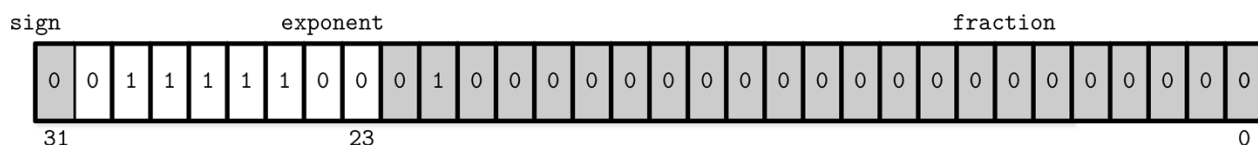


Figure 3-2. Binary representation of single-precision float number

Hình 3-2. Biểu diễn nhị phân của số float độ chính xác đơn

Since a floating-point value is calculated using fractions, the number this representation yields is just an approximation. Discussing a complete conversion algorithm is out of the scope of this book, and we only cover representation basics.

Vì một giá trị dấu phẩy động được tính bằng các phân số, nên con số mà biểu diễn này tạo ra chỉ là một giá trị xấp xỉ. Việc bàn về một thuật toán chuyển đổi đầy đủ nằm ngoài phạm vi cuốn sách này, và chúng ta chỉ đề cập tới những điều cơ bản về biểu diễn.

The double represents a double-precision floating-point value [SAVARD05]. Most programming languages have means for encoding and decoding floating-point values to and from their binary representation in their standard libraries.

Kiểu double biểu diễn một giá trị dấu phẩy động độ chính xác kép (double-precision) [SAVARD05]. Hầu hết các ngôn ngữ lập trình đều có sẵn các phương tiện để mã hóa và giải mã các giá trị dấu phẩy động sang và từ biểu diễn nhị phân của chúng trong thư viện chuẩn.

1 Depending on the platform (macOS, Solaris, Aix, or one of the BSD flavors, or Windows), the `kLittleEndian` variable is set to whether or not the platform supports little-endian.

1 Tùy thuộc vào nền tảng (macOS, Solaris, Aix, hoặc một trong các biến thể BSD, hay Windows), biến `kLittleEndian` được đặt theo việc nền tảng có hỗ trợ little-endian hay không.

Strings and Variable-Size Data — Chuỗi và dữ liệu kích thước thay đổi

All primitive numeric types have a fixed size. Composing more complex values together is much like struct 2 in C. You can combine primitive values into structures and use fixed-size arrays or pointers to other memory regions.

Tất cả các kiểu số nguyên thủy đều có kích thước cố định. Việc kết hợp các giá trị phức tạp hơn lại với nhau khá giống với struct 2 trong C. Bạn có thể kết hợp các giá trị nguyên thủy thành các cấu trúc và dùng các mảng kích thước cố định hoặc các con trỏ tới những vùng bộ nhớ khác.

Strings and other **variable-size data** types (such as arrays of fixed-size data) can be serialized as a number, representing the length of the array or string, followed by size bytes: the actual data. For strings, this representation is often called UCS2 String or Pascal String, named after the popular implementation of the Pascal programming language. We can express it in pseudocode as follows:

Chuỗi và các kiểu dữ liệu kích thước thay đổi khác (chẳng hạn như mảng các dữ liệu kích thước cố định) có thể được tuần tự hóa thành một con số, biểu diễn độ dài của mảng hoặc chuỗi, theo sau là số byte tương ứng: dữ liệu thực tế. Đối với chuỗi, biểu diễn này thường được gọi là chuỗi UCS2 (UCS2 String) hoặc chuỗi Pascal (Pascal String), đặt theo tên bản

hiện thực phổ biến của ngôn ngữ lập trình Pascal. Chúng ta có thể diễn đạt nó bằng mã giả như sau:

```
String
{
    size    uint_16
    data    byte[size]
}
```

An alternative to Pascal strings is null-terminated strings, where the reader consumes the string byte-wise until the end-of-string symbol is reached. The Pascal string approach has several advantages: it allows finding out a length of a string in constant time, instead of iterating through string contents, and a language-specific string can be composed by slicing size bytes from memory and passing the byte array to a string constructor.

Một giải pháp thay thế cho chuỗi Pascal là chuỗi kết thúc bằng null (null-terminated strings), trong đó bộ đọc tiêu thụ chuỗi theo từng byte cho tới khi gặp ký hiệu kết thúc chuỗi. Cách tiếp cận chuỗi Pascal có một số ưu điểm: nó cho phép tìm ra độ dài của chuỗi trong thời gian hằng số, thay vì phải lặp qua nội dung chuỗi, và một chuỗi đặc thù theo ngôn ngữ có thể được dựng bằng cách cắt ra số byte tương ứng từ bộ nhớ rồi truyền mảng byte đó cho hàm khởi tạo chuỗi.

Bit-Packed Data: Booleans, Enums, and Flags — Dữ liệu đóng gói bit: Boolean, Enum và Flag

Booleans can be represented either by using a single byte, or encoding true and false as 1 and 0 values. Since a boolean has only two values, using an entire byte for its representation is wasteful, and developers often batch boolean values together in groups of eight, each boolean occupying just one bit. We say that every 1 bit is set and every 0 bit is unset or empty.

Boolean có thể được biểu diễn hoặc bằng cách dùng một byte đơn, hoặc mã hóa true và false thành các giá trị 1 và 0. Vì một boolean chỉ có hai giá trị, việc dùng cả một byte để biểu diễn nó là lãng phí, nên lập trình viên thường gom các giá trị boolean lại thành nhóm tám cái, mỗi boolean chỉ chiếm một bit. Chúng ta nói rằng mỗi bit 1 là được đặt (set) và mỗi bit 0 là không được đặt (unset) hoặc rỗng (empty).

Enums, short for enumerated types, can be represented as integers and are often used in binary formats and communication protocols. Enums are used to represent often-repeated low-cardinality values. For example, we can encode a B-Tree **node** type using an enum:

Enum, viết tắt của kiểu liệt kê (enumerated types), có thể được biểu diễn dưới dạng số nguyên và thường được dùng trong các định dạng nhị phân cùng các giao thức truyền thông. Enum được dùng để biểu diễn các giá trị có lực lượng thấp (low-cardinality) thường lặp lại. Ví dụ, chúng ta có thể mã hóa kiểu của một nút B-Tree bằng một enum:

```
enum NodeType {
    ROOT,      // 0x00h
    INTERNAL,  // 0x01h
    LEAF       // 0x02h
};
```

It's worth noting that compilers can add padding to structures, which is also architecture dependent. This may break the assumptions about the exact byte offsets and locations. You can read more about

structure packing here: <https://databass.dev/links/58> .

2 Cần lưu ý rằng các trình biên dịch có thể thêm phần đệm (padding) vào các cấu trúc, điều này cũng phụ thuộc vào kiến trúc. Việc này có thể phá vỡ các giả định về các độ dài byte và vị trí chính xác. Bạn có thể đọc thêm về việc đóng gói cấu trúc (structure packing) tại đây: <https://databass.dev/links/58>.

Another closely related concept is flags , kind of a combination of packed booleans and enums. Flags can represent nonmutually exclusive named boolean parameters. For example, we can use flags to denote whether or not the page holds value cells, whether the values are fixed-size or variable-size, and whether or not there are overflow pages associated with this node. Since every bit represents a flag value, we can only use power-of-two values for masks (since powers of two in binary always have a single set bit; for example, $2^3 == 8 == 1000b$, $2^4 == 16 == 0001\ 0000b$, etc.):

Một khái niệm khác có liên quan mật thiết là flag, một dạng kết hợp giữa các boolean đóng gói và enum. Flag có thể biểu diễn các tham số boolean có tên nhưng không loại trừ lẫn nhau (nonmutually exclusive). Ví dụ, chúng ta có thể dùng flag để biểu thị liệu trang có chứa các ô giá trị (value cell) hay không, liệu các giá trị có kích thước cố định hay kích thước thay đổi, và liệu có các trang tràn (overflow page) gắn với nút này hay không. Vì mỗi bit biểu diễn một giá trị flag, chúng ta chỉ có thể dùng các giá trị là lũy thừa của hai làm mặt nạ (mask) (vì các lũy thừa của hai ở dạng nhị phân luôn có đúng một bit được đặt; ví dụ, $2^3 == 8 == 1000b$, $2^4 == 16 == 0001\ 0000b$, v.v.):

```
int IS_LEAF_MASK          = 0x01h; // bit #1
int VARIABLE_SIZE_VALUES = 0x02h; // bit #2
int HAS_OVERFLOW_PAGES    = 0x04h; // bit #3
```

Just like packed booleans, flag values can be read and written from the packed value using bitmasks and bitwise operators. For example, in order to set a bit responsible for one of the flags, we can use bitwise OR (`|`) and a bitmask. Instead of a bitmask, we can use bitshift (`<<`) and a bit index. To unset the bit, we can use bitwise AND (`&`) and the bitwise negation operator (`~`). To test whether or not the bit `n` is set, we can compare the result of a bitwise AND with 0 :

Giống như các boolean đóng gói, các giá trị flag có thể được đọc và ghi từ giá trị đóng gói bằng cách dùng các mặt nạ bit và các toán tử thao tác bit (bitwise). Ví dụ, để đặt một bit chịu trách nhiệm cho một trong các flag, chúng ta có thể dùng phép OR bit (`|`) và một mặt nạ bit. Thay vì mặt nạ bit, chúng ta có thể dùng phép dịch bit (`<<`) và một chỉ số bit. Để bỏ đặt bit, chúng ta có thể dùng phép AND bit (`&`) và toán tử phủ định bit (`~`). Để kiểm tra xem bit `n` có được đặt hay không, chúng ta có thể so sánh kết quả của phép AND bit với 0:

```
// Set the bit
flags |= HAS_OVERFLOW_PAGES;
flags |= (1 << 2);
// Unset the bit
flags &= ~HAS_OVERFLOW_PAGES;
flags &= ~(1 << 2);
// Test whether or not the bit is set
is_set = (flags & HAS_OVERFLOW_PAGES) != 0;
is_set = (flags & (1 << 2)) != 0;
```


General Principles — Các nguyên lý chung

Usually, you start designing a file format by deciding how the addressing is going to be done: whether the file is going to be split into same-sized pages, which are represented by a single block or multiple contiguous blocks. Most in-place update storage structures use pages of the same size, since it significantly simplifies read and write access. Append-only storage structures often write data page-wise, too: records are appended one after the other and, as soon as the page fills up in memory, it is flushed on disk.

Thông thường, bạn bắt đầu thiết kế một định dạng tệp bằng cách quyết định việc đánh địa chỉ sẽ được thực hiện như thế nào: liệu tệp có được chia thành các trang cùng kích thước hay không, mỗi trang được biểu diễn bằng một khối đơn hoặc nhiều khối liền kề. Hầu hết các cấu trúc lưu trữ cập nhật tại chỗ (in-place update) dùng các trang cùng kích thước, vì điều đó đơn giản hóa đáng kể việc truy cập đọc và ghi. Các cấu trúc lưu trữ chỉ-thêm (append-only) cũng thường ghi dữ liệu theo từng trang: các bản ghi được nối tiếp nhau và, ngay khi trang lấp đầy trong bộ nhớ, nó được xả (flush) xuống đĩa.

The file usually starts with a fixed-size header and may end with a fixed-size trailer, which hold auxiliary information that should be accessed quickly or is required for decoding the rest of the file. The rest of the file is split into pages. Figure 3-3 shows this file organization schematically.

Tệp thường bắt đầu bằng một phần đầu (header) có kích thước cố định và có thể kết thúc bằng một phần đuôi (trailer) có kích thước cố định, chứa các thông tin phụ trợ cần được truy cập nhanh hoặc cần thiết để giải mã phần còn lại của tệp. Phần còn lại của tệp được chia thành các trang. Hình 3-3 mô tả cách tổ chức tệp này một cách lược đồ.

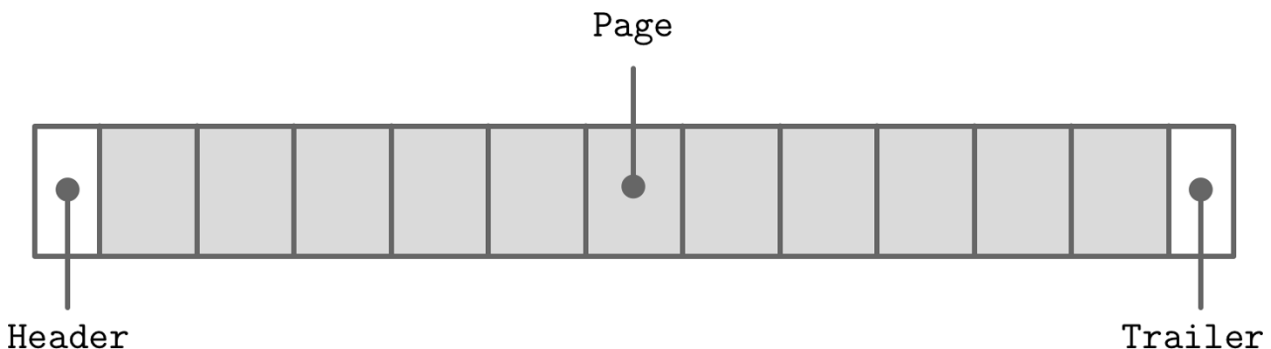


Figure 3-3. File organization

Hình 3-3. Tổ chức tệp

Many data stores have a fixed schema, specifying the number, order, and type of fields the table can hold. Having a fixed schema helps to reduce the amount of data stored on disk: instead of repeatedly writing field names, we can use their positional identifiers.

Nhiều kho dữ liệu có một lược đồ (schema) cố định, chỉ định số lượng, thứ tự và kiểu của các trường mà bảng có thể chứa. Việc có một lược đồ cố định giúp giảm lượng dữ liệu lưu trên đĩa: thay vì lặp đi lặp lại việc ghi tên trường, chúng ta có thể dùng các định danh theo vị trí của chúng.

If we wanted to design a format for the company directory, storing names, birth dates, tax numbers, and genders for each employee, we could use several approaches. We could store the fixed-size fields (such as birth date and tax number) in the head of the structure, followed by the variable-size ones:

Nếu chúng ta muốn thiết kế một định dạng cho danh bạ công ty, lưu trữ tên, ngày sinh, mã

số thuế và giới tính cho mỗi nhân viên, chúng ta có thể dùng một vài cách tiếp cận. Chúng ta có thể lưu các trường kích thước cố định (như ngày sinh và mã số thuế) ở đầu cấu trúc, theo sau là các trường kích thước thay đổi:

Fixed-size fields:

(4 bytes) employee_id	
(4 bytes) tax_number	
(3 bytes) date	
(1 byte) gender	
(2 bytes) first_name_length	
(2 bytes) last_name_length	

Variable-size fields:

(first_name_length bytes) first_name	
(last_name_length bytes) last_name	

Now, to access `first_name`, we can slice `first_name_length` bytes after the fixed-size area. To access `last_name`, we can locate its starting position by checking the sizes of the variable-size fields that precede it. To avoid calculations involving multiple fields, we can encode both offset and length to the fixed-size area. In this case, we can locate any variable-size field separately.

Giờ đây, để truy cập `first_name`, chúng ta có thể cắt ra `first_name_length` byte sau vùng kích thước cố định. Để truy cập `last_name`, chúng ta có thể xác định vị trí bắt đầu của nó bằng cách kiểm tra kích thước của các trường kích thước thay đổi đứng trước nó. Để tránh các phép tính liên quan tới nhiều trường, chúng ta có thể mã hóa cả độ dời lẫn độ dài vào vùng kích thước cố định. Trong trường hợp này, chúng ta có thể xác định vị trí bất kỳ trường kích thước thay đổi nào một cách độc lập.

Building more complex structures usually involves building hierarchies: fields composed out of primitives, cells composed of fields, pages composed of cells, sections composed of pages, regions composed of sections, and so on. There are no strict rules you have to follow here, and it all depends on what kind of data you need to create a format for.

Việc xây dựng các cấu trúc phức tạp hơn thường liên quan tới việc dựng nên các phân cấp: các trường được tạo từ các nguyên thủy, các ô được tạo từ các trường, các trang được tạo từ các ô, các phân đoạn (section) được tạo từ các trang, các vùng (region) được tạo từ các phân đoạn, v.v. Không có quy tắc nghiêm ngặt nào bạn phải tuân theo ở đây, và tất cả phụ thuộc vào việc bạn cần tạo định dạng cho loại dữ liệu nào.

Database files often consist of multiple parts, with a lookup table aiding navigation and pointing to the start offsets of these parts written either in the file header, trailer, or in the separate file.

Các tệp cơ sở dữ liệu thường gồm nhiều phần, với một bảng tra cứu (lookup table) hỗ trợ điều hướng và trỏ tới các độ dời bắt đầu của các phần này, được ghi hoặc trong phần đầu tệp, phần đuôi, hoặc trong một tệp riêng.

Page Structure — Cấu trúc trang

Database systems store data records in data and index files. These files are partitioned into fixed-size units called pages, which often have a size of multiple filesystem blocks. Page sizes usually range from 4 to 16 Kb.

Các hệ cơ sở dữ liệu lưu trữ các bản ghi dữ liệu trong các tệp dữ liệu và tệp chỉ mục. Các tệp này được phân hoạch thành các đơn vị kích thước cố định gọi là trang (page), mà thường có

kích thước là bội số của các khối hệ thống tệp (filesystem block). Kích thước trang thường nằm trong khoảng từ 4 đến 16 Kb.

Let's take a look at the example of an on-disk B-Tree node. From a structure perspective, in B-Trees, we distinguish between the leaf nodes that hold keys and data records pairs, and nonleaf nodes that hold keys and pointers to other nodes. Each B-Tree node occupies one page or multiple pages linked together, so in the context of B-Trees the terms node and page (and even block) are often used interchangeably.

Hãy cùng xem ví dụ về một nút B-Tree trên đĩa. Xét về mặt cấu trúc, trong B-Tree, chúng ta phân biệt giữa các nút lá (leaf node) chứa các cặp khóa và bản ghi dữ liệu, và các nút không-phải-lá (nonleaf node) chứa các khóa và con trỏ tới các nút khác. Mỗi nút B-Tree chiếm một trang hoặc nhiều trang liên kết với nhau, nên trong ngữ cảnh B-Tree, các thuật ngữ nút (node) và trang (page) (và thậm chí cả khối, block) thường được dùng thay thế cho nhau.

The original B-Tree paper [BAYER72] describes a simple page organization for fixed-size data records, where each page is just a concatenation of triplets, as shown in Figure 3-4 : keys are denoted by k , associated values are denoted by v , and pointers to child pages are denoted by p .

Bài báo gốc về B-Tree [BAYER72] mô tả một cách tổ chức trang đơn giản cho các bản ghi dữ liệu kích thước cố định, trong đó mỗi trang chỉ là một phép nối các bộ ba (triplet), như trong Hình 3-4: các khóa được ký hiệu bằng k , các giá trị liên kết được ký hiệu bằng v , và các con trỏ tới các trang con được ký hiệu bằng p .

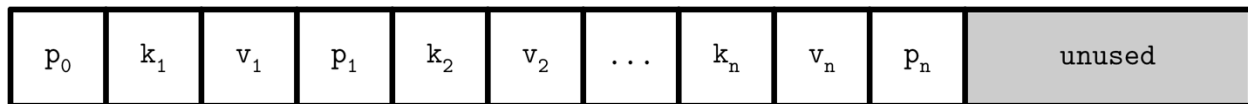


Figure 3-4. Page organization for fixed-size records

Hình 3-4. Cách tổ chức trang cho các bản ghi kích thước cố định

This approach is easy to follow, but has some downsides:

Cách tiếp cận này dễ theo dõi, nhưng có một số nhược điểm:

- Appending a key anywhere but the right side requires relocating elements.
- It doesn't allow managing or accessing variable-size records efficiently and works only for fixed-size data.
 - Việc thêm một khóa ở bất kỳ đâu ngoài phía bên phải đòi hỏi phải di dời các phần tử.
 - Nó không cho phép quản lý hoặc truy cập các bản ghi kích thước thay đổi một cách hiệu quả và chỉ hoạt động với dữ liệu kích thước cố định.

Slotted Pages — Trang có khe (Slotted Pages)

When storing variable-size records, the main problem is free space management: reclaiming the space occupied by removed records. If we attempt to put a record of size n into the space previously occupied by the record of size m , unless $m == n$ or we can find another record that has a size exactly $m - n$, this space will remain unused. Similarly, a segment of size m cannot be used to store a record of size k if k is larger than m , so it will be inserted without reclaiming the unused space.

Khi lưu trữ các bản ghi kích thước thay đổi, vấn đề chính là quản lý không gian trống: thu hồi lại không gian bị chiếm bởi các bản ghi đã bị xóa. Nếu chúng ta cố đặt một bản ghi kích

thước n vào không gian trước đó bị chiếm bởi một bản ghi kích thước m , thì trừ khi $m == n$ hoặc chúng ta có thể tìm được một bản ghi khác có kích thước đúng bằng $m - n$, không gian này sẽ vẫn không được dùng. Tương tự, một đoạn kích thước m không thể được dùng để lưu một bản ghi kích thước k nếu k lớn hơn m , nên bản ghi đó sẽ được chèn vào mà không thu hồi được không gian chưa dùng.

To simplify space management for variable-size records, we can split the page into fixed-size segments. However, we end up wasting space if we do that, too. For example, if we use a segment size of 64 bytes, unless the record size is a multiple of 64, we waste $64 - (n \bmod 64)$ bytes, where n is the size of the inserted record. In other words, unless the record is a multiple of 64, one of the blocks will be only partially filled.

Để đơn giản hóa việc quản lý không gian cho các bản ghi kích thước thay đổi, chúng ta có thể chia trang thành các đoạn kích thước cố định. Tuy nhiên, làm vậy thì chúng ta cũng lãng phí không gian. Ví dụ, nếu chúng ta dùng kích thước đoạn là 64 byte, thì trừ khi kích thước bản ghi là bội số của 64, chúng ta lãng phí $64 - (n \bmod 64)$ byte, với n là kích thước của bản ghi được chèn. Nói cách khác, trừ khi bản ghi là bội số của 64, một trong các khối sẽ chỉ được lấp đầy một phần.

Space reclamation can be done by simply rewriting the page and moving the records around, but we need to preserve record offsets, since out-of-page pointers might be using these offsets. It is desirable to do that while minimizing space waste, too.

Việc thu hồi không gian có thể được thực hiện bằng cách đơn giản là viết lại trang và di chuyển các bản ghi xung quanh, nhưng chúng ta cần bảo toàn các độ dời của bản ghi, vì các con trỏ từ ngoài trang có thể đang dùng những độ dời này. Cũng nên làm điều đó trong khi tối thiểu hóa lãng phí không gian.

To summarize, we need a page format that allows us to:

Tóm lại, chúng ta cần một định dạng trang cho phép chúng ta:

- Store variable-size records with a minimal overhead.
- Reclaim space occupied by the removed records.
- Reference records in the page without regard to their exact locations.
 - Lưu trữ các bản ghi kích thước thay đổi với chi phí phụ trội (overhead) tối thiểu.
 - Thu hồi không gian bị chiếm bởi các bản ghi đã bị xóa.
 - Tham chiếu các bản ghi trong trang mà không cần quan tâm tới vị trí chính xác của chúng.

To efficiently store variable-size records such as strings, binary large objects (BLOBs), etc., we can use an organization technique called **slotted page** (i.e., a page with slots) [SILBERSCHATZ10] or slot directory [RAMAKRISHNAN03]. This approach is used by many databases, for example, PostgreSQL.

Để lưu trữ hiệu quả các bản ghi kích thước thay đổi như chuỗi, đối tượng nhị phân lớn (binary large objects, BLOB), v.v., chúng ta có thể dùng một kỹ thuật tổ chức gọi là trang có khe (slotted page, tức là trang có các khe) [SILBERSCHATZ10] hoặc thư mục khe (slot directory) [RAMAKRISHNAN03]. Cách tiếp cận này được nhiều cơ sở dữ liệu sử dụng, chẳng hạn như PostgreSQL.

We organize the page into a collection of slots or cells and split out pointers and cells in two independent memory regions residing on different sides of the page. This means that we only need to reorganize pointers addressing the cells to preserve the order, and deleting a record can be done either by nullifying its pointer or removing it.

Chúng ta tổ chức trang thành một tập hợp các khe (slot) hoặc ô (cell) và tách riêng các con trỏ và các ô thành hai vùng bộ nhớ độc lập nằm ở hai phía khác nhau của trang. Điều này có nghĩa là chúng ta chỉ cần tổ chức lại các con trỏ trỏ tới các ô để bảo toàn thứ tự, và việc xóa một bản ghi có thể được thực hiện hoặc bằng cách làm rỗng con trỏ của nó hoặc bằng cách loại bỏ nó.

A slotted page has a fixed-size header that holds important information about the page and cells (see “Page Header” on page 61). Cells may differ in size and can hold arbitrary data: keys, pointers, data records, etc. Figure 3-5 shows a slotted page organization, where every page has a maintenance region (header), cells, and pointers to them.

Một trang có khe có một phần đầu kích thước cố định chứa các thông tin quan trọng về trang và các ô (xem “Page Header” ở trang 61). Các ô có thể khác nhau về kích thước và có thể chứa dữ liệu tùy ý: khóa, con trỏ, bản ghi dữ liệu, v.v. Hình 3-5 thể hiện cách tổ chức trang có khe, trong đó mỗi trang có một vùng bảo trì (phần đầu), các ô, và các con trỏ tới chúng.

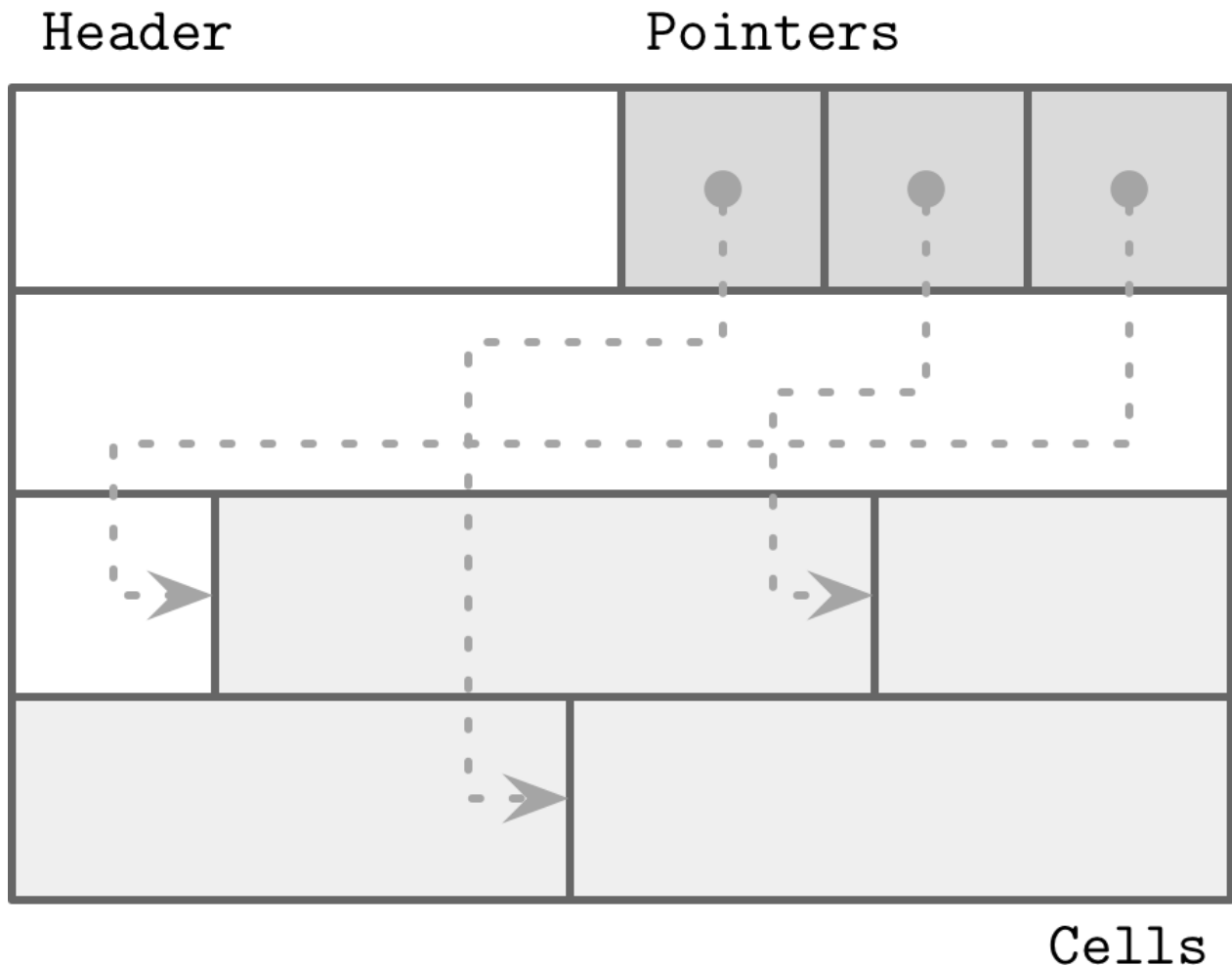


Figure 3-5. Slotted page Let’s see how this approach fixes the problems we stated in the beginning of this section:

Hình 3-5. Trang có khe Hãy xem cách tiếp cận này giải quyết các vấn đề mà chúng ta đã nêu ở đầu phần này như thế nào:

- Minimal overhead: the only overhead incurred by slotted pages is a pointer array holding

offsets to the exact positions where the records are stored.

- Space reclamation: space can be reclaimed by defragmenting and rewriting the page.
- Dynamic layout: from outside the page, slots are referenced only by their IDs, so the exact location is internal to the page.
 - Chi phí phụ trội tối thiểu: chi phí phụ trội duy nhất mà trang có thể gây ra là một mảng con trỏ chứa các độ dời tới vị trí chính xác nơi các bản ghi được lưu trữ.
 - Thu hồi không gian: không gian có thể được thu hồi bằng cách chống phân mảnh và viết lại trang.
 - Bố trí động: từ bên ngoài trang, các khe chỉ được tham chiếu bằng ID của chúng, nên vị trí chính xác là nội bộ của trang.

Cell Layout — Bố trí ô (Cell Layout)

Using flags, enums, and primitive values, we can start designing the **cell** layout, then combine cells into pages, and compose a tree out of the pages. On a cell level, we have a distinction between key and key-value cells. Key cells hold a **separator key** and a pointer to the page between two neighboring pointers. Key-value cells hold keys and data records associated with them.

Dùng các flag, enum và các giá trị nguyên thủy, chúng ta có thể bắt đầu thiết kế bố trí ô, rồi kết hợp các ô thành các trang, và dựng nên một cây từ các trang. Ở mức ô, chúng ta phân biệt giữa ô khóa (key cell) và ô khóa-giá trị (key-value cell). Ô khóa chứa một khóa phân tách (separator key) và một con trỏ tới trang nằm giữa hai con trỏ kề nhau. Ô khóa-giá trị chứa các khóa và các bản ghi dữ liệu liên kết với chúng.

We assume that all cells within the page are uniform (for example, all cells can hold either just keys or both keys and values; similarly, all cells hold either fixed-size or variable-size data, but not a mix of both). This means we can store metadata describing cells once on the page level, instead of duplicating it in every cell.

Chúng ta giả định rằng tất cả các ô trong một trang đều đồng nhất (ví dụ, tất cả các ô có thể chỉ chứa khóa hoặc chứa cả khóa và giá trị; tương tự, tất cả các ô đều chứa dữ liệu kích thước cố định hoặc kích thước thay đổi, nhưng không trộn lẫn cả hai). Điều này có nghĩa là chúng ta có thể lưu siêu dữ liệu (metadata) mô tả các ô một lần ở mức trang, thay vì lặp lại nó trong từng ô.

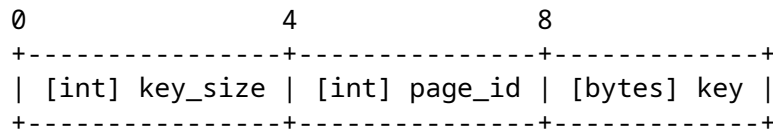
To compose a key cell, we need to know:

Để dựng một ô khóa, chúng ta cần biết:

- Cell type (can be inferred from the page metadata)
- Key size
- ID of the child page this cell is pointing to
- Key bytes
 - Kiểu ô (có thể suy ra từ siêu dữ liệu của trang)
 - Kích thước khóa
 - ID của trang con mà ô này trỏ tới
 - Các byte của khóa

A variable-size key cell layout might look something like this (a fixed-size one would have no size specifier on the cell level):

Một bố trí ô khóa kích thước thay đổi có thể trông giống như thế này (một bố trí kích thước cố định sẽ không có phần chỉ định kích thước ở mức ô):



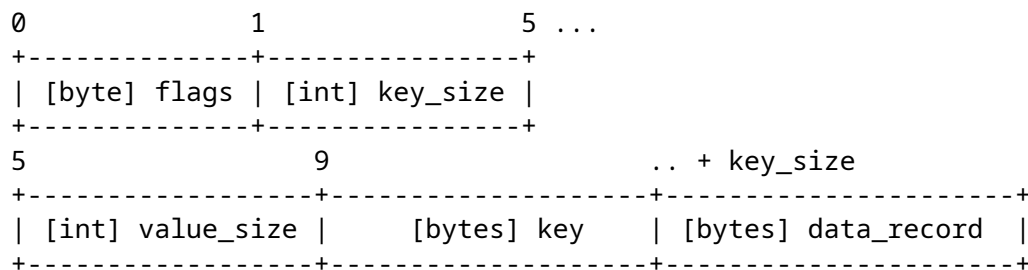
We have grouped fixed-size data fields together, followed by `key_size` bytes. This is not strictly necessary but can simplify offset calculation, since all fixed-size fields can be accessed by using static, precomputed offsets, and we need to calculate the offsets only for the variable-size data.

Chúng ta đã nhóm các trường dữ liệu kích thước cố định lại với nhau, theo sau là `key_size` byte. Việc này không bắt buộc một cách nghiêm ngặt nhưng có thể đơn giản hóa việc tính độ dời, vì tất cả các trường kích thước cố định có thể được truy cập bằng cách dùng các độ dời tĩnh, được tính trước, và chúng ta chỉ cần tính độ dời cho dữ liệu kích thước thay đổi.

The key-value cells hold data records instead of the child page IDs. Otherwise, their structure is similar:

Các ô khóa-giá trị chứa các bản ghi dữ liệu thay vì ID của các trang con. Ngoài ra, cấu trúc của chúng tương tự nhau:

- Cell type (can be inferred from page metadata)
- Key size
- Value size
- Key bytes
- Data record bytes
 - Kiểu ô (có thể suy ra từ siêu dữ liệu của trang)
 - Kích thước khóa
 - Kích thước giá trị
 - Các byte của khóa
 - Các byte của bản ghi dữ liệu



You might have noticed the distinction between the offset and page ID here. Since pages have a fixed size and are managed by the **page cache** (see “Buffer Management” on page 81), we only need to store the page ID, which is later translated to the actual offset in the file using the lookup table. Cell offsets are page-local and are relative to the page start offset: this way we can use a smaller cardinality integer to keep the representation more compact.

Bạn có thể đã để ý tới sự phân biệt giữa độ dời và ID trang ở đây. Vì các trang có kích thước cố định và được quản lý bởi page cache (xem “Buffer Management” ở trang 81), chúng ta chỉ cần lưu ID trang, sau đó được dịch thành độ dời thực tế trong tệp bằng cách dùng bảng tra cứu. Các độ dời của ô là cục bộ trong trang (page-local) và tương đối so với độ dời bắt đầu của trang: theo cách này chúng ta có thể dùng một số nguyên có lực lượng nhỏ hơn để giữ cho biểu diễn gọn nhẹ hơn.

Variable-Size Data — Dữ liệu kích thước thay đổi

It is not necessary for the key and value in the cell to have a fixed size. Both the key and value can have a variable size. Their locations can be calculated from the fixed-size cell header using offsets.

Không nhất thiết khóa và giá trị trong ô phải có kích thước cố định. Cả khóa lẫn giá trị đều có thể có kích thước thay đổi. Vị trí của chúng có thể được tính từ phần đầu ô kích thước cố định bằng cách dùng các độ dời.

To locate the key, we skip the header and read `key_size` bytes. Similarly, to locate the value, we can skip the header plus `key_size` more bytes and read `value_size` bytes.

Để xác định vị trí khóa, chúng ta bỏ qua phần đầu và đọc `key_size` byte. Tương tự, để xác định vị trí giá trị, chúng ta có thể bỏ qua phần đầu cộng thêm `key_size` byte nữa và đọc `value_size` byte.

There are different ways to do the same; for example, by storing a total size and calculating the value size by subtraction. It all boils down to having enough information to slice the cell into subparts and reconstruct the encoded data.

Có những cách khác nhau để làm cùng một việc; ví dụ, bằng cách lưu tổng kích thước và tính kích thước giá trị bằng phép trừ. Tất cả quy về việc có đủ thông tin để cắt ô thành các phần con và tái dựng dữ liệu đã được mã hóa.

Combining Cells into Slotted Pages — Kết hợp các ô thành trang có khe

To organize cells into pages, we can use the slotted page technique that we discussed in “Page Structure” on page 52 . We append cells to the right side of the page (toward its end) and keep cell offsets/pointers in the left side of the page, as shown in Figure 3-6 .

Để tổ chức các ô thành các trang, chúng ta có thể dùng kỹ thuật trang có khe mà chúng ta đã thảo luận trong “Page Structure” ở trang 52. Chúng ta nối các ô vào phía bên phải của trang (về phía cuối trang) và giữ các độ dời/con trỏ của ô ở phía bên trái của trang, như trong Hình 3-6.

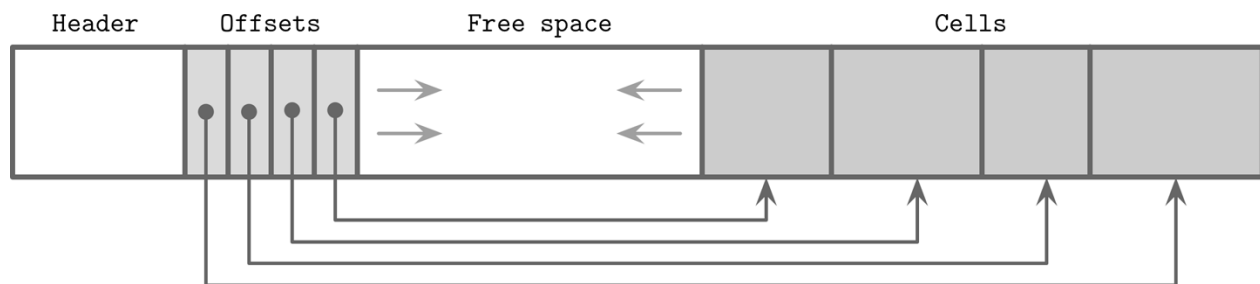


Figure 3-6. Offset and cell growth direction

Hình 3-6. Hướng tăng trưởng của độ dời và ô

Keys can be inserted out of order and their logical sorted order is kept by sorting cell offset pointers in key order. This design allows appending cells to the page with minimal effort, since cells don't have to be relocated during insert, update, or delete operations.

Các khóa có thể được chèn không theo thứ tự và thứ tự được sắp xếp về mặt logic của chúng được giữ bằng cách sắp xếp các con trỏ độ dời của ô theo thứ tự khóa. Thiết kế này cho phép nối các ô vào trang với nỗ lực tối thiểu, vì các ô không phải bị di dời trong các thao tác chèn, cập nhật hoặc xóa.

Let's consider an example of a page that holds names. Two names are added to the page, and their insertion order is: Tom and Leslie . As you can see in Figure 3-7 , their logical order (in this case, alphabetical), does not match insertion order (order in which they were appended to the page). Cells are laid out in insertion order, but offsets are re-sorted to allow using binary search.

Hãy xét một ví dụ về một trang chứa các tên. Hai tên được thêm vào trang, và thứ tự chèn của chúng là: Tom và Leslie. Như bạn có thể thấy trong Hình 3-7, thứ tự logic của chúng (trong trường hợp này là theo bảng chữ cái) không khớp với thứ tự chèn (thứ tự mà chúng được nối vào trang). Các ô được bố trí theo thứ tự chèn, nhưng các độ dời được sắp xếp lại để cho phép dùng tìm kiếm nhị phân.



Figure 3-7. Records appended in random order: Tom, Leslie

Hình 3-7. Các bản ghi được nối theo thứ tự ngẫu nhiên: Tom, Leslie

Now, we'd like to add one more name to this page: Ron . New data is appended at the upper boundary of the free space of the page, but cell offsets have to preserve the lexicographical key order: Leslie , Ron , Tom . To do that, we have to reorder cell offsets: pointers after the insertion point are shifted to the right to make space for the new pointer to the Ron cell, as you can see in Figure 3-8 .

Giờ đây, chúng ta muốn thêm một tên nữa vào trang này: Ron. Dữ liệu mới được nối vào ranh giới trên của không gian trống của trang, nhưng các độ dời của ô phải bảo toàn thứ tự khóa theo từ điển: Leslie, Ron, Tom. Để làm điều đó, chúng ta phải sắp xếp lại các độ dời của ô: các con trỏ sau điểm chèn được dịch sang phải để tạo chỗ cho con trỏ mới tới ô Ron, như bạn có thể thấy trong Hình 3-8.

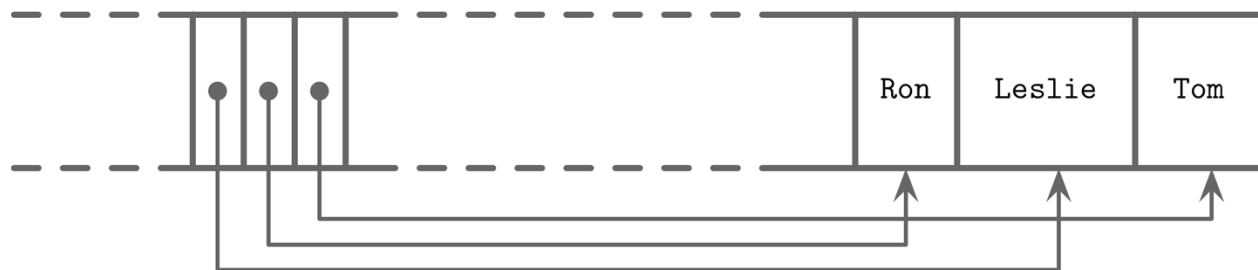


Figure 3-8. Appending one more record: Ron

Hình 3-8. Nối thêm một bản ghi nữa: Ron

Managing Variable-Size Data — Quản lý dữ liệu kích thước thay đổi

Removing an item from the page does not have to remove the actual cell and shift other cells to reoccupy the freed space. Instead, the cell can be marked as deleted and an in-memory availability list can be updated with the amount of freed memory and a pointer to the freed value. The availability list stores offsets of freed segments and their sizes. When inserting a new cell, we first check the availability list to find if there's a segment where it may fit. You can see an example of the fragmented page with available segments in Figure 3-9 .

Việc loại bỏ một mục khỏi trang không nhất thiết phải xóa ô thực tế và dịch các ô khác để chiếm lại không gian đã giải phóng. Thay vào đó, ô có thể được đánh dấu là đã bị xóa và một danh sách khả dụng (availability list) trong bộ nhớ có thể được cập nhật với lượng bộ nhớ đã giải phóng và một con trỏ tới giá trị đã giải phóng. Danh sách khả dụng lưu các độ dời của các đoạn đã giải phóng và kích thước của chúng. Khi chèn một ô mới, đầu tiên chúng ta kiểm tra danh sách khả dụng để tìm xem có một đoạn nào mà nó có thể vừa hay không. Bạn có thể thấy một ví dụ về trang bị phân mảnh với các đoạn khả dụng trong Hình 3-9.

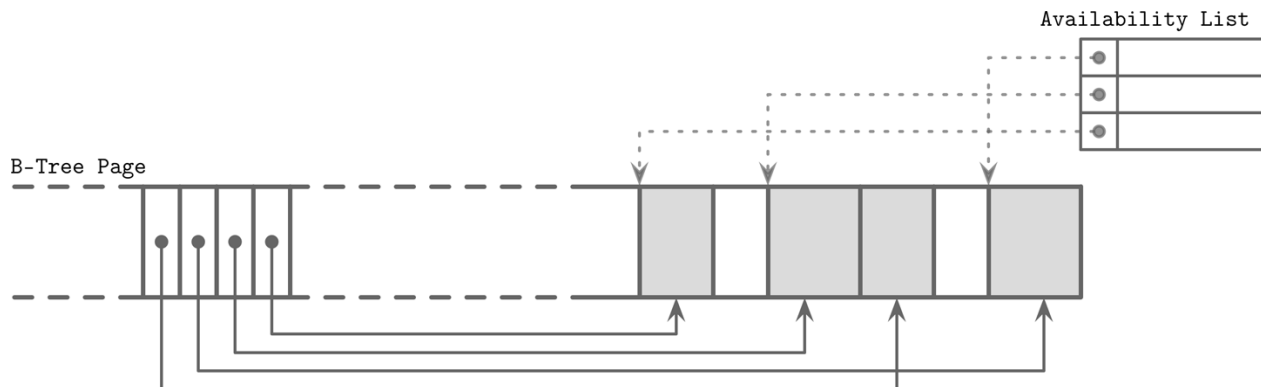


Figure 3-9. Fragmented page and availability list. Occupied pages are shown in gray. Dotted lines represent pointers to unoccupied memory regions from the availability list.

Hình 3-9. Trang bị phân mảnh và danh sách khả dụng. Các trang bị chiếm được tô màu xám. Các đường nét đứt biểu diễn các con trỏ tới các vùng bộ nhớ chưa bị chiếm từ danh sách khả dụng.

SQLite calls unoccupied segments freeblocks and stores a pointer to the first freeblock in the page header . Additionally, it stores a total number of available bytes within the page to quickly check whether or not we can fit a new element into the page after defragmenting it.

SQLite gọi các đoạn chưa bị chiếm là freeblock và lưu một con trỏ tới freeblock đầu tiên trong phần đầu trang. Ngoài ra, nó lưu tổng số byte khả dụng trong trang để nhanh chóng kiểm tra xem chúng ta có thể nhét vừa một phần tử mới vào trang sau khi chống phân mảnh nó hay không.

Fit is calculated based on the strategy :

Việc tính độ vừa (fit) được dựa trên chiến lược:

First fit This might cause a larger overhead, since the space remaining after reusing the first suitable segment might be too small to fit any other cell, so it will be effectively wasted.

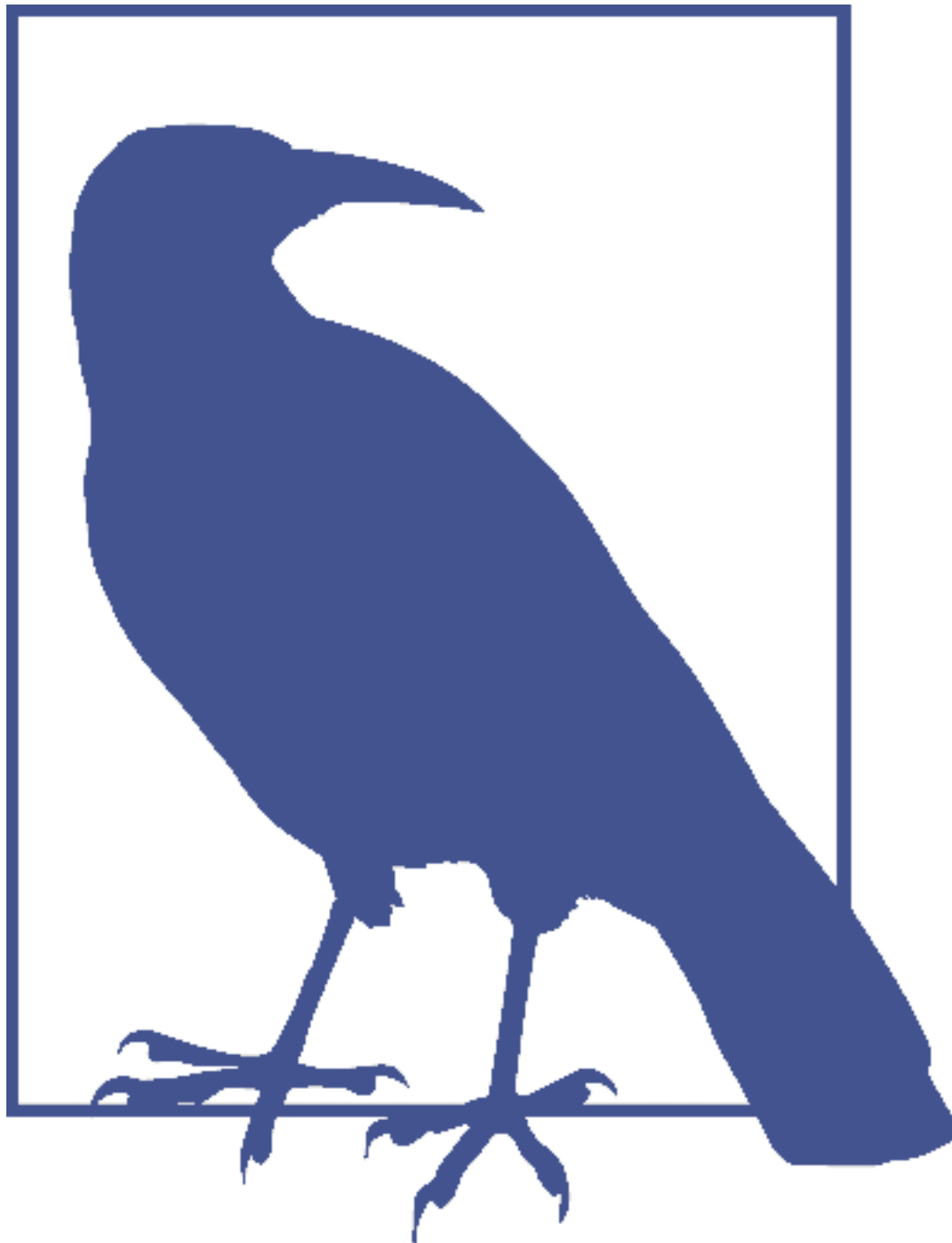
Vừa-đầu-tiên (First fit) Cách này có thể gây ra chi phí phụ trội lớn hơn, vì không gian còn lại sau khi tái sử dụng đoạn phù hợp đầu tiên có thể quá nhỏ để nhét vừa bất kỳ ô nào khác, nên nó sẽ thực sự bị lãng phí.

Best fit For best fit, we try to find a segment for which insertion leaves the smallest remainder.

Vừa-nhất (Best fit) Đối với vừa-nhất, chúng ta cố tìm một đoạn mà việc chèn vào để lại phần dư nhỏ nhất.

If we cannot find enough consecutive bytes to fit the new cell but there are enough fragmented bytes available, live cells are read and rewritten, defragmenting the page and reclaiming space for new writes. If there's not enough free space even after **defragmentation**, we have to create an **overflow page** (see “Overflow Pages” on page 65).

Nếu chúng ta không thể tìm đủ số byte liên tiếp để nhét vừa ô mới nhưng có đủ số byte phân mảnh khả dụng, các ô còn sống (live cell) được đọc và viết lại, chống phân mảnh trang và thu hồi không gian cho các lần ghi mới. Nếu vẫn không có đủ không gian trống ngay cả sau khi chống phân mảnh, chúng ta phải tạo một trang tràn (xem “Overflow Pages” ở trang 65).



To improve locality (especially when keys are small in size), some implementations store keys and values separately on the leaf level. Keeping keys together can improve the locality during the search. After the searched key is located, its value can be found in a value cell with a corresponding index. With variable-size keys, this requires us to calculate and store an additional value cell pointer.

Để cải thiện tính cục bộ (locality) (đặc biệt khi các khóa có kích thước nhỏ), một số bản hiện thực lưu trữ khóa và giá trị riêng biệt ở mức lá. Việc giữ các khóa cùng nhau có thể cải thiện tính cục bộ trong quá trình tìm kiếm. Sau khi khóa được tìm kiếm đã được xác định vị trí, giá trị của nó có thể được tìm thấy trong một ô giá trị với một chỉ số tương ứng. Với các khóa kích thước thay đổi, điều này đòi hỏi chúng ta tính và lưu thêm một con trỏ

tới ô giá trị.

In summary, to simplify B-Tree layout, we assume that each node occupies a single page. A page consists of a fixed-size header, cell pointer block, and cells. Cells hold keys and pointers to the pages representing child nodes or associated data records. B-Trees use simple pointer hierarchies: page identifiers to locate the child nodes in the tree file, and cell offsets to locate cells within the page.

Tóm lại, để đơn giản hóa bố trí B-Tree, chúng ta giả định rằng mỗi nút chiếm một trang đơn. Một trang gồm một phần đầu kích thước cố định, một khối con trỏ ô, và các ô. Các ô chứa các khóa và các con trỏ tới các trang biểu diễn các nút con hoặc các bản ghi dữ liệu liên kết. B-Tree dùng các phân cấp con trỏ đơn giản: các định danh trang để xác định vị trí các nút con trong tệp cây, và các độ dời của ô để xác định vị trí các ô bên trong trang.

Versioning — Phiên bản hóa (Versioning)

Database systems constantly evolve, and developers work to add features, and to fix bugs and performance issues. As a result of that, the binary file format can change. Most of the time, any **storage engine** version has to support more than one serialization format (e.g., current and one or more legacy formats for backward compatibility). To support that, we have to be able to find out which version of the file we're up against.

Các hệ cơ sở dữ liệu liên tục tiến hóa, và các lập trình viên làm việc để thêm tính năng, sửa lỗi và các vấn đề về hiệu năng. Kết quả là, định dạng tệp nhị phân có thể thay đổi. Hầu hết thời gian, bất kỳ phiên bản nào của bộ máy lưu trữ đều phải hỗ trợ nhiều hơn một định dạng tuần tự hóa (ví dụ, định dạng hiện tại và một hoặc nhiều định dạng cũ để tương thích ngược). Để hỗ trợ điều đó, chúng ta phải có khả năng tìm ra mình đang đối mặt với phiên bản nào của tệp.

This can be done in several ways. For example, Apache Cassandra is using version prefixes in filenames. This way, you can tell which version the file has without even opening it. As of version 4.0, a **data file** name has the `na` prefix, such as `na-1-big-Data.db`. Older files have different prefixes: files written in version 3.0 have the `ma` prefix.

Việc này có thể được thực hiện theo nhiều cách. Ví dụ, Apache Cassandra dùng các tiền tố phiên bản trong tên tệp. Theo cách này, bạn có thể biết tệp thuộc phiên bản nào mà thậm chí không cần mở nó ra. Kể từ phiên bản 4.0, một tên tệp dữ liệu có tiền tố `na`, chẳng hạn như `na-1-big-Data.db`. Các tệp cũ hơn có các tiền tố khác: các tệp được ghi trong phiên bản 3.0 có tiền tố `ma`.

Alternatively, the version can be stored in a separate file. For example, PostgreSQL stores the version in the `PG_VERSION` file.

Một cách khác, phiên bản có thể được lưu trong một tệp riêng. Ví dụ, PostgreSQL lưu phiên bản trong tệp `PG_VERSION`.

The version can also be stored directly in the **index file** header. In this case, a part of the header (or an entire header) has to be encoded in a format that does not change between versions. After finding out which version the file is encoded with, we can create a version-specific reader to interpret the contents. Some file formats identify the version using magic numbers, which we discuss in more detail in “Magic Numbers” on page 62.

Phiên bản cũng có thể được lưu trực tiếp trong phần đầu của tệp chỉ mục. Trong trường hợp này, một phần của phần đầu (hoặc toàn bộ phần đầu) phải được mã hóa ở một định dạng không thay đổi giữa các phiên bản. Sau khi tìm ra tệp được mã hóa bằng phiên bản

nào, chúng ta có thể tạo một bộ đọc đặc thù theo phiên bản để diễn giải nội dung. Một số định dạng tệp nhận diện phiên bản bằng cách dùng các số ma thuật (magic number), mà chúng ta thảo luận chi tiết hơn trong “Magic Numbers” ở trang 62.

Checksumming — Tổng kiểm (Checksumming)

Files on disk may get damaged or corrupted by software bugs and hardware failures. To identify these problems preemptively and avoid propagating corrupt data to other subsystems or even nodes, we can use checksums and cyclic redundancy checks (CRCs).

Các tệp trên đĩa có thể bị hư hỏng hoặc hỏng hóc bởi các lỗi phần mềm và các sự cố phần cứng. Để nhận diện các vấn đề này một cách phòng ngừa và tránh lan truyền dữ liệu hỏng tới các phân hệ khác hoặc thậm chí các nút khác, chúng ta có thể dùng các tổng kiểm (checksum) và kiểm tra dư thừa vòng (cyclic redundancy check, CRC).

Some sources make no distinction between cryptographic and noncryptographic hash functions, CRCs, and checksums. What they all have in common is that they reduce a large chunk of data to a small number, but their use cases, purposes, and guarantees are different.

Một số nguồn không phân biệt giữa các hàm băm mật mã và phi mật mã, CRC, và tổng kiểm. Điểm chung của tất cả chúng là chúng thu gọn một khối dữ liệu lớn thành một con số nhỏ, nhưng các trường hợp dùng, mục đích và các bảo đảm của chúng thì khác nhau.

Checksums provide the weakest form of guarantee and aren’t able to detect corruption in multiple bits. They’re usually computed by using XOR with parity checks or summation [KOOPMAN15].

Tổng kiểm cung cấp dạng bảo đảm yếu nhất và không thể phát hiện hư hỏng ở nhiều bit. Chúng thường được tính bằng cách dùng phép XOR với các kiểm tra chẵn lẻ (parity check) hoặc phép tính tổng [KOOPMAN15].

CRCs can help detect burst errors (e.g., when multiple consecutive bits got corrupted) and their implementations usually use lookup tables and polynomial division [STONE98]. Multibit errors are crucial to detect, since a significant percentage of failures in communication networks and storage devices manifest this way.

CRC có thể giúp phát hiện các lỗi cụm (burst error) (ví dụ, khi nhiều bit liên tiếp bị hỏng) và các bản hiện thực của chúng thường dùng các bảng tra cứu và phép chia đa thức [STONE98]. Các lỗi đa bit là cực kỳ quan trọng để phát hiện, vì một tỷ lệ đáng kể các sự cố trong các mạng truyền thông và thiết bị lưu trữ biểu hiện theo cách này.



Noncryptographic hashes and CRCs should not be used to verify whether or not the data has been tampered with. For this, you should always use strong cryptographic hashes designed for security. The main goal of CRC is to make sure that there were no unintended and accidental changes in data. These algorithms are not designed to resist attacks and intentional changes in data.

Các hàm băm phi mật mã và CRC không nên được dùng để xác minh xem dữ liệu có bị can thiệp hay không. Đối với việc đó, bạn luôn nên dùng các hàm băm mật mã mạnh được thiết kế cho mục đích bảo mật. Mục tiêu chính của CRC là đảm bảo rằng không có những thay đổi ngoài ý muốn và tình cờ trong dữ liệu. Các thuật toán này không được thiết kế để chống lại các cuộc tấn công và những thay đổi cố ý trong dữ liệu.

Before writing the data on disk, we compute its **checksum** and write it together with the data. When reading it back, we compute the checksum again and compare it with the written one. If there's a checksum mismatch, we know that corruption has occurred and we should not use the data that was read.

Trước khi ghi dữ liệu lên đĩa, chúng ta tính tổng kiểm của nó và ghi nó cùng với dữ liệu. Khi đọc lại nó, chúng ta tính lại tổng kiểm và so sánh nó với cái đã được ghi. Nếu có sự không khớp tổng kiểm, chúng ta biết rằng hư hỏng đã xảy ra và chúng ta không nên dùng dữ liệu đã được đọc.

Since computing a checksum over the whole file is often impractical and it is unlikely we're going to read the entire content every time we access it, page checksums are usually computed on pages and placed in the page header. This way, checksums can be more robust (since they are performed on a small subset of the data), and the whole file doesn't have to be discarded if corruption is contained in a single page.

Vì việc tính tổng kiểm trên toàn bộ tệp thường không thực tế và khó có khả năng chúng ta sẽ đọc toàn bộ nội dung mỗi khi truy cập nó, các tổng kiểm trang thường được tính trên các trang và được đặt trong phần đầu trang. Theo cách này, các tổng kiểm có thể vững chắc hơn (vì chúng được thực hiện trên một tập con nhỏ của dữ liệu), và toàn bộ tệp không phải bị loại bỏ nếu hư hỏng chỉ giới hạn trong một trang đơn.

Summary — Tóm tắt

In this chapter, we learned about binary data organization: how to serialize primitive data types, combine them into cells, build slotted pages out of cells, and navigate these structures.

Trong chương này, chúng ta đã học về cách tổ chức dữ liệu nhị phân: cách tuần tự hóa các kiểu dữ liệu nguyên thủy, kết hợp chúng thành các ô, dựng các trang có khe từ các ô, và điều hướng các cấu trúc này.

We learned how to handle variable-size data types such as strings, byte sequences, and arrays, and compose special cells that hold a size of values contained in them.

Chúng ta đã học cách xử lý các kiểu dữ liệu kích thước thay đổi như chuỗi, chuỗi byte, và mảng, và dựng các ô đặc biệt chứa kích thước của các giá trị nằm trong chúng.

We discussed the slotted page format, which allows us to reference individual cells from outside the page by cell ID, store records in the insertion order, and preserve the key order by sorting cell offsets.

Chúng ta đã thảo luận về định dạng trang có khe, cho phép chúng ta tham chiếu các ô riêng lẻ từ bên ngoài trang bằng ID ô, lưu các bản ghi theo thứ tự chèn, và bảo toàn thứ tự khóa bằng cách sắp xếp các độ dời của ô.

These principles can be used to compose binary formats for on-disk structures and network protocols.

Các nguyên lý này có thể được dùng để dựng các định dạng nhị phân cho các cấu trúc trên đĩa và các giao thức mạng.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

File organization techniques Folk, Michael J., Greg Riccardi, and Bill Zoellick. 1997. File Structures: An Object- Oriented Approach with C++ (3rd Ed.) . Boston: Addison-Wesley Longman.

Giampaolo, Dominic. 1998. Practical File System Design with the Be File System (1st Ed.) . San Francisco: Morgan Kaufmann.

Vitter, Jeffrey Scott. 2008. "Algorithms and data structures for external memory." Foundations and Trends in Theoretical Computer Science 2, no. 4 (January): 305-474. <https://doi.org/10.1561/04000000014>

.

CHAPTER 4 Implementing B-Trees —

CHƯƠNG 4: Hiện thực B-Tree

In the previous chapter, we talked about general principles of binary format composition, and learned how to create cells, build hierarchies, and connect them to pages using pointers. These concepts are applicable for both in-place update and append-only storage structures. In this chapter, we discuss some concepts specific to B-Trees.

Trong chương trước, chúng ta đã bàn về các nguyên tắc chung khi tạo bố cục định dạng nhị phân, và học cách tạo các ô (cell), xây dựng phân cấp, và kết nối chúng với các trang (page) bằng con trỏ. Những khái niệm này áp dụng được cho cả cấu trúc lưu trữ cập-nhật-tại-chỗ lẫn cấu trúc chỉ-ghi-thêm. Trong chương này, chúng ta bàn về một số khái niệm đặc thù của B-Tree.

The sections in this chapter are split into three logical groups. First, we discuss organization: how to establish relationships between keys and pointers, and how to implement headers and links between pages.

Các mục trong chương này được chia thành ba nhóm logic. Trước hết, chúng ta bàn về tổ chức: cách thiết lập mối quan hệ giữa khóa và con trỏ, và cách hiện thực header cùng các liên kết giữa các trang.

Next, we discuss processes that occur during root-to-leaf descends, namely how to perform binary search and how to collect breadcrumbs and keep **track** of parent nodes in case we later have to split or **merge** nodes.

Tiếp theo, chúng ta bàn về những quá trình xảy ra trong lúc đi xuống từ gốc tới lá (root-to-leaf descent), cụ thể là cách thực hiện tìm kiếm nhị phân (binary search) và cách thu thập dấu vết đường đi (breadcrumb) cùng theo dõi các nút cha phòng khi về sau ta phải tách (split) hoặc gộp (merge) nút.

Lastly, we discuss optimization techniques (**rebalancing**, right-only appends, and **bulk loading**), maintenance processes, and **garbage collection**.

Cuối cùng, chúng ta bàn về các kỹ thuật tối ưu (tái cân bằng, ghi-thêm-chỉ-bên-phải, và nạp hàng loạt), các quá trình bảo trì, và thu gom rác (garbage collection).

Page Header — Page Header (Header của trang)

The **page** header holds information about the page that can be used for navigation, maintenance, and optimizations. It usually contains flags that describe page contents and layout, number of cells

in the page, lower and upper offsets marking the empty space (used to append **cell** offsets and data), and other useful metadata.

Header của trang chứa thông tin về trang có thể dùng cho điều hướng, bảo trì và tối ưu. Nó thường chứa các cờ mô tả nội dung và bố cục của trang, số ô trong trang, độ dời (offset) dưới và trên đánh dấu vùng trống (dùng để ghi thêm các offset của ô và dữ liệu), cùng các siêu dữ liệu hữu ích khác.

For example, PostgreSQL stores the page size and layout version in the header. In MySQL InnoDB , page header holds the number of heap records, level, and some other implementation-specific values. In SQLite , page header stores the number of cells and a **rightmost pointer**.

Ví dụ, PostgreSQL lưu kích thước trang và phiên bản bố cục trong header. Ở MySQL InnoDB, header của trang chứa số bản ghi heap, mức (level), và một số giá trị khác đặc thù cho hiện thực. Ở SQLite, header của trang lưu số ô và một con trỏ cực phải.

61

61

Magic Numbers — Magic Numbers (Số ma thuật)

One of the values often placed in the file or page header is a magic number. Usually, it's a multibyte **block**, containing a constant value that can be used to signal that the block represents a page, specify its kind, or identify its version.

Một trong những giá trị thường được đặt vào header của tệp hoặc trang là một magic number. Thông thường, đó là một khối nhiều byte, chứa một giá trị hằng có thể dùng để báo hiệu rằng khối đó biểu diễn một trang, xác định loại của nó, hoặc nhận diện phiên bản của nó.

Magic numbers are often used for validation and sanity checks [GIAMPAOLO98] . It's very improbable that the byte sequence at a random **offset** would exactly match the magic number. If it did match, there's a good chance the offset is correct. For example, to verify that the page is loaded and aligned correctly, during write we can place the magic number 50 41 47 45 (hex for PAGE) into the header. During the **read**, we validate the page by comparing the four bytes from the read header with the expected byte sequence.

Magic number thường được dùng cho việc xác thực và kiểm tra tính hợp lý [GIAMPAOLO98]. Rất khó xảy ra việc chuỗi byte tại một offset ngẫu nhiên lại trùng khớp chính xác với magic number. Nếu nó trùng khớp, nhiều khả năng offset đó là đúng. Ví dụ, để xác minh rằng trang được nạp và căn chỉnh đúng, trong lúc ghi ta có thể đặt magic number 50 41 47 45 (mã hex của PAGE) vào header. Trong lúc đọc, ta xác thực trang bằng cách so sánh bốn byte từ header đã đọc với chuỗi byte kỳ vọng.

Sibling Links — Sibling Links (Liên kết anh em)

Some implementations store forward and backward links, pointing to the left and right sibling pages. These links help to locate neighboring nodes without having to ascend back to the parent. This approach adds some complexity to split and merge operations, as the sibling offsets have to be updated as well. For example, when a non-rightmost **node** is split, its right sibling's backward pointer (previously pointing to the node that was split) has to be re-bounded to point to the newly created node.

Một số hiện thực lưu các liên kết xuôi và ngược, trở tới các trang anh em bên trái và bên phải. Các liên kết này giúp định vị các nút lân cận mà không phải đi ngược lên cha. Cách tiếp cận này làm cho thao tác tách và gộp phức tạp hơn đôi chút, vì các offset của anh em cũng phải được cập nhật. Ví dụ, khi một nút không phải nút cực phải bị tách, con trở ngược của anh em bên phải của nó (trước đó trở tới nút bị tách) phải được gắn lại để trở tới nút mới tạo.

In Figure 4-1 you can see that to locate a sibling node, unless the siblings are linked, we have to refer to the parent node. This operation might ascend all the way up to the root, since the direct parent can only help to address its own children. If we store sibling links directly in the header, we can simply follow them to locate the previous or next node on the same level.

Trong Hình 4-1, bạn có thể thấy rằng để định vị một nút anh em, nếu các anh em không được liên kết với nhau, ta phải tham chiếu tới nút cha. Thao tác này có thể phải đi ngược lên tới tận gốc, vì cha trực tiếp chỉ có thể giúp định địa chỉ các con của chính nó. Nếu ta lưu liên kết anh em (sibling link) trực tiếp trong header, ta chỉ cần đi theo chúng để định vị nút trước hoặc nút sau ở cùng mức.

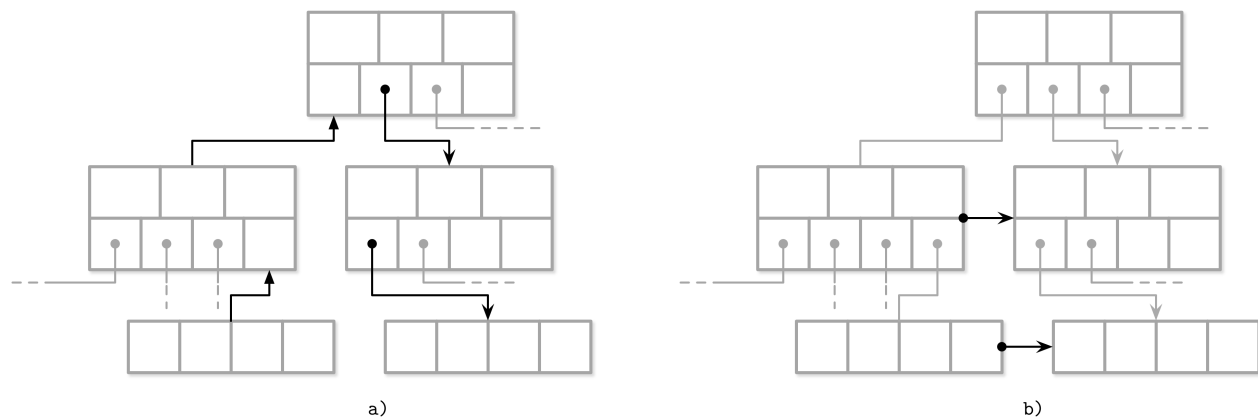


Figure 4-1. Locating a sibling by following parent links (a) versus sibling links (b) One of the downsides of storing sibling links is that they have to be updated during splits and merges. Since updates have to happen in a sibling node, not in a splitting/merging node, it may require additional locking. We discuss how sibling links can be useful in a concurrent **B-Tree** implementation in “Blink-Trees” on page 107 .

Hình 4-1. Định vị một nút anh em bằng cách đi theo liên kết cha (a) so với liên kết anh em (b). Một trong những nhược điểm của việc lưu liên kết anh em là chúng phải được cập nhật trong lúc tách và gộp. Vì cập nhật phải xảy ra ở nút anh em, chứ không phải ở nút đang tách/gộp, nó có thể đòi hỏi thêm khóa (lock). Chúng ta sẽ bàn cách liên kết anh em có thể hữu ích trong một hiện thực B-Tree tương tranh ở mục “Blink-Trees” trang 107.

Rightmost Pointers — Rightmost Pointers (Con trở cực phải)

B-Tree separator keys have strict invariants: they’re used to split the tree into subtrees and navigate them, so there is always one more pointer to child pages than there are keys. That’s where the +1 mentioned in “Counting Keys” on page 38 is coming from.

Các khóa phân tách (separator key) của B-Tree có những bất biến chặt chẽ: chúng được dùng để chia cây thành các cây con và điều hướng trong đó, nên luôn có nhiều hơn một

con trỏ tới trang con so với số khóa. Đó chính là chỗ phát sinh ra số +1 được nhắc tới trong mục “Counting Keys” trang 38.

In “Separator Keys” on page 36 , we described **separator key** invariants. In many implementations, nodes look more like the ones displayed in Figure 4-2 : each separator key has a child pointer, while the last pointer is stored separately, since it’s not paired with any key. You can compare this to Figure 2-10 .

Trong mục “Separator Keys” trang 36, chúng ta đã mô tả các bất biến của khóa phân tách. Trong nhiều hiện thực, các nút trông giống như những nút hiển thị trong Hình 4-2: mỗi khóa phân tách có một con trỏ con, còn con trỏ cuối cùng được lưu riêng, vì nó không ghép cặp với khóa nào. Bạn có thể so sánh điều này với Hình 2-10.

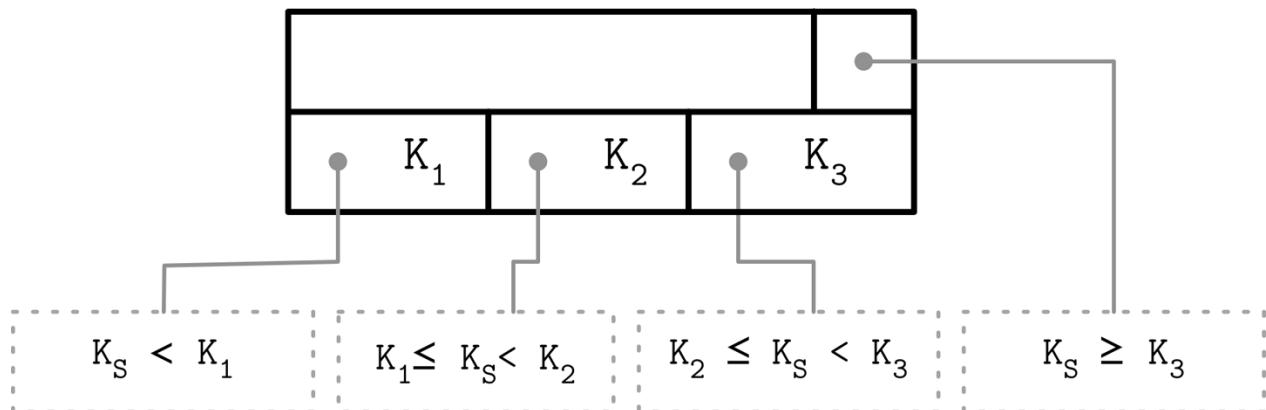


Figure 4-2. Rightmost pointer

Hình 4-2. Con trỏ cực phải

This extra pointer can be stored in the header as, for example, it is implemented in SQLite .

Con trỏ phụ này có thể được lưu trong header, ví dụ như cách nó được hiện thực trong SQLite.

If the rightmost child is split and the new cell is appended to its parent, the rightmost child pointer has to be reassigned. As shown in Figure 4-3 , after the split, the cell appended to the parent (shown in gray) holds the promoted key and points to the split node. The pointer to the new node is assigned instead of the previous rightmost pointer. A similar approach is described and implemented in SQLite.¹

Nếu con cực phải bị tách và ô mới được ghi thêm vào cha của nó, con trỏ con cực phải phải được gán lại. Như trong Hình 4-3, sau khi tách, ô được ghi thêm vào cha (hiển thị màu xám) chứa khóa được đẩy lên và trỏ tới nút bị tách. Con trỏ tới nút mới được gán thay cho con trỏ cực phải trước đó. Một cách tiếp cận tương tự được mô tả và hiện thực trong SQLite.¹

¹ You can find this algorithm in the `balance_deeper` function in the project repository .

¹ Bạn có thể tìm thấy thuật toán này trong hàm `balance_deeper` trong kho mã của dự án.

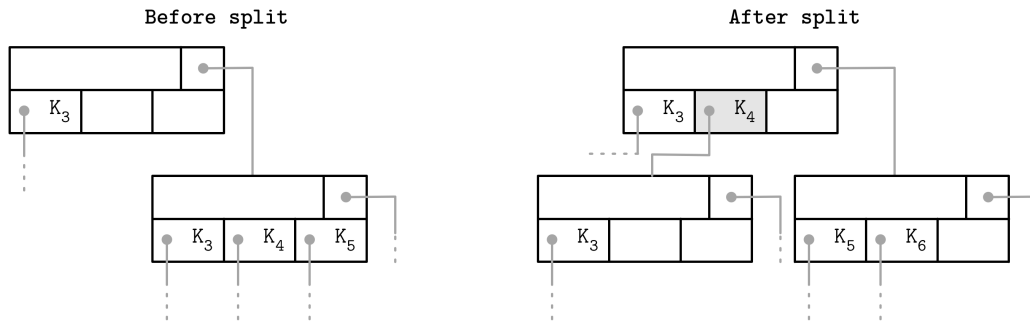


Figure 4-3. Rightmost pointer update during **node split**. The promoted key is shown in gray.

Hình 4-3. Cập nhật con trỏ cực phải trong lúc tách nút. Khóa được đẩy lên hiển thị màu xám.

Node High Keys — Node High Keys (Khóa biên của nút)

We can take a slightly different approach and store the rightmost pointer in the cell along with the node **high key**. The high key represents the highest possible key that can be present in the subtree under the current node. This approach is used by PostgreSQL and is called B **link**-Trees (for concurrency implications of this approach, see “Blink-Trees” on page 107).

Chúng ta có thể áp dụng một cách tiếp cận hơi khác và lưu con trỏ cực phải trong ô cùng với khóa biên (high key) của nút. Khóa biên biểu diễn khóa lớn nhất có thể hiện diện trong cây con dưới nút hiện tại. Cách tiếp cận này được PostgreSQL sử dụng và được gọi là Blink-Tree (về hệ quả tương tranh của cách tiếp cận này, xem mục “Blink-Trees” trang 107).

B-Trees have N keys (denoted with K) and $N + 1$ pointers (denoted with P). In each

B-Tree có N khóa (ký hiệu là K) và $N + 1$ con trỏ (ký hiệu là P). Trong mỗi subtree, keys are bounded by $K \leq K < K$. The $K = -\infty$ is implicit and is not

cây con, các khóa bị chặn bởi $K \leq K < K$. Khóa $K = -\infty$ là ngầm định và không present in the node.

hiện diện trong nút.

B link-Trees add a K key to each node. It specifies an upper bound of keys that can be

Blink-Tree thêm một khóa K vào mỗi nút. Nó xác định cận trên của các khóa có thể được stored in the subtree to which the pointer P points, and therefore is an upper bound

lưu trong cây con mà con trỏ P trỏ tới, và do đó là cận trên

of values that can be stored in the current subtree. Both approaches are shown in Figure 4-4 : (a) shows a node without a high key, and (b) shows a node with a high key.

của các giá trị có thể được lưu trong cây con hiện tại. Cả hai cách tiếp cận được hiển thị trong Hình 4-4: (a) thể hiện một nút không có khóa biên, và (b) thể hiện một nút có khóa biên.

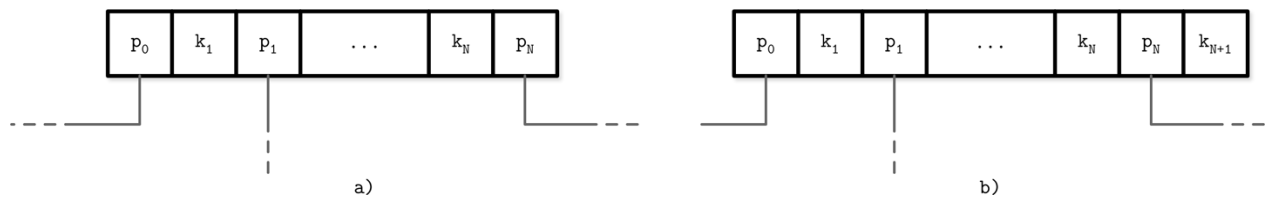


Figure 4-4. B-Trees without (a) and with (b) a high key

Hình 4-4. B-Tree không có (a) và có (b) khóa biên

In this case, pointers can be stored pairwise, and each cell can have a corresponding pointer, which might simplify rightmost pointer handling as there are not as many edge cases to consider.

Trong trường hợp này, các con trỏ có thể được lưu theo cặp, và mỗi ô có thể có một con trỏ tương ứng, điều này có thể đơn giản hóa việc xử lý con trỏ cực phải vì không còn nhiều trường hợp biên cần cân nhắc.

In Figure 4-5, you can see schematic page structure for both approaches and how the search space is split differently for these cases: going up to $+\infty$ in the first case, and up to the upper bound of K in the second.

Trong Hình 4-5, bạn có thể thấy cấu trúc trang dạng sơ đồ cho cả hai cách tiếp cận và cách không gian tìm kiếm được chia khác nhau ở hai trường hợp này: đi lên tới $+\infty$ ở trường hợp đầu, và đi lên tới cận trên của K ở trường hợp sau.

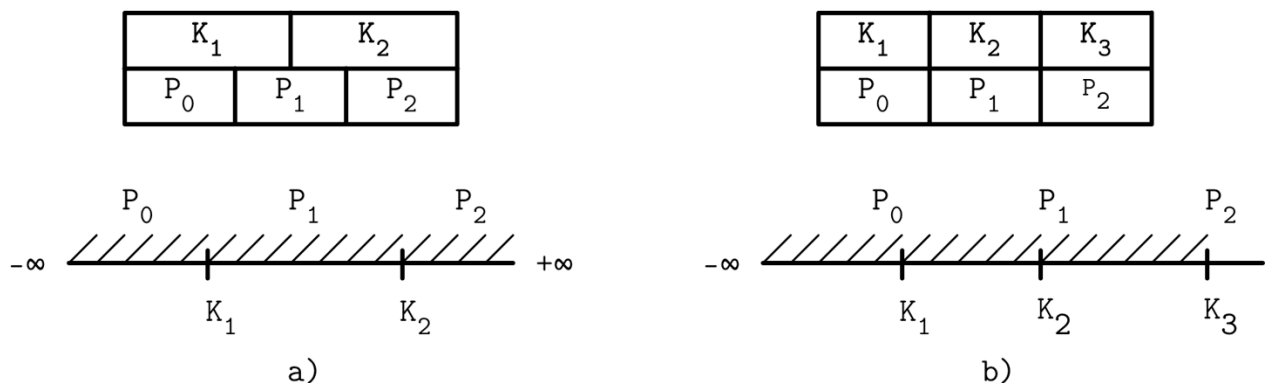


Figure 4-5. Using $+\infty$ as a virtual key (a) versus storing the high key (b)

Hình 4-5. Dùng $+\infty$ làm khóa ảo (a) so với lưu khóa biên (b)

Overflow Pages — Overflow Pages (Trang tràn)

Node size and tree **fanout** values are fixed and do not change dynamically. It would also be difficult to come up with a value that would be universally optimal: if variable-size values are present in the tree and they are large enough, only a few of them can fit into the page. If the values are tiny, we end up wasting the reserved space.

Kích thước nút và giá trị độ phân nhánh (fanout) của cây là cố định và không thay đổi động. Cũng sẽ khó nghĩ ra một giá trị tối ưu phổ quát: nếu trong cây có các giá trị kích thước thay đổi và chúng đủ lớn, thì chỉ một vài giá trị mới vừa với trang. Còn nếu các giá trị quá nhỏ, ta lại lãng phí không gian đã dành sẵn.

The B-Tree algorithm specifies that every node keeps a specific number of items. Since some values have different sizes, we may end up in a situation where, according to the B-Tree algorithm, the node

is not full yet, but there's no more free space on the fixed-size page that holds this node. Resizing the page requires copying already written data to the new region and is often impractical. However, we still need to find a way to increase or extend the page size.

Thuật toán B-Tree quy định rằng mỗi nút giữ một số lượng phần tử nhất định. Vì một số giá trị có kích thước khác nhau, ta có thể rơi vào tình huống mà theo thuật toán B-Tree thì nút chưa đầy, nhưng trên trang kích thước cố định chứa nút đó lại không còn không gian trống. Việc đổi kích thước trang đòi hỏi sao chép dữ liệu đã ghi sang vùng mới và thường không thực tế. Tuy nhiên, ta vẫn cần tìm cách tăng hoặc mở rộng kích thước trang.

To implement variable-size nodes without copying data to the new contiguous region, we can build nodes from multiple linked pages. For example, the default page size is 4 K, and after inserting a few values, its data size has grown over 4 K. Instead of allowing arbitrary sizes, nodes are allowed to grow in 4 K increments, so we allocate a 4 K extension page and link it from the original one. These linked page extensions are called overflow pages . For clarity, we call the original page the primary page in the scope of this section.

Để hiện thực các nút kích thước thay đổi mà không phải sao chép dữ liệu sang vùng liên kề mới, ta có thể dựng nút từ nhiều trang được liên kết với nhau. Ví dụ, kích thước trang mặc định là 4 K, và sau khi chèn vài giá trị, kích thước dữ liệu của nó đã vượt 4 K. Thay vì cho phép kích thước tùy ý, các nút được phép tăng theo từng nấc 4 K, nên ta cấp phát một trang mở rộng 4 K và liên kết nó từ trang gốc. Các phần mở rộng trang được liên kết này được gọi là trang tràn (overflow page). Để rõ ràng, trong phạm vi mục này ta gọi trang gốc là trang chính (primary page).

Most B-Tree implementations allow storing only up to a fixed number of payload bytes in the B-Tree node directly and spilling the rest to the **overflow page**. This value is calculated by dividing the node size by fanout. Using this approach, we cannot end up in a situation where the page has no free space, as it will always have at least `max_payload_size` bytes. For more information on overflow pages in SQLite, see the SQLite source code repository ; also check out the MySQL InnoDB documentation .

Phần lớn các hiện thực B-Tree chỉ cho phép lưu trực tiếp tối đa một số byte tải trọng cố định trong nút B-Tree và tràn phần còn lại sang trang tràn. Giá trị này được tính bằng cách chia kích thước nút cho fanout. Dùng cách này, ta không thể rơi vào tình huống trang không còn không gian trống, vì nó sẽ luôn có ít nhất `max_payload_size` byte. Để biết thêm về trang tràn trong SQLite, xem kho mã nguồn SQLite; cũng nên xem tài liệu MySQL InnoDB.

When the inserted payload is larger than `max_payload_size` , the node is checked for whether or not it already has any associated overflow pages. If an overflow page already exists and has enough space available, extra bytes from the payload are spilled there. Otherwise, a new overflow page is allocated.

Khi tải trọng được chèn lớn hơn `max_payload_size`, nút được kiểm tra xem nó đã có trang tràn liên kết nào hay chưa. Nếu trang tràn đã tồn tại và còn đủ không gian, các byte dư từ tải trọng được tràn vào đó. Nếu không, một trang tràn mới được cấp phát.

In Figure 4-6 , you can see a primary page and an overflow page with records pointing from the primary page to the overflow one, where their payload continues.

Trong Hình 4-6, bạn có thể thấy một trang chính và một trang tràn với các bản ghi trỏ từ trang chính sang trang tràn, nơi tải trọng của chúng tiếp tục.

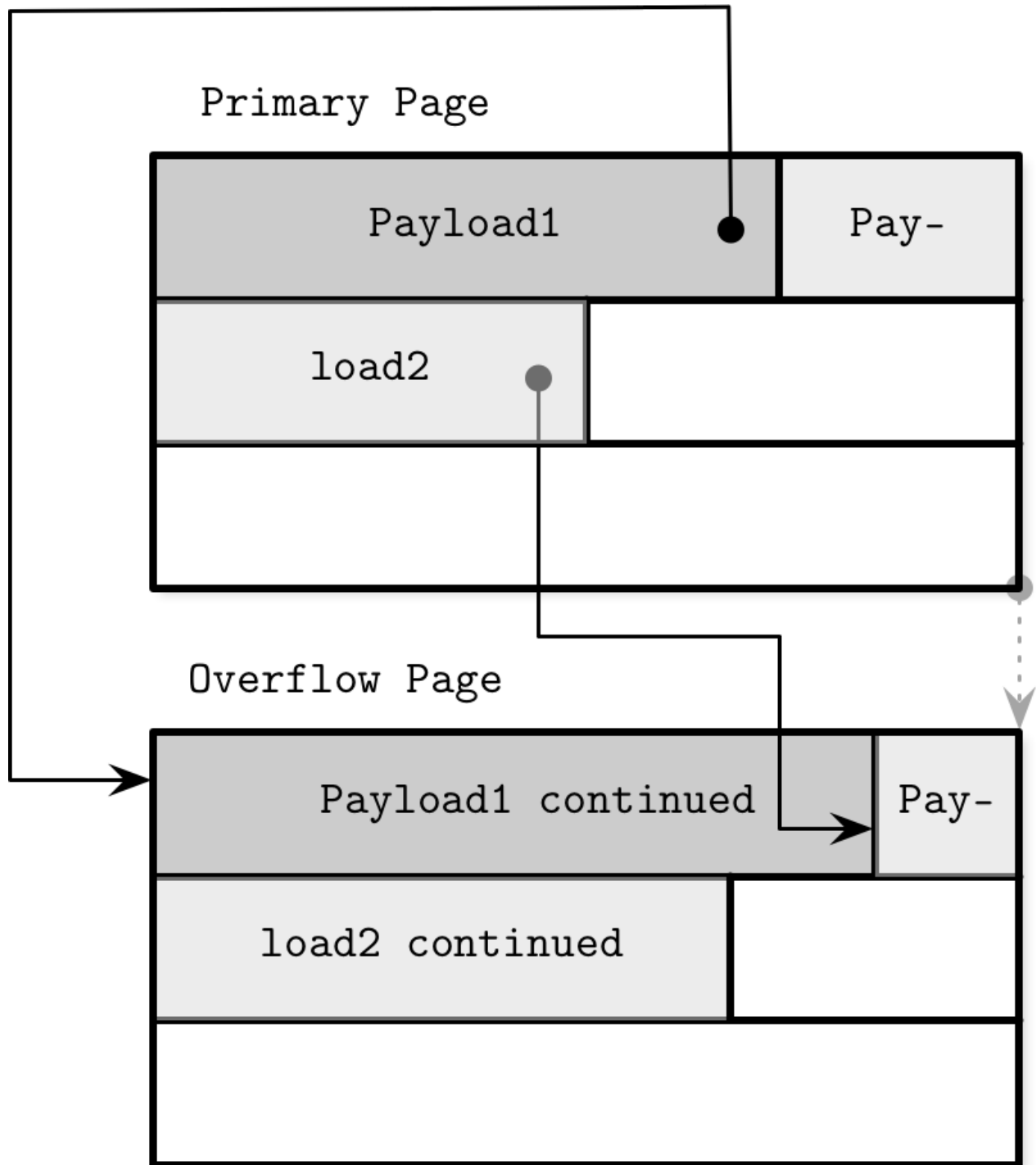


Figure 4-6. Overflow pages

Hình 4-6. Trang tràn

Overflow pages require some extra bookkeeping, since they may get fragmented as well as primary pages, and we have to be able to reclaim this space to write new data, or discard the overflow page if it's not needed anymore.

Trang tràn đòi hỏi thêm một chút công việc theo dõi, vì chúng cũng có thể bị phân mảnh

giống như trang chính, và ta phải có khả năng thu hồi không gian này để ghi dữ liệu mới, hoặc loại bỏ trang tràn nếu không còn cần nữa.

When the first overflow page is allocated, its page ID is stored in the header of the primary page. If a single overflow page is not enough, multiple overflow pages are linked together by storing the next overflow page ID in the previous one's header. Several pages may have to be traversed to locate the overflow part for the given payload.

Khi trang tràn đầu tiên được cấp phát, ID trang của nó được lưu trong header của trang chính. Nếu một trang tràn duy nhất không đủ, nhiều trang tràn được liên kết với nhau bằng cách lưu ID của trang tràn kế tiếp trong header của trang trước. Có thể phải duyệt qua vài trang để định vị phần tràn cho một tải trọng nhất định.

Since keys usually have high cardinality, storing a portion of a key makes sense, as most of the comparisons can be made on the trimmed key part that resides in the primary page.

Vì các khóa thường có lực lượng cao (high cardinality), việc lưu một phần của khóa là hợp lý, vì hầu hết các phép so sánh có thể thực hiện trên phần khóa đã cắt bớt nằm ở trang chính.

For data records, we have to locate their overflow parts to return them to the user. However, this doesn't matter much, since it's an infrequent operation. If all data records are oversize, it is worth considering specialized blob storage for large values.

Đối với các bản ghi dữ liệu, ta phải định vị phần tràn của chúng để trả về cho người dùng. Tuy nhiên, điều này không quan trọng lắm, vì đó là thao tác không thường xuyên. Nếu tất cả các bản ghi dữ liệu đều quá khổ, đáng để cân nhắc dùng kho lưu blob chuyên dụng cho các giá trị lớn.

Binary Search — Binary Search (Tìm kiếm nhị phân)

We've already discussed the B-Tree lookup algorithm (see "B-Tree Lookup Algorithm" on page 38) and mentioned that we locate a searched key within the node using the binary search algorithm. Binary search works only for sorted data. If keys are not ordered, they can't be binary searched. This is why keeping keys in order and maintaining a sorted invariant is essential.

Chúng ta đã bàn về thuật toán tra cứu của B-Tree (xem "B-Tree Lookup Algorithm" trang 38) và đã nhắc rằng ta định vị khóa cần tìm trong nút bằng thuật toán tìm kiếm nhị phân (binary search). Tìm kiếm nhị phân chỉ hoạt động với dữ liệu đã sắp xếp. Nếu các khóa không có thứ tự, chúng không thể được tìm kiếm nhị phân. Đây là lý do vì sao việc giữ các khóa theo thứ tự và duy trì bất biến đã-sắp-xếp là thiết yếu.

The binary search algorithm receives an array of sorted items and a searched key, and returns a number. If the returned number is positive, we know that the searched key was found and the number specifies its position in the input array. A negative return value indicates that the searched key is not present in the input array and gives us an insertion point.

Thuật toán tìm kiếm nhị phân nhận một mảng các phần tử đã sắp xếp và một khóa cần tìm, rồi trả về một con số. Nếu số trả về là dương, ta biết rằng khóa cần tìm đã được tìm thấy và con số đó xác định vị trí của nó trong mảng đầu vào. Một giá trị trả về âm cho biết khóa cần tìm không có mặt trong mảng đầu vào và cho ta một điểm chèn (insertion point).

The insertion point is the index of the first element that is greater than the given key. An absolute value of this number is the index at which the searched key can be inserted to preserve order.

Insertion can be done by shifting elements over one position, starting from an insertion point, to make space for the inserted element [SEEDGE- WICK11] .

Điểm chèn là chỉ số của phần tử đầu tiên lớn hơn khóa đã cho. Giá trị tuyệt đối của con số này là chỉ số mà tại đó khóa cần tìm có thể được chèn vào để bảo toàn thứ tự. Việc chèn có thể được thực hiện bằng cách dịch các phần tử sang một vị trí, bắt đầu từ điểm chèn, để tạo chỗ trống cho phần tử được chèn [SEEDGEWICK11].

The majority of searches on higher levels do not result in exact matches, and we're interested in the search direction, in which case we have to find the first value that is greater than the searched one and follow the corresponding child link into the associated subtree.

Phần lớn các phép tìm kiếm ở các mức cao hơn không cho ra kết quả khớp chính xác, và ta quan tâm tới hướng tìm kiếm, trong trường hợp đó ta phải tìm giá trị đầu tiên lớn hơn giá trị cần tìm và đi theo liên kết con tương ứng vào cây con liên quan.

Binary Search with Indirection Pointers — Binary Search with Indirection Pointers (Tìm kiếm nhị phân với con trỏ gián tiếp)

Cells in the B-Tree page are stored in the insertion order, and only cell offsets preserve the logical element order. To perform binary search through page cells, we pick the middle cell offset, follow its pointer to locate the cell, compare the key from this cell with the searched key to decide whether the search should continue left or right, and continue this **process** recursively until the searched element or the insertion point is found, as shown in Figure 4-7 .

Các ô trong trang B-Tree được lưu theo thứ tự chèn, và chỉ có các offset của ô mới bảo toàn thứ tự logic của phần tử. Để thực hiện tìm kiếm nhị phân qua các ô của trang, ta chọn offset của ô ở giữa, đi theo con trỏ của nó để định vị ô, so sánh khóa từ ô này với khóa cần tìm để quyết định xem nên tiếp tục tìm sang trái hay sang phải, và tiếp tục quá trình này một cách đệ quy cho tới khi tìm thấy phần tử cần tìm hoặc điểm chèn, như trong Hình 4-7.

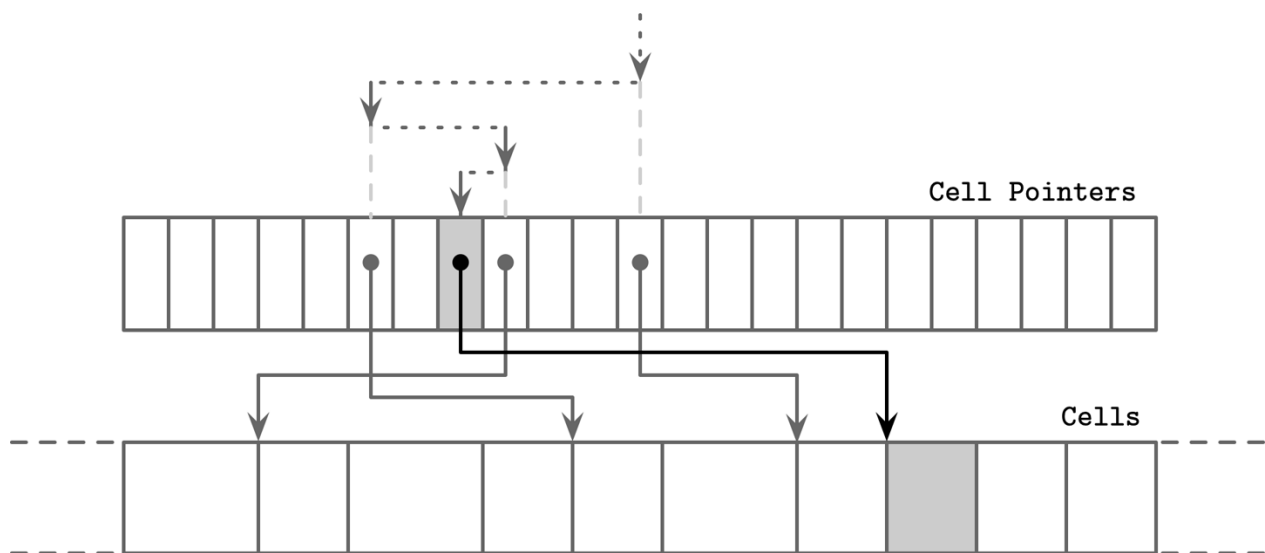


Figure 4-7. Binary search with indirection pointers. The searched element is shown in gray. Dotted arrows represent binary search through cell pointers. Solid lines represent accesses that follow the cell pointers, necessary to compare the cell value with a searched key.

Hình 4-7. Tìm kiếm nhị phân với con trỏ gián tiếp. Phần tử cần tìm hiển thị màu xám. Các mũi tên đứt nét biểu diễn quá trình tìm kiếm nhị phân qua các con trỏ ô. Các đường liền biểu diễn các truy cập đi theo con trỏ ô, cần thiết để so sánh giá trị của ô với khóa cần tìm.

Propagating Splits and Merges — Propagating Splits and Merges (Lan truyền tách và gộp)

As we’ve discussed in previous chapters, B-Tree splits and merges can propagate to higher levels. For that, we need to be able to traverse a chain back to the root node from the splitting leaf or a pair of merging leaves.

Như đã bàn trong các chương trước, các thao tác tách và gộp của B-Tree có thể lan truyền lên các mức cao hơn. Để làm điều đó, ta cần có khả năng đi ngược một chuỗi về tới nút gốc từ lá đang tách hoặc một cặp lá đang gộp.

B-Tree nodes may include parent node pointers. Since pages from lower levels are always paged in when they’re referenced from a higher level, it is not even necessary to persist this information on disk.

Các nút B-Tree có thể bao gồm con trỏ tới nút cha. Vì các trang ở mức thấp hơn luôn được nạp vào bộ nhớ khi chúng được tham chiếu từ mức cao hơn, nên thậm chí không cần lưu bền thông tin này trên đĩa.

Just like sibling pointers (see “Sibling Links” on page 62), parent pointers have to be updated whenever the parent changes. This happens in all the cases when the separator key with the page identifier is transferred from one node to another: during the parent node splits, merges, or rebalancing of the parent node.

Cũng giống như con trỏ anh em (xem “Sibling Links” trang 62), con trỏ cha phải được cập nhật mỗi khi cha thay đổi. Điều này xảy ra trong mọi trường hợp khi khóa phân tách cùng với mã định danh trang được chuyển từ nút này sang nút khác: trong lúc tách nút cha, gộp, hoặc tái cân bằng nút cha.

Some implementations (for example, WiredTiger) use parent pointers for leaf traversal to avoid deadlocks, which may happen when using sibling pointers (see [MILLER78] , [LEHMAN81]). Instead of using sibling pointers to traverse leaf nodes, the algorithm employs parent pointers, much like we saw in Figure 4-1 .

Một số hiện thực (ví dụ, WiredTiger) dùng con trỏ cha để duyệt lá nhằm tránh bế tắc (deadlock), vốn có thể xảy ra khi dùng con trỏ anh em (xem [MILLER78], [LEHMAN81]). Thay vì dùng con trỏ anh em để duyệt các nút lá, thuật toán sử dụng con trỏ cha, khá giống điều ta đã thấy trong Hình 4-1.

To address and locate a sibling, we can follow a pointer from the parent node and recursively descend back to the lower level. Whenever we reach the end of the parent node after traversing all the siblings sharing the parent, the search continues upward recursively, eventually reaching up to the root and continuing back down to the leaf level.

Để định địa chỉ và định vị một nút anh em, ta có thể đi theo một con trỏ từ nút cha và đệ quy đi xuống về mức thấp hơn. Mỗi khi ta tới cuối nút cha sau khi đã duyệt hết các anh em chung cha, việc tìm kiếm tiếp tục đi lên một cách đệ quy, cuối cùng lên tới gốc rồi đi ngược trở xuống mức lá.

Breadcrumbs — Breadcrumbs (Dấu vết đường đi)

Instead of storing and maintaining parent node pointers, it is possible to keep track of nodes traversed on the path to the target leaf node, and follow the chain of parent nodes in reverse order in case of cascading splits during inserts, or merges during deletes.

Thay vì lưu và duy trì con trỏ tới nút cha, ta có thể theo dõi các nút đã duyệt qua trên đường đi tới nút lá đích, và đi theo chuỗi nút cha theo thứ tự ngược lại trong trường hợp xảy ra tách dây chuyền khi chèn, hoặc gộp khi xóa.

During operations that may result in structural changes of the B-Tree (insert or delete), we first traverse the tree from the root to the leaf to find the target node and the insertion point. Since we do not always know up front whether or not the operation will result in a split or merge (at least not until the target leaf node is located), we have to collect breadcrumbs .

Trong các thao tác có thể dẫn tới thay đổi cấu trúc của B-Tree (chèn hoặc xóa), trước tiên ta duyệt cây từ gốc tới lá để tìm nút đích và điểm chèn. Vì ta không phải lúc nào cũng biết trước thao tác có dẫn tới tách hay gộp hay không (ít nhất là cho tới khi định vị được nút lá đích), nên ta phải thu thập dấu vết đường đi (breadcrumb).

Breadcrumbs contain references to the nodes followed from the root and are used to backtrack them in reverse when propagating splits or merges. The most natural data structure for this is a stack. For example, PostgreSQL stores breadcrumbs in a stack, internally referenced as BTStack. 2

Dấu vết đường đi chứa các tham chiếu tới những nút đã đi theo từ gốc và được dùng để lần ngược lại chúng khi lan truyền tách hoặc gộp. Cấu trúc dữ liệu tự nhiên nhất cho việc này là ngăn xếp (stack). Ví dụ, PostgreSQL lưu dấu vết đường đi trong một ngăn xếp, được tham chiếu nội bộ là BTStack.2

If the node is split or merged, breadcrumbs can be used to find insertion points for the keys pulled to the parent and to walk back up the tree to propagate structural changes to the higher-level nodes, if necessary. This stack is maintained in memory.

Nếu nút bị tách hoặc gộp, dấu vết đường đi có thể được dùng để tìm các điểm chèn cho những khóa được kéo lên cha và để đi ngược lên cây nhằm lan truyền các thay đổi cấu trúc lên các nút mức cao hơn, nếu cần. Ngăn xếp này được duy trì trong bộ nhớ.

Figure 4-8 shows an example of root-to-leaf traversal, collecting breadcrumbs containing pointers to the visited nodes and cell indices. If the target leaf node is split, the item on top of the stack is popped to locate its immediate parent. If the parent node has enough space, a new cell is appended to it at the cell index from the **breadcrumb** (assuming the index is still valid). Otherwise, the parent node is split as well. This process continues recursively until either the stack is empty and we have reached the root, or there was no split on the level.

Hình 4-8 thể hiện một ví dụ về việc duyệt từ gốc tới lá, thu thập dấu vết đường đi chứa con trỏ tới các nút đã thăm và chỉ số ô. Nếu nút lá đích bị tách, phần tử trên đỉnh ngăn xếp được lấy ra (pop) để định vị cha trực tiếp của nó. Nếu nút cha còn đủ không gian, một ô mới được ghi thêm vào nó tại chỉ số ô lấy từ dấu vết đường đi (giả định chỉ số đó vẫn còn hợp lệ). Nếu không, nút cha cũng bị tách. Quá trình này tiếp tục một cách đệ quy cho tới khi hoặc ngăn xếp rỗng và ta đã tới gốc, hoặc không có sự tách nào ở mức đó.

2 You can read more about it in the project repository: <https://databass.dev/links/21> .

2 Bạn có thể đọc thêm về nó trong kho mã của dự án: <https://databass.dev/links/21>.

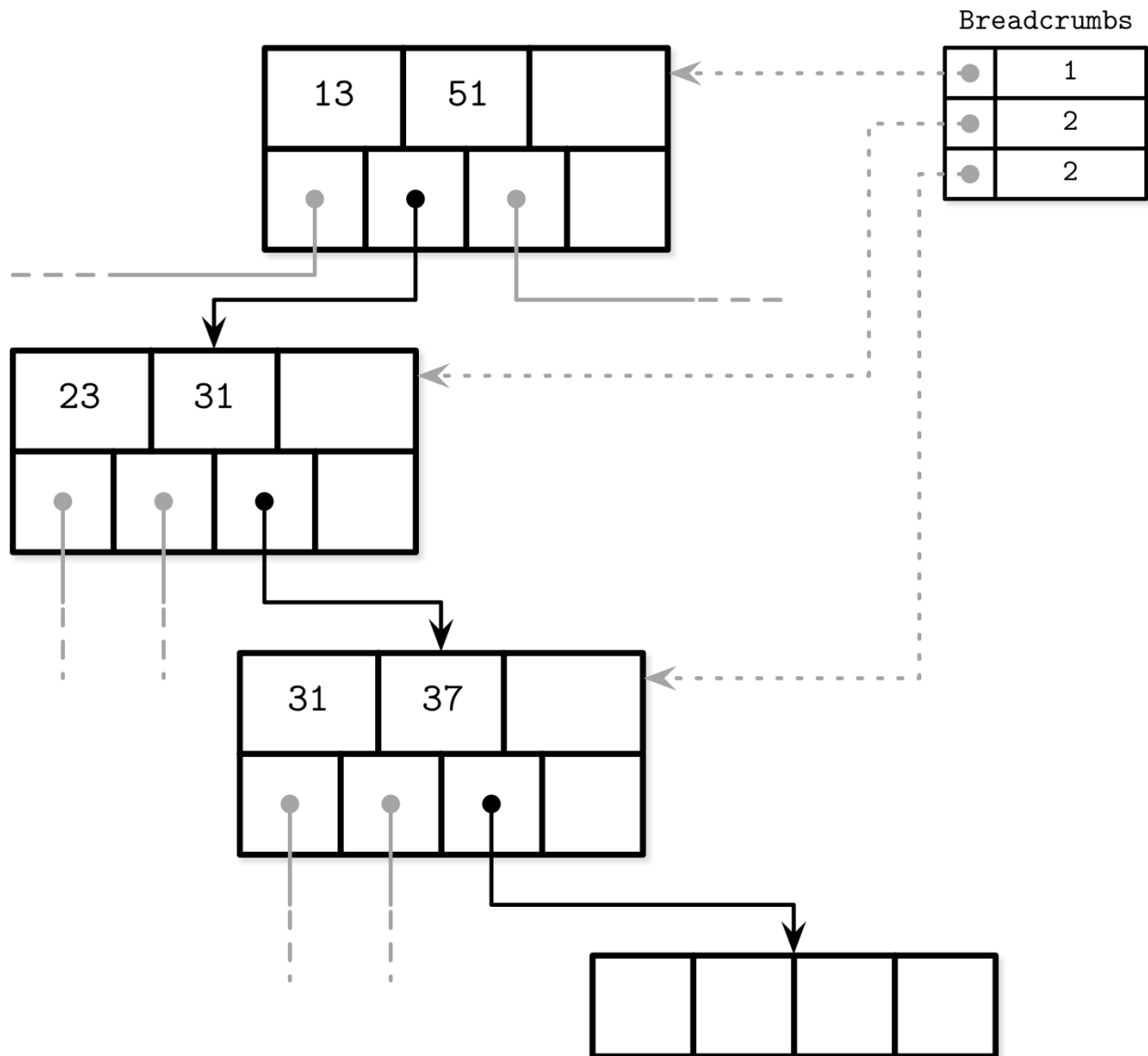


Figure 4-8. Breadcrumbs collected during lookup, containing traversed nodes and cell indices. Dotted lines represent logical links to visited nodes. Numbers in the breadcrumbs table represent indices of the followed child pointers.

Hình 4-8. Dấu vết đường đi được thu thập trong lúc tra cứu, chứa các nút đã duyệt và chỉ số ô. Các đường đứt nét biểu diễn liên kết logic tới các nút đã thăm. Các con số trong bảng dấu vết đường đi biểu diễn chỉ số của các con trỏ con đã đi theo.

Rebalancing — Rebalancing (Tái cân bằng)

Some B-Tree implementations attempt to postpone split and merge operations to amortize their costs by rebalancing elements within the level, or moving elements from more occupied nodes to less occupied ones for as long as possible before finally performing a split or merge. This helps to improve node **occupancy** and may reduce the number of levels within the tree at a potentially higher maintenance cost of rebalancing.

Một số hiện thực B-Tree cố gắng hoãn các thao tác tách và gộp để san sẻ chi phí của chúng bằng cách tái cân bằng các phần tử trong cùng một mức, hoặc chuyển phần tử từ các nút lấp đầy nhiều hơn sang các nút lấp đầy ít hơn càng lâu càng tốt trước khi cuối cùng buộc phải tách hoặc gộp. Điều này giúp cải thiện độ lấp đầy (occupancy) của nút và có thể giảm số mức trong cây, đổi lại có thể tốn chi phí bảo trì tái cân bằng cao hơn.

Load balancing can be performed during insert and delete operations [GRAEFE11]. To improve space utilization, instead of splitting the node on overflow, we can transfer some of the elements to one of the sibling nodes and make space for the insertion. Similarly, during delete, instead of merging the sibling nodes, we may choose to move some of the elements from the neighboring nodes to ensure the node is at least half full.

Cân bằng tải có thể được thực hiện trong các thao tác chèn và xóa [GRAEFE11]. Để cải thiện việc tận dụng không gian, thay vì tách nút khi tràn, ta có thể chuyển một số phần tử sang một trong các nút anh em và tạo chỗ trống cho việc chèn. Tương tự, khi xóa, thay vì gộp các nút anh em, ta có thể chọn chuyển một số phần tử từ các nút lân cận để đảm bảo nút ít nhất đầy một nửa.

B*-Trees keep distributing data between the neighboring nodes until both siblings are full [KNUTH98]. Then, instead of splitting a single node into two half-empty ones, the algorithm splits two nodes into three nodes, each of which is two-thirds full. SQLite uses this variant in the implementation. This approach improves an average occupancy by postponing splits, but requires additional tracking and balancing logic. Higher utilization also means more efficient searches, because the height of the tree is smaller and fewer pages have to be traversed on the path to the searched leaf.

B*-Tree tiếp tục phân phối dữ liệu giữa các nút lân cận cho tới khi cả hai anh em đều đầy [KNUTH98]. Khi đó, thay vì tách một nút thành hai nút đầy một nửa, thuật toán tách hai nút thành ba nút, mỗi nút đầy hai phần ba. SQLite dùng biến thể này trong hiện thực. Cách tiếp cận này cải thiện độ lấp đầy trung bình bằng cách hoãn tách, nhưng đòi hỏi thêm logic theo dõi và cân bằng. Mức tận dụng cao hơn cũng có nghĩa là tìm kiếm hiệu quả hơn, vì chiều cao của cây nhỏ hơn và phải duyệt qua ít trang hơn trên đường tới lá cần tìm.

Figure 4-9 shows distributing elements between the neighboring nodes, where the left sibling contains more elements than the right one. Elements from the more occupied node are moved to the less occupied one. Since balancing changes the min/max invariant of the sibling nodes, we have to update keys and pointers at the parent node to preserve it.

Hình 4-9 thể hiện việc phân phối các phần tử giữa các nút lân cận, trong đó anh em bên trái chứa nhiều phần tử hơn anh em bên phải. Các phần tử từ nút lấp đầy nhiều hơn được chuyển sang nút lấp đầy ít hơn. Vì việc cân bằng làm thay đổi bất biến min/max của các nút anh em, ta phải cập nhật khóa và con trỏ ở nút cha để bảo toàn nó.

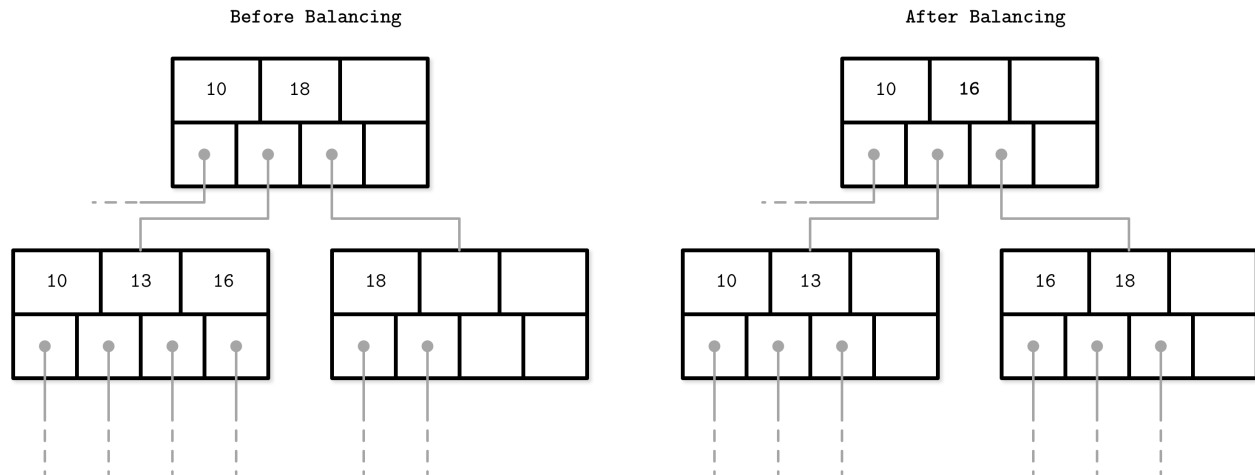


Figure 4-9. **B-Tree balancing**: Distributing elements between the more occupied node and the less occupied one

Hình 4-9. Cân bằng B-Tree: Phân phối các phần tử giữa nút lấp đầy nhiều hơn và nút lấp đầy ít hơn

Load balancing is a useful technique used in many database implementations. For example, SQLite implements the balance-siblings algorithm, which is somewhat close to what we have described in this section. Balancing might add some complexity to the code, but since its use cases are isolated, it can be implemented as an optimization at a later stage.

Cân bằng tải là một kỹ thuật hữu ích được dùng trong nhiều hiện thực cơ sở dữ liệu. Ví dụ, SQLite hiện thực thuật toán balance-siblings, khá gần với những gì ta đã mô tả trong mục này. Cân bằng có thể làm mã phức tạp thêm đôi chút, nhưng vì các tình huống sử dụng của nó là tách biệt, nó có thể được hiện thực như một tối ưu ở giai đoạn về sau.

Right-Only Appends — Right-Only Appends (Ghi thêm chỉ bên phải)

Many database systems use auto-incremented monotonically increasing values as **primary index** keys. This case opens up an opportunity for an optimization, since all the insertions are happening toward the end of the index (in the rightmost leaf), so most of the splits occur on the rightmost node on each level. Moreover, since the keys are monotonically incremented, given that the ratio of appends versus updates and deletes is low, nonleaf pages are also less fragmented than in the case of randomly ordered keys.

Nhiều hệ cơ sở dữ liệu dùng các giá trị tự tăng đơn điệu làm khóa chỉ mục chính (primary index). Trường hợp này mở ra cơ hội tối ưu, vì mọi phép chèn đều xảy ra về phía cuối của chỉ mục (ở lá cực phải), nên hầu hết các phép tách xảy ra ở nút cực phải tại mỗi mức. Hơn nữa, vì các khóa tăng đơn điệu, với điều kiện tỷ lệ giữa ghi-thêm so với cập-nhật và xóa thấp, các trang không-phải-lá cũng ít bị phân mảnh hơn so với trường hợp khóa có thứ tự ngẫu nhiên.

PostgreSQL is calling this case a fastpath. When the inserted key is strictly greater than the first key in the rightmost page, and the rightmost page has enough space to hold the newly inserted entry, the new entry is inserted into the appropriate location in the cached rightmost leaf, and the whole read path can be skipped.

PostgreSQL gọi trường hợp này là fastpath. Khi khóa được chèn lớn hơn hẳn khóa đầu tiên trong trang cực phải, và trang cực phải còn đủ không gian để chứa mục mới chèn, mục mới được chèn vào vị trí thích hợp trong lá cực phải đã được cache, và toàn bộ đường đọc có thể được bỏ qua.

SQLite has a similar concept and calls it quickbalance . When the entry is inserted on the far right end and the target node is full (i.e., it becomes the largest entry in the tree upon insertion), instead of rebalancing or splitting the node, it allocates the new rightmost node and adds its pointer to the parent (for more on implementing balancing in SQLite, see “Rebalancing” on page 70). Even though this leaves the newly created page nearly empty (instead of half empty in the case of a node split), it is very likely that the node will get filled up shortly.

SQLite có một khái niệm tương tự và gọi là quickbalance. Khi một mục được chèn ở đầu cùng bên phải và nút đích đã đầy (tức là nó trở thành mục lớn nhất trong cây khi được chèn), thay vì tái cân bằng hoặc tách nút, nó cấp phát nút cực phải mới và thêm con trỏ của nút này vào cha (để biết thêm về hiện thực cân bằng trong SQLite, xem “Rebalancing” trang 70). Mặc dù điều này khiến trang mới tạo gần như trống (thay vì trống một nửa như trong trường hợp tách nút), rất có khả năng nút sẽ sớm được lấp đầy.

Bulk Loading — Bulk Loading (Nạp hàng loạt)

If we have presorted data and want to bulk load it, or have to rebuild the tree (for example, for **defragmentation**), we can take the idea with right-only appends even further. Since the data required for tree creation is already sorted, during bulk loading we only need to append the items at the rightmost location in the tree.

Nếu ta có dữ liệu đã sắp xếp sẵn và muốn nạp hàng loạt (bulk load) nó, hoặc phải dựng lại cây (ví dụ, để chống phân mảnh), ta có thể đẩy ý tưởng ghi-thêm-chỉ-bên-phải đi xa hơn nữa. Vì dữ liệu cần để tạo cây đã được sắp xếp, trong lúc nạp hàng loạt ta chỉ cần ghi thêm các phần tử ở vị trí cực phải trong cây.

In this case, we can avoid splits and merges altogether and compose the tree from the bottom up, writing it out level by level, or writing out higher-level nodes as soon as we have enough pointers to already written lower-level nodes.

Trong trường hợp này, ta có thể tránh hoàn toàn việc tách và gộp và xây dựng cây từ dưới lên, ghi ra từng mức một, hoặc ghi ra các nút mức cao hơn ngay khi ta có đủ con trỏ tới các nút mức thấp hơn đã được ghi.

One approach for implementing bulk loading is to write presorted data on the leaf level page-wise (rather than inserting individual elements). After the leaf page is written, we propagate its first key to the parent and use a normal algorithm for building higher B-Tree levels [RAMAKRISHNAN03] . Since appended keys are given in the sorted order, all splits in this case occur on the rightmost node.

Một cách tiếp cận để hiện thực nạp hàng loạt là ghi dữ liệu đã sắp xếp sẵn ở mức lá theo từng trang (thay vì chèn từng phần tử riêng lẻ). Sau khi trang lá được ghi, ta lan truyền khóa đầu tiên của nó lên cha và dùng thuật toán thông thường để dựng các mức B-Tree cao hơn [RAMAKRISHNAN03]. Vì các khóa được ghi thêm theo thứ tự đã sắp xếp, mọi phép tách trong trường hợp này đều xảy ra ở nút cực phải.

Since B-Trees are always built starting from the bottom (leaf) level, the complete leaf level can be written out before any higher-level nodes are composed. This allows having all child pointers at hand by the time the higher levels are constructed. The main benefits of this approach are that we do not

have to perform any splits or merges on disk and, at the same time, have to keep only a minimal part of the tree (i.e., all parents of the currently filling leaf node) in memory for the time of construction.

Vì B-Tree luôn được dựng bắt đầu từ mức dưới cùng (mức lá), toàn bộ mức lá có thể được ghi ra trước khi bất kỳ nút mức cao hơn nào được tạo. Điều này cho phép có sẵn toàn bộ con trỏ con vào thời điểm các mức cao hơn được dựng. Lợi ích chính của cách tiếp cận này là ta không phải thực hiện bất kỳ phép tách hay gộp nào trên đĩa và đồng thời chỉ phải giữ một phần tối thiểu của cây (tức là tất cả các cha của nút lá đang được lấp đầy hiện thời) trong bộ nhớ trong suốt thời gian xây dựng.

Immutable B-Trees can be created in the same manner but, unlike mutable B-Trees, they require no space overhead for subsequent modifications, since all operations on a tree are final. All pages can be completely filled up, improving occupancy and resulting into better performance.

B-Tree bất biến có thể được tạo theo cùng cách thức này nhưng, không như B-Tree khả biến, chúng không đòi hỏi chi phí không gian dôi dư cho các sửa đổi về sau, vì mọi thao tác trên cây đều là cuối cùng. Mọi trang có thể được lấp đầy hoàn toàn, cải thiện độ lấp đầy và đem lại hiệu năng tốt hơn.

Compression — Compression (Nén)

Storing the raw, uncompressed data can induce significant overhead, and many databases offer ways to compress it to save space. The apparent trade-off here is between access speed and compression ratio: larger compression ratios can improve data size, allowing you to fetch more data in a single access, but might require more RAM and CPU cycles to compress and decompress it.

Lưu dữ liệu thô, chưa nén có thể gây ra chi phí dôi dư đáng kể, và nhiều cơ sở dữ liệu cung cấp các cách để nén nó nhằm tiết kiệm không gian. Sự đánh đổi rõ rệt ở đây là giữa tốc độ truy cập và tỷ lệ nén: tỷ lệ nén cao hơn có thể cải thiện kích thước dữ liệu, cho phép bạn lấy về nhiều dữ liệu hơn trong một lần truy cập, nhưng có thể đòi hỏi nhiều RAM và chu kỳ CPU hơn để nén và giải nén nó.

Compression can be done at different granularity levels. Even though compressing entire files can yield better compression ratios, it has limited application as a whole file has to be recompressed on an update, and more granular compression is usually better-suited for larger datasets. Compressing an entire **index file** is both impractical and hard to implement efficiently: to address a particular page, the whole file (or its section containing compression metadata) has to be accessed (in order to locate a compressed section), decompressed, and made available.

Nén có thể được thực hiện ở các mức độ chi tiết khác nhau. Mặc dù nén toàn bộ tệp có thể đem lại tỷ lệ nén tốt hơn, nó có ứng dụng hạn chế vì toàn bộ tệp phải được nén lại khi có cập nhật, và nén chi tiết hơn thường phù hợp hơn với các tệp dữ liệu lớn. Nén toàn bộ một tệp chỉ mục vừa không thực tế vừa khó hiện thực hiệu quả: để định địa chỉ một trang cụ thể, toàn bộ tệp (hoặc phần của nó chứa siêu dữ liệu nén) phải được truy cập (để định vị một phần đã nén), giải nén, và đưa ra sẵn dùng.

An alternative is to compress data page-wise. It fits our discussion well, since the algorithms we've been discussing so far use fixed-size pages. Pages can be compressed and uncompressed independently from one another, allowing you to couple compression with page loading and flushing. However, a compressed page in this case can occupy only a fraction of a disk block and, since transfers are usually done in units of disk blocks, it might be necessary to page in extra bytes [RAY95]. In Figure 4-10, you can see a compressed page (a) taking less space than the disk block.

When we load this page, we also page in additional bytes that belong to the other page. With pages that span multiple disk blocks, like (b) in the same image, we have to read an additional block.

Một lựa chọn thay thế là nén dữ liệu theo từng trang. Nó phù hợp với phần thảo luận của chúng ta, vì các thuật toán mà ta đã bàn từ trước tới giờ đều dùng các trang kích thước cố định. Các trang có thể được nén và giải nén độc lập với nhau, cho phép bạn gắn việc nén với việc nạp và xả (flush) trang. Tuy nhiên, một trang đã nén trong trường hợp này có thể chỉ chiếm một phần của khối đĩa và, vì các phép truyền thường được thực hiện theo đơn vị khối đĩa, có thể cần phải nạp thêm các byte dôi dư [RAY95]. Trong Hình 4-10, bạn có thể thấy một trang đã nén (a) chiếm ít không gian hơn khối đĩa. Khi ta nạp trang này, ta cũng nạp vào thêm các byte thuộc về trang khác. Với các trang trải dài qua nhiều khối đĩa, như (b) trong cùng hình, ta phải đọc thêm một khối.

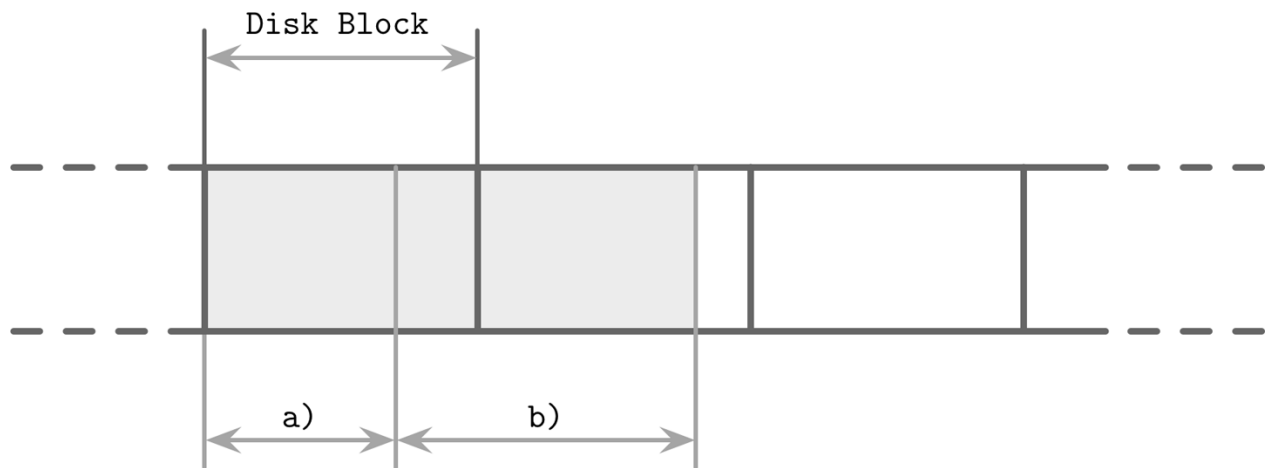


Figure 4-10. Compression and block padding

Hình 4-10. Nén và đệm khối (block padding)

Another approach is to compress data only, either row-wise (compressing entire data records) or column-wise (compressing columns individually). In this case, page management and compression are decoupled.

Một cách tiếp cận khác là chỉ nén dữ liệu, hoặc theo hàng (nén toàn bộ bản ghi dữ liệu) hoặc theo cột (nén từng cột riêng lẻ). Trong trường hợp này, việc quản lý trang và việc nén được tách rời.

Most of the open source databases reviewed while writing this book have pluggable compression methods, using available libraries such as Snappy , zLib , lz4 , and many others.

Hầu hết các cơ sở dữ liệu mã nguồn mở được khảo sát khi viết cuốn sách này đều có các phương pháp nén cắm-được (pluggable), sử dụng các thư viện sẵn có như Snappy, zLib, lz4, và nhiều thư viện khác.

As compression algorithms yield different results depending on a dataset and potential objectives (e.g., compression ratio, performance, or memory overhead), we will not go into comparison and implementation details in this book. There are many overviews available that evaluate different compression algorithms for different block sizes (for example, Squash Compression Benchmark), usually focusing on four metrics: memory overhead, compression performance, decompression performance, and compression ratio. These metrics are important to consider when picking a compression library.

Vì các thuật toán nén cho ra kết quả khác nhau tùy theo tập dữ liệu và các mục tiêu tiềm

năng (ví dụ, tỷ lệ nén, hiệu năng, hoặc chi phí bộ nhớ dôi dư), chúng ta sẽ không đi vào chi tiết so sánh và hiện thực trong cuốn sách này. Có nhiều bài tổng quan sẵn có đánh giá các thuật toán nén khác nhau cho các kích thước khối khác nhau (ví dụ, Squash Compression Benchmark), thường tập trung vào bốn chỉ số: chi phí bộ nhớ dôi dư, hiệu năng nén, hiệu năng giải nén, và tỷ lệ nén. Những chỉ số này quan trọng cần cân nhắc khi chọn một thư viện nén.

Vacuum and Maintenance — Vacuum and Maintenance (Dọn dẹp và bảo trì)

So far we've been mostly talking about user-facing operations in B-Trees. However, there are other processes that happen in parallel with queries that maintain storage integrity, reclaim space, reduce overhead, and keep pages in order. Performing these operations in the background allows us to save some time and avoid paying the price of cleanup during inserts, updates, and deletes.

Cho tới giờ chúng ta phần lớn nói về các thao tác hướng tới người dùng trong B-Tree. Tuy nhiên, còn có các quá trình khác xảy ra song song với các truy vấn nhằm duy trì tính toàn vẹn của kho lưu trữ, thu hồi không gian, giảm chi phí dôi dư, và giữ các trang gọn gàng. Thực hiện những thao tác này ở chế độ nền cho phép ta tiết kiệm thời gian và tránh phải trả giá cho việc dọn dẹp trong lúc chèn, cập nhật và xóa.

The described design of slotted pages (see “Slotted Pages” on page 52) requires maintenance to be performed on pages to keep them in good shape. For example, subsequent splits and merges in internal nodes or inserts, updates, and deletes on the leaf level can result in a page that has enough logical space but does not have enough contiguous space, since it is fragmented. Figure 4-11 shows an example of such a situation: the page still has some logical space available, but it's fragmented and is split between the two deleted (garbage) records and some remaining free space between the header/cell pointers and cells.

Thiết kế đã mô tả về trang có khe (slotted page) (xem “Slotted Pages” trang 52) đòi hỏi phải thực hiện bảo trì trên các trang để giữ chúng ở tình trạng tốt. Ví dụ, các phép tách và gộp về sau ở các nút trong, hoặc các phép chèn, cập nhật và xóa ở mức lá có thể dẫn tới một trang có đủ không gian logic nhưng không có đủ không gian liền kề, vì nó bị phân mảnh. Hình 4-11 thể hiện một ví dụ về tình huống như vậy: trang vẫn còn một ít không gian logic, nhưng nó bị phân mảnh và bị chia tách giữa hai bản ghi đã xóa (rác) và một ít không gian trống còn lại giữa header/con trỏ ô và các ô.

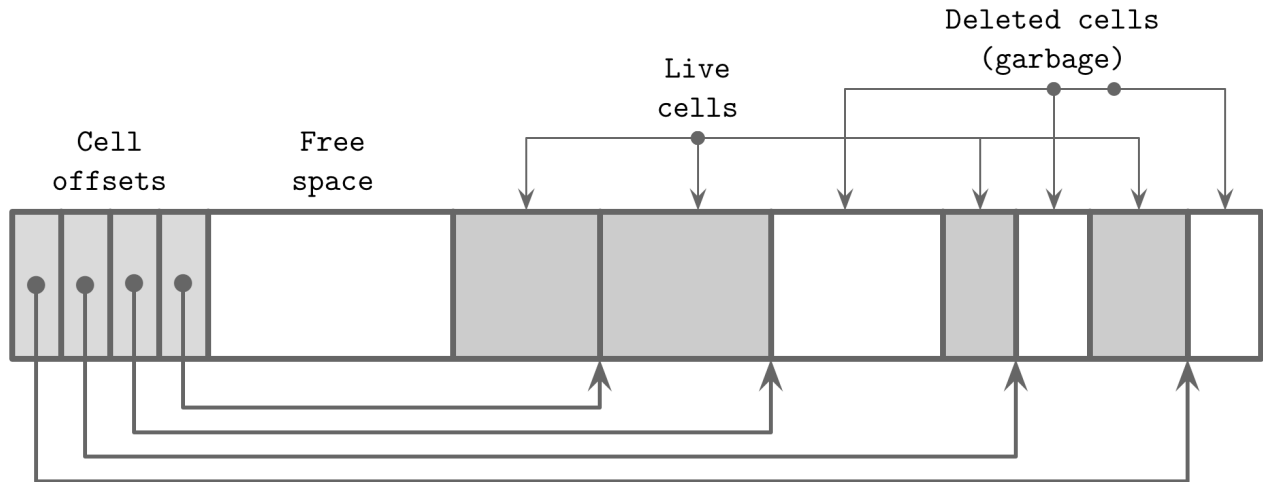


Figure 4-11. An example of a fragmented page B-Trees are navigated from the root level. Data records that can be reached by following pointers down from the root node are live (addressable). Nonaddressable data records are said to be garbage : these records are not referenced anywhere and cannot be read or interpreted, so their contents are as good as nullified.

Hình 4-11. Một ví dụ về trang bị phân mảnh. B-Tree được điều hướng từ mức gốc. Các bản ghi dữ liệu có thể tới được bằng cách đi theo con trỏ xuống từ nút gốc là bản ghi sống (live, định địa chỉ được). Các bản ghi dữ liệu không định địa chỉ được thì được gọi là rác (garbage): các bản ghi này không được tham chiếu ở đâu cả và không thể được đọc hay diễn giải, nên nội dung của chúng coi như đã bị vô hiệu.

You can see this distinction in Figure 4-11 : cells that still have pointers to them are addressable, unlike the removed or overwritten ones. Zero-filling of garbage areas is often skipped for performance reasons, as eventually these areas are overwritten by the new data anyway.

Bạn có thể thấy sự phân biệt này trong Hình 4-11: các ô vẫn còn con trỏ tới chúng là định địa chỉ được, không như các ô đã bị gỡ bỏ hoặc ghi đè. Việc lấp-không (zero-filling) các vùng rác thường được bỏ qua vì lý do hiệu năng, vì rốt cuộc các vùng này dù sao cũng sẽ bị ghi đè bởi dữ liệu mới.

Fragmentation Caused by Updates and Deletes — Fragmentation Caused by Updates and Deletes (Phân mảnh do cập nhật và xóa)

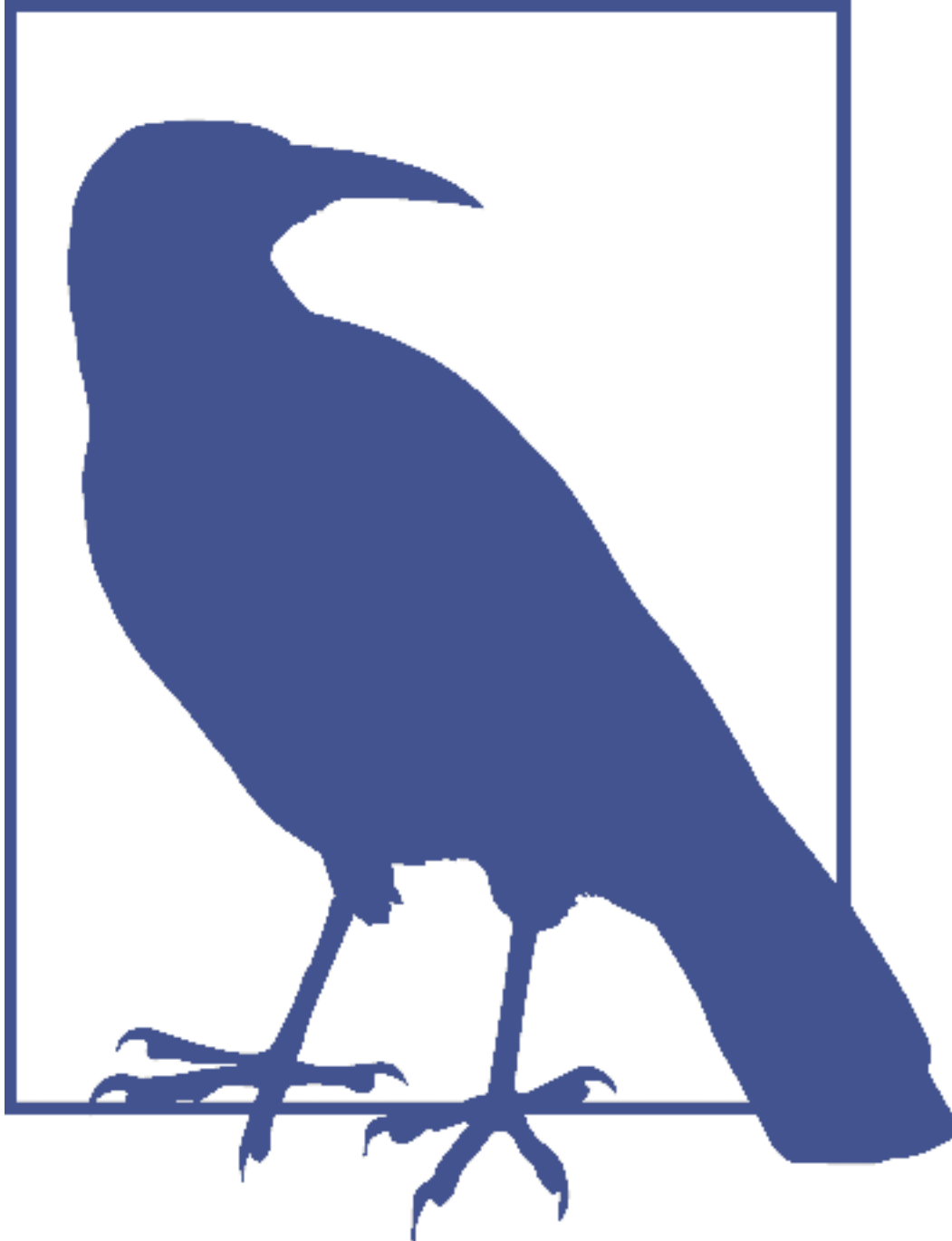
Let's consider under which circumstances pages get into the state where they have nonaddressable data and have to be compacted. On the leaf level, deletes only remove cell offsets from the header, leaving the cell itself intact. After this is done, the cell is not addressable anymore, its contents will not appear in the query results, and nullifying it or moving neighboring cells is not necessary.

Hãy xét xem trong những hoàn cảnh nào các trang rơi vào trạng thái có dữ liệu không định địa chỉ được và phải được nén-gộp lại. Ở mức lá, việc xóa chỉ gỡ bỏ các offset của ô khỏi header, để nguyên bản thân ô. Sau khi làm xong điều này, ô không còn định địa chỉ được nữa, nội dung của nó sẽ không xuất hiện trong kết quả truy vấn, và không cần phải vô hiệu nó hay dời các ô lân cận.

When the page is split, only offsets are trimmed and, since the rest of the page is not addressable, cells whose offsets were truncated are not reachable, so they will be overwritten whenever the new

data arrives, or garbage-collected when the **vacuum** process kicks in.

Khi trang bị tách, chỉ các offset bị cắt bớt và, vì phần còn lại của trang không định địa chỉ được, các ô có offset bị cắt bỏ sẽ không tới được, nên chúng sẽ bị ghi đè mỗi khi có dữ liệu mới tới, hoặc bị thu gom rác khi quá trình dọn dẹp (vacuum) khởi động.



Some databases rely on garbage collection, and leave removed and updated cells in place for multiversion **concurrency control** (see “Multiversion Concurrency Control” on page 99). Cells remain accessible for the concurrently executing transactions until the update is complete, and can be collected as soon as no other thread accesses them. Some databases maintain structures that track ghost records, which are collected as soon as all transactions that may have seen them

complete [WEIKUM01] .

Một số cơ sở dữ liệu dựa vào thu gom rác, và để nguyên tại chỗ các ô đã gỡ bỏ và đã cập nhật để phục vụ điều khiển tương tranh đa phiên bản (MVCC) (xem “Multiversion Concurrency Control” trang 99). Các ô vẫn truy cập được đối với các giao dịch đang chạy đồng thời cho tới khi việc cập nhật hoàn tất, và có thể được thu gom ngay khi không còn luồng nào truy cập chúng. Một số cơ sở dữ liệu duy trì các cấu trúc theo dõi bản ghi ma (ghost record), vốn được thu gom ngay khi tất cả các giao dịch có thể đã thấy chúng hoàn tất [WEIKUM01].

Since deletes only discard cell offsets and do not relocate remaining cells or physically remove the target cells to occupy the freed space, freed bytes might end up scattered across the page. In this case, we say that the page is fragmented and requires defragmentation.

Vì việc xóa chỉ loại bỏ các offset của ô và không dời chỗ các ô còn lại hay loại bỏ vật lý các ô đích để chiếm dụng không gian được giải phóng, các byte được giải phóng có thể rất cuộc rải rác khắp trang. Trong trường hợp này, ta nói trang bị phân mảnh và cần chống phân mảnh (defragmentation).

To make a write, we often need a contiguous block of free bytes where the cell fits. To put the freed fragments back together and fix this situation, we have to rewrite the page.

Để thực hiện một phép ghi, ta thường cần một khối byte trống liền kề đủ chỗ cho ô. Để ráp lại các mảnh đã giải phóng và khắc phục tình huống này, ta phải ghi lại trang.

Insert operations leave tuples in their insertion order. This does not have as significant an impact, but having naturally sorted tuples can help with cache prefetch during **sequential** reads.

Các thao tác chèn để lại các bộ (tuple) theo thứ tự chèn của chúng. Điều này không gây tác động đáng kể lắm, nhưng việc có các bộ được sắp xếp tự nhiên có thể giúp ích cho việc nạp trước (prefetch) cache trong các phép đọc tuần tự.

Updates are mostly applicable to the leaf level: internal page keys are used for guided navigation and only define subtree boundaries. Additionally, updates are performed on a per-key basis, and generally do not result in structural changes in the tree, apart from the creation of overflow pages. On the leaf level, however, update operations do not change cell order and attempt to avoid page rewrite. This means that multiple versions of the cell, only one of which is addressable, may end up being stored.

Các phép cập nhật chủ yếu áp dụng ở mức lá: khóa của trang trong được dùng cho điều hướng có hướng dẫn và chỉ xác định biên của các cây con. Ngoài ra, cập nhật được thực hiện trên cơ sở từng-khóa, và nói chung không dẫn tới thay đổi cấu trúc trong cây, ngoại trừ việc tạo trang tràn. Tuy nhiên, ở mức lá, các thao tác cập nhật không thay đổi thứ tự ô và cố gắng tránh việc ghi lại trang. Điều này có nghĩa là nhiều phiên bản của ô, trong đó chỉ một phiên bản là định địa chỉ được, có thể rất cuộc bị lưu lại.

Page Defragmentation — Page Defragmentation (Chống phân mảnh trang)

The process that takes care of space reclamation and page rewrites is called **compaction** , vacuum , or just maintenance . Page rewrites can be done synchronously on write if the page does not have enough free physical space (to avoid creating unnecessary overflow pages), but compaction is mostly referred to as a distinct, **asynchronous** process of walking through pages, performing garbage collection, and rewriting their contents.

Quá trình lo việc thu hồi không gian và ghi lại trang được gọi là nén-gộp (compaction), dọn dẹp (vacuum), hoặc đơn giản là bảo trì. Việc ghi lại trang có thể được thực hiện đồng bộ khi ghi nếu trang không có đủ không gian vật lý trống (để tránh tạo các trang tràn không cần thiết), nhưng nén-gộp chủ yếu được hiểu là một quá trình riêng biệt, bất đồng bộ gồm việc đi qua các trang, thực hiện thu gom rác, và ghi lại nội dung của chúng.

This process reclaims the space occupied by dead cells, and rewrites cells in their logical order. When pages are rewritten, they may also get relocated to new positions in the file. Unused in-memory pages become available and are returned to the **page cache**. IDs of the newly available on-disk pages are added to the free page list (sometimes called a freelist 3). This information has to be persisted to survive node crashes and restarts, and to make sure free space is not lost or leaked.

Quá trình này thu hồi không gian bị chiếm bởi các ô chết, và ghi lại các ô theo thứ tự logic của chúng. Khi các trang được ghi lại, chúng cũng có thể được dời tới các vị trí mới trong tệp. Các trang không dùng trong bộ nhớ trở nên sẵn dùng và được trả về cho page cache. ID của các trang trên đĩa vừa trở nên sẵn dùng được thêm vào danh sách trang trống (đôi khi gọi là freelist3). Thông tin này phải được lưu bền để sống sót qua các sự cố sập nút và khởi động lại, và để đảm bảo không gian trống không bị mất hay rò rỉ.

Summary — Summary (Tóm tắt)

In this chapter, we discussed the concepts specific to on-disk B-Tree implementations, such as:

Trong chương này, chúng ta đã bàn về các khái niệm đặc thù của các hiện thực B-Tree trên đĩa, chẳng hạn như:

Page header What information is usually stored there.

Page header: Thông tin gì thường được lưu ở đó.

Rightmost pointers These are not paired with separator keys, and how to handle them.

Con trở cực phải: Chúng không ghép cặp với khóa phân tách, và cách xử lý chúng.

High keys Determine the maximum allowed key that can be stored in the node.

Khóa biên: Xác định khóa lớn nhất được phép lưu trong nút.

Overflow pages Allow you to store oversize and variable-size records using fixed-size pages.

Trang tràn: Cho phép bạn lưu các bản ghi quá khổ và kích thước thay đổi bằng các trang kích thước cố định.

3 For example, SQLite maintains a list of pages that are not used by the database, where trunk pages are held in a linked list and hold addresses of freed pages.

3 Ví dụ, SQLite duy trì một danh sách các trang không được cơ sở dữ liệu sử dụng, trong đó các trang gốc (trunk page) được giữ trong một danh sách liên kết và chứa địa chỉ của các trang đã được giải phóng.

After that, we went through some details related to root-to-leaf traversals:

Sau đó, chúng ta đi qua một số chi tiết liên quan tới các phép duyệt từ gốc tới lá:

- How to perform binary search with indirection pointers
- How to keep track of tree hierarchies using parent pointers or breadcrumbs
 - Cách thực hiện tìm kiếm nhị phân với con trở gián tiếp

- Cách theo dõi các phân cấp của cây bằng con trỏ cha hoặc dấu vết đường đi

Lastly, we went through some optimization and maintenance techniques:

Cuối cùng, chúng ta đi qua một số kỹ thuật tối ưu và bảo trì:

Rebalancing Moves elements between neighboring nodes to reduce a number of splits and merges.

Tái cân bằng: Chuyển các phần tử giữa các nút lân cận để giảm số phép tách và gộp.

Right-only appends Appends the new rightmost cell instead of splitting it under the assumption that it will quickly fill up.

Ghi thêm chỉ bên phải: Ghi thêm ô cực phải mới thay vì tách nó, dựa trên giả định rằng nó sẽ nhanh chóng được lấp đầy.

Bulk loading A technique for efficiently building B-Trees from scratch from sorted data.

Nạp hàng loạt: Một kỹ thuật để xây dựng B-Tree từ đầu một cách hiệu quả từ dữ liệu đã sắp xếp.

Garbage collection A process that rewrites pages, puts cells in key order, and reclaims space occupied by unaddressable cells.

Thu gom rác: Một quá trình ghi lại các trang, đặt các ô theo thứ tự khóa, và thu hồi không gian bị chiếm bởi các ô không định địa chỉ được.

These concepts should bridge the gap between the basic B-Tree algorithm and a real-world implementation, and help you better understand how B-Tree-based storage systems work.

Những khái niệm này sẽ giúp bắc cầu khoảng cách giữa thuật toán B-Tree cơ bản và một hiện thực trong thực tế, và giúp bạn hiểu rõ hơn cách các hệ lưu trữ dựa trên B-Tree hoạt động.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Disk-based B-Trees Graefe, Goetz. 2011. "Modern B-Tree Techniques." Foundations and Trends in Databases 3, no. 4 (April): 203-402. <https://doi.org/10.1561/19000000028> .

Healey, Christopher G. 2016. Disk-Based Algorithms for Big Data (1st Ed.) . Boca Raton: CRC Press.

CHAPTER 5 Transaction Processing and Recovery — CHƯƠNG 5 Xử lý giao dịch và Phục hồi

In this book, we've taken a bottom-up approach to database system concepts: we first learned about storage structures. Now, we're ready to move to the higher-level components responsible for buffer management, **lock** management, and **recovery**, which are the prerequisites for understanding database transactions.

Trong cuốn sách này, chúng ta đã tiếp cận các khái niệm hệ cơ sở dữ liệu theo hướng từ dưới lên: trước tiên ta tìm hiểu về các cấu trúc lưu trữ. Bây giờ, chúng ta đã sẵn sàng chuyển sang các thành phần ở mức cao hơn, chịu trách nhiệm quản lý vùng đệm, quản lý khóa, và phục hồi — đó là những điều kiện tiên quyết để hiểu các giao dịch trong cơ sở dữ liệu.

A transaction is an indivisible logical unit of work in a database management system, allowing you to represent multiple operations as a single step. Operations executed by transactions include reading and writing database records. A database transaction has to preserve **atomicity**, consistency, **isolation**, and **durability**. These properties are commonly referred as **ACID** [HAERDER83] :

Một giao dịch là một đơn vị công việc logic không thể chia tách trong hệ quản trị cơ sở dữ liệu (DBMS), cho phép bạn biểu diễn nhiều thao tác như một bước duy nhất. Các thao tác do giao dịch thực thi bao gồm đọc và ghi các bản ghi cơ sở dữ liệu. Một giao dịch cơ sở dữ liệu phải bảo toàn tính nguyên tử (atomicity), tính nhất quán (consistency), tính cô lập (isolation), và tính bền vững (durability). Các tính chất này thường được gọi chung là ACID [HAERDER83]:

Atomicity Transaction steps are indivisible , which means that either all the steps associated with the transaction execute successfully or none of them do. In other words, transactions should not be applied partially. Each transaction can either commit (make all changes from write operations executed during the transaction visible), or abort (roll back all transaction side effects that haven't yet been made visible). Commit is a final operation. After an abort, the transaction can be retried.

Tính nguyên tử Các bước của giao dịch là không thể chia tách, nghĩa là hoặc tất cả các bước gắn với giao dịch đều thực thi thành công, hoặc không bước nào thực thi cả. Nói cách khác, các giao dịch không được áp dụng một phần. Mỗi giao dịch có thể hoặc commit (làm cho mọi thay đổi từ các thao tác ghi thực thi trong giao dịch trở nên hữu hình), hoặc abort (cuộn lại tất cả các tác dụng phụ của giao dịch chưa được làm hữu hình). Commit là một thao tác cuối cùng. Sau khi abort, giao dịch có thể được thử lại.

Consistency Consistency is an application-specific guarantee; a transaction should only bring the

database from one valid state to another valid state, maintaining all database invariants (such as constraints, referential integrity, and others). Consistency is the most weakly defined property, possibly because it is the only property that is controlled by the user and not only by the database itself.

Tính nhất quán Tính nhất quán là một bảo đảm đặc thù cho từng ứng dụng; một giao dịch chỉ nên đưa cơ sở dữ liệu từ một trạng thái hợp lệ sang một trạng thái hợp lệ khác, duy trì tất cả các bất biến của cơ sở dữ liệu (như các ràng buộc, tính toàn vẹn tham chiếu, v.v.). Tính nhất quán là tính chất được định nghĩa lỏng lẻo nhất, có lẽ vì đây là tính chất duy nhất do người dùng kiểm soát chứ không chỉ do bản thân cơ sở dữ liệu kiểm soát.

Isolation Multiple concurrently executing transactions should be able to run without interference, as if there were no other transactions executing at the same time.

Tính cô lập Nhiều giao dịch thực thi đồng thời phải có khả năng chạy mà không gây nhiễu lẫn nhau, như thể không có giao dịch nào khác đang thực thi cùng lúc.

79 Isolation defines when the changes to the database state may become visible, and what changes may become visible to the concurrent transactions. Many databases use isolation levels that are weaker than the given definition of isolation for performance reasons. Depending on the methods and approaches used for **concurrency control**, changes made by a transaction may or may not be visible to other concurrent transactions (see “Isolation Levels” on **page 96**).

Tính cô lập định nghĩa khi nào các thay đổi đối với trạng thái cơ sở dữ liệu có thể trở nên hữu hình, và những thay đổi nào có thể trở nên hữu hình đối với các giao dịch đồng thời. Nhiều cơ sở dữ liệu sử dụng các mức cô lập (isolation level) yếu hơn định nghĩa nói trên về tính cô lập vì lý do hiệu năng. Tùy vào các phương pháp và cách tiếp cận được dùng cho điều khiển tương tranh, các thay đổi do một giao dịch thực hiện có thể hữu hình hoặc không đối với các giao dịch đồng thời khác (xem “Isolation Levels” ở trang 96).

Durability Once a transaction has been committed, all database state modifications have to be persisted on disk and be able to survive power outages, system failures, and crashes.

Tính bền vững Một khi giao dịch đã được commit, mọi sửa đổi trạng thái cơ sở dữ liệu phải được lưu bền trên đĩa và phải có khả năng tồn tại qua mất điện, lỗi hệ thống, và sự cố sập (crash).

Implementing transactions in a database system, in addition to a storage structure that organizes and persists data on disk, requires several components to work together. On the **node** locally, the transaction manager coordinates, schedules, and tracks transactions and their individual steps.

Việc hiện thực giao dịch trong một hệ cơ sở dữ liệu, ngoài một cấu trúc lưu trữ tổ chức và lưu bền dữ liệu trên đĩa, còn đòi hỏi nhiều thành phần phải phối hợp với nhau. Trên cục bộ từng nút, bộ quản lý giao dịch (transaction manager) điều phối, lập lịch, và theo dõi các giao dịch cùng từng bước riêng lẻ của chúng.

The lock manager guards access to these resources and prevents concurrent accesses that would violate data integrity. Whenever a lock is requested, the lock manager checks if it is already held by any other transaction in shared or exclusive mode, and grants access to it if the requested access level results in no contradiction. Since exclusive locks can be held by at most one transaction at any given moment, other transactions requesting them have to wait until locks are released, or abort and retry later. As soon as the lock is released or whenever the transaction terminates, the lock manager notifies one of the pending transactions, letting it acquire the lock and continue.

Bộ quản lý khóa (lock manager) bảo vệ quyền truy cập vào các tài nguyên này và ngăn các truy cập đồng thời có thể vi phạm tính toàn vẹn dữ liệu. Mỗi khi một khóa (lock) được yêu

cầu, bộ quản lý khóa kiểm tra xem nó đã được giữ bởi giao dịch nào khác ở chế độ chia sẻ hay độc quyền chưa, và cấp quyền truy cập nếu mức truy cập được yêu cầu không gây mâu thuẫn. Vì khóa độc quyền tại bất kỳ thời điểm nào cũng chỉ có thể được giữ bởi nhiều nhất một giao dịch, các giao dịch khác yêu cầu chúng phải chờ cho tới khi khóa được giải phóng, hoặc abort rồi thử lại sau. Ngay khi khóa được giải phóng hoặc bất cứ khi nào giao dịch kết thúc, bộ quản lý khóa thông báo cho một trong các giao dịch đang chờ, cho phép nó giành khóa và tiếp tục.

The **page cache** serves as an intermediary between persistent storage (disk) and the rest of the **storage engine**. It stages state changes in main memory and serves as a cache for the pages that haven't been synchronized with persistent storage. All changes to a database state are first applied to the cached pages.

Page cache đóng vai trò trung gian giữa lưu trữ bền (đĩa) và phần còn lại của bộ máy lưu trữ (storage engine). Nó tổ chức tạm thời các thay đổi trạng thái trong bộ nhớ chính và đóng vai trò cache cho các trang chưa được đồng bộ với lưu trữ bền. Mọi thay đổi đối với trạng thái cơ sở dữ liệu trước hết đều được áp dụng lên các trang đã được cache.

The log manager holds a history of operations (log entries) applied to cached pages but not yet synchronized with persistent storage to guarantee they won't be lost in case of a crash. In other words, the log is used to reapply these operations and reconstruct the cached state during startup. Log entries can also be used to **undo** changes done by the aborted transactions.

Bộ quản lý nhật ký (log manager) lưu giữ lịch sử các thao tác (các bản ghi nhật ký) đã áp dụng lên các trang được cache nhưng chưa được đồng bộ với lưu trữ bền, nhằm bảo đảm chúng không bị mất trong trường hợp sự cố sập. Nói cách khác, nhật ký được dùng để áp dụng lại các thao tác này và tái dựng trạng thái đã cache trong lúc khởi động. Các bản ghi nhật ký cũng có thể được dùng để undo các thay đổi do các giao dịch bị abort thực hiện.

Distributed (multipartition) transactions require additional coordination and remote execution. We discuss **distributed transaction** protocols in Chapter 13 .

Các giao dịch phân tán (đa phân vùng) đòi hỏi sự điều phối bổ sung và thực thi từ xa. Chúng ta thảo luận các giao thức giao dịch phân tán ở Chương 13.

Buffer Management — Quản lý vùng đệm

Most databases are built using a two-level memory hierarchy: slower persistent storage (disk) and faster main memory (RAM). To reduce the number of accesses to persistent storage, pages are cached in memory. When the page is requested again by the storage layer, its cached copy is returned.

Hầu hết các cơ sở dữ liệu được xây dựng dựa trên một phân cấp bộ nhớ hai mức: lưu trữ bền chậm hơn (đĩa) và bộ nhớ chính nhanh hơn (RAM). Để giảm số lần truy cập vào lưu trữ bền, các trang được cache trong bộ nhớ. Khi một trang được lớp lưu trữ yêu cầu lại, bản sao đã cache của nó được trả về.

Cached pages available in memory can be reused under the assumption that no other **process** has modified the data on disk. This approach is sometimes referenced as virtual disk [BAYER72] . A virtual disk **read** accesses physical storage only if no copy of the page is already available in memory. A more common name for the same concept is page cache or **buffer pool** . The page cache is responsible for caching pages read from disk in memory. In case of a database system crash or unordered shutdown, cached contents are lost.

Các trang được cache sẵn trong bộ nhớ có thể được tái sử dụng dựa trên giả định rằng không có tiến trình nào khác đã sửa đổi dữ liệu trên đĩa. Cách tiếp cận này đôi khi được gọi là đĩa ảo (virtual disk) [BAYER72]. Một thao tác đọc đĩa ảo chỉ truy cập lưu trữ vật lý nếu chưa có bản sao nào của trang sẵn có trong bộ nhớ. Một tên gọi phổ biến hơn cho cùng khái niệm này là page cache hay vùng đệm trang (buffer pool). Page cache chịu trách nhiệm cache trong bộ nhớ các trang được đọc từ đĩa. Trong trường hợp hệ cơ sở dữ liệu sập hoặc tắt không trật tự, nội dung đã cache sẽ bị mất.

Since the **term** page cache better reflects the purpose of this structure, this book defaults to this name. The term buffer pool sounds like its primary purpose is to pool and reuse empty buffers, without sharing their contents, which can be a useful part of a page cache or even as a separate component, but does not reflect the entire purpose as precisely.

Vì thuật ngữ page cache phản ánh mục đích của cấu trúc này tốt hơn, cuốn sách này mặc định dùng tên gọi đó. Thuật ngữ buffer pool nghe như thể mục đích chính của nó là gom và tái sử dụng các vùng đệm rỗng, mà không chia sẻ nội dung của chúng — điều này có thể là một phần hữu ích của một page cache hoặc thậm chí là một thành phần riêng, nhưng không phản ánh trọn vẹn mục đích một cách chính xác.

The problem of caching pages is not limited in scope to databases. Operating systems have the concept of a page cache, too. Operating systems utilize unused memory segments to transparently cache disk contents to improve performance of I/O syscalls.

Vấn đề cache các trang không chỉ giới hạn trong phạm vi cơ sở dữ liệu. Các hệ điều hành cũng có khái niệm page cache. Các hệ điều hành tận dụng những đoạn bộ nhớ chưa dùng để cache nội dung đĩa một cách trong suốt nhằm cải thiện hiệu năng của các lời gọi hệ thống I/O.

Uncached pages are said to be paged in when they're loaded from disk. If any changes are made to the cached page, it is said to be dirty, until these changes are flushed back on disk.

Các trang chưa được cache được gọi là được nạp vào (paged in) khi chúng được tải từ đĩa. Nếu có bất kỳ thay đổi nào được thực hiện lên trang đã cache, nó được gọi là bẩn (dirty), cho tới khi các thay đổi này được flush trở lại đĩa.

Since the memory region where cached pages are held is usually substantially smaller than an entire dataset, the page cache eventually fills up and, in order to page in a new page, one of the cached pages has to be evicted.

Vì vùng bộ nhớ giữ các trang đã cache thường nhỏ hơn đáng kể so với toàn bộ tập dữ liệu, page cache rất cuộc sẽ đầy và, để nạp vào một trang mới, một trong các trang đã cache phải bị tống ra (evicted).

In Figure 5-1, you can see the relation between the logical representation of **B-Tree** pages, their cached versions, and the pages on disk. The page cache loads pages into free slots out of order, so there's no direct mapping between how pages are ordered on disk and in memory.

Trong Hình 5-1, bạn có thể thấy mối quan hệ giữa biểu diễn logic của các trang B-Tree, các phiên bản đã cache của chúng, và các trang trên đĩa. Page cache nạp các trang vào các khe trống mà không theo thứ tự, nên không có ánh xạ trực tiếp giữa cách các trang được sắp xếp trên đĩa và trong bộ nhớ.

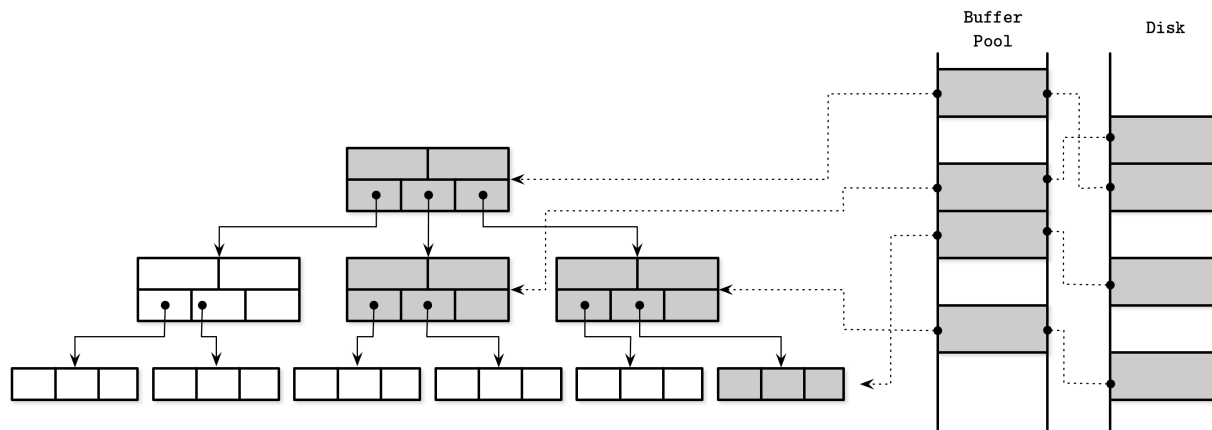


Figure 5-1. Page cache

Hình 5-1. Page cache

The primary functions of a page cache can be summarized as:

Các chức năng chính của page cache có thể tóm tắt như sau:

- It keeps cached page contents in memory.
 - It allows modifications to on-disk pages to be buffered together and performed against their cached versions.
 - When a requested page isn't present in memory and there's enough space available for it, it is paged in by the page cache, and its cached version is returned.
 - If an already cached page is requested, its cached version is returned.
 - If there's not enough space available for the new page, some other page is evicted and its contents are flushed to disk.
- Nó giữ nội dung trang đã cache trong bộ nhớ.
 - Nó cho phép các sửa đổi đối với các trang trên đĩa được đệm chung lại với nhau và thực hiện trên các phiên bản đã cache của chúng.
 - Khi một trang được yêu cầu không hiện diện trong bộ nhớ và có đủ không gian cho nó, nó được page cache nạp vào, và phiên bản đã cache của nó được trả về.
 - Nếu một trang đã được cache được yêu cầu, phiên bản đã cache của nó được trả về.
 - Nếu không có đủ không gian cho trang mới, một trang khác nào đó bị tống ra và nội dung của nó được flush xuống đĩa.

Bypassing the Kernel Page Cache — Đi vòng qua Page Cache của nhân

Many database systems open files using `O_DIRECT` flag . This flag allows I/O system calls to bypass the kernel page cache, access the disk directly, and use database-specific buffer management. This is sometimes frowned upon by the operating systems folks.

Nhiều hệ cơ sở dữ liệu mở các tệp bằng cờ `O_DIRECT`. Cờ này cho phép các lời gọi hệ thống I/O đi vòng qua page cache của nhân (kernel), truy cập đĩa trực tiếp, và sử dụng cơ chế quản lý vùng đệm đặc thù cho cơ sở dữ liệu. Điều này đôi khi bị giới làm hệ điều hành phản đối.

Linus Torvalds has criticized usage of `O_DIRECT` since it's not **asynchronous** and has no readahead

or other means for instructing the kernel about access patterns. However, until operating systems start offering better mechanisms, O_DIRECT is still going to be useful.

Linus Torvalds đã phê phán việc dùng O_DIRECT vì nó không bất đồng bộ và không có cơ chế đọc trước (readahead) hay các phương tiện khác để chỉ dẫn cho nhân về các mẫu truy cập. Tuy nhiên, cho tới khi các hệ điều hành bắt đầu cung cấp những cơ chế tốt hơn, O_DIRECT vẫn còn hữu dụng.

We can gain some control over how the kernel evicts pages from its cache is by using `fsync`, but this only allows us to ask the kernel to consider our opinion and does not guarantee it will actually happen. To avoid syscalls when performing I/O, we can use memory mapping, but then we lose control over caching.

Chúng ta có thể giành được phần nào quyền kiểm soát cách nhân tổng các trang khỏi cache của nó bằng cách dùng `fsync`, nhưng cách này chỉ cho phép ta yêu cầu nhân cân nhắc ý kiến của ta chứ không bảo đảm điều đó sẽ thực sự xảy ra. Để tránh các lời gọi hệ thống khi thực hiện I/O, ta có thể dùng ánh xạ bộ nhớ (memory mapping), nhưng khi đó ta mất quyền kiểm soát việc cache.

Caching Semantics — Ngữ nghĩa của việc cache

All changes made to buffers are kept in memory until they are eventually written back to disk. As no other process is allowed to make changes to the backing file, this synchronization is a one-way process: from memory to disk, and not vice versa. The page cache allows the database to have more control over memory management and disk accesses. You can think of it as an application-specific equivalent of the kernel page cache: it accesses the **block** device directly, implements similar functionality, and serves a similar purpose. It abstracts disk accesses and decouples logical write operations from the physical ones.

Mọi thay đổi được thực hiện lên các vùng đệm đều được giữ trong bộ nhớ cho tới khi rốt cuộc chúng được ghi trở lại đĩa. Vì không tiến trình nào khác được phép thực hiện thay đổi lên tệp nền, sự đồng bộ này là một quá trình một chiều: từ bộ nhớ ra đĩa, chứ không ngược lại. Page cache cho phép cơ sở dữ liệu có nhiều quyền kiểm soát hơn đối với việc quản lý bộ nhớ và các truy cập đĩa. Bạn có thể coi nó như một bản tương đương đặc thù ứng dụng của page cache của nhân: nó truy cập thiết bị khối trực tiếp, hiện thực chức năng tương tự, và phục vụ một mục đích tương tự. Nó trừu tượng hóa các truy cập đĩa và tách rời các thao tác ghi logic khỏi các thao tác ghi vật lý.

Caching pages helps to keep the tree partially in memory without making additional changes to the algorithm and materializing objects in memory. All we have to do is replace disk accesses by the calls to the page cache.

Việc cache các trang giúp giữ một phần cây trong bộ nhớ mà không cần thực hiện thêm thay đổi nào đối với thuật toán cũng như không cần hiện thực hóa các đối tượng trong bộ nhớ. Tất cả những gì ta phải làm là thay các truy cập đĩa bằng các lời gọi đến page cache.

When the storage engine accesses (in other words, requests) the page, we first check if its contents are already cached, in which case the cached page contents are returned. If the page contents are not yet cached, the cache translates the logical page address or page number to its physical address, loads its contents in memory, and returns its cached version to the storage engine. Once returned, the buffer with cached page contents is said to be referenced, and the storage engine has to hand it back to the page cache or dereference it once it's done. The page cache can be instructed to avoid evicting pages by pinning them.

Khi bộ máy lưu trữ truy cập (nói cách khác, yêu cầu) trang, trước hết ta kiểm tra xem nội dung của nó đã được cache chưa, và nếu rồi thì nội dung trang đã cache được trả về. Nếu nội dung trang chưa được cache, cache dịch địa chỉ trang logic hoặc số trang sang địa chỉ vật lý của nó, tải nội dung của nó vào bộ nhớ, và trả về phiên bản đã cache của nó cho bộ máy lưu trữ. Sau khi được trả về, vùng đệm chứa nội dung trang đã cache được gọi là đang được tham chiếu (referenced), và bộ máy lưu trữ phải trả nó lại cho page cache hoặc bỏ tham chiếu (dereference) một khi xong việc. Page cache có thể được chỉ dẫn để tránh tống ra các trang bằng cách ghim (pinning) chúng.

If the page is modified (for example, a **cell** was appended to it), it is marked as dirty. A dirty flag set on the page indicates that its contents are out of sync with the disk and have to be flushed for durability.

Nếu trang bị sửa đổi (ví dụ, một ô (cell) được nối thêm vào nó), nó được đánh dấu là bẩn. Một cờ bẩn được đặt trên trang cho biết nội dung của nó không còn đồng bộ với đĩa và phải được flush để đảm bảo tính bền vững.

Cache Eviction — Tổng trang khỏi Cache

Keeping caches populated is good: we can serve more reads without going to persistent storage, and more same-page writes can be buffered together. However, the page cache has a limited capacity and, sooner or later, to serve the new contents, old pages have to be evicted. If page contents are in sync with the disk (i.e., were already flushed or were never modified) and the page is not pinned or referenced, it can be evicted right away. Dirty pages have to be flushed before they can be evicted. Referenced pages should not be evicted while some other thread is using them.

Giữ cho các cache được lấp đầy là điều tốt: ta có thể phục vụ nhiều lượt đọc hơn mà không cần đến lưu trữ bền, và nhiều lượt ghi cùng-trang có thể được đệm chung lại với nhau. Tuy nhiên, page cache có dung lượng giới hạn và, sớm hay muộn, để phục vụ nội dung mới, các trang cũ phải bị tống ra. Nếu nội dung trang đồng bộ với đĩa (tức đã được flush hoặc chưa bao giờ bị sửa đổi) và trang không bị ghim hay tham chiếu, nó có thể bị tống ra ngay lập tức. Các trang bẩn phải được flush trước khi có thể bị tống ra. Các trang đang được tham chiếu không nên bị tống ra trong khi có luồng nào khác đang dùng chúng.

Since triggering a flush on every eviction might be bad for performance, some databases use a separate background process that cycles through the dirty pages that are likely to be evicted, updating their disk versions. For example, PostgreSQL has a background flush writer that does just that.

Vì việc kích hoạt một lượt flush trên mỗi lần tống ra có thể gây hại cho hiệu năng, một số cơ sở dữ liệu dùng một tiến trình nền riêng để duyệt qua các trang bẩn có khả năng bị tống ra, cập nhật các phiên bản trên đĩa của chúng. Ví dụ, PostgreSQL có một bộ ghi flush nền làm đúng việc đó.

Another important property to keep in mind is durability : if the database has crashed, all data that was not flushed is lost. To make sure that all changes are persisted, flushes are coordinated by the checkpoint process. The checkpoint process controls the write-ahead log (**WAL**) and page cache, and ensures that they work in lockstep. Only log records associated with operations applied to cached pages that were flushed can be discarded from the WAL. Dirty pages cannot be evicted until this process completes.

Một tính chất quan trọng khác cần ghi nhớ là tính bền vững: nếu cơ sở dữ liệu đã sập, mọi dữ liệu chưa được flush sẽ bị mất. Để bảo đảm rằng mọi thay đổi đều được lưu bền, các

lượt flush được điều phối bởi tiến trình điểm kiểm tra (checkpoint). Tiến trình điểm kiểm tra điều khiển nhật ký ghi-trước (WAL) và page cache, và bảo đảm chúng hoạt động ăn khớp nhịp nhàng. Chỉ những bản ghi nhật ký gắn với các thao tác đã áp dụng lên những trang đã cache và đã được flush mới có thể bị loại bỏ khỏi WAL. Các trang bản không thể bị tổng ra cho tới khi quá trình này hoàn tất.

This means there is always a trade-off between several objectives:

Điều này nghĩa là luôn có một sự đánh đổi giữa nhiều mục tiêu:

- Postpone flushes to reduce the number of disk accesses
- Preemptively flush pages to allow quick eviction
- Pick pages for eviction and flush in the optimal order
- Keep cache size within its memory bounds
- Avoid losing the data as it is not persisted to the primary storage
 - Trì hoãn các lượt flush để giảm số lần truy cập đĩa
 - Flush các trang trước hạn để cho phép tổng ra nhanh chóng
 - Chọn các trang để tổng ra và flush theo thứ tự tối ưu
 - Giữ kích thước cache trong giới hạn bộ nhớ của nó
 - Tránh làm mất dữ liệu khi nó chưa được lưu bền vào lưu trữ chính

We explore several techniques that help us to improve the first three characteristics while keeping us within the boundaries of the other two.

Chúng ta khảo sát một vài kỹ thuật giúp cải thiện ba đặc tính đầu trong khi vẫn giữ ta trong ranh giới của hai đặc tính còn lại.

Locking Pages in Cache — Khóa các trang trong Cache

Having to perform disk I/O on each read or write is impractical: subsequent reads may request the same page, just as subsequent writes may modify the same page. Since B-Tree gets “narrower” toward the top, higher-level nodes (ones that are closer to the root) are hit for most of the reads. Splits and merges also eventually propagate to the higher-level nodes. This means there’s always at least a part of a tree that can significantly benefit from being cached.

Việc phải thực hiện I/O đĩa trên mỗi lượt đọc hoặc ghi là không thực tế: các lượt đọc kế tiếp có thể yêu cầu cùng một trang, cũng như các lượt ghi kế tiếp có thể sửa đổi cùng một trang. Vì B-Tree “hẹp dần” về phía đỉnh, các nút ở mức cao hơn (những nút gần gốc hơn) được truy cập trong phần lớn các lượt đọc. Các thao tác tách (split) và gộp (merge) rốt cuộc cũng lan truyền lên các nút ở mức cao hơn. Điều này nghĩa là luôn có ít nhất một phần của cây có thể hưởng lợi đáng kể từ việc được cache.

We can “lock” pages that have a high probability of being used in the nearest time. Locking pages in the cache is called pinning . Pinned pages are kept in memory for a longer time, which helps to reduce the number of disk accesses and improve performance [GRAEFE11] .

Chúng ta có thể “khóa” các trang có xác suất cao được dùng trong thời gian gần nhất. Việc khóa các trang trong cache được gọi là ghim (pinning). Các trang được ghim được giữ trong bộ nhớ lâu hơn, điều này giúp giảm số lần truy cập đĩa và cải thiện hiệu năng [GRAEFE11].

Since each lower B-Tree node level has exponentially more nodes than the higher one, and higher-level nodes represent just a small fraction of the tree, this part of the tree can reside in memory permanently, and other parts can be paged in on demand. This means that, in order to perform a query, we won’t have to make h disk accesses (as discussed in “B-Tree Lookup Complexity” on page

37, h is the height of the tree), but only hit the disk for the lower levels, for which pages are not cached.

Vì mỗi mức nút B-Tree thấp hơn có số nút nhiều hơn theo cấp số nhân so với mức cao hơn, và các nút ở mức cao chỉ chiếm một phần nhỏ của cây, phần này của cây có thể thường trú trong bộ nhớ, còn các phần khác có thể được nạp vào theo nhu cầu. Điều này nghĩa là, để thực hiện một truy vấn, ta sẽ không phải thực hiện h lần truy cập đĩa (như đã thảo luận trong “B-Tree Lookup Complexity” ở trang 37, h là chiều cao của cây), mà chỉ phải chạm đĩa cho các mức thấp hơn, là những mức có trang chưa được cache.

Operations performed against a subtree may result in structural changes that contradict each other—for example, multiple delete operations causing merges followed by writes causing splits, or vice versa. Likewise for structural changes that propagate from different subtrees (structural changes occurring close to each other in time, in different parts of the tree, propagating up). These operations can be buffered together by applying changes only in memory, which can reduce the number of disk writes and amortize the operation costs, since only one write can be performed instead of multiple writes.

Các thao tác thực hiện trên một cây con có thể dẫn đến những thay đổi cấu trúc mâu thuẫn với nhau — ví dụ, nhiều thao tác xóa gây ra các lượt gộp, sau đó các lượt ghi gây ra các lượt tách, hoặc ngược lại. Tương tự đối với các thay đổi cấu trúc lan truyền từ các cây con khác nhau (các thay đổi cấu trúc xảy ra gần nhau về thời gian, ở những phần khác nhau của cây, lan truyền lên trên). Những thao tác này có thể được đệm chung lại với nhau bằng cách chỉ áp dụng các thay đổi trong bộ nhớ, điều này có thể giảm số lượt ghi đĩa và phân bổ chi phí thao tác (amortize), vì chỉ cần một lượt ghi thay vì nhiều lượt ghi.

Prefetching and Immediate Eviction — Đọc trước và Tổng ra ngay lập tức

The page cache also allows the storage engine to have fine-grained control over prefetching and eviction. It can be instructed to load pages ahead of time, before they are accessed. For example, when the leaf nodes are traversed in a range scan, the next leaves can be preloaded. Similarly, if a maintenance process loads the page, it can be evicted immediately after the process finishes, since it's unlikely to be useful for the in-flight queries. Some databases, for example, PostgreSQL, use a circular buffer (in other words, FIFO page replacement policy) for large **sequential** scans.

Page cache cũng cho phép bộ máy lưu trữ có quyền kiểm soát chi tiết đối với việc đọc trước (prefetching) và tổng ra. Nó có thể được chỉ dẫn để tải các trang trước thời điểm, trước khi chúng được truy cập. Ví dụ, khi các nút lá được duyệt trong một lượt quét theo dải (range scan), các lá kế tiếp có thể được nạp trước. Tương tự, nếu một tiến trình bảo trì tải trang, nó có thể bị tổng ra ngay sau khi tiến trình kết thúc, vì khả năng nó còn hữu ích cho các truy vấn đang chạy là thấp. Một số cơ sở dữ liệu, ví dụ PostgreSQL, dùng một vùng đệm vòng (nói cách khác, chính sách thay thế trang FIFO) cho các lượt quét tuần tự lớn.

Page Replacement — Thay thế trang

When cache capacity is reached, to load new pages, old ones have to be evicted. However, unless we evict pages that are least likely to be accessed again soon, we might end up loading them several times subsequently even though we could've just kept them in memory for all that time. We need to find a way to estimate the likelihood of subsequent page access to optimize this.

Khi đạt đến dung lượng cache, để tải các trang mới, các trang cũ phải bị tống ra. Tuy nhiên, trừ khi ta tống ra những trang ít có khả năng được truy cập lại sớm nhất, ta có thể rất cuộc phải tải chúng vài lần liên tiếp dù lẽ ra ta đã có thể giữ chúng trong bộ nhớ suốt thời gian đó. Ta cần tìm cách ước lượng khả năng truy cập trang kế tiếp để tối ưu việc này.

For this, we can say that pages should be evicted according to the eviction policy (also sometimes called the page-replacement policy). It attempts to find pages that are least likely to be accessed again any time soon. When the page is evicted from the cache, the new page can be loaded in its place.

Vì vậy, ta có thể nói rằng các trang nên bị tống ra theo chính sách tống ra (eviction policy) (đôi khi cũng gọi là chính sách thay thế trang). Nó cố gắng tìm những trang ít có khả năng được truy cập lại sớm nhất. Khi trang bị tống ra khỏi cache, trang mới có thể được tải vào vị trí của nó.

For a page cache implementation to be performant, it needs an efficient page-replacement algorithm. An ideal page-replacement strategy would require a crystal ball that would predict the order in which pages are going to be accessed and evict only pages that will not be touched for the longest time. Since requests do not necessarily follow any specific pattern or distribution, precisely predicting behavior can be complicated, but using a right page replacement strategy can help to reduce the number of evictions.

Để một hiện thực page cache đạt hiệu năng cao, nó cần một thuật toán thay thế trang hiệu quả. Một chiến lược thay thế trang lý tưởng sẽ đòi hỏi một quả cầu pha lê có thể dự đoán thứ tự các trang sẽ được truy cập và chỉ tống ra những trang sẽ không bị chạm đến trong thời gian lâu nhất. Vì các yêu cầu không nhất thiết tuân theo bất kỳ mẫu hay phân phối cụ thể nào, việc dự đoán hành vi một cách chính xác có thể phức tạp, nhưng dùng một chiến lược thay thế trang đúng đắn có thể giúp giảm số lần tống ra.

It seems logical that we can reduce the number of evictions by simply using a larger cache. However, this does not appear to be the case. One of the examples demonstrating this dilemma this is called Bélády's anomaly [BEDALY69] . It shows that increasing the number of pages might increase the number of evictions if the used page-replacement algorithm is not optimal. When pages that might be required soon are evicted and then loaded again, pages start competing for space in the cache. Because of that, we need to wisely consider the algorithm we're using, so that it would improve the situation, not make it worse.

Có vẻ hợp lý khi cho rằng ta có thể giảm số lần tống ra bằng cách đơn giản là dùng một cache lớn hơn. Tuy nhiên, điều này hóa ra không phải vậy. Một trong những ví dụ minh họa tình thế nan giải này được gọi là dị thường Bélády (Bélády's anomaly) [BEDALY69]. Nó cho thấy rằng việc tăng số trang có thể làm tăng số lần tống ra nếu thuật toán thay thế trang được dùng không tối ưu. Khi những trang có thể sớm cần đến bị tống ra rồi lại được tải lại, các trang bắt đầu tranh giành không gian trong cache. Vì lẽ đó, ta cần cân nhắc một cách khôn ngoan thuật toán mà ta đang dùng, sao cho nó cải thiện tình hình chứ không làm tệ đi.

FIFO and LRU

FIFO và LRU

The most naïve page-replacement strategy is first in, first out (FIFO). FIFO maintains a queue of page IDs in their insertion order, adding new pages to the tail of the queue. Whenever the page cache is full, it takes the element from the head of the queue to find the page that was paged in at the farthest point in time. Since it does not account for subsequent page accesses, only for page-in events, this proves to be impractical for the most real-world systems. For example, the root and topmost-level pages are paged in first and, according to this algorithm, are the first candidates for eviction, even

though it's clear from the tree structure that these pages are likely to be paged in again soon, if not immediately.

Chiến lược thay thế trang ngây thơ nhất là vào trước, ra trước (first in, first out — FIFO). FIFO duy trì một hàng đợi các ID trang theo thứ tự chen vào, thêm các trang mới vào đuôi hàng đợi. Mỗi khi page cache đầy, nó lấy phần tử ở đầu hàng đợi để tìm trang đã được nạp vào tại thời điểm xa nhất. Vì nó không tính đến các lượt truy cập trang kế tiếp mà chỉ tính đến các sự kiện nạp vào, điều này hóa ra là không thực tế đối với hầu hết các hệ thống thực tế. Ví dụ, trang gốc và các trang ở mức trên cùng được nạp vào đầu tiên và, theo thuật toán này, là những ứng viên đầu tiên để bị tống ra, dù từ cấu trúc cây ta thấy rõ rằng những trang này có khả năng sẽ sớm được nạp lại, nếu không phải là ngay lập tức.

A natural extension of the FIFO algorithm is least-recently used (LRU) [TANEN-BAUM14]. It also maintains a queue of eviction candidates in insertion order, but allows you to place a page back to the tail of the queue on repeated accesses, as if this was the first time it was paged in. However, updating references and relinking nodes on every access can become expensive in a concurrent environment.

Một mở rộng tự nhiên của thuật toán FIFO là ít được dùng gần đây nhất (least-recently used — LRU) [TANENBAUM14]. Nó cũng duy trì một hàng đợi các ứng viên tống ra theo thứ tự chen vào, nhưng cho phép bạn đặt một trang trở lại đuôi hàng đợi khi truy cập lặp lại, như thể đây là lần đầu tiên nó được nạp vào. Tuy nhiên, việc cập nhật các tham chiếu và liên kết lại các nút trên mỗi lần truy cập có thể trở nên đắt đỏ trong một môi trường tương tranh.

There are other LRU-based cache eviction strategies. For example, 2Q (Two-Queue LRU) maintains two queues and puts pages into the first queue during the initial access and moves them to the second hot queue on subsequent accesses, allowing you to distinguish between the recently and frequently accessed pages [JONSON94]. LRU-K identifies frequently referenced pages by keeping **track** of the last K accesses, and using this information to estimate access times on a page basis [ONEIL93].

Có những chiến lược tống ra cache dựa trên LRU khác. Ví dụ, 2Q (Two-Queue LRU) duy trì hai hàng đợi và đưa các trang vào hàng đợi thứ nhất trong lần truy cập ban đầu rồi chuyển chúng sang hàng đợi nóng thứ hai khi truy cập kế tiếp, cho phép bạn phân biệt giữa các trang được truy cập gần đây và các trang được truy cập thường xuyên [JONSON94]. LRU-K nhận diện các trang được tham chiếu thường xuyên bằng cách theo dõi K lần truy cập gần nhất, và dùng thông tin này để ước lượng thời điểm truy cập trên cơ sở từng trang [ONEIL93].

CLOCK

CLOCK

In some situations, efficiency may be more important than precision. CLOCK algorithm variants are often used as compact, cache-friendly, and concurrent alternatives to LRU [SOUNDARARAJAN06]. Linux, for example, uses a variant of the CLOCK algorithm.

Trong một số tình huống, hiệu quả có thể quan trọng hơn độ chính xác. Các biến thể của thuật toán CLOCK thường được dùng như những lựa chọn thay thế cho LRU — gọn nhẹ, thân thiện với cache, và hỗ trợ tương tranh [SOUNDARARAJAN06]. Ví dụ, Linux dùng một biến thể của thuật toán CLOCK.

CLOCK-sweep holds references to pages and associated access bits in a circular buffer. Some variants use counters instead of bits to account for frequency. Every time the page is accessed, its access bit is set to 1. The algorithm works by going around the circular buffer, checking access bits:

CLOCK-sweep giữ các tham chiếu đến các trang và các bit truy cập liên quan trong một vùng đệm vòng. Một số biến thể dùng bộ đếm thay vì bit để tính đến tần suất. Mỗi khi trang được truy cập, bit truy cập của nó được đặt thành 1. Thuật toán hoạt động bằng cách đi vòng quanh vùng đệm vòng, kiểm tra các bit truy cập:

- If the access bit is 1 , and the page is unreferenced, it is set to 0 , and the next page is inspected.
- If the access bit is already 0 , the page becomes a candidate and is scheduled for eviction.
- If the page is currently referenced, its access bit remains unchanged. It is assumed that the access bit of an accessed page cannot be 0, so it cannot be evicted. This makes referenced pages less likely to be replaced.
 - Nếu bit truy cập là 1, và trang không được tham chiếu, nó được đặt thành 0, và trang kế tiếp được kiểm tra.
 - Nếu bit truy cập đã là 0, trang trở thành ứng viên và được lập lịch để bị tống ra.
 - Nếu trang hiện đang được tham chiếu, bit truy cập của nó giữ nguyên. Người ta giả định rằng bit truy cập của một trang đang được truy cập không thể là 0, nên nó không thể bị tống ra. Điều này khiến các trang đang được tham chiếu ít có khả năng bị thay thế hơn.

Figure 5-2 shows a circular buffer with access bits.

Hình 5-2 cho thấy một vùng đệm vòng với các bit truy cập.

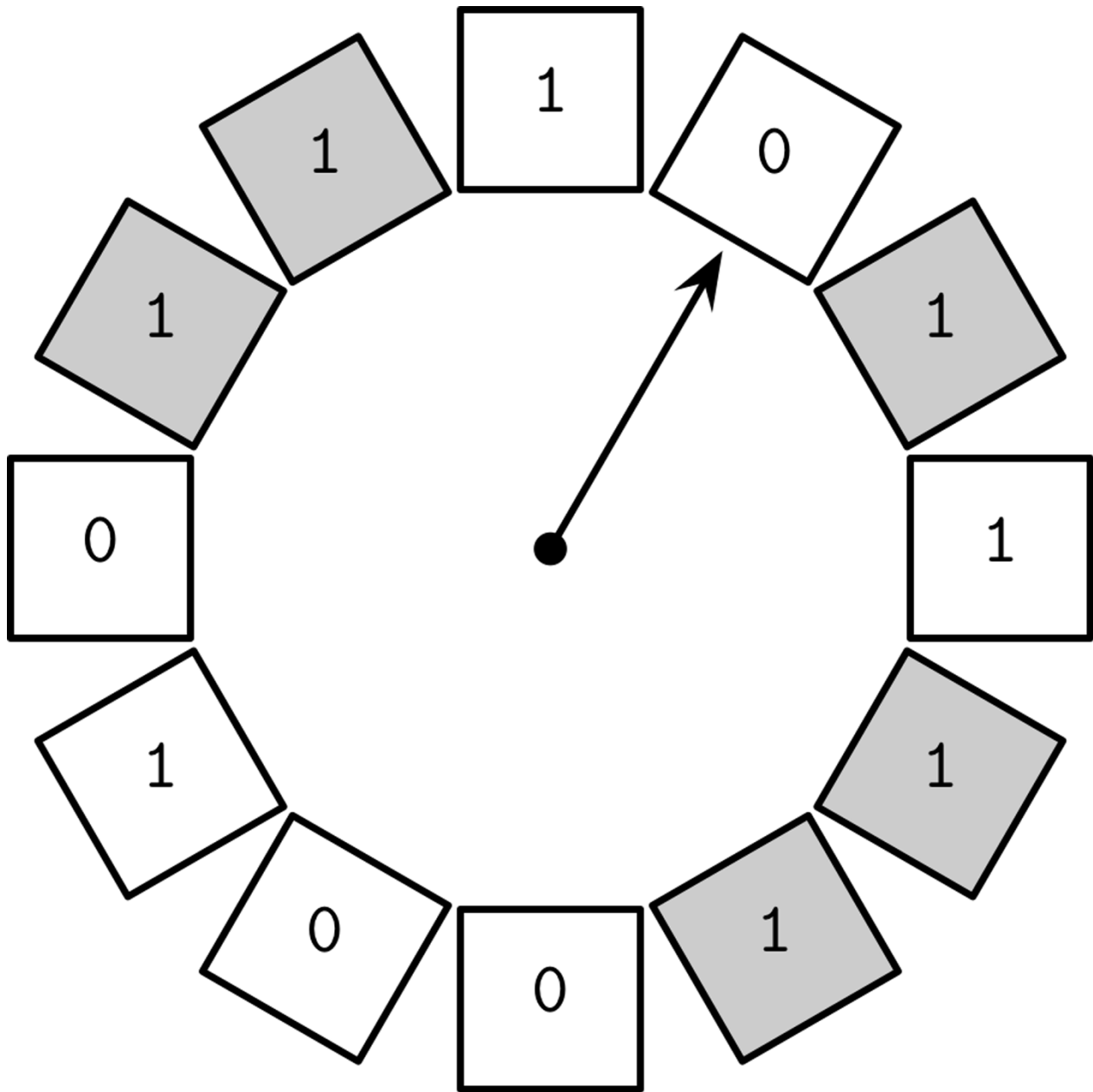


Figure 5-2. CLOCK-sweep example. Counters for currently referenced pages are shown in gray. Counters for unreferenced pages are shown in white. The arrow points to the element that will be checked next.

Hình 5-2. Ví dụ CLOCK-sweep. Các bộ đếm cho những trang đang được tham chiếu được tô màu xám. Các bộ đếm cho những trang không được tham chiếu được để trắng. Mũi tên trỏ tới phần tử sẽ được kiểm tra tiếp theo.

An advantage of using a circular buffer is that both the clock hand pointer and contents can be modified using compare-and-swap operations, and do not require additional locking mechanisms. The algorithm is easy to understand and implement and is often used in both textbooks [TANENBAUM14] and real-world systems.

Một ưu điểm của việc dùng vùng đệm vòng là cả con trỏ kim đồng hồ lẫn nội dung đều có thể được sửa đổi bằng các thao tác so-sánh-rồi-tráo (compare-and-swap), và không đòi

hỏi cơ chế khóa bổ sung nào. Thuật toán dễ hiểu và dễ hiện thực, và thường được dùng cả trong sách giáo khoa [TANENBAUM14] lẫn các hệ thống thực tế.

LRU is not always the best replacement strategy for a database system. Sometimes, it may be more practical to consider usage frequency rather than recency as a predictive factor. In the end, for a database system under a heavy load, recency might not be very indicative as it only represents the order in which items were accessed.

LRU không phải lúc nào cũng là chiến lược thay thế tốt nhất cho một hệ cơ sở dữ liệu. Đôi khi, xét tần suất sử dụng thay vì mức độ gần đây như một yếu tố dự đoán có thể thực tế hơn. Rốt cuộc, đối với một hệ cơ sở dữ liệu chịu tải nặng, mức độ gần đây có thể không mấy biểu thị vì nó chỉ phản ánh thứ tự mà các mục được truy cập.

LFU

LFU

To improve the situation, we can start tracking page reference events rather than page-in events . One of the approaches allowing us to do this tracks least-frequently used (LFU) pages.

Để cải thiện tình hình, ta có thể bắt đầu theo dõi các sự kiện tham chiếu trang thay vì các sự kiện nạp vào. Một trong các cách tiếp cận cho phép ta làm điều này là theo dõi các trang ít được dùng thường xuyên nhất (least-frequently used — LFU).

TinyLFU, a frequency-based page-eviction policy [EINZIGER17] , does precisely this: instead of evicting pages based on page-in recency , it orders pages by usage frequency . It is implemented in the popular Java library called Caffeine .

TinyLFU, một chính sách tổng trang dựa trên tần suất [EINZIGER17], làm đúng điều này: thay vì tổng các trang dựa trên mức độ gần đây của lần nạp vào, nó sắp xếp các trang theo tần suất sử dụng. Nó được hiện thực trong thư viện Java phổ biến tên là Caffeine.

TinyLFU uses a frequency histogram [CORMODE11] to maintain compact cache access history, since preserving an entire history might be prohibitively expensive for practical purposes.

TinyLFU dùng một biểu đồ tần suất (frequency histogram) [CORMODE11] để duy trì một lịch sử truy cập cache gọn nhẹ, vì việc bảo toàn toàn bộ lịch sử có thể đắt đỏ đến mức không khả thi cho các mục đích thực tế.

Elements can be in one of the three queues:

Các phần tử có thể nằm ở một trong ba hàng đợi:

- Admission , maintaining newly added elements, implemented using LRU policy.
- Probation , holding elements most likely to get evicted.
- Protected , holding elements that are to stay in the queue for a longer time.

- Kết nạp (Admission), giữ các phần tử mới được thêm vào, được hiện thực bằng chính sách LRU.
- Thử thách (Probation), giữ các phần tử có khả năng cao nhất bị tổng ra.
- Bảo vệ (Protected), giữ các phần tử sẽ ở lại hàng đợi trong thời gian lâu hơn.

Rather than choosing which elements to evict every time, this approach chooses which ones to promote for retention. Only the items that have a frequency larger than the item that would be evicted as a result of promoting them, can be moved to the probation queue. On subsequent accesses, items can get moved from probation to the protected queue. If the protected queue is full, one of the elements from it may have to be placed back into probation. More frequently accessed items have a higher chance of retention, and less frequently used ones are more likely to be evicted.

Thay vì chọn các phần tử nào để tổng ra mỗi lần, cách tiếp cận này chọn các phần tử nào để thăng lên cho việc giữ lại. Chỉ những mục có tần suất lớn hơn mục sẽ bị tổng ra do hậu quả của việc thăng chúng lên mới có thể được chuyển sang hàng đợi thử thách. Trong các lần truy cập kế tiếp, các mục có thể được chuyển từ thử thách sang hàng đợi bảo vệ. Nếu hàng đợi bảo vệ đầy, một trong các phần tử của nó có thể phải bị đặt trở lại hàng đợi thử thách. Các mục được truy cập thường xuyên hơn có cơ hội được giữ lại cao hơn, còn các mục ít được dùng hơn có khả năng bị tổng ra cao hơn.

Figure 5-3 shows the logical connections between the admission, probation, and protected queues, the frequency filter, and eviction.

Hình 5-3 cho thấy các kết nối logic giữa các hàng đợi kết nạp, thử thách, và bảo vệ, bộ lọc tần suất, và việc tổng ra.

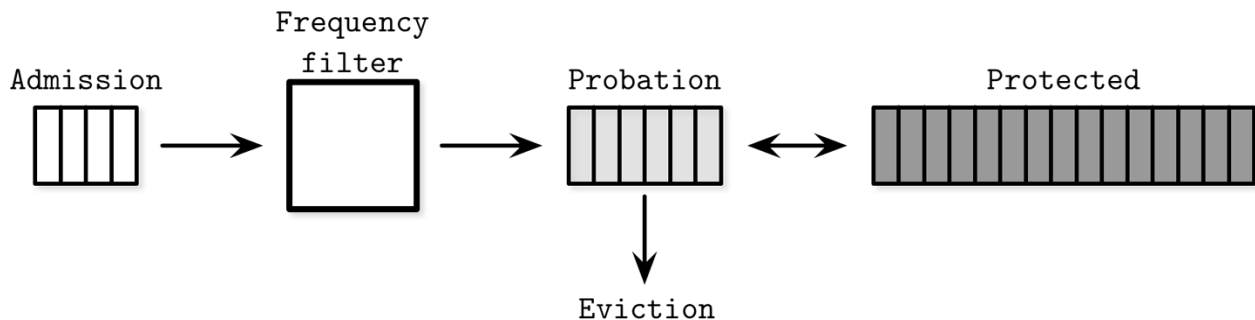


Figure 5-3. TinyLFU admission, protected, and probation queues

Hình 5-3. Các hàng đợi kết nạp, bảo vệ, và thử thách của TinyLFU

There are many other algorithms that can be used for optimal cache eviction. The choice of a page-replacement strategy has a significant impact on latency and the number of performed I/O operations, and has to be taken into consideration.

Có nhiều thuật toán khác có thể được dùng để tổng cache tối ưu. Việc lựa chọn một chiến lược thay thế trang có tác động đáng kể đến độ trễ và số thao tác I/O được thực hiện, và phải được đưa vào cân nhắc.

Recovery — Phục hồi

Database systems are built on top of several hardware and software layers that can have their own stability and reliability problems. Database systems themselves, as well as the underlying software and hardware components, may fail. Database implementers have to consider these failure scenarios and make sure that the data that was “promised” to be written is, in fact, written.

Các hệ cơ sở dữ liệu được xây dựng trên nhiều lớp phần cứng và phần mềm vốn có thể có những vấn đề riêng về độ ổn định và độ tin cậy. Bản thân các hệ cơ sở dữ liệu, cũng như các thành phần phần mềm và phần cứng nền tảng bên dưới, đều có thể lỗi. Người hiện thực cơ sở dữ liệu phải cân nhắc các kịch bản lỗi này và bảo đảm rằng dữ liệu đã được “hứa” sẽ ghi thì thực sự được ghi.

A write-ahead log (WAL for short, also known as a commit log) is an append-only auxiliary disk-resident structure used for crash and transaction recovery. The page cache allows **buffering** changes to page contents in memory. Until the cached contents are flushed back to disk, the only disk-resident copy preserving the operation history is stored in the WAL. Many database systems use append-only write-ahead logs; for example, PostgreSQL and MySQL .

Một nhật ký ghi-trước (write-ahead log — viết tắt WAL, còn được gọi là commit log) là một cấu trúc phụ trợ chỉ-nối-thêm thường trú trên đĩa được dùng cho việc phục hồi sau sự cố sập và phục hồi giao dịch. Page cache cho phép đệm các thay đổi đối với nội dung trang trong bộ nhớ. Cho tới khi nội dung đã cache được flush trở lại đĩa, bản sao duy nhất thường trú trên đĩa bảo toàn lịch sử thao tác được lưu trong WAL. Nhiều hệ cơ sở dữ liệu dùng các nhật ký ghi-trước chỉ-nối-thêm; ví dụ, PostgreSQL và MySQL.

The main functionality of a write-ahead log can be summarized as:

Chức năng chính của một nhật ký ghi-trước có thể tóm tắt như sau:

- Allow the page cache to buffer updates to disk-resident pages while ensuring durability semantics in the larger context of a database system.
- Persist all operations on disk until the cached copies of pages affected by these operations are synchronized on disk. Every operation that modifies the database state has to be logged on disk before the contents of the associated pages can be modified.
- Allow lost in-memory changes to be reconstructed from the operation log in case of a crash.
 - Cho phép page cache đệm các cập nhật đối với các trang thường trú trên đĩa trong khi vẫn bảo đảm ngữ nghĩa bền vững trong bối cảnh rộng hơn của một hệ cơ sở dữ liệu.
 - Lưu bền mọi thao tác trên đĩa cho tới khi các bản sao đã cache của các trang bị ảnh hưởng bởi các thao tác này được đồng bộ trên đĩa. Mọi thao tác sửa đổi trạng thái cơ sở dữ liệu đều phải được ghi nhật ký trên đĩa trước khi nội dung của các trang liên quan có thể bị sửa đổi.
 - Cho phép các thay đổi trong bộ nhớ bị mất được tái dựng từ nhật ký thao tác trong trường hợp sự cố sập.

In addition to this functionality, the write-ahead log plays an important role in **transaction processing**. It is hard to overstate the importance of the WAL as it ensures that data makes it to the persistent storage and is available in case of a crash, as uncommitted data is replayed from the log and the pre-crash database state is fully restored. In this section, we will often refer to **ARIES** (Algorithm for Recovery and Isolation Exploiting Semantics), a state-of-the-art recovery algorithm that is widely used and cited [MOHAN92].

Ngoài chức năng này, nhật ký ghi-trước đóng một vai trò quan trọng trong việc xử lý giao dịch. Khó mà nói quá tầm quan trọng của WAL vì nó bảo đảm rằng dữ liệu đến được lưu trữ bền và sẵn có trong trường hợp sự cố sập, vì dữ liệu chưa commit được phát lại từ nhật ký và trạng thái cơ sở dữ liệu trước khi sập được khôi phục hoàn toàn. Trong phần này, chúng ta sẽ thường nhắc đến ARIES (Algorithm for Recovery and Isolation Exploiting Semantics), một thuật toán phục hồi tối tân được dùng và trích dẫn rộng rãi [MOHAN92].

PostgreSQL Versus fsync() — PostgreSQL với fsync()

PostgreSQL uses checkpoints to ensure that index and data files have been updated with all information up to a certain record in the logfile. Flushing all dirty (modified) pages at once is done periodically by the checkpoint process. Synchronizing dirty page contents with disk is done by making the fsync() kernel call, which is supposed to sync dirty pages to disk, and unset the dirty flag on the kernel pages. As you would expect, fsync returns with an error if it isn't able to flush pages on disk.

PostgreSQL dùng các điểm kiểm tra để bảo đảm rằng các tệp chỉ mục và tệp dữ liệu đã được cập nhật với mọi thông tin tính đến một bản ghi nhất định trong tệp nhật ký. Việc

flush tất cả các trang bẩn (đã sửa đổi) cùng một lúc được tiến trình điểm kiểm tra thực hiện định kỳ. Việc đồng bộ nội dung trang bẩn với đĩa được thực hiện bằng cách gọi lời gọi nhân `fsync()`, vốn được kỳ vọng đồng bộ các trang bẩn xuống đĩa, và bỏ đặt cờ bẩn trên các trang của nhân. Như bạn dự đoán, `fsync` trả về kèm lỗi nếu nó không thể flush các trang xuống đĩa.

In Linux and a few other operating systems, `fsync` unsets the dirty flag even from unsuccessfully flushed pages after I/O errors. Additionally, errors will be reported only to the file descriptors that were open at the time of failure, so `fsync` will not return any errors that have occurred before the descriptor it was called upon was opened [CORBET18].

Trong Linux và một vài hệ điều hành khác, `fsync` bỏ đặt cờ bẩn ngay cả từ các trang flush không thành công sau khi có lỗi I/O. Ngoài ra, các lỗi sẽ chỉ được báo cáo cho những bộ mô tả tệp (file descriptor) đang mở tại thời điểm xảy ra lỗi, nên `fsync` sẽ không trả về bất kỳ lỗi nào đã xảy ra trước khi bộ mô tả mà nó được gọi lên được mở [CORBET18].

Since the checkpointer doesn't keep all files open at any given point in time, it may happen that it misses error notifications. Because dirty page flags are cleared, the checkpointer will assume that data has successfully made it on disk while, in fact, it might have not been written.

Vì bộ tạo điểm kiểm tra không giữ tất cả các tệp mở tại bất kỳ thời điểm nào, có thể xảy ra việc nó bỏ lỡ các thông báo lỗi. Vì các cờ trang bẩn bị xóa, bộ tạo điểm kiểm tra sẽ giả định rằng dữ liệu đã đến đĩa thành công trong khi, thực tế, có thể nó chưa được ghi.

A combination of these behaviors can be a source of data loss or database corruption in the presence of potentially recoverable failures. Such behaviors can be difficult to detect and some of the states they lead to may be unrecoverable. Sometimes, even triggering such behavior can be nontrivial. When working on recovery mechanisms, we should always take extra care and think through and attempt to test every possible failure scenario.

Một sự kết hợp của các hành vi này có thể là nguồn gây mất dữ liệu hoặc hỏng cơ sở dữ liệu khi có các lỗi lẻ ra có thể phục hồi. Những hành vi như vậy có thể khó phát hiện và một số trạng thái mà chúng dẫn đến có thể không thể phục hồi. Đôi khi, ngay cả việc kích hoạt hành vi như vậy cũng không hề tầm thường. Khi làm việc với các cơ chế phục hồi, ta nên luôn hết sức cẩn trọng và suy nghĩ thấu đáo cũng như cố gắng kiểm thử mọi kịch bản lỗi có thể có.

Log Semantics — Ngữ nghĩa của nhật ký

The write-ahead log is append-only and its written contents are **immutable**, so all writes to the log are sequential. Since the WAL is an immutable, append-only data structure, readers can safely access its contents up to the latest write threshold while the writer continues appending data to the log tail.

Nhật ký ghi-trước là chỉ-nối-thêm và nội dung đã ghi của nó là bất biến, nên mọi lượt ghi vào nhật ký đều tuần tự. Vì WAL là một cấu trúc dữ liệu bất biến, chỉ-nối-thêm, các bên đọc có thể truy cập nội dung của nó một cách an toàn tới ngưỡng ghi mới nhất trong khi bên ghi tiếp tục nối thêm dữ liệu vào đuôi nhật ký.

The WAL consists of log records. Every record has a unique, monotonically increasing log sequence number (LSN). Usually, the LSN is represented by an internal counter or a timestamp. Since log records do not necessarily occupy an entire disk block, their contents are cached in the log buffer and are flushed on disk in a force operation. Forces happen as the log buffers fill up, and can be requested by the transaction manager or a page cache. All log records have to be flushed on disk in LSN order.

WAL gồm các bản ghi nhật ký. Mỗi bản ghi có một số thứ tự nhật ký (log sequence number — LSN) duy nhất, tăng đơn điệu. Thông thường, LSN được biểu diễn bằng một bộ đếm nội bộ hoặc một dấu thời gian. Vì các bản ghi nhật ký không nhất thiết chiếm trọn một khối đĩa, nội dung của chúng được cache trong vùng đệm nhật ký và được flush xuống đĩa trong một thao tác force. Các thao tác force xảy ra khi các vùng đệm nhật ký đầy, và có thể được bộ quản lý giao dịch hoặc một page cache yêu cầu. Mọi bản ghi nhật ký đều phải được flush xuống đĩa theo thứ tự LSN.

Besides individual operation records, the WAL holds records indicating transaction completion. A transaction can't be considered committed until the log is forced up to the LSN of its commit record.

Bên cạnh các bản ghi thao tác riêng lẻ, WAL còn giữ các bản ghi chỉ ra sự hoàn tất của giao dịch. Một giao dịch không thể được coi là đã commit cho tới khi nhật ký được force tới LSN của bản ghi commit của nó.

To make sure the system can continue functioning correctly after a crash during rollback or recovery, some systems use compensation log records (CLR) during undo and store them in the log.

Để bảo đảm hệ thống có thể tiếp tục hoạt động đúng đắn sau một sự cố sập trong lúc cuộn lại (rollback) hoặc phục hồi, một số hệ thống dùng các bản ghi nhật ký bù trừ (compensation log records — CLR) trong lúc undo và lưu chúng trong nhật ký.

The WAL is usually coupled with a primary storage structure by the interface that allows trimming it whenever a checkpoint is reached. Logging is one of the most critical correctness aspects of the database, which is somewhat tricky to get right: even the slightest disagreements between log trimming and ensuring that the data has made it to the primary storage structure may cause data loss.

WAL thường được ghép với một cấu trúc lưu trữ chính thông qua giao diện cho phép cắt gọt nó mỗi khi đạt đến một điểm kiểm tra. Ghi nhật ký là một trong những khía cạnh về tính đúng đắn quan trọng nhất của cơ sở dữ liệu, mà lại khá khó để làm cho đúng: ngay cả những bất nhất nhỏ nhất giữa việc cắt gọt nhật ký và việc bảo đảm rằng dữ liệu đã đến được cấu trúc lưu trữ chính cũng có thể gây mất dữ liệu.

Checkpoints are a way for a log to know that log records up to a certain mark are fully persisted and aren't required anymore, which significantly reduces the amount of work required during the database startup. A process that forces all dirty pages to be flushed on disk is generally called a sync checkpoint, as it fully synchronizes the primary storage structure.

Các điểm kiểm tra là một cách để nhật ký biết rằng các bản ghi nhật ký tính đến một mốc nhất định đã được lưu bền hoàn toàn và không còn cần đến nữa, điều này giảm đáng kể khối lượng công việc cần thực hiện trong lúc khởi động cơ sở dữ liệu. Một tiến trình ép buộc mọi trang bẩn phải được flush xuống đĩa thường được gọi là điểm kiểm tra đồng bộ (sync checkpoint), vì nó đồng bộ hoàn toàn cấu trúc lưu trữ chính.

Flushing the entire contents on disk is rather impractical and would require pausing all running operations until the checkpoint is done, so most database systems implement fuzzy checkpoints. In this case, the last_checkpoint pointer stored in the log header contains the information about the last successful checkpoint. A fuzzy checkpoint begins with a special begin_checkpoint log record specifying its start, and ends with end_checkpoint log record, containing information about the dirty pages, and the contents of a transaction table. Until all the pages specified by this record are flushed, the checkpoint is considered to be incomplete. Pages are flushed asynchronously and, once this is done, the last_checkpoint record is updated with the LSN of the begin_checkpoint record and, in case of a crash, the recovery process will start from there [MOHAN92].

Việc flush toàn bộ nội dung xuống đĩa khá là không thực tế và sẽ đòi hỏi tạm dừng mọi thao tác đang chạy cho tới khi điểm kiểm tra hoàn tất, nên hầu hết các hệ cơ sở dữ liệu hiện thực các điểm kiểm tra mờ (fuzzy checkpoint). Trong trường hợp này, con trỏ `last_checkpoint` lưu trong phần đầu của nhật ký chứa thông tin về điểm kiểm tra thành công gần nhất. Một điểm kiểm tra mờ bắt đầu bằng một bản ghi nhật ký `begin_checkpoint` đặc biệt chỉ định điểm bắt đầu của nó, và kết thúc bằng bản ghi nhật ký `end_checkpoint`, chứa thông tin về các trang bẩn, và nội dung của một bảng giao dịch. Cho tới khi mọi trang được bản ghi này chỉ định đều được flush, điểm kiểm tra được coi là chưa hoàn tất. Các trang được flush một cách bất đồng bộ và, một khi việc này xong, bản ghi `last_checkpoint` được cập nhật với LSN của bản ghi `begin_checkpoint` và, trong trường hợp sự cố sập, tiến trình phục hồi sẽ bắt đầu từ đó [MOHAN92].

Operation Versus Data Log — Nhật ký thao tác với nhật ký dữ liệu

Some database systems, for example System R [CHAMBERLIN81], use shadow paging : a **copy-on-write** technique ensuring data durability and transaction atomicity. New contents are placed into the new unpublished shadow page and made visible with a pointer flip, from the old page to the one holding updated contents.

Một số hệ cơ sở dữ liệu, ví dụ System R [CHAMBERLIN81], dùng phân trang bóng (shadow paging): một kỹ thuật sao-khi-ghi (copy-on-write) bảo đảm tính bền vững của dữ liệu và tính nguyên tử của giao dịch. Nội dung mới được đặt vào trang bóng mới chưa công bố và được làm hữu hình bằng một cú lật con trỏ, từ trang cũ sang trang chứa nội dung đã cập nhật.

Any state change can be represented by a before-image and an after-image or by corresponding **redo** and undo operations. Applying a redo operation to a before-image produces an after-image . Similarly, applying an undo operation to an after-image produces a before-image .

Bất kỳ thay đổi trạng thái nào cũng có thể được biểu diễn bằng một ảnh-trước và một ảnh-sau hoặc bằng các thao tác redo và undo tương ứng. Áp dụng một thao tác redo lên một ảnh-trước tạo ra một ảnh-sau. Tương tự, áp dụng một thao tác undo lên một ảnh-sau tạo ra một ảnh-trước.

We can use a physical log (that stores complete page state or byte-wise changes to it) or a logical log (that stores operations that have to be performed against the current state) to move records or pages from one state to the other, both backward and forward in time. It is important to track the exact state of the pages that physical and logical log records can be applied to.

Ta có thể dùng một nhật ký vật lý (lưu trạng thái trang đầy đủ hoặc các thay đổi theo từng byte của nó) hoặc một nhật ký logic (lưu các thao tác phải được thực hiện trên trạng thái hiện tại) để chuyển các bản ghi hoặc trang từ trạng thái này sang trạng thái khác, cả lùi lẫn tiến theo thời gian. Việc theo dõi chính xác trạng thái của các trang mà các bản ghi nhật ký vật lý và logic có thể áp dụng lên là điều quan trọng.

Physical logging records before and after images, requiring entire pages affected by the operation to be logged. A logical log specifies which operations have to be applied to the page, such as “insert a data record X for key Y”, and a corresponding undo operation, such as “remove the value associated with Y” .

Ghi nhật ký vật lý ghi lại các ảnh-trước và ảnh-sau, đòi hỏi toàn bộ các trang bị ảnh hưởng bởi thao tác phải được ghi nhật ký. Một nhật ký logic chỉ định các thao tác nào phải được

áp dụng lên trang, chẳng hạn “chèn một bản ghi dữ liệu X cho khóa Y”, và một thao tác undo tương ứng, chẳng hạn “xóa giá trị gắn với Y”.

In practice, many database systems use a combination of these two approaches, using logical logging to perform an undo (for concurrency and performance) and physical logging to perform a redo (to improve recovery time) [MOHAN92] .

Trong thực tế, nhiều hệ cơ sở dữ liệu dùng kết hợp hai cách tiếp cận này, dùng ghi nhật ký logic để thực hiện undo (vì tính tương tranh và hiệu năng) và ghi nhật ký vật lý để thực hiện redo (để cải thiện thời gian phục hồi) [MOHAN92].

Steal and Force Policies — Chính sách Steal và Force

To determine when the changes made in memory have to be flushed on disk, database management systems define **steal**/no-steal and force/**no-force** policies. These policies are mostly applicable to the page cache, but they’re better discussed in the context of recovery, since they have a significant impact on which recovery approaches can be used in combination with them.

Để xác định khi nào các thay đổi được thực hiện trong bộ nhớ phải được flush xuống đĩa, các hệ quản trị cơ sở dữ liệu định nghĩa các chính sách steal/no-steal và force/no-force. Các chính sách này chủ yếu áp dụng cho page cache, nhưng tốt hơn là nên thảo luận trong bối cảnh phục hồi, vì chúng có tác động đáng kể đến những cách tiếp cận phục hồi nào có thể được dùng kết hợp với chúng.

A recovery method that allows flushing a page modified by the transaction even before the transaction has committed is called a steal policy. A no-steal policy does not allow flushing any uncommitted transaction contents on disk. To steal a dirty page here means flushing its in-memory contents to disk and loading a different page from disk in its place.

Một phương pháp phục hồi cho phép flush một trang bị một giao dịch sửa đổi ngay cả trước khi giao dịch đó đã commit được gọi là chính sách steal. Một chính sách no-steal không cho phép flush bất kỳ nội dung giao dịch chưa commit nào xuống đĩa. “Steal” một trang bẩn ở đây nghĩa là flush nội dung trong bộ nhớ của nó xuống đĩa và tải một trang khác từ đĩa vào vị trí của nó.

A force policy requires all pages modified by the transactions to be flushed on disk before the transaction commits. On the other hand, a no-force policy allows a transaction to commit even if some pages modified during this transaction were not yet flushed on disk. To force a dirty page here means to flush it on disk before the commit.

Một chính sách force đòi hỏi mọi trang bị các giao dịch sửa đổi đều phải được flush xuống đĩa trước khi giao dịch commit. Mặt khác, một chính sách no-force cho phép một giao dịch commit ngay cả khi một số trang bị sửa đổi trong giao dịch này chưa được flush xuống đĩa. “Force” một trang bẩn ở đây nghĩa là flush nó xuống đĩa trước khi commit.

Steal and force policies are important to understand, since they have implications for transaction undo and redo. Undo rolls back updates to forced pages for committed transactions, while redo applies changes performed by committed transactions on disk.

Các chính sách steal và force quan trọng để hiểu, vì chúng có hệ quả đối với việc undo và redo giao dịch. Undo cuộn lại các cập nhật đối với các trang đã được force của những giao dịch đã commit, còn redo áp dụng các thay đổi do những giao dịch đã commit thực hiện lên đĩa.

Using the no-steal policy allows implementing recovery using only redo entries: old copy is contained in the page on disk and modification is stored in the log [WEI-KUM01]. With no-force, we potentially can buffer several updates to pages by deferring them. Since page contents have to be cached in memory for that time, a larger page cache may be needed.

Dùng chính sách no-steal cho phép hiện thực việc phục hồi chỉ dùng các bản ghi redo: bản sao cũ được chứa trong trang trên đĩa và sửa đổi được lưu trong nhật ký [WEIKUM01]. Với no-force, ta có khả năng đệm vài cập nhật đối với các trang bằng cách trì hoãn chúng. Vì nội dung trang phải được cache trong bộ nhớ trong thời gian đó, có thể cần một page cache lớn hơn.

When the force policy is used, crash recovery doesn't need any additional work to reconstruct the results of committed transactions, since pages modified by these transactions are already flushed. A major drawback of using this approach is that transactions take longer to commit due to the necessary I/O.

Khi dùng chính sách force, việc phục hồi sau sự cố sập không cần bất kỳ công việc bổ sung nào để tái dựng kết quả của các giao dịch đã commit, vì các trang bị các giao dịch này sửa đổi đã được flush. Một nhược điểm lớn của cách tiếp cận này là các giao dịch mất nhiều thời gian hơn để commit do I/O cần thiết.

More generally, until the transaction commits, we need to have enough information to undo its results. If any pages touched by the transaction are flushed, we need to keep undo information in the log until it commits to be able to roll it back. Otherwise, we have to keep redo records in the log until it commits. In both cases, transaction cannot commit until either undo or redo records are written to the logfile.

Tổng quát hơn, cho tới khi giao dịch commit, ta cần có đủ thông tin để undo kết quả của nó. Nếu bất kỳ trang nào bị giao dịch chạm đến được flush, ta cần giữ thông tin undo trong nhật ký cho tới khi nó commit để có thể cuộn lại. Ngược lại, ta phải giữ các bản ghi redo trong nhật ký cho tới khi nó commit. Trong cả hai trường hợp, giao dịch không thể commit cho tới khi các bản ghi undo hoặc redo được ghi vào tệp nhật ký.

ARIES — ARIES

ARIES is a steal/no-force recovery algorithm. It uses physical redo to improve performance during recovery (since changes can be installed quicker) and logical undo to improve concurrency during normal operation (since logical undo operations can be applied to pages independently). It uses WAL records to implement repeating history during recovery, to completely reconstruct the database state before undoing uncommitted transactions, and creates compensation log records during undo [MOHAN92].

ARIES là một thuật toán phục hồi steal/no-force. Nó dùng redo vật lý để cải thiện hiệu năng trong lúc phục hồi (vì các thay đổi có thể được lắp đặt nhanh hơn) và undo logic để cải thiện tính tương tranh trong lúc vận hành bình thường (vì các thao tác undo logic có thể được áp dụng lên các trang một cách độc lập). Nó dùng các bản ghi WAL để hiện thực việc lặp lại lịch sử (repeating history) trong lúc phục hồi, để tái dựng hoàn toàn trạng thái cơ sở dữ liệu trước khi undo các giao dịch chưa commit, và tạo các bản ghi nhật ký bù trừ trong lúc undo [MOHAN92].

When the database system restarts after the crash, recovery proceeds in three phases:

Khi hệ cơ sở dữ liệu khởi động lại sau sự cố sập, việc phục hồi diễn ra theo ba pha:

1. The analysis phase identifies dirty pages in the page cache and transactions that were in progress at the time of a crash. Information about dirty pages is used to identify the starting point for the redo phase. A list of in-progress transactions is used during the undo phase to roll back incomplete transactions.

1. Pha phân tích nhận diện các trang bẩn trong page cache và các giao dịch đang dang dở tại thời điểm sập. Thông tin về các trang bẩn được dùng để xác định điểm bắt đầu cho pha redo. Một danh sách các giao dịch đang dang dở được dùng trong pha undo để cuộn lại các giao dịch chưa hoàn tất.

2. The redo phase repeats the history up to the point of a crash and restores the database to the previous state. This phase is done for incomplete transactions as well as ones that were committed but whose contents weren't flushed to persistent storage.

3. The undo phase rolls back all incomplete transactions and restores the database to the last consistent state. All operations are rolled back in reverse chronological order. In case the database crashes again during recovery, operations that undo transactions are logged as well to avoid repeating them.

1. Pha redo lặp lại lịch sử tới điểm sập và khôi phục cơ sở dữ liệu về trạng thái trước đó. Pha này được thực hiện cho các giao dịch chưa hoàn tất cũng như những giao dịch đã commit nhưng nội dung của chúng chưa được flush vào lưu trữ bền.

2. Pha undo cuộn lại tất cả các giao dịch chưa hoàn tất và khôi phục cơ sở dữ liệu về trạng thái nhất quán gần nhất. Mọi thao tác đều được cuộn lại theo thứ tự thời gian ngược. Trong trường hợp cơ sở dữ liệu lại sập trong lúc phục hồi, các thao tác undo các giao dịch cũng được ghi nhật ký để tránh lặp lại chúng.

ARIES uses LSNs for identifying log records, tracks pages modified by running transactions in the dirty page table, and uses physical redo, logical undo, and fuzzy checkpointing. Even though the paper describing this system was released in 1992, most concepts, approaches, and paradigms are still relevant in transaction processing and recovery today.

ARIES dùng LSN để nhận diện các bản ghi nhật ký, theo dõi các trang bị các giao dịch đang chạy sửa đổi trong bảng trang bẩn (dirty page table), và dùng redo vật lý, undo logic, và điểm kiểm tra mờ. Mặc dù bài báo mô tả hệ thống này được công bố vào năm 1992, hầu hết các khái niệm, cách tiếp cận, và mô thức vẫn còn liên quan trong việc xử lý giao dịch và phục hồi ngày nay.

Concurrency Control — Điều khiển tương tranh

When discussing database management system architecture in “**DBMS Architecture**” on page 8 , we mentioned that the transaction manager and lock manager work together to handle concurrency control . Concurrency control is a set of techniques for handling interactions between concurrently executing transactions. These techniques can be roughly grouped into the following categories:

Khi thảo luận về kiến trúc hệ quản trị cơ sở dữ liệu trong “DBMS Architecture” ở trang 8, chúng ta đã đề cập rằng bộ quản lý giao dịch và bộ quản lý khóa phối hợp với nhau để xử lý điều khiển tương tranh (concurrency control). Điều khiển tương tranh là một tập các kỹ thuật để xử lý các tương tác giữa các giao dịch thực thi đồng thời. Các kỹ thuật này có thể được gom đại khái vào các nhóm sau:

Optimistic concurrency control (OCC) Allows transactions to execute concurrent read and write operations, and determines whether or not the result of the combined execution is serializable. In other words, transactions do not block each other, maintain histories of their operations, and check

these histories for possible conflicts before commit. If execution results in a conflict, one of the conflicting transactions is aborted.

Điều khiển tương tranh lạc quan (optimistic concurrency control — OCC) Cho phép các giao dịch thực thi các thao tác đọc và ghi đồng thời, và xác định xem kết quả của việc thực thi kết hợp có khả thi tuần tự (serializable) hay không. Nói cách khác, các giao dịch không chặn lẫn nhau, duy trì lịch sử các thao tác của chúng, và kiểm tra các lịch sử này để tìm các xung đột khả dĩ trước khi commit. Nếu việc thực thi dẫn đến một xung đột, một trong các giao dịch xung đột sẽ bị abort.

Multiversion concurrency control (**MVCC**) Guarantees a consistent view of the database at some point in the past identified by the timestamp by allowing multiple timestamped versions of the record to be present. MVCC can be implemented using validation techniques, allowing only one of the updating or committing transactions to win, as well as with lockless techniques such as timestamp ordering, or lock-based ones, such as two-phase locking.

Điều khiển tương tranh đa phiên bản (MVCC) Bảo đảm một góc nhìn nhất quán về cơ sở dữ liệu tại một thời điểm nào đó trong quá khứ được nhận diện bằng dấu thời gian, bằng cách cho phép nhiều phiên bản của bản ghi gắn dấu thời gian cùng hiện diện. MVCC có thể được hiện thực bằng các kỹ thuật kiểm định (validation), chỉ cho phép một trong các giao dịch đang cập nhật hoặc đang commit thắng, cũng như bằng các kỹ thuật phi khóa như sắp thứ tự dấu thời gian, hoặc các kỹ thuật dựa trên khóa, như khóa hai pha.

Pessimistic (also known as conservative) concurrency control (PCC) There are both lock-based and nonlocking conservative methods, which differ in how they manage and grant access to shared resources. Lock-based approaches require transactions to maintain locks on database records to prevent other transactions from modifying locked records and assessing records that are being modified until the transaction releases its locks. Nonlocking approaches maintain read and write operation lists and restrict execution, depending on the schedule of unfinished transactions. Pessimistic schedules can result in a **deadlock** when multiple transactions wait for each other to release a lock in order to proceed.

Điều khiển tương tranh bi quan (còn gọi là bảo thủ) (PCC) Có cả các phương pháp bảo thủ dựa trên khóa lẫn phi khóa, khác nhau ở cách chúng quản lý và cấp quyền truy cập vào các tài nguyên dùng chung. Các cách tiếp cận dựa trên khóa đòi hỏi các giao dịch duy trì khóa trên các bản ghi cơ sở dữ liệu để ngăn các giao dịch khác sửa đổi các bản ghi đã khóa và truy cập các bản ghi đang bị sửa đổi cho tới khi giao dịch giải phóng khóa của nó. Các cách tiếp cận phi khóa duy trì các danh sách thao tác đọc và ghi và hạn chế việc thực thi, tùy vào lịch trình của các giao dịch chưa hoàn tất. Các lịch trình bi quan có thể dẫn đến bế tắc (deadlock) khi nhiều giao dịch chờ nhau giải phóng khóa để có thể tiến tới.

In this chapter, we concentrate on node-local concurrency control techniques. In Chapter 13 , you can find information about distributed transactions and other approaches, such as deterministic concurrency control (see “Distributed Transactions with **Calvin**” on page 266).

Trong chương này, chúng ta tập trung vào các kỹ thuật điều khiển tương tranh cục bộ trên từng nút. Trong Chương 13, bạn có thể tìm thấy thông tin về các giao dịch phân tán và các cách tiếp cận khác, chẳng hạn điều khiển tương tranh tất định (xem “Distributed Transactions with Calvin” ở trang 266).

Before we can further discuss concurrency control, we need to define a set of problems we’re trying to solve and discuss how transaction operations overlap and what consequences this overlapping has.

Trước khi có thể thảo luận sâu hơn về điều khiển tương tranh, chúng ta cần định nghĩa

một tập các vấn đề mà ta đang cố giải quyết và thảo luận xem các thao tác giao dịch chồng lấn ra sao và sự chồng lấn này có những hệ quả gì.

Serializability — Tính khả tuần tự

Transactions consist of read and write operations executed against the database state, and business logic (transformations, applied to the read contents). A schedule is a list of operations required to execute a set of transactions from the database-system perspective (i.e., only ones that interact with the database state, such as read, write, commit, or abort operations), since all other operations are assumed to be side-effect free (in other words, have no impact on the database state) [MOLINA08] .

Các giao dịch gồm các thao tác đọc và ghi thực thi trên trạng thái cơ sở dữ liệu, và logic nghiệp vụ (các phép biến đổi áp dụng lên nội dung đã đọc). Một lịch trình (schedule) là một danh sách các thao tác cần thiết để thực thi một tập các giao dịch dưới góc nhìn của hệ cơ sở dữ liệu (tức là chỉ những thao tác tương tác với trạng thái cơ sở dữ liệu, chẳng hạn các thao tác đọc, ghi, commit, hoặc abort), vì mọi thao tác khác được giả định là không có tác dụng phụ (nói cách khác, không có tác động đến trạng thái cơ sở dữ liệu) [MOLINA08].

A schedule is complete if contains all operations from every transaction executed in it. Correct schedules are logical equivalents to the original lists of operations, but their parts can be executed in parallel or get reordered for optimization purposes, as long as this does not violate ACID properties and the correctness of the results of individual transactions [WEIKUM01] .

Một lịch trình là đầy đủ nếu nó chứa mọi thao tác từ mọi giao dịch được thực thi trong nó. Các lịch trình đúng đắn tương đương logic với các danh sách thao tác gốc, nhưng các phần của chúng có thể được thực thi song song hoặc bị sắp xếp lại vì mục đích tối ưu, miễn là điều này không vi phạm các tính chất ACID và tính đúng đắn của kết quả của từng giao dịch riêng lẻ [WEIKUM01].

A schedule is said to be serial when transactions in it are executed completely independently and without any interleaving: every preceding transaction is fully executed before the next one starts. Serial execution is easy to reason about, as contrasted with all possible interleavings between several multistep transactions. However, always executing transactions one after another would significantly limit the system throughput and hurt performance.

Một lịch trình được gọi là tuần tự (serial) khi các giao dịch trong đó được thực thi hoàn toàn độc lập và không có bất kỳ sự đan xen nào: mỗi giao dịch trước được thực thi hoàn toàn trước khi giao dịch tiếp theo bắt đầu. Việc thực thi tuần tự dễ suy luận, trái ngược với mọi cách đan xen khả dĩ giữa nhiều giao dịch nhiều bước. Tuy nhiên, việc luôn thực thi các giao dịch lần lượt cái này sau cái kia sẽ hạn chế đáng kể thông lượng (throughput) của hệ thống và làm tổn hại hiệu năng.

We need to find a way to execute transaction operations concurrently, while maintaining the correctness and simplicity of a serial schedule. We can achieve this with serializable schedules. A schedule is serializable if it is equivalent to some complete serial schedule over the same set of transactions. In other words, it produces the same result as if we executed a set of transactions one after another in some order. Figure 5-4 shows three concurrent transactions, and possible execution histories ($3! = 6$ possibilities, in every possible order).

Chúng ta cần tìm cách thực thi các thao tác giao dịch một cách đồng thời, trong khi vẫn duy trì tính đúng đắn và tính đơn giản của một lịch trình tuần tự. Ta có thể đạt được điều này bằng các lịch trình khả tuần tự. Một lịch trình là khả tuần tự nếu nó tương đương với một lịch trình tuần tự đầy đủ nào đó trên cùng tập các giao dịch. Nói cách khác, nó tạo ra

cùng kết quả như thể ta đã thực thi một tập các giao dịch lần lượt cái này sau cái kia theo một thứ tự nào đó. Hình 5-4 cho thấy ba giao dịch đồng thời, và các lịch sử thực thi khả dĩ ($3! = 6$ khả năng, theo mọi thứ tự có thể).

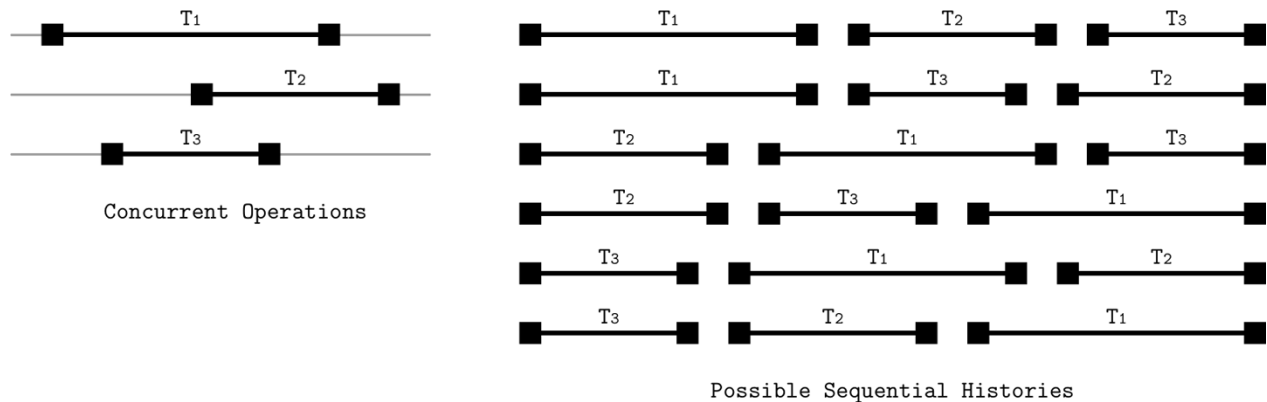


Figure 5-4. Concurrent transactions and their possible sequential execution histories

Hình 5-4. Các giao dịch đồng thời và các lịch sử thực thi tuần tự khả dĩ của chúng

Transaction Isolation — Tính cô lập của giao dịch

Transactional database systems allow different isolation levels. An **isolation level** specifies how and when parts of the transaction can and should become visible to other transactions. In other words, isolation levels describe the degree to which transactions are isolated from other concurrently executing transactions, and what kinds of anomalies can be encountered during execution.

Các hệ cơ sở dữ liệu giao dịch cho phép các mức cô lập khác nhau. Một mức cô lập chỉ định cách thức và thời điểm các phần của giao dịch có thể và nên trở nên hữu hình đối với các giao dịch khác. Nói cách khác, các mức cô lập mô tả mức độ mà các giao dịch được cô lập khỏi các giao dịch khác thực thi đồng thời, và những loại dị thường nào có thể gặp phải trong lúc thực thi.

Achieving isolation comes at a cost: to prevent incomplete or temporary writes from propagating over transaction boundaries, we need additional coordination and synchronization, which negatively impacts the performance.

Đạt được tính cô lập là có cái giá của nó: để ngăn các lượt ghi chưa hoàn tất hoặc tạm thời lan truyền qua các ranh giới giao dịch, ta cần thêm sự điều phối và đồng bộ, điều này tác động tiêu cực đến hiệu năng.

Read and Write Anomalies — Dị thường đọc và ghi

The SQL standard [MELTON06] refers to and describes read anomalies that can occur during execution of concurrent transactions: dirty, nonrepeatable, and phantom reads.

Tiêu chuẩn SQL [MELTON06] nhắc đến và mô tả các dị thường đọc có thể xảy ra trong lúc thực thi các giao dịch đồng thời: đọc bẩn, đọc không lặp lại, và đọc bóng ma.

A **dirty read** is a situation in which a transaction can read uncommitted changes from other transactions. For example, transaction T updates a user record with a new

Độc bẩn (dirty read) là tình huống trong đó một giao dịch có thể đọc các thay đổi chưa commit từ các giao dịch khác. Ví dụ, giao dịch T1 cập nhật một bản ghi người dùng với một

value for the address field, and transaction T reads the updated address before T

giá trị mới cho trường địa chỉ, và giao dịch T2 đọc địa chỉ đã cập nhật trước khi T1

commits. Transaction T aborts and rolls back its execution results. However, T has

commit. Giao dịch T1 abort và cuộn lại các kết quả thực thi của nó. Tuy nhiên, T2 đã

already been able to read this value, so it has accessed the value that has never been committed.

có thể đọc được giá trị này rồi, nên nó đã truy cập một giá trị chưa bao giờ được commit.

A nonrepeatable read (sometimes called a fuzzy read) is a situation in which a transaction queries the same row twice and gets different results. For example, this can happen even if transaction T reads a row, then transaction T modifies it and commits

Độc không lặp lại (nonrepeatable read) (đôi khi gọi là độc mờ) là tình huống trong đó một giao dịch truy vấn cùng một hàng hai lần và nhận được các kết quả khác nhau. Ví dụ, điều này có thể xảy ra ngay cả khi giao dịch T1 đọc một hàng, rồi giao dịch T2 sửa đổi nó và commit

this change. If T requests the same row again before finishing its execution, the result

thay đổi này. Nếu T1 yêu cầu cùng hàng đó lần nữa trước khi kết thúc việc thực thi của nó, kết quả

will differ from the previous run.

sẽ khác với lần chạy trước.

If we use range reads during the transaction (i.e., read not a single data record, but a range of records), we might see phantom records. A **phantom read** is when a transaction queries the same set of rows twice and receives different results. It is similar to a nonrepeatable read, but holds for range queries.

Nếu ta dùng các lượt đọc theo dải trong giao dịch (tức đọc không phải một bản ghi dữ liệu đơn lẻ, mà một dải các bản ghi), ta có thể thấy các bản ghi bóng ma. Đọc bóng ma (phantom read) là khi một giao dịch truy vấn cùng một tập hàng hai lần và nhận được các kết quả khác nhau. Nó tương tự độc không lặp lại, nhưng đúng cho các truy vấn theo dải.

There are also write anomalies with similar semantics: lost update, dirty write, and write skew.

Cũng có các dị thường ghi với ngữ nghĩa tương tự: mất cập nhật, ghi bẩn, và lệch ghi.

A lost update occurs when transactions T and T both attempt to update the value of

Mất cập nhật (lost update) xảy ra khi cả hai giao dịch T1 và T2 đều cố cập nhật giá trị của

V. T and T read the value of V. T updates V and commits, and T updates V after that

V. T1 và T2 đọc giá trị của V. T1 cập nhật V và commit, và T2 cập nhật V sau đó

and commits as well. Since the transactions are not aware about each other's existence, if both of them are allowed to commit, the results of T will be overwritten by

và cũng commit. Vì các giao dịch không hề biết đến sự tồn tại của nhau, nếu cả hai đều được phép commit, các kết quả của T1 sẽ bị ghi đè bởi

the results of T, and the update from T will be lost.

các kết quả của T2, và cập nhật từ T1 sẽ bị mất.

A dirty write is a situation in which one of the transactions takes an uncommitted value (i.e., dirty read), modifies it, and saves it. In other words, when transaction results are based on the values that have never been committed.

Ghi bẩn (dirty write) là tình huống trong đó một trong các giao dịch lấy một giá trị chưa commit (tức một lượt đọc bẩn), sửa đổi nó, và lưu lại. Nói cách khác, khi các kết quả giao dịch dựa trên các giá trị chưa bao giờ được commit.

A write skew occurs when each individual transaction respects the required invariants, but their combination does not satisfy these invariants. For example, transactions T and T modify values of two accounts A and A . A starts with 100\$ and A

Lệch ghi (write skew) xảy ra khi mỗi giao dịch riêng lẻ tôn trọng các bất biến được yêu cầu, nhưng sự kết hợp của chúng không thỏa các bất biến này. Ví dụ, các giao dịch T1 và T2 sửa đổi giá trị của hai tài khoản A1 và A2. A1 bắt đầu với 100\$ và A2

starts with 150\$. The account value is allowed to be negative, as long as the sum of the two accounts is nonnegative: $A + A \geq 0$. T and T each attempt to withdraw 200\$

bắt đầu với 150\$. Giá trị tài khoản được phép âm, miễn là tổng của hai tài khoản không âm: $A1 + A2 \geq 0$. T1 và T2 mỗi giao dịch cố rút 200\$

from A and A , respectively. Since at the time these transactions start $A + A = 250$$,

từ A1 và A2 tương ứng. Vì tại thời điểm các giao dịch này bắt đầu $A1 + A2 = 250$$,

250\$ is available in total. Both transactions assume they're preserving the invariant and are allowed to commit. After the commit, A has -100\$ and A has -50\$, which

tổng cộng có sẵn 250\$. Cả hai giao dịch đều giả định chúng đang bảo toàn bất biến và được phép commit. Sau khi commit, A1 có -100\$ và A2 có -50\$, điều này

clearly violates the requirement to keep a sum of the accounts positive [FEKETE04] .

rõ ràng vi phạm yêu cầu giữ tổng của các tài khoản dương [FEKETE04].

Isolation Levels — Các mức cô lập

The lowest (in other words, weakest) isolation level is read uncommitted . Under this isolation level, the transactional system allows one transaction to observe uncommitted changes of other concurrent transactions. In other words, dirty reads are allowed.

Mức cô lập thấp nhất (nói cách khác, yếu nhất) là đọc chưa commit (read uncommitted). Dưới mức cô lập này, hệ thống giao dịch cho phép một giao dịch quan sát các thay đổi chưa commit của các giao dịch đồng thời khác. Nói cách khác, các lượt đọc bẩn được cho phép.

We can avoid some of the anomalies. For example, we can make sure that any read performed by the specific transaction can only read already committed changes. However, it is not guaranteed that if the transaction attempts to read the same data record once again at a later stage, it will see the same value. If there was a committed modification between two reads, two queries in the same transaction would yield different results. In other words, dirty reads are not permitted, but phantom and nonrepeatable reads are. This isolation level is called read committed . If we further disallow nonrepeatable reads, we get a repeatable read isolation level.

Chúng ta có thể tránh một số dị thường. Ví dụ, ta có thể bảo đảm rằng bất kỳ lượt đọc nào do giao dịch cụ thể thực hiện chỉ có thể đọc các thay đổi đã được commit. Tuy nhiên, không có gì bảo đảm rằng nếu giao dịch cố đọc cùng một bản ghi dữ liệu lần nữa ở giai đoạn sau, nó sẽ thấy cùng giá trị. Nếu có một sửa đổi đã commit giữa hai lượt đọc, hai truy vấn trong cùng giao dịch sẽ cho các kết quả khác nhau. Nói cách khác, các lượt đọc bản không được cho phép, nhưng đọc bóng ma và đọc không lặp lại thì có. Mức cô lập này được gọi là đọc đã commit (read committed). Nếu ta cấm thêm cả đọc không lặp lại, ta có mức cô lập đọc lặp lại (repeatable read).

The strongest isolation level is **serializability**. As we already discussed in “Serializability” on page 94 , it guarantees that transaction outcomes will appear in some order as if transactions were executed serially (i.e., without overlapping in time). Disallowing concurrent execution would have a substantial negative impact on the database performance. Transactions can get reordered, as long as their internal invariants hold and can be executed concurrently, but their outcomes have to appear in some serial order.

Mức cô lập mạnh nhất là tính khả tuần tự. Như ta đã thảo luận trong “Serializability” ở trang 94, nó bảo đảm rằng các kết cục của giao dịch sẽ xuất hiện theo một thứ tự nào đó như thể các giao dịch được thực thi tuần tự (tức là không chồng lấn về thời gian). Việc cấm thực thi đồng thời sẽ có tác động tiêu cực đáng kể đến hiệu năng cơ sở dữ liệu. Các giao dịch có thể bị sắp xếp lại, miễn là các bất biến nội bộ của chúng được giữ vững và chúng có thể được thực thi đồng thời, nhưng các kết cục của chúng phải xuất hiện theo một thứ tự tuần tự nào đó.

Figure 5-5 shows isolation levels and the anomalies they allow.

Hình 5-5 cho thấy các mức cô lập và các dị thường mà chúng cho phép.

	Dirty	Non-Repeatable	Phantom
Read Uncommitted	Allowed	Allowed	Allowed
Read Committed	-	Allowed	Allowed
Repeatable Read	-	-	Allowed
Serializable	-	-	-

Figure 5-5. Isolation levels and allowed anomalies

Hình 5-5. Các mức cô lập và các dị thường được cho phép

Transactions that do not have dependencies can be executed in any order since their results are fully independent. Unlike **linearizability** (which we discuss in the context of distributed systems; see “Linearizability” on page 223), serializability is a property of multiple operations executed in arbitrary order. It does not imply or attempt to impose any particular order on executing transactions. Isolation in ACID terms means serializability [BAILIS14a] . Unfortunately, implementing serializability requires coordination. In other words, transactions executing concurrently have to coordinate to preserve invariants and impose a serial order on conflicting executions [BAILIS14b] .

Các giao dịch không có phụ thuộc có thể được thực thi theo bất kỳ thứ tự nào vì kết quả của chúng hoàn toàn độc lập. Khác với tính tuyến tính hóa (linearizability) (mà ta thảo luận trong bối cảnh các hệ phân tán; xem “Linearizability” ở trang 223), tính khả tuần tự là một tính chất của nhiều thao tác được thực thi theo thứ tự tùy ý. Nó không ngụ ý cũng không cố áp đặt bất kỳ thứ tự cụ thể nào lên việc thực thi các giao dịch. Tính cô lập theo

nghĩa ACID đồng nghĩa với tính khả tuần tự [BAILIS14a]. Đáng tiếc, việc hiện thực tính khả tuần tự đòi hỏi sự điều phối. Nói cách khác, các giao dịch thực thi đồng thời phải điều phối để bảo toàn các bất biến và áp đặt một thứ tự tuần tự lên các lượt thực thi xung đột [BAILIS14b].

Some databases use snapshot isolation . Under snapshot isolation, a transaction can observe the state changes performed by all transactions that were committed by the time it has started. Each transaction takes a snapshot of data and executes queries against it. This snapshot cannot change during transaction execution. The transaction commits only if the values it has modified did not change while it was executing. Otherwise, it is aborted and rolled back.

Một số cơ sở dữ liệu dùng cô lập ảnh chụp (snapshot isolation). Dưới cô lập ảnh chụp, một giao dịch có thể quan sát các thay đổi trạng thái được thực hiện bởi mọi giao dịch đã commit tính đến thời điểm nó bắt đầu. Mỗi giao dịch chụp một ảnh chụp dữ liệu và thực thi các truy vấn trên đó. Ảnh chụp này không thể thay đổi trong lúc giao dịch thực thi. Giao dịch chỉ commit nếu các giá trị nó đã sửa đổi không thay đổi trong lúc nó đang thực thi. Ngược lại, nó bị abort và cuộn lại.

If two transactions attempt to modify the same value, only one of them is allowed to commit. This precludes a lost update anomaly. For example, transactions T and T

Nếu hai giao dịch cố sửa đổi cùng một giá trị, chỉ một trong chúng được phép commit. Điều này loại trừ dị thường mất cập nhật. Ví dụ, các giao dịch T1 và T2

both attempt to modify V . They read the current value of V from the snapshot that contains changes from all transactions that were committed before they started. Whichever transaction attempts to commit first, will commit, and the other one will have to abort. The failed transactions will retry instead of overwriting the value.

đều cố sửa đổi V. Chúng đọc giá trị hiện tại của V từ ảnh chụp chứa các thay đổi từ mọi giao dịch đã được commit trước khi chúng bắt đầu. Giao dịch nào cố commit trước sẽ commit, và giao dịch kia sẽ phải abort. Các giao dịch thất bại sẽ thử lại thay vì ghi đè giá trị.

A write skew anomaly is possible under snapshot isolation, since if two transactions read from local state, modify independent records, and preserve local invariants, they both are allowed to commit [FEKETE04] . We discuss snapshot isolation in more detail in the context of distributed transactions in “Distributed Transactions with **Percolator**” on page 272 .

Một dị thường lệch ghi vẫn có thể xảy ra dưới cô lập ảnh chụp, vì nếu hai giao dịch đọc từ trạng thái cục bộ, sửa đổi các bản ghi độc lập, và bảo toàn các bất biến cục bộ, cả hai đều được phép commit [FEKETE04]. Chúng ta thảo luận cô lập ảnh chụp chi tiết hơn trong bối cảnh các giao dịch phân tán trong “Distributed Transactions with Percolator” ở trang 272.

Optimistic Concurrency Control — Điều khiển tương tranh lạc quan

Optimistic concurrency control assumes that transaction conflicts occur rarely and, instead of using locks and blocking transaction execution, we can validate transactions to prevent read/write conflicts with concurrently executing transactions and ensure serializability before committing their results. Generally, transaction execution is split into three phases [WEIKUM01] :

Điều khiển tương tranh lạc quan giả định rằng các xung đột giao dịch hiếm khi xảy ra và, thay vì dùng các khóa và chặn việc thực thi giao dịch, ta có thể kiểm định các giao dịch để ngăn các xung đột đọc/ghi với các giao dịch thực thi đồng thời và bảo đảm tính khả tuần tự

trước khi commit kết quả của chúng. Nhìn chung, việc thực thi giao dịch được chia thành ba pha [WEIKUM01]:

Read phase The transaction executes its steps in its own private context, without making any of the changes visible to other transactions. After this step, all transaction dependencies (read set) are known, as well as the side effects the transaction produces (write set).

Pha đọc Giao dịch thực thi các bước của nó trong bối cảnh riêng của nó, không làm cho bất kỳ thay đổi nào trở nên hữu hình đối với các giao dịch khác. Sau bước này, mọi phụ thuộc của giao dịch (tập đọc) đã được biết, cũng như các tác dụng phụ mà giao dịch tạo ra (tập ghi).

Validation phase Read and write sets of concurrent transactions are checked for the presence of possible conflicts between their operations that might violate serializability. If some of the data the transaction was reading is now out-of-date, or it would overwrite some of the values written by transactions that committed during its read phase, its private context is cleared and the read phase is restarted. In other words, the validation phase determines whether or not committing the transaction preserves ACID properties.

Pha kiểm định Các tập đọc và ghi của các giao dịch đồng thời được kiểm tra để tìm sự hiện diện của các xung đột khả dĩ giữa các thao tác của chúng có thể vi phạm tính khả tuần tự. Nếu một phần dữ liệu mà giao dịch đang đọc giờ đã lỗi thời, hoặc nó sẽ ghi đè một số giá trị được ghi bởi các giao dịch đã commit trong lúc pha đọc của nó, thì bối cảnh riêng của nó bị xóa và pha đọc được khởi động lại. Nói cách khác, pha kiểm định xác định xem việc commit giao dịch có bảo toàn các tính chất ACID hay không.

Write phase If the validation phase hasn't determined any conflicts, the transaction can commit its write set from the private context to the database state.

Pha ghi Nếu pha kiểm định không xác định được xung đột nào, giao dịch có thể commit tập ghi của nó từ bối cảnh riêng vào trạng thái cơ sở dữ liệu.

Validation can be done by checking for conflicts with the transactions that have already been committed (backward-oriented), or with the transactions that are currently in the validation phase (forward-oriented). Validation and write phases of different transactions should be done atomically. No transaction is allowed to commit while some other transaction is being validated. Since validation and write phases are generally shorter than the read phase, this is an acceptable compromise.

Việc kiểm định có thể được thực hiện bằng cách kiểm tra các xung đột với các giao dịch đã được commit (hướng-lùi), hoặc với các giao dịch hiện đang ở pha kiểm định (hướng-tiến). Các pha kiểm định và ghi của các giao dịch khác nhau nên được thực hiện một cách nguyên tử. Không giao dịch nào được phép commit trong khi một giao dịch khác đang được kiểm định. Vì các pha kiểm định và ghi nhìn chung ngắn hơn pha đọc, đây là một thỏa hiệp chấp nhận được.

Backward-oriented concurrency control ensures that for any pair of transactions T

Điều khiển tương tranh hướng-lùi bảo đảm rằng đối với bất kỳ cặp giao dịch T1

and T , the following properties hold:

và T2 nào, các tính chất sau được giữ vững:

- T was committed before the read phase of T began, so T is allowed to commit.
- T1 đã được commit trước khi pha đọc của T2 bắt đầu, nên T1 được phép commit.

- T was committed before the T write phase, and the write set of T doesn't intersect with the T read set. In other words, T hasn't written any values T should have seen.
 - T1 đã được commit trước pha ghi của T2, và tập ghi của T1 không giao với tập đọc của T2. Nói cách khác, T1 chưa ghi bất kỳ giá trị nào mà T2 lẽ ra đã phải thấy.
- The read phase of T has completed before the read phase of T, and the write set of T doesn't intersect with the read or write sets of T. In other words, transactions have operated on independent sets of data records, so both are allowed to commit.
 - Pha đọc của T1 đã hoàn tất trước pha đọc của T2, và tập ghi của T1 không giao với các tập đọc hoặc ghi của T2. Nói cách khác, các giao dịch đã thao tác trên các tập bản ghi dữ liệu độc lập, nên cả hai đều được phép commit.

This approach is efficient if validation usually succeeds and transactions don't have to be retried, since retries have a significant negative impact on performance. Of course, optimistic concurrency still has a critical section, which transactions can enter one at a time. Another approach that allows nonexclusive ownership for some operations is to use readers-writer locks (to allow shared access for readers) and upgradeable locks (to allow conversion of shared locks to exclusive when needed).

Cách tiếp cận này hiệu quả nếu việc kiểm định thường thành công và các giao dịch không phải thử lại, vì các lần thử lại có tác động tiêu cực đáng kể đến hiệu năng. Tất nhiên, tương tranh lạc quan vẫn có một vùng tới hạn (critical section), mà mỗi lần chỉ một giao dịch có thể vào. Một cách tiếp cận khác cho phép quyền sở hữu không độc quyền cho một số thao tác là dùng khóa đọc-ghi (readers-writer lock) (để cho phép truy cập chia sẻ cho các bên đọc) và khóa có thể nâng cấp (upgradeable lock) (để cho phép chuyển khóa chia sẻ thành độc quyền khi cần).

Multiversion Concurrency Control — Điều khiển tương tranh đa phiên bản

Multiversion concurrency control is a way to achieve transactional consistency in database management systems by allowing multiple record versions and using monotonically incremented transaction IDs or timestamps. This allows reads and writes to proceed with a minimal coordination on the storage level, since reads can continue accessing older values until the new ones are committed.

Điều khiển tương tranh đa phiên bản là một cách để đạt được tính nhất quán giao dịch trong các hệ quản trị cơ sở dữ liệu bằng cách cho phép nhiều phiên bản bản ghi và dùng các ID giao dịch hoặc dấu thời gian tăng đơn điệu. Điều này cho phép các lượt đọc và ghi tiến hành với sự điều phối tối thiểu ở mức lưu trữ, vì các lượt đọc có thể tiếp tục truy cập các giá trị cũ hơn cho tới khi các giá trị mới được commit.

MVCC distinguishes between committed and uncommitted versions, which correspond to value versions of committed and uncommitted transactions. The last committed version of the value is

assumed to be current . Generally, the goal of the transaction manager in this case is to have at most one uncommitted value at a time.

MVCC phân biệt giữa các phiên bản đã commit và chưa commit, vốn tương ứng với các phiên bản giá trị của các giao dịch đã commit và chưa commit. Phiên bản đã commit gần nhất của giá trị được giả định là hiện hành. Nhìn chung, mục tiêu của bộ quản lý giao dịch trong trường hợp này là có nhiều nhất một giá trị chưa commit tại một thời điểm.

Depending on the isolation level implemented by the database system, read operations may or may not be allowed to access uncommitted values [WEIKUM01] . Multiversion concurrency can be implemented using locking, scheduling, and conflict resolution techniques (such as two-phase locking), or timestamp ordering. One of the major use cases for MVCC for implementing snapshot isolation [HELLERSTEIN07] .

Tùy vào mức cô lập được hệ cơ sở dữ liệu hiện thực, các thao tác đọc có thể được hoặc không được phép truy cập các giá trị chưa commit [WEIKUM01]. Tương tranh đa phiên bản có thể được hiện thực bằng các kỹ thuật khóa, lập lịch, và giải quyết xung đột (chẳng hạn khóa hai pha), hoặc sắp thứ tự dấu thời gian. Một trong những trường hợp sử dụng chính của MVCC là để hiện thực cô lập ảnh chụp [HELLERSTEIN07].

Pessimistic Concurrency Control — Điều khiển tương tranh bi quan

Pessimistic concurrency control schemes are more conservative than optimistic ones. These schemes determine transaction conflicts while they're running and block or abort their execution.

Các sơ đồ điều khiển tương tranh bi quan bảo thủ hơn các sơ đồ lạc quan. Các sơ đồ này xác định các xung đột giao dịch trong khi chúng đang chạy và chặn hoặc abort việc thực thi của chúng.

One of the simplest pessimistic (lock-free) concurrency control schemes is timestamp ordering , where each transaction has a timestamp. Whether or not transaction operations are allowed to be executed is determined by whether or not any transaction with an earlier timestamp has already been committed. To implement that, the transaction manager has to maintain `max_read_timestamp` and `max_write_timestamp` per value, describing read and write operations executed by concurrent transactions.

Một trong những sơ đồ điều khiển tương tranh bi quan (phi khóa) đơn giản nhất là sắp thứ tự dấu thời gian (timestamp ordering), trong đó mỗi giao dịch có một dấu thời gian. Việc các thao tác giao dịch có được phép thực thi hay không được xác định bởi việc có giao dịch nào có dấu thời gian sớm hơn đã được commit hay chưa. Để hiện thực điều đó, bộ quản lý giao dịch phải duy trì `max_read_timestamp` và `max_write_timestamp` cho mỗi giá trị, mô tả các thao tác đọc và ghi được thực thi bởi các giao dịch đồng thời.

Read operations that attempt to read a value with a timestamp lower than `max_write_timestamp` cause the transaction they belong to be aborted, since there's already a newer value, and allowing this operation would violate the transaction order.

Các thao tác đọc cố đọc một giá trị với dấu thời gian thấp hơn `max_write_timestamp` sẽ khiến giao dịch chứa chúng bị abort, vì đã có một giá trị mới hơn, và cho phép thao tác này sẽ vi phạm thứ tự giao dịch.

Similarly, write operations with a timestamp lower than `max_read_timestamp` would conflict with a more recent read. However, write operations with a timestamp lower than `max_write_timestamp`

are allowed, since we can safely ignore the outdated written values. This conjecture is commonly called the Thomas Write Rule [THOMAS79] . As soon as read or write operations are performed, the corresponding maximum timestamp values are updated. Aborted transactions restart with a new timestamp, since otherwise they're guaranteed to be aborted again [RAMAKRISHNAN03] .

Tương tự, các thao tác ghi với dấu thời gian thấp hơn `max_read_timestamp` sẽ xung đột với một lượt đọc gần đây hơn. Tuy nhiên, các thao tác ghi với dấu thời gian thấp hơn `max_write_timestamp` được cho phép, vì ta có thể an toàn bỏ qua các giá trị đã ghi lỗi thời. Phỏng đoán này thường được gọi là Quy tắc Ghi Thomas (Thomas Write Rule) [THOMAS79]. Ngay khi các thao tác đọc hoặc ghi được thực hiện, các giá trị dấu thời gian cực đại tương ứng được cập nhật. Các giao dịch bị abort khởi động lại với một dấu thời gian mới, vì nếu không chúng chắc chắn sẽ lại bị abort [RAMAKRISHNAN03].

Lock-Based Concurrency Control — Điều khiển tương tranh dựa trên khóa

Lock-based concurrency control schemes are a form of pessimistic concurrency control that uses explicit locks on the database objects rather than resolving schedules, like protocols such as timestamp ordering do. Some of the downsides of using locks are contention and scalability issues [REN16] .

Các sơ đồ điều khiển tương tranh dựa trên khóa là một dạng điều khiển tương tranh bị quan dùng các khóa tương minh trên các đối tượng cơ sở dữ liệu thay vì giải quyết các lịch trình, như các giao thức kiểu sắp thứ tự dấu thời gian làm. Một số nhược điểm của việc dùng khóa là các vấn đề về tranh chấp và khả năng mở rộng [REN16].

One of the most widespread lock-based techniques is two-phase locking (2PL), which separates lock management into two phases:

Một trong những kỹ thuật dựa trên khóa phổ biến nhất là khóa hai pha (two-phase locking — 2PL), vốn tách việc quản lý khóa thành hai pha:

- The growing phase (also called the expanding phase), during which all locks required by the transaction are acquired and no locks are released.
- The shrinking phase , during which all locks acquired during the growing phase are released.
 - Pha tăng trưởng (còn gọi là pha mở rộng), trong đó mọi khóa mà giao dịch cần đều được giành lấy và không khóa nào được giải phóng.
 - Pha co lại, trong đó mọi khóa giành được trong pha tăng trưởng đều được giải phóng.

A rule that follows from these two definitions is that a transaction cannot acquire any locks as soon as it has released at least one of them. It's important to note that 2PL does not preclude transactions from executing steps during either one of these phases; however, some 2PL variants (such as conservative 2PL) do impose these limitations.

Một quy tắc rút ra từ hai định nghĩa này là một giao dịch không thể giành thêm bất kỳ khóa nào ngay khi nó đã giải phóng ít nhất một trong số đó. Cần lưu ý rằng 2PL không ngăn các giao dịch thực thi các bước trong một trong hai pha này; tuy nhiên, một số biến thể 2PL (như 2PL bảo thủ) có áp đặt các giới hạn này.



Despite similar names, two-phase locking is a concept that is entirely different from **two-phase commit** (see “Two-Phase Commit” on page 259). Two-phase commit is a protocol used for distributed multipartition transactions, while two-phase locking is a concurrency control mechanism often used to implement serializability.

Dù tên gọi tương tự, khóa hai pha là một khái niệm hoàn toàn khác với commit hai pha (xem “Two-Phase Commit” ở trang 259). Commit hai pha là một giao thức được dùng cho các giao dịch phân tán đa phân vùng, còn khóa hai pha là một cơ chế điều khiển tương tranh thường được dùng để hiện thực tính khả tuần tự.

Deadlocks

Các bế tắc

In locking protocols, transactions attempt to acquire locks on the database objects and, in case a lock cannot be granted immediately, a transaction has to wait until the lock is released. A situation may occur when two transactions, while attempting to acquire locks they require in order to proceed

with execution, end up waiting for each other to release the other locks they hold. This situation is called a deadlock .

Trong các giao thức khóa, các giao dịch cố giành các khóa trên các đối tượng cơ sở dữ liệu và, trong trường hợp một khóa không thể được cấp ngay lập tức, một giao dịch phải chờ cho tới khi khóa được giải phóng. Có thể xảy ra tình huống khi hai giao dịch, trong lúc cố giành các khóa mà chúng cần để có thể tiến hành thực thi, rốt cuộc cùng chờ nhau giải phóng các khóa khác mà chúng đang giữ. Tình huống này được gọi là bế tắc.

Figure 5-6 shows an example of a deadlock: T holds lock L and waits for lock L to be

Hình 5-6 cho thấy một ví dụ về bế tắc: T₁ giữ khóa L₁ và chờ khóa L₂ được released, while T holds lock L and waits for L to be released.

giải phóng, trong khi T₂ giữ khóa L₂ và chờ L₁ được giải phóng.

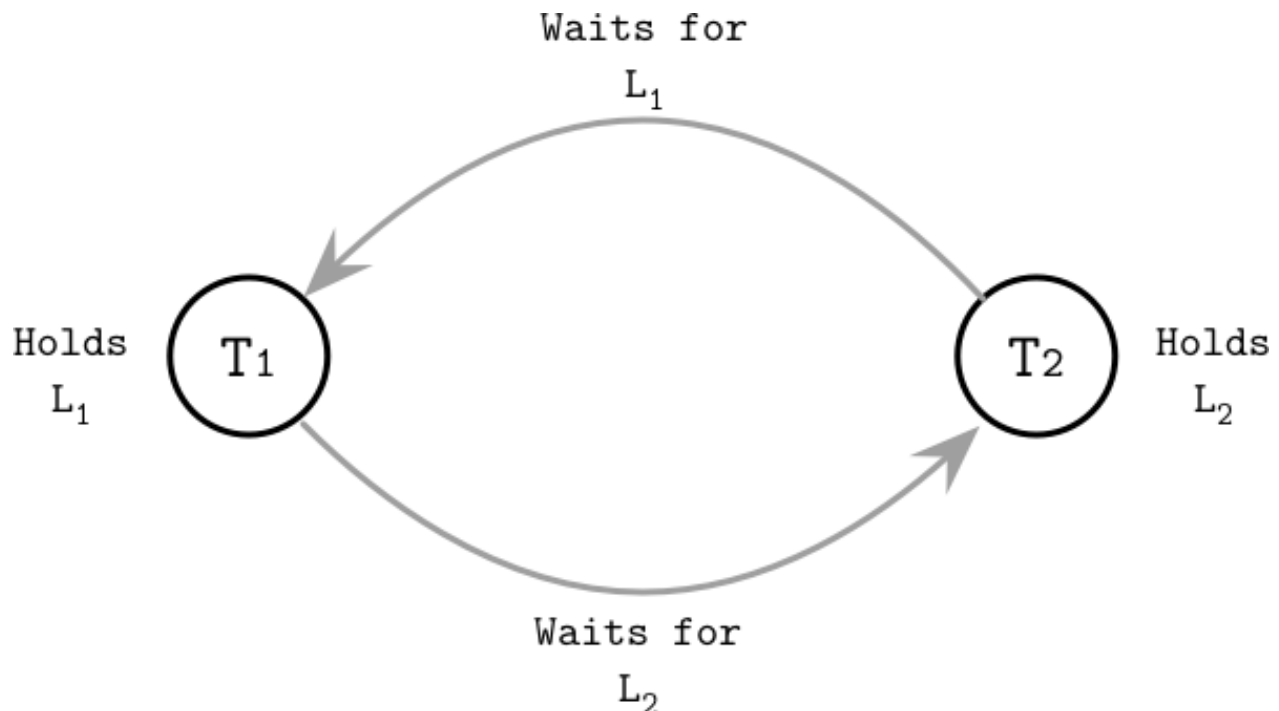


Figure 5-6. Example of a deadlock

Hình 5-6. Ví dụ về một bế tắc

The simplest way to handle deadlocks is to introduce timeouts and abort long-running transactions under the assumption that they might be in a deadlock. Another strategy, conservative 2PL, requires transactions to acquire all the locks before they can execute any of their operations and abort if they cannot. However, these approaches significantly limit system concurrency, and database systems mostly use a transaction manager to detect or avoid (in other words, prevent) deadlocks.

Cách đơn giản nhất để xử lý các bế tắc là đưa vào các thời gian chờ (timeout) và abort các giao dịch chạy lâu dưới giả định rằng chúng có thể đang ở trong một bế tắc. Một chiến lược khác, 2PL bảo thủ, đòi hỏi các giao dịch giành tất cả các khóa trước khi chúng có thể thực thi bất kỳ thao tác nào của mình và abort nếu chúng không thể. Tuy nhiên, các cách tiếp cận này hạn chế đáng kể tính tương tranh của hệ thống, và các hệ cơ sở dữ liệu chủ yếu dùng một bộ quản lý giao dịch để phát hiện hoặc tránh (nói cách khác, ngăn ngừa) các bế tắc.

Detecting deadlocks is generally done using a waits-for graph , which tracks relationships between the in-flight transactions and establishes waits-for relationships between them.

Việc phát hiện các bế tắc nhìn chung được thực hiện bằng một đồ thị chờ-cho (waits-for graph), vốn theo dõi các quan hệ giữa các giao dịch đang chạy và thiết lập các quan hệ chờ-cho giữa chúng.

Cycles in this graph indicate the presence of a deadlock: transaction T is waiting for

Các chu trình trong đồ thị này chỉ ra sự hiện diện của một bế tắc: giao dịch T1 đang chờ

T which, in turn, waits for T . Deadlock detection can be done periodically (once per

T2, đến lượt nó, lại chờ T1. Việc phát hiện bế tắc có thể được thực hiện định kỳ (mỗi khoảng time interval) or continuously (every time the waits-for graph is updated) [WEI- KUM01] . One of the transactions (usually, the one that attempted to acquire the lock more recently) is aborted.

thời gian một lần) hoặc liên tục (mỗi khi đồ thị chờ-cho được cập nhật) [WEIKUM01]. Một trong các giao dịch (thường là giao dịch đã cố giành khóa gần đây hơn) bị abort.

To avoid deadlocks and restrict lock acquisition to cases that will not result in a deadlock, the transaction manager can use transaction timestamps to determine their priority . A lower timestamp usually implies higher priority and vice versa.

Để tránh các bế tắc và hạn chế việc giành khóa vào những trường hợp sẽ không dẫn đến một bế tắc, bộ quản lý giao dịch có thể dùng các dấu thời gian giao dịch để xác định độ ưu tiên của chúng. Một dấu thời gian thấp hơn thường ngụ ý độ ưu tiên cao hơn và ngược lại.

If transaction T attempts to acquire a lock currently held by T , and T has higher pri-

Nếu giao dịch T2 cố giành một khóa hiện đang được T1 giữ, và T1 có độ ưu tiên

ority (it started before T), we can use one of the following restrictions to avoid dead-

cao hơn (nó bắt đầu trước T2), ta có thể dùng một trong các hạn chế sau để tránh các bế

locks [RAMAKRISHNAN03] :

tắc [RAMAKRISHNAN03]:

Wait-die T is allowed to block and wait for the lock. Otherwise, T is aborted and restar- 1 1 ted. In other words, a transaction can be blocked only by a transaction with a higher timestamp.

Wait-die T2 được phép chặn và chờ khóa. Ngược lại, T2 bị abort và khởi động lại. Nói cách khác, một giao dịch chỉ có thể bị chặn bởi một giao dịch có dấu thời gian cao hơn.

Wound-wait T is aborted and restarted (T wounds T). Otherwise (if T has started before T),

Wound-wait T1 bị abort và khởi động lại (T2 làm bị thương T1). Ngược lại (nếu T2 bắt đầu trước T1),

T is allowed to wait. In other words, a transaction can be blocked only by a

T2 được phép chờ. Nói cách khác, một giao dịch chỉ có thể bị chặn bởi một

transaction with a lower timestamp.

giao dịch có dấu thời gian thấp hơn.

Transaction processing requires a scheduler to handle deadlocks. At the same time, latches (see “Latches” on page 103) rely on the programmer to ensure that deadlocks cannot happen and do not rely on deadlock avoidance mechanisms.

Việc xử lý giao dịch đòi hỏi một bộ lập lịch để xử lý các bế tắc. Đồng thời, các chốt (latch) (xem “Latches” ở trang 103) dựa vào người lập trình để bảo đảm rằng các bế tắc không thể xảy ra và không dựa vào các cơ chế tránh bế tắc.

Locks

Các khóa

If two transactions are submitted concurrently, modifying overlapping segments of data, neither one of them should observe partial results of the other one, hence maintaining logical consistency. Similarly, two threads from the same transaction have to observe the same database contents, and have access to each other's data.

Nếu hai giao dịch được gửi đến đồng thời, sửa đổi các đoạn dữ liệu chồng lấn nhau, thì không giao dịch nào trong hai nên quan sát các kết quả từng phần của giao dịch kia, qua đó duy trì tính nhất quán logic. Tương tự, hai luồng từ cùng một giao dịch phải quan sát cùng nội dung cơ sở dữ liệu, và có quyền truy cập dữ liệu của nhau.

In transaction processing, there's a distinction between the mechanisms that guard the logical and physical data integrity. The two concepts responsible logical and physical integrity are, correspondingly, locks and latches. The naming is somewhat unfortunate since what's called a **latch** here is usually referred to as a lock in systems programming, but we'll clarify the distinction and implications in this section.

Trong việc xử lý giao dịch, có sự phân biệt giữa các cơ chế bảo vệ tính toàn vẹn dữ liệu logic và vật lý. Hai khái niệm chịu trách nhiệm về tính toàn vẹn logic và vật lý lần lượt là khóa (lock) và chốt (latch). Cách đặt tên có phần không may vì cái được gọi là chốt ở đây thường được gọi là lock trong lập trình hệ thống, nhưng chúng ta sẽ làm rõ sự phân biệt và các hệ quả trong phần này.

Locks are used to isolate and schedule overlapping transactions and manage database contents but not the internal storage structure, and are acquired on the key. Locks can guard either a specific key (whether it's existing or nonexistent) or a range of keys. Locks are generally stored and managed outside of the tree implementation and represent a higher-level concept, managed by the database lock manager.

Các khóa được dùng để cô lập và lập lịch các giao dịch chồng lấn và quản lý nội dung cơ sở dữ liệu chứ không phải cấu trúc lưu trữ nội bộ, và được giành trên khóa (key). Các khóa có thể bảo vệ hoặc một khóa cụ thể (dù nó tồn tại hay không tồn tại) hoặc một dải các khóa. Các khóa nhìn chung được lưu và quản lý bên ngoài hiện thực cây và biểu thị một khái niệm ở mức cao hơn, do bộ quản lý khóa của cơ sở dữ liệu quản lý.

Locks are more heavyweight than latches and are held for the duration of the transaction.

Các khóa nặng nề hơn các chốt và được giữ trong suốt thời gian của giao dịch.

Latches

Các chốt

On the other hand, latches guard the physical representation: leaf page contents are modified during insert, update, and delete operations. Nonleaf page contents and a tree structure are modified during operations resulting in splits and merges that propagate from leaf under- and overflows. Latches guard the physical tree representation (page contents and the tree structure) during these operations and are obtained on the page level. Any page has to be latched to allow safe concurrent access to it. Lockless concurrency control techniques still have to use latches.

Mặt khác, các chốt bảo vệ biểu diễn vật lý: nội dung trang lá bị sửa đổi trong các thao tác chèn, cập nhật, và xóa. Nội dung trang không phải lá và cấu trúc cây bị sửa đổi trong các thao tác dẫn đến tách và gộp lan truyền từ tình trạng thiếu và tràn của lá. Các chốt bảo vệ biểu diễn cây vật lý (nội dung trang và cấu trúc cây) trong các thao tác này và được giành ở mức trang. Bất kỳ trang nào cũng phải được chốt để cho phép truy cập đồng thời an toàn vào nó. Các kỹ thuật điều khiển tương tranh phi khóa vẫn phải dùng các chốt.

Since a single modification on the leaf level might propagate to higher levels of the B-Tree, latches might have to be obtained on multiple levels. Executing queries should not be able to observe pages in an inconsistent state, such as incomplete writes or partial node splits, during which data might be present in both the source and target node, or not yet propagated to the parent.

Vì một sửa đổi đơn lẻ ở mức lá có thể lan truyền lên các mức cao hơn của B-Tree, các chốt có thể phải được giành trên nhiều mức. Các truy vấn đang thực thi không nên có khả năng quan sát các trang ở trạng thái không nhất quán, chẳng hạn các lượt ghi chưa hoàn tất hoặc các lượt tách nút từng phần, trong đó dữ liệu có thể hiện diện ở cả nút nguồn lẫn nút đích, hoặc chưa được lan truyền lên nút cha.

The same rules apply to parent or **sibling pointer** updates. A general rule is to hold a latch for the smallest possible duration—namely, when the page is read or updated—to increase concurrency.

Cùng các quy tắc đó áp dụng cho các cập nhật con trở cha hoặc con trở anh em (sibling). Một quy tắc chung là giữ một chốt trong khoảng thời gian ngắn nhất có thể — cụ thể, khi trang được đọc hoặc cập nhật — để tăng tính tương tranh.

Interferences between concurrent operations can be roughly grouped into three categories:

Các nhiễu giữa các thao tác đồng thời có thể được gom đại khái vào ba nhóm:

- Concurrent reads , when several threads access the same page without modifying it.
- Concurrent updates , when several threads attempt to make modifications to the same page.
- Reading while writing , when one of the threads is trying to modify the page contents, and the other one is trying to access the same page for a read.

- Đọc đồng thời, khi nhiều luồng truy cập cùng một trang mà không sửa đổi nó.
- Cập nhật đồng thời, khi nhiều luồng cố thực hiện các sửa đổi lên cùng một trang.
- Đọc trong khi ghi, khi một trong các luồng đang cố sửa đổi nội dung trang, và luồng kia đang cố truy cập cùng trang đó để đọc.

These scenarios also apply to accesses that overlap with database maintenance (such as background processes, as described in “**Vacuum and Maintenance**” on page 74).

Các kịch bản này cũng áp dụng cho các truy cập chồng lấn với việc bảo trì cơ sở dữ liệu (chẳng hạn các tiến trình nền, như mô tả trong “**Vacuum and Maintenance**” ở trang 74).

Readers-writer lock

Khóa đọc-ghi

The simplest latch implementation would grant exclusive read/write access to the requesting thread. However, most of the time, we do not need to isolate all the processes from each other. For example, reads can access pages concurrently without causing any trouble, so we only need to make sure that multiple concurrent writers do not overlap, and readers do not overlap with writers . To achieve this level of granularity, we can use a readers-writer lock or RW lock.

Hiện thực chốt đơn giản nhất sẽ cấp quyền đọc/ghi độc quyền cho luồng yêu cầu. Tuy nhiên, hầu hết thời gian, ta không cần cô lập mọi tiến trình khỏi nhau. Ví dụ, các lượt đọc có thể truy cập các trang đồng thời mà không gây rắc rối gì, nên ta chỉ cần bảo đảm rằng

nhều bên ghi đồng thời không chồng lấn, và các bên đọc không chồng lấn với các bên ghi. Để đạt mức độ chi tiết này, ta có thể dùng một khóa đọc-ghi hay khóa RW.

An RW lock allows multiple readers to access the object concurrently, and only writers (which we usually have fewer of) have to obtain exclusive access to the object. Figure 5-7 shows the compatibility table for readers-writer locks: only readers can share lock ownership, while all other combinations of readers and writers should obtain exclusive ownership.

Một khóa RW cho phép nhiều bên đọc truy cập đối tượng đồng thời, và chỉ các bên ghi (mà ta thường có ít hơn) mới phải giành quyền truy cập độc quyền vào đối tượng. Hình 5-7 cho thấy bảng tương thích của các khóa đọc-ghi: chỉ các bên đọc mới có thể chia sẻ quyền sở hữu khóa, còn mọi tổ hợp khác của các bên đọc và bên ghi đều phải giành quyền sở hữu độc quyền.

	Reader	Writer
Reader	Shared	Exclusive
Writer	Exclusive	Exclusive

Figure 5-7. Readers-writer lock compatibility table

Hình 5-7. Bảng tương thích của khóa đọc-ghi

In Figure 5-8 (a), we have multiple readers accessing the object, while the writer is waiting for its turn, since it can't modify the page while readers access it. In Figure 5-8 (b), writer 1 holds an exclusive lock on the object, while another writer and three readers have to wait.

Trong Hình 5-8 (a), ta có nhiều bên đọc truy cập đối tượng, trong khi bên ghi đang chờ tới lượt, vì nó không thể sửa đổi trang trong khi các bên đọc truy cập nó. Trong Hình 5-8 (b), bên ghi 1 giữ một khóa độc quyền trên đối tượng, trong khi một bên ghi khác và ba bên đọc phải chờ.

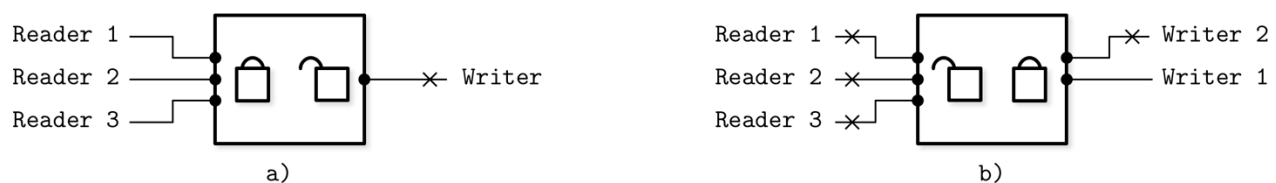


Figure 5-8. Readers-writer locks

Hình 5-8. Các khóa đọc-ghi

Since two overlapping reads attempting to access the same page do not require synchronization other than preventing the page from being fetched from disk by the page cache twice, reads can be safely executed concurrently in shared mode. As soon as writes come into play, we need to isolate them from both concurrent reads and other writes.

Vì hai lượt đọc chồng lấn cố truy cập cùng một trang không đòi hỏi sự đồng bộ nào khác ngoài việc ngăn trang bị page cache nạp từ đĩa hai lần, các lượt đọc có thể được thực thi

an toàn một cách đồng thời ở chế độ chia sẻ. Ngay khi các lượt ghi xuất hiện, ta cần cô lập chúng khỏi cả các lượt đọc đồng thời lẫn các lượt ghi khác.

Busy-Wait and Queueing Techniques — Các kỹ thuật Busy-Wait và Xếp hàng

To manage shared access to pages, we can either use blocking algorithms, which de-schedule threads and wake them up as soon as they can proceed, or use busy-wait algorithms. Busy-wait algorithms allow threads to wait for insignificant amounts of time instead of handing control back to the scheduler.

Để quản lý truy cập dùng chung vào các trang, ta có thể hoặc dùng các thuật toán chặn (blocking), vốn loại các luồng khỏi lịch và đánh thức chúng ngay khi chúng có thể tiến hành, hoặc dùng các thuật toán busy-wait. Các thuật toán busy-wait cho phép các luồng chờ trong những khoảng thời gian không đáng kể thay vì trả quyền điều khiển lại cho bộ lập lịch.

Queueing is usually implemented using compare-and-swap instructions, used to perform operations guaranteeing lock acquisition and queue update atomicity. If the queue is empty, the thread obtains access immediately. Otherwise, the thread appends itself to the waiting queue and spins on the variable that can be updated only by the thread preceding it in the queue. This helps to reduce the amount of CPU traffic for lock acquisition and release [MELLORCRUMMEY91].

Việc xếp hàng thường được hiện thực bằng các lệnh so-sánh-rồi-tráo, được dùng để thực hiện các thao tác bảo đảm tính nguyên tử của việc giành khóa và cập nhật hàng đợi. Nếu hàng đợi rỗng, luồng giành được quyền truy cập ngay lập tức. Ngược lại, luồng tự nối thêm mình vào hàng đợi chờ và quay (spin) trên biến vốn chỉ có thể được cập nhật bởi luồng đứng trước nó trong hàng đợi. Điều này giúp giảm lượng lưu lượng CPU cho việc giành và giải phóng khóa [MELLORCRUMMEY91].

Latch crabbing

Latch crabbing

The most straightforward approach for latch acquisition is to grab all the latches on the way from the root to the target leaf. This creates a concurrency bottleneck and can be avoided in most cases. The time during which a latch is held should be minimized. One of the optimizations that can be used to achieve that is called latch crabbing (or latch coupling) [RAMAKRISHNAN03].

Cách tiếp cận trực tiếp nhất để giành chốt là chộp tất cả các chốt trên đường đi từ gốc tới lá đích. Điều này tạo ra một nút thắt cổ chai về tương tranh và có thể tránh được trong hầu hết trường hợp. Khoảng thời gian một chốt được giữ nên được giảm thiểu. Một trong các tối ưu hóa có thể được dùng để đạt điều đó được gọi là latch crabbing (hay latch coupling) [RAMAKRISHNAN03].

Latch crabbing is a rather simple method that allows holding latches for less time and releasing them as soon as it's clear that the executing operation does not require them anymore. On the read path, as soon as the child node is located and its latch is acquired, the parent node's latch can be released.

Latch crabbing là một phương pháp khá đơn giản cho phép giữ các chốt trong thời gian ngắn hơn và giải phóng chúng ngay khi rõ ràng rằng thao tác đang thực thi không còn cần đến chúng nữa. Trên đường đọc, ngay khi nút con được định vị và chốt của nó được giành, chốt của nút cha có thể được giải phóng.

During insert, the parent latch can be released if the operation is guaranteed not to result in structural changes that can propagate to it. In other words, the parent latch can be released if the child node is not full.

Trong lúc chèn, chốt cha có thể được giải phóng nếu thao tác được bảo đảm sẽ không dẫn đến các thay đổi cấu trúc có thể lan truyền lên nó. Nói cách khác, chốt cha có thể được giải phóng nếu nút con không đầy.

Similarly, during deletes, if the child node holds enough elements and the operation will not cause sibling nodes to **merge**, the latch on the parent node is released.

Tương tự, trong lúc xóa, nếu nút con giữ đủ phần tử và thao tác sẽ không khiến các nút anh em gộp lại, chốt trên nút cha được giải phóng.

Figure 5-9 shows a root-to-leaf pass during insert:

Hình 5-9 cho thấy một lượt đi từ gốc-tới-lá trong lúc chèn:

- a) The write latch is acquired on the root level.
- b) The next-level node is located, and its write latch is acquired. The node is checked for potential structural changes. Since the node is not full, the parent latch can be released.
- c) The operation descends to the next level. The write latch is acquired, the target leaf node is checked for potential structural changes, and the parent latch is released.
 - a) Chốt ghi được giành ở mức gốc.
 - b) Nút ở mức tiếp theo được định vị, và chốt ghi của nó được giành. Nút được kiểm tra về các thay đổi cấu trúc tiềm tàng. Vì nút không đầy, chốt cha có thể được giải phóng.
 - c) Thao tác đi xuống mức tiếp theo. Chốt ghi được giành, nút lá đích được kiểm tra về các thay đổi cấu trúc tiềm tàng, và chốt cha được giải phóng.

This approach is optimistic: most insert and delete operations do not cause structural changes that propagate multiple levels up. In fact, the probability of structural changes decreases at higher levels. Most of the operations only require the latch on the target node, and the number of cases when the parent latch has to be retained is relatively small.

Cách tiếp cận này là lạc quan: hầu hết các thao tác chèn và xóa không gây ra các thay đổi cấu trúc lan truyền lên nhiều mức. Thực tế, xác suất xảy ra các thay đổi cấu trúc giảm dần ở các mức cao hơn. Hầu hết các thao tác chỉ cần chốt trên nút đích, và số trường hợp mà chốt cha phải được giữ lại là tương đối nhỏ.

If the child page is still not loaded in the page cache, we can either latch a future loading page, or release a parent latch and restart the root-to-leaf pass after the page is loaded to reduce contention. Restarting root-to-leaf traversal sounds rather expensive, but in reality, we have to perform it rather infrequently, and can employ mechanisms to detect whether or not there were any structural changes at higher levels since the time of traversal [GRAEFE10] .

Nếu trang con vẫn chưa được nạp trong page cache, ta có thể hoặc chốt một trang sắp được nạp trong tương lai, hoặc giải phóng chốt cha và khởi động lại lượt đi từ gốc-tới-lá sau khi trang được nạp để giảm tranh chấp. Việc khởi động lại lượt duyệt từ gốc-tới-lá nghe có vẻ khá đắt đỏ, nhưng trên thực tế, ta phải thực hiện nó khá hiếm khi, và có thể sử dụng các cơ chế để phát hiện xem có bất kỳ thay đổi cấu trúc nào ở các mức cao hơn kể từ thời điểm duyệt hay không [GRAEFE10].

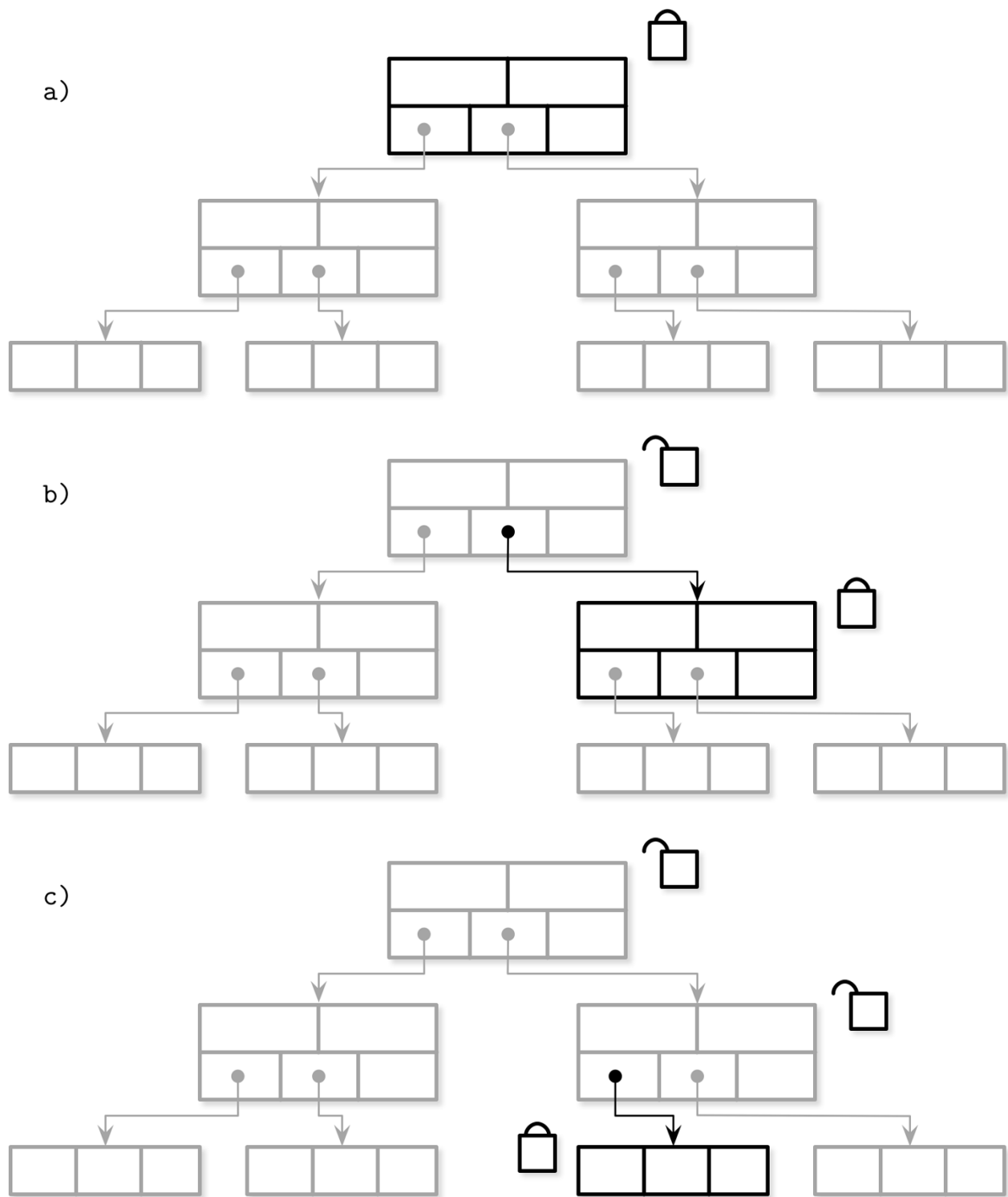


Figure 5-9. Latch crabbing during insert

Hình 5-9. Latch crabbing trong lúc chèn

Latch Upgrading and Pointer Chasing — Nâng cấp chốt và Đuổi theo con trở

Instead of acquiring latches during traversal in an exclusive mode right away, latch upgrading can be employed instead. This approach involves acquisition of shared locks along the search path and upgrading them to exclusive locks when necessary.

Thay vì giành các chốt ngay lập tức ở chế độ độc quyền trong lúc duyệt, có thể sử dụng việc nâng cấp chốt thay thế. Cách tiếp cận này bao gồm việc giành các khóa chia sẻ dọc theo đường tìm kiếm và nâng cấp chúng thành các khóa độc quyền khi cần.

Write operations first acquire exclusive locks only at the leaf level. If the leaf has to be split or merged, the algorithm walks up the tree and attempts to upgrade a shared lock that the parent holds, acquiring exclusive ownership of latches for the affected portion of the tree (i.e., nodes that will also be split or merged as a result of that operation). Since multiple threads might attempt to acquire exclusive locks on one of the higher levels, one of them has to wait or restart.

Các thao tác ghi trước hết chỉ giành các khóa độc quyền ở mức lá. Nếu lá phải bị tách hoặc gộp, thuật toán đi ngược lên cây và cố nâng cấp một khóa chia sẻ mà nút cha đang giữ, giành quyền sở hữu độc quyền các chốt cho phần cây bị ảnh hưởng (tức là các nút cũng sẽ bị tách hoặc gộp do hậu quả của thao tác đó). Vì nhiều luồng có thể cố giành các khóa độc quyền trên một trong các mức cao hơn, một trong số chúng phải chờ hoặc khởi động lại.

You might have noticed that the mechanisms described so far all start by acquiring a latch on the root node. Every request has to go through the root node, and it quickly becomes a bottleneck. At the same time, the root is always the last to be split, since all of its children have to fill up first. This means that the root node can always be latched optimistically, and the price of a retry (pointer chasing) is seldom paid.

Bạn có thể đã để ý rằng các cơ chế được mô tả từ đầu đến giờ đều bắt đầu bằng việc giành một chốt trên nút gốc. Mọi yêu cầu đều phải đi qua nút gốc, và nó nhanh chóng trở thành một nút thắt cổ chai. Đồng thời, gốc luôn là nút cuối cùng bị tách, vì mọi nút con của nó phải đầy lên trước đã. Điều này nghĩa là nút gốc luôn có thể được chốt một cách lạc quan, và cái giá của một lần thử lại (đuổi theo con trở — pointer chasing) hiếm khi phải trả.

Blink-Trees

Blink-Tree

B link -Trees build on top of B*-Trees (see “**Rebalancing**” on page 70) and add high keys (see “Node High Keys” on page 64) and sibling link pointers [LEHMAN81] . A **high key** indicates the highest possible subtree key. Every node but root in a B link -Tree has two pointers: a child pointer descending from the parent and a sibling link from the left node residing on the same level.

B-link-Tree được xây dựng dựa trên B*-Tree (xem “Rebalancing” ở trang 70) và bổ sung các khóa biên (high key) (xem “Node High Keys” ở trang 64) và các con trở liên kết anh em [LEHMAN81]. Một khóa biên chỉ ra khóa cây con cao nhất có thể có. Mọi nút trong một B-link-Tree, trừ gốc, đều có hai con trở: một con trở con đi xuống từ nút cha và một liên kết anh em từ nút bên trái nằm trên cùng mức.

B link -Trees allow a state called half-split , where the node is already referenced by the sibling pointer, but not by the child pointer from its parent. Half-split is identified by checking the node high key. If the search key exceeds the high key of the node (which violates the high key invariant), the lookup algorithm concludes that the structure has been changed concurrently and follows the sibling link to proceed with the search.

B-link-Tree cho phép một trạng thái gọi là tách-nửa (half-split), trong đó nút đã được con trở anh em tham chiếu, nhưng chưa được con trở con từ nút cha của nó tham chiếu. Tách-nửa được nhận diện bằng cách kiểm tra khóa biên của nút. Nếu khóa tìm kiếm vượt quá khóa biên của nút (điều này vi phạm bất biến khóa biên), thuật toán tra cứu kết luận rằng cấu trúc đã bị thay đổi một cách đồng thời và đi theo liên kết anh em để tiếp tục việc tìm kiếm.

The pointer has to be quickly added to the parent guarantee the best performance, but the search process doesn't have to be aborted and restarted, since all elements in the tree are accessible. The advantage here is that we do not have to hold the parent lock when descending to the child level, even if the child is going to be split: we can make a new node visible through its sibling link and update the parent pointer lazily without sacrificing correctness [GRAEFE10] .

Con trở phải được thêm nhanh vào nút cha để bảo đảm hiệu năng tốt nhất, nhưng quá trình tìm kiếm không phải bị abort và khởi động lại, vì mọi phần tử trong cây đều có thể truy cập. Ưu điểm ở đây là ta không phải giữ khóa cha khi đi xuống mức con, ngay cả khi nút con sắp bị tách: ta có thể làm cho một nút mới trở nên hữu hình thông qua liên kết anh em của nó và cập nhật con trở cha một cách lười (lazy) mà không hy sinh tính đúng đắn [GRAEFE10].

While this is slightly less efficient than descending directly from the parent and requires accessing an extra page, this results in correct root-to-leaf descent while simplifying concurrent access. Since splits are a relatively infrequent operation and B- Trees rarely shrink, this case is exceptional, and its cost is insignificant. This approach has quite a few benefits: it reduces contention, prevents holding a parent lock during splits, and reduces the number of locks held during tree structure modification to a constant number. More importantly, it allows reads concurrent to structural tree changes, and prevents deadlocks otherwise resulting from concurrent modifications ascending to the parent nodes.

Mặc dù cách này kém hiệu quả hơn đôi chút so với việc đi xuống trực tiếp từ nút cha và đòi hỏi truy cập một trang phụ, nó vẫn cho ra một lượt đi từ gốc-tới-lá đúng đắn trong khi đơn giản hóa truy cập đồng thời. Vì các lượt tách là một thao tác tương đối hiếm và các B-Tree hiếm khi co lại, trường hợp này là ngoại lệ, và cái giá của nó không đáng kể. Cách tiếp cận này có khá nhiều lợi ích: nó giảm tranh chấp, ngăn việc giữ một khóa cha trong lúc tách, và giảm số khóa được giữ trong lúc sửa đổi cấu trúc cây xuống một con số hằng. Quan trọng hơn, nó cho phép các lượt đọc đồng thời với các thay đổi cấu trúc cây, và ngăn các bế tắc mà nếu không sẽ phát sinh từ các sửa đổi đồng thời lan lên các nút cha.

Summary — Tóm tắt

In this chapter, we discussed the storage engine components responsible for transaction processing and recovery. When implementing transaction processing, we are presented with two problems:

Trong chương này, chúng ta đã thảo luận các thành phần của bộ máy lưu trữ chịu trách nhiệm xử lý giao dịch và phục hồi. Khi hiện thực việc xử lý giao dịch, chúng ta đối mặt với hai vấn đề:

- To improve efficiency, we need to allow concurrent transaction execution.
- To preserve correctness, we have to ensure that concurrently executing transactions preserve ACID properties.
 - Để cải thiện hiệu quả, ta cần cho phép thực thi giao dịch đồng thời.

- Để bảo toàn tính đúng đắn, ta phải bảo đảm rằng các giao dịch thực thi đồng thời bảo toàn các tính chất ACID.

Concurrent transaction execution can cause different kinds of read and write anomalies. Presence or absence of these anomalies is described and limited by implementing different isolation levels. Concurrency control approaches determine how transactions are scheduled and executed.

Việc thực thi giao dịch đồng thời có thể gây ra nhiều loại dị thường đọc và ghi khác nhau. Sự hiện diện hay vắng mặt của các dị thường này được mô tả và giới hạn bằng việc hiện thực các mức cô lập khác nhau. Các cách tiếp cận điều khiển tương tranh xác định cách các giao dịch được lập lịch và thực thi.

The page cache is responsible for reducing the number of disk accesses: it caches pages in memory and allows read and write access to them. When the cache reaches its capacity, pages are evicted and flushed back on disk. To make sure that unflushed changes are not lost in case of node crashes and to support transaction rollback, we use write-ahead logs. The page cache and write-ahead logs are coordinated using force and steal policies, ensuring that every transaction can be executed efficiently and rolled back without sacrificing durability.

Page cache chịu trách nhiệm giảm số lần truy cập đĩa: nó cache các trang trong bộ nhớ và cho phép truy cập đọc và ghi vào chúng. Khi cache đạt đến dung lượng của nó, các trang bị tống ra và flush trở lại đĩa. Để bảo đảm rằng các thay đổi chưa flush không bị mất trong trường hợp các nút sập và để hỗ trợ việc cuộn lại giao dịch, ta dùng các nhật ký ghi-trước. Page cache và các nhật ký ghi-trước được điều phối bằng các chính sách force và steal, bảo đảm rằng mọi giao dịch đều có thể được thực thi hiệu quả và cuộn lại mà không hy sinh tính bền vững.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Transaction processing and recovery, generally Weikum, Gerhard, and Gottfried Vossen. 2001. Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery . San Francisco: Morgan Kaufmann Publishers Inc.

Bernstein, Philip A. and Eric Newcomer. 2009. Principles of Transaction Processing . San Francisco: Morgan Kaufmann.

Graefe, Goetz, Guy, Wey & Sauer, Caetano. 2016. "Instant Recovery with Write- Ahead Logging: Page Repair, System Restart, Media Restore, and System Failover, (2nd Ed.)" in Synthesis Lectures on Data Management 8, 1-113. 10.2200/ S00710ED2V01Y201603DTM044.

Mohan, C., Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz.

1992. "ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging." Transactions on Database Systems 17, no. 1 (March): 94-162. <https://doi.org/10.1145/128765.128770> .

Concurrency control in B-Trees Wang, Paul. 1991. "An In-Depth Analysis of Concurrent B-Tree Algorithms." MIT Technical Report. <https://apps.dtic.mil/dtic/tr/fulltext/u2/a232287.pdf> .

Goetz Graefe. 2010. A survey of B-tree locking techniques. ACM Trans. Database Syst. 35, 3, Article 16 (July 2010), 26 pages.

Parallel and concurrent data structures McKenney, Paul E. 2012. “Is Parallel Programming Hard, And, If So, What Can You Do About It?” <https://arxiv.org/abs/1701.00854> .

Herlihy, Maurice and Nir Shavit. 2012. *The Art of Multiprocessor Programming*, Revised Reprint (1st Ed.) . San Francisco: Morgan Kaufmann.

Chronological developments in the field of transaction processing Diaconu, Cristian, Craig Freedman, Erik Ismert, Per-Åke Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. 2013. “Hekaton: SQL Server’s Memory-Optimized OLTP Engine.” In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD ’13)* , 1243-1254. New York: Association for Computing Machinery. <https://doi.org/10.1145/2463676.2463710> .

Kimura, Hideaki. 2015. “FOEDUS: OLTP Engine for a Thousand Cores and NVRAM.” In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD ’15)* , 691-706. <https://doi.org/10.1145/2723372.2746480> .

Yu, Xiangyao, Andrew Pavlo, Daniel Sanchez, and Srinivas Devadas. 2016. “Tic- Toc: Time Traveling Optimistic Concurrency Control.” In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD ’16)* , 1629-1642. <https://doi.org/10.1145/2882903.2882935> .

Kim, Kangnyeon, Tianzheng Wang, Ryan Johnson, and Ippokratis Pandis. 2016. “ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads.” In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD ’16)* , 1675-1687. <https://doi.org/10.1145/2882903.2882905> .

Lim, Hyeontaek, Michael Kaminsky, and David G. Andersen. 2017. “Cicada: Dependably Fast Multi-Core In-Memory Transactions.” In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD ’17)* , 21-35. <https://doi.org/10.1145/3035918.3064015> .

CHAPTER 6 B-Tree Variants —

Chương 6: Các biến thể của B-Tree

B-Tree variants have a few things in common: tree structure, balancing through splits and merges, and lookup and delete algorithms. Other details, related to concurrency, on-disk **page** representation, links between sibling nodes, and maintenance processes, may vary between implementations.

Các biến thể của B-Tree có một số điểm chung: cấu trúc cây, cân bằng cây (tree balancing) thông qua tách và gộp nút (node split/merge), và các thuật toán tra cứu cùng xóa. Những chi tiết khác, liên quan đến tính tương tranh, biểu diễn trang trên đĩa, các liên kết giữa các nút anh em (sibling), và các tiến trình bảo trì, có thể khác nhau giữa các cách hiện thực.

In this chapter, we'll discuss several techniques that can be used to implement efficient B-Trees and structures that employ them:

Trong chương này, chúng ta sẽ thảo luận một số kỹ thuật có thể được dùng để hiện thực các B-Tree hiệu quả cùng những cấu trúc tận dụng chúng:

- **Copy-on-write** B-Trees are structured like B-Trees, but their nodes are **immutable** and are not updated in place. Instead, pages are copied, updated, and written to new locations.
- **Lazy B-Trees** reduce the number of I/O requests from subsequent same-**node** writes by **buffering** updates to nodes. In the next chapter, we also cover two-component LSM trees (see “Two-component **LSM Tree**” on page 132), which take buffering a step further to implement fully immutable B-Trees.
- **FD-Trees** take a different approach to buffering, somewhat similar to LSM Trees (see “LSM Trees” on page 130). FD-Trees buffer updates in a small B-Tree. As soon as this tree fills up, its contents are written into an immutable run. Updates propagate between levels of immutable runs in a cascading manner, from higher levels to lower ones.
- **Bw-Trees** separate B-Tree nodes into several smaller parts that are written in an append-only manner. This reduces costs of small writes by batching updates to the different nodes together.
- **Cache-oblivious B-Trees** allow treating on-disk data structures in a way that is very similar to how we build in-memory ones.

- B-Tree sao-khi-ghi (copy-on-write) được tổ chức giống B-Tree, nhưng các nút của chúng là bất biến (immutable) và không được cập nhật tại chỗ. Thay vào đó, các trang được sao chép, cập nhật, rồi ghi vào vị trí mới.
- B-Tree lười (lazy B-Tree) giảm số lượng yêu cầu I/O từ các lần ghi kế tiếp vào cùng một nút bằng cách đệm (buffering) các bản cập nhật cho các nút. Trong chương tiếp theo, chúng ta cũng đề cập đến LSM tree hai thành phần (xem “Two-component LSM Tree” ở trang 132), kỹ thuật này đẩy việc đệm tiến thêm một bước để hiện thực các B-Tree hoàn toàn bất biến.
- FD-Tree áp dụng cách tiếp cận khác cho việc đệm, phần nào tương tự LSM Tree (xem “LSM Trees” ở trang 130). FD-Tree đệm các bản cập nhật trong một B-Tree nhỏ. Ngay khi cây này đầy, nội dung của nó được ghi vào một run bất biến. Các bản cập nhật lan truyền giữa các tầng run bất biến theo kiểu xếp tầng, từ tầng cao xuống tầng thấp.
- Bw-Tree tách các nút B-Tree thành nhiều phần

nhỏ hơn được ghi theo kiểu chỉ-thêm (append-only). Điều này giảm chi phí của các lần ghi nhỏ bằng cách gộp theo lô các bản cập nhật cho các nút khác nhau lại với nhau. • B-Tree không phụ thuộc bộ nhớ đệm (cache-oblivious B-Tree) cho phép xử lý các cấu trúc dữ liệu trên đĩa theo cách rất giống với cách chúng ta xây dựng các cấu trúc trong bộ nhớ.

111

111

Copy-on-Write — Sao-khi-ghi

Some databases, rather than building complex latching mechanisms, use the copy-on-write technique to guarantee data integrity in the presence of concurrent operations. In this case, whenever the page is about to be modified, its contents are copied, the copied page is modified instead of the original one, and a parallel tree hierarchy is created.

Một số cơ sở dữ liệu, thay vì xây dựng các cơ chế chốt (latch) phức tạp, sử dụng kỹ thuật sao-khi-ghi để bảo đảm tính toàn vẹn dữ liệu khi có các thao tác diễn ra đồng thời. Trong trường hợp này, mỗi khi một trang sắp bị sửa đổi, nội dung của nó được sao chép, trang sao chép được sửa đổi thay cho trang gốc, và một cây phân cấp song song được tạo ra.

Old tree versions remain accessible for readers that run concurrently to the writer, while writers accessing modified pages have to wait until preceding write operations are complete. After the new page hierarchy is created, the pointer to the topmost page is atomically updated. In Figure 6-1, you can see a new tree being created parallel to the old one, reusing the untouched pages.

Các phiên bản cây cũ vẫn truy cập được cho những bên đọc đang chạy đồng thời với bên ghi, trong khi những bên ghi truy cập các trang đã sửa đổi phải chờ cho đến khi các thao tác ghi trước đó hoàn tất. Sau khi cây phân cấp trang mới được tạo, con trỏ tới trang trên cùng được cập nhật một cách nguyên tử. Trong Hình 6-1, bạn có thể thấy một cây mới đang được tạo song song với cây cũ, tái sử dụng các trang không bị động đến.

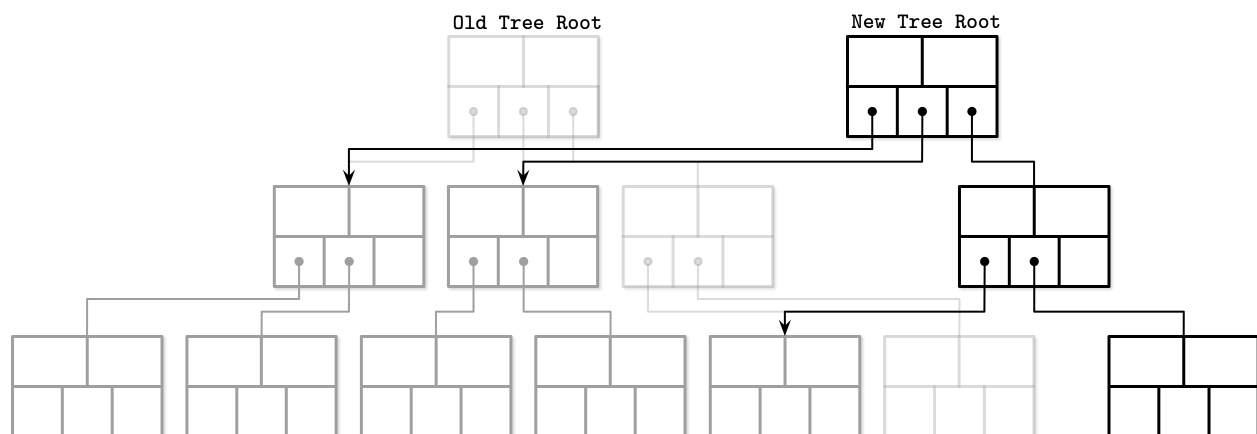


Figure 6-1. Copy-on-write B-Trees

Hình 6-1. B-Tree sao-khi-ghi

An obvious downside of this approach is that it requires more space (even though old versions are retained only for brief time periods, since pages can be reclaimed immediately after concurrent operations using the old pages complete) and processor time, as entire page contents have to be

copied. Since B-Trees are generally shallow, the simplicity and advantages of this approach often still outweigh the downsides.

Một nhược điểm rõ ràng của cách tiếp cận này là nó đòi hỏi nhiều không gian hơn (mặc dù các phiên bản cũ chỉ được giữ lại trong khoảng thời gian ngắn, vì các trang có thể được thu hồi ngay sau khi những thao tác đồng thời dùng các trang cũ hoàn tất) và nhiều thời gian xử lý hơn, vì toàn bộ nội dung trang phải được sao chép. Do B-Tree thường nông, sự đơn giản và những ưu điểm của cách tiếp cận này thường vẫn lớn hơn các nhược điểm.

The biggest advantage of this approach is that readers require no synchronization, because written pages are immutable and can be accessed without additional latching. Since writes are performed against copied pages, readers do not **block** writers. No operation can observe a page in an incomplete state, and a system crash cannot leave pages in a corrupted state, since the topmost pointer is switched only when all page modifications are done.

Ưu điểm lớn nhất của cách tiếp cận này là các bên đọc không cần đồng bộ hóa, vì các trang đã ghi là bất biến và có thể được truy cập mà không cần chốt bổ sung. Do các lần ghi được thực hiện trên các trang đã sao chép, các bên đọc không chặn các bên ghi. Không thao tác nào có thể quan sát một trang ở trạng thái chưa hoàn chỉnh, và một sự cố sập hệ thống không thể để lại các trang ở trạng thái hỏng, vì con trỏ trên cùng chỉ được chuyển khi tất cả các sửa đổi trang đã hoàn tất.

Implementing Copy-on-Write: LMDB — Hiện thực Sao-khi-ghi: LMDB

One of the storage engines using copy-on-write is the Lightning Memory-Mapped Database (LMDB), which is a key-value store used by the OpenLDAP project. Due to its design and architecture, LMDB doesn't require a **page cache**, a write-ahead log, checkpointing, or **compaction**.¹

Một trong những bộ máy lưu trữ (storage engine) sử dụng sao-khi-ghi là Lightning Memory-Mapped Database (LMDB), một kho khóa-giá trị được dự án OpenLDAP sử dụng. Nhờ thiết kế và kiến trúc của mình, LMDB không cần page cache, nhật ký ghi-trước (WAL), điểm kiểm tra (checkpointing), hay nén-gộp (compaction).¹

LMDB is implemented as a single-level data store, which means that **read** and write operations are satisfied directly through the memory map, without additional application-level caching in between. This also means that pages require no additional materialization and reads can be served directly from the memory map without copying data to the intermediate buffer. During the update, every branch node on the path from the root to the target leaf is copied and potentially modified: nodes for which updates propagate are changed, and the rest of the nodes remain intact.

LMDB được hiện thực dưới dạng một kho dữ liệu một tầng, nghĩa là các thao tác đọc và ghi được phục vụ trực tiếp thông qua ánh xạ bộ nhớ (memory map), không có lớp đệm cấp ứng dụng nào ở giữa. Điều này cũng có nghĩa các trang không cần được vật chất hóa thêm và các lần đọc có thể được phục vụ trực tiếp từ ánh xạ bộ nhớ mà không cần sao chép dữ liệu sang vùng đệm trung gian. Trong quá trình cập nhật, mỗi nút nhánh trên đường đi từ gốc đến lá đích đều được sao chép và có thể bị sửa đổi: các nút mà bản cập nhật lan truyền tới sẽ bị thay đổi, còn các nút còn lại giữ nguyên.

LMDB holds only two versions of the root node: the latest version, and the one where new changes are going to be committed. This is sufficient since all writes have to go through the root node. After the new root is created, the old one becomes unavailable for new reads and writes. As soon as the reads referencing old tree sections complete, their pages are reclaimed and can be reused. Because

of LMDB's append-only design, it does not use sibling pointers and has to ascend back to the parent node during **sequential** scans.

LMDB chỉ giữ hai phiên bản của nút gốc: phiên bản mới nhất, và phiên bản nơi các thay đổi mới sẽ được commit. Như vậy là đủ vì mọi lần ghi đều phải đi qua nút gốc. Sau khi nút gốc mới được tạo, nút cũ trở nên không khả dụng cho các lần đọc và ghi mới. Ngay khi các lần đọc tham chiếu đến các phần cây cũ hoàn tất, các trang của chúng được thu hồi và có thể tái sử dụng. Do thiết kế chỉ-thêm của LMDB, nó không dùng con trỏ anh em (sibling) và phải đi ngược lên nút cha trong các lần quét tuần tự.

With this design, leaving stale data in copied nodes is impractical: there is already a copy that can be used for **MVCC** and satisfy ongoing read transactions. The database structure is inherently multiversioned, and readers can run without any locks as they do not interfere with writers in any way.

Với thiết kế này, việc để lại dữ liệu cũ trong các nút đã sao chép là không thực tế: đã có sẵn một bản sao có thể dùng cho MVCC và phục vụ các giao dịch đọc đang diễn ra. Cấu trúc cơ sở dữ liệu vốn dĩ đa phiên bản, và các bên đọc có thể chạy mà không cần khóa nào vì chúng không can thiệp vào các bên ghi theo bất kỳ cách nào.

Abstracting Node Updates — Trừu tượng hóa việc cập nhật nút

To update the page on disk, one way or the other, we have to first update its in-memory representation. However, there are a few ways to represent a node in memory: we can access the cached version of the node directly, do it through the wrapper object, or create its in-memory representation native to the implementation language.

Để cập nhật trang trên đĩa, dù bằng cách nào đi nữa, trước tiên chúng ta phải cập nhật biểu diễn của nó trong bộ nhớ. Tuy nhiên, có một vài cách để biểu diễn một nút trong bộ nhớ: chúng ta có thể truy cập trực tiếp phiên bản đã cache của nút, làm điều đó thông qua một đối tượng bao bọc (wrapper), hoặc tạo biểu diễn trong bộ nhớ của nó theo kiểu bản địa của ngôn ngữ hiện thực.

In languages with an unmanaged memory model, raw binary data stored in B-Tree nodes can be reinterpreted and native pointers can be used to manipulate it. In this case, the node is defined in terms of structures, which use raw binary data behind the pointer and runtime casts. Most often, they point to the memory area managed by the page cache or use memory mapping.

Trong các ngôn ngữ với mô hình bộ nhớ không quản lý, dữ liệu nhị phân thô được lưu trong các nút B-Tree có thể được diễn giải lại và các con trỏ bản địa có thể được dùng để thao tác trên nó. Trong trường hợp này, nút được định nghĩa thông qua các cấu trúc, vốn dùng dữ liệu nhị phân thô đằng sau con trỏ cùng các phép ép kiểu lúc chạy (runtime cast). Thường thì chúng trỏ tới vùng nhớ do page cache quản lý hoặc dùng ánh xạ bộ nhớ.

1 To learn more about LMDB, see the code comments and the presentation .

1 Để tìm hiểu thêm về LMDB, xem các chú thích trong mã nguồn và bài thuyết trình.

Alternatively, B-Tree nodes can be materialized into objects or structures native to the language. These structures can be used for inserts, updates, and deletes. During flush, changes are applied to pages in memory and, subsequently, on disk. This approach has the advantage of simplifying concurrent accesses since changes to underlying raw pages are managed separately from accesses to intermediate objects, but results in a higher memory overhead, since we have to store two versions (raw binary and language-native) of the same page in memory.

Một cách khác, các nút B-Tree có thể được vật chất hóa thành các đối tượng hoặc cấu trúc bản địa của ngôn ngữ. Các cấu trúc này có thể được dùng cho các thao tác thêm, cập nhật, và xóa. Trong quá trình flush, các thay đổi được áp dụng vào các trang trong bộ nhớ và sau đó xuống đĩa. Cách tiếp cận này có ưu điểm là đơn giản hóa các truy cập đồng thời vì các thay đổi vào các trang thô bên dưới được quản lý tách biệt với các truy cập vào các đối tượng trung gian, nhưng lại dẫn đến chi phí bộ nhớ cao hơn, vì chúng ta phải lưu hai phiên bản (nhị phân thô và bản địa của ngôn ngữ) của cùng một trang trong bộ nhớ.

The third approach is to provide access to the buffer backing the node through the wrapper object that materializes changes in the B-Tree as soon as they're performed. This approach is most often used in languages with a managed memory model. Wrapper objects apply the changes to the backing buffers.

Cách tiếp cận thứ ba là cung cấp quyền truy cập vào vùng đệm hậu thuẫn cho nút thông qua đối tượng bao bọc, đối tượng này vật chất hóa các thay đổi trong B-Tree ngay khi chúng được thực hiện. Cách tiếp cận này thường được dùng nhất trong các ngôn ngữ với mô hình bộ nhớ có quản lý. Các đối tượng bao bọc áp dụng các thay đổi vào các vùng đệm hậu thuẫn.

Managing on-disk pages, their cached versions, and their in-memory representations separately allows them to have different life cycles. For example, we can buffer insert, update, and delete operations, and reconcile changes made in memory with the original on-disk versions during reads.

Việc quản lý riêng biệt các trang trên đĩa, các phiên bản đã cache của chúng, và các biểu diễn trong bộ nhớ của chúng cho phép chúng có các vòng đời khác nhau. Ví dụ, chúng ta có thể đệm các thao tác thêm, cập nhật, và xóa, rồi đối chiếu các thay đổi đã thực hiện trong bộ nhớ với các phiên bản gốc trên đĩa trong lúc đọc.

Lazy B-Trees — B-Tree lười

Some algorithms (in the scope of this book, we call them lazy B-Trees 2) reduce costs of updating the B-Tree and use more lightweight, concurrency- and update-friendly in-memory structures to buffer updates and propagate them with a delay.

Một số thuật toán (trong phạm vi cuốn sách này, chúng ta gọi chúng là B-Tree lười²) giảm chi phí cập nhật B-Tree và sử dụng các cấu trúc trong bộ nhớ nhẹ hơn, thân thiện với tương tranh và cập nhật, để đệm các bản cập nhật và lan truyền chúng với độ trễ.

WiredTiger — WiredTiger

Let's take a look at how we can use buffering to implement a **lazy B-Tree**. For that, we can materialize B-Tree nodes in memory as soon as they are paged in and use this structure to store updates until we're ready to flush them.

Hãy xem cách chúng ta có thể dùng việc đệm để hiện thực một B-Tree lười. Để làm điều đó, chúng ta có thể vật chất hóa các nút B-Tree trong bộ nhớ ngay khi chúng được nạp trang vào và dùng cấu trúc này để lưu các bản cập nhật cho đến khi sẵn sàng flush chúng.

A similar approach is used by WiredTiger , a now-default MongoDB **storage engine**. Its row store B-Tree implementation uses different formats for in-memory and on-disk pages. Before in-memory pages are persisted, they have to go through the reconciliation **process**.

Một cách tiếp cận tương tự được WiredTiger sử dụng, nay là bộ máy lưu trữ mặc định của MongoDB. Cách hiện thực B-Tree dạng row store của nó dùng các định dạng khác nhau cho các trang trong bộ nhớ và trên đĩa. Trước khi các trang trong bộ nhớ được lưu bền, chúng phải trải qua quá trình đối chiếu (reconciliation).

2 This is not a commonly recognized name, but since the B-Tree variants we’re discussing here share one property—buffering B-Tree updates in intermediate structures instead of applying them to the tree directly—we’ll use the **term** lazy, which rather precisely defines this property.

2 Đây không phải là một tên được công nhận rộng rãi, nhưng vì các biến thể B-Tree chúng ta đang thảo luận ở đây có chung một đặc tính—đệm các bản cập nhật B-Tree trong các cấu trúc trung gian thay vì áp dụng trực tiếp vào cây—chúng ta sẽ dùng thuật ngữ lười (lazy), thuật ngữ này mô tả khá chính xác đặc tính đó.

In Figure 6-2, you can see a schematic representation of WiredTiger pages and their composition in a B-Tree. A clean page consists of just an index, initially constructed from the on-disk page image. Updates are first saved into the update buffer.

Trong Hình 6-2, bạn có thể thấy biểu diễn sơ đồ các trang của WiredTiger và cách chúng cấu thành trong một B-Tree. Một trang sạch chỉ gồm một chỉ mục, ban đầu được dựng từ ảnh trang trên đĩa. Các bản cập nhật trước tiên được lưu vào vùng đệm cập nhật (update buffer).

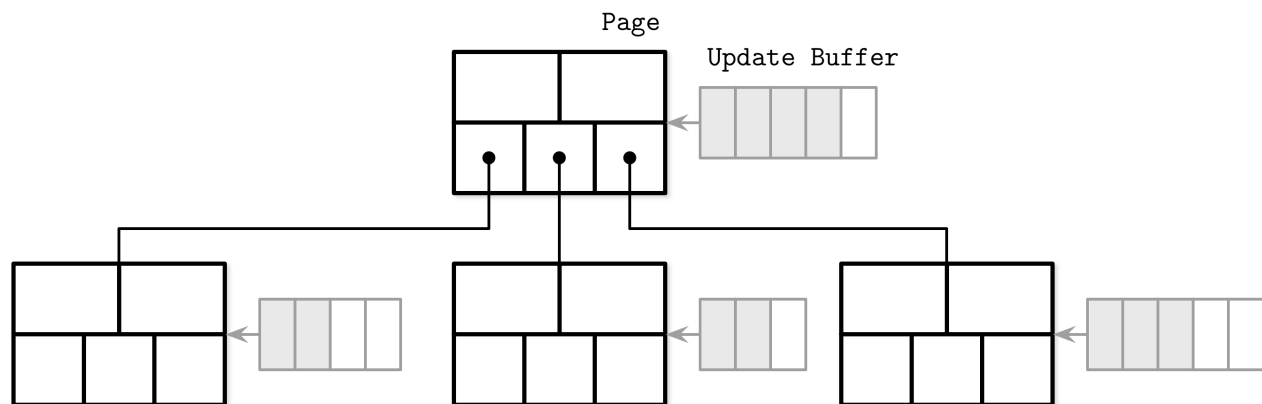


Figure 6-2. WiredTiger: high-level overview

Hình 6-2. WiredTiger: tổng quan ở mức cao

Update buffers are accessed during reads: their contents are merged with the original on-disk page contents to return the most recent data. When the page is flushed, update buffer contents are reconciled with page contents and persisted on disk, overwriting the original page. If the size of the reconciled page is greater than the maximum, it is split into multiple pages. Update buffers are implemented using skiplists, which have a complexity similar to search trees [PAPADAKIS93] but have a better concurrency profile [PUGH90a].

Các vùng đệm cập nhật được truy cập trong lúc đọc: nội dung của chúng được trộn với nội dung trang gốc trên đĩa để trả về dữ liệu mới nhất. Khi trang được flush, nội dung vùng đệm cập nhật được đối chiếu với nội dung trang và được lưu bền xuống đĩa, ghi đè lên trang gốc. Nếu kích thước trang đã đối chiếu lớn hơn mức tối đa, nó được tách thành nhiều trang. Các vùng đệm cập nhật được hiện thực bằng skiplist, vốn có độ phức tạp tương tự cây tìm kiếm [PAPADAKIS93] nhưng có đặc tính tương tranh tốt hơn [PUGH90a].

Figure 6-3 shows that both clean and dirty pages in WiredTiger have in-memory versions, and

reference a base image on disk. Dirty pages have an update buffer in addition to that.

Hình 6-3 cho thấy cả các trang sạch lẫn các trang bẩn trong WiredTiger đều có phiên bản trong bộ nhớ, và tham chiếu đến một ảnh nền trên đĩa. Các trang bẩn còn có thêm một vùng đệm cập nhật.

The main advantage here is that the page updates and structural modifications (splits and merges) are performed by the background thread, and read/write processes do not have to wait for them to complete.

Ưu điểm chính ở đây là các bản cập nhật trang và các sửa đổi cấu trúc (tách và gộp) được thực hiện bởi luồng nền, và các tiến trình đọc/ghi không phải chờ chúng hoàn tất.

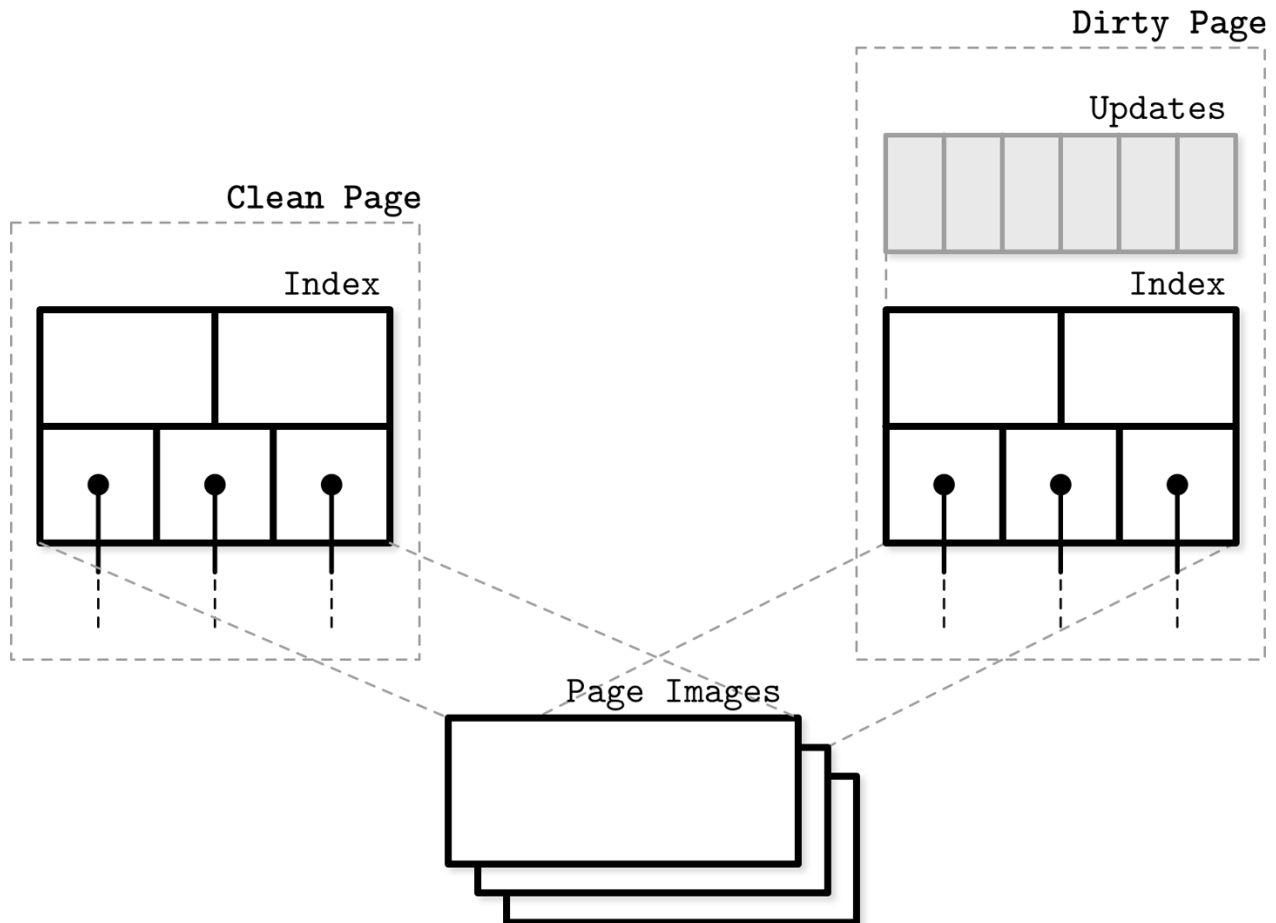


Figure 6-3. WiredTiger pages

Hình 6-3. Các trang của WiredTiger

Lazy-Adaptive Tree — Cây thích nghi-lười

Rather than buffering updates to individual nodes, we can group nodes into subtrees, and attach an update buffer for batching operations to each subtree. Update buffers in this case will **track** all operations performed against the subtree top node and its descendants. This algorithm is called Lazy-Adaptive Tree (LA-Tree) [AGRAWAL09].

Thay vì đệm các bản cập nhật cho từng nút riêng lẻ, chúng ta có thể nhóm các nút thành

các cây con, và gắn một vùng đệm cập nhật để gộp theo lô các thao tác cho từng cây con. Vùng đệm cập nhật trong trường hợp này sẽ theo dõi tất cả các thao tác được thực hiện trên nút đỉnh của cây con và các nút con cháu của nó. Thuật toán này được gọi là Cây thích nghi-lười (Lazy-Adaptive Tree, LA-Tree) [AGRAWAL09].

When inserting a data record, a new entry is first added to the root node update buffer. When this buffer becomes full, it is emptied by copying and propagating the changes to the buffers in the lower tree levels. This operation can continue recursively if the lower levels fill up as well, until it finally reaches the leaf nodes.

Khi chèn một bản ghi dữ liệu, một mục mới trước tiên được thêm vào vùng đệm cập nhật của nút gốc. Khi vùng đệm này đầy, nó được làm rỗng bằng cách sao chép và lan truyền các thay đổi tới các vùng đệm ở các tầng cây thấp hơn. Thao tác này có thể tiếp tục đệ quy nếu các tầng thấp hơn cũng đầy, cho đến khi cuối cùng đến các nút lá.

In Figure 6-4, you see an LA-Tree with cascaded buffers for nodes grouped in corresponding subtrees. Gray boxes represent changes that propagated from the root buffer.

Trong Hình 6-4, bạn thấy một LA-Tree với các vùng đệm xếp tầng cho các nút được nhóm trong các cây con tương ứng. Các hộp màu xám biểu diễn các thay đổi đã lan truyền từ vùng đệm gốc.

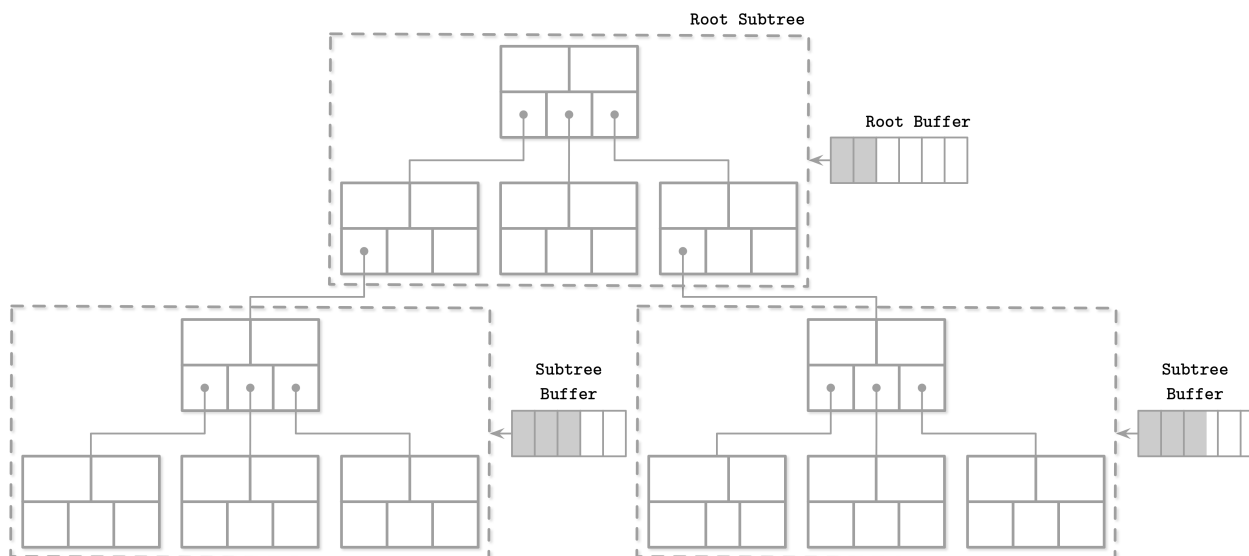


Figure 6-4. LA-Tree

Hình 6-4. LA-Tree

Buffers have hierarchical dependencies and are cascaded : all the updates propagate from higher-level buffers to the lower-level ones. When the updates reach the leaf level, batched insert, update, and delete operations are performed there, applying all changes to the tree contents and its structure at once. Instead of performing subsequent updates on pages separately, pages can be updated in a single run, requiring fewer disk accesses and structural changes, since splits and merges propagate to the higher levels in batches as well.

Các vùng đệm có các phụ thuộc phân cấp và được xếp tầng: tất cả các bản cập nhật lan truyền từ các vùng đệm tầng cao xuống các vùng đệm tầng thấp. Khi các bản cập nhật đến tầng lá, các thao tác thêm, cập nhật, và xóa gộp theo lô được thực hiện tại đó, áp dụng tất cả các thay đổi vào nội dung cây và cấu trúc của nó cùng một lúc. Thay vì thực hiện các

bản cập nhật kế tiếp lên từng trang riêng rẽ, các trang có thể được cập nhật trong một lượt duy nhất, đòi hỏi ít truy cập đĩa và ít sửa đổi cấu trúc hơn, vì các thao tác tách và gộp cũng lan truyền lên các tầng cao hơn theo lô.

The buffering approaches described here optimize tree update time by batching write operations, but in slightly different ways. Both algorithms require additional lookups in in-memory buffering structures and **merge/reconciliation** with stale disk data.

Các cách tiếp cận đệm được mô tả ở đây tối ưu thời gian cập nhật cây bằng cách gộp theo lô các thao tác ghi, nhưng theo những cách hơi khác nhau. Cả hai thuật toán đều đòi hỏi các lần tra cứu bổ sung trong các cấu trúc đệm trong bộ nhớ và việc trộn/đối chiếu với dữ liệu cũ trên đĩa.

FD-Trees — FD-Tree

Buffering is one of the ideas that is widely used in database storage: it helps to avoid many small random writes and performs a single larger write instead. On HDDs, random writes are slow because of the head positioning. On SSDs, there are no moving parts, but the extra write I/O imposes an additional **garbage collection** penalty.

Đệm là một trong những ý tưởng được dùng rộng rãi trong lưu trữ cơ sở dữ liệu: nó giúp tránh nhiều lần ghi ngẫu nhiên nhỏ và thay vào đó thực hiện một lần ghi lớn duy nhất. Trên HDD, ghi ngẫu nhiên chậm vì phải định vị đầu đọc. Trên SSD không có bộ phận chuyển động, nhưng I/O ghi thêm lại áp đặt một khoản phạt thu gom rác (garbage collection) bổ sung.

Maintaining a B-Tree requires a lot of random writes—leaf-level writes, splits, and merges propagating to the parents—but what if we could avoid random writes and node updates altogether?

Việc bảo trì một B-Tree đòi hỏi rất nhiều lần ghi ngẫu nhiên—các lần ghi ở tầng lá, các thao tác tách, và gộp lan truyền lên các nút cha—nhưng nếu chúng ta có thể tránh hoàn toàn các lần ghi ngẫu nhiên và các bản cập nhật nút thì sao?

So far we’ve discussed buffering updates to individual nodes or groups of nodes by creating auxiliary buffers. An alternative approach is to group updates targeting different nodes together by using append-only storage and merge processes, an idea that has also inspired LSM Trees (see “LSM Trees” on page 130). This means that any write we perform does not require locating a target node for the write: all updates are simply appended. One of the examples of using this approach for indexing is called Flash Disk Tree (**FD-Tree**) [LI10] .

Cho đến giờ, chúng ta đã thảo luận việc đệm các bản cập nhật cho từng nút riêng lẻ hoặc các nhóm nút bằng cách tạo các vùng đệm phụ trợ. Một cách tiếp cận khác là nhóm các bản cập nhật nhằm vào các nút khác nhau lại với nhau bằng cách dùng lưu trữ chỉ-thêm và các tiến trình trộn, một ý tưởng cũng đã truyền cảm hứng cho LSM Tree (xem “LSM Trees” ở trang 130). Điều này có nghĩa là bất kỳ lần ghi nào chúng ta thực hiện đều không cần định vị một nút đích cho lần ghi: tất cả các bản cập nhật đơn giản là được nối thêm. Một trong những ví dụ về việc dùng cách tiếp cận này cho lập chỉ mục được gọi là Flash Disk Tree (FD-Tree) [LI10].

An FD-Tree consists of a small mutable head tree and multiple immutable sorted runs. This approach limits the surface area, where random write I/O is required, to the head tree: a small B-Tree buffering the updates. As soon as the head tree fills up, its contents are transferred to the immutable run . If the size of the newly written run exceeds the threshold, its contents are merged with the next level, gradually propagating data records from upper to lower levels.

Một FD-Tree gồm một cây đầu nhỏ có thể thay đổi và nhiều run đã sắp xếp bất biến. Cách tiếp cận này giới hạn bề mặt cần đến I/O ghi ngẫu nhiên xuống còn cây đầu: một B-Tree nhỏ đệm các bản cập nhật. Ngay khi cây đầu đầy, nội dung của nó được chuyển sang run bất biến. Nếu kích thước của run vừa ghi vượt ngưỡng, nội dung của nó được trộn với tầng tiếp theo, dần dần lan truyền các bản ghi dữ liệu từ tầng trên xuống tầng dưới.

Fractional Cascading — Xếp tầng phân đoạn

To maintain pointers between the levels, FD-Trees use a technique called fractional cascading [CHAZELLE86]. This approach helps to reduce the cost of locating an item in the cascade of sorted arrays: you perform $\log n$ steps to find the searched item in the first array, but subsequent searches are significantly cheaper, since they start the search from the closest match from the previous level.

Để duy trì các con trỏ giữa các tầng, FD-Tree dùng một kỹ thuật gọi là xếp tầng phân đoạn (fractional cascading) [CHAZELLE86]. Cách tiếp cận này giúp giảm chi phí định vị một phần tử trong chuỗi xếp tầng các mảng đã sắp xếp: bạn thực hiện $\log n$ bước để tìm phần tử cần tìm trong mảng đầu tiên, nhưng các lần tìm kiếm tiếp theo rẻ hơn đáng kể, vì chúng bắt đầu tìm từ điểm khớp gần nhất từ tầng trước.

Shortcuts between the levels are made by building bridges between the neighbor-level arrays to minimize the gaps: element groups without pointers from higher levels. Bridges are built by pulling elements from lower levels to the higher ones, if they don't already exist there, and pointing to the location of the pulled element in the lower-level array.

Các lối tắt giữa các tầng được tạo bằng cách xây các cây cầu (bridge) giữa các mảng ở tầng kề nhau nhằm giảm thiểu các khoảng hở (gap): các nhóm phần tử không có con trỏ từ các tầng cao hơn. Các cây cầu được xây bằng cách kéo các phần tử từ tầng thấp lên tầng cao, nếu chúng chưa tồn tại ở đó, và trỏ tới vị trí của phần tử đã kéo trong mảng tầng thấp.

Since [CHAZELLE86] solves a search problem in computational geometry, it describes bidirectional bridges, and an algorithm for restoring the gap size invariant that we won't be covering here. We describe only the parts that are applicable to database storage and FD-Trees in particular.

Vì [CHAZELLE86] giải quyết một bài toán tìm kiếm trong hình học tính toán, nó mô tả các cây cầu hai chiều, và một thuật toán khôi phục bất biến kích thước khoảng hở mà chúng ta sẽ không đề cập ở đây. Chúng ta chỉ mô tả những phần áp dụng được cho lưu trữ cơ sở dữ liệu và FD-Tree nói riêng.

We could create a mapping from every element of the higher-level array to the closest element on the next level, but that would cause too much overhead for pointers and their maintenance. If we were to map only the items that already exist on a higher level, we could end up in a situation where the gaps between the elements are too large. To solve this problem, we pull every N th item from the lower-level array to the higher one.

Chúng ta có thể tạo một ánh xạ từ mọi phần tử của mảng tầng cao tới phần tử gần nhất ở tầng kế tiếp, nhưng điều đó sẽ gây ra quá nhiều phụ phí cho các con trỏ và việc bảo trì chúng. Nếu chúng ta chỉ ánh xạ những mục đã tồn tại ở tầng cao hơn, chúng ta có thể rơi vào tình huống các khoảng hở giữa các phần tử quá lớn. Để giải quyết vấn đề này, chúng ta kéo mỗi phần tử thứ N từ mảng tầng thấp lên tầng cao.

For example, if we have multiple sorted arrays:

Ví dụ, nếu chúng ta có nhiều mảng đã sắp xếp:

A1 = [12, 24, 32, 34, 39]
 A2 = [22, 25, 28, 30, 35]
 A3 = [11, 16, 24, 26, 30]

We can bridge the gaps between elements by pulling every other element from the array with a higher index to the one with a lower index in order to simplify searches:

Chúng ta có thể nối các khoảng hở giữa các phần tử bằng cách kéo cứ một phần tử cách một từ mảng có chỉ số cao hơn lên mảng có chỉ số thấp hơn để đơn giản hóa các lần tìm kiếm:

A1 = [12, 24, 25, 30, 32, 34, 39]
 A2 = [16, 22, 25, 26, 28, 30, 35]
 A3 = [11, 16, 24, 26, 30]

Now, we can use these pulled elements to create bridges (or fences as the FD-Tree paper calls them): pointers from higher-level elements to their counterparts on the lower levels, as Figure 6-5 shows.

Bây giờ, chúng ta có thể dùng các phần tử đã kéo này để tạo các cây cầu (hoặc hàng rào—fence—như bài báo FD-Tree gọi): các con trỏ từ các phần tử tầng cao tới các phần tử đối ứng của chúng ở các tầng thấp hơn, như Hình 6-5 minh họa.

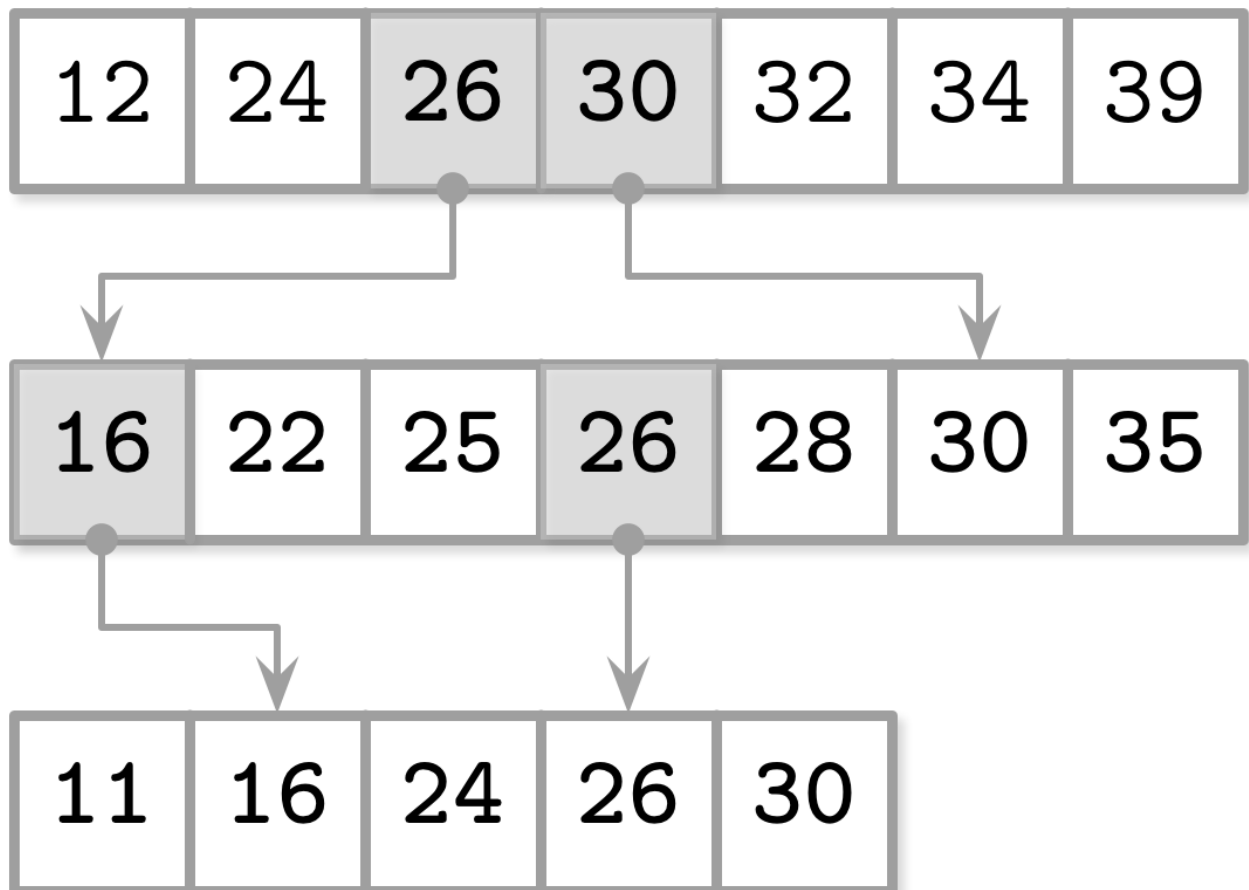


Figure 6-5. Fractional cascading

Hình 6-5. Xếp tầng phân đoạn

To search for elements in all these arrays, we perform a binary search on the highest level, and

the search space on the next level is reduced significantly, since now we are forwarded to the approximate location of the searched item by following a bridge. This allows us to connect multiple sorted runs and reduce the costs of searching in them.

Để tìm kiếm các phần tử trong tất cả các mảng này, chúng ta thực hiện một lần tìm kiếm nhị phân ở tầng cao nhất, và không gian tìm kiếm ở tầng kế tiếp được thu hẹp đáng kể, vì giờ đây chúng ta được chuyển tới vị trí gần đúng của phần tử cần tìm bằng cách đi theo một cây cầu. Điều này cho phép chúng ta kết nối nhiều run đã sắp xếp và giảm chi phí tìm kiếm trong chúng.

Logarithmic Runs — Các run logarit

An FD-Tree combines fractional cascading with creating logarithmically sized sorted runs : immutable sorted arrays with sizes increasing by a factor of k , created by merging the previous level with the current one.

Một FD-Tree kết hợp xếp tầng phân đoạn với việc tạo các run đã sắp xếp có kích thước theo cấp logarit: các mảng đã sắp xếp bất biến với kích thước tăng theo hệ số k , được tạo bằng cách trộn tầng trước với tầng hiện tại.

The highest-level run is created when the head tree becomes full: its leaf contents are written to the first level. As soon as the head tree fills up again, its contents are merged with the first-level items. The merged result replaces the old version of the first run. The lower-level runs are created when the sizes of the higher-level ones reach a threshold. If a lower-level run already exists, it is replaced by the result of merging its contents with the contents of a higher level. This process is quite similar to compaction in LSM Trees, where immutable table contents are merged to create larger tables.

Run ở tầng cao nhất được tạo khi cây đầu đầy: nội dung lá của nó được ghi xuống tầng đầu tiên. Ngay khi cây đầu lại đầy, nội dung của nó được trộn với các mục ở tầng đầu tiên. Kết quả đã trộn thay thế phiên bản cũ của run đầu tiên. Các run ở tầng thấp hơn được tạo khi kích thước của các run tầng cao hơn đạt ngưỡng. Nếu một run tầng thấp hơn đã tồn tại, nó được thay thế bởi kết quả trộn nội dung của nó với nội dung của tầng cao hơn. Quá trình này khá giống với nén-gộp trong LSM Tree, nơi nội dung các bảng bất biến được trộn để tạo các bảng lớn hơn.

Figure 6-6 shows a schematic representation of an FD-Tree, with a head B-Tree on the top, two logarithmic runs L1 and L2, and bridges between them.

Hình 6-6 cho thấy biểu diễn sơ đồ của một FD-Tree, với một B-Tree đầu ở trên cùng, hai run logarit L1 và L2, cùng các cây cầu giữa chúng.

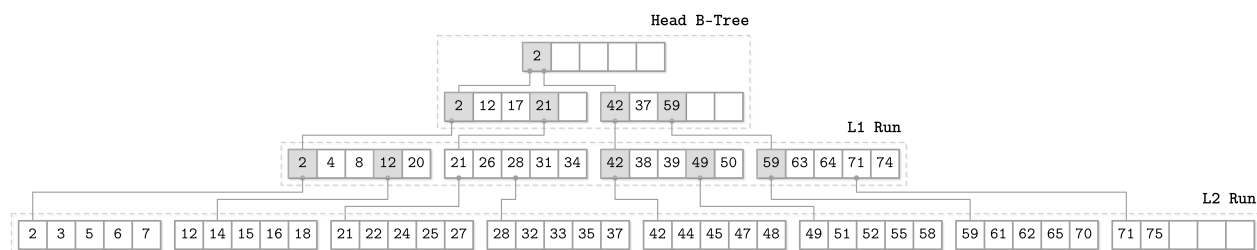


Figure 6-6. Schematic FD-Tree overview

Hình 6-6. Tổng quan sơ đồ FD-Tree

To keep items in all sorted runs addressable, FD-Trees use an adapted version of fractional cascading, where head elements from lower-level pages are propagated as pointers to the higher levels. Using these pointers, the cost of searching in lower-level trees is reduced, since the search was already partially done on a higher level and can continue from the closest match.

Để giữ các mục trong tất cả các run đã sắp xếp luôn định địa chỉ được, FD-Tree dùng một phiên bản thích nghi của xếp tầng phân đoạn, trong đó các phần tử đầu từ các trang tầng thấp được lan truyền dưới dạng con trỏ lên các tầng cao hơn. Dùng các con trỏ này, chi phí tìm kiếm trong các cây tầng thấp được giảm, vì việc tìm kiếm đã được thực hiện một phần ở tầng cao hơn và có thể tiếp tục từ điểm khớp gần nhất.

Since FD-Trees do not update pages in place, and it may happen that data records for the same key are present on several levels, the FD-Trees delete work by inserting tombstones (the FD-Tree paper calls them filter entries) that indicate that the data record associated with a corresponding key is marked for deletion, and all data records for that key in the lower levels have to be discarded. When tombstones propagate all the way to the lowest level, they can be discarded, since it is guaranteed that there are no items they can shadow anymore.

Vì FD-Tree không cập nhật các trang tại chỗ, và có thể xảy ra việc các bản ghi dữ liệu cho cùng một khóa hiện diện ở nhiều tầng, nên FD-Tree thực hiện xóa bằng cách chèn các bia mộ (tombstone) (bài báo FD-Tree gọi chúng là mục lọc—filter entry) chỉ ra rằng bản ghi dữ liệu gắn với khóa tương ứng được đánh dấu để xóa, và tất cả các bản ghi dữ liệu cho khóa đó ở các tầng thấp hơn phải bị loại bỏ. Khi các bia mộ lan truyền tới tận tầng thấp nhất, chúng có thể bị loại bỏ, vì được bảo đảm rằng không còn mục nào chúng có thể che khuất nữa.

Bw-Trees — Bw-Tree

Write amplification is one of the most significant problems with in-place update implementations of B-Trees: subsequent updates to a B-Tree page may require updating a disk-resident page copy on every update. The second problem is **space amplification**: we reserve extra space to make updates possible. This also means that for each transferred useful byte carrying the requested data, we have to transfer some empty bytes and the rest of the page. The third problem is complexity in solving concurrency problems and dealing with latches.

Khuếch đại ghi (write amplification) là một trong những vấn đề đáng kể nhất với các cách hiện thực B-Tree cập nhật tại chỗ: các lần cập nhật kế tiếp vào một trang B-Tree có thể đòi hỏi cập nhật một bản sao trang nằm trên đĩa ở mỗi lần cập nhật. Vấn đề thứ hai là khuếch đại không gian (space amplification): chúng ta dành thêm không gian để làm cho các bản cập nhật khả thi. Điều này cũng có nghĩa là với mỗi byte hữu ích được chuyển mang dữ liệu được yêu cầu, chúng ta phải chuyển một số byte rỗng cùng phần còn lại của trang. Vấn đề thứ ba là sự phức tạp trong việc giải quyết các vấn đề tương tranh và xử lý các chốt (latch).

To solve all three problems at once, we have to take an approach entirely different from the ones we've discussed so far. Buffering updates helps with write and space amplification, but offers no solution to concurrency issues.

Để giải quyết cả ba vấn đề cùng một lúc, chúng ta phải áp dụng một cách tiếp cận hoàn toàn khác với những cách đã thảo luận từ trước đến giờ. Việc đệm các bản cập nhật giúp ích cho khuếch đại ghi và khuếch đại không gian, nhưng không đưa ra giải pháp nào cho các vấn đề tương tranh.

We can batch updates to different nodes by using append-only storage, **link** nodes together into chains, and use an in-memory data structure that allows installing pointers between the nodes with a single compare-and-swap operation, making the tree **lock**-free. This approach is called a Buzzword-Tree (**Bw-Tree**) [LEVAN- DOSKI14] .

Chúng ta có thể gộp theo lô các bản cập nhật vào các nút khác nhau bằng cách dùng lưu trữ chỉ-thêm, liên kết các nút lại với nhau thành các chuỗi, và dùng một cấu trúc dữ liệu trong bộ nhớ cho phép cài đặt các con trỏ giữa các nút bằng một thao tác so-sánh-và-hoán-đổi (compare-and-swap) duy nhất, làm cho cây không cần khóa. Cách tiếp cận này được gọi là Buzzword-Tree (Bw-Tree) [LEVANDOSKI14].

Update Chains — Các chuỗi cập nhật

A Bw-Tree writes a base node separately from its modifications. Modifications (delta nodes) form a chain: a linked list from the newest modification, through older ones, with the base node in the end. Each update can be stored separately, without needing to rewrite the existing node on disk. Delta nodes can represent inserts, updates (which are indistinguishable from inserts), or deletes.

Một Bw-Tree ghi một nút cơ sở (base node) tách biệt với các sửa đổi của nó. Các sửa đổi (nút delta) tạo thành một chuỗi: một danh sách liên kết từ sửa đổi mới nhất, qua các sửa đổi cũ hơn, với nút cơ sở ở cuối. Mỗi bản cập nhật có thể được lưu riêng, mà không cần ghi lại nút hiện có trên đĩa. Các nút delta có thể biểu diễn các thao tác thêm, cập nhật (vốn không phân biệt được với thêm), hoặc xóa.

Since the sizes of base and delta nodes are unlikely to be page aligned, it makes sense to store them contiguously, and because neither base nor delta nodes are modified during update (all modifications just prepend a node to the existing linked list), we do not need to reserve any extra space.

Vì kích thước của các nút cơ sở và nút delta khó có thể căn theo trang, việc lưu chúng liên kề nhau là hợp lý, và vì cả nút cơ sở lẫn nút delta đều không bị sửa đổi trong lúc cập nhật (mọi sửa đổi chỉ nối thêm một nút vào đầu danh sách liên kết hiện có), chúng ta không cần dành thêm bất kỳ không gian dư nào.

Having a node as a logical, rather than physical, entity is an interesting paradigm change: we do not need to pre-allocate space, require nodes to have a fixed size, or even keep them in contiguous memory segments. This certainly has a downside: during a read, all deltas have to be traversed and applied to the base node to reconstruct the actual node state. This is somewhat similar to what LA-Trees do (see “Lazy- Adaptive Tree” on page 116): keeping updates separate from the main structure and replaying them on read.

Việc xem một nút như một thực thể logic, thay vì vật lý, là một sự thay đổi mô hình thú vị: chúng ta không cần cấp phát trước không gian, không yêu cầu các nút có kích thước cố định, hay thậm chí không cần giữ chúng trong các phân đoạn bộ nhớ liên kề. Điều này chắc chắn có một nhược điểm: trong lúc đọc, tất cả các delta phải được duyệt qua và áp dụng vào nút cơ sở để tái dựng trạng thái nút thực sự. Điều này phần nào tương tự với những gì LA-Tree làm (xem “Lazy-Adaptive Tree” ở trang 116): giữ các bản cập nhật tách biệt với cấu trúc chính và phát lại chúng khi đọc.

Taming Concurrency with Compare-and-Swap — Thuận hóa tương tranh bằng So-sánh-và-Hoán-đổi

It would be quite costly to maintain an on-disk tree structure that allows prepending items to child nodes: it would require us to constantly update parent nodes with pointers to the freshest delta. This is why Bw-Tree nodes, consisting of a chain of deltas and the base node, have logical identifiers and use an in-memory mapping table from the identifiers to their locations on disk. Using this mapping also helps us to get rid of latches: instead of having exclusive ownership during write time, the Bw-Tree uses compare-and-swap operations on physical offsets in the mapping table.

Sẽ khá tốn kém nếu duy trì một cấu trúc cây trên đĩa cho phép nối thêm các mục vào đầu các nút con: nó sẽ buộc chúng ta liên tục cập nhật các nút cha với con trỏ tới delta mới nhất. Đây là lý do các nút Bw-Tree, gồm một chuỗi delta và nút cơ sở, có các định danh logic và dùng một bảng ánh xạ trong bộ nhớ từ các định danh tới vị trí của chúng trên đĩa. Việc dùng ánh xạ này cũng giúp chúng ta loại bỏ các chốt: thay vì có quyền sở hữu độc quyền trong lúc ghi, Bw-Tree dùng các thao tác so-sánh-và-hoán-đổi trên các độ dời vật lý trong bảng ánh xạ.

Figure 6-7 shows a simple Bw-Tree. Each logical node consists of a single base node and multiple linked delta nodes.

Hình 6-7 cho thấy một Bw-Tree đơn giản. Mỗi nút logic gồm một nút cơ sở duy nhất và nhiều nút delta được liên kết.

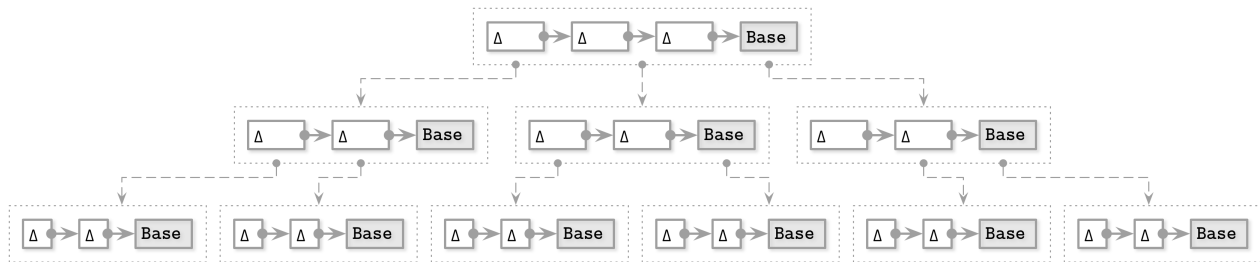


Figure 6-7. Bw-Tree. Dotted lines represent virtual links between the nodes, resolved using the mapping table. Solid lines represent actual data pointers between the nodes.

Hình 6-7. Bw-Tree. Các đường nét đứt biểu diễn các liên kết ảo giữa các nút, được phân giải bằng bảng ánh xạ. Các đường nét liền biểu diễn các con trỏ dữ liệu thực sự giữa các nút.

To update a Bw-Tree node, the algorithm executes the following steps:

Để cập nhật một nút Bw-Tree, thuật toán thực hiện các bước sau:

1. The target logical leaf node is located by traversing the tree from root to leaf. The mapping table contains virtual links to target base nodes or the latest delta nodes in the update chain.
2. A new delta node is created with a pointer to the base node (or to the latest delta node) located during step 1.
3. The mapping table is updated with a pointer to the new delta node created during step 2.
 1. Nút lá logic đích được định vị bằng cách duyệt cây từ gốc đến lá. Bảng ánh xạ chứa các liên kết ảo tới các nút cơ sở đích hoặc các nút delta mới nhất trong chuỗi cập nhật.
 2. Một nút delta mới được tạo với một con trỏ tới nút cơ sở (hoặc tới nút delta mới nhất) đã định vị trong bước 1.
 3. Bảng ánh xạ được cập nhật với một con trỏ tới nút delta mới được tạo trong bước 2.

An update operation during step 3 can be done using compare-and-swap, which is an atomic operation, so all reads, concurrent to the pointer update, are ordered either before or after the write, without blocking either the readers or the writer. Reads ordered before follow the old pointer and do not see the new delta node, since it was not yet installed. Reads ordered after follow the new pointer, and observe the update. If two threads attempt to install a new delta node to the same logical node, only one of them can succeed, and the other one has to retry the operation.

Một thao tác cập nhật trong bước 3 có thể được thực hiện bằng so-sánh-và-hoán-đổi, vốn là một thao tác nguyên tử, nên tất cả các lần đọc diễn ra đồng thời với việc cập nhật con trỏ đều được sắp thứ tự hoặc trước hoặc sau lần ghi, mà không chặn cả bên đọc lẫn bên ghi. Các lần đọc được sắp trước đi theo con trỏ cũ và không thấy nút delta mới, vì nó chưa được cài đặt. Các lần đọc được sắp sau đi theo con trỏ mới, và quan sát thấy bản cập nhật. Nếu hai luồng cố gắng cài đặt một nút delta mới vào cùng một nút logic, chỉ một trong số chúng có thể thành công, và luồng còn lại phải thử lại thao tác.

Structural Modification Operations — Các thao tác sửa đổi cấu trúc

A Bw-Tree is logically structured like a B-Tree, which means that nodes still might grow to be too large (overflow) or shrink to be almost empty (underflow) and require structure modification operations (SMOs), such as splits and merges. The semantics of splits and merges here are similar to those of B-Trees (see “B-Tree Node Splits” on page 39 and “B-Tree Node Merges” on page 41), but their implementation is different.

Một Bw-Tree về mặt logic được tổ chức giống một B-Tree, nghĩa là các nút vẫn có thể phình ra quá lớn (tràn) hoặc co lại gần như rỗng (thiếu hụt) và đòi hỏi các thao tác sửa đổi cấu trúc (structure modification operations, SMO), như tách và gộp. Ngữ nghĩa của tách và gộp ở đây tương tự như của B-Tree (xem “B-Tree Node Splits” ở trang 39 và “B-Tree Node Merges” ở trang 41), nhưng cách hiện thực của chúng thì khác.

Split SMOs start by consolidating the logical contents of the splitting node, applying deltas to its base node, and creating a new page containing elements to the right of the split point. After this, the process proceeds in two steps [WANG18] :

Các SMO tách bắt đầu bằng việc hợp nhất nội dung logic của nút đang tách, áp dụng các delta vào nút cơ sở của nó, và tạo một trang mới chứa các phần tử nằm bên phải điểm tách. Sau đó, tiến trình diễn ra theo hai bước [WANG18]:

1. Split —A special split delta node is appended to the splitting node to notify the readers about the ongoing split. The split delta node holds a midpoint **separator key** to invalidate records in the splitting node, and a link to the new logical sibling node.
 2. Parent update —At this point, the situation is similar to that of the B link -Tree half-split (see “Blink-Trees” on page 107), since the node is available through the split delta node pointer, but is not yet referenced by the parent, and readers have to go through the old node and then traverse the **sibling pointer** to reach the newly created sibling node. A new node is added as a child to the parent node, so that readers can directly reach it instead of being redirected through the splitting node, and the split completes.
1. Tách (Split)—Một nút delta tách đặc biệt được nối thêm vào nút đang tách để thông báo cho các bên đọc về việc tách đang diễn ra. Nút delta tách giữ một khóa phân tách (separator key) ở điểm giữa để vô hiệu hóa các bản ghi trong nút đang tách, và một liên kết tới nút anh em logic mới.
 2. Cập nhật cha (Parent update)—Tại thời điểm này,

tình huống tương tự với nửa-tách của B link-Tree (xem “Blink-Trees” ở trang 107), vì nút khả dụng thông qua con trỏ nút delta tách, nhưng chưa được nút cha tham chiếu, và các bên đọc phải đi qua nút cũ rồi duyệt con trỏ anh em để đến nút anh em mới được tạo. Một nút mới được thêm làm con của nút cha, để các bên đọc có thể đến nó trực tiếp thay vì bị chuyển hướng qua nút đang tách, và việc tách hoàn tất.

Updating the parent pointer is a performance optimization: all nodes and their elements remain accessible even if the parent pointer is never updated. Bw-Trees are **latch**-free, so any thread can encounter an incomplete SMO. The thread is required to cooperate by picking up and finishing a multistep SMO before proceeding. The next thread will follow the installed parent pointer and won't have to go through the sibling pointer.

Việc cập nhật con trỏ cha là một tối ưu hiệu năng: tất cả các nút và các phần tử của chúng vẫn truy cập được ngay cả khi con trỏ cha không bao giờ được cập nhật. Bw-Tree không cần chốt, nên bất kỳ luồng nào cũng có thể gặp một SMO chưa hoàn chỉnh. Luồng đó bắt buộc phải hợp tác bằng cách tiếp nhận và hoàn tất một SMO nhiều bước trước khi tiếp tục. Luồng tiếp theo sẽ đi theo con trỏ cha đã cài đặt và sẽ không phải đi qua con trỏ anh em.

Merge SMOs work in a similar way:

Các SMO gộp hoạt động theo cách tương tự:

1. Remove sibling —A special remove delta node is created and appended to the right sibling, indicating the start of the merge SMO and marking the right sibling for deletion.
2. Merge —A merge delta node is created on the left sibling to point to the contents of the right sibling and making it a logical part of the left sibling.
3. Parent update —At that point, the right sibling node contents are accessible from the left one. To finish the merge process, the link to the right sibling has to be removed from the parent.

1. Loại bỏ anh em (Remove sibling)—Một nút delta loại bỏ đặc biệt được tạo và nối thêm vào nút anh em bên phải, báo hiệu việc bắt đầu SMO gộp và đánh dấu nút anh em bên phải để xóa.
2. Gộp (Merge)—Một nút delta gộp được tạo trên nút anh em bên trái để trỏ tới nội dung của nút anh em bên phải và biến nó thành một phần logic của nút anh em bên trái.
3. Cập nhật cha (Parent update)—Tại thời điểm đó, nội dung nút anh em bên phải truy cập được từ nút bên trái. Để hoàn tất quá trình gộp, liên kết tới nút anh em bên phải phải bị loại khỏi nút cha.

Concurrent SMOs require an additional abort delta node to be installed on the parent to prevent concurrent splits and merges [WANG18]. An abort delta works similarly to a write lock: only one thread can have write access at a time, and any thread that attempts to append a new record to this delta node will abort. On SMO completion, the abort delta can be removed from the parent.

Các SMO đồng thời đòi hỏi cài đặt thêm một nút delta hủy bỏ (abort delta) trên nút cha để ngăn các thao tác tách và gộp đồng thời [WANG18]. Một delta hủy bỏ hoạt động tương tự một khóa ghi: chỉ một luồng có thể có quyền truy cập ghi tại một thời điểm, và bất kỳ luồng nào cố gắng nối thêm một bản ghi mới vào nút delta này sẽ bị hủy bỏ. Khi SMO hoàn tất, delta hủy bỏ có thể được loại khỏi nút cha.

The Bw-Tree height grows during the root node splits. When the root node gets too big, it is split in two, and a new root is created in place of the old one, with the old root and a newly created sibling as its children.

Chiều cao Bw-Tree tăng lên trong các lần tách nút gốc. Khi nút gốc trở nên quá lớn, nó được tách làm đôi, và một nút gốc mới được tạo thay cho nút cũ, với nút gốc cũ và một nút anh em mới được tạo làm các con của nó.

Consolidation and Garbage Collection — Hợp nhất và Thu gom rác

Delta chains can get arbitrarily long without any additional action. Since reads are getting more expensive as the delta chain gets longer, we need to try to keep the delta chain length within reasonable bounds. When it reaches a configurable threshold, we rebuild the node by merging the base node contents with all of the deltas, consolidating them to one new base node. The new node is then written to the new location on disk and the node pointer in the mapping table is updated to point to it. We discuss this process in more detail in “LLAMA and Mindful Stacking” on page 160 , as the underlying **log-structured storage** is responsible for garbage collection, node consolidation, and relocation.

Các chuỗi delta có thể dài tùy ý mà không cần bất kỳ hành động bổ sung nào. Vì các lần đọc trở nên đắt đỏ hơn khi chuỗi delta dài ra, chúng ta cần cố gắng giữ độ dài chuỗi delta trong giới hạn hợp lý. Khi nó đạt một ngưỡng có thể cấu hình, chúng ta xây dựng lại nút bằng cách trộn nội dung nút cơ sở với tất cả các delta, hợp nhất chúng thành một nút cơ sở mới duy nhất. Nút mới sau đó được ghi vào vị trí mới trên đĩa và con trỏ nút trong bảng ánh xạ được cập nhật để trỏ tới nó. Chúng ta thảo luận quá trình này chi tiết hơn trong “LLAMA and Mindful Stacking” ở trang 160, vì lưu trữ dạng nhật ký (log-structured) bên dưới chịu trách nhiệm thu gom rác, hợp nhất nút, và tái định vị.

As soon as the node is consolidated, its old contents (the base node and all of the delta nodes) are no longer addressed from the mapping table. However, we cannot free the memory they occupy right away, because some of them might be still used by ongoing operations. Since there are no latches held by readers (readers did not have to pass through or register at any sort of barrier to access the node), we need to find other means to track live pages.

Ngay khi nút được hợp nhất, nội dung cũ của nó (nút cơ sở và tất cả các nút delta) không còn được định địa chỉ từ bảng ánh xạ nữa. Tuy nhiên, chúng ta không thể giải phóng vùng nhớ chúng chiếm dụng ngay lập tức, vì một số trong chúng có thể vẫn đang được các thao tác đang diễn ra sử dụng. Vì không có chốt nào do các bên đọc nắm giữ (các bên đọc không phải đi qua hay đăng ký tại bất kỳ loại rào chắn nào để truy cập nút), chúng ta cần tìm các phương tiện khác để theo dõi các trang còn sống.

To separate threads that might have encountered a specific node from those that couldn't have possibly seen it, Bw-Trees use a technique known as epoch-based reclamation . If some nodes and deltas are removed from the mapping table due to consolidations that replaced them during some epoch, original nodes are preserved until every reader that started during the same epoch or the earlier one is finished. After that, they can be safely garbage collected, since later readers are guaranteed to have never seen those nodes, as they were not addressable by the time those readers started.

Để tách các luồng có thể đã gặp một nút cụ thể khỏi những luồng không thể nào nhìn thấy nó, Bw-Tree dùng một kỹ thuật gọi là thu hồi dựa trên kỷ nguyên (epoch-based reclamation). Nếu một số nút và delta bị loại khỏi bảng ánh xạ do các lần hợp nhất đã thay thế chúng trong một kỷ nguyên nào đó, các nút gốc được bảo tồn cho đến khi mọi bên đọc khởi động trong cùng kỷ nguyên đó hoặc kỷ nguyên trước đó kết thúc. Sau đó, chúng có thể được thu gom rác một cách an toàn, vì các bên đọc về sau được bảo đảm chưa bao giờ nhìn thấy các nút đó, do chúng không còn định địa chỉ được vào thời điểm các bên đọc đó khởi động.

The Bw-Tree is an interesting B-Tree variant, making improvements on several important aspects: write amplification, nonblocking access, and cache friendliness. A modified version

was implemented in Sled, an experimental storage engine. The CMU Database Group has developed an in-memory version of the Bw-Tree called OpenBw-Tree and released a practical implementation guide [WANG18].

Bw-Tree là một biến thể B-Tree thú vị, đem lại cải tiến ở vài khía cạnh quan trọng: khuếch đại ghi, truy cập không chặn, và thân thiện với cache. Một phiên bản đã sửa đổi được hiện thực trong Sled, một bộ máy lưu trữ thử nghiệm. Nhóm Database Group của CMU đã phát triển một phiên bản trong bộ nhớ của Bw-Tree gọi là OpenBw-Tree và phát hành một hướng dẫn hiện thực thực tế [WANG18].

We’ve only touched on higher-level Bw-Tree concepts related to B-Trees in this chapter, and we continue the discussion about them in “LLAMA and Mindful Stacking” on page 160, including the discussion about the underlying log-structured storage.

Trong chương này, chúng ta mới chỉ chạm tới các khái niệm Bw-Tree ở mức cao liên quan đến B-Tree, và chúng ta tiếp tục thảo luận về chúng trong “LLAMA and Mindful Stacking” ở trang 160, bao gồm cả thảo luận về lưu trữ dạng nhật ký bên dưới.

Cache-Oblivious B-Trees — B-Tree không phụ thuộc bộ nhớ đệm

Block size, node size, cache line alignments, and other configurable parameters influence B-Tree performance. A new class of data structures called cache-oblivious structures [DEMAINE02] give asymptotically optimal performance regardless of the underlying memory hierarchy and a need to tune these parameters. This means that the algorithm is not required to know the sizes of the cache lines, filesystem blocks, and disk pages. Cache-oblivious structures are designed to perform well without modification on multiple machines with different configurations.

Kích thước khối, kích thước nút, căn lề dòng cache, và các tham số có thể cấu hình khác đều ảnh hưởng đến hiệu năng B-Tree. Một lớp cấu trúc dữ liệu mới gọi là các cấu trúc không phụ thuộc bộ nhớ đệm (cache-oblivious structure) [DEMAINE02] mang lại hiệu năng tối ưu tiệm cận bất kể hệ phân cấp bộ nhớ bên dưới và bất kể nhu cầu tinh chỉnh các tham số này. Điều này có nghĩa là thuật toán không cần biết kích thước của các dòng cache, các khối hệ thống tệp, và các trang đĩa. Các cấu trúc không phụ thuộc bộ nhớ đệm được thiết kế để hoạt động tốt mà không cần sửa đổi trên nhiều máy với các cấu hình khác nhau.

So far, we’ve been mostly looking at B-Trees from a two-level memory hierarchy (with the exception of LMDB described in “Copy-on-Write” on page 112). B-Tree nodes are stored in disk-resident pages, and the page cache is used to allow efficient access to them in main memory.

Cho đến giờ, chúng ta hầu như xem xét B-Tree từ một hệ phân cấp bộ nhớ hai tầng (ngoại trừ LMDB được mô tả trong “Copy-on-Write” ở trang 112). Các nút B-Tree được lưu trong các trang nằm trên đĩa, và page cache được dùng để cho phép truy cập hiệu quả tới chúng trong bộ nhớ chính.

The two levels of this hierarchy are page cache (which is faster, but is limited in space) and disk (which is generally slower, but has a larger capacity) [AGGARWAL88]. Here, we have only two parameters, which makes it rather easy to design algorithms as we only have to have two level-specific code modules that take care of all the details relevant to that level.

Hai tầng của hệ phân cấp này là page cache (nhanh hơn, nhưng giới hạn về không gian) và đĩa (nhìn chung chậm hơn, nhưng có dung lượng lớn hơn) [AGGARWAL88]. Ở đây, chúng ta chỉ có hai tham số, điều này khiến việc thiết kế các thuật toán khá dễ dàng vì chúng ta chỉ phải có hai mô-đun mã đặc thù cho từng tầng, lo liệu mọi chi tiết liên quan đến tầng đó.

The disk is partitioned into blocks, and data is transferred between disk and cache in blocks: even when the algorithm has to locate a single item within the block, an entire block has to be loaded. This approach is cache-aware .

Đĩa được phân chia thành các khối, và dữ liệu được chuyển giữa đĩa và cache theo khối: ngay cả khi thuật toán phải định vị một mục đơn lẻ bên trong khối, toàn bộ khối vẫn phải được nạp. Cách tiếp cận này là phụ thuộc bộ nhớ đệm (cache-aware).

When developing performance-critical software, we often program for a more complex model, taking into consideration CPU caches, and sometimes even disk hierarchies (like hot/cold storage or build **HDD/SSD/NVM** hierarchies, and phase off data from one level to the other). Most of the time such efforts are difficult to generalize. In “Memory- Versus Disk-Based **DBMS**” on page 10 , we talked about the fact that accessing disk is several orders of magnitude slower than accessing main memory, which has motivated database implementers to optimize for this difference.

Khi phát triển phần mềm chú trọng hiệu năng, chúng ta thường lập trình cho một mô hình phức tạp hơn, có tính đến các cache CPU, và đôi khi cả các hệ phân cấp đĩa (như lưu trữ nóng/lạnh hoặc xây các hệ phân cấp HDD/SSD/NVM, và dồn dữ liệu từ tầng này sang tầng khác). Hầu hết thời gian, những nỗ lực như vậy khó tổng quát hóa. Trong “Memory- Versus Disk-Based DBMS” ở trang 10, chúng ta đã nói về việc truy cập đĩa chậm hơn truy cập bộ nhớ chính vài bậc độ lớn, điều này đã thúc đẩy các nhà hiện thực cơ sở dữ liệu tối ưu cho sự khác biệt này.

Cache-oblivious algorithms allow reasoning about data structures in terms of a two-level memory model while providing the benefits of a multilevel hierarchy model. This approach allows having no platform-specific parameters, yet guarantees that the number of transfers between the two levels of the hierarchy is within a constant factor. If the data structure is optimized to perform optimally for any two levels of memory hierarchy, it also works optimally for the two adjacent hierarchy levels. This is achieved by working at the highest cache level as much as possible.

Các thuật toán không phụ thuộc bộ nhớ đệm cho phép suy luận về các cấu trúc dữ liệu theo một mô hình bộ nhớ hai tầng trong khi vẫn mang lại lợi ích của một mô hình phân cấp nhiều tầng. Cách tiếp cận này cho phép không có tham số đặc thù nền tảng nào, nhưng vẫn bảo đảm rằng số lần chuyển giữa hai tầng của hệ phân cấp nằm trong một hệ số hằng. Nếu cấu trúc dữ liệu được tối ưu để hoạt động tối ưu cho bất kỳ hai tầng nào của hệ phân cấp bộ nhớ, nó cũng hoạt động tối ưu cho hai tầng phân cấp kề nhau. Điều này đạt được bằng cách làm việc ở tầng cache cao nhất càng nhiều càng tốt.

van Emde Boas Layout — Bố cục van Emde Boas

A **cache-oblivious B-Tree** consists of a static B-Tree and a packed array structure [BENDER05] . A static B-Tree is built using the van Emde Boas layout. It splits the tree at the middle level of the edges. Then each subtree is split recursively in a similar manner, resulting in subtrees of $\sqrt{N}$ size. The key idea of this layout is that any recursive tree is stored in a contiguous block of memory.

Một B-Tree không phụ thuộc bộ nhớ đệm gồm một B-Tree tĩnh và một cấu trúc mảng nén (packed array) [BENDER05]. Một B-Tree tĩnh được xây dựng dùng bố cục van Emde Boas. Nó tách cây tại tầng giữa của các cạnh. Sau đó mỗi cây con được tách đệ quy theo cách tương tự, cho ra các cây con kích thước $\sqrt{N}$. Ý tưởng cốt lõi của bố cục này là bất kỳ cây đệ quy nào cũng được lưu trong một khối bộ nhớ liền kề.

In Figure 6-8 , you can see an example of a van Emde Boas layout. Nodes, logically grouped together, are placed closely together. On top, you can see a logical layout representation (i.e., how nodes form

a tree), and on the bottom you can see how tree nodes are laid out in memory and on disk.

Trong Hình 6-8, bạn có thể thấy một ví dụ về bố cục van Emde Boas. Các nút, được nhóm với nhau về mặt logic, được đặt gần nhau. Ở trên cùng, bạn có thể thấy biểu diễn bố cục logic (tức là cách các nút tạo thành một cây), và ở dưới cùng bạn có thể thấy cách các nút cây được sắp đặt trong bộ nhớ và trên đĩa.

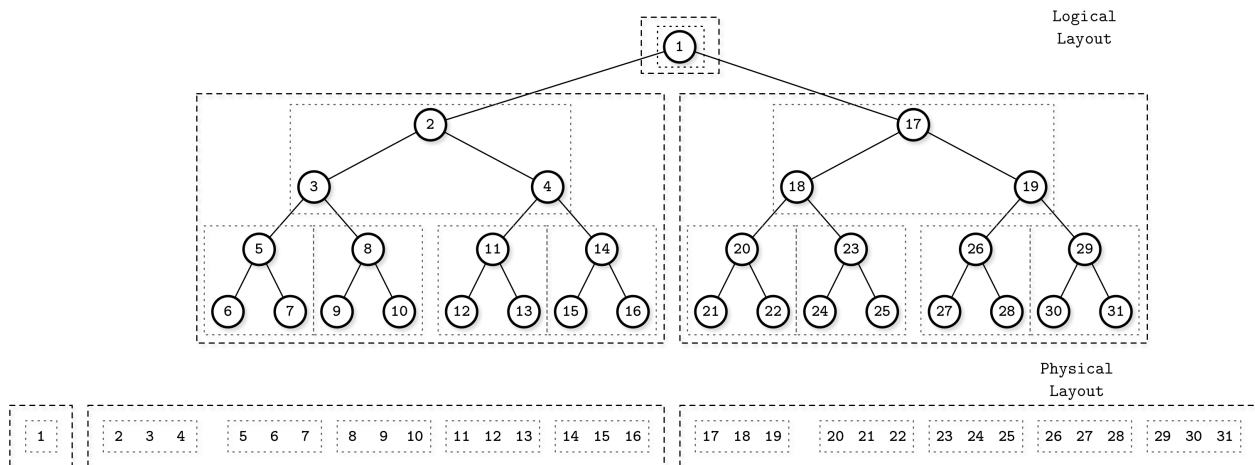


Figure 6-8. van Emde Boas layout

Hình 6-8. Bố cục van Emde Boas

To make the data structure dynamic (i.e., allow inserts, updates, and deletes), cache-oblivious trees use a packed array data structure, which uses contiguous memory segments for storing elements, but contains gaps reserved for future inserted elements. Gaps are spaced based on the density threshold. Figure 6-9 shows a packed array structure, where elements are spaced to create gaps.

Để làm cho cấu trúc dữ liệu trở nên động (tức là cho phép thêm, cập nhật, và xóa), các cây không phụ thuộc bộ nhớ đệm dùng một cấu trúc dữ liệu mảng nén, vốn dùng các phân đoạn bộ nhớ liền kề để lưu các phần tử, nhưng chứa các khoảng hở được dành sẵn cho các phần tử sẽ được chèn trong tương lai. Các khoảng hở được phân bố dựa trên ngưỡng mật độ (density threshold). Hình 6-9 cho thấy một cấu trúc mảng nén, nơi các phần tử được phân bố để tạo các khoảng hở.

Figure 6-9. Packed array

Hình 6-9. Mảng nén

This approach allows inserting items into the tree with fewer relocations. Items have to be relocated just to create a gap for the newly inserted element, if the gap is not already present. When the packed array becomes too densely or sparsely populated, the structure has to be rebuilt to grow or shrink the array.

Cách tiếp cận này cho phép chèn các mục vào cây với ít lần tái định vị hơn. Các mục chỉ phải được tái định vị để tạo một khoảng hở cho phần tử mới được chèn, nếu khoảng hở đó chưa có sẵn. Khi mảng nén trở nên quá dày đặc hoặc quá thưa thớt, cấu trúc phải được xây dựng lại để mở rộng hoặc thu nhỏ mảng.

The static tree is used as an index for the bottom-level packed array, and has to be updated in accordance with relocated elements to point to correct elements on the bottom level.

Cây tính được dùng làm chỉ mục cho mảng nén ở tầng đáy, và phải được cập nhật phù hợp với các phần tử đã tái định vị để trở tới các phần tử đúng ở tầng đáy.

This is an interesting approach, and ideas from it can be used to build efficient B-Tree implementations. It allows constructing on-disk structures in ways that are very similar to how main memory ones are constructed. However, as of the date of writing, I'm not aware of any nonacademic cache-oblivious B-Tree implementations.

Đây là một cách tiếp cận thú vị, và các ý tưởng từ nó có thể được dùng để xây dựng các cách hiện thực B-Tree hiệu quả. Nó cho phép xây dựng các cấu trúc trên đĩa theo những cách rất giống với cách các cấu trúc bộ nhớ chính được xây dựng. Tuy nhiên, tính đến thời điểm viết, tôi không biết đến bất kỳ cách hiện thực B-Tree không phụ thuộc bộ nhớ đệm phi học thuật nào.

A possible reason for that is an assumption that when cache loading is abstracted away, while data is loaded and written back in blocks, paging and eviction still have a negative impact on the result. Another possible reason is that in terms of block transfers, the complexity of cache-oblivious B-Trees is the same as their cache-aware counterpart. This may change when more efficient nonvolatile byte-addressable storage devices become more widespread.

Một lý do khả dĩ cho điều đó là một giả định rằng khi việc nạp cache được trừu tượng hóa đi, trong khi dữ liệu được nạp và ghi lại theo khối, thì việc phân trang và đuổi trang (eviction) vẫn có tác động tiêu cực lên kết quả. Một lý do khả dĩ khác là xét về mặt chuyển khối, độ phức tạp của B-Tree không phụ thuộc bộ nhớ đệm bằng với độ phức tạp của phiên bản đối ứng phụ thuộc bộ nhớ đệm. Điều này có thể thay đổi khi các thiết bị lưu trữ không mất dữ liệu khi mất điện (nonvolatile), định địa chỉ theo byte và hiệu quả hơn trở nên phổ biến hơn.

Summary — Tóm tắt

The original B-Tree design has several shortcomings that might have worked well on spinning disks, but make it less efficient when used on SSDs. B-Trees have high write amplification (caused by page rewrites) and high space overhead since B-Trees have to reserve space in nodes for future writes.

Thiết kế B-Tree gốc có vài hạn chế có thể đã hoạt động tốt trên các đĩa quay, nhưng khiến nó kém hiệu quả hơn khi dùng trên SSD. B-Tree có khuếch đại ghi cao (do việc ghi lại trang) và phụ phí không gian cao vì B-Tree phải dành sẵn không gian trong các nút cho các lần ghi tương lai.

Write amplification can be reduced by using buffering. Lazy B-Trees, such as WiredTiger and LA-Trees, attach in-memory buffers to individual nodes or groups of nodes to reduce the number of required I/O operations by buffering subsequent updates to pages in memory.

Khuếch đại ghi có thể được giảm bằng cách dùng đệm. Các B-Tree lười, chẳng hạn WiredTiger và LA-Tree, gắn các vùng đệm trong bộ nhớ vào từng nút riêng lẻ hoặc các nhóm nút để giảm số thao tác I/O cần thiết bằng cách đệm các bản cập nhật kế tiếp vào các trang trong bộ nhớ.

To reduce space amplification, FD-Trees use **immutability**: data records are stored in the immutable sorted runs, and the size of a mutable B-Tree is limited.

Để giảm khuếch đại không gian, FD-Tree dùng tính bất biến: các bản ghi dữ liệu được lưu trong các run đã sắp xếp bất biến, và kích thước của một B-Tree có thể thay đổi bị giới hạn.

Bw-Trees solve space amplification by using immutability, too. B-Tree nodes and updates to them are stored in separate on-disk locations and persisted in the log-structured store. Write amplification is reduced compared to the original B-Tree design, since reconciling contents that belong to a single logical node is relatively infrequent. Bw-Trees do not require latches for protecting pages from concurrent accesses, as the virtual pointers between the logical nodes are stored in memory.

Bw-Tree cũng giải quyết khuếch đại không gian bằng cách dùng tính bất biến. Các nút B-Tree và các bản cập nhật vào chúng được lưu ở các vị trí riêng biệt trên đĩa và được lưu bền trong kho lưu trữ dạng nhật ký. Khuếch đại ghi được giảm so với thiết kế B-Tree gốc, vì việc đối chiếu nội dung thuộc về một nút logic duy nhất tương đối không thường xuyên. Bw-Tree không cần các chốt để bảo vệ các trang khỏi các truy cập đồng thời, vì các con trỏ ảo giữa các nút logic được lưu trong bộ nhớ.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Copy-on-Write B-Trees Driscoll, J. R., N. Sarnak, D. D. Sleator, and R. E. Tarjan. 1986. "Making data structures persistent." In Proceedings of the eighteenth annual ACM symposium on Theory of computing (STOC '86) , 109-121. [https://dx.doi.org/10.1016/0022-0000\(89\)90034-2](https://dx.doi.org/10.1016/0022-0000(89)90034-2) .

Lazy-Adaptive Trees Agrawal, Devesh, Deepak Ganesan, Ramesh Sitaraman, Yanlei Diao, and Shashi Singh. 2009. "Lazy-Adaptive Tree: an optimized index structure for flash devices." Proceedings of the VLDB Endowment 2, no. 1 (January): 361-372. <https://doi.org/10.14778/1687627.1687669> .

FD-Trees Li, Yinan, Bingsheng He, Robin Jun Yang, Qiong Luo, and Ke Yi. 2010. "Tree Indexing on Solid State Drives." Proceedings of the VLDB Endowment 3, no. 1-2 (September): 1195-1206. <https://doi.org/10.14778/1920841.1920990> .

Bw-Trees Wang, Ziqi, Andrew Pavlo, Hyeontaek Lim, Viktor Leis, Huanchen Zhang, Michael Kaminsky, and David G. Andersen. 2018. "Building a Bw-Tree Takes More Than Just Buzz Words." Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18) , 473-488. <https://doi.org/10.1145/3183713.3196895>

Levandowski, Justin J., David B. Lomet, and Sudipta Sengupta. 2013. "The Bw- Tree: A B-tree for new hardware platforms." In Proceedings of the 2013 IEEE International Conference on Data Engineering (ICDE '13) , 302-313. IEEE. <https://doi.org/10.1109/ICDE.2013.6544834> .

Cache-Oblivious B-Trees Bender, Michael A., Erik D. Demaine, and Martin Farach-Colton. 2005. "Cache- Oblivious B-Trees." SIAM Journal on Computing 35, no. 2 (August): 341-358. <https://doi.org/10.1137/S0097539701389956> .

CHAPTER 7 Log-Structured Storage

— CHƯƠNG 7: Lưu trữ dạng nhật ký (Log-Structured Storage)

Accountants don't use erasers or they end up in jail. —Pat Helland

Kế toán viên không dùng tẩy, nếu không họ sẽ vào tù. —Pat Helland

When accountants have to modify the record, instead of erasing the existing value, they create a new record with a correction. When the quarterly report is published, it may contain minor modifications, correcting the previous quarter results. To derive the bottom line, you have to go through the records and calculate a subtotal [HEL- LAND15].

Khi kế toán viên phải sửa một bản ghi, thay vì xóa giá trị hiện có, họ tạo một bản ghi mới với phần đính chính. Khi báo cáo quý được công bố, nó có thể chứa những sửa đổi nhỏ, đính chính kết quả của quý trước. Để rút ra con số cuối cùng, bạn phải duyệt qua các bản ghi và tính tổng phụ [HEL-LAND15].

Similarly, **immutable** storage structures do not allow modifications to the existing files: tables are written once and are never modified again. Instead, new records are appended to the new file and, to find the final value (or conclude its absence), records have to be reconstructed from multiple files. In contrast, mutable storage structures modify records on disk in place.

Tương tự, các cấu trúc lưu trữ bất biến (immutable) không cho phép sửa đổi các tệp hiện có: bảng được ghi một lần và không bao giờ bị sửa lại. Thay vào đó, các bản ghi mới được nối thêm vào tệp mới và, để tìm ra giá trị cuối cùng (hoặc kết luận rằng nó không tồn tại), các bản ghi phải được tái dựng từ nhiều tệp. Ngược lại, các cấu trúc lưu trữ khả biến (mutable) sửa đổi bản ghi tại chỗ trên đĩa.

Immutable data structures are often used in functional programming languages and are getting more popular because of their safety characteristics: once created, an immutable structure doesn't change, all of its references can be accessed concurrently, and its integrity is guaranteed by the fact that it cannot be modified.

Các cấu trúc dữ liệu bất biến thường được dùng trong các ngôn ngữ lập trình hàm và đang trở nên phổ biến hơn nhờ các đặc tính an toàn của chúng: một khi đã được tạo, một cấu trúc bất biến không thay đổi, mọi tham chiếu đến nó có thể được truy cập đồng thời, và tính toàn vẹn của nó được bảo đảm bởi việc nó không thể bị sửa đổi.

On a high level, there is a strict distinction between how data is treated inside a storage structure and outside of it. Internally, immutable files can hold multiple copies, more recent ones overwriting the older ones, while mutable files generally hold only the most recent value instead. When accessed,

immutable files are processed, redundant copies are reconciled, and the most recent ones are returned to the client.

Ở mức cao, có một sự phân biệt nghiêm ngặt giữa cách dữ liệu được xử lý bên trong một cấu trúc lưu trữ và bên ngoài nó. Bên trong, các tệp bất biến có thể giữ nhiều bản sao, bản mới hơn ghi đè lên bản cũ hơn, trong khi các tệp khả biến nói chung chỉ giữ giá trị mới nhất. Khi được truy cập, các tệp bất biến được xử lý, các bản sao dư thừa được hòa giải, và những bản mới nhất được trả về cho client.

As do other books and papers on the subject, we use B-Trees as a typical example of mutable structure and Log-Structured **Merge** Trees (LSM Trees) as an example of an immutable structure. Immutable LSM Trees use append-only storage and merge

Giống như các sách và bài báo khác về chủ đề này, chúng ta dùng B-Tree làm ví dụ điển hình cho cấu trúc khả biến và Log-Structured Merge Trees (LSM Tree) làm ví dụ cho cấu trúc bất biến. LSM Tree bất biến dùng lưu trữ chỉ-nối-thêm (append-only) và gộp (merge)

129 reconciliation, and B-Trees locate data records on disk and update pages at their original offsets in the file.

hòa giải (reconciliation), còn B-Tree định vị các bản ghi dữ liệu trên đĩa và cập nhật trang tại độ dời (offset) gốc của chúng trong tệp.

In-place update storage structures are optimized for **read** performance [GRAEFE04] : after locating data on disk, the record can be returned to the client. This comes at the expense of write performance: to update the data record in place, it first has to be located on disk. On the other hand, append-only storage is optimized for write performance. Writes do not have to locate records on disk to overwrite them. However, this is done at the expense of reads, which have to retrieve multiple data record versions and reconcile them.

Các cấu trúc lưu trữ cập-nhật-tại-chỗ được tối ưu cho hiệu năng đọc [GRAEFE04]: sau khi định vị dữ liệu trên đĩa, bản ghi có thể được trả về cho client. Điều này đánh đổi bằng hiệu năng ghi: để cập nhật bản ghi dữ liệu tại chỗ, trước tiên phải định vị nó trên đĩa. Mặt khác, lưu trữ chỉ-nối-thêm được tối ưu cho hiệu năng ghi. Việc ghi không cần phải định vị bản ghi trên đĩa để ghi đè. Tuy nhiên, điều này đánh đổi bằng việc đọc, vốn phải truy xuất nhiều phiên bản bản ghi dữ liệu và hòa giải chúng.

So far we've mostly talked about mutable storage structures. We've touched on the subject of **immutability** while discussing **copy-on-write** B-Trees (see "Copy-on-Write" on **page** 112), FD-Trees (see "FD-Trees" on page 117), and Bw-Trees (see "Bw-Trees" on page 120). But there are more ways to implement immutable structures.

Cho đến giờ chúng ta chủ yếu nói về các cấu trúc lưu trữ khả biến. Chúng ta đã chạm tới chủ đề tính bất biến (immutability) khi thảo luận về B-Tree sao-khi-ghi (copy-on-write) (xem "Copy-on-Write" ở trang 112), FD-Tree (xem "FD-Trees" ở trang 117), và Bw-Tree (xem "Bw-Trees" ở trang 120). Nhưng còn nhiều cách khác để hiện thực các cấu trúc bất biến.

Because of the structure and construction approach taken by mutable B-Trees, most I/O operations during reads, writes, and maintenance are random . Each write operation first needs to locate a page that holds a data record and only then can modify it. B-Trees require **node** splits and merges that relocate already written records. After some time, **B-Tree** pages may require maintenance. Pages are fixed in size, and some free space is reserved for future writes. Another problem is that even when only one **cell** in the page is modified, an entire page has to be rewritten.

Vì cấu trúc và cách xây dựng của B-Tree khả biến, hầu hết các thao tác I/O trong khi đọc, ghi và bảo trì đều là ngẫu nhiên. Mỗi thao tác ghi trước tiên cần định vị trang giữ bản ghi

dữ liệu rồi mới có thể sửa nó. B-Tree đòi hỏi tách và gộp nút (node split/merge) khiến phải di dời các bản ghi đã ghi. Sau một thời gian, các trang B-Tree có thể cần bảo trì. Các trang có kích thước cố định, và một phần dung lượng trống được dành cho các lần ghi tương lai. Một vấn đề khác là ngay cả khi chỉ một ô (cell) trong trang bị sửa, toàn bộ trang phải được ghi lại.

There are alternative approaches that can help to mitigate these problems, make some of the I/O operations **sequential**, and avoid page rewrites during modifications. One of the ways to do this is to use immutable structures. In this chapter, we'll focus on LSM Trees: how they're built, what their properties are, and how they are different from B-Trees.

Có những cách tiếp cận thay thế giúp giảm nhẹ các vấn đề này, biến một số thao tác I/O thành tuần tự, và tránh ghi lại trang trong khi sửa đổi. Một trong những cách đó là dùng các cấu trúc bất biến. Trong chương này, chúng ta sẽ tập trung vào LSM Tree: cách chúng được xây dựng, các thuộc tính của chúng, và chúng khác B-Tree như thế nào.

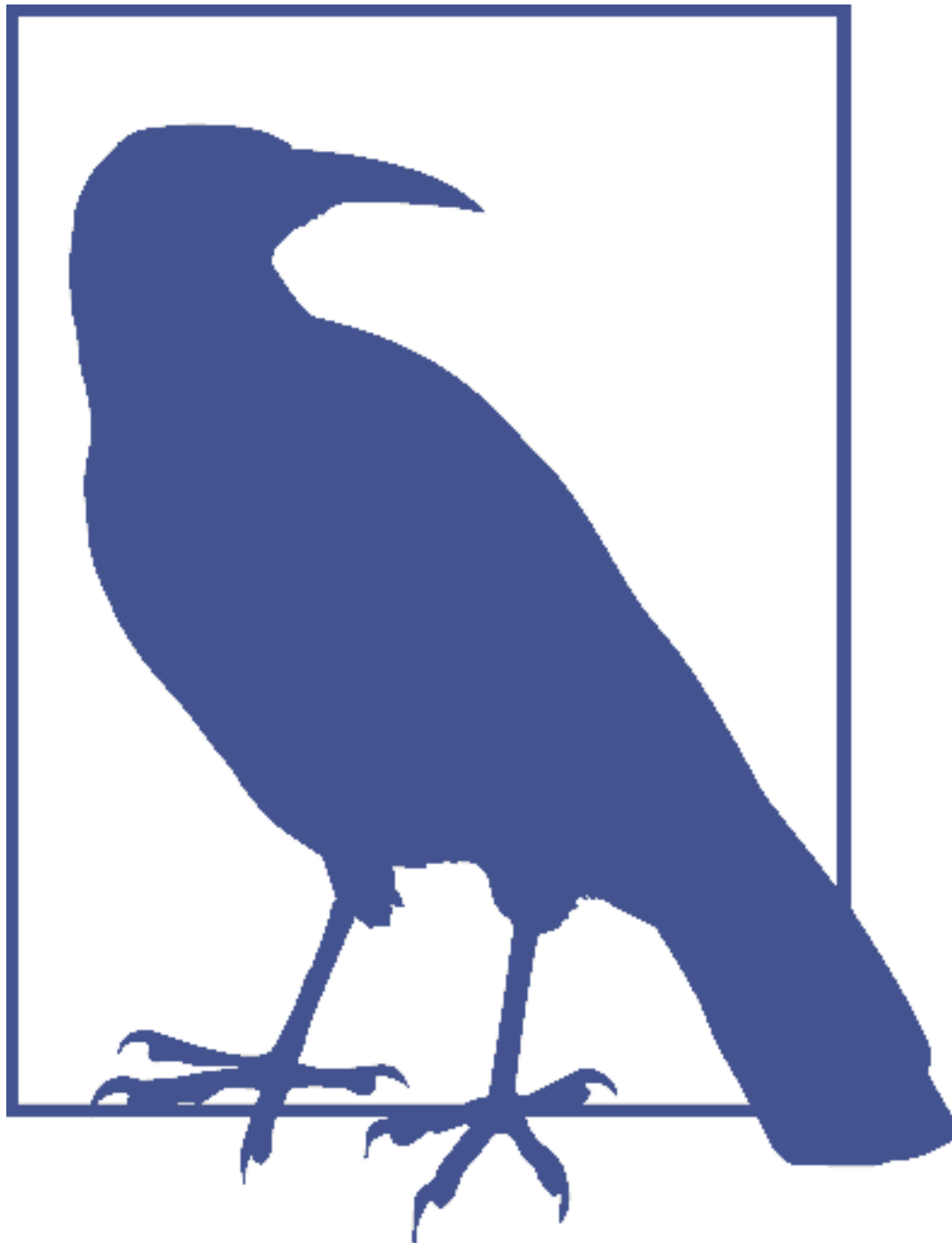
LSM Trees — LSM Tree

When talking about B-Trees, we concluded that space overhead and **write amplification** can be improved by using **buffering**. Generally, there are two ways buffering can be applied in different storage structures: to postpone propagating writes to disk-resident pages (as we've seen with “FD-Trees” on page 117 and “WiredTiger” on page 114), and to make write operations sequential.

Khi nói về B-Tree, chúng ta đã kết luận rằng chi phí không gian (space overhead) và khuếch đại ghi (write amplification) có thể được cải thiện bằng cách dùng đệm (buffering). Nói chung, có hai cách áp dụng đệm trong các cấu trúc lưu trữ khác nhau: để hoãn việc truyền các lần ghi xuống các trang nằm trên đĩa (như ta đã thấy với “FD-Trees” ở trang 117 và “WiredTiger” ở trang 114), và để biến các thao tác ghi thành tuần tự.

One of the most popular immutable on-disk storage structures, **LSM Tree** uses buffering and append-only storage to achieve sequential writes. The LSM Tree is a variant of a disk-resident structure similar to a B-Tree, where nodes are fully occupied, optimized for sequential disk access. This concept was first introduced in a paper by Patrick O’Neil and Edward Cheng [ONEIL96] . Log-structured merge trees take their name from log-structured filesystems, which write all modifications on disk in a log-like file [ROSENBLUM92] .

Là một trong những cấu trúc lưu trữ bất biến trên đĩa phổ biến nhất, LSM Tree dùng đệm và lưu trữ chỉ-nối-thêm để đạt được các lần ghi tuần tự. LSM Tree là một biến thể của cấu trúc nằm trên đĩa tương tự B-Tree, trong đó các nút được lấp đầy hoàn toàn, được tối ưu cho truy cập đĩa tuần tự. Khái niệm này lần đầu được giới thiệu trong một bài báo của Patrick O’Neil và Edward Cheng [ONEIL96]. Cây gộp dạng nhật ký (log-structured merge tree) lấy tên từ các hệ thống tệp dạng nhật ký, vốn ghi mọi sửa đổi lên đĩa trong một tệp giống nhật ký [ROSENBLUM92].



LSM Trees write immutable files and merge them together over time. These files usually contain an index of their own to help readers efficiently locate data. Even though LSM Trees are often presented as an alternative to B-Trees, it is common for B-Trees to be used as the internal indexing structure for an LSM Tree's immutable files.

LSM Tree ghi các tệp bất biến và gộp chúng lại theo thời gian. Các tệp này thường chứa một chỉ mục riêng để giúp người đọc định vị dữ liệu hiệu quả. Mặc dù LSM Tree thường được trình bày như một giải pháp thay thế cho B-Tree, việc dùng B-Tree làm cấu trúc lập chỉ mục nội bộ cho các tệp bất biến của LSM Tree là điều phổ biến.

The word “merge” in LSM Trees indicates that, due to their immutability, tree contents are merged

using an approach similar to merge sort. This happens during maintenance to reclaim space occupied by the redundant copies, and during reads, before contents can be returned to the user.

Từ “merge” (gộp) trong LSM Tree cho thấy rằng, do tính bất biến của chúng, nội dung cây được gộp bằng một cách tiếp cận tương tự sắp xếp trộn (merge sort). Điều này xảy ra trong khi bảo trì để thu hồi không gian bị các bản sao dư thừa chiếm giữ, và trong khi đọc, trước khi nội dung có thể được trả về cho người dùng.

LSM Trees defer **data file** writes and buffer changes in a memory-resident table. These changes are then propagated by writing their contents out to the immutable disk files. All data records remain accessible in memory until the files are fully persisted.

LSM Tree hoãn việc ghi tệp dữ liệu và đệm các thay đổi trong một bảng nằm trong bộ nhớ. Các thay đổi này sau đó được truyền bằng cách ghi nội dung của chúng ra các tệp đĩa bất biến. Mọi bản ghi dữ liệu vẫn truy cập được trong bộ nhớ cho tới khi các tệp được lưu bền hoàn toàn.

Keeping data files immutable favors sequential writes: data is written on the disk in a single pass and files are append-only. Mutable structures can pre-allocate blocks in a single pass (for example, indexed sequential access method (ISAM) [RAMAK- RISHNAN03] [LARSON81]), but subsequent accesses still require random reads and writes. Immutable structures allow us to lay out data records sequentially to prevent **fragmentation**. Additionally, immutable files have higher density : we do not reserve any extra space for data records that are going to be written later, or for the cases when updated records require more space than the originally written ones.

Giữ cho các tệp dữ liệu bất biến ưu ái các lần ghi tuần tự: dữ liệu được ghi lên đĩa trong một lượt duy nhất và các tệp là chỉ-nối-thêm. Các cấu trúc khả biến có thể cấp phát trước các khối (block) trong một lượt duy nhất (ví dụ, phương pháp truy cập tuần tự được lập chỉ mục (ISAM) [RAMAK-RISHNAN03] [LARSON81]), nhưng các lần truy cập tiếp theo vẫn đòi hỏi đọc và ghi ngẫu nhiên. Các cấu trúc bất biến cho phép chúng ta sắp xếp các bản ghi dữ liệu một cách tuần tự để ngăn phân mảnh (fragmentation). Ngoài ra, các tệp bất biến có mật độ cao hơn: chúng ta không dành thêm bất kỳ không gian nào cho các bản ghi dữ liệu sẽ được ghi sau này, hoặc cho trường hợp các bản ghi đã cập nhật cần nhiều không gian hơn các bản đã ghi ban đầu.

Since files are immutable, insert, update, and delete operations do not need to locate data records on disk, which significantly improves write performance and throughput. Instead, duplicate contents are allowed, and conflicts are resolved during the read time. LSM Trees are particularly useful for applications where writes are far more common than reads, which is often the case in modern data-intensive systems, given ever-growing amounts of data and ingest rates.

Vì các tệp là bất biến, các thao tác chèn, cập nhật và xóa không cần định vị bản ghi dữ liệu trên đĩa, điều này cải thiện đáng kể hiệu năng và thông lượng (throughput) ghi. Thay vào đó, nội dung trùng lặp được cho phép, và xung đột được giải quyết tại thời điểm đọc. LSM Tree đặc biệt hữu ích cho các ứng dụng mà việc ghi diễn ra thường xuyên hơn nhiều so với việc đọc, vốn là trường hợp phổ biến trong các hệ thống thâm dụng dữ liệu hiện đại, do lượng dữ liệu và tốc độ nạp ngày càng tăng.

Reads and writes do not intersect by design, so data on disk can be read and written without segment locking, which significantly simplifies concurrent access. In contrast, mutable structures employ hierarchical locks and latches (you can find more information about locks and latches in “**Concurrency Control**” on page 93) to ensure on-disk data structure integrity, and allow multiple concurrent readers but require exclusive subtree ownership for writers. LSM-based storage engines use linearizable in-memory views of data and index files, and only have to guard concurrent access

to the structures managing them.

Đọc và ghi không giao nhau theo thiết kế, nên dữ liệu trên đĩa có thể được đọc và ghi mà không cần khóa phân đoạn, điều này đơn giản hóa đáng kể truy cập đồng thời. Ngược lại, các cấu trúc khả biến dùng khóa (lock) và chốt (latch) phân cấp (bạn có thể tìm thêm thông tin về khóa và chốt trong “Concurrency Control” ở trang 93) để bảo đảm tính toàn vẹn của cấu trúc dữ liệu trên đĩa, và cho phép nhiều người đọc đồng thời nhưng đòi hỏi quyền sở hữu độc quyền cây con đối với người ghi. Các bộ máy lưu trữ (storage engine) dựa trên LSM dùng các khung nhìn trong bộ nhớ có tính tuyến tính hóa (linearizable) cho dữ liệu và tệp chỉ mục, và chỉ phải bảo vệ truy cập đồng thời tới các cấu trúc quản lý chúng.

Both B-Trees and LSM Trees require some housekeeping to optimize performance, but for different reasons. Since the number of allocated files steadily grows, LSM Trees have to merge and rewrite files to make sure that the smallest possible number of files is accessed during the read, as requested data records might be spread across multiple files. On the other hand, mutable files may have to be rewritten partially or wholly to decrease fragmentation and reclaim space occupied by updated or deleted records. Of course, the exact scope of work done by the housekeeping **process** heavily depends on the concrete implementation.

Cả B-Tree lẫn LSM Tree đều đòi hỏi một số công việc dọn dẹp để tối ưu hiệu năng, nhưng vì những lý do khác nhau. Vì số lượng tệp được cấp phát liên tục tăng, LSM Tree phải gộp và ghi lại các tệp để bảo đảm rằng số lượng tệp được truy cập trong khi đọc là nhỏ nhất có thể, bởi các bản ghi dữ liệu được yêu cầu có thể nằm rải rác trên nhiều tệp. Mặt khác, các tệp khả biến có thể phải được ghi lại một phần hoặc toàn bộ để giảm phân mảnh và thu hồi không gian bị các bản ghi đã cập nhật hoặc đã xóa chiếm giữ. Tất nhiên, phạm vi công việc chính xác của tiến trình dọn dẹp phụ thuộc nhiều vào hiện thực cụ thể.

LSM Tree Structure — Cấu trúc của LSM Tree

We start with ordered LSM Trees [ONEIL96] , where files hold sorted data records. Later, in “Unordered LSM Storage” on page 152 , we’ll also discuss structures that allow storing data records in insertion order, which has some obvious advantages on the write path.

Chúng ta bắt đầu với LSM Tree có thứ tự [ONEIL96], trong đó các tệp giữ các bản ghi dữ liệu đã được sắp xếp. Sau này, trong “Unordered LSM Storage” ở trang 152, chúng ta cũng sẽ thảo luận các cấu trúc cho phép lưu các bản ghi dữ liệu theo thứ tự chèn, vốn có một số ưu điểm rõ ràng trên đường ghi.

As we just discussed, LSM Trees consist of smaller memory-resident and larger disk-resident components. To write out immutable file contents on disk, it is necessary to first buffer them in memory and sort their contents.

Như ta vừa thảo luận, LSM Tree gồm các thành phần nhỏ hơn nằm trong bộ nhớ và các thành phần lớn hơn nằm trên đĩa. Để ghi nội dung tệp bất biến ra đĩa, trước tiên cần đệm chúng trong bộ nhớ và sắp xếp nội dung của chúng.

A memory-resident component (often called a **memtable**) is mutable: it buffers data records and serves as a target for read and write operations. Memtable contents are persisted on disk when its size grows up to a configurable threshold. Memtable updates incur no disk access and have no associated I/O costs. A separate write-ahead log file, similar to what we discussed in “**Recovery**” on page 88 , is required to guarantee **durability** of data records. Data records are appended to the log and committed in memory before the operation is acknowledged to the client.

Một thành phần nằm trong bộ nhớ (thường gọi là memtable) là khả biến: nó đệm các bản ghi dữ liệu và phục vụ như đích cho các thao tác đọc và ghi. Nội dung memtable được lưu bền trên đĩa khi kích thước của nó tăng tới một ngưỡng có thể cấu hình. Các cập nhật memtable không phát sinh truy cập đĩa và không có chi phí I/O liên quan. Một tệp nhật ký ghi-trước (write-ahead log) riêng, tương tự như những gì ta đã thảo luận trong “Recovery” ở trang 88, là cần thiết để bảo đảm tính bền vững (durability) của các bản ghi dữ liệu. Các bản ghi dữ liệu được nối vào nhật ký và được commit trong bộ nhớ trước khi thao tác được xác nhận với client.

Buffering is done in memory: all read and write operations are applied to a memory-resident table that maintains a sorted data structure allowing concurrent access, usually some form of an in-memory sorted tree, or any data structure that can give similar performance characteristics.

Việc đệm được thực hiện trong bộ nhớ: mọi thao tác đọc và ghi được áp dụng lên một bảng nằm trong bộ nhớ duy trì một cấu trúc dữ liệu đã sắp xếp cho phép truy cập đồng thời, thường là một dạng cây sắp xếp trong bộ nhớ, hoặc bất kỳ cấu trúc dữ liệu nào có thể cho đặc tính hiệu năng tương tự.

Disk-resident components are built by flushing contents buffered in memory to disk. Disk-resident components are used only for reads: buffered contents are persisted, and files are never modified. This allows us to think in terms of simple operations: writes against an in-memory table, and reads against disk and memory-based tables, merges, and file removals.

Các thành phần nằm trên đĩa được xây dựng bằng cách xả (flush) nội dung đã đệm trong bộ nhớ xuống đĩa. Các thành phần nằm trên đĩa chỉ được dùng để đọc: nội dung đã đệm được lưu bền, và các tệp không bao giờ bị sửa đổi. Điều này cho phép ta tư duy theo các thao tác đơn giản: ghi vào bảng trong bộ nhớ, và đọc từ các bảng trên đĩa và trong bộ nhớ, các lần gộp, và việc xóa tệp.

Throughout this chapter, we will be using the word table as a shortcut for disk-resident table . Since we’re discussing semantics of a **storage engine**, this **term** is not ambiguous with a table concept in the wider context of a database management system.

Xuyên suốt chương này, chúng ta sẽ dùng từ bảng như một cách viết tắt cho bảng nằm trên đĩa. Vì chúng ta đang thảo luận ngữ nghĩa của một bộ máy lưu trữ, thuật ngữ này không bị nhập nhằng với khái niệm bảng trong ngữ cảnh rộng hơn của một hệ quản trị cơ sở dữ liệu (DBMS).

Two-component LSM Tree

LSM Tree hai thành phần

We distinguish between two- and multicomponent LSM Trees. Two-component LSM Trees have only one disk component, comprised of immutable segments. The disk component here is organized as a B-Tree, with 100% node **occupancy** and read-only pages.

Chúng ta phân biệt giữa LSM Tree hai thành phần và đa thành phần. LSM Tree hai thành phần chỉ có một thành phần đĩa, gồm các phân đoạn bất biến. Thành phần đĩa ở đây được tổ chức như một B-Tree, với độ lấp đầy (occupancy) nút 100% và các trang chỉ-đọc.

Memory-resident tree contents are flushed on disk in parts. During a flush, for each flushed in-memory subtree, we find a corresponding subtree on disk and write out the merged contents of a memory-resident segment and disk-resident subtree into the new segment on disk. Figure 7-1 shows in-memory and disk-resident trees before a merge.

Nội dung cây nằm trong bộ nhớ được xả xuống đĩa theo từng phần. Trong khi xả, với mỗi cây con trong bộ nhớ được xả, ta tìm một cây con tương ứng trên đĩa và ghi ra nội dung

đã gộp của một phân đoạn nằm trong bộ nhớ và cây con nằm trên đĩa vào phân đoạn mới trên đĩa. Hình 7-1 cho thấy các cây nằm trong bộ nhớ và nằm trên đĩa trước khi gộp.

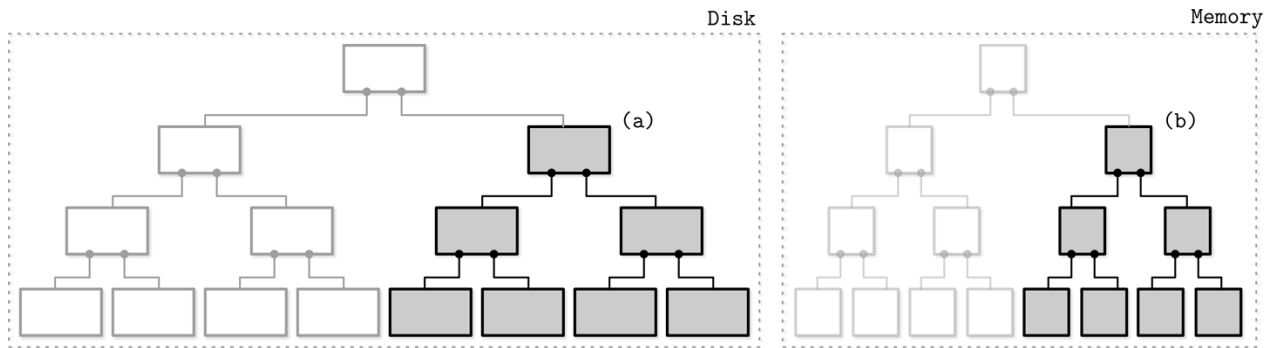


Figure 7-1. Two-component LSM Tree before a flush. Flushing memory- and disk-resident segments are shown in gray.

Hình 7-1. LSM Tree hai thành phần trước khi xả. Các phân đoạn đang xả nằm trong bộ nhớ và nằm trên đĩa được hiển thị bằng màu xám.

After the subtree is flushed, superseded memory-resident and disk-resident subtrees are discarded and replaced with the result of their merge, which becomes addressable from the preexisting sections of the disk-resident tree. Figure 7-2 shows the result of a merge process, already written to the new location on disk and attached to the rest of the tree.

Sau khi cây con được xả, các cây con nằm trong bộ nhớ và nằm trên đĩa đã bị thay thế được loại bỏ và thay bằng kết quả gộp của chúng, kết quả này trở nên có thể địa chỉ hóa được từ các phần đã tồn tại sẵn của cây nằm trên đĩa. Hình 7-2 cho thấy kết quả của tiến trình gộp, đã được ghi vào vị trí mới trên đĩa và gắn vào phần còn lại của cây.

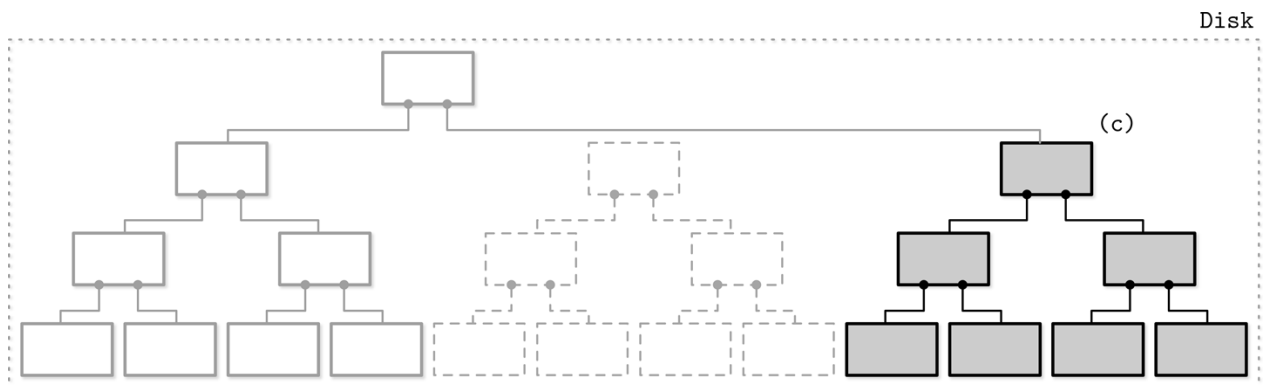


Figure 7-2. Two-component LSM Tree after a flush. Merged contents are shown in gray. Boxes with dashed lines depict discarded on-disk segments.

Hình 7-2. LSM Tree hai thành phần sau khi xả. Nội dung đã gộp được hiển thị bằng màu xám. Các hộp viền nét đứt mô tả các phân đoạn trên đĩa đã bị loại bỏ.

A merge can be implemented by advancing iterators reading the disk-resident leaf nodes and contents of the in-memory tree in lockstep. Since both sources are sorted, to produce a sorted merged result, we only need to know the current values of both iterators during each step of the merge process.

Một lần gộp có thể được hiện thực bằng cách cho các bộ lặp đọc các nút lá nằm trên đĩa và nội dung của cây trong bộ nhớ tiến lên đồng bộ. Vì cả hai nguồn đều đã được sắp xếp,

để tạo ra một kết quả gộp đã sắp xếp, ta chỉ cần biết các giá trị hiện tại của cả hai bộ lặp tại mỗi bước của tiến trình gộp.

This approach is a logical extension and continuation of our conversation on immutable B-Trees. Copy-on-write B-Trees (see “Copy-on-Write” on page 112) use B-Tree structure, but their nodes are not fully occupied, and they require copying pages on the root-leaf path and creating a parallel tree structure. Here, we do something similar, but since we buffer writes in memory, we amortize the costs of the disk-resident tree update.

Cách tiếp cận này là một mở rộng và tiếp nối hợp lý của cuộc trò chuyện của chúng ta về B-Tree bất biến. B-Tree sao-khi-ghi (xem “Copy-on-Write” ở trang 112) dùng cấu trúc B-Tree, nhưng các nút của chúng không được lấp đầy hoàn toàn, và chúng đòi hỏi sao chép các trang trên đường gốc-lá và tạo một cấu trúc cây song song. Ở đây, ta làm điều tương tự, nhưng vì ta đệm các lần ghi trong bộ nhớ, ta phân bổ chi phí cập nhật cây nằm trên đĩa.

When implementing subtree merges and flushes, we have to make sure of three things:

Khi hiện thực việc gộp và xả cây con, ta phải bảo đảm ba điều:

1. As soon as the flush process starts, all new writes have to go to the new memtable. 2. During the subtree flush, both the disk-resident and flushing memory-resident subtree have to remain accessible for reads. 3. After the flush, publishing merged contents, and discarding unmerged disk- and memory-resident contents have to be performed atomically.
1. Ngay khi tiến trình xả bắt đầu, mọi lần ghi mới phải đi tới memtable mới. 2. Trong khi xả cây con, cả cây con nằm trên đĩa lẫn cây con nằm trong bộ nhớ đang xả phải vẫn truy cập được để đọc. 3. Sau khi xả, việc công bố nội dung đã gộp, và loại bỏ nội dung trên đĩa và trong bộ nhớ chưa gộp phải được thực hiện một cách nguyên tử.

Even though two-component LSM Trees can be useful for maintaining index files, no implementations are known to the author as of time of writing. This can be explained by the write amplification characteristics of this approach: merges are relatively frequent, as they are triggered by memtable flushes.

Mặc dù LSM Tree hai thành phần có thể hữu ích để duy trì các tệp chỉ mục, tác giả chưa biết hiện thực nào tại thời điểm viết. Điều này có thể được giải thích bằng đặc tính khuếch đại ghi của cách tiếp cận này: các lần gộp tương đối thường xuyên, vì chúng được kích hoạt bởi các lần xả memtable.

Multicomponent LSM Trees

LSM Tree đa thành phần

Let’s consider an alternative design, multicomponent LSM Trees that have more than just one disk-resident table. In this case, entire memtable contents are flushed in a single run.

Hãy xét một thiết kế thay thế, LSM Tree đa thành phần có nhiều hơn chỉ một bảng nằm trên đĩa. Trong trường hợp này, toàn bộ nội dung memtable được xả trong một lượt chạy duy nhất.

It quickly becomes evident that after multiple flushes we’ll end up with multiple disk-resident tables, and their number will only grow over time. Since we do not always know exactly which tables are holding required data records, we might have to access multiple files to locate the searched data.

Nhanh chóng trở nên rõ ràng rằng sau nhiều lần xả ta sẽ có nhiều bảng nằm trên đĩa, và số lượng của chúng sẽ chỉ tăng theo thời gian. Vì ta không phải lúc nào cũng biết chính xác bảng nào đang giữ các bản ghi dữ liệu cần thiết, ta có thể phải truy cập nhiều tệp để định vị dữ liệu được tìm.

Having to read from multiple sources instead of just one might get expensive. To mitigate this problem and keep the number of tables to minimum, a periodic merge process called **compaction** (see “Maintenance in LSM Trees” on page 141) is triggered. Compaction picks several tables, reads their contents, merges them, and writes the merged result out to the new combined file. Old tables are discarded simultaneously with the appearance of the new merged table.

Việc phải đọc từ nhiều nguồn thay vì chỉ một có thể trở nên tốn kém. Để giảm nhẹ vấn đề này và giữ số lượng bảng ở mức tối thiểu, một tiến trình gộp định kỳ gọi là nén-gộp (compaction) (xem “Maintenance in LSM Trees” ở trang 141) được kích hoạt. Compaction chọn vài bảng, đọc nội dung của chúng, gộp chúng, và ghi kết quả đã gộp ra tệp kết hợp mới. Các bảng cũ bị loại bỏ đồng thời với sự xuất hiện của bảng đã gộp mới.

Figure 7-3 shows the multicomponent LSM Tree data life cycle. Data is first buffered in a memory-resident component. When it gets too large, its contents are flushed on disk to create disk-resident tables. Later, multiple tables are merged together to create larger tables.

Hình 7-3 cho thấy vòng đời dữ liệu của LSM Tree đa thành phần. Dữ liệu trước tiên được đệm trong một thành phần nằm trong bộ nhớ. Khi nó trở nên quá lớn, nội dung của nó được xả xuống đĩa để tạo các bảng nằm trên đĩa. Sau này, nhiều bảng được gộp lại với nhau để tạo các bảng lớn hơn.

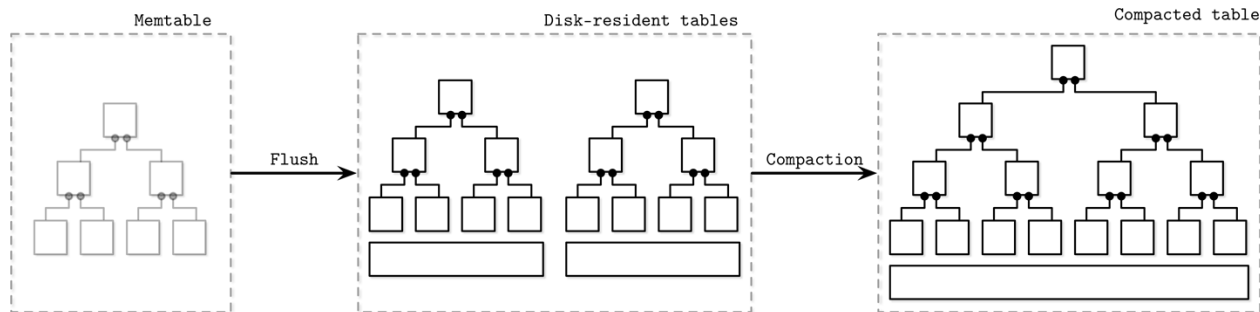


Figure 7-3. Multicomponent LSM Tree data life cycle

Hình 7-3. Vòng đời dữ liệu của LSM Tree đa thành phần

The rest of this chapter is dedicated to multicomponent LSM Trees, building blocks, and their maintenance processes.

Phần còn lại của chương này dành cho LSM Tree đa thành phần, các khối xây dựng của chúng, và các tiến trình bảo trì của chúng.

In-memory tables

Các bảng trong bộ nhớ

Memtable flushes can be triggered periodically, or by using a size threshold. Before it can be flushed, the memtable has to be switched : a new memtable is allocated, and it becomes a target for all new writes, while the old one moves to the flushing state. These two steps have to be performed atomically. The flushing memtable remains available for reads until its contents are fully flushed. After this, the old memtable is discarded in favor of a newly written disk-resident table, which becomes available for reads.

Các lần xả memtable có thể được kích hoạt định kỳ, hoặc bằng cách dùng một ngưỡng kích thước. Trước khi có thể được xả, memtable phải được chuyển đổi: một memtable mới được cấp phát, và nó trở thành đích cho mọi lần ghi mới, trong khi memtable cũ chuyển sang trạng thái đang xả. Hai bước này phải được thực hiện một cách nguyên tử. Memtable đang

xả vẫn khả dụng để đọc cho tới khi nội dung của nó được xả hoàn toàn. Sau đó, memtable cũ được loại bỏ để nhường chỗ cho một bảng nằm trên đĩa mới được ghi, bảng này trở nên khả dụng để đọc.

In Figure 7-4 , you see the components of the LSM Tree, relationships between them, and operations that fulfill transitions between them:

Trong Hình 7-4, bạn thấy các thành phần của LSM Tree, các mối quan hệ giữa chúng, và các thao tác hoàn thành các chuyển tiếp giữa chúng:

Current memtable Receives writes and serves reads.

Memtable hiện tại Nhận các lần ghi và phục vụ các lần đọc.

Flushing memtable Available for reads.

Memtable đang xả Khả dụng để đọc.

On-disk flush target Does not participate in reads, as its contents are incomplete.

Đích xả trên đĩa Không tham gia vào các lần đọc, vì nội dung của nó chưa hoàn chỉnh.

Flushed tables Available for reads as soon as the flushed memtable is discarded.

Các bảng đã xả Khả dụng để đọc ngay khi memtable đã xả bị loại bỏ.

Compacting tables Currently merging disk-resident tables.

Các bảng đang nén-gộp Hiện đang gộp các bảng nằm trên đĩa.

Compacted tables Created from flushed or other compacted tables.

Các bảng đã nén-gộp Được tạo từ các bảng đã xả hoặc các bảng đã nén-gộp khác.

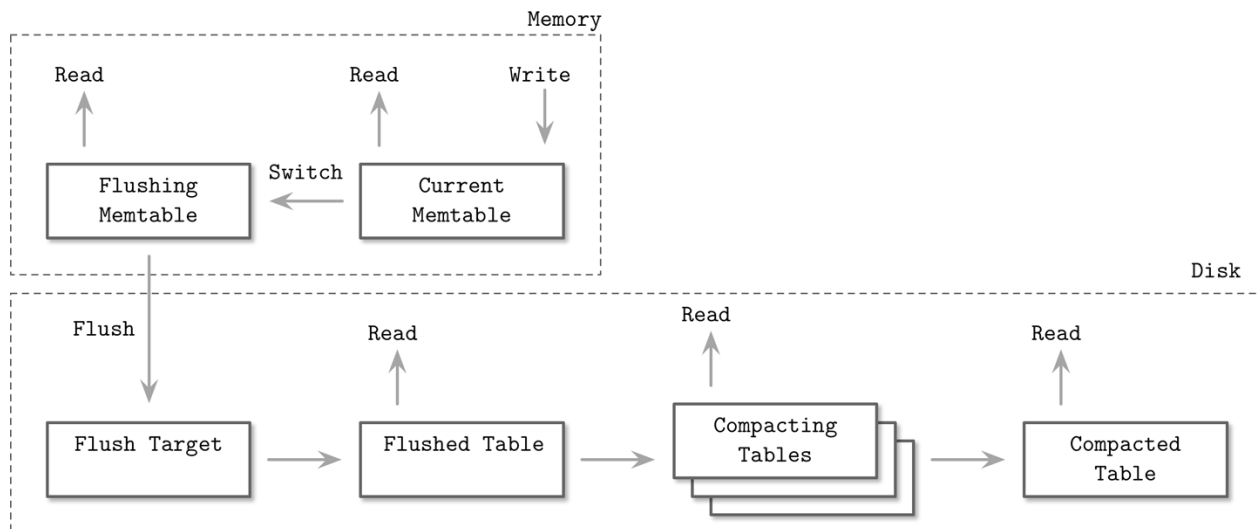


Figure 7-4. LSM component structure

Hình 7-4. Cấu trúc thành phần LSM

Data is already sorted in memory, so a disk-resident table can be created by sequentially writing out memory-resident contents to disk. During a flush, both the flushing memtable and the current memtable are available for read.

Dữ liệu đã được sắp xếp trong bộ nhớ, nên một bảng nằm trên đĩa có thể được tạo bằng cách ghi tuần tự nội dung nằm trong bộ nhớ xuống đĩa. Trong khi xả, cả memtable đang xả lẫn memtable hiện tại đều khả dụng để đọc.

Until the memtable is fully flushed, the only disk-resident version of its contents is stored in the write-ahead log. When memtable contents are fully flushed on disk, the log can be trimmed, and the log section, holding operations applied to the flushed memtable, can be discarded.

Cho tới khi memtable được xả hoàn toàn, phiên bản nằm trên đĩa duy nhất của nội dung của nó được lưu trong nhật ký ghi-trước. Khi nội dung memtable được xả hoàn toàn trên đĩa, nhật ký có thể được cắt bớt, và phần nhật ký giữ các thao tác đã áp dụng lên memtable đã xả có thể được loại bỏ.

Updates and Deletes — Cập nhật và Xóa

In LSM Trees, insert, update, and delete operations do not require locating data records on disk. Instead, redundant records are reconciled during the read.

Trong LSM Tree, các thao tác chèn, cập nhật và xóa không đòi hỏi định vị các bản ghi dữ liệu trên đĩa. Thay vào đó, các bản ghi dư thừa được hòa giải trong khi đọc.

Removing data records from the memtable is not enough, since other disk or memory resident tables may hold data records for the same key. If we were to implement deletes by just removing items from the memtable, we would end up with deletes that either have no impact or would resurrect the previous values.

Việc gỡ các bản ghi dữ liệu khỏi memtable là không đủ, vì các bảng khác nằm trên đĩa hoặc trong bộ nhớ có thể giữ các bản ghi dữ liệu cho cùng một khóa. Nếu ta hiện thực việc xóa chỉ bằng cách gỡ các mục khỏi memtable, ta sẽ kết thúc với các thao tác xóa hoặc không có tác dụng gì hoặc làm sống lại các giá trị trước đó.

Let's consider an example. The flushed disk-resident table contains data record v1 associated with a key k1, and the memtable holds its new value v2:

Hãy xét một ví dụ. Bảng nằm trên đĩa đã xả chứa bản ghi dữ liệu v1 liên kết với khóa k1, và memtable giữ giá trị mới của nó v2:

Disk Table	Memtable
k1 v1	k1 v2

If we just remove v2 from the memtable and flush it, we effectively resurrect v1, since it becomes the only value associated with that key:

Nếu ta chỉ gỡ v2 khỏi memtable và xả nó, ta thực chất làm sống lại v1, vì nó trở thành giá trị duy nhất liên kết với khóa đó:

Disk Table	Memtable
k1 v1	∅

Because of that, deletes need to be recorded explicitly. This can be done by inserting a special delete entry (sometimes called a tombstone or a dormant certificate), indicating removal of the data record associated with a specific key:

Vì vậy, các thao tác xóa cần được ghi nhận một cách tường minh. Điều này có thể được thực hiện bằng cách chèn một mục xóa đặc biệt (đôi khi gọi là tombstone hoặc dormant certificate), chỉ ra việc gỡ bỏ bản ghi dữ liệu liên kết với một khóa cụ thể:

Disk Table	Memtable
k1 v1	k1 <tombstone>

The reconciliation process picks up tombstones, and filters out the shadowed values.

Tiến trình hòa giải nhận diện các tombstone, và lọc bỏ các giá trị bị che khuất.

Sometimes it might be useful to remove a consecutive range of keys rather than just a single key. This can be done using predicate deletes, which work by appending a delete entry with a predicate that sorts according to regular record-sorting rules. During reconciliation, data records matching the predicate are skipped and not returned to the client.

Đôi khi việc gỡ bỏ một dải khóa liên tiếp thay vì chỉ một khóa đơn lẻ có thể hữu ích. Điều này có thể được thực hiện bằng cách dùng xóa theo vị từ (predicate deletes), hoạt động bằng cách nối thêm một mục xóa kèm một vị từ sắp xếp theo các quy tắc sắp xếp bản ghi thông thường. Trong khi hòa giải, các bản ghi dữ liệu khớp vị từ được bỏ qua và không được trả về cho client.

Predicates can take a form of `DELETE FROM table WHERE key ≥ "k2" AND key < "k4"` and can receive any range matchers. Apache Cassandra implements this approach and calls it range tombstones. A range tombstone covers a range of keys rather than just a single key.

“k4” và có thể nhận bất kỳ bộ so khớp dải nào. Apache Cassandra hiện thực cách tiếp cận này và gọi nó là range tombstones. Một range tombstone bao trùm một dải khóa thay vì chỉ một khóa đơn lẻ.

When using range tombstones, resolution rules have to be carefully considered because of overlapping ranges and disk-resident table boundaries. For example, the following combination will hide data records associated with k2 and k3 from the final result:

Khi dùng range tombstones, các quy tắc phân giải phải được cân nhắc kỹ vì các dải chồng lấn và ranh giới của các bảng nằm trên đĩa. Ví dụ, tổ hợp sau sẽ ẩn các bản ghi dữ liệu liên kết với k2 và k3 khỏi kết quả cuối cùng:

Disk Table 1	Disk Table 2
k1 v1	k2 <start_tombstone_inclusive>
k2 v2	k4 <end_tombstone_exclusive>
k3 v3	
k4 v4	

LSM Tree Lookups — Tra cứu trong LSM Tree

LSM Trees consist of multiple components. During lookups, more than one component is usually accessed, so their contents have to be merged and reconciled before they can be returned to the client. To better understand the merge process, let's see how tables are iterated during the merge and how conflicting records are combined.

LSM Tree gồm nhiều thành phần. Trong khi tra cứu, thường nhiều hơn một thành phần được truy cập, nên nội dung của chúng phải được gộp và hòa giải trước khi có thể được trả về cho client. Để hiểu rõ hơn tiến trình gộp, hãy xem các bảng được lặp qua như thế nào trong khi gộp và các bản ghi xung đột được kết hợp ra sao.

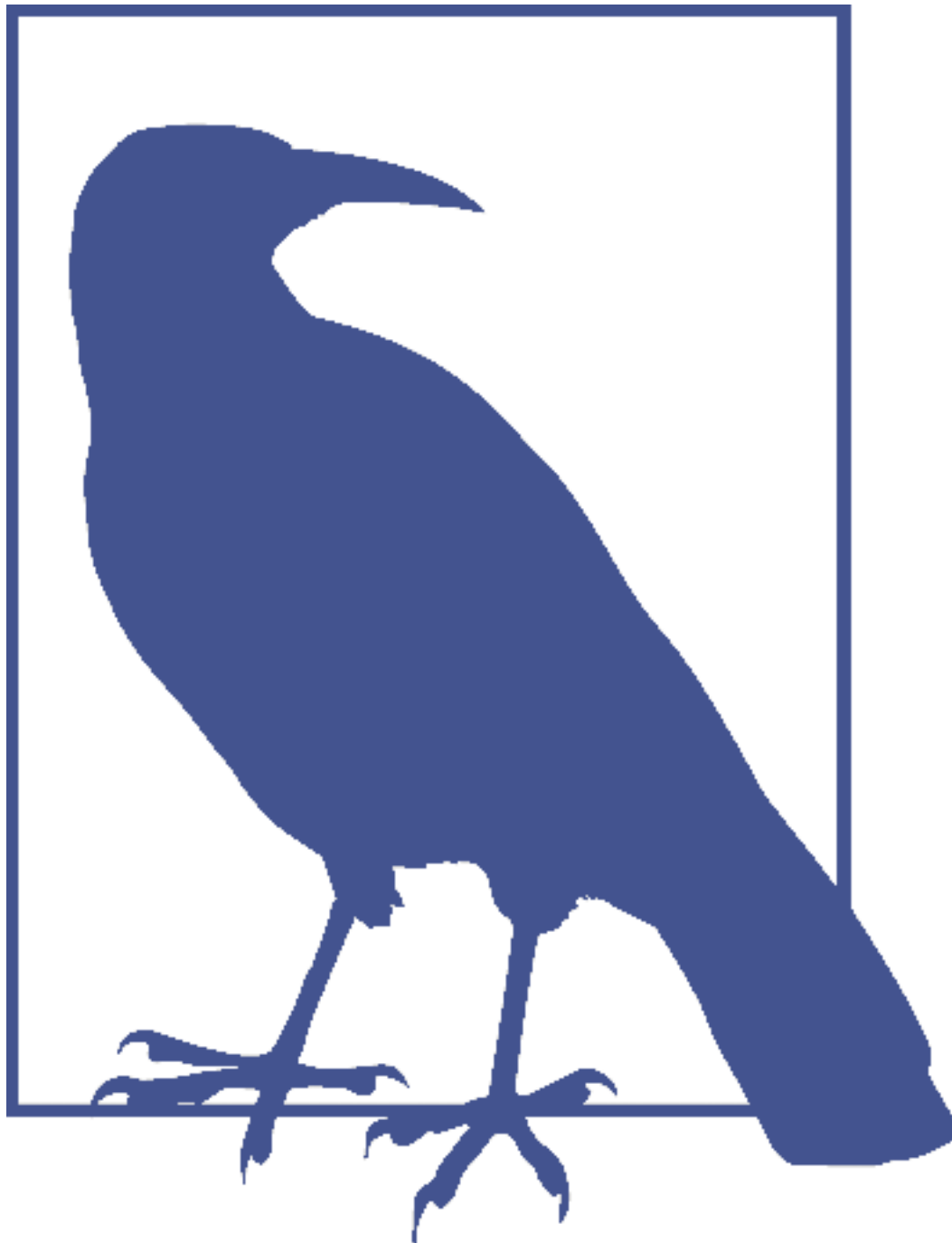
Merge-Iteration — Lặp-gộp (Merge-Iteration)

Since contents of disk-resident tables are sorted, we can use a multiway merge-sort algorithm. For example, we have three sources: two disk-resident tables and one memtable. Usually, storage engines offer a cursor or an iterator to navigate through file contents. This cursor holds the **offset** of the last consumed data record, can be checked for whether or not iteration has finished, and can be used to retrieve the next data record.

Vì nội dung của các bảng nằm trên đĩa đã được sắp xếp, ta có thể dùng một thuật toán sắp xếp trộn nhiều đường (multiway merge-sort). Ví dụ, ta có ba nguồn: hai bảng nằm trên đĩa và một memtable. Thông thường, các bộ máy lưu trữ cung cấp một con trỏ (cursor) hoặc một bộ lặp (iterator) để điều hướng qua nội dung tệp. Con trỏ này giữ độ dời của bản ghi dữ liệu được tiêu thụ cuối cùng, có thể được kiểm tra xem việc lặp đã kết thúc hay chưa, và có thể được dùng để truy xuất bản ghi dữ liệu tiếp theo.

A multiway merge-sort uses a priority queue, such as min-heap [SEdGEWICK11], that holds up to N elements (where N is the number of iterators), which sorts its contents and prepares the next-in-line smallest element to be returned. The head of each iterator is placed into the queue. An element in the head of the queue is then the minimum of all iterators.

Sắp xếp trộn nhiều đường dùng một hàng đợi ưu tiên (priority queue), chẳng hạn min-heap [SEdGEWICK11], giữ tới N phần tử (với N là số bộ lặp), sắp xếp nội dung của nó và chuẩn bị phần tử nhỏ nhất kế tiếp để trả về. Phần đầu của mỗi bộ lặp được đặt vào hàng đợi. Phần tử ở đầu hàng đợi khi đó là phần tử nhỏ nhất trong tất cả các bộ lặp.



A priority queue is a data structure used for maintaining an ordered queue of items. While a regular queue retains items in order of their addition (first in, first out), a priority queue re-sorts items on insertion and the item with the highest (or lowest) priority is placed in the head of the queue. This is particularly useful for merge-iteration, since we have to output elements in a sorted order.

Hàng đợi ưu tiên là một cấu trúc dữ liệu dùng để duy trì một hàng đợi có thứ tự các mục. Trong khi một hàng đợi thông thường giữ các mục theo thứ tự thêm vào của chúng (vào trước, ra trước), một hàng đợi ưu tiên sắp xếp lại các mục khi chèn và mục có độ ưu tiên cao nhất (hoặc thấp nhất) được đặt ở đầu hàng đợi. Điều này đặc biệt hữu ích cho lặp-gộp, vì ta phải xuất các phần tử theo thứ tự đã sắp xếp.

When the smallest element is removed from the queue, the iterator associated with it is checked for the next value, which is then placed into the queue, which is re-sorted to preserve the order.

Khi phần tử nhỏ nhất được gỡ khỏi hàng đợi, bộ lặp liên kết với nó được kiểm tra giá trị tiếp theo, giá trị này sau đó được đặt vào hàng đợi, và hàng đợi được sắp xếp lại để bảo toàn thứ tự.

Since all iterator contents are sorted, reinserting a value from the iterator that held the previous smallest value of all iterator heads also preserves an invariant that the queue still holds the smallest elements from all iterators. Whenever one of the iterators is exhausted, the algorithm proceeds without reinserting the next iterator head. The algorithm continues until either query conditions are satisfied or all iterators are exhausted.

Vì mọi nội dung bộ lặp đều đã được sắp xếp, việc chèn lại một giá trị từ bộ lặp đã giữ giá trị nhỏ nhất trước đó trong tất cả các phần đầu bộ lặp cũng bảo toàn một bất biến rằng hàng đợi vẫn giữ các phần tử nhỏ nhất từ tất cả các bộ lặp. Bất cứ khi nào một trong các bộ lặp cạn kiệt, thuật toán tiếp tục mà không chèn lại phần đầu bộ lặp tiếp theo. Thuật toán tiếp tục cho tới khi hoặc các điều kiện truy vấn được thỏa mãn hoặc mọi bộ lặp đều cạn kiệt.

Figure 7-5 shows a schematic representation of the merge process just described: head elements (light gray items in source tables) are placed to the priority queue. Elements from the priority queue are returned to the output iterator. The resulting output is sorted.

Hình 7-5 cho thấy một biểu diễn sơ đồ của tiến trình gộp vừa được mô tả: các phần tử đầu (các mục màu xám nhạt trong các bảng nguồn) được đặt vào hàng đợi ưu tiên. Các phần tử từ hàng đợi ưu tiên được trả về cho bộ lặp xuất. Kết quả xuất ra đã được sắp xếp.

It may happen that we encounter more than one data record for the same key during merge-iteration. From the priority queue and iterator invariants, we know that if each iterator only holds a single data record per key, and we end up with multiple records for the same key in the queue, these data records must have come from the different iterators.

Có thể xảy ra việc ta gặp nhiều hơn một bản ghi dữ liệu cho cùng một khóa trong khi lặp-gộp. Từ các bất biến của hàng đợi ưu tiên và bộ lặp, ta biết rằng nếu mỗi bộ lặp chỉ giữ một bản ghi dữ liệu duy nhất cho mỗi khóa, và ta kết thúc với nhiều bản ghi cho cùng một khóa trong hàng đợi, các bản ghi dữ liệu này hẳn phải đến từ các bộ lặp khác nhau.

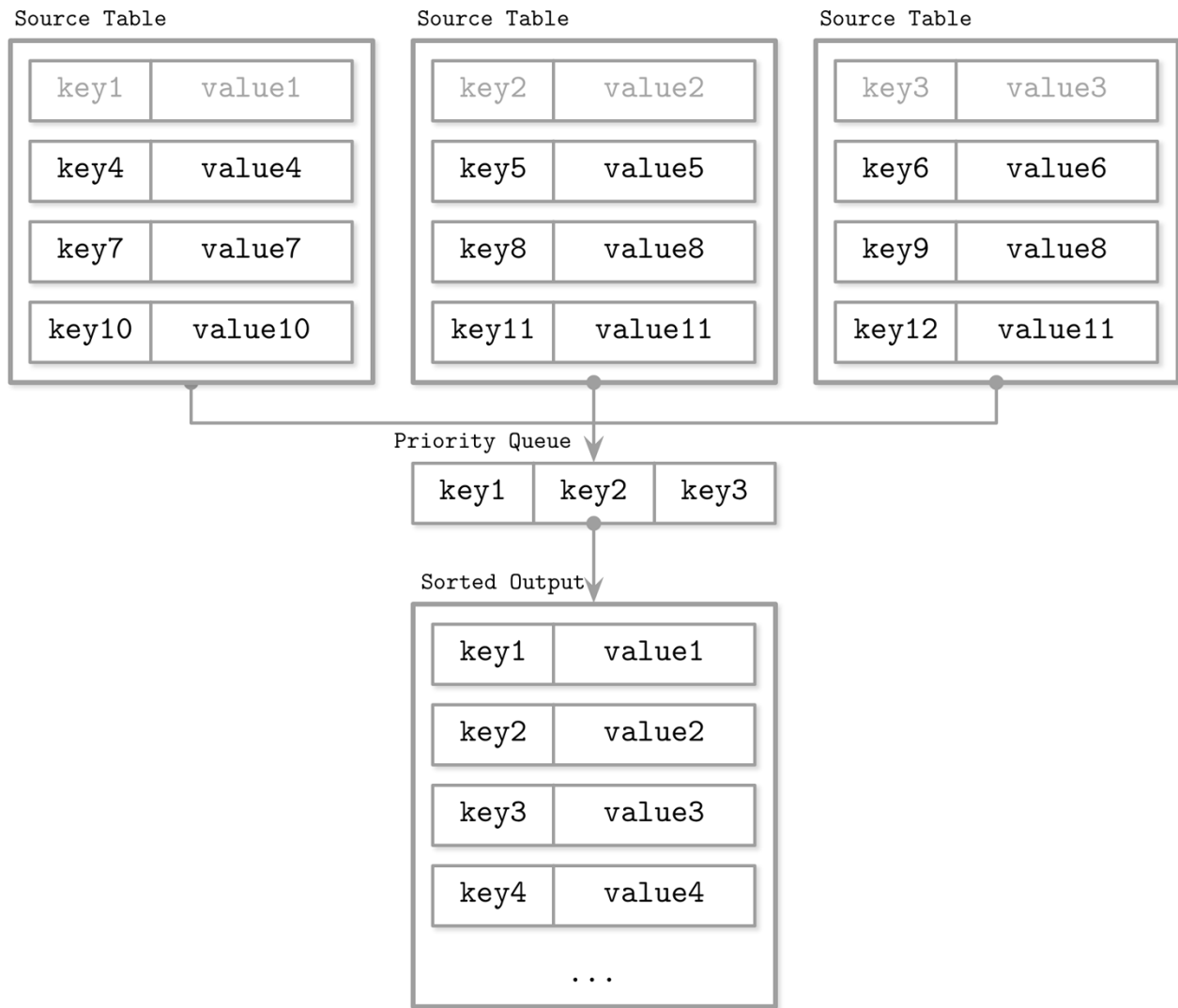


Figure 7-5. LSM merge mechanics

Hình 7-5. Cơ chế gộp của LSM

Let's follow through one example step-by-step. As input data, we have iterators over two disk-resident tables:

Hãy lần theo từng bước một ví dụ. Là dữ liệu đầu vào, ta có các bộ lặp trên hai bảng nằm trên đĩa:

Iterator 1: {k2: v1} {k4: v2} Iterator 2: {k1: v3} {k2: v4} {k3: v5}

The priority queue is filled from the iterator heads:

Hàng đợi ưu tiên được điền từ các phần đầu bộ lặp:

Iterator 1: {k4: v2} Iterator 2: {k2: v4} {k3: v5} Priority queue: {k1: v3} {k2: v1}

Key k1 is the smallest key in the queue and is appended to the result. Since it came from Iterator 2, we refill the queue from it:

Khóa k1 là khóa nhỏ nhất trong hàng đợi và được nối vào kết quả. Vì nó đến từ Iterator 2, ta điền lại hàng đợi từ nó:

Iterator 1:	Iterator 2:	Priority queue:	Merged Result:
{k4: v2}	{k3: v5}	{k2: v1} {k2: v4}	{k1: v3}

Now, we have two records for the k2 key in the queue. We can be sure there are no other records with the same key in any iterator because of the aforementioned invariants. Same-key records are merged and appended to the merged result.

Bây giờ, ta có hai bản ghi cho khóa k2 trong hàng đợi. Ta có thể chắc chắn không có bản ghi nào khác với cùng khóa trong bất kỳ bộ lặp nào nhờ các bất biến đã nêu. Các bản ghi cùng-khóa được gộp và nối vào kết quả đã gộp.

The queue is refilled with data from both iterators:

Hàng đợi được điền lại với dữ liệu từ cả hai bộ lặp:

Iterator 1:	Iterator 2:	Priority queue:	Merged Result:
{}	{}	{k3: v5} {k4: v2}	{k1: v3} {k2: v4}

Since all iterators are now empty, we append the remaining queue contents to the output:

Vì mọi bộ lặp giờ đều rỗng, ta nối nội dung còn lại của hàng đợi vào kết quả xuất:

Merged Result:
{k1: v3} {k2: v4} {k3: v5} {k4: v2}

In summary, the following steps have to be repeated to create a combined iterator:

Tóm lại, các bước sau phải được lặp lại để tạo một bộ lặp kết hợp:

1. Initially, fill the queue with the first items from each iterator.
2. Take the smallest element (head) from the queue.
3. Refill the queue from the corresponding iterator, unless this iterator is exhausted.
 1. Ban đầu, điền hàng đợi với các mục đầu tiên từ mỗi bộ lặp.
 2. Lấy phần tử nhỏ nhất (đầu) khỏi hàng đợi.
 3. Điền lại hàng đợi từ bộ lặp tương ứng, trừ khi bộ lặp này đã cạn kiệt.

In terms of complexity, merging iterators is the same as merging sorted collections. It has $O(N)$ memory overhead, where N is the number of iterators. A sorted collection of iterator heads is maintained with $O(\log N)$ (average case) [KNUTH98].

Về mặt độ phức tạp, việc gộp các bộ lặp tương tự việc gộp các tập hợp đã sắp xếp. Nó có chi phí bộ nhớ $O(N)$, với N là số bộ lặp. Một tập hợp đã sắp xếp các phần đầu bộ lặp được duy trì với $O(\log N)$ (trường hợp trung bình) [KNUTH98].

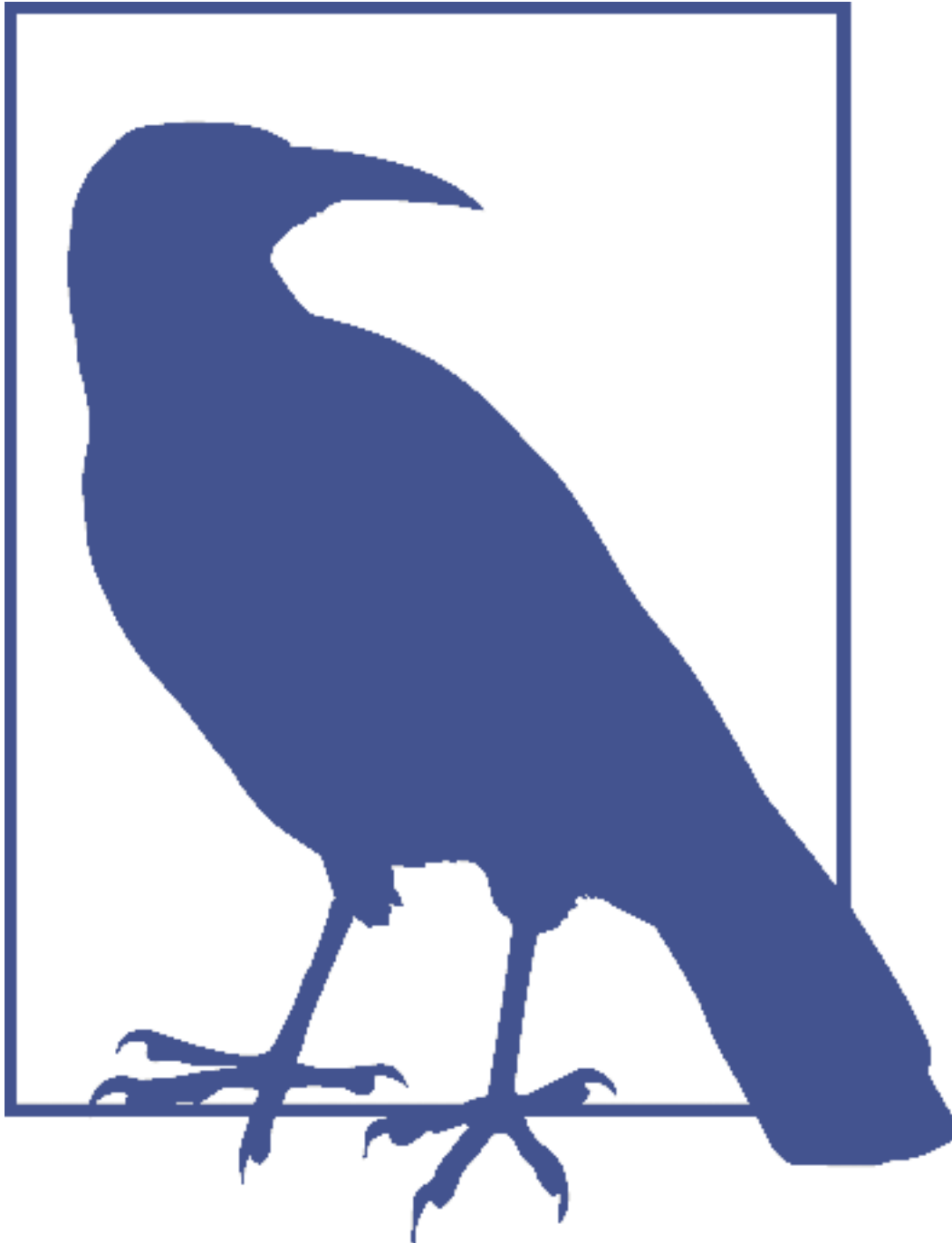
Reconciliation — Hòa giải (Reconciliation)

Merge-iteration is just a single aspect of what has to be done to merge data from multiple sources. Another important aspect is reconciliation and conflict resolution of the data records associated with the same key.

Lập-gộp chỉ là một khía cạnh của những gì phải làm để gộp dữ liệu từ nhiều nguồn. Một khía cạnh quan trọng khác là hòa giải và giải quyết xung đột của các bản ghi dữ liệu liên kết với cùng một khóa.

Different tables might hold data records for the same key, such as updates and deletes, and their contents have to be reconciled. The priority queue implementation from the preceding example must be able to allow multiple values associated with the same key and trigger reconciliation.

Các bảng khác nhau có thể giữ các bản ghi dữ liệu cho cùng một khóa, chẳng hạn các cập nhật và các xóa, và nội dung của chúng phải được hòa giải. Hiện thực hàng đợi ưu tiên từ ví dụ trước phải có khả năng cho phép nhiều giá trị liên kết với cùng một khóa và kích hoạt hòa giải.



An operation that inserts the record to the database if it does not exist, and updates an existing one otherwise, is called an upsert . In LSM Trees, insert and update operations are indistinguishable,

since they do not attempt to locate data records previously associated with the key in all sources and reassign its value, so we can say that we upsert records by default.

Một thao tác chèn bản ghi vào cơ sở dữ liệu nếu nó chưa tồn tại, và cập nhật bản ghi hiện có nếu ngược lại, được gọi là upsert. Trong LSM Tree, các thao tác chèn và cập nhật không thể phân biệt được, vì chúng không cố gắng định vị các bản ghi dữ liệu đã liên kết trước đó với khóa trong tất cả các nguồn và gán lại giá trị của nó, nên ta có thể nói rằng theo mặc định ta upsert các bản ghi.

To reconcile data records, we need to understand which one of them takes precedence. Data records hold metadata necessary for this, such as timestamps. To establish the order between the items coming from multiple sources and find out which one is more recent, we can compare their timestamps.

Để hòa giải các bản ghi dữ liệu, ta cần hiểu bản ghi nào trong số chúng được ưu tiên. Các bản ghi dữ liệu giữ siêu dữ liệu cần thiết cho việc này, chẳng hạn dấu thời gian (timestamp). Để thiết lập thứ tự giữa các mục đến từ nhiều nguồn và tìm ra mục nào mới hơn, ta có thể so sánh dấu thời gian của chúng.

Records shadowed by the records with higher timestamps are not returned to the client or written during compaction.

Các bản ghi bị che khuất bởi các bản ghi có dấu thời gian cao hơn không được trả về cho client hoặc ghi trong khi nén-gộp.

Maintenance in LSM Trees — Bảo trì trong LSM Tree

Similar to mutable B-Trees, LSM Trees require maintenance. The nature of these processes is heavily influenced by the invariants these algorithms preserve.

Tương tự B-Tree khả biến, LSM Tree đòi hỏi bảo trì. Bản chất của các tiến trình này chịu ảnh hưởng nặng nề bởi các bất biến mà các thuật toán này bảo toàn.

In B-Trees, the maintenance process collects unreferenced cells and defragments the pages, reclaiming the space occupied by removed and shadowed records. In LSM Trees, the number of disk-resident tables is constantly growing, but can be reduced by triggering periodic compaction.

Trong B-Tree, tiến trình bảo trì thu gom các ô không còn được tham chiếu và chống phân mảnh các trang, thu hồi không gian bị các bản ghi đã gỡ bỏ và bị che khuất chiếm giữ. Trong LSM Tree, số lượng các bảng nằm trên đĩa liên tục tăng, nhưng có thể được giảm bằng cách kích hoạt nén-gộp định kỳ.

Compaction picks multiple disk-resident tables, iterates over their entire contents using the aforementioned merge and reconciliation algorithms, and writes out the results into the newly created table.

Compaction chọn nhiều bảng nằm trên đĩa, lặp qua toàn bộ nội dung của chúng bằng các thuật toán gộp và hòa giải đã nêu, và ghi kết quả ra bảng mới được tạo.

Since disk-resident table contents are sorted, and because of the way merge-sort works, compaction has a theoretical memory usage upper bound, since it should only hold iterator heads in memory. All table contents are consumed sequentially, and the resulting merged data is also written out sequentially. These details may vary between implementations due to additional optimizations.

Vì nội dung các bảng nằm trên đĩa đã được sắp xếp, và vì cách sắp xếp trộn hoạt động, nén-gộp có một cận trên lý thuyết về việc dùng bộ nhớ, vì nó chỉ phải giữ các phần đầu bộ

lập trong bộ nhớ. Mọi nội dung bảng được tiêu thụ tuần tự, và dữ liệu đã gộp kết quả cũng được ghi ra tuần tự. Các chi tiết này có thể khác nhau giữa các hiện thực do các tối ưu bổ sung.

Compacting tables remain available for reads until the compaction process finishes, which means that for the duration of compaction, it is required to have enough free space available on disk for a compacted table to be written.

Các bảng đang nén-gộp vẫn khả dụng để đọc cho tới khi tiến trình nén-gộp kết thúc, nghĩa là trong suốt thời gian nén-gộp, cần có đủ không gian trống trên đĩa để ghi một bảng đã nén-gộp.

At any given time, multiple compactions can be executed in the system. However, these concurrent compactions usually work on nonintersecting sets of tables. A compaction writer can both merge several tables into one and partition one table into multiple tables.

Tại bất kỳ thời điểm nào, nhiều lần nén-gộp có thể được thực thi trong hệ thống. Tuy nhiên, các lần nén-gộp đồng thời này thường làm việc trên các tập bảng không giao nhau. Một bộ ghi nén-gộp có thể vừa gộp vài bảng thành một vừa phân hoạch một bảng thành nhiều bảng.

Tombstones and Compaction — Tombstone và Nén-gộp

Tombstones represent an important piece of information required for correct reconciliation, as some other table might still hold an outdated data record shadowed by the tombstone.

Tombstone biểu diễn một mẫu thông tin quan trọng cần thiết cho việc hòa giải chính xác, vì một bảng nào đó khác có thể vẫn giữ một bản ghi dữ liệu lỗi thời bị che khuất bởi tombstone.

During compaction, tombstones are not dropped right away. They are preserved until the storage engine can be certain that no data record for the same key with a smaller timestamp is present in any other table. RocksDB keeps tombstones until they reach the bottommost level. Apache Cassandra keeps tombstones until the GC (**garbage collection**) grace period is reached because of the eventually consistent nature of the database, ensuring that other nodes observe the tombstone. Preserving tombstones during compaction is important to avoid data resurrection.

Trong khi nén-gộp, tombstone không bị loại bỏ ngay lập tức. Chúng được bảo toàn cho tới khi bộ máy lưu trữ có thể chắc chắn rằng không có bản ghi dữ liệu nào cho cùng một khóa với dấu thời gian nhỏ hơn hiện diện trong bất kỳ bảng nào khác. RocksDB giữ tombstone cho tới khi chúng đạt mức thấp nhất (bottommost level). Apache Cassandra giữ tombstone cho tới khi đạt khoảng thời gian ân hạn GC (thu gom rác) do bản chất nhất quán cuối cùng (eventual consistency) của cơ sở dữ liệu, bảo đảm rằng các nút khác quan sát được tombstone. Việc bảo toàn tombstone trong khi nén-gộp là quan trọng để tránh làm sống lại dữ liệu.

Leveled compaction

Nén-gộp theo mức (Leveled compaction)

Compaction opens up multiple opportunities for optimizations, and there are many different compaction strategies. One of the frequently implemented compaction strategies is called leveled compaction. For example, it is used by RocksDB.

Nén-gộp mở ra nhiều cơ hội tối ưu, và có nhiều chiến lược nén-gộp khác nhau. Một trong những chiến lược nén-gộp thường được hiện thực gọi là nén-gộp theo mức. Ví dụ, nó được dùng bởi RocksDB.

Leveled compaction separates disk-resident tables into levels. Tables on each level have target sizes, and each level has a corresponding index number (identifier). Somewhat counterintuitively, the level with the highest index is called the bottommost level. For clarity, this section avoids using terms higher and lower level and uses the same qualifiers for level index. That is, since 2 is larger than 1, level 2 has a higher index than level 1. The terms previous and next have the same order semantics as level indexes.

Nén-gộp theo mức tách các bảng nằm trên đĩa thành các mức. Các bảng trên mỗi mức có kích thước mục tiêu, và mỗi mức có một số chỉ số (định danh) tương ứng. Có phần phân trực giác, mức có chỉ số cao nhất được gọi là mức thấp nhất (bottommost level). Để rõ ràng, phần này tránh dùng các thuật ngữ mức cao hơn và mức thấp hơn và dùng cùng các từ định tính cho chỉ số mức. Tức là, vì 2 lớn hơn 1, mức 2 có chỉ số cao hơn mức 1. Các thuật ngữ trước và tiếp theo có cùng ngữ nghĩa thứ tự như các chỉ số mức.

Level-0 tables are created by flushing memtable contents. Tables in level 0 may contain overlapping key ranges. As soon as the number of tables on level 0 reaches a threshold, their contents are merged, creating new tables for level 1.

Các bảng mức-0 được tạo bằng cách xả nội dung memtable. Các bảng ở mức 0 có thể chứa các dải khóa chồng lấn. Ngay khi số bảng ở mức 0 đạt một ngưỡng, nội dung của chúng được gộp, tạo các bảng mới cho mức 1.

Key ranges for the tables on level 1 and all levels with a higher index do not overlap, so level-0 tables have to be partitioned during compaction, split into ranges, and merged with tables holding corresponding key ranges. Alternatively, compaction can include all level-0 and level-1 tables, and output partitioned level-1 tables.

Các dải khóa cho các bảng ở mức 1 và mọi mức có chỉ số cao hơn không chồng lấn, nên các bảng mức-0 phải được phân hoạch trong khi nén-gộp, chia thành các dải, và gộp với các bảng giữ các dải khóa tương ứng. Một cách khác, nén-gộp có thể bao gồm mọi bảng mức-0 và mức-1, và xuất ra các bảng mức-1 đã phân hoạch.

Compactions on the levels with the higher indexes pick tables from two consecutive levels with overlapping ranges and produce a new table on a higher level. Figure 7-6 schematically shows how the compaction process migrates data between the levels. The process of compacting level-1 and level-2 tables will produce a new table on level

Các lần nén-gộp ở các mức có chỉ số cao hơn chọn các bảng từ hai mức liên tiếp có các dải chồng lấn và tạo ra một bảng mới ở mức cao hơn. Hình 7-6 cho thấy một cách sơ đồ tiến trình nén-gộp di chuyển dữ liệu giữa các mức như thế nào. Tiến trình nén-gộp các bảng mức-1 và mức-2 sẽ tạo ra một bảng mới ở mức

2. Depending on how tables are partitioned, multiple tables from one level can be picked for compaction.

2. Tùy thuộc vào cách các bảng được phân hoạch, nhiều bảng từ một mức có thể được chọn để nén-gộp.

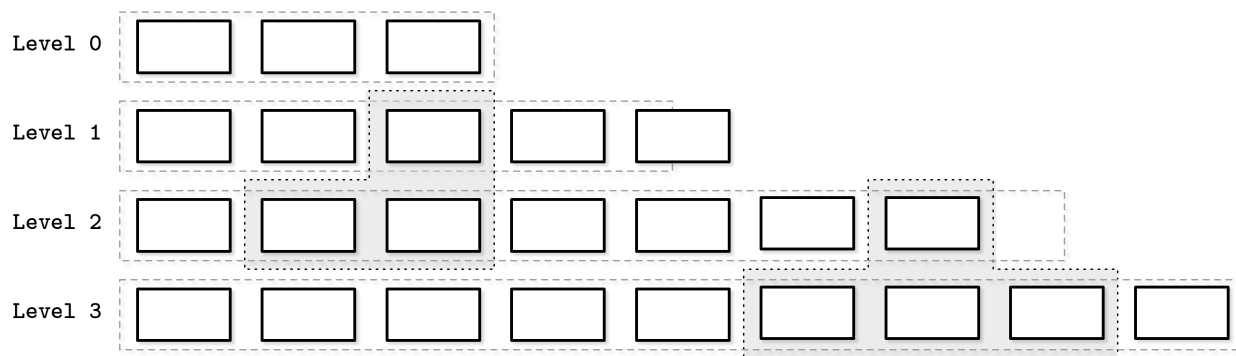


Figure 7-6. Compaction process. Gray boxes with dashed lines represent currently compacting tables. Level-wide boxes represent the target data size limit on the level. Level 1 is over the limit.

Hình 7-6. Tiến trình nén-gộp. Các hộp màu xám viền nét đứt biểu diễn các bảng hiện đang nén-gộp. Các hộp rộng hơn mức biểu diễn giới hạn kích thước dữ liệu mục tiêu của mức. Mức 1 vượt quá giới hạn.

Keeping different key ranges in the distinct tables reduces the number of tables accessed during the read. This is done by inspecting the table metadata and filtering out the tables whose ranges do not contain a searched key.

Việc giữ các dải khóa khác nhau trong các bảng riêng biệt làm giảm số lượng bảng được truy cập trong khi đọc. Điều này được thực hiện bằng cách kiểm tra siêu dữ liệu bảng và loại bỏ các bảng có dải không chứa khóa được tìm.

Each level has a limit on the table size and the maximum number of tables. As soon as the number of tables on level 1 or any level with a higher index reaches a threshold, tables from the current level are merged with tables on the next level holding the overlapping key range.

Mỗi mức có một giới hạn về kích thước bảng và số lượng bảng tối đa. Ngay khi số bảng ở mức 1 hoặc bất kỳ mức nào có chỉ số cao hơn đạt một ngưỡng, các bảng từ mức hiện tại được gộp với các bảng ở mức tiếp theo giữ dải khóa chồng lấn.

Sizes grow exponentially between the levels: tables on each next level are exponentially larger than tables on the previous one. This way, the freshest data is always on the level with the lowest index, and older data gradually migrates to the higher ones.

Kích thước tăng theo cấp số nhân giữa các mức: các bảng ở mỗi mức tiếp theo lớn hơn theo cấp số nhân so với các bảng ở mức trước. Theo cách này, dữ liệu mới nhất luôn ở mức có chỉ số thấp nhất, và dữ liệu cũ hơn dần dần di chuyển lên các mức cao hơn.

Size-tiered compaction

Nén-gộp theo tầng kích thước (Size-tiered compaction)

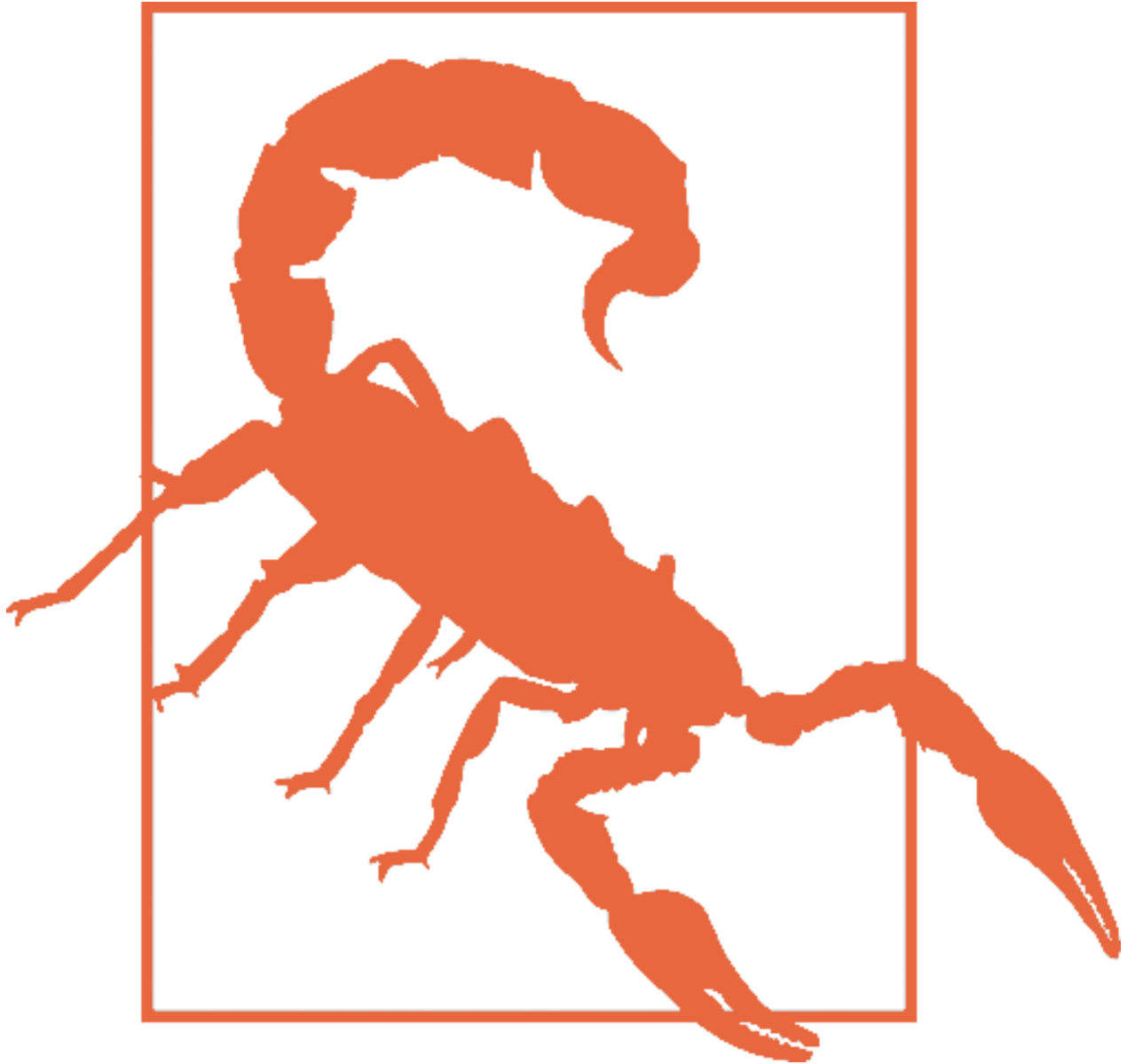
Another popular compaction strategy is called size-tiered compaction . In size-tiered compaction, rather than grouping disk-resident tables based on their level, they're grouped by size: smaller tables are grouped with smaller ones, and bigger tables are grouped with bigger ones.

Một chiến lược nén-gộp phổ biến khác gọi là nén-gộp theo tầng kích thước. Trong nén-gộp theo tầng kích thước, thay vì nhóm các bảng nằm trên đĩa dựa trên mức của chúng, chúng được nhóm theo kích thước: các bảng nhỏ hơn được nhóm với các bảng nhỏ hơn, và các bảng lớn hơn được nhóm với các bảng lớn hơn.

Level 0 holds the smallest tables that were either flushed from memtables or created by the compaction process. When the tables are compacted, the resulting merged table is written to the

level holding tables with corresponding sizes. The process continues recursively incrementing levels, compacting and promoting larger tables to higher levels, and demoting smaller tables to lower levels.

Mức 0 giữ các bảng nhỏ nhất hoặc được xả từ các memtable hoặc được tạo bởi tiến trình nén-gộp. Khi các bảng được nén-gộp, bảng đã gộp kết quả được ghi vào mức giữ các bảng có kích thước tương ứng. Tiến trình tiếp tục đệ quy tăng dần các mức, nén-gộp và đẩy các bảng lớn hơn lên các mức cao hơn, và hạ các bảng nhỏ hơn xuống các mức thấp hơn.



One of the problems with size-tiered compaction is called table starvation : if compacted tables are still small enough after compaction (e.g., records were shadowed by the tombstones and did not make it to the merged table), higher levels may get starved of compaction and their tombstones will not be taken into consideration, increasing the cost of reads. In this case, compaction has to be forced for a level, even if it doesn't contain enough tables.

Một trong những vấn đề của nén-gộp theo tầng kích thước được gọi là sự bỏ đói bảng (table

starvation): nếu các bảng đã nén-gộp vẫn còn đủ nhỏ sau khi nén-gộp (ví dụ, các bản ghi bị che khuất bởi các tombstone và không lọt vào bảng đã gộp), các mức cao hơn có thể bị bỏ đối nén-gộp và các tombstone của chúng sẽ không được xét đến, làm tăng chi phí đọc. Trong trường hợp này, nén-gộp phải được ép buộc cho một mức, ngay cả khi nó không chứa đủ số bảng.

There are other commonly implemented compaction strategies that might optimize for different workloads. For example, Apache Cassandra also implements a time window compaction strategy, which is particularly useful for time-series workloads with records for which time-to-live is set (in other words, items have to be expired after a given time period).

Có các chiến lược nén-gộp thường được hiện thực khác có thể tối ưu cho các tải công việc khác nhau. Ví dụ, Apache Cassandra cũng hiện thực một chiến lược nén-gộp theo cửa sổ thời gian (time window compaction strategy), vốn đặc biệt hữu ích cho các tải công việc chuỗi thời gian với các bản ghi có thời gian sống (time-to-live) được đặt (nói cách khác, các mục phải hết hạn sau một khoảng thời gian cho trước).

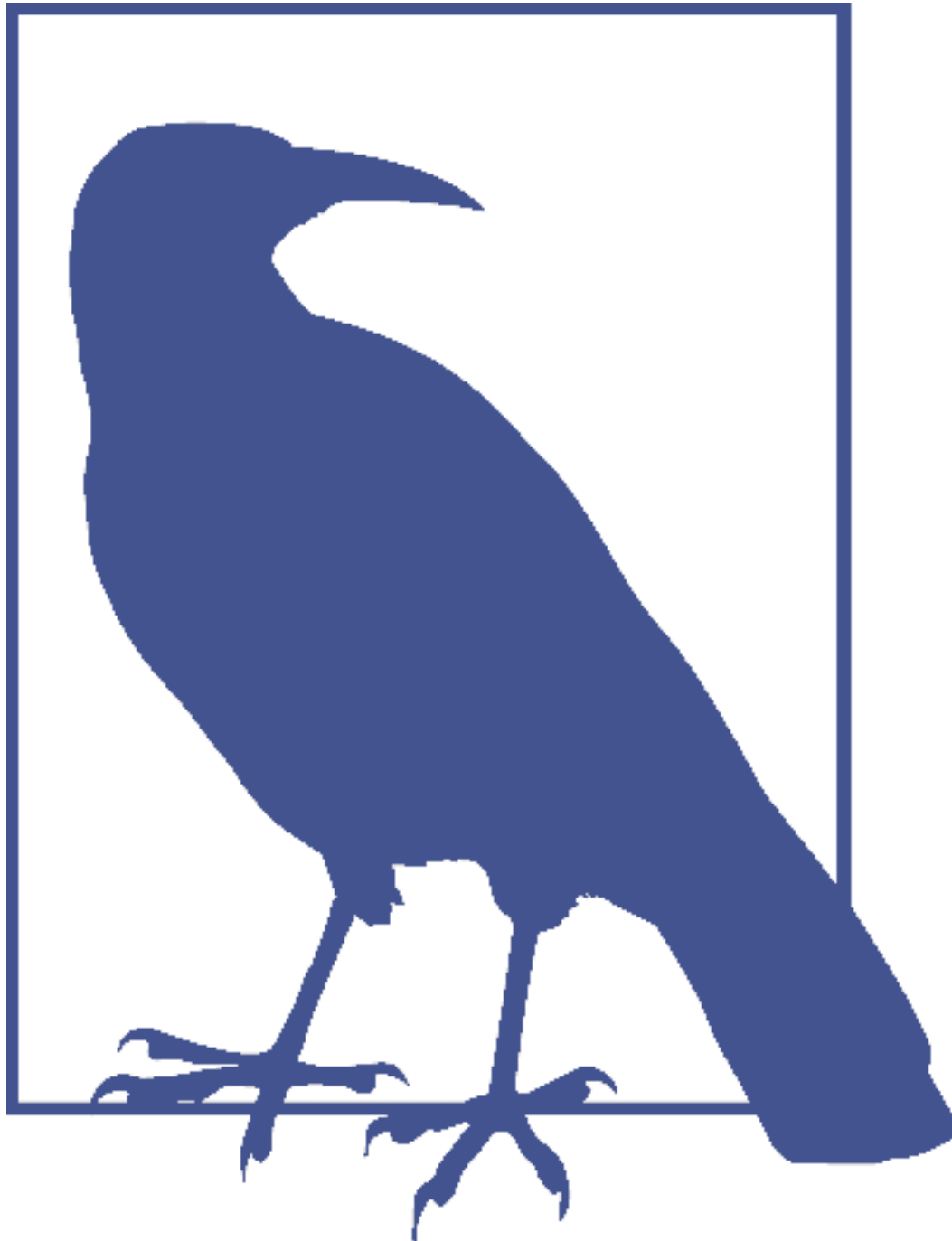
The time window compaction strategy takes write timestamps into consideration and allows dropping entire files that hold data for an already expired time range without requiring us to compact and rewrite their contents.

Chiến lược nén-gộp theo cửa sổ thời gian xét đến các dấu thời gian ghi và cho phép loại bỏ toàn bộ các tệp giữ dữ liệu cho một dải thời gian đã hết hạn mà không cần ta nén-gộp và ghi lại nội dung của chúng.

Read, Write, and Space Amplification — Khuếch đại đọc, ghi và không gian

When implementing an optimal compaction strategy, we have to take multiple factors into consideration. One approach is to reclaim space occupied by duplicate records and reduce space overhead, which results in higher write amplification caused by re-writing tables continuously. The alternative is to avoid rewriting the data continuously, which increases **read amplification** (overhead from reconciling data records associated with the same key during the read), and **space amplification** (since redundant records are preserved for a longer time).

Khi hiện thực một chiến lược nén-gộp tối ưu, ta phải xét đến nhiều yếu tố. Một cách tiếp cận là thu hồi không gian bị các bản ghi trùng lặp chiếm giữ và giảm chi phí không gian, điều này dẫn đến khuếch đại ghi cao hơn do liên tục ghi lại các bảng. Cách thay thế là tránh ghi lại dữ liệu liên tục, điều này làm tăng khuếch đại đọc (read amplification) (chi phí từ việc hòa giải các bản ghi dữ liệu liên kết với cùng một khóa trong khi đọc), và khuếch đại không gian (space amplification) (vì các bản ghi dư thừa được bảo toàn trong thời gian dài hơn).



One of the big disputes in the database community is whether B- Trees or LSM Trees have lower write amplification. It is extremely important to understand the source of write amplification in both cases. In B-Trees, it comes from writeback operations and subsequent updates to the same node. In LSM Trees, write amplification is caused by migrating data from one file to the other during compaction. Comparing the two directly may lead to incorrect assumptions.

Một trong những tranh luận lớn trong cộng đồng cơ sở dữ liệu là liệu B-Tree hay LSM Tree có khuếch đại ghi thấp hơn. Hiểu rõ nguồn gốc của khuếch đại ghi trong cả hai trường hợp là cực kỳ quan trọng. Trong B-Tree, nó đến từ các thao tác ghi ngược và các cập nhật tiếp theo lên cùng một nút. Trong LSM Tree, khuếch đại ghi gây ra bởi việc di chuyển dữ liệu

từ tệp này sang tệp khác trong khi nén-gộp. So sánh hai cái này một cách trực tiếp có thể dẫn đến các giả định sai.

In summary, when storing data on disk in an immutable fashion, we face three problems:

Tóm lại, khi lưu dữ liệu trên đĩa theo cách bất biến, ta đối mặt với ba vấn đề:

Read amplification Resulting from a need to address multiple tables to retrieve data.

Khuếch đại đọc Phát sinh từ nhu cầu phải xử lý nhiều bảng để truy xuất dữ liệu.

Write amplification Caused by continuous rewrites by the compaction process.

Khuếch đại ghi Gây ra bởi các lần ghi lại liên tục của tiến trình nén-gộp.

Space amplification Arising from storing multiple records associated with the same key.

Khuếch đại không gian Phát sinh từ việc lưu nhiều bản ghi liên kết với cùng một khóa.

We'll be addressing each one of these throughout the rest of the chapter.

Chúng ta sẽ giải quyết từng vấn đề này xuyên suốt phần còn lại của chương.

RUM Conjecture — Giả thuyết RUM (RUM Conjecture)

One of the popular cost models for storage structures takes three factors into consideration: Read, Update, and Memory overheads. It is called RUM Conjecture [ATHA- NASSOULIS16].

Một trong những mô hình chi phí phổ biến cho các cấu trúc lưu trữ xét đến ba yếu tố: chi phí Đọc (Read), Cập nhật (Update), và Bộ nhớ (Memory). Nó được gọi là Giả thuyết RUM [ATHA-NASSOULIS16].

RUM Conjecture states that reducing two of these overheads inevitably leads to change for the worse in the third one, and that optimizations can be done only at the expense of one of the three parameters. We can compare different storage engines in terms of these three parameters to understand which ones they optimize for, and which potential trade-offs this may imply.

Giả thuyết RUM phát biểu rằng việc giảm hai trong số các chi phí này tất yếu dẫn đến sự thay đổi theo hướng xấu đi ở chi phí thứ ba, và rằng các tối ưu chỉ có thể được thực hiện bằng cách đánh đổi một trong ba tham số. Ta có thể so sánh các bộ máy lưu trữ khác nhau theo ba tham số này để hiểu chúng tối ưu cho cái nào, và những đánh đổi tiềm năng nào điều này có thể ngụ ý.

An ideal solution would provide the lowest read cost while maintaining low memory and write overheads, but in reality, this is not achievable, and we are presented with a trade-off.

Một giải pháp lý tưởng sẽ cung cấp chi phí đọc thấp nhất trong khi duy trì chi phí bộ nhớ và ghi thấp, nhưng trong thực tế, điều này không thể đạt được, và ta phải đối mặt với một đánh đổi.

B-Trees are read-optimized. Writes to the B-Tree require locating a record on disk, and subsequent writes to the same page might have to update the page on disk multiple times. Reserved extra space for future updates and deletes increases space overhead.

B-Tree được tối ưu cho đọc. Việc ghi vào B-Tree đòi hỏi định vị một bản ghi trên đĩa, và các lần ghi tiếp theo lên cùng một trang có thể phải cập nhật trang trên đĩa nhiều lần. Không gian dự dành cho các cập nhật và xóa tương lai làm tăng chi phí không gian.

LSM Trees do not require locating the record on disk during write and do not reserve extra space for future writes. There is still some space overhead resulting from storing redundant records. In a default configuration, reads are more expensive, since multiple tables have to be accessed to return complete results. However, optimizations we discuss in this chapter help to mitigate this problem.

LSM Tree không đòi hỏi định vị bản ghi trên đĩa trong khi ghi và không dành thêm không gian cho các lần ghi tương lai. Vẫn còn một số chi phí không gian phát sinh từ việc lưu các bản ghi dư thừa. Trong cấu hình mặc định, việc đọc tốn kém hơn, vì nhiều bảng phải được truy cập để trả về các kết quả hoàn chỉnh. Tuy nhiên, các tối ưu mà ta thảo luận trong chương này giúp giảm nhẹ vấn đề này.

As we've seen in the chapters about B-Trees, and will see in this chapter, there are ways to improve these characteristics by applying different optimizations.

Như ta đã thấy trong các chương về B-Tree, và sẽ thấy trong chương này, có những cách để cải thiện các đặc tính này bằng cách áp dụng các tối ưu khác nhau.

This cost model is not perfect, as it does not take into account other important metrics such as latency, access patterns, implementation complexity, maintenance overhead, and hardware-related specifics. Higher-level concepts important for distributed databases, such as consistency implications and replication overhead, are also not considered. However, this model can be used as a first approximation and a rule of thumb as it helps understand what the storage engine has to offer.

Mô hình chi phí này không hoàn hảo, vì nó không xét đến các chỉ số quan trọng khác như độ trễ (latency), mẫu truy cập, độ phức tạp hiện thực, chi phí bảo trì, và các đặc thù liên quan đến phần cứng. Các khái niệm mức cao hơn quan trọng đối với các cơ sở dữ liệu phân tán, chẳng hạn các hệ quả về nhất quán và chi phí sao bản (replication), cũng không được xét đến. Tuy nhiên, mô hình này có thể được dùng như một xấp xỉ ban đầu và một quy tắc kinh nghiệm vì nó giúp hiểu bộ máy lưu trữ phải cung cấp những gì.

Implementation Details — Chi tiết hiện thực

We've covered the basic dynamics of LSM Trees: how data is read, written, and compacted. However, there are some other things that many LSM Tree implementations have in common that are worth discussing: how memory- and disk-resident tables are implemented, how secondary indexes work, how to reduce the number of disk-resident tables accessed during read and, finally, new ideas related to **log-structured storage**.

Chúng ta đã đề cập đến động lực học cơ bản của LSM Tree: cách dữ liệu được đọc, ghi và nén-gộp. Tuy nhiên, có một số điều khác mà nhiều hiện thực LSM Tree có chung đáng để thảo luận: cách các bảng nằm trong bộ nhớ và nằm trên đĩa được hiện thực, cách các chỉ mục phụ (secondary index) hoạt động, cách giảm số lượng bảng nằm trên đĩa được truy cập trong khi đọc, và cuối cùng, các ý tưởng mới liên quan đến lưu trữ dạng nhật ký.

Sorted String Tables — Sorted String Tables

So far we've discussed the hierarchical and logical structure of LSM Trees (that they consist of multiple memory- and disk-resident components), but have not yet discussed how disk-resident tables are implemented and how their design plays together with the rest of the system.

Cho đến giờ chúng ta đã thảo luận cấu trúc phân cấp và logic của LSM Tree (rằng chúng gồm nhiều thành phần nằm trong bộ nhớ và nằm trên đĩa), nhưng chưa thảo luận cách các bảng nằm trên đĩa được hiện thực và cách thiết kế của chúng phối hợp với phần còn lại của hệ thống.

Disk-resident tables are often implemented using Sorted String Tables (SSTables). As the name suggests, data records in SSTables are sorted and laid out in key order. SSTables usually consist of two components: index files and data files. Index files are implemented using some structure allowing logarithmic lookups, such as B-Trees, or constant-time lookups, such as hashtables.

Các bảng nằm trên đĩa thường được hiện thực bằng Sorted String Tables (SSTable). Như tên gọi gợi ý, các bản ghi dữ liệu trong SSTable được sắp xếp và bố trí theo thứ tự khóa. SSTable thường gồm hai thành phần: các tệp chỉ mục và các tệp dữ liệu. Các tệp chỉ mục được hiện thực bằng một cấu trúc nào đó cho phép tra cứu logarit, chẳng hạn B-Tree, hoặc tra cứu thời gian hằng số, chẳng hạn bảng băm (hashtable).

Since data files hold records in key order, using hashtables for indexing does not prevent us from implementing range scans, as a hashtable is only accessed to locate the first key in the range, and the range itself can be read from the data file sequentially while the range predicate still matches.

Vì các tệp dữ liệu giữ các bản ghi theo thứ tự khóa, việc dùng bảng băm để lập chỉ mục không ngăn ta hiện thực quét dải (range scan), bởi bảng băm chỉ được truy cập để định vị khóa đầu tiên trong dải, và bản thân dải có thể được đọc từ tệp dữ liệu một cách tuần tự chừng nào vị từ dải vẫn còn khớp.

The index component holds keys and data entries (offsets in the data file where the actual data records are located). The data component consists of concatenated key-value pairs. The cell design and data record formats we discussed in Chapter 3 are largely applicable to SSTables. The main difference here is that cells are written sequentially and are not modified during the life cycle of the **SSTable**. Since the index files hold pointers to the data records stored in the data file, their offsets have to be known by the time the index is created.

Thành phần chỉ mục giữ các khóa và các mục dữ liệu (các độ dời trong tệp dữ liệu nơi các bản ghi dữ liệu thực sự nằm). Thành phần dữ liệu gồm các cặp khóa-giá trị được nối liền. Thiết kế ô và các định dạng bản ghi dữ liệu mà ta đã thảo luận trong Chương 3 phần lớn áp dụng được cho SSTable. Khác biệt chính ở đây là các ô được ghi tuần tự và không bị sửa đổi trong vòng đời của SSTable. Vì các tệp chỉ mục giữ các con trỏ tới các bản ghi dữ liệu được lưu trong tệp dữ liệu, các độ dời của chúng phải được biết tại thời điểm chỉ mục được tạo.

During compaction, data files can be read sequentially without addressing the index component, as data records in them are already ordered. Since tables merged during compaction have the same order, and merge-iteration is order-preserving, the resulting merged table is also created by writing data records sequentially in a single run. As soon as the file is fully written, it is considered immutable, and its disk-resident contents are not modified.

Trong khi nén-gộp, các tệp dữ liệu có thể được đọc tuần tự mà không cần xử lý thành phần chỉ mục, vì các bản ghi dữ liệu trong chúng đã được sắp xếp. Vì các bảng được gộp trong khi nén-gộp có cùng thứ tự, và lập-gộp bảo toàn thứ tự, bảng đã gộp kết quả cũng được tạo bằng cách ghi các bản ghi dữ liệu tuần tự trong một lượt chạy duy nhất. Ngay khi tệp được ghi hoàn toàn, nó được coi là bất biến, và nội dung nằm trên đĩa của nó không bị sửa đổi.

SSTable-Attached Secondary Indexes — Chỉ mục phụ gắn với SSTable (SSTable-Attached Secondary Indexes)

One of the interesting developments in the area of LSM Tree indexing is SSTable-Attached Secondary Indexes (SASI) implemented in Apache Cassandra. To allow indexing table contents not just by the primary key, but also by any other field, index structures and their life cycles are coupled with the SSTable life cycle, and an index is created per SSTable. When the memtable is flushed, its contents are written to disk, and **secondary index** files are created along with the SSTable primary key index.

Một trong những phát triển thú vị trong lĩnh vực lập chỉ mục LSM Tree là Chỉ mục phụ gắn với SSTable (SASI) được hiện thực trong Apache Cassandra. Để cho phép lập chỉ mục nội dung bảng không chỉ theo khóa chính, mà còn theo bất kỳ trường nào khác, các cấu trúc chỉ mục và vòng đời của chúng được ghép với vòng đời SSTable, và một chỉ mục được tạo cho mỗi SSTable. Khi memtable được xả, nội dung của nó được ghi xuống đĩa, và các tệp chỉ mục phụ được tạo cùng với chỉ mục khóa chính của SSTable.

Since LSM Trees buffer data in memory and indexes have to work for memory-resident contents as well as the disk-resident ones, SASI maintains a separate in-memory structure, indexing memtable contents.

Vì LSM Tree đệm dữ liệu trong bộ nhớ và các chỉ mục phải hoạt động cho cả nội dung nằm trong bộ nhớ lẫn nội dung nằm trên đĩa, SASI duy trì một cấu trúc riêng nằm trong bộ nhớ, lập chỉ mục nội dung memtable.

During a read, primary keys of searched records are located by searching and merging index contents, and data records are merged and reconciled similar to how lookups usually work in LSM Trees.

Trong khi đọc, các khóa chính của các bản ghi được tìm được định vị bằng cách tìm kiếm và gộp nội dung chỉ mục, và các bản ghi dữ liệu được gộp và hòa giải tương tự cách tra cứu thường hoạt động trong LSM Tree.

One of the advantages of piggybacking the SSTable life cycle is that indexes can be created during memtable flush or compaction.

Một trong những ưu điểm của việc ăn theo vòng đời SSTable là các chỉ mục có thể được tạo trong khi xả memtable hoặc nén-gộp.

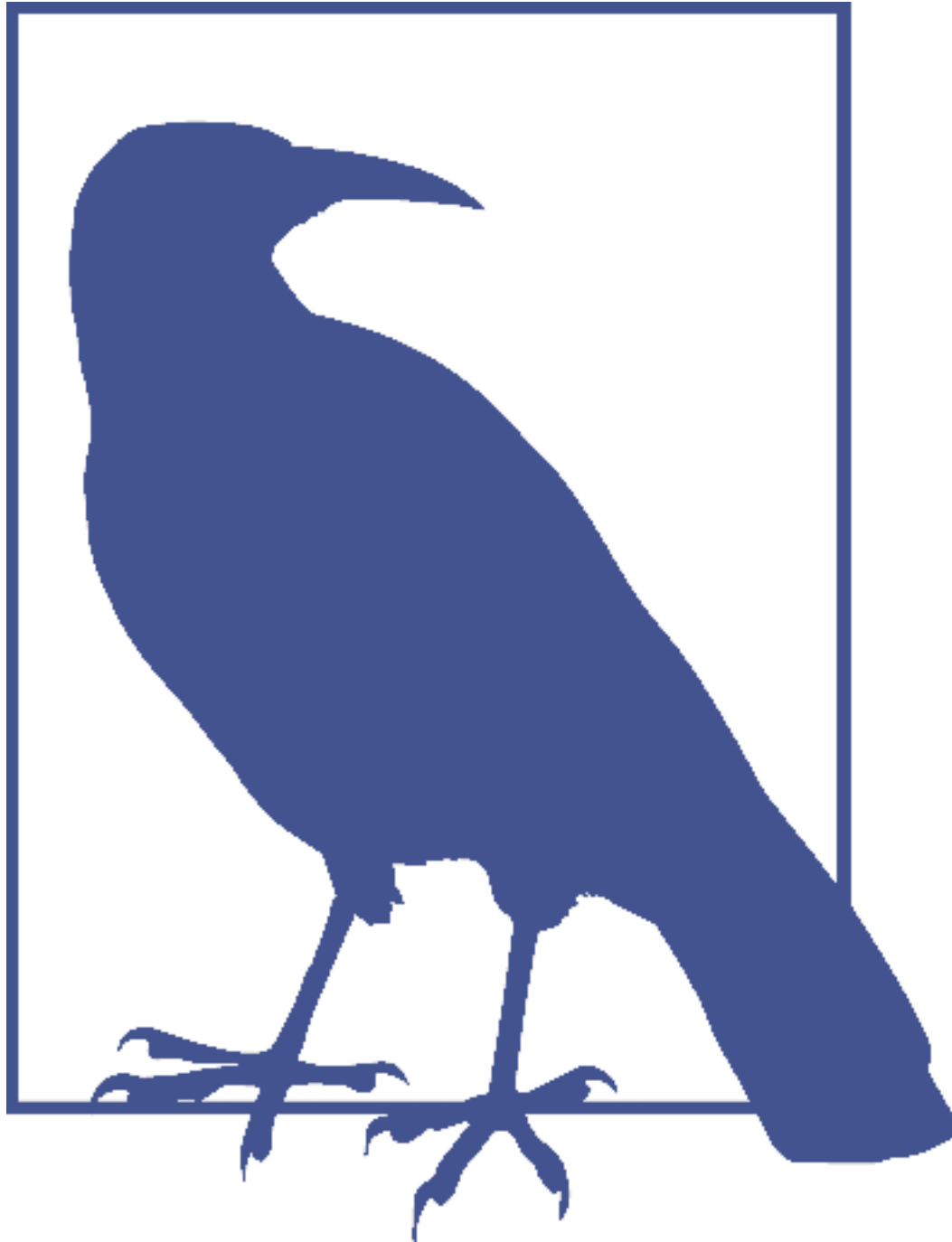
Bloom Filters — Bộ lọc Bloom (Bloom Filters)

The source of read amplification in LSM Trees is that we have to address multiple disk-resident tables for the read operation to complete. This happens because we do not always know up front whether or not a disk-resident table contains a data record for the searched key.

Nguồn gốc của khuếch đại đọc trong LSM Tree là việc ta phải xử lý nhiều bảng nằm trên đĩa để hoàn thành thao tác đọc. Điều này xảy ra vì ta không phải lúc nào cũng biết trước liệu một bảng nằm trên đĩa có chứa một bản ghi dữ liệu cho khóa được tìm hay không.

One of the ways to prevent table lookup is to store its key range (smallest and largest keys stored in the given table) in metadata, and check if the searched key belongs to the range of that table. This information is imprecise and can only tell us if the data record can be present in the table. To improve this situation, many implementations, including Apache Cassandra and RocksDB, use a data structure called a **Bloom filter**.

Một trong những cách để ngăn việc tra cứu bảng là lưu dải khóa của nó (khóa nhỏ nhất và lớn nhất được lưu trong bảng đó) trong siêu dữ liệu, và kiểm tra xem khóa được tìm có thuộc dải của bảng đó hay không. Thông tin này không chính xác và chỉ có thể cho ta biết liệu bản ghi dữ liệu có thể hiện diện trong bảng. Để cải thiện tình huống này, nhiều hiện thực, bao gồm Apache Cassandra và RocksDB, dùng một cấu trúc dữ liệu gọi là bộ lọc Bloom (Bloom filter).



Probabilistic data structures are generally more space efficient than their “regular” counterparts. For example, to check set membership, cardinality (find out the number of distinct elements in a set), or frequency (find out how many times a certain element has been encountered), we would have to

store all set elements and go through the entire dataset to find the result. Probabilistic structures allow us to store approximate information and perform queries that yield results with an element of uncertainty. Some commonly known examples of such data structures are a Bloom filter (for set membership), HyperLogLog (for cardinality estimation) [FLAJO-LET12], and Count-Min Sketch (for frequency estimation) [COR-MODE12].

Các cấu trúc dữ liệu xác suất nói chung tiết kiệm không gian hơn các đối tác “thông thường” của chúng. Ví dụ, để kiểm tra tư cách thành viên tập hợp, lực lượng tập hợp (tìm ra số phần tử phân biệt trong một tập hợp), hoặc tần suất (tìm ra một phần tử nào đó đã được gặp bao nhiêu lần), ta sẽ phải lưu tất cả các phần tử của tập hợp và duyệt qua toàn bộ tập dữ liệu để tìm kết quả. Các cấu trúc xác suất cho phép ta lưu thông tin xấp xỉ và thực hiện các truy vấn cho ra kết quả có một yếu tố không chắc chắn. Một số ví dụ thường biết của các cấu trúc dữ liệu như vậy là bộ lọc Bloom (cho tư cách thành viên tập hợp), HyperLogLog (cho ước lượng lực lượng) [FLAJO-LET12], và Count-Min Sketch (cho ước lượng tần suất) [COR-MODE12].

A Bloom filter, conceived by Burton Howard Bloom in 1970 [BLOOM70], is a space-efficient probabilistic data structure that can be used to test whether the element is a member of the set or not. It can produce false-positive matches (say that the element is a member of the set, while it is not present there), but cannot produce false negatives (if a negative match is returned, the element is guaranteed not to be a member of the set).

Bộ lọc Bloom, do Burton Howard Bloom hình thành năm 1970 [BLOOM70], là một cấu trúc dữ liệu xác suất tiết kiệm không gian có thể được dùng để kiểm tra xem phần tử có là thành viên của tập hợp hay không. Nó có thể tạo ra các kết quả khớp dương tính giả (nói rằng phần tử là thành viên của tập hợp, trong khi nó không hiện diện ở đó), nhưng không thể tạo ra âm tính giả (nếu một kết quả khớp âm tính được trả về, phần tử được bảo đảm không phải là thành viên của tập hợp).

In other words, a Bloom filter can be used to tell if the key might be in the table or is definitely not in the table. Files for which a Bloom filter returns a negative match are skipped during the query. The rest of the files are accessed to find out if the data record is actually present. Using Bloom filters associated with disk-resident tables helps to significantly reduce the number of tables accessed during a read.

Nói cách khác, một bộ lọc Bloom có thể được dùng để cho biết liệu khóa có thể có trong bảng hoặc chắc chắn không có trong bảng. Các tệp mà bộ lọc Bloom trả về kết quả khớp âm tính được bỏ qua trong khi truy vấn. Các tệp còn lại được truy cập để tìm ra liệu bản ghi dữ liệu có thực sự hiện diện. Việc dùng các bộ lọc Bloom liên kết với các bảng nằm trên đĩa giúp giảm đáng kể số lượng bảng được truy cập trong khi đọc.

A Bloom filter uses a large bit array and multiple hash functions. Hash functions are applied to keys of the records in the table to find indices in the bit array, bits for which are set to 1. Bits set to 1 in all positions determined by the hash functions indicate a presence of the key in the set. During lookup, when checking for element presence in a Bloom filter, hash functions are calculated for the key again and, if bits determined by all hash functions are 1, we return the positive result stating that item is a member of the set with a certain probability. If at least one of the bits is 0, we can precisely say that element is not present in the set.

Một bộ lọc Bloom dùng một mảng bit lớn và nhiều hàm băm. Các hàm băm được áp dụng lên các khóa của các bản ghi trong bảng để tìm các chỉ số trong mảng bit, các bit ở những vị trí đó được đặt thành 1. Các bit được đặt thành 1 ở tất cả các vị trí do các hàm băm xác định cho thấy sự hiện diện của khóa trong tập hợp. Trong khi tra cứu, khi kiểm tra sự hiện diện của phần tử trong một bộ lọc Bloom, các hàm băm được tính lại cho khóa và, nếu các

bit do tất cả các hàm băm xác định đều là 1, ta trả về kết quả dương tính khẳng định rằng mục là thành viên của tập hợp với một xác suất nhất định. Nếu ít nhất một trong các bit là 0, ta có thể nói chính xác rằng phần tử không hiện diện trong tập hợp.

Hash functions applied to different keys can return the same bit position and result in a hash collision, and 1 bits only imply that some hash function has yielded this bit position for some key.

Các hàm băm áp dụng lên các khóa khác nhau có thể trả về cùng một vị trí bit và dẫn đến một va chạm băm (hash collision), và các bit 1 chỉ ngụ ý rằng một hàm băm nào đó đã cho ra vị trí bit này cho một khóa nào đó.

Probability of false positives is managed by configuring the size of the bit set and the number of hash functions: in a larger bit set, there's a smaller chance of collision; similarly, having more hash functions, we can check more bits and have a more precise outcome.

Xác suất dương tính giả được quản lý bằng cách cấu hình kích thước của tập bit và số lượng hàm băm: trong một tập bit lớn hơn, có ít khả năng va chạm hơn; tương tự, có nhiều hàm băm hơn, ta có thể kiểm tra nhiều bit hơn và có một kết quả chính xác hơn.

The larger bit set occupies more memory, and computing results of more hash functions may have a negative performance impact, so we have to find a reasonable middle ground between acceptable probability and incurred overhead. Probability can be calculated from the expected set size. Since tables in LSM Trees are immutable, set size (number of keys in the table) is known up front.

Tập bit lớn hơn chiếm nhiều bộ nhớ hơn, và việc tính kết quả của nhiều hàm băm hơn có thể có tác động tiêu cực đến hiệu năng, nên ta phải tìm một điểm trung gian hợp lý giữa xác suất chấp nhận được và chi phí phát sinh. Xác suất có thể được tính từ kích thước tập hợp kỳ vọng. Vì các bảng trong LSM Tree là bất biến, kích thước tập hợp (số khóa trong bảng) được biết trước.

Let's take a look at a simple example, shown in Figure 7-7. We have a 16-way bit array and 3 hash functions, which yield values 3, 5, and 10 for key1. We now set bits at these positions. The next key is added and hash functions yield values of 5, 8, and 14 for key2, for which we set bits, too.

Hãy xem một ví dụ đơn giản, được thể hiện trong Hình 7-7. Ta có một mảng bit 16-đường và 3 hàm băm, cho ra các giá trị 3, 5 và 10 cho key1. Bây giờ ta đặt các bit tại các vị trí này. Khóa tiếp theo được thêm và các hàm băm cho ra các giá trị 5, 8 và 14 cho key2, ta cũng đặt các bit cho nó.

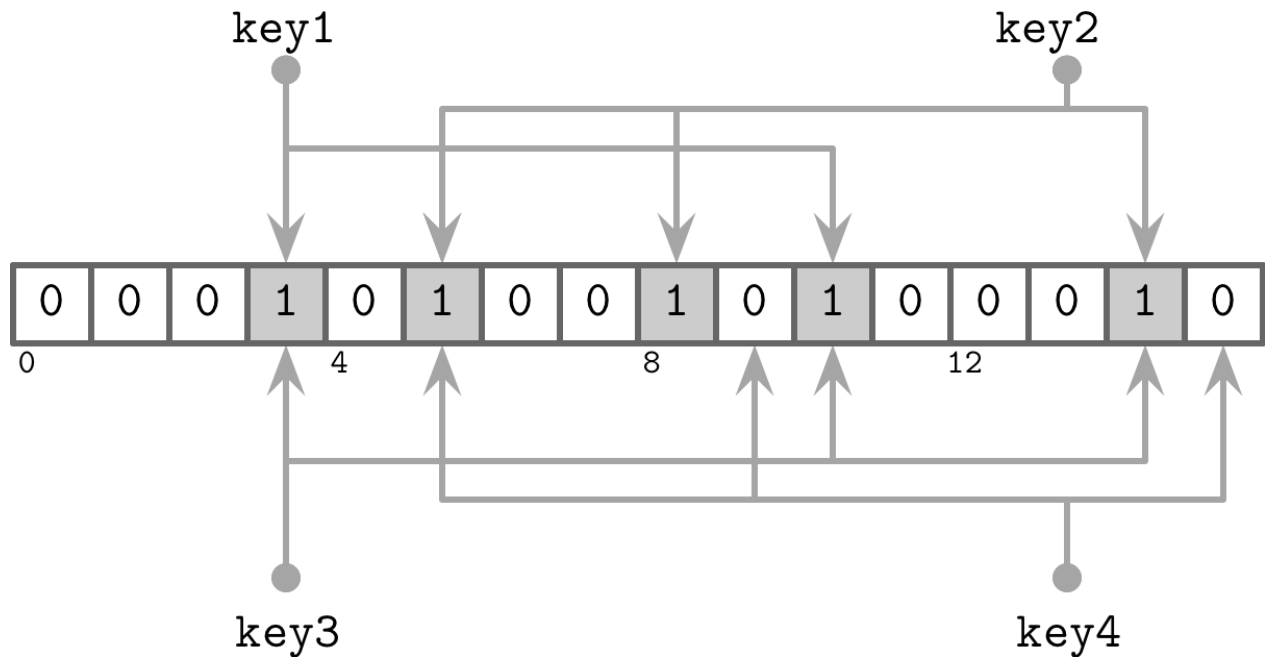


Figure 7-7. Bloom filter

Hình 7-7. Bộ lọc Bloom

Now, we're trying to check whether or not key3 is present in the set, and hash functions yield 3, 10, and 14. Since all three bits were set when adding key1 and key2, we have a situation in which the Bloom filter returns a false positive: key3 was never appended there, yet all of the calculated bits are set. However, since the Bloom filter only claims that element might be in the table, this result is acceptable.

Bây giờ, ta đang cố kiểm tra liệu key3 có hiện diện trong tập hợp hay không, và các hàm băm cho ra 3, 10 và 14. Vì cả ba bit đã được đặt khi thêm key1 và key2, ta có một tình huống trong đó bộ lọc Bloom trả về một dương tính giả: key3 chưa bao giờ được nối vào đó, vậy mà tất cả các bit đã tính đều được đặt. Tuy nhiên, vì bộ lọc Bloom chỉ khẳng định rằng phần tử có thể có trong bảng, kết quả này là chấp nhận được.

If we try to perform a lookup for key4 and receive values of 5, 9, and 15, we find that only bit 5 is set, and the other two bits are unset. If even one of the bits is unset, we know for sure that the element was never appended to the filter.

Nếu ta thử thực hiện một tra cứu cho key4 và nhận các giá trị 5, 9 và 15, ta thấy rằng chỉ bit 5 được đặt, và hai bit còn lại không được đặt. Nếu thậm chí một trong các bit không được đặt, ta biết chắc chắn rằng phần tử chưa bao giờ được nối vào bộ lọc.

Skiplist — Skiplist

There are many different data structures for keeping sorted data in memory, and one that has been getting more popular recently because of its simplicity is called a skiplist [PUGH90b]. Implementation-wise, a skiplist is not much more complex than a singly-linked list, and its probabilistic complexity guarantees are close to those of search trees.

Có nhiều cấu trúc dữ liệu khác nhau để giữ dữ liệu đã sắp xếp trong bộ nhớ, và một cấu trúc gần đây trở nên phổ biến hơn nhờ tính đơn giản của nó được gọi là skiplist [PUGH90b].

Về mặt hiện thực, một skiplist không phức tạp hơn nhiều so với một danh sách liên kết đơn, và các bảo đảm độ phức tạp xác suất của nó gần với các bảo đảm của cây tìm kiếm.

Skiplists do not require rotation or relocation for inserts and updates, and use probabilistic balancing instead. Skiplists are generally less cache-friendly than in-memory B-Trees, since skiplist nodes are small and randomly allocated in memory. Some implementations improve the situation by using unrolled linked lists .

Skiplist không đòi hỏi xoay hay di dời cho các lần chèn và cập nhật, mà dùng cân bằng xác suất thay thế. Skiplist nói chung kém thân thiện với cache hơn B-Tree trong bộ nhớ, vì các nút skiplist nhỏ và được cấp phát ngẫu nhiên trong bộ nhớ. Một số hiện thực cải thiện tình huống bằng cách dùng các danh sách liên kết được mở cuộn (unrolled linked lists).

A skiplist consists of a series of nodes of a different height , building linked hierarchies allowing to skip ranges of items. Each node holds a key, and, unlike the nodes in a linked list, some nodes have more than just one successor. A node of height h is linked from one or more predecessor nodes of a height up to h . Nodes on the lowest level can be linked from nodes of any height.

Một skiplist gồm một loạt các nút có chiều cao khác nhau, xây dựng các phân cấp liên kết cho phép bỏ qua các dải mục. Mỗi nút giữ một khóa, và, khác với các nút trong một danh sách liên kết, một số nút có nhiều hơn chỉ một nút kế nhiệm. Một nút chiều cao h được liên kết từ một hoặc nhiều nút tiền nhiệm có chiều cao tới h . Các nút ở mức thấp nhất có thể được liên kết từ các nút có bất kỳ chiều cao nào.

Node height is determined by a random function and is computed during insert. Nodes that have the same height form a level . The number of levels is capped to avoid infinite growth, and a maximum height is chosen based on how many items can be held by the structure. There are exponentially fewer nodes on each next level.

Chiều cao nút được xác định bởi một hàm ngẫu nhiên và được tính trong khi chèn. Các nút có cùng chiều cao tạo thành một mức. Số mức bị giới hạn để tránh tăng trưởng vô hạn, và một chiều cao tối đa được chọn dựa trên số mục mà cấu trúc có thể giữ. Có ít nút hơn theo cấp số nhân trên mỗi mức tiếp theo.

Lookups work by following the node pointers on the highest level. As soon as the search encounters the node that holds a key that is greater than the searched one, its predecessor's **link** to the node on the next level is followed. In other words, if the searched key is greater than the current node key, the search continues forward. If the searched key is smaller than the current node key, the search continues from the predecessor node on the next level. This process is repeated recursively until the searched key or its predecessor is located.

Các tra cứu hoạt động bằng cách đi theo các con trỏ nút trên mức cao nhất. Ngay khi tìm kiếm gặp nút giữ một khóa lớn hơn khóa được tìm, liên kết của nút tiền nhiệm của nó tới nút ở mức tiếp theo được đi theo. Nói cách khác, nếu khóa được tìm lớn hơn khóa của nút hiện tại, tìm kiếm tiếp tục tiến về phía trước. Nếu khóa được tìm nhỏ hơn khóa của nút hiện tại, tìm kiếm tiếp tục từ nút tiền nhiệm ở mức tiếp theo. Tiến trình này được lặp lại đệ quy cho tới khi khóa được tìm hoặc tiền nhiệm của nó được định vị.

For example, searching for key 7 in the skiplist shown in Figure 7-8 can be done as follows:

Ví dụ, tìm kiếm khóa 7 trong skiplist được thể hiện trong Hình 7-8 có thể được thực hiện như sau:

1. Follow the pointer on the highest level, to the node that holds key 10 .
2. Since the searched key 7 is smaller than 10 , the next-level pointer from the head node is followed, locating a node holding key 5 .

3. The highest-level pointer on this node is followed, locating the node holding key 10 again.
4. The searched key 7 is smaller than 10, and the next-level pointer from the node holding key 5 is followed, locating a node holding the searched key 7.

1. Đi theo con trỏ trên mức cao nhất, tới nút giữ khóa 10.
2. Vì khóa được tìm 7 nhỏ hơn 10, con trỏ mức tiếp theo từ nút đầu được đi theo, định vị một nút giữ khóa 5.
3. Con trỏ mức cao nhất trên nút này được đi theo, định vị lại nút giữ khóa 10.
4. Khóa được tìm 7 nhỏ hơn 10, và con trỏ mức tiếp theo từ nút giữ khóa 5 được đi theo, định vị một nút giữ khóa 7 được tìm.

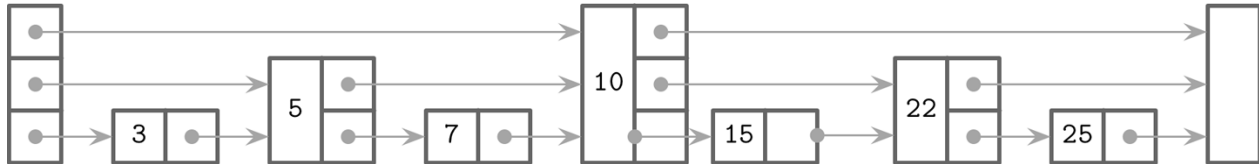


Figure 7-8. Skip list During insert, an insertion point (node holding a key or its predecessor) is found using the aforementioned algorithm, and a new node is created. To build a tree-like hierarchy and keep balance, the height of the node is determined using a random number, generated based on a probability distribution. Pointers in predecessor nodes holding keys smaller than the key in a newly created node are linked to point to that node. Their higher-level pointers remain intact. Pointers in the newly created node are linked to corresponding successors on each level.

Hình 7-8. Skip list. Trong khi chèn, một điểm chèn (nút giữ một khóa hoặc tiền nhiệm của nó) được tìm bằng thuật toán đã nêu, và một nút mới được tạo. Để xây dựng một phân cấp giống cây và giữ cân bằng, chiều cao của nút được xác định bằng một số ngẫu nhiên, được sinh dựa trên một phân phối xác suất. Các con trỏ trong các nút tiền nhiệm giữ các khóa nhỏ hơn khóa trong một nút mới được tạo được liên kết để trỏ tới nút đó. Các con trỏ mức cao hơn của chúng giữ nguyên. Các con trỏ trong nút mới được tạo được liên kết tới các nút kế nhiệm tương ứng trên mỗi mức.

During delete, forward pointers of the removed node are placed to predecessor nodes on corresponding levels.

Trong khi xóa, các con trỏ tiến của nút bị gỡ được đặt vào các nút tiền nhiệm trên các mức tương ứng.

We can create a concurrent version of a skip list by implementing a **linearizability** scheme that uses an additional `fully_linked` flag that determines whether or not the node pointers are fully updated. This flag can be set using compare-and-swap [HER- LIHY10]. This is required because the node pointers have to be updated on multiple levels to fully restore the skip list structure.

Ta có thể tạo một phiên bản đồng thời của skip list bằng cách hiện thực một lược đồ tuyến tính hóa dùng một cờ `fully_linked` bổ sung xác định liệu các con trỏ nút đã được cập nhật hoàn toàn hay chưa. Cờ này có thể được đặt bằng compare-and-swap [HER-LIHY10]. Điều này là cần thiết vì các con trỏ nút phải được cập nhật trên nhiều mức để khôi phục hoàn toàn cấu trúc skip list.

In languages with an unmanaged memory model, reference counting or hazard pointers can be used to ensure that currently referenced nodes are not freed while they are accessed concurrently [RUSSEL12]. This algorithm is **deadlock**-free, since nodes are always accessed from higher levels.

Trong các ngôn ngữ với mô hình bộ nhớ không được quản lý, đếm tham chiếu (reference counting) hoặc con trỏ hazard (hazard pointers) có thể được dùng để bảo đảm rằng các nút hiện đang được tham chiếu không bị giải phóng trong khi chúng đang được truy cập

đồng thời [RUSSEL12]. Thuật toán này không có bế tắc (deadlock), vì các nút luôn được truy cập từ các mức cao hơn.

Apache Cassandra uses skiplists for the secondary index memtable implementation . WiredTiger uses skiplists for some in-memory operations.

Apache Cassandra dùng skiplist cho hiện thực memtable của chỉ mục phụ. WiredTiger dùng skiplist cho một số thao tác trong bộ nhớ.

Disk Access — Truy cập đĩa

Since most of the table contents are disk-resident, and storage devices generally allow accessing data blockwise, many LSM Tree implementations rely on the **page cache** for disk accesses and intermediate caching. Many techniques described in “Buffer Management” on page 81 , such as page eviction and pinning, still apply to log-structured storage.

Vì hầu hết nội dung bảng nằm trên đĩa, và các thiết bị lưu trữ nói chung cho phép truy cập dữ liệu theo khối, nhiều hiện thực LSM Tree dựa vào page cache cho các lần truy cập đĩa và lưu đệm trung gian. Nhiều kỹ thuật được mô tả trong “Buffer Management” ở trang 81, chẳng hạn loại bỏ trang (page eviction) và ghim trang (pinning), vẫn áp dụng được cho lưu trữ dạng nhật ký.

The most notable difference is that in-memory contents are immutable and therefore require no additional locks or latches for concurrent access. Reference counting is applied to make sure that currently accessed pages are not evicted from memory, and in-flight requests complete before underlying files are removed during compaction.

Khác biệt đáng chú ý nhất là nội dung trong bộ nhớ là bất biến và do đó không đòi hỏi các khóa hoặc chốt bổ sung cho truy cập đồng thời. Đếm tham chiếu được áp dụng để bảo đảm rằng các trang hiện đang được truy cập không bị loại bỏ khỏi bộ nhớ, và các yêu cầu đang trong tiến trình hoàn thành trước khi các tệp nền tảng bị gỡ bỏ trong khi nén-gộp.

Another difference is that data records in LSM Trees are not necessarily page aligned, and pointers can be implemented using absolute offsets rather than page IDs for addressing. In Figure 7-9 , you can see records with contents that are not aligned with disk blocks. Some records cross the page boundaries and require loading several pages in memory.

Một khác biệt khác là các bản ghi dữ liệu trong LSM Tree không nhất thiết được căn theo trang, và các con trỏ có thể được hiện thực bằng các độ dời tuyệt đối thay vì ID trang để địa chỉ hóa. Trong Hình 7-9, bạn có thể thấy các bản ghi với nội dung không được căn với các khối đĩa. Một số bản ghi vượt qua ranh giới trang và đòi hỏi nạp vài trang vào bộ nhớ.

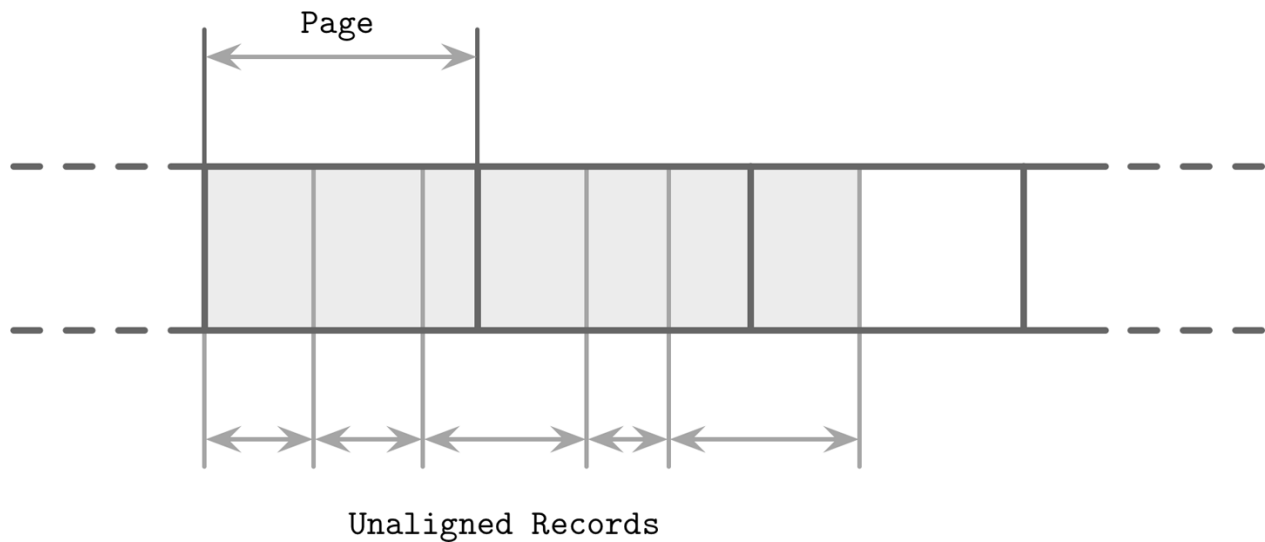


Figure 7-9. Unaligned data records

Hình 7-9. Các bản ghi dữ liệu không được căn

Compression — Nén dữ liệu (Compression)

We’ve discussed compression already in context of B-Trees (see “Compression” on page 73). Similar ideas are also applicable to LSM Trees. The main difference here is that LSM Tree tables are immutable, and are generally written in a single pass. When compressing data page-wise, compressed pages are not page aligned, as their sizes are smaller than that of uncompressed ones.

Chúng ta đã thảo luận nén dữ liệu rồi trong ngữ cảnh B-Tree (xem “Compression” ở trang 73). Các ý tưởng tương tự cũng áp dụng được cho LSM Tree. Khác biệt chính ở đây là các bảng LSM Tree là bất biến, và nói chung được ghi trong một lượt duy nhất. Khi nén dữ liệu theo trang, các trang đã nén không được căn theo trang, vì kích thước của chúng nhỏ hơn kích thước của các trang chưa nén.

To be able to address compressed pages, we need to keep **track** of the address boundaries when writing their contents. We could fill compressed pages with zeros, aligning them to the page size, but then we’d lose the benefits of compression.

Để có thể địa chỉ hóa các trang đã nén, ta cần theo dõi các ranh giới địa chỉ khi ghi nội dung của chúng. Ta có thể điền các trang đã nén bằng các số 0, căn chúng theo kích thước trang, nhưng khi đó ta sẽ mất các lợi ích của việc nén.

To make compressed pages addressable, we need an indirection layer which stores offsets and sizes of compressed pages. Figure 7-10 shows the mapping between compressed and uncompressed blocks. Compressed pages are always smaller than the originals, since otherwise there’s no point in compressing them.

Để làm cho các trang đã nén có thể địa chỉ hóa, ta cần một lớp gián tiếp lưu các độ dời và kích thước của các trang đã nén. Hình 7-10 cho thấy ánh xạ giữa các khối đã nén và chưa nén. Các trang đã nén luôn nhỏ hơn các trang gốc, vì nếu không thì việc nén chúng không có ý nghĩa.

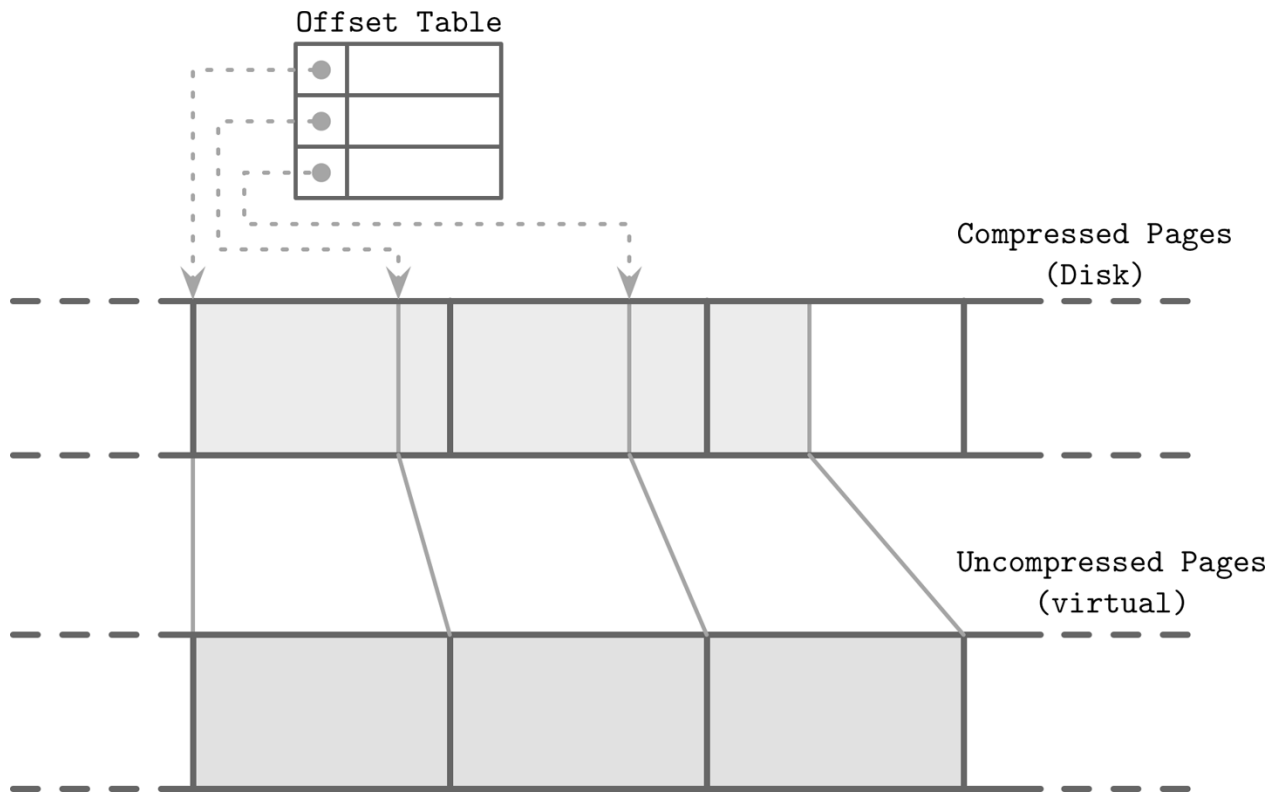


Figure 7-10. Reading compressed blocks. Dotted lines represent pointers from the mapping table to the offsets of compressed pages on disk. Uncompressed pages generally reside in the page cache.

Hình 7-10. Đọc các khối đã nén. Các đường chấm chấm biểu diễn các con trỏ từ bảng ánh xạ tới các độ dời của các trang đã nén trên đĩa. Các trang chưa nén nói chung nằm trong page cache.

During compaction and flush, compressed pages are appended sequentially, and compression information (the original uncompressed page offset and the actual compressed page offset) is stored in a separate file segment. During the read, the compressed page offset and its size are looked up, and the page can be uncompressed and materialized in memory.

Trong khi nén-gộp và xả, các trang đã nén được nối tuần tự, và thông tin nén (độ dời trang chưa nén gốc và độ dời trang đã nén thực tế) được lưu trong một phân đoạn tệp riêng. Trong khi đọc, độ dời trang đã nén và kích thước của nó được tra cứu, và trang có thể được giải nén và hiện thực hóa trong bộ nhớ.

Unordered LSM Storage — Lưu trữ LSM không có thứ tự (Unordered LSM Storage)

Most of the storage structures discussed so far store data in order . Mutable and immutable B-Tree pages, sorted runs in FD-Trees, and SSTables in LSM Trees store data records in key order. The order in these structures is preserved differently: B- Tree pages are updated in place, **FD-Tree** runs are created by merging contents of two runs, and SSTables are created by buffering and sorting data records in memory.

Hầu hết các cấu trúc lưu trữ được thảo luận cho đến giờ lưu dữ liệu theo thứ tự. Các trang B-Tree khả biến và bất biến, các lượt chạy đã sắp xếp trong FD-Tree, và các SSTable trong

LSM Tree lưu các bản ghi dữ liệu theo thứ tự khóa. Thứ tự trong các cấu trúc này được bảo toàn khác nhau: các trang B-Tree được cập nhật tại chỗ, các lượt chạy FD-Tree được tạo bằng cách gộp nội dung của hai lượt chạy, và các SSTable được tạo bằng cách đệm và sắp xếp các bản ghi dữ liệu trong bộ nhớ.

In this section, we discuss structures that store records in random order. Unordered stores generally do not require a separate log and allow us to reduce the cost of writes by storing data records in insertion order.

Trong phần này, chúng ta thảo luận các cấu trúc lưu các bản ghi theo thứ tự ngẫu nhiên. Các kho không có thứ tự nói chung không đòi hỏi một nhật ký riêng và cho phép ta giảm chi phí ghi bằng cách lưu các bản ghi dữ liệu theo thứ tự chèn.

Bitcask — Bitcask

Bitcask, one of the storage engines used in Riak, is an unordered log-structured storage engine [SHEEHY10b]. Unlike the log-structured storage implementations discussed so far, it does not use memtables for buffering, and stores data records directly in logfiles.

Bitcask, một trong các bộ máy lưu trữ được dùng trong Riak, là một bộ máy lưu trữ dạng nhật ký không có thứ tự [SHEEHY10b]. Khác với các hiện thực lưu trữ dạng nhật ký được thảo luận cho đến giờ, nó không dùng memtable để đệm, và lưu các bản ghi dữ liệu trực tiếp trong các tệp nhật ký.

To make values searchable, Bitcask uses a data structure called `keydir`, which holds references to the latest data records for the corresponding keys. Old data records may still be present on disk, but are not referenced from `keydir`, and are garbage-collected during compaction. `Keydir` is implemented as an in-memory hashmap and has to be rebuilt from the logfiles during startup.

Để làm cho các giá trị có thể tìm kiếm được, Bitcask dùng một cấu trúc dữ liệu gọi là `keydir`, giữ các tham chiếu tới các bản ghi dữ liệu mới nhất cho các khóa tương ứng. Các bản ghi dữ liệu cũ có thể vẫn hiện diện trên đĩa, nhưng không được tham chiếu từ `keydir`, và được thu gom rác trong khi nén-gộp. `Keydir` được hiện thực như một hashmap trong bộ nhớ và phải được xây dựng lại từ các tệp nhật ký trong khi khởi động.

During a write, a key and a data record are appended to the logfile sequentially, and the pointer to the newly written data record location is placed in `keydir`.

Trong khi ghi, một khóa và một bản ghi dữ liệu được nối vào tệp nhật ký một cách tuần tự, và con trỏ tới vị trí bản ghi dữ liệu mới được ghi được đặt trong `keydir`.

Reads check the `keydir` to locate the searched key and follow the associated pointer to the logfile, locating the data record. Since at any given moment there can be only one value associated with the key in the `keydir`, point queries do not have to merge data from multiple sources.

Các lần đọc kiểm tra `keydir` để định vị khóa được tìm và đi theo con trỏ liên kết tới tệp nhật ký, định vị bản ghi dữ liệu. Vì tại bất kỳ thời điểm nào chỉ có thể có một giá trị liên kết với khóa trong `keydir`, các truy vấn điểm không phải gộp dữ liệu từ nhiều nguồn.

Figure 7-11 shows mapping between the keys and records in data files in Bitcask. Logfiles hold data records, and `keydir` points to the latest live data record associated with each key. Shadowed records in data files (ones that were superseded by later writes or deletes) are shown in gray.

Hình 7-11 cho thấy ánh xạ giữa các khóa và các bản ghi trong các tệp dữ liệu trong Bitcask. Các tệp nhật ký giữ các bản ghi dữ liệu, và `keydir` trỏ tới bản ghi dữ liệu còn sống mới nhất

liên kết với mỗi khóa. Các bản ghi bị che khuất trong các tệp dữ liệu (những bản ghi đã bị thay thế bởi các lần ghi hoặc xóa sau này) được hiển thị bằng màu xám.

Keydir

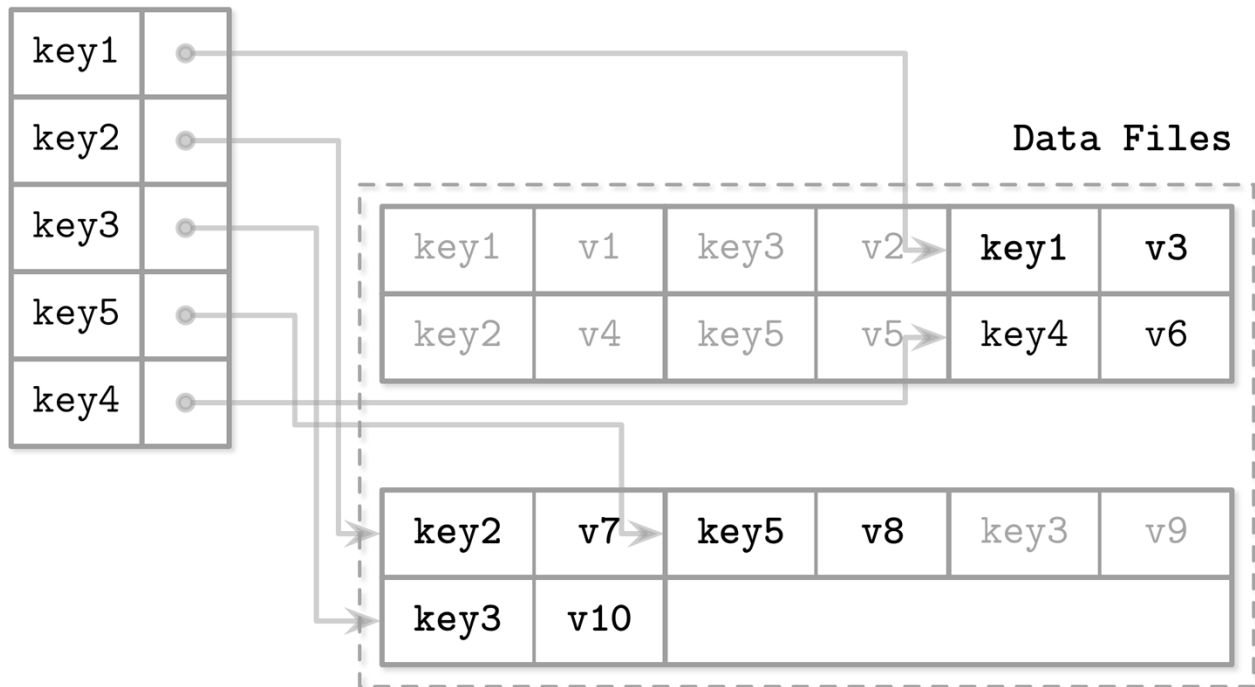


Figure 7-11. Mapping between keydir and data files in Bitcask. Solid lines represent pointers from the key to the latest value associated with it. Shadowed key/value pairs are shown in light gray.

Hình 7-11. Ảnh xạ giữa keydir và các tệp dữ liệu trong Bitcask. Các đường nét liên biểu diễn các con trỏ từ khóa tới giá trị mới nhất liên kết với nó. Các cặp khóa/giá trị bị che khuất được hiển thị bằng màu xám nhạt.

During compaction, contents of all logfiles are read sequentially, merged, and written to a new location, preserving only live data records and discarding the shadowed ones. Keydir is updated with new pointers to relocated data records.

Trong khi nén-gộp, nội dung của tất cả các tệp nhật ký được đọc tuần tự, gộp, và ghi vào một vị trí mới, chỉ bảo toàn các bản ghi dữ liệu còn sống và loại bỏ các bản ghi bị che khuất. Keydir được cập nhật với các con trỏ mới tới các bản ghi dữ liệu đã được di dời.

Data records are stored directly in logfiles, so a separate write-ahead log doesn't have to be maintained, which reduces both space overhead and write amplification. A downside of this approach is that it offers only point queries and doesn't allow range scans, since items are unordered both in keydir and in data files.

Các bản ghi dữ liệu được lưu trực tiếp trong các tệp nhật ký, nên một nhật ký ghi-trước riêng không phải được duy trì, điều này giảm cả chi phí không gian lẫn khuếch đại ghi. Một nhược điểm của cách tiếp cận này là nó chỉ cung cấp các truy vấn điểm và không cho phép quét dải, vì các mục không có thứ tự cả trong keydir lẫn trong các tệp dữ liệu.

Advantages of this approach are simplicity and great point query performance. Even though multiple versions of data records exist, only the latest one is addressed by keydir. However, having to keep all keys in memory and rebuilding keydir on startup are limitations that might be a deal breaker for

some use cases. While this approach is great for point queries, it does not offer any support for range queries.

Các ưu điểm của cách tiếp cận này là tính đơn giản và hiệu năng truy vấn điểm tuyệt vời. Mặc dù nhiều phiên bản của các bản ghi dữ liệu tồn tại, chỉ phiên bản mới nhất được keydir địa chỉ hóa. Tuy nhiên, việc phải giữ tất cả các khóa trong bộ nhớ và xây dựng lại keydir khi khởi động là các giới hạn có thể là yếu tố loại trừ cho một số trường hợp sử dụng. Trong khi cách tiếp cận này tuyệt vời cho các truy vấn điểm, nó không cung cấp bất kỳ hỗ trợ nào cho các truy vấn dải.

WiscKey — WiscKey

Range queries are important for many applications, and it would be great to have a storage structure that could have the write and space advantages of unordered storage, while still allowing us to perform range scans.

Các truy vấn dải là quan trọng đối với nhiều ứng dụng, và sẽ thật tuyệt nếu có một cấu trúc lưu trữ có thể có các ưu điểm về ghi và không gian của lưu trữ không có thứ tự, trong khi vẫn cho phép ta thực hiện quét dải.

WiscKey [LU16] decouples sorting from garbage collection by keeping the keys sorted in LSM Trees, and keeping data records in unordered append-only files called vLogs (value logs). This approach can solve two problems mentioned while discussing Bitcask: a need to keep all keys in memory and to rebuild a hashtable on startup.

WiscKey [LU16] tách rời việc sắp xếp khỏi thu gom rác bằng cách giữ các khóa được sắp xếp trong LSM Tree, và giữ các bản ghi dữ liệu trong các tệp chỉ-nối-thêm không có thứ tự gọi là vLog (value logs). Cách tiếp cận này có thể giải quyết hai vấn đề được nêu khi thảo luận Bitcask: nhu cầu giữ tất cả các khóa trong bộ nhớ và xây dựng lại một bảng băm khi khởi động.

Figure 7-12 shows key components of WiscKey, and mapping between keys and log files. vLog files hold unordered data records. Keys are stored in sorted LSM Trees, pointing to the latest data records in the logfiles.

Hình 7-12 cho thấy các thành phần chính của WiscKey, và ánh xạ giữa các khóa và các tệp nhật ký. Các tệp vLog giữ các bản ghi dữ liệu không có thứ tự. Các khóa được lưu trong các LSM Tree đã sắp xếp, trỏ tới các bản ghi dữ liệu mới nhất trong các tệp nhật ký.

Since keys are typically much smaller than the data records associated with them, compacting them is significantly more efficient. This approach can be particularly useful for use cases with a low rate of updates and deletes, where garbage collection won't free up as much disk space.

Vì các khóa thường nhỏ hơn nhiều so với các bản ghi dữ liệu liên kết với chúng, việc nén-gộp chúng hiệu quả hơn đáng kể. Cách tiếp cận này có thể đặc biệt hữu ích cho các trường hợp sử dụng với tốc độ cập nhật và xóa thấp, nơi thu gom rác sẽ không giải phóng được nhiều không gian đĩa.

The main challenge here is that because vLog data is unsorted, range scans require **random I/O**. WiscKey uses internal **SSD** parallelism to prefetch blocks in parallel during range scans and reduce random I/O costs. In terms of **block** transfers, the costs are still high: to fetch a single data record during the range scan, the entire page where it is located has to be read.

Thách thức chính ở đây là vì dữ liệu vLog không được sắp xếp, các lần quét dải đòi hỏi I/O ngẫu nhiên. WiscKey dùng tính song song nội bộ của SSD để nạp trước các khối song song

trong khi quét dải và giảm chi phí I/O ngẫu nhiên. Về mặt truyền khối, các chi phí vẫn cao: để lấy một bản ghi dữ liệu đơn lẻ trong khi quét dải, toàn bộ trang nơi nó nằm phải được đọc.

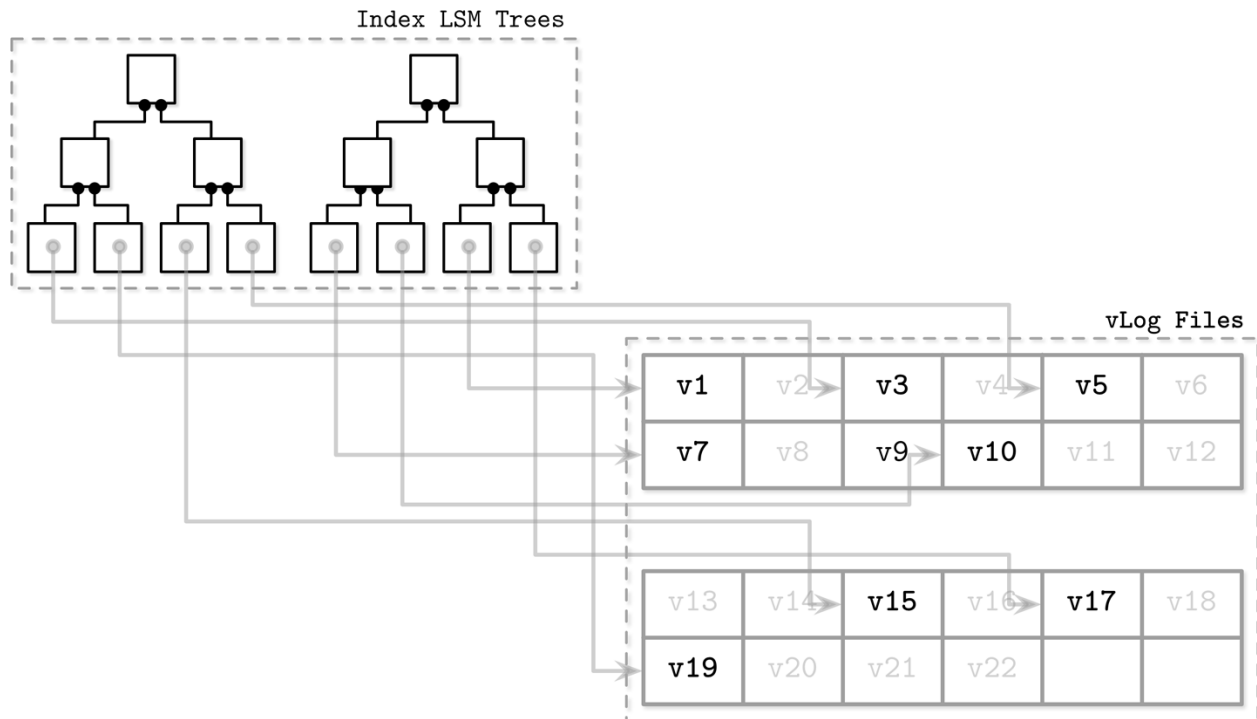


Figure 7-12. Key components of WiscKey: index LSM Trees and vLog files, and relationships between them. Shaded records in data files (ones that were superseded by later writes or deletes) are shown in gray. Solid lines represent pointers from the key in the LSM tree to the latest value in the log file.

Hình 7-12. Các thành phần chính của WiscKey: các LSM Tree chỉ mục và các tệp vLog, và các mối quan hệ giữa chúng. Các bản ghi bị che khuất trong các tệp dữ liệu (những bản ghi đã bị thay thế bởi các lần ghi hoặc xóa sau này) được hiển thị bằng màu xám. Các đường nét liền biểu diễn các con trỏ từ khóa trong LSM Tree tới giá trị mới nhất trong tệp nhật ký.

During compaction, vLog file contents are read sequentially, merged, and written to a new location. Pointers (values in a key LSM Tree) are updated to point to these new locations. To avoid scanning entire vLog contents, WiscKey uses head and tail pointers, holding information about vLog segments that hold live keys.

Trong khi nén-gộp, nội dung tệp vLog được đọc tuần tự, gộp, và ghi vào một vị trí mới. Các con trỏ (các giá trị trong một LSM Tree khóa) được cập nhật để trỏ tới các vị trí mới này. Để tránh quét toàn bộ nội dung vLog, WiscKey dùng các con trỏ đầu và đuôi, giữ thông tin về các phân đoạn vLog giữ các khóa còn sống.

Since data in vLog is unsorted and contains no liveness information, the key tree has to be scanned to find which values are still live. Performing these checks during garbage collection introduces additional complexity: traditional LSM Trees can resolve file contents during compaction without addressing the key index.

Vì dữ liệu trong vLog không được sắp xếp và không chứa thông tin về tính còn sống, cây khóa phải được quét để tìm ra các giá trị nào vẫn còn sống. Việc thực hiện các kiểm tra

này trong khi thu gom rác đưa vào độ phức tạp bổ sung: các LSM Tree truyền thống có thể phân giải nội dung tệp trong khi nén-gộp mà không cần xử lý chỉ mục khóa.

Concurrency in LSM Trees — Tương tranh trong LSM Tree

The main concurrency challenges in LSM Trees are related to switching table views (collections of memory- and disk-resident tables that change during flush and compaction) and log synchronization. Memtables are also generally accessed concurrently (except core-partitioned stores such as ScyllaDB), but concurrent in-memory data structures are out of the scope of this book.

Các thách thức tương tranh chính trong LSM Tree liên quan đến việc chuyển đổi các khung nhìn bảng (các tập hợp bảng nằm trong bộ nhớ và nằm trên đĩa thay đổi trong khi xả và nén-gộp) và đồng bộ hóa nhật ký. Các memtable cũng nói chung được truy cập đồng thời (trừ các kho được phân hoạch theo lõi như ScyllaDB), nhưng các cấu trúc dữ liệu đồng thời trong bộ nhớ nằm ngoài phạm vi của cuốn sách này.

During flush, the following rules have to be followed:

Trong khi xả, các quy tắc sau phải được tuân theo:

- The new memtable has to become available for reads and writes.
- The old (flushing) memtable has to remain visible for reads.
- The flushing memtable has to be written on disk.
- Discarding a flushed memtable and making a flushed disk-resident table have to be performed as an atomic operation.
- The write-ahead log segment, holding log entries of operations applied to the flushed memtable, has to be discarded.
 - Memtable mới phải trở nên khả dụng để đọc và ghi.
 - Memtable cũ (đang xả) phải vẫn hiển thị để đọc.
 - Memtable đang xả phải được ghi lên đĩa.
 - Việc loại bỏ một memtable đã xả và tạo một bảng nằm trên đĩa đã xả phải được thực hiện như một thao tác nguyên tử.
 - Phân đoạn nhật ký ghi-trước, giữ các mục nhật ký của các thao tác đã áp dụng lên memtable đã xả, phải được loại bỏ.

For example, Apache Cassandra solves these problems by using operation order barriers : all operations that were accepted for write will be waited upon prior to the memtable flush. This way the flush process (serving as a consumer) knows which other processes (acting as producers) depend on it.

Ví dụ, Apache Cassandra giải quyết các vấn đề này bằng cách dùng các rào cản thứ tự thao tác (operation order barriers): tất cả các thao tác đã được chấp nhận để ghi sẽ được chờ đợi trước khi xả memtable. Theo cách này, tiến trình xả (đóng vai trò người tiêu thụ) biết các tiến trình khác nào (đóng vai trò người sản xuất) phụ thuộc vào nó.

More generally, we have the following synchronization points:

Tổng quát hơn, ta có các điểm đồng bộ hóa sau:

Memtable switch After this, all writes go only to the new memtable, making it primary, while the old one is still available for reads.

Chuyển đổi memtable Sau điểm này, mọi lần ghi chỉ đi tới memtable mới, làm cho nó trở thành chính, trong khi memtable cũ vẫn khả dụng để đọc.

Flush finalization Replaces the old memtable with a flushed disk-resident table in the table view.

Hoàn tất xả Thay thế memtable cũ bằng một bảng nằm trên đĩa đã xả trong khung nhìn bảng.

Write-ahead log truncation Discards a log segment holding records associated with a flushed memtable.

Cắt bớt nhật ký ghi-trước Loại bỏ một phân đoạn nhật ký giữ các bản ghi liên kết với một memtable đã xả.

These operations have severe correctness implications. Continuing writes to the old memtable might result in data loss; for example, if the write is made into a memtable section that was already flushed. Similarly, failing to leave the old memtable available for reads until its disk-resident counterpart is ready will result in incomplete results.

Các thao tác này có các hệ quả nghiêm trọng về tính đúng đắn. Việc tiếp tục ghi vào memtable cũ có thể dẫn đến mất dữ liệu; ví dụ, nếu lần ghi được thực hiện vào một phần memtable đã được xả. Tương tự, việc không để memtable cũ khả dụng để đọc cho tới khi đối tác nằm trên đĩa của nó sẵn sàng sẽ dẫn đến các kết quả không hoàn chỉnh.

During compaction, the table view is also changed, but here the process is slightly more straightforward: old disk-resident tables are discarded, and the compacted version is added instead. Old tables have to remain accessible for reads until the new one is fully written and is ready to replace them for reads. Situations in which the same tables participate in multiple compactions running in parallel have to be avoided as well.

Trong khi nén-gộp, khung nhìn bảng cũng bị thay đổi, nhưng ở đây tiến trình hơi đơn giản hơn một chút: các bảng nằm trên đĩa cũ bị loại bỏ, và phiên bản đã nén-gộp được thêm vào thay thế. Các bảng cũ phải vẫn truy cập được để đọc cho tới khi bảng mới được ghi hoàn toàn và sẵn sàng thay thế chúng cho việc đọc. Các tình huống trong đó cùng các bảng tham gia vào nhiều lần nén-gộp chạy song song cũng phải được tránh.

In B-Trees, log truncation has to be coordinated with flushing dirty pages from the page cache to guarantee durability. In LSM Trees, we have a similar requirement: writes are buffered in a memtable, and their contents are not durable until fully flushed, so log truncation has to be coordinated with memtable flushes. As soon as the flush is complete, the log manager is given the information about the latest flushed log segment, and its contents can be safely discarded.

Trong B-Tree, việc cắt bớt nhật ký phải được phối hợp với việc xả các trang bẩn từ page cache để bảo đảm tính bền vững. Trong LSM Tree, ta có một yêu cầu tương tự: các lần ghi được đệm trong một memtable, và nội dung của chúng không bền vững cho tới khi được xả hoàn toàn, nên việc cắt bớt nhật ký phải được phối hợp với các lần xả memtable. Ngay khi việc xả hoàn tất, trình quản lý nhật ký được cho thông tin về phân đoạn nhật ký đã xả mới nhất, và nội dung của nó có thể được loại bỏ an toàn.

Not synchronizing log truncations with flushes will also result in data loss: if a log segment is discarded before the flush is complete, and the node crashes, log contents will not be replayed, and data from this segment won't be restored.

Việc không đồng bộ hóa các lần cắt bớt nhật ký với các lần xả cũng sẽ dẫn đến mất dữ liệu: nếu một phân đoạn nhật ký bị loại bỏ trước khi việc xả hoàn tất, và nút sập, nội dung nhật ký sẽ không được phát lại, và dữ liệu từ phân đoạn này sẽ không được khôi phục.

Log Stacking — Chồng tầng nhật ký (Log Stacking)

Many modern filesystems are log structured: they buffer writes in a memory segment and flush its contents on disk when it becomes full in an append-only manner. SSDs use log-structured storage, too, to deal with small random writes, minimize write overhead, improve wear leveling, and increase device lifetime.

Nhiều hệ thống tệp hiện đại có dạng nhật ký: chúng đệm các lần ghi trong một phân đoạn bộ nhớ và xả nội dung của nó lên đĩa khi nó đầy theo cách chỉ-nối-thêm. SSD cũng dùng lưu trữ dạng nhật ký, để xử lý các lần ghi ngẫu nhiên nhỏ, giảm thiểu chi phí ghi, cải thiện cân bằng hao mòn (wear leveling), và tăng tuổi thọ thiết bị.

Log-structured storage (LSS) systems started gaining popularity around the time SSDs were becoming more affordable. LSM Trees and SSDs are a good match, since sequential workloads and append-only writes help to reduce amplification from in-place updates, which negatively affect performance on SSDs.

Các hệ thống lưu trữ dạng nhật ký (LSS) bắt đầu trở nên phổ biến vào khoảng thời điểm SSD trở nên giá phải chăng hơn. LSM Tree và SSD là một cặp đôi phù hợp, vì các tải công việc tuần tự và các lần ghi chỉ-nối-thêm giúp giảm khuếch đại từ các cập nhật tại chỗ, vốn ảnh hưởng tiêu cực đến hiệu năng trên SSD.

If we stack multiple log-structured systems on top each other, we can run into several problems that we were trying to solve using LSS, including write amplification, fragmentation, and poor performance. At the very least, we need to keep the SSD flash translation layer and the filesystem in mind when developing our applications [YANG14].

Nếu ta chồng nhiều hệ thống dạng nhật ký lên trên nhau, ta có thể gặp phải vài vấn đề mà ta đã cố giải quyết bằng LSS, bao gồm khuếch đại ghi, phân mảnh, và hiệu năng kém. Ít nhất, ta cần lưu ý đến lớp dịch flash của SSD và hệ thống tệp khi phát triển các ứng dụng của mình [YANG14].

Flash Translation Layer — Lớp dịch flash (Flash Translation Layer)

Using a log-structuring mapping layer in SSDs is motivated by two factors: small random writes have to be batched together in a physical page, and the fact that SSDs work by using program/erase cycles. Writes can be done only into previously erased pages. This means that a page cannot be programmed (in other words, written) unless it is empty (in other words, was erased).

Việc dùng một lớp ánh xạ dạng nhật ký trong SSD được thúc đẩy bởi hai yếu tố: các lần ghi ngẫu nhiên nhỏ phải được gộp lại với nhau trong một trang vật lý, và việc SSD hoạt động bằng cách dùng các chu kỳ ghi-chương-trình/xóa (program/erase cycles). Các lần ghi chỉ có thể được thực hiện vào các trang đã được xóa trước đó. Điều này nghĩa là một trang không thể được ghi (nói cách khác, được lập trình) trừ khi nó rỗng (nói cách khác, đã được xóa).

A single page cannot be erased, and only groups of pages in a block (typically holding 64 to 512 pages) can be erased together. Figure 7-13 shows a schematic representation of pages, grouped into blocks. The flash translation layer (FTL) translates logical page addresses to their physical locations and keeps track of page states (live, discarded, or empty). When FTL runs out of free pages, it has to perform garbage collection and erase discarded pages.

Một trang đơn lẻ không thể bị xóa, và chỉ các nhóm trang trong một khối (thường giữ 64 đến 512 trang) có thể bị xóa cùng nhau. Hình 7-13 cho thấy một biểu diễn sơ đồ của các trang, được nhóm thành các khối. Lớp dịch flash (FTL) dịch các địa chỉ trang logic sang các vị trí vật lý của chúng và theo dõi các trạng thái trang (còn sống, bị loại bỏ, hoặc rỗng). Khi FTL hết các trang trống, nó phải thực hiện thu gom rác và xóa các trang bị loại bỏ.

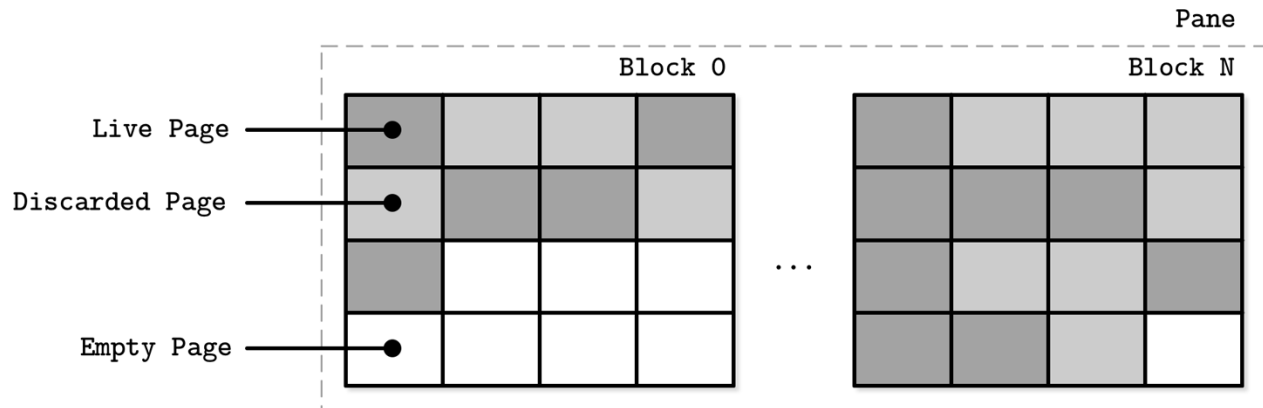


Figure 7-13. SSD pages, grouped into blocks

Hình 7-13. Các trang SSD, được nhóm thành các khối

There are no guarantees that all pages in the block that is about to be erased are discarded. Before the block can be erased, FTL has to relocate its live pages to one of the blocks containing empty pages. Figure 7-14 shows the process of moving live pages from one block to new locations.

Không có bảo đảm rằng tất cả các trang trong khối sắp bị xóa đều đã bị loại bỏ. Trước khi khối có thể bị xóa, FTL phải di dời các trang còn sống của nó sang một trong các khối chứa các trang rỗng. Hình 7-14 cho thấy tiến trình di chuyển các trang còn sống từ một khối sang các vị trí mới.

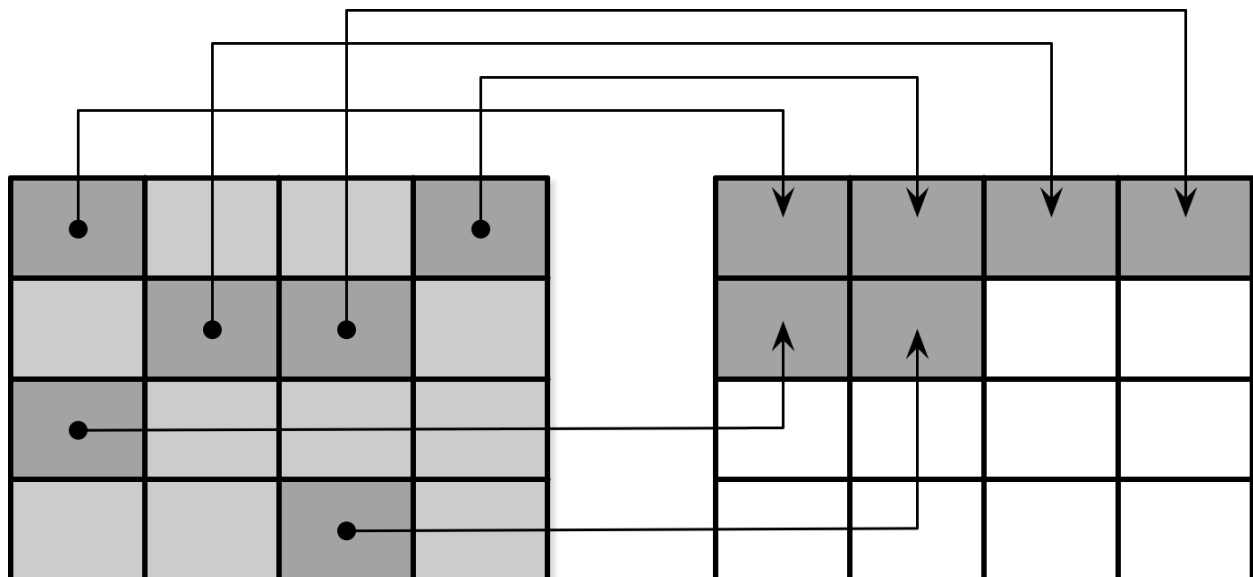


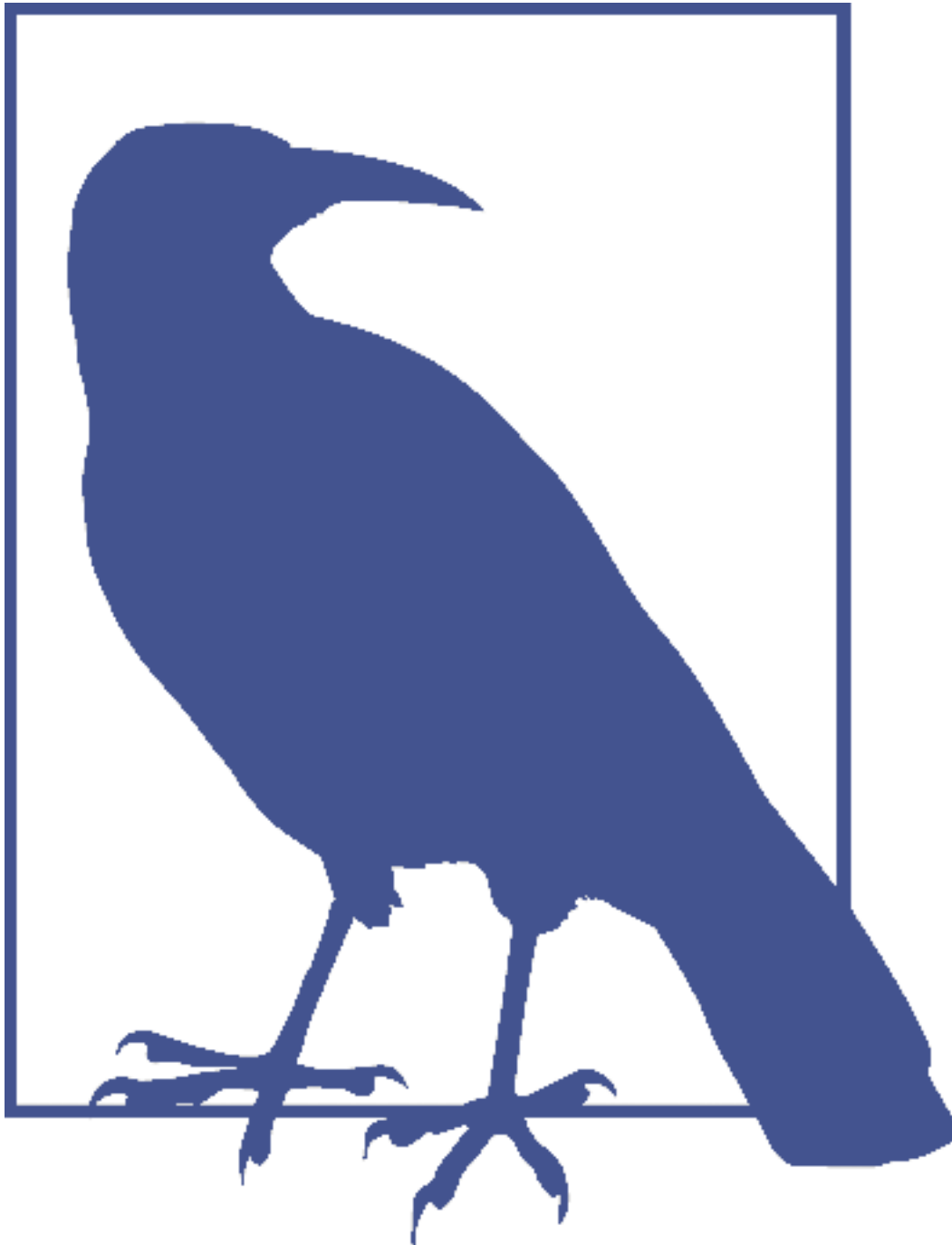
Figure 7-14. Page relocation during garbage collection

Hình 7-14. Di dời trang trong khi thu gom rác

When all live pages are relocated, the block can be safely erased, and its empty pages become

available for writes. Since FTL is aware of page states and state transitions and has all the necessary information, it is also responsible for SSD wear leveling .

Khi tất cả các trang còn sống được di dời, khối có thể được xóa an toàn, và các trang rỗng của nó trở nên khả dụng để ghi. Vì FTL nhận biết các trạng thái trang và các chuyển tiếp trạng thái và có tất cả thông tin cần thiết, nó cũng chịu trách nhiệm về cân bằng hao mòn SSD.



Wear leveling distributes the load evenly across the medium, avoiding hotspots, where blocks fail prematurely because of a high number of program-erase cycles. It is required, since flash memory cells can go through only a limited number of program-erase cycles, and using memory cells evenly

helps to extend the lifetime of the device.

Cân bằng hao mòn phân bố tải đều khắp môi trường lưu trữ, tránh các điểm nóng, nơi các khối hỏng sớm vì số lượng chu kỳ ghi-chương-trình/xóa cao. Nó là cần thiết, vì các ô bộ nhớ flash chỉ có thể trải qua một số lượng giới hạn các chu kỳ ghi-chương-trình/xóa, và việc dùng các ô bộ nhớ đều giúp kéo dài tuổi thọ của thiết bị.

In summary, the motivation for using log-structured storage on SSDs is to amortize I/O costs by batching small random writes together, which generally results in a smaller number of operations and, subsequently, reduces the number of times the garbage collection is triggered.

Tóm lại, động cơ để dùng lưu trữ dạng nhật ký trên SSD là để phân bổ chi phí I/O bằng cách gộp các lần ghi ngẫu nhiên nhỏ lại với nhau, điều này nói chung dẫn đến một số lượng thao tác nhỏ hơn và, do đó, giảm số lần thu gom rác được kích hoạt.

Filesystem Logging — Ghi nhật ký hệ thống tệp (Filesystem Logging)

On top of that, we get filesystems, many of which also use logging techniques for write buffering to reduce write amplification and use the underlying hardware optimally.

Trên hết, ta có các hệ thống tệp, nhiều trong số đó cũng dùng các kỹ thuật ghi nhật ký để đệm ghi nhằm giảm khuếch đại ghi và dùng phần cứng nền tảng một cách tối ưu.

Log stacking manifests in a few different ways. First, each layer has to perform its own bookkeeping, and most often the underlying log does not expose the information necessary to avoid duplicating the efforts.

Chồng tầng nhật ký biểu hiện theo vài cách khác nhau. Thứ nhất, mỗi lớp phải thực hiện công việc ghi sổ của riêng nó, và thường thì nhật ký nền tảng không phơi bày thông tin cần thiết để tránh trùng lặp các nỗ lực.

Figure 7-15 shows a mapping between a higher-level log (for example, the application) and a lower-level log (for example, the filesystem) resulting in redundant logging and different garbage collection patterns [YANG14]. Misaligned segment writes can make the situation even worse, since discarding a higher-level log segment may cause fragmentation and relocation of the neighboring segments' parts.

Hình 7-15 cho thấy một ánh xạ giữa một nhật ký mức cao hơn (ví dụ, ứng dụng) và một nhật ký mức thấp hơn (ví dụ, hệ thống tệp) dẫn đến ghi nhật ký dư thừa và các mẫu thu gom rác khác nhau [YANG14]. Các lần ghi phân đoạn lệch căn có thể làm tình huống thậm chí tồi tệ hơn, vì việc loại bỏ một phân đoạn nhật ký mức cao hơn có thể gây phân mảnh và di dời các phần của các phân đoạn lân cận.

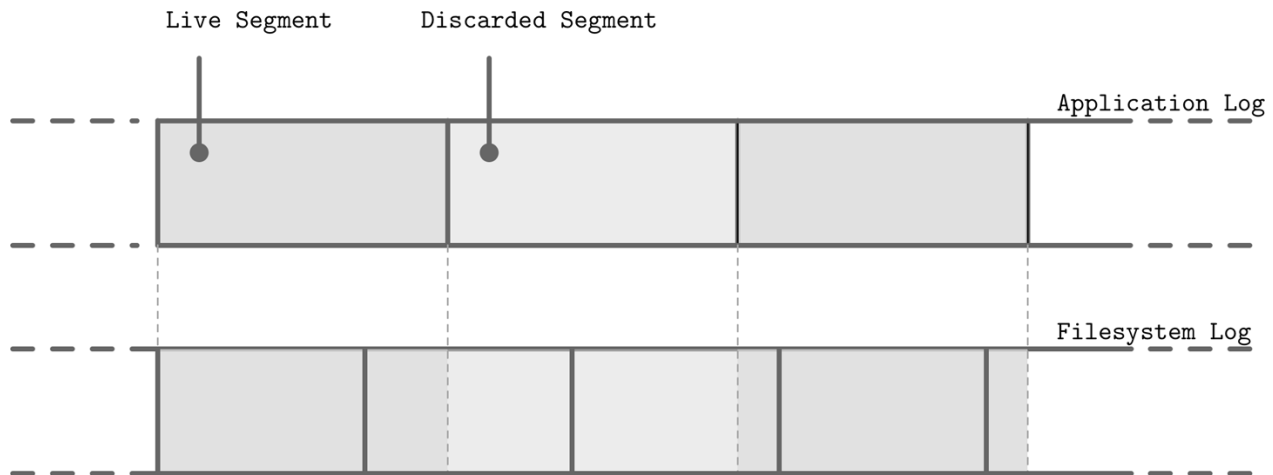


Figure 7-15. Misaligned writes and discarding of a higher-level log segment

Hình 7-15. Các lần ghi lệch căn và việc loại bỏ một phân đoạn nhật ký mức cao hơn

Because layers do not communicate LSS-related scheduling (for example, discarding or relocating segments), lower-level subsystems might perform redundant operations on discarded data or the data that is about to be discarded. Similarly, because there's no single, standard segment size, it may happen that unaligned higher-level segments occupy multiple lower-level segments. All these overheads can be reduced or completely avoided.

Vì các lớp không truyền đạt việc lập lịch liên quan đến LSS (ví dụ, loại bỏ hoặc di dời các phân đoạn), các phân hệ mức thấp hơn có thể thực hiện các thao tác dư thừa trên dữ liệu đã bị loại bỏ hoặc dữ liệu sắp bị loại bỏ. Tương tự, vì không có một kích thước phân đoạn duy nhất, chuẩn, có thể xảy ra việc các phân đoạn mức cao hơn không được căn chỉnh chiếm nhiều phân đoạn mức thấp hơn. Tất cả các chi phí này có thể được giảm hoặc tránh hoàn toàn.

Even though we say that log-structured storage is all about sequential I/O, we have to keep in mind that database systems may have multiple write streams (for example, log writes parallel to data record writes) [YANG14]. When considered on a hardware level, interleaved sequential write streams may not translate into the same sequential pattern: blocks are not necessarily going to be placed in write order. Figure 7-16 shows multiple streams overlapping in time, writing records that have sizes not aligned with the underlying hardware page size.

Mặc dù ta nói rằng lưu trữ dạng nhật ký toàn về I/O tuần tự, ta phải nhớ rằng các hệ thống cơ sở dữ liệu có thể có nhiều luồng ghi (ví dụ, các lần ghi nhật ký song song với các lần ghi bản ghi dữ liệu) [YANG14]. Khi được xét ở mức phần cứng, các luồng ghi tuần tự xen kẽ có thể không chuyển thành cùng một mẫu tuần tự: các khối không nhất thiết được đặt theo thứ tự ghi. Hình 7-16 cho thấy nhiều luồng chồng lấn theo thời gian, ghi các bản ghi có kích thước không được căn với kích thước trang của phần cứng nền tảng.

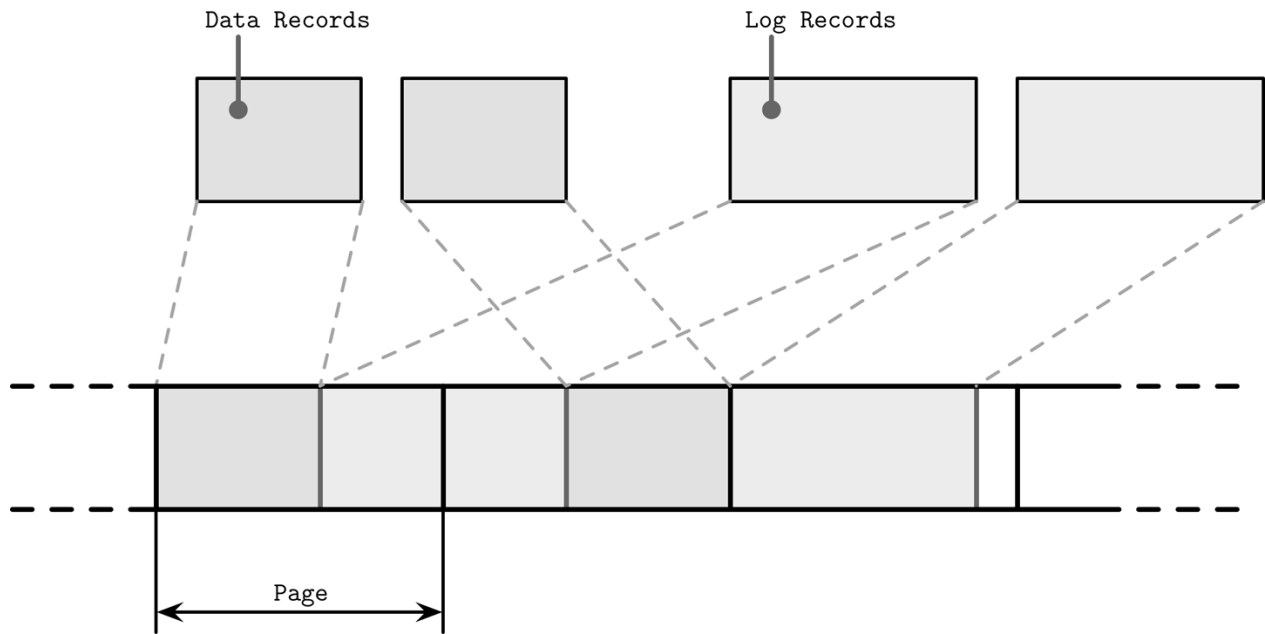


Figure 7-16. Unaligned multistream writes

Hình 7-16. Các lần ghi đa luồng không được căn

This results in fragmentation that we tried to avoid. To reduce interleaving, some database vendors recommend keeping the log on a separate device to isolate workloads and be able to reason about their performance and access patterns independently. However, it is more important to keep partitions aligned to the underlying hardware [INTEL14] and keep writes aligned to page size [KIM12].

Điều này dẫn đến phân mảnh mà ta đã cố tránh. Để giảm xen kẽ, một số nhà cung cấp cơ sở dữ liệu khuyến nghị giữ nhật ký trên một thiết bị riêng để cô lập các tải công việc và có thể suy luận về hiệu năng và các mẫu truy cập của chúng một cách độc lập. Tuy nhiên, quan trọng hơn là giữ các phân vùng được căn theo phân cứng nền tảng [INTEL14] và giữ các lần ghi được căn theo kích thước trang [KIM12].

LLAMA and Mindful Stacking — LLAMA và chồng tầng có ý thức (LLAMA and Mindful Stacking)

Well, you'll never believe this, but that llama you're looking at was once a human being. And not just any human being. That guy was an emperor. A rich, powerful ball of charisma. —Kuzco from The Emperor's New Groove

Ồ, bạn sẽ không bao giờ tin điều này đâu, nhưng con lạc đà không bướu mà bạn đang nhìn từng là một con người. Và không phải con người bình thường nào. Gã đó từng là một hoàng đế. Một khối uy lực giàu có, quyền lực, đầy sức hút. —Kuzco trong The Emperor's New Groove

In “Bw-Trees” on page 120, we discussed an immutable B-Tree version called Bw-Tree. **Bw-Tree** is layered on top of a **latch-free**, log-structured, access-method aware (LLAMA) storage subsystem. This layering allows Bw-Trees to grow and shrink dynamically, while leaving garbage collection and page management transparent for the tree. Here, we're most interested in the access-method aware part, demonstrating the benefits of coordination between the software layers.

Trong “Bw-Trees” ở trang 120, chúng ta đã thảo luận một phiên bản B-Tree bất biến gọi là Bw-Tree. Bw-Tree được phân lớp trên một phân hệ lưu trữ không-chốt, dạng nhật ký, nhận biết phương pháp truy cập (latch-free, log-structured, access-method aware - LLAMA). Việc phân lớp này cho phép Bw-Tree tăng và giảm động, trong khi để thu gom rác và quản lý trang trong suốt đối với cây. Ở đây, ta quan tâm nhất đến phần nhận biết phương pháp truy cập, thể hiện các lợi ích của sự phối hợp giữa các lớp phần mềm.

To recap, a logical Bw-Tree node consists of a linked list of physical delta nodes, a chain of updates from the newest one to the oldest one, ending in a base node. Logical nodes are linked using an in-memory mapping table, pointing to the location of the latest update on disk. Keys and values are added to and removed from the logical nodes, but their physical representations remain immutable.

Để tóm tắt, một nút Bw-Tree logic gồm một danh sách liên kết các nút delta vật lý, một chuỗi các cập nhật từ cập nhật mới nhất tới cập nhật cũ nhất, kết thúc ở một nút cơ sở. Các nút logic được liên kết bằng một bảng ánh xạ trong bộ nhớ, trỏ tới vị trí của cập nhật mới nhất trên đĩa. Các khóa và giá trị được thêm vào và gỡ khỏi các nút logic, nhưng biểu diễn vật lý của chúng giữ nguyên bất biến.

Log-structured storage buffers node updates (delta nodes) together in 4 Mb flush buffers. As soon as the page fills up, it's flushed on disk. Periodically, garbage collection reclaims space occupied by the unused delta and base nodes, and relocates the live ones to free up fragmented pages.

Lưu trữ dạng nhật ký đệm các cập nhật nút (các nút delta) lại với nhau trong các vùng đệm xả 4 Mb. Ngay khi trang đầy, nó được xả lên đĩa. Định kỳ, thu gom rác thu hồi không gian bị các nút delta và nút cơ sở không dùng đến chiếm giữ, và di dời các nút còn sống để giải phóng các trang bị phân mảnh.

Without access-method awareness, interleaved delta nodes that belong to different logical nodes will be written in their insertion order. Bw-Tree awareness in LLAMA allows for the consolidation of several delta nodes into a single contiguous physical location. If two updates in delta nodes cancel each other (for example, an insert followed by delete), their logical consolidation can be performed as well, and only the latter delete can be persisted.

Không có sự nhận biết phương pháp truy cập, các nút delta xen kẽ thuộc về các nút logic khác nhau sẽ được ghi theo thứ tự chèn của chúng. Sự nhận biết Bw-Tree trong LLAMA cho phép hợp nhất vài nút delta vào một vị trí vật lý liên tục duy nhất. Nếu hai cập nhật trong các nút delta triệt tiêu lẫn nhau (ví dụ, một lần chèn theo sau bởi một lần xóa), việc hợp nhất logic của chúng cũng có thể được thực hiện, và chỉ lần xóa sau cùng có thể được lưu bền.

LSS garbage collection can also take care of consolidating the logical Bw-Tree node contents. This means that garbage collection will not only reclaim the free space, but also significantly reduce the physical node fragmentation. If garbage collection only rewrote several delta nodes contiguously, they would still take the same amount of space, and readers would need to perform the work of applying the delta updates to the base node. At the same time, if a higher-level system consolidated the nodes and wrote them contiguously to the new locations, LSS would still have to garbage-collect the old versions.

Thu gom rác LSS cũng có thể lo việc hợp nhất nội dung nút Bw-Tree logic. Điều này nghĩa là thu gom rác sẽ không chỉ thu hồi không gian trống, mà còn giảm đáng kể phân mảnh nút vật lý. Nếu thu gom rác chỉ ghi lại vài nút delta một cách liên tục, chúng vẫn sẽ chiếm cùng một lượng không gian, và người đọc sẽ cần thực hiện công việc áp dụng các cập nhật delta lên nút cơ sở. Đồng thời, nếu một hệ thống mức cao hơn hợp nhất các nút và ghi chúng một cách liên tục vào các vị trí mới, LSS vẫn sẽ phải thu gom rác các phiên bản cũ.

By being aware of Bw-Tree semantics, several deltas may be rewritten as a single base node with all deltas already applied during garbage collection. This reduces the total space used to represent this Bw-Tree node and the latency required to read the page while reclaiming the space occupied by discarded pages.

Bằng cách nhận biết ngữ nghĩa của Bw-Tree, vài nút delta có thể được ghi lại như một nút cơ sở duy nhất với tất cả các delta đã được áp dụng trong khi thu gom rác. Điều này giảm tổng không gian dùng để biểu diễn nút Bw-Tree này và độ trễ cần để đọc trang trong khi thu hồi không gian bị các trang đã loại bỏ chiếm giữ.

You can see that, when considered carefully, stacking can yield many benefits. It is not necessary to always build tightly coupled single-level structures. Good APIs and exposing the right information can significantly improve efficiency.

Bạn có thể thấy rằng, khi được xét kỹ lưỡng, chồng tầng có thể mang lại nhiều lợi ích. Không nhất thiết phải luôn xây dựng các cấu trúc đơn-lớp ghép chặt. Các API tốt và việc phơi bày thông tin đúng có thể cải thiện đáng kể hiệu quả.

Open-Channel SSDs — Open-Channel SSD

An alternative to stacking software layers is to skip all indirection layers and use the hardware directly. For example, it is possible to avoid using a filesystem and flash translation layer by developing for Open-Channel SSDs. This way, we can avoid at least two layers of logs and have more control over wear-leveling, garbage collection, data placement, and scheduling. One of the implementations that uses this approach is LOCS (LSM Tree-based KV Store on Open-Channel SSD) [ZHANG13] . Another example using Open-Channel SSDs is LightNVM, implemented in the Linux kernel [BJØRLING17] .

Một giải pháp thay thế cho việc chồng các lớp phần mềm là bỏ qua tất cả các lớp gián tiếp và dùng phần cứng trực tiếp. Ví dụ, có thể tránh dùng một hệ thống tệp và lớp dịch flash bằng cách phát triển cho Open-Channel SSD. Theo cách này, ta có thể tránh ít nhất hai lớp nhật ký và có nhiều kiểm soát hơn đối với cân bằng hao mòn, thu gom rác, bố trí dữ liệu, và lập lịch. Một trong các hiện thực dùng cách tiếp cận này là LOCS (LSM Tree-based KV Store on Open-Channel SSD) [ZHANG13]. Một ví dụ khác dùng Open-Channel SSD là LightNVM, được hiện thực trong nhân Linux [BJØRLING17].

The flash translation layer usually handles data placement, garbage collection, and page relocation. Open-Channel SSDs expose their internals, drive management, and I/O scheduling without needing to go through the FTL. While this certainly requires much more attention to detail from the developer's perspective, this approach may yield significant performance improvements. You can draw a parallel with using the `O_DIRECT` flag to bypass the kernel page cache, which gives better control, but requires manual page management.

Lớp dịch flash thường xử lý bố trí dữ liệu, thu gom rác, và di dời trang. Open-Channel SSD phơi bày phần nội bộ, quản lý ổ đĩa, và lập lịch I/O của chúng mà không cần đi qua FTL. Mặc dù điều này chắc chắn đòi hỏi nhiều sự chú ý đến chi tiết hơn từ góc nhìn của nhà phát triển, cách tiếp cận này có thể mang lại các cải thiện hiệu năng đáng kể. Bạn có thể vẽ một sự tương đồng với việc dùng cờ `O_DIRECT` để bỏ qua page cache của nhân, vốn cho kiểm soát tốt hơn, nhưng đòi hỏi quản lý trang thủ công.

Software Defined Flash (SDF) [OUYANG14] , a hardware/software codesigned Open- Channel SSDs system, exposes an asymmetric I/O interface that takes SSD specifics into consideration. Sizes of read and write units are different, and write unit size corresponds to erase unit size (block), which greatly

reduces write amplification. This setting is ideal for log-structured storage, since there's only one software layer that performs garbage collection and relocates pages. Additionally, developers have access to internal SSD parallelism, since every channel in SDF is exposed as a separate block device, which can be used to further improve performance.

Software Defined Flash (SDF) [OUYANG14], một hệ thống Open-Channel SSD được đồng thiết kế phần cứng/phần mềm, phơi bày một giao diện I/O bất đối xứng xét đến các đặc thù của SSD. Các kích thước của đơn vị đọc và ghi là khác nhau, và kích thước đơn vị ghi tương ứng với kích thước đơn vị xóa (khối), điều này giảm đáng kể khuếch đại ghi. Thiết lập này lý tưởng cho lưu trữ dạng nhật ký, vì chỉ có một lớp phần mềm thực hiện thu gom rác và di dời các trang. Ngoài ra, các nhà phát triển có quyền truy cập tính song song nội bộ của SSD, vì mỗi kênh trong SDF được phơi bày như một thiết bị khối riêng, có thể được dùng để cải thiện hiệu năng hơn nữa.

Hiding complexity behind a simple API might sound compelling, but can cause complications in cases in which software layers have different semantics. Exposing some underlying system internals may be beneficial for better integration.

Ẩn độ phức tạp sau một API đơn giản nghe có vẻ hấp dẫn, nhưng có thể gây ra các phức tạp trong các trường hợp mà các lớp phần mềm có ngữ nghĩa khác nhau. Phơi bày một số phần nội bộ của hệ thống nền tảng có thể có lợi cho việc tích hợp tốt hơn.

Summary — Tóm tắt

Log-structured storage is used everywhere: from the flash translation layer, to filesystems and database systems. It helps to reduce write amplification by batching small random writes together in memory. To reclaim space occupied by removed segments, LSS periodically triggers garbage collection.

Lưu trữ dạng nhật ký được dùng ở khắp mọi nơi: từ lớp dịch flash, tới các hệ thống tệp và các hệ thống cơ sở dữ liệu. Nó giúp giảm khuếch đại ghi bằng cách gộp các lần ghi ngẫu nhiên nhỏ lại với nhau trong bộ nhớ. Để thu hồi không gian bị các phân đoạn đã gỡ bỏ chiếm giữ, LSS định kỳ kích hoạt thu gom rác.

LSM Trees take some ideas from LSS and help to build index structures managed in a log-structured manner: writes are batched in memory and flushed on disk; shadowed data records are cleaned up during compaction.

LSM Tree lấy một số ý tưởng từ LSS và giúp xây dựng các cấu trúc chỉ mục được quản lý theo cách dạng nhật ký: các lần ghi được gộp trong bộ nhớ và xả lên đĩa; các bản ghi dữ liệu bị che khuất được dọn dẹp trong khi nén-gộp.

It is important to remember that many software layers use LSS, and make sure that layers are stacked optimally. Alternatively, we can skip the filesystem level altogether and access hardware directly.

Quan trọng là nhớ rằng nhiều lớp phần mềm dùng LSS, và bảo đảm rằng các lớp được chồng một cách tối ưu. Một cách khác, ta có thể bỏ qua hoàn toàn mức hệ thống tệp và truy cập phần cứng trực tiếp.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Overview Luo, Chen, and Michael J. Carey. 2019. “LSM-based Storage Techniques: A Survey.” The VLDB Journal <https://doi.org/10.1007/s00778-019-00555-y>.

LSM Trees O’Neil, Patrick, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. 1996. “The log-structured merge-tree (LSM-tree).” Acta Informatica 33, no. 4: 351-385. <https://doi.org/10.1007/s002360050048>.

Bitcask Justin Sheehy, David Smith. “Bitcask: A Log-Structured Hash Table for Fast Key/ Value Data.” 2010.

WiscKey Lanyue Lu, Thanumalayan Sankaranarayanan Pillai, Hariharan Gopalakrishnan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2017. “WiscKey: Separating Keys from Values in SSD-Conscious Storage.” ACM Trans. Storage 13, 1, Article 5 (March 2017), 28 pages.

LOCS Peng Wang, Guangyu Sun, Song Jiang, Jian Ouyang, Shiding Lin, Chen Zhang, and Jason Cong. 2014. “An efficient design and implementation of LSM-tree based key-value store on open-channel SSD.” In Proceedings of the Ninth European Conference on Computer Systems (EuroSys ’14). ACM, New York, NY, USA, Article 16, 14 pages.

LLAMA Justin Levandoski, David Lomet, and Sudipta Sengupta. 2013. “LLAMA: a cache/ storage subsystem for modern hardware.” Proc. VLDB Endow. 6, 10 (August 2013), 877-888.

Part I Conclusion — Kết luận Phần I

In Part I, we’ve been talking about storage engines. We started from high-level database system architecture and classification, learned how to implement on-disk storage structures, and how they fit into the full picture with other components.

Trong Phần I, chúng ta đã bàn về các bộ máy lưu trữ (storage engine). Chúng ta bắt đầu từ kiến trúc và phân loại hệ cơ sở dữ liệu ở mức cao, học cách hiện thực hóa các cấu trúc lưu trữ trên đĩa, và cách chúng ghép vào bức tranh tổng thể cùng các thành phần khác.

We’ve seen several storage structures, starting from B-Trees. The discussed structures do not represent an entire field, and there are many other interesting developments. However, these examples are still a good illustration of the three properties we identified at the beginning of this part: **buffering**, **immutability**, and **ordering**. These properties are useful for describing, memorizing, and expressing different aspects of the storage structures.

Chúng ta đã thấy nhiều cấu trúc lưu trữ, bắt đầu từ B-Tree. Các cấu trúc được thảo luận không đại diện cho toàn bộ lĩnh vực, và còn nhiều phát triển thú vị khác nữa. Tuy nhiên, những ví dụ này vẫn là minh họa tốt cho ba thuộc tính mà chúng ta đã xác định ở đầu phần này: đệm (buffering), tính bất biến (immutability) và sắp thứ tự (ordering). Các thuộc tính này hữu ích để mô tả, ghi nhớ và biểu đạt những khía cạnh khác nhau của các cấu trúc lưu trữ.

Figure I-1 summarizes the discussed storage structures and shows whether or not they’re using these properties.

Hình I-1 tóm tắt các cấu trúc lưu trữ đã thảo luận và cho thấy chúng có sử dụng những thuộc tính này hay không.

Adding in-memory buffers always has a positive impact on **write amplification**. In in-place update structures like WiredTiger and LA-Trees, in-memory buffering helps to amortize the cost of multiple same-**page** writes by combining them. In other words, buffering helps to reduce write amplification.

Thêm vùng đệm trong bộ nhớ luôn có tác động tích cực tới khuếch đại ghi (write amplification). Trong các cấu trúc cập nhật tại chỗ (in-place update) như WiredTiger và LA-Tree, việc đệm trong bộ nhớ giúp khấu hao chi phí của nhiều lần ghi cùng một trang bằng cách gộp chúng lại. Nói cách khác, đệm giúp giảm khuếch đại ghi.

In **immutable** structures, such as multicomponent LSM Trees and FD-Trees, buffering has a similar positive effect, but at a cost of future rewrites when moving data from one immutable level to the other. In other words, using immutability may lead to deferred write amplification. At the same time, using immutability has a positive impact on concurrency and **space amplification**, since most of the discussed immutable structures use fully occupied pages.

Trong các cấu trúc bất biến, chẳng hạn LSM Tree đa thành phần và FD-Tree, đệm cũng có tác động tích cực tương tự, nhưng phải trả giá bằng việc ghi lại trong tương lai khi di chuyển dữ liệu từ một tầng bất biến này sang tầng khác. Nói cách khác, dùng tính bất biến có thể dẫn tới khuếch đại ghi bị trì hoãn (deferred). Đồng thời, dùng tính bất biến lại có tác động tích cực tới tính tương tranh và khuếch đại không gian, vì hầu hết các cấu trúc bất biến được thảo luận đều dùng các trang được lấp đầy hoàn toàn.

When using immutability, unless we also use buffering, we end up with unordered storage structures like Bitcask and WiscKey (with the exception of **copy-on-write** B- Trees, which copy, re-sort, and relocate their pages). WiscKey stores only keys in sorted LSM Trees and allows retrieving records in key order using the key index. In Bw- Trees, some of the nodes (ones that were consolidated) hold data records in key order,

Khi dùng tính bất biến, trừ khi ta cũng dùng đệm, ta sẽ thu được các cấu trúc lưu trữ không sắp thứ tự như Bitcask và WiscKey (ngoại lệ là B-Tree sao-khi-ghi (copy-on-write), vốn sao chép, sắp xếp lại và di dời các trang của nó). WiscKey chỉ lưu các khóa trong LSM Tree đã được sắp xếp và cho phép truy xuất các bản ghi theo thứ tự khóa nhờ chỉ mục khóa. Trong Bw-Tree, một số nút (những nút đã được hợp nhất) giữ các bản ghi dữ liệu theo thứ tự khóa,

165 while the rest of the logical **Bw-Tree** nodes may have their delta updates scattered across different pages.

165 trong khi các nút logic Bw-Tree còn lại có thể có các bản cập nhật delta của chúng rải rác trên những trang khác nhau.

	Buffered	Mutable	Ordered
B+Trees	No	Yes	Yes
WiredTiger	Yes	Yes	Yes
LA-Trees	Yes	Yes	Yes
COW B-Trees	No	No	Yes
2C LSM Trees	Yes	No	Yes
MC LSM Trees	Yes	No	Yes
FD-Trees	Yes	No	Yes
BitCask	No	No	No
WiscKey	Yes (1)	No	Yes (1)
BW-Trees	No	No	No (2)

Figure I-1. Buffering, immutability, and ordering properties of discussed storage structures. (1) WiscKey uses buffering only for keeping keys sorted order. (2) Only consolidated nodes in Bw-Trees hold ordered records.

Hình I-1. Các thuộc tính đệm, bất biến và sắp thứ tự của những cấu trúc lưu trữ đã thảo luận. (1) WiscKey chỉ dùng đệm để giữ các khóa theo thứ tự đã sắp xếp. (2) Chỉ những nút đã được hợp nhất trong Bw-Tree mới giữ các bản ghi theo thứ tự.

You see that these three properties can be mixed and matched in order to achieve the desired characteristics. Unfortunately, **storage engine** design usually involves trade-offs: you increase the cost of one operation in favor of the other.

Bạn thấy rằng ba thuộc tính này có thể được pha trộn và kết hợp để đạt được những đặc tính mong muốn. Tiếc thay, thiết kế bộ máy lưu trữ thường kéo theo các đánh đổi: bạn tăng chi phí của một thao tác để đổi lấy lợi ích cho thao tác khác.

Using this knowledge, you should be able to start looking closer at the code of most modern database systems. Some of the code references and starting points can be found across the entire book. Knowing and understanding the terminology will make this **process** easier for you.

Với kiến thức này, bạn sẽ có thể bắt đầu xem xét kỹ hơn mã nguồn của hầu hết các hệ cơ sở dữ liệu hiện đại. Một số tham chiếu mã nguồn và điểm khởi đầu có thể tìm thấy rải rác xuyên suốt cuốn sách. Việc biết và hiểu thuật ngữ sẽ làm cho quá trình này dễ dàng hơn với bạn.

Many modern database systems are powered by probabilistic data structures [FLAJO- LET12] [CORMODE04] , and there's new research being done on bringing ideas from machine learning

into database systems [KRASKA18] . We're about to experience further changes in research and industry as nonvolatile and byte-addressable storage becomes more prevalent and widely available [VENKATARAMAN11] .

Nhiều hệ cơ sở dữ liệu hiện đại được vận hành nhờ các cấu trúc dữ liệu xác suất (probabilistic) [FLAJO-LET12] [CORMODE04], và đang có những nghiên cứu mới nhằm đưa các ý tưởng từ học máy vào hệ cơ sở dữ liệu [KRASKA18]. Chúng ta sắp trải nghiệm thêm những thay đổi trong nghiên cứu và công nghiệp khi lưu trữ phi biến đổi (nonvolatile) và định địa chỉ theo byte (byte-addressable) trở nên phổ biến và sẵn có rộng rãi hơn [VENKATARAMAN11].

Knowing the fundamental concepts described in this book should help you to understand and implement newer research, since it borrows from, builds upon, and is inspired by the same concepts. The major advantage of knowing the theory and history is that there's nothing entirely new and, as the narrative of this book shows, progress is incremental.

Biết các khái niệm nền tảng được mô tả trong cuốn sách này sẽ giúp bạn hiểu và hiện thực hóa những nghiên cứu mới hơn, vì chúng vay mượn, xây dựng trên và được truyền cảm hứng từ chính những khái niệm đó. Lợi thế lớn nhất của việc nắm vững lý thuyết và lịch sử là không có gì hoàn toàn mới mẻ, và như mạch tự sự của cuốn sách này cho thấy, tiến bộ là tích lũy dần dần.

PART II Distributed Systems — PHẦN II Hệ phân tán

A **distributed system** is one in which the failure of a computer you didn't even know existed can render your own computer unusable. —Leslie Lamport

Hệ phân tán là hệ mà ở đó sự cố của một máy tính mà bạn thậm chí không biết là nó tồn tại lại có thể khiến chính máy tính của bạn trở nên vô dụng. —Leslie Lamport

Without distributed systems, we wouldn't be able to make phone calls, transfer money, or exchange information over long distances. We use distributed systems daily. Sometimes, even without acknowledging it: any client/server application is a distributed system.

Không có hệ phân tán, chúng ta sẽ không thể gọi điện thoại, chuyển tiền, hay trao đổi thông tin qua những khoảng cách xa. Chúng ta sử dụng hệ phân tán hằng ngày. Đôi khi, thậm chí mà không nhận ra: bất kỳ ứng dụng client/server nào cũng là một hệ phân tán.

For many modern software systems, vertical scaling (scaling by running the same software on a bigger, faster machine with more CPU, RAM, or faster disks) isn't viable. Bigger machines are more expensive, harder to replace, and may require special maintenance. An alternative is to scale horizontally : to run software on multiple machines connected over the network and working as a single logical entity.

Đối với nhiều hệ phần mềm hiện đại, mở rộng theo chiều dọc (vertical scaling — mở rộng bằng cách chạy cùng phần mềm đó trên một máy lớn hơn, nhanh hơn với nhiều CPU, RAM hơn, hoặc đĩa nhanh hơn) là không khả thi. Máy lớn hơn thì đắt hơn, khó thay thế hơn, và có thể đòi hỏi bảo trì đặc biệt. Một phương án thay thế là mở rộng theo chiều ngang (horizontally): chạy phần mềm trên nhiều máy được kết nối qua mạng và hoạt động như một thực thể logic duy nhất.

Distributed systems might differ both in size, from a handful to hundreds of machines, and in characteristics of their participants, from small handheld or sensor devices to high-performance computers.

Các hệ phân tán có thể khác nhau cả về quy mô, từ vài máy đến hàng trăm máy, lẫn về đặc tính của các bên tham gia, từ các thiết bị cầm tay nhỏ hoặc cảm biến cho tới các máy tính hiệu năng cao.

The time when database systems were mainly running on a single **node** is long gone, and most modern database systems have multiple nodes connected in clusters to increase storage capacity, improve performance, and enhance availability.

Thời mà các hệ cơ sở dữ liệu chủ yếu chạy trên một nút đơn đã qua từ lâu, và hầu hết các hệ cơ sở dữ liệu hiện đại đều có nhiều nút được kết nối thành cụm (cluster) để tăng dung lượng lưu trữ, cải thiện hiệu năng và nâng cao tính sẵn sàng.

Even though some of the theoretical breakthroughs in distributed computing aren't new, most of their practical application happened relatively recently. Today, we see increasing interest in the subject, more research, and new development being done.

Mặc dù một số đột phá lý thuyết trong tính toán phân tán không phải là mới, hầu hết ứng dụng thực tiễn của chúng mới chỉ diễn ra tương đối gần đây. Ngày nay, chúng ta thấy sự quan tâm ngày càng tăng đối với chủ đề này, nhiều nghiên cứu hơn và nhiều phát triển mới đang được thực hiện.

Basic definitions — Các định nghĩa cơ bản

In a distributed system, we have several participants (sometimes called processes , nodes , or replicas). Each **participant** has its own local state . Participants communicate by exchanging messages using communication links between them.

Trong một hệ phân tán, chúng ta có nhiều bên tham gia (đôi khi gọi là tiến trình, nút, hoặc bản sao). Mỗi bên tham gia có trạng thái cục bộ (local state) riêng. Các bên tham gia giao tiếp bằng cách trao đổi thông điệp qua những liên kết truyền thông (communication link) giữa chúng.

Processes can access the time using a clock , which can be logical or physical . Logical clocks are implemented using a kind of monotonically growing counter. Physical clocks, also called wall clocks , are bound to a notion of time in the physical world and are accessible through process-local means; for example, through an operating system.

Các tiến trình có thể truy cập thời gian thông qua một đồng hồ (clock), đồng hồ này có thể là logic hoặc vật lý. Đồng hồ logic được hiện thực bằng một dạng bộ đếm tăng đơn điệu. Đồng hồ vật lý, còn gọi là đồng hồ treo tường (wall clock), gắn với khái niệm thời gian trong thế giới vật lý và truy cập được thông qua các phương tiện cục bộ của tiến trình; ví dụ, thông qua hệ điều hành.

It's impossible to talk about distributed systems without mentioning the inherent difficulties caused by the fact that its parts are located apart from each other. Remote processes communicate through links that can be slow and unreliable, which makes knowing the exact state of the remote process more complicated.

Không thể bàn về hệ phân tán mà không nhắc tới những khó khăn cố hữu phát sinh từ việc các bộ phận của nó nằm cách xa nhau. Các tiến trình từ xa giao tiếp qua các liên kết vốn có thể chậm và không đáng tin cậy, điều này khiến việc biết chính xác trạng thái của tiến trình từ xa trở nên phức tạp hơn.

Most of the research in the distributed systems field is related to the fact that nothing is entirely reliable: communication channels may delay, reorder, or fail to deliver the messages; processes may

pause, slow down, crash, go out of control, or suddenly stop responding.

Phần lớn nghiên cứu trong lĩnh vực hệ phân tán liên quan đến thực tế là không có gì hoàn toàn đáng tin cậy: các kênh truyền thông có thể làm trễ, đảo thứ tự, hoặc không gửi được thông điệp; các tiến trình có thể tạm dừng, chậm lại, sập, mất kiểm soát, hoặc đột ngột ngừng phản hồi.

There are many themes in common in the fields of concurrent and distributed programming, since CPUs are tiny distributed systems with links, processors, and communication protocols. You'll see many parallels with concurrent programming in “Consistency Models” on page 222 . However, most of the primitives can't be reused directly because of the costs of communication between remote parties, and the unreliability of links and processes.

Có nhiều chủ đề chung giữa hai lĩnh vực lập trình tương tranh (concurrent) và lập trình phân tán, vì CPU chính là những hệ phân tán tí hon với các liên kết, bộ xử lý và giao thức truyền thông. Bạn sẽ thấy nhiều điểm tương đồng với lập trình tương tranh trong phần “Các mô hình nhất quán” (Consistency Models) ở trang 222. Tuy nhiên, hầu hết các nguyên thủy (primitive) không thể tái sử dụng trực tiếp được vì chi phí giao tiếp giữa các bên ở xa, cùng với sự kém tin cậy của các liên kết và tiến trình.

To overcome the difficulties of the distributed environment, we need to use a particular class of algorithms, distributed algorithms , which have notions of local and remote state and execution and work despite unreliable networks and component failures. We describe algorithms in terms of state and steps (or phases), with transitions between them. Each process executes the algorithm steps locally, and a combination of local executions and process interactions constitutes a distributed algorithm.

Để vượt qua những khó khăn của môi trường phân tán, chúng ta cần dùng một lớp thuật toán riêng — các thuật toán phân tán (distributed algorithm) — vốn có khái niệm về trạng thái và việc thực thi cục bộ lẫn từ xa, và hoạt động bất chấp mạng không tin cậy và các thành phần gặp lỗi. Chúng ta mô tả các thuật toán theo trạng thái và các bước (hay pha), với các chuyển dịch giữa chúng. Mỗi tiến trình thực thi các bước của thuật toán một cách cục bộ, và sự kết hợp của các lần thực thi cục bộ cùng các tương tác giữa tiến trình tạo nên một thuật toán phân tán.

Distributed algorithms describe the local behavior and interaction of multiple independent nodes. Nodes communicate by sending messages to each other. Algorithms define participant roles, exchanged messages, states, transitions, executed steps, properties of the delivery medium, timing assumptions, failure models, and other characteristics that describe processes and their interactions.

Các thuật toán phân tán mô tả hành vi cục bộ và tương tác của nhiều nút độc lập. Các nút giao tiếp bằng cách gửi thông điệp cho nhau. Thuật toán định nghĩa vai trò của các bên tham gia, các thông điệp được trao đổi, các trạng thái, các chuyển dịch, các bước được thực thi, thuộc tính của môi trường truyền tin, các giả định về thời gian, các mô hình lỗi, và những đặc tính khác mô tả các tiến trình cùng tương tác của chúng.

Distributed algorithms serve many different purposes:

Các thuật toán phân tán phục vụ nhiều mục đích khác nhau:

Coordination A process that supervises the actions and behavior of several workers.

Điều phối (Coordination) Một tiến trình giám sát các hành động và hành vi của nhiều tiến trình thợ (worker).

Cooperation Multiple participants relying on one another for finishing their tasks.

Hợp tác (Cooperation) Nhiều bên tham gia dựa vào nhau để hoàn thành các nhiệm vụ của mình.

Dissemination Processes cooperating in spreading the information to all interested parties quickly and reliably.

Lan truyền (Dissemination) Các tiến trình hợp tác trong việc phát tán thông tin đến mọi bên quan tâm một cách nhanh chóng và đáng tin cậy.

Consensus Achieving agreement among multiple processes.

Đồng thuận (Consensus) Đạt được sự nhất trí giữa nhiều tiến trình.

In this book, we talk about algorithms in the context of their usage and prefer a practical approach over purely academic material. First, we cover all necessary abstractions, the processes and the connections between them, and progress to building more complex communication patterns. We start with UDP, where the sender doesn't have any guarantees on whether or not its message has reached its destination; and finally, to achieve consensus, where multiple processes agree on a specific value.

Trong cuốn sách này, chúng ta bàn về các thuật toán trong ngữ cảnh sử dụng của chúng và ưu tiên cách tiếp cận thực tiễn hơn là tài liệu thuần túy hàn lâm. Trước hết, chúng ta trình bày mọi trừu tượng cần thiết — các tiến trình và những kết nối giữa chúng — rồi tiến tới xây dựng những khuôn mẫu giao tiếp phức tạp hơn. Chúng ta bắt đầu với UDP, nơi bên gửi không có bất kỳ bảo đảm nào về việc thông điệp của nó đã tới đích hay chưa; và cuối cùng, để đạt được đồng thuận, nơi nhiều tiến trình đồng ý về một giá trị cụ thể.

CHAPTER 8 Introduction and Overview — Chương 8: Giới thiệu và Tổng quan

What makes distributed systems inherently different from single-**node** systems? Let's take a look at a simple example and try to see. In a single-threaded program, we define variables and the execution **process** (a set of steps).

Điều gì khiến hệ phân tán (distributed system) về bản chất lại khác với hệ thống đơn nút (single-node)? Hãy cùng xem một ví dụ đơn giản và thử tìm hiểu. Trong một chương trình đơn luồng, chúng ta định nghĩa các biến và quá trình thực thi (một tập các bước).

For example, we can define a variable and perform simple arithmetic operations over it:

Ví dụ, chúng ta có thể định nghĩa một biến và thực hiện các phép toán số học đơn giản trên nó:

```
int i = 1;
i += 2;
i *= 2;
```

We have a single execution history: we declare a variable, increment it by two, then multiply it by two, and get the result: 6 . Let's say that, instead of having one execution thread performing these operations, we have two threads that have **read** and write access to variable x .

Chúng ta có một lịch sử thực thi duy nhất: khai báo một biến, tăng nó lên hai đơn vị, rồi nhân nó với hai, và nhận kết quả: 6. Giả sử rằng, thay vì chỉ có một luồng thực thi thực hiện các thao tác này, ta có hai luồng cùng có quyền đọc và ghi vào biến x.

Concurrent Execution — Thực thi tương tranh

As soon as two execution threads are allowed to access the variable, the exact outcome of the concurrent step execution is unpredictable, unless the steps are synchronized between the threads. Instead of a single possible outcome, we end up with four, as Figure 8-1 shows. 1

Ngay khi hai luồng thực thi được phép truy cập vào biến, kết quả chính xác của việc thực thi các bước theo kiểu tương tranh trở nên không thể dự đoán được, trừ khi các bước được đồng bộ giữa các luồng. Thay vì một kết quả khả dĩ duy nhất, ta thu được tới bốn kết quả, như Hình 8-1 cho thấy. 1

1 Interleaving, where the multiplier reads before the adder, is left out for brevity, since it yields the same result as a).

1 Cách xen kẽ (interleaving) trong đó bộ nhân đọc trước bộ cộng được bỏ qua cho gọn, vì nó cho cùng kết quả như trường hợp a).

171

171

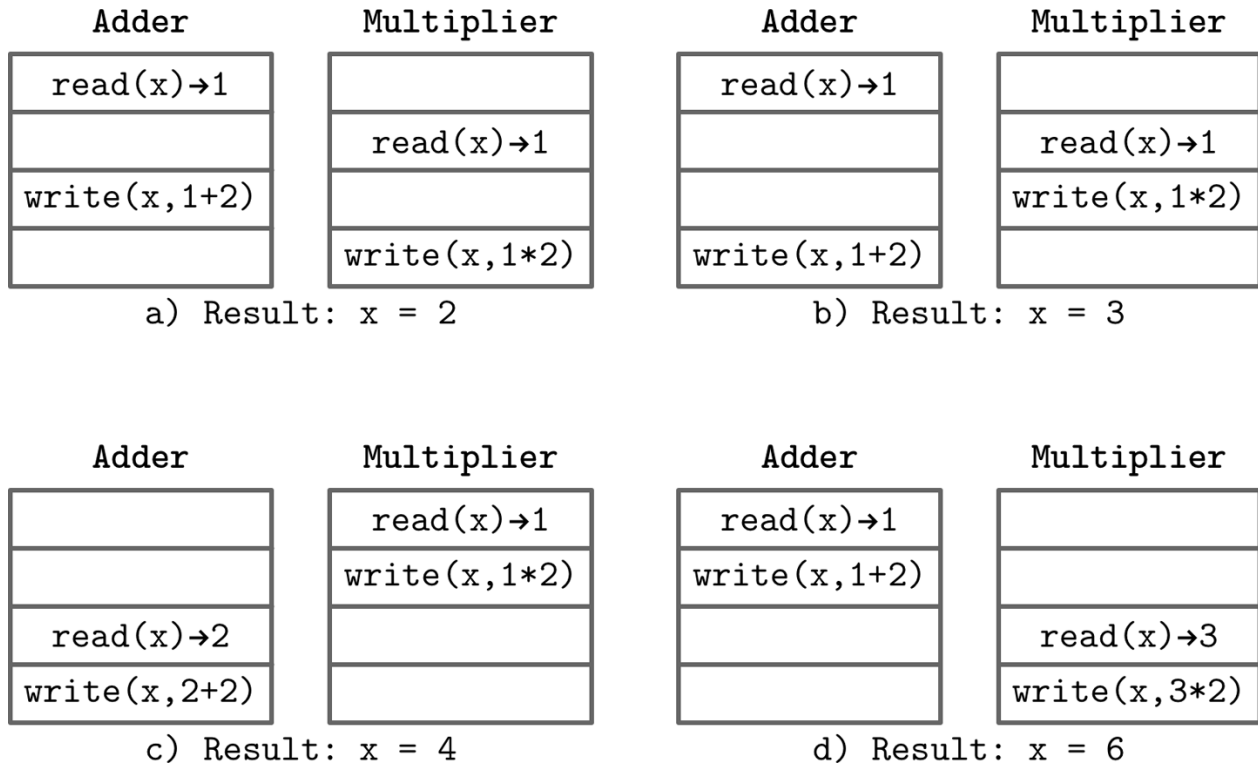


Figure 8-1. Possible interleavings of concurrent executions

Hình 8-1. Các cách xen kẽ khả dĩ của các lần thực thi tương tranh

- a) $x = 2$, if both threads read an initial value, the adder writes its value, but it is overwritten with the multiplication result.
- b) $x = 3$, if both threads read an initial value, the multiplier writes its value, but it is overwritten with the addition result.
- c) $x = 4$, if the multiplier can read the initial value and execute its operation before the adder starts.
- d) $x = 6$, if the adder can read the initial value and execute its operation before the multiplier starts.
- a) $x = 2$, nếu cả hai luồng đều đọc giá trị ban đầu, bộ cộng ghi giá trị của nó, nhưng giá trị đó bị ghi đè bởi kết quả của phép nhân.
- b) $x = 3$, nếu cả hai luồng đều đọc giá trị ban đầu, bộ nhân ghi giá trị của nó, nhưng giá trị đó bị ghi đè bởi kết quả của phép cộng.
- c) $x = 4$, nếu bộ nhân có thể đọc giá trị ban đầu và thực hiện phép toán của nó trước khi bộ cộng bắt đầu.

- d) $x = 6$, nếu bộ cộng có thể đọc giá trị ban đầu và thực hiện phép toán của nó trước khi bộ nhân bắt đầu.

Even before we can cross a single node boundary, we encounter the first problem in distributed systems: concurrency . Every concurrent program has some properties of a **distributed system**. Threads access the shared state, perform some operations locally, and propagate the results back to the shared variables.

Ngay cả trước khi vượt qua ranh giới của một nút duy nhất, chúng ta đã gặp vấn đề đầu tiên trong hệ phân tán: tính tương tranh (concurrency). Mọi chương trình tương tranh đều mang một số đặc tính của hệ phân tán. Các luồng truy cập trạng thái dùng chung, thực hiện một số thao tác cục bộ, rồi lan truyền kết quả trở lại các biến dùng chung.

To define execution histories precisely and reduce the number of possible outcomes, we need consistency models . Consistency models describe concurrent executions and establish an order in which operations can be executed and made visible to the participants. Using different consistency models, we can constraint or relax the number of states the system can be in.

Để định nghĩa lịch sử thực thi một cách chính xác và giảm số lượng kết quả khả dĩ, chúng ta cần đến các mô hình nhất quán (consistency model). Các mô hình nhất quán mô tả các lần thực thi tương tranh và thiết lập một thứ tự mà các thao tác có thể được thực thi và được làm cho hiển thị với các bên tham gia. Bằng cách sử dụng những mô hình nhất quán khác nhau, ta có thể ràng buộc hoặc nới lỏng số lượng trạng thái mà hệ thống có thể rơi vào.

There is a lot of overlap in terminology and research in the areas of distributed systems and concurrent computing, but there are also some differences. In a concurrent system, we can have shared memory , which processors can use to exchange the information. In a distributed system, each processor has its local state and participants communicate by passing messages.

Có rất nhiều điểm trùng lặp về thuật ngữ và nghiên cứu giữa hai lĩnh vực hệ phân tán và tính toán tương tranh, nhưng cũng có một số khác biệt. Trong một hệ tương tranh, ta có thể có bộ nhớ dùng chung (shared memory) mà các bộ xử lý có thể dùng để trao đổi thông tin. Trong một hệ phân tán, mỗi bộ xử lý có trạng thái cục bộ riêng và các bên tham gia giao tiếp bằng cách truyền thông điệp (message passing).

Concurrent and Parallel — Tương tranh và song song

We often use the terms concurrent and parallel computing interchangeably, but these concepts have a slight semantic difference. When two sequences of steps execute concurrently, both of them are in progress, but only one of them is executed at any moment. If two sequences execute in parallel, their steps can be executed simultaneously. Concurrent operations overlap in time, while parallel operations are executed by multiple processors [WEIKUM01] .

Chúng ta thường dùng các thuật ngữ tính toán tương tranh (concurrent) và tính toán song song (parallel) thay thế cho nhau, nhưng các khái niệm này có một khác biệt nhỏ về ngữ nghĩa. Khi hai chuỗi bước thực thi một cách tương tranh, cả hai đều đang được tiến hành, nhưng tại bất kỳ thời điểm nào cũng chỉ có một chuỗi đang được thực thi. Nếu hai chuỗi thực thi song song, các bước của chúng có thể được thực thi đồng thời. Các thao tác tương tranh chồng lấn nhau về mặt thời gian, trong khi các thao tác song song được thực thi bởi nhiều bộ xử lý [WEIKUM01].

Joe Armstrong, creator of the Erlang programming language, gave an example : concurrent execution is like having two queues to a single coffee machine, while parallel execution is like having two

queues to two coffee machines. That said, the vast majority of sources use the **term** concurrency to describe systems with several parallel execution threads, and the term parallelism is rarely used.

Joe Armstrong, người tạo ra ngôn ngữ lập trình Erlang, đã đưa ra một ví dụ: thực thi tương tranh giống như có hai hàng người xếp hàng cho một máy pha cà phê duy nhất, còn thực thi song song giống như có hai hàng người xếp hàng cho hai máy pha cà phê. Dẫu vậy, đại đa số các nguồn tài liệu đều dùng thuật ngữ tương tranh (concurrency) để mô tả các hệ thống có nhiều luồng thực thi song song, còn thuật ngữ song song (parallelism) thì hiếm khi được dùng.

Shared State in a Distributed System — Trạng thái dùng chung trong hệ phân tán

We can try to introduce some notion of shared memory to a distributed system, for example, a single source of information, such as database. Even if we solve the problems with concurrent access to it, we still cannot guarantee that all processes are in sync.

Chúng ta có thể thử đưa một khái niệm bộ nhớ dùng chung nào đó vào hệ phân tán, ví dụ như một nguồn thông tin duy nhất, chẳng hạn một cơ sở dữ liệu. Ngay cả khi đã giải quyết được các vấn đề về truy cập tương tranh tới nó, ta vẫn không thể đảm bảo rằng tất cả các tiến trình đều đồng bộ với nhau.

To access this database, processes have to go over the communication medium by sending and receiving messages to query or modify the state. However, what happens if one of the processes does not receive a response from the database for a longer time? To answer this question, we first have to define what longer even means. To do this, the system has to be described in terms of synchrony : whether the communication is fully **asynchronous**, or whether there are some timing assumptions. These timing assumptions allow us to introduce operation timeouts and retries.

Để truy cập cơ sở dữ liệu này, các tiến trình phải đi qua môi trường truyền thông bằng cách gửi và nhận thông điệp nhằm truy vấn hoặc sửa đổi trạng thái. Tuy nhiên, điều gì xảy ra nếu một trong các tiến trình không nhận được phản hồi từ cơ sở dữ liệu trong một khoảng thời gian khá dài? Để trả lời câu hỏi này, trước hết ta phải định nghĩa khá dài thực ra nghĩa là gì. Muốn vậy, hệ thống phải được mô tả theo phương diện tính đồng bộ (synchrony): liệu việc giao tiếp là hoàn toàn bất đồng bộ, hay có một số giả định về thời gian. Những giả định về thời gian này cho phép ta đưa vào cơ chế thời gian chờ (timeout) và thử lại (retry) cho các thao tác.

We do not know whether the database hasn't responded because it's overloaded, unavailable, or slow, or because of some problems with the network on the way to it. This describes a nature of a crash: processes may crash by failing to participate in further algorithm steps, having a temporary failure, or by omitting some of the messages. We need to define a failure model and describe ways in which failures can occur before we decide how to treat them.

Chúng ta không biết liệu cơ sở dữ liệu chưa phản hồi là vì nó bị quá tải, không khả dụng, hay chậm, hay vì một số vấn đề với mạng trên đường đi tới nó. Điều này mô tả bản chất của một sự cố sập: các tiến trình có thể sập do không tham gia vào các bước tiếp theo của thuật toán, do gặp lỗi tạm thời, hoặc do bỏ sót một số thông điệp. Chúng ta cần định nghĩa một mô hình lỗi (failure model) và mô tả những cách mà lỗi có thể xảy ra trước khi quyết định cách xử lý chúng.

A property that describes system reliability and whether or not it can continue operating correctly in the presence of failures is called fault tolerance . Failures are inevitable, so we need to build systems

with reliable components, and eliminating a single point of failure in the form of the aforementioned single-node database can be the first step in this direction. We can do this by introducing some redundancy and adding a backup database. However, now we face a different problem: how do we keep multiple copies of shared state in sync?

Một đặc tính mô tả độ tin cậy của hệ thống và liệu nó có thể tiếp tục vận hành đúng đắn khi có lỗi hay không được gọi là khả năng dung lỗi (fault tolerance). Lỗi là điều không thể tránh khỏi, nên ta cần xây dựng các hệ thống bằng những thành phần đáng tin cậy, và loại bỏ điểm lỗi đơn lẻ (single point of failure) dưới dạng cơ sở dữ liệu đơn nút nói trên có thể là bước đầu tiên theo hướng này. Ta có thể làm điều đó bằng cách đưa vào một mức dư thừa nào đó và thêm một cơ sở dữ liệu dự phòng. Tuy nhiên, giờ đây ta lại đối mặt với một vấn đề khác: làm sao để giữ cho nhiều bản sao của trạng thái dùng chung được đồng bộ với nhau?

So far, trying to introduce shared state to our simple system has left us with more questions than answers. We now know that sharing state is not as simple as just introducing a database, and have to take a more granular approach and describe interactions in terms of independent processes and passing messages between them.

Cho đến nay, việc cố gắng đưa trạng thái dùng chung vào hệ thống đơn giản của chúng ta đã để lại nhiều câu hỏi hơn là câu trả lời. Giờ ta biết rằng việc chia sẻ trạng thái không đơn giản như chỉ cần thêm một cơ sở dữ liệu, và phải áp dụng một cách tiếp cận chi tiết hơn, mô tả các tương tác theo phương diện các tiến trình độc lập và việc truyền thông điệp giữa chúng.

Fallacies of Distributed Computing — Những ngộ nhận của tính toán phân tán

In an ideal case, when two computers talk over the network, everything works just fine: a process opens up a connection, sends the data, gets responses, and everyone is happy. Assuming that operations always succeed and nothing can go wrong is dangerous, since when something does break and our assumptions turn out to be wrong, systems behave in ways that are hard or impossible to predict.

Trong trường hợp lý tưởng, khi hai máy tính trò chuyện qua mạng, mọi thứ diễn ra suôn sẻ: một tiến trình mở một kết nối, gửi dữ liệu, nhận phản hồi, và ai cũng hài lòng. Giả định rằng các thao tác luôn thành công và không có gì có thể trục trặc là điều nguy hiểm, vì khi có thứ gì đó hỏng và các giả định của ta hóa ra sai, hệ thống sẽ hành xử theo những cách khó hoặc không thể dự đoán.

Most of the time, assuming that the network is reliable is a reasonable thing to do. It has to be reliable to at least some extent to be useful. We've all been in the situation when we tried to establish a connection to the remote server and got a Network is Unreachable error instead. But even if it is possible to establish a connection, a successful initial connection to the server does not guarantee that the **link** is stable, and the connection can get interrupted at any time. The message might've reached the remote party, but the response could've gotten lost, or the connection was interrupted before the response was delivered.

Phần lớn thời gian, giả định rằng mạng là đáng tin cậy là một điều hợp lý để làm. Mạng phải đáng tin cậy ở mức nào đó thì mới có ích. Tất cả chúng ta đều từng ở trong tình huống cố thiết lập kết nối tới máy chủ từ xa nhưng thay vào đó lại nhận được lỗi "Network is Unreachable". Nhưng ngay cả khi có thể thiết lập kết nối, một lần kết nối ban đầu thành

công tới máy chủ cũng không đảm bảo rằng đường liên kết là ổn định, và kết nối có thể bị gián đoạn bất cứ lúc nào. Thông điệp có thể đã tới được bên kia, nhưng phản hồi có thể đã bị mất, hoặc kết nối đã bị gián đoạn trước khi phản hồi được chuyển tới.

Network switches break, cables get disconnected, and network configurations can change at any time. We should build our system by handling all of these scenarios gracefully.

Bộ chuyển mạch mạng hỏng, cáp bị ngắt, và cấu hình mạng có thể thay đổi bất cứ lúc nào. Chúng ta nên xây dựng hệ thống của mình sao cho xử lý được tất cả các kịch bản này một cách nhẹ nhàng.

A connection can be stable, but we can't expect remote calls to be as fast as the local ones. We should make as few assumptions about latency as possible and never assume that latency is zero . For our message to reach a remote server, it has to go through several software layers, and a physical medium such as optic fiber or a cable. All of these operations are not instantaneous.

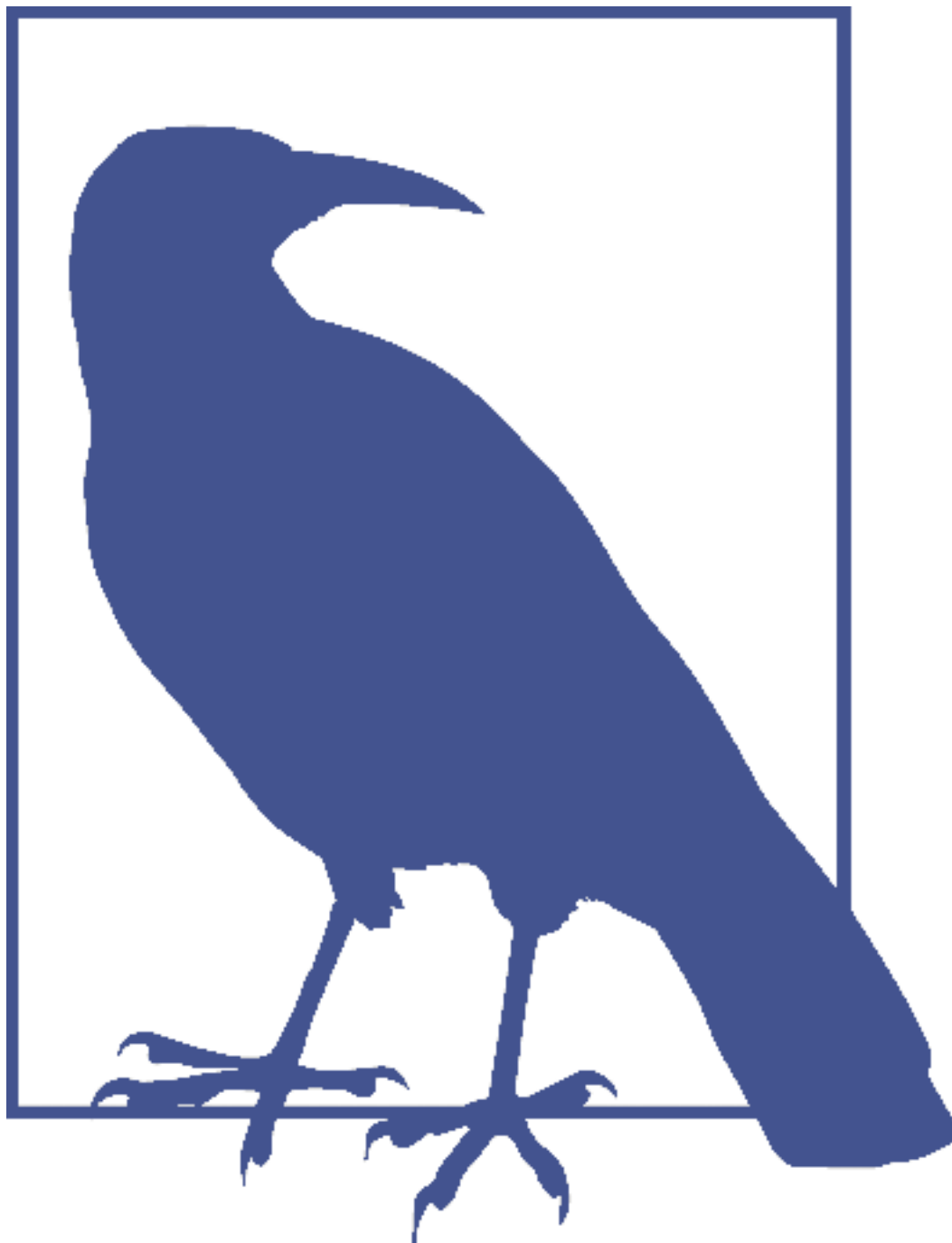
Một kết nối có thể ổn định, nhưng ta không thể kỳ vọng các lời gọi từ xa cũng nhanh như các lời gọi cục bộ. Ta nên đưa ra càng ít giả định về độ trễ (latency) càng tốt và không bao giờ giả định rằng độ trễ bằng không. Để thông điệp của ta tới được máy chủ từ xa, nó phải đi qua một số tầng phần mềm, và một môi trường vật lý như cáp quang hay dây cáp. Tất cả các thao tác này đều không xảy ra tức thời.

Michael Lewis, in his Flash Boys book (Simon and Schuster), tells a story about companies spending millions of dollars to reduce latency by several milliseconds to be able to access stock exchanges faster than the competition. This is a great example of using latency as a competitive advantage, but it's worth mentioning that, according to some other studies, such as [BARTLETT16] , the chance of stale-quote arbitrage (the ability to profit from being able to know prices and execute orders faster than the competition) doesn't give fast traders the ability to exploit markets.

Michael Lewis, trong cuốn sách Flash Boys (Simon and Schuster) của mình, kể một câu chuyện về việc các công ty chi hàng triệu đô-la để giảm độ trễ vài mili-giây nhằm có thể truy cập các sàn giao dịch chứng khoán nhanh hơn đối thủ cạnh tranh. Đây là một ví dụ tuyệt vời về việc dùng độ trễ như một lợi thế cạnh tranh, nhưng cũng đáng nhắc tới rằng, theo một số nghiên cứu khác, chẳng hạn [BARTLETT16], khả năng đầu cơ chênh lệch giá báo cũ (stale-quote arbitrage, tức khả năng kiếm lời từ việc biết giá và thực hiện lệnh nhanh hơn đối thủ) không cho phép các nhà giao dịch tốc độ cao trục lợi từ thị trường.

Learning our lessons, we've added retries, reconnects, and removed the assumptions about instantaneous execution, but this still turns out not to be enough. When increasing the number, rates, and sizes of exchanged messages, or adding new processes to the existing network, we should not assume that bandwidth is infinite .

Rút ra bài học, chúng ta đã thêm cơ chế thử lại, kết nối lại, và loại bỏ giả định về việc thực thi tức thời, nhưng điều này vẫn hóa ra là chưa đủ. Khi tăng số lượng, tần suất và kích thước của các thông điệp trao đổi, hoặc thêm các tiến trình mới vào mạng hiện có, ta không nên giả định rằng băng thông là vô hạn.



In 1994, Peter Deutsch published a now-famous list of assertions, titled “Fallacies of distributed computing,” describing the aspects of distributed computing that are easy to overlook. In addition to network reliability, latency, and bandwidth assumptions, he describes some other problems. For example, network security, the possible presence of adversarial parties, intentional and unintentional topology changes that can break our assumptions about presence and location of specific resources, transport costs in terms of both time and resources, and, finally, the existence of a single authority having knowledge and control over the entire network.

Năm 1994, Peter Deutsch công bố một danh sách các khẳng định nay đã trở nên nổi tiếng, với tựa đề “Fallacies of distributed computing” (Những ngộ nhận của tính toán phân tán),

mô tả những khía cạnh của tính toán phân tán mà người ta dễ bỏ qua. Bên cạnh các giả định về độ tin cậy của mạng, độ trễ và băng thông, ông còn mô tả một số vấn đề khác. Ví dụ, an ninh mạng, sự hiện diện khả dĩ của các bên đối nghịch, những thay đổi tô-pô có chủ ý lẫn vô ý có thể phá vỡ các giả định của ta về sự hiện diện và vị trí của những tài nguyên cụ thể, chi phí truyền vận xét cả về thời gian lẫn tài nguyên, và cuối cùng là sự tồn tại của một thực thể quản lý duy nhất nắm hiểu biết và quyền kiểm soát toàn bộ mạng.

Deutsch's list of distributed computing fallacies is pretty exhaustive, but it focuses on what can go wrong when we send messages from one process to another through the link. These concerns are valid and describe the most general and low-level complications, but unfortunately, there are many other assumptions we make about the distributed systems while designing and implementing them that can cause problems when operating them.

Danh sách các ngộ nhận về tính toán phân tán của Deutsch khá đầy đủ, nhưng nó tập trung vào những gì có thể trục trặc khi ta gửi thông điệp từ tiến trình này sang tiến trình khác qua đường liên kết. Những mối lo này là chính đáng và mô tả các phức tạp tổng quát nhất, ở mức thấp nhất, nhưng không may là còn rất nhiều giả định khác mà ta đặt ra về các hệ phân tán trong khi thiết kế và triển khai chúng, có thể gây ra vấn đề khi vận hành chúng.

Processing — Xử lý

Before a remote process can send a response to the message it just received, it needs to perform some work locally, so we cannot assume that processing is instantaneous. Taking network latency into consideration is not enough, as operations performed by the remote processes aren't immediate, either.

Trước khi một tiến trình từ xa có thể gửi phản hồi cho thông điệp mà nó vừa nhận, nó cần thực hiện một số công việc cục bộ, nên ta không thể giả định rằng việc xử lý là tức thời. Cân nhắc độ trễ mạng thôi là chưa đủ, vì các thao tác do các tiến trình từ xa thực hiện cũng không diễn ra ngay lập tức.

Moreover, there's no guarantee that processing starts as soon as the message is delivered. The message may land in the pending queue on the remote server, and will have to wait there until all the messages that arrived before it are processed.

Hơn nữa, không có gì đảm bảo rằng việc xử lý bắt đầu ngay khi thông điệp được chuyển tới. Thông điệp có thể nằm trong hàng đợi chờ trên máy chủ từ xa, và sẽ phải đợi ở đó cho đến khi tất cả các thông điệp đến trước nó được xử lý xong.

Nodes can be located closer or further from one another, have different CPUs, amounts of RAM, different disks, or be running different software versions and configurations. We cannot expect them to process requests at the same rate. If we have to wait for several remote servers working in parallel to respond to complete the task, the execution as a whole is as slow as the slowest remote server.

Các nút có thể nằm gần hoặc xa nhau, có CPU khác nhau, dung lượng RAM khác nhau, ổ đĩa khác nhau, hoặc đang chạy các phiên bản và cấu hình phần mềm khác nhau. Ta không thể kỳ vọng chúng xử lý yêu cầu với cùng tốc độ. Nếu ta phải chờ nhiều máy chủ từ xa làm việc song song phản hồi để hoàn thành tác vụ, thì việc thực thi xét tổng thể chỉ chậm bằng máy chủ từ xa chậm nhất.

Contrary to the widespread belief, queue capacity is not infinite and piling up more requests won't do the system any good. Backpressure is a strategy that allows us to cope with producers that publish messages at a rate that is faster than the rate at which consumers can process them by slowing

down the producers. Backpressure is one of the least appreciated and applied concepts in distributed systems, often built post hoc instead of being an integral part of the system design.

Trái với niềm tin phổ biến, dung lượng hàng đợi không phải là vô hạn và việc chất chồng thêm nhiều yêu cầu sẽ chẳng có lợi gì cho hệ thống. Áp lực ngược (backpressure) là một chiến lược cho phép ta đối phó với các bên sản xuất phát hành thông điệp ở tốc độ nhanh hơn tốc độ mà các bên tiêu thụ có thể xử lý, bằng cách làm chậm các bên sản xuất lại. Áp lực ngược là một trong những khái niệm ít được trân trọng và áp dụng nhất trong hệ phân tán, thường được xây dựng chắp vá về sau (post hoc) thay vì là một phần không thể tách rời của thiết kế hệ thống.

Even though increasing the queue capacity might sound like a good idea and can help to pipeline, parallelize, and effectively schedule requests, nothing is happening to the messages while they're sitting in the queue and waiting for their turn. Increasing the queue size may negatively impact latency, since changing it has no effect on the processing rate.

Mặc dù việc tăng dung lượng hàng đợi nghe có vẻ là một ý hay và có thể giúp đường ống hóa (pipeline), song song hóa và lập lịch yêu cầu hiệu quả, nhưng không có gì xảy ra với các thông điệp trong khi chúng đang nằm trong hàng đợi và chờ đến lượt. Việc tăng kích thước hàng đợi có thể tác động tiêu cực đến độ trễ, vì thay đổi nó không có ảnh hưởng gì đến tốc độ xử lý.

In general, process-local queues are used to achieve the following goals:

Nhìn chung, các hàng đợi cục bộ trong tiến trình được dùng để đạt được những mục tiêu sau:

Decoupling Receipt and processing are separated in time and happen independently.

Tách rời (Decoupling) Việc tiếp nhận và việc xử lý được tách rời về mặt thời gian và diễn ra độc lập.

Pipelining Requests in different stages are processed by independent parts of the system. The subsystem responsible for receiving messages doesn't have to **block** until the previous message is fully processed.

Đường ống hóa (Pipelining) Các yêu cầu ở những giai đoạn khác nhau được xử lý bởi các bộ phận độc lập của hệ thống. Phân hệ chịu trách nhiệm tiếp nhận thông điệp không phải chặn để chờ cho đến khi thông điệp trước đó được xử lý hoàn toàn.

Absorbing short-time bursts System load tends to vary, but request inter-arrival times are hidden from the component responsible for request processing. Overall system latency increases because of the time spent in the queue, but this is usually still better than responding with a failure and retrying the request.

Hấp thụ các đợt bùng phát ngắn hạn Tải của hệ thống có xu hướng dao động, nhưng khoảng thời gian giữa các lần đến của yêu cầu được che giấu khỏi thành phần chịu trách nhiệm xử lý yêu cầu. Độ trễ tổng thể của hệ thống tăng lên do thời gian nằm trong hàng đợi, nhưng điều này thường vẫn tốt hơn là phản hồi bằng một lỗi rồi thử lại yêu cầu.

Queue size is workload- and application-specific. For relatively stable workloads, we can size queues by measuring task processing times and the average time each task spends in the queue before it is processed, and making sure that latency remains within acceptable bounds while throughput increases. In this case, queue sizes are relatively small. For unpredictable workloads, when tasks get submitted in bursts, queues should be sized to account for bursts and high load as well.

Kích thước hàng đợi phụ thuộc vào khối lượng công việc (workload) và vào ứng dụng cụ thể. Với các khối lượng công việc tương đối ổn định, ta có thể định cỡ hàng đợi bằng cách đo thời gian xử lý tác vụ và thời gian trung bình mà mỗi tác vụ nằm trong hàng đợi trước khi được xử lý, và đảm bảo rằng độ trễ vẫn nằm trong các giới hạn chấp nhận được trong khi thông lượng (throughput) tăng lên. Trong trường hợp này, kích thước hàng đợi tương đối nhỏ. Với các khối lượng công việc khó dự đoán, khi các tác vụ được gửi đến theo từng đợt bùng phát, hàng đợi nên được định cỡ để tính đến cả các đợt bùng phát lẫn tải cao.

The remote server can work through requests quickly, but it doesn't mean that we always get a positive response from it. It can respond with a failure: it couldn't make a write, the searched value was not present, or it could've hit a bug. In summary, even the most favorable scenario still requires some attention from our side.

Máy chủ từ xa có thể xử lý xong các yêu cầu một cách nhanh chóng, nhưng điều đó không có nghĩa là ta luôn nhận được phản hồi tích cực từ nó. Nó có thể phản hồi bằng một lỗi: nó không thực hiện được một thao tác ghi, giá trị được tìm kiếm không tồn tại, hoặc nó có thể đã gặp một lỗi (bug). Tóm lại, ngay cả kịch bản thuận lợi nhất vẫn đòi hỏi một chút lưu tâm từ phía chúng ta.

Clocks and Time — Đồng hồ và thời gian

Time is an illusion. Lunchtime doubly so. —Ford Prefect, *The Hitchhiker's Guide to the Galaxy*

Thời gian là một ảo ảnh. Giờ ăn trưa lại càng ảo ảnh gấp đôi. —Ford Prefect, *The Hitchhiker's Guide to the Galaxy*

Assuming that clocks on remote machines run in sync can also be dangerous. Combined with latency is zero and processing is instantaneous, it leads to different idiosyncrasies, especially in time-series and real-time data processing. For example, when collecting and aggregating data from participants with a different perception of time, you should understand time drifts between them and normalize times accordingly, rather than relying on the source timestamp. Unless you use specialized high-precision time sources, you should not rely on timestamps for synchronization or ordering. Of course this doesn't mean we cannot or should not rely on time at all: in the end, any **synchronous system** uses local clocks for timeouts.

Giả định rằng đồng hồ trên các máy ở xa chạy đồng bộ với nhau cũng có thể nguy hiểm. Kết hợp với các giả định độ trễ bằng không và xử lý tức thời, nó dẫn đến những hành vi kỳ quặc khác nhau, đặc biệt là trong xử lý dữ liệu chuỗi thời gian (time-series) và dữ liệu thời gian thực. Ví dụ, khi thu thập và tổng hợp dữ liệu từ các bên tham gia có cảm nhận về thời gian khác nhau, bạn nên hiểu các độ lệch thời gian (time drift) giữa chúng và chuẩn hóa thời gian cho phù hợp, thay vì dựa vào dấu thời gian (timestamp) của nguồn. Trừ khi bạn dùng các nguồn thời gian chuyên dụng có độ chính xác cao, bạn không nên dựa vào dấu thời gian để đồng bộ hóa hay sắp thứ tự. Tất nhiên điều này không có nghĩa là ta không thể hoặc không nên dựa vào thời gian chút nào: rốt cuộc, bất kỳ hệ đồng bộ nào cũng dùng đồng hồ cục bộ cho thời gian chờ.

It's essential to always account for the possible time differences between the processes and the time required for the messages to get delivered and processed. For example, **Spanner** (see “Distributed Transactions with Spanner” on **page** 268) uses a special time API that returns a timestamp and uncertainty bounds to impose a strict transaction order. Some failure-detection algorithms rely on a shared notion of time and a guarantee that the clock drift is always within allowed bounds for correctness [GUPTA01] .

Việc luôn tính đến những khác biệt thời gian khả dĩ giữa các tiến trình cũng như thời gian cần thiết để các thông điệp được chuyển tới và xử lý là điều thiết yếu. Ví dụ, Spanner (xem “Distributed Transactions with Spanner” ở trang 268) dùng một API thời gian đặc biệt trả về một dấu thời gian cùng các biên độ bất định để áp đặt một thứ tự giao dịch nghiêm ngặt. Một số thuật toán phát hiện lỗi dựa vào một khái niệm thời gian dùng chung và một đảm bảo rằng độ lệch đồng hồ (clock drift) luôn nằm trong các giới hạn cho phép để đảm bảo tính đúng đắn [GUPTA01].

Besides the fact that clock synchronization in a distributed system is hard, the current time is constantly changing: you can request a current POSIX timestamp from the operating system, and request another current timestamp after executing several steps, and the two will be different. This is a rather obvious observation, but understanding both a source of time and which exact moment the timestamp captures is crucial.

Bên cạnh việc đồng bộ hóa đồng hồ trong một hệ phân tán là khó, thời gian hiện tại còn liên tục thay đổi: bạn có thể yêu cầu một dấu thời gian POSIX hiện tại từ hệ điều hành, rồi yêu cầu một dấu thời gian hiện tại khác sau khi thực thi vài bước, và hai giá trị sẽ khác nhau. Đây là một quan sát khá hiển nhiên, nhưng việc hiểu cả nguồn thời gian lẫn việc dấu thời gian nắm bắt đúng khoảnh khắc nào là điều then chốt.

Understanding whether the clock source is monotonic (i.e., that it won't ever go backward) and how much the scheduled time-related operations might drift can be helpful, too.

Hiểu được liệu nguồn đồng hồ có đơn điệu (monotonic) hay không (tức là nó sẽ không bao giờ đi lùi) và việc các thao tác liên quan đến thời gian được lập lịch có thể lệch đi bao nhiêu cũng có thể hữu ích.

State Consistency — Tính nhất quán của trạng thái

Most of the previous assumptions fall into the almost always false category, but there are some that are better described as not always true : when it's easy to take a mental shortcut and simplify the model by thinking of it a specific way, ignoring some tricky edge cases.

Hầu hết các giả định trước đó rơi vào loại gần như luôn sai, nhưng có một số được mô tả chính xác hơn là không phải lúc nào cũng đúng: khi ta dễ đi đường tắt trong tư duy và đơn giản hóa mô hình bằng cách nghĩ về nó theo một cách cụ thể, bỏ qua một số trường hợp biên học búa.

Distributed algorithms do not always guarantee strict state consistency. Some approaches have looser constraints and allow state divergence between replicas, and rely on conflict resolution (an ability to detect and resolve diverged states within the system) and read-time data repair (bringing replicas back in sync during reads in cases where they respond with different results). You can find more information about these concepts in Chapter 12 . Assuming that the state is fully consistent across the nodes may lead to subtle bugs.

Các thuật toán phân tán không phải lúc nào cũng đảm bảo tính nhất quán nghiêm ngặt của trạng thái. Một số cách tiếp cận có ràng buộc lỏng lẻo hơn và cho phép trạng thái giữa các bản sao (replica) phân kỳ, và dựa vào cơ chế giải quyết xung đột (conflict resolution, khả năng phát hiện và giải quyết các trạng thái đã phân kỳ trong hệ thống) cùng cơ chế sửa dữ liệu lúc đọc (read-time data repair, đưa các bản sao trở lại đồng bộ trong lúc đọc, trong những trường hợp chúng phản hồi với các kết quả khác nhau). Bạn có thể tìm thêm thông tin về các khái niệm này trong Chương 12. Giả định rằng trạng thái là hoàn toàn nhất quán trên khắp các nút có thể dẫn đến những lỗi tinh vi.

An eventually consistent distributed database system might have the logic to handle replica disagreement by querying a **quorum** of nodes during reads, but assume that the database schema and the view of the cluster are strongly consistent. Unless we enforce consistency of this information, relying on that assumption may have severe consequences.

Một hệ cơ sở dữ liệu phân tán nhất quán cuối cùng (eventually consistent) có thể có logic để xử lý sự bất đồng giữa các bản sao bằng cách truy vấn một quorum các nút trong lúc đọc, nhưng lại giả định rằng lược đồ (schema) cơ sở dữ liệu và góc nhìn về cụm (cluster) là nhất quán mạnh. Trừ khi ta cưỡng bức tính nhất quán cho thông tin này, việc dựa vào giả định đó có thể gây ra những hậu quả nghiêm trọng.

For example, there was a bug in Apache Cassandra, caused by the fact that schema changes propagate to servers at different times. If you tried to read from the database while the schema was propagating, there was a chance of corruption, since one server encoded results assuming one schema and the other one decoded them using a different schema.

Ví dụ, từng có một lỗi trong Apache Cassandra, do thực tế là các thay đổi lược đồ lan truyền tới các máy chủ ở những thời điểm khác nhau. Nếu bạn cố đọc từ cơ sở dữ liệu trong lúc lược đồ đang lan truyền, có khả năng xảy ra hỏng dữ liệu, vì một máy chủ mã hóa kết quả theo giả định một lược đồ còn máy chủ kia giải mã chúng bằng một lược đồ khác.

Another example is a bug caused by the divergent view of the ring: if one of the nodes assumes that the other node holds data records for a key, but this other node has a different view of the cluster, reading or writing the data can result in misplacing data records or getting an empty response while data records are in fact happily present on the other node.

Một ví dụ khác là một lỗi gây ra bởi góc nhìn phân kỳ về vòng (ring): nếu một trong các nút giả định rằng nút kia nắm giữ các bản ghi dữ liệu cho một khóa, nhưng nút kia này lại có góc nhìn khác về cụm, thì việc đọc hoặc ghi dữ liệu có thể dẫn đến đặt nhầm chỗ các bản ghi dữ liệu hoặc nhận được phản hồi rỗng trong khi các bản ghi dữ liệu thực ra vẫn đang nằm yên ổn trên nút kia.

It is better to think about the possible problems in advance, even if a complete solution is costly to implement. By understanding and handling these cases, you can embed safeguards or change the design in a way that makes the solution more natural.

Tốt hơn là nên nghĩ trước về các vấn đề khả dĩ, ngay cả khi một giải pháp trọn vẹn tốn kém để triển khai. Bằng cách hiểu và xử lý các trường hợp này, bạn có thể nhúng vào các biện pháp bảo vệ hoặc thay đổi thiết kế theo cách làm cho giải pháp trở nên tự nhiên hơn.

Local and Remote Execution — Thực thi cục bộ và từ xa

Hiding complexity behind an API might be dangerous. For example, if you have an iterator over the local dataset, you can reasonably predict what's going on behind the scenes, even if the **storage engine** is unfamiliar. Understanding the process of iteration over the remote dataset is an entirely different problem: you need to understand consistency and delivery semantics, data reconciliation, paging, merges, concurrent access implications, and many other things.

Việc che giấu sự phức tạp đằng sau một API có thể nguy hiểm. Ví dụ, nếu bạn có một bộ lặp (iterator) trên tập dữ liệu cục bộ, bạn có thể dự đoán hợp lý điều gì đang diễn ra ở hậu trường, ngay cả khi bộ máy lưu trữ (storage engine) là lạ lẫm. Hiểu được quá trình lặp trên một tập dữ liệu từ xa lại là một vấn đề hoàn toàn khác: bạn cần hiểu ngữ nghĩa nhất quán và phân phối, sự đối soát dữ liệu, phân trang, các thao tác gộp, các hệ quả của truy cập tương tranh, và nhiều thứ khác.

Simply hiding both behind the same interface, however useful, might be misleading. Additional API parameters may be necessary for debugging, configuration, and observability. We should always keep in mind that local and remote execution are not the same [WALDO96] .

Tuy nhiên, việc đơn thuần che giấu cả hai đằng sau cùng một giao diện, dù hữu ích đến đâu, có thể gây hiểu lầm. Có thể cần thêm các tham số API để gỡ lỗi, cấu hình và khả năng quan sát. Ta nên luôn ghi nhớ rằng thực thi cục bộ và thực thi từ xa không giống nhau [WALDO96].

The most apparent problem with hiding remote calls is latency: remote invocation is many times more costly than the local one, since it involves two-way network transport, serialization/deserialization, and many other steps. Interleaving local and blocking remote calls may lead to performance degradation and unintended side effects [VINOSKI08] .

Vấn đề rõ ràng nhất của việc che giấu các lời gọi từ xa là độ trễ: lời gọi từ xa tốn kém hơn lời gọi cục bộ nhiều lần, vì nó liên quan đến việc truyền vận mạng hai chiều, tuần tự hóa/giải tuần tự hóa (serialization/deserialization) và nhiều bước khác. Việc xen kẽ các lời gọi cục bộ và các lời gọi từ xa có tính chặn (blocking) có thể dẫn đến suy giảm hiệu năng và các tác dụng phụ ngoài ý muốn [VINOSKI08].

Need to Handle Failures — Cần phải xử lý lỗi

It's OK to start working on a system assuming that all nodes are up and functioning normally, but thinking this is the case all the time is dangerous. In a long-running system, nodes can be taken down for maintenance (which usually involves a graceful shutdown) or crash for various reasons: software problems, out-of-memory killer [KERRISK10] , runtime bugs, hardware issues, etc. Processes do fail, and the best thing you can do is be prepared for failures and understand how to handle them.

Bắt đầu làm việc trên một hệ thống với giả định rằng tất cả các nút đều đang chạy và hoạt động bình thường thì không sao, nhưng nghĩ rằng tình trạng này lúc nào cũng đúng là điều nguy hiểm. Trong một hệ thống chạy lâu dài, các nút có thể bị hạ xuống để bảo trì (thường liên quan đến việc tắt máy nhẹ nhàng) hoặc sập vì nhiều lý do: các vấn đề phần mềm, bộ diệt tiến trình khi hết bộ nhớ (out-of-memory killer) [KERRISK10], lỗi lúc chạy (runtime bug), vấn đề phần cứng, v.v. Các tiến trình đúng là có sập, và điều tốt nhất bạn có thể làm là chuẩn bị sẵn sàng cho lỗi và hiểu cách xử lý chúng.

If the remote server doesn't respond, we do not always know the exact reason for it. It could be caused by the crash, a network failure, the remote process, or the link to it being slow. Some distributed algorithms use **heartbeat** protocols and failure detectors to form a hypothesis about which participants are alive and reachable.

Nếu máy chủ từ xa không phản hồi, ta không phải lúc nào cũng biết lý do chính xác của việc đó. Nó có thể do sự cố sập, do lỗi mạng, do tiến trình từ xa, hoặc do đường liên kết tới nó bị chậm. Một số thuật toán phân tán dùng các giao thức nhịp tim (heartbeat) và các bộ phát hiện lỗi (failure detector) để hình thành giả thuyết về việc bên tham gia nào còn sống và còn tiếp cận được.

Network Partitions and Partial Failures — Phân vùng mạng và lỗi cục bộ

When two or more servers cannot communicate with each other, we call the situation **network partition**. In “Perspectives on the CAP Theorem” [GILBERT12], Seth Gilbert and Nancy Lynch draw a distinction between the case when two participants cannot communicate with each other and when several groups of participants are isolated from one another, cannot exchange messages, and proceed with the algorithm.

Khi hai hay nhiều máy chủ không thể giao tiếp với nhau, ta gọi tình huống đó là phân vùng mạng (network partition). Trong “Perspectives on the CAP Theorem” [GILBERT12], Seth Gilbert và Nancy Lynch phân biệt giữa trường hợp hai bên tham gia không thể giao tiếp với nhau và trường hợp nhiều nhóm bên tham gia bị cô lập khỏi nhau, không thể trao đổi thông điệp, và vẫn tiếp tục chạy thuật toán.

General unreliability of the network (packet loss, retransmission, latencies that are hard to predict) are annoying but tolerable, while network partitions can cause much more trouble, since independent groups can proceed with execution and produce conflicting results. Network links can also fail asymmetrically: messages can still be getting delivered from one process to the other one, but not vice versa.

Sự không tin cậy chung của mạng (mất gói, truyền lại, độ trễ khó dự đoán) thì gây phiền toái nhưng vẫn chịu đựng được, trong khi phân vùng mạng có thể gây rắc rối nhiều hơn, vì các nhóm độc lập có thể tiếp tục thực thi và tạo ra những kết quả mâu thuẫn. Các đường liên kết mạng cũng có thể hỏng một cách bất đối xứng: thông điệp vẫn có thể được chuyển từ tiến trình này sang tiến trình kia, nhưng không theo chiều ngược lại.

To build a system that is robust in the presence of failure of one or multiple processes, we have to consider cases of partial failures [TANENBAUM06] and how the system can continue operating even though a part of it is unavailable or functioning incorrectly.

Để xây dựng một hệ thống vững vàng khi có lỗi của một hoặc nhiều tiến trình, ta phải cân nhắc các trường hợp lỗi cục bộ (partial failure) [TANENBAUM06] và việc hệ thống có thể tiếp tục vận hành như thế nào ngay cả khi một phần của nó không khả dụng hoặc hoạt động không đúng đắn.

Failures are hard to detect and aren’t always visible in the same way from different parts of the system. When designing highly available systems, one should always think about edge cases: what if we did replicate the data, but received no acknowledgments? Do we need to retry? Is the data still going to be available for reads on the nodes that have sent acknowledgments?

Lỗi rất khó phát hiện và không phải lúc nào cũng hiển thị theo cùng một cách từ những phần khác nhau của hệ thống. Khi thiết kế các hệ thống có tính sẵn sàng cao, ta nên luôn nghĩ về các trường hợp biên: nếu ta đã nhân bản dữ liệu, nhưng không nhận được xác nhận nào thì sao? Ta có cần thử lại không? Dữ liệu có còn khả dụng cho việc đọc trên các nút đã gửi xác nhận hay không?

Murphy’s Law 2 tells us that the failures do happen. Programming folklore adds that the failures will happen in the worst way possible, so our job as distributed systems engineers is to make sure we reduce the number of scenarios where things go wrong and prepare for failures in a way that contains the damage they can cause.

Định luật Murphy 2 cho ta biết rằng lỗi đúng là có xảy ra. Truyền thuyết dân gian của giới lập trình bổ sung thêm rằng lỗi sẽ xảy ra theo cách tồi tệ nhất có thể, nên công việc của

chúng ta với tư cách là kỹ sư hệ phân tán là đảm bảo giảm bớt số lượng các kịch bản mà mọi thứ trục trặc và chuẩn bị sẵn cho lỗi theo cách kiểm chế được thiệt hại mà chúng có thể gây ra.

It's impossible to prevent all failures, but we can still build a resilient system that functions correctly in their presence. The best way to design for failures is to test for them. It's close to impossible to think through every possible failure scenario and predict the behaviors of multiple processes. Setting up testing harnesses that create partitions,

Không thể ngăn chặn mọi lỗi, nhưng ta vẫn có thể xây dựng một hệ thống có khả năng chống chịu (resilient), vận hành đúng đắn ngay cả khi có lỗi. Cách tốt nhất để thiết kế cho lỗi là kiểm thử cho lỗi. Gần như không thể nghĩ thấu đáo mọi kịch bản lỗi khả dĩ và dự đoán hành vi của nhiều tiến trình. Việc thiết lập các giàn kiểm thử (testing harness) tạo ra các phân vùng,

2 Murphy's Law is an adage that can be summarized as "Anything that can go wrong, will go wrong," which was popularized and is often used as an idiom in popular culture.

2 Định luật Murphy là một câu châm ngôn có thể tóm tắt là "Bất cứ điều gì có thể trục trặc thì sẽ trục trặc," vốn đã được phổ biến rộng rãi và thường được dùng như một thành ngữ trong văn hóa đại chúng.

simulate bit rot [GRAY05] , increase latencies, diverge clocks, and magnify relative processing speeds is the best way to go about it. Real-world distributed system setups can be quite adversarial, unfriendly, and "creative" (however, in a very hostile way), so the testing effort should attempt to cover as many scenarios as possible.

mô phỏng sự mục rữa bit (bit rot) [GRAY05], tăng độ trễ, làm lệch đồng hồ và phóng đại tốc độ xử lý tương đối là cách tốt nhất để tiến hành việc này. Các thiết lập hệ phân tán trong thế giới thực có thể khá đối nghịch, thù địch và "sáng tạo" (theo một cách rất ác ý), nên nỗ lực kiểm thử nên cố gắng bao phủ càng nhiều kịch bản càng tốt.



Over the last few years, we've seen a few open source projects that help to recreate different failure scenarios. Toxiproxy can help to simulate network problems: limit the bandwidth, introduce latency, timeouts, and more. Chaos Monkey takes a more radical approach and exposes engineers to production failures by randomly shutting down services. CharybdeFS helps to simulate filesystem and hardware errors and failures. You can use these tools to test your software and make sure it

behaves correctly in the presence of these failures. CrashMonkey , a filesystem agnostic record-replay-and-test framework, helps test data and metadata consistency for persistent files.

Trong vài năm qua, chúng ta đã thấy một vài dự án mã nguồn mở giúp tái tạo các kịch bản lỗi khác nhau. Toxiproxy có thể giúp mô phỏng các vấn đề mạng: giới hạn băng thông, đưa vào độ trễ, thời gian chờ, và hơn thế nữa. Chaos Monkey áp dụng một cách tiếp cận triệt để hơn và phơi bày các kỹ sư trước những lỗi sản xuất bằng cách tắt ngẫu nhiên các dịch vụ. CharybdeFS giúp mô phỏng các lỗi và sự cố của hệ thống tệp và phần cứng. Bạn có thể dùng các công cụ này để kiểm thử phần mềm của mình và đảm bảo nó hành xử đúng đắn khi có những lỗi này. CrashMonkey, một khung ghi-phát lại-và-kiểm thử (record-replay-and-test) không phụ thuộc hệ thống tệp, giúp kiểm thử tính nhất quán của dữ liệu và siêu dữ liệu (metadata) cho các tệp bền vững.

When working with distributed systems, we have to take fault tolerance, resilience, possible failure scenarios, and edge cases seriously. Similar to “given enough eyeballs, all bugs are shallow,” we can say that a large enough cluster will eventually hit every possible issue. At the same time, given enough testing, we will be able to eventually find every existing problem.

Khi làm việc với hệ phân tán, ta phải xem xét nghiêm túc khả năng dung lỗi, khả năng chống chịu, các kịch bản lỗi khả dĩ và các trường hợp biên. Tương tự như câu “đủ nhiều cặp mắt thì mọi lỗi đều nông cạn,” ta có thể nói rằng một cụm đủ lớn rất cuộc sẽ gặp mọi vấn đề khả dĩ. Đồng thời, với đủ nhiều kiểm thử, ta rất cuộc sẽ có thể tìm ra mọi vấn đề đang tồn tại.

Cascading Failures — Lỗi dây chuyền

We cannot always wholly isolate failures: a process tipping over under a high load increases the load for the rest of cluster, making it even more probable for the other nodes to fail. Cascading failures can propagate from one part of the system to the other, increasing the scope of the problem.

Ta không phải lúc nào cũng có thể cô lập hoàn toàn các lỗi: một tiến trình gục ngã dưới tải cao làm tăng tải cho phần còn lại của cụm, khiến các nút khác càng có khả năng hỏng. Lỗi dây chuyền (cascading failure) có thể lan truyền từ phần này của hệ thống sang phần khác, làm tăng phạm vi của vấn đề.

Sometimes, cascading failures can even be initiated by perfectly good intentions. For example, a node was offline for a while and did not receive the most recent updates. After it comes back online, helpful peers would like to help it to catch up with recent happenings and start streaming the data it’s missing over to it, exhausting network resources or causing the node to fail shortly after the startup.

Đôi khi, lỗi dây chuyền thậm chí có thể được khởi phát bởi những ý định hoàn toàn tốt đẹp. Ví dụ, một nút đã ngoại tuyến một thời gian và không nhận được các cập nhật gần đây nhất. Sau khi nó trở lại trực tuyến, các nút lân cận tốt bụng muốn giúp nó bắt kịp những diễn biến gần đây và bắt đầu phát luồng dữ liệu mà nó đang thiếu sang cho nó, làm cạn kiệt tài nguyên mạng hoặc khiến nút đó hỏng ngay sau khi khởi động.



To protect a system from propagating failures and treat failure scenarios gracefully, circuit breakers can be used. In electrical engineering, circuit breakers protect expensive and hard-to-replace parts from overload or short circuit by interrupting the current flow. In software development, circuit breakers monitor failures and allow fallback mechanisms that can protect the system by steering away from the failing service, giving it some time to recover, and handling failing calls gracefully.

Để bảo vệ một hệ thống khỏi việc lan truyền lỗi và xử lý các kịch bản lỗi một cách nhẹ nhàng, ta có thể dùng cầu dao ngắt mạch (circuit breaker). Trong kỹ thuật điện, cầu dao ngắt mạch bảo vệ những bộ phận đắt tiền và khó thay thế khỏi quá tải hoặc đoản mạch bằng cách ngắt dòng điện. Trong phát triển phần mềm, cầu dao ngắt mạch giám sát các lỗi và cho phép các cơ chế dự phòng có thể bảo vệ hệ thống bằng cách lèo lái tránh xa dịch vụ đang hỏng, cho nó một chút thời gian để phục hồi, và xử lý các lời gọi thất bại một cách nhẹ nhàng.

When the connection to one of the servers fails or the server does not respond, the client starts a reconnection loop. By that point, an overloaded server already has a hard time catching up with new connection requests, and client-side retries in a tight loop don't help the situation. To avoid that, we can use a backoff strategy. Instead of retrying immediately, clients wait for some time. Backoff can help us to avoid amplifying problems by scheduling retries and increasing the time window between subsequent requests.

Khi kết nối tới một trong các máy chủ thất bại hoặc máy chủ không phản hồi, máy khách bắt đầu một vòng lặp kết nối lại. Đến thời điểm đó, một máy chủ đã quá tải vốn đã chật vật bắt kịp các yêu cầu kết nối mới, và việc máy khách thử lại trong một vòng lặp chặt chẽ chẳng giúp ích gì cho tình hình. Để tránh điều đó, ta có thể dùng một chiến lược lui lại (backoff). Thay vì thử lại ngay lập tức, các máy khách chờ một khoảng thời gian. Lui lại có thể giúp ta tránh khuếch đại các vấn đề bằng cách lập lịch cho các lần thử lại và tăng khoảng thời gian giữa các yêu cầu liên tiếp.

Backoff is used to increase time periods between requests from a single client. However, different clients using the same backoff strategy can produce substantial load as well. To prevent different clients from retrying all at once after the backoff period, we can introduce jitter . Jitter adds small random time periods to backoff and reduces the probability of clients waking up and retrying at the same time.

Lui lại được dùng để tăng các khoảng thời gian giữa các yêu cầu từ một máy khách đơn lẻ. Tuy nhiên, các máy khách khác nhau cùng dùng một chiến lược lui lại cũng có thể tạo ra một tải đáng kể. Để ngăn các máy khách khác nhau cùng thử lại một lúc sau khoảng thời gian lui lại, ta có thể đưa vào độ dao động (jitter). Độ dao động thêm vào các khoảng thời gian ngẫu nhiên nhỏ cho việc lui lại và làm giảm xác suất các máy khách thức dậy và thử lại cùng một lúc.

Hardware failures, bit rot, and software errors can result in corruption that can propagate through standard delivery mechanisms. For example, corrupted data records can get replicated to the other nodes if they are not validated. Without validation mechanisms in place, a system can propagate corrupted data to the other nodes, potentially overwriting noncorrupted data records. To avoid that, we should use **checksumming** and validation to verify the integrity of any content exchanged between the nodes.

Các lỗi phần cứng, sự mục rữa bit và các lỗi phần mềm có thể dẫn đến hỏng dữ liệu mà có thể lan truyền qua các cơ chế phân phối tiêu chuẩn. Ví dụ, các bản ghi dữ liệu bị hỏng có thể bị nhân bản sang các nút khác nếu chúng không được kiểm chứng. Khi không có sẵn các cơ chế kiểm chứng, một hệ thống có thể lan truyền dữ liệu hỏng sang các nút khác, có khả năng ghi đè lên các bản ghi dữ liệu không bị hỏng. Để tránh điều đó, ta nên dùng cơ chế tổng kiểm (checksum) và kiểm chứng để xác minh tính toàn vẹn của bất kỳ nội dung nào được trao đổi giữa các nút.

Overload and hotspotting can be avoided by planning and coordinating execution. Instead of letting peers execute operation steps independently, we can use a **coordinator** that prepares an **execution plan** based on the available resources and predicts the load based on the past execution

data available to it.

Tình trạng quá tải và điểm nóng (hotspotting) có thể được tránh bằng cách lập kế hoạch và điều phối việc thực thi. Thay vì để các nút lân cận thực thi các bước thao tác một cách độc lập, ta có thể dùng một bên điều phối (coordinator) chuẩn bị một kế hoạch thực thi dựa trên các tài nguyên sẵn có và dự đoán tải dựa trên dữ liệu thực thi trong quá khứ mà nó có được.

In summary, we should always consider cases in which failures in one part of the system can cause problems elsewhere. We should equip our systems with circuit breakers, backoff, validation, and coordination mechanisms. Handling small isolated problems is always more straightforward than trying to recover from a large outage.

Tóm lại, ta nên luôn xem xét các trường hợp mà lỗi ở một phần của hệ thống có thể gây ra vấn đề ở nơi khác. Ta nên trang bị cho hệ thống của mình các cầu dao ngắt mạch, cơ chế lui lại, kiểm chứng và điều phối. Xử lý các vấn đề nhỏ, biệt lập luôn dễ hơn việc cố gắng phục hồi từ một sự cố ngừng hoạt động trên diện rộng.

We've just spent an entire section discussing problems and potential failure scenarios in distributed systems, but we should see this as a warning and not as something that should scare us away.

Chúng ta vừa dành cả một mục để bàn về các vấn đề và các kịch bản lỗi khả dĩ trong hệ phân tán, nhưng ta nên xem đây như một lời cảnh báo chứ không phải như một thứ gì đó nên làm ta sợ mà bỏ chạy.

Understanding what can go wrong, and carefully designing and testing our systems makes them more robust and resilient. Being aware of these issues can help you to identify and find potential sources of problems during development, as well as debug them in production.

Hiểu được điều gì có thể trục trặc, đồng thời thiết kế và kiểm thử các hệ thống của ta một cách cẩn thận, sẽ làm cho chúng vững chắc và có khả năng chống chịu hơn. Việc nhận thức được những vấn đề này có thể giúp bạn nhận diện và tìm ra những nguồn gốc tiềm tàng của vấn đề trong quá trình phát triển, cũng như gỡ lỗi chúng trong môi trường sản xuất.

Distributed Systems Abstractions — Các trừu tượng hóa của hệ phân tán

When talking about programming languages, we use common terminology and define our programs in terms of functions, operators, classes, variables, and pointers.

Khi nói về các ngôn ngữ lập trình, ta dùng thuật ngữ chung và định nghĩa các chương trình của mình theo phương diện hàm, toán tử, lớp, biến và con trỏ.

Having a common vocabulary helps us to avoid inventing new words every time we describe anything. The more precise and less ambiguous our definitions are, the easier it is for our listeners to understand us.

Việc có một bộ từ vựng chung giúp ta tránh phải phát minh từ mới mỗi khi mô tả bất cứ điều gì. Các định nghĩa của ta càng chính xác và càng ít mơ hồ thì người nghe càng dễ hiểu ta.

Before we move to algorithms, we first have to cover the distributed systems vocabulary: definitions you'll frequently encounter in talks, books, and papers.

Trước khi chuyển sang các thuật toán, trước hết ta phải bao quát bộ từ vựng của hệ phân tán: các định nghĩa mà bạn sẽ thường xuyên gặp trong các bài nói chuyện, sách vở và các bài báo.

Links — Các kênh liên kết

Networks are not reliable: messages can get lost, delayed, and reordered. Now, with this thought in our minds, we will try to build several communication protocols. We'll start with the least reliable and robust ones, identifying the states they can be in, and figuring out the possible additions to the protocol that can provide better guarantees.

Mạng không đáng tin cậy: thông điệp có thể bị mất, bị trễ và bị sắp xếp lại thứ tự. Giờ đây, với ý nghĩ này trong đầu, ta sẽ thử xây dựng vài giao thức giao tiếp. Ta sẽ bắt đầu với những giao thức kém tin cậy và kém vững chắc nhất, nhận diện các trạng thái mà chúng có thể rơi vào, và tìm ra những bổ sung khả dĩ cho giao thức có thể đem lại các đảm bảo tốt hơn.

Fair-loss link

Kênh fair-loss

We can start with two processes, connected with a link. Processes can send messages to each other, as shown in Figure 8-2. Any communication medium is imperfect, and messages can get lost or delayed.

Ta có thể bắt đầu với hai tiến trình, được nối với nhau bằng một kênh liên kết (link). Các tiến trình có thể gửi thông điệp cho nhau, như Hình 8-2 cho thấy. Mọi môi trường giao tiếp đều không hoàn hảo, và thông điệp có thể bị mất hoặc bị trễ.

Let's see what kind of guarantees we can get. After the message M is sent, from the senders' perspective, it can be in one of the following states:

Hãy xem ta có thể nhận được loại đảm bảo nào. Sau khi thông điệp M được gửi đi, từ góc nhìn của bên gửi, nó có thể ở một trong các trạng thái sau:

- Not yet delivered to process B (but will be, at some point in time)
- Irrecoverably lost during transport
- Successfully delivered to the remote process
 - Chưa được chuyển tới tiến trình B (nhưng sẽ được chuyển tới, vào một thời điểm nào đó)
 - Bị mất không thể khôi phục trong khi truyền vận
 - Được chuyển thành công tới tiến trình từ xa

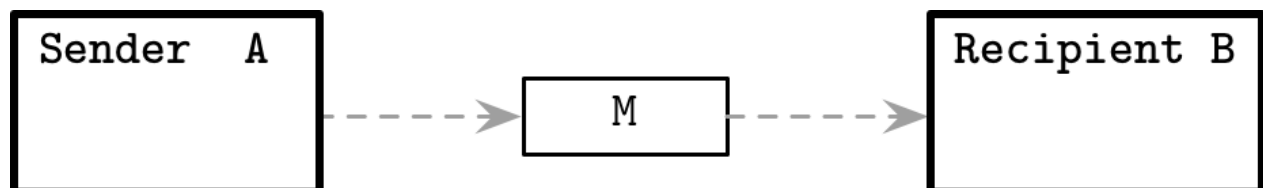


Figure 8-2. Simplest, unreliable form of communication

Hình 8-2. Dạng giao tiếp đơn giản nhất, không đáng tin cậy

Notice that the sender does not have any way to find out if the message is already delivered. In distributed systems terminology, this kind of link is called fair-loss . The properties of this kind of link are:

Lưu ý rằng bên gửi không có cách nào để biết được liệu thông điệp đã được chuyển tới hay chưa. Theo thuật ngữ hệ phân tán, loại kênh liên kết này được gọi là fair-loss. Các đặc tính của loại kênh này là:

Fair loss If both sender and recipient are correct and the sender keeps retransmitting the message infinitely many times, it will eventually be delivered. 3

Mất công bằng (Fair loss) Nếu cả bên gửi lẫn bên nhận đều đúng đắn và bên gửi cứ truyền lại thông điệp vô số lần, thì rốt cuộc nó sẽ được chuyển tới. 3

Finite duplication Sent messages won't be delivered infinitely many times.

Trùng lặp hữu hạn (Finite duplication) Các thông điệp được gửi sẽ không được chuyển tới vô số lần.

No creation A link will not come up with messages; in other words, it won't deliver the message that was never sent.

Không tạo mới (No creation) Một kênh liên kết sẽ không tự nghĩ ra thông điệp; nói cách khác, nó sẽ không chuyển tới một thông điệp chưa bao giờ được gửi đi.

A fair-loss link is a useful abstraction and a first building block for communication protocols with strong guarantees. We can assume that this link is not losing messages between communicating parties systematically and doesn't create new messages. But, at the same time, we cannot entirely rely on it. This might remind you of the User Datagram Protocol (UDP) , which allows us to send messages from one process to the other, but does not have reliable delivery semantics on the protocol level.

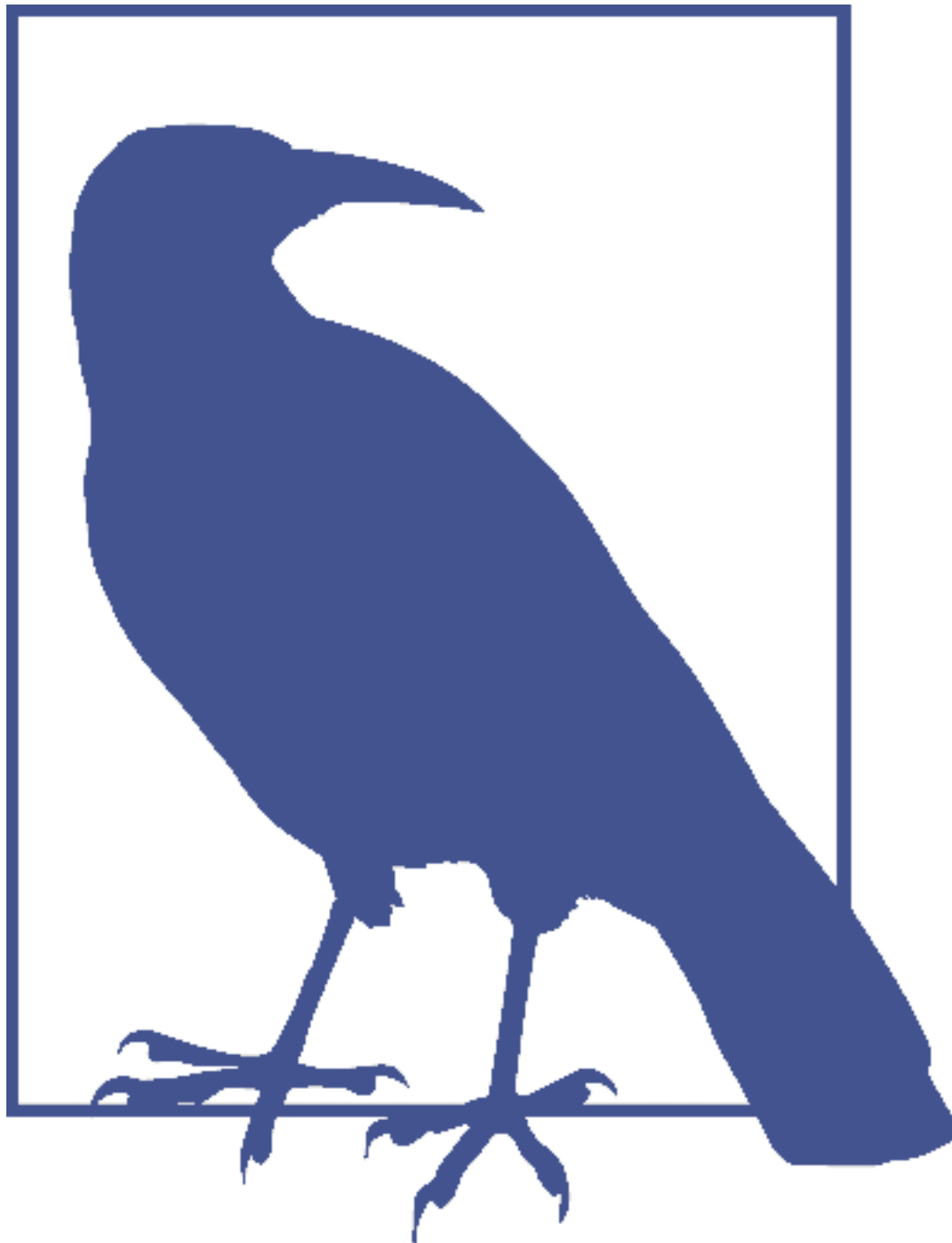
Kênh fair-loss là một trừu tượng hóa hữu ích và là viên gạch xây dựng đầu tiên cho các giao thức giao tiếp có những đảm bảo mạnh. Ta có thể giả định rằng kênh này không làm mất thông điệp giữa các bên giao tiếp một cách có hệ thống và không tạo ra thông điệp mới. Nhưng đồng thời, ta không thể hoàn toàn dựa vào nó. Điều này có thể làm bạn liên tưởng đến Giao thức Datagram Người dùng (UDP), vốn cho phép ta gửi thông điệp từ tiến trình này sang tiến trình khác, nhưng không có ngữ nghĩa chuyển tin đáng tin cậy ở mức giao thức.

Message acknowledgments

Xác nhận thông điệp

To improve the situation and get more clarity in terms of message status, we can introduce acknowledgments : a way for the recipient to notify the sender that it has received the message. For that, we need to use bidirectional communication channels and add some means that allow us to distinguish differences between the messages; for example, sequence numbers , which are unique monotonically increasing message identifiers.

Để cải thiện tình hình và có được sự rõ ràng hơn về trạng thái thông điệp, ta có thể đưa vào các xác nhận (acknowledgment): một cách để bên nhận thông báo cho bên gửi rằng nó đã nhận được thông điệp. Để làm vậy, ta cần dùng các kênh giao tiếp hai chiều và thêm một số phương tiện cho phép ta phân biệt sự khác nhau giữa các thông điệp; ví dụ, số thứ tự (sequence number), tức các định danh thông điệp tăng đơn điệu và duy nhất.



It is enough to have a unique identifier for every message. Sequence numbers are just a particular case of a unique identifier, where we achieve uniqueness by drawing identifiers from a counter. When using hash algorithms to identify messages uniquely, we should account for possible collisions and make sure we can still disambiguate messages.

Chỉ cần có một định danh duy nhất cho mỗi thông điệp là đủ. Số thứ tự chỉ là một trường hợp đặc biệt của định danh duy nhất, trong đó ta đạt được tính duy nhất bằng cách lấy các định danh ra từ một bộ đếm. Khi dùng các thuật toán băm để định danh duy nhất các thông điệp, ta nên tính đến các va chạm khả dĩ và đảm bảo rằng ta vẫn có thể phân định rạch ròi các thông điệp.

Now, process A can send a message $M(n)$, where n is a monotonically increasing message counter. As soon as B receives the message, it sends an acknowledgment $ACK(n)$ back to A. Figure 8-3 shows this form of communication.

Giờ đây, tiến trình A có thể gửi một thông điệp $M(n)$, trong đó n là một bộ đếm thông điệp tăng đơn điệu. Ngay khi B nhận được thông điệp, nó gửi một xác nhận $ACK(n)$ trở lại cho A. Hình 8-3 cho thấy dạng giao tiếp này.

3 A more precise definition is that if a correct process A sends a message to a correct process B infinitely often, it will be delivered infinitely often ([CACHIN11]).

3 Một định nghĩa chính xác hơn là nếu một tiến trình đúng đắn A gửi một thông điệp cho một tiến trình đúng đắn B vô số lần, thì nó sẽ được chuyển tới vô số lần ([CACHIN11]).

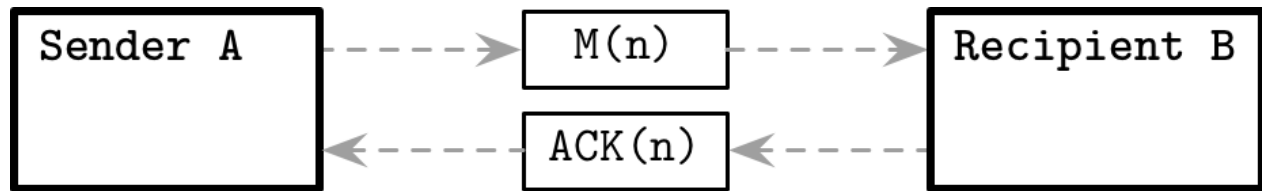


Figure 8-3. Sending a message with an acknowledgment

Hình 8-3. Gửi một thông điệp kèm xác nhận

The acknowledgment, as well as the original message, may get lost on the way. The number of states the message can be in changes slightly. Until A receives an acknowledgment, the message is still in one of the three states we mentioned previously, but as soon as A receives the acknowledgment, it can be confident that the message is delivered to B.

Xác nhận, cũng như thông điệp gốc, có thể bị mất trên đường đi. Số lượng trạng thái mà thông điệp có thể ở thay đổi đôi chút. Cho đến khi A nhận được một xác nhận, thông điệp vẫn ở một trong ba trạng thái mà ta đã đề cập trước đó, nhưng ngay khi A nhận được xác nhận, nó có thể tự tin rằng thông điệp đã được chuyển tới B.

Message retransmits

Truyền lại thông điệp

Adding acknowledgments is still not enough to call this communication protocol reliable: a sent message may still get lost, or the remote process may fail before acknowledging it. To solve this problem and provide delivery guarantees, we can try retransmits instead. Retransmits are a way for the sender to retry a potentially failed operation. We say potentially failed, because the sender doesn't really know whether it has failed or not, since the type of link we're about to discuss does not use acknowledgments.

Thêm các xác nhận vẫn chưa đủ để gọi giao thức giao tiếp này là đáng tin cậy: một thông điệp đã gửi vẫn có thể bị mất, hoặc tiến trình từ xa có thể hỏng trước khi xác nhận nó. Để giải quyết vấn đề này và cung cấp các đảm bảo chuyển tin, ta có thể thử dùng truyền lại (retransmit) để thay thế. Truyền lại là một cách để bên gửi thử lại một thao tác có khả năng đã thất bại. Ta nói có khả năng đã thất bại, vì bên gửi thực ra không biết liệu nó đã thất bại hay chưa, do loại kênh liên kết mà ta sắp bàn không dùng xác nhận.

After process A sends message M , it waits until timeout T is triggered and tries to send the same message again. Assuming the link between processes stays intact, network partitions between the processes are not infinite, and not all packets are lost, we can state that, from the sender's perspective,

the message is either not yet delivered to process B or is successfully delivered to process B . Since A keeps trying to send the message, we can say that it cannot get irrecoverably lost during transport.

Sau khi tiến trình A gửi thông điệp M, nó chờ cho đến khi thời gian chờ T được kích hoạt rồi thử gửi lại cùng thông điệp đó. Với giả định rằng kênh liên kết giữa các tiến trình vẫn còn nguyên vẹn, các phân vùng mạng giữa các tiến trình không phải là vô hạn, và không phải tất cả các gói đều bị mất, ta có thể khẳng định rằng, từ góc nhìn của bên gửi, thông điệp hoặc là chưa được chuyển tới tiến trình B hoặc là đã được chuyển thành công tới tiến trình B. Vì A cứ tiếp tục thử gửi thông điệp, ta có thể nói rằng nó không thể bị mất không thể khôi phục trong khi truyền vận.

In distributed systems terminology, this abstraction is called a stubborn link . It's called stubborn because the sender keeps resending the message again and again indefinitely, but, since this sort of abstraction would be highly impractical, we need to combine retries with acknowledgments.

Theo thuật ngữ hệ phân tán, trừu tượng hóa này được gọi là kênh bền bỉ (stubborn link). Nó được gọi là bền bỉ vì bên gửi cứ gửi lại thông điệp hết lần này đến lần khác một cách vô tận, nhưng vì loại trừu tượng hóa kiểu này sẽ rất không thực tế, ta cần kết hợp các lần thử lại với các xác nhận.

Problem with retransmits

Vấn đề với truyền lại

Whenever we send the message, until we receive an acknowledgment from the remote process, we do not know whether it has already been processed, it will be processed shortly, it has been lost, or the remote process has crashed before receiving it— any one of these states is possible. We can retry the operation and send the message again, but this can result in message duplicates. Processing duplicates is only safe if the operation we're about to perform is idempotent.

Mỗi khi ta gửi thông điệp, cho đến khi ta nhận được một xác nhận từ tiến trình từ xa, ta không biết liệu nó đã được xử lý rồi, sắp được xử lý, đã bị mất, hay tiến trình từ xa đã sập trước khi nhận được nó—bất kỳ trạng thái nào trong số này đều khả dĩ. Ta có thể thử lại thao tác và gửi lại thông điệp, nhưng điều này có thể dẫn đến các thông điệp trùng lặp. Việc xử lý các thông điệp trùng lặp chỉ an toàn nếu thao tác mà ta sắp thực hiện là lũy đẳng (idempotent).

An idempotent operation is one that can be executed multiple times, yielding the same result without producing additional side effects. For example, a server shutdown operation can be idempotent, the first call initiates the shutdown, and all subsequent calls do not produce any additional effects.

Một thao tác lũy đẳng là thao tác có thể được thực thi nhiều lần, cho ra cùng một kết quả mà không tạo ra các tác dụng phụ bổ sung. Ví dụ, một thao tác tắt máy chủ có thể là lũy đẳng: lời gọi đầu tiên khởi động việc tắt máy, và tất cả các lời gọi tiếp theo không tạo ra bất kỳ tác dụng bổ sung nào.

If every operation was idempotent, we could think less about delivery semantics, rely more on retransmits for fault tolerance, and build systems in an entirely reactive way: triggering an action as a response to some signal, without causing unintended side effects. However, operations are not necessarily idempotent, and merely assuming that they are might lead to cluster-wide side effects. For example, charging a customer's credit card is not idempotent, and charging it multiple times is definitely undesirable.

Nếu mọi thao tác đều lũy đẳng, ta có thể bớt phải nghĩ về ngữ nghĩa chuyển tin, dựa nhiều hơn vào truyền lại để dung lỗi, và xây dựng các hệ thống theo một cách hoàn toàn phản ứng (reactive): kích hoạt một hành động như một sự đáp lại trước một tín hiệu nào đó, mà

không gây ra các tác dụng phụ ngoài ý muốn. Tuy nhiên, các thao tác không nhất thiết là lũy đẳng, và việc chỉ đơn thuần giả định rằng chúng lũy đẳng có thể dẫn đến các tác dụng phụ trên phạm vi toàn cục. Ví dụ, việc tính tiền vào thẻ tín dụng của khách hàng không phải là lũy đẳng, và việc tính tiền nhiều lần thì chắc chắn là điều không mong muốn.

Idempotence is particularly important in the presence of partial failures and network partitions, since we cannot always find out the exact status of a remote operation— whether it has succeeded, failed, or will be executed shortly—and we just have to wait longer. Since guaranteeing that each executed operation is idempotent is an unrealistic requirement, we need to provide guarantees equivalent to idempotence without changing the underlying operation semantics. To achieve this, we can use deduplication and avoid processing messages more than once.

Tính lũy đẳng đặc biệt quan trọng khi có lỗi cục bộ và phân vùng mạng, vì ta không phải lúc nào cũng tìm ra được tình trạng chính xác của một thao tác từ xa—liệu nó đã thành công, đã thất bại, hay sắp được thực thi—và ta chỉ còn cách phải chờ lâu hơn. Vì đảm bảo rằng mỗi thao tác được thực thi đều lũy đẳng là một yêu cầu phi thực tế, ta cần cung cấp các đảm bảo tương đương với tính lũy đẳng mà không thay đổi ngữ nghĩa của thao tác bên dưới. Để đạt được điều này, ta có thể dùng cơ chế khử trùng lặp (deduplication) và tránh xử lý các thông điệp nhiều hơn một lần.

Message order

Thứ tự thông điệp

Unreliable networks present us with two problems: messages can arrive out of order and, because of retransmits, some messages may arrive more than once. We have already introduced sequence numbers, and we can use these message identifiers on the recipient side to ensure first-in, first-out (FIFO) ordering. Since every message has a sequence number, the receiver can **track**:

Các mạng không đáng tin cậy đặt ra cho ta hai vấn đề: thông điệp có thể đến không đúng thứ tự và, do truyền lại, một số thông điệp có thể đến nhiều hơn một lần. Ta đã đưa vào các số thứ tự, và ta có thể dùng các định danh thông điệp này ở phía bên nhận để đảm bảo thứ tự vào-trước-ra-trước (FIFO). Vì mỗi thông điệp đều có một số thứ tự, bên nhận có thể theo dõi:

- n , specifying the highest sequence number, up to which it has seen all

- n , chỉ định số thứ tự cao nhất, tính tới đó nó đã thấy tất cả các

messages. Messages up to this number can be put back in order. • n , specifying the highest sequence number, up to which messages were put

thông điệp. Các thông điệp tính tới số này có thể được đặt trở lại đúng thứ tự. • n , chỉ định số thứ tự cao nhất, tính tới đó các thông điệp đã được đặt

back in their original order and processed . This number can be used for deduplication.

trở lại đúng thứ tự gốc của chúng và đã được xử lý. Số này có thể được dùng để khử trùng lặp.

If the received message has a nonconsecutive sequence number, the receiver puts it into the reordering buffer. For example, it receives a message with a sequence number 5 after receiving one with 3 , and we know that 4 is still missing, so we need to put 5 aside until 4 comes, and we can reconstruct the message order. Since we're building on top of a fair-loss link, we assume that messages between n and n will

Nếu thông điệp nhận được có một số thứ tự không liên tiếp, bên nhận đặt nó vào vùng đệm sắp xếp lại thứ tự (reordering buffer). Ví dụ, nó nhận được một thông điệp có số thứ

tự 5 sau khi nhận được một thông điệp số 3, và ta biết rằng số 4 vẫn còn thiếu, nên ta cần để số 5 sang một bên cho đến khi số 4 đến, và ta có thể tái dựng thứ tự thông điệp. Vì ta đang xây dựng trên nền một kênh fair-loss, ta giả định rằng các thông điệp giữa n và n rất cuộc

eventually be delivered.

sẽ được chuyển tới.

The recipient can safely discard the messages with sequence numbers up to n

Bên nhận có thể an toàn loại bỏ các thông điệp có số thứ tự tính tới n

that it receives, since they're guaranteed to be already delivered.

mà nó nhận được, vì chúng được đảm bảo là đã được chuyển tới rồi.

Deduplication works by checking if the message with a sequence number n has already been processed (passed down the stack by the receiver) and discarding already processed messages.

Khử trùng lặp hoạt động bằng cách kiểm tra xem thông điệp có số thứ tự n đã được xử lý hay chưa (đã được bên nhận chuyển xuống dưới ngăn xếp) và loại bỏ những thông điệp đã được xử lý.

In distributed systems terms, this type of link is called a perfect link, which provides the following guarantees [CACHIN11]:

Theo thuật ngữ hệ phân tán, loại kênh liên kết này được gọi là kênh hoàn hảo (perfect link), cung cấp các đảm bảo sau [CACHIN11]:

Reliable delivery Every message sent once by the correct process A to the correct process B , will eventually be delivered.

Chuyển tin đáng tin cậy (Reliable delivery) Mỗi thông điệp được gửi một lần bởi tiến trình đúng đắn A tới tiến trình đúng đắn B sẽ rất cuộc được chuyển tới.

No duplication No message is delivered more than once.

Không trùng lặp (No duplication) Không thông điệp nào được chuyển tới nhiều hơn một lần.

No creation Same as with other types of links, it can only deliver the messages that were actually sent.

Không tạo mới (No creation) Giống như với các loại kênh khác, nó chỉ có thể chuyển tới những thông điệp đã thực sự được gửi đi.

This might remind you of the TCP 4 protocol (however, reliable delivery in TCP is guaranteed only in the scope of a single session). Of course, this model is just a simplified representation we use for illustration purposes only. TCP has a much more sophisticated model for dealing with acknowledgments, which groups acknowledgments and reduces the protocol-level overhead. In addition, TCP has selective acknowledgments, flow control, congestion control, error detection, and many other features that are out of the scope of our discussion.

Điều này có thể làm bạn liên tưởng đến giao thức TCP 4 (tuy nhiên, việc chuyển tin đáng tin cậy trong TCP chỉ được đảm bảo trong phạm vi một phiên duy nhất). Tất nhiên, mô hình này chỉ là một biểu diễn được đơn giản hóa mà ta dùng cho mục đích minh họa mà thôi. TCP có một mô hình tinh vi hơn nhiều để xử lý các xác nhận, mô hình này gom nhóm các xác nhận và giảm bớt phụ phí ở mức giao thức. Ngoài ra, TCP còn có xác nhận chọn lọc,

kiểm soát luồng, kiểm soát tắc nghẽn, phát hiện lỗi và nhiều tính năng khác nằm ngoài phạm vi thảo luận của chúng ta.

Exactly-once delivery

Chuyển tin chính-xác-một-lần

There are only two hard problems in distributed systems: 2. Exactly-once delivery 1. Guaranteed order of messages 2. Exactly-once delivery. —Mathias Verraes

Trong hệ phân tán chỉ có hai vấn đề khó: 2. Chuyển tin chính-xác-một-lần 1. Đảm bảo thứ tự của các thông điệp 2. Chuyển tin chính-xác-một-lần. —Mathias Verraes

There have been many discussions about whether or not exactly-once delivery is possible. Here, semantics and precise wording are essential. Since there might be a link failure preventing the message from being delivered from the first try, most of the real-world systems employ at-least-once delivery, which ensures that the sender retries until it receives an acknowledgment, otherwise the message is not considered to be received. Another delivery semantic is at-most-once: the sender sends the message and doesn't expect any delivery confirmation.

Đã có nhiều cuộc thảo luận về việc liệu chuyển tin chính-xác-một-lần (exactly-once delivery) có khả thi hay không. Ở đây, ngữ nghĩa và cách dùng từ chính xác là thiết yếu. Vì có thể có một lỗi kênh liên kết ngăn thông điệp được chuyển tới ngay từ lần thử đầu tiên, hầu hết các hệ thống trong thế giới thực dùng chuyển tin ít-nhất-một-lần (at-least-once delivery), vốn đảm bảo rằng bên gửi cứ thử lại cho đến khi nó nhận được một xác nhận, nếu không thì thông điệp không được coi là đã nhận. Một ngữ nghĩa chuyển tin khác là nhiều-nhất-một-lần (at-most-once): bên gửi gửi thông điệp đi và không kỳ vọng bất kỳ xác nhận chuyển tin nào.

The TCP protocol works by breaking down messages into packets, transmitting them one by one, and stitching them back together on the receiving side. TCP might attempt to retransmit some of the packets, and more than one transmission attempt

Giao thức TCP hoạt động bằng cách chia nhỏ các thông điệp thành các gói, truyền chúng đi từng cái một, và ghép chúng lại với nhau ở phía bên nhận. TCP có thể cố truyền lại một số gói, và nhiều hơn một lần thử truyền

4 See <https://databass.dev/links/53>.

4 Xem <https://databass.dev/links/53>.

may succeed. Since TCP marks each packet with a sequence number, even though some packets were transmitted more than once, it can deduplicate the packets and guarantee that the recipient will see the message and process it only once. In TCP, this guarantee is valid only for a single session: if the message is acknowledged and processed, but the sender didn't receive the acknowledgment before the connection was interrupted, the application is not aware of this delivery and, depending on its logic, it might attempt to send the message once again.

có thể thành công. Vì TCP đánh dấu mỗi gói bằng một số thứ tự, nên ngay cả khi một số gói được truyền nhiều hơn một lần, nó vẫn có thể khử trùng lặp các gói và đảm bảo rằng bên nhận sẽ thấy thông điệp và chỉ xử lý nó đúng một lần. Trong TCP, đảm bảo này chỉ có hiệu lực trong phạm vi một phiên duy nhất: nếu thông điệp được xác nhận và xử lý, nhưng bên gửi không nhận được xác nhận trước khi kết nối bị gián đoạn, thì ứng dụng không hề biết về lần chuyển tin này và, tùy vào logic của nó, nó có thể cố gắng gửi lại thông điệp một lần nữa.

This means that exactly-once processing is what's interesting here since duplicate deliveries (or packet transmissions) have no side effects and are merely an artifact of the best effort by the link. For example, if the database node has only received the record, but hasn't persisted it, delivery has occurred, but it'll be of no use unless the record can be retrieved (in other words, unless it was both delivered and processed).

Điều này có nghĩa là xử lý chính-xác-một-lần mới chính là điều đáng quan tâm ở đây, vì các lần chuyển tin trùng lặp (hay truyền gói trùng lặp) không có tác dụng phụ và chỉ đơn thuần là một sản phẩm phụ của nỗ lực hết sức (best effort) của kênh liên kết. Ví dụ, nếu một nút cơ sở dữ liệu mới chỉ nhận được bản ghi, nhưng chưa lưu trữ bền vững nó, thì lần chuyển tin đã xảy ra, nhưng nó sẽ chẳng có ích gì trừ khi bản ghi có thể được truy xuất (nói cách khác, trừ khi nó vừa được chuyển tới vừa được xử lý).

For the exactly-once guarantee to hold, nodes should have a common knowledge [HALPERN90] : everyone knows about some fact, and everyone knows that everyone else also knows about that fact. In simplified terms, nodes have to agree on the state of the record: both nodes agree that it either was or was not persisted. As you will see later in this chapter, this is theoretically impossible, but in practice we still use this notion by relaxing coordination requirements.

Để đảm bảo chính-xác-một-lần thành lập, các nút nên có hiểu biết chung (common knowledge) [HALPERN90]: mọi nút đều biết về một sự kiện nào đó, và mọi nút đều biết rằng mọi nút khác cũng biết về sự kiện đó. Nói một cách đơn giản, các nút phải đồng thuận về trạng thái của bản ghi: cả hai nút đều đồng ý rằng nó hoặc đã hoặc chưa được lưu trữ bền vững. Như bạn sẽ thấy ở phần sau của chương này, điều này về mặt lý thuyết là bất khả thi, nhưng trong thực tế ta vẫn dùng khái niệm này bằng cách nói lỏng các yêu cầu phối hợp.

Any misunderstanding about whether or not exactly-once delivery is possible most likely comes from approaching the problem from different protocol and abstraction levels and the definition of “delivery.” It's not possible to build a reliable link without ever transferring any message more than once, but we can create the illusion of exactly-once delivery from the sender's perspective by processing the message once and ignoring duplicates.

Bất kỳ sự hiểu lầm nào về việc liệu chuyển tin chính-xác-một-lần có khả thi hay không nhiều khả năng đến từ việc tiếp cận vấn đề từ những mức giao thức và trừu tượng hóa khác nhau và từ định nghĩa của “chuyển tin”. Không thể xây dựng một kênh liên kết đáng tin cậy mà không bao giờ phải truyền bất kỳ thông điệp nào nhiều hơn một lần, nhưng ta có thể tạo ra ảo giác về chuyển tin chính-xác-một-lần từ góc nhìn của bên gửi bằng cách xử lý thông điệp một lần và phớt lờ các bản trùng lặp.

Now, as we have established the means for reliable communication, we can move ahead and look for ways to achieve uniformity and agreement between processes in the distributed system.

Giờ đây, khi ta đã thiết lập được các phương tiện cho giao tiếp đáng tin cậy, ta có thể tiến lên và tìm cách đạt được tính đồng nhất và sự đồng thuận giữa các tiến trình trong hệ phân tán.

Two Generals' Problem — Bài toán hai vị tướng

One of the most prominent descriptions of an agreement in a distributed system is a thought experiment widely known as the Two Generals' Problem .

Một trong những mô tả nổi bật nhất về sự đồng thuận trong một hệ phân tán là một thí nghiệm tư duy được biết đến rộng rãi với tên gọi Bài toán hai vị tướng (Two Generals'

Problem).

This thought experiment shows that it is impossible to achieve an agreement between two parties if communication is asynchronous in the presence of link failures. Even though TCP exhibits properties of a perfect link, it's important to remember that perfect links, despite the name, do not guarantee perfect delivery. They also can't guarantee that participants will be alive the whole time, and are concerned only with transport.

Thí nghiệm tư duy này cho thấy rằng không thể đạt được sự đồng thuận giữa hai bên nếu việc giao tiếp là bất đồng bộ khi có các lỗi kênh liên kết. Mặc dù TCP thể hiện các đặc tính của một kênh hoàn hảo, điều quan trọng cần nhớ là các kênh hoàn hảo, dù mang tên như vậy, không đảm bảo chuyển tin hoàn hảo. Chúng cũng không thể đảm bảo rằng các bên tham gia sẽ còn sống suốt cả thời gian, và chỉ quan tâm đến việc truyền vận mà thôi.

Imagine two armies, led by two generals, preparing to attack a fortified city. The armies are located on two sides of the city and can succeed in their siege only if they attack simultaneously.

Hãy hình dung hai đạo quân, do hai vị tướng chỉ huy, đang chuẩn bị tấn công một thành phố được phòng thủ kiên cố. Hai đạo quân nằm ở hai phía của thành phố và chỉ có thể thành công trong cuộc vây hãm nếu họ tấn công đồng thời.

The generals can communicate by sending messengers, and already have devised an attack plan. The only thing they now have to agree on is whether or not to carry out the plan. Variants of this problem are when one of the generals has a higher rank, but needs to make sure the attack is coordinated; or that the generals need to agree on the exact time. These details do not change the problem definition: the generals have to come to an agreement.

Các vị tướng có thể giao tiếp bằng cách gửi sứ giả, và đã vạch ra một kế hoạch tấn công. Điều duy nhất họ phải thống nhất bây giờ là có thực hiện kế hoạch hay không. Các biến thể của bài toán này là khi một trong các vị tướng có cấp bậc cao hơn, nhưng cần đảm bảo rằng cuộc tấn công được phối hợp; hoặc rằng các vị tướng cần thống nhất về thời điểm chính xác. Những chi tiết này không làm thay đổi định nghĩa của bài toán: các vị tướng phải đi đến một sự đồng thuận.

The army generals only have to agree on the fact that they both will proceed with the attack. Otherwise, the attack cannot succeed. General A sends a message MSG(N) , stating an intention to proceed with the attack at a specified time, if the other party agrees to proceed as well.

Các vị tướng chỉ cần thống nhất về việc cả hai sẽ cùng tiến hành tấn công. Bằng không, cuộc tấn công không thể thành công. Tướng A gửi một thông điệp MSG(N), nêu rõ ý định tiến hành tấn công vào một thời điểm cụ thể, nếu bên kia cũng đồng ý tiến hành.

After A sends the messenger, he doesn't know whether the messenger has arrived or not: the messenger can get captured and fail to deliver the message. When general B receives the message, he has to send an acknowledgment ACK(MSG(N)) . Figure 8-4 shows that a message is sent one way and acknowledged by the other party.

Sau khi A cử sứ giả đi, ông không biết liệu sứ giả đã đến nơi hay chưa: sứ giả có thể bị bắt và không chuyển được thông điệp. Khi tướng B nhận được thông điệp, ông phải gửi một xác nhận ACK(MSG(N)). Hình 8-4 cho thấy một thông điệp được gửi theo một chiều và được bên kia xác nhận.

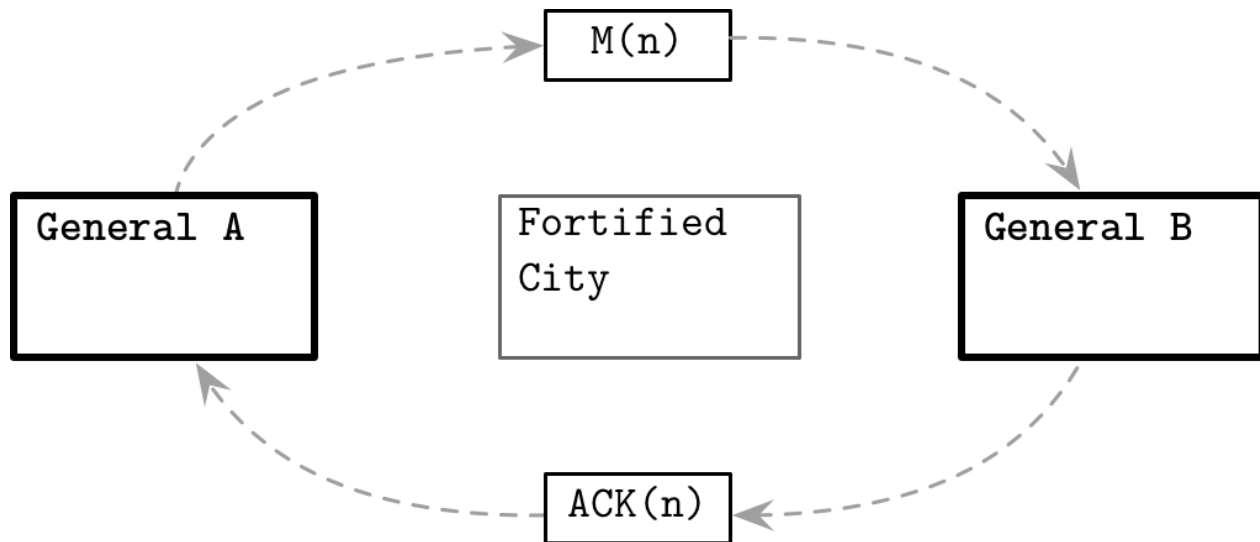


Figure 8-4. Two Generals' Problem illustrated

Hình 8-4. Minh họa Bài toán hai vị tướng

The messenger carrying this acknowledgment might get captured or fail to deliver it, as well. B doesn't have any way of knowing if the messenger has successfully delivered the acknowledgment.

Sứ giả mang xác nhận này cũng có thể bị bắt hoặc không chuyển được nó. B không có cách nào để biết liệu sứ giả đã chuyển xác nhận thành công hay chưa.

To be sure about it, B has to wait for $ACK(ACK(MSG(N)))$, a second-order acknowledgment stating that A received an acknowledgment for the acknowledgment.

Để chắc chắn về điều đó, B phải chờ một $ACK(ACK(MSG(N)))$, một xác nhận cấp hai nêu rõ rằng A đã nhận được một xác nhận cho cái xác nhận kia.

No matter how many further confirmations the generals send to each other, they will always be one ACK away from knowing if they can safely proceed with the attack. The generals are doomed to wonder if the message carrying this last acknowledgment has reached the destination.

Cho dù các vị tướng có gửi thêm bao nhiêu xác nhận cho nhau đi nữa, vẫn luôn còn thiếu đúng một cái ACK nữa thì họ mới biết chắc liệu có thể an toàn tiến hành tấn công hay không. Các vị tướng cứ phải băn khoăn mãi liệu thông điệp mang cái xác nhận cuối cùng này đã tới được đích hay chưa.

Notice that we did not make any timing assumptions: communication between generals is fully asynchronous. There is no upper time bound set on how long the generals can take to respond.

Lưu ý rằng ta đã không đưa ra bất kỳ giả định nào về thời gian: việc giao tiếp giữa các vị tướng là hoàn toàn bất đồng bộ. Không có một giới hạn thời gian trên nào được đặt ra cho việc các vị tướng có thể mất bao lâu để phản hồi.

FLP Impossibility — Bất khả thi FLP

In a paper by Fisher, Lynch, and Paterson, the authors describe a problem famously known as the **FLP Impossibility** Problem [FISCHER85] (derived from the first letters of authors' last names), wherein they discuss a form of **consensus** in which processes start with an initial value and attempt to agree

on a new value. After the algorithm completes, this new value has to be the same for all nonfaulty processes.

Trong một bài báo của Fisher, Lynch và Paterson, các tác giả mô tả một bài toán nổi tiếng được gọi là Bài toán bất khả thi FLP [FISCHER85] (lấy từ các chữ cái đầu trong họ của các tác giả), trong đó họ thảo luận về một dạng đồng thuận mà các tiến trình bắt đầu với một giá trị ban đầu và cố gắng thống nhất về một giá trị mới. Sau khi thuật toán hoàn tất, giá trị mới này phải giống nhau cho tất cả các tiến trình không bị lỗi.

Reaching an agreement on a specific value is straightforward if the network is entirely reliable; but in reality, systems are prone to many different sorts of failures, such as message loss, duplication, network partitions, and slow or crashed processes.

Đạt được sự thống nhất về một giá trị cụ thể là điều đơn giản nếu mạng hoàn toàn đáng tin cậy; nhưng trong thực tế, các hệ thống dễ gặp nhiều loại lỗi khác nhau, chẳng hạn như mất thông điệp, trùng lặp, phân vùng mạng và các tiến trình chậm hoặc bị sập.

A consensus protocol describes a system that, given multiple processes starting at its initial state, brings all of the processes to the decision state. For a consensus protocol to be correct, it has to preserve three properties:

Một giao thức đồng thuận mô tả một hệ thống mà, cho trước nhiều tiến trình bắt đầu ở trạng thái ban đầu của nó, đưa tất cả các tiến trình đến trạng thái quyết định. Để một giao thức đồng thuận là đúng đắn, nó phải bảo toàn ba đặc tính:

Agreement The decision the protocol arrives at has to be unanimous: each process decides on some value, and this has to be the same for all processes. Otherwise, we have not reached a consensus.

Sự nhất trí (Agreement) Quyết định mà giao thức đi đến phải nhất trí: mỗi tiến trình quyết định trên một giá trị nào đó, và giá trị này phải giống nhau cho tất cả các tiến trình. Bằng không, ta đã không đạt được sự đồng thuận.

Validity The agreed value has to be proposed by one of the participants, which means that the system should not just “come up” with the value. This also implies nontriviality of the value: processes should not always decide on some predefined default value.

Tính hợp lệ (Validity) Giá trị được thống nhất phải do một trong các bên tham gia đề xuất, điều này có nghĩa là hệ thống không được tự “nghĩ ra” giá trị. Điều này cũng ngụ ý tính không tầm thường của giá trị: các tiến trình không nên lúc nào cũng quyết định trên một giá trị mặc định định sẵn nào đó.

Termination An agreement is final only if there are no processes that did not reach the decision state.

Tính kết thúc (Termination) Một sự thống nhất chỉ là cuối cùng nếu không có tiến trình nào chưa đạt đến trạng thái quyết định.

[FISCHER85] assumes that processing is entirely asynchronous; there’s no shared notion of time between the processes. Algorithms in such systems cannot be based on timeouts, and there’s no way for a process to find out whether the other process has crashed or is simply running too slow. The paper shows that, given these assumptions, there exists no protocol that can guarantee consensus in a bounded time. No completely asynchronous consensus algorithm can tolerate the unannounced crash of even a single remote process.

[FISCHER85] giả định rằng việc xử lý là hoàn toàn bất đồng bộ; không có khái niệm thời gian dùng chung nào giữa các tiến trình. Các thuật toán trong các hệ thống như vậy không thể dựa trên thời gian chờ, và không có cách nào để một tiến trình tìm ra liệu tiến trình kia đã sập hay chỉ đơn giản là đang chạy quá chậm. Bài báo cho thấy rằng, với những giả

định này, không tồn tại giao thức nào có thể đảm bảo đồng thuận trong một thời gian có giới hạn. Không một thuật toán đồng thuận hoàn toàn bất đồng bộ nào có thể dung được sự sập không báo trước của dù chỉ một tiến trình từ xa duy nhất.

If we do not consider an upper time bound for the process to complete the algorithm steps, process failures can't be reliably detected, and there's no deterministic algorithm to reach a consensus.

Nếu ta không xét đến một giới hạn thời gian trên cho việc tiến trình hoàn thành các bước của thuật toán, thì các lỗi tiến trình không thể được phát hiện một cách đáng tin cậy, và không có thuật toán tất định nào để đạt được sự đồng thuận.

However, FLP Impossibility does not mean we have to pack our things and go home, as reaching consensus is not possible. It only means that we cannot always reach consensus in an asynchronous system in bounded time. In practice, systems exhibit at least some degree of synchrony, and the solution to this problem requires a more refined model.

Tuy nhiên, Bất khả thi FLP không có nghĩa là ta phải thu xếp đồ đạc về nhà vì việc đạt được đồng thuận là bất khả thi. Nó chỉ có nghĩa là ta không phải lúc nào cũng có thể đạt được đồng thuận trong một hệ bất đồng bộ trong thời gian có giới hạn. Trong thực tế, các hệ thống thể hiện ít nhất một mức độ đồng bộ nào đó, và giải pháp cho vấn đề này đòi hỏi một mô hình tinh tế hơn.

System Synchrony — Tính đồng bộ của hệ thống

From FLP Impossibility, you can see that the timing assumption is one of the critical characteristics of the distributed system. In an asynchronous system, we do not know the relative speeds of processes, and cannot guarantee message delivery in a bounded time or a particular order. The process might take indefinitely long to respond, and process failures can't always be reliably detected.

Từ Bất khả thi FLP, bạn có thể thấy rằng giả định về thời gian là một trong những đặc điểm then chốt của hệ phân tán. Trong một hệ bất đồng bộ, ta không biết tốc độ tương đối của các tiến trình, và không thể đảm bảo việc chuyển tin trong một thời gian có giới hạn hay theo một thứ tự cụ thể. Tiến trình có thể mất một thời gian dài vô hạn để phản hồi, và các lỗi tiến trình không phải lúc nào cũng có thể được phát hiện một cách đáng tin cậy.

The main criticism of asynchronous systems is that these assumptions are not realistic: processes can't have arbitrarily different processing speeds, and links don't take indefinitely long to deliver messages. Relying on time both simplifies reasoning and helps to provide upper-bound timing guarantees.

Lời chỉ trích chính dành cho các hệ bất đồng bộ là những giả định này không thực tế: các tiến trình không thể có tốc độ xử lý khác nhau một cách tùy ý, và các kênh liên kết không mất một thời gian dài vô hạn để chuyển các thông điệp. Việc dựa vào thời gian vừa đơn giản hóa việc suy luận vừa giúp cung cấp các đảm bảo về thời gian có giới hạn trên.

It is not always possible to solve a consensus problem in an asynchronous model [FISCHER85]. Moreover, designing an efficient synchronous algorithm is not always achievable, and for some tasks the practical solutions are more likely to be time-dependent [ARJOMANDI83].

Không phải lúc nào cũng có thể giải được một bài toán đồng thuận trong một mô hình bất đồng bộ [FISCHER85]. Hơn nữa, việc thiết kế một thuật toán đồng bộ hiệu quả không phải lúc nào cũng khả thi, và với một số tác vụ thì các giải pháp thực tiễn nhiều khả năng phụ thuộc vào thời gian hơn [ARJOMANDI83].

These assumptions can be loosened up, and the system can be considered to be synchronous . For that, we introduce the notion of timing. It is much easier to reason about the system under the synchronous model. It assumes that processes are progressing at comparable rates, that transmission delays are bounded, and message delivery cannot take arbitrarily long.

Những giả định này có thể được nói lỏng ra, và hệ thống có thể được coi là đồng bộ. Để làm vậy, ta đưa vào khái niệm thời điểm. Suy luận về hệ thống dưới mô hình đồng bộ thì dễ hơn nhiều. Nó giả định rằng các tiến trình đang tiến triển ở những tốc độ tương đương nhau, rằng các độ trễ truyền tin là có giới hạn, và việc chuyển tin không thể mất một thời gian dài tùy ý.

A synchronous system can also be represented in terms of synchronized process-local clocks: there is an upper time bound in time difference between the two process-local time sources [CACHIN11] .

Một hệ đồng bộ cũng có thể được biểu diễn theo phương diện các đồng hồ cục bộ của tiến trình được đồng bộ hóa: có một giới hạn thời gian trên cho khác biệt thời gian giữa hai nguồn thời gian cục bộ của tiến trình [CACHIN11].

Designing systems under a synchronous model allows us to use timeouts. We can build more complex abstractions, such as **leader election**, consensus, failure detection, and many others on top of them. This makes the best-case scenarios more robust, but results in a failure if the timing assumptions don't hold up. For example, in the **Raft** consensus algorithm (see “Raft” on page 300), we may end up with multiple processes believing they're leaders, which is resolved by forcing the lagging process to accept the other process as a **leader**; failure-detection algorithms (see Chapter 9) can wrongly identify a live process as failed or vice versa. When designing our systems, we should make sure to consider these possibilities.

Thiết kế các hệ thống dưới một mô hình đồng bộ cho phép ta dùng thời gian chờ. Ta có thể xây dựng các trừu tượng hóa phức tạp hơn, chẳng hạn như bầu chọn leader, đồng thuận, phát hiện lỗi, và nhiều thứ khác, trên nền chúng. Điều này làm cho các kịch bản trong trường hợp tốt nhất trở nên vững chắc hơn, nhưng dẫn đến một lỗi nếu các giả định về thời gian không thành lập. Ví dụ, trong thuật toán đồng thuận Raft (xem “Raft” ở trang 300), ta có thể gặp tình huống nhiều tiến trình tin rằng chúng là leader, vấn đề này được giải quyết bằng cách buộc tiến trình tụt hậu chấp nhận tiến trình kia làm leader; các thuật toán phát hiện lỗi (xem Chương 9) có thể nhận diện sai một tiến trình còn sống là đã hỏng hoặc ngược lại. Khi thiết kế các hệ thống của mình, ta nên đảm bảo cân nhắc những khả năng này.

Properties of both asynchronous and synchronous models can be combined, and we can think of a system as partially synchronous . A partially synchronous system exhibits some of the properties of the synchronous system, but the bounds of message delivery, clock drift, and relative processing speeds might not be exact and hold only most of the time [DWORK88] .

Các đặc tính của cả mô hình bất đồng bộ lẫn đồng bộ có thể được kết hợp, và ta có thể nghĩ về một hệ thống như là đồng bộ một phần (partially synchronous). Một hệ đồng bộ một phần thể hiện một số đặc tính của hệ đồng bộ, nhưng các giới hạn về chuyển tin, độ lệch đồng hồ và tốc độ xử lý tương đối có thể không chính xác và chỉ thành lập trong phần lớn thời gian [DWORK88].

Synchrony is an essential property of the distributed system: it has an impact on performance, scalability, and general solvability, and has many factors necessary for the correct functioning of our systems. Some of the algorithms we discuss in this book operate under the assumptions of synchronous systems.

Tính đồng bộ là một đặc tính thiết yếu của hệ phân tán: nó có tác động đến hiệu năng, khả

năng mở rộng và khả năng giải được nói chung, và có nhiều yếu tố cần thiết cho việc vận hành đúng đắn của các hệ thống của chúng ta. Một số thuật toán mà ta bàn trong cuốn sách này vận hành dưới các giả định của các hệ đồng bộ.

Failure Models — Các mô hình lỗi

We keep mentioning failures , but so far it has been a rather broad and generic concept that might capture many meanings. Similar to how we can make different timing assumptions, we can assume the presence of different types of failures. A failure model describes exactly how processes can crash in a distributed system, and algorithms are developed using these assumptions. For example, we can assume that a process can crash and never recover, or that it is expected to recover after some time passes, or that it can fail by spinning out of control and supplying incorrect values.

Chúng ta cứ liên tục nhắc đến lỗi, nhưng cho đến nay nó vẫn là một khái niệm khá rộng và chung chung có thể nắm bắt nhiều ý nghĩa. Tương tự như cách ta có thể đưa ra các giả định khác nhau về thời gian, ta có thể giả định sự hiện diện của các loại lỗi khác nhau. Một mô hình lỗi mô tả chính xác cách mà các tiến trình có thể sập trong một hệ phân tán, và các thuật toán được phát triển dựa trên các giả định này. Ví dụ, ta có thể giả định rằng một tiến trình có thể sập và không bao giờ phục hồi, hoặc rằng nó được kỳ vọng sẽ phục hồi sau khi qua một khoảng thời gian nào đó, hoặc rằng nó có thể hỏng bằng cách quay cuồng mất kiểm soát và cung cấp các giá trị sai.

In distributed systems, processes rely on one another for executing an algorithm, so failures can result in incorrect execution across the whole system.

Trong hệ phân tán, các tiến trình dựa vào nhau để thực thi một thuật toán, nên các lỗi có thể dẫn đến việc thực thi không đúng đắn trên toàn hệ thống.

We'll discuss multiple failure models present in distributed systems, such as crash , omission , and arbitrary faults. This list is not exhaustive, but it covers most of the cases applicable and important in real-life systems.

Chúng ta sẽ thảo luận về nhiều mô hình lỗi hiện diện trong hệ phân tán, chẳng hạn như lỗi sập (crash), lỗi bỏ sót (omission), và lỗi tùy tiện (arbitrary). Danh sách này không phải là đầy đủ, nhưng nó bao quát hầu hết các trường hợp áp dụng được và quan trọng trong các hệ thống thực tế.

Crash Faults — Lỗi sập

Normally, we expect the process to be executing all steps of an algorithm correctly. The simplest way for a process to crash is by stopping the execution of any further steps required by the algorithm and not sending any messages to other processes. In other words, the process crashes . Most of the time, we assume a crash-stop process abstraction, which prescribes that, once the process has crashed, it remains in this state.

Thông thường, ta kỳ vọng tiến trình thực thi tất cả các bước của một thuật toán một cách đúng đắn. Cách đơn giản nhất để một tiến trình sập là bằng cách dừng việc thực thi bất kỳ bước nào tiếp theo mà thuật toán yêu cầu và không gửi bất kỳ thông điệp nào tới các tiến trình khác. Nói cách khác, tiến trình sập. Hầu hết thời gian, ta giả định một trừu tượng hóa tiến trình kiểu sập-dừng (crash-stop), vốn quy định rằng, một khi tiến trình đã sập, nó duy trì ở trạng thái này.

This model does not assume that it is impossible for the process to recover, and does not discourage **recovery** or try to prevent it. It only means that the algorithm does not rely on recovery for correctness or liveness. Nothing prevents processes from recovering, catching up with the system state, and participating in the next instance of the algorithm.

Mô hình này không giả định rằng việc tiến trình phục hồi là bất khả thi, và không ngăn cản việc phục hồi hay cố gắng ngăn chặn nó. Nó chỉ có nghĩa là thuật toán không dựa vào việc phục hồi để đảm bảo tính đúng đắn hay tính sống động (liveness). Không gì ngăn cản các tiến trình phục hồi, bắt kịp trạng thái của hệ thống, và tham gia vào lần chạy tiếp theo của thuật toán.

Failed processes are not able to continue participating in the current round of negotiations during which they failed. Assigning the recovering process a new, different identity does not make the model equivalent to crash-recovery (discussed next), since most algorithms use predefined lists of processes and clearly define failure semantics in terms of how many failures they can tolerate [CACHIN11] .

Các tiến trình đã hỏng không thể tiếp tục tham gia vào vòng đàm phán hiện tại mà trong đó chúng đã hỏng. Việc gán cho tiến trình đang phục hồi một danh tính mới, khác đi không làm cho mô hình này tương đương với sập-phục-hồi (crash-recovery, bàn ở phần kế tiếp), vì hầu hết các thuật toán dùng các danh sách tiến trình định sẵn và định nghĩa rõ ràng ngữ nghĩa lỗi theo phương diện chúng có thể dung được bao nhiêu lỗi [CACHIN11].

Crash-recovery is a different process abstraction, under which the process stops executing the steps required by the algorithm, but recovers at a later point and tries to execute further steps. The possibility of recovery requires introducing a durable state and recovery protocol into the system [SKEEN83] . Algorithms that allow crash-recovery need to take all possible recovery states into consideration, since the recovering process may attempt to continue execution from the last step known to it.

Sập-phục-hồi là một trừu tượng hóa tiến trình khác, theo đó tiến trình dừng việc thực thi các bước mà thuật toán yêu cầu, nhưng phục hồi vào một thời điểm sau đó và cố gắng thực thi các bước tiếp theo. Khả năng phục hồi đòi hỏi phải đưa vào hệ thống một trạng thái bền vững và một giao thức phục hồi [SKEEN83]. Các thuật toán cho phép sập-phục-hồi cần cân nhắc tất cả các trạng thái phục hồi khả dĩ, vì tiến trình đang phục hồi có thể cố gắng tiếp tục thực thi từ bước cuối cùng mà nó biết.

Algorithms, aiming to exploit recovery, have to take both state and identity into account. Crash-recovery, in this case, can also be viewed as a special case of omission failure, since from the other process's perspective there's no distinction between the process that was unreachable and the one that has crashed and recovered.

Các thuật toán nhằm khai thác việc phục hồi phải tính đến cả trạng thái lẫn danh tính. Sập-phục-hồi, trong trường hợp này, cũng có thể được xem như một trường hợp đặc biệt của lỗi bỏ sót, vì từ góc nhìn của tiến trình khác thì không có sự phân biệt nào giữa một tiến trình không tiếp cận được và một tiến trình đã sập rồi phục hồi.

Omission Faults — Lỗi bỏ sót

Another failure model is omission fault . This model assumes that the process skips some of the algorithm steps, or is not able to execute them, or this execution is not visible to other participants, or it cannot send or receive messages to and from other participants. Omission fault captures network partitions between the processes caused by faulty network links, switch failures, or network congestion. Network partitions can be represented as omissions of messages between individual

processes or process groups. A crash can be simulated by completely omitting any messages to and from the process.

Một mô hình lỗi khác là lỗi bỏ sót (omission fault). Mô hình này giả định rằng tiến trình bỏ qua một số bước của thuật toán, hoặc không thể thực thi chúng, hoặc việc thực thi này không hiển thị với các bên tham gia khác, hoặc nó không thể gửi hay nhận các thông điệp đến và đi từ các bên tham gia khác. Lỗi bỏ sót nắm bắt các phân vùng mạng giữa các tiến trình gây ra bởi các đường liên kết mạng lỗi, sự cố bộ chuyển mạch, hoặc tắc nghẽn mạng. Các phân vùng mạng có thể được biểu diễn dưới dạng việc bỏ sót các thông điệp giữa các tiến trình riêng lẻ hoặc các nhóm tiến trình. Một sự sập có thể được mô phỏng bằng cách hoàn toàn bỏ sót bất kỳ thông điệp nào đến và đi từ tiến trình.

When the process is operating slower than the other participants and sends responses much later than expected, for the rest of the system it may look like it is forgetful. Instead of stopping completely, a slow node attempts to send its results out of sync with other nodes.

Khi tiến trình đang hoạt động chậm hơn các bên tham gia khác và gửi phản hồi muộn hơn nhiều so với kỳ vọng, thì đối với phần còn lại của hệ thống, nó có thể trông như đang đờ đẫn. Thay vì dừng hẳn, một nút chậm cố gắng gửi các kết quả của nó ra ngoài một cách không đồng bộ với các nút khác.

Omission failures occur when the algorithm that was supposed to execute certain steps either skips them or the results of this execution are not visible. For example, this may happen if the message is lost on the way to the recipient, and the sender fails to send it again and continues to operate as if it was successfully delivered, even though it was irrecoverably lost. Omission failures can also be caused by intermittent hangs, overloaded networks, full queues, etc.

Các lỗi bỏ sót xảy ra khi thuật toán lẽ ra phải thực thi một số bước nhất định lại bỏ qua chúng hoặc các kết quả của việc thực thi này không hiển thị. Ví dụ, điều này có thể xảy ra nếu thông điệp bị mất trên đường tới bên nhận, và bên gửi không gửi lại nó và tiếp tục hoạt động như thể nó đã được chuyển tới thành công, mặc dù nó đã bị mất không thể khôi phục. Các lỗi bỏ sót cũng có thể gây ra bởi những lần treo gián đoạn, các mạng bị quá tải, các hàng đợi đầy, v.v.

Arbitrary Faults — Lỗi tùy tiện

The hardest class of failures to overcome is arbitrary or Byzantine faults: a process continues executing the algorithm steps, but in a way that contradicts the algorithm (for example, if a process in a consensus algorithm decides on a value that no other **participant** has ever proposed).

Lớp lỗi khó vượt qua nhất là các lỗi tùy tiện hay lỗi Byzantine: một tiến trình tiếp tục thực thi các bước của thuật toán, nhưng theo một cách mâu thuẫn với thuật toán (ví dụ, nếu một tiến trình trong một thuật toán đồng thuận quyết định trên một giá trị mà chưa bên tham gia nào từng đề xuất).

Such failures can happen due to bugs in software, or due to processes running different versions of the algorithm, in which case failures are easier to find and understand. It can get much more difficult when we do not have control over all processes, and one of the processes is intentionally misleading other processes.

Những lỗi như vậy có thể xảy ra do các lỗi trong phần mềm, hoặc do các tiến trình đang chạy những phiên bản khác nhau của thuật toán, trong trường hợp đó lỗi dễ tìm và dễ hiểu hơn. Nó có thể trở nên khó khăn hơn nhiều khi ta không có quyền kiểm soát tất cả các tiến trình, và một trong các tiến trình đang cố tình đánh lừa các tiến trình khác.

You might have heard of **Byzantine fault tolerance** from the airspace industry: airplane and spacecraft systems do not take responses from subcomponents at face value and cross-validate their results. Another widespread application is cryptocurrencies [GILAD17], where there is no central authority, different parties control the nodes, and adversary participants have a material incentive to forge values and attempt to game the system by providing faulty responses.

Bạn có thể đã nghe nói về dung lỗi Byzantine từ ngành hàng không vũ trụ: các hệ thống máy bay và tàu vũ trụ không tiếp nhận các phản hồi từ các bộ phận con theo đúng bề ngoài của chúng mà kiểm chứng chéo các kết quả của chúng. Một ứng dụng phổ biến khác là tiền mã hóa [GILAD17], nơi không có thực thể quản lý trung tâm, các bên khác nhau kiểm soát các nút, và các bên tham gia đối nghịch có động cơ vật chất để giả mạo các giá trị và cố gắng lũng đoạn hệ thống bằng cách cung cấp các phản hồi lỗi.

Handling Failures — Xử lý lỗi

We can mask failures by forming process groups and introducing redundancy into the algorithm: even if one of the processes fails, the user will not notice this failure [CHRISTIAN91].

Ta có thể che giấu các lỗi bằng cách hình thành các nhóm tiến trình và đưa sự dư thừa vào thuật toán: ngay cả khi một trong các tiến trình hỏng, người dùng sẽ không nhận ra lỗi này [CHRISTIAN91].

There might be some performance penalty related to failures: normal execution relies on processes being responsive, and the system has to fall back to the slower execution path for error handling and correction. Many failures can be prevented on the software level by code reviews, extensive testing, ensuring message delivery by introducing timeouts and retries, and making sure that steps are executed in order locally.

Có thể có một số tổn thất hiệu năng liên quan đến lỗi: việc thực thi bình thường dựa vào việc các tiến trình có khả năng đáp ứng, và hệ thống phải lui về con đường thực thi chậm hơn để xử lý và sửa lỗi. Nhiều lỗi có thể được ngăn chặn ở mức phần mềm bằng cách rà soát mã (code review), kiểm thử rộng rãi, đảm bảo việc chuyển tin bằng cách đưa vào các thời gian chờ và các lần thử lại, và đảm bảo rằng các bước được thực thi theo đúng thứ tự tại chỗ.

Most of the algorithms we're going to cover here assume the crash-failure model and work around failures by introducing redundancy. These assumptions help to create algorithms that perform better and are easier to understand and implement.

Hầu hết các thuật toán mà ta sắp đề cập ở đây đều giả định mô hình lỗi-sập và xoay xở với các lỗi bằng cách đưa vào sự dư thừa. Những giả định này giúp tạo ra các thuật toán có hiệu năng tốt hơn và dễ hiểu, dễ triển khai hơn.

Summary — Tóm tắt

In this chapter, we discussed some of the distributed systems terminology and introduced some basic concepts. We've talked about the inherent difficulties and complications caused by the unreliability of the system components: links may fail to deliver messages, processes may crash, or the network may get partitioned.

Trong chương này, chúng ta đã thảo luận một số thuật ngữ của hệ phân tán và giới thiệu một số khái niệm cơ bản. Chúng ta đã nói về những khó khăn và phức tạp cố hữu gây ra

bởi sự không tin cậy của các thành phần hệ thống: các kênh liên kết có thể thất bại trong việc chuyển thông điệp, các tiến trình có thể sập, hoặc mạng có thể bị phân vùng.

This terminology should be enough for us to continue the discussion. The rest of the book talks about the solutions commonly used in distributed systems: we think back to what can go wrong and see what options we have available.

Bộ thuật ngữ này hẳn là đủ để ta tiếp tục cuộc thảo luận. Phần còn lại của cuốn sách bàn về các giải pháp thường được dùng trong hệ phân tán: ta nghĩ lại về những gì có thể trục trặc và xem ta có sẵn những lựa chọn nào.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Distributed systems abstractions, failure models, and timing assumptions Lynch, Nancy A. 1996. Distributed Algorithms . San Francisco: Morgan Kaufmann.

Tanenbaum, Andrew S. and Maarten van Steen. 2006. Distributed Systems: Principles and Paradigms (2nd Ed). Boston: Pearson.

Cachin, Christian, Rachid Guerraoui, and Lus Rodrigues. 2011. Introduction to Reliable and Secure Distributed Programming (2nd Ed.). New York: Springer.

CHAPTER 9 Failure Detection —

Chương 9: Phát hiện lỗi

If a tree falls in a forest and no one is around to hear it, does it make a sound? —Unknown Author

Nếu một cái cây đổ trong rừng mà không có ai ở quanh để nghe, thì nó có tạo ra âm thanh không? —Tác giả khuyết danh

In order for a system to appropriately react to failures, failures should be detected in a timely manner. A faulty **process** might get contacted even though it won't be able to respond, increasing latencies and reducing overall system availability.

Để một hệ thống phản ứng đúng đắn với các lỗi, lỗi cần được phát hiện kịp thời. Một tiến trình bị hỏng có thể vẫn bị liên hệ tới ngay cả khi nó không thể phản hồi, làm tăng độ trễ (latency) và giảm tính sẵn sàng tổng thể của hệ thống.

Detecting failures in **asynchronous** distributed systems (i.e., without making any timing assumptions) is extremely difficult as it's impossible to tell whether the process has crashed, or is running slowly and taking an indefinitely long time to respond. We discussed a problem related to this one in “**FLP Impossibility**” on **page 189**.

Phát hiện lỗi trong các hệ phân tán bất đồng bộ (tức là không đưa ra bất kỳ giả định nào về thời gian) là cực kỳ khó, vì không thể biết được liệu một tiến trình đã sụp đổ hay chỉ đang chạy chậm và mất một khoảng thời gian dài vô hạn định để phản hồi. Chúng ta đã bàn về một vấn đề liên quan tới điều này trong mục “Bất khả thi FLP (FLP Impossibility)” ở trang 189.

Terms such as *dead*, *failed*, and *crashed* are usually used to describe a process that has stopped executing its steps completely. Terms such as *unresponsive*, *faulty*, and *slow* are used to describe suspected processes, which may actually be dead.

Các thuật ngữ như *dead* (chết), *failed* (hỏng) và *crashed* (sụp đổ) thường được dùng để mô tả một tiến trình đã ngừng hẳn việc thực thi các bước của nó. Các thuật ngữ như *unresponsive* (không phản hồi), *faulty* (bị lỗi) và *slow* (chậm) được dùng để mô tả những tiến trình bị nghi ngờ, mà thực ra có thể đã chết.

Failures may occur on the **link** level (messages between processes are lost or delivered slowly), or on the process level (the process crashes or is running slowly), and slowness may not always be distinguishable from failure. This means there's always a trade-off between wrongly suspecting alive processes as dead (producing false-positives), and delaying marking an unresponsive process as dead, giving it the benefit of doubt and expecting it to respond eventually (producing false-negatives).

Lỗi có thể xảy ra ở mức kênh (kênh liên kết — các thông điệp giữa các tiến trình bị mất hoặc được chuyển giao chậm), hoặc ở mức tiến trình (tiến trình sụp đổ hoặc đang chạy chậm), và sự chậm chạp không phải lúc nào cũng phân biệt được với lỗi. Điều này nghĩa là luôn tồn tại một sự đánh đổi giữa việc nghi ngờ sai một tiến trình còn sống là đã chết (tạo ra *dương tính giả* — false-positive), và việc trì hoãn đánh dấu một tiến trình không phản hồi là đã chết, cho nó hưởng lợi từ sự nghi ngờ và kỳ vọng nó cuối cùng sẽ phản hồi (tạo ra *âm tính giả* — false-negative).

A **failure detector** is a local subsystem responsible for identifying failed or unreachable processes to exclude them from the algorithm and guarantee liveness while preserving safety.

Một bộ phát hiện lỗi (failure detector) là một phân hệ cục bộ chịu trách nhiệm nhận dạng các tiến trình bị hỏng hoặc không thể tiếp cận để loại chúng khỏi thuật toán, đảm bảo tính tiến triển (liveness) trong khi vẫn giữ tính an toàn (safety).

Liveness and safety are the properties that describe an algorithm's ability to solve a specific problem and the correctness of its output. More formally, liveness is a property that guarantees that a specific intended event must occur. For example, if one of

Tính tiến triển (liveness) và tính an toàn (safety) là các thuộc tính mô tả khả năng của một thuật toán trong việc giải một bài toán cụ thể và tính đúng đắn của đầu ra của nó. Một cách hình thức hơn, tính tiến triển là một thuộc tính bảo đảm rằng một sự kiện mong muốn cụ thể chắc chắn phải xảy ra. Ví dụ, nếu một trong các

195 the processes has failed, a failure detector must detect that failure. Safety guarantees that unintended events will not occur. For example, if a failure detector has marked a process as dead, this process had to be, in fact, dead [LAMPORT77] [RAYNAL99] [FREILING11].

195 tiến trình đã hỏng, thì bộ phát hiện lỗi phải phát hiện được sự cố đó. Tính an toàn bảo đảm rằng các sự kiện không mong muốn sẽ không xảy ra. Ví dụ, nếu một bộ phát hiện lỗi đã đánh dấu một tiến trình là đã chết, thì trên thực tế tiến trình đó phải đã chết [LAMPORT77] [RAYNAL99] [FREILING11].

From a practical perspective, excluding failed processes helps to avoid unnecessary work and prevents error propagation and cascading failures, while reducing availability when excluding potentially suspected alive processes.

Từ góc nhìn thực tiễn, việc loại bỏ các tiến trình bị hỏng giúp tránh công việc không cần thiết và ngăn ngừa sự lan truyền lỗi cùng các lỗi dây chuyền (cascading failures), trong khi việc loại bỏ những tiến trình còn sống nhưng bị nghi ngờ lại làm giảm tính sẵn sàng.

Failure-detection algorithms should exhibit several essential properties. First of all, every nonfaulty member should eventually notice the process failure, and the algorithm should be able to make progress and eventually reach its final result. This property is called completeness.

Các thuật toán phát hiện lỗi cần thể hiện một vài thuộc tính thiết yếu. Trước hết, mọi thành viên không bị lỗi cuối cùng phải nhận ra được sự cố của tiến trình, và thuật toán phải có khả năng tiến triển và cuối cùng đạt tới kết quả cuối cùng của nó. Thuộc tính này được gọi là *tính đầy đủ* (completeness).

We can judge the quality of the algorithm by its efficiency : how fast the failure detector can identify process failures. Another way to do this is to look at the accuracy of the algorithm: whether or not the process failure was precisely detected. In other words, an algorithm is not accurate if it falsely accuses a live process of being failed or is not able to detect the existing failures.

Chúng ta có thể đánh giá chất lượng của thuật toán qua *tính hiệu quả* (efficiency): bộ phát hiện lỗi có thể nhận dạng các sự cố tiến trình nhanh tới mức nào. Một cách khác là xét

tính chính xác (accuracy) của thuật toán: liệu sự cố tiến trình có được phát hiện một cách chuẩn xác hay không. Nói cách khác, một thuật toán là không chính xác nếu nó buộc tội sai một tiến trình còn sống là đã hỏng, hoặc không thể phát hiện các sự cố đang tồn tại.

We can think of the relationship between efficiency and accuracy as a tunable parameter: a more efficient algorithm might be less precise, and a more accurate algorithm is usually less efficient. It is provably impossible to build a failure detector that is both accurate and efficient. At the same time, failure detectors are allowed to produce false-positives (i.e., falsely identify live processes as failed and vice versa) [CHAN- DRA96] .

Chúng ta có thể coi mối quan hệ giữa tính hiệu quả và tính chính xác như một tham số điều chỉnh được: một thuật toán hiệu quả hơn có thể kém chuẩn xác hơn, và một thuật toán chính xác hơn thường kém hiệu quả hơn. Có thể chứng minh được rằng không thể xây dựng một bộ phát hiện lỗi vừa chính xác vừa hiệu quả. Đồng thời, các bộ phát hiện lỗi được phép tạo ra dương tính giả (tức là nhận dạng sai các tiến trình còn sống là đã hỏng và ngược lại) [CHANDRA96].

Failure detectors are an essential prerequisite and an integral part of many **consensus** and **atomic broadcast** algorithms, which we'll be discussing later in this book.

Bộ phát hiện lỗi là một điều kiện tiên quyết thiết yếu và là một phần không thể tách rời của nhiều thuật toán đồng thuận (consensus) và quảng bá nguyên tử (atomic broadcast) mà chúng ta sẽ bàn tới ở phần sau của cuốn sách này.

Many distributed systems implement failure detectors by using heartbeats . This approach is quite popular because of its simplicity and strong completeness. Algorithms we discuss here assume the absence of Byzantine failures: processes do not attempt to intentionally lie about their state or states of their neighbors.

Nhiều hệ phân tán hiện thực bộ phát hiện lỗi bằng cách dùng nhịp tim (heartbeat). Cách tiếp cận này khá phổ biến vì tính đơn giản và tính đầy đủ mạnh của nó. Các thuật toán mà chúng ta bàn ở đây giả định không có lỗi Byzantine: các tiến trình không cố tình nói dối về trạng thái của chính nó hay trạng thái của các tiến trình lân cận.

Heartbeats and Pings — Nhịp tim và Ping

We can query the state of remote processes by triggering one of two periodic processes:

Chúng ta có thể truy vấn trạng thái của các tiến trình ở xa bằng cách kích hoạt một trong hai tiến trình định kỳ:

- We can trigger a ping, which sends messages to remote processes, checking if they are still alive by expecting a response within a specified time period.
- We can trigger a **heartbeat** when the process is actively notifying its peers that it's still running by sending messages to them.
 - *Chúng ta có thể kích hoạt một *ping*, tức gửi các thông điệp tới các tiến trình ở xa, kiểm tra xem chúng còn sống hay không bằng cách kỳ vọng có một phản hồi trong một khoảng thời gian xác định.*
 - *Chúng ta có thể kích hoạt một *nhịp tim* (heartbeat) khi tiến trình chủ động thông báo cho các bên ngang hàng rằng nó vẫn đang chạy, bằng cách gửi các thông điệp tới chúng.*

We'll use pings as an example here, but the same problem can be solved using heartbeats, producing similar results.

Ở đây chúng ta sẽ dùng ping làm ví dụ, nhưng cùng một bài toán có thể được giải bằng nhịp tim, cho ra kết quả tương tự.

Each process maintains a list of other processes (alive, dead, and suspected ones) and updates it with the last response time for each process. If a process fails to respond to a ping message for a longer time, it is marked as suspected .

Mỗi tiến trình duy trì một danh sách các tiến trình khác (còn sống, đã chết, và bị nghi ngờ) và cập nhật danh sách đó với thời điểm phản hồi gần nhất của mỗi tiến trình. Nếu một tiến trình không phản hồi một thông điệp ping trong một khoảng thời gian đủ dài, nó sẽ bị đánh dấu là *bị nghi ngờ* (suspected).

Figure 9-1 shows the normal functioning of a system: process P1 is querying the state of neighboring **node** P2 , which responds with an acknowledgment.

Hình 9-1 cho thấy hoạt động bình thường của một hệ thống: tiến trình P1 đang truy vấn trạng thái của nút lân cận P2, nút này phản hồi bằng một xác nhận (acknowledgment).

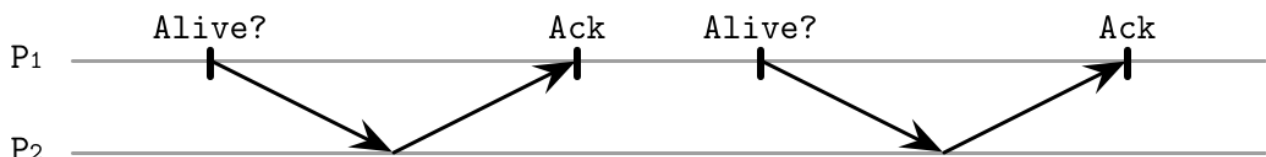


Figure 9-1. Pings for failure detection: normal functioning, no message delays

Hình 9-1. Ping để phát hiện lỗi: hoạt động bình thường, không có trễ thông điệp

In contrast, Figure 9-2 shows how acknowledgment messages are delayed, which might result in marking the active process as down.

Ngược lại, Hình 9-2 cho thấy các thông điệp xác nhận bị trễ như thế nào, điều có thể dẫn tới việc đánh dấu một tiến trình đang hoạt động là đã ngừng.

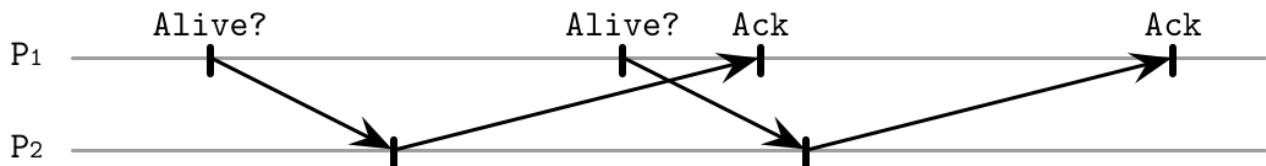


Figure 9-2. Pings for failure detection: responses are delayed, coming after the next message is sent

Hình 9-2. Ping để phát hiện lỗi: các phản hồi bị trễ, đến sau khi thông điệp tiếp theo đã được gửi đi

Many failure-detection algorithms are based on heartbeats and timeouts. For example, Akka, a popular framework for building distributed systems, has an implementation of a deadline failure detector , which uses heartbeats and reports a process failure if it has failed to register within a fixed time interval.

Nhiều thuật toán phát hiện lỗi dựa trên nhịp tim và thời gian chờ (timeout). Ví dụ, Akka — một framework phổ biến để xây dựng các hệ phân tán — có một hiện thực của *bộ phát hiện lỗi theo hạn chót* (deadline failure detector), dùng nhịp tim và báo cáo một sự cố tiến trình nếu tiến trình đó không đăng ký được trong một khoảng thời gian cố định.

This approach has several potential downsides: its precision relies on the careful selection of ping frequency and timeout, and it does not capture process visibility from the perspective of other processes (see “Outsourced Heartbeats” on page 198).

Cách tiếp cận này có vài nhược điểm tiềm tàng: độ chuẩn xác của nó phụ thuộc vào việc lựa chọn cẩn thận tần suất ping và thời gian chờ, và nó không nắm bắt được khả năng nhìn thấy tiến trình từ góc nhìn của các tiến trình khác (xem “Nhịp tim thuê ngoài (Outsourced Heartbeats)” ở trang 198).

Timeout-Free Failure Detector — Bộ phát hiện lỗi không cần thời gian chờ

Some algorithms avoid relying on timeouts for detecting failures. For example, Heartbeat, a timeout-free failure detector [AGUILERA97], is an algorithm that only counts heartbeats and allows the application to detect process failures based on the data in the heartbeat counter vectors. Since this algorithm is timeout-free, it operates under asynchronous system assumptions.

Một số thuật toán tránh phụ thuộc vào thời gian chờ để phát hiện lỗi. Ví dụ, Heartbeat — một bộ phát hiện lỗi không cần thời gian chờ [AGUILERA97] — là một thuật toán chỉ đếm các nhịp tim và cho phép ứng dụng phát hiện các sự cố tiến trình dựa trên dữ liệu trong các vector bộ đếm nhịp tim. Vì thuật toán này không cần thời gian chờ, nó vận hành dưới các giả định của hệ bất đồng bộ.

The algorithm assumes that any two correct processes are connected to each other with a fair path, which contains only fair links (i.e., if a message is sent over this link infinitely often, it is also received infinitely often), and each process is aware of the existence of all other processes in the network.

Thuật toán giả định rằng bất kỳ hai tiến trình đúng đắn nào cũng được kết nối với nhau bằng một *đường công bằng* (fair path), tức đường chỉ chứa các *kênh công bằng* (fair link) (nghĩa là nếu một thông điệp được gửi qua kênh này vô hạn lần thì nó cũng được nhận vô hạn lần), và mỗi tiến trình đều biết về sự tồn tại của tất cả các tiến trình khác trong mạng.

Each process maintains a list of neighbors and counters associated with them. Processes start by sending heartbeat messages to their neighbors. Each message contains a path that the heartbeat has traveled so far. The initial message contains the first sender in the path and a unique identifier that can be used to avoid broadcasting the same message multiple times.

Mỗi tiến trình duy trì một danh sách các tiến trình lân cận và các bộ đếm gắn với chúng. Các tiến trình khởi đầu bằng việc gửi các thông điệp nhịp tim tới các tiến trình lân cận. Mỗi thông điệp chứa một đường đi (path) mà nhịp tim đã đi qua cho tới thời điểm đó. Thông điệp ban đầu chứa bên gửi đầu tiên trong đường đi và một định danh duy nhất có thể dùng để tránh quảng bá cùng một thông điệp nhiều lần.

When the process receives a new heartbeat message, it increments counters for all participants present in the path and sends the heartbeat to the ones that are not present there, appending itself to the path. Processes stop propagating messages as soon as they see that all the known processes have already received it (in other words, process IDs appear in the path).

Khi một tiến trình nhận được một thông điệp nhịp tim mới, nó tăng bộ đếm cho tất cả các bên tham gia có mặt trong đường đi và gửi nhịp tim tới những bên chưa có mặt ở đó, đồng thời tự gắn mình vào đường đi. Các tiến trình ngừng lan truyền thông điệp ngay khi chúng thấy tất cả các tiến trình đã biết đều đã nhận được nó (nói cách khác, các ID tiến trình xuất hiện trong đường đi).

Since messages are propagated through different processes, and heartbeat paths contain aggregated information received from the neighbors, we can (correctly) mark an unreachable process as alive even when the direct link between the two processes is faulty.

Vì các thông điệp được lan truyền qua nhiều tiến trình khác nhau, và các đường đi nhịp tim chứa thông tin tổng hợp nhận được từ các tiến trình lân cận, nên chúng ta có thể (một cách đúng đắn) đánh dấu một tiến trình không thể tiếp cận là còn sống ngay cả khi kênh liên kết trực tiếp giữa hai tiến trình bị lỗi.

Heartbeat counters represent a global and normalized view of the system. This view captures how the heartbeats are propagated relative to one another, allowing us to compare processes. However, one of the shortcomings of this approach is that interpreting heartbeat counters may be quite tricky: we need to pick a threshold that can yield reliable results. Unless we can do that, the algorithm will falsely mark active processes as suspected.

Các bộ đếm nhịp tim biểu diễn một góc nhìn toàn cục và đã chuẩn hóa về hệ thống. Góc nhìn này nắm bắt cách các nhịp tim được lan truyền tương đối so với nhau, cho phép chúng ta so sánh các tiến trình. Tuy nhiên, một trong những điểm yếu của cách tiếp cận này là việc diễn giải các bộ đếm nhịp tim có thể khá rắc rối: chúng ta cần chọn một ngưỡng có thể cho ra kết quả đáng tin cậy. Trừ khi làm được điều đó, thuật toán sẽ đánh dấu sai các tiến trình đang hoạt động là bị nghi ngờ.

Outsourced Heartbeats — Nhịp tim thuê ngoài

An alternative approach, used by the Scalable Weakly Consistent Infection-style Process Group Membership Protocol (SWIM) [GUPTA01] is to use outsourced heartbeats to improve reliability using information about the process liveness from the perspective of its neighbors. This approach does not require processes to be aware of all other processes in the network, only a subset of connected peers.

Một cách tiếp cận thay thế, được dùng bởi Scalable Weakly Consistent Infection-style Process Group Membership Protocol (SWIM) [GUPTA01], là dùng nhịp tim thuê ngoài (outsourced heartbeat) để cải thiện độ tin cậy bằng cách dùng thông tin về tính còn sống của tiến trình từ góc nhìn của các tiến trình lân cận. Cách tiếp cận này không đòi hỏi các tiến trình phải biết về tất cả các tiến trình khác trong mạng, mà chỉ cần một tập con các bên ngang hàng được kết nối.

As shown in Figure 9-3, process P sends a ping message to process P. P doesn't

Như Hình 9-3 cho thấy, tiến trình P gửi một thông điệp ping tới tiến trình P. P không respond to the message, so P proceeds by selecting multiple random members (P

phản hồi thông điệp đó, nên P tiếp tục bằng cách chọn nhiều thành viên ngẫu nhiên (P and P). These random members try sending heartbeat messages to P and, if it

và P). Các thành viên ngẫu nhiên này thử gửi các thông điệp nhịp tim tới P và, nếu P responds, forward acknowledgments back to P.

phản hồi, chuyển tiếp các xác nhận trở lại cho P.

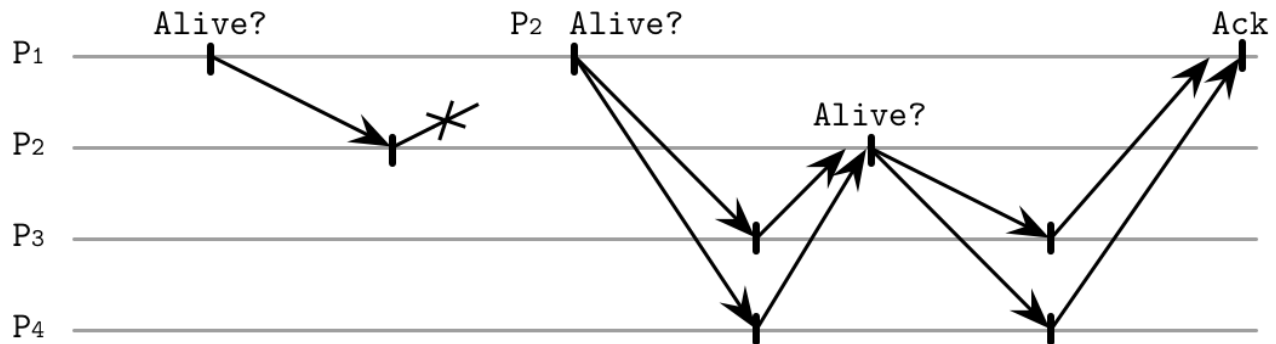


Figure 9-3. “Outsourcing” heartbeats

Hình 9-3. “Thuê ngoài” nhịp tim

This allows accounting for both direct and indirect reachability. For example, if we have processes P_1 , P_2 , and P_3 , we can check the state of P_3 from the perspective of both

Điều này cho phép tính đến cả khả năng tiếp cận trực tiếp lẫn gián tiếp. Ví dụ, nếu ta có các tiến trình P_1 , P_2 và P_3 , ta có thể kiểm tra trạng thái của P_3 từ góc nhìn của cả

P_1 and P_2 .

P_1 và P_2 .

Outsourced heartbeats allow reliable failure detection by distributing responsibility for deciding across the group of members. This approach does not require broadcasting messages to a broad group of peers. Since outsourced heartbeat requests can be triggered in parallel, this approach can collect more information about suspected processes quickly, and allow us to make more accurate decisions.

Nhịp tim thuê ngoài cho phép phát hiện lỗi đáng tin cậy bằng cách phân tán trách nhiệm ra quyết định cho cả nhóm thành viên. Cách tiếp cận này không đòi hỏi phải quảng bá thông điệp tới một nhóm rộng các bên ngang hàng. Vì các yêu cầu nhịp tim thuê ngoài có thể được kích hoạt song song, cách tiếp cận này có thể nhanh chóng thu thập được nhiều thông tin hơn về các tiến trình bị nghi ngờ, và cho phép chúng ta đưa ra các quyết định chính xác hơn.

Phi-Accrual Failure Detector — Bộ phát hiện lỗi Phi-accrual

Instead of treating node failure as a binary problem, where the process can be only in two states: up or down, a phi-accrual (ϕ -accrual) failure detector [HAYASHIBARA04] has a continuous scale, capturing the probability of the monitored process’s crash. It works by maintaining a sliding window, collecting arrival times of the most recent heartbeats from the peer processes. This information is used to approximate arrival time of the next heartbeat, compare this approximation with the actual arrival time, and compute the suspicion level ϕ : how certain the failure detector is about the failure, given the current network conditions.

Thay vì coi sự cố nút là một bài toán nhị phân, trong đó tiến trình chỉ có thể ở một trong hai trạng thái — sống hoặc chết — một bộ phát hiện lỗi phi-accrual (ϕ -accrual) [HAYASHIBARA04] có một thang đo liên tục, nắm bắt xác suất sụp đổ của tiến trình được giám sát. Nó hoạt động bằng cách duy trì một cửa sổ trượt, thu thập các thời điểm đến của những nhịp tim gần nhất từ các tiến trình ngang hàng. Thông tin này được dùng để xấp xỉ thời điểm đến của nhịp tim tiếp theo, so sánh giá trị xấp xỉ này với thời điểm đến

thực tế, và tính ra mức nghi ngờ ϕ : bộ phát hiện lỗi chắc chắn tới mức nào về sự cố, với các điều kiện mạng hiện tại.

The algorithm works by collecting and sampling arrival times, creating a view that can be used to make a reliable judgment about node health. It uses these samples to compute the value of ϕ : if this value reaches a threshold, the node is marked as down. This failure detector dynamically adapts to changing network conditions by adjusting the scale on which the node can be marked as a suspect.

Thuật toán hoạt động bằng cách thu thập và lấy mẫu các thời điểm đến, tạo ra một góc nhìn có thể dùng để đưa ra phán đoán đáng tin cậy về sức khỏe của nút. Nó dùng các mẫu này để tính giá trị ϕ : nếu giá trị này chạm ngưỡng, nút sẽ bị đánh dấu là đã ngừng. Bộ phát hiện lỗi này thích nghi động với các điều kiện mạng đang thay đổi bằng cách điều chỉnh thang đo mà trên đó nút có thể bị đánh dấu là bị nghi ngờ.

From the architecture perspective, a **phi-accrual failure detector** can be viewed as a combination of three subsystems:

Từ góc nhìn kiến trúc, một bộ phát hiện lỗi phi-accrual có thể được xem như sự kết hợp của ba phân hệ:

Monitoring Collecting liveness information through pings, heartbeats, or request-response sampling.

Giám sát (Monitoring) Thu thập thông tin về tính còn sống thông qua ping, nhịp tim, hoặc lấy mẫu yêu cầu-phản hồi.

Interpretation Making a decision on whether or not the process should be marked as suspected.

Diễn giải (Interpretation) Ra quyết định về việc tiến trình có nên bị đánh dấu là bị nghi ngờ hay không.

Action A callback executed whenever the process is marked as suspected.

Hành động (Action) Một callback được thực thi mỗi khi tiến trình bị đánh dấu là bị nghi ngờ.

The monitoring process collects and stores data samples (which are assumed to follow a normal distribution) in a fixed-size window of heartbeat arrival times. Newer arrivals are added to the window, and the oldest heartbeat data points are discarded.

Tiến trình giám sát thu thập và lưu trữ các mẫu dữ liệu (vốn được giả định là tuân theo phân phối chuẩn) trong một cửa sổ kích thước cố định gồm các thời điểm đến của nhịp tim. Các lần đến mới hơn được thêm vào cửa sổ, và các điểm dữ liệu nhịp tim cũ nhất bị loại bỏ.

Distribution parameters are estimated from the sampling window by determining the mean and variance of samples. This information is used to compute the probability of arrival of the message within t time units after the previous one. Given this information, we compute ϕ , which describes how likely we are to make a correct decision about a process's liveness. In other words, how likely it is to make a mistake and receive a heartbeat that will contradict the calculated assumptions.

Các tham số phân phối được ước lượng từ cửa sổ lấy mẫu bằng cách xác định kỳ vọng (mean) và phương sai (variance) của các mẫu. Thông tin này được dùng để tính xác suất một thông điệp đến trong vòng t đơn vị thời gian sau thông điệp trước đó. Với thông tin này, chúng ta tính ϕ , đại lượng mô tả mức độ chúng ta có khả năng đưa ra một quyết định đúng đắn về tính còn sống của một tiến trình. Nói cách khác, mức độ có khả năng phạm sai lầm và nhận được một nhịp tim mâu thuẫn với các giả định đã tính toán.

This approach was developed by researchers from the Japan Advanced Institute of Science and Technology, and is now used in many distributed systems; for example, Cassandra and Akka (along with the aforementioned deadline failure detector).

Cách tiếp cận này được phát triển bởi các nhà nghiên cứu từ Japan Advanced Institute of Science and Technology, và nay được dùng trong nhiều hệ phân tán; ví dụ, Cassandra và Akka (cùng với bộ phát hiện lỗi theo hạn chót đã nhắc tới ở trên).

Gossip and Failure Detection — Gossip và phát hiện lỗi

Another approach that avoids relying on a single-node view to make a decision is a gossip-style failure detection service [VANRENESSE98], which uses gossip (see “Gossip **Dissemination**” on page 250) to collect and distribute states of neighboring processes.

Một cách tiếp cận khác tránh phụ thuộc vào góc nhìn của một nút đơn lẻ để ra quyết định là dịch vụ phát hiện lỗi kiểu gossip (gossip-style failure detection service) [VANRENESSE98], vốn dùng gossip (xem “Lan truyền Gossip (Gossip Dissemination)” ở trang 250) để thu thập và phân phối trạng thái của các tiến trình lân cận.

Each member maintains a list of other members, their heartbeat counters, and timestamps, specifying when the heartbeat counter was incremented for the last time. Periodically, each member increments its heartbeat counter and distributes its list to a random neighbor. Upon the message receipt, the neighboring node merges the list with its own, updating heartbeat counters for the other neighbors.

Mỗi thành viên duy trì một danh sách các thành viên khác, các *bộ đếm nhịp tim* của chúng, và các nhãn thời gian (timestamp) cho biết lần cuối cùng bộ đếm nhịp tim được tăng là khi nào. Định kỳ, mỗi thành viên tăng bộ đếm nhịp tim của nó và phân phối danh sách của nó tới một tiến trình lân cận ngẫu nhiên. Khi nhận được thông điệp, nút lân cận gộp danh sách đó với danh sách của chính mình, cập nhật các bộ đếm nhịp tim cho các tiến trình lân cận khác.

Nodes also periodically check the list of states and heartbeat counters. If any node did not update its counter for long enough, it is considered failed. This timeout period should be chosen carefully to minimize the probability of false-positives. How often members have to communicate with each other (in other words, worst-case bandwidth) is capped, and can grow at most linearly with a number of processes in the system.

Các nút cũng định kỳ kiểm tra danh sách trạng thái và các bộ đếm nhịp tim. Nếu một nút nào đó không cập nhật bộ đếm của nó trong đủ lâu, nó bị coi là đã hỏng. Khoảng thời gian chờ này cần được chọn cẩn thận để giảm thiểu xác suất dương tính giả. Tần suất các thành viên phải giao tiếp với nhau (nói cách khác, băng thông trong trường hợp xấu nhất) bị giới hạn trần, và tối đa chỉ có thể tăng tuyến tính theo số tiến trình trong hệ thống.

Figure 9-4 shows three communicating processes sharing their heartbeat counters:

Hình 9-4 cho thấy ba tiến trình đang giao tiếp chia sẻ các bộ đếm nhịp tim của chúng:

- a) All three can communicate and update their timestamps.
- b) P3 isn’t able to communicate with P1, but its timestamp t can still be propagated.
 - a) Cả ba đều có thể giao tiếp và cập nhật các nhãn thời gian của chúng.
 - b) P3 không thể giao tiếp với P1, nhưng nhãn thời gian t của nó vẫn có thể được lan truyền

ted through P2 .

qua P2.

- c) P3 crashes. Since it doesn't send updates anymore, it is detected as failed by other processes.

- c) P3 sập đổ. Vì nó không còn gửi cập nhật nữa, nó bị các tiến trình khác phát hiện là đã hỏng.

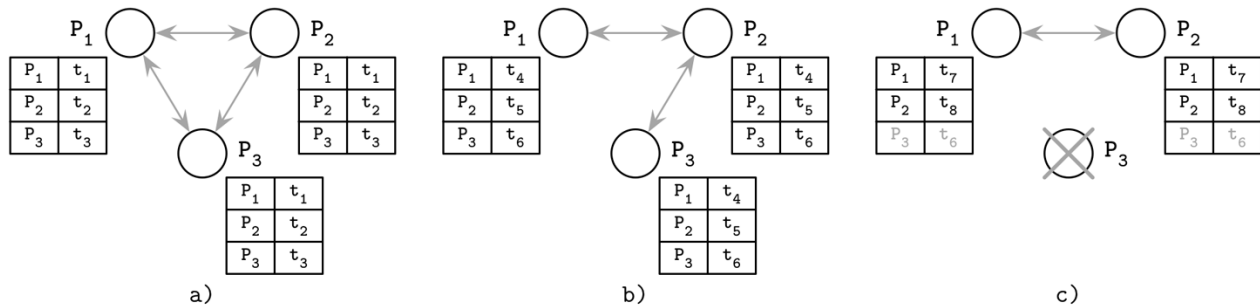


Figure 9-4. Replicated heartbeat table for failure detection

Hình 9-4. Bảng nhịp tim được nhân bản để phát hiện lỗi

This way, we can detect crashed nodes, as well as the nodes that are unreachable by any other cluster member. This decision is reliable, since the view of the cluster is an aggregate from multiple nodes. If there's a link failure between the two hosts, heartbeats can still propagate through other processes. Using gossip for propagating system states increases the number of messages in the system, but allows information to spread more reliably.

Bằng cách này, chúng ta có thể phát hiện các nút đã sập đổ, cũng như các nút không thể tiếp cận được bởi bất kỳ thành viên cụm nào khác. Quyết định này là đáng tin cậy, vì góc nhìn về cụm là một tổng hợp từ nhiều nút. Nếu có một sự cố kênh liên kết giữa hai máy chủ, các nhịp tim vẫn có thể lan truyền qua các tiến trình khác. Việc dùng gossip để lan truyền trạng thái hệ thống làm tăng số lượng thông điệp trong hệ thống, nhưng cho phép thông tin lan tỏa một cách đáng tin cậy hơn.

Reversing Failure Detection Problem Statement — Đảo ngược cách phát biểu bài toán phát hiện lỗi

Since propagating the information about failures is not always possible, and propagating it by notifying every member might be expensive, one of the approaches, called FUSE (failure notification service) [DUNAGAN04] , focuses on reliable and cheap failure propagation that works even in cases of network partitions.

Vì việc lan truyền thông tin về các sự cố không phải lúc nào cũng khả thi, và việc lan truyền nó bằng cách thông báo cho mọi thành viên có thể tốn kém, nên một trong các cách tiếp cận — gọi là FUSE (dịch vụ thông báo lỗi — failure notification service) [DUNAGAN04] — tập trung vào việc lan truyền lỗi một cách đáng tin cậy và rẻ, hoạt động được ngay cả trong các trường hợp phân vùng mạng (network partition).

To detect process failures, this approach arranges all active processes in groups. If one of the groups becomes unavailable, all participants detect the failure. In other words, every time a single process

failure is detected, it is converted and propagated as a group failure . This allows detecting failures in the presence of any pattern of disconnects, partitions, and node failures.

Để phát hiện các sự cố tiến trình, cách tiếp cận này sắp xếp tất cả các tiến trình đang hoạt động thành các nhóm. Nếu một trong các nhóm trở nên không sẵn sàng, tất cả các bên tham gia đều phát hiện ra sự cố. Nói cách khác, mỗi khi một sự cố tiến trình đơn lẻ được phát hiện, nó được chuyển đổi và lan truyền thành một *sự cố nhóm* (group failure). Điều này cho phép phát hiện sự cố trong sự hiện diện của bất kỳ kiểu ngắt kết nối, phân vùng và sự cố nút nào.

Processes in the group periodically send ping messages to other members, querying whether they're still alive. If one of the members cannot respond to this message because of a crash, **network partition**, or link failure, the member that has initiated this ping will, in turn, stop responding to ping messages itself.

Các tiến trình trong nhóm định kỳ gửi các thông điệp ping tới các thành viên khác, truy vấn xem chúng có còn sống hay không. Nếu một trong các thành viên không thể phản hồi thông điệp này vì sập đổ, phân vùng mạng, hoặc sự cố kênh liên kết, thì thành viên đã khởi xướng ping này đến lượt mình cũng sẽ ngừng phản hồi các thông điệp ping.

Figure 9-5 shows four communicating processes:

Hình 9-5 cho thấy bốn tiến trình đang giao tiếp:

- a) Initial state: all processes are alive and can communicate.
- b) P crashes and stops responding to ping messages.
 - a) Trạng thái ban đầu: tất cả các tiến trình đều còn sống và có thể giao tiếp.
 - b) P sập đổ và ngừng phản hồi các thông điệp ping.
- c) P detects the failure of P and stops responding to ping messages itself.
 - c) P phát hiện sự cố của P và đến lượt mình cũng ngừng phản hồi các thông điệp ping.
- d) Eventually, P and P notice that both P and P do not respond, and process
 - d) Cuối cùng, P và P nhận ra rằng cả P và P đều không phản hồi, và sự cố tiến trình

failure propagates to the entire group.

lan truyền ra toàn bộ nhóm.

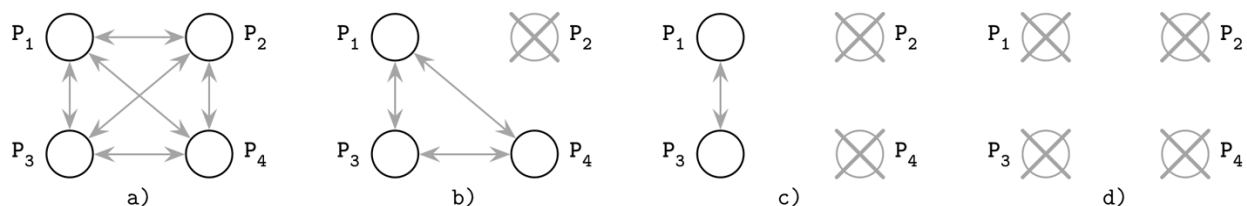


Figure 9-5. FUSE failure detection

Hình 9-5. Phát hiện lỗi FUSE

All failures are propagated through the system from the source of failure to all other participants. Participants gradually stop responding to pings, converting from the individual node failure to the group failure.

Tất cả các sự cố được lan truyền qua hệ thống từ nguồn của sự cố tới tất cả các bên tham gia khác. Các bên tham gia dần dần ngừng phản hồi các ping, chuyển đổi từ sự cố nút đơn lẻ thành sự cố nhóm.

Here, we use the absence of communication as a means of propagation. An advantage of using this approach is that every member is guaranteed to learn about group failure and adequately react to it. One of the downsides is that a link failure separating a single process from other ones can be converted to the group failure as well, but this can be seen as an advantage, depending on the use case. Applications can use their own definitions of propagated failures to account for this scenario.

Ở đây, chúng ta dùng sự vắng mặt của giao tiếp như một phương tiện lan truyền. Một ưu điểm của cách tiếp cận này là mọi thành viên được bảo đảm sẽ biết về sự cố nhóm và phản ứng thỏa đáng với nó. Một trong các nhược điểm là một sự cố kênh liên kết tách một tiến trình đơn lẻ khỏi các tiến trình khác cũng có thể bị chuyển đổi thành sự cố nhóm, nhưng điều này có thể được xem là một ưu điểm, tùy thuộc vào tình huống sử dụng. Các ứng dụng có thể dùng định nghĩa riêng của chúng về các sự cố được lan truyền để tính đến tình huống này.

Summary — Tóm tắt

Failure detectors are an essential part of any **distributed system**. As shown by the FLP Impossibility result, no protocol can guarantee consensus in an asynchronous system. Failure detectors help to augment the model, allowing us to solve a consensus problem by making a trade-off between accuracy and completeness. One of the significant findings in this area, proving the usefulness of failure detectors, was described in [CHANDRA96], which shows that solving consensus is possible even with a failure detector that makes an infinite number of mistakes.

Bộ phát hiện lỗi là một phần thiết yếu của bất kỳ hệ phân tán nào. Như đã được chỉ ra bởi kết quả Bất khả thi FLP, không có giao thức nào có thể bảo đảm sự đồng thuận trong một hệ bất đồng bộ. Các bộ phát hiện lỗi giúp bổ trợ cho mô hình, cho phép chúng ta giải bài toán đồng thuận bằng cách thực hiện một sự đánh đổi giữa tính chính xác và tính đầy đủ. Một trong những phát hiện quan trọng trong lĩnh vực này, chứng minh sự hữu ích của các bộ phát hiện lỗi, được mô tả trong [CHANDRA96], cho thấy rằng việc giải bài toán đồng thuận là khả thi ngay cả với một bộ phát hiện lỗi phạm vô hạn số lần sai lầm.

We've covered several algorithms for failure detection, each using a different approach: some focus on detecting failures by direct communication, some use broadcast or gossip for spreading the information around, and some opt out by using quiescence (in other words, absence of communication) as a means of propagation. We now know that we can use heartbeats or pings, hard deadlines, or continuous scales. Each one of these approaches has its own upsides: simplicity, accuracy, or precision.

Chúng ta đã đề cập tới một vài thuật toán phát hiện lỗi, mỗi thuật toán dùng một cách tiếp cận khác nhau: một số tập trung vào việc phát hiện sự cố qua giao tiếp trực tiếp, một số dùng quảng bá hoặc gossip để lan tỏa thông tin xung quanh, và một số chọn cách dùng sự tĩnh lặng (quiescence) — nói cách khác, sự vắng mặt của giao tiếp — như một phương tiện lan truyền. Giờ chúng ta đã biết rằng có thể dùng nhịp tim hoặc ping, các hạn chót cứng, hoặc các thang đo liên tục. Mỗi cách tiếp cận này đều có ưu điểm riêng: tính đơn giản, tính chính xác, hoặc độ chuẩn xác.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Failure detection and algorithms Chandra, Tushar Deepak and Sam Toueg. 1996. "Unreliable failure detectors for reliable distributed systems." *Journal of the ACM* 43, no. 2 (March): 225-267. <https://doi.org/10.1145/226643.226647> .

Freiling, Felix C., Rachid Guerraoui, and Petr Kuznetsov. 2011. "The failure detector abstraction." *ACM Computing Surveys* 43, no. 2 (January): Article 9. <https://doi.org/10.1145/1883612.1883616> .

Phan-Ba, Michael. 2015. "A literature review of failure detection within the context of solving the problem of distributed consensus." https://www.cs.ubc.ca/~best_chai/theses/michael-phan-ba-msc-essay-2015.pdf

CHAPTER 10 Leader Election —

Chương 10: Bầu chọn Leader

Synchronization can be quite costly: if each algorithm step involves contacting each other **participant**, we can end up with a significant communication overhead. This is particularly true in large and geographically distributed networks. To reduce synchronization overhead and the number of message round-trips required to reach a decision, some algorithms rely on the existence of the **leader** (sometimes called **coordinator**) **process**, responsible for executing or coordinating steps of a distributed algorithm.

Đồng bộ hóa có thể khá tốn kém: nếu mỗi bước của thuật toán đều phải liên lạc với tất cả các bên tham gia (participant) khác, ta có thể gánh chịu chi phí truyền thông đáng kể. Điều này đặc biệt đúng trong các mạng lớn và phân tán theo địa lý. Để giảm chi phí đồng bộ hóa và số vòng truyền thông điệp (message round-trip) cần thiết để đi đến quyết định, một số thuật toán dựa vào sự tồn tại của một tiến trình leader (đôi khi gọi là tiến trình điều phối, coordinator), chịu trách nhiệm thực thi hoặc điều phối các bước của một thuật toán phân tán.

Generally, processes in distributed systems are uniform, and any process can take over the leadership role. Processes assume leadership for long periods of time, but this is not a permanent role. Usually, the process remains a leader until it crashes. After the crash, any other process can start a new election round, assume leadership, if it gets elected, and continue the failed leader's work.

Nhìn chung, các tiến trình trong hệ phân tán là đồng nhất, và bất kỳ tiến trình nào cũng có thể đảm nhận vai trò leader. Các tiến trình giữ vai trò leader trong thời gian dài, nhưng đây không phải là vai trò vĩnh viễn. Thông thường, một tiến trình vẫn là leader cho đến khi nó sập (crash). Sau khi sập, bất kỳ tiến trình nào khác cũng có thể khởi động một vòng bầu chọn mới, đảm nhận vai trò leader nếu được bầu, và tiếp tục công việc của leader đã hỏng.

The liveness of the election algorithm guarantees that most of the time there will be a leader, and the election will eventually complete (i.e., the system should not be in the election state indefinitely).

Tính sống (liveness) của thuật toán bầu chọn đảm bảo rằng hầu hết thời gian sẽ luôn có một leader, và việc bầu chọn rồi sẽ hoàn tất (nghĩa là hệ thống không nên ở trạng thái đang bầu chọn vô thời hạn).

Ideally, we'd like to assume safety, too, and guarantee there may be at most one leader at a time, and completely eliminate the possibility of a **split brain** situation (when two leaders serving the same purpose are elected but unaware of each other). However, in practice, many **leader election** algorithms violate this agreement.

Lý tưởng nhất, ta cũng muốn giả định cả tính an toàn (safety) nữa, đảm bảo tại mỗi thời

điểm có nhiều nhất một leader, và loại trừ hoàn toàn khả năng xảy ra tình huống split brain (khi hai leader phục vụ cùng một mục đích cùng được bầu nhưng không biết về sự tồn tại của nhau). Tuy nhiên, trong thực tế, nhiều thuật toán bầu chọn leader vi phạm thỏa thuận này.

Leader processes can be used, for example, to achieve a total order of messages in a broadcast. The leader collects and holds the global state, receives messages, and disseminates them among the processes. It can also be used to coordinate system reorganization after the failure, during initialization, or when important state changes happen.

Các tiến trình leader có thể được dùng, chẳng hạn, để đạt được thứ tự toàn cục của các thông điệp trong một phép quảng bá (broadcast). Leader thu thập và nắm giữ trạng thái toàn cục, nhận các thông điệp, và lan truyền (disseminate) chúng giữa các tiến trình. Nó cũng có thể được dùng để điều phối việc tái tổ chức hệ thống sau khi xảy ra lỗi, trong lúc khởi tạo, hoặc khi có các thay đổi trạng thái quan trọng xảy ra.

205 Election is triggered when the system initializes, and the leader is elected for the first time, or when the previous leader crashes or fails to communicate. Election has to be deterministic: exactly one leader has to emerge from the process. This decision needs to be effective for all participants.

Việc bầu chọn được kích hoạt khi hệ thống khởi tạo và leader được bầu lần đầu tiên, hoặc khi leader trước đó bị sập hoặc không liên lạc được. Việc bầu chọn phải có tính tất định: phải có đúng một leader xuất hiện từ quá trình này. Quyết định này cần có hiệu lực với tất cả các bên tham gia.

Even though leader election and distributed locking (i.e., exclusive ownership over a shared resource) might look alike from a theoretical perspective, they are slightly different. If one process holds a **lock** for executing a critical section, it is unimportant for other processes to know who exactly is holding a lock right now, as long as the liveness property is satisfied (i.e., the lock will be eventually released, allowing others to acquire it). In contrast, the elected process has some special properties and has to be known to all other participants, so the newly elected leader has to notify its peers about its role.

Mặc dù bầu chọn leader và khóa phân tán (distributed locking, tức quyền sở hữu độc quyền trên một tài nguyên dùng chung) có thể trông giống nhau từ góc nhìn lý thuyết, chúng vẫn hơi khác nhau. Nếu một tiến trình giữ một khóa (lock) để thực thi một vùng găng (critical section), thì việc các tiến trình khác biết chính xác ai đang giữ khóa lúc này là không quan trọng, miễn là tính sống được thỏa mãn (tức khóa rồi sẽ được giải phóng, cho phép tiến trình khác giành lấy nó). Ngược lại, tiến trình được bầu có một số thuộc tính đặc biệt và phải được tất cả các bên tham gia khác biết đến, nên leader mới được bầu phải thông báo cho các tiến trình ngang hàng (peer) của nó về vai trò của mình.

If a distributed locking algorithm has any sort of preference toward some process or group of processes, it will eventually starve nonpreferred processes from the shared resource, which contradicts the liveness property. In contrast, the leader can remain in its role until it stops or crashes, and long-lived leaders are preferred.

Nếu một thuật toán khóa phân tán có bất kỳ sự thiên vị nào đối với một tiến trình hay một nhóm tiến trình, thì rất cuộc nó sẽ bỏ đói (starve) các tiến trình không được ưu ái khỏi tài nguyên dùng chung, điều này mâu thuẫn với tính sống. Ngược lại, leader có thể giữ vai trò của mình cho đến khi nó dừng hoặc sập, và những leader tồn tại lâu dài được ưu tiên.

Having a stable leader in the system helps to avoid state synchronization between remote participants, reduce the number of exchanged messages, and drive execution from a single process instead of requiring peer-to-peer coordination. One of the potential problems in systems with a

notion of leadership is that the leader can become a bottleneck. To overcome that, many systems partition data in non-intersecting independent replica sets (see “Database Partitioning” on **page 270**). Instead of having a single system-wide leader, each replica set has its own leader. One of the systems that uses this approach is **Spanner** (see “Distributed Transactions with Spanner” on page 268).

Việc có một leader ổn định trong hệ thống giúp tránh đồng bộ trạng thái giữa các bên tham gia ở xa, giảm số thông điệp trao đổi, và điều khiển việc thực thi từ một tiến trình duy nhất thay vì đòi hỏi sự phối hợp ngang hàng (peer-to-peer). Một trong những vấn đề tiềm tàng trong các hệ thống có khái niệm leader là leader có thể trở thành nút cổ chai (bottleneck). Để khắc phục điều đó, nhiều hệ thống phân hoạch (partition) dữ liệu thành các tập bản sao (replica set) độc lập không giao nhau (xem “Database Partitioning” ở trang 270). Thay vì có một leader duy nhất cho toàn hệ thống, mỗi tập bản sao có leader riêng của nó. Một trong những hệ thống dùng cách tiếp cận này là Spanner (xem “Distributed Transactions with Spanner” ở trang 268).

Because every leader process will eventually fail, failure has to be detected, reported, and reacted upon: a system has to elect another leader to replace the failed one.

Bởi vì mọi tiến trình leader rốt cuộc đều sẽ hỏng, lỗi phải được phát hiện, báo cáo, và xử lý: hệ thống phải bầu một leader khác để thay thế leader đã hỏng.

Some algorithms, such as **ZAB** (see “Zookeeper **Atomic Broadcast** (ZAB)” on page 283), **Multi-Paxos** (see “Multi-**Paxos**” on page 291), or **Raft** (see “Raft” on page 300), use temporary leaders to reduce the number of messages required to reach an agreement between the participants. However, these algorithms use their own algorithm-specific means for leader election, failure detection, and resolving conflicts between the competing leader processes.

Một số thuật toán, chẳng hạn ZAB (xem “Zookeeper Atomic Broadcast (ZAB)” ở trang 283), Multi-Paxos (xem “Multi-Paxos” ở trang 291), hoặc Raft (xem “Raft” ở trang 300), sử dụng các leader tạm thời để giảm số thông điệp cần thiết để đạt được thỏa thuận giữa các bên tham gia. Tuy nhiên, các thuật toán này sử dụng những phương tiện riêng đặc thù của thuật toán cho việc bầu chọn leader, phát hiện lỗi, và giải quyết xung đột giữa các tiến trình leader đang cạnh tranh.

Bully Algorithm — Thuật toán Bully

One of the leader election algorithms, known as the **bully algorithm** , uses process ranks to identify the new leader. Each process gets a unique rank assigned to it. During the election, the process with the highest rank becomes a leader [MOLINA82] .

Một trong các thuật toán bầu chọn leader, gọi là thuật toán bully, dùng thứ hạng (rank) của các tiến trình để xác định leader mới. Mỗi tiến trình được gán một thứ hạng duy nhất. Trong lúc bầu chọn, tiến trình có thứ hạng cao nhất trở thành leader [MOLINA82].

This algorithm is known for its simplicity. The algorithm is named bully because the highest-ranked **node** “bullies” other nodes into accepting it. It is also known as monarchical leader election: the highest-ranked sibling becomes a monarch after the previous one ceases to exist.

Thuật toán này nổi tiếng về sự đơn giản. Nó được đặt tên là bully (kẻ bắt nạt) vì nút có thứ hạng cao nhất “bắt nạt” các nút khác phải chấp nhận nó. Nó còn được gọi là bầu chọn leader kiểu quân chủ (monarchical): nút anh em (sibling) có thứ hạng cao nhất trở thành quân vương sau khi nút quân vương trước đó không còn tồn tại.

Election starts if one of the processes notices that there's no leader in the system (it was never initialized) or the previous leader has stopped responding to requests, and proceeds in three steps:

1

Việc bầu chọn bắt đầu nếu một trong các tiến trình nhận thấy rằng không có leader trong hệ thống (nó chưa từng được khởi tạo) hoặc leader trước đó đã ngừng phản hồi các yêu cầu, và tiến hành theo ba bước:

1. The process sends election messages to processes with higher identifiers.
2. The process waits, allowing higher-ranked processes to respond. If no higher-ranked process responds, it proceeds with step 3. Otherwise, the process notifies the highest-ranked process it has heard from, and allows it to proceed with step 3.
3. The process assumes that there are no active processes with a higher rank, and notifies all lower-ranked processes about the new leader.
 1. Tiến trình gửi các thông điệp bầu chọn (Election) tới các tiến trình có định danh cao hơn.
 2. Tiến trình chờ, để các tiến trình thứ hạng cao hơn phản hồi. Nếu không có tiến trình thứ hạng cao hơn nào phản hồi, nó tiến hành bước 3. Ngược lại, tiến trình thông báo cho tiến trình thứ hạng cao nhất mà nó nghe được, và để tiến trình đó tiến hành bước 3.
 3. Tiến trình giả định rằng không có tiến trình hoạt động nào có thứ hạng cao hơn, và thông báo cho tất cả các tiến trình thứ hạng thấp hơn về leader mới.

Figure 10-1 illustrates the bully leader election algorithm:

Hình 10-1 minh họa thuật toán bầu chọn leader bully:

- a) Process 3 notices that the previous leader 6 has crashed and starts a new election by sending Election messages to processes with higher identifiers.
- b) 4 and 5 respond with Alive , as they have a higher rank than 3 .
- c) 3 notifies the highest-ranked process 5 that has responded during this round.
- d) 5 is elected as a new leader. It broadcasts Elected messages, notifying lower-ranked processes about the election results.
 - a) Tiến trình 3 nhận thấy leader trước đó là 6 đã sập và bắt đầu một cuộc bầu chọn mới bằng cách gửi các thông điệp Election tới các tiến trình có định danh cao hơn.
 - b) 4 và 5 phản hồi bằng Alive, vì chúng có thứ hạng cao hơn 3.
 - c) 3 thông báo cho tiến trình thứ hạng cao nhất là 5 đã phản hồi trong vòng này.
 - d) 5 được bầu làm leader mới. Nó quảng bá các thông điệp Elected, thông báo cho các tiến trình thứ hạng thấp hơn về kết quả bầu chọn.

1 These steps describe the modified bully election algorithm [KORDAFSHARI05] as it's more compact and clear.

1 Các bước này mô tả thuật toán bầu chọn bully đã được sửa đổi [KORDAFSHARI05] vì nó cô đọng và rõ ràng hơn.

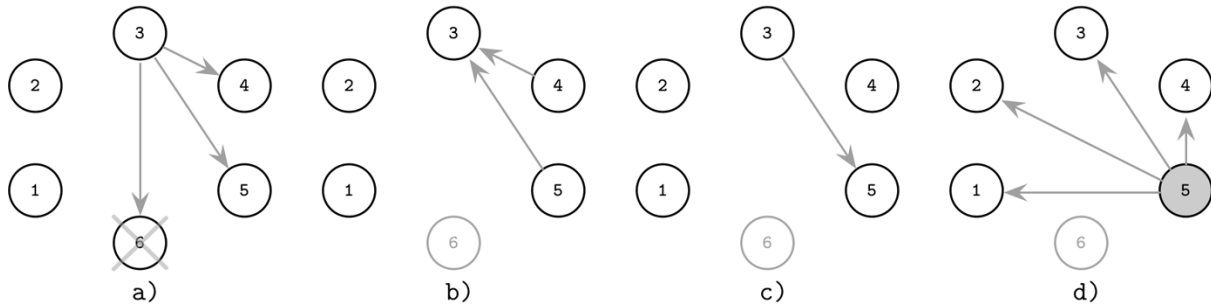


Figure 10-1. Bully algorithm: previous leader (6) fails and process 3 starts the new election

Hình 10-1. Thuật toán bully: leader trước đó (6) hỏng và tiến trình 3 bắt đầu cuộc bầu chọn mới

One of the apparent problems with this algorithm is that it violates the safety guarantee (that at most one leader can be elected at a time) in the presence of network partitions. It is quite easy to end up in the situation where nodes get split into two or more independently functioning subsets, and each subset elects its leader. This situation is called split brain .

Một trong những vấn đề rõ ràng của thuật toán này là nó vi phạm bảo đảm an toàn (rằng tại mỗi thời điểm có nhiều nhất một leader được bầu) khi có phân vùng mạng (network partition). Khá dễ rơi vào tình huống các nút bị chia thành hai hoặc nhiều tập con hoạt động độc lập, và mỗi tập con bầu leader riêng của nó. Tình huống này được gọi là split brain.

Another problem with this algorithm is a strong preference toward high-ranked nodes, which becomes an issue if they are unstable and can lead to a permanent state of reelection. An unstable high-ranked node proposes itself as a leader, fails shortly thereafter, wins reelection, fails again, and the whole process repeats. This problem can be solved by distributing host quality metrics and taking them into consideration during the election.

Một vấn đề khác của thuật toán này là sự thiên vị mạnh đối với các nút thứ hạng cao, điều này trở thành rắc rối nếu chúng không ổn định và có thể dẫn đến trạng thái bầu lại (reelection) vĩnh viễn. Một nút thứ hạng cao nhưng không ổn định tự đề cử mình làm leader, hỏng ngay sau đó, thắng cuộc bầu lại, lại hỏng, và toàn bộ quá trình lặp lại. Vấn đề này có thể được giải quyết bằng cách phân phát các chỉ số chất lượng máy chủ và đưa chúng vào xem xét trong lúc bầu chọn.

Next-In-Line Failover — Dự phòng Next-In-Line

There are many versions of the bully algorithm that improve its various properties. For example, we can use multiple next-in-line alternative processes as a failover to shorten reelections [GHOLIPOUR09] .

Có nhiều phiên bản của thuật toán bully cải thiện các thuộc tính khác nhau của nó. Chẳng hạn, ta có thể dùng nhiều tiến trình thay thế kế tiếp (next-in-line) làm dự phòng (failover) để rút ngắn các cuộc bầu lại [GHOLIPOUR09].

Each elected leader provides a list of failover nodes. When one of the processes detects a leader failure, it starts a new election round by sending a message to the highest-ranked alternative from the list provided by the failed leader. If one of the proposed alternatives is up, it becomes a new leader without having to go through the complete election round.

Mỗi leader được bầu cung cấp một danh sách các nút dự phòng. Khi một trong các tiến trình phát hiện leader hỏng, nó bắt đầu một vòng bầu chọn mới bằng cách gửi thông điệp tới phương án thay thế thứ hạng cao nhất trong danh sách do leader đã hỏng cung cấp. Nếu một trong các phương án được đề xuất còn hoạt động, nó trở thành leader mới mà không phải trải qua trọn vẹn vòng bầu chọn.

If the process that has detected the leader failure is itself the highest ranked process from the list, it can notify the processes about the new leader right away.

Nếu tiến trình đã phát hiện leader hỏng tự nó là tiến trình thứ hạng cao nhất trong danh sách, nó có thể thông báo ngay cho các tiến trình về leader mới.

Figure 10-2 shows the process with this optimization in place:

Hình 10-2 cho thấy quá trình với tối ưu hóa này được áp dụng:

- a) 6, a leader with designated alternatives {5,4}, crashes. 3 notices this failure and contacts 5, the alternative from the list with the highest rank.
 - a) 6, một leader với các phương án thay thế được chỉ định là {5,4}, bị sập. 3 nhận thấy lỗi này và liên lạc với 5, phương án thay thế có thứ hạng cao nhất trong danh sách.
- b) 5 responds to 3 that it's alive to prevent it from contacting other nodes from the alternatives list.
- c) 5 notifies other nodes that it's a new leader.
 - b) 5 phản hồi cho 3 rằng nó còn sống để ngăn 3 liên lạc với các nút khác trong danh sách thay thế.
 - c) 5 thông báo cho các nút khác rằng nó là leader mới.

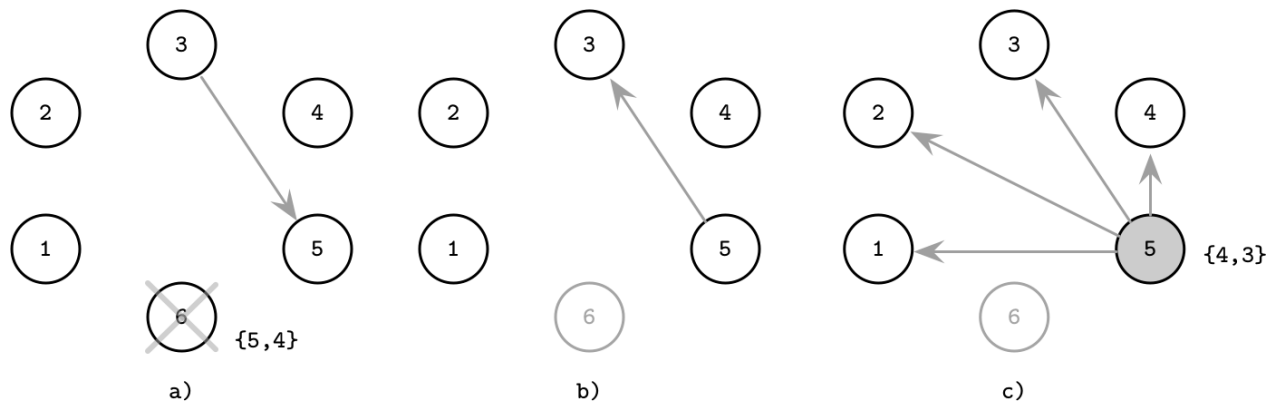


Figure 10-2. Bully algorithm with failover: previous leader (6) fails and process 3 starts the new election by contacting the highest-ranked alternative

Hình 10-2. Thuật toán bully với dự phòng: leader trước đó (6) hỏng và tiến trình 3 bắt đầu cuộc bầu chọn mới bằng cách liên lạc với phương án thay thế có thứ hạng cao nhất

As a result, we require fewer steps during the election if the next-in-line process is alive.

Kết quả là, ta cần ít bước hơn trong lúc bầu chọn nếu tiến trình kế tiếp còn sống.

Candidate/Ordinary Optimization — Tối ưu hóa Candidate/Ordinary

Another algorithm attempts to lower requirements on the number of messages by splitting the nodes into two subsets, candidate and ordinary , where only one of the candidate nodes can eventually become a leader [MURSHED12] .

Một thuật toán khác cố gắng hạ thấp yêu cầu về số thông điệp bằng cách chia các nút thành hai tập con, ứng viên (candidate) và thường (ordinary), trong đó chỉ một trong các nút ứng viên tốt cuộc có thể trở thành leader [MURSHED12].

The ordinary process initiates election by contacting candidate nodes, collecting responses from them, picking the highest-ranked alive candidate as a new leader, and then notifying the rest of the nodes about the election results.

Tiến trình thường khởi xướng bầu chọn bằng cách liên lạc với các nút ứng viên, thu thập phản hồi từ chúng, chọn ứng viên còn sống có thứ hạng cao nhất làm leader mới, rồi thông báo cho các nút còn lại về kết quả bầu chọn.

To solve the problem with multiple simultaneous elections, the algorithm proposes to use a tiebreaker variable δ , a process-specific delay, varying significantly between the nodes, that allows one of the nodes to initiate the election before the other ones. The tiebreaker time is generally greater than the message round-trip time. Nodes with higher priorities have a lower δ , and vice versa.

Để giải quyết vấn đề có nhiều cuộc bầu chọn xảy ra đồng thời, thuật toán đề xuất dùng một biến phân định thắng thua δ , một độ trễ đặc thù theo từng tiến trình, biến thiên đáng kể giữa các nút, cho phép một trong các nút khởi xướng bầu chọn trước các nút khác. Thời gian phân định thắng thua nhìn chung lớn hơn thời gian một vòng truyền thông điệp. Các nút có độ ưu tiên cao hơn có δ thấp hơn, và ngược lại.

Figure 10-3 shows the steps of the election process:

Hình 10-3 cho thấy các bước của quá trình bầu chọn:

- a) Process 4 from the ordinary set notices the failure of leader process 6 . It starts a new election round by contacting all remaining processes from the candidate set.
- b) Candidate processes respond to notify 4 that they're still alive.
- c) 4 notifies all processes about the new leader: 2 .
 - a) Tiến trình 4 thuộc tập thường nhận thấy lỗi của tiến trình leader 6. Nó bắt đầu một vòng bầu chọn mới bằng cách liên lạc với tất cả các tiến trình còn lại trong tập ứng viên.
 - b) Các tiến trình ứng viên phản hồi để thông báo cho 4 rằng chúng vẫn còn sống.
 - c) 4 thông báo cho tất cả các tiến trình về leader mới: 2.

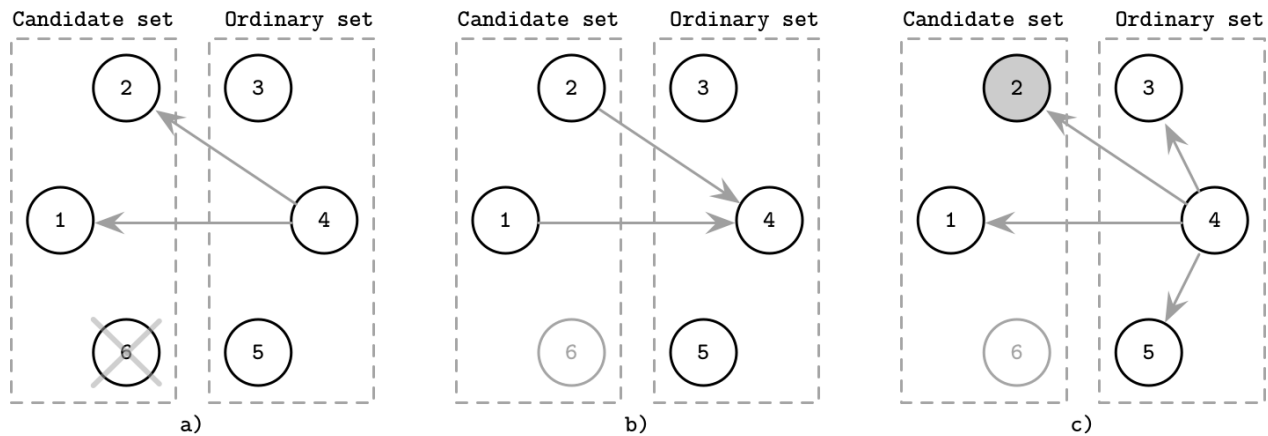


Figure 10-3. Candidate/ordinary modification of the bully algorithm: previous leader (6) fails and process 4 starts the new election

Hình 10-3. Biến thể candidate/ordinary của thuật toán bully: leader trước đó (6) hỏng và tiến trình 4 bắt đầu cuộc bầu chọn mới

Invitation Algorithm — Thuật toán Invitation

An invitation algorithm allows processes to “invite” other processes to join their groups instead of trying to outrank them. This algorithm allows multiple leaders by definition, since each group has its own leader.

Một thuật toán mời (invitation) cho phép các tiến trình “mời” các tiến trình khác gia nhập nhóm của mình thay vì cố vượt thứ hạng chúng. Thuật toán này theo định nghĩa cho phép nhiều leader, vì mỗi nhóm có leader riêng.

Each process starts as a leader of a new group, where the only member is the process itself. Group leaders contact peers that do not belong to their groups, inviting them to join. If the peer process is a leader itself, two groups are merged. Otherwise, the contacted process responds with a group leader ID, allowing two group leaders to establish contact and **merge** groups in fewer steps.

Mỗi tiến trình bắt đầu với tư cách là leader của một nhóm mới, mà thành viên duy nhất là chính tiến trình đó. Các leader nhóm liên lạc với những tiến trình ngang hàng không thuộc nhóm của mình, mời chúng gia nhập. Nếu tiến trình ngang hàng tự nó là một leader, hai nhóm được gộp lại. Ngược lại, tiến trình được liên lạc phản hồi bằng ID của leader nhóm, cho phép hai leader nhóm thiết lập liên lạc và gộp các nhóm trong ít bước hơn.

Figure 10-4 shows the execution steps of the invitation algorithm:

Hình 10-4 cho thấy các bước thực thi của thuật toán invitation:

- a) Four processes start as leaders of groups containing one member each. 1 invites 2 to join its group, and 3 invites 4 to join its group.
- b) 2 joins a group with process 1, and 4 joins a group with process 3. 1, the leader of the first group, contacts 3, the leader of the other group. Remaining group members (4, in this case) are notified about the new group leader.
- c) Two groups are merged and 1 becomes a leader of an extended group.
 - a) Bốn tiến trình bắt đầu với tư cách leader của các nhóm mỗi nhóm chứa một thành viên. 1 mời 2 gia nhập nhóm của nó, và 3 mời 4 gia nhập nhóm của nó.

- b) 2 gia nhập nhóm với tiến trình 1, và 4 gia nhập nhóm với tiến trình 3. 1, leader của nhóm thứ nhất, liên lạc với 3, leader của nhóm kia. Các thành viên còn lại của nhóm (trong trường hợp này là 4) được thông báo về leader nhóm mới.
- c) Hai nhóm được gộp lại và 1 trở thành leader của một nhóm mở rộng.

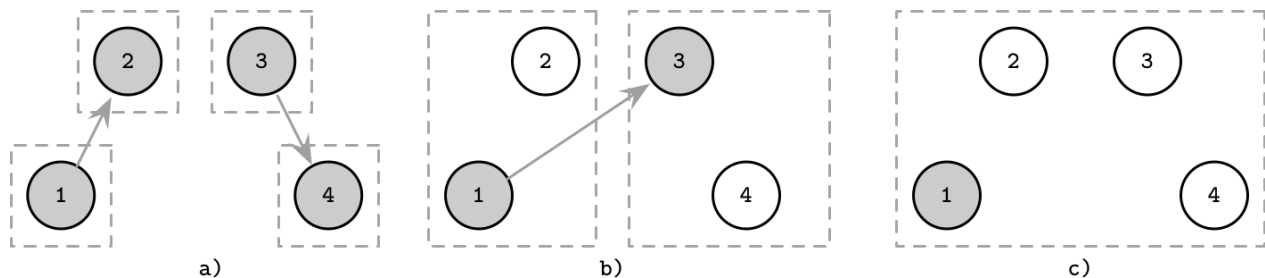


Figure 10-4. Invitation algorithm Since groups are merged, it doesn't matter whether the process that suggested the group merge becomes a new leader or the other one does. To keep the number of messages required to merge groups to a minimum, a leader of a larger group can become a leader for a new group. This way only the processes from the smaller group have to be notified about the change of leader.

Hình 10-4. Thuật toán invitation. Vì các nhóm được gộp lại, không quan trọng việc tiến trình đề xuất gộp nhóm trở thành leader mới hay tiến trình kia. Để giữ số thông điệp cần thiết để gộp các nhóm ở mức tối thiểu, leader của nhóm lớn hơn có thể trở thành leader cho nhóm mới. Bằng cách này, chỉ các tiến trình từ nhóm nhỏ hơn phải được thông báo về việc thay đổi leader.

Similar to the other discussed algorithms, this algorithm allows processes to settle in multiple groups and have multiple leaders. The invitation algorithm allows creating process groups and merging them without having to trigger a new election from scratch, reducing the number of messages required to finish the election.

Tương tự các thuật toán khác đã bàn, thuật toán này cho phép các tiến trình ổn định trong nhiều nhóm và có nhiều leader. Thuật toán invitation cho phép tạo các nhóm tiến trình và gộp chúng mà không phải kích hoạt một cuộc bầu chọn mới từ đầu, giảm số thông điệp cần thiết để kết thúc việc bầu chọn.

Ring Algorithm — Thuật toán Ring

In the ring algorithm [CHANG79], all nodes in the system form a ring and are aware of the ring topology (i.e., their predecessors and successors in the ring). When the process detects the leader failure, it starts the new election. The election message is forwarded across the ring: each process contacts its successor (the next node closest to it in the ring). If this node is unavailable, the process skips the unreachable node and attempts to contact the nodes after it in the ring, until eventually one of them responds.

Trong thuật toán ring (vòng) [CHANG79], tất cả các nút trong hệ thống tạo thành một vòng và nhận biết được tô-pô vòng (tức biết tiền nhiệm và kế nhiệm của chúng trong vòng). Khi tiến trình phát hiện leader hỏng, nó bắt đầu cuộc bầu chọn mới. Thông điệp bầu chọn được chuyển tiếp dọc theo vòng: mỗi tiến trình liên lạc với nút kế nhiệm của nó (nút gần nó nhất tiếp theo trong vòng). Nếu nút này không khả dụng, tiến trình bỏ qua nút không liên lạc được và cố liên lạc với các nút sau nó trong vòng, cho đến khi một nút trong số chúng phản hồi.

Nodes contact their siblings, following around the ring and collecting the live node set, adding themselves to the set before passing it over to the next node, similar to the failure-detection algorithm described in “Timeout-Free **Failure Detector**” on page 197 , where nodes append their identifiers to the path before passing it to the next node.

Các nút liên lạc với các nút anh em của mình, đi vòng quanh vòng và thu thập tập các nút còn sống, tự thêm mình vào tập trước khi chuyển nó sang nút kế tiếp, tương tự thuật toán phát hiện lỗi được mô tả trong “Timeout-Free Failure Detector” ở trang 197, nơi các nút nối thêm định danh của mình vào đường đi trước khi chuyển nó sang nút kế tiếp.

The algorithm proceeds by fully traversing the ring. When the message comes back to the node that started the election, the highest-ranked node from the live set is chosen as a leader. In Figure 10-5 , you can see an example of such a traversal:

Thuật toán tiến hành bằng cách duyệt trọn vẹn vòng. Khi thông điệp quay lại nút đã khởi xướng cuộc bầu chọn, nút có thứ hạng cao nhất trong tập còn sống được chọn làm leader. Trong Hình 10-5, bạn có thể thấy một ví dụ về một lượt duyệt như vậy:

- a) Previous leader 6 has failed and each process has a view of the ring from its perspective.
 - b) 3 initiates an election round by starting traversal. On each step, there's a set of nodes traversed on the path so far. 5 can't reach 6 , so it skips it and goes straight to 1 .
 - c) Since 5 was the node with the highest rank, 3 initiates another round of messages, distributing the information about the new leader.
- a) Leader trước đó 6 đã hỏng và mỗi tiến trình có một góc nhìn riêng về vòng từ vị trí của nó.
 - b) 3 khởi xướng một vòng bầu chọn bằng cách bắt đầu duyệt. Ở mỗi bước, có một tập các nút đã được duyệt trên đường đi cho đến thời điểm đó. 5 không liên lạc được với 6, nên nó bỏ qua 6 và đi thẳng tới 1.
 - c) Vì 5 là nút có thứ hạng cao nhất, 3 khởi xướng một vòng thông điệp khác, phân phát thông tin về leader mới.

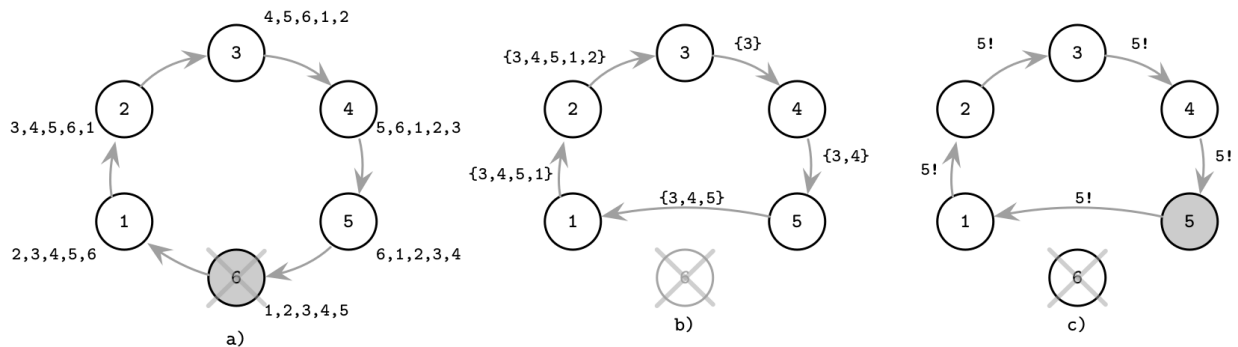


Figure 10-5. Ring algorithm: previous leader (6) fails and 3 starts the election process

Hình 10-5. Thuật toán ring: leader trước đó (6) hỏng và 3 bắt đầu quá trình bầu chọn

Variants of this algorithm include collecting a single highest-ranked identifier instead of a set of active nodes to save space: since the max function is commutative, it is enough to know a current maximum. When the algorithm comes back to the node that has started the election, the last known highest identifier is circulated across the ring once again.

Các biến thể của thuật toán này bao gồm việc chỉ thu thập một định danh có thứ hạng cao nhất thay vì cả một tập các nút đang hoạt động để tiết kiệm không gian: vì hàm max có tính giao hoán, chỉ cần biết một giá trị cực đại hiện tại là đủ. Khi thuật toán quay lại nút đã khởi xướng cuộc bầu chọn, định danh cao nhất được biết gần nhất lại được luân chuyển một lần nữa khắp vòng.

Since the ring can be partitioned in two or more parts, with each part potentially electing its own leader, this approach doesn't hold a safety property, either.

Vì vòng có thể bị phân vùng thành hai hay nhiều phần, mỗi phần có khả năng bầu leader riêng của nó, cách tiếp cận này cũng không nắm giữ được thuộc tính an toàn.

As you can see, for a system with a leader to function correctly, we need to know the status of the current leader (whether it is alive or not), since to keep processes organized and for execution to continue, the leader has to be alive and reachable to perform its duties. To detect leader crashes, we can use failure-detection algorithms (see Chapter 9).

Như bạn thấy, để một hệ thống có leader hoạt động đúng, ta cần biết trạng thái của leader hiện tại (nó còn sống hay không), bởi để giữ các tiến trình có tổ chức và để việc thực thi tiếp diễn, leader phải còn sống và liên lạc được nhằm thực hiện nhiệm vụ của mình. Để phát hiện các sự sập của leader, ta có thể dùng các thuật toán phát hiện lỗi (xem Chương 9).

Summary — Tóm tắt

Leader election is an important subject in distributed systems, since using a designated leader helps to reduce coordination overhead and improve the algorithm's performance. Election rounds might be costly but, since they're infrequent, they do not have a negative impact on the overall system performance. A single leader can become a bottleneck, but most of the time this is solved by partitioning data and using per-partition leaders or using different leaders for different actions.

Bầu chọn leader là một chủ đề quan trọng trong hệ phân tán, vì việc dùng một leader được chỉ định giúp giảm chi phí phối hợp và cải thiện hiệu năng của thuật toán. Các vòng bầu chọn có thể tốn kém, nhưng vì chúng không xảy ra thường xuyên nên chúng không gây tác động tiêu cực đến hiệu năng tổng thể của hệ thống. Một leader duy nhất có thể trở thành nút cổ chai, nhưng hầu hết thời gian điều này được giải quyết bằng cách phân hoạch dữ liệu và dùng leader cho từng phân hoạch hoặc dùng các leader khác nhau cho các hành động khác nhau.

Unfortunately, all the algorithms we've discussed in this chapter are prone to the split brain problem: we can end up with two leaders in independent subnets that are not aware of each other's existence. To avoid split brain, we have to obtain a cluster-wide majority of votes.

Đáng tiếc, tất cả các thuật toán ta đã bàn trong chương này đều dễ gặp vấn đề split brain: ta có thể rơi vào tình huống có hai leader trong các mạng con độc lập mà không biết về sự tồn tại của nhau. Để tránh split brain, ta phải có được đa số phiếu trên phạm vi toàn cụm.

Many **consensus** algorithms, including Multi-Paxos and Raft, rely on a leader for coordination. But isn't leader election the same as consensus? To elect a leader, we need to reach a consensus about its identity. If we can reach consensus about the leader identity, we can use the same means to reach consensus on anything else [ABRAHAM13].

Nhiều thuật toán đồng thuận (consensus), bao gồm Multi-Paxos và Raft, dựa vào một leader để phối hợp. Nhưng chẳng phải bầu chọn leader cũng giống như đồng thuận sao?

Để bầu một leader, ta cần đạt được sự đồng thuận về danh tính của nó. Nếu ta có thể đạt được đồng thuận về danh tính leader, ta có thể dùng chính phương tiện đó để đạt đồng thuận về bất kỳ điều gì khác [ABRAHAM13].

The identity of a leader may change without processes knowing about it, so the question is whether the process-local knowledge about the leader is still valid. To achieve that, we need to combine leader election with failure detection. For example, the stable leader election algorithm uses rounds with a unique stable leader and timeout-based failure detection to guarantee that the leader can retain its position for as long as it doesn't crash and is accessible [AGUILERA01].

Danh tính của một leader có thể thay đổi mà các tiến trình không hay biết, nên câu hỏi đặt ra là liệu hiểu biết cục bộ tại tiến trình về leader có còn hợp lệ hay không. Để đạt được điều đó, ta cần kết hợp bầu chọn leader với phát hiện lỗi. Chẳng hạn, thuật toán bầu chọn leader ổn định (stable leader election) dùng các vòng với một leader ổn định duy nhất và phát hiện lỗi dựa trên timeout để đảm bảo rằng leader có thể giữ vị trí của mình chừng nào nó không sập và còn truy cập được [AGUILERA01].

Algorithms that rely on leader election often allow the existence of multiple leaders and attempt to resolve conflicts between the leaders as quickly as possible. For example, this is true for Multi-Paxos (see “Multi-Paxos” on page 291), where only one of the two conflicting leaders (proposers) can proceed, and these conflicts are resolved by collecting a second **quorum**, guaranteeing that the values from two different proposers won't be accepted.

Các thuật toán dựa vào bầu chọn leader thường cho phép sự tồn tại của nhiều leader và cố gắng giải quyết xung đột giữa các leader nhanh nhất có thể. Chẳng hạn, điều này đúng với Multi-Paxos (xem “Multi-Paxos” ở trang 291), nơi chỉ một trong hai leader (bên đề xuất, proposer) xung đột có thể tiến hành, và các xung đột này được giải quyết bằng cách thu thập một quorum thứ hai, đảm bảo rằng các giá trị từ hai bên đề xuất khác nhau sẽ không được chấp nhận.

In Raft (see “Raft” on page 300), a leader can discover that its **term** is out-of-date, which implies the presence of a different leader in the system, and update its term to the more recent one.

Trong Raft (xem “Raft” ở trang 300), một leader có thể phát hiện ra rằng nhiệm kỳ (term) của nó đã lỗi thời, điều này hàm ý sự hiện diện của một leader khác trong hệ thống, và cập nhật nhiệm kỳ của nó lên nhiệm kỳ gần đây hơn.

In both cases, having a leader is a way to ensure liveness (if the current leader has failed, we need a new one), and processes should not take indefinitely long to understand whether or not it has really failed. Lack of safety and allowing multiple leaders is a performance optimization: algorithms can proceed with a replication phase, and safety is guaranteed by detecting and resolving the conflicts.

Trong cả hai trường hợp, việc có một leader là một cách để đảm bảo tính sống (nếu leader hiện tại đã hỏng, ta cần một leader mới), và các tiến trình không nên mất quá nhiều thời gian một cách vô hạn định để hiểu rằng nó có thực sự đã hỏng hay không. Việc thiếu tính an toàn và cho phép nhiều leader là một tối ưu hóa hiệu năng: các thuật toán có thể tiến hành một pha sao chép (replication), và tính an toàn được đảm bảo bằng cách phát hiện và giải quyết các xung đột.

We discuss consensus and leader election in the context of consensus in more detail in Chapter 14.

Ta bàn về đồng thuận và bầu chọn leader trong bối cảnh đồng thuận chi tiết hơn ở Chương 14.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Leader election algorithms Lynch, Nancy and Boaz Patt-Shamir. 1993. "Distributed algorithms." Lecture notes for 6.852 . Cambridge, MA: MIT.

Attiya, Hagit and Jennifer Welch. 2004. Distributed Computing: Fundamentals, Simulations and Advanced Topics . USA: John Wiley & Sons.

Tanenbaum, Andrew S. and Maarten van Steen. 2006. Distributed Systems: Principles and Paradigms (2nd Ed.). Upper Saddle River, NJ: Prentice-Hall.

CHAPTER 11 Replication and Consistency — Chương 11: Nhân bản và Nhất quán

Before we move on to discuss **consensus** and atomic commitment algorithms, let's put together the last piece required for their in-depth understanding: consistency models. Consistency models are important, since they explain visibility semantics and behavior of the system in the presence of multiple copies of data.

Trước khi chuyển sang bàn về các thuật toán đồng thuận (consensus) và commit nguyên tử (atomic commitment), hãy ghép nốt mảnh ghép cuối cùng cần thiết để hiểu sâu chúng: các mô hình nhất quán (consistency model). Mô hình nhất quán rất quan trọng, bởi chúng giải thích ngữ nghĩa hiển thị và hành vi của hệ thống khi tồn tại nhiều bản sao của dữ liệu.

Fault tolerance is a property of a system that can continue operating correctly in the presence of failures of its components. Making a system fault-tolerant is not an easy task, and it may be difficult to add fault tolerance to the existing system. The primary goal is to remove a single point of failure from the system and make sure that we have redundancy in mission-critical components. Usually, redundancy is entirely transparent for the user.

Dung lỗi (fault tolerance) là thuộc tính của một hệ thống có thể tiếp tục hoạt động đúng đắn ngay cả khi các thành phần của nó gặp sự cố. Làm cho một hệ thống dung lỗi không phải việc dễ dàng, và có thể rất khó để thêm khả năng dung lỗi vào một hệ thống đã tồn tại. Mục tiêu chính là loại bỏ điểm lỗi đơn (single point of failure) khỏi hệ thống và bảo đảm rằng ta có sự dư thừa (redundancy) ở các thành phần trọng yếu. Thông thường, sự dư thừa hoàn toàn trong suốt đối với người dùng.

A system can continue operating correctly by storing multiple copies of data so that, when one of the machines fails, the other one can serve as a failover. In systems with a single source of truth (for example, primary/replica databases), failover can be done explicitly, by promoting a replica to become a new master. Other systems do not require explicit reconfiguration and ensure consistency by collecting responses from multiple participants during **read** and write queries.

Một hệ thống có thể tiếp tục hoạt động đúng đắn bằng cách lưu nhiều bản sao của dữ liệu, để khi một trong các máy gặp sự cố thì máy kia có thể đóng vai trò chuyển dự phòng (failover). Trong các hệ thống có một nguồn chân lý duy nhất (chẳng hạn cơ sở dữ liệu primary/replica), failover có thể được thực hiện tường minh, bằng cách thăng cấp một replica thành master mới. Các hệ thống khác không cần cấu hình lại tường minh và bảo đảm nhất quán bằng cách thu thập phản hồi từ nhiều bên tham gia trong các truy vấn đọc và ghi.

Data replication is a way of introducing redundancy by maintaining multiple copies of data in the system. However, since updating multiple copies of data atomically is a problem equivalent to consensus [MILOSEVIC11], it might be quite costly to perform this operation for every operation in the database. We can explore some more cost-effective and flexible ways to make data look consistent from the user's perspective, while allowing some degree of divergence between participants.

Nhân bản dữ liệu (data replication) là cách đưa vào sự dư thừa bằng cách duy trì nhiều bản sao của dữ liệu trong hệ thống. Tuy nhiên, vì việc cập nhật nhiều bản sao dữ liệu một cách nguyên tử là một bài toán tương đương với đồng thuận [MILOSEVIC11], nên việc thực hiện thao tác này cho mọi phép toán trong cơ sở dữ liệu có thể khá tốn kém. Ta có thể khám phá một số cách tiết kiệm chi phí hơn và linh hoạt hơn để dữ liệu trông nhất quán dưới góc nhìn của người dùng, đồng thời cho phép một mức độ phân kỳ nhất định giữa các bên tham gia.

Replication is particularly important in multidatacenter deployments. Geo-replication, in this case, serves multiple purposes: it increases availability and the ability to withstand a failure of one or more datacenters by providing redundancy. It

Nhân bản đặc biệt quan trọng trong các triển khai đa trung tâm dữ liệu. Trong trường hợp này, nhân bản theo địa lý (geo-replication) phục vụ nhiều mục đích: nó tăng tính sẵn sàng và khả năng chịu đựng sự cố của một hoặc nhiều trung tâm dữ liệu bằng cách cung cấp sự dư thừa. Nó

215 can also help to reduce the latency by placing a copy of data physically closer to the client.

cũng có thể giúp giảm độ trễ (latency) bằng cách đặt một bản sao dữ liệu ở vị trí vật lý gần với client hơn.

When data records are modified, their copies have to be updated accordingly. When talking about replication, we care most about three events: write, replica update, and read. These operations trigger a sequence of events initiated by the client. In some cases, updating replicas can happen after the write has finished from the client perspective, but this still does not change the fact that the client has to be able to observe operations in a particular order.

Khi các bản ghi dữ liệu bị sửa đổi, các bản sao của chúng phải được cập nhật tương ứng. Khi nói về nhân bản, ta quan tâm nhất đến ba sự kiện: ghi (write), cập nhật replica (replica update), và đọc (read). Các thao tác này kích hoạt một chuỗi sự kiện do client khởi xướng. Trong một số trường hợp, việc cập nhật replica có thể diễn ra sau khi thao tác ghi đã hoàn tất dưới góc nhìn của client, nhưng điều này vẫn không thay đổi thực tế rằng client phải có khả năng quan sát các thao tác theo một thứ tự cụ thể.

Achieving Availability — Đạt được tính sẵn sàng

We've talked about the fallacies of distributed systems and have identified many things that can go wrong. In the real world, nodes aren't always alive or able to communicate with one another. However, intermittent failures should not impact availability: from the user's perspective, the system as a whole has to continue operating as if nothing has happened.

Ta đã bàn về các ngộ nhận của hệ phân tán và đã xác định nhiều thứ có thể trục trặc. Trong thế giới thực, các nút không phải lúc nào cũng sống hoặc có thể giao tiếp với nhau. Tuy nhiên, các sự cố gián đoạn không nên ảnh hưởng đến tính sẵn sàng (availability): dưới góc nhìn của người dùng, toàn bộ hệ thống phải tiếp tục hoạt động như thể không có gì xảy ra.

System availability is an incredibly important property: in software engineering, we always strive for high availability, and try to minimize downtime. Engineering teams brag about their uptime metrics. We care so much about availability for several reasons: software has become an integral part of our society, and many important things cannot happen without it: bank transactions, communication, travel, and so on.

Tính sẵn sàng của hệ thống là một thuộc tính cực kỳ quan trọng: trong kỹ nghệ phần mềm, ta luôn nỗ lực đạt tính sẵn sàng cao và cố gắng giảm thiểu thời gian ngừng hoạt động. Các đội kỹ thuật khoe khoang về các chỉ số uptime của họ. Ta quan tâm rất nhiều đến tính sẵn sàng vì nhiều lý do: phần mềm đã trở thành một phần không thể thiếu của xã hội, và nhiều việc quan trọng không thể diễn ra nếu thiếu nó: giao dịch ngân hàng, liên lạc, đi lại, v.v.

For companies, lack of availability can mean losing customers or money: you can't shop in the online store if it's down, or transfer the money if your bank's website isn't responding.

Đối với các công ty, việc thiếu tính sẵn sàng có thể đồng nghĩa với mất khách hàng hoặc mất tiền: bạn không thể mua sắm trong cửa hàng trực tuyến nếu nó ngừng hoạt động, hay chuyển tiền nếu trang web của ngân hàng không phản hồi.

To make the system highly available, we need to design it in a way that allows handling failures or unavailability of one or more participants gracefully. For that, we need to introduce redundancy and replication. However, as soon as we add redundancy, we face the problem of keeping several copies of data in sync and have to implement **recovery** mechanisms.

Để làm cho hệ thống có tính sẵn sàng cao, ta cần thiết kế nó theo cách cho phép xử lý một cách uyển chuyển sự cố hoặc tình trạng không sẵn sàng của một hay nhiều bên tham gia. Vì vậy, ta cần đưa vào sự dư thừa và nhân bản. Tuy nhiên, ngay khi thêm sự dư thừa, ta phải đối mặt với bài toán giữ đồng bộ nhiều bản sao dữ liệu và phải hiện thực các cơ chế phục hồi (recovery).

Infamous CAP — CAP khét tiếng

Availability is a property that measures the ability of the system to serve a response for every request successfully. The theoretical definition of availability mentions eventual response, but of course, in a real-world system, we'd like to avoid services that take indefinitely long to respond.

Tính sẵn sàng là thuộc tính đo lường khả năng hệ thống phục vụ thành công một phản hồi cho mỗi yêu cầu. Định nghĩa lý thuyết của tính sẵn sàng đề cập đến phản hồi rốt cuộc cũng đến, nhưng dĩ nhiên, trong một hệ thống thực tế, ta muốn tránh các dịch vụ mất thời gian vô hạn để phản hồi.

Ideally, we'd like every operation to be consistent . Consistency is defined here as atomic or linearizable consistency (see “**Linearizability**” on **page** 223). Linearizable history can be expressed as a sequence of instantaneous operations that preserves the original operation order. Linearizability simplifies reasoning about the possible system states and makes a **distributed system** appear as if it was running on a single machine.

Một cách lý tưởng, ta muốn mọi thao tác đều nhất quán. Ở đây, nhất quán được định nghĩa là nhất quán nguyên tử hoặc tuyến tính hóa (xem “Linearizability” ở trang 223). Lịch sử tuyến tính hóa có thể được biểu diễn dưới dạng một chuỗi các thao tác tức thời mà bảo toàn thứ tự thao tác gốc. Tính tuyến tính hóa (linearizability) đơn giản hóa việc lập luận về các trạng thái khả dĩ của hệ thống và làm cho một hệ phân tán trông như thể nó đang chạy trên một máy đơn lẻ.

We would like to achieve both consistency and availability while tolerating network partitions. The network can get split into several parts where processes are not able to communicate with each other: some of the messages sent between partitioned nodes won't reach their destinations.

Ta muốn đạt được cả nhất quán lẫn tính sẵn sàng trong khi vẫn chịu đựng được phân vùng mạng (network partition). Mạng có thể bị chia thành nhiều phần mà các tiến trình không thể giao tiếp với nhau: một số thông điệp gửi giữa các nút bị phân vùng sẽ không đến được đích.

Availability requires any nonfailing **node** to deliver results, while consistency requires results to be linearizable. CAP conjecture, formulated by Eric Brewer, discusses trade-offs between Consistency, Availability, and Partition tolerance [BREWER00] .

Tính sẵn sàng đòi hỏi bất kỳ nút không lỗi nào cũng phải trả về kết quả, trong khi nhất quán đòi hỏi kết quả phải tuyến tính hóa. Phỏng đoán CAP, do Eric Brewer phát biểu, bàn về các đánh đổi giữa Nhất quán (Consistency), Tính sẵn sàng (Availability), và Khả năng chịu phân vùng (Partition tolerance) [BREWER00].

Availability requirement is impossible to satisfy in an **asynchronous** system, and we cannot implement a system that simultaneously guarantees both availability and consistency in the presence of network partitions [GILBERT02] . We can build systems that guarantee strong consistency while providing best effort availability, or guarantee availability while providing best effort consistency [GILBERT12] . Best effort here implies that if everything works, the system will not purposefully violate any guarantees, but guarantees are allowed to be weakened and violated in the case of network partitions.

Yêu cầu về tính sẵn sàng là bất khả thi để thỏa mãn trong một hệ bất đồng bộ, và ta không thể hiện thực một hệ thống đồng thời bảo đảm cả tính sẵn sàng lẫn nhất quán khi có phân vùng mạng [GILBERT02]. Ta có thể xây dựng các hệ thống bảo đảm nhất quán mạnh trong khi cung cấp tính sẵn sàng theo kiểu nỗ lực tối đa, hoặc bảo đảm tính sẵn sàng trong khi cung cấp nhất quán theo kiểu nỗ lực tối đa [GILBERT12]. “Nỗ lực tối đa” ở đây hàm ý rằng nếu mọi thứ hoạt động, hệ thống sẽ không cố ý vi phạm bất kỳ bảo đảm nào, nhưng các bảo đảm được phép bị làm yếu đi và bị vi phạm trong trường hợp phân vùng mạng.

In other words, CAP describes a continuum of potential choices, where on different sides of the spectrum we have systems that are:

Nói cách khác, CAP mô tả một dải liên tục các lựa chọn khả dĩ, trong đó ở các phía khác nhau của phổ ta có các hệ thống là:

Consistent and partition tolerant CP systems prefer failing requests to serving potentially inconsistent data.

Nhất quán và chịu phân vùng Các hệ thống CP ưu tiên cho yêu cầu thất bại hơn là phục vụ dữ liệu có thể không nhất quán.

Available and partition tolerant AP systems loosen the consistency requirement and allow serving potentially inconsistent values during the request.

Sẵn sàng và chịu phân vùng Các hệ thống AP nới lỏng yêu cầu nhất quán và cho phép phục vụ các giá trị có thể không nhất quán trong suốt yêu cầu.

An example of a CP system is an implementation of a consensus algorithm, requiring a majority of nodes for progress: always consistent, but might be unavailable in the case of a **network partition**. A database always accepting writes and serving reads as long as even a single replica is up is an example of an AP system, which may end up losing data or serving inconsistent results.

Một ví dụ về hệ thống CP là một hiện thực của thuật toán đồng thuận, đòi hỏi đa số các nút để tiến triển: luôn nhất quán, nhưng có thể không sẵn sàng trong trường hợp phân vùng mạng. Một cơ sở dữ liệu luôn chấp nhận ghi và phục vụ đọc miễn là còn dù chỉ một replica sống là một ví dụ về hệ thống AP, vốn có thể rất cuộc làm mất dữ liệu hoặc phục vụ kết quả không nhất quán.

PACELEC conjecture [ABADI12], an extension of CAP, states that in presence of network partitions there's a choice between consistency and availability (PAC). Else (E), even if the system is running normally, we still have to make a choice between latency and consistency.

Phỏng đoán PACELC [ABADI12], một mở rộng của CAP, phát biểu rằng khi có phân vùng mạng thì có một lựa chọn giữa nhất quán và tính sẵn sàng (PAC). Còn lại (E), ngay cả khi hệ thống đang chạy bình thường, ta vẫn phải lựa chọn giữa độ trễ và nhất quán.

Use CAP Carefully — Dùng CAP một cách thận trọng

It's important to note that CAP discusses network partitions rather than node crashes or any other type of failure (such as crash-recovery). A node, partitioned from the rest of the cluster, can serve inconsistent requests, but a crashed node will not respond at all. On the one hand, this implies that it's not necessary to have any nodes down to face consistency problems. On the other hand, this isn't the case in the real world: there are many different failure scenarios (some of which can be simulated with network partitions).

Cần lưu ý rằng CAP bàn về phân vùng mạng chứ không phải về việc nút sập hay bất kỳ loại sự cố nào khác (chẳng hạn crash-recovery). Một nút bị phân vùng khỏi phần còn lại của cụm có thể phục vụ các yêu cầu không nhất quán, nhưng một nút đã sập sẽ không phản hồi gì cả. Một mặt, điều này hàm ý rằng không nhất thiết phải có nút nào ngừng hoạt động thì mới gặp các vấn đề nhất quán. Mặt khác, điều này lại không đúng trong thế giới thực: có rất nhiều kịch bản sự cố khác nhau (một số trong đó có thể được mô phỏng bằng phân vùng mạng).

CAP implies that we can face consistency problems even if all the nodes are up, but there are connectivity issues between them since we expect every nonfailed node to respond correctly, with no regard to how many nodes may be down.

CAP hàm ý rằng ta có thể gặp các vấn đề nhất quán ngay cả khi tất cả các nút đều sống, nhưng giữa chúng có vấn đề kết nối, vì ta kỳ vọng mọi nút không lỗi đều phản hồi đúng đắn, bất kể có bao nhiêu nút có thể đang ngừng hoạt động.

CAP conjecture is sometimes illustrated as a triangle, as if we could turn a knob and have more or less of all of the three parameters. However, while we can turn a knob and trade consistency for availability, partition tolerance is a property we cannot realistically tune or trade for anything [HALE10].

Phỏng đoán CAP đôi khi được minh họa như một tam giác, như thể ta có thể vặn một núm xoay để có nhiều hơn hay ít hơn cả ba tham số. Tuy nhiên, dù ta có thể vặn núm xoay và đánh đổi nhất quán lấy tính sẵn sàng, thì khả năng chịu phân vùng là một thuộc tính mà ta không thể chỉnh hay đánh đổi lấy bất cứ thứ gì một cách thực tế [HALE10].



Consistency in CAP is defined quite differently from what **ACID** (see Chapter 5) defines as consistency. ACID consistency describes transaction consistency: transaction brings the database from one valid state to another, maintaining all the database invariants (such as uniqueness constraints and referential integrity). In CAP, it means that operations are atomic (operations succeed or fail in their entirety) and consistent (operations never leave the data in an inconsistent

state).

Nhất quán trong CAP được định nghĩa khá khác với điều mà ACID (xem Chương 5) định nghĩa là nhất quán. Nhất quán ACID mô tả nhất quán giao dịch: một giao dịch đưa cơ sở dữ liệu từ trạng thái hợp lệ này sang trạng thái hợp lệ khác, duy trì mọi bất biến của cơ sở dữ liệu (chẳng hạn ràng buộc duy nhất và toàn vẹn tham chiếu). Trong CAP, nó nghĩa là các thao tác là nguyên tử (thao tác thành công hoặc thất bại trọn vẹn) và nhất quán (thao tác không bao giờ để dữ liệu ở trạng thái không nhất quán).

Availability in CAP is also different from the aforementioned high availability [KLEPPMANN15]. The CAP definition puts no bounds on execution latency. Additionally, availability in databases, contrary to CAP, doesn't require every nonfailed node to respond to every request.

Tính sẵn sàng trong CAP cũng khác với tính sẵn sàng cao đã nói ở trên [KLEPPMANN15]. Định nghĩa CAP không đặt giới hạn nào lên độ trễ thực thi. Ngoài ra, tính sẵn sàng trong cơ sở dữ liệu, trái với CAP, không đòi hỏi mọi nút không lỗi phải phản hồi cho mọi yêu cầu.

CAP conjecture is used to explain distributed systems, reason about failure scenarios, and evaluate possible situations, but it's important to remember that there's a fine line between giving up consistency and serving unpredictable results.

Phỏng đoán CAP được dùng để giải thích các hệ phân tán, lập luận về các kịch bản sự cố, và đánh giá các tình huống khả dĩ, nhưng cần nhớ rằng có một ranh giới mong manh giữa việc từ bỏ nhất quán và việc phục vụ các kết quả khó lường.

Databases that claim to be on the availability side, when used correctly, are still able to serve consistent results from replicas, given there are enough replicas alive. Of course, there are more complicated failure scenarios and CAP conjecture is just a rule of thumb, and it doesn't necessarily tell the whole truth. 1

Các cơ sở dữ liệu tự nhận thuộc phía tính sẵn sàng, khi được dùng đúng cách, vẫn có thể phục vụ kết quả nhất quán từ các replica, miễn là có đủ replica còn sống. Dĩ nhiên, có những kịch bản sự cố phức tạp hơn và phỏng đoán CAP chỉ là một quy tắc kinh nghiệm, không nhất thiết kể hết toàn bộ sự thật. 1

Harvest and Yield — Thu hoạch và Năng suất

CAP conjecture discusses consistency and availability only in their strongest forms: linearizability and the ability of the system to eventually respond to every request.

Phỏng đoán CAP bàn về nhất quán và tính sẵn sàng chỉ ở các dạng mạnh nhất của chúng: tuyến tính hóa và khả năng hệ thống rốt cuộc phản hồi cho mọi yêu cầu.

1 **Quorum** reads and writes in the context of eventually consistent stores, which are discussed in more detail in “**Eventual Consistency**” on page 234.

1 Đọc và ghi theo quorum trong bối cảnh các kho lưu nhất quán cuối cùng, được bàn chi tiết hơn ở “**Eventual Consistency**” trang 234.

This forces us to make a hard trade-off between the two properties. However, some applications can benefit from slightly relaxed assumptions and we can think about these properties in their weaker forms.

Điều này buộc ta phải thực hiện một đánh đổi gay gắt giữa hai thuộc tính. Tuy nhiên, một số ứng dụng có thể hưởng lợi từ các giả định được nới lỏng đôi chút và ta có thể nghĩ về các thuộc tính này ở dạng yếu hơn của chúng.

Instead of being either consistent or available, systems can provide relaxed guarantees. We can define two tunable metrics: harvest and yield, choosing between which still constitutes correct behavior [FOX99]:

Thay vì hoặc nhất quán hoặc sẵn sàng, các hệ thống có thể cung cấp các bảo đảm được nói lỏng. Ta có thể định nghĩa hai chỉ số có thể điều chỉnh: thu hoạch (harvest) và năng suất (yield), việc lựa chọn giữa chúng vẫn cấu thành hành vi đúng đắn [FOX99]:

Harvest Defines how complete the query is: if the query has to return 100 rows, but can fetch only 99 due to unavailability of some nodes, it still can be better than failing the query completely and returning nothing.

Thu hoạch Định nghĩa mức độ đầy đủ của truy vấn: nếu truy vấn phải trả về 100 hàng, nhưng chỉ có thể lấy được 99 do một số nút không sẵn sàng, thì điều đó vẫn có thể tốt hơn là để truy vấn thất bại hoàn toàn và không trả về gì.

Yield Specifies the number of requests that were completed successfully, compared to the total number of attempted requests. Yield is different from the uptime, since, for example, a busy node is not down, but still can fail to respond to some of the requests.

Năng suất Chỉ định số lượng yêu cầu được hoàn thành thành công, so với tổng số yêu cầu đã thử. Năng suất khác với uptime, vì, ví dụ, một nút bận thì không phải là ngừng hoạt động, nhưng vẫn có thể không phản hồi được một số yêu cầu.

This shifts the focus of the trade-off from the absolute to the relative terms. We can trade harvest for yield and allow some requests to return incomplete data. One of the ways to increase yield is to return query results only from the available partitions (see “Database Partitioning” on page 270). For example, if a subset of nodes storing records of some users is down, we can still continue serving requests for other users. Alternatively, we can require the critical application data to be returned only in its entirety, but allow some deviations for other requests.

Điều này dịch chuyển trọng tâm của đánh đổi từ thuật ngữ tuyệt đối sang thuật ngữ tương đối. Ta có thể đánh đổi thu hoạch lấy năng suất và cho phép một số yêu cầu trả về dữ liệu không đầy đủ. Một trong những cách để tăng năng suất là chỉ trả về kết quả truy vấn từ các phân vùng sẵn sàng (xem “Database Partitioning” trang 270). Ví dụ, nếu một tập con các nút lưu bản ghi của một số người dùng bị ngừng hoạt động, ta vẫn có thể tiếp tục phục vụ yêu cầu cho những người dùng khác. Ngoài ra, ta có thể yêu cầu rằng dữ liệu ứng dụng trọng yếu chỉ được trả về khi đầy đủ trọn vẹn, nhưng cho phép một số sai lệch đối với các yêu cầu khác.

Defining, measuring, and making a conscious choice between harvest and yield helps us to build systems that are more resilient to failures.

Việc định nghĩa, đo lường, và lựa chọn có ý thức giữa thu hoạch và năng suất giúp ta xây dựng các hệ thống bền bỉ hơn trước sự cố.

Shared Memory — Bộ nhớ chia sẻ

For a client, the distributed system storing the data acts as if it has shared storage, similar to a single-node system. Internode communication and **message passing** are abstracted away and happen behind the scenes. This creates an illusion of a shared memory.

Đối với một client, hệ phân tán lưu trữ dữ liệu hành xử như thể nó có bộ lưu trữ chia sẻ, tương tự một hệ thống đơn nút. Giao tiếp giữa các nút và truyền thông điệp được trừu

tượng hóa đi và diễn ra phía sau hậu trường. Điều này tạo ra ảo giác về một bộ nhớ chia sẻ.

A single unit of storage, accessible by read or write operations, is usually called a register . We can view shared memory in a distributed database as an array of such registers.

Một đơn vị lưu trữ duy nhất, có thể truy cập bằng các thao tác đọc hoặc ghi, thường được gọi là một thanh ghi (register). Ta có thể xem bộ nhớ chia sẻ trong một cơ sở dữ liệu phân tán như một mảng các thanh ghi như vậy.

We identify every operation by its invocation and completion events. We define an operation as failed if the **process** that invoked it crashes before it completes. If both invocation and completion events for one operation happen before the other operation is invoked, we say that this operation precedes the other one, and these two operations are **sequential** . Otherwise, we say that they are concurrent

Ta nhận diện mỗi thao tác qua các sự kiện gọi (invocation) và hoàn tất (completion) của nó. Ta định nghĩa một thao tác là thất bại nếu tiến trình gọi nó bị sập trước khi nó hoàn tất. Nếu cả sự kiện gọi lẫn sự kiện hoàn tất của một thao tác đều diễn ra trước khi thao tác kia được gọi, ta nói rằng thao tác này đứng trước thao tác kia, và hai thao tác đó là tuần tự (sequential). Ngược lại, ta nói chúng là đồng thời (concurrent).

In Figure 11-1 , you can see processes P and P executing different operations:

Trong Hình 11-1, bạn có thể thấy các tiến trình P và P thực thi các thao tác khác nhau:

- a) The operation performed by process P starts after the operation executed by
 - a) Thao tác do tiến trình P thực hiện bắt đầu sau khi thao tác do

P has already finished, and the two operations are sequential .

P thực thi đã kết thúc, và hai thao tác là tuần tự.

- b) There's an overlap between the two operations, so these operations are concurrent .
- c) The operation executed by P starts after and completes before the operation
 - b) Có sự chồng lấn giữa hai thao tác, nên các thao tác này là đồng thời.
 - c) Thao tác do P thực thi bắt đầu sau và hoàn tất trước thao tác

executed by P . These operations are concurrent , too.

do P thực thi. Các thao tác này cũng là đồng thời.

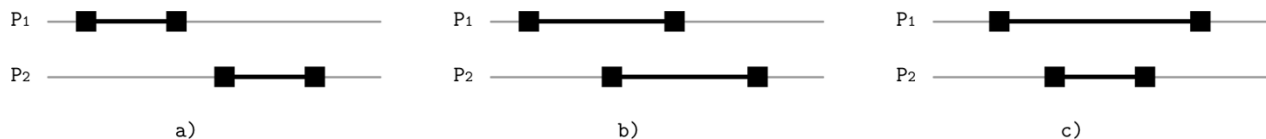


Figure 11-1. Sequential and concurrent operations

Hình 11-1. Các thao tác tuần tự và đồng thời

Multiple readers or writers can access the register simultaneously. Read and write operations on registers are not immediate and take some time. Concurrent read/write operations performed by different processes are not serial : depending on how registers behave when operations overlap, they might be ordered differently and may produce different results. Depending on how the register behaves in the presence of concurrent operations, we distinguish among three types of registers:

Nhiều bên đọc hoặc bên ghi có thể truy cập thanh ghi đồng thời. Các thao tác đọc và ghi trên thanh ghi không tức thời mà tốn một khoảng thời gian. Các thao tác đọc/ghi đồng thời do các tiến trình khác nhau thực hiện không phải là tuần tự (serial): tùy vào cách các thanh ghi hành xử khi các thao tác chồng lấn, chúng có thể được sắp thứ tự khác nhau và có thể tạo ra các kết quả khác nhau. Tùy vào cách thanh ghi hành xử khi có các thao tác đồng thời, ta phân biệt ba loại thanh ghi:

Safe Reads to the safe registers may return arbitrary values within the range of the register during a concurrent write operation (which does not sound very practical, but might describe the semantics of an asynchronous system that does not impose the order). Safe registers with binary values might appear to be flickering (i.e., returning results alternating between the two values) during reads concurrent to writes.

An toàn Các thao tác đọc tới thanh ghi an toàn có thể trả về các giá trị tùy ý trong phạm vi của thanh ghi trong khi một thao tác ghi đang diễn ra đồng thời (điều này nghe có vẻ không thực dụng lắm, nhưng có thể mô tả ngữ nghĩa của một hệ bất đồng bộ không áp đặt thứ tự). Các thanh ghi an toàn với giá trị nhị phân có thể trông như đang nhấp nháy (tức là trả về kết quả thay phiên giữa hai giá trị) trong các lần đọc đồng thời với lần ghi.

Regular For regular registers, we have slightly stronger guarantees: a read operation can return only the value written by the most recent completed write or the value written by the write operation that overlaps with the current read. In this case, the system has some notion of order, but write results are not visible to all the readers simultaneously (for example, this may happen in a replicated database, where the master accepts writes and replicates them to workers serving reads).

Thông thường Đối với thanh ghi thông thường, ta có các bảo đảm mạnh hơn đôi chút: một thao tác đọc chỉ có thể trả về giá trị đã được ghi bởi lần ghi hoàn tất gần nhất hoặc giá trị được ghi bởi thao tác ghi chồng lấn với lần đọc hiện tại. Trong trường hợp này, hệ thống có một khái niệm thứ tự nào đó, nhưng kết quả ghi không hiển thị cho tất cả các bên đọc cùng lúc (ví dụ, điều này có thể xảy ra trong một cơ sở dữ liệu được nhân bản, nơi master chấp nhận ghi và nhân bản chúng tới các worker phục vụ đọc).

Atomic Atomic registers guarantee linearizability: every write operation has a single moment before which every read operation returns an old value and after which every read operation returns a new one. **Atomicity** is a fundamental property that simplifies reasoning about the system state.

Nguyên tử Thanh ghi nguyên tử bảo đảm tính tuyến tính hóa: mỗi thao tác ghi có một thời điểm duy nhất mà trước nó mọi thao tác đọc trả về giá trị cũ và sau nó mọi thao tác đọc trả về giá trị mới. Tính nguyên tử là một thuộc tính nền tảng giúp đơn giản hóa việc lập luận về trạng thái hệ thống.

Ordering — Sắp thứ tự

When we see a sequence of events, we have some intuition about their execution order. However, in a distributed system it's not always that easy, because it's hard to know when exactly something has happened and have this information available instantly across the cluster. Each **participant** may have its view of the state, so we have to look at every operation and define it in terms of its invocation and completion events and describe the operation bounds.

Khi ta thấy một chuỗi sự kiện, ta có một trực giác nào đó về thứ tự thực thi của chúng. Tuy nhiên, trong một hệ phân tán thì điều đó không phải lúc nào cũng dễ, bởi rất khó biết chính xác khi nào một việc gì đó đã xảy ra và có thông tin này sẵn có tức thời trên toàn cụm. Mỗi bên tham gia có thể có góc nhìn riêng về trạng thái, nên ta phải xem xét mọi

thao tác và định nghĩa nó theo các sự kiện gọi và hoàn tất của nó cũng như mô tả các biên của thao tác.

Let's define a system in which processes can execute `read(register)` and `write(register, value)` operations on shared registers. Each process executes its own set of operations sequentially (i.e., every invoked operation has to complete before it can start the next one). The combination of sequential process executions forms a global history, in which operations can be executed concurrently.

Hãy định nghĩa một hệ thống trong đó các tiến trình có thể thực thi các thao tác `read(register)` và `write(register, value)` trên các thanh ghi chia sẻ. Mỗi tiến trình thực thi tập thao tác riêng của nó một cách tuần tự (tức là, mỗi thao tác được gọi phải hoàn tất trước khi nó có thể bắt đầu thao tác kế tiếp). Sự kết hợp các thực thi tuần tự của các tiến trình tạo thành một lịch sử toàn cục (global history), trong đó các thao tác có thể được thực thi đồng thời.

The simplest way to think about consistency models is in terms of read and write operations and ways they can overlap: read operations have no side effects, while writes change the register state. This helps to reason about when exactly data becomes readable after the write. For example, consider a history in which two processes execute the following events concurrently:

Cách đơn giản nhất để nghĩ về các mô hình nhất quán là theo các thao tác đọc và ghi cùng cách chúng có thể chồng lấn: các thao tác đọc không có hiệu ứng phụ, trong khi các thao tác ghi thay đổi trạng thái thanh ghi. Điều này giúp lập luận về thời điểm chính xác mà dữ liệu trở nên đọc được sau khi ghi. Ví dụ, hãy xét một lịch sử trong đó hai tiến trình thực thi các sự kiện sau đây một cách đồng thời:

Process 1:	Process 2:
<code>write(x, 1)</code>	<code>read(x)</code>
	<code>read(x)</code>

When looking at these events, it's unclear what is an outcome of the `read(x)` operations in both cases. We have several possible histories:

Khi nhìn vào các sự kiện này, không rõ kết quả của các thao tác `read(x)` trong cả hai trường hợp là gì. Ta có một số lịch sử khả dĩ:

- Write completes before both reads.
- Write and two reads can get interleaved, and can be executed between the reads.
- Both reads complete before the write.
 - Lần ghi hoàn tất trước cả hai lần đọc.
 - Lần ghi và hai lần đọc có thể đan xen nhau, và lần ghi có thể được thực thi giữa hai lần đọc.
 - Cả hai lần đọc hoàn tất trước lần ghi.

There's no simple answer to what should happen if we have just one copy of data. In a replicated system, we have more combinations of possible states, and it can get even more complicated when we have multiple processes reading and writing the data.

Không có câu trả lời đơn giản cho việc điều gì nên xảy ra nếu ta chỉ có một bản sao dữ liệu. Trong một hệ thống được nhân bản, ta có nhiều tổ hợp trạng thái khả dĩ hơn, và nó còn có thể phức tạp hơn nữa khi ta có nhiều tiến trình đọc và ghi dữ liệu.

If all of these operations were executed by the single process, we could enforce a strict order of events, but it's harder to do so with multiple processes. We can group the potential difficulties into two groups:

Nếu tất cả các thao tác này được thực thi bởi một tiến trình duy nhất, ta có thể áp đặt một thứ tự sự kiện chặt chẽ, nhưng việc đó khó hơn với nhiều tiến trình. Ta có thể gộp các khó khăn tiềm tàng thành hai nhóm:

- Operations may overlap.
- Effects of the nonoverlapping calls might not be visible immediately.
 - Các thao tác có thể chồng lấn.
 - Hiệu ứng của các lời gọi không chồng lấn có thể không hiển thị ngay lập tức.

To reason about the operation order and have nonambiguous descriptions of possible outcomes, we have to define consistency models. We discuss concurrency in distributed systems in terms of shared memory and concurrent systems, since most of the definitions and rules defining consistency still apply. Even though a lot of terminology between concurrent and distributed systems overlap, we can't directly apply most of the concurrent algorithms, because of differences in communication patterns, performance, and reliability.

Để lập luận về thứ tự thao tác và có những mô tả không nhập nhằng về các kết cục khả dĩ, ta phải định nghĩa các mô hình nhất quán. Ta bàn về tương tranh trong các hệ phân tán theo thuật ngữ bộ nhớ chia sẻ và các hệ thống tương tranh, vì hầu hết các định nghĩa và quy tắc xác định nhất quán vẫn áp dụng được. Mặc dù nhiều thuật ngữ giữa các hệ tương tranh và hệ phân tán trùng nhau, ta không thể áp dụng trực tiếp hầu hết các thuật toán tương tranh, do những khác biệt về kiểu mẫu giao tiếp, hiệu năng, và độ tin cậy.

Consistency Models — Các mô hình nhất quán

Since operations on shared memory registers are allowed to overlap, we should define clear semantics: what happens if multiple clients read or modify different copies of data simultaneously or within a short period. There's no single right answer to that question, since these semantics are different depending on the application, but they are well studied in the context of consistency models.

Vì các thao tác trên thanh ghi bộ nhớ chia sẻ được phép chồng lấn, ta nên định nghĩa ngữ nghĩa rõ ràng: điều gì xảy ra nếu nhiều client đọc hoặc sửa đổi các bản sao dữ liệu khác nhau đồng thời hoặc trong một khoảng thời gian ngắn. Không có một câu trả lời đúng duy nhất cho câu hỏi đó, vì các ngữ nghĩa này khác nhau tùy theo ứng dụng, nhưng chúng đã được nghiên cứu kỹ trong bối cảnh các mô hình nhất quán.

Consistency models provide different semantics and guarantees. You can think of a **consistency model** as a contract between the participants: what each replica has to do to satisfy the required semantics, and what users can expect when issuing read and write operations.

Các mô hình nhất quán cung cấp các ngữ nghĩa và bảo đảm khác nhau. Bạn có thể nghĩ về một mô hình nhất quán như một hợp đồng giữa các bên tham gia: mỗi replica phải làm gì để thỏa mãn ngữ nghĩa được yêu cầu, và người dùng có thể kỳ vọng gì khi phát ra các thao tác đọc và ghi.

Consistency models describe what expectations clients might have in terms of possible returned values despite the existence of multiple copies of data and concurrent accesses to it. In this section, we will discuss single-operation consistency models.

Các mô hình nhất quán mô tả những kỳ vọng mà client có thể có về các giá trị khả dĩ được trả về, bất chấp sự tồn tại của nhiều bản sao dữ liệu và các truy cập đồng thời tới nó. Trong phần này, ta sẽ bàn về các mô hình nhất quán đơn thao tác (single-operation).

Each model describes how far the behavior of the system is from the behavior we might expect or find natural. It helps us to distinguish between “all possible histories” of interleaving operations and “histories permissible under model X,” which significantly simplifies reasoning about the visibility of state changes.

Mỗi mô hình mô tả hành vi của hệ thống cách bao xa so với hành vi mà ta có thể kỳ vọng hoặc thấy là tự nhiên. Nó giúp ta phân biệt giữa “tất cả các lịch sử khả dĩ” của các thao tác đan xen và “các lịch sử được phép theo mô hình X,” điều này đơn giản hóa đáng kể việc lập luận về tính hiển thị của các thay đổi trạng thái.

We can think about consistency from the perspective of state, describe which state invariants are acceptable, and establish allowable relationships between copies of the data placed onto different replicas. Alternatively, we can consider operation consistency, which provides an outside view on the data store, describes operations, and puts constraints on the order in which they occur [TANENBAUM06] [AGUI- LERA16].

Ta có thể nghĩ về nhất quán dưới góc nhìn trạng thái (state), mô tả những bất biến trạng thái nào là chấp nhận được, và thiết lập các quan hệ cho phép giữa các bản sao dữ liệu đặt trên các replica khác nhau. Ngoài ra, ta có thể xét nhất quán thao tác, vốn cung cấp một góc nhìn bên ngoài về kho dữ liệu, mô tả các thao tác, và đặt các ràng buộc lên thứ tự xảy ra của chúng [TANENBAUM06] [AGUILERA16].

Without a global clock, it is difficult to give distributed operations a precise and deterministic order. It's like a Special Relativity Theory for data: every participant has its own perspective on state and time.

Không có đồng hồ toàn cục, rất khó để gán cho các thao tác phân tán một thứ tự chính xác và xác định. Nó giống như Thuyết Tương đối Hẹp dành cho dữ liệu: mỗi bên tham gia có góc nhìn riêng của mình về trạng thái và thời gian.

Theoretically, we could grab a system-wide **lock** every time we want to change the system state, but it'd be highly impractical. Instead, we use a set of rules, definitions, and restrictions that limit the number of possible histories and outcomes.

Về lý thuyết, ta có thể chiếm một khóa phạm vi toàn hệ thống mỗi khi muốn thay đổi trạng thái hệ thống, nhưng điều đó sẽ rất phi thực tế. Thay vào đó, ta dùng một tập các quy tắc, định nghĩa, và hạn chế nhằm giới hạn số lượng lịch sử và kết cục khả dĩ.

Consistency models add another dimension to what we discussed in “Infamous CAP” on page 216. Now we have to juggle not only consistency and availability, but also consider consistency in terms of synchronization costs [ATTIYA94]. Synchronization costs may include latency, additional CPU cycles spent executing additional operations, disk I/O used to persist recovery information, wait time, network I/O, and everything else that can be prevented by avoiding synchronization.

Các mô hình nhất quán bổ sung một chiều khác vào những gì ta đã bàn ở “Infamous CAP” trang 216. Giờ ta phải tung hứng không chỉ nhất quán và tính sẵn sàng, mà còn phải xét nhất quán theo thuật ngữ chi phí đồng bộ hóa [ATTIYA94]. Chi phí đồng bộ hóa có thể bao gồm độ trễ, các chu kỳ CPU bổ sung dành cho việc thực thi các thao tác bổ sung, I/O đĩa dùng để lưu bền thông tin phục hồi, thời gian chờ, I/O mạng, và mọi thứ khác có thể được ngăn ngừa bằng cách tránh đồng bộ hóa.

First, we'll focus on visibility and propagation of operation results. Coming back to the example with concurrent reads and writes, we'll be able to limit the number of possible histories by either positioning dependent writes after one another or defining a point at which the new value is propagated.

Trước hết, ta sẽ tập trung vào tính hiển thị và sự lan truyền của các kết quả thao tác. Quay lại ví dụ với các lần đọc và ghi đồng thời, ta sẽ có thể giới hạn số lượng lịch sử khả dĩ bằng cách hoặc đặt các lần ghi phụ thuộc nối tiếp nhau hoặc định nghĩa một điểm mà tại đó giá trị mới được lan truyền.

We discuss consistency models in terms of processes (clients) issuing read and write operations against the database state. Since we discuss consistency in the context of replicated data, we assume that the database can have multiple replicas.

Ta bàn về các mô hình nhất quán theo các tiến trình (client) phát ra các thao tác đọc và ghi đối với trạng thái cơ sở dữ liệu. Vì ta bàn về nhất quán trong bối cảnh dữ liệu được nhân bản, ta giả định rằng cơ sở dữ liệu có thể có nhiều replica.

Strict Consistency — Nhất quán nghiêm ngặt

Strict consistency is the equivalent of complete replication transparency: any write by any process is instantly available for the subsequent reads by any process. It involves the concept of a global clock and, if there was a write($x, 1$) at instant t , any

Nhất quán nghiêm ngặt (strict consistency) tương đương với sự trong suốt nhân bản hoàn toàn: bất kỳ lần ghi nào bởi bất kỳ tiến trình nào đều tức thời sẵn có cho các lần đọc kế tiếp bởi bất kỳ tiến trình nào. Nó kéo theo khái niệm về một đồng hồ toàn cục và, nếu có một write($x, 1$) tại thời điểm t , thì bất kỳ

read(x) will return a newly written value 1 at any instant $t > t$.

read(x) nào cũng sẽ trả về giá trị 1 mới được ghi tại bất kỳ thời điểm $t > t$ nào.

Unfortunately, this is just a theoretical model, and it's impossible to implement, as the laws of physics and the way distributed systems work set limits on how fast things may happen [SINHA97].

Tiếc thay, đây chỉ là một mô hình lý thuyết, và bất khả thi để hiện thực, vì các định luật vật lý và cách các hệ phân tán hoạt động đặt ra giới hạn về tốc độ mà mọi việc có thể diễn ra [SINHA97].

Linearizability — Tính tuyến tính hóa

Linearizability is the strongest single-object, single-operation consistency model. Under this model, effects of the write become visible to all readers exactly once at some point in time between its start and end, and no client can observe state transitions or side effects of partial (i.e., unfinished, still in-flight) or incomplete (i.e., interrupted before completion) write operations [LEE15].

Tính tuyến tính hóa là mô hình nhất quán đơn-đối-tượng, đơn-thao-tác mạnh nhất. Dưới mô hình này, hiệu ứng của lần ghi trở nên hiển thị cho tất cả các bên đọc đúng một lần tại một thời điểm nào đó giữa lúc nó bắt đầu và kết thúc, và không client nào có thể quan sát các chuyển trạng thái hay hiệu ứng phụ của các thao tác ghi cục bộ (tức là chưa xong, vẫn đang dang dở) hoặc không hoàn chỉnh (tức là bị gián đoạn trước khi hoàn tất) [LEE15].

Concurrent operations are represented as one of the possible sequential histories for which visibility properties hold. There is some indeterminism in linearizability, as there may exist more than one way in which the events can be ordered [HER- LIHY90].

Các thao tác đồng thời được biểu diễn như một trong những lịch sử tuần tự khả dĩ mà các thuộc tính hiển thị vẫn đúng. Có một sự bất định nào đó trong tính tuyến tính hóa, vì có thể tồn tại nhiều hơn một cách mà các sự kiện có thể được sắp thứ tự [HERLIHY90].

If two operations overlap, they may take effect in any order. All read operations that occur after write operation completion can observe the effects of this operation. As soon as a single read operation returns a particular value, all reads that come after it return the value at least as recent as the one it returns [BAILIS14a].

Nếu hai thao tác chồng lấn, chúng có thể có hiệu lực theo bất kỳ thứ tự nào. Mọi thao tác đọc xảy ra sau khi thao tác ghi hoàn tất đều có thể quan sát được hiệu ứng của thao tác này. Ngay khi một thao tác đọc đơn lẻ trả về một giá trị cụ thể, mọi lần đọc đến sau nó đều trả về giá trị ít nhất cũng mới như giá trị mà nó trả về [BAILIS14a].

There is some flexibility in terms of the order in which concurrent events occur in a global history, but they cannot be reordered arbitrarily. Operation results should not become effective before the operation starts as that would require an oracle able to predict future operations. At the same time, results have to take effect before completion, since otherwise, we cannot define a linearization point.

Có một sự linh hoạt nào đó về thứ tự xảy ra của các sự kiện đồng thời trong một lịch sử toàn cục, nhưng chúng không thể bị sắp lại tùy tiện. Kết quả của thao tác không nên trở nên có hiệu lực trước khi thao tác bắt đầu, vì điều đó sẽ đòi hỏi một nhà tiên tri có khả năng dự đoán các thao tác tương lai. Đồng thời, kết quả phải có hiệu lực trước khi hoàn tất, vì nếu không, ta không thể định nghĩa một điểm tuyến tính hóa.

Linearizability respects both sequential process-local operation order and the order of operations running in parallel relative to other processes, and defines a total order of the events.

Tính tuyến tính hóa tôn trọng cả thứ tự thao tác cục bộ theo tiến trình lẫn thứ tự của các thao tác chạy song song tương đối so với các tiến trình khác, và định nghĩa một thứ tự toàn phần của các sự kiện.

This order should be consistent, which means that every read of the shared value should return the latest value written to this shared variable preceding this read, or the value of a write that overlaps with this read. Linearizable write access to a shared variable also implies mutual exclusion: between the two concurrent writes, only one can go first.

Thứ tự này phải nhất quán, nghĩa là mỗi lần đọc giá trị chia sẻ phải trả về giá trị mới nhất được ghi vào biến chia sẻ này trước lần đọc đó, hoặc giá trị của một lần ghi chồng lấn với lần đọc này. Truy cập ghi tuyến tính hóa tới một biến chia sẻ cũng hàm ý loại trừ tương hỗ (mutual exclusion): giữa hai lần ghi đồng thời, chỉ một lần có thể đi trước.

Even though operations are concurrent and have some overlap, their effects become visible in a way that makes them appear sequential. No operation happens instantaneously, but still appears to be atomic.

Mặc dù các thao tác là đồng thời và có sự chồng lấn nào đó, hiệu ứng của chúng trở nên hiển thị theo cách khiến chúng trông như tuần tự. Không thao tác nào xảy ra tức thời, nhưng vẫn trông như nguyên tử.

Let's consider the following history:

Hãy xét lịch sử sau đây:

Process 1:	Process 2:	Process 3:
write(x, 1)	write(x, 2)	read(x)
		read(x)

read(x)

In Figure 11-2, we have three processes, two of which perform write operations on the register x, which has an initial value of $\emptyset$. Read operations can observe these writes in one of the following ways:

Trong Hình 11-2, ta có ba tiến trình, hai trong số đó thực hiện các thao tác ghi trên thanh ghi x, vốn có giá trị ban đầu là $\emptyset$. Các thao tác đọc có thể quan sát các lần ghi này theo một trong các cách sau:

- a) The first read operation can return 1, 2, or $\emptyset$ (the initial value, a state before both writes), since both writes are still in-flight. The first read can get ordered before both writes, between the first and second writes, and after both writes.
- b) The second read operation can return only 1 and 2, since the first write has completed, but the second write didn't return yet.
- c) The third read can only return 2, since the second write is ordered after the first.
 - a) Thao tác đọc đầu tiên có thể trả về 1, 2, hoặc $\emptyset$ (giá trị ban đầu, một trạng thái trước cả hai lần ghi), vì cả hai lần ghi vẫn đang dang dở. Lần đọc đầu tiên có thể được sắp thứ tự trước cả hai lần ghi, giữa lần ghi thứ nhất và thứ hai, và sau cả hai lần ghi.
 - b) Thao tác đọc thứ hai chỉ có thể trả về 1 và 2, vì lần ghi thứ nhất đã hoàn tất, nhưng lần ghi thứ hai chưa trả về.
 - c) Lần đọc thứ ba chỉ có thể trả về 2, vì lần ghi thứ hai được sắp thứ tự sau lần ghi thứ nhất.

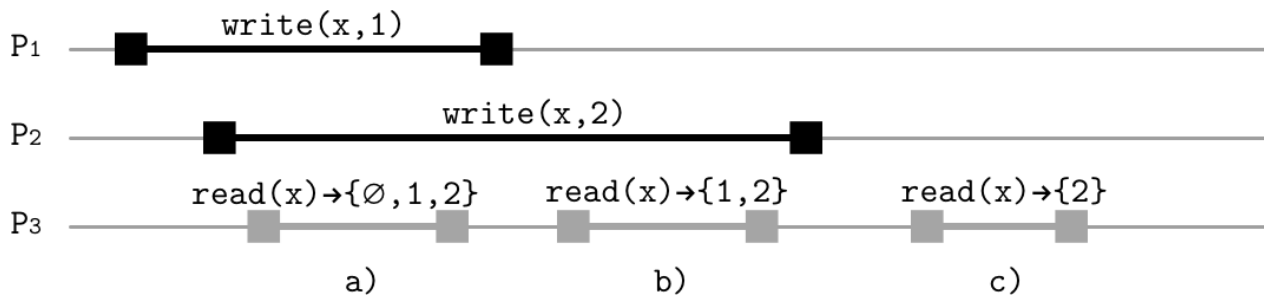


Figure 11-2. Example of linearizability Linearization point

Hình 11-2. Ví dụ về tính tuyến tính hóa. Điểm tuyến tính hóa

One of the most important traits of linearizability is visibility: once the operation is complete, everyone must see it, and the system can't "travel back in time," reverting it or making it invisible for some participants. In other words, linearization prohibits stale reads and requires reads to be monotonic.

Một trong những đặc điểm quan trọng nhất của tính tuyến tính hóa là tính hiển thị: một khi thao tác đã hoàn tất, mọi bên phải thấy nó, và hệ thống không thể "du hành ngược thời gian," hoàn tác nó hay làm cho nó vô hình đối với một số bên tham gia. Nói cách khác, tuyến tính hóa cấm các lần đọc cũ (stale read) và đòi hỏi các lần đọc phải đơn điệu (monotonic).

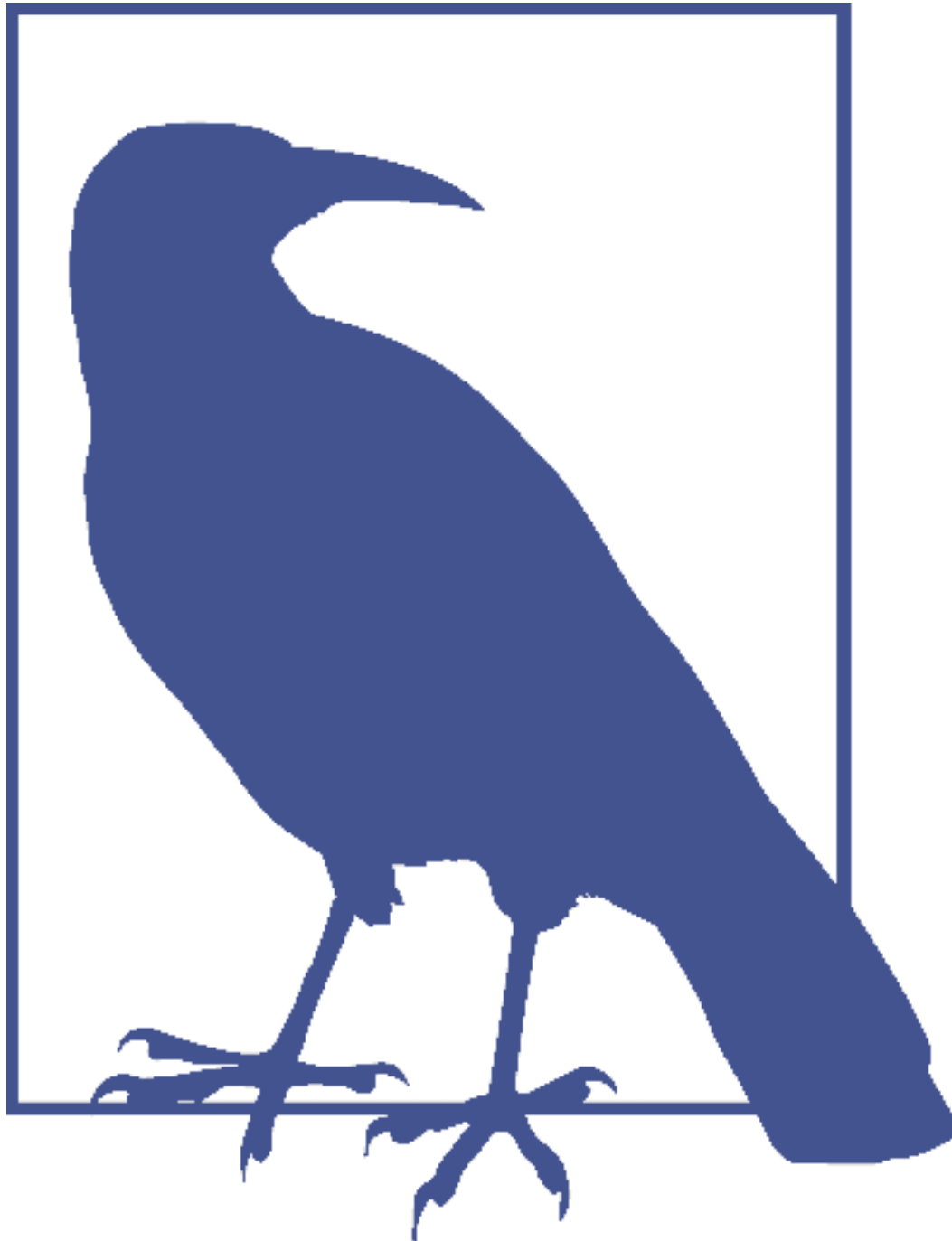
This consistency model is best explained in terms of atomic (i.e., uninterruptible, indivisible) operations. Operations do not have to be instantaneous (also because there's no such thing), but

their effects have to become visible at some point in time, making an illusion that they were instantaneous. This moment is called a linearization point .

Mô hình nhất quán này được giải thích tốt nhất theo các thao tác nguyên tử (tức là không gián đoạn, không thể chia tách). Các thao tác không nhất thiết phải tức thời (cũng vì không có thứ gì như vậy cả), nhưng hiệu ứng của chúng phải trở nên hiển thị tại một thời điểm nào đó, tạo ảo giác rằng chúng tức thời. Thời điểm này được gọi là điểm tuyến tính hóa (linearization point).

Past the linearization point of the write operation (in other words, when the value becomes visible for other processes) every process has to see either the value this operation wrote or some later value, if some additional write operations are ordered after it. A visible value should remain stable until the next one becomes visible after it, and the register should not alternate between the two recent states.

Sau điểm tuyến tính hóa của thao tác ghi (nói cách khác, khi giá trị trở nên hiển thị cho các tiến trình khác), mọi tiến trình phải thấy hoặc giá trị mà thao tác này đã ghi, hoặc một giá trị mới hơn nào đó, nếu có một số thao tác ghi bổ sung được sắp thứ tự sau nó. Một giá trị hiển thị phải giữ ổn định cho đến khi giá trị kế tiếp trở nên hiển thị sau nó, và thanh ghi không nên thay phiên giữa hai trạng thái gần đây.



Most of the programming languages these days offer atomic primitives that allow atomic write and compare-and-swap (CAS) operations. Atomic write operations do not consider current register values, unlike CAS, that move from one value to the next only when the previous value is unchanged [HERLIHY94] . Reading the value, modifying it, and then writing it with CAS is more complex than simply checking and setting the value, because of the possible ABA problem [DECHEV10] : if CAS expects the value A to be present in the register, it will be installed even if the value B was set and then switched back to A by the other two concurrent write operations. In other words, the presence of the value A alone does not guarantee that the value hasn't been changed since the last read.

Hầu hết các ngôn ngữ lập trình ngày nay cung cấp các nguyên thủy nguyên tử cho phép

các thao tác ghi nguyên tử và so-sánh-rồi-hoán-đổi (compare-and-swap, CAS). Các thao tác ghi nguyên tử không xét giá trị thanh ghi hiện tại, khác với CAS, vốn chỉ chuyển từ giá trị này sang giá trị kế khi giá trị trước đó không đổi [HERLIHY94]. Đọc giá trị, sửa đổi nó, rồi ghi nó bằng CAS thì phức tạp hơn là chỉ kiểm tra và đặt giá trị, vì có thể gặp vấn đề ABA [DECHEV10]: nếu CAS kỳ vọng giá trị A có mặt trong thanh ghi, nó sẽ được cài đặt ngay cả khi giá trị B đã được đặt rồi sau đó chuyển lại về A bởi hai thao tác ghi đồng thời khác. Nói cách khác, sự hiện diện của riêng giá trị A không bảo đảm rằng giá trị chưa từng bị thay đổi kể từ lần đọc trước.

The linearization point serves as a cutoff, after which operation effects become visible. We can implement it by using locks to guard a critical section, atomic read/write, or read-modify-write primitives.

Điểm tuyến tính hóa đóng vai trò một mốc cắt, sau đó các hiệu ứng thao tác trở nên hiển thị. Ta có thể hiện thực nó bằng cách dùng khóa để canh giữ một vùng găng (critical section), bằng đọc/ghi nguyên tử, hoặc bằng các nguyên thủy đọc-sửa-ghi.

Figure 11-3 shows that linearizability assumes hard time bounds and the clock is real time, so the operation effects have to become visible between t_1 , when the operation

Hình 11-3 cho thấy tính tuyến tính hóa giả định các giới hạn thời gian cứng và đồng hồ là thời gian thực, nên hiệu ứng thao tác phải trở nên hiển thị giữa t_1 , khi yêu cầu thao tác

request was issued, and t_2 , when the process received a response.

được phát ra, và t_2 , khi tiến trình nhận được phản hồi.



Figure 11-3. Time bounds of a linearizable operation Figure 11-4 illustrates that the linearization point cuts the history into before and after. Before the linearization point, the old value is visible, after it, the new value is visible.

Hình 11-3. Các giới hạn thời gian của một thao tác tuyến tính hóa. Hình 11-4 minh họa rằng điểm tuyến tính hóa cắt lịch sử thành trước và sau. Trước điểm tuyến tính hóa, giá trị cũ hiển thị; sau nó, giá trị mới hiển thị.

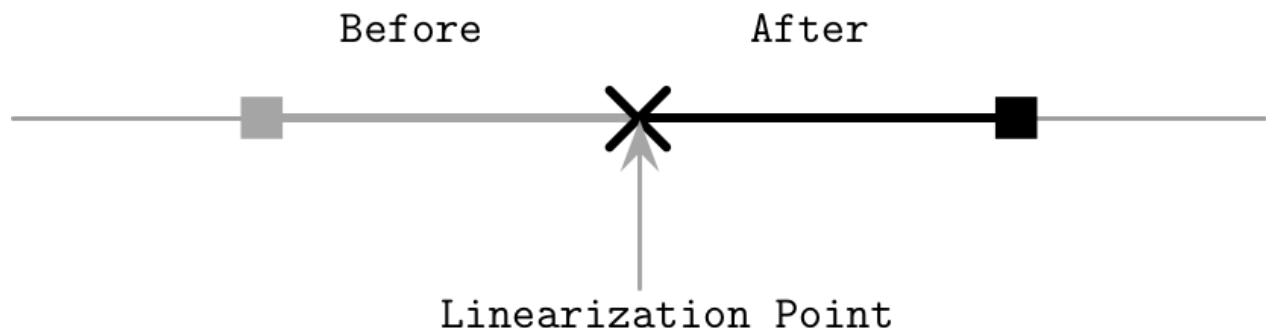


Figure 11-4. Linearization point

Hình 11-4. Điểm tuyến tính hóa

Cost of linearizability

Chi phí của tính tuyến tính hóa

Many systems avoid implementing linearizability today. Even CPUs do not offer linearizability when accessing main memory by default. This has happened because synchronization instructions are expensive, slow, and involve cross-node CPU traffic and cache invalidations. However, it is possible to implement linearizability using low-level primitives [MCKENNEY05a] , [MCKENNEY05b] .

Nhiều hệ thống ngày nay tránh hiện thực tính tuyến tính hóa. Ngay cả các CPU cũng không cung cấp tính tuyến tính hóa khi truy cập bộ nhớ chính theo mặc định. Điều này xảy ra vì các chỉ thị đồng bộ hóa tốn kém, chậm, và kéo theo lưu lượng CPU liên-nút cùng việc vô hiệu hóa cache. Tuy nhiên, vẫn có thể hiện thực tính tuyến tính hóa bằng các nguyên thủy cấp thấp [MCKENNEY05a], [MCKENNEY05b].

In concurrent programming, you can use compare-and-swap operations to introduce linearizability. Many algorithms work by preparing results and then using CAS for swapping pointers and publishing them. For example, we can implement a concurrent queue by creating a linked list node and then atomically appending it to the tail of the list [KHANCHANDANI18] .

Trong lập trình tương tranh, bạn có thể dùng các thao tác so-sánh-rồi-hoán-đổi để đưa vào tính tuyến tính hóa. Nhiều thuật toán hoạt động bằng cách chuẩn bị kết quả rồi dùng CAS để hoán đổi các con trỏ và công bố chúng. Ví dụ, ta có thể hiện thực một hàng đợi tương tranh bằng cách tạo một nút danh sách liên kết rồi nối nó một cách nguyên tử vào đuôi danh sách [KHANCHANDANI18].

In distributed systems, linearizability requires coordination and ordering. It can be implemented using consensus : clients interact with a replicated store using messages, and the consensus module is responsible for ensuring that applied operations are consistent and identical across the cluster. Each write operation will appear instantaneously, exactly once at some point between its invocation and completion events [HOWARD14] .

Trong các hệ phân tán, tính tuyến tính hóa đòi hỏi điều phối và sắp thứ tự. Nó có thể được hiện thực bằng đồng thuận: các client tương tác với một kho được nhân bản bằng các thông điệp, và mô-đun đồng thuận chịu trách nhiệm bảo đảm rằng các thao tác được áp dụng là nhất quán và đồng nhất trên toàn cụm. Mỗi thao tác ghi sẽ xuất hiện tức thời, đúng một lần tại một thời điểm nào đó giữa các sự kiện gọi và hoàn tất của nó [HOWARD14].

Interestingly, linearizability in its traditional understanding is regarded as a local property and implies composition of independently implemented and verified elements. Combining linearizable histories produces a history that is also linearizable [HERLIHY90] . In other words, a system in which all objects are linearizable, is also linearizable. This is a very useful property, but we should remember that its scope is limited to a single object and, even though operations on two independent objects are linearizable, operations that involve both objects have to rely on additional synchronization means.

Điều thú vị là, tính tuyến tính hóa theo cách hiểu truyền thống được coi là một thuộc tính cục bộ và hàm ý sự kết hợp của các phần tử được hiện thực và kiểm chứng độc lập. Kết hợp các lịch sử tuyến tính hóa tạo ra một lịch sử cũng tuyến tính hóa [HERLIHY90]. Nói cách khác, một hệ thống mà trong đó mọi đối tượng đều tuyến tính hóa thì cũng tuyến tính hóa. Đây là một thuộc tính rất hữu ích, nhưng ta nên nhớ rằng phạm vi của nó giới hạn ở một đối tượng đơn lẻ và, dù các thao tác trên hai đối tượng độc lập đều tuyến tính hóa, các thao tác liên quan đến cả hai đối tượng vẫn phải dựa vào các phương tiện đồng bộ hóa bổ sung.

Reusable Infrastructure for Linearizability — Hạ tầng tái sử dụng cho Tính tuyến tính hóa

Reusable Infrastructure for Linearizability (RIFL), is a mechanism for implementing linearizable remote procedure calls (RPCs) [LEE15] . In RIFL, messages are uniquely identified with the client ID and a client-local monotonically increasing sequence number.

Hạ tầng Tái sử dụng cho Tính tuyến tính hóa (Reusable Infrastructure for Linearizability, RIFL), là một cơ chế để hiện thực các lời gọi thủ tục từ xa (RPC) tuyến tính hóa [LEE15]. Trong RIFL, các thông điệp được nhận diện duy nhất bằng ID client và một số thứ tự đơn điệu tăng dần cục bộ theo client.

To assign client IDs, RIFL uses leases , issued by the system-wide service: unique identifiers used to establish uniqueness and break sequence number ties. If the failed client tries to execute an operation using an expired lease, its operation will not be committed: the client has to receive a new lease and retry.

Để gán ID client, RIFL dùng các lease (giấy thuê), do dịch vụ phạm vi toàn hệ thống cấp phát: các định danh duy nhất dùng để thiết lập tính duy nhất và phá vỡ các trùng số thứ tự. Nếu một client bị lỗi cố thực thi một thao tác bằng một lease đã hết hạn, thao tác của nó sẽ không được commit: client phải nhận một lease mới và thử lại.

If the server crashes before it can acknowledge the write, the client may attempt to retry this operation without knowing that it has already been applied. We can even end up in a situation in which client C1 writes value V1 , but doesn't receive an acknowledgment. Meanwhile, client C2 writes value V2 . If C1 retries its operation and successfully writes V1 , the write of C2 would be lost. To avoid this, the system needs to prevent repeated execution of retried operations. When the client retries the operation, instead of reapplying it, RIFL returns a completion object, indicating that the operation it's associated with has already been executed, and returns its result.

Nếu server sập trước khi nó có thể xác nhận (acknowledge) lần ghi, client có thể cố thử lại thao tác này mà không biết rằng nó đã được áp dụng rồi. Ta thậm chí có thể rơi vào tình huống mà client C1 ghi giá trị V1, nhưng không nhận được xác nhận. Trong khi đó, client C2 ghi giá trị V2. Nếu C1 thử lại thao tác của nó và ghi thành công V1, lần ghi của C2 sẽ bị mất. Để tránh điều này, hệ thống cần ngăn việc thực thi lặp lại các thao tác được thử lại. Khi client thử lại thao tác, thay vì áp dụng lại nó, RIFL trả về một đối tượng hoàn tất (completion object), cho biết rằng thao tác mà nó gắn với đã được thực thi rồi, và trả về kết quả của thao tác đó.

Completion objects are stored in a durable storage, along with the actual data records. However, their lifetimes are different: the completion object should exist until either the issuing client promises it won't retry the operation associated with it, or until the server detects a client crash, in which case all completion objects associated with it can be safely removed. Creating a completion object should be atomic with the mutation of the data record it is associated with.

Các đối tượng hoàn tất được lưu trong bộ lưu trữ bền, cùng với các bản ghi dữ liệu thực tế. Tuy nhiên, vòng đời của chúng khác nhau: đối tượng hoàn tất nên tồn tại cho đến khi hoặc client phát hành nó hứa rằng sẽ không thử lại thao tác gắn với nó, hoặc cho đến khi server phát hiện một client bị sập, trong trường hợp đó mọi đối tượng hoàn tất gắn với nó có thể được loại bỏ một cách an toàn. Việc tạo một đối tượng hoàn tất nên là nguyên tử cùng với việc đột biến bản ghi dữ liệu mà nó gắn với.

Clients have to periodically renew their leases to signal their liveness. If the client fails to renew its lease, it is marked as crashed and all the data associated with its lease is garbage collected. Leases

have a limited lifetime to make sure that operations that belong to the failed process won't be retained in the log forever. If the failed client tries to continue operation using an expired lease, its results will not be committed and the client will have to start from scratch.

Các client phải định kỳ gia hạn các lease của mình để báo hiệu chúng còn sống. Nếu client không gia hạn được lease, nó bị đánh dấu là đã sập và mọi dữ liệu gắn với lease của nó được thu gom rác. Các lease có vòng đời giới hạn để bảo đảm rằng các thao tác thuộc về một tiến trình bị lỗi sẽ không bị giữ lại trong nhật ký mãi mãi. Nếu một client bị lỗi cố tiếp tục thao tác bằng một lease đã hết hạn, kết quả của nó sẽ không được commit và client sẽ phải bắt đầu lại từ đầu.

The advantage of RIFL is that, by guaranteeing that the RPC cannot be executed more than once, an operation can be made linearizable by ensuring that its results are made visible atomically, and most of its implementation details are independent from the underlying storage system.

Lợi thế của RIFL là, bằng cách bảo đảm rằng RPC không thể được thực thi nhiều hơn một lần, một thao tác có thể được làm cho tuyến tính hóa bằng cách bảo đảm rằng kết quả của nó được làm cho hiển thị một cách nguyên tử, và hầu hết các chi tiết hiện thực của nó độc lập với hệ thống lưu trữ bên dưới.

Sequential Consistency — Nhất quán tuần tự

Achieving linearizability might be too expensive, but it is possible to relax the model, while still providing rather strong consistency guarantees. Sequential consistency allows ordering operations as if they were executed in some sequential order, while requiring operations of each individual process to be executed in the same order they were performed by the process.

Đạt được tính tuyến tính hóa có thể quá tốn kém, nhưng vẫn có thể nới lỏng mô hình mà vẫn cung cấp các bảo đảm nhất quán khá mạnh. Nhất quán tuần tự (sequential consistency) cho phép sắp thứ tự các thao tác như thể chúng được thực thi theo một thứ tự tuần tự nào đó, đồng thời đòi hỏi các thao tác của mỗi tiến trình riêng lẻ phải được thực thi theo đúng thứ tự mà tiến trình đã thực hiện chúng.

Processes can observe operations executed by other participants in the order consistent with their own history, but this view can be arbitrarily stale from the global perspective [KINGSBURY18a]. Order of execution between processes is undefined, as there's no shared notion of time.

Các tiến trình có thể quan sát các thao tác do các bên tham gia khác thực thi theo thứ tự nhất quán với lịch sử của riêng chúng, nhưng góc nhìn này có thể cũ một cách tùy tiện so với góc nhìn toàn cục [KINGSBURY18a]. Thứ tự thực thi giữa các tiến trình là không xác định, vì không có khái niệm thời gian chia sẻ.

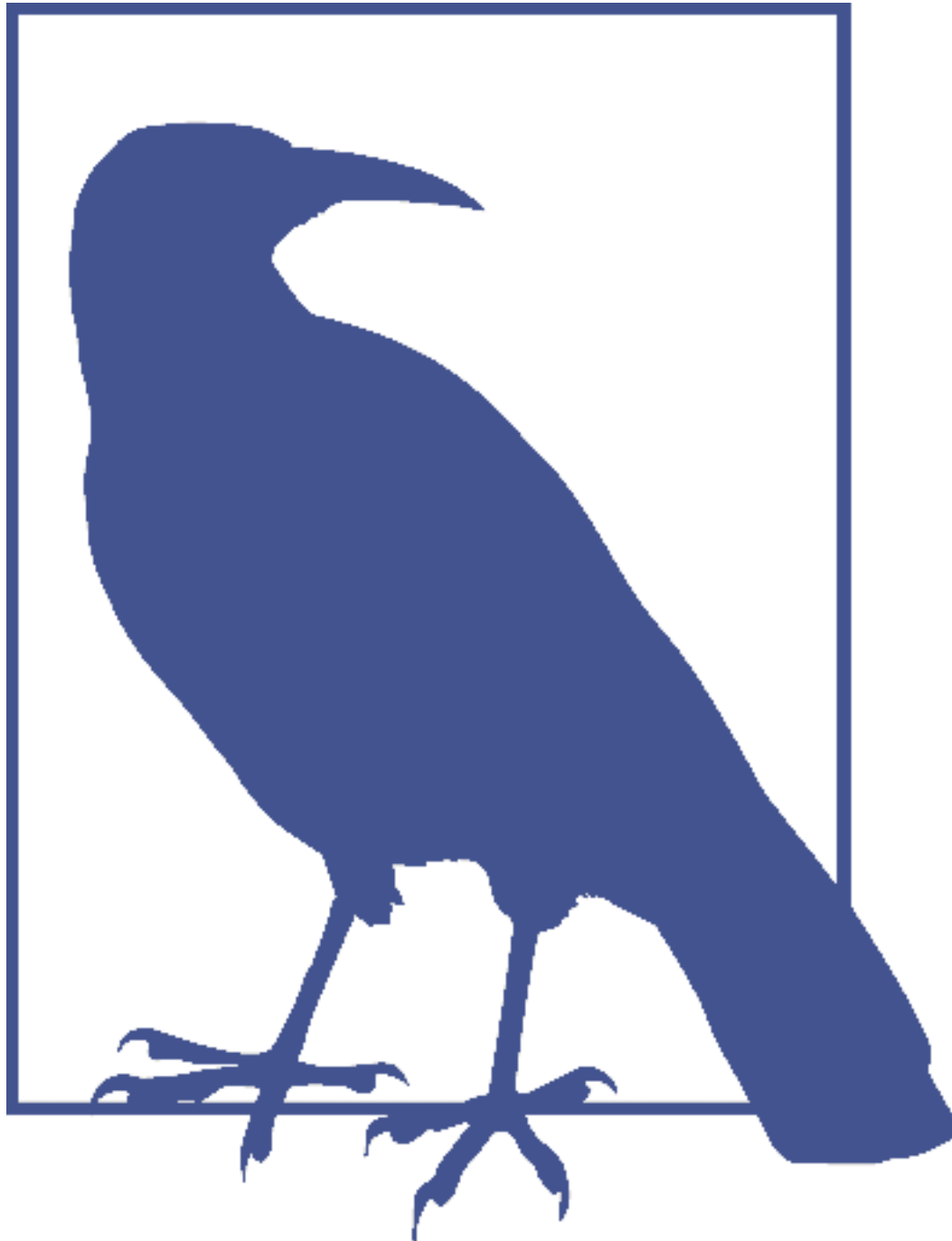
Sequential consistency was initially introduced in the context of concurrency, describing it as a way to execute multiprocessor programs correctly. The original description required memory requests to the same **cell** to be ordered in the queue (FIFO, arrival order), did not impose global ordering on the overlapping writes to independent memory cells, and allowed reads to fetch the value from the memory cell, or the latest value from the queue if the queue was nonempty [LAMPORT79]. This example helps to understand the semantics of sequential consistency. Operations can be ordered in different ways (depending on the arrival order, or even arbitrarily in case two writes arrive simultaneously), but all processes observe the operations in the same order.

Nhất quán tuần tự ban đầu được giới thiệu trong bối cảnh tương tranh, mô tả nó như một cách để thực thi đúng đắn các chương trình đa vi xử lý. Mô tả gốc đòi hỏi các yêu cầu bộ nhớ tới cùng một ô phải được sắp thứ tự trong hàng đợi (FIFO, theo thứ tự đến), không

áp đặt sắp thứ tự toàn cục lên các lần ghi chồng lấn tới các ô bộ nhớ độc lập, và cho phép các lần đọc lấy giá trị từ ô bộ nhớ, hoặc giá trị mới nhất từ hàng đợi nếu hàng đợi không rỗng [LAMPOR79]. Ví dụ này giúp hiểu ngữ nghĩa của nhất quán tuần tự. Các thao tác có thể được sắp thứ tự theo các cách khác nhau (tùy theo thứ tự đến, hoặc thậm chí tùy tiện trong trường hợp hai lần ghi đến đồng thời), nhưng tất cả các tiến trình đều quan sát các thao tác theo cùng một thứ tự.

Each process can issue read and write requests in an order specified by its own program, which is very intuitive. Any nonconcurrent, single-threaded program executes its steps this way: one after another. All write operations propagating from the same process appear in the order they were submitted by this process. Operations propagating from different sources may be ordered arbitrarily, but this order will be consistent from the readers' perspective.

Mỗi tiến trình có thể phát ra các yêu cầu đọc và ghi theo một thứ tự được chỉ định bởi chương trình của riêng nó, điều này rất trực giác. Bất kỳ chương trình đơn luồng, không đồng thời nào cũng thực thi các bước của nó theo cách này: lần lượt nối tiếp nhau. Mọi thao tác ghi lan truyền từ cùng một tiến trình đều xuất hiện theo thứ tự mà tiến trình này đã đệ trình chúng. Các thao tác lan truyền từ các nguồn khác nhau có thể được sắp thứ tự tùy tiện, nhưng thứ tự này sẽ nhất quán dưới góc nhìn của các bên đọc.



Sequential consistency is often confused with linearizability since both have similar semantics. Sequential consistency, just as linearizability, requires operations to be globally ordered, but linearizability requires the local order of each process and global order to be consistent. In other words, linearizability respects a real-time operation order. Under sequential consistency, ordering holds only for the same-origin writes [VIOTTI16] . Another important distinction is composition: we can combine linearizable histories and still expect results to be linearizable, while sequentially consistent schedules are not composable [ATTIYA94] .

Nhất quán tuần tự thường bị nhầm lẫn với tính tuyến tính hóa vì cả hai có ngữ nghĩa tương tự nhau. Nhất quán tuần tự, cũng như tính tuyến tính hóa, đòi hỏi các thao tác được

sắp thứ tự toàn cục, nhưng tính tuyến tính hóa đòi hỏi thứ tự cục bộ của mỗi tiến trình và thứ tự toàn cục phải nhất quán với nhau. Nói cách khác, tính tuyến tính hóa tôn trọng thứ tự thao tác theo thời gian thực. Dưới nhất quán tuần tự, sắp thứ tự chỉ đúng đối với các lần ghi cùng nguồn [VIOTTI16]. Một phân biệt quan trọng khác là tính kết hợp: ta có thể kết hợp các lịch sử tuyến tính hóa và vẫn kỳ vọng kết quả là tuyến tính hóa, trong khi các lịch trình nhất quán tuần tự thì không kết hợp được [ATTIYA94].

Figure 11-5 shows how $\text{write}(x,1)$ and $\text{write}(x,2)$ can become visible to P and P .

Hình 11-5 cho thấy $\text{write}(x,1)$ và $\text{write}(x,2)$ có thể trở nên hiển thị cho P và P như thế nào.

Even though in wall-clock terms, 1 was written before 2 , it can get ordered after 2 . At the same time, while P already reads the value 1 , P can still read 2 . However, both

Mặc dù theo thời gian đồng hồ tường, 1 được ghi trước 2, nó có thể được sắp thứ tự sau 2. Đồng thời, trong khi P đã đọc giá trị 1, P vẫn có thể đọc 2. Tuy nhiên, cả hai

orders, $1 \rightarrow 2$ and $2 \rightarrow 1$, are valid, as long as they're consistent for different readers. What's important here is that both P and P have observed values in the same order :

thứ tự, $1 \rightarrow 2$ và $2 \rightarrow 1$, đều hợp lệ, miễn là chúng nhất quán đối với các bên đọc khác nhau. Điều quan trọng ở đây là cả P và P đều đã quan sát các giá trị theo cùng một thứ tự:

first 2 , and then 1 [TANENBAUM14] .

đầu tiên là 2, rồi đến 1 [TANENBAUM14].

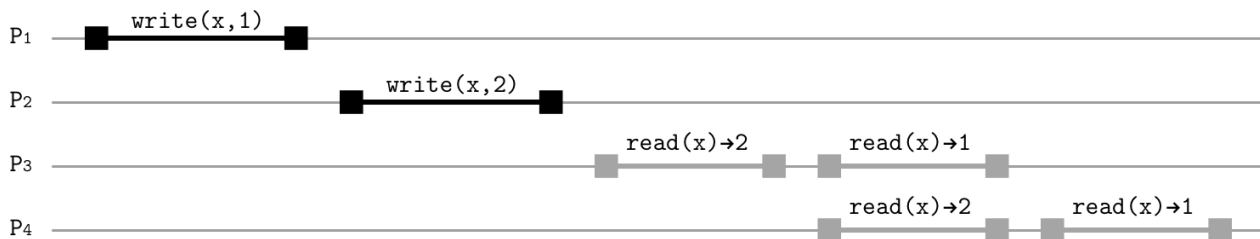


Figure 11-5. Ordering in sequential consistency

Hình 11-5. Sắp thứ tự trong nhất quán tuần tự

Stale reads can be explained, for example, by replica divergence: even though writes propagate to different replicas in the same order, they can arrive there at different times.

Các lần đọc cũ có thể được giải thích, chẳng hạn, bằng sự phân kỳ của replica: mặc dù các lần ghi lan truyền tới các replica khác nhau theo cùng một thứ tự, chúng có thể đến đó vào những thời điểm khác nhau.

The main difference with linearizability is the absence of globally enforced time bounds. Under linearizability, an operation has to become effective within its wall-clock time bounds. By the time the write W_1 operation completes, its results have to be applied, and every reader should be able to see the value at least as recent as one written by W_1 . Similarly, after a read operation R_1 returns, any read operation that happens after it should return the value that R_1 has seen or a later value (which, of course, has to follow the same rule).

Khác biệt chính so với tính tuyến tính hóa là sự thiếu vắng các giới hạn thời gian được áp đặt toàn cục. Dưới tính tuyến tính hóa, một thao tác phải có hiệu lực trong các giới hạn thời gian đồng hồ tường của nó. Vào lúc thao tác ghi W_1 hoàn tất, kết quả của nó phải được áp dụng, và mọi bên đọc phải có thể thấy giá trị ít nhất cũng mới như giá trị do W_1 ghi. Tương tự, sau khi một thao tác đọc R_1 trả về, bất kỳ thao tác đọc nào xảy ra sau nó cũng

nên trả về giá trị mà R_1 đã thấy hoặc một giá trị mới hơn (vốn, dĩ nhiên, phải tuân theo cùng quy tắc).

Sequential consistency relaxes this requirement: an operation's results can become visible after its completion, as long as the order is consistent from the individual processors' perspective. Same-origin writes can't "jump" over each other: their program order, relative to their own executing process, has to be preserved. The other restriction is that the order in which operations have appeared must be consistent for all readers.

Nhất quán tuần tự nói lỏng yêu cầu này: kết quả của một thao tác có thể trở nên hiển thị sau khi nó hoàn tất, miễn là thứ tự nhất quán dưới góc nhìn của từng vi xử lý riêng lẻ. Các lần ghi cùng nguồn không thể "nhảy" qua nhau: thứ tự chương trình của chúng, tương đối so với tiến trình đang thực thi của chính chúng, phải được bảo toàn. Hạn chế còn lại là thứ tự mà các thao tác đã xuất hiện phải nhất quán đối với tất cả các bên đọc.

Similar to linearizability, modern CPUs do not guarantee sequential consistency by default and, since the processor can reorder instructions, we should use memory barriers (also called fences) to make sure that writes become visible to concurrently running threads in order [DREPPER07] [GEORGOPOULOS16].

Tương tự tính tuyến tính hóa, các CPU hiện đại không bảo đảm nhất quán tuần tự theo mặc định và, vì vi xử lý có thể sắp lại các chỉ thị, ta nên dùng các rào cản bộ nhớ (memory barrier, còn gọi là fence) để bảo đảm rằng các lần ghi trở nên hiển thị cho các luồng chạy đồng thời theo đúng thứ tự [DREPPER07] [GEORGOPOULOS16].

Causal Consistency — Nhất quán nhân quả

You see, there is only one constant, one universal, it is the only real truth: causality. Action. Reaction. Cause and effect. —Merovingian from The Matrix Reloaded

Bạn thấy đấy, chỉ có một hằng số, một thứ phổ quát, đó là chân lý thực sự duy nhất: nhân quả. Hành động. Phản ứng. Nguyên nhân và kết quả. —Merovingian trong phim The Matrix Reloaded

Even though having a global operation order is often unnecessary, it might be necessary to establish order between some operations. Under the **causal consistency** model, all processes have to see causally related operations in the same order. Concurrent writes with no causal relationship can be observed in a different order by different processors.

Mặc dù việc có một thứ tự thao tác toàn cục thường là không cần thiết, có thể cần thiết phải thiết lập thứ tự giữa một số thao tác. Dưới mô hình nhất quán nhân quả (causal consistency), tất cả các tiến trình phải thấy các thao tác có quan hệ nhân quả theo cùng một thứ tự. Các lần ghi đồng thời không có quan hệ nhân quả có thể được quan sát theo thứ tự khác nhau bởi các vi xử lý khác nhau.

First, let's take a look at why we need causality and how writes that have no causal relationship can propagate. In Figure 11-6, processes P and P make writes that aren't

Trước hết, hãy xem tại sao ta cần nhân quả và cách các lần ghi không có quan hệ nhân quả có thể lan truyền. Trong Hình 11-6, các tiến trình P và P thực hiện các lần ghi không

causally ordered. The results of these operations can propagate to readers at different times and out of order. Process P will see the value 1 before it sees 2, while P will first

được sắp thứ tự theo nhân quả. Kết quả của các thao tác này có thể lan truyền tới các bên đọc vào những thời điểm khác nhau và không theo thứ tự. Tiến trình P sẽ thấy giá trị 1 trước khi nó thấy 2, trong khi P sẽ đầu tiên

see 2 , and then 1 .

thấy 2, rồi đến 1.

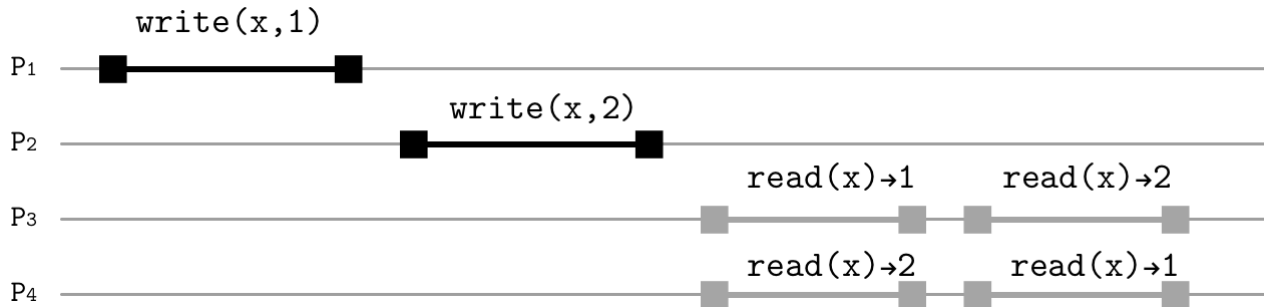


Figure 11-6. Write operations with no causal relationship

Hình 11-6. Các thao tác ghi không có quan hệ nhân quả

Figure 11-7 shows an example of causally related writes. In addition to a written value, we now have to specify a **logical clock** value that would establish a causal order between operations. P starts with a write operation `write(x,  $\emptyset$ , 1) → t` , which starts

Hình 11-7 cho thấy một ví dụ về các lần ghi có quan hệ nhân quả. Bên cạnh một giá trị được ghi, giờ ta phải chỉ định một giá trị đồng hồ logic vốn sẽ thiết lập một thứ tự nhân quả giữa các thao tác. P bắt đầu với một thao tác ghi `write(x,  $\emptyset$ , 1) → t`, vốn khởi đầu

from the initial value $\emptyset$. P performs another write operation, `write(x, t, 2)` , and

từ giá trị ban đầu $\emptyset$. P thực hiện một thao tác ghi khác, `write(x, t, 2)`, và

specifies that it is logically ordered after t , requiring operations to propagate only in

chỉ định rằng nó được sắp thứ tự logic sau t, đòi hỏi các thao tác chỉ lan truyền theo

the order established by the logical clock.

thứ tự được thiết lập bởi đồng hồ logic.

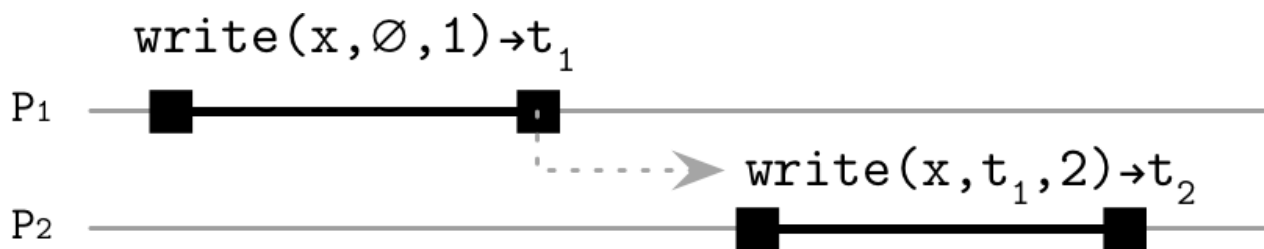


Figure 11-7. Causally related write operations

Hình 11-7. Các thao tác ghi có quan hệ nhân quả

This establishes a causal order between these operations. Even if the latter write propagates faster than the former one, it isn't made visible until all of its dependencies arrive, and the event order is reconstructed from their logical timestamps. In other words, a happened-before relationship is established logically, without using physical clocks, and all processes agree on this order.

Điều này thiết lập một thứ tự nhân quả giữa các thao tác này. Ngay cả khi lần ghi sau lan truyền nhanh hơn lần ghi trước, nó cũng không được làm cho hiển thị cho đến khi tất cả các phụ thuộc của nó đến, và thứ tự sự kiện được dừng lại từ các dấu thời gian logic của chúng. Nói cách khác, một quan hệ xảy-ra-trước (happens-before) được thiết lập một cách logic, không dùng đồng hồ vật lý, và tất cả các tiến trình đồng thuận về thứ tự này.

Figure 11-8 shows processes P and P making causally related writes, which propa-

Hình 11-8 cho thấy các tiến trình P và P thực hiện các lần ghi có quan hệ nhân quả, vốn lan

gate to P and P in their logical order. This prevents us from the situation shown in

truyền tới P và P theo thứ tự logic của chúng. Điều này ngăn ta khỏi tình huống được trình bày trong

Figure 11-6 ; you can compare histories of P and P in both figures.

Hình 11-6; bạn có thể so sánh các lịch sử của P và P trong cả hai hình.

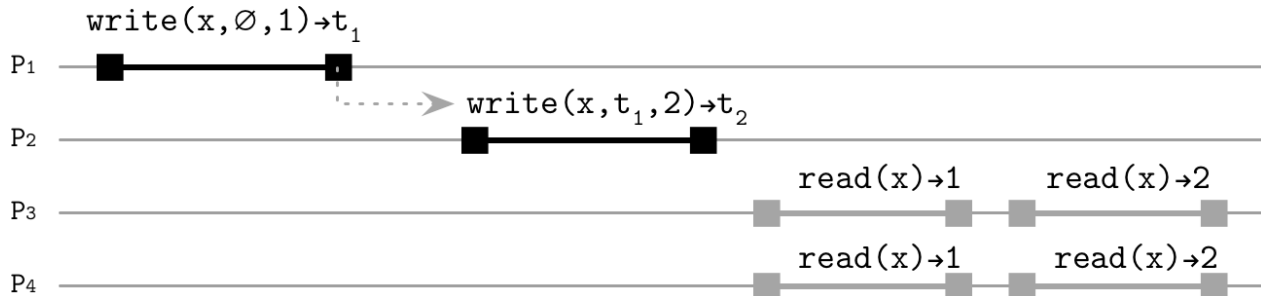


Figure 11-8. Write operations with causal relationship

Hình 11-8. Các thao tác ghi có quan hệ nhân quả

You can think of this in terms of communication on some online forum: you post something online, someone sees your post and responds to it, and a third person sees this response and continues the conversation thread. It is possible for conversation threads to diverge: you can choose to respond to one of the conversations in the thread and continue the chain of events, but some threads will have only a few messages in common, so there might be no single history for all the messages.

Bạn có thể nghĩ về điều này theo kiểu giao tiếp trên một diễn đàn trực tuyến: bạn đăng thứ gì đó lên mạng, ai đó thấy bài của bạn và phản hồi nó, rồi một người thứ ba thấy phản hồi này và tiếp tục chuỗi hội thoại (conversation thread). Các chuỗi hội thoại có thể phân kỳ: bạn có thể chọn phản hồi một trong các cuộc hội thoại trong chuỗi và tiếp tục chuỗi sự kiện, nhưng một số chuỗi sẽ chỉ có chung vài thông điệp, nên có thể không có một lịch sử duy nhất cho tất cả các thông điệp.

In a causally consistent system, we get **session guarantees** for the application, ensuring the view of the database is consistent with its own actions, even if it executes read and write requests against different, potentially inconsistent, servers [TERRY94] . These guarantees are: monotonic reads, monotonic writes, read-your-writes, writes-follow-reads. You can find more information on these session models in “Session Models” on page 233 .

Trong một hệ thống nhất quán nhân quả, ta nhận được các bảo đảm phiên cho ứng dụng, bảo đảm rằng góc nhìn về cơ sở dữ liệu nhất quán với chính các hành động của nó, ngay cả khi nó thực thi các yêu cầu đọc và ghi đối với các server khác nhau, có thể không nhất quán [TERRY94]. Các bảo đảm này là: đọc đơn điệu (monotonic reads), ghi đơn điệu (monotonic

writes), đọc-thấy-ghi-của-mình (read-your-writes), ghi-theo-sau-đọc (writes-follow-reads).
Bạn có thể tìm thêm thông tin về các mô hình phiên này ở “Session Models” trang 233.

Causal consistency can be implemented using logical clocks [LAMPORT78] and sending context metadata with every message, summarizing which operations logically precede the current one. When the update is received from the server, it contains the latest version of the context. Any operation can be processed only if all operations preceding it have already been applied. Messages for which contexts do not match are buffered on the server as it is too early to deliver them.

Nhất quán nhân quả có thể được hiện thực bằng đồng hồ logic [LAMPORT78] và bằng cách gửi metadata ngữ cảnh cùng với mỗi thông điệp, tóm tắt những thao tác nào về mặt logic đứng trước thao tác hiện tại. Khi cập nhật được nhận từ server, nó chứa phiên bản mới nhất của ngữ cảnh. Bất kỳ thao tác nào chỉ có thể được xử lý nếu tất cả các thao tác đứng trước nó đã được áp dụng rồi. Các thông điệp mà ngữ cảnh không khớp được đệm lại trên server vì còn quá sớm để giao chúng.

The two prominent and frequently cited projects implementing causal consistency are Clusters of Order-Preserving Servers (COPS) [LLOYD11] and Eiger [LLOYD13]. Both projects implement causality through a library (implemented as a frontend server that users connect to) and **track** dependencies to ensure consistency. COPS tracks dependencies through key versions, while Eiger establishes operation order instead (operations in Eiger can depend on operations executed on the other nodes; for example, in the case of multipartition transactions). Both projects do not expose out-of-order operations like eventually consistent stores might do. Instead, they detect and handle conflicts: in COPS, this is done by checking the key order and using application-specific functions, while Eiger implements the last-write-wins rule.

Hai dự án nổi bật và thường được trích dẫn hiện thực nhất quán nhân quả là Clusters of Order-Preserving Servers (COPS) [LLOYD11] và Eiger [LLOYD13]. Cả hai dự án đều hiện thực nhân quả thông qua một thư viện (được hiện thực dưới dạng một server frontend mà người dùng kết nối tới) và theo dõi các phụ thuộc để bảo đảm nhất quán. COPS theo dõi các phụ thuộc qua các phiên bản khóa, trong khi Eiger thiết lập thứ tự thao tác thay vào đó (các thao tác trong Eiger có thể phụ thuộc vào các thao tác được thực thi trên các nút khác; ví dụ, trong trường hợp các giao dịch đa phân vùng). Cả hai dự án đều không phơi bày các thao tác không-theo-thứ-tự như các kho lưu nhất quán cuối cùng có thể làm. Thay vào đó, chúng phát hiện và xử lý các xung đột: trong COPS, việc này được thực hiện bằng cách kiểm tra thứ tự khóa và dùng các hàm tùy theo ứng dụng, trong khi Eiger hiện thực quy tắc ghi-sau-cùng-thắng.

Vector clocks

Đồng hồ vector

Establishing causal order allows the system to reconstruct the sequence of events even if messages are delivered out of order, fill the gaps between the messages, and avoid publishing operation results in case some messages are still missing. For example, if messages $\{M1(\emptyset, t1), M2(M1, t2), M3(M2, t3)\}$, each specifying their dependencies, are causally related and were propagated out of order, the process buffers them until it can collect all operation dependencies and restore their causal order [KINGS- BURY18b]. Many databases, for example, Dynamo [DECANDIA07] and Riak [SHEEHY10a], use vector clocks [LAMPORT78] [MATTERN88] for establishing causal order.

Việc thiết lập thứ tự nhân quả cho phép hệ thống dựng lại trình tự các sự kiện ngay cả khi các thông điệp được giao không theo thứ tự, lấp các khoảng trống giữa các thông điệp, và tránh công bố kết quả thao tác trong trường hợp một số thông điệp vẫn còn thiếu. Ví dụ, nếu các thông điệp $\{M1(\emptyset, t1), M2(M1, t2), M3(M2, t3)\}$, mỗi thông điệp chỉ định các

phụ thuộc của nó, có quan hệ nhân quả và được lan truyền không theo thứ tự, thì tiến trình đệm chúng cho đến khi nó có thể thu thập tất cả các phụ thuộc của thao tác và khôi phục thứ tự nhân quả của chúng [KINGSBURY18b]. Nhiều cơ sở dữ liệu, ví dụ, Dynamo [DECANDIA07] và Riak [SHEEHY10a], dùng đồng hồ vector (vector clock) [LAMPORT78] [MATTERN88] để thiết lập thứ tự nhân quả.

A **vector clock** is a structure for establishing a partial order between the events, detecting and resolving divergence between the event chains. With vector clocks, we can simulate common time, global state, and represent asynchronous events as synchronous ones. Processes maintain vectors of logical clocks, with one clock per process. Every clock starts at the initial value and is incremented every time a new event arrives (for example, a write occurs). When receiving clock vectors from other processes, a process updates its local vector to the highest clock values per process from the received vectors (i.e., highest clock values the transmitting node has ever seen).

Một đồng hồ vector là một cấu trúc để thiết lập một thứ tự bộ phận (partial order) giữa các sự kiện, phát hiện và giải quyết sự phân kỳ giữa các chuỗi sự kiện. Với đồng hồ vector, ta có thể mô phỏng thời gian chung, trạng thái toàn cục, và biểu diễn các sự kiện bất đồng bộ như các sự kiện đồng bộ. Các tiến trình duy trì các vector của các đồng hồ logic, với một đồng hồ cho mỗi tiến trình. Mỗi đồng hồ bắt đầu ở giá trị ban đầu và được tăng lên mỗi khi một sự kiện mới đến (ví dụ, một lần ghi xảy ra). Khi nhận các vector đồng hồ từ các tiến trình khác, một tiến trình cập nhật vector cục bộ của nó tới các giá trị đồng hồ cao nhất theo từng tiến trình từ các vector đã nhận (tức là các giá trị đồng hồ cao nhất mà nút truyền từng thấy).

To use vector clocks for conflict resolution, whenever we make a write to the database, we first check if the value for the written key already exists locally. If the previous value already exists, we append a new version to the version vector and establish the causal relationship between the two writes. Otherwise, we start a new chain of events and initialize the value with a single version.

Để dùng đồng hồ vector cho giải quyết xung đột, mỗi khi ta ghi vào cơ sở dữ liệu, ta trước tiên kiểm tra xem giá trị cho khóa được ghi đã tồn tại cục bộ chưa. Nếu giá trị trước đã tồn tại, ta nối thêm một phiên bản mới vào vector phiên bản và thiết lập quan hệ nhân quả giữa hai lần ghi. Ngược lại, ta bắt đầu một chuỗi sự kiện mới và khởi tạo giá trị với một phiên bản đơn lẻ.

We were talking about consistency in terms of access to shared memory registers and wall-clock operation ordering, and first mentioned potential replica divergence when talking about sequential consistency. Since only write operations to the same memory location have to be ordered, we cannot end up in a situation where we have a write conflict if values are independent [LAMPORT79].

Ta đã bàn về nhất quán theo thuật ngữ truy cập tới các thanh ghi bộ nhớ chia sẻ và sắp thứ tự thao tác theo đồng hồ tường, và lần đầu đề cập đến sự phân kỳ tiềm tàng của replica khi bàn về nhất quán tuần tự. Vì chỉ các thao tác ghi tới cùng một vị trí bộ nhớ phải được sắp thứ tự, ta không thể rơi vào tình huống có một xung đột ghi nếu các giá trị độc lập [LAMPORT79].

Since we're looking for a consistency model that would improve availability and performance, we have to allow replicas to diverge not only by serving stale reads but also by accepting potentially conflicting writes, so the system is allowed to create two independent chains of events. Figure 11-9 shows such a divergence: from the perspective of one replica, we see history as 1, 5, 7, 8 and the other one reports 1, 5, 3. Riak allows users to see and resolve divergent histories [DAILY13].

Vì ta đang tìm một mô hình nhất quán cải thiện tính sẵn sàng và hiệu năng, ta phải cho phép các replica phân kỳ không chỉ bằng cách phục vụ các lần đọc cũ mà còn bằng cách

chấp nhận các lần ghi có thể xung đột, nên hệ thống được phép tạo ra hai chuỗi sự kiện độc lập. Hình 11-9 cho thấy một sự phân kỳ như vậy: từ góc nhìn của một replica, ta thấy lịch sử là 1, 5, 7, 8 còn replica kia báo cáo 1, 5, 3. Riak cho phép người dùng thấy và giải quyết các lịch sử phân kỳ [DAILY13].

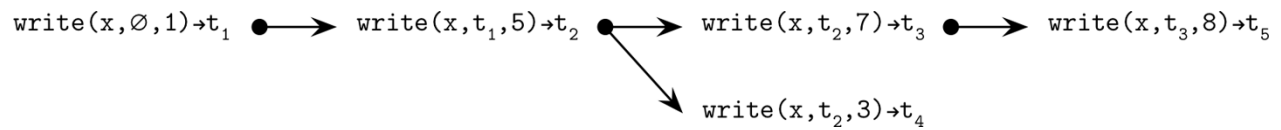
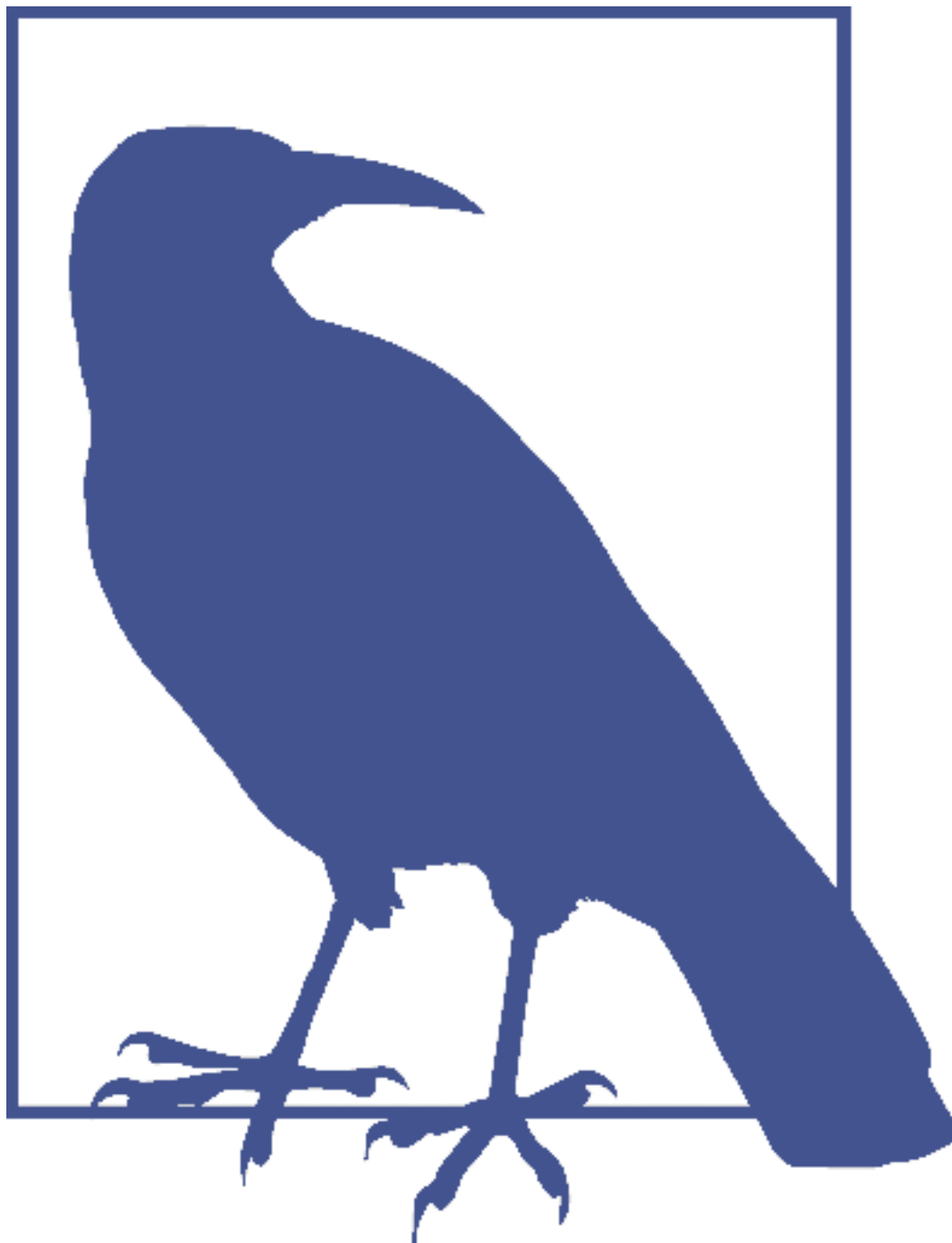


Figure 11-9. Divergent histories under causal consistency

Hình 11-9. Các lịch sử phân kỳ dưới nhất quán nhân quả



To implement causal consistency, we have to store causal history, add **garbage collection**, and ask the user to reconcile divergent histories in case of a conflict. Vector clocks can tell you that the conflict has occurred, but do not propose exactly how to resolve it, since resolution semantics are often application-specific. Because of that, some eventually consistent databases, for example, Apache Cassandra, do not order operations causally and use the last-write-wins rule for conflict resolution instead [ELLIS13] .

Để hiện thực nhất quán nhân quả, ta phải lưu lịch sử nhân quả, thêm thu gom rác, và yêu cầu người dùng đối chiếu các lịch sử phân kỳ trong trường hợp có xung đột. Đồng hồ vector có thể cho bạn biết rằng xung đột đã xảy ra, nhưng không đề xuất chính xác

cách giải quyết nó, vì ngữ nghĩa giải quyết thường tùy theo ứng dụng. Vì vậy, một số cơ sở dữ liệu nhất quán cuối cùng, ví dụ, Apache Cassandra, không sắp thứ tự các thao tác theo nhân quả mà dùng quy tắc ghi-sau-cùng-thắng (last-write-wins) để giải quyết xung đột thay vào đó [ELLIS13].

Session Models — Các mô hình phiên

Thinking about consistency in terms of value propagation is useful for database developers, since it helps to understand and impose required data invariants, but some things are easier understood and explained from the client point of view. We can look at our distributed system from the perspective of a single client instead of multiple clients.

Nghĩ về nhất quán theo thuật ngữ lan truyền giá trị thì hữu ích cho các nhà phát triển cơ sở dữ liệu, vì nó giúp hiểu và áp đặt các bất biến dữ liệu cần thiết, nhưng một số thứ dễ hiểu và dễ giải thích hơn từ góc nhìn của client. Ta có thể nhìn hệ phân tán của mình từ góc nhìn của một client đơn lẻ thay vì nhiều client.

Session models [VIOTTI16] (also called client-centric consistency models [TANEN-BAUM06]) help to reason about the state of the distributed system from the client perspective: how each client observes the state of the system while issuing read and write operations.

Các mô hình phiên (session model) [VIOTTI16] (còn gọi là các mô hình nhất quán lấy client làm trung tâm [TANENBAUM06]) giúp lập luận về trạng thái của hệ phân tán từ góc nhìn của client: mỗi client quan sát trạng thái của hệ thống như thế nào khi phát ra các thao tác đọc và ghi.

If other consistency models we discussed so far focus on explaining operation ordering in the presence of concurrent clients, client-centric consistency focuses on how a single client interacts with the system. We still assume that each client's operations are sequential: it has to finish one operation before it can start executing the next one. If the client crashes or loses connection to the server before its operation completes, we do not make any assumptions about the state of incomplete operations.

Nếu các mô hình nhất quán khác mà ta đã bàn cho đến nay tập trung vào việc giải thích sắp thứ tự thao tác khi có các client đồng thời, thì nhất quán lấy client làm trung tâm tập trung vào cách một client đơn lẻ tương tác với hệ thống. Ta vẫn giả định rằng các thao tác của mỗi client là tuần tự: nó phải hoàn tất một thao tác trước khi có thể bắt đầu thực thi thao tác kế tiếp. Nếu client sập hoặc mất kết nối tới server trước khi thao tác của nó hoàn tất, ta không đưa ra giả định nào về trạng thái của các thao tác chưa hoàn chỉnh.

In a distributed system, clients often can connect to any available replica and, if the results of the recent write against one replica did not propagate to the other one, the client might not be able to observe the state change it has made.

Trong một hệ phân tán, các client thường có thể kết nối tới bất kỳ replica sẵn sàng nào và, nếu kết quả của lần ghi gần đây đối với một replica chưa lan truyền tới replica kia, client có thể không thể quan sát được thay đổi trạng thái mà nó đã thực hiện.

One of the reasonable expectations is that every write issued by the client is visible to it. This assumption holds under the read-own-writes consistency model, which states that every read operation following the write on the same or the other replica has to observe the updated value. For example, `read(x)` that was executed immediately after `write(x,V)` will return the value `V`.

Một trong những kỳ vọng hợp lý là mọi lần ghi do client phát ra đều hiển thị với chính nó. Giả định này đúng dưới mô hình nhất quán đọc-thấy-ghi-của-mình (read-own-writes), vốn phát biểu rằng mọi thao tác đọc theo sau lần ghi trên cùng một hoặc một replica khác đều phải quan sát được giá trị đã cập nhật. Ví dụ, $\text{read}(x)$ được thực thi ngay sau $\text{write}(x,V)$ sẽ trả về giá trị V .

The monotonic reads model restricts the value visibility and states that if the $\text{read}(x)$ has observed the value V , the following reads have to observe a value at least as recent as V or some later value.

Mô hình đọc đơn điệu (monotonic reads) hạn chế tính hiển thị giá trị và phát biểu rằng nếu $\text{read}(x)$ đã quan sát giá trị V , thì các lần đọc theo sau phải quan sát một giá trị ít nhất cũng mới như V hoặc một giá trị mới hơn nào đó.

The monotonic writes model assumes that values originating from the same client appear in the order this client has executed them. If, according to the client session order, $\text{write}(x,V2)$ was made after $\text{write}(x,V1)$, their effects have to become visible in the same order (i.e., $V1$ first, and then $V2$) to all other processes. Without this assumption, old data can be “resurrected,” resulting in data loss.

Mô hình ghi đơn điệu (monotonic writes) giả định rằng các giá trị bắt nguồn từ cùng một client xuất hiện theo thứ tự mà client này đã thực thi chúng. Nếu, theo thứ tự phiên của client, $\text{write}(x,V2)$ được thực hiện sau $\text{write}(x,V1)$, thì hiệu ứng của chúng phải trở nên hiển thị theo cùng thứ tự (tức là, $V1$ trước, rồi đến $V2$) cho tất cả các tiến trình khác. Không có giả định này, dữ liệu cũ có thể bị “hồi sinh,” dẫn đến mất dữ liệu.

Writes-follow-reads (sometimes referred as session causality) ensures that writes are ordered after writes that were observed by previous read operations. For example, if $\text{write}(x,V2)$ is ordered after $\text{read}(x)$ that has returned $V1$, $\text{write}(x,V2)$ will be ordered after $\text{write}(x,V1)$.

Ghi-theo-sau-đọc (writes-follow-reads, đôi khi được gọi là nhân quả phiên) bảo đảm rằng các lần ghi được sắp thứ tự sau các lần ghi đã được quan sát bởi các thao tác đọc trước đó. Ví dụ, nếu $\text{write}(x,V2)$ được sắp thứ tự sau $\text{read}(x)$ vốn đã trả về $V1$, thì $\text{write}(x,V2)$ sẽ được sắp thứ tự sau $\text{write}(x,V1)$.



Session models make no assumptions about operations made by different processes (clients) or from the different logical session [TANENBAUM14] . These models describe operation ordering from the point of view of a single process. However, the same guarantees have to hold for every process in the system. In other words, if P can read its own writes, P should be able to read its own

Các mô hình phiên không đưa ra giả định nào về các thao tác được thực hiện bởi các tiến trình (client) khác nhau hoặc từ phiên logic khác nhau [TANENBAUM14]. Các mô hình này mô tả sắp thứ tự thao tác từ góc nhìn của một tiến trình đơn lẻ. Tuy nhiên, các bảo đảm tương tự phải đúng cho mọi tiến trình trong hệ thống. Nói cách khác, nếu P có thể đọc các lần ghi của chính nó, thì P cũng nên có thể đọc các lần ghi

writes, too.

của chính nó.

Combining monotonic reads, monotonic writes, and read-own-writes gives Pipelined RAM (PRAM) consistency [LIPTON88] [BRZEZINSKI03] , also known as FIFO consistency. PRAM guarantees that

write operations originating from one process will propagate in the order they were executed by this process. Unlike under sequential consistency, writes from different processes can be observed in different order.

Kết hợp đọc đơn điệu, ghi đơn điệu, và đọc-thấy-ghi-của-mình cho ta nhất quán Pipelined RAM (PRAM) [LIPTON88] [BRZEZINSKI03], còn gọi là nhất quán FIFO. PRAM bảo đảm rằng các thao tác ghi bắt nguồn từ một tiến trình sẽ lan truyền theo thứ tự mà tiến trình này đã thực thi chúng. Khác với dưới nhất quán tuần tự, các lần ghi từ các tiến trình khác nhau có thể được quan sát theo thứ tự khác nhau.

The properties described by client-centric consistency models are desirable and, in the majority of cases, are used by distributed systems developers to validate their systems and simplify their usage.

Các thuộc tính được mô tả bởi các mô hình nhất quán lấy client làm trung tâm là đáng mong muốn và, trong phần lớn các trường hợp, được các nhà phát triển hệ phân tán sử dụng để xác thực các hệ thống của họ và đơn giản hóa việc sử dụng chúng.

Eventual Consistency — Nhất quán cuối cùng

Synchronization is expensive, both in multiprocessor programming and in distributed systems. As we discussed in “Consistency Models” on page 222, we can relax consistency guarantees and use models that allow some divergence between the nodes. For example, sequential consistency allows reads to be propagated at different speeds.

Đồng bộ hóa tốn kém, cả trong lập trình đa vi xử lý lẫn trong các hệ phân tán. Như ta đã bàn ở “Consistency Models” trang 222, ta có thể nới lỏng các bảo đảm nhất quán và dùng các mô hình cho phép một mức độ phân kỳ nào đó giữa các nút. Ví dụ, nhất quán tuần tự cho phép các lần đọc được lan truyền với tốc độ khác nhau.

Under eventual consistency, updates propagate through the system asynchronously. Formally, it states that if there are no additional updates performed against the data item, eventually all accesses return the latest written value [VOGELS09]. In case of a conflict, the notion of latest value might change, as the values from diverged replicas are reconciled using a conflict resolution strategy, such as last-write-wins or using vector clocks (see “Vector clocks” on page 232).

Dưới nhất quán cuối cùng (eventual consistency), các cập nhật lan truyền qua hệ thống một cách bất đồng bộ. Một cách hình thức, nó phát biểu rằng nếu không có cập nhật bổ sung nào được thực hiện đối với mục dữ liệu, thì rốt cuộc mọi truy cập sẽ trả về giá trị được ghi mới nhất [VOGELS09]. Trong trường hợp có xung đột, khái niệm về giá trị mới nhất có thể thay đổi, vì các giá trị từ các replica phân kỳ được đối chiếu bằng một chiến lược giải quyết xung đột, chẳng hạn ghi-sau-cùng-thắng hoặc dùng đồng hồ vector (xem “Vector clocks” trang 232).

Eventually is an interesting **term** to describe value propagation, since it specifies no hard time bound in which it has to happen. If the delivery service provides nothing more than an “eventually” guarantee, it doesn’t sound like it can be relied upon.

“Rốt cuộc” (eventually) là một thuật ngữ thú vị để mô tả sự lan truyền giá trị, vì nó không chỉ định giới hạn thời gian cứng nào mà trong đó nó phải xảy ra. Nếu dịch vụ giao hàng không cung cấp gì hơn một bảo đảm “rốt cuộc,” thì nghe có vẻ không thể dựa vào được.

However, in practice, this works well, and many databases these days are described as eventually consistent.

Tuy nhiên, trên thực tế, điều này hoạt động tốt, và nhiều cơ sở dữ liệu ngày nay được mô tả là nhất quán cuối cùng.

Tunable Consistency — Nhất quán điều chỉnh được

Eventually consistent systems are sometimes described in CAP terms: you can trade availability for consistency or vice versa (see “Infamous CAP” on page 216). From the server-side perspective, eventually consistent systems usually implement **tunable consistency**, where data is replicated, read, and written using three variables:

Các hệ thống nhất quán cuối cùng đôi khi được mô tả theo thuật ngữ CAP: bạn có thể đánh đổi tính sẵn sàng lấy nhất quán hoặc ngược lại (xem “Infamous CAP” trang 216). Từ góc nhìn phía server, các hệ thống nhất quán cuối cùng thường hiện thực nhất quán điều chỉnh được (tunable consistency), trong đó dữ liệu được nhân bản, đọc, và ghi bằng ba biến số:

Replication Factor N Number of nodes that will store a copy of data.

Hệ số nhân bản N Số nút sẽ lưu một bản sao dữ liệu.

Write Consistency W Number of nodes that have to acknowledge a write for it to succeed.

Nhất quán ghi W Số nút phải xác nhận một lần ghi để nó thành công.

Read Consistency R Number of nodes that have to respond to a read operation for it to succeed.

Nhất quán đọc R Số nút phải phản hồi một thao tác đọc để nó thành công.

Choosing consistency levels where $(R + W > N)$, the system can guarantee returning the most recent written value, because there’s always an overlap between read and write sets. For example, if $N = 3$, $W = 2$, and $R = 2$, the system can tolerate a failure of just one node. Two nodes out of three must acknowledge the write. In the ideal scenario, the system also asynchronously replicates the write to the third node. If the third node is down, **anti-entropy** mechanisms (see Chapter 12) eventually propagate it.

Bằng cách chọn các mức nhất quán mà $(R + W > N)$, hệ thống có thể bảo đảm trả về giá trị được ghi mới nhất, vì luôn có một sự chồng lấn giữa tập đọc và tập ghi. Ví dụ, nếu $N = 3$, $W = 2$, và $R = 2$, hệ thống có thể chịu đựng sự cố của chỉ một nút. Hai trong ba nút phải xác nhận lần ghi. Trong kịch bản lý tưởng, hệ thống cũng nhân bản lần ghi tới nút thứ ba một cách bất đồng bộ. Nếu nút thứ ba ngừng hoạt động, các cơ chế anti-entropy (xem Chương 12) rất cuộc sẽ lan truyền nó.

During the read, two replicas out of three have to be available to serve the request for us to respond with consistent results. Any combination of nodes will give us at least one node that will have the most up-to-date record for a given key.

Trong lúc đọc, hai trong ba replica phải sẵn sàng để phục vụ yêu cầu để ta phản hồi với kết quả nhất quán. Bất kỳ tổ hợp nút nào cũng sẽ cho ta ít nhất một nút có bản ghi cập nhật nhất cho một khóa cho trước.



When performing a write, the **coordinator** should submit it to

Khi thực hiện một lần ghi, bên điều phối nên đệ trình nó tới

nodes, but can wait for only W nodes before it proceeds (or $W - 1$ in case the coordinator is also a replica). The rest of the write operations can complete asynchronously or fail. Similarly, when

performing a read, the coordinator has to collect at least R responses. Some databases use speculative execution and submit extra read requests to reduce coordinator response latency. This means if one of the originally submitted read requests fails or arrives slowly, speculative requests can be counted toward R instead.

N nút, nhưng chỉ có thể chờ W nút trước khi nó tiến tới (hoặc $W - 1$ trong trường hợp bên điều phối cũng là một replica). Phần còn lại của các thao tác ghi có thể hoàn tất bất đồng bộ hoặc thất bại. Tương tự, khi thực hiện một lần đọc, bên điều phối phải thu thập ít nhất R phản hồi. Một số cơ sở dữ liệu dùng thực thi suy đoán và đệ trình các yêu cầu đọc bổ sung để giảm độ trễ phản hồi của bên điều phối. Điều này nghĩa là nếu một trong các yêu cầu đọc được đệ trình ban đầu thất bại hoặc đến chậm, các yêu cầu suy đoán có thể được tính vào R thay thế.

Write-heavy systems may sometimes pick $W = 1$ and $R = N$, which allows writes to be acknowledged by just one node before they succeed, but would require all the replicas (even potentially failed ones) to be available for reads. The same is true for the $W = N$, $R = 1$ combination: the latest value can be read from any node, as long as writes succeed only after being applied on all replicas.

Các hệ thống nặng về ghi đôi khi có thể chọn $W = 1$ và $R = N$, vốn cho phép các lần ghi được xác nhận bởi chỉ một nút trước khi chúng thành công, nhưng sẽ đòi hỏi tất cả các replica (kể cả những replica có thể đã bị lỗi) phải sẵn sàng cho việc đọc. Điều tương tự cũng đúng cho tổ hợp $W = N$, $R = 1$: giá trị mới nhất có thể được đọc từ bất kỳ nút nào, miễn là các lần ghi chỉ thành công sau khi được áp dụng trên tất cả các replica.

Increasing read or write consistency levels increases latencies and raises requirements for node availability during requests. Decreasing them improves system availability while sacrificing consistency.

Tăng các mức nhất quán đọc hoặc ghi làm tăng độ trễ và nâng cao yêu cầu về tính sẵn sàng của nút trong các yêu cầu. Giảm chúng cải thiện tính sẵn sàng của hệ thống trong khi hy sinh nhất quán.

Quorums — Quorum

A consistency level that consists of $\lfloor N/2 \rfloor + 1$ nodes is called a quorum, a majority of nodes. In the case of a network partition or node failures, in a system with $2f + 1$ nodes, live nodes can continue accepting writes or reads, if up to f nodes are unavailable, until the rest of the cluster is available again. In other words, such systems can tolerate at most f node failures.

Một mức nhất quán bao gồm $\lfloor N/2 \rfloor + 1$ nút được gọi là một quorum, một đa số các nút. Trong trường hợp có phân vùng mạng hoặc sự cố nút, trong một hệ thống có $2f + 1$ nút, các nút còn sống có thể tiếp tục chấp nhận ghi hoặc đọc, nếu có tới f nút không sẵn sàng, cho đến khi phần còn lại của cụm lại sẵn sàng. Nói cách khác, các hệ thống như vậy có thể chịu đựng tối đa f sự cố nút.

When executing read and write operations using quorums, a system cannot tolerate failures of the majority of nodes. For example, if there are three replicas in total, and two of them are down, read and write operations won't be able to achieve the number of nodes necessary for read and write consistency, since only one node out of three will be able to respond to the request.

Khi thực thi các thao tác đọc và ghi bằng quorum, một hệ thống không thể chịu đựng sự cố của đa số các nút. Ví dụ, nếu tổng cộng có ba replica, và hai trong số đó ngừng hoạt động, các thao tác đọc và ghi sẽ không thể đạt được số nút cần thiết cho nhất quán đọc và ghi, vì chỉ một trong ba nút sẽ có thể phản hồi yêu cầu.

Reading and writing using quorums does not guarantee monotonicity in cases of incomplete writes. If some write operation has failed after writing a value to one replica out of three, depending on the contacted replicas, a quorum read can return either the result of the incomplete operation, or the old value. Since subsequent same-value reads are not required to contact the same replicas, values they return can alternate. To achieve read monotonicity (at the cost of availability), we have to use blocking read-repair (see “**Read Repair**” on page 245).

Đọc và ghi bằng quorum không bảo đảm tính đơn điệu trong các trường hợp ghi không hoàn chỉnh. Nếu một thao tác ghi nào đó đã thất bại sau khi ghi một giá trị vào một trong ba replica, thì tùy vào các replica được liên hệ, một lần đọc theo quorum có thể trả về hoặc kết quả của thao tác không hoàn chỉnh, hoặc giá trị cũ. Vì các lần đọc cùng-giá-trị kế tiếp không bắt buộc phải liên hệ cùng các replica, các giá trị mà chúng trả về có thể thay phiên nhau. Để đạt được tính đơn điệu đọc (với cái giá là tính sẵn sàng), ta phải dùng sửa-khi-đọc kiểu chặn (blocking read-repair) (xem “Read Repair” trang 245).

Witness Replicas — Replica nhân chứng

Using quorums for read consistency helps to improve availability: even if some of the nodes are down, a database system can still accept reads and serve writes. The majority requirement guarantees that, since there’s an overlap of at least one node in any majority, any quorum read will observe the most recent completed quorum write. However, using replication and majorities increases storage costs: we have to store a copy of the data on each replica. If our replication factor is five, we have to store five copies.

Dùng quorum cho nhất quán đọc giúp cải thiện tính sẵn sàng: ngay cả khi một số nút ngừng hoạt động, một hệ cơ sở dữ liệu vẫn có thể chấp nhận đọc và phục vụ ghi. Yêu cầu đa số bảo đảm rằng, vì có sự chồng lấn ít nhất một nút trong bất kỳ đa số nào, nên bất kỳ lần đọc theo quorum nào cũng sẽ quan sát được lần ghi theo quorum hoàn tất gần nhất. Tuy nhiên, việc dùng nhân bản và đa số làm tăng chi phí lưu trữ: ta phải lưu một bản sao dữ liệu trên mỗi replica. Nếu hệ số nhân bản của ta là năm, ta phải lưu năm bản sao.

We can improve storage costs by using a concept called witness replicas . Instead of storing a copy of the record on each replica, we can split replicas into copy and witness subsets. Copy replicas still hold data records as previously. Under normal operation, witness replicas merely store the record indicating the fact that the write operation occurred. However, a situation might occur when the number of copy replicas is too low. For example, if we have three copy replicas and two witness ones, and two copy replicas go down, we end up with a quorum of one copy and two witness replicas.

Ta có thể cải thiện chi phí lưu trữ bằng cách dùng một khái niệm gọi là replica nhân chứng (witness replica). Thay vì lưu một bản sao của bản ghi trên mỗi replica, ta có thể chia các replica thành tập bản sao (copy) và tập nhân chứng (witness). Các replica bản sao vẫn giữ các bản ghi dữ liệu như trước. Trong điều kiện hoạt động bình thường, các replica nhân chứng chỉ đơn thuần lưu bản ghi cho biết sự kiện rằng thao tác ghi đã xảy ra. Tuy nhiên, có thể xảy ra một tình huống khi số replica bản sao quá thấp. Ví dụ, nếu ta có ba replica bản sao và hai replica nhân chứng, và hai replica bản sao ngừng hoạt động, ta rất cuộc có một quorum gồm một bản sao và hai replica nhân chứng.

In cases of write timeouts or copy replica failures, witness replicas can be upgraded to temporarily store the record in place of failed or timed-out copy replicas. As soon as the original copy replicas recover, upgraded replicas can revert to their previous state, or recovered replicas can become witnesses.

Trong các trường hợp ghi quá thời gian hoặc replica bản sao gặp sự cố, các replica nhân chứng có thể được nâng cấp để tạm thời lưu bản ghi thay cho các replica bản sao bị lỗi hoặc quá thời gian. Ngay khi các replica bản sao gốc phục hồi, các replica đã nâng cấp có thể trở về trạng thái trước đó của chúng, hoặc các replica đã phục hồi có thể trở thành nhân chứng.

Let's consider a replicated system with three nodes, two of which are holding copies of data and the third serves as a witness: [1c, 2c, 3w]. We attempt to make a write, but 2c is temporarily unavailable and cannot complete the operation. In this case, we temporarily store the record on the witness replica 3w. Whenever 2c comes back up, repair mechanisms can bring it back up-to-date and remove redundant copies from witnesses.

Hãy xét một hệ thống được nhân bản với ba nút, hai trong số đó đang giữ các bản sao dữ liệu và nút thứ ba đóng vai trò nhân chứng: [1c, 2c, 3w]. Ta cố thực hiện một lần ghi, nhưng 2c tạm thời không sẵn sàng và không thể hoàn tất thao tác. Trong trường hợp này, ta tạm thời lưu bản ghi trên replica nhân chứng 3w. Mỗi khi 2c trở lại hoạt động, các cơ chế sửa chữa có thể đưa nó về trạng thái cập nhật và loại bỏ các bản sao dư thừa khỏi các nhân chứng.

In a different scenario, we can attempt to perform a read, and the record is present on 1c and 3w, but not on 2c. Since any two replicas are enough to constitute a quorum, if any subset of nodes of size two is available, whether it's two copy replicas [1c, 2c], or one copy replica and one witness [1c, 3w] or [2c, 3w], we can guarantee to serve consistent results. If we read from [1c, 2c], we fetch the latest record from 1c and can replicate it to 2c, since the value is missing there. In case only [2c, 3w] are available, the latest record can be fetched from 3w. To restore the original configuration and bring 2c up-to-date, the record can be replicated to it, and removed from the witness.

Trong một kịch bản khác, ta có thể cố thực hiện một lần đọc, và bản ghi có mặt trên 1c và 3w, nhưng không có trên 2c. Vì bất kỳ hai replica nào cũng đủ để cấu thành một quorum, nếu bất kỳ tập con nút nào kích thước hai sẵn sàng, dù là hai replica bản sao [1c, 2c], hay một replica bản sao và một nhân chứng [1c, 3w] hoặc [2c, 3w], ta có thể bảo đảm phục vụ kết quả nhất quán. Nếu ta đọc từ [1c, 2c], ta lấy bản ghi mới nhất từ 1c và có thể nhân bản nó tới 2c, vì giá trị thiếu ở đó. Trong trường hợp chỉ [2c, 3w] sẵn sàng, bản ghi mới nhất có thể được lấy từ 3w. Để khôi phục cấu hình gốc và đưa 2c về trạng thái cập nhật, bản ghi có thể được nhân bản tới nó, và loại bỏ khỏi nhân chứng.

More generally, having n copy and m witness replicas has same availability guarantees as $n + m$ copies, given that we follow two rules:

Tổng quát hơn, có n replica bản sao và m replica nhân chứng có cùng các bảo đảm về tính sẵn sàng như $n + m$ bản sao, miễn là ta tuân theo hai quy tắc:

- Read and write operations are performed using majorities (i.e., with $N/2 + 1$ participants)
- At least one of the replicas in this quorum is necessarily a copy one
 - Các thao tác đọc và ghi được thực hiện bằng đa số (tức là, với $N/2 + 1$ bên tham gia)
 - Ít nhất một trong các replica trong quorum này nhất thiết phải là một bản sao

This works because data is guaranteed to be either on the copy or witness replicas. Copy replicas are brought up-to-date by the repair mechanism in case of a failure, and witness replicas store the data in the interim.

Điều này hoạt động được vì dữ liệu được bảo đảm là nằm trên các replica bản sao hoặc nhân chứng. Các replica bản sao được đưa về trạng thái cập nhật bởi cơ chế sửa chữa trong trường hợp có sự cố, và các replica nhân chứng lưu dữ liệu trong khoảng thời gian

chuyển tiếp.

Using witness replicas helps to reduce storage costs while preserving consistency invariants. There are several implementations of this approach; for example, **Spanner** [CORBETT12] and Apache Cassandra .

Dùng replica nhân chứng giúp giảm chi phí lưu trữ trong khi vẫn bảo toàn các bất biến nhất quán. Có một số hiện thực của cách tiếp cận này; ví dụ, Spanner [CORBETT12] và Apache Cassandra.

Strong Eventual Consistency and CRDTs — Nhất quán cuối cùng mạnh và CRDT

We’ve discussed several strong consistency models, such as linearizability and **serializability**, and a form of weak consistency: eventual consistency. A possible middle ground between the two, offering some benefits of both models, is strong eventual consistency . Under this model, updates are allowed to propagate to servers late or out of order, but when all updates finally propagate to target nodes, conflicts between them can be resolved and they can be merged to produce the same valid state [GOMES17] .

Ta đã bàn về một số mô hình nhất quán mạnh, chẳng hạn tính tuyến tính hóa và tính khả tuần tự, và một dạng nhất quán yếu: nhất quán cuối cùng. Một điểm trung gian khả dĩ giữa hai cái, mang lại một số lợi ích của cả hai mô hình, là nhất quán cuối cùng mạnh (strong eventual consistency). Dưới mô hình này, các cập nhật được phép lan truyền tới các server muộn hoặc không theo thứ tự, nhưng khi tất cả các cập nhật rốt cuộc lan truyền tới các nút đích, các xung đột giữa chúng có thể được giải quyết và chúng có thể được gộp lại để tạo ra cùng một trạng thái hợp lệ [GOMES17].

Under some conditions, we can relax our consistency requirements by allowing operations to preserve additional state that allows the diverged states to be reconciled (in other words, merged) after execution. One of the most prominent examples of such an approach is Conflict-Free Replicated Data Types (CRDTs, [SHAPIRO11a]) implemented, for example, in Redis [BIYIKOGLU13] .

Dưới một số điều kiện, ta có thể nới lỏng các yêu cầu nhất quán của mình bằng cách cho phép các thao tác bảo toàn trạng thái bổ sung mà cho phép các trạng thái phân kỳ được đối chiếu (nói cách khác, được gộp) sau khi thực thi. Một trong những ví dụ nổi bật nhất của cách tiếp cận như vậy là các Kiểu dữ liệu được nhân bản không-xung-đột (Conflict-Free Replicated Data Types, CRDT, [SHAPIRO11a]) được hiện thực, chẳng hạn, trong Redis [BIYIKOGLU13].

CRDTs are specialized data structures that preclude the existence of conflict and allow operations on these data types to be applied in any order without changing the result. This property can be extremely useful in a distributed system. For example, in a multinode system that uses conflict-free replicated counters, we can increment counter values on each node independently, even if they cannot communicate with one another due to a network partition. As soon as communication is restored, results from all nodes can be reconciled, and none of the operations applied during the partition will be lost.

CRDT là các cấu trúc dữ liệu chuyên biệt loại trừ sự tồn tại của xung đột và cho phép các thao tác trên các kiểu dữ liệu này được áp dụng theo bất kỳ thứ tự nào mà không thay đổi kết quả. Thuộc tính này có thể cực kỳ hữu ích trong một hệ phân tán. Ví dụ, trong một hệ thống đa nút dùng các bộ đếm được nhân bản không-xung-đột, ta có thể tăng giá trị bộ đếm trên mỗi nút một cách độc lập, ngay cả khi chúng không thể giao tiếp với nhau do

một phân vùng mạng. Ngay khi giao tiếp được khôi phục, kết quả từ tất cả các nút có thể được đối chiếu, và không thao tác nào được áp dụng trong lúc phân vùng sẽ bị mất.

This makes CRDTs useful in eventually consistent systems, since replica states in such systems are allowed to temporarily diverge. Replicas can execute operations locally, without prior synchronization with other nodes, and operations eventually propagate to all other replicas, potentially out of order. CRDTs allow us to reconstruct the complete system state from local individual states or operation sequences.

Điều này khiến CRDT hữu ích trong các hệ thống nhất quán cuối cùng, vì các trạng thái replica trong các hệ thống như vậy được phép tạm thời phân kỳ. Các replica có thể thực thi các thao tác cục bộ, không cần đồng bộ trước với các nút khác, và các thao tác rốt cuộc lan truyền tới tất cả các replica khác, có thể không theo thứ tự. CRDT cho phép ta dựng lại trạng thái hệ thống đầy đủ từ các trạng thái cục bộ riêng lẻ hoặc từ các trình tự thao tác.

The simplest example of CRDTs is operation-based Commutative Replicated Data Types (CmRDTs). For CmRDTs to work, we need the allowed operations to be:

Ví dụ đơn giản nhất về CRDT là các Kiểu dữ liệu được nhân bản giao hoán dựa trên thao tác (Commutative Replicated Data Types, CmRDT). Để CmRDT hoạt động, ta cần các thao tác được phép phải:

Side-effect free Their application does not change the system state.

Không có hiệu ứng phụ Việc áp dụng chúng không thay đổi trạng thái hệ thống.

Commutative Argument order does not matter: $x \bullet y = y \bullet x$. In other words, it doesn't matter whether x is merged with y , or y is merged with x .

Giao hoán Thứ tự đối số không quan trọng: $x \bullet y = y \bullet x$. Nói cách khác, không quan trọng việc x được gộp với y , hay y được gộp với x .

Causally ordered Their successful delivery depends on the precondition, which ensures that the system has reached the state the operation can be applied to.

Được sắp thứ tự theo nhân quả Việc giao chúng thành công phụ thuộc vào điều kiện tiên quyết, vốn bảo đảm rằng hệ thống đã đạt được trạng thái mà thao tác có thể được áp dụng vào.

For example, we could implement a grow-only counter. Each server can hold a state vector consisting of last known counter updates from all other participants, initialized with zeros. Each server is only allowed to modify its own value in the vector. When updates are propagated, the function **merge**(state1, state2) merges the states from the two servers.

Ví dụ, ta có thể hiện thực một bộ đếm chỉ-tăng (grow-only counter). Mỗi server có thể giữ một vector trạng thái bao gồm các cập nhật bộ đếm được biết gần nhất từ tất cả các bên tham gia khác, được khởi tạo bằng các số không. Mỗi server chỉ được phép sửa đổi giá trị của riêng nó trong vector. Khi các cập nhật được lan truyền, hàm **merge**(state1, state2) gộp các trạng thái từ hai server.

For example, we have three servers, with initial state vectors initialized:

Ví dụ, ta có ba server, với các vector trạng thái ban đầu được khởi tạo:

Node 1:	Node 2:	Node 3:
[0, 0, 0]	[0, 0, 0]	[0, 0, 0]

If we update counters on the first and third nodes, their states change as follows:

Nếu ta cập nhật các bộ đếm trên nút thứ nhất và thứ ba, trạng thái của chúng thay đổi như sau:

Node 1:	Node 2:	Node 3:
[1, 0, 0]	[0, 0, 0]	[0, 0, 1]

When updates propagate, we use a merge function to combine the results by picking the maximum value for each slot:

Khi các cập nhật lan truyền, ta dùng một hàm gộp để kết hợp các kết quả bằng cách chọn giá trị lớn nhất cho mỗi ô:

```
Node 1 (Node 3 state vector propagated):  
merge([1, 0, 0], [0, 0, 1]) = [1, 0, 1]  
Node 2 (Node 1 state vector propagated):  
merge([0, 0, 0], [1, 0, 0]) = [1, 0, 0]  
Node 2 (Node 3 state vector propagated):  
merge([1, 0, 0], [0, 0, 1]) = [1, 0, 1]  
Node 3 (Node 1 state vector propagated):  
merge([0, 0, 1], [1, 0, 0]) = [1, 0, 1]
```

To determine the current vector state, the sum of values in all slots is computed: $\text{sum}([1, 0, 1]) = 2$. The merge function is commutative. Since servers are only allowed to update their own values and these values are independent, no additional coordination is required.

Để xác định trạng thái vector hiện tại, tổng các giá trị trong tất cả các ô được tính: $\text{sum}([1, 0, 1]) = 2$. Hàm gộp có tính giao hoán. Vì các server chỉ được phép cập nhật các giá trị của riêng chúng và các giá trị này độc lập, không cần điều phối bổ sung.

It is possible to produce a Positive-Negative-Counter (PN-Counter) that supports both increments and decrements by using payloads consisting of two vectors: P, which nodes use for increments, and N, where they store decrements. In a larger system, to avoid propagating huge vectors, we can use super-peers. Super-peers replicate counter states and help to avoid constant peer-to-peer chatter [SHAPIRO11b].

Có thể tạo ra một Bộ đếm Dương-Âm (Positive-Negative-Counter, PN-Counter) hỗ trợ cả tăng và giảm bằng cách dùng các payload gồm hai vector: P, mà các nút dùng cho việc tăng, và N, nơi chúng lưu các việc giảm. Trong một hệ thống lớn hơn, để tránh lan truyền các vector khổng lồ, ta có thể dùng các super-peer. Các super-peer nhân bản các trạng thái bộ đếm và giúp tránh sự huyền não peer-to-peer liên tục [SHAPIRO11b].

To save and replicate values, we can use registers. The simplest version of the register is the last-write-wins register (LWW register), which stores a unique, globally ordered timestamp attached to each value to resolve conflicts. In case of a conflicting write, we preserve only the one with the larger timestamp. The merge operation (picking the value with the largest timestamp) here is also commutative, since it relies on the timestamp. If we cannot allow values to be discarded, we can supply application-specific merge logic and use a multivalue register, which stores all values that were written and allows the application to pick the right one.

Để lưu và nhân bản các giá trị, ta có thể dùng các thanh ghi. Phiên bản đơn giản nhất của thanh ghi là thanh ghi ghi-sau-cùng-thắng (LWW register), vốn lưu một dấu thời gian duy nhất, được sắp thứ tự toàn cục gắn với mỗi giá trị để giải quyết xung đột. Trong trường hợp có một lần ghi xung đột, ta chỉ giữ lại cái có dấu thời gian lớn hơn. Thao tác gộp (chọn giá trị có dấu thời gian lớn nhất) ở đây cũng giao hoán, vì nó dựa vào dấu thời gian. Nếu ta không thể cho phép các giá trị bị loại bỏ, ta có thể cung cấp logic gộp tùy theo ứng dụng

và dùng một thanh ghi đa-giá-trị, vốn lưu tất cả các giá trị đã được ghi và cho phép ứng dụng chọn giá trị đúng.

Another example of CRDTs is an unordered grow-only set (G-Set). Each node maintains its local state and can append elements to it. Adding elements produces a valid set. Merging two sets is also a commutative operation. Similar to counters, we can use two sets to support both additions and removals. In this case, we have to preserve an invariant: only the values contained in the addition set can be added into the removal set. To reconstruct the current state of the set, all elements contained in the removal set are subtracted from the addition set [SHAPIRO11b].

Một ví dụ khác về CRDT là một tập chỉ-tăng không có thứ tự (G-Set). Mỗi nút duy trì trạng thái cục bộ của nó và có thể nối thêm các phần tử vào đó. Việc thêm các phần tử tạo ra một tập hợp lệ. Gộp hai tập cũng là một thao tác giao hoán. Tương tự các bộ đếm, ta có thể dùng hai tập để hỗ trợ cả việc thêm và xóa. Trong trường hợp này, ta phải bảo toàn một bất biến: chỉ các giá trị nằm trong tập thêm mới có thể được thêm vào tập xóa. Để dựng lại trạng thái hiện tại của tập, tất cả các phần tử nằm trong tập xóa được trừ khỏi tập thêm [SHAPIRO11b].

An example of a conflict-free type that combines more complex structures is a conflict-free replicated JSON data type, allowing modifications such as insertions, deletions, and assignments on deeply nested JSON documents with list and map types. This algorithm performs merge operations on the client side and does not require operations to be propagated in any specific order [KLEPPMANN14].

Một ví dụ về kiểu không-xung-đột kết hợp các cấu trúc phức tạp hơn là một kiểu dữ liệu JSON được nhân bản không-xung-đột, cho phép các sửa đổi như chèn, xóa, và gán trên các tài liệu JSON lồng sâu với các kiểu danh sách và bản đồ. Thuật toán này thực hiện các thao tác gộp ở phía client và không đòi hỏi các thao tác được lan truyền theo bất kỳ thứ tự cụ thể nào [KLEPPMANN14].

There are quite a few possibilities CRDTs provide us with, and we can see more data stores using this concept to provide Strong Eventual Consistency (SEC). This is a powerful concept that we can add to our arsenal of tools for building fault-tolerant distributed systems.

Có khá nhiều khả năng mà CRDT mang lại cho ta, và ta có thể thấy ngày càng nhiều kho dữ liệu dùng khái niệm này để cung cấp Nhất quán cuối cùng mạnh (Strong Eventual Consistency, SEC). Đây là một khái niệm mạnh mẽ mà ta có thể thêm vào kho công cụ của mình để xây dựng các hệ phân tán dung lỗi.

Summary — Tóm tắt

Fault-tolerant systems use replication to improve availability: even if some processes fail or are unresponsive, the system as a whole can continue functioning correctly. However, keeping multiple copies in sync requires additional coordination.

Các hệ thống dung lỗi dùng nhân bản để cải thiện tính sẵn sàng: ngay cả khi một số tiến trình gặp sự cố hoặc không phản hồi, toàn bộ hệ thống vẫn có thể tiếp tục hoạt động đúng đắn. Tuy nhiên, việc giữ đồng bộ nhiều bản sao đòi hỏi điều phối bổ sung.

We've discussed several single-operation consistency models, ordered from the one with the most guarantees to the one with the least: 2

Ta đã bàn về một số mô hình nhất quán đơn thao tác, được sắp từ cái có nhiều bảo đảm nhất đến cái có ít bảo đảm nhất: 2

Linearizability Operations appear to be applied instantaneously, and the real-time operation order is maintained.

Tính tuyến tính hóa Các thao tác trông như được áp dụng tức thời, và thứ tự thao tác theo thời gian thực được duy trì.

Sequential consistency Operation effects are propagated in some total order, and this order is consistent with the order they were executed by the individual processes.

Nhất quán tuần tự Hiệu ứng thao tác được lan truyền theo một thứ tự toàn phần nào đó, và thứ tự này nhất quán với thứ tự mà chúng được thực thi bởi từng tiến trình riêng lẻ.

Causal consistency Effects of the causally related operations are visible in the same order to all processes.

Nhất quán nhân quả Hiệu ứng của các thao tác có quan hệ nhân quả hiển thị theo cùng một thứ tự cho tất cả các tiến trình.

2 These short definitions are given for recap only, the reader is advised to refer to the complete definitions for context.

2 Các định nghĩa ngắn gọn này được đưa ra chỉ để ôn lại, người đọc nên tham khảo các định nghĩa đầy đủ để có ngữ cảnh.

PRAM/FIFO consistency Operation effects become visible in the same order they were executed by individual processes. Writes from different processes can be observed in different orders.

Nhất quán PRAM/FIFO Hiệu ứng thao tác trở nên hiển thị theo cùng thứ tự mà chúng được thực thi bởi từng tiến trình riêng lẻ. Các lần ghi từ các tiến trình khác nhau có thể được quan sát theo các thứ tự khác nhau.

After that, we discussed multiple session models:

Sau đó, ta đã bàn về nhiều mô hình phiên:

Read-own-writes Read operations reflect the previous writes. Writes propagate through the system and become available for later reads that come from the same client.

Đọc-thấy-ghi-của-mình Các thao tác đọc phản ánh các lần ghi trước đó. Các lần ghi lan truyền qua hệ thống và trở nên sẵn có cho các lần đọc sau đó đến từ cùng một client.

Monotonic reads Any read that has observed a value cannot observe a value that is older than the observed one.

Đọc đơn điệu Bất kỳ lần đọc nào đã quan sát một giá trị thì không thể quan sát một giá trị cũ hơn giá trị đã được quan sát.

Monotonic writes Writes coming from the same client propagate to other clients in the order they were made by this client.

Ghi đơn điệu Các lần ghi đến từ cùng một client lan truyền tới các client khác theo thứ tự mà chúng được thực hiện bởi client này.

Writes-follow-reads Write operations are ordered after the writes whose effects were observed by the previous reads executed by the same client.

Ghi-theo-sau-đọc Các thao tác ghi được sắp thứ tự sau các lần ghi mà hiệu ứng của chúng đã được quan sát bởi các lần đọc trước đó do cùng một client thực thi.

Knowing and understanding these concepts can help you to understand the guarantees of the underlying systems and use them for application development. Consistency models describe rules

that operations on data have to follow, but their scope is limited to a specific system. Stacking systems with weaker guarantees on top of ones with stronger guarantees or ignoring consistency implications of underlying systems may lead to unrecoverable inconsistencies and data loss.

Biết và hiểu các khái niệm này có thể giúp bạn hiểu các bảo đảm của các hệ thống bên dưới và dùng chúng cho việc phát triển ứng dụng. Các mô hình nhất quán mô tả các quy tắc mà các thao tác trên dữ liệu phải tuân theo, nhưng phạm vi của chúng giới hạn ở một hệ thống cụ thể. Chồng các hệ thống có bảo đảm yếu hơn lên trên các hệ thống có bảo đảm mạnh hơn, hoặc bỏ qua các hệ quả nhất quán của các hệ thống bên dưới, có thể dẫn đến những bất nhất không thể phục hồi và mất dữ liệu.

We also discussed the concept of eventual and tunable consistency. Quorum-based systems use majorities to serve consistent data. Witness replicas can be used to reduce storage costs.

Ta cũng đã bàn về khái niệm nhất quán cuối cùng và điều chỉnh được. Các hệ thống dựa trên quorum dùng đa số để phục vụ dữ liệu nhất quán. Các replica nhân chứng có thể được dùng để giảm chi phí lưu trữ.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Consistency models Perrin, Matthieu. 2017. Distributed Systems: Concurrency and Consistency (1st Ed.). Elsevier, UK: ISTE Press.

Viotti, Paolo and Marko Vukolić. 2016. "Consistency in Non-Transactional Distributed Storage Systems." ACM Computing Surveys 49, no. 1 (July): Article 19. <https://doi.org/0.1145/2926965> .

Bailis, Peter, Aaron Davidson, Alan Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2013. "Highly available transactions: virtues and limitations." Proceedings of the VLDB Endowment 7, no. 3 (November): 181-192. <https://doi.org/10.14778/2732232.2732237> .

Aguilera, M.K., and D.B. Terry. 2016. "The Many Faces of Consistency." Bulletin of the Technical Committee on Data Engineering 39, no. 1 (March): 3-13.

CHAPTER 12 Anti-Entropy and Dissemination — Chương 12: Anti-Entropy và Lan truyền

Most of the communication patterns we've been discussing so far were either peer-to-peer or one-to-many (**coordinator** and replicas). To reliably propagate data records throughout the system, we need the propagating **node** to be available and able to reach the other nodes, but even then the throughput is limited to a single machine.

Hầu hết các mẫu giao tiếp mà ta đã bàn đến cho tới giờ đều là kiểu ngang hàng (peer-to-peer) hoặc một-tới-nhiều (bên điều phối và các bản sao). Để lan truyền các bản ghi dữ liệu xuyên suốt hệ thống một cách đáng tin cậy, ta cần nút lan truyền phải sẵn sàng và có khả năng liên hệ với các nút khác, nhưng kể cả khi đó thì thông lượng vẫn bị giới hạn ở một máy đơn lẻ.

Quick and reliable propagation may be less applicable to data records and more important for the cluster-wide metadata, such as membership information (joining and leaving nodes), node states, failures, schema changes, etc. Messages containing this information are generally infrequent and small, but have to be propagated as quickly and reliably as possible.

Việc lan truyền nhanh và đáng tin cậy có lẽ ít cần thiết với các bản ghi dữ liệu hơn, nhưng lại quan trọng hơn với siêu dữ liệu phạm vi toàn cụm, chẳng hạn như thông tin thành viên (các nút gia nhập và rời đi), trạng thái nút, các lỗi, các thay đổi lược đồ, v.v. Những thông điệp chứa thông tin loại này nhìn chung không thường xuyên và có kích thước nhỏ, nhưng phải được lan truyền nhanh và đáng tin cậy nhất có thể.

Such updates can generally be propagated to all nodes in the cluster using one of the three broad groups of approaches [DEMERS87] ; schematic depictions of these communication patterns are shown in Figure 12-1 :

Những cập nhật như vậy nhìn chung có thể được lan truyền tới mọi nút trong cụm bằng một trong ba nhóm tiếp cận lớn [DEMERS87]; mô tả sơ đồ của các mẫu giao tiếp này được trình bày trong Hình 12-1:

- a) Notification broadcast from one **process** to all others.
- b) Periodic peer-to-peer information exchange. Peers connect pairwise and exchange messages.
- c) Cooperative broadcast, where message recipients become broadcasters and help to spread the information quicker and more reliably.

- a) Quảng bá thông báo từ một tiến trình tới tất cả các tiến trình khác.
- b) Trao đổi thông tin ngang hàng định kỳ. Các nút ngang hàng kết nối theo cặp và trao đổi thông điệp.
- c) Quảng bá hợp tác, trong đó bên nhận thông điệp trở thành bên quảng bá và giúp lan truyền thông tin nhanh và đáng tin cậy hơn.

243

243

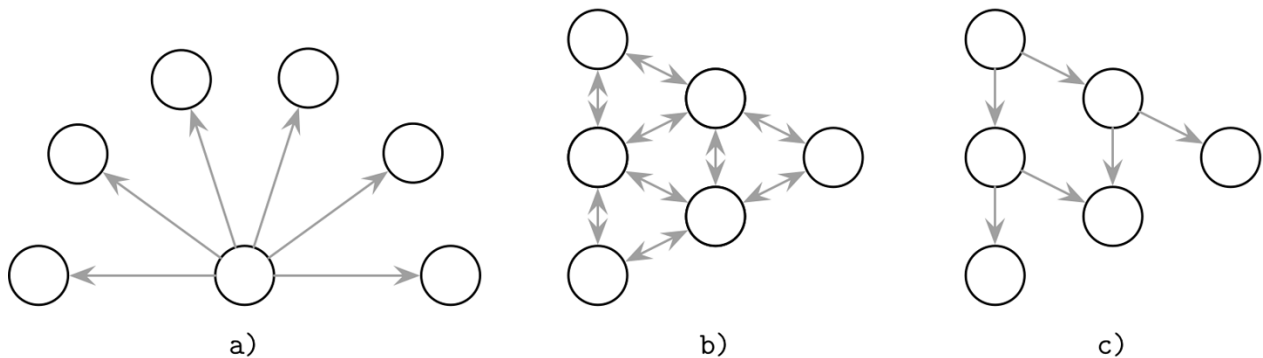


Figure 12-1. Broadcast (a), **anti-entropy** (b), and gossip (c)

Hình 12-1. Quảng bá (a), anti-entropy (b), và gossip (c)

Broadcasting the message to all other processes is the most straightforward approach that works well when the number of nodes in the cluster is small, but in large clusters it can get expensive because of the number of nodes, and unreliable because of overdependence on a single process. Individual processes may not always know about the existence of all other processes in the network. Moreover, there has to be some overlap in time during which both the broadcasting process and each one of its recipients are up, which might be difficult to achieve in some cases.

Quảng bá thông điệp tới tất cả các tiến trình khác là cách tiếp cận trực tiếp nhất, hoạt động tốt khi số nút trong cụm nhỏ, nhưng trong các cụm lớn nó có thể trở nên tốn kém vì số lượng nút, và kém tin cậy vì phụ thuộc quá mức vào một tiến trình duy nhất. Các tiến trình riêng lẻ có thể không phải lúc nào cũng biết về sự tồn tại của mọi tiến trình khác trong mạng. Hơn nữa, phải có một khoảng chồng lấn về thời gian trong đó cả tiến trình quảng bá lẫn từng bên nhận của nó đều đang hoạt động, điều này trong một số trường hợp có thể khó đạt được.

To relax these constraints, we can assume that some updates may fail to propagate. The coordinator will do its best and deliver the messages to all available participants, and then anti-entropy mechanisms will bring nodes back in sync in case there were any failures. This way, the responsibility for delivering messages is shared by all nodes in the system, and is split into two steps: primary delivery and periodic sync.

Để nới lỏng những ràng buộc này, ta có thể giả định rằng một số cập nhật có thể lan truyền thất bại. Bên điều phối sẽ cố gắng hết sức và giao thông điệp tới tất cả các bên tham gia sẵn sàng, rồi sau đó các cơ chế anti-entropy sẽ đưa các nút trở lại đồng bộ trong trường hợp có lỗi nào đó xảy ra. Theo cách này, trách nhiệm giao thông điệp được chia sẻ bởi mọi nút trong hệ thống, và được tách thành hai bước: giao chính (primary delivery) và đồng bộ định kỳ.

Entropy is a property that represents the measure of disorder in the system. In a **distributed system**, entropy represents a degree of state divergence between the nodes. Since this property is undesired and its amount should be kept to a minimum, there are many techniques that help to deal with entropy.

Entropy là một tính chất biểu diễn thước đo độ hỗn loạn trong hệ thống. Trong một hệ phân tán, entropy biểu diễn mức độ phân kỳ trạng thái giữa các nút. Vì tính chất này là không mong muốn và lượng của nó nên được giữ ở mức tối thiểu, có nhiều kỹ thuật giúp xử lý entropy.

Anti-entropy is usually used to bring the nodes back up-to-date in case the primary delivery mechanism has failed. The system can continue functioning correctly even if the coordinator fails at some point, since the other nodes will continue spreading the information. In other words, anti-entropy is used to lower the convergence time bounds in eventually consistent systems.

Anti-entropy thường được dùng để đưa các nút trở lại trạng thái cập nhật mới nhất trong trường hợp cơ chế giao chính đã thất bại. Hệ thống có thể tiếp tục hoạt động đúng đắn ngay cả khi bên điều phối gặp lỗi tại một thời điểm nào đó, vì các nút khác sẽ tiếp tục lan truyền thông tin. Nói cách khác, anti-entropy được dùng để hạ thấp giới hạn thời gian hội tụ trong các hệ nhất quán cuối cùng (eventually consistent).

To keep nodes in sync, anti-entropy triggers a background or a foreground process that compares and reconciles missing or conflicting records. Background anti-entropy processes use auxiliary structures such as Merkle trees and update logs to identify divergence. Foreground anti-entropy processes piggyback **read** or write requests: **hinted handoff**, read repairs, etc.

Để giữ các nút đồng bộ, anti-entropy kích hoạt một tiến trình nền (background) hoặc tiền cảnh (foreground) để so sánh và hòa giải các bản ghi bị thiếu hoặc xung đột. Các tiến trình anti-entropy nền dùng các cấu trúc phụ trợ như cây Merkle và nhật ký cập nhật để nhận diện phân kỳ. Các tiến trình anti-entropy tiền cảnh đi nhờ (piggyback) trên các yêu cầu đọc hoặc ghi: hinted handoff, sửa-khi-đọc (read repair), v.v.

If replicas diverge in a replicated system, to restore consistency and bring them back in sync, we have to find and repair missing records by comparing replica states pairwise. For large datasets, this can be very costly: we have to read the whole dataset on both nodes and notify replicas about more recent state changes that weren't yet propagated. To reduce this cost, we can consider ways in which replicas can get out-of-date and patterns in which data is accessed.

Nếu các bản sao phân kỳ trong một hệ có nhân bản, để khôi phục nhất quán và đưa chúng trở lại đồng bộ, ta phải tìm và sửa các bản ghi bị thiếu bằng cách so sánh trạng thái các bản sao theo cặp. Với các tập dữ liệu lớn, việc này có thể rất tốn kém: ta phải đọc toàn bộ tập dữ liệu trên cả hai nút và thông báo cho các bản sao về những thay đổi trạng thái mới hơn mà chưa được lan truyền. Để giảm chi phí này, ta có thể xem xét những cách mà các bản sao có thể trở nên lỗi thời cùng các mẫu truy cập dữ liệu.

Read Repair — Sửa-khi-đọc (Read Repair)

It is easiest to detect divergence between the replicas during the read, since at that point we can contact replicas, request the queried state from each one of them, and see whether or not their responses match. Note that in this case we do not query an entire dataset stored on each replica, and we limit our goal to just the data that was requested by the client.

Dễ nhất là phát hiện phân kỳ giữa các bản sao trong lúc đọc, vì tại thời điểm đó ta có thể liên hệ với các bản sao, yêu cầu trạng thái được truy vấn từ từng bản sao, và xem các

phản hồi của chúng có khớp hay không. Lưu ý rằng trong trường hợp này ta không truy vấn toàn bộ tập dữ liệu lưu trên mỗi bản sao, mà giới hạn mục tiêu chỉ ở dữ liệu mà máy khách yêu cầu.

The coordinator performs a distributed read, optimistically assuming that replicas are in sync and have the same information available. If replicas send different responses, the coordinator sends missing updates to the replicas where they're missing.

Bên điều phối thực hiện một lần đọc phân tán, lạc quan giả định rằng các bản sao đang đồng bộ và có cùng thông tin sẵn có. Nếu các bản sao gửi các phản hồi khác nhau, bên điều phối gửi các cập nhật bị thiếu tới những bản sao đang thiếu chúng.

This mechanism is called **read repair**. It is often used to detect and eliminate inconsistencies. During read repair, the coordinator node makes a request to replicas, waits for their responses, and compares them. In case some of the replicas have missed the recent updates and their responses differ, the coordinator detects inconsistencies and sends updates back to the replicas [DECANDIA07]

Cơ chế này gọi là sửa-khi-đọc. Nó thường được dùng để phát hiện và loại bỏ các bất nhất quán. Trong lúc sửa-khi-đọc, nút điều phối gửi yêu cầu tới các bản sao, chờ phản hồi của chúng, và so sánh chúng. Trong trường hợp một số bản sao đã bỏ lỡ các cập nhật gần đây và phản hồi của chúng khác nhau, bên điều phối phát hiện các bất nhất quán và gửi các cập nhật trở lại cho các bản sao [DECANDIA07].

Some Dynamo-style databases choose to lift the requirement of contacting all replicas and use **tunable consistency** levels instead. To return consistent results, we do not have to contact and repair all the replicas, but only the number of nodes that satisfies the consistency level. If we do **quorum** reads and writes, we still get consistent results, but some of the replicas still might not contain all the writes.

Một số cơ sở dữ liệu kiểu Dynamo chọn bỏ yêu cầu phải liên hệ với tất cả các bản sao và thay vào đó dùng các mức nhất quán điều chỉnh được. Để trả về kết quả nhất quán, ta không cần liên hệ và sửa tất cả các bản sao, mà chỉ cần số nút thỏa mãn mức nhất quán. Nếu ta thực hiện đọc và ghi theo quorum, ta vẫn nhận được kết quả nhất quán, nhưng một số bản sao vẫn có thể không chứa tất cả các lần ghi.

Read repair can be implemented as a blocking or **asynchronous** operation. During blocking read repair, the original client request has to wait until the coordinator “repairs” the replicas. Asynchronous read repair simply schedules a task that can be executed after results are returned to the user.

Sửa-khi-đọc có thể được hiện thực như một thao tác chặn (blocking) hoặc bất đồng bộ. Trong sửa-khi-đọc chặn, yêu cầu gốc của máy khách phải chờ cho tới khi bên điều phối “sửa” xong các bản sao. Sửa-khi-đọc bất đồng bộ chỉ đơn giản lập lịch một tác vụ có thể được thực thi sau khi kết quả đã được trả về cho người dùng.

Blocking read repair ensures read monotonicity (see “Session Models” on **page 233**) for quorum reads: as soon as the client reads a specific value, subsequent reads return the value at least as recent as the one it has seen, since replica states were repaired. If we're not using quorums for reads, we lose this monotonicity guarantee as data might have not been propagated to the target node by the time of a subsequent read. At the same time, blocking read repair sacrifices availability, since repairs should be acknowledged by the target replicas and the read cannot return until they respond.

Sửa-khi-đọc chặn bảo đảm tính đơn điệu đọc (read monotonicity) (xem “Session Models” ở trang 233) cho các lần đọc quorum: ngay khi máy khách đọc một giá trị cụ thể, các lần

đọc tiếp theo trả về giá trị ít nhất cũng mới bằng giá trị nó đã thấy, vì trạng thái các bản sao đã được sửa. Nếu ta không dùng quorum cho các lần đọc, ta mất bảo đảm đơn điệu này vì dữ liệu có thể chưa được lan truyền tới nút đích vào thời điểm của một lần đọc tiếp theo. Đồng thời, sửa-khi-đọc chặn hy sinh tính sẵn sàng, vì các lần sửa phải được các bản sao đích xác nhận và lần đọc không thể trả về cho tới khi chúng phản hồi.

To detect exactly which records differ between replica responses, some databases (for example, Apache Cassandra) use specialized iterators with **merge** listeners, which reconstruct differences between the merged result and individual inputs. Its output is then used by the coordinator to notify replicas about the missing data.

Để phát hiện chính xác bản ghi nào khác nhau giữa các phản hồi của bản sao, một số cơ sở dữ liệu (ví dụ Apache Cassandra) dùng các bộ lặp (iterator) chuyên dụng với các merge listener, để tái dựng các khác biệt giữa kết quả đã gộp và các đầu vào riêng lẻ. Đầu ra của nó sau đó được bên điều phối dùng để thông báo cho các bản sao về dữ liệu bị thiếu.

Read repair assumes that replicas are mostly in sync and we do not expect every request to fall back to a blocking repair. Because of the read monotonicity of blocking repairs, we can also expect subsequent requests to return the same consistent results, as long as there was no write operation that has completed in the interim.

Sửa-khi-đọc giả định rằng các bản sao phần lớn đang đồng bộ và ta không kỳ vọng mọi yêu cầu đều phải lùi về một lần sửa chặn. Nhờ tính đơn điệu đọc của các lần sửa chặn, ta cũng có thể kỳ vọng các yêu cầu tiếp theo trả về cùng kết quả nhất quán, miễn là không có thao tác ghi nào hoàn tất trong khoảng thời gian xen giữa.

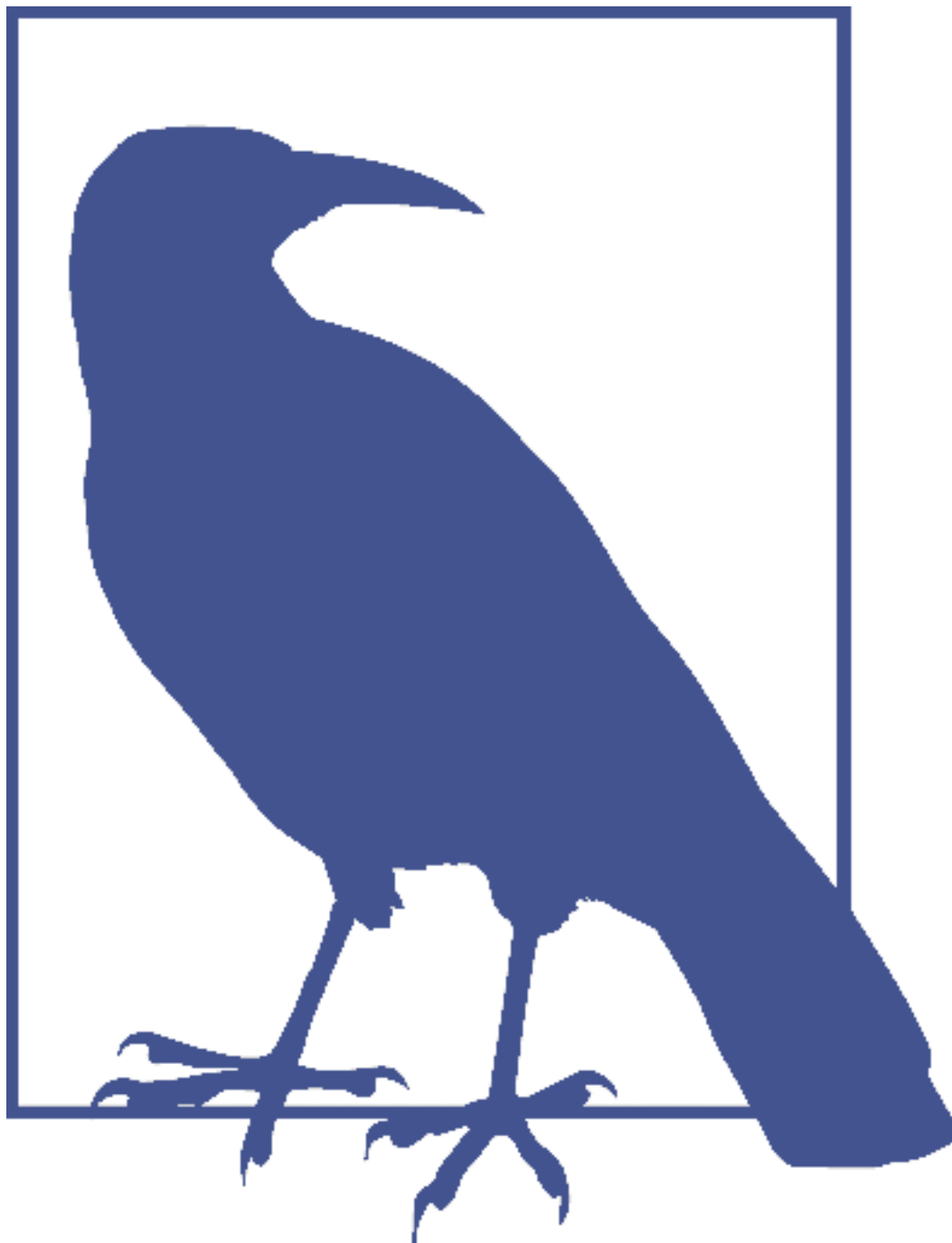
Digest Reads — Đọc Tóm tắt (Digest Reads)

Instead of issuing a full read request to each node, the coordinator can issue only one full read request and send only digest requests to the other replicas. A digest request reads the replica-local data and, instead of returning a full snapshot of the requested data, it computes a hash of this response. Now, the coordinator can compute a hash of the full read and compare it to digests from all other nodes. If all the digests match, it can be confident that the replicas are in sync.

Thay vì phát một yêu cầu đọc đầy đủ tới mỗi nút, bên điều phối có thể chỉ phát một yêu cầu đọc đầy đủ và gửi các yêu cầu tóm tắt (digest) tới các bản sao còn lại. Một yêu cầu tóm tắt đọc dữ liệu cục bộ của bản sao và, thay vì trả về ảnh chụp đầy đủ của dữ liệu được yêu cầu, nó tính một mã băm (hash) của phản hồi này. Bây giờ, bên điều phối có thể tính một mã băm của lần đọc đầy đủ và so sánh nó với các tóm tắt từ tất cả các nút khác. Nếu tất cả các tóm tắt khớp nhau, nó có thể tin chắc rằng các bản sao đang đồng bộ.

In case digests do not match, the coordinator does not know which replicas are ahead, and which ones are behind. To bring lagging replicas back in sync with the rest of the nodes, the coordinator has to issue full reads to any replicas that responded with different digests, compare their responses, reconcile the data, and send updates to the lagging replicas.

Trong trường hợp các tóm tắt không khớp, bên điều phối không biết bản sao nào đang đi trước và bản sao nào đang đi sau. Để đưa các bản sao tụt hậu trở lại đồng bộ với các nút còn lại, bên điều phối phải phát các lần đọc đầy đủ tới bất kỳ bản sao nào đã phản hồi bằng các tóm tắt khác nhau, so sánh phản hồi của chúng, hòa giải dữ liệu, và gửi các cập nhật tới các bản sao tụt hậu.



Digests are usually computed using a noncryptographic hash function, such as MD5, since it has to be computed quickly to make the “happy path” performant. Hash functions can have collisions, but their probability is negligible for most real-world systems. Since databases often use more than just one anti-entropy mechanism, we can expect that, even in the unlikely event of a hash collision, data will be reconciled by the different subsystem.

Các tóm tắt thường được tính bằng một hàm băm phi mật mã (noncryptographic), chẳng hạn MD5, vì nó phải được tính nhanh để làm cho “đường đi thuận lợi” (happy path) hiệu năng cao. Các hàm băm có thể có va chạm (collision), nhưng xác suất của chúng là không đáng kể với hầu hết các hệ thống thực tế. Vì các cơ sở dữ liệu thường dùng nhiều hơn một

cơ chế anti-entropy, ta có thể kỳ vọng rằng, ngay cả trong sự kiện va chạm băm khó xảy ra, dữ liệu sẽ được hòa giải bởi phân hệ khác.

Hinted Handoff — Hinted Handoff

Another anti-entropy approach is called hinted handoff [DECANDIA07] , a write-side repair mechanism. If the target node fails to acknowledge the write, the write coordinator or one of the replicas stores a special record, called a hint , which is replayed to the target node as soon as it comes back up.

Một cách tiếp cận anti-entropy khác gọi là hinted handoff [DECANDIA07], một cơ chế sửa ở phía ghi. Nếu nút đích không xác nhận lần ghi, bên điều phối ghi hoặc một trong các bản sao lưu một bản ghi đặc biệt, gọi là gợi ý (hint), được phát lại tới nút đích ngay khi nó hoạt động trở lại.

In Apache Cassandra, unless the ANY consistency level is in use [ELLIS11] , hinted writes aren't counted toward the replication factor (see “Tunable Consistency” on page 235), since the data in the hint log isn't accessible for reads and is only used to help the lagging participants catch up.

Trong Apache Cassandra, trừ khi mức nhất quán ANY được dùng [ELLIS11], các lần ghi có gợi ý không được tính vào hệ số nhân bản (replication factor) (xem “Tunable Consistency” ở trang 235), vì dữ liệu trong nhật ký gợi ý không truy cập được để đọc và chỉ được dùng để giúp các bên tham gia tụt hậu bắt kịp.

Some databases, for example Riak, use sloppy quorums together with hinted handoff. With sloppy quorums, in case of replica failures, write operations can use additional healthy nodes from the node list, and these nodes do not have to be target replicas for the executed operations.

Một số cơ sở dữ liệu, ví dụ Riak, dùng quorum lỏng (sloppy quorum) cùng với hinted handoff. Với quorum lỏng, trong trường hợp có lỗi bản sao, các thao tác ghi có thể dùng thêm các nút khỏe mạnh khác từ danh sách nút, và những nút này không nhất thiết phải là bản sao đích cho các thao tác được thực thi.

For example, say we have a five-node cluster with nodes {A, B, C, D, E} , where {A, B, C} are replicas for the executed write operation, and node B is down. A , being the coordinator for the query, picks node D to satisfy the sloppy quorum and maintain the desired availability and **durability** guarantees. Now, data is replicated to {A, D, C} . However, the record at D will have a hint in its metadata, since the write was originally intended for B . As soon as B recovers, D will attempt to forward a hint back to it. Once the hint is replayed on B , it can be safely removed without reducing the total number of replicas [DECANDIA07] .

Ví dụ, giả sử ta có một cụm năm nút với các nút {A, B, C, D, E}, trong đó {A, B, C} là các bản sao cho thao tác ghi được thực thi, và nút B đang hỏng. A, với vai trò bên điều phối cho truy vấn, chọn nút D để thỏa mãn quorum lỏng và duy trì các bảo đảm sẵn sàng và bền vững mong muốn. Bây giờ, dữ liệu được nhân bản tới {A, D, C}. Tuy nhiên, bản ghi tại D sẽ có một gợi ý trong siêu dữ liệu của nó, vì lần ghi ban đầu vốn dành cho B. Ngay khi B phục hồi, D sẽ cố gắng chuyển tiếp một gợi ý trở lại cho nó. Một khi gợi ý được phát lại trên B, nó có thể được loại bỏ an toàn mà không làm giảm tổng số bản sao [DECANDIA07].

Under similar circumstances, if nodes {B, C} are briefly separated from the rest of the cluster by the **network partition**, and a sloppy quorum write was done against {A, D, E} , a read on {B, C} , immediately following this write, would not observe the latest read [DOWNEY12] . In other words, sloppy quorums improve availability at the cost of consistency.

Trong những hoàn cảnh tương tự, nếu các nút {B, C} bị tách khỏi phần còn lại của cụm trong thời gian ngắn bởi phân vùng mạng (network partition), và một lần ghi quorum lỏng được thực hiện trên {A, D, E}, thì một lần đọc trên {B, C} ngay sau lần ghi này sẽ không quan sát được giá trị đọc mới nhất [DOWNEY12]. Nói cách khác, quorum lỏng cải thiện tính sẵn sàng nhưng đánh đổi bằng tính nhất quán.

Merkle Trees — Cây Merkle

Since read repair can only fix inconsistencies on the currently queried data, we should use different mechanisms to find and repair inconsistencies in the data that is not actively queried.

Vì sửa-khi-đọc chỉ có thể sửa các bất nhất quán trên dữ liệu hiện đang được truy vấn, ta nên dùng các cơ chế khác để tìm và sửa các bất nhất quán trong dữ liệu không được truy vấn tích cực.

As we already discussed, finding exactly which rows have diverged between the replicas requires exchanging and comparing the data records pairwise. This is highly impractical and expensive. Many databases employ Merkle trees [MERKLE87] to reduce the cost of reconciliation.

Như đã bàn, việc tìm chính xác hàng nào đã phân kỳ giữa các bản sao đòi hỏi trao đổi và so sánh các bản ghi dữ liệu theo cặp. Việc này rất không thực tế và tốn kém. Nhiều cơ sở dữ liệu dùng cây Merkle [MERKLE87] để giảm chi phí hòa giải.

Merkle trees compose a compact hashed representation of the local data, building a tree of hashes. The lowest level of this hash tree is built by scanning an entire table holding data records, and computing hashes of record ranges. Higher tree levels contain hashes of the lower-level hashes, building a hierarchical representation that allows us to quickly detect inconsistencies by comparing the hashes, following the hash tree nodes recursively to narrow down inconsistent ranges. This can be done by exchanging and comparing subtrees level-wise, or by exchanging and comparing entire trees.

Cây Merkle tạo nên một biểu diễn băm gọn của dữ liệu cục bộ, xây dựng một cây các mã băm. Tầng thấp nhất của cây băm này được dựng bằng cách quét toàn bộ một bảng chứa các bản ghi dữ liệu và tính các mã băm của các dải bản ghi (record range). Các tầng cao hơn của cây chứa các mã băm của các mã băm tầng thấp hơn, xây dựng một biểu diễn phân cấp cho phép ta nhanh chóng phát hiện các bất nhất quán bằng cách so sánh các mã băm, đi theo các nút của cây băm một cách đệ quy để thu hẹp các dải không nhất quán. Việc này có thể được thực hiện bằng cách trao đổi và so sánh các cây con theo từng tầng, hoặc bằng cách trao đổi và so sánh toàn bộ cây.

Figure 12-2 shows a composition of a **Merkle tree**. The lowest level consists of the hashes of data record ranges. Hashes for each higher level are computed by hashing underlying level hashes, repeating this process recursively up to the tree root.

Hình 12-2 trình bày cấu thành của một cây Merkle. Tầng thấp nhất gồm các mã băm của các dải bản ghi dữ liệu. Các mã băm cho mỗi tầng cao hơn được tính bằng cách băm các mã băm tầng bên dưới, lặp lại tiến trình này một cách đệ quy lên tới gốc cây.

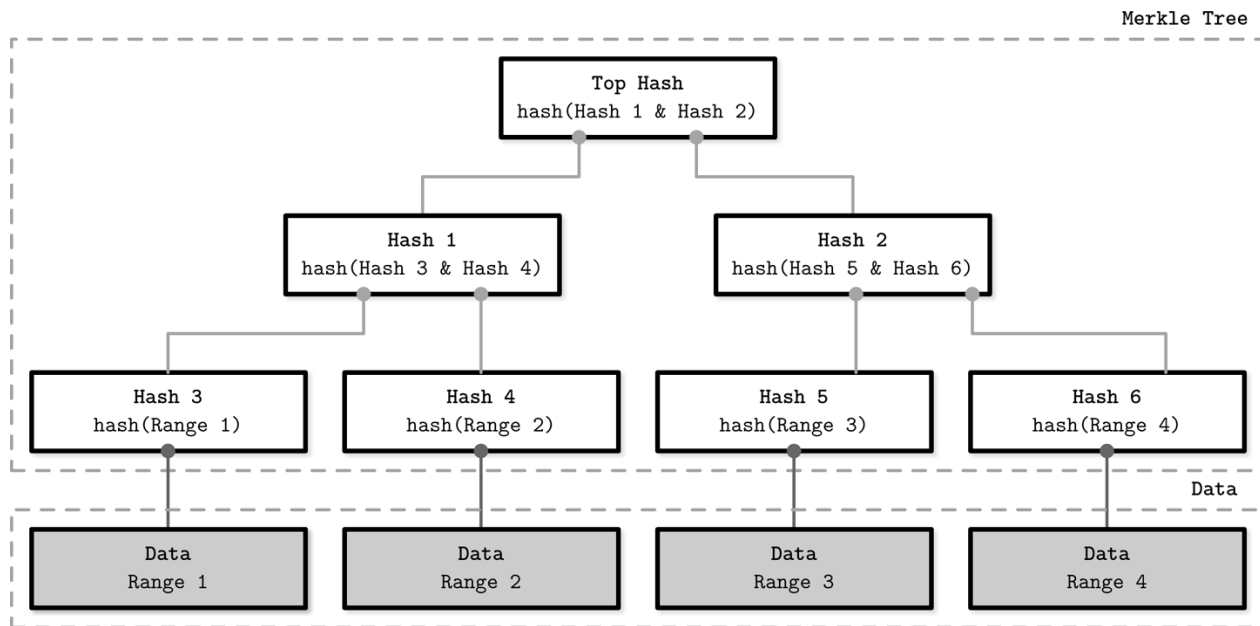


Figure 12-2. Merkle tree. Gray boxes represent data record ranges. White boxes represent a hash tree hierarchy.

Hình 12-2. Cây Merkle. Các hộp xám biểu diễn các dải bản ghi dữ liệu. Các hộp trắng biểu diễn một phân cấp cây băm.

To determine whether or not there's an inconsistency between the two replicas, we only need to compare the root-level hashes from their Merkle trees. By comparing hashes pairwise from top to bottom, it is possible to locate ranges holding differences between the nodes, and repair data records contained in them.

Để xác định liệu có hay không một bất nhất quán giữa hai bản sao, ta chỉ cần so sánh các mã băm ở tầng gốc của các cây Merkle của chúng. Bằng cách so sánh các mã băm theo cặp từ trên xuống dưới, ta có thể định vị các dải chứa khác biệt giữa các nút, và sửa các bản ghi dữ liệu nằm trong chúng.

Since Merkle trees are calculated recursively from the bottom to the top, a change in data triggers recomputation of the entire subtree. There's also a trade-off between the size of a tree (consequently, sizes of exchanged messages) and its precision (how small and exact data ranges are).

Vì cây Merkle được tính đệ quy từ dưới lên trên, một thay đổi trong dữ liệu sẽ kích hoạt việc tính lại toàn bộ cây con. Cũng có một đánh đổi giữa kích thước của cây (kéo theo kích thước các thông điệp trao đổi) và độ chính xác của nó (các dải dữ liệu nhỏ và chính xác đến mức nào).

Bitmap Version Vectors — Vector Phiên bản Bitmap (Bitmap Version Vectors)

More recent research on this subject introduces **bitmap** version vectors [GON- ÇALVES15], which can be used to resolve data conflicts based on recency: each node keeps a per-peer log of operations that have occurred locally or were replicated. During anti-entropy, logs are compared, and missing data is replicated to the target node.

Nghiên cứu gần đây hơn về chủ đề này giới thiệu vector phiên bản bitmap [GONÇALVES15], có thể được dùng để giải quyết các xung đột dữ liệu dựa trên tính mới gần đây (recency): mỗi nút giữ một nhật ký các thao tác theo từng bên ngang hàng đã xảy ra cục bộ hoặc đã được nhân bản. Trong lúc anti-entropy, các nhật ký được so sánh, và dữ liệu bị thiếu được nhân bản tới nút đích.

Each write, coordinated by a node, is represented by a dot (i,n) : an event with a node-local sequence number i coordinated by the node n . The sequence number i starts with 1 and is incremented each time the node executes a write operation.

Mỗi lần ghi, do một nút điều phối, được biểu diễn bằng một chấm (dot) (i,n) : một sự kiện với số thứ tự cục bộ của nút là i được điều phối bởi nút n . Số thứ tự i bắt đầu từ 1 và được tăng mỗi khi nút thực thi một thao tác ghi.

To **track** replica states, we use node-local logical clocks. Each clock represents a set of dots, representing writes this node has seen directly (coordinated by the node itself), or transitively (coordinated by and replicated from the other nodes).

Để theo dõi trạng thái các bản sao, ta dùng các đồng hồ logic cục bộ của nút. Mỗi đồng hồ biểu diễn một tập hợp các chấm, biểu diễn các lần ghi mà nút này đã thấy trực tiếp (do chính nó điều phối), hoặc gián tiếp (được điều phối bởi và nhân bản từ các nút khác).

In the node **logical clock**, events coordinated by the node itself will have no gaps. If some writes aren't replicated from the other nodes, the clock will contain gaps. To get two nodes back in sync, they can exchange logical clocks, identify gaps represented by the missing dots, and then replicate data records associated with them. To do this, we need to reconstruct the data records each dot refers to. This information is stored in a dotted causal container (DCC), which maps dots to causal information for a given key. This way, conflict resolution captures causal relationships between the writes.

Trong đồng hồ logic của nút, các sự kiện do chính nút điều phối sẽ không có khoảng trống. Nếu một số lần ghi không được nhân bản từ các nút khác, đồng hồ sẽ chứa các khoảng trống. Để đưa hai nút trở lại đồng bộ, chúng có thể trao đổi các đồng hồ logic, nhận diện các khoảng trống được biểu diễn bởi các chấm bị thiếu, rồi nhân bản các bản ghi dữ liệu liên kết với chúng. Để làm vậy, ta cần tái dựng các bản ghi dữ liệu mà mỗi chấm tham chiếu tới. Thông tin này được lưu trong một bộ chứa nhân quả có chấm (dotted causal container, DCC), ánh xạ các chấm tới thông tin nhân quả cho một khóa cho trước. Theo cách này, việc giải quyết xung đột nắm bắt được các quan hệ nhân quả giữa các lần ghi.

Figure 12-3 (adapted from [GONÇALVES15]) shows an example of the state representation of three nodes in the system, P , P and P , from the perspective of P , track-

Hình 12-3 (phỏng theo [GONÇALVES15]) trình bày một ví dụ về biểu diễn trạng thái của ba nút trong hệ thống, P , P và P , từ góc nhìn của P , theo dõi

ing which values it has seen. Each time P makes a write or receives a replicated value,

những giá trị mà nó đã thấy. Mỗi lần P thực hiện một lần ghi hoặc nhận một giá trị được nhân bản,

it updates this table.

nó cập nhật bảng này.

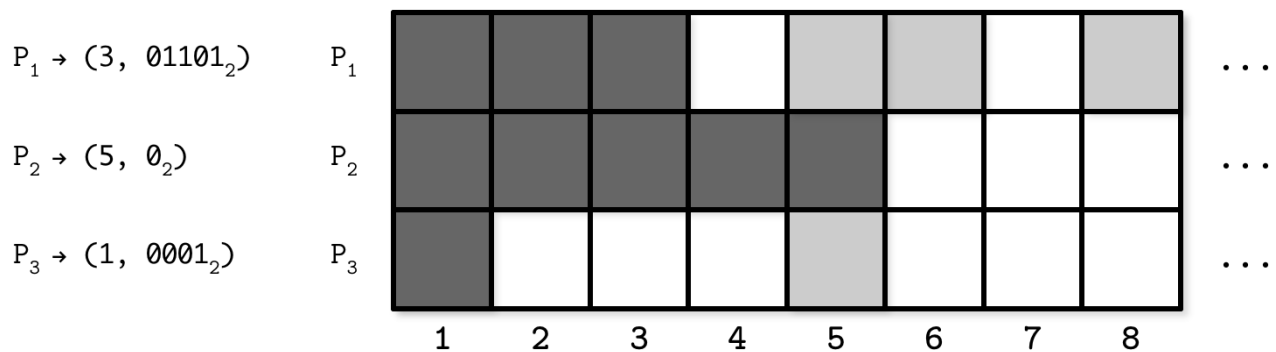


Figure 12-3. Bitmap version vector example

Hình 12-3. Ví dụ về vector phiên bản bitmap

During replication, P creates a compact representation of this state and creates a map

Trong lúc nhân bản, P tạo một biểu diễn gọn của trạng thái này và tạo một ánh xạ

from the node identifier to a pair of latest values, up to which it has seen consecutive writes, and a bitmap where other seen writes are encoded as 1. (3, 01101) here

từ định danh nút tới một cặp gồm giá trị mới nhất, tới ngưỡng mà nó đã thấy các lần ghi liên tiếp, và một bitmap nơi các lần ghi khác đã thấy được mã hóa thành 1. (3, 01101) ở đây

means that node P has seen consecutive updates up to the third value, and it has seen

nghĩa là nút P đã thấy các cập nhật liên tiếp tới giá trị thứ ba, và nó đã thấy

values on the second, third, and fifth position relative to 3 (i.e., it has seen the values with sequence numbers 5, 6, and 8).

các giá trị ở vị trí thứ hai, thứ ba và thứ năm tương đối so với 3 (tức là nó đã thấy các giá trị với số thứ tự 5, 6 và 8).

During exchange with other nodes, it will receive the missing updates the other node has seen. As soon as all the nodes in the system have seen consecutive values up to the index i , the version vector can be truncated up to this index.

Trong lúc trao đổi với các nút khác, nó sẽ nhận các cập nhật bị thiếu mà nút kia đã thấy. Ngay khi tất cả các nút trong hệ thống đã thấy các giá trị liên tiếp tới chỉ số i , vector phiên bản có thể được cắt ngắn tới chỉ số này.

An advantage of this approach is that it captures the causal relation between the value writes and allows nodes to precisely identify the data points missing on the other nodes. A possible downside is that, if the node was down for an extended time period, peer nodes can't truncate the log, since data still has to be replicated to the lagging node once it comes back up.

Một ưu điểm của cách tiếp cận này là nó nắm bắt được quan hệ nhân quả giữa các lần ghi giá trị và cho phép các nút nhận diện chính xác các điểm dữ liệu bị thiếu trên các nút khác. Một nhược điểm có thể có là, nếu một nút bị hỏng trong một khoảng thời gian kéo dài, các nút ngang hàng không thể cắt ngắn nhật ký, vì dữ liệu vẫn phải được nhân bản tới nút tụt hậu một khi nó hoạt động trở lại.

Gossip Dissemination — Lan truyền Gossip (Gossip Dissemination)

Masses are always breeding grounds of psychic epidemics. —Carl Jung

Đám đông luôn là mảnh đất màu mỡ cho các dịch bệnh tâm lý. —Carl Jung

To involve other nodes, and propagate updates with the reach of a broadcast and the reliability of anti-entropy, we can use gossip protocols.

Để lôi kéo các nút khác tham gia, và lan truyền các cập nhật với tầm với của quảng bá và độ tin cậy của anti-entropy, ta có thể dùng các giao thức gossip.

Gossip protocols are probabilistic communication procedures based on how rumors are spread in human society or how diseases propagate in the population. Rumors and epidemics provide rather illustrative ways to describe how these protocols work: rumors spread while the population still has an interest in hearing them; diseases propagate until there are no more susceptible members in the population.

Các giao thức gossip là các thủ tục giao tiếp xác suất dựa trên cách tin đồn lan truyền trong xã hội loài người hoặc cách bệnh tật lan truyền trong quần thể. Tin đồn và dịch bệnh cho ta những cách khá hình tượng để mô tả các giao thức này hoạt động ra sao: tin đồn lan truyền chừng nào quần thể còn hứng thú nghe chúng; dịch bệnh lan truyền cho tới khi không còn thành viên nào dễ nhiễm trong quần thể.

The main objective of gossip protocols is to use cooperative propagation to disseminate information from one process to the rest of the cluster. Just as a virus spreads through the human population by being passed from one individual to another, potentially increasing in scope with each step, information is relayed through the system, getting more processes involved.

Mục tiêu chính của các giao thức gossip là dùng sự lan truyền hợp tác để phổ biến thông tin từ một tiến trình tới phần còn lại của cụm. Giống như một virus lan truyền qua quần thể loài người bằng cách được truyền từ cá thể này sang cá thể khác, có khả năng mở rộng phạm vi qua mỗi bước, thông tin được chuyển tiếp qua hệ thống, lôi kéo thêm nhiều tiến trình tham gia.

A process that holds a record that has to be spread around is said to be infective . Any process that hasn't received the update yet is then susceptible . Infective processes not willing to propagate the new state after a period of active **dissemination** are said to be removed [DEMERS87] . All processes start in a susceptible state. Whenever an update for some data record arrives, a process that received it moves to the infective state and starts disseminating the update to other random neighboring processes, infecting them. As soon as the infective processes become certain that the update was propagated, they move to the removed state.

Một tiến trình giữ một bản ghi cần được lan truyền được gọi là đang lây nhiễm (infective). Bất kỳ tiến trình nào chưa nhận được cập nhật thì được gọi là dễ nhiễm (susceptible). Các tiến trình lây nhiễm không còn sẵn lòng lan truyền trạng thái mới sau một thời gian lan truyền tích cực được gọi là đã loại bỏ (removed) [DEMERS87]. Mọi tiến trình bắt đầu ở trạng thái dễ nhiễm. Mỗi khi một cập nhật cho bản ghi dữ liệu nào đó đến, tiến trình nhận được nó chuyển sang trạng thái lây nhiễm và bắt đầu phổ biến cập nhật tới các tiến trình lân cận ngẫu nhiên khác, lây nhiễm cho chúng. Ngay khi các tiến trình lây nhiễm trở nên chắc chắn rằng cập nhật đã được lan truyền, chúng chuyển sang trạng thái đã loại bỏ.

To avoid explicit coordination and maintaining a global list of recipients and requiring a single coordinator to broadcast messages to each other **participant** in the system, this class of algorithms models completeness using the loss of interest function. The protocol efficiency is then determined

by how quickly it can infect as many nodes as possible, while keeping overhead caused by redundant messages to a minimum.

Để tránh điều phối tường minh và việc duy trì một danh sách bên nhận toàn cục cũng như yêu cầu một bên điều phối duy nhất quảng bá thông điệp tới từng bên tham gia khác trong hệ thống, lớp thuật toán này mô hình hóa tính đầy đủ bằng hàm mất hứng thú (loss of interest function). Hiệu suất của giao thức khi đó được xác định bởi việc nó có thể lây nhiễm cho càng nhiều nút càng nhanh đến mức nào, đồng thời giữ chi phí phụ trội do các thông điệp dư thừa gây ra ở mức tối thiểu.

Gossip can be used for asynchronous message delivery in homogeneous decentralized systems, where nodes may not have long-term membership or be organized in any topology. Since gossip protocols generally do not require explicit coordination, they can be useful in systems with flexible membership (where nodes are joining and leaving frequently) or mesh networks.

Gossip có thể được dùng cho việc giao thông điệp bất đồng bộ trong các hệ phi tập trung đồng nhất, nơi các nút có thể không có tư cách thành viên dài hạn hoặc không được tổ chức theo một cấu trúc liên kết (topology) nào. Vì các giao thức gossip nhìn chung không yêu cầu điều phối tường minh, chúng có thể hữu ích trong các hệ có tư cách thành viên linh hoạt (nơi các nút gia nhập và rời đi thường xuyên) hoặc các mạng lưới (mesh network).

Gossip protocols are very robust and help to achieve high reliability in the presence of failures inherent to distributed systems. Since messages are relayed in a randomized manner, they still can be delivered even if some communication components between them fail, just through the different paths. It can be said that the system adapts to failures.

Các giao thức gossip rất bền bỉ và giúp đạt độ tin cậy cao trong sự hiện diện của các lỗi vốn có trong các hệ phân tán. Vì các thông điệp được chuyển tiếp theo cách ngẫu nhiên hóa, chúng vẫn có thể được giao ngay cả khi một số thành phần giao tiếp giữa chúng gặp lỗi, chỉ là qua các đường đi khác. Có thể nói rằng hệ thống thích nghi với các lỗi.

Gossip Mechanics — Cơ chế Gossip

Processes periodically select f peers at random (where f is a configurable parameter, called **fanout**) and exchange currently “hot” information with them. Whenever the process learns about a new piece of information from its peers, it will attempt to pass it on further. Because peers are selected probabilistically, there will always be some overlap, and messages will get delivered repeatedly and may continue circulating for some time. Message redundancy is a metric that captures the overhead incurred by repeated delivery. Redundancy is an important property, and it is crucial to how gossip works.

Các tiến trình định kỳ chọn ngẫu nhiên f nút ngang hàng (trong đó f là một tham số cấu hình được, gọi là độ phân nhánh (fanout)) và trao đổi với chúng những thông tin hiện đang “nóng”. Mỗi khi tiến trình biết về một mẫu thông tin mới từ các nút ngang hàng của nó, nó sẽ cố gắng chuyển tiếp tiếp. Vì các nút ngang hàng được chọn theo xác suất, sẽ luôn có một chút chồng lấn, và các thông điệp sẽ được giao lặp lại và có thể tiếp tục lưu hành trong một thời gian. Độ dư thừa thông điệp (message redundancy) là một độ đo nắm bắt chi phí phụ trội do việc giao lặp lại gây ra. Độ dư thừa là một tính chất quan trọng, và nó có ý nghĩa then chốt với cách gossip hoạt động.

The amount of time the system requires to reach convergence is called latency. There’s a slight difference between reaching convergence (stopping the gossip process) and delivering the message to all peers, since there might be a short period during which all peers are notified, but gossip

continues. Fanout and latency depend on the system size: in a larger system, we either have to increase the fanout to keep latency stable, or allow higher latency.

Lượng thời gian hệ thống cần để đạt hội tụ được gọi là độ trễ (latency). Có một khác biệt nhỏ giữa việc đạt hội tụ (dừng tiến trình gossip) và việc giao thông điệp tới tất cả các nút ngang hàng, vì có thể có một khoảng thời gian ngắn trong đó tất cả các nút ngang hàng đều đã được thông báo, nhưng gossip vẫn tiếp tục. Độ phân nhánh và độ trễ phụ thuộc vào kích thước hệ thống: trong một hệ lớn hơn, ta hoặc phải tăng độ phân nhánh để giữ độ trễ ổn định, hoặc cho phép độ trễ cao hơn.

Over time, as the nodes notice they've been receiving the same information again and again, the message will start losing importance and nodes will have to eventually stop relaying it. Interest loss can be computed either probabilistically (the probability of propagation stop is computed for each process on every step) or using a threshold (the number of received duplicates is counted, and propagation is stopped when this number is too high). Both approaches have to take the cluster size and fanout into consideration. Counting duplicates to measure convergence can improve latency and reduce redundancy [DEMERS87] .

Theo thời gian, khi các nút nhận ra chúng cứ nhận đi nhận lại cùng một thông tin, thông điệp sẽ bắt đầu mất tầm quan trọng và các nút rốt cuộc sẽ phải dừng chuyển tiếp nó. Việc mất hứng thú có thể được tính theo xác suất (xác suất dừng lan truyền được tính cho mỗi tiến trình ở mỗi bước) hoặc dùng một ngưỡng (đếm số bản trùng lặp nhận được, và việc lan truyền bị dừng khi con số này quá cao). Cả hai cách tiếp cận đều phải xem xét đến kích thước cụm và độ phân nhánh. Việc đếm các bản trùng lặp để đo hội tụ có thể cải thiện độ trễ và giảm độ dư thừa [DEMERS87].

In terms of consistency, gossip protocols offer convergent consistency [BIRMAN07] : nodes have a higher probability to have the same view of the events that occurred further in the past.

Về mặt nhất quán, các giao thức gossip cung cấp nhất quán hội tụ (convergent consistency) [BIRMAN07]: các nút có xác suất cao hơn để có cùng góc nhìn về những sự kiện đã xảy ra xa hơn trong quá khứ.

Overlay Networks — Mạng Phủ (Overlay Networks)

Even though gossip protocols are important and useful, they're usually applied for a narrow set of problems. Nonepidemic approaches can distribute the message with nonprobabilistic certainty, less redundancy, and generally in a more optimal way [BIRMAN07] . Gossip algorithms are often praised for their scalability and the fact it is possible to distribute a message within $\log N$ message rounds (where N is the size of the cluster) [KREMARREC07] , but it's important to keep the number of redundant messages generated during gossip rounds in mind as well. To achieve reliability, gossip-based protocols produce some duplicate message deliveries.

Mặc dù các giao thức gossip quan trọng và hữu ích, chúng thường được áp dụng cho một tập hẹp các vấn đề. Các cách tiếp cận phi dịch tễ (nonepidemic) có thể phân phối thông điệp với độ chắc chắn phi xác suất, ít dư thừa hơn, và nhìn chung theo cách tối ưu hơn [BIRMAN07]. Các thuật toán gossip thường được ca ngợi vì tính mở rộng của chúng và vì có thể phân phối một thông điệp trong vòng $\log N$ vòng thông điệp (trong đó N là kích thước của cụm) [KREMARREC07], nhưng cũng cần lưu ý đến số lượng thông điệp dư thừa được sinh ra trong các vòng gossip. Để đạt độ tin cậy, các giao thức dựa trên gossip tạo ra một số lần giao thông điệp trùng lặp.

Selecting nodes at random greatly improves system robustness : if there is a network partition,

messages will be delivered eventually if there are links that indirectly connect two processes. The obvious downside of this approach is that it is not message-optimal: to guarantee robustness, we have to maintain redundant connections between the peers and send redundant messages.

Việc chọn ngẫu nhiên các nút cải thiện đáng kể tính bền bỉ của hệ thống: nếu có một phân vùng mạng, các thông điệp rất cuộc sẽ được giao nếu có các liên kết gián tiếp kết nối hai tiến trình. Nhược điểm rõ ràng của cách tiếp cận này là nó không tối ưu về thông điệp: để bảo đảm tính bền bỉ, ta phải duy trì các kết nối dư thừa giữa các nút ngang hàng và gửi các thông điệp dư thừa.

A middle ground between the two approaches is to construct a temporary fixed topology in a gossip system. This can be achieved by creating an overlay network of peers: nodes can sample their peers and select the best contact points based on proximity (usually measured by the latency).

Một điểm trung dung giữa hai cách tiếp cận là xây dựng một cấu trúc liên kết cố định tạm thời trong một hệ gossip. Điều này có thể đạt được bằng cách tạo một mạng phủ (overlay network) của các nút ngang hàng: các nút có thể lấy mẫu các nút ngang hàng của chúng và chọn các điểm liên hệ tốt nhất dựa trên độ gần (thường được đo bằng độ trễ).

Nodes in the system can form spanning trees : unidirected, loop-free graphs with distinct edges, covering the whole network. Having such a graph, messages can be distributed in a fixed number of steps.

Các nút trong hệ thống có thể tạo thành các cây bao trùm (spanning tree): các đồ thị vô hướng, không vòng lặp với các cạnh phân biệt, bao phủ toàn bộ mạng. Có được một đồ thị như vậy, các thông điệp có thể được phân phối trong một số bước cố định.

Figure 12-4 shows an example of a spanning tree: 1

Hình 12-4 trình bày một ví dụ về cây bao trùm: 1

- a) We achieve full connectivity between the points without using all the edges.
- b) We can lose connectivity to the entire subtree if just a single **link** is broken.
 - a) Ta đạt được tính liên thông đầy đủ giữa các điểm mà không dùng tất cả các cạnh.
 - b) Ta có thể mất tính liên thông tới toàn bộ cây con nếu chỉ một liên kết bị hỏng.

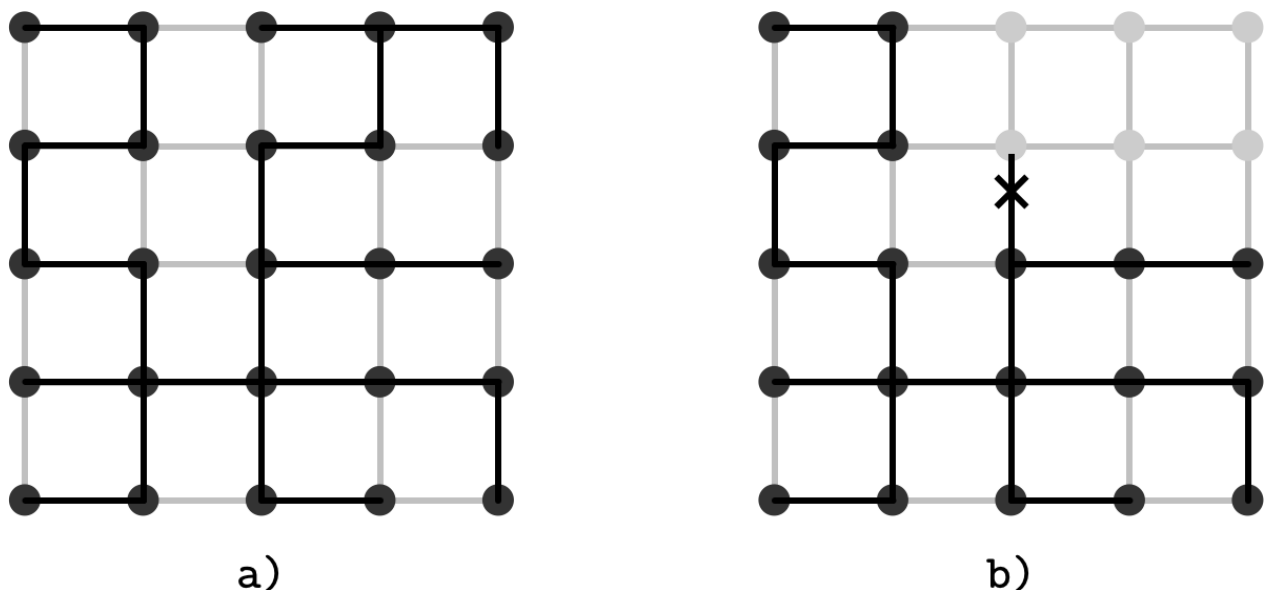


Figure 12-4. Spanning tree. Dark points represent nodes. Dark lines represent an overlay network. Gray lines represent other possible existing connections between the nodes.

Hình 12-4. Cây bao trùm. Các điểm đậm biểu diễn các nút. Các đường đậm biểu diễn một mạng phủ. Các đường xám biểu diễn các kết nối có thể đang tồn tại khác giữa các nút.

One of the potential downsides of this approach is that it might lead to forming interconnected “islands” of peers having strong preferences toward each other.

Một trong những nhược điểm tiềm tàng của cách tiếp cận này là nó có thể dẫn tới việc hình thành các “hòn đảo” các nút ngang hàng liên thông với nhau, có thiên hướng mạnh ưu ái lẫn nhau.

To keep the number of messages low, while allowing quick **recovery** in case of a connectivity loss, we can mix both approaches—fixed topologies and tree-based broadcast—when the system is in a stable state, and fall back to gossip for failover and system recovery.

Để giữ số lượng thông điệp thấp, đồng thời cho phép phục hồi nhanh trong trường hợp mất liên thông, ta có thể kết hợp cả hai cách tiếp cận—các cấu trúc liên kết cố định và quảng bá dựa trên cây—khi hệ thống ở trạng thái ổn định, và lùi về gossip cho việc chuyển đổi dự phòng (failover) và phục hồi hệ thống.

1 This example is only used for illustration: nodes in the network are generally not arranged in a grid.

1 Ví dụ này chỉ được dùng để minh họa: các nút trong mạng nhìn chung không được sắp xếp theo dạng lưới.

Hybrid Gossip — Gossip Lai (Hybrid Gossip)

Push/lazy-push multicast trees (Plumtrees) [LEITAO07] make a trade-off between epidemic and tree-based broadcast primitives. Plumtrees work by creating a spanning tree overlay of nodes to actively distribute messages with the smallest overhead. Under normal conditions, nodes send full messages to just a small subset of peers provided by the peer sampling service.

Các cây phát đa hướng push/lazy-push (Plumtree) [LEITAO07] tạo ra một đánh đổi giữa các nguyên thủy quảng bá dịch tễ và dựa trên cây. Plumtree hoạt động bằng cách tạo một mạng phủ cây bao trùm của các nút để chủ động phân phối thông điệp với chi phí phụ trội nhỏ nhất. Trong điều kiện bình thường, các nút gửi thông điệp đầy đủ chỉ tới một tập con nhỏ các nút ngang hàng do dịch vụ lấy mẫu nút ngang hàng cung cấp.

Each node sends the full message to the small subset of nodes, and for the rest of the nodes, it lazily forwards only the message ID. If the node receives the identifier of a message it has never seen, it can query its peers to get it. This lazy-push step ensures high reliability and provides a way to quickly heal the broadcast tree. In case of failures, protocol falls back to the gossip approach through lazy-push steps, broadcasting the message and repairing the overlay.

Mỗi nút gửi thông điệp đầy đủ tới tập con nhỏ các nút, và với các nút còn lại, nó lười (lazily) chỉ chuyển tiếp ID của thông điệp. Nếu nút nhận được định danh của một thông điệp nó chưa từng thấy, nó có thể truy vấn các nút ngang hàng của nó để lấy thông điệp đó. Bước lazy-push này bảo đảm độ tin cậy cao và cung cấp một cách để nhanh chóng vá lành cây quảng bá. Trong trường hợp có lỗi, giao thức lùi về cách tiếp cận gossip thông qua các bước lazy-push, quảng bá thông điệp và sửa chữa mạng phủ.

Due to the nature of distributed systems, any node or link between the nodes might fail at any time, making it impossible to traverse the tree when the segment becomes unreachable. The lazy gossip network helps to notify peers about seen messages in order to construct and repair the tree.

Do bản chất của các hệ phân tán, bất kỳ nút hoặc liên kết nào giữa các nút đều có thể gặp lỗi bất kỳ lúc nào, khiến không thể duyệt cây khi đoạn đó trở nên không thể tiếp cận. Mạng gossip lười giúp thông báo cho các nút ngang hàng về các thông điệp đã thấy nhằm xây dựng và sửa chữa cây.

Figure 12-5 shows an illustration of such double connectivity: nodes are connected with an optimal spanning tree (solid lines) and the lazy gossip network (dotted lines). This illustration does not represent any particular network topology, but only connections between the nodes.

Hình 12-5 trình bày một minh họa về tính liên thông kép như vậy: các nút được kết nối bằng một cây bao trùm tối ưu (các đường liền) và mạng gossip lười (các đường chấm). Minh họa này không biểu diễn bất kỳ cấu trúc liên kết mạng cụ thể nào, mà chỉ biểu diễn các kết nối giữa các nút.

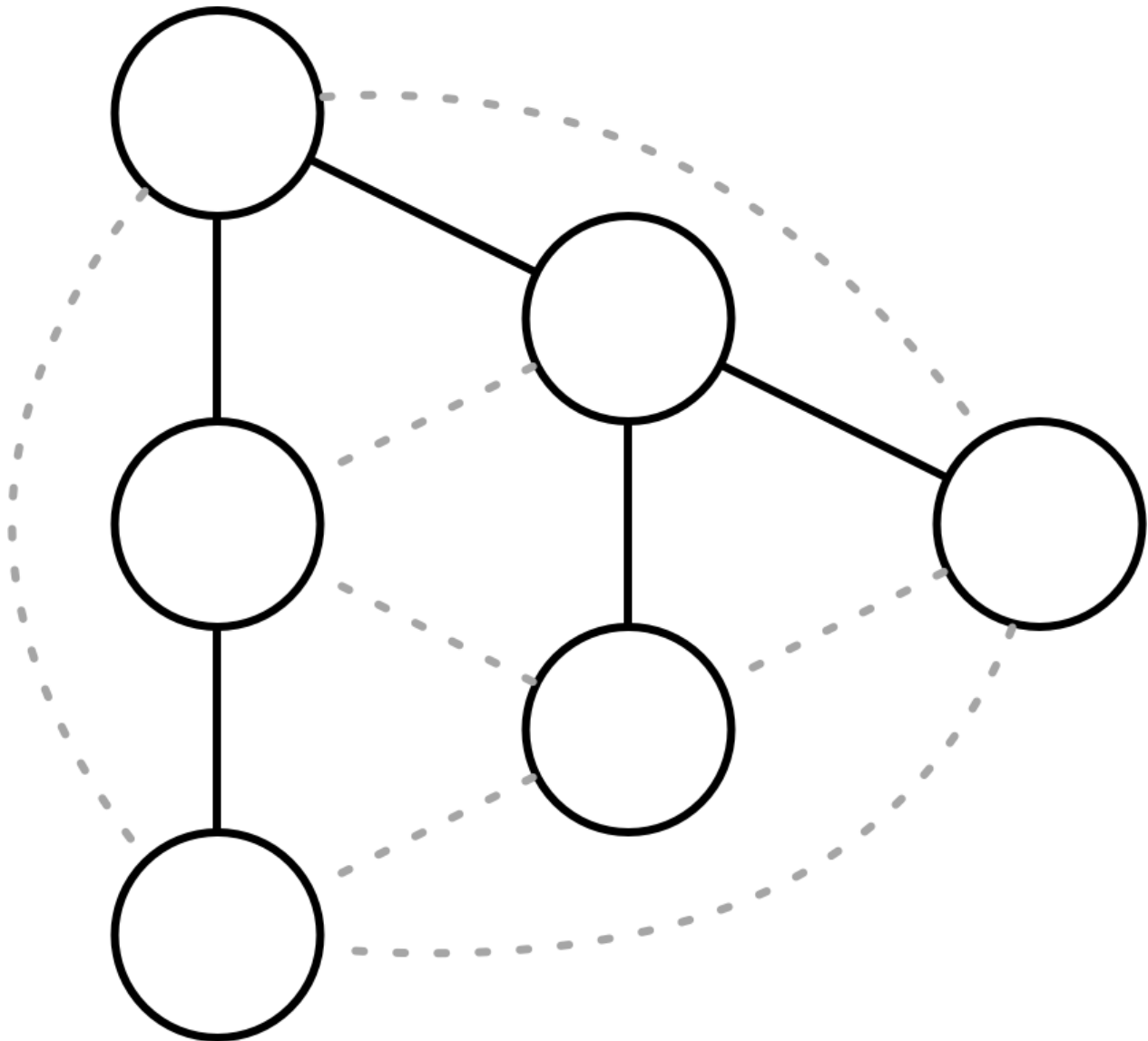


Figure 12-5. Lazy and eager push networks. Solid lines represent a broadcast tree. Dotted lines represent lazy gossip connections.

Hình 12-5. Các mạng push lười và push tích cực (eager). Các đường liền biểu diễn một cây quảng bá. Các đường chấm biểu diễn các kết nối gossip lười.

One of the advantages of using the lazy-push mechanism for tree construction and repair is that in a network with constant load, it will tend to generate a tree that also minimizes message latency, since nodes that are first to respond are added to the broadcast tree.

Một trong những ưu điểm của việc dùng cơ chế lazy-push để xây dựng và sửa chữa cây là trong một mạng với tải không đổi, nó sẽ có xu hướng sinh ra một cây cũng tối thiểu hóa độ trễ thông điệp, vì các nút phản hồi đầu tiên được thêm vào cây quảng bá.

Partial Views — Góc nhìn Cục bộ (Partial Views)

Broadcasting messages to all known peers and maintaining a full view of the cluster can get expensive and impractical, especially if the churn (measure of the number of joining and leaving nodes in the system) is high. To avoid this, gossip protocols often use a peer sampling service. This service maintains a partial view of the cluster, which is periodically refreshed using gossip. Partial views overlap, as some degree of redundancy is desired in gossip protocols, but too much redundancy means we're doing extra work.

Việc quảng bá thông điệp tới tất cả các nút ngang hàng đã biết và duy trì một góc nhìn đầy đủ của cụm có thể trở nên tốn kém và không thực tế, đặc biệt nếu mức biến động (churn — thước đo số lượng nút gia nhập và rời đi trong hệ thống) cao. Để tránh điều này, các giao thức gossip thường dùng một dịch vụ lấy mẫu nút ngang hàng (peer sampling service). Dịch vụ này duy trì một góc nhìn cục bộ của cụm, được làm mới định kỳ bằng gossip. Các góc nhìn cục bộ chồng lấn nhau, vì một mức độ dư thừa nào đó là điều mong muốn trong các giao thức gossip, nhưng quá nhiều dư thừa nghĩa là ta đang làm thừa việc.

For example, the Hybrid Partial View (HyParView) protocol [LEITAO07] maintains a small active view and a larger passive view of the cluster. Nodes from the active view create an overlay that can be used for dissemination. Passive view is used to maintain a list of nodes that can be used to replace the failed ones from the active view.

Ví dụ, giao thức Hybrid Partial View (HyParView) [LEITAO07] duy trì một góc nhìn chủ động (active view) nhỏ và một góc nhìn thụ động (passive view) lớn hơn của cụm. Các nút từ góc nhìn chủ động tạo một mạng phủ có thể được dùng để lan truyền. Góc nhìn thụ động được dùng để duy trì một danh sách các nút có thể được dùng để thay thế các nút bị lỗi từ góc nhìn chủ động.

Periodically, nodes perform a shuffle operation, during which they exchange their active and passive views. During this exchange, nodes add the members from both passive and active views they receive from their peers to their passive views, cycling out the oldest values to cap the list size.

Định kỳ, các nút thực hiện một thao tác xáo trộn (shuffle), trong đó chúng trao đổi các góc nhìn chủ động và thụ động của mình. Trong lúc trao đổi này, các nút thêm các thành viên từ cả góc nhìn thụ động và chủ động mà chúng nhận được từ các nút ngang hàng vào góc nhìn thụ động của mình, loại bỏ dần các giá trị cũ nhất để giới hạn kích thước danh sách.

The active view is updated depending on the state changes of nodes in this view and requests from peers. If a process P suspects that P, one of the peers from its active

Góc nhìn chủ động được cập nhật tùy theo các thay đổi trạng thái của các nút trong góc nhìn này và các yêu cầu từ các nút ngang hàng. Nếu một tiến trình P nghi ngờ rằng P, một trong các nút ngang hàng từ góc nhìn chủ động của nó,

view, has failed, P removes P from its active view and attempts to establish a connec-

đã gặp lỗi, P loại bỏ P khỏi góc nhìn chủ động của nó và cố gắng thiết lập một kết nối

tion with a replacement process P from the passive view. If the connection fails, P is

với một tiến trình thay thế P từ góc nhìn thụ động. Nếu kết nối thất bại, P bị

removed from the passive view of P .

loại bỏ khỏi góc nhìn thụ động của P.

Depending on the number of processes in P's active view, P may choose to decline

Tùy theo số lượng tiến trình trong góc nhìn chủ động của P, P có thể chọn từ chối

the connection if its active view is already full. If P's view is empty, P has to replace

kết nối nếu góc nhìn chủ động của nó đã đầy. Nếu góc nhìn của P trống, P phải thay thế

one of its current active view peers with P . This helps bootstrapping or recovering

một trong các nút ngang hàng góc nhìn chủ động hiện tại của nó bằng P. Điều này giúp các nút đang khởi động (bootstrapping) hoặc đang phục hồi

nodes to quickly become effective members of the cluster at the cost of cycling some connections.

nhANH chóng trở thành thành viên hữu hiệu của cụm với cái giá là phải xoay vòng một số kết nối.

This approach helps to reduce the number of messages in the system by using only active view nodes for dissemination, while maintaining high reliability by using passive views as a recovery mechanism. One of the performance and quality measures is how quickly a peer sampling service converges to a stable overlay in cases of topology reorganization [JELASITY04] . HyParView scores rather high here, because of how the views are maintained and since it gives priority to bootstrapping processes.

Cách tiếp cận này giúp giảm số lượng thông điệp trong hệ thống bằng cách chỉ dùng các nút góc nhìn chủ động để lan truyền, đồng thời duy trì độ tin cậy cao bằng cách dùng các góc nhìn thụ động làm cơ chế phục hồi. Một trong các thước đo hiệu năng và chất lượng là một dịch vụ lấy mẫu nút ngang hàng hội tụ tới một mạng phủ ổn định nhanh đến mức nào trong các trường hợp tái tổ chức cấu trúc liên kết [JELASITY04]. HyParView đạt điểm khá cao ở đây, nhờ cách các góc nhìn được duy trì và vì nó ưu tiên các tiến trình đang khởi động.

HyParView and Plumtree use a hybrid gossip approach: using a small subset of peers for broadcasting messages and falling back to a wider network of peers in case of failures and network partitions. Both systems do not rely on a global view that includes all the peers, which can be helpful not only because of a large number of nodes in the system (which is not the case most of the time), but also because of costs associated with maintaining an up-to-date list of members on every node. Partial views allow nodes to actively communicate with only a small subset of neighboring nodes.

HyParView và Plumtree dùng một cách tiếp cận gossip lai: dùng một tập con nhỏ các nút ngang hàng để quảng bá thông điệp và lùi về một mạng rộng hơn các nút ngang hàng trong trường hợp có lỗi và phân vùng mạng. Cả hai hệ thống đều không dựa vào một góc

nhìn toàn cục bao gồm tất cả các nút ngang hàng, điều này có thể hữu ích không chỉ vì số lượng nút lớn trong hệ thống (vốn không phải là trường hợp thường gặp), mà còn vì các chi phí liên quan tới việc duy trì một danh sách thành viên cập nhật mới nhất trên mỗi nút. Các góc nhìn cục bộ cho phép các nút chủ động giao tiếp chỉ với một tập con nhỏ các nút lân cận.

Summary — Tóm tắt

Eventually consistent systems allow replica state divergence. Tunable consistency allows us to trade consistency for availability and vice versa. Replica divergence can be resolved using one of the anti-entropy mechanisms:

Các hệ nhất quán cuối cùng cho phép trạng thái các bản sao phân kỳ. Nhất quán điều chỉnh được cho phép ta đánh đổi nhất quán lấy tính sẵn sàng và ngược lại. Sự phân kỳ bản sao có thể được giải quyết bằng một trong các cơ chế anti-entropy:

Hinted handoff Temporarily store writes on neighboring nodes in case the target is down, and replay them on the target as soon as it comes back up.

Hinted handoff Tạm thời lưu các lần ghi trên các nút lân cận trong trường hợp nút đích hỏng, và phát lại chúng trên nút đích ngay khi nó hoạt động trở lại.

Read-repair Reconcile requested data ranges during the read by comparing responses, detecting missing records, and sending them to lagging replicas.

Sửa-khi-đọc Hòa giải các dải dữ liệu được yêu cầu trong lúc đọc bằng cách so sánh các phản hồi, phát hiện các bản ghi bị thiếu, và gửi chúng tới các bản sao tụt hậu.

Merkle trees Detect data ranges that require repair by computing and exchanging hierarchical trees of hashes.

Cây Merkle Phát hiện các dải dữ liệu cần sửa bằng cách tính và trao đổi các cây băm phân cấp.

Bitmap version vectors Detect missing replica writes by maintaining compact records containing information about the most recent writes.

Vector phiên bản bitmap Phát hiện các lần ghi bản sao bị thiếu bằng cách duy trì các bản ghi gọn chứa thông tin về các lần ghi gần đây nhất.

These anti-entropy approaches optimize for one of the three parameters: scope reduction, recency, or completeness. We can reduce the scope of anti-entropy by only synchronizing the data that is being actively queried (read-repairs) or individual missing writes (hinted handoff). If we assume that most failures are temporary and participants recover from them as quickly as possible, we can store the log of the most recent diverged events and know exactly what to synchronize in the event of failure (bitmap version vectors). If we need to compare entire datasets on multiple nodes pairwise and efficiently locate differences between them, we can hash the data and compare hashes (Merkle trees).

Các cách tiếp cận anti-entropy này tối ưu cho một trong ba tham số: giảm phạm vi, tính mới gần đây, hoặc tính đầy đủ. Ta có thể giảm phạm vi của anti-entropy bằng cách chỉ đồng bộ dữ liệu đang được truy vấn tích cực (sửa-khi-đọc) hoặc từng lần ghi riêng lẻ bị thiếu (hinted handoff). Nếu ta giả định rằng hầu hết các lỗi là tạm thời và các bên tham gia phục hồi từ chúng nhanh nhất có thể, ta có thể lưu nhật ký các sự kiện phân kỳ gần đây nhất và biết chính xác cần đồng bộ cái gì trong sự kiện lỗi (vector phiên bản bitmap).

Nếu ta cần so sánh toàn bộ các tập dữ liệu trên nhiều nút theo cặp và định vị các khác biệt giữa chúng một cách hiệu quả, ta có thể băm dữ liệu và so sánh các mã băm (cây Merkle).

To reliably distribute information in a large-scale system, gossip protocols can be used. Hybrid gossip protocols reduce the number of exchanged messages while remaining resistant to network partitions, when possible.

Để phân phối thông tin một cách đáng tin cậy trong một hệ quy mô lớn, các giao thức gossip có thể được dùng. Các giao thức gossip lai giảm số lượng thông điệp trao đổi trong khi vẫn kháng được các phân vùng mạng, khi có thể.

Many modern systems use gossip for failure detection and membership information [DECANDIA07]. HyParView is used in Partisan, the high-performance, high-scalability distributed computing framework. Plumtree was used in the Riak core for cluster-wide information.

Nhiều hệ thống hiện đại dùng gossip để phát hiện lỗi và lấy thông tin thành viên [DECANDIA07]. HyParView được dùng trong Partisan, framework điện toán phân tán hiệu năng cao, khả mở rộng cao. Plumtree được dùng trong nhân (core) của Riak cho thông tin phạm vi toàn cụm.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Gossip protocols Shah, Devavrat. 2009. "Gossip Algorithms." *Foundations and Trends in Networking* 3, no. 1 (January): 1-125. <https://doi.org/10.1561/13000000014>.

Jelasity, Márk. 2003. "Gossip-based Protocols for Large-scale Distributed Systems." Dissertation. <http://www.inf.u-szeged.hu/~jelasity/dr/doktori-mu.pdf>.

Demers, Alan, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. 1987. "Epidemic algorithms for replicated database maintenance." In *Proceedings of the sixth annual ACM Symposium on Principles of distributed computing (PODC '87)*, 1-12. New York: Association for Computing Machinery. <https://doi.org/10.1145/41840.41841>.

CHAPTER 13 Distributed Transactions — Chương 13: Giao dịch phân tán

To maintain order in a **distributed system**, we have to guarantee at least some consistency. In “Consistency Models” on **page 222**, we talked about single-object, single-operation consistency models that help us to reason about the individual operations. However, in databases we often need to execute multiple operations atomically.

Để duy trì trật tự trong một hệ phân tán, chúng ta phải bảo đảm ít nhất một mức nhất quán nào đó. Trong phần “Consistency Models” ở trang 222, ta đã bàn về các mô hình nhất quán đơn-đối-tượng, đơn-thao-tác, giúp ta lập luận về từng thao tác riêng lẻ. Tuy nhiên, trong cơ sở dữ liệu ta thường cần thực thi nhiều thao tác một cách nguyên tử.

Atomic operations are explained in terms of state transitions: the database was in state A before a particular transaction was started; by the time it finished, the state went from A to B. In operation terms, this is simple to understand, since transactions have no predetermined attached state. Instead, they apply operations to data records starting at some point in time. This gives us some flexibility in terms of scheduling and execution: transactions can be reordered and even retried.

Các thao tác nguyên tử được giải thích qua các chuyển trạng thái: cơ sở dữ liệu ở trạng thái A trước khi một giao dịch (transaction) cụ thể bắt đầu; tới khi giao dịch hoàn tất, trạng thái chuyển từ A sang B. Xét theo góc độ thao tác, điều này dễ hiểu, vì giao dịch không có trạng thái gắn sẵn được định trước. Thay vào đó, chúng áp dụng các thao tác lên các bản ghi dữ liệu bắt đầu tại một thời điểm nào đó. Điều này cho ta sự linh hoạt về mặt lập lịch và thực thi: các giao dịch có thể được sắp xếp lại thứ tự, và thậm chí được thử lại.

The main focus of **transaction processing** is to determine permissible histories, to model and represent possible interleaving execution scenarios. History, in this case, represents a dependency graph: which transactions have been executed prior to execution of the current transaction. History is said to be serializable if it is equivalent (i.e., has the same dependency graph) to some history that executes these transactions sequentially. You can review concepts of histories, their equivalence, **serializability**, and other concepts in “Serializability” on page 94. Generally, this chapter is a distributed systems counterpart of Chapter 5, where we discussed **node**-local transaction processing.

Trọng tâm chính của xử lý giao dịch là xác định các lịch sử (history) hợp lệ, để mô hình hóa và biểu diễn các kịch bản thực thi đan xen có thể xảy ra. Trong trường hợp này, lịch sử biểu diễn một đồ thị phụ thuộc: những giao dịch nào đã được thực thi trước khi giao dịch hiện tại được thực thi. Một lịch sử được gọi là khả tuần tự (serializable) nếu nó tương

đương (tức là có cùng đồ thị phụ thuộc) với một lịch sử nào đó thực thi các giao dịch này một cách tuần tự. Bạn có thể ôn lại các khái niệm về lịch sử, sự tương đương giữa chúng, tính khả tuần tự, và các khái niệm khác trong phần “Serializability” ở trang 94. Nói chung, chương này là phần tương ứng trong hệ phân tán của Chương 5, nơi ta đã bàn về xử lý giao dịch cục bộ trên một nút.

Single-partition transactions involve the **pessimistic (lock-based or tracking)** or optimistic (try and validate) **concurrency control** schemes that we discussed in Chapter 5 , but neither one of these approaches solves the problem of multipartition transactions, which require coordination between different servers, distributed commit, and rollback protocols.

Các giao dịch đơn-phân-vùng dùng các cơ chế điều khiển tương tranh bi quan (dựa trên khóa hoặc theo dõi) hoặc lạc quan (thử rồi kiểm chứng) mà ta đã bàn ở Chương 5, nhưng không cách nào trong số này giải quyết được bài toán giao dịch đa-phân-vùng, vốn đòi hỏi sự phối hợp giữa các máy chủ khác nhau cùng các giao thức commit và rollback phân tán.

257 Generally speaking, when transferring money from one account to another, you’d like to both credit the first account and debit the second one simultaneously . However, if we break down the transaction into individual steps, even debiting or crediting doesn’t look atomic at first sight: we need to **read** the old balance, add or subtract the required amount, and save this result. Each one of these substeps involves several operations: the node receives a request, parses it, locates the data on disk, makes a write and, finally, acknowledges it. Even this is a rather high-level view: to execute a simple write, we have to perform hundreds of small steps.

Nói chung, khi chuyển tiền từ tài khoản này sang tài khoản khác, bạn muốn vừa ghi có vào tài khoản thứ nhất vừa ghi nợ tài khoản thứ hai một cách đồng thời. Tuy nhiên, nếu ta phân rã giao dịch thành các bước riêng lẻ, thì ngay cả việc ghi nợ hay ghi có thoạt nhìn cũng không có vẻ nguyên tử: ta cần đọc số dư cũ, cộng hoặc trừ lượng tiền cần thiết, rồi lưu kết quả này lại. Mỗi bước con đó lại bao gồm nhiều thao tác: nút nhận yêu cầu, phân tích nó, định vị dữ liệu trên đĩa, thực hiện một lần ghi và cuối cùng xác nhận lại. Ngay cả đây cũng chỉ là góc nhìn khá cao cấp: để thực thi một lần ghi đơn giản, ta phải thực hiện hàng trăm bước nhỏ.

This means that we have to first execute the transaction and only then make its results visible . But let’s first define what transactions are. A transaction is a set of operations, an atomic unit of execution. Transaction **atomicity** implies that all its results become visible or none of them do. For example, if we modify several rows, or even tables in a single transaction, either all or none of the modifications will be applied.

Điều này có nghĩa là ta phải thực thi giao dịch trước rồi mới làm cho kết quả của nó trở nên hiển thị. Nhưng trước hết hãy định nghĩa giao dịch là gì. Một giao dịch là một tập các thao tác, một đơn vị thực thi nguyên tử. Tính nguyên tử của giao dịch hàm ý rằng hoặc tất cả kết quả của nó trở nên hiển thị, hoặc không kết quả nào hiển thị. Ví dụ, nếu ta sửa đổi vài hàng, hay thậm chí vài bảng trong một giao dịch, thì hoặc tất cả hoặc không sửa đổi nào được áp dụng.

To ensure atomicity, transactions should be recoverable . In other words, if the transaction cannot complete, is aborted, or times out, its results have to be rolled back completely. A nonrecoverable, partially executed transaction can leave the database in an inconsistent state. In summary, in case of unsuccessful transaction execution, the database state has to be reverted to its previous state, as if this transaction was never tried in the first place.

Để bảo đảm tính nguyên tử, các giao dịch phải có thể phục hồi được. Nói cách khác, nếu giao dịch không thể hoàn tất, bị hủy bỏ, hoặc hết thời gian chờ, kết quả của nó phải được

rollback hoàn toàn. Một giao dịch không thể phục hồi, thực thi dở dang có thể để cơ sở dữ liệu ở trạng thái không nhất quán. Tóm lại, trong trường hợp giao dịch thực thi không thành công, trạng thái cơ sở dữ liệu phải được khôi phục về trạng thái trước đó, như thể giao dịch này chưa từng được thử ngay từ đầu.

Another important aspect is network partitions and node failures: nodes in the system fail and recover independently, but their states have to remain consistent. This means that the atomicity requirement holds not only for the local operations, but also for operations executed on other nodes: changes have to be durably propagated to all of the nodes involved in the transaction or none of them [LAMPSON79] .

Một khía cạnh quan trọng khác là phân vùng mạng (network partition) và lỗi nút: các nút trong hệ thống lỗi và phục hồi một cách độc lập, nhưng trạng thái của chúng phải nhất quán với nhau. Điều này nghĩa là yêu cầu về tính nguyên tử không chỉ áp dụng cho các thao tác cục bộ, mà cả cho các thao tác thực thi trên các nút khác: các thay đổi phải được lan truyền một cách bền vững tới tất cả các nút liên quan trong giao dịch, hoặc không tới nút nào cả [LAMPSON79].

Making Operations Appear Atomic — Làm cho các thao tác có vẻ nguyên tử

To make multiple operations appear atomic, especially if some of them are remote, we need to use a class of algorithms called atomic commitment . Atomic commitment doesn't allow disagreements between the participants: a transaction will not commit if even one of the participants votes against it. At the same time, this means that failed processes have to reach the same conclusion as the rest of the **cohort**. Another important implication of this fact is that atomic commitment algorithms do not work in the presence of Byzantine failures: when the **process** lies about its state or decides on an arbitrary value, since it contradicts unanimity [HADZILACOS05] .

Để làm cho nhiều thao tác có vẻ nguyên tử, đặc biệt khi một số trong đó là từ xa, ta cần dùng một lớp thuật toán gọi là cam kết nguyên tử (atomic commitment). Cam kết nguyên tử không cho phép bất đồng giữa các bên tham gia: một giao dịch sẽ không commit nếu dù chỉ một bên tham gia bỏ phiếu chống lại nó. Đồng thời, điều này nghĩa là các tiến trình bị lỗi phải đi đến cùng một kết luận như phần còn lại của nhóm tham gia (cohort). Một hệ quả quan trọng khác của thực tế này là các thuật toán cam kết nguyên tử không hoạt động được khi có lỗi Byzantine: khi tiến trình nói dối về trạng thái của nó hoặc quyết định trên một giá trị tùy tiện, vì điều đó mâu thuẫn với sự nhất trí [HADZILACOS05].

The problem that atomic commitment is trying to solve is reaching an agreement on whether or not to execute the proposed transaction. Cohorts cannot choose, influence, or change the proposed transaction or propose any alternative: they can only give their vote on whether or not they are willing to execute it [ROBINSON08] .

Bài toán mà cam kết nguyên tử cố giải quyết là đạt được sự đồng thuận về việc có thực thi giao dịch được đề xuất hay không. Các bên tham gia không thể chọn, tác động, hay thay đổi giao dịch được đề xuất, cũng không thể đề xuất bất kỳ phương án thay thế nào: họ chỉ có thể bỏ phiếu về việc liệu mình có sẵn lòng thực thi nó hay không [ROBINSON08].

Atomic commitment algorithms do not set strict requirements for the semantics of transaction prepare , commit , or rollback operations. Database implementers have to decide on:

Các thuật toán cam kết nguyên tử không đặt ra yêu cầu nghiêm ngặt cho ngữ nghĩa của các thao tác prepare, commit, hay rollback của giao dịch. Người triển khai cơ sở dữ liệu

phải tự quyết định về:

- When the data is considered ready to commit, and they're just a pointer swap away from making the changes public.
- How to perform the commit itself to make transaction results visible in the shortest timeframe possible.
- How to roll back the changes made by the transaction if the algorithm decides not to commit.
 - Khi nào dữ liệu được xem là sẵn sàng để commit, và họ chỉ còn cách một thao tác hoán đổi con trỏ nữa là có thể công khai các thay đổi.
 - Cách thực hiện commit nhằm làm cho kết quả giao dịch hiển thị trong khoảng thời gian ngắn nhất có thể.
 - Cách rollback các thay đổi do giao dịch thực hiện nếu thuật toán quyết định không commit.

We discussed node-local implementations of these processes in Chapter 5 .

Ta đã bàn về các cách triển khai cục bộ trên nút của các tiến trình này ở Chương 5.

Many distributed systems use atomic commitment algorithms—for example, MySQL (for distributed transactions) and Kafka (for producer and consumer interaction [MEHTA17]).

Nhiều hệ phân tán dùng thuật toán cam kết nguyên tử — ví dụ MySQL (cho giao dịch phân tán) và Kafka (cho tương tác producer và consumer [MEHTA17]).

In databases, distributed transactions are executed by the component commonly known as a transaction manager . The transaction manager is a subsystem responsible for scheduling, coordinating, executing, and tracking transactions. In a distributed environment, the transaction manager is responsible for ensuring that node-local visibility guarantees are consistent with the visibility prescribed by distributed atomic operations. In other words, transactions commit in all partitions, and for all replicas.

Trong cơ sở dữ liệu, giao dịch phân tán được thực thi bởi thành phần thường gọi là bộ quản lý giao dịch (transaction manager). Bộ quản lý giao dịch là một phân hệ chịu trách nhiệm lập lịch, phối hợp, thực thi, và theo dõi các giao dịch. Trong môi trường phân tán, bộ quản lý giao dịch chịu trách nhiệm bảo đảm rằng các bảo đảm về tính hiển thị cục bộ trên nút nhất quán với tính hiển thị do các thao tác nguyên tử phân tán quy định. Nói cách khác, các giao dịch commit ở tất cả các phân vùng, và cho tất cả các bản sao.

We will discuss two atomic commitment algorithms: **two-phase commit**, which solves a commitment problem, but doesn't allow for failures of the **coordinator** process; and three-phase commit [SKEEN83] , which solves a nonblocking atomic commitment problem, 1 and allows participants proceed even in case of coordinator failures [BABAUGLU93] .

Ta sẽ bàn về hai thuật toán cam kết nguyên tử: commit hai pha (2PC), giải quyết bài toán cam kết nhưng không cho phép xảy ra lỗi của tiến trình điều phối; và commit ba pha (3PC) [SKEEN83], giải quyết bài toán cam kết nguyên tử không chặn, và cho phép các bên tham gia tiếp tục ngay cả khi bên điều phối bị lỗi [BABAUGLU93].

Two-Phase Commit — Commit hai pha

Let's start with the most straightforward protocol for a distributed commit that allows multipartition atomic updates. (For more information on partitioning, you can refer to “Database Partitioning” on page 270 .) Two-phase commit (2PC) is usually discussed in the context of database transactions. 2PC executes in two phases. During the first phase, the decided value is distributed, and votes are

collected. During the second phase, nodes just flip the switch, making the results of the first phase visible.

Hãy bắt đầu với giao thức đơn giản nhất cho một commit phân tán cho phép cập nhật nguyên tử đa-phân-vùng. (Để biết thêm thông tin về phân vùng, bạn có thể tham khảo “Database Partitioning” ở trang 270.) Commit hai pha (2PC) thường được bàn trong bối cảnh giao dịch cơ sở dữ liệu. 2PC thực thi qua hai pha. Trong pha thứ nhất, giá trị đã quyết định được phân phối, và các phiếu bầu được thu thập. Trong pha thứ hai, các nút chỉ việc gạt công tắc, làm cho kết quả của pha thứ nhất trở nên hiển thị.

1 The fine print says “assuming a highly reliable network.” In other words, a network that precludes partitions [ALHOUMAILY10]. Implications of this assumption are discussed in the paper’s section about algorithm description.

1 Phần in nhỏ ghi rõ “giả định một mạng có độ tin cậy cao”. Nói cách khác, một mạng loại trừ các phân vùng [ALHOUMAILY10]. Các hàm ý của giả định này được bàn trong phần mô tả thuật toán của bài báo.

2PC assumes the presence of a **leader** (or coordinator) that holds the state, collects votes, and is a primary point of reference for the agreement round. The rest of the nodes are called cohorts . Cohorts, in this case, are usually partitions that operate over disjoint datasets, against which transactions are performed. The coordinator and every cohort keep local operation logs for each executed step. Participants vote to accept or reject some value , proposed by the coordinator. Most often, this value is an identifier of the **distributed transaction** that has to be executed, but 2PC can be used in other contexts as well.

2PC giả định sự hiện diện của một leader (hay bên điều phối, coordinator) nắm giữ trạng thái, thu thập phiếu bầu, và là điểm tham chiếu chính cho vòng đồng thuận. Các nút còn lại được gọi là các bên tham gia (cohort). Trong trường hợp này, các bên tham gia thường là các phân vùng vận hành trên các tập dữ liệu rời nhau, mà trên đó giao dịch được thực hiện. Bên điều phối và mỗi bên tham gia đều giữ nhật ký thao tác cục bộ cho mỗi bước đã thực thi. Các bên tham gia bỏ phiếu chấp nhận hoặc từ chối một giá trị nào đó do bên điều phối đề xuất. Thông thường nhất, giá trị này là một định danh của giao dịch phân tán cần được thực thi, nhưng 2PC cũng có thể được dùng trong các bối cảnh khác.

The coordinator can be a node that received a request to execute the transaction, or it can be picked at random, using a leader-election algorithm, assigned manually, or even fixed throughout the lifetime of the system. The protocol does not place restrictions on the coordinator role, and the role can be transferred to another **participant** for reliability or performance.

Bên điều phối có thể là một nút đã nhận được yêu cầu thực thi giao dịch, hoặc có thể được chọn ngẫu nhiên, dùng thuật toán bầu chọn leader, gán thủ công, hoặc thậm chí cố định trong suốt vòng đời của hệ thống. Giao thức không đặt ra ràng buộc nào về vai trò bên điều phối, và vai trò này có thể được chuyển sang một bên tham gia khác vì độ tin cậy hoặc hiệu năng.

As the name suggests, a two-phase commit is executed in two steps:

Đúng như tên gọi, commit hai pha được thực thi qua hai bước:

Prepare The coordinator notifies cohorts about the new transaction by sending a Propose message. Cohorts make a decision on whether or not they can commit the part of the transaction that applies to them. If a cohort decides that it can commit, it notifies the coordinator about the positive vote. Otherwise, it responds to the coordinator, asking it to abort the transaction. All decisions taken by cohorts are persisted in the coordinator log, and each cohort keeps a copy of its decision locally.

Prepare Bên điều phối thông báo cho các bên tham gia về giao dịch mới bằng cách gửi một thông điệp Propose. Các bên tham gia ra quyết định về việc liệu họ có thể commit phần giao dịch áp dụng cho mình hay không. Nếu một bên tham gia quyết định rằng nó có thể commit, nó thông báo cho bên điều phối về phiếu thuận. Ngược lại, nó phản hồi bên điều phối, yêu cầu hủy bỏ giao dịch. Mọi quyết định của các bên tham gia đều được lưu bên trong nhật ký của bên điều phối, và mỗi bên tham gia giữ một bản sao quyết định của mình cục bộ.

Commit/abort Operations within a transaction can change state across different partitions (each represented by a cohort). If even one of the cohorts votes to abort the transaction, the coordinator sends the Abort message to all of them. Only if all cohorts have voted positively does the coordinator send them a final Commit message.

Commit/abort Các thao tác trong một giao dịch có thể thay đổi trạng thái trên nhiều phân vùng khác nhau (mỗi phân vùng đại diện bởi một bên tham gia). Nếu dù chỉ một bên tham gia bỏ phiếu hủy bỏ giao dịch, bên điều phối gửi thông điệp Abort tới tất cả. Chỉ khi tất cả các bên tham gia đã bỏ phiếu thuận thì bên điều phối mới gửi cho họ thông điệp Commit cuối cùng.

This process is shown in Figure 13-1 .

Quá trình này được minh họa ở Hình 13-1.

During the prepare phase, the coordinator distributes the proposed value and collects votes from the participants on whether or not this proposed value should be committed. Cohorts may choose to reject the coordinator's proposal if, for example, another conflicting transaction has already committed a different value.

Trong pha prepare, bên điều phối phân phối giá trị được đề xuất và thu thập phiếu bầu từ các bên tham gia về việc liệu giá trị đề xuất này có nên được commit hay không. Các bên tham gia có thể chọn từ chối đề xuất của bên điều phối nếu, chẳng hạn, một giao dịch xung đột khác đã commit một giá trị khác.

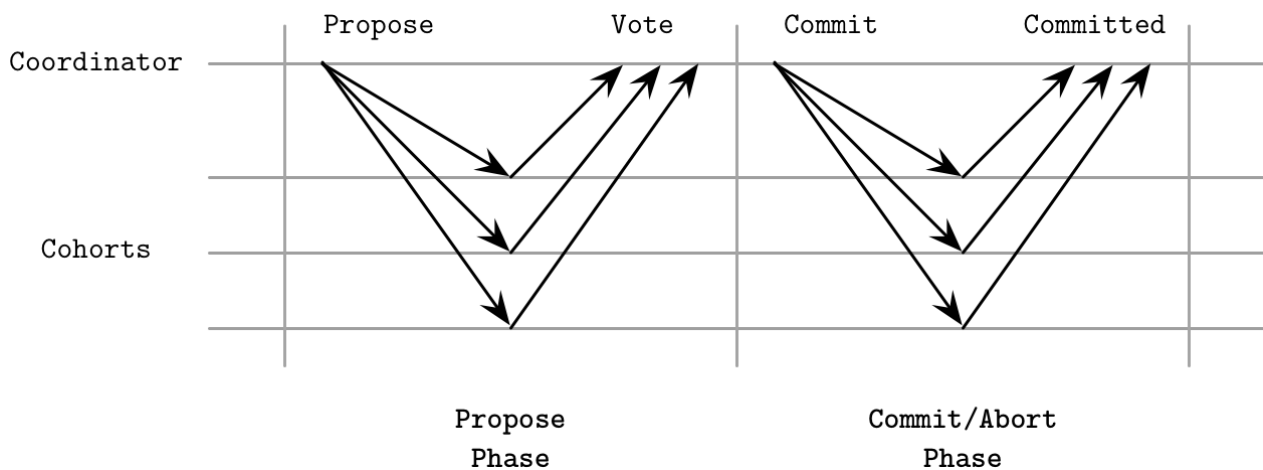


Figure 13-1. Two-phase commit protocol. During the first phase, cohorts are notified about the new transaction. During the second phase, the transaction is committed or aborted.

Hình 13-1. Giao thức commit hai pha. Trong pha thứ nhất, các bên tham gia được thông báo về giao dịch mới. Trong pha thứ hai, giao dịch được commit hoặc hủy bỏ.

After the coordinator has collected the votes, it can make a decision on whether to commit the transaction or abort it. If all cohorts have voted positively, it decides to commit and notifies them

by sending a Commit message. Otherwise, the coordinator sends an Abort message to all cohorts and the transaction gets rolled back. In other words, if one node rejects the proposal, the whole round is aborted.

Sau khi bên điều phối đã thu thập các phiếu bầu, nó có thể ra quyết định commit giao dịch hay hủy bỏ nó. Nếu tất cả các bên tham gia đã bỏ phiếu thuận, nó quyết định commit và thông báo cho họ bằng cách gửi thông điệp Commit. Ngược lại, bên điều phối gửi thông điệp Abort tới tất cả các bên tham gia và giao dịch bị rollback. Nói cách khác, nếu một nút từ chối đề xuất, cả vòng đó bị hủy bỏ.

During each step the coordinator and cohorts have to write the results of each operation to durable storage to be able to reconstruct the state and recover in case of local failures, and be able to forward and replay results for other participants.

Trong mỗi bước, bên điều phối và các bên tham gia phải ghi kết quả của từng thao tác vào bộ lưu trữ bền vững để có thể tái dựng trạng thái và phục hồi trong trường hợp lỗi cục bộ, và có thể chuyển tiếp cũng như phát lại kết quả cho các bên tham gia khác.

In the context of database systems, each 2PC round is usually responsible for a single transaction. During the prepare phase, transaction contents (operations, identifiers, and other metadata) are transferred from the coordinator to the cohorts. The transaction is executed by the cohorts locally and is left in a partially committed state (sometimes called precommitted), making it ready for the coordinator to finalize execution during the next phase by either committing or aborting it. By the time the transaction commits, its contents are already stored durably on all other nodes [BERNSTEIN09] .

Trong bối cảnh hệ cơ sở dữ liệu, mỗi vòng 2PC thường chịu trách nhiệm cho một giao dịch duy nhất. Trong pha prepare, nội dung giao dịch (các thao tác, định danh, và metadata khác) được chuyển từ bên điều phối tới các bên tham gia. Giao dịch được các bên tham gia thực thi cục bộ và được để ở trạng thái commit một phần (đôi khi gọi là precommitted), làm cho nó sẵn sàng để bên điều phối hoàn tất việc thực thi trong pha kế tiếp bằng cách commit hoặc hủy bỏ nó. Tới khi giao dịch commit, nội dung của nó đã được lưu bền trên tất cả các nút khác [BERNSTEIN09].

Cohort Failures in 2PC — Lỗi bên tham gia trong 2PC

Let's consider several failure scenarios. For example, as Figure 13-2 shows, if one of the cohorts fails during the propose phase, the coordinator cannot proceed with a commit, since it requires all votes to be positive. If one of the cohorts is unavailable, the coordinator will abort the transaction. This requirement has a negative impact on availability: failure of a single node can prevent transactions from happening. Some systems, for example, **Spanner** (see “Distributed Transactions with Spanner” on page 268), perform 2PC over **Paxos** groups rather than individual nodes to improve protocol availability.

Hãy xét vài kịch bản lỗi. Ví dụ, như Hình 13-2 cho thấy, nếu một trong các bên tham gia bị lỗi trong pha propose, bên điều phối không thể tiến hành commit, vì nó đòi hỏi tất cả các phiếu bầu phải là thuận. Nếu một trong các bên tham gia không khả dụng, bên điều phối sẽ hủy bỏ giao dịch. Yêu cầu này có tác động tiêu cực tới tính sẵn sàng: lỗi của một nút duy nhất có thể ngăn các giao dịch diễn ra. Một số hệ thống, ví dụ Spanner (xem “Distributed Transactions with Spanner” ở trang 268), thực hiện 2PC trên các nhóm Paxos thay vì trên từng nút riêng lẻ để cải thiện tính sẵn sàng của giao thức.

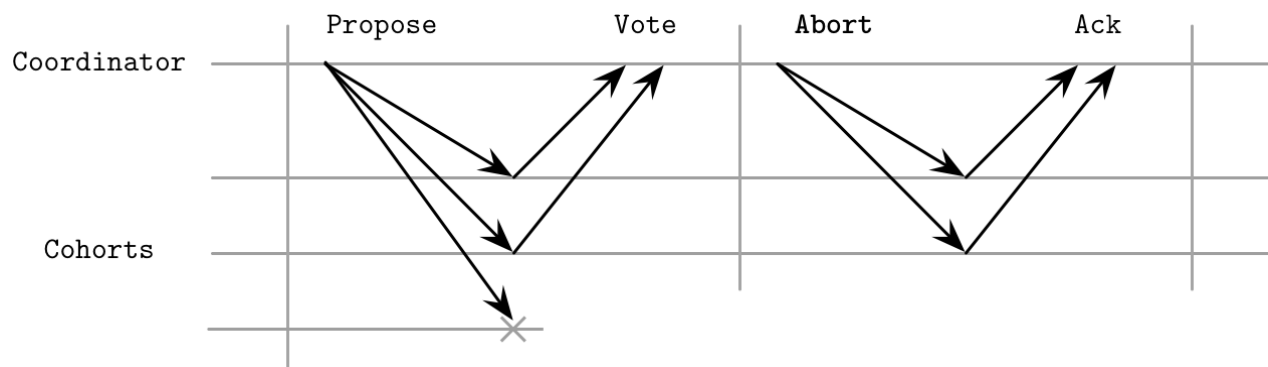


Figure 13-2. Cohort failure during the propose phase

Hình 13-2. Lỗi bên tham gia trong pha propose

The main idea behind 2PC is a promise by a cohort that, once it has positively responded to the proposal, it will not go back on its decision, so only the coordinator can abort the transaction.

Ý tưởng chính đằng sau 2PC là một lời hứa của bên tham gia rằng, một khi nó đã phản hồi thuận đối với đề xuất, nó sẽ không đảo ngược quyết định của mình, nên chỉ bên điều phối mới có thể hủy bỏ giao dịch.

If one of the cohorts has failed after accepting the proposal, it has to learn about the actual outcome of the vote before it can serve values correctly, since the coordinator might have aborted the commit due to the other cohorts' decisions. When a cohort node recovers, it has to get up to speed with a final coordinator decision. Usually, this is done by persisting the decision log on the coordinator side and replicating decision values to the failed participants. Until then, the cohort cannot serve requests because it is in an inconsistent state.

Nếu một trong các bên tham gia bị lỗi sau khi chấp nhận đề xuất, nó phải biết được kết quả thực tế của cuộc bỏ phiếu trước khi có thể phục vụ giá trị một cách đúng đắn, vì bên điều phối có thể đã hủy bỏ commit do quyết định của các bên tham gia khác. Khi một nút tham gia phục hồi, nó phải cập nhật cho kịp quyết định cuối cùng của bên điều phối. Thông thường, việc này được thực hiện bằng cách lưu bên nhật ký quyết định ở phía bên điều phối và nhân bản các giá trị quyết định tới các bên tham gia bị lỗi. Cho tới lúc đó, bên tham gia không thể phục vụ các yêu cầu vì nó đang ở trạng thái không nhất quán.

Since the protocol has multiple spots where processes are waiting for the other participants (when the coordinator collects votes, or when the cohort is waiting for the commit/abort phase), **link** failures might lead to message loss, and this wait will continue indefinitely. If the coordinator does not receive a response from the replica during the propose phase, it can trigger a timeout and abort the transaction.

Vì giao thức có nhiều điểm mà các tiến trình phải chờ các bên tham gia khác (khi bên điều phối thu thập phiếu bầu, hoặc khi bên tham gia đang chờ pha commit/abort), lỗi kênh có thể dẫn đến mất thông điệp, và sự chờ đợi này sẽ kéo dài vô hạn. Nếu bên điều phối không nhận được phản hồi từ bản sao trong pha propose, nó có thể kích hoạt một time-out và hủy bỏ giao dịch.

Coordinator Failures in 2PC — Lỗi bên điều phối trong 2PC

If one of the cohorts does not receive a commit or abort command from the coordinator during the second phase, as shown in Figure 13-3, it should attempt to find out which decision was made by the

coordinator. The coordinator might have decided upon the value but wasn't able to communicate it to the particular replica. In such cases, information about the decision can be replicated from the peers' transaction logs or from the backup coordinator. Replicating commit decisions is safe since it's always unanimous: the whole point of 2PC is to either commit or abort on all sites, and commit on one cohort implies that all other cohorts have to commit.

Nếu một trong các bên tham gia không nhận được lệnh commit hoặc abort từ bên điều phối trong pha thứ hai, như được minh họa ở Hình 13-3, nó nên cố gắng tìm hiểu xem bên điều phối đã ra quyết định nào. Bên điều phối có thể đã quyết định trên giá trị đó nhưng không thể truyền đạt được tới bản sao cụ thể đó. Trong các trường hợp như vậy, thông tin về quyết định có thể được nhân bản từ nhật ký giao dịch của các nút ngang hàng hoặc từ bên điều phối dự phòng. Việc nhân bản các quyết định commit là an toàn vì nó luôn nhất trí: toàn bộ mục đích của 2PC là commit hoặc abort trên tất cả các điểm, và commit trên một bên tham gia hàm ý rằng tất cả các bên tham gia khác đều phải commit.

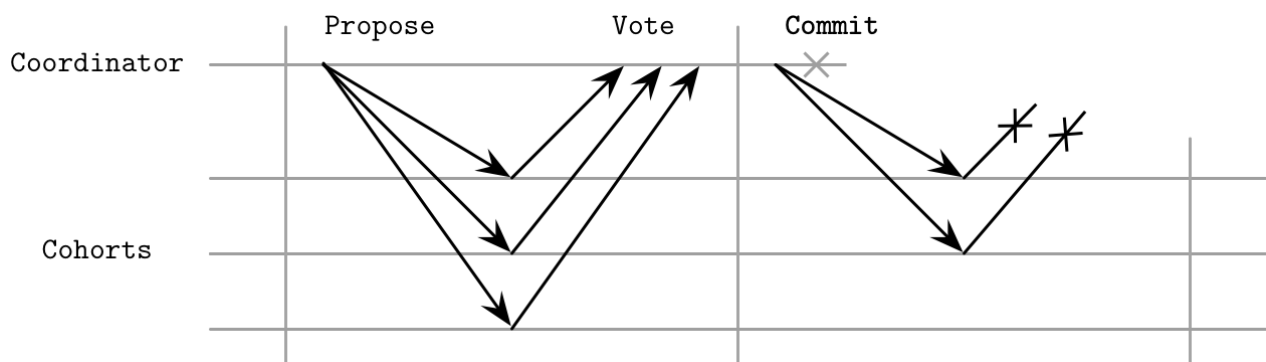


Figure 13-3. Coordinator failure after the propose phase

Hình 13-3. Lỗi bên điều phối sau pha propose

During the first phase, the coordinator collects votes and, subsequently, promises from cohorts, that they will wait for its explicit commit or abort command. If the coordinator fails after collecting the votes, but before broadcasting vote results, the cohorts end up in a state of uncertainty. This is shown in Figure 13-4. Cohorts do not know what precisely the coordinator has decided, and whether or not any of the participants (potentially also unreachable) might have been notified about the transaction results [BERNSTEIN87].

Trong pha thứ nhất, bên điều phối thu thập phiếu bầu và, sau đó, lời hứa từ các bên tham gia rằng họ sẽ chờ lệnh commit hoặc abort tường minh của nó. Nếu bên điều phối bị lỗi sau khi thu thập phiếu bầu, nhưng trước khi quảng bá kết quả bỏ phiếu, các bên tham gia rơi vào trạng thái bất định. Điều này được minh họa ở Hình 13-4. Các bên tham gia không biết chính xác bên điều phối đã quyết định gì, và liệu có bên tham gia nào (cũng có thể không tiếp cận được) đã được thông báo về kết quả giao dịch hay chưa [BERNSTEIN87].

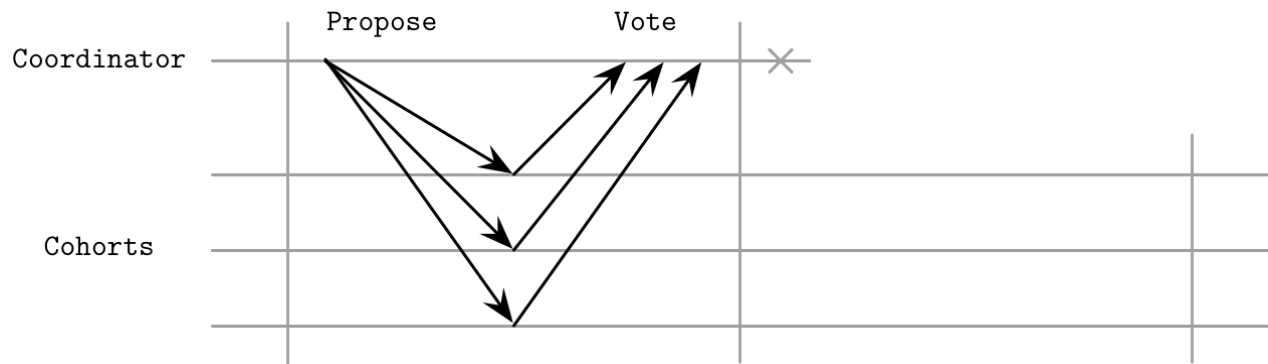


Figure 13-4. Coordinator failure before it could contact any cohorts

Hình 13-4. Lỗi bên điều phối trước khi nó kịp liên lạc với bất kỳ bên tham gia nào

Inability of the coordinator to proceed with a commit or abort leaves the cluster in an undecided state. This means that cohorts will not be able to learn about the final decision in case of a permanent coordinator failure. Because of this property, we say that 2PC is a blocking atomic commitment algorithm. If the coordinator never recovers, its replacement has to collect votes for a given transaction again, and proceed with a final decision.

Việc bên điều phối không thể tiến hành commit hoặc abort để cụm ở trạng thái chưa quyết định. Điều này nghĩa là các bên tham gia sẽ không thể biết được quyết định cuối cùng trong trường hợp bên điều phối lỗi vĩnh viễn. Vì tính chất này, ta nói rằng 2PC là một thuật toán cam kết nguyên tử có chặn. Nếu bên điều phối không bao giờ phục hồi, bên thay thế nó phải thu thập lại phiếu bầu cho một giao dịch nhất định, rồi tiến hành đến quyết định cuối cùng.

Many databases use 2PC: MySQL, PostgreSQL, MongoDB, 2 and others. Two-phase commit is often used to implement distributed transactions because of its simplicity (it is easy to reason about, implement, and debug) and low overhead (message com-

Nhiều cơ sở dữ liệu dùng 2PC: MySQL, PostgreSQL, MongoDB, và các hệ khác. Commit hai pha thường được dùng để triển khai giao dịch phân tán vì sự đơn giản của nó (dễ lập luận, triển khai, và gỡ lỗi) và chi phí phụ thấp (độ phức tạp về thông điệp

2 However, the documentation says that as of v3.6, 2PC provides only transaction-like semantics: <https://data.bass.dev/links/7>.

2 Tuy nhiên, tài liệu nói rằng tính tới phiên bản v3.6, 2PC chỉ cung cấp ngữ nghĩa kiểu-giao-dịch: <https://data.bass.dev/links/7>.

plexity and the number of round-trips of the protocol are low). It is important to implement proper **recovery** mechanisms and have backup coordinator nodes to reduce the chance of the failures just described.

và số vòng khứ hồi của giao thức đều thấp). Điều quan trọng là phải triển khai các cơ chế phục hồi đúng đắn và có các nút điều phối dự phòng để giảm khả năng xảy ra những lỗi vừa được mô tả.

Three-Phase Commit — Commit ba pha

To make an atomic commitment protocol robust against coordinator failures and avoid undecided states, the three-phase commit (**3PC**) protocol adds an extra step, and timeouts on both sides that can

allow cohorts to proceed with either commit or abort in the event of coordinator failure, depending on the system state. 3PC assumes a synchronous model and that communication failures are not possible [BABAOG- GLU93] .

Để làm cho giao thức cam kết nguyên tử bền vững trước lỗi bên điều phối và tránh các trạng thái chưa quyết định, giao thức commit ba pha (3PC) thêm một bước phụ, cùng các time-out ở cả hai phía, cho phép các bên tham gia tiến hành commit hoặc abort trong trường hợp bên điều phối lỗi, tùy theo trạng thái hệ thống. 3PC giả định một mô hình đồng bộ và rằng lỗi truyền thông là không thể xảy ra [BABAOG- GLU93].

3PC adds a prepare phase before the commit/abort step, which communicates cohort states collected by the coordinator during the propose phase, allowing the protocol to carry on even if the coordinator fails. All other properties of 3PC and a requirement to have a coordinator for the round are similar to its two-phase sibling. Another useful addition to 3PC is timeouts on the cohort side. Depending on which step the process is currently executing, either a commit or abort decision is forced on timeout.

3PC thêm một pha prepare trước bước commit/abort, pha này truyền đạt các trạng thái của bên tham gia mà bên điều phối thu thập được trong pha propose, cho phép giao thức tiếp tục ngay cả khi bên điều phối lỗi. Tất cả các tính chất khác của 3PC và yêu cầu phải có một bên điều phối cho vòng đó đều tương tự với người anh em hai pha của nó. Một bổ sung hữu ích khác của 3PC là các time-out ở phía bên tham gia. Tùy thuộc vào bước nào tiến trình đang thực thi, quyết định commit hoặc abort được áp đặt khi hết thời gian chờ.

As Figure 13-5 shows, the three-phase commit round consists of three steps:

Như Hình 13-5 cho thấy, vòng commit ba pha gồm ba bước:

Propose The coordinator sends out a proposed value and collects the votes.

Propose Bên điều phối gửi đi một giá trị được đề xuất và thu thập các phiếu bầu.

Prepare The coordinator notifies cohorts about the vote results. If the vote has passed and all cohorts have decided to commit, the coordinator sends a Prepare message, instructing them to prepare to commit. Otherwise, an Abort message is sent and the round completes.

Prepare Bên điều phối thông báo cho các bên tham gia về kết quả bỏ phiếu. Nếu cuộc bỏ phiếu đã thông qua và tất cả các bên tham gia đã quyết định commit, bên điều phối gửi một thông điệp Prepare, chỉ thị họ chuẩn bị commit. Ngược lại, một thông điệp Abort được gửi và vòng đó kết thúc.

Commit Cohorts are notified by the coordinator to commit the transaction.

Commit Các bên tham gia được bên điều phối thông báo để commit giao dịch.

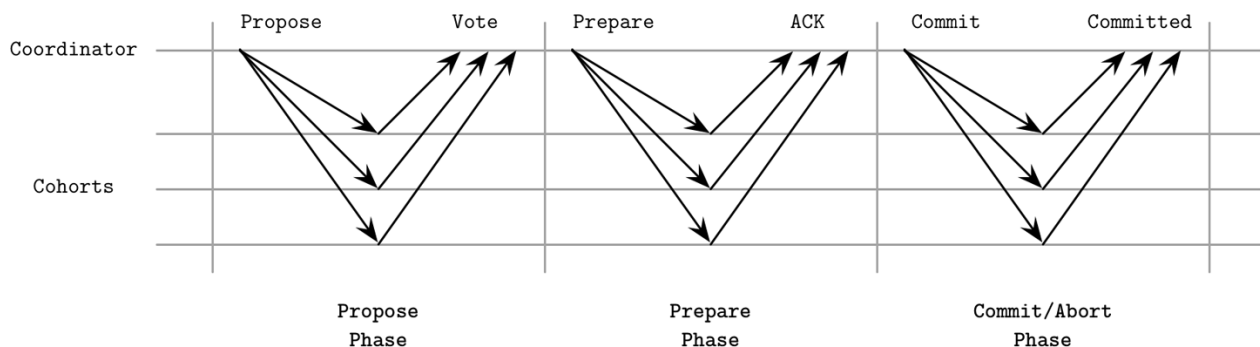


Figure 13-5. Three-phase commit During the propose step, similar to 2PC, the coordinator distributes the proposed value and collects votes from cohorts, as shown in Figure 13-5 . If the coordinator crashes during this phase and the operation times out, or if one of the cohorts votes negatively, the transaction will be aborted.

Hình 13-5. Commit ba pha. Trong bước propose, tương tự 2PC, bên điều phối phân phối giá trị được đề xuất và thu thập phiếu bầu từ các bên tham gia, như được minh họa ở Hình 13-5. Nếu bên điều phối sập trong pha này và thao tác hết thời gian chờ, hoặc nếu một trong các bên tham gia bỏ phiếu chống, giao dịch sẽ bị hủy bỏ.

After collecting the votes, the coordinator makes a decision. If the coordinator decides to proceed with a transaction, it issues a Prepare command. It may happen that the coordinator cannot distribute prepare messages to all cohorts or it fails to receive their acknowledgments. In this case, cohorts may abort the transaction after timeout, since the algorithm hasn't moved all the way to the prepared state.

Sau khi thu thập phiếu bầu, bên điều phối ra quyết định. Nếu bên điều phối quyết định tiến hành một giao dịch, nó phát ra một lệnh Prepare. Có thể xảy ra việc bên điều phối không phân phối được thông điệp prepare tới tất cả các bên tham gia hoặc không nhận được xác nhận của họ. Trong trường hợp này, các bên tham gia có thể hủy bỏ giao dịch sau khi hết thời gian chờ, vì thuật toán chưa đi hết tới trạng thái prepared.

As soon as all the cohorts successfully move into the prepared state and the coordinator has received their prepare acknowledgments, the transaction will be committed if either side fails. This can be done since all participants at this stage have the same view of the state.

Ngay khi tất cả các bên tham gia chuyển thành công vào trạng thái prepared và bên điều phối đã nhận được xác nhận prepare của họ, giao dịch sẽ được commit nếu một trong hai bên bị lỗi. Điều này có thể làm được vì tất cả các bên tham gia ở giai đoạn này đều có cùng góc nhìn về trạng thái.

During commit , the coordinator communicates the results of the prepare phase to all the participants, resetting their timeout counters and effectively finishing the transaction.

Trong bước commit, bên điều phối truyền đạt kết quả của pha prepare tới tất cả các bên tham gia, đặt lại bộ đếm thời gian chờ của họ và thực sự kết thúc giao dịch.

Coordinator Failures in 3PC — Lỗi bên điều phối trong 3PC

All state transitions are coordinated, and cohorts can't move on to the next phase until everyone is done with the previous one: the coordinator has to wait for the replicas to continue. Cohorts can eventually abort the transaction if they do not hear from the coordinator before the timeout, if they didn't move past the prepare phase.

Mọi chuyển trạng thái đều được phối hợp, và các bên tham gia không thể chuyển sang pha kế tiếp cho tới khi mọi người đã hoàn tất pha trước: bên điều phối phải chờ các bản sao mới tiếp tục được. Các bên tham gia rốt cuộc có thể hủy bỏ giao dịch nếu họ không nhận được tin từ bên điều phối trước thời gian chờ, miễn là họ chưa vượt qua pha prepare.

As we discussed previously, 2PC cannot recover from coordinator failures, and cohorts may get stuck in a nondeterministic state until the coordinator comes back. 3PC avoids blocking the processes in this case and allows cohorts to proceed with a deterministic decision.

Như ta đã bàn trước đó, 2PC không thể phục hồi từ lỗi bên điều phối, và các bên tham gia có thể bị kẹt ở một trạng thái phi tất định cho tới khi bên điều phối quay lại. 3PC tránh

được việc chặn các tiến trình trong trường hợp này và cho phép các bên tham gia tiến hành một quyết định tất định.

The worst-case scenario for the 3PC is a **network partition**, shown in Figure 13-6 . Some nodes successfully move to the prepared state, and now can proceed with commit after the timeout. Some can't communicate with the coordinator, and will abort after the timeout. This results in a **split brain**: some nodes proceed with a commit and some abort, all according to the protocol, leaving participants in an inconsistent and contradictory state.

Kịch bản tệ nhất cho 3PC là phân vùng mạng, được minh họa ở Hình 13-6. Một số nút chuyển thành công vào trạng thái prepared, và giờ có thể tiến hành commit sau thời gian chờ. Một số khác không thể liên lạc với bên điều phối, và sẽ abort sau thời gian chờ. Điều này dẫn đến split brain (não phân liệt): một số nút tiến hành commit và một số abort, tất cả đều theo đúng giao thức, để các bên tham gia ở trạng thái không nhất quán và mâu thuẫn.

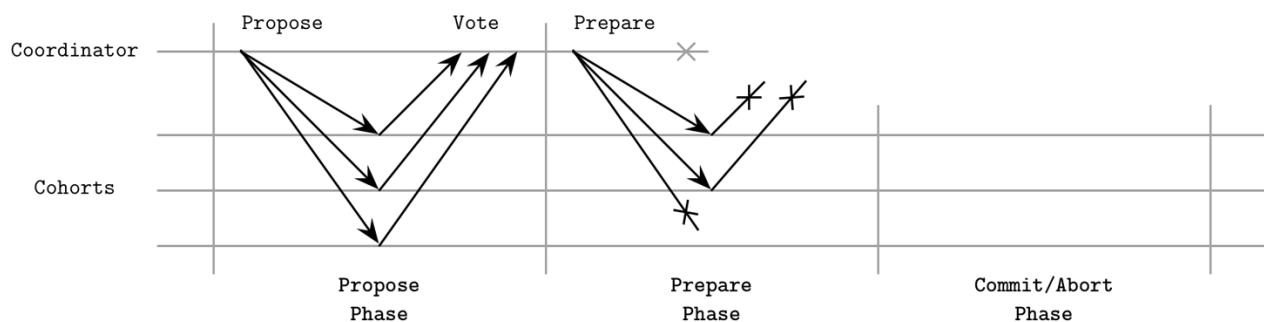


Figure 13-6. Coordinator failure during the second phase

Hình 13-6. Lỗi bên điều phối trong pha thứ hai

While in theory 3PC does, to a degree, solve the problem with 2PC blocking, it has a larger message overhead, introduces potential contradictions, and does not work well in the presence of network partitions. This might be the primary reason 3PC is not widely used in practice.

Mặc dù về lý thuyết 3PC có, ở một mức độ nào đó, giải quyết được vấn đề chặn của 2PC, nó lại có chi phí phụ về thông điệp lớn hơn, gây ra các mâu thuẫn tiềm tàng, và không hoạt động tốt khi có phân vùng mạng. Đây có thể là lý do chính khiến 3PC không được dùng rộng rãi trong thực tế.

Distributed Transactions with Calvin — Giao dịch phân tán với Calvin

We've already touched on the subject of synchronization costs and several ways around it. But there are other ways to reduce contention and the total amount of time during which transactions hold locks. One of the ways to do this is to let replicas agree on the execution order and transaction boundaries before acquiring locks and proceeding with execution. If we can achieve this, node failures do not cause transaction aborts, since nodes can recover state from other participants that execute the same transaction in parallel.

Ta đã đề cập đến chủ đề chi phí đồng bộ hóa và vài cách để né tránh nó. Nhưng còn những cách khác để giảm tranh chấp và tổng thời gian mà trong đó các giao dịch giữ khóa. Một trong các cách làm điều này là để các bản sao thống nhất về thứ tự thực thi và ranh giới giao dịch trước khi giành khóa và tiến hành thực thi. Nếu ta đạt được điều này, lỗi nút

không gây hủy bỏ giao dịch, vì các nút có thể phục hồi trạng thái từ các bên tham gia khác đang thực thi cùng giao dịch một cách song song.

Traditional database systems execute transactions using two-phase locking or **optimistic concurrency control** and have no deterministic transaction order. This means that nodes have to be coordinated to preserve order. Deterministic transaction order removes coordination overhead during the execution phase and, since all replicas get the same inputs, they also produce equivalent outputs. This approach is commonly known as **Calvin**, a fast distributed transaction protocol [THOMSON12] . One of the prominent examples implementing distributed transactions using Calvin is FaunaDB .

Các hệ cơ sở dữ liệu truyền thống thực thi giao dịch dùng khóa hai pha hoặc điều khiển tương tranh lạc quan và không có thứ tự giao dịch tất định. Điều này nghĩa là các nút phải được phối hợp để giữ gìn thứ tự. Thứ tự giao dịch tất định loại bỏ chi phí phối hợp trong pha thực thi và, vì tất cả các bản sao nhận cùng đầu vào, chúng cũng tạo ra đầu ra tương đương. Phương pháp này thường được biết đến với tên Calvin, một giao thức giao dịch phân tán nhanh [THOMSON12]. Một trong các ví dụ nổi bật triển khai giao dịch phân tán dùng Calvin là FaunaDB.

To achieve deterministic order, Calvin uses a sequencer : an entry point for all transactions. The sequencer determines the order in which transactions are executed, and establishes a global transaction input sequence. To minimize contention and batch decisions, the timeline is split into epochs . The sequencer collects transactions and groups them into short time windows (the original paper mentions 10-millisecond batches), which also become replication units, so transactions do not have to be communicated separately.

Để đạt thứ tự tất định, Calvin dùng một bộ phân chuỗi (sequencer): một điểm vào cho tất cả các giao dịch. Bộ phân chuỗi xác định thứ tự thực thi các giao dịch, và thiết lập một chuỗi đầu vào giao dịch toàn cục. Để giảm thiểu tranh chấp và gom nhóm các quyết định, dòng thời gian được chia thành các kỷ nguyên (epoch). Bộ phân chuỗi thu thập các giao dịch và gom chúng vào các cửa sổ thời gian ngắn (bài báo gốc đề cập đến các lô 10 mili-giây), những cửa sổ này cũng trở thành đơn vị nhân bản, nên các giao dịch không phải được truyền đạt riêng lẻ.

As soon as a transaction batch is successfully replicated, sequencer forwards it to the scheduler , which orchestrates transaction execution. The scheduler uses a deterministic scheduling protocol that executes parts of transaction in parallel, while preserving the serial execution order specified by the sequencer. Since applying transaction to a specific state is guaranteed to produce only changes specified by the transaction and transaction order is predetermined, replicas do not have to further communicate with the sequencer.

Ngay khi một lô giao dịch được nhân bản thành công, bộ phân chuỗi chuyển tiếp nó tới bộ lập lịch (scheduler), bộ này điều phối việc thực thi giao dịch. Bộ lập lịch dùng một giao thức lập lịch tất định, thực thi các phần của giao dịch một cách song song, đồng thời giữ gìn thứ tự thực thi tuần tự do bộ phân chuỗi quy định. Vì việc áp dụng giao dịch lên một trạng thái cụ thể được bảo đảm chỉ tạo ra các thay đổi do giao dịch quy định và thứ tự giao dịch là định trước, các bản sao không phải tiếp tục liên lạc với bộ phân chuỗi nữa.

Each transaction in Calvin has a read set (its dependencies, which is a collection of data records from the current database state required to execute it) and a write set (results of the transaction execution; in other words, its side effects). Calvin does not natively support transactions that rely on additional reads that would determine read and write sets.

Mỗi giao dịch trong Calvin có một tập đọc (read set, là các phụ thuộc của nó, vốn là tập

hợp các bản ghi dữ liệu từ trạng thái cơ sở dữ liệu hiện tại cần để thực thi nó) và một tập ghi (write set, là kết quả của việc thực thi giao dịch; nói cách khác, là các tác dụng phụ của nó). Calvin không hỗ trợ một cách bản địa các giao dịch dựa vào các phép đọc bổ sung để xác định tập đọc và tập ghi.

A worker thread, managed by the scheduler, proceeds with execution in four steps:

Một luồng worker, do bộ lập lịch quản lý, tiến hành thực thi qua bốn bước:

1. It analyzes the transaction's read and write sets, determines node-local data records from the read set, and creates the list of active participants (i.e., ones that hold the elements of the write set, and will perform modifications on the data).
 2. It collects the local data required to execute the transaction, in other words, the read set records that happen to reside on that node. The collected data records are forwarded to the corresponding active participants.
 3. If this worker thread is executing on an active participant node, it receives data records forwarded from the other participants, as a counterpart of the operations executed during step 2.
 4. Finally, it executes a batch of transactions, persisting results into local storage. It does not have to forward execution results to the other nodes, as they receive the same inputs for transactions and execute and persist results locally themselves.
1. Nó phân tích tập đọc và tập ghi của giao dịch, xác định các bản ghi dữ liệu cục bộ trên nút từ tập đọc, và tạo danh sách các bên tham gia hoạt động (tức là những bên giữ các phần tử của tập ghi, và sẽ thực hiện sửa đổi trên dữ liệu).
 2. Nó thu thập dữ liệu cục bộ cần để thực thi giao dịch, nói cách khác, là các bản ghi của tập đọc tình cờ nằm trên nút đó. Các bản ghi dữ liệu đã thu thập được chuyển tiếp tới các bên tham gia hoạt động tương ứng.
 3. Nếu luồng worker này đang thực thi trên một nút tham gia hoạt động, nó nhận các bản ghi dữ liệu được chuyển tiếp từ các bên tham gia khác, như là phần đối ứng của các thao tác được thực thi ở bước 2.
 4. Cuối cùng, nó thực thi một lô giao dịch, lưu bền kết quả vào bộ lưu trữ cục bộ. Nó không phải chuyển tiếp kết quả thực thi tới các nút khác, vì chúng nhận cùng đầu vào cho các giao dịch và tự thực thi cũng như lưu bền kết quả một cách cục bộ.

A typical Calvin implementation colocates sequencer, scheduler, worker, and storage subsystems, as Figure 13-7 shows. To make sure that sequencers reach **consensus** on exactly which transactions make it into the current epoch/batch, Calvin uses the Paxos consensus algorithm (see “Paxos” on page 285) or **asynchronous** replication, in which a dedicated replica serves as a leader. While using a leader can improve latency, it comes with a higher cost of recovery as nodes have to reproduce the state of the failed leader in order to proceed.

Một triển khai Calvin điển hình đặt chung các phân hệ bộ phân chuỗi, bộ lập lịch, worker, và lưu trữ, như Hình 13-7 cho thấy. Để chắc chắn rằng các bộ phân chuỗi đạt được đồng thuận về chính xác những giao dịch nào lọt vào kỷ nguyên/lô hiện tại, Calvin dùng thuật toán đồng thuận Paxos (xem “Paxos” ở trang 285) hoặc nhân bản bất đồng bộ, trong đó một bản sao chuyên trách đóng vai trò leader. Mặc dù dùng một leader có thể cải thiện độ trễ, nó đi kèm chi phí phục hồi cao hơn vì các nút phải tái tạo trạng thái của leader bị lỗi để có thể tiếp tục.

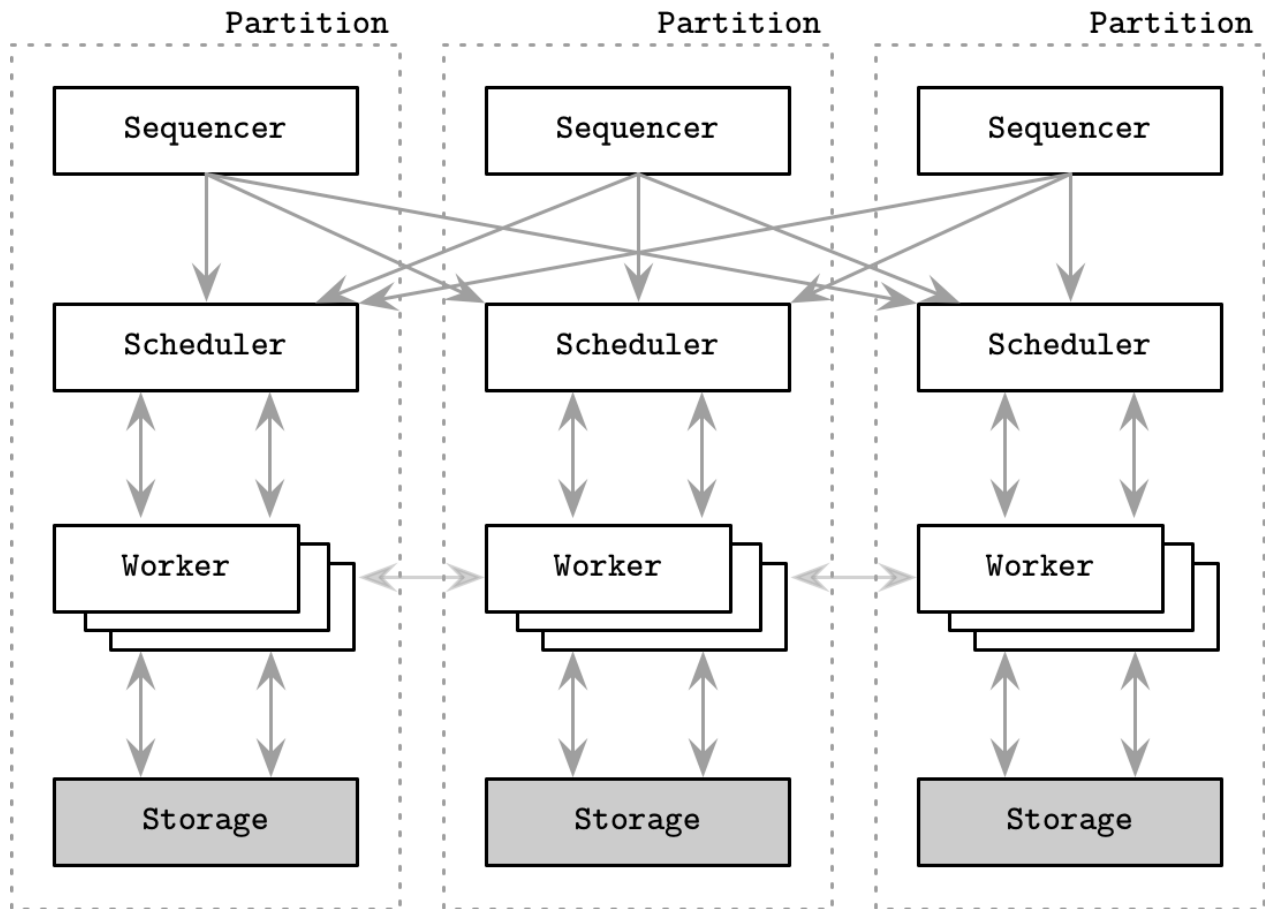


Figure 13-7. Calvin architecture

Hình 13-7. Kiến trúc Calvin

Distributed Transactions with Spanner — Giao dịch phân tán với Spanner

Calvin is often contrasted with another approach for distributed transaction management called Spanner [CORBETT12]. Its implementations (or derivatives) include several open source databases, most prominently CockroachDB and YugaByte DB. While Calvin establishes the global transaction execution order by reaching consensus on sequencers, Spanner uses two-phase commit over consensus groups per partition (in other words, per shard). Spanner has a rather complex setup, and we only cover high-level details in the scope of this book.

Calvin thường được đối chiếu với một phương pháp khác để quản lý giao dịch phân tán gọi là Spanner [CORBETT12]. Các triển khai (hoặc dẫn xuất) của nó bao gồm vài cơ sở dữ liệu mã nguồn mở, nổi bật nhất là CockroachDB và YugaByte DB. Trong khi Calvin thiết lập thứ tự thực thi giao dịch toàn cục bằng cách đạt đồng thuận trên các bộ phân chuỗi, thì Spanner dùng commit hai pha trên các nhóm đồng thuận theo từng phân vùng (nói cách khác, theo từng shard). Spanner có một thiết lập khá phức tạp, và ta chỉ đề cập đến các chi tiết cao cấp trong phạm vi cuốn sách này.

To achieve consistency and impose transaction order, Spanner uses TrueTime : a high-precision wall-clock API that also exposes an uncertainty bound, allowing local operations to introduce artificial

slowdowns to wait for the uncertainty bound to pass.

Để đạt được tính nhất quán và áp đặt thứ tự giao dịch, Spanner dùng TrueTime: một API đồng hồ thực (wall-clock) độ chính xác cao, đồng thời phơi bày cả biên độ bất định, cho phép các thao tác cục bộ chủ động trì hoãn một cách nhân tạo để chờ cho biên độ bất định này trôi qua.

Spanner offers three main operation types: read-write transactions , read-only transactions , and snapshot reads . Read-write transactions require locks, pessimistic concurrency control, and presence of the leader replica. Read-only transactions are lock-free and can be executed at any replica. A leader is required only for reads at the latest timestamp, which takes the latest committed value from the Paxos group. Reads at the specific timestamp are consistent, since values are versioned and snapshot contents can't be changed once written. Each data record has a timestamp assigned, which holds a value of the transaction commit time. This also implies that multiple timestamped versions of the record can be stored.

Spanner cung cấp ba loại thao tác chính: giao dịch đọc-ghi (read-write), giao dịch chỉ-đọc (read-only), và phép đọc snapshot (snapshot read). Giao dịch đọc-ghi đòi hỏi khóa, điều khiển tương tranh bi quan, và sự hiện diện của bản sao leader. Giao dịch chỉ-đọc không cần khóa và có thể được thực thi tại bất kỳ bản sao nào. Một leader chỉ cần cho các phép đọc tại dấu thời gian mới nhất, vốn lấy giá trị đã commit mới nhất từ nhóm Paxos. Các phép đọc tại dấu thời gian cụ thể là nhất quán, vì các giá trị được phiên-bản-hóa và nội dung snapshot không thể bị thay đổi một khi đã ghi. Mỗi bản ghi dữ liệu được gán một dấu thời gian, dấu này giữ giá trị thời điểm commit của giao dịch. Điều này cũng hàm ý rằng nhiều phiên bản được gán dấu thời gian của bản ghi có thể được lưu trữ.

Figure 13-8 shows the Spanner architecture. Each spanserver (replica, a server instance that serves data to clients) holds several tablets , with Paxos (see “Paxos” on page 285) state machines attached to them. Replicas are grouped into replica sets called Paxos groups, a unit of data placement and replication. Each Paxos group has a long-lived leader (see “**Multi-Paxos**” on page 291). Leaders communicate with each other during multishard transactions.

Hình 13-8 cho thấy kiến trúc Spanner. Mỗi spanserver (bản sao, một thực thể máy chủ phục vụ dữ liệu cho client) giữ vài tablet, với các máy trạng thái Paxos (xem “Paxos” ở trang 285) gắn vào chúng. Các bản sao được gom thành các tập bản sao gọi là nhóm Paxos, một đơn vị bố trí và nhân bản dữ liệu. Mỗi nhóm Paxos có một leader sống lâu (xem “Multi-Paxos” ở trang 291). Các leader liên lạc với nhau trong các giao dịch đa-shard.

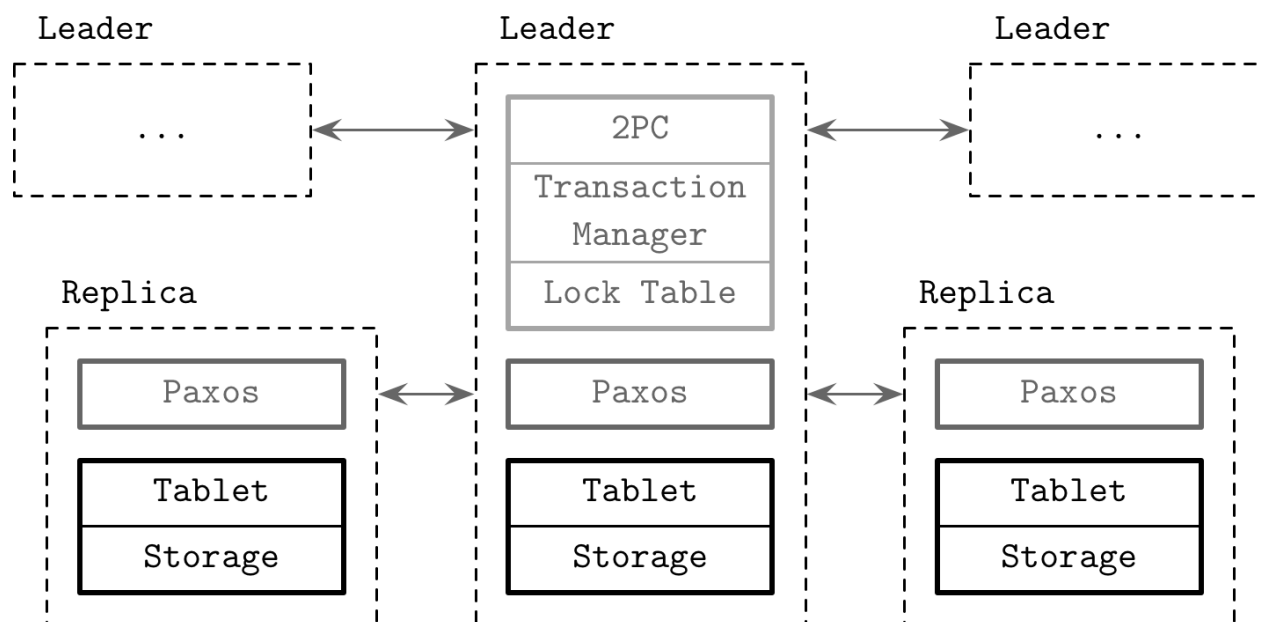


Figure 13-8. Spanner architecture

Hình 13-8. Kiến trúc Spanner

Every write has to go through the Paxos group leader, while reads can be served directly from the tablet on up-to-date replicas. The leader holds a lock table that is used to implement concurrency control using the two-phase locking (see “Lock- Based Concurrency Control” on page 100) mechanism and a transaction manager that is responsible for multishard distributed transactions. Operations that require synchronization (such as writes and reads within a transaction) have to acquire the locks from the lock table, while other operations (snapshot reads) can access the data directly.

Mọi lần ghi đều phải đi qua leader của nhóm Paxos, trong khi các phép đọc có thể được phục vụ trực tiếp từ tablet trên các bản sao đã cập nhật. Leader giữ một bảng khóa được dùng để triển khai điều khiển tương tranh bằng cơ chế khóa hai pha (xem “Lock-Based Concurrency Control” ở trang 100) và một bộ quản lý giao dịch chịu trách nhiệm cho các giao dịch phân tán đa-shard. Các thao tác đòi hỏi đồng bộ (như ghi và đọc trong một giao dịch) phải giành khóa từ bảng khóa, trong khi các thao tác khác (phép đọc snapshot) có thể truy cập dữ liệu trực tiếp.

For multishard transactions, group leaders have to coordinate and perform a two-phase commit to ensure consistency, and use two-phase locking to ensure **isolation**. Since the 2PC algorithm requires the presence of all participants for a successful commit, it hurts availability. Spanner solves this by using Paxos groups rather than individual nodes as cohorts. This means that 2PC can continue operating even if some of the members of the group are down. Within the Paxos group, 2PC contacts only the node that serves as a leader.

Đối với các giao dịch đa-shard, các leader của nhóm phải phối hợp và thực hiện commit hai pha để bảo đảm tính nhất quán, và dùng khóa hai pha để bảo đảm tính cô lập. Vì thuật toán 2PC đòi hỏi sự hiện diện của tất cả các bên tham gia để commit thành công, nó làm tổn hại tính sẵn sàng. Spanner giải quyết điều này bằng cách dùng các nhóm Paxos thay vì các nút riêng lẻ làm các bên tham gia. Điều này nghĩa là 2PC có thể tiếp tục vận hành ngay cả khi một số thành viên của nhóm bị sập. Trong nhóm Paxos, 2PC chỉ liên lạc với nút đóng vai trò leader.

Paxos groups are used to consistently replicate transaction manager states across multiple nodes. The Paxos leader first acquires write locks, and chooses a write timestamp that is guaranteed to be larger than any previous transactions' timestamp, and records a 2PC prepare entry through Paxos. The transaction coordinator collects timestamps and generates a commit timestamp that is greater than any of the prepare timestamps, and logs a commit entry through Paxos. It then waits until after the timestamp it has chosen for commit, since it has to guarantee that clients will only see transaction results whose timestamps are in the past. After that, it sends this timestamp to the client and leaders, which log the commit record with the new timestamp in their local Paxos group and are now free to release the locks.

Các nhóm Paxos được dùng để nhân bản một cách nhất quán trạng thái của bộ quản lý giao dịch trên nhiều nút. Leader Paxos trước hết giành các khóa ghi, và chọn một dấu thời gian ghi được bảo đảm lớn hơn dấu thời gian của bất kỳ giao dịch trước đó nào, rồi ghi một mục prepare 2PC thông qua Paxos. Bên điều phối giao dịch thu thập các dấu thời gian và sinh ra một dấu thời gian commit lớn hơn bất kỳ dấu thời gian prepare nào, rồi ghi một mục commit thông qua Paxos. Sau đó nó chờ cho tới sau dấu thời gian mà nó đã chọn cho commit, vì nó phải bảo đảm rằng các client sẽ chỉ thấy các kết quả giao dịch có dấu thời gian nằm trong quá khứ. Sau đó, nó gửi dấu thời gian này tới client và các leader; các leader này ghi bản ghi commit với dấu thời gian mới trong nhóm Paxos cục bộ của mình và giữ được tự do nhả khóa.

Single-shard transactions do not have to consult the transaction manager (and, subsequently, do not have to perform a cross-partition two-phase commit), since consulting a Paxos group and a lock table is enough to guarantee transaction order and consistency within the shard.

Các giao dịch đơn-shard không phải tham vấn bộ quản lý giao dịch (và do đó, không phải thực hiện commit hai pha xuyên-phân-vùng), vì việc tham vấn một nhóm Paxos và một bảng khóa là đủ để bảo đảm thứ tự giao dịch và tính nhất quán trong shard.

Spanner read-write transactions offer a serialization order called external consistency : transaction timestamps reflect serialization order, even in cases of distributed transactions. External consistency has real-time properties equivalent to **linearizability**: if transaction T commits before T' starts, T's timestamp is smaller than the timestamp

Các giao dịch đọc-ghi của Spanner cung cấp một thứ tự tuần tự hóa gọi là tính nhất quán bên ngoài (external consistency): dấu thời gian giao dịch phản ánh thứ tự tuần tự hóa, ngay cả trong các trường hợp giao dịch phân tán. Tính nhất quán bên ngoài có các tính chất thời gian thực tương đương với tính tuyến tính hóa: nếu giao dịch T commit trước khi T' bắt đầu, thì dấu thời gian của T nhỏ hơn dấu thời gian

of T'.

của T'.

To summarize, Spanner uses Paxos for consistent transaction log replication, two-phase commit for cross-shard transactions, and TrueTime for deterministic transaction ordering. This means that multipartition transactions have a higher cost due to an additional two-phase commit round, compared to Calvin [ABADI17]. Both approaches are important to understand since they allow us to perform transactions in partitioned distributed data stores.

Tóm lại, Spanner dùng Paxos cho việc nhân bản nhật ký giao dịch một cách nhất quán, commit hai pha cho các giao dịch xuyên-shard, và TrueTime cho việc sắp thứ tự giao dịch một cách tất định. Điều này nghĩa là các giao dịch đa-phân-vùng có chi phí cao hơn do một vòng commit hai pha bổ sung, so với Calvin [ABADI17]. Cả hai phương pháp đều quan

trọng để hiểu vì chúng cho phép ta thực hiện giao dịch trong các kho dữ liệu phân tán được phân vùng.

Database Partitioning — Phân vùng cơ sở dữ liệu

While discussing Spanner and Calvin, we've been using the **term** partitioning quite heavily. Let's now discuss it in more detail. Since storing all database records on a single node is rather unrealistic for the majority of modern applications, many databases use partitioning: a logical division of data into smaller manageable segments.

Trong khi bàn về Spanner và Calvin, ta đã dùng thuật ngữ phân vùng (partitioning) khá nhiều. Giờ hãy bàn về nó chi tiết hơn. Vì lưu toàn bộ bản ghi cơ sở dữ liệu trên một nút duy nhất là khá phi thực tế đối với đa số ứng dụng hiện đại, nhiều cơ sở dữ liệu dùng phân vùng: một sự chia tách logic dữ liệu thành các đoạn nhỏ hơn, dễ quản lý hơn.

The most straightforward way to partition data is by splitting it into ranges and allowing replica sets to manage only specific ranges (partitions). When executing queries, clients (or query coordinators) have to route requests based on the routing key to the correct replica set for both reads and writes. This partitioning scheme is typically called sharding : every replica set acts as a single source for a subset of data.

Cách phân vùng dữ liệu đơn giản nhất là chia nó thành các khoảng (range) và cho phép các tập bản sao chỉ quản lý các khoảng (phân vùng) cụ thể. Khi thực thi truy vấn, các client (hoặc các bên điều phối truy vấn) phải định tuyến các yêu cầu dựa trên khóa định tuyến tới đúng tập bản sao cho cả đọc và ghi. Sơ đồ phân vùng này thường được gọi là sharding: mỗi tập bản sao đóng vai trò là nguồn duy nhất cho một tập con của dữ liệu.

To use partitions most effectively, they have to be sized, taking the load and value distribution into consideration. This means that frequently accessed, read/write heavy ranges can be split into smaller partitions to spread the load between them. At the same time, if some value ranges are more dense than other ones, it might be a good idea to split them into smaller partitions as well. For example, if we pick zip code as a routing key, since the country population is unevenly spread, some zip code ranges can have more data (e.g., people and orders) assigned to them.

Để dùng các phân vùng hiệu quả nhất, chúng phải được định cỡ, có tính đến tải và phân bố giá trị. Điều này nghĩa là các khoảng được truy cập thường xuyên, nặng về đọc/ghi có thể được chia thành các phân vùng nhỏ hơn để dàn tải giữa chúng. Đồng thời, nếu một số khoảng giá trị dày đặc hơn các khoảng khác, có thể cũng nên chia chúng thành các phân vùng nhỏ hơn. Ví dụ, nếu ta chọn mã bưu chính (zip code) làm khóa định tuyến, vì dân số của một quốc gia phân bố không đều, một số khoảng mã bưu chính có thể có nhiều dữ liệu (ví dụ người và đơn hàng) được gán cho chúng.

When nodes are added to or removed from the cluster, the database has to re-partition the data to maintain the balance. To ensure consistent movements, we should relocate the data before we update the cluster metadata and start routing requests to the new targets. Some databases perform auto-sharding and relocate the data using placement algorithms that determine optimal partitioning. These algorithms use information about read, write loads, and amounts of data in each shard.

Khi các nút được thêm vào hoặc gỡ khỏi cụm, cơ sở dữ liệu phải tái phân vùng dữ liệu để duy trì sự cân bằng. Để bảo đảm các lần di chuyển nhất quán, ta nên dời chỗ dữ liệu trước khi cập nhật metadata của cụm và bắt đầu định tuyến các yêu cầu tới các đích mới. Một số cơ sở dữ liệu thực hiện tự-động-sharding và dời chỗ dữ liệu dùng các thuật toán bố trí xác

định sự phân vùng tối ưu. Các thuật toán này dùng thông tin về tải đọc, tải ghi, và lượng dữ liệu trong mỗi shard.

To find a target node from the routing key, some database systems compute a hash of the key, and use some form of mapping from the hash value to the node ID. One of the advantages of using the hash functions for determining replica placement is that it can help to reduce range hot-spotting, since hash values do not sort the same way as the original values. While two lexicographically close routing keys would be placed at the same replica set, using hashed values would place them on different ones.

Để tìm một nút đích từ khóa định tuyến, một số hệ cơ sở dữ liệu tính một hàm băm (hash) của khóa, và dùng một dạng ánh xạ nào đó từ giá trị băm tới ID của nút. Một trong các ưu điểm của việc dùng các hàm băm để xác định việc bố trí bản sao là nó có thể giúp giảm điểm nóng theo khoảng (range hot-spotting), vì các giá trị băm không sắp xếp giống như các giá trị gốc. Trong khi hai khóa định tuyến gần nhau về mặt từ điển sẽ được đặt tại cùng một tập bản sao, thì việc dùng các giá trị đã băm sẽ đặt chúng trên các tập bản sao khác nhau.

The most straightforward way to map hash values to node IDs is by taking a remainder of the division of the hash value by the size of the cluster (modulo). If we have N nodes in the system, the target node ID is picked by computing $\text{hash}(v) \bmod N$. The main problem with this approach is that whenever nodes are added or removed and the cluster size changes from N to N' , many values returned by $\text{hash}(v) \bmod N'$ will differ from the original ones. This means that most of the data will have to be moved.

Cách đơn giản nhất để ánh xạ các giá trị băm tới các ID nút là lấy phần dư của phép chia giá trị băm cho kích thước của cụm (lấy mô-đun). Nếu ta có N nút trong hệ thống, ID nút đích được chọn bằng cách tính $\text{hash}(v) \bmod N$. Vấn đề chính với cách tiếp cận này là bất cứ khi nào các nút được thêm hoặc gỡ và kích thước cụm thay đổi từ N sang N' , nhiều giá trị do $\text{hash}(v) \bmod N'$ trả về sẽ khác với các giá trị gốc. Điều này nghĩa là hầu hết dữ liệu sẽ phải được di chuyển.

Consistent Hashing — Băm nhất quán

In order to mitigate this problem, some databases, such as Apache Cassandra and Riak (among others), use a different partitioning scheme called consistent hashing. As previously mentioned, routing key values are hashed. Values returned by the hash function are mapped to a ring, so that after the largest possible value, it wraps around to its smallest value. Each node gets its own position on the ring and becomes responsible for the range of values, between its predecessor's and its own positions.

Để giảm nhẹ vấn đề này, một số cơ sở dữ liệu, như Apache Cassandra và Riak (cùng nhiều hệ khác), dùng một sơ đồ phân vùng khác gọi là băm nhất quán (consistent hashing). Như đã đề cập trước đó, các giá trị khóa định tuyến được băm. Các giá trị do hàm băm trả về được ánh xạ lên một vòng (ring), sao cho sau giá trị lớn nhất có thể, nó quấn vòng lại về giá trị nhỏ nhất của mình. Mỗi nút có vị trí riêng của nó trên vòng và trở nên chịu trách nhiệm cho khoảng các giá trị nằm giữa vị trí của nút tiền nhiệm và vị trí của chính nó.

Using consistent hashing helps to reduce the number of relocations required for maintaining balance: a change in the ring affects only the immediate neighbors of the leaving or joining node, and not an entire cluster. The word consistent in the definition implies that, when the hash table is resized, if we have K possible hash keys and n nodes, on average we have to relocate only K/n keys.

In other words, a consistent hash function output changes minimally as the function range changes [KARGER97] .

Việc dùng băm nhất quán giúp giảm số lần dời chỗ cần thiết để duy trì cân bằng: một thay đổi trong vòng chỉ ảnh hưởng đến các láng giềng kề ngay của nút rời đi hoặc gia nhập, chứ không ảnh hưởng đến toàn bộ cụm. Từ “nhất quán” trong định nghĩa hàm ý rằng, khi bảng băm được thay đổi kích thước, nếu ta có K khóa băm có thể có và n nút, thì trung bình ta chỉ phải dời chỗ K/n khóa. Nói cách khác, đầu ra của một hàm băm nhất quán thay đổi ở mức tối thiểu khi miền của hàm thay đổi [KARGER97].

Distributed Transactions with Percolator — Giao dịch phân tán với Percolator

Coming back to the subject of distributed transactions, isolation levels might be difficult to reason about because of the allowed read and write anomalies. If serializability is not required by the application, one of the ways to avoid the write anomalies described in SQL-92 is to use a transactional model called snapshot isolation (SI).

Quay lại chủ đề giao dịch phân tán, các mức cô lập có thể khó lập luận về vì các dị thường đọc và ghi được cho phép. Nếu ứng dụng không đòi hỏi tính khả tuần tự, một trong các cách tránh các dị thường ghi được mô tả trong SQL-92 là dùng một mô hình giao dịch gọi là cô lập snapshot (snapshot isolation, SI).

Snapshot isolation guarantees that all reads made within the transaction are consistent with a snapshot of the database. The snapshot contains all values that were committed before the transaction's start timestamp. If there's a write-write conflict (i.e., when two concurrently running transactions attempt to make a write to the same **cell**), only one of them will commit. This characteristic is usually referred to as first committer wins .

Cô lập snapshot bảo đảm rằng tất cả các phép đọc thực hiện trong giao dịch là nhất quán với một snapshot của cơ sở dữ liệu. Snapshot chứa tất cả các giá trị đã được commit trước dấu thời gian bắt đầu của giao dịch. Nếu có một xung đột ghi-ghi (tức là khi hai giao dịch chạy đồng thời cố ghi vào cùng một ô), chỉ một trong số chúng sẽ commit. Đặc tính này thường được gọi là “người commit đầu tiên thắng” (first committer wins).

Snapshot isolation prevents read skew , an anomaly permitted under the read-committed **isolation level**. For example, a sum of x and y is supposed to be 100 . Transaction T1 performs an operation read(x) , and reads the value 70 . T2 updates two values write(x, 50) and write(y, 50) , and commits. If T1 attempts to run read(y) , and proceeds with transaction execution based on the value of y (50), newly committed by T2 , it will lead to an inconsistency. The value of x that T1 has read before T2 committed and the new value of y aren't consistent with each other. Since snapshot isolation only makes values up to a specific timestamp visible for transactions, the new value of y , 50 , won't be visible to T1 [BERENSON95] .

Cô lập snapshot ngăn được đọc lệch (read skew), một dị thường được cho phép dưới mức cô lập read-committed. Ví dụ, tổng của x và y được kỳ vọng là 100. Giao dịch T1 thực hiện một thao tác read(x), và đọc giá trị 70. T2 cập nhật hai giá trị write(x, 50) và write(y, 50), rồi commit. Nếu T1 cố chạy read(y), và tiến hành thực thi giao dịch dựa trên giá trị của y (50), vừa được T2 commit, nó sẽ dẫn đến sự không nhất quán. Giá trị của x mà T1 đã đọc trước khi T2 commit và giá trị mới của y không nhất quán với nhau. Vì cô lập snapshot chỉ làm cho các giá trị tới một dấu thời gian cụ thể trở nên hiển thị cho các giao dịch, nên giá trị mới của y, là 50, sẽ không hiển thị với T1 [BERENSON95].

Snapshot isolation has several convenient properties:

Cô lập snapshot có vài tính chất tiện lợi:

- It allows only repeatable reads of committed data.
- Values are consistent, as they're read from the snapshot at a specific timestamp.
- Conflicting writes are aborted and retried to prevent inconsistencies.
 - Nó chỉ cho phép các phép đọc lặp lại được (repeatable read) trên dữ liệu đã commit.
 - Các giá trị là nhất quán, vì chúng được đọc từ snapshot tại một dấu thời gian cụ thể.
 - Các phép ghi xung đột bị hủy bỏ và thử lại để ngăn sự không nhất quán.

Despite that, histories under snapshot isolation are not serializable. Since only conflicting writes to the same cells are aborted, we can still end up with a write skew (see “Read and Write Anomalies” on page 95). Write skew occurs when two transactions modify disjoint sets of values, each preserving invariants for the data it writes. Both transactions are allowed to commit, but a combination of writes performed by these transactions may violate these invariants.

Mặc dù vậy, các lịch sử dưới cô lập snapshot không khả tuần tự. Vì chỉ các phép ghi xung đột vào cùng các ô bị hủy bỏ, ta vẫn có thể kết thúc với một ghi lệch (write skew) (xem “Read and Write Anomalies” ở trang 95). Ghi lệch xảy ra khi hai giao dịch sửa đổi các tập giá trị rời nhau, mỗi giao dịch giữ gìn các bất biến cho dữ liệu mà nó ghi. Cả hai giao dịch đều được phép commit, nhưng một sự kết hợp các phép ghi do các giao dịch này thực hiện có thể vi phạm các bất biến đó.

Snapshot isolation provides semantics that can be useful for many applications and has the major advantage of efficient reads, because no locks have to be acquired since snapshot data cannot be changed.

Cô lập snapshot cung cấp ngữ nghĩa có thể hữu ích cho nhiều ứng dụng và có ưu điểm lớn là đọc hiệu quả, bởi không phải giành khóa nào vì dữ liệu snapshot không thể bị thay đổi.

Percolator is a library that implements a transactional API on top of the distributed database Bigtable (see “Wide Column Stores” on page 15). This is a great example of building a transaction API on top of the existing system. Percolator stores data records, committed data point locations (write metadata), and locks in different columns. To avoid race conditions and reliably lock tables in a single RPC call, it uses a conditional mutation Bigtable API that allows it to perform read-modify-write operations with a single remote call.

Percolator là một thư viện triển khai một API giao dịch trên nền cơ sở dữ liệu phân tán Bigtable (xem “Wide Column Stores” ở trang 15). Đây là một ví dụ tuyệt vời về việc xây dựng một API giao dịch trên nền một hệ thống đã có. Percolator lưu các bản ghi dữ liệu, vị trí của các điểm dữ liệu đã commit (metadata ghi), và các khóa trong các cột khác nhau. Để tránh tình trạng tranh đua (race condition) và khóa các bảng một cách đáng tin cậy trong một lời gọi RPC duy nhất, nó dùng một API biến đổi có điều kiện (conditional mutation) của Bigtable cho phép nó thực hiện các thao tác đọc-sửa-ghi với một lời gọi từ xa duy nhất.

Each transaction has to consult the timestamp oracle (a source of clusterwide-consistent monotonically increasing timestamps) twice: for a transaction start timestamp, and during commit. Writes are buffered and committed using a client-driven two-phase commit (see “Two-Phase Commit” on page 259).

Mỗi giao dịch phải tham vấn timestamp oracle (một nguồn các dấu thời gian tăng đơn điệu nhất quán toàn cụm) hai lần: để lấy dấu thời gian bắt đầu giao dịch, và trong lúc commit. Các phép ghi được đệm và được commit dùng một commit hai pha do client điều khiển (xem “Two-Phase Commit” ở trang 259).

Figure 13-9 shows how the contents of the table change during execution of the transaction steps:

Hình 13-9 cho thấy nội dung của bảng thay đổi như thế nào trong khi thực thi các bước của giao dịch:

- a) Initial state. After the execution of the previous transaction, TS1 is the latest timestamp for both accounts. No locks are held.
- b) The first phase, called prewrite . The transaction attempts to acquire locks for all cells written during the transaction. One of the locks is marked as primary and is used for client recovery. The transaction checks for the possible conflicts: if any other transaction has already written any data with a later timestamp or there are unreleased locks at any timestamp. If any conflict is detected, the transaction aborts.
- c) If all locks were successfully acquired and the possibility of conflict is ruled out, the transaction can continue. During the second phase, the client releases its locks, starting with the primary one. It publishes its write by replacing the lock with a write record, updating write metadata with the timestamp of the latest data point.
 - a) Trạng thái ban đầu. Sau khi thực thi giao dịch trước đó, TS1 là dấu thời gian mới nhất cho cả hai tài khoản. Không có khóa nào được giữ.
 - b) Pha thứ nhất, gọi là prewrite. Giao dịch cố giành các khóa cho tất cả các ô được ghi trong giao dịch. Một trong các khóa được đánh dấu là chính (primary) và được dùng cho việc phục hồi của client. Giao dịch kiểm tra các xung đột có thể xảy ra: nếu bất kỳ giao dịch nào khác đã ghi bất kỳ dữ liệu nào với một dấu thời gian muộn hơn hoặc có các khóa chưa được nhả tại bất kỳ dấu thời gian nào. Nếu phát hiện bất kỳ xung đột nào, giao dịch hủy bỏ.
 - c) Nếu tất cả các khóa được giành thành công và khả năng xung đột bị loại trừ, giao dịch có thể tiếp tục. Trong pha thứ hai, client nhả các khóa của nó, bắt đầu từ khóa chính. Nó công bố phép ghi của mình bằng cách thay khóa bằng một bản ghi ghi (write record), cập nhật metadata ghi với dấu thời gian của điểm dữ liệu mới nhất.

Since the client may fail while trying to commit the transaction, we need to make sure that partial transactions are finalized or rolled back. If a later transaction encounters an incomplete state, it should attempt to release the primary lock and commit the transaction. If the primary lock is already released, transaction contents have to be committed. Only one transaction can hold a lock at a time and all state transitions are atomic, so situations in which two transactions attempt to perform operations on the contents are not possible.

Vì client có thể bị lỗi trong khi cố commit giao dịch, ta cần chắc chắn rằng các giao dịch một phần được hoàn tất hoặc được rollback. Nếu một giao dịch muộn hơn gặp một trạng thái chưa hoàn chỉnh, nó nên cố nhả khóa chính và commit giao dịch. Nếu khóa chính đã được nhả, nội dung giao dịch phải được commit. Mỗi lần chỉ một giao dịch có thể giữ một khóa và mọi chuyển trạng thái đều nguyên tử, nên các tình huống mà hai giao dịch cố thực hiện các thao tác trên cùng nội dung là không thể xảy ra.

		Data	Locks	Write Metadata
Account1	TS2	-	-	TS1 is latest
	TS1	\$100	-	-
Account2	TS2	-	-	TS1 is latest
	TS1	\$200	-	-

a) Initial State before moving \$150 from Account2 to Account1

		Data	Locks	Write Metadata
Account1	TS3	\$250	Primary	-
	TS2	-	-	TS1 is latest
	TS1	\$100	-	-
Account2	TS3	\$50	Primary at Account1	-
	TS2	-	-	TS1 is latest
	TS1	\$200	-	-

b) State after taking locks and updating accounts

		Data	Locks	Write Metadata
Account1	TS4	-	-	TS3 is latest
	TS3	\$250	-	-
	TS2	-	-	TS1 is latest
	TS1	\$100	-	-
Account2	TS4	-	-	TS3 is latest
	TS3	\$50	Primary at Account1	-
	TS2	-	-	TS1 is latest
	TS1	\$200	-	-

c) Transaction commit releases locks and updates metadata with latest timestamp

Figure 13-9. Percolator transaction execution steps. Transaction credits \$150 from Account2 and debits it to Account1.

Hình 13-9. Các bước thực thi giao dịch Percolator. Giao dịch ghi có \$150 từ Account2 và ghi nợ nó vào Account1.

Snapshot isolation is an important and useful abstraction, commonly used in transaction processing. Since it simplifies semantics, precludes some of the anomalies, and opens up an opportunity to improve concurrency and performance, many **MVCC** systems offer this isolation level.

Cô lập snapshot là một sự trừu tượng hóa quan trọng và hữu ích, thường được dùng trong xử lý giao dịch. Vì nó đơn giản hóa ngữ nghĩa, loại trừ một số dị thường, và mở ra cơ hội cải thiện tính tương tranh và hiệu năng, nhiều hệ MVCC cung cấp mức cô lập này.

One of the examples of databases based on the Percolator model is TiDB (“Ti” stands for Titanium).

TiDB is a strongly consistent, highly available, and horizontally scalable open source database, compatible with MySQL.

Một trong các ví dụ về cơ sở dữ liệu dựa trên mô hình Percolator là TiDB (“Ti” là viết tắt của Titanium). TiDB là một cơ sở dữ liệu mã nguồn mở nhất quán mạnh, có tính sẵn sàng cao, và mở rộng quy mô được theo chiều ngang, tương thích với MySQL.

Coordination Avoidance — Né tránh phối hợp

One more example, discussing costs of serializability and attempting to reduce the amount of coordination while still providing strong consistency guarantees, is coordination avoidance [BAILIS14b]. Coordination can be avoided, while preserving data integrity constraints, if operations are invariant confluent. Invariant Confluence (I-Confluence) is defined as a property that ensures that two invariant-valid but diverged database states can be merged into a single valid, final state. Invariants in this case preserve consistency in **ACID** terms.

Một ví dụ nữa, bàn về chi phí của tính khả tuần tự và cố giảm lượng phối hợp trong khi vẫn cung cấp các bảo đảm nhất quán mạnh, là né tránh phối hợp (coordination avoidance) [BAILIS14b]. Việc phối hợp có thể được né tránh, trong khi vẫn giữ gìn các ràng buộc toàn vẹn dữ liệu, nếu các thao tác là bất-biến-hợp-lưu (invariant confluent). Hợp lưu bất biến (Invariant Confluence, I-Confluence) được định nghĩa là một tính chất bảo đảm rằng hai trạng thái cơ sở dữ liệu hợp lệ về bất biến nhưng đã phân kỳ có thể được gộp thành một trạng thái cuối cùng hợp lệ duy nhất. Trong trường hợp này các bất biến giữ gìn tính nhất quán theo nghĩa ACID.

Because any two valid states can be merged into a valid state, I-Confluent operations can be executed without additional coordination, which significantly improves performance characteristics and scalability potential.

Vì bất kỳ hai trạng thái hợp lệ nào cũng có thể được gộp thành một trạng thái hợp lệ, các thao tác I-Confluent có thể được thực thi mà không cần phối hợp bổ sung, điều này cải thiện đáng kể các đặc tính hiệu năng và tiềm năng mở rộng quy mô.

To preserve this invariant, in addition to defining an operation that brings our database to the new state, we have to define a **merge** function that accepts two states. This function is used in case states were updated independently and bring diverged states back to convergence.

Để giữ gìn bất biến này, ngoài việc định nghĩa một thao tác đưa cơ sở dữ liệu của ta tới trạng thái mới, ta phải định nghĩa một hàm gộp (merge function) nhận vào hai trạng thái. Hàm này được dùng trong trường hợp các trạng thái được cập nhật độc lập và đưa các trạng thái đã phân kỳ trở lại hội tụ.

Transactions are executed against the local database versions (snapshots). If a transaction requires any state from other partitions for execution, this state is made available for it locally. If a transaction commits, resulting changes made to the local snapshot are migrated and merged with the snapshots on the other nodes. A system model that allows coordination avoidance has to guarantee the following properties:

Các giao dịch được thực thi trên các phiên bản cơ sở dữ liệu cục bộ (snapshot). Nếu một giao dịch cần bất kỳ trạng thái nào từ các phân vùng khác để thực thi, trạng thái đó được làm cho khả dụng cho nó một cách cục bộ. Nếu một giao dịch commit, các thay đổi kết quả thực hiện trên snapshot cục bộ được di chuyển và gộp với các snapshot trên các nút khác. Một mô hình hệ thống cho phép né tránh phối hợp phải bảo đảm các tính chất sau:

Global validity Required invariants are always satisfied, for both merged and divergent committed database states, and transactions cannot observe invalid states.

Tính hợp lệ toàn cục Các bất biến yêu cầu luôn được thỏa mãn, cho cả các trạng thái cơ sở dữ liệu đã commit gộp lẫn phân kỳ, và các giao dịch không thể quan sát thấy các trạng thái không hợp lệ.

Availability If all nodes holding states are reachable by the client, the transaction has to reach a commit decision, or abort, if committing it would violate one of the transaction invariants.

Tính sẵn sàng Nếu tất cả các nút giữ trạng thái đều tiếp cận được bởi client, giao dịch phải đi đến một quyết định commit, hoặc abort, nếu việc commit nó sẽ vi phạm một trong các bất biến của giao dịch.

Convergence Nodes can maintain their local states independently, but in the absence of further transactions and indefinite network partitions, they have to be able to reach the same state.

Tính hội tụ Các nút có thể duy trì trạng thái cục bộ của mình một cách độc lập, nhưng khi không có thêm giao dịch nào và không có phân vùng mạng vô hạn định, chúng phải có khả năng đạt tới cùng một trạng thái.

Coordination freedom Local transaction execution is independent from the operations against the local states performed on behalf of the other nodes.

Tính tự do về phối hợp Việc thực thi giao dịch cục bộ là độc lập với các thao tác trên các trạng thái cục bộ được thực hiện thay mặt cho các nút khác.

One of the examples of implementing coordination avoidance is Read-Atomic Multi Partition (RAMP) transactions [BAILIS14c]. RAMP uses multiversion concurrency control and metadata of current in-flight operations to fetch any missing state updates from other nodes, allowing read and write operations to be executed concurrently. For example, readers that overlap with some writer modifying the same entry can be detected and, if necessary, repaired by retrieving required information from the in-flight write metadata in an additional round of communication.

Một trong các ví dụ về việc triển khai né tránh phối hợp là các giao dịch Read-Atomic Multi Partition (RAMP) [BAILIS14c]. RAMP dùng điều khiển tương tranh đa phiên bản (MVCC) và metadata của các thao tác đang diễn ra (in-flight) hiện tại để lấy bất kỳ cập nhật trạng thái còn thiếu nào từ các nút khác, cho phép các thao tác đọc và ghi được thực thi đồng thời. Ví dụ, các bên đọc trùng lặp với một bên ghi nào đó đang sửa đổi cùng một mục có thể được phát hiện và, nếu cần, được sửa chữa bằng cách truy xuất thông tin cần thiết từ metadata ghi đang diễn ra trong một vòng truyền thông bổ sung.

Using lock-based approaches in a distributed environment might be not the best idea, and instead of doing that, RAMP provides two properties:

Việc dùng các cách tiếp cận dựa trên khóa trong môi trường phân tán có thể không phải là ý tưởng tốt nhất, và thay vì làm vậy, RAMP cung cấp hai tính chất:

Synchronization independence One client's transactions won't stall, abort, or force the other client's transactions to wait.

Tính độc lập về đồng bộ Các giao dịch của một client sẽ không bị đình trệ, hủy bỏ, hay buộc các giao dịch của client khác phải chờ.

Partition independence Clients do not have to contact partitions whose values aren't involved in their transactions.

Tính độc lập về phân vùng Các client không phải liên lạc với các phân vùng có giá trị không liên quan tới giao dịch của họ.

RAMP introduces the read atomic isolation level: transactions cannot observe any in-process state changes from in-flight, uncommitted, and aborted transactions. In other words, all (or none) transaction updates are visible to concurrent transactions. By that definition, the read atomic isolation level also precludes fractured reads : when a transaction observes only a subset of writes executed by some other transaction.

RAMP giới thiệu mức cô lập đọc nguyên tử (read atomic): các giao dịch không thể quan sát thấy bất kỳ thay đổi trạng thái đang xử lý nào từ các giao dịch đang diễn ra, chưa commit, và đã hủy bỏ. Nói cách khác, tất cả (hoặc không có) cập nhật giao dịch nào hiển thị với các giao dịch đồng thời. Theo định nghĩa đó, mức cô lập đọc nguyên tử cũng loại trừ các phép đọc rạn nứt (fractured read): khi một giao dịch chỉ quan sát thấy một tập con các phép ghi được một giao dịch khác thực thi.

RAMP offers atomic write visibility without requiring mutual exclusion, which other solutions, such as distributed locks, often couple together. This means that transactions can proceed without stalling each other.

RAMP cung cấp tính hiển thị ghi nguyên tử mà không đòi hỏi loại trừ tương hỗ (mutual exclusion), thứ mà các giải pháp khác, như khóa phân tán, thường ghép chung lại với nhau. Điều này nghĩa là các giao dịch có thể tiến hành mà không làm đình trệ lẫn nhau.

RAMP distributes transaction metadata that allows reads to detect concurrent in-flight writes. By using this metadata, transactions can detect the presence of newer record versions, find and fetch the latest ones, and operate on them. To avoid coordination, all local commit decisions must also be valid globally. In RAMP, this is solved by requiring that, by the time a write becomes visible in one partition, writes from the same transaction in all other involved partitions are also visible for readers in those partitions.

RAMP phân phối metadata giao dịch cho phép các phép đọc phát hiện các phép ghi đồng thời đang diễn ra. Bằng cách dùng metadata này, các giao dịch có thể phát hiện sự hiện diện của các phiên bản ghi mới hơn, tìm và lấy các phiên bản mới nhất, và thao tác trên chúng. Để né tránh phối hợp, mọi quyết định commit cục bộ cũng phải hợp lệ ở quy mô toàn cục. Trong RAMP, điều này được giải quyết bằng cách đòi hỏi rằng, tới khi một phép ghi trở nên hiển thị trong một phân vùng, các phép ghi từ cùng giao dịch đó trong tất cả các phân vùng liên quan khác cũng hiển thị đối với các bên đọc trong các phân vùng đó.

To allow readers and writers to proceed without blocking other concurrent readers and writers, while maintaining the read atomic isolation level both locally and system-wide (in all other partitions modified by the committing transaction), writes in RAMP are installed and made visible using two-phase commit:

Để cho phép các bên đọc và bên ghi tiến hành mà không chặn các bên đọc và bên ghi đồng thời khác, trong khi vẫn duy trì mức cô lập đọc nguyên tử cả cục bộ lẫn toàn hệ thống (trong tất cả các phân vùng khác bị sửa đổi bởi giao dịch đang commit), các phép ghi trong RAMP được cài đặt và làm cho hiển thị dùng commit hai pha:

Prepare The first phase prepares and places writes to their respective target partitions without making them visible.

Prepare Pha thứ nhất chuẩn bị và đặt các phép ghi vào các phân vùng đích tương ứng của chúng mà không làm chúng hiển thị.

Commit/abort The second phase publishes the state changes made by the write operation of the committing transaction, making them available atomically across all partitions, or rolls back the changes.

Commit/abort Pha thứ hai công bố các thay đổi trạng thái do thao tác ghi của giao dịch đang commit thực hiện, làm cho chúng khả dụng một cách nguyên tử trên tất cả các phân vùng, hoặc rollback các thay đổi.

RAMP allows multiple versions of the same record to be present at any given moment: latest value, in-flight uncommitted changes, and stale versions, overwritten by later transactions. Stale versions have to be kept around only for in-progress read requests. As soon as all concurrent readers complete, stale values can be discarded.

RAMP cho phép nhiều phiên bản của cùng một bản ghi hiện diện tại bất kỳ thời điểm nào: giá trị mới nhất, các thay đổi chưa commit đang diễn ra, và các phiên bản cũ, bị ghi đè bởi các giao dịch muộn hơn. Các phiên bản cũ chỉ phải được giữ lại cho các yêu cầu đọc đang tiến hành. Ngay khi tất cả các bên đọc đồng thời hoàn tất, các giá trị cũ có thể bị loại bỏ.

Making distributed transactions performant and scalable is difficult because of the coordination overhead associated with preventing, detecting, and avoiding conflicts for the concurrent operations. The larger the system, or the more transactions it attempts to serve, the more overhead it incurs. The approaches described in this section attempt to reduce the amount of coordination by using invariants to determine where coordination can be avoided, and only paying the full price if it's absolutely necessary.

Làm cho các giao dịch phân tán có hiệu năng cao và mở rộng được là điều khó, vì chi phí phối hợp gắn với việc ngăn ngừa, phát hiện, và né tránh xung đột cho các thao tác đồng thời. Hệ thống càng lớn, hoặc càng cố phục vụ nhiều giao dịch, nó càng phát sinh nhiều chi phí phụ. Các cách tiếp cận được mô tả trong phần này cố giảm lượng phối hợp bằng cách dùng các bất biến để xác định nơi có thể né tránh phối hợp, và chỉ trả cái giá đầy đủ nếu điều đó thật sự cần thiết.

Summary — Tóm tắt

In this chapter, we discussed several ways of implementing distributed transactions. First, we discussed two atomic commitment algorithms: two- and three-phase commits. The big advantage of these algorithms is that they're easy to understand and implement, but have several shortcomings. In 2PC, a coordinator (or at least its substitute) has to be alive for the length of the commitment process, which significantly reduces availability. 3PC lifts this requirement for some cases, but is prone to split brain in case of network partition.

Trong chương này, ta đã bàn về vài cách triển khai giao dịch phân tán. Trước hết, ta đã bàn về hai thuật toán cam kết nguyên tử: commit hai pha và ba pha. Ưu điểm lớn của các thuật toán này là chúng dễ hiểu và dễ triển khai, nhưng có vài nhược điểm. Trong 2PC, một bên điều phối (hoặc ít nhất là bên thay thế nó) phải còn sống suốt độ dài của quá trình cam kết, điều này làm giảm đáng kể tính sẵn sàng. 3PC gỡ bỏ yêu cầu này trong một số trường hợp, nhưng dễ bị split brain trong trường hợp phân vùng mạng.

Distributed transactions in modern database systems are often implemented using consensus algorithms, which we're going to discuss in the next chapter. For example, both Calvin and Spanner, discussed in this chapter, use Paxos.

Các giao dịch phân tán trong các hệ cơ sở dữ liệu hiện đại thường được triển khai dùng các thuật toán đồng thuận, mà ta sẽ bàn ở chương kế tiếp. Ví dụ, cả Calvin và Spanner, được

bàn trong chương này, đều dùng Paxos.

Consensus algorithms are more involved than atomic commit ones, but have much better fault-tolerance properties, and decouple decisions from their initiators and allow participants to decide on a value rather than on whether or not to accept the value [GRAY04] .

Các thuật toán đồng thuận phức tạp hơn các thuật toán cam kết nguyên tử, nhưng có các tính chất dung lỗi tốt hơn nhiều, và tách rời các quyết định khỏi những bên khởi xướng chúng cũng như cho phép các bên tham gia quyết định trên một giá trị thay vì quyết định về việc có chấp nhận giá trị đó hay không [GRAY04].

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Atomic commitment integration with local transaction processing and recovery subsystems Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. 2010. Database Systems Concepts (6th Ed.). New York: McGraw-Hill.

Garcia-Molina, Hector, Jeffrey D. Ullman, and Jennifer Widom. 2008. Database Systems: The Complete Book (2nd Ed.). Boston: Pearson.

Recent progress in the area of distributed transactions (ordered chronologically; this list is not intended to be exhaustive) Cowling, James and Barbara Liskov. 2012. “Granola: low-overhead distributed transaction coordination.” In Proceedings of the 2012 USENIX conference on Annual Technical Conference (USENIX ATC '12) : 21-21. USENIX.

Balakrishnan, Mahesh, Dahlia Malkhi, Ted Wobber, Ming Wu, Vijayan Prabhakaran, Michael Wei, John D. Davis, Sriram Rao, Tao Zou, and Aviad Zuck. 2013. “Tango: distributed data structures over a shared log.” In Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP '13) : 324-340.

Ding, Bailu, Lucja Kot, Alan Demers, and Johannes Gehrke. 2015. “Centiman: elastic, high performance optimistic concurrency control by watermarking.” In Proceedings of the Sixth ACM Symposium on Cloud Computing (SoCC '15) : 262-275.

Dragojević, Aleksandar, Dushyanth Narayanan, Edmund B. Nightingale, Matthew Renzelmann, Alex Shamis, Anirudh Badam, and Miguel Castro. 2015. “No compromises: distributed transactions with consistency, availability, and performance.” In Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15) : 54-70.

Zhang, Irene, Naveen Kr. Sharma, Adriana Szekeres, Arvind Krishnamurthy, and Dan R. K. Ports. 2015. “Building consistent transactions with inconsistent replication.” In Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15) : 263-278.

CHAPTER 14 Consensus — CHƯƠNG 14 Đồng thuận

We’ve discussed quite a few concepts in distributed systems, starting with basics, such as links and processes, problems with distributed computing; then going through failure models, failure detectors, and **leader election**; discussed consistency models; and we’re finally ready to put it all together for a pinnacle of distributed systems research: distributed **consensus**.

Chúng ta đã thảo luận khá nhiều khái niệm trong hệ phân tán (distributed system), bắt đầu từ những điều cơ bản như kênh (link) và tiến trình (process), các vấn đề của tính toán phân tán; rồi đi qua các mô hình lỗi, bộ phát hiện lỗi (failure detector) và bầu chọn leader (leader election); thảo luận các mô hình nhất quán; và cuối cùng chúng ta đã sẵn sàng ghép tất cả lại với nhau cho đỉnh cao của nghiên cứu hệ phân tán: đồng thuận phân tán (distributed consensus).

Consensus algorithms in distributed systems allow multiple processes to reach an agreement on a value. **FLP impossibility** (see “FLP Impossibility” on **page 189**) shows that it is impossible to guarantee consensus in a completely **asynchronous** system in a bounded time. Even if message delivery is guaranteed, it is impossible for one **process** to know whether the other one has crashed or is running slowly.

Các thuật toán đồng thuận (consensus) trong hệ phân tán cho phép nhiều tiến trình đạt được một sự thống nhất về một giá trị. Bất khả thi FLP (FLP impossibility) (xem “FLP Impossibility” ở trang 189) cho thấy rằng không thể bảo đảm đồng thuận trong một hệ hoàn toàn bất đồng bộ (asynchronous system) trong thời gian hữu hạn. Ngay cả khi việc chuyển giao thông điệp được bảo đảm, một tiến trình vẫn không thể biết được tiến trình khác đã sập đổ hay chỉ đang chạy chậm.

In Chapter 9 , we discussed that there’s a trade-off between failure-detection accuracy and how quickly the failure can be detected. Consensus algorithms assume an asynchronous model and guarantee safety, while an external **failure detector** can provide information about other processes, guaranteeing liveness [CHANDRA96] . Since failure detection is not always fully accurate, there will be situations when a consensus algorithm waits for a process failure to be detected, or when the algorithm is restarted because some process is incorrectly suspected to be faulty.

Trong Chương 9, chúng ta đã thảo luận rằng có một sự đánh đổi giữa độ chính xác của việc phát hiện lỗi và tốc độ phát hiện lỗi. Các thuật toán đồng thuận giả định một mô hình bất đồng bộ và bảo đảm tính an toàn (safety), trong khi một bộ phát hiện lỗi bên ngoài có thể cung cấp thông tin về các tiến trình khác, bảo đảm tính sống còn (liveness) [CHANDRA96]. Vì việc phát hiện lỗi không phải lúc nào cũng hoàn toàn chính xác, sẽ có những tình huống thuật toán đồng thuận phải chờ một lỗi tiến trình được phát hiện, hoặc thuật toán bị khởi động lại vì một tiến trình nào đó bị nghi ngờ nhầm là đã hỏng.

Processes have to agree on some value proposed by one of the participants, even if some of them happen to crash. A process is said to be correct if hasn't crashed and continues executing algorithm steps. Consensus is extremely useful for putting events in a particular order, and ensuring consistency among the participants. Using consensus, we can have a system where processes move from one value to the next one without losing certainty about which values the clients observe.

Các tiến trình phải thống nhất về một giá trị nào đó do một trong các bên tham gia đề xuất, ngay cả khi một số trong chúng tình cờ sập đổ. Một tiến trình được gọi là đúng (correct) nếu nó chưa sập đổ và tiếp tục thực hiện các bước của thuật toán. Đồng thuận cực kỳ hữu ích cho việc sắp xếp các sự kiện theo một thứ tự cụ thể, và bảo đảm tính nhất quán giữa các bên tham gia. Sử dụng đồng thuận, chúng ta có thể có một hệ thống trong đó các tiến trình chuyển từ giá trị này sang giá trị kế tiếp mà không mất đi sự chắc chắn về những giá trị mà các client quan sát được.

279 From a theoretical perspective, consensus algorithms have three properties:

279 Từ góc nhìn lý thuyết, các thuật toán đồng thuận có ba thuộc tính:

Agreement The decision value is the same for all correct processes.

Thống nhất (Agreement) Giá trị quyết định là giống nhau cho mọi tiến trình đúng.

Validity The decided value was proposed by one of the processes.

Hợp lệ (Validity) Giá trị được quyết định là một giá trị đã được đề xuất bởi một trong các tiến trình.

Termination All correct processes eventually reach the decision.

Kết thúc (Termination) Tất cả các tiến trình đúng cuối cùng đều đạt tới quyết định.

Each one of these properties is extremely important. The agreement is embedded in the human understanding of consensus. The dictionary definition of consensus has the word “unanimity” in it. This means that upon the agreement, no process is allowed to have a different opinion about the outcome. Think of it as an agreement to meet at a particular time and place with your friends: all of you would like to meet, and only the specifics of the event are being agreed upon.

Mỗi thuộc tính trong số này đều cực kỳ quan trọng. Sự thống nhất được lồng ghép trong cách hiểu của con người về đồng thuận. Định nghĩa trong từ điển về đồng thuận có chứa từ “nhất trí” (unanimity). Điều này có nghĩa là khi đã thống nhất, không tiến trình nào được phép có ý kiến khác về kết quả. Hãy hình dung nó như một thỏa thuận gặp nhau ở một thời điểm và địa điểm cụ thể với bạn bè của bạn: tất cả các bạn đều muốn gặp mặt, và chỉ những chi tiết cụ thể của sự kiện là đang được thống nhất.

Validity is essential, because without it consensus can be trivial. Consensus algorithms require all processes to agree on some value. If processes use some predetermined, arbitrary default value as a decision output regardless of the proposed values, they will reach unanimity, but the output of such an algorithm will not be valid and it wouldn't be useful in reality.

Tính hợp lệ là thiết yếu, bởi vì nếu không có nó thì đồng thuận có thể trở nên tầm thường. Các thuật toán đồng thuận yêu cầu mọi tiến trình thống nhất về một giá trị nào đó. Nếu các tiến trình dùng một giá trị mặc định tùy tiện, định sẵn làm kết quả quyết định bất kể các giá trị được đề xuất, chúng sẽ đạt được sự nhất trí, nhưng kết quả của một thuật toán như vậy sẽ không hợp lệ và sẽ chẳng hữu ích gì trong thực tế.

Without termination, our algorithm will continue forever without reaching any conclusion or will wait indefinitely for a crashed process to come back, which is not very useful, either. Processes have to agree eventually and, for a consensus algorithm to be practical, this has to happen rather quickly.

Nếu không có tính kết thúc, thuật toán của chúng ta sẽ tiếp tục mãi mãi mà không đạt tới bất kỳ kết luận nào, hoặc sẽ chờ vô thời hạn một tiến trình đã sụp đổ quay trở lại, điều này cũng không hữu ích lắm. Các tiến trình cuối cùng phải thống nhất, và để một thuật toán đồng thuận thực dụng, điều này phải xảy ra khá nhanh.

Broadcast — Quảng bá (Broadcast)

A broadcast is a communication abstraction often used in distributed systems. Broadcast algorithms are used to disseminate information among a set of processes. There exist many broadcast algorithms, making different assumptions and providing different guarantees. Broadcast is an important primitive and is used in many places, including consensus algorithms. We’ve discussed one of the forms of broadcast—gossip **dissemination**—already (see “Gossip Dissemination” on page 250).

Quảng bá (broadcast) là một sự trừu tượng hóa giao tiếp thường được dùng trong hệ phân tán. Các thuật toán quảng bá được dùng để lan truyền (dissemination) thông tin giữa một tập hợp các tiến trình. Có nhiều thuật toán quảng bá tồn tại, với các giả định khác nhau và cung cấp các bảo đảm khác nhau. Quảng bá là một nguyên hàm quan trọng và được dùng ở nhiều nơi, kể cả trong các thuật toán đồng thuận. Chúng ta đã thảo luận một trong các dạng quảng bá — lan truyền gossip (gossip dissemination) — rồi (xem “Gossip Dissemination” ở trang 250).

Broadcasts are often used for database replication when a single **coordinator node** has to distribute the data to all other participants. However, making this process reliable is not a trivial matter: if the coordinator crashes after distributing the message to some nodes but not the other ones, it leaves the system in an inconsistent state: some of the nodes observe a new message and some do not.

Quảng bá thường được dùng cho việc nhân bản cơ sở dữ liệu khi một nút điều phối đơn lẻ phải phân phối dữ liệu tới tất cả các bên tham gia khác. Tuy nhiên, làm cho quá trình này trở nên đáng tin cậy không phải là chuyện đơn giản: nếu bên điều phối sụp đổ sau khi đã phân phối thông điệp tới một số nút nhưng chưa tới các nút khác, nó để lại hệ thống ở trạng thái không nhất quán: một số nút quan sát được thông điệp mới còn một số thì không.

The simplest and the most straightforward way to broadcast messages is through a best effort broadcast [CACHIN11] . In this case, the sender is responsible for ensuring message delivery to all the targets. If it fails, the other participants do not try to rebroadcast the message, and in the case of coordinator crash, this type of broadcast will fail silently.

Cách quảng bá thông điệp đơn giản và trực tiếp nhất là thông qua quảng bá nỗ lực tối đa (best effort broadcast) [CACHIN11]. Trong trường hợp này, bên gửi chịu trách nhiệm bảo đảm việc chuyển giao thông điệp tới tất cả các đích. Nếu nó thất bại, các bên tham gia khác không cố quảng bá lại thông điệp, và trong trường hợp bên điều phối sụp đổ, kiểu quảng bá này sẽ thất bại trong im lặng.

For a broadcast to be reliable , it needs to guarantee that all correct processes receive the same messages, even if the sender crashes during transmission.

Để một quảng bá trở nên đáng tin cậy (reliable), nó cần bảo đảm rằng tất cả các tiến trình đúng nhận được cùng các thông điệp, ngay cả khi bên gửi sụp đổ trong lúc truyền.

To implement a naive version of a reliable broadcast, we can use a failure detector and a fallback mechanism. The most straightforward fallback mechanism is to allow every process that received the message to forward it to every other process it’s aware of. When the source process fails, other

processes detect the failure and continue broadcasting the message, effectively flooding the network with N^2 messages (as shown in Figure 14-1). Even if the sender has crashed, messages still are picked up and delivered by the rest of the system, improving its reliability, and allowing all receivers to see the same messages [CACHIN11].

Để hiện thực một phiên bản ngây thơ của quảng bá đáng tin cậy, chúng ta có thể dùng một bộ phát hiện lỗi và một cơ chế dự phòng. Cơ chế dự phòng trực tiếp nhất là cho phép mọi tiến trình đã nhận được thông điệp chuyển tiếp nó tới mọi tiến trình khác mà nó biết. Khi tiến trình nguồn thất bại, các tiến trình khác phát hiện lỗi và tiếp tục quảng bá thông điệp, về cơ bản làm tràn ngập mạng với N^2 thông điệp (như minh họa trong Hình 14-1). Ngay cả khi bên gửi đã sụp đổ, các thông điệp vẫn được phần còn lại của hệ thống tiếp nhận và chuyển giao, cải thiện độ tin cậy của nó, và cho phép tất cả các bên nhận thấy cùng các thông điệp [CACHIN11].

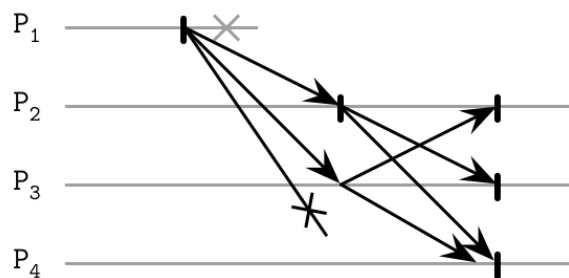


Figure 14-1. Broadcast

Hình 14-1. Quảng bá

One of the downsides of this approach is the fact that it uses N^2 messages, where N is the number of remaining recipients (since every broadcasting process excludes the original process and itself). Ideally, we'd want to reduce the number of messages required for a reliable broadcast.

Một trong những nhược điểm của cách tiếp cận này là việc nó dùng N^2 thông điệp, trong đó N là số người nhận còn lại (vì mỗi tiến trình quảng bá loại trừ tiến trình gốc và chính nó). Lý tưởng nhất, chúng ta muốn giảm số thông điệp cần thiết cho một quảng bá đáng tin cậy.

Atomic Broadcast — Quảng bá nguyên tử (Atomic Broadcast)

Even though the flooding algorithm just described can ensure message delivery, it does not guarantee delivery in any particular order. Messages reach their destination eventually, at an unknown time. If we need to deliver messages in order, we have to use the **atomic broadcast** (also called the total order multicast), which guarantees both reliable delivery and total order.

Mặc dù thuật toán làm tràn ngập vừa mô tả có thể đảm bảo việc chuyển giao thông điệp, nó không bảo đảm chuyển giao theo bất kỳ thứ tự cụ thể nào. Các thông điệp cuối cùng cũng tới đích, vào một thời điểm không xác định. Nếu chúng ta cần chuyển giao thông điệp theo thứ tự, chúng ta phải dùng quảng bá nguyên tử (atomic broadcast) (còn gọi là multicast thứ tự toàn cục), nó bảo đảm cả việc chuyển giao đáng tin cậy lẫn thứ tự toàn cục.

While a reliable broadcast ensures that the processes agree on the set of messages delivered, an atomic broadcast also ensures they agree on the same sequence of messages (i.e., message delivery

order is the same for every target).

Trong khi một quảng bá đáng tin cậy bảo đảm rằng các tiến trình thống nhất về tập hợp các thông điệp được chuyển giao, một quảng bá nguyên tử cũng bảo đảm chúng thống nhất về cùng một trình tự các thông điệp (tức là thứ tự chuyển giao thông điệp là như nhau với mọi đích).

In summary, an atomic broadcast has to ensure two essential properties:

Tóm lại, một quảng bá nguyên tử phải bảo đảm hai thuộc tính thiết yếu:

Atomicity Processes have to agree on the set of received messages. Either all nonfailed processes deliver the message, or none do.

Tính nguyên tử (Atomicity) Các tiến trình phải thống nhất về tập hợp các thông điệp đã nhận. Hoặc tất cả các tiến trình không bị lỗi đều chuyển giao thông điệp, hoặc không tiến trình nào chuyển giao.

Order All nonfailed processes deliver the messages in the same order.

Thứ tự (Order) Tất cả các tiến trình không bị lỗi chuyển giao các thông điệp theo cùng một thứ tự.

Messages here are delivered atomically : every message is either delivered to all processes or none of them and, if the message is delivered, every other message is ordered before or after this message.

Thông điệp ở đây được chuyển giao một cách nguyên tử: mỗi thông điệp hoặc được chuyển giao tới tất cả các tiến trình hoặc không tới tiến trình nào, và nếu thông điệp được chuyển giao, mọi thông điệp khác đều được sắp thứ tự trước hoặc sau thông điệp này.

Virtual Synchrony — Đồng bộ ảo (Virtual Synchrony)

One of the frameworks for group communication using broadcast is called **virtual synchrony** . An atomic broadcast helps to deliver totally ordered messages to a static group of processes, and virtual synchrony delivers totally ordered messages to a dynamic group of peers.

Một trong các khuôn khổ cho giao tiếp nhóm sử dụng quảng bá được gọi là đồng bộ ảo (virtual synchrony). Một quảng bá nguyên tử giúp chuyển giao các thông điệp được sắp thứ tự toàn phần tới một nhóm tiến trình tĩnh, còn đồng bộ ảo chuyển giao các thông điệp được sắp thứ tự toàn phần tới một nhóm peer động.

Virtual synchrony organizes processes into groups. As long as the group exists, messages are delivered to all of its members in the same order. In this case, the order is not specified by the model, and some implementations can take this to their advantage for performance gains, as long as the order they provide is consistent across all members [BIRMAN10] .

Đồng bộ ảo tổ chức các tiến trình thành các nhóm. Chừng nào nhóm còn tồn tại, các thông điệp được chuyển giao tới tất cả các thành viên của nó theo cùng một thứ tự. Trong trường hợp này, thứ tự không được mô hình quy định, và một số hiện thực có thể tận dụng điều này để đạt lợi ích về hiệu năng, miễn là thứ tự mà chúng cung cấp nhất quán trên tất cả các thành viên [BIRMAN10].

Processes have the same view of the group, and messages are associated with the group identity: processes can see the identical messages only as long as they belong to the same group.

Các tiến trình có cùng góc nhìn về nhóm, và các thông điệp được gắn với danh tính của nhóm: các tiến trình chỉ có thể nhìn thấy các thông điệp giống hệt nhau chừng nào chúng còn thuộc cùng một nhóm.

As soon as one of the participants joins, leaves the group, or fails and is forced out of it, the group view changes. This happens by announcing the group change to all its members. Each message is uniquely associated with the group it has originated from.

Ngay khi một trong các bên tham gia gia nhập, rời khỏi nhóm, hoặc bị lỗi và bị buộc ra khỏi nhóm, góc nhìn nhóm thay đổi. Điều này xảy ra bằng cách thông báo sự thay đổi nhóm tới tất cả các thành viên của nó. Mỗi thông điệp được gắn duy nhất với nhóm mà nó bắt nguồn.

Virtual synchrony distinguishes between the message receipt (when a group member receives the message) and its delivery (which happens when all the group members receive the message). If the message was sent in one view, it can be delivered only in the same view, which can be determined by comparing the current group with the group the message is associated with. Received messages remain pending in the queue until the process is notified about successful delivery.

Đồng bộ ảo phân biệt giữa việc tiếp nhận thông điệp (receipt) (khi một thành viên nhóm nhận được thông điệp) và việc chuyển giao (delivery) của nó (xảy ra khi tất cả các thành viên nhóm nhận được thông điệp). Nếu thông điệp được gửi trong một góc nhìn, nó chỉ có thể được chuyển giao trong cùng góc nhìn đó, điều này có thể được xác định bằng cách so sánh nhóm hiện tại với nhóm mà thông điệp gắn với. Các thông điệp đã nhận vẫn còn chờ trong hàng đợi cho tới khi tiến trình được thông báo về việc chuyển giao thành công.

Since every message belongs to a specific group, unless all processes in the group have received it before the view change, no group member can consider this message delivered . This implies that all messages are sent and delivered between the view changes, which gives us atomic delivery guarantees. In this case, group views serve as a barrier that message broadcasts cannot pass.

Vì mỗi thông điệp thuộc về một nhóm cụ thể, trừ khi tất cả các tiến trình trong nhóm đã nhận được nó trước khi góc nhìn thay đổi, không thành viên nhóm nào có thể coi thông điệp này là đã được chuyển giao. Điều này hàm ý rằng tất cả các thông điệp được gửi và chuyển giao giữa các lần thay đổi góc nhìn, điều này cho chúng ta các bảo đảm chuyển giao nguyên tử. Trong trường hợp này, các góc nhìn nhóm đóng vai trò như một rào chắn mà các lần quảng bá thông điệp không thể vượt qua.

Some total broadcast algorithms order messages by using a single process (sequencer) that is responsible for determining it. Such algorithms can be easier to implement, but rely on detecting the **leader** failures for liveness. Using a sequencer can improve performance, since we do not need to establish consensus between processes for every message, and can use a sequencer-local view instead. This approach can still scale by partitioning the requests.

Một số thuật toán quảng bá toàn cục sắp thứ tự các thông điệp bằng cách dùng một tiến trình đơn lẻ (sequencer) chịu trách nhiệm xác định thứ tự đó. Các thuật toán như vậy có thể dễ hiện thực hơn, nhưng dựa vào việc phát hiện lỗi của leader để bảo đảm tính sống còn. Dùng một sequencer có thể cải thiện hiệu năng, vì chúng ta không cần thiết lập đồng thuận giữa các tiến trình cho mỗi thông điệp, và có thể dùng góc nhìn cục bộ của sequencer thay vào đó. Cách tiếp cận này vẫn có thể mở rộng quy mô bằng cách phân hoạch các yêu cầu.

Despite its technical soundness, virtual synchrony has not received broad adoption and isn't commonly used in end-user commercial systems [BIRMAN06] .

Mặc dù vững chắc về mặt kỹ thuật, đồng bộ ảo đã không được áp dụng rộng rãi và không thường được dùng trong các hệ thương mại dành cho người dùng cuối [BIRMAN06].

Zookeeper Atomic Broadcast (ZAB) — Zookeeper Atomic Broadcast (ZAB)

One of the most popular and widely known implementations of the atomic broadcast is **ZAB** used by Apache Zookeeper [HUNT10] [JUNQUEIRA11], a hierarchical distributed key-value store, where it's used to ensure the total order of events and atomic delivery necessary to maintain consistency between the replica states.

Một trong những hiện thực phổ biến và được biết đến rộng rãi nhất của quảng bá nguyên tử là ZAB được dùng bởi Apache Zookeeper [HUNT10] [JUNQUEIRA11], một kho khóa-giá trị phân tán phân cấp, nơi nó được dùng để bảo đảm thứ tự toàn cục của các sự kiện và việc chuyển giao nguyên tử cần thiết để duy trì tính nhất quán giữa các trạng thái bản sao.

Processes in ZAB can take on one of two roles: leader and follower. Leader is a temporary role. It drives the process by executing algorithm steps, broadcasts messages to followers, and establishes the event order. To write new records and execute reads that observe the most recent values, clients connect to one of the nodes in the cluster. If the node happens to be a leader, it will handle the request. Otherwise, it forwards the request to the leader.

Các tiến trình trong ZAB có thể đảm nhận một trong hai vai trò: leader và follower. Leader là một vai trò tạm thời. Nó điều khiển tiến trình bằng cách thực hiện các bước thuật toán, quảng bá thông điệp tới các follower, và thiết lập thứ tự sự kiện. Để ghi các bản ghi mới và thực hiện các thao tác đọc quan sát được những giá trị mới nhất, các client kết nối tới một trong các nút trong cụm. Nếu nút đó tình cờ là một leader, nó sẽ xử lý yêu cầu. Ngược lại, nó chuyển tiếp yêu cầu tới leader.

To guarantee leader uniqueness, the protocol timeline is split into epochs, identified with a unique monotonically- and incrementally-sequenced number. During any epoch, there can be only one leader. The process starts from finding a prospective leader using any election algorithm, as long as it chooses a process that is up with a high probability. Since safety is guaranteed by the further algorithm steps, determining a prospective leader is more of a performance optimization. A prospective leader can also emerge as a consequence of the previous leader's failure.

Để bảo đảm tính duy nhất của leader, dòng thời gian của giao thức được chia thành các kỷ nguyên (epoch), được định danh bằng một số được đánh thứ tự tăng dần đơn điệu và duy nhất. Trong bất kỳ kỷ nguyên nào, chỉ có thể có một leader. Quá trình bắt đầu từ việc tìm một leader tiềm năng bằng bất kỳ thuật toán bầu chọn nào, miễn là nó chọn một tiến trình đang hoạt động với xác suất cao. Vì tính an toàn được bảo đảm bởi các bước tiếp theo của thuật toán, việc xác định một leader tiềm năng chỉ mang tính tối ưu hóa hiệu năng. Một leader tiềm năng cũng có thể xuất hiện như là hệ quả của việc lỗi của leader trước đó.

As soon as a prospective leader is established, it executes a protocol in three phases:

Ngay khi một leader tiềm năng được thiết lập, nó thực hiện một giao thức trong ba pha:

Discovery The prospective leader learns about the latest epoch known by every other process, and proposes a new epoch that is greater than the current epoch of any follower. Followers respond to the epoch proposal with the identifier of the latest transaction seen in the previous epoch. After this step, no process will accept broadcast proposals for the earlier epochs.

Khám phá (Discovery) Leader tiềm năng tìm hiểu về kỷ nguyên mới nhất mà mọi tiến trình khác biết, và đề xuất một kỷ nguyên mới lớn hơn kỷ nguyên hiện tại của bất kỳ follower nào. Các follower phản hồi đề xuất kỷ nguyên với định danh của giao dịch mới nhất được nhìn thấy trong kỷ nguyên trước. Sau bước này, không tiến trình nào sẽ chấp nhận các đề xuất quảng bá cho các kỷ nguyên trước đó.

Synchronization This phase is used to recover from the previous leader's failure and bring lagging followers up to speed. The prospective leader sends a message to the followers proposing itself as a leader for the new epoch and collects their acknowledgments. As soon as acknowledgments are received, the leader is established. After this step, followers will not accept attempts to become the epoch leader from any other processes. During synchronization, the new leader ensures that followers have the same history and delivers committed proposals from the established leaders of earlier epochs. These proposals are delivered before any proposal from the new epoch is delivered.

Đồng bộ hóa (Synchronization) Pha này được dùng để phục hồi từ lỗi của leader trước đó và đưa các follower bị tụt hậu lên kịp tiến độ. Leader tiềm năng gửi một thông điệp tới các follower đề xuất chính nó làm leader cho kỷ nguyên mới và thu thập các xác nhận của chúng. Ngay khi các xác nhận được nhận, leader được thiết lập. Sau bước này, các follower sẽ không chấp nhận các nỗ lực trở thành leader kỷ nguyên từ bất kỳ tiến trình nào khác. Trong lúc đồng bộ hóa, leader mới đảm bảo các follower có cùng lịch sử và chuyển giao các đề xuất đã được commit từ các leader đã được thiết lập của các kỷ nguyên trước đó. Các đề xuất này được chuyển giao trước khi bất kỳ đề xuất nào từ kỷ nguyên mới được chuyển giao.

Broadcast As soon as the followers are back in sync, active messaging starts. During this phase, the leader receives client messages, establishes their order, and broadcasts them to the followers: it sends a new proposal, waits for a **quorum** of followers to respond with acknowledgments and, finally, commits. This process is similar to a **two-phase commit** without aborts: votes are just acknowledgments, and the client cannot vote against a valid leader's proposal. However, proposals from the leaders from incorrect epochs should not be acknowledged. The broadcast phase continues until the leader crashes, is partitioned from the followers, or is suspected to be crashed due to the message delay.

Quảng bá (Broadcast) Ngay khi các follower trở lại đồng bộ, việc trao đổi thông điệp tích cực bắt đầu. Trong pha này, leader nhận các thông điệp của client, thiết lập thứ tự của chúng, và quảng bá chúng tới các follower: nó gửi một đề xuất mới, chờ một quorum các follower phản hồi với các xác nhận, và cuối cùng commit nó. Quá trình này tương tự commit hai pha (2PC) nhưng không có abort: các phiếu chỉ là xác nhận, và client không thể bỏ phiếu chống lại đề xuất của một leader hợp lệ. Tuy nhiên, các đề xuất từ các leader của các kỷ nguyên không đúng không nên được xác nhận. Pha quảng bá tiếp tục cho tới khi leader sụp đổ, bị phân vùng khỏi các follower, hoặc bị nghi ngờ đã sụp đổ do độ trễ thông điệp.

Figure 14-2 shows the three phases of the ZAB algorithm, and messages exchanged during each step.

Hình 14-2 thể hiện ba pha của thuật toán ZAB, và các thông điệp được trao đổi trong mỗi bước.

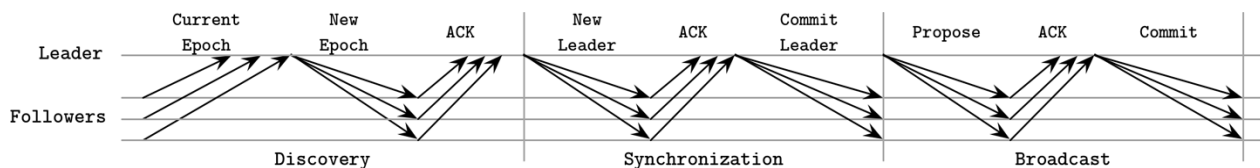


Figure 14-2. ZAB protocol summary

Hình 14-2. Tóm tắt giao thức ZAB

The safety of this protocol is guaranteed if followers ensure they accept proposals only from the leader of the established epoch. Two processes may attempt to get elected, but only one of them can win and establish itself as an epoch leader. It is also assumed that processes perform the prescribed steps in good faith and follow the protocol.

Tính an toàn của giao thức này được bảo đảm nếu các follower bảo đảm rằng chúng chỉ chấp nhận các đề xuất từ leader của kỷ nguyên đã được thiết lập. Hai tiến trình có thể cố gắng được bầu chọn, nhưng chỉ một trong số chúng có thể thắng và tự thiết lập mình làm leader kỷ nguyên. Người ta cũng giả định rằng các tiến trình thực hiện các bước được quy định một cách thiện chí và tuân theo giao thức.

Both the leader and followers rely on heartbeats to determine the liveness of the remote processes. If the leader does not receive heartbeats from the quorum of followers, it steps down as a leader, and restarts the election process. Similarly, if one of the followers has determined the leader crashed, it starts a new election process.

Cả leader lẫn follower đều dựa vào các nhịp tim (heartbeat) để xác định tính sống còn của các tiến trình từ xa. Nếu leader không nhận được nhịp tim từ quorum các follower, nó từ bỏ vai trò leader, và khởi động lại quá trình bầu chọn. Tương tự, nếu một trong các follower xác định rằng leader đã sập đổ, nó bắt đầu một quá trình bầu chọn mới.

Messages are totally ordered, and the leader will not attempt to send the next message until the message that preceded it was acknowledged. Even if some messages are received by a follower more than once, their repeated application do not produce additional side effects, as long as delivery order is followed. ZAB is able to handle multiple outstanding concurrent state changes from clients, since a unique leader will receive write requests, establish the event order, and broadcast the changes.

Các thông điệp được sắp thứ tự toàn phần, và leader sẽ không cố gắng gửi thông điệp kế tiếp cho tới khi thông điệp đứng trước nó đã được xác nhận. Ngay cả khi một số thông điệp được một follower nhận hơn một lần, việc áp dụng lặp lại chúng không tạo ra tác dụng phụ bổ sung nào, miễn là thứ tự chuyển giao được tuân theo. ZAB có khả năng xử lý nhiều thay đổi trạng thái tương tranh chưa hoàn thành từ các client, vì một leader duy nhất sẽ nhận các yêu cầu ghi, thiết lập thứ tự sự kiện, và quảng bá các thay đổi.

Total message order also allows ZAB to improve **recovery** efficiency. During the synchronization phase, followers respond with a highest committed proposal. The leader can simply choose the node with the highest proposal for recovery, and this can be the only node messages have to be copied from.

Thứ tự toàn cục của thông điệp cũng cho phép ZAB cải thiện hiệu quả phục hồi. Trong pha đồng bộ hóa, các follower phản hồi với đề xuất đã được commit cao nhất. Leader có thể đơn giản chọn nút có đề xuất cao nhất để phục hồi, và đây có thể là nút duy nhất mà các thông điệp phải được sao chép từ đó.

One of the advantages of ZAB is its efficiency: the broadcast process requires only two rounds of messages, and leader failures can be recovered from by streaming the missing messages from a single up-to-date process. Having a long-lived leader can have a positive impact on performance: we do not require additional consensus rounds to establish a history of events, since the leader can sequence them based on its local view.

Một trong những ưu điểm của ZAB là hiệu quả của nó: quá trình quảng bá chỉ yêu cầu hai vòng thông điệp, và các lỗi của leader có thể được phục hồi bằng cách truyền các thông điệp còn thiếu từ một tiến trình cập nhật duy nhất. Việc có một leader tồn tại lâu dài có

thể có tác động tích cực đến hiệu năng: chúng ta không cần các vòng đồng thuận bổ sung để thiết lập một lịch sử sự kiện, vì leader có thể sắp trình tự chúng dựa trên góc nhìn cục bộ của nó.

Paxos — Paxos

An atomic broadcast is a problem equivalent to consensus in an asynchronous system with crash failures [CHANDRA96], since participants have to agree on the message order and must be able to learn about it. You will see many similarities in both motivation and implementation between atomic broadcast and consensus algorithms.

Quảng bá nguyên tử là một bài toán tương đương với đồng thuận trong một hệ bất đồng bộ với các lỗi sụp đổ [CHANDRA96], vì các bên tham gia phải thống nhất về thứ tự thông điệp và phải có khả năng tìm hiểu về nó. Bạn sẽ thấy nhiều điểm tương đồng cả trong động cơ lẫn hiện thực giữa quảng bá nguyên tử và các thuật toán đồng thuận.

Probably the most widely known consensus algorithm is **Paxos**. It was first introduced by Leslie Lamport in “The Part-Time Parliament” paper [LAMPORT98]. In this paper, consensus is described in terms of terminology inspired by the legislative and voting process on the Aegian island of Paxos. In 2001, the author released a follow-up paper titled “Paxos Made Simple” [LAMPORT01] that introduced simpler terms, which are now commonly used to explain this algorithm.

Có lẽ thuật toán đồng thuận được biết đến rộng rãi nhất là Paxos. Nó được Leslie Lamport giới thiệu lần đầu trong bài báo “The Part-Time Parliament” [LAMPORT98]. Trong bài báo này, đồng thuận được mô tả bằng các thuật ngữ lấy cảm hứng từ quá trình lập pháp và bỏ phiếu trên hòn đảo Paxos ở vùng biển Aegean. Năm 2001, tác giả công bố một bài báo tiếp nối có tiêu đề “Paxos Made Simple” [LAMPORT01] giới thiệu các thuật ngữ đơn giản hơn, hiện đang được dùng phổ biến để giải thích thuật toán này.

Participants in Paxos can take one of three roles: proposers, acceptors, or learners:

Các bên tham gia trong Paxos có thể đảm nhận một trong ba vai trò: bên đề xuất (proposer), bên chấp nhận (acceptor), hoặc bên học (learner):

Proposers Receive values from clients, create proposals to accept these values, and attempt to collect votes from acceptors.

Bên đề xuất (Proposers) Nhận các giá trị từ client, tạo các đề xuất để chấp nhận các giá trị này, và cố gắng thu thập phiếu từ các bên chấp nhận.

Acceptors Vote to accept or reject the values proposed by the **proposer**. For fault tolerance, the algorithm requires the presence of multiple acceptors, but for liveness, only a quorum (majority) of **acceptor** votes is required to accept the proposal.

Bên chấp nhận (Acceptors) Bỏ phiếu để chấp nhận hoặc từ chối các giá trị được đề xuất bởi bên đề xuất. Để dung lỗi, thuật toán yêu cầu sự hiện diện của nhiều bên chấp nhận, nhưng để bảo đảm tính sống còn, chỉ cần một quorum (đa số) phiếu của bên chấp nhận là đủ để chấp nhận đề xuất.

Learners Take the role of replicas, storing the outcomes of the accepted proposals.

Bên học (Learners) Đảm nhận vai trò của các bản sao, lưu trữ kết quả của các đề xuất đã được chấp nhận.

Any **participant** can take any role, and most implementations colocate them: a single process can simultaneously be a proposer, an acceptor, and a **learner**.

Bất kỳ bên tham gia nào cũng có thể đảm nhận bất kỳ vai trò nào, và hầu hết các hiện thực đặt chúng cùng chỗ: một tiến trình đơn lẻ có thể đồng thời là một bên đề xuất, một bên chấp nhận, và một bên học.

Every proposal consists of a value , proposed by the client, and a unique monotonically increasing proposal number. This number is then used to ensure a total order of executed operations and establish happened-before/after relationships among them. Proposal numbers are often implemented using an (id, timestamp) pair, where node IDs are also comparable and can be used to break ties for timestamps.

Mỗi đề xuất bao gồm một giá trị (value), được đề xuất bởi client, và một số đề xuất tăng đơn điệu duy nhất. Số này sau đó được dùng để bảo đảm thứ tự toàn cục của các thao tác được thực hiện và thiết lập các quan hệ xảy-ra-trước/sau giữa chúng. Các số đề xuất thường được hiện thực bằng cặp (id, timestamp), trong đó các ID nút cũng có thể so sánh được và có thể được dùng để phân định khi các timestamp bằng nhau.

Paxos Algorithm — Thuật toán Paxos

The Paxos algorithm can be generally split into two phases: voting (or propose phase) and replication . During the voting phase, proposers compete to establish their leadership. During replication, the proposer distributes the value to the acceptors.

Thuật toán Paxos nói chung có thể được chia thành hai pha: bỏ phiếu (voting) (hay pha đề xuất) và nhân bản (replication). Trong pha bỏ phiếu, các bên đề xuất cạnh tranh để thiết lập vị thế lãnh đạo của mình. Trong lúc nhân bản, bên đề xuất phân phối giá trị tới các bên chấp nhận.

The proposer is an initial point of contact for the client. It receives a value that should be decided upon, and attempts to collect votes from the quorum of acceptors. When this is done, acceptors distribute the information about the agreed value to the learners, ratifying the result. Learners increase the replication factor of the value that's been agreed on.

Bên đề xuất là điểm tiếp xúc ban đầu cho client. Nó nhận một giá trị cần được quyết định, và cố gắng thu thập phiếu từ quorum các bên chấp nhận. Khi việc này hoàn tất, các bên chấp nhận phân phối thông tin về giá trị đã thống nhất tới các bên học, phê chuẩn kết quả. Các bên học làm tăng hệ số nhân bản của giá trị đã được thống nhất.

Only one proposer can collect the majority of votes. Under some circumstances, votes may get split evenly between the proposers, and neither one of them will be able to collect a majority during this round, forcing them to restart. We discuss this and other scenarios of competing proposers in “Failure Scenarios” on page 288 .

Chỉ một bên đề xuất có thể thu thập được đa số phiếu. Trong một số hoàn cảnh, phiếu có thể bị chia đều giữa các bên đề xuất, và không bên nào trong số chúng có thể thu thập được đa số trong vòng này, buộc chúng phải khởi động lại. Chúng ta thảo luận tình huống này và các kịch bản khác của các bên đề xuất cạnh tranh trong “Failure Scenarios” ở trang 288.

During the propose phase, the proposer sends a Prepare(n) message (where n is a proposal number) to a majority of acceptors and attempts to collect their votes.

Trong pha đề xuất, bên đề xuất gửi một thông điệp Prepare(n) (trong đó n là một số đề xuất) tới đa số các bên chấp nhận và cố gắng thu thập phiếu của chúng.

When the acceptor receives the prepare request, it has to respond, preserving the following invariants [LAMP-ORT01] :

Khi bên chấp nhận nhận được yêu cầu prepare, nó phải phản hồi, giữ gìn các bất biến sau [LAMP-ORT01]:

- If this acceptor hasn't responded to a prepare request with a higher sequence number yet, it promises that it will not accept any proposal with a lower sequence number.
- If this acceptor has already accepted (received an `Accept!(m,v)` message)
 - Nếu bên chấp nhận này chưa phản hồi một yêu cầu prepare nào với số trình tự cao hơn, nó hứa rằng nó sẽ không chấp nhận bất kỳ đề xuất nào với số trình tự thấp hơn.
 - Nếu bên chấp nhận này đã chấp nhận (đã nhận một thông điệp `Accept!(m,v)`)

any other proposal earlier, it responds with a `Promise(m, v)` message,

bất kỳ đề xuất nào khác trước đó, nó phản hồi với một thông điệp `Promise(m, v)`,

notifying the proposer that it has already accepted the proposal with a sequence number `m`.

thông báo cho bên đề xuất rằng nó đã chấp nhận đề xuất với số trình tự `m`.

- If this acceptor has already responded to a prepare request with a higher sequence number, it notifies the proposer about the existence of a higher-numbered proposal.
- Acceptor can respond to more than one prepare request, as long as the later one has a higher sequence number .
 - Nếu bên chấp nhận này đã phản hồi một yêu cầu prepare với số trình tự cao hơn, nó thông báo cho bên đề xuất về sự tồn tại của một đề xuất có số cao hơn.
 - Bên chấp nhận có thể phản hồi nhiều hơn một yêu cầu prepare, miễn là cái sau có số trình tự cao hơn.

During the replication phase, after collecting a majority of votes, the proposer can start the replication, where it commits the proposal by sending acceptors an `Accept!(n, v)` message with value `v` and proposal number `n`. `v` is the value associated with the highest-numbered proposal among the responses it received from acceptors, or any value of its own if their responses did not contain old accepted proposals.

Trong pha nhân bản, sau khi thu thập được đa số phiếu, bên đề xuất có thể bắt đầu nhân bản, trong đó nó commit đề xuất bằng cách gửi cho các bên chấp nhận một thông điệp `Accept!(n, v)` với giá trị `v` và số đề xuất `n`. `v` là giá trị gắn với đề xuất có số cao nhất trong số các phản hồi nó nhận được từ các bên chấp nhận, hoặc bất kỳ giá trị nào của riêng nó nếu các phản hồi của chúng không chứa các đề xuất cũ đã được chấp nhận.

The acceptor accepts the proposal with a number `n`, unless during the propose phase it has already responded to `Prepare(m)`, where `m` is greater than `n`. If the acceptor rejects the proposal, it notifies the proposer about it by sending the highest sequence number it has seen along with the request to help the proposer catch up [LAMP-ORT01].

Bên chấp nhận chấp nhận đề xuất với số `n`, trừ khi trong pha đề xuất nó đã phản hồi `Prepare(m)`, trong đó `m` lớn hơn `n`. Nếu bên chấp nhận từ chối đề xuất, nó thông báo cho bên đề xuất về điều đó bằng cách gửi số trình tự cao nhất mà nó đã thấy kèm theo yêu cầu để giúp bên đề xuất bắt kịp [LAMP-ORT01].

You can see a generalized depiction of a Paxos round in Figure 14-3.

Bạn có thể thấy một mô tả tổng quát hóa của một vòng Paxos trong Hình 14-3.

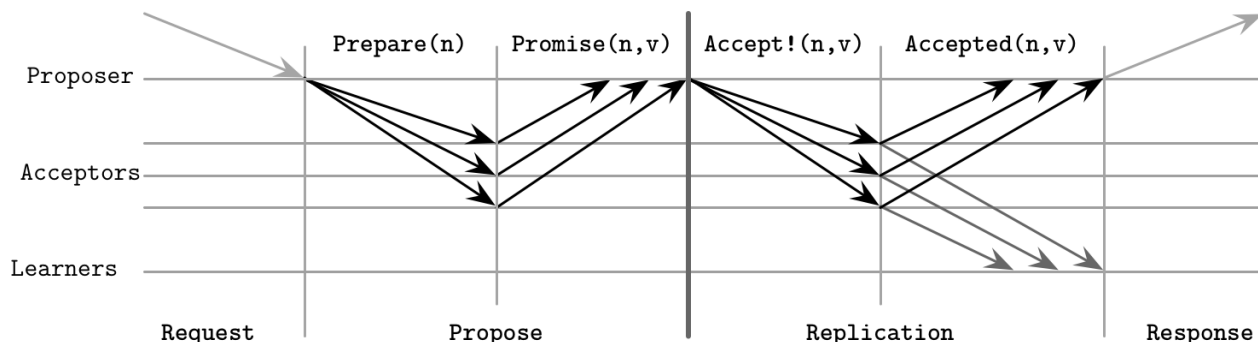


Figure 14-3. Paxos algorithm: normal execution

Hình 14-3. Thuật toán Paxos: thực thi thông thường

Once a consensus was reached on the value (in other words, it was accepted by at least one acceptor), future proposers have to decide on the same value to guarantee the agreement. This is why acceptors respond with the latest value they've accepted. If no acceptor has seen a previous value, the proposer is free to choose its own value.

Một khi đồng thuận đã đạt được trên một giá trị (nói cách khác, nó đã được chấp nhận bởi ít nhất một bên chấp nhận), các bên đề xuất tương lai phải quyết định trên cùng giá trị đó để bảo đảm sự thống nhất. Đây là lý do tại sao các bên chấp nhận phản hồi với giá trị mới nhất mà chúng đã chấp nhận. Nếu không bên chấp nhận nào đã thấy một giá trị trước đó, bên đề xuất được tự do chọn giá trị của riêng mình.

A learner has to find out the value that has been decided, which it can know after receiving notification from the majority of acceptors. To let the learner know about the new value as soon as possible, acceptors can notify it about the value as soon as they accept it. If there's more than one learner, each acceptor will have to notify each learner. One or more learners can be distinguished , in which case it will notify other learners about accepted values.

Một bên học phải tìm ra giá trị đã được quyết định, điều mà nó có thể biết sau khi nhận được thông báo từ đa số các bên chấp nhận. Để cho bên học biết về giá trị mới càng sớm càng tốt, các bên chấp nhận có thể thông báo cho nó về giá trị ngay khi chúng chấp nhận nó. Nếu có nhiều hơn một bên học, mỗi bên chấp nhận sẽ phải thông báo cho mỗi bên học. Một hoặc nhiều bên học có thể được phân biệt (distinguished), trong trường hợp đó nó sẽ thông báo cho các bên học khác về các giá trị đã được chấp nhận.

In summary, the goal of the first algorithm phase is to establish a leader for the round and understand which value is going to be accepted, allowing the leader to proceed with the second phase: broadcasting the value. For the purpose of the base algorithm, we assume that we have to perform both phases every time we'd like to decide on a value. In practice, we'd like to reduce the number of steps in the algorithm, so we allow the proposer to propose more than one value. We discuss this in more detail later in “**Multi-Paxos**” on page 291 .

Tóm lại, mục tiêu của pha thứ nhất của thuật toán là thiết lập một leader cho vòng và hiểu giá trị nào sẽ được chấp nhận, cho phép leader tiến hành với pha thứ hai: quảng bá giá trị. Đối với mục đích của thuật toán cơ sở, chúng ta giả định rằng chúng ta phải thực hiện cả hai pha mỗi lần chúng ta muốn quyết định một giá trị. Trong thực tế, chúng ta muốn giảm số bước trong thuật toán, nên chúng ta cho phép bên đề xuất đề xuất nhiều hơn một giá trị. Chúng ta thảo luận chi tiết hơn điều này sau trong “Multi-Paxos” ở trang 291.

Quorums in Paxos — Quorum trong Paxos

Quorums are used to make sure that some of the participants can fail, but we still can proceed as long as we can collect votes from the alive ones. A quorum is the minimum number of votes required for the operation to be performed. This number usually constitutes a majority of participants. The main idea behind quorums is that even if participants fail or happen to be separated by the **network partition**, there's at least one participant that acts as an arbiter, ensuring protocol correctness.

Quorum được dùng để bảo đảm rằng một số bên tham gia có thể bị lỗi, nhưng chúng ta vẫn có thể tiến hành miễn là chúng ta có thể thu thập phiếu từ những bên còn sống. Một quorum là số phiếu tối thiểu cần thiết để thao tác được thực hiện. Con số này thường tạo thành một đa số các bên tham gia. Ý tưởng chính đằng sau quorum là ngay cả khi các bên tham gia bị lỗi hoặc tình cờ bị tách ra bởi phân vùng mạng (network partition), thì luôn có ít nhất một bên tham gia đóng vai trò như một trọng tài, bảo đảm tính đúng đắn của giao thức.

Once a sufficient number of participants accept the proposal, the value is guaranteed to be accepted by the protocol, since any two majorities have at least one participant in common.

Một khi đủ số bên tham gia chấp nhận đề xuất, giá trị được bảo đảm sẽ được giao thức chấp nhận, vì bất kỳ hai đa số nào cũng có ít nhất một bên tham gia chung.

Paxos guarantees safety in the presence of any number of failures. There's no configuration that can produce incorrect or inconsistent states since this would contradict the definition of consensus.

Paxos bảo đảm tính an toàn trước bất kỳ số lượng lỗi nào. Không có cấu hình nào có thể tạo ra các trạng thái không đúng hoặc không nhất quán, vì điều này sẽ mâu thuẫn với định nghĩa của đồng thuận.

Liveness is guaranteed in the presence of f failed processes. For that, the protocol requires $2f + 1$ processes in total so that, if f processes happen to fail, there are still $f + 1$ processes able to proceed. By using quorums, rather than requiring the presence of all processes, Paxos (and other consensus algorithms) guarantee results even when f process failures occur. In “Flexible Paxos” on page 296, we talk about quorums in slightly different terms and describe how to build protocols requiring quorum intersection between algorithm steps only.

Tính sống còn được bảo đảm trước sự hiện diện của f tiến trình bị lỗi. Để làm được điều đó, giao thức yêu cầu tổng cộng $2f + 1$ tiến trình sao cho, nếu f tiến trình tình cờ bị lỗi, vẫn còn $f + 1$ tiến trình có khả năng tiến hành. Bằng cách dùng quorum, thay vì yêu cầu sự hiện diện của tất cả các tiến trình, Paxos (và các thuật toán đồng thuận khác) bảo đảm các kết quả ngay cả khi f lỗi tiến trình xảy ra. Trong “Flexible Paxos” ở trang 296, chúng ta nói về quorum theo những thuật ngữ hơi khác và mô tả cách xây dựng các giao thức chỉ yêu cầu giao của quorum giữa các bước thuật toán.



It is important to remember that quorums only describe the blocking properties of the system. To guarantee safety, for each step we have to wait for responses from at least a quorum of nodes. We can send proposals and accept commands to more nodes; we just do not have to wait for their responses to proceed. We may send messages to more nodes (some systems use speculative execution : issuing redundant queries that help to achieve the required response count in case of node failures), but to

guarantee liveness, we can proceed as soon as we hear from the quorum.

Quan trọng cần nhớ rằng quorum chỉ mô tả các thuộc tính chặn (blocking) của hệ thống. Để bảo đảm tính an toàn, ở mỗi bước chúng ta phải chờ phản hồi từ ít nhất một quorum các nút. Chúng ta có thể gửi các đề xuất và lệnh accept tới nhiều nút hơn; chúng ta chỉ không phải chờ phản hồi của chúng để tiến hành. Chúng ta có thể gửi thông điệp tới nhiều nút hơn (một số hệ thống dùng thực thi đầu cơ (speculative execution): phát đi các truy vấn dự thừa giúp đạt được số phản hồi cần thiết trong trường hợp nút bị lỗi), nhưng để bảo đảm tính sống còn, chúng ta có thể tiến hành ngay khi nghe được từ quorum.

Failure Scenarios — Các kịch bản lỗi

Discussing distributed algorithms gets particularly interesting when failures are discussed. One of the failure scenarios, demonstrating fault tolerance, is when the proposer fails during the second phase, before it is able to broadcast the value to all the acceptors (a similar situation can happen if the proposer is alive but is slow or cannot communicate with some acceptors). In this case, the new proposer may pick up and commit the value, distributing it to the other participants.

Việc thảo luận các thuật toán phân tán trở nên đặc biệt thú vị khi bàn tới các lỗi. Một trong các kịch bản lỗi, minh họa khả năng dung lỗi, là khi bên đề xuất bị lỗi trong pha thứ hai, trước khi nó có thể quảng bá giá trị tới tất cả các bên chấp nhận (một tình huống tương tự có thể xảy ra nếu bên đề xuất còn sống nhưng chậm hoặc không thể giao tiếp với một số bên chấp nhận). Trong trường hợp này, bên đề xuất mới có thể tiếp nhận và commit giá trị đó, phân phối nó tới các bên tham gia khác.

Figure 14-4 shows this situation:

Hình 14-4 thể hiện tình huống này:

- Proposer P goes through the election phase with a proposal number 1, but fails
 - Bên đề xuất P đi qua pha bầu chọn với số đề xuất 1, nhưng bị lỗi

after sending the value V1 to just one acceptor A.

sau khi gửi giá trị V1 chỉ tới một bên chấp nhận A.

- Another proposer P starts a new round with a higher proposal number 2, collects a quorum of acceptor responses (A and A in this case), and proceeds by
 - Một bên đề xuất khác P bắt đầu một vòng mới với số đề xuất cao hơn 2, thu

lects a quorum of acceptor responses (A and A in this case), and proceeds by

thập một quorum các phản hồi của bên chấp nhận (A và A trong trường hợp này), và tiến hành bằng cách

committing the old value V1, proposed by P.

commit giá trị cũ V1, được đề xuất bởi P.

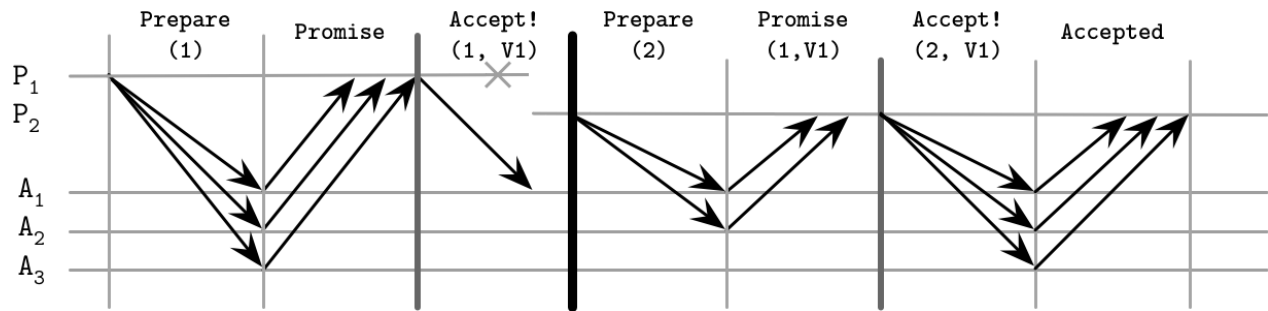


Figure 14-4. Paxos failure scenario: proposer failure, deciding on the old value

Hình 14-4. Kịch bản lỗi Paxos: lỗi bên đề xuất, quyết định trên giá trị cũ

Since the algorithm state is replicated to multiple nodes, proposer failure does not result in failure to reach a consensus. If the current proposer fails after even a single acceptor A has accepted the value, its proposal can be picked by the next proposer.

Vì trạng thái của thuật toán được nhân bản tới nhiều nút, lỗi của bên đề xuất không dẫn tới việc không đạt được đồng thuận. Nếu bên đề xuất hiện tại bị lỗi sau khi thậm chí chỉ một bên chấp nhận A đã chấp nhận giá trị, đề xuất của nó có thể được bên đề xuất kế tiếp tiếp nhận.

This also implies that all of it may happen without the original proposer knowing about it.

Điều này cũng hàm ý rằng tất cả những điều đó có thể xảy ra mà bên đề xuất ban đầu không hề biết.

In a client/server application, where the client is connected only to the original proposer, this might lead to situations where the client doesn't know about the result of the Paxos round execution. 1

Trong một ứng dụng client/server, nơi client chỉ kết nối với bên đề xuất ban đầu, điều này có thể dẫn tới các tình huống mà client không biết về kết quả của việc thực thi vòng Paxos. 1

However, other scenarios are possible, too, as Figure 14-5 shows. For example:

Tuy nhiên, các kịch bản khác cũng có thể xảy ra, như Hình 14-5 thể hiện. Ví dụ:

- P has failed just like in the previous example, after sending the value V1 only to

- P đã bị lỗi giống như trong ví dụ trước, sau khi gửi giá trị V1 chỉ tới

A.

A.

- The next proposer, P, starts a new round with a higher proposal number 2, and

- Bên đề xuất kế tiếp, P, bắt đầu một vòng mới với số đề xuất cao hơn 2, và

collects a quorum of acceptor responses, but this time A and A are first to

thu thập một quorum các phản hồi của bên chấp nhận, nhưng lần này A và A là những bên đầu tiên

respond. After collecting a quorum, P commits its own value despite the fact that

phản hồi. Sau khi thu thập được một quorum, P commit giá trị của riêng mình mặc dù thực tế là

theoretically there's a different committed value on A .

về mặt lý thuyết có một giá trị đã commit khác trên A.

1 For example, such a situation was described in <https://databass.dev/links/68> .

1 Ví dụ, một tình huống như vậy được mô tả trong <https://databass.dev/links/68> .

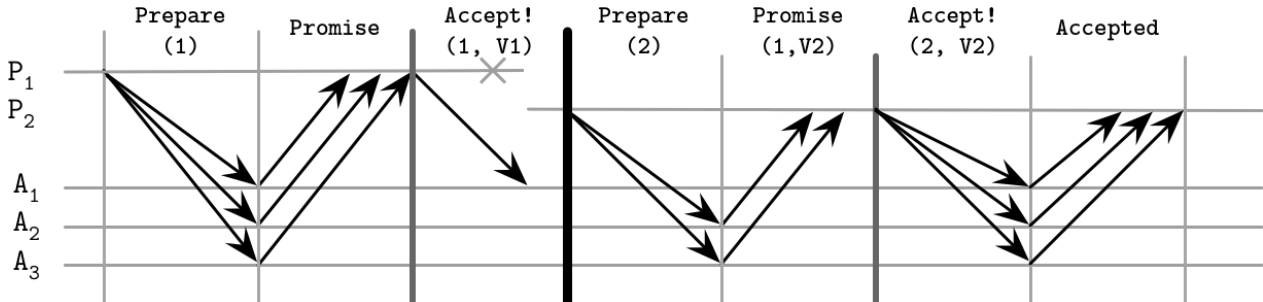


Figure 14-5. Paxos failure scenario: proposer failure, deciding on the new value

Hình 14-5. Kịch bản lỗi Paxos: lỗi bên đề xuất, quyết định trên giá trị mới

There's one more possibility here, shown in Figure 14-6 :

Còn một khả năng nữa ở đây, được thể hiện trong Hình 14-6:

- Proposer P fails after only one acceptor A accepts the value V1 . A fails shortly
 - Bên đề xuất P bị lỗi sau khi chỉ một bên chấp nhận A chấp nhận giá trị V1. A bị lỗi ngay

after accepting the proposal, before it can notify the next proposer about its value.

sau khi chấp nhận đề xuất, trước khi nó có thể thông báo cho bên đề xuất kế tiếp về giá trị của nó.

- Proposer P , which started the round after P failed, does not overlap with A and
 - Bên đề xuất P, vốn bắt đầu vòng sau khi P bị lỗi, không giao với A và

proceeds to commit its value instead.

tiến hành commit giá trị của riêng nó thay vào đó.

- Any proposer that comes after this round that will overlap with A , will ignore A 's
 - Bất kỳ bên đề xuất nào đến sau vòng này mà giao với A, sẽ bỏ qua giá trị của A

value and choose a more recent accepted proposal instead.

và chọn một đề xuất đã chấp nhận gần đây hơn thay vào đó.

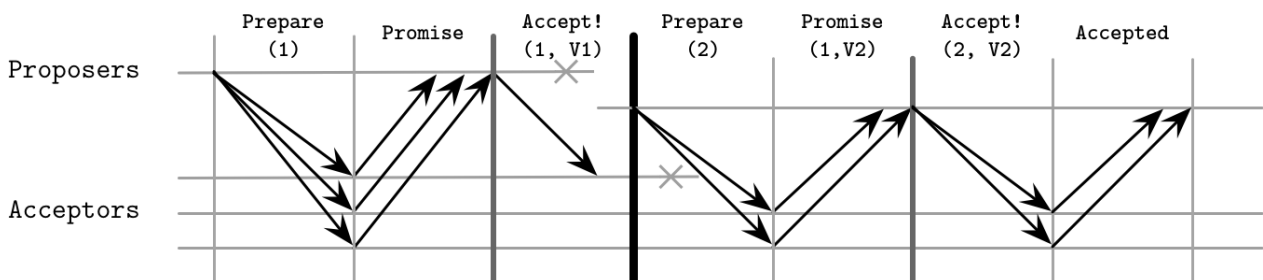


Figure 14-6. Paxos failure scenario: proposer failure, followed by the acceptor failure

Hình 14-6. Kịch bản lỗi Paxos: lỗi bên đề xuất, tiếp theo là lỗi bên chấp nhận

Another failure scenario is when two or more proposers start competing, each trying to get through the propose phase, but keep failing to collect a majority because the other one beat them to it.

Một kịch bản lỗi khác là khi hai hay nhiều bên đề xuất bắt đầu cạnh tranh, mỗi bên cố gắng vượt qua pha đề xuất, nhưng cứ liên tục thất bại trong việc thu thập đa số vì bên kia đã đánh bại chúng trong việc đó.

While acceptors promise not to accept any proposals with a lower number, they still may respond to multiple prepare requests, as long as the later one has a higher sequence number. When a proposer tries to commit the value, it might find that acceptors have already responded to a prepare request with a higher sequence number. This may lead to multiple proposers constantly retrying and preventing each other from further progress. This problem is usually solved by incorporating a random backoff, which eventually lets one of the proposers proceed while the other one sleeps.

Trong khi các bên chấp nhận hứa không chấp nhận bất kỳ đề xuất nào có số thấp hơn, chúng vẫn có thể phản hồi nhiều yêu cầu prepare, miễn là cái sau có số trình tự cao hơn. Khi một bên đề xuất cố commit giá trị, nó có thể phát hiện rằng các bên chấp nhận đã phản hồi một yêu cầu prepare với số trình tự cao hơn. Điều này có thể dẫn tới nhiều bên đề xuất liên tục thử lại và ngăn cản nhau tiến triển thêm. Vấn đề này thường được giải quyết bằng cách kết hợp một độ trễ ngẫu nhiên (random backoff), điều này cuối cùng cho phép một trong các bên đề xuất tiến hành trong khi bên kia ngủ.

The Paxos algorithm can tolerate acceptor failures, but only if there are still enough acceptors alive to form a majority.

Thuật toán Paxos có thể chịu đựng các lỗi của bên chấp nhận, nhưng chỉ khi vẫn còn đủ bên chấp nhận sống để tạo thành đa số.

Multi-Paxos — Multi-Paxos

So far we discussed the classic Paxos algorithm, where we pick an arbitrary proposer and attempt to start a Paxos round. One of the problems with this approach is that a propose round is required for each replication round that occurs in the system. Only after the proposer is established for the round, which happens after a majority of acceptors respond with a Promise to the proposer's Prepare, can it start the replication. To avoid repeating the propose phase and let the proposer reuse its recognized position, we can use Multi-Paxos, which introduces the concept of a leader: a distinguished proposer [LAMPORT01]. This is a crucial addition, significantly improving algorithm efficiency.

Cho đến giờ chúng ta đã thảo luận thuật toán Paxos cổ điển, nơi chúng ta chọn một bên đề xuất tùy ý và cố gắng bắt đầu một vòng Paxos. Một trong những vấn đề với cách tiếp cận này là một vòng đề xuất là cần thiết cho mỗi vòng nhân bản xảy ra trong hệ thống. Chỉ sau khi bên đề xuất được thiết lập cho vòng đó, điều xảy ra sau khi đa số các bên chấp nhận phản hồi bằng một Promise cho thông điệp Prepare của bên đề xuất, thì nó mới có thể bắt đầu nhân bản. Để tránh lặp lại pha đề xuất và cho phép bên đề xuất tái sử dụng vị thế đã được công nhận của nó, chúng ta có thể dùng Multi-Paxos, nó giới thiệu khái niệm leader: một bên đề xuất được phân biệt [LAMPORT01]. Đây là một bổ sung quan trọng, cải thiện đáng kể hiệu quả của thuật toán.

Having an established leader, we can skip the propose phase and proceed straight to replication: distributing a value and collecting acceptor acknowledgments.

Khi có một leader đã được thiết lập, chúng ta có thể bỏ qua pha đề xuất và tiến thẳng tới nhân bản: phân phối một giá trị và thu thập các xác nhận của bên chấp nhận.

In the classic Paxos algorithm, reads can be implemented by running a Paxos round that would collect any values from incomplete rounds if they're present. This has to be done because the last known proposer is not guaranteed to hold the most recent data, since there might have been a different proposer that has modified state without the proposer knowing about it.

Trong thuật toán Paxos cổ điển, các thao tác đọc có thể được hiện thực bằng cách chạy một vòng Paxos để thu thập bất kỳ giá trị nào từ các vòng chưa hoàn thành nếu chúng có mặt. Điều này phải được thực hiện vì bên đề xuất được biết đến cuối cùng không được bảo đảm giữ dữ liệu mới nhất, vì có thể đã có một bên đề xuất khác đã sửa đổi trạng thái mà bên đề xuất không hề biết.

A similar situation may occur in Multi-Paxos: we're trying to perform a **read** from the known leader after the other leader is already elected, returning stale data, which contradicts the **linearizability** guarantees of consensus. To avoid that and guarantee that no other process can successfully submit values, some Multi-Paxos implementations use leases. The leader periodically contacts the participants, notifying them that it is still alive, effectively prolonging its lease. Participants have to respond and allow the leader to continue operation, promising that they will not accept proposals from other leaders for the period of the lease [CHANDRA07].

Một tình huống tương tự có thể xảy ra trong Multi-Paxos: chúng ta đang cố gắng thực hiện một thao tác đọc từ leader đã biết sau khi leader khác đã được bầu chọn, trả về dữ liệu cũ, điều này mâu thuẫn với các bảo đảm tuyến tính hóa (linearizability) của đồng thuận. Để tránh điều đó và bảo đảm rằng không tiến trình nào khác có thể gửi giá trị thành công, một số hiện thực Multi-Paxos dùng hợp đồng thuê (lease). Leader định kỳ liên hệ với các bên tham gia, thông báo cho chúng rằng nó vẫn còn sống, về cơ bản kéo dài hợp đồng thuê của nó. Các bên tham gia phải phản hồi và cho phép leader tiếp tục hoạt động, hứa rằng chúng sẽ không chấp nhận các đề xuất từ các leader khác trong suốt thời gian của hợp đồng thuê [CHANDRA07].

Leases are not a correctness guarantee, but a performance optimization that allows reads from the active leader without collecting a quorum. To guarantee safety, leases rely on the bounded clock synchrony between the participants. If their clocks drift too much and the leader assumes its lease is still valid while other participants think its lease has expired, linearizability cannot be guaranteed.

Hợp đồng thuê không phải là một bảo đảm về tính đúng đắn, mà là một tối ưu hóa hiệu năng cho phép các thao tác đọc từ leader đang hoạt động mà không cần thu thập một quorum. Để bảo đảm tính an toàn, hợp đồng thuê dựa vào tính đồng bộ đồng hồ có giới hạn giữa các bên tham gia. Nếu đồng hồ của chúng trôi lệch quá nhiều và leader cho rằng hợp đồng thuê của nó vẫn còn hiệu lực trong khi các bên tham gia khác nghĩ rằng hợp đồng thuê của nó đã hết hạn, thì tính tuyến tính hóa không thể được bảo đảm.

Multi-Paxos is sometimes described as a replicated log of operations applied to some structure. The algorithm is oblivious to the semantics of this structure and is only concerned with consistently replicating values that will be appended to this log. To preserve the state in case of process crashes, participants keep a durable log of received messages.

Multi-Paxos đôi khi được mô tả như một nhật ký nhân bản các thao tác được áp dụng lên một cấu trúc nào đó. Thuật toán không quan tâm đến ngữ nghĩa của cấu trúc này và chỉ quan tâm đến việc nhân bản một cách nhất quán các giá trị sẽ được nối thêm vào nhật ký này. Để bảo toàn trạng thái trong trường hợp các tiến trình sụp đổ, các bên tham gia giữ một nhật ký bền vững các thông điệp đã nhận.

To prevent a log from growing indefinitely large, its contents should be applied to the aforementioned structure. After the log contents are synchronized with a primary structure, creating a snapshot, the log can be truncated. Log and state snapshots should be mutually consistent, and snapshot changes should be applied atomically with truncation of the log segment [CHANDRA07].

Để ngăn nhật ký phát triển lớn vô hạn, nội dung của nó nên được áp dụng lên cấu trúc đã nói ở trên. Sau khi nội dung nhật ký được đồng bộ với một cấu trúc chính, tạo ra một ảnh chụp (snapshot), nhật ký có thể được cắt ngắn. Nhật ký và các ảnh chụp trạng thái nên nhất quán lẫn nhau, và các thay đổi ảnh chụp nên được áp dụng một cách nguyên tử cùng với việc cắt ngắn đoạn nhật ký [CHANDRA07].

We can think of single-decree Paxos as a write-once register : we have a slot where we can put a value, and as soon as we've written the value there, no subsequent modifications are possible. During the first step, proposers compete for ownership of the register, and during the second phase, one of them writes the value. At the same time, Multi-Paxos can be thought of as an append-only log, consisting of a sequence of such values: we can write one value at a time, all values are strictly ordered, and we cannot modify already written values [RYSTSOV16]. There are examples of consensus algorithms that offer collections of read-modify-write registers and use state sharing rather than replicated state machines, such as Active Disk Paxos [CHOCKLER15] and CASPaxos [RYSTSOV18].

Chúng ta có thể nghĩ về Paxos đơn-quyết-định (single-decree) như một thanh ghi ghi-một-lần (write-once register): chúng ta có một khe nơi chúng ta có thể đặt một giá trị, và ngay khi chúng ta đã ghi giá trị ở đó, không có sửa đổi nào về sau là khả thi. Trong bước đầu tiên, các bên đề xuất cạnh tranh giành quyền sở hữu thanh ghi, và trong pha thứ hai, một trong số chúng ghi giá trị. Đồng thời, Multi-Paxos có thể được nghĩ tới như một nhật ký chỉ-nối-thêm (append-only log), gồm một trình tự các giá trị như vậy: chúng ta có thể ghi một giá trị tại một thời điểm, tất cả các giá trị được sắp thứ tự chặt chẽ, và chúng ta không thể sửa đổi các giá trị đã ghi [RYSTSOV16]. Có những ví dụ về các thuật toán đồng thuận cung cấp các tập hợp các thanh ghi đọc-sửa-ghi (read-modify-write) và dùng việc chia sẻ trạng thái thay vì các máy trạng thái nhân bản, chẳng hạn như Active Disk Paxos [CHOCKLER15] và CASPaxos [RYSTSOV18].

Fast Paxos — Fast Paxos

We can reduce the number of round-trips by one, compared to the classic Paxos algorithm, by letting any proposer contact acceptors directly rather than going through the leader. For this, we need to increase the quorum size to $2f + 1$ (where f is the number of processes allowed to fail), compared to $f + 1$ in classic Paxos, and a total number of acceptors to $3f + 1$ [JUNQUEIRA07]. This optimization is called **Fast Paxos** [LAMPORTE06].

Chúng ta có thể giảm số vòng khứ hồi đi một, so với thuật toán Paxos cổ điển, bằng cách cho phép bất kỳ bên đề xuất nào liên hệ trực tiếp với các bên chấp nhận thay vì đi qua leader. Để làm được điều này, chúng ta cần tăng kích thước quorum lên $2f + 1$ (trong đó f là số tiến trình được phép bị lỗi), so với $f + 1$ trong Paxos cổ điển, và tổng số bên chấp nhận lên $3f + 1$ [JUNQUEIRA07]. Tối ưu hóa này được gọi là Fast Paxos [LAMPORTE06].

The classic Paxos algorithm has a condition, where during the replication phase, the proposer can pick any value it has collected during the propose phase. Fast Paxos has two types of rounds: classic, where the algorithm proceeds the same way as the classic version, and fast, where it allows acceptors to accept other values.

Thuật toán Paxos cổ điển có một điều kiện, trong đó trong pha nhân bản, bên đề xuất có

thể chọn bất kỳ giá trị nào nó đã thu thập trong pha đề xuất. Fast Paxos có hai loại vòng: cổ điển (classic), nơi thuật toán tiến hành theo cùng cách như phiên bản cổ điển, và nhanh (fast), nơi nó cho phép các bên chấp nhận các giá trị khác.

While describing this algorithm, we will refer to the proposer that has collected a sufficient number of responses during the propose phase as a coordinator, and reserve **term** proposer for all other proposers. Some Fast Paxos descriptions say that clients can contact acceptors directly [ZHAO15].

Khi mô tả thuật toán này, chúng ta sẽ gọi bên đề xuất đã thu thập đủ số lượng phản hồi trong pha đề xuất là một bên điều phối (coordinator), và dành riêng thuật ngữ bên đề xuất cho tất cả các bên đề xuất khác. Một số mô tả Fast Paxos nói rằng các client có thể liên hệ trực tiếp với các bên chấp nhận [ZHAO15].

In a fast round, if the coordinator is permitted to pick its own value during the replication phase, it can instead issue a special Any message to acceptors. Acceptors, in this case, are allowed to treat any proposer's value as if it is a classic round and they received a message with this value from the coordinator. In other words, acceptors independently decide on values they receive from different proposers.

Trong một vòng nhanh, nếu bên điều phối được phép chọn giá trị của riêng nó trong pha nhân bản, thay vào đó nó có thể phát ra một thông điệp Any đặc biệt tới các bên chấp nhận. Các bên chấp nhận, trong trường hợp này, được phép coi giá trị của bất kỳ bên đề xuất nào như thể đó là một vòng cổ điển và chúng đã nhận một thông điệp với giá trị này từ bên điều phối. Nói cách khác, các bên chấp nhận độc lập quyết định trên các giá trị mà chúng nhận được từ các bên đề xuất khác nhau.

Figure 14-7 shows an example of classic and fast rounds in Fast Paxos. From the image it might look like the fast round has more execution steps, but keep in mind that in a classic round, in order to submit its value, the proposer would need to go through the coordinator to get its value committed.

Hình 14-7 thể hiện một ví dụ về các vòng cổ điển và nhanh trong Fast Paxos. Từ hình ảnh có thể trông như vòng nhanh có nhiều bước thực thi hơn, nhưng hãy nhớ rằng trong một vòng cổ điển, để gửi giá trị của mình, bên đề xuất sẽ cần đi qua bên điều phối để giá trị của nó được commit.

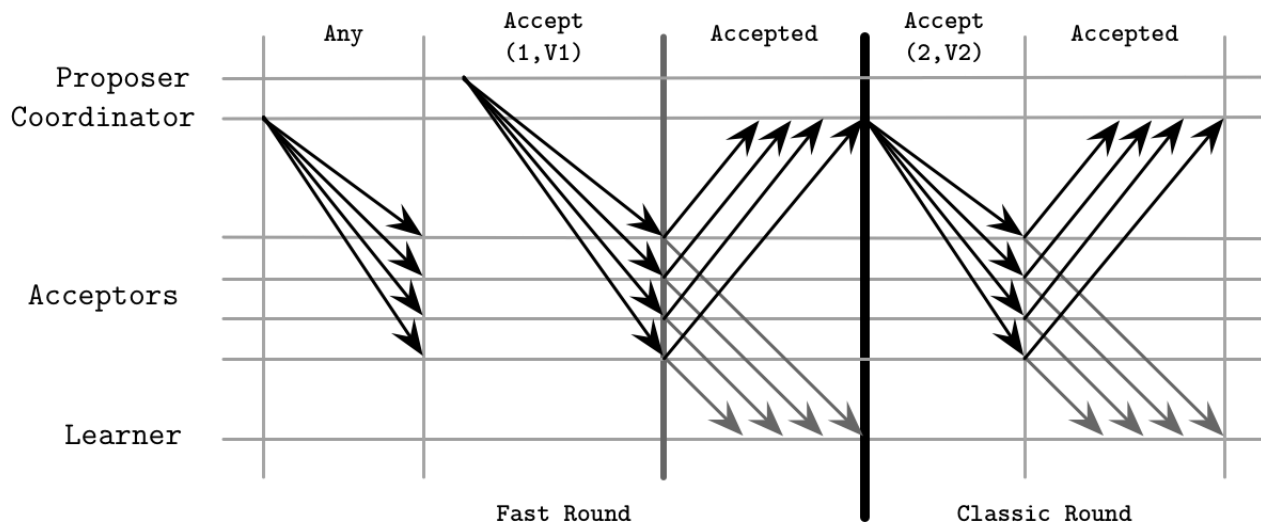


Figure 14-7. Fast Paxos algorithm: fast and classic rounds

Hình 14-7. Thuật toán Fast Paxos: các vòng nhanh và cổ điển

This algorithm is prone to collisions, which occur if two or more proposers attempt to use the fast step and reduce the number of round-trips, and acceptors receive different values. The coordinator has to intervene and start recovery by initiating a new round.

Thuật toán này dễ bị va chạm (collision), xảy ra nếu hai hay nhiều bên đề xuất cố gắng dùng bước nhanh và giảm số vòng khứ hồi, và các bên chấp nhận nhận được các giá trị khác nhau. Bên điều phối phải can thiệp và bắt đầu phục hồi bằng cách khởi tạo một vòng mới.

This means that acceptors, after receiving values from different proposers, may decide on conflicting values. When the coordinator detects a conflict (value collision), it has to reinitiate a Propose phase to let acceptors converge to a single value.

Điều này có nghĩa là các bên chấp nhận, sau khi nhận các giá trị từ các bên đề xuất khác nhau, có thể quyết định trên các giá trị xung đột. Khi bên điều phối phát hiện một xung đột (va chạm giá trị), nó phải khởi tạo lại một pha Propose để cho các bên chấp nhận hội tụ về một giá trị duy nhất.

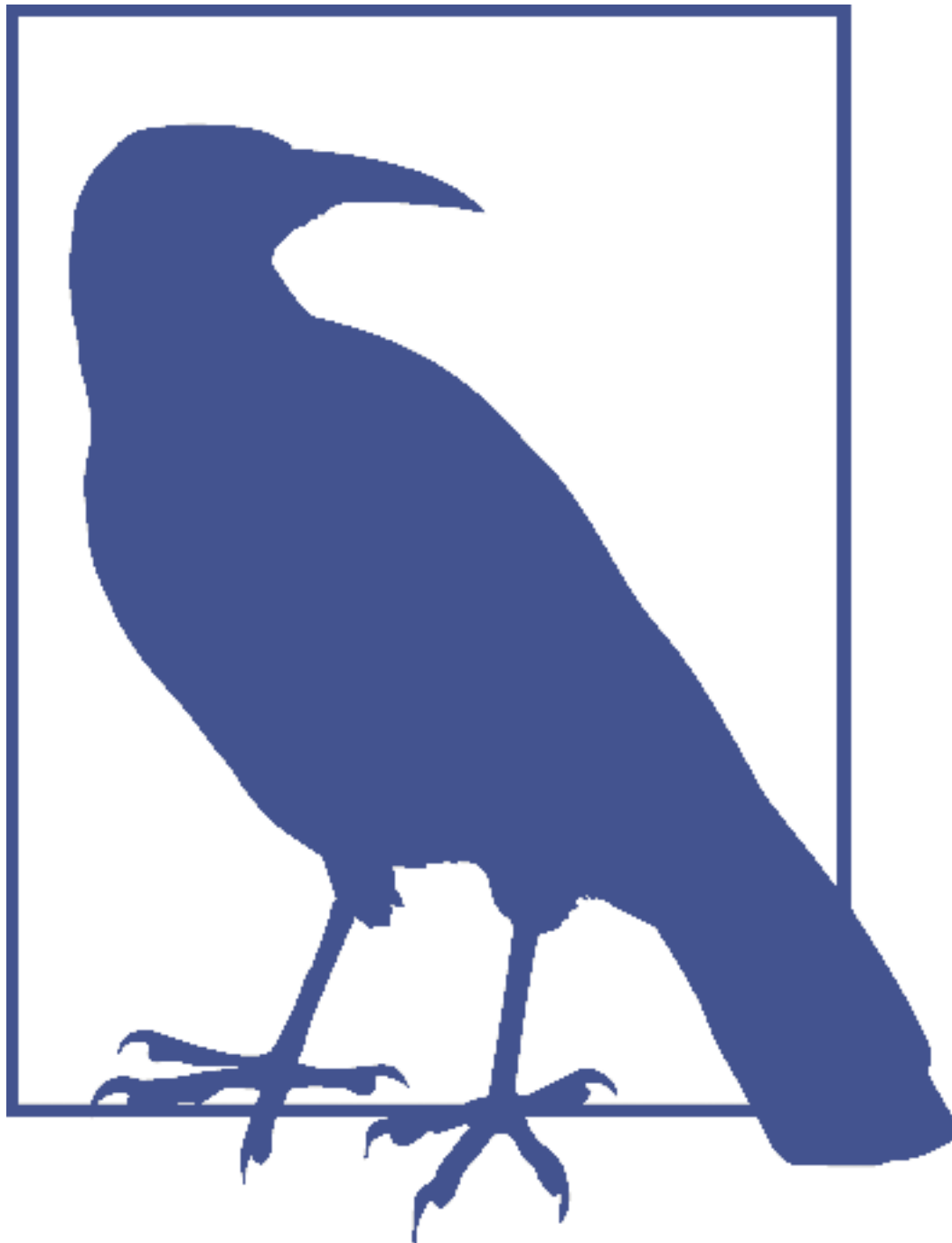
One of the disadvantages of Fast Paxos is the increased number of round-trips and request latency on collisions if the request rate is high. [JUNQUEIRA07] shows that, due to the increased number of replicas and, subsequently, messages exchanged between the participants, despite a reduced number of steps, Fast Paxos can have higher latencies than its classic counterpart.

Một trong những nhược điểm của Fast Paxos là số vòng khứ hồi và độ trễ yêu cầu tăng lên khi va chạm nếu tỷ lệ yêu cầu cao. [JUNQUEIRA07] cho thấy rằng, do số bản sao tăng lên và kéo theo đó là các thông điệp được trao đổi giữa các bên tham gia, mặc dù số bước giảm xuống, Fast Paxos có thể có độ trễ cao hơn so với phiên bản cổ điển tương ứng của nó.

Egalitarian Paxos — Egalitarian Paxos

Using a distinguished proposer as a leader makes a system prone to failures: as soon as the leader fails, the system has to elect a new one before it can proceed with further steps. Another problem is that having a leader can put a disproportionate load on it, impairing system performance.

Việc dùng một bên đề xuất được phân biệt làm leader khiến một hệ thống dễ bị lỗi: ngay khi leader bị lỗi, hệ thống phải bầu chọn một leader mới trước khi nó có thể tiến hành với các bước tiếp theo. Một vấn đề khác là việc có một leader có thể đặt một tải không cân xứng lên nó, làm suy giảm hiệu năng hệ thống.



One of the ways to avoid putting an entire system load on the leader is partitioning . Many systems split the range of possible values into smaller segments and allow a part of the system to be responsible for a specific range without having to worry about the other parts. This helps with availability (by isolating failures to a single partition and preventing propagation to other parts of the system), performance (since segments serving different values are nonoverlapping), and scalability (since we can scale the system by increasing the number of partitions). It is important to keep in mind that performing an operation against multiple partitions will require an atomic commitment.

Một trong những cách để tránh đặt toàn bộ tải hệ thống lên leader là phân hoạch

(partitioning). Nhiều hệ thống chia khoảng các giá trị có thể có thành các đoạn nhỏ hơn và cho phép một phần của hệ thống chịu trách nhiệm cho một khoảng cụ thể mà không phải lo lắng về các phần khác. Điều này giúp ích cho tính sẵn sàng (bằng cách cô lập các lỗi vào một phân hoạch duy nhất và ngăn chúng lan truyền sang các phần khác của hệ thống), hiệu năng (vì các đoạn phục vụ các giá trị khác nhau không chồng lấn), và khả năng mở rộng (vì chúng ta có thể mở rộng hệ thống bằng cách tăng số phân hoạch). Quan trọng cần nhớ rằng việc thực hiện một thao tác trên nhiều phân hoạch sẽ yêu cầu một sự cam kết nguyên tử.

Instead of using a leader and proposal numbers for sequencing commands, we can use a leader responsible for the commit of the specific command, and establish the order by looking up and setting dependencies. This approach is commonly called Egalitarian Paxos, or EPaxos [MORARU11]. The idea of allowing nonconflicting writes to be committed to the replicated state machine independently was first introduced in [LAMPORT05] and called Generalized Paxos. EPaxos is a first implementation of Generalized Paxos.

Thay vì dùng một leader và các số đề xuất để sắp trình tự các lệnh, chúng ta có thể dùng một leader chịu trách nhiệm cho việc commit của lệnh cụ thể, và thiết lập thứ tự bằng cách tra cứu và thiết lập các phụ thuộc. Cách tiếp cận này thường được gọi là Egalitarian Paxos, hay EPaxos [MORARU11]. Ý tưởng cho phép các thao tác ghi không xung đột được commit độc lập vào máy trạng thái nhân bản lần đầu được giới thiệu trong [LAMPORT05] và được gọi là Generalized Paxos. EPaxos là một hiện thực đầu tiên của Generalized Paxos.

EPaxos attempts to offer benefits of both the classic Paxos algorithm and Multi-Paxos. Classic Paxos offers high availability, since a leader is established during each round, but has a higher message complexity. Multi-Paxos offers high throughput and requires fewer messages, but a leader may become a bottleneck.

EPaxos cố gắng cung cấp các lợi ích của cả thuật toán Paxos cổ điển lẫn Multi-Paxos. Paxos cổ điển cung cấp tính sẵn sàng cao, vì một leader được thiết lập trong mỗi vòng, nhưng có độ phức tạp thông điệp cao hơn. Multi-Paxos cung cấp thông lượng cao và yêu cầu ít thông điệp hơn, nhưng một leader có thể trở thành nút thắt cổ chai.

EPaxos starts with a Pre-Accept phase, during which a process becomes a leader for the specific proposal. Every proposal has to include:

EPaxos bắt đầu với một pha Pre-Accept, trong đó một tiến trình trở thành leader cho đề xuất cụ thể. Mỗi đề xuất phải bao gồm:

Dependencies All commands that potentially interfere with a current proposal, but are not necessarily already committed.

Các phụ thuộc (Dependencies) Tất cả các lệnh có khả năng can thiệp vào một đề xuất hiện tại, nhưng không nhất thiết đã được commit.

A sequence number This breaks cycles between the dependencies. Set it with a value larger than any sequence number of the known dependencies.

Một số trình tự (A sequence number) Cái này phá vỡ các chu trình giữa các phụ thuộc. Đặt nó với một giá trị lớn hơn bất kỳ số trình tự nào của các phụ thuộc đã biết.

After collecting this information, it forwards a Pre-Accept message to a fast quorum of replicas. A fast quorum is $\lceil 3f/4 \rceil$ replicas, where f is the number of tolerated failures.

Sau khi thu thập thông tin này, nó chuyển tiếp một thông điệp Pre-Accept tới một quorum nhanh (fast quorum) các bản sao. Một quorum nhanh là $\lceil 3f/4 \rceil$ bản sao, trong đó f là số lỗi được chịu đựng.

Replicas check their local command logs, update the proposal dependencies based on their view of potentially conflicting proposals, and send this information back to the leader. If the leader receives responses from a fast quorum of replicas, and their dependency lists are in agreement with each other and the leader itself, it can commit the command.

Các bản sao kiểm tra nhật ký lệnh cục bộ của chúng, cập nhật các phụ thuộc của đề xuất dựa trên góc nhìn của chúng về các đề xuất có khả năng xung đột, và gửi thông tin này trở lại cho leader. Nếu leader nhận được phản hồi từ một quorum nhanh các bản sao, và các danh sách phụ thuộc của chúng thống nhất với nhau và với chính leader, nó có thể commit lệnh.

If the leader does not receive enough responses or if the command lists received from the replicas differ and contain interfering commands, it updates its proposal with a new dependency list and a sequence number. The new dependency list is based on previous replica responses and combines all collected dependencies. The new sequence number has to be larger than the highest sequence number seen by the replicas. After that, the leader sends the new, updated command to $\lfloor f/2 \rfloor + 1$ replicas. After this is done, the leader can finally commit the proposal.

Nếu leader không nhận được đủ phản hồi hoặc nếu các danh sách lệnh nhận từ các bản sao khác nhau và chứa các lệnh can thiệp, nó cập nhật đề xuất của mình với một danh sách phụ thuộc mới và một số trình tự. Danh sách phụ thuộc mới dựa trên các phản hồi trước đó của các bản sao và kết hợp tất cả các phụ thuộc đã thu thập. Số trình tự mới phải lớn hơn số trình tự cao nhất mà các bản sao đã thấy. Sau đó, leader gửi lệnh mới, đã cập nhật tới $\lfloor f/2 \rfloor + 1$ bản sao. Sau khi việc này hoàn tất, leader cuối cùng có thể commit đề xuất.

Effectively, we have two possible scenarios:

Trên thực tế, chúng ta có hai kịch bản khả dĩ:

Fast path When dependencies match and the leader can safely proceed with the commit phase with only a fast quorum of replicas.

Đường nhanh (Fast path) Khi các phụ thuộc khớp nhau và leader có thể tiến hành an toàn pha commit chỉ với một quorum nhanh các bản sao.

Slow path When there's a disagreement between the replicas, and their command lists have to be updated before the leader can proceed with a commit.

Đường chậm (Slow path) Khi có một sự bất đồng giữa các bản sao, và các danh sách lệnh của chúng phải được cập nhật trước khi leader có thể tiến hành một commit.

Figure 14-8 shows these scenarios— P initiating a fast path run, and P initiating a

Hình 14-8 thể hiện các kịch bản này — P khởi tạo một lần chạy đường nhanh, và P khởi tạo một

slow path run:

lần chạy đường chậm:

- P starts with proposal number 1 and no dependencies, and sends a PreAccept(1, $\emptyset$) message. Since the command logs of P and P are empty, P can proceed with a commit.
- P bắt đầu với số đề xuất 1 và không có phụ thuộc, và gửi một thông điệp PreAccept(1, $\emptyset$). Vì các nhật ký lệnh của P và P trống, P có thể tiến

hành với một commit.

- P creates a proposal with sequence number 2 . Since its command log is empty by

- P tạo một đề xuất với số trình tự 2. Vì nhật ký lệnh của nó trống tại

that point, it also declares no dependencies and sends a $\text{PreAccept}(2, \emptyset)$ message. P is not aware of the committed proposal 1 , but P notifies P about the

thời điểm đó, nó cũng tuyên bố không có phụ thuộc và gửi một thông điệp $\text{PreAccept}(2, \emptyset)$.
P không hề biết về đề xuất 1 đã được commit, nhưng P thông báo cho P về

conflict and sends its command log: {1} .

xung đột và gửi nhật ký lệnh của nó: {1}.

- P updates its local dependency list and sends a message to make sure replicas

- P cập nhật danh sách phụ thuộc cục bộ của nó và gửi một thông điệp để bảo đảm các bản sao

have the same dependencies: $\text{Accept}(2, \{1\})$. As soon as the replicas respond, it can commit the value.

có cùng các phụ thuộc: $\text{Accept}(2, \{1\})$. Ngay khi các bản sao phản hồi, nó có thể commit giá trị.

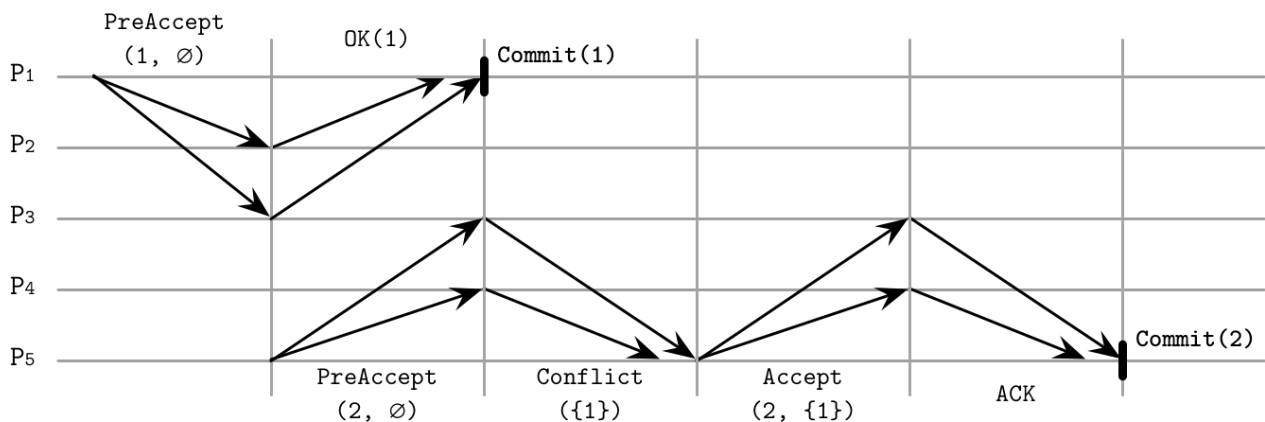


Figure 14-8. EPaxos algorithm run Two commands, A and B , interfere only if their execution order matters; in other words, if executing A before B and executing B before A produce different results.

Hình 14-8. Một lần chạy thuật toán EPaxos Hai lệnh, A và B, can thiệp lẫn nhau chỉ khi thứ tự thực thi của chúng quan trọng; nói cách khác, nếu việc thực thi A trước B và việc thực thi B trước A tạo ra các kết quả khác nhau.

Commit is done by responding to the client and asynchronously notifying replicas with a Commit message. Commands are executed after they're committed.

Việc commit được thực hiện bằng cách phản hồi cho client và thông báo bất đồng bộ cho các bản sao bằng một thông điệp Commit. Các lệnh được thực thi sau khi chúng được commit.

Since dependencies are collected during the Pre-Accept phase, by the time requests are executed, the command order is already established and no command can suddenly appear somewhere in-between: it can only get appended after the command with the largest sequence number.

Vì các phụ thuộc được thu thập trong pha Pre-Accept, đến lúc các yêu cầu được thực thi, thứ tự lệnh đã được thiết lập và không lệnh nào có thể đột nhiên xuất hiện ở đâu đó ở giữa: nó chỉ có thể được nối thêm sau lệnh có số trình tự lớn nhất.

To execute a command, replicas build a dependency graph and execute all commands in a reverse dependency order. In other words, before a command can be executed, all its dependencies (and, subsequently, all their dependencies) have to be executed. Since only interfering commands have to depend on each other, this situation should be relatively rare for most workloads [MORARU13].

Để thực thi một lệnh, các bản sao xây dựng một đồ thị phụ thuộc và thực thi tất cả các lệnh theo thứ tự phụ thuộc đảo ngược. Nói cách khác, trước khi một lệnh có thể được thực thi, tất cả các phụ thuộc của nó (và, kéo theo đó, tất cả các phụ thuộc của chúng) phải được thực thi. Vì chỉ các lệnh can thiệp lẫn nhau mới phải phụ thuộc lẫn nhau, tình huống này nên tương đối hiếm gặp đối với hầu hết các khối lượng công việc [MORARU13].

Similar to Paxos, EPaxos uses proposal numbers, which prevent stale messages from being propagated. Sequence numbers consist of an epoch (identifier of the current cluster configuration that changes when nodes leave and join the cluster), a monotonically incremented node-local counter, and a replica ID. If a replica receives a proposal with a sequence number lower than one it has already seen, it negatively acknowledges the proposal, and sends the highest sequence number and an updated command list known to it in response.

Tương tự như Paxos, EPaxos dùng các số đề xuất, vốn ngăn các thông điệp cũ bị lan truyền. Các số trình tự bao gồm một kỷ nguyên (định danh của cấu hình cụm hiện tại, thay đổi khi các nút rời và gia nhập cụm), một bộ đếm cục bộ của nút được tăng đơn điệu, và một ID bản sao. Nếu một bản sao nhận được một đề xuất với số trình tự thấp hơn cái nó đã thấy, nó xác nhận phủ định đề xuất, và gửi số trình tự cao nhất và một danh sách lệnh đã cập nhật mà nó biết để phản hồi.

Flexible Paxos — Flexible Paxos

A quorum is usually defined as a majority of processes. By definition, we have an intersection between two quorums no matter how we pick nodes: there's always at least one node that can break ties.

Một quorum thường được định nghĩa là một đa số các tiến trình. Theo định nghĩa, chúng ta có một giao giữa hai quorum bất kể chúng ta chọn các nút như thế nào: luôn có ít nhất một nút có thể phân định.

We have to answer two important questions:

Chúng ta phải trả lời hai câu hỏi quan trọng:

- Is it necessary to contact the majority of servers during every execution step?
- Do all quorums have to intersect? In other words, does a quorum we use to pick a distinguished proposer (first phase), a quorum we use to decide on a value (second phase), and every execution instance (for example, if multiple instances of the second step are executed concurrently), have to have nodes in common?
 - Có cần thiết phải liên hệ với đa số các máy chủ trong mỗi bước thực thi không?
 - Tất cả các quorum có phải giao nhau không? Nói cách khác, một quorum mà chúng ta dùng để chọn một bên đề xuất được phân biệt (pha thứ nhất), một quorum mà chúng ta dùng để quyết định trên một giá trị (pha thứ hai), và mọi thể hiện thực thi (ví dụ,

nếu nhiều thể hiện của bước thứ hai được thực thi tương tranh), có phải có các nút chung không?

Since we're still talking about consensus, we cannot change any safety definitions: the algorithm has to guarantee the agreement.

Vì chúng ta vẫn đang nói về đồng thuận, chúng ta không thể thay đổi bất kỳ định nghĩa an toàn nào: thuật toán phải bảo đảm sự thống nhất.

In Multi-Paxos, the leader election phase is infrequent, and the distinguished proposer is allowed to commit several values without rerunning the election phase, potentially staying in the lead for a longer period. In “**Tunable Consistency**” on page 235 , we discussed formulae that help us to find configurations where we have intersections between the node sets. One of the examples was to wait for just one node to acknowledge the write (and let the requests to the rest of nodes finish asynchronously), and read from all the nodes. In other words, as long as we keep $R + W > N$, there's at least one node in common between read and write sets.

Trong Multi-Paxos, pha bầu chọn leader là không thường xuyên, và bên đề xuất được phân biệt được phép commit nhiều giá trị mà không cần chạy lại pha bầu chọn, có khả năng giữ vị thế dẫn đầu trong một thời gian dài hơn. Trong “Tunable Consistency” ở trang 235, chúng ta đã thảo luận các công thức giúp chúng ta tìm các cấu hình nơi chúng ta có các giao giữa các tập nút. Một trong các ví dụ là chờ chỉ một nút xác nhận thao tác ghi (và để các yêu cầu tới phần còn lại của các nút hoàn thành bất đồng bộ), và đọc từ tất cả các nút. Nói cách khác, miễn là chúng ta giữ $R + W > N$, thì có ít nhất một nút chung giữa các tập đọc và ghi.

Can we use a similar logic for consensus? It turns out that we can, and in Paxos we only require the group of nodes from the first phase (that elects a leader) to overlap with the group from the second phase (that participates in accepting proposals).

Chúng ta có thể dùng một logic tương tự cho đồng thuận không? Hóa ra là chúng ta có thể, và trong Paxos chúng ta chỉ yêu cầu nhóm nút từ pha thứ nhất (vốn bầu chọn một leader) giao với nhóm từ pha thứ hai (vốn tham gia chấp nhận các đề xuất).

In other words, a quorum doesn't have to be defined as a majority, but only as a non-empty group of nodes. If we define a total number of participants as N , the number of nodes required for a propose phase to succeed as Q_1 , and the number of nodes required for the accept phase to succeed as Q_2 , we only need to ensure that $Q_1 + Q_2 > N$. Since the second phase is usually more common than the first one, Q_2 can contain only $N/2$ acceptors, as long as Q_1 is adjusted to be correspondingly larger ($Q_1 = N - Q_2 + 1$). This finding is an important observation crucial for understanding consensus. The algorithm that uses this approach is called Flexible Paxos [HOWARD16] .

Nói cách khác, một quorum không nhất thiết phải được định nghĩa là một đa số, mà chỉ là một nhóm nút không rỗng. Nếu chúng ta định nghĩa tổng số bên tham gia là N , số nút cần thiết để một pha đề xuất thành công là Q_1 , và số nút cần thiết để pha accept thành công là Q_2 , chúng ta chỉ cần bảo đảm rằng $Q_1 + Q_2 > N$. Vì pha thứ hai thường phổ biến hơn pha thứ nhất, Q_2 có thể chỉ chứa $N/2$ bên chấp nhận, miễn là Q_1 được điều chỉnh để lớn hơn tương ứng ($Q_1 = N - Q_2 + 1$). Phát hiện này là một quan sát quan trọng then chốt cho việc hiểu đồng thuận. Thuật toán dùng cách tiếp cận này được gọi là Flexible Paxos [HOWARD16].

For example, if we have five acceptors, as long as we require collecting votes from four of them to win the election round, we can allow the leader to wait for responses from two nodes during the replication stage. Moreover, since there's an overlap between any subset consisting of two acceptors with the leader election quorum, we can submit proposals to disjoint sets of acceptors. Intuitively,

this works because whenever a new leader is elected without the current one being aware of it, there will always be at least one acceptor that knows about the existence of the new leader.

Ví dụ, nếu chúng ta có năm bên chấp nhận, miễn là chúng ta yêu cầu thu thập phiếu từ bốn trong số chúng để thắng vòng bầu chọn, chúng ta có thể cho phép leader chờ phần hồi từ hai nút trong giai đoạn nhân bản. Hơn nữa, vì có một giao giữa bất kỳ tập con nào gồm hai bên chấp nhận với quorum bầu chọn leader, chúng ta có thể gửi các đề xuất tới các tập bên chấp nhận rời nhau. Một cách trực giác, điều này hoạt động vì bất cứ khi nào một leader mới được bầu chọn mà leader hiện tại không hề biết về nó, sẽ luôn có ít nhất một bên chấp nhận biết về sự tồn tại của leader mới.

Flexible Paxos allows trading availability for latency: we reduce the number of nodes participating in the second phase but have to collect more votes, requiring more participants to be available during the leader election phase. The good news is that this configuration can continue the replication phase and tolerate failures of up to $N - Q_2$ nodes, as long as the current leader is stable and a new election round is not required.

Flexible Paxos cho phép đánh đổi tính sẵn sàng lấy độ trễ: chúng ta giảm số nút tham gia trong pha thứ hai nhưng phải thu thập nhiều phiếu hơn, yêu cầu nhiều bên tham gia hơn phải sẵn sàng trong pha bầu chọn leader. Tin tốt là cấu hình này có thể tiếp tục pha nhân bản và chịu đựng các lỗi của tối đa $N - Q_2$ nút, miễn là leader hiện tại ổn định và một vòng bầu chọn mới không được yêu cầu.

Another Paxos variant using the idea of intersecting quorums is Vertical Paxos. Vertical Paxos distinguishes between read and write quorums. These quorums must intersect. A leader has to collect a smaller read quorum for one or more lower-numbered proposals, and a larger write quorum for its own proposal [LAMPORT09]. [LAMP-SON01] also distinguishes between the out and decision quorums, which translate to prepare and accept phases, and gives a quorum definition similar to Flexible Paxos.

Một biến thể Paxos khác dùng ý tưởng các quorum giao nhau là Vertical Paxos. Vertical Paxos phân biệt giữa các quorum đọc và ghi. Các quorum này phải giao nhau. Một leader phải thu thập một quorum đọc nhỏ hơn cho một hoặc nhiều đề xuất có số thấp hơn, và một quorum ghi lớn hơn cho đề xuất của riêng nó [LAMPORT09]. [LAMPSON01] cũng phân biệt giữa các quorum out và decision, vốn tương ứng với các pha prepare và accept, và đưa ra một định nghĩa quorum tương tự như Flexible Paxos.

Generalized Solution to Consensus — Lời giải tổng quát cho Đồng thuận

Paxos might sometimes be a bit difficult to reason about: multiple roles, steps, and all the possible variations are hard to keep **track** of. But we can think of it in simpler terms. Instead of splitting roles between the participants and having decision rounds, we can use a simple set of concepts and rules to achieve guarantees of a single-decree Paxos. We discuss this approach only briefly as this is a relatively new development [HOWARD19] —it's important to know, but we've yet to see its implementations and practical applications.

Paxos đôi khi có thể hơi khó để lập luận về nó: nhiều vai trò, bước, và tất cả các biến thể khả dĩ thì khó theo dõi. Nhưng chúng ta có thể nghĩ về nó theo những thuật ngữ đơn giản hơn. Thay vì phân chia các vai trò giữa các bên tham gia và có các vòng quyết định, chúng ta có thể dùng một tập đơn giản các khái niệm và quy tắc để đạt được các bảo đảm của Paxos đơn-quyết-định. Chúng ta chỉ thảo luận ngắn gọn cách tiếp cận này vì đây là một

phát triển tương đối mới [HOWARD19] — điều quan trọng là phải biết, nhưng chúng ta vẫn chưa thấy các hiện thực và các ứng dụng thực tế của nó.

We have a client and a set of servers. Each server has multiple registers . A register has an index identifying it, can be written only once, and it can be in one of three states: unwritten, containing a value , and containing nil (a special empty value).

Chúng ta có một client và một tập các máy chủ. Mỗi máy chủ có nhiều thanh ghi (register). Một thanh ghi có một chỉ số định danh nó, chỉ có thể được ghi một lần, và nó có thể ở một trong ba trạng thái: chưa ghi, chứa một giá trị, và chứa nil (một giá trị rỗng đặc biệt).

Registers with the same index located on different servers form a register set . Each register set can have one or more quorums. Depending on the state of the registers in it, a quorum can be in one of the undecided (Any and Maybe v), or decided (None and Decided v) states:

Các thanh ghi có cùng chỉ số nằm trên các máy chủ khác nhau tạo thành một tập thanh ghi (register set). Mỗi tập thanh ghi có thể có một hoặc nhiều quorum. Tùy thuộc vào trạng thái của các thanh ghi trong nó, một quorum có thể ở một trong các trạng thái chưa-quyết-định (Any và Maybe v), hoặc đã-quyết-định (None và Decided v):

Depending on future operations, this quorum set can decide on any value.

Tùy thuộc vào các thao tác tương lai, tập quorum này có thể quyết định trên bất kỳ giá trị nào.

If this quorum reaches a decision, its decision can only be v .

Nếu quorum này đạt tới một quyết định, quyết định của nó chỉ có thể là v.

This quorum cannot decide on the value.

Quorum này không thể quyết định trên giá trị.

This quorum has decided on the value v .

Quorum này đã quyết định trên giá trị v.

The client exchanges messages with the servers and maintains a state table, where it keeps track of values and registers, and can infer decisions made by the quorums.

Client trao đổi thông điệp với các máy chủ và duy trì một bảng trạng thái, nơi nó theo dõi các giá trị và thanh ghi, và có thể suy ra các quyết định đã được đưa ra bởi các quorum.

To maintain correctness, we have to limit how clients can interact with servers and which values they may write and which they may not. In terms of reading values, the client can output the decided value only if it has read it from the quorum of servers in the same register set.

Để duy trì tính đúng đắn, chúng ta phải giới hạn cách các client có thể tương tác với các máy chủ và các giá trị nào chúng có thể ghi và không thể ghi. Về mặt đọc các giá trị, client chỉ có thể xuất ra giá trị đã quyết định nếu nó đã đọc nó từ quorum các máy chủ trong cùng một tập thanh ghi.

The writing rules are slightly more involved because to guarantee algorithm safety, we have to preserve several invariants. First, we have to make sure that the client doesn't just come up with new values: it is allowed to write a specific value to the register only if it has received it as input or has read it from a register. Clients cannot write values that allow different quorums in the same register to decide on different values. Lastly, clients cannot write values that override previous decisions made in the previous register sets (decisions made in register sets up to $r - 1$ have to be None , Maybe v , or

Các quy tắc ghi hơi phức tạp hơn một chút vì để bảo đảm tính an toàn của thuật toán, chúng ta phải giữ gìn một số bất biến. Thứ nhất, chúng ta phải bảo đảm rằng client không tự bịa ra các giá trị mới: nó chỉ được phép ghi một giá trị cụ thể vào thanh ghi nếu nó đã nhận giá trị đó làm đầu vào hoặc đã đọc nó từ một thanh ghi. Các client không thể ghi các giá trị cho phép các quorum khác nhau trong cùng một thanh ghi quyết định trên các giá trị khác nhau. Cuối cùng, các client không thể ghi các giá trị ghi đè lên các quyết định trước đó đã được đưa ra trong các tập thanh ghi trước đó (các quyết định được đưa ra trong các tập thanh ghi đến tận $r - 1$ phải là None, Maybe v, hoặc

Decided v).

Generalized Paxos algorithm

Thuật toán Generalized Paxos

Putting all these rules together, we can implement a generalized Paxos algorithm that achieves consensus over a single value using write-once registers [HOWARD19]. Let's say we have three servers $[S_0, S_1, S_2]$, registers $[R_0, R_1, \dots]$, and clients $[C_0, C_1, \dots]$, where the client can only write to the assigned subset of registers. We use simple majority quorums for all registers ($\{S_0, S_1\}$, $\{S_0, S_2\}$, $\{S_1, S_2\}$).

Ghép tất cả các quy tắc này lại với nhau, chúng ta có thể hiện thực một thuật toán generalized Paxos đạt được đồng thuận trên một giá trị đơn lẻ bằng cách dùng các thanh ghi ghi-một-lần [HOWARD19]. Giả sử chúng ta có ba máy chủ $[S_0, S_1, S_2]$, các thanh ghi $[R_0, R_1, \dots]$, và các client $[C_0, C_1, \dots]$, trong đó client chỉ có thể ghi vào tập con các thanh ghi được gán cho nó. Chúng ta dùng các quorum đa số đơn giản cho tất cả các thanh ghi ($\{S_0, S_1\}$, $\{S_0, S_2\}$, $\{S_1, S_2\}$).

The decision process here consists of two phases. The first phase ensures that it is safe to write a value to the register, and the second phase writes the value to the register:

Quá trình quyết định ở đây gồm hai pha. Pha thứ nhất bảo đảm rằng việc ghi một giá trị vào thanh ghi là an toàn, và pha thứ hai ghi giá trị vào thanh ghi:

During phase 1 The client checks if the register it is about to write is unwritten by sending a P1 (register) command to the server. If the register is unwritten, all registers

Trong pha 1 Client kiểm tra xem thanh ghi mà nó sắp ghi có chưa được ghi hay không bằng cách gửi một lệnh P1(register) tới máy chủ. Nếu thanh ghi chưa được ghi, tất cả các thanh ghi

up to register - 1 are set to nil, which prevents clients from writing to previous registers. The server responds with a set of registers written so far. If it receives responses from the majority of servers, the client chooses either the nonempty value from the register with the largest index or its own value in case no value is present. Otherwise, it restarts the first phase.

đến tận register - 1 được đặt thành nil, điều này ngăn các client ghi vào các thanh ghi trước đó. Máy chủ phản hồi với một tập các thanh ghi đã được ghi cho đến giờ. Nếu nó nhận được phản hồi từ đa số các máy chủ, client chọn hoặc giá trị khác rỗng từ thanh ghi có chỉ số lớn nhất hoặc giá trị của riêng nó trong trường hợp không có giá trị nào hiện diện. Ngược lại, nó khởi động lại pha thứ nhất.

During phase 2 The client notifies all servers about the value it has picked during the first phase by sending them P2 (register, value). If the majority of servers respond to

Trong pha 2 Client thông báo cho tất cả các máy chủ về giá trị mà nó đã chọn trong pha thứ nhất bằng cách gửi cho chúng P2(register, value). Nếu đa số các máy chủ phản hồi

this message, it can output the decision value. Otherwise, it starts again from phase 1.

thông điệp này, nó có thể xuất ra giá trị quyết định. Ngược lại, nó bắt đầu lại từ pha 1.

Figure 14-9 shows this generalization of Paxos (adapted from [HOWARD19]). Client C0 tries to commit value V. During the first step, its state table is empty, and servers S0 and S1 respond with the empty register set, indicating that no registers were written so far. During the second step, it can submit its value V, since no other value was written.

Hình 14-9 thể hiện sự tổng quát hóa này của Paxos (phỏng theo [HOWARD19]). Client C0 cố commit giá trị V. Trong bước đầu tiên, bảng trạng thái của nó trống, và các máy chủ S0 và S1 phản hồi với tập thanh ghi rỗng, cho thấy rằng không thanh ghi nào được ghi cho đến giờ. Trong bước thứ hai, nó có thể gửi giá trị V của nó, vì không giá trị nào khác đã được ghi.

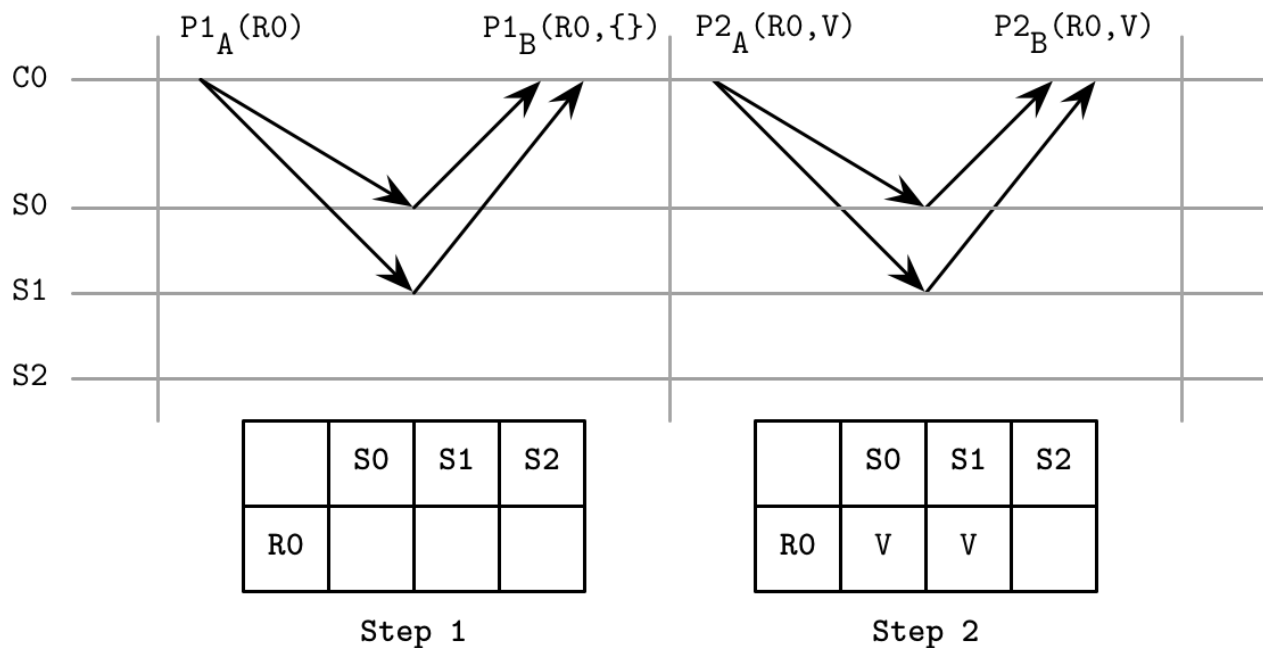


Figure 14-9. Generalization of Paxos At that point, any other client can query servers to find out the current state. Quorum {S0, S1} has reached Decided A state, and quorums {S0, S2} and {S1, S2} have reached the Maybe V state for R0, so C1 chooses the value V. At that point, no client can decide on a value other than V.

Hình 14-9. Sự tổng quát hóa của Paxos Tại thời điểm đó, bất kỳ client nào khác có thể truy vấn các máy chủ để tìm ra trạng thái hiện tại. Quorum {S0, S1} đã đạt tới trạng thái Decided A, và các quorum {S0, S2} và {S1, S2} đã đạt tới trạng thái Maybe V cho R0, nên C1 chọn giá trị V. Tại thời điểm đó, không client nào có thể quyết định trên một giá trị khác V.

This approach helps to understand the semantics of Paxos. Instead of thinking about the state from the perspective of interactions of remote actors (e.g., a proposer finding out whether or not an acceptor has already accepted a different proposal), we can think in terms of the last known state, making our decision process simple and removing possible ambiguities. **Immutable** state and **message passing** can also be easier to implement correctly.

Cách tiếp cận này giúp hiểu ngữ nghĩa của Paxos. Thay vì nghĩ về trạng thái từ góc nhìn của các tương tác của các tác nhân từ xa (ví dụ, một bên đề xuất tìm hiểu xem một bên chấp nhận đã chấp nhận một đề xuất khác hay chưa), chúng ta có thể nghĩ theo các thuật ngữ về trạng thái được biết đến cuối cùng, làm cho quá trình quyết định của chúng ta đơn

giản và loại bỏ các mơ hồ khả dĩ. Trạng thái bất biến và truyền thông điệp cũng có thể dễ hiện thực một cách đúng đắn hơn.

We can also draw parallels with original Paxos. For example, in a scenario in which the client finds that one of the previous register sets has the Maybe V decision, it picks up V and attempts to commit it again, which is similar to how a proposer in Paxos can propose the value after the failure of the previous proposer that was able to commit the value to at least one acceptor. Similarly, if in Paxos leader conflicts are resolved by restarting the vote with a higher proposal number, in the generalized algorithm any unwritten lower-ranked registers are set to nil .

Chúng ta cũng có thể vẽ những điểm tương đồng với Paxos gốc. Ví dụ, trong một kịch bản mà client phát hiện rằng một trong các tập thanh ghi trước đó có quyết định Maybe V, nó tiếp nhận V và cố commit nó lại, điều này tương tự cách một bên đề xuất trong Paxos có thể đề xuất giá trị sau lỗi của bên đề xuất trước đó vốn đã có thể commit giá trị tới ít nhất một bên chấp nhận. Tương tự, nếu trong Paxos các xung đột leader được giải quyết bằng cách khởi động lại cuộc bỏ phiếu với một số đề xuất cao hơn, thì trong thuật toán tổng quát hóa bất kỳ thanh ghi có hạng thấp hơn nào chưa được ghi đều được đặt thành nil.

Raft — Raft

Paxos was the consensus algorithm for over a decade, but in the distributed systems community it's been known as difficult to reason about. In 2013, a new algorithm called **Raft** appeared. The researchers who developed it wanted to create an algorithm that's easy to understand and implement. It was first presented in a paper titled “In Search of an Understandable Consensus Algorithm” [ONGARO14] .

Paxos đã là thuật toán đồng thuận trong hơn một thập kỷ, nhưng trong cộng đồng hệ phân tán nó đã được biết đến là khó để lập luận. Năm 2013, một thuật toán mới gọi là Raft xuất hiện. Các nhà nghiên cứu phát triển nó muốn tạo ra một thuật toán dễ hiểu và dễ hiện thực. Nó được trình bày lần đầu trong một bài báo có tiêu đề “In Search of an Understandable Consensus Algorithm” [ONGARO14].

There's enough inherent complexity in distributed systems, and having simpler algorithms is very desirable. Along with a paper, the authors have released a reference implementation called LogCabin to resolve possible ambiguities and help future implementors to gain a better understanding.

Có đủ độ phức tạp cố hữu trong các hệ phân tán, và việc có các thuật toán đơn giản hơn là rất đáng mong muốn. Cùng với một bài báo, các tác giả đã công bố một hiện thực tham chiếu gọi là LogCabin để giải quyết các mơ hồ khả dĩ và giúp các nhà hiện thực tương lai có được sự hiểu biết tốt hơn.

Locally, participants store a log containing the sequence of commands executed by the state machine. Since inputs that processes receive are identical and logs contain the same commands in the same order, applying these commands to the state machine guarantees the same output. Raft simplifies consensus by making the concept of leader a first-class citizen. A leader is used to coordinate state machine manipulation and replication. There are many similarities between Raft and atomic broadcast algorithms, as well as Multi-Paxos: a single leader emerges from replicas, makes atomic decisions, and establishes the message order.

Một cách cục bộ, các bên tham gia lưu trữ một nhật ký chứa trình tự các lệnh được thực thi bởi máy trạng thái. Vì các đầu vào mà các tiến trình nhận được là giống hệt nhau và các nhật ký chứa cùng các lệnh theo cùng một thứ tự, việc áp dụng các lệnh này lên máy trạng thái bảo đảm cùng một đầu ra. Raft đơn giản hóa đồng thuận bằng cách làm cho

khái niệm leader trở thành một công dân hạng nhất. Một leader được dùng để điều phối việc thao tác và nhân bản máy trạng thái. Có nhiều điểm tương đồng giữa Raft và các thuật toán quảng bá nguyên tử, cũng như Multi-Paxos: một leader duy nhất xuất hiện từ các bản sao, đưa ra các quyết định nguyên tử, và thiết lập thứ tự thông điệp.

Each participant in Raft can take one of three roles:

Mỗi bên tham gia trong Raft có thể đảm nhận một trong ba vai trò:

Candidate Leadership is a temporary condition, and any participant can take this role. To become a leader, the node first has to transition into a candidate state, and attempt to collect a majority of votes. If a candidate neither wins nor loses the election (the vote is split between multiple candidates and none of them has a majority of votes), the new term is slated and election restarts.

Ứng viên (Candidate) Vị thế lãnh đạo là một điều kiện tạm thời, và bất kỳ bên tham gia nào cũng có thể đảm nhận vai trò này. Để trở thành một leader, nút trước tiên phải chuyển sang trạng thái ứng viên, và cố gắng thu thập đa số phiếu. Nếu một ứng viên không thắng cũng không thua cuộc bầu chọn (phiếu bị chia giữa nhiều ứng viên và không ai trong số chúng có đa số phiếu), nhiệm kỳ (term) mới được sắp đặt và cuộc bầu chọn khởi động lại.

Leader A current, temporary cluster leader that handles client requests and interacts with a replicated state machine. The leader is elected for a period called a term . Each term is identified by a monotonically increasing number and may continue for an arbitrary time period. A new leader is elected if the current one crashes, becomes unresponsive, or is suspected by other processes to have failed, which can happen because of network partitions and message delays.

Leader Một leader cụm hiện tại, tạm thời, xử lý các yêu cầu của client và tương tác với một máy trạng thái nhân bản. Leader được bầu chọn cho một khoảng thời gian gọi là nhiệm kỳ (term). Mỗi nhiệm kỳ được định danh bằng một số tăng đơn điệu và có thể kéo dài trong một khoảng thời gian tùy ý. Một leader mới được bầu chọn nếu leader hiện tại sụp đổ, trở nên không phản hồi, hoặc bị các tiến trình khác nghi ngờ đã bị lỗi, điều có thể xảy ra do các phân vùng mạng và độ trễ thông điệp.

Follower A passive participant that persists log entries and responds to requests from the leader and candidates. Follower in Raft is a role similar to acceptor and learner from Paxos. Every process begins as a follower.

Follower Một bên tham gia thụ động vốn lưu giữ bền vững các mục nhật ký và phản hồi các yêu cầu từ leader và các ứng viên. Follower trong Raft là một vai trò tương tự bên chấp nhận và bên học từ Paxos. Mọi tiến trình bắt đầu như một follower.

To guarantee global partial ordering without relying on clock synchronization, time is divided into terms (also called epoch), during which the leader is unique and stable. Terms are monotonically numbered, and each command is uniquely identified by the term number and the message number within the term [HOWARD14] .

Để bảo đảm thứ tự từng phần trên phạm vi toàn cục mà không dựa vào sự đồng bộ đồng hồ, thời gian được chia thành các nhiệm kỳ (cũng gọi là kỷ nguyên), trong suốt đó leader là duy nhất và ổn định. Các nhiệm kỳ được đánh số đơn điệu, và mỗi lệnh được định danh duy nhất bằng số nhiệm kỳ và số thông điệp trong phạm vi nhiệm kỳ [HOWARD14].

It may happen that different participants disagree on which term is current , since they can find out about the new term at different times, or could have missed the leader election for one or multiple terms. Since each message contains a term identifier, if one of the participants discovers that its term is out-of-date, it updates the term to the higher-numbered one [ONGARO14] . This means that there may be several terms in flight at any given point in time, but the higher-numbered one wins in case

of a conflict. A node updates the term only if it starts a new election process or finds out that its term is out-of-date.

Có thể xảy ra việc các bên tham gia khác nhau bất đồng về việc nhiệm kỳ nào là hiện tại, vì chúng có thể tìm hiểu về nhiệm kỳ mới vào những thời điểm khác nhau, hoặc có thể đã bỏ lỡ cuộc bầu chọn leader cho một hoặc nhiều nhiệm kỳ. Vì mỗi thông điệp chứa một định danh nhiệm kỳ, nếu một trong các bên tham gia phát hiện rằng nhiệm kỳ của nó đã lỗi thời, nó cập nhật nhiệm kỳ lên cái có số cao hơn [ONGARO14]. Điều này có nghĩa là có thể có nhiều nhiệm kỳ đang lưu hành tại bất kỳ thời điểm nào, nhưng cái có số cao hơn thắng trong trường hợp xung đột. Một nút cập nhật nhiệm kỳ chỉ khi nó bắt đầu một quá trình bầu chọn mới hoặc phát hiện rằng nhiệm kỳ của nó đã lỗi thời.

On startup, or whenever a follower doesn't receive messages from the leader and suspects that it has crashed, it starts the leader election process. A participant attempts to become a leader by transitioning into the candidate state and collecting votes from the majority of nodes.

Khi khởi động, hoặc bất cứ khi nào một follower không nhận được thông điệp từ leader và nghi ngờ rằng nó đã sập đổ, nó bắt đầu quá trình bầu chọn leader. Một bên tham gia cố gắng trở thành một leader bằng cách chuyển sang trạng thái ứng viên và thu thập phiếu từ đa số các nút.

Figure 14-10 shows a sequence diagram representing the main components of the Raft algorithm:

Hình 14-10 thể hiện một sơ đồ trình tự đại diện cho các thành phần chính của thuật toán Raft:

Leader election Candidate P1 sends a RequestVote message to the other processes. This message includes the candidate's term, the last term known by it, and the ID of the last log entry it has observed. After collecting a majority of votes, the candidate is successfully elected as a leader for the term. Each process can give its vote to at most one candidate.

Bầu chọn leader Ứng viên P1 gửi một thông điệp RequestVote tới các tiến trình khác. Thông điệp này bao gồm nhiệm kỳ của ứng viên, nhiệm kỳ cuối cùng được nó biết đến, và ID của mục nhật ký cuối cùng nó đã quan sát. Sau khi thu thập đa số phiếu, ứng viên được bầu chọn thành công làm leader cho nhiệm kỳ đó. Mỗi tiến trình có thể trao phiếu của mình cho tối đa một ứng viên.

Periodic heartbeats The protocol uses a **heartbeat** mechanism to ensure the liveness of participants. The leader periodically sends heartbeats to all followers to maintain its term. If a follower doesn't receive new heartbeats for a period called an election timeout, it assumes that the leader has failed and starts a new election.

Các nhịp tim định kỳ Giao thức dùng một cơ chế nhịp tim để bảo đảm tính sống còn của các bên tham gia. Leader định kỳ gửi các nhịp tim tới tất cả các follower để duy trì nhiệm kỳ của nó. Nếu một follower không nhận được các nhịp tim mới trong một khoảng thời gian gọi là thời gian chờ bầu chọn (election timeout), nó cho rằng leader đã bị lỗi và bắt đầu một cuộc bầu chọn mới.

Log replication / broadcast The leader can repeatedly append new values to the replicated log by sending AppendEntries messages. The message includes the leader's term, index, and term of the log entry that immediately precedes the ones it's currently sending, and one or more entries to store.

Nhân bản nhật ký / quảng bá Leader có thể lặp đi lặp lại nối thêm các giá trị mới vào nhật ký nhân bản bằng cách gửi các thông điệp AppendEntries. Thông điệp bao gồm nhiệm kỳ của leader, chỉ số, và nhiệm kỳ của mục nhật ký đứng ngay trước những mục mà nó đang gửi, và một hoặc nhiều mục để lưu trữ.

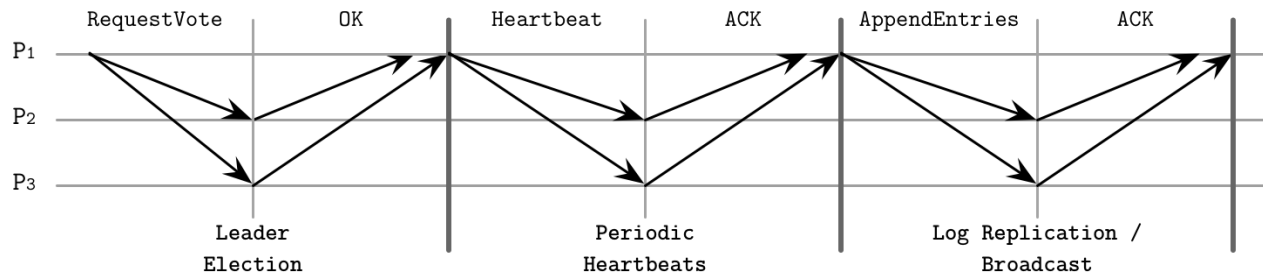


Figure 14-10. Raft consensus algorithm summary

Hình 14-10. Tóm tắt thuật toán đồng thuận Raft

Leader Role in Raft — Vai trò Leader trong Raft

A leader can be elected only from the nodes holding all committed entries: if during the election, the follower's log information is more up-to-date (in other words, has a higher term ID, or a longer log entry sequence, if terms are equal) than the candidate's, its vote is denied.

Một leader chỉ có thể được bầu chọn từ các nút giữ tất cả các mục đã được commit: nếu trong lúc bầu chọn, thông tin nhật ký của follower mới hơn (nói cách khác, có ID nhiệm kỳ cao hơn, hoặc một trình tự mục nhật ký dài hơn, nếu các nhiệm kỳ bằng nhau) so với của ứng viên, thì phiếu của nó bị từ chối.

To win the vote, a candidate has to collect a majority of votes. Entries are always replicated in order, so it is always enough to compare IDs of the latest entries to understand whether or not one of the participants is up-to-date.

Để thắng cuộc bỏ phiếu, một ứng viên phải thu thập đa số phiếu. Các mục luôn được nhân bản theo thứ tự, nên luôn đủ để so sánh các ID của các mục mới nhất để hiểu liệu một trong các bên tham gia có cập nhật hay không.

Once elected, the leader has to accept client requests (which can also be forwarded to it from other nodes) and replicate them to the followers. This is done by appending the entry to its log and sending it to all the followers in parallel.

Một khi được bầu chọn, leader phải chấp nhận các yêu cầu của client (vốn cũng có thể được chuyển tiếp tới nó từ các nút khác) và nhân bản chúng tới các follower. Điều này được thực hiện bằng cách nối thêm mục vào nhật ký của nó và gửi nó tới tất cả các follower một cách song song.

When a follower receives an AppendEntries message, it appends the entries from the message to the local log, and acknowledges the message, letting the leader know that it was persisted. As soon as enough replicas send their acknowledgments, the entry is considered committed and is marked correspondingly in the leader log.

Khi một follower nhận một thông điệp AppendEntries, nó nối thêm các mục từ thông điệp vào nhật ký cục bộ, và xác nhận thông điệp, cho leader biết rằng nó đã được lưu giữ bền vững. Ngay khi đủ số bản sao gửi các xác nhận của chúng, mục được coi là đã được commit và được đánh dấu tương ứng trong nhật ký của leader.

Since only the most up-to-date candidates can become a leader, followers never have to bring the leader up-to-date, and log entries are only flowing from leader to follower and not vice versa.

Vì chỉ các ứng viên cập nhật nhất mới có thể trở thành một leader, các follower không bao giờ phải đưa leader lên cập nhật, và các mục nhật ký chỉ chảy từ leader sang follower chứ không ngược lại.

Figure 14-11 shows this process:

Hình 14-11 thể hiện quá trình này:

- a) A new command $x = 8$ is appended to the leader's log.
- b) Before the value can be committed, it has to be replicated to the majority of participants.
- c) As soon as the leader is done with replication, it commits the value locally.
- d) The commit decision is replicated to the followers.
 - a) Một lệnh mới $x = 8$ được nối thêm vào nhật ký của leader.
 - b) Trước khi giá trị có thể được commit, nó phải được nhân bản tới đa số các bên tham gia.
 - c) Ngay khi leader hoàn tất việc nhân bản, nó commit giá trị một cách cục bộ.
 - d) Quyết định commit được nhân bản tới các follower.

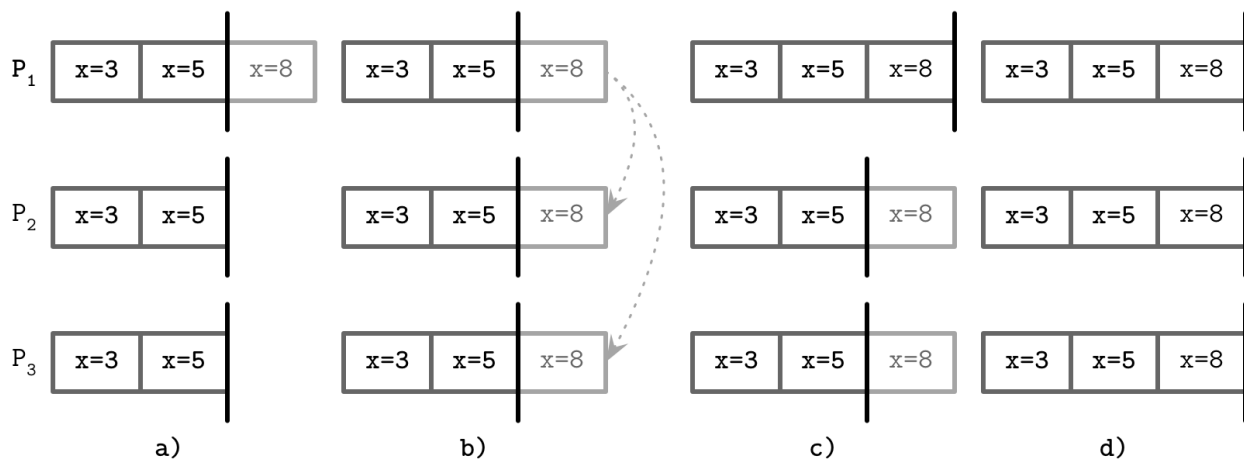


Figure 14-11. Procedure of a commit in Raft with P as a leader

Hình 14-11. Quy trình của một commit trong Raft với P là leader

Figure 14-12 shows an example of a consensus round where P₁ is a leader, which has the most recent view of the events. The leader proceeds by replicating the entries to the followers, and committing them after collecting acknowledgments. Committing an entry also commits all entries preceding it in the log. Only the leader can make a decision on whether or not the entry can be committed. Each log entry is marked with a term ID (a number in the top-right corner of each log entry box) and a log index, identifying its position in the log. Committed entries are guaranteed to be replicated to the quorum of participants and are safe to be applied to the state machine in the order they appear in the log.

Hình 14-12 thể hiện một ví dụ về một vòng đồng thuận trong đó P₁ là một leader, vốn có góc nhìn gần đây nhất về các sự kiện. Leader tiến hành bằng cách nhân bản các mục tới các follower, và commit chúng sau khi thu thập các xác nhận. Việc commit một mục cũng commit tất cả các mục đứng trước nó trong nhật ký. Chỉ leader mới có thể đưa ra quyết định về việc liệu mục có thể được commit hay không. Mỗi mục nhật ký được đánh dấu

bằng một ID nhiệm kỳ (một số ở góc trên bên phải của mỗi ô mục nhật ký) và một chỉ số nhật ký, định danh vị trí của nó trong nhật ký. Các mục đã commit được bảo đảm được nhân bản tới quorum các bên tham gia và an toàn để được áp dụng lên máy trạng thái theo thứ tự chúng xuất hiện trong nhật ký.

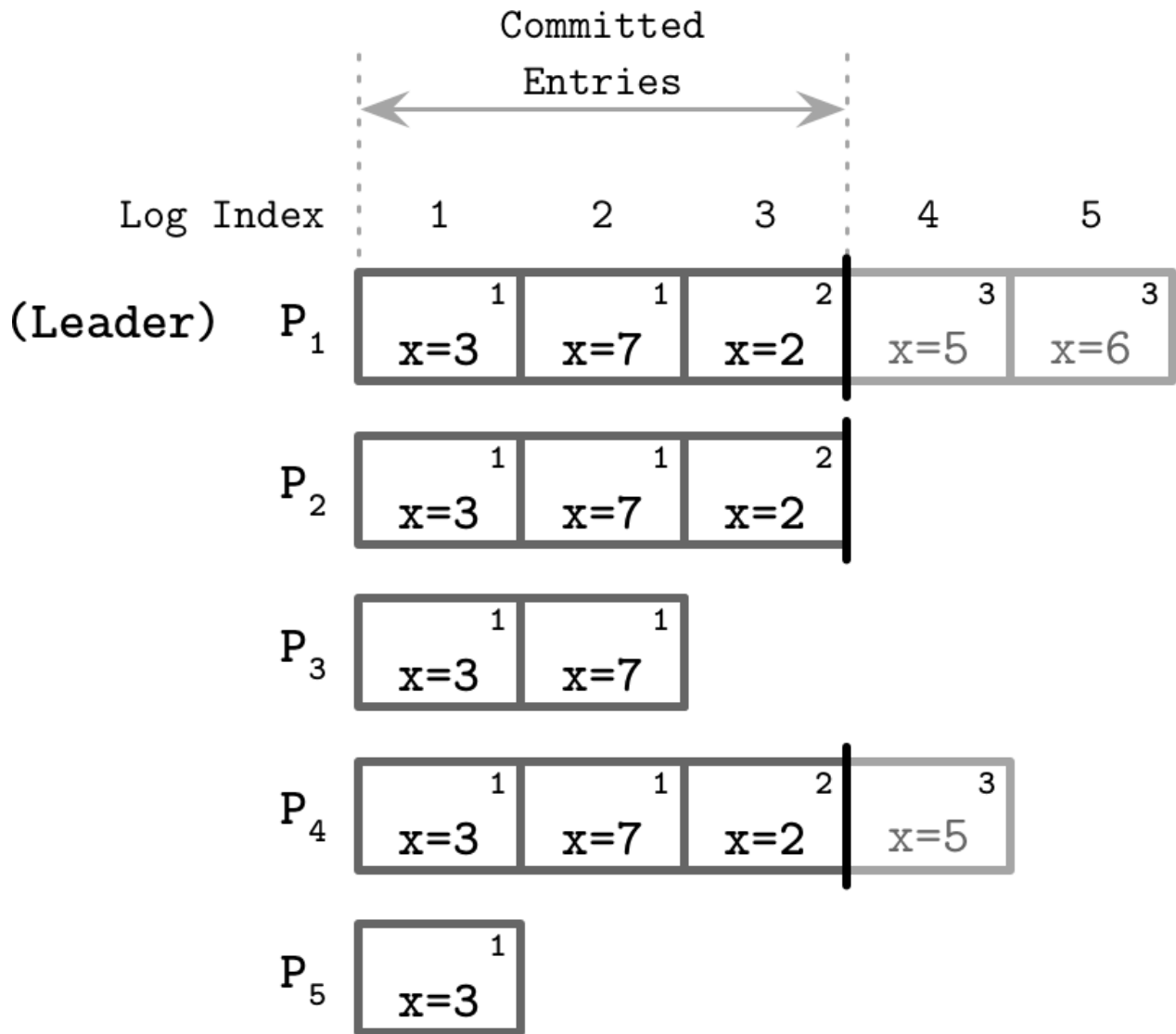


Figure 14-12. Raft state machine

Hình 14-12. Máy trạng thái Raft

Failure Scenarios — Các kịch bản lỗi

When multiple followers decide to become candidates, and no candidate can collect a majority of votes, the situation is called a split vote. Raft uses randomized timers to reduce the probability of multiple subsequent elections ending up in a split vote. One of the candidates can start the next election round earlier and collect enough votes, while the others sleep and give way to it. This approach speeds up the election without requiring any additional coordination between candidates.

Khi nhiều follower quyết định trở thành ứng viên, và không ứng viên nào có thể thu thập

đa số phiếu, tình huống này được gọi là phiếu chia (split vote). Raft dùng các bộ định thời ngẫu nhiên để giảm xác suất nhiều cuộc bầu chọn liên tiếp kết thúc trong một phiếu chia. Một trong các ứng viên có thể bắt đầu vòng bầu chọn kế tiếp sớm hơn và thu thập đủ phiếu, trong khi những ứng viên khác ngủ và nhường đường cho nó. Cách tiếp cận này tăng tốc cuộc bầu chọn mà không yêu cầu bất kỳ sự điều phối bổ sung nào giữa các ứng viên.

Followers may be down or slow to respond, and the leader has to make the best effort to ensure message delivery. It can try sending messages again if it doesn't receive an acknowledgment within the expected time bounds. As a performance optimization, it can send multiple messages in parallel.

Các follower có thể bị ngừng hoạt động hoặc chậm phản hồi, và leader phải nỗ lực hết sức để bảo đảm việc chuyển giao thông điệp. Nó có thể thử gửi lại thông điệp nếu nó không nhận được một xác nhận trong các giới hạn thời gian mong đợi. Như một tối ưu hóa hiệu năng, nó có thể gửi nhiều thông điệp một cách song song.

Since entries replicated by the leader are uniquely identified, repeated message delivery is guaranteed not to break the log order. Followers deduplicate messages using their sequence IDs, ensuring that double delivery has no undesired side effects.

Vì các mục được nhân bản bởi leader được định danh duy nhất, việc chuyển giao thông điệp lặp lại được bảo đảm không phá vỡ thứ tự nhật ký. Các follower khử trùng lặp các thông điệp bằng cách dùng các ID trình tự của chúng, bảo đảm rằng việc chuyển giao kép không có tác dụng phụ không mong muốn.

Sequence IDs are also used to ensure the log ordering. A follower rejects a higher-numbered entry if the ID and term of the entry that immediately precedes it, sent by the leader, do not match the highest entry according to its own records. If entries in two logs on different replicas have the same term and the same index, they store the same command and all entries that precede them are the same.

Các ID trình tự cũng được dùng để bảo đảm thứ tự nhật ký. Một follower từ chối một mục có số cao hơn nếu ID và nhiệm kỳ của mục đứng ngay trước nó, được leader gửi, không khớp với mục cao nhất theo các bản ghi của riêng nó. Nếu các mục trong hai nhật ký trên các bản sao khác nhau có cùng nhiệm kỳ và cùng chỉ số, chúng lưu cùng một lệnh và tất cả các mục đứng trước chúng đều giống nhau.

Raft guarantees to never show an uncommitted message as a committed one, but, due to network or replica slowness, already committed messages can still be seen as in progress, which is a rather harmless property and can be worked around by retrying a client command until it is finally committed [HOWARD14].

Raft bảo đảm không bao giờ hiển thị một thông điệp chưa commit như một thông điệp đã commit, nhưng, do sự chậm chạp của mạng hoặc bản sao, các thông điệp đã commit vẫn có thể được nhìn thấy như đang trong tiến trình (in progress), đây là một thuộc tính khá vô hại và có thể được khắc phục bằng cách thử lại một lệnh của client cho đến khi nó cuối cùng được commit [HOWARD14].

For failure detection, the leader has to send heartbeats to the followers. This way, the leader maintains its term. When one of the nodes notices that the current leader is down, it attempts to initiate the election. The newly elected leader has to restore the state of the cluster to the last known up-to-date log entry. It does so by finding a common ground (the highest log entry on which both the leader and follower agree), and ordering followers to discard all (uncommitted) entries appended after this point. It then sends the most recent entries from its log, overwriting the followers' history. The leader's own log records are never removed or overwritten: it can only append entries to its own log.

Để phát hiện lỗi, leader phải gửi các nhịp tim tới các follower. Bằng cách này, leader duy trì nhiệm kỳ của nó. Khi một trong các nút nhận thấy rằng leader hiện tại đã ngừng hoạt động, nó cố gắng khởi tạo cuộc bầu chọn. Leader vừa được bầu chọn phải khôi phục trạng thái của cụm về mục nhật ký cập nhật được biết đến cuối cùng. Nó làm điều đó bằng cách tìm một điểm chung (mục nhật ký cao nhất mà cả leader lẫn follower đều thống nhất), và ra lệnh cho các follower loại bỏ tất cả các mục (chưa commit) được nối thêm sau điểm này. Sau đó nó gửi các mục gần đây nhất từ nhật ký của nó, ghi đè lên lịch sử của các follower. Các bản ghi nhật ký của riêng leader không bao giờ bị xóa hoặc ghi đè: nó chỉ có thể nối thêm các mục vào nhật ký của riêng mình.

Summing up, the Raft algorithm provides the following guarantees:

Tóm lại, thuật toán Raft cung cấp các bảo đảm sau:

- Only one leader can be elected at a time for a given term; no two leaders can be active during the same term.
- The leader does not remove or reorder its log contents; it only appends new messages to it.
- Committed log entries are guaranteed to be present in logs for subsequent leaders and cannot get reverted, since before the entry is committed it is known to be replicated by the leader.
- All messages are identified uniquely by the message and term IDs; neither current nor subsequent leaders can reuse the same identifier for the different entry.
 - Chỉ một leader có thể được bầu chọn tại một thời điểm cho một nhiệm kỳ nhất định; không thể có hai leader cùng hoạt động trong cùng một nhiệm kỳ.
 - Leader không xóa hoặc sắp xếp lại nội dung nhật ký của nó; nó chỉ nối thêm các thông điệp mới vào đó.
 - Các mục nhật ký đã commit được bảo đảm hiện diện trong các nhật ký cho các leader kế tiếp và không thể bị hoàn tác, vì trước khi mục được commit thì nó được biết là đã được leader nhân bản.
 - Tất cả các thông điệp được định danh duy nhất bằng các ID thông điệp và nhiệm kỳ; cả leader hiện tại lẫn các leader kế tiếp đều không thể tái sử dụng cùng một định danh cho mục khác.

Since its appearance, Raft has become very popular and is currently used in many databases and other distributed systems, including CockroachDB , Etcd , and Consul . This can be attributed to its simplicity, but also may mean that Raft lives up to the promise of being a reliable consensus algorithm.

Kể từ khi xuất hiện, Raft đã trở nên rất phổ biến và hiện được dùng trong nhiều cơ sở dữ liệu và các hệ phân tán khác, bao gồm CockroachDB, Etcd, và Consul. Điều này có thể được quy cho sự đơn giản của nó, nhưng cũng có thể có nghĩa là Raft xứng đáng với lời hứa là một thuật toán đồng thuận đáng tin cậy.

Byzantine Consensus — Đồng thuận Byzantine

All the consensus algorithms we have been discussing so far assume non-Byzantine failures (see “Arbitrary Faults” on page 193). In other words, nodes execute the algorithm in “good faith” and do not try to exploit it or forge the results.

Tất cả các thuật toán đồng thuận chúng ta đã thảo luận cho đến giờ đều giả định các lỗi không phải Byzantine (xem “Arbitrary Faults” ở trang 193). Nói cách khác, các nút thực hiện thuật toán một cách “thiện chí” và không cố khai thác nó hoặc giả mạo các kết quả.

As we will see, this assumption allows achieving consensus with a smaller number of available participants and with fewer round-trips required for a commit. However, distributed systems are sometimes deployed in potentially adversarial environments, where the nodes are not controlled by the same entity, and we need algorithms that can ensure a system can function correctly even if some nodes behave erratically or even maliciously. Besides ill intentions, Byzantine failures can also be caused by bugs, misconfiguration, hardware issues, or data corruption.

Như chúng ta sẽ thấy, giả định này cho phép đạt được đồng thuận với một số lượng bên tham gia khả dụng nhỏ hơn và với ít vòng khứ hồi hơn cần thiết cho một commit. Tuy nhiên, các hệ phân tán đôi khi được triển khai trong các môi trường tiềm ẩn tính thù địch, nơi các nút không được kiểm soát bởi cùng một thực thể, và chúng ta cần các thuật toán có thể bảo đảm một hệ thống hoạt động đúng đắn ngay cả khi một số nút hành xử thất thường hoặc thậm chí độc hại. Bên cạnh các ý định xấu, các lỗi Byzantine cũng có thể bị gây ra bởi các lỗi phần mềm (bug), cấu hình sai, các vấn đề phần cứng, hoặc hỏng dữ liệu.

Most Byzantine consensus algorithms require N^2 messages to complete an algorithm step, where N is the size of the quorum, since each node in the quorum has to communicate with each other. This is required to cross-validate each step against other nodes, since nodes cannot rely on each other or on the leader and have to verify other nodes' behaviors by comparing returned results with the majority responses.

Hầu hết các thuật toán đồng thuận Byzantine yêu cầu N^2 thông điệp để hoàn thành một bước thuật toán, trong đó N là kích thước của quorum, vì mỗi nút trong quorum phải giao tiếp với mỗi nút khác. Điều này là cần thiết để kiểm chứng chéo mỗi bước với các nút khác, vì các nút không thể dựa vào nhau hoặc vào leader và phải xác minh hành vi của các nút khác bằng cách so sánh các kết quả trả về với các phản hồi đa số.

We'll only discuss one Byzantine consensus algorithm here, Practical **Byzantine Fault Tolerance** (PBFT) [CASTRO99]. PBFT assumes independent node failures (i.e., failures can be coordinated, but the entire system cannot be taken over at once, or at least with the same exploit method). The system makes weak synchrony assumptions, like how you would expect a network to behave normally: failures may occur, but they are not indefinite and are eventually recovered from.

Chúng ta sẽ chỉ thảo luận một thuật toán đồng thuận Byzantine ở đây, Practical Byzantine Fault Tolerance (PBFT) [CASTRO99]. PBFT giả định các lỗi nút độc lập (tức là các lỗi có thể được điều phối, nhưng toàn bộ hệ thống không thể bị chiếm lấy cùng một lúc, hoặc ít nhất là với cùng một phương pháp khai thác). Hệ thống đưa ra các giả định đồng bộ yếu, giống như cách bạn mong đợi một mạng hành xử bình thường: các lỗi có thể xảy ra, nhưng chúng không kéo dài vô thời hạn và cuối cùng được phục hồi.

All communication between the nodes is encrypted, which serves to prevent message forging and network attacks. Replicas know one another's public keys to verify identities and encrypt messages. Faulty nodes may leak information from inside the system, since, even though encryption is used, every node needs to interpret message contents to react upon them. This doesn't undermine the algorithm, since it serves a different purpose.

Mọi giao tiếp giữa các nút đều được mã hóa, điều này phục vụ để ngăn việc giả mạo thông điệp và các cuộc tấn công mạng. Các bản sao biết khóa công khai của nhau để xác minh danh tính và mã hóa thông điệp. Các nút lỗi có thể rò rỉ thông tin từ bên trong hệ thống, vì, mặc dù mã hóa được dùng, mỗi nút cần diễn giải nội dung thông điệp để phản ứng dựa trên chúng. Điều này không làm suy yếu thuật toán, vì nó phục vụ một mục đích khác.

PBFT Algorithm — Thuật toán PBFT

For PBFT to guarantee both safety and liveness, no more than $(n - 1)/3$ replicas can be faulty (where n is the total number of participants). For a system to sustain f compromised nodes, it is required to have at least $n = 3f + 1$ nodes. This is the case because a majority of nodes have to agree on the value: f replicas might be faulty, and there might be f replicas that are not responding but may not be faulty (for example, due to a network partition, power failure, or maintenance). The algorithm has to be able to collect enough responses from nonfaulty replicas to still outnumber those from the faulty ones.

Để PBFT bảo đảm cả tính an toàn lẫn tính sống còn, không quá $(n - 1)/3$ bản sao có thể bị lỗi (trong đó n là tổng số bên tham gia). Để một hệ thống chịu đựng được f nút bị xâm phạm, nó cần có ít nhất $n = 3f + 1$ nút. Đây là trường hợp như vậy vì một đa số các nút phải thống nhất về giá trị: f bản sao có thể bị lỗi, và có thể có f bản sao không phản hồi nhưng có thể không bị lỗi (ví dụ, do một phân vùng mạng, mất điện, hoặc bảo trì). Thuật toán phải có khả năng thu thập đủ phản hồi từ các bản sao không lỗi để vẫn đông hơn các phản hồi từ những bản sao bị lỗi.

Consensus properties for PBFT are similar to those of other consensus algorithms: all nonfaulty replicas have to agree both on the set of received values and their order, despite the possible failures.

Các thuộc tính đồng thuận cho PBFT tương tự như của các thuật toán đồng thuận khác: tất cả các bản sao không lỗi phải thống nhất cả về tập các giá trị đã nhận lẫn thứ tự của chúng, bất chấp các lỗi có thể xảy ra.

To distinguish between cluster configurations, PBFT uses views. In each view, one of the replicas is a primary and the rest of them are considered backups. All nodes are numbered consecutively, and the index of the primary node is $v \bmod N$, where v is the view ID, and N is the number of nodes in the current configuration. The view can change in cases when the primary fails. Clients execute their operations against the primary. The primary broadcasts the requests to the backups, which execute the requests and send a response back to the client. The client waits for $f + 1$ replicas to respond with the same result for any operation to succeed.

Để phân biệt giữa các cấu hình cụm, PBFT dùng các góc nhìn (views). Trong mỗi góc nhìn, một trong các bản sao là một bản chính (primary) và phần còn lại được coi là các bản sao lưu (backups). Tất cả các nút được đánh số liên tiếp, và chỉ số của nút chính là $v \bmod N$, trong đó v là ID góc nhìn, và N là số nút trong cấu hình hiện tại. Góc nhìn có thể thay đổi trong các trường hợp khi bản chính bị lỗi. Các client thực hiện các thao tác của họ trên bản chính. Bản chính quảng bá các yêu cầu tới các bản sao lưu, vốn thực thi các yêu cầu và gửi một phản hồi trở lại cho client. Client chờ $f + 1$ bản sao phản hồi với cùng một kết quả để bất kỳ thao tác nào thành công.

After the primary receives a client request, protocol execution proceeds in three phases:

Sau khi bản chính nhận một yêu cầu của client, việc thực thi giao thức tiến hành trong ba pha:

Pre-prepare The primary broadcasts a message containing a view ID, a unique monotonically increasing identifier, a payload (client request), and a payload digest. Digests are computed using a strong collision-resistant hash function, and are signed by the sender. The backup accepts the message if its view matches with the primary view and the client request hasn't been tampered with: the calculated payload digest matches the received one.

Tiền chuẩn bị (Pre-prepare) Bản chính quảng bá một thông điệp chứa một ID góc nhìn, một định danh tăng đơn điệu duy nhất, một tải trọng (yêu cầu của client), và một bản

tóm tắt tải trọng (payload digest). Các bản tóm tắt được tính bằng một hàm băm chống va chạm mạnh, và được ký bởi bên gửi. Bản sao lưu chấp nhận thông điệp nếu góc nhìn của nó khớp với góc nhìn của bản chính và yêu cầu của client chưa bị giả mạo: bản tóm tắt tải trọng được tính khớp với cái đã nhận.

Prepare If the backup accepts the pre-prepare message, it enters the prepare phase and starts broadcasting Prepare messages, containing a view ID, message ID, and a payload digest, but without the payload itself, to all other replicas (including the primary). Replicas can move past the prepare state only if they receive $2f$ prepares from different backups that match the message received during pre-prepare: they have to have the same view, same ID, and a digest.

Chuẩn bị (Prepare) Nếu bản sao lưu chấp nhận thông điệp pre-prepare, nó bước vào pha prepare và bắt đầu quảng bá các thông điệp Prepare, chứa một ID góc nhìn, ID thông điệp, và một bản tóm tắt tải trọng, nhưng không có chính tải trọng, tới tất cả các bản sao khác (bao gồm cả bản chính). Các bản sao chỉ có thể vượt qua trạng thái prepare nếu chúng nhận được $2f$ prepare từ các bản sao lưu khác nhau khớp với thông điệp đã nhận trong lúc pre-prepare: chúng phải có cùng góc nhìn, cùng ID, và một bản tóm tắt.

Commit After that, the backup moves to the commit phase, where it broadcasts Commit messages to all other replicas and waits to collect $2f + 1$ matching Commit messages (possibly including its own) from the other participants.

Commit Sau đó, bản sao lưu chuyển sang pha commit, nơi nó quảng bá các thông điệp Commit tới tất cả các bản sao khác và chờ thu thập $2f + 1$ thông điệp Commit khớp nhau (có thể bao gồm cả của chính nó) từ các bên tham gia khác.

A digest in this case is used to reduce the message size during the prepare phase, since it's not necessary to rebroadcast an entire payload for verification, as the digest serves as a payload summary. Cryptographic hash functions are resistant to collisions: it is difficult to produce two values that have the same digest, let alone two messages with matching digests that make sense in the context of the system. In addition, digests are signed to make sure that the digest itself is coming from a trusted source.

Một bản tóm tắt trong trường hợp này được dùng để giảm kích thước thông điệp trong pha prepare, vì không cần quảng bá lại toàn bộ tải trọng để xác minh, vì bản tóm tắt đóng vai trò như một bản tóm lược tải trọng. Các hàm băm mật mã chống va chạm: rất khó để tạo ra hai giá trị có cùng bản tóm tắt, chứ đừng nói đến hai thông điệp với các bản tóm tắt khớp nhau mà có ý nghĩa trong ngữ cảnh của hệ thống. Ngoài ra, các bản tóm tắt được ký để bảo đảm rằng chính bản tóm tắt đến từ một nguồn đáng tin cậy.

The number $2f$ is important, since the algorithm has to make sure that at least $f + 1$ nonfaulty replicas respond to the client.

Con số $2f$ là quan trọng, vì thuật toán phải bảo đảm rằng ít nhất $f + 1$ bản sao không lỗi phản hồi cho client.

Figure 14-13 shows a sequence diagram of a normal-case PBFT algorithm round: the client sends a request to P, and nodes move between phases by collecting a sufficient

Hình 14-13 thể hiện một sơ đồ trình tự của một vòng thuật toán PBFT trường hợp bình thường: client gửi một yêu cầu tới P, và các nút di chuyển giữa các pha bằng cách thu thập một số

number of matching responses from properly behaving peers. P may have failed or

lượng đủ các phản hồi khớp nhau từ các peer hành xử đúng đắn. P có thể đã bị lỗi hoặc

could've responded with unmatching messages, so its responses wouldn't have been counted.

có thể đã phản hồi với các thông điệp không khớp, nên các phản hồi của nó sẽ không được tính.

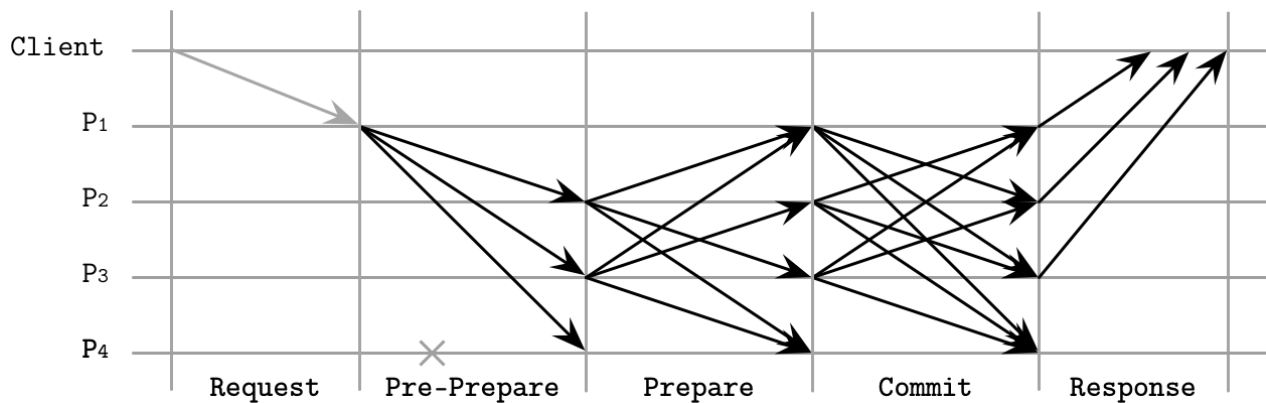


Figure 14-13. PBFT consensus, normal-case operation

Hình 14-13. Đồng thuận PBFT, hoạt động trường hợp bình thường

During the prepare and commit phases, nodes communicate by sending messages to each other node and waiting for the messages from the corresponding number of other nodes, to check if they match and make sure that incorrect messages are not broadcasted. Peers cross-validate all messages so that only nonfaulty nodes can successfully commit messages. If a sufficient number of matching messages cannot be collected, the node doesn't move to the next step.

Trong các pha prepare và commit, các nút giao tiếp bằng cách gửi các thông điệp tới mỗi nút khác và chờ các thông điệp từ số lượng tương ứng các nút khác, để kiểm tra xem chúng có khớp không và bảo đảm rằng các thông điệp không đúng không được quảng bá. Các peer kiểm chứng chéo tất cả các thông điệp để chỉ các nút không lỗi mới có thể commit các thông điệp thành công. Nếu không thể thu thập đủ số lượng các thông điệp khớp nhau, nút không chuyển sang bước kế tiếp.

When replicas collect enough commit messages, they notify the client, finishing the round. The client cannot be certain about whether or not execution was fulfilled correctly until it receives $f + 1$ matching responses.

Khi các bản sao thu thập đủ các thông điệp commit, chúng thông báo cho client, hoàn thành vòng. Client không thể chắc chắn về việc liệu việc thực thi có được thực hiện đúng đắn hay không cho đến khi nó nhận được $f + 1$ phản hồi khớp nhau.

View changes occur when replicas notice that the primary is inactive, and suspect that it might have failed. Nodes that detect a primary failure stop responding to further messages (apart from checkpoint and view-change related ones), broadcast a view change notification, and wait for confirmations. When the primary of the new view receives $2f$ view change events, it initiates a new view.

Các thay đổi góc nhìn (view changes) xảy ra khi các bản sao nhận thấy rằng bản chính không hoạt động, và nghi ngờ rằng nó có thể đã bị lỗi. Các nút phát hiện một lỗi bản chính ngừng phản hồi các thông điệp tiếp theo (ngoài các thông điệp liên quan đến checkpoint và view-change), quảng bá một thông báo thay đổi góc nhìn, và chờ các xác nhận. Khi bản chính của góc nhìn mới nhận được $2f$ sự kiện thay đổi góc nhìn, nó khởi tạo một góc nhìn mới.

To reduce the number of messages in the protocol, clients can collect $2f + 1$ matching responses from nodes that tentatively execute a request (e.g., after they've collected a sufficient number of matching Prepared messages). If the client cannot collect enough matching tentative responses, it retries and waits for $f + 1$ nontentative responses as described previously.

Để giảm số thông điệp trong giao thức, các client có thể thu thập $2f + 1$ phản hồi khớp nhau từ các nút thực thi một yêu cầu một cách tạm thời (ví dụ, sau khi chúng đã thu thập đủ số lượng các thông điệp Prepared khớp nhau). Nếu client không thể thu thập đủ các phản hồi tạm thời khớp nhau, nó thử lại và chờ $f + 1$ phản hồi không tạm thời như đã mô tả trước đó.

Read-only operations in PBFT can be done in just one round-trip. The client sends a read request to all replicas. Replicas execute the request in their tentative states, after all ongoing state changes to the read value are committed, and respond to the client. After collecting $2f + 1$ responses with the same value from different replicas, the operation completes.

Các thao tác chỉ-đọc trong PBFT có thể được thực hiện chỉ trong một vòng khứ hồi. Client gửi một yêu cầu đọc tới tất cả các bản sao. Các bản sao thực thi yêu cầu trong các trạng thái tạm thời của chúng, sau khi tất cả các thay đổi trạng thái đang diễn ra lên giá trị đọc được commit, và phản hồi cho client. Sau khi thu thập $2f + 1$ phản hồi với cùng một giá trị từ các bản sao khác nhau, thao tác hoàn thành.

Recovery and Checkpointing — Phục hồi và Tạo điểm kiểm tra

Replicas save accepted messages in a stable log. Every message has to be kept until it has been executed by at least $f + 1$ nodes. This log can be used to get other replicas up to speed in case of a network partition, but recovering replicas need some means of verifying that the state they receive is correct, since otherwise recovery can be used as an attack vector.

Các bản sao lưu các thông điệp đã chấp nhận trong một nhật ký ổn định. Mỗi thông điệp phải được giữ cho đến khi nó đã được thực thi bởi ít nhất $f + 1$ nút. Nhật ký này có thể được dùng để đưa các bản sao khác lên kịp tiến độ trong trường hợp một phân vùng mạng, nhưng các bản sao đang phục hồi cần một số phương tiện để xác minh rằng trạng thái chúng nhận được là đúng, vì nếu không thì việc phục hồi có thể bị dùng như một véc-tơ tấn công.

To show that the state is correct, nodes compute a digest of the state for messages up to a given sequence number. Nodes can compare digests, verify state integrity, and make sure that messages they received during recovery add up to a correct final state. This process is too expensive to perform on every request.

Để cho thấy rằng trạng thái là đúng, các nút tính một bản tóm tắt của trạng thái cho các thông điệp đến tận một số trình tự nhất định. Các nút có thể so sánh các bản tóm tắt, xác minh tính toàn vẹn trạng thái, và bảo đảm rằng các thông điệp chúng nhận được trong lúc phục hồi cộng lại thành một trạng thái cuối cùng đúng. Quá trình này quá tốn kém để thực hiện trên mỗi yêu cầu.

After every N requests, where N is a configurable constant, the primary makes a stable checkpoint, where it broadcasts the latest sequence number of the latest request whose execution is reflected in the state, and the digest of this state. It then waits for $2f + 1$ replicas to respond. These responses constitute a proof for this checkpoint, and a guarantee that replicas can safely discard state for all pre-prepare, prepare, commit, and checkpoint messages up to the given sequence number.

Sau mỗi N yêu cầu, trong đó N là một hằng số có thể cấu hình, bản chính tạo một điểm kiểm tra ổn định (stable checkpoint), nơi nó quảng bá số trình tự mới nhất của yêu cầu gần nhất mà việc thực thi của nó được phản ánh trong trạng thái, và bản tóm tắt của trạng thái này. Sau đó nó chờ $2f + 1$ bản sao phản hồi. Các phản hồi này tạo thành một bằng chứng cho điểm kiểm tra này, và một bảo đảm rằng các bản sao có thể an toàn loại bỏ trạng thái cho tất cả các thông điệp pre-prepare, prepare, commit, và checkpoint đến tận số trình tự nhất định.

Byzantine fault tolerance is essential to understand and is used in storage systems deployed in potentially adversarial networks. Most of the time, it is enough to authenticate and encrypt internode communication, but when there's no trust between the parts of the system, algorithms similar to PBFT have to be employed.

Dung lỗi Byzantine là điều thiết yếu để hiểu và được dùng trong các hệ thống lưu trữ được triển khai trong các mạng tiềm ẩn tính thù địch. Hầu hết thời gian, việc xác thực và mã hóa giao tiếp giữa các nút là đủ, nhưng khi không có sự tin cậy giữa các phần của hệ thống, các thuật toán tương tự PBFT phải được sử dụng.

Since algorithms resistant to Byzantine faults impose significant overhead in terms of the number of exchanged messages, it is important to understand their use cases. Other protocols, such as the ones described in [BAUDET19] and [BUCHMAN18], attempt to optimize the PBFT algorithm for systems with a large number of participants.

Vì các thuật toán chống lại các lỗi Byzantine áp đặt một chi phí đáng kể về số lượng các thông điệp được trao đổi, điều quan trọng là phải hiểu các trường hợp sử dụng của chúng. Các giao thức khác, chẳng hạn như các giao thức được mô tả trong [BAUDET19] và [BUCHMAN18], cố gắng tối ưu hóa thuật toán PBFT cho các hệ thống có một số lượng lớn các bên tham gia.

Summary — Tóm tắt

Consensus algorithms are one of the most interesting yet most complex subjects in distributed systems. Over the last few years, new algorithms and many implementations of the existing algorithms have emerged, which proves the rising importance and popularity of the subject.

Các thuật toán đồng thuận là một trong những chủ đề thú vị nhất nhưng cũng phức tạp nhất trong các hệ phân tán. Trong vài năm qua, các thuật toán mới và nhiều hiện thực của các thuật toán hiện có đã xuất hiện, điều này chứng tỏ tầm quan trọng và sự phổ biến ngày càng tăng của chủ đề này.

In this chapter, we discussed the classic Paxos algorithm, and several variants of Paxos, each one improving its different properties:

Trong chương này, chúng ta đã thảo luận thuật toán Paxos cổ điển, và một vài biến thể của Paxos, mỗi biến thể cải thiện các thuộc tính khác nhau của nó:

Multi-Paxos Allows a proposer to retain its role and replicate multiple values instead of just one.

Multi-Paxos Cho phép một bên đề xuất giữ lại vai trò của nó và nhân bản nhiều giá trị thay vì chỉ một.

Fast Paxos Allows us to reduce a number of messages by using fast rounds, when acceptors can proceed with messages from proposers other than the established leader.

Fast Paxos Cho phép chúng ta giảm số thông điệp bằng cách dùng các vòng nhanh, khi các bên chấp nhận có thể tiến hành với các thông điệp từ các bên đề xuất khác với leader đã được thiết lập.

EPaxos Establishes event order by resolving dependencies between submitted messages.

EPaxos Thiết lập thứ tự sự kiện bằng cách giải quyết các phụ thuộc giữa các thông điệp đã gửi.

Flexible Paxos Relaxes quorum requirements and only requires a quorum for the first phase (voting) to intersect with a quorum for the second phase (replication).

Flexible Paxos Nói lỏng các yêu cầu quorum và chỉ yêu cầu một quorum cho pha thứ nhất (bỏ phiếu) giao với một quorum cho pha thứ hai (nhân bản).

Raft simplifies the terms in which consensus is described, and makes leadership a first-class citizen in the algorithm. Raft separates log replication, leader election, and safety.

Raft đơn giản hóa các thuật ngữ mà đồng thuận được mô tả, và làm cho vị thế lãnh đạo trở thành một công dân hạng nhất trong thuật toán. Raft tách biệt nhân bản nhật ký, bầu chọn leader, và tính an toàn.

To guarantee consensus safety in adversarial environments, Byzantine fault-tolerant algorithms should be used; for example, PBFT. In PBFT, participants cross-validate one another's responses and only proceed with execution steps when there's enough nodes that obey the prescribed algorithm rules.

Để bảo đảm tính an toàn của đồng thuận trong các môi trường thù địch, các thuật toán dung lỗi Byzantine nên được dùng; ví dụ, PBFT. Trong PBFT, các bên tham gia kiểm chứng chéo các phản hồi của nhau và chỉ tiến hành với các bước thực thi khi có đủ số nút tuân theo các quy tắc thuật toán đã được quy định.

Further Reading

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Atomic broadcast Junqueira, Flavio P., Benjamin C. Reed, and Marco Serafini. "Zab: High-performance broadcast for primary-backup systems." 2011. In Proceedings of the 2011 IEEE/IFIP 41st International Conference on Dependable Systems & Networks (DSN '11) : 245-256.

Hunt, Patrick, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. 2010. "ZooKeeper: wait-free coordination for internet-scale systems." In Proceedings of the 2010 USENIX conference on USENIX annual technical conference (USENIX- ATC'10) : 11.

Oki, Brian M., and Barbara H. Liskov. 1988. "Viewstamped Replication: A New Primary Copy Method to Support Highly-Available Distributed Systems." In Proceedings of the seventh annual ACM Symposium on Principles of distributed computing (PODC '88) : 8-17.

Van Renesse, Robbert, Nicolas Schiper, and Fred B. Schneider. 2014. "Vive la Différence: Paxos vs. Viewstamped Replication vs. Zab."

Classic Paxos Lamport, Leslie. 1998. "The part-time parliament." ACM Transactions on Computer Systems 16, no. 2 (May): 133-169.

Lamport, Leslie. 2001. "Paxos made simple." ACM SIGACT News 32, no. 4: 51-58.

- Lamport, Leslie. 2005. “Generalized Consensus and Paxos.” Technical Report MSR-TR-2005-33. Microsoft Research, Mountain View, CA.
- Primi, Marco. 2009. “Paxos made code: Implementing a high throughput Atomic Broadcast.” (Libpaxos code: <https://bitbucket.org/sciascid/libpaxos/src/master/>).
- Fast Paxos Lamport, Leslie. 2005. “Fast Paxos.” 14 July 2005. Microsoft Research.
- Multi-Paxos Chandra, Tushar D., Robert Griesemer, and Joshua Redstone. 2007. “Paxos made live: an engineering perspective.” In Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing (PODC '07) : 398-407.
- Van Renesse, Robbert and Deniz Altinbuken. 2015. “Paxos Made Moderately Complex.” ACM Computing Surveys 47, no. 3 (February): Article 42. <https://doi.org/10.1145/2673577>.
- EPaxos Moraru, Iulian, David G. Andersen, and Michael Kaminsky. 2013. “There is more consensus in Egalitarian parliaments.” In Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP '13) : 358-372.
- Moraru, I., D. G. Andersen, and M. Kaminsky. 2013. “A proof of correctness for Egalitarian Paxos.” Technical report, Parallel Data Laboratory, Carnegie Mellon University, Aug. 2013.
- Raft Ongaro, Diego, and John Ousterhout. 2014. “In search of an understandable consensus algorithm.” In Proceedings of the 2014 USENIX conference on USENIX Annual Technical Conference (USENIX ATC'14) , Garth Gibson and Nickolai Zeldovich (Eds.): 305-320.
- Howard, H. 2014. “ARC: Analysis of Raft Consensus.” Technical Report UCAM- CL-TR-857, University of Cambridge, Computer Laboratory, July 2014.
- Howard, Heidi, Malte Schwarzkopf, Anil Madhavapeddy, and Jon Crowcroft. 2015. “Raft Refloated: Do We Have Consensus?” SIGOPS Operating Systems Review 49, no. 1 (January): 12-21. <https://doi.org/10.1145/2723872.2723876>.
- Recent developments Howard, Heidi and Richard Mortier. 2019. “A Generalised Solution to Distributed Consensus.” 18 Feb 2019.

Part II Conclusion — Kết luận Phần II

Performance and scalability are important properties of any database system. The **storage engine** and **node-local read-write** path can have a larger impact on performance of the system: how quickly it can **process** requests locally. At the same time, a subsystem responsible for communication in the cluster often has a larger impact on the scalability of the database system: maximum cluster size and capacity. However, the storage engine can only be used for a limited number of use cases if it's not scalable and its performance degrades as the dataset grows. At the same time, putting a slow atomic commit protocol on top of the fastest storage engine will not yield good results.

Hiệu năng và khả năng mở rộng là những thuộc tính quan trọng của bất kỳ hệ cơ sở dữ liệu nào. Bộ máy lưu trữ (storage engine) cùng đường đọc-ghi cục bộ trên từng nút có thể tác động lớn hơn tới hiệu năng của hệ thống: tức tốc độ xử lý các yêu cầu tại chỗ. Đồng thời, phân hệ chịu trách nhiệm về giao tiếp trong cụm thường có tác động lớn hơn tới khả năng mở rộng của hệ cơ sở dữ liệu: kích thước cụm tối đa và năng lực phục vụ. Tuy nhiên, bộ máy lưu trữ chỉ dùng được cho một số ít trường hợp nếu nó không thể mở rộng và hiệu năng của nó suy giảm khi tập dữ liệu lớn dần. Mặt khác, đặt một giao thức commit nguyên tử chậm chạp lên trên bộ máy lưu trữ nhanh nhất cũng sẽ không cho kết quả tốt.

Distributed, cluster-wide, and node-local processes are interconnected, and have to be considered holistically. When designing a database system, you have to consider how different subsystems fit and work together.

Các tiến trình phân tán, ở phạm vi toàn cụm, và cục bộ trên từng nút đều liên kết với nhau và phải được xem xét một cách tổng thể. Khi thiết kế một hệ cơ sở dữ liệu, bạn phải cân nhắc cách các phân hệ khác nhau ăn khớp và phối hợp với nhau ra sao.

Part II began with a discussion of how distributed systems are different from single-node applications, and which difficulties are to be expected in such environments.

Phần II bắt đầu bằng việc bàn luận về điểm khác biệt giữa hệ phân tán và các ứng dụng đơn nút, cùng những khó khăn cần lường trước trong các môi trường như vậy.

We discussed the basic **distributed system** building blocks, different consistency models, and several important classes of distributed algorithms, some of which can be used to implement these consistency models:

Chúng ta đã thảo luận về các khối xây dựng cơ bản của hệ phân tán, các mô hình nhất quán khác nhau, và một số lớp thuật toán phân tán quan trọng, trong đó có những thuật toán có thể dùng để hiện thực hóa các mô hình nhất quán này:

Failure detection Identify remote process failures accurately and efficiently.

Phát hiện lỗi (Failure detection) Nhận biết lỗi của các tiến trình ở xa một cách chính xác và hiệu quả.

Leader election Quickly and reliably choose a single process to temporarily serve as a **coordinator**.

Bầu chọn leader (Leader election) Chọn nhanh chóng và đáng tin cậy một tiến trình duy nhất để tạm thời đóng vai trò bên điều phối.

Dissemination Reliably distribute information using peer-to-peer communication.

Lan truyền (Dissemination) Phân phối thông tin một cách đáng tin cậy bằng giao tiếp ngang hàng (peer-to-peer).

Anti-entropy Identify and repair state divergence between the nodes.

Anti-entropy Nhận biết và sửa chữa sự phân kỳ trạng thái giữa các nút.

313 Distributed transactions Execute series of operations against multiple partitions atomically.

Giao dịch phân tán (Distributed transactions) Thực thi một loạt thao tác trên nhiều phân vùng một cách nguyên tử.

Consensus Reach an agreement between remote participants while tolerating process failures.

Đồng thuận (Consensus) Đạt được thỏa thuận giữa các bên tham gia ở xa trong khi vẫn dung được lỗi tiến trình.

These algorithms are used in many database systems, message queues, schedulers, and other important infrastructure software. Using the knowledge from this book, you'll be able to better understand how they work, which, in turn, will help to make better decisions about which software to use, and identify potential problems.

Các thuật toán này được dùng trong nhiều hệ cơ sở dữ liệu, hàng đợi thông điệp, bộ lập lịch và các phần mềm hạ tầng quan trọng khác. Với kiến thức từ cuốn sách này, bạn sẽ có thể hiểu rõ hơn cách chúng vận hành, điều này đến lượt nó sẽ giúp bạn đưa ra những

quyết định tốt hơn về việc nên dùng phần mềm nào, cũng như nhận diện các vấn đề tiềm ẩn.

Further Reading

At the end of each chapter, you can find resources related to the material presented in the chapter. Here, you'll find books you can address for further study, covering both concepts mentioned in this book and other concepts. This list is not meant to be complete, but these sources contain a lot of important and useful information relevant for database systems enthusiasts, some of which is not covered in this book:

Database systems Bernstein, Philip A., Vassco Hadzilacos, and Nathan Goodman. 1987. *Concurrency Control and Recovery in Database Systems*. Boston: Addison-Wesley Longman.

Korth, Henry F. and Abraham Silberschatz. 1986. *Database System Concepts*. New York: McGraw-Hill.

Gray, Jim and Andreas Reuter. 1992. *Transaction Processing: Concepts and Techniques* (1st Ed.). San Francisco: Morgan Kaufmann.

Stonebraker, Michael and Joseph M. Hellerstein (Eds.). 1998. *Readings in Database Systems* (3rd Ed.). San Francisco: Morgan Kaufmann.

Weikum, Gerhard and Gottfried Vossen. 2001. *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery*. San Francisco: Morgan Kaufmann.

Ramakrishnan, Raghu and Johannes Gehrke. 2002. *Database Management Systems* (3 Ed.). New York: McGraw-Hill.

Garcia-Molina, Hector, Jeffrey D. Ullman, and Jennifer Widom. 2008. *Database Systems: The Complete Book* (2 Ed.). Upper Saddle River, NJ: Prentice Hall.

Bernstein, Philip A. and Eric Newcomer. 2009. *Principles of Transaction Processing* (2nd Ed.). San Francisco: Morgan Kaufmann.

Elmasri, Ramez and Shamkant Navathe. 2010. *Fundamentals of Database Systems* (6th Ed.). Boston: Addison-Wesley.

Lake, Peter and Paul Crowther. 2013. *Concise Guide to Databases: A Practical Introduction*. New York: Springer.

Härder, Theo, Caetano Sauer, Goetz Graefe, and Wey Guy. 2015. *Instant recovery with write-ahead logging*. Datenbank-Spektrum.

Distributed systems Lynch, Nancy A. *Distributed Algorithms*. 1996. San Francisco: Morgan Kaufmann.

Attiya, Hagit, and Jennifer Welch. 2004. *Distributed Computing: Fundamentals, Simulations and Advanced Topics*. Hoboken, NJ: John Wiley & Sons.

Birman, Kenneth P. 2005. *Reliable Distributed Systems: Technologies, Web Services, and Applications*. Berlin: Springer-Verlag.

Cachin, Christian, Rachid Guerraoui, and Lus Rodrigues. 2011. *Introduction to Reliable and Secure Distributed Programming* (2nd Ed.). New York: Springer.

Fokkink, Wan. 2013. *Distributed Algorithms: An Intuitive Approach*. The MIT Press.

Ghosh, Sukumar. Distributed Systems: An Algorithmic Approach (2nd Ed.). Chapman & Hall/CRC.

Tanenbaum Andrew S. and Maarten van Steen. 2017. Distributed Systems: Principles and Paradigms (3rd Ed.). Boston: Pearson.

Operating databases Beyer, Betsy, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016 Site Reliability Engineering: How Google Runs Production Systems (1st Ed.). Boston: O'Reilly Media.

Blank-Edelman, David N. 2018. Seeking SRE . Boston: O'Reilly Media.

Campbell, Laine and Charity Majors. 2017. Database Reliability Engineering: Designing and Operating Resilient Database Systems (1st Ed.). Boston: O'Reilly Media. +Sridharan, Cindy. 2018. Distributed Systems Observability: A Guide to Building Robust Systems . Boston: O'Reilly Media.

APPENDIX A Bibliography

1. [ABADI12] Abadi, Daniel. 2012. "Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story." Computer 45, no. 2 (February): 37-42. <https://doi.org/10.1109/MC.2012.33> . 2. [ABADI17] Abadi, Daniel. 2017. "Distributed consistency at scale: Spanner vs. Calvin." Fauna (blog). April 6, 2017. <https://fauna.com/blog/distributed-consistency-at-scale-spanner-vs-calvin> . 3. [ABADI13] Abadi, Daniel, Peter Boncz, Stavros Harizopoulos, Stratos Idreos, and Samuel Madden. 2013. The Design and Implementation of Modern Column- Oriented Database Systems . Hanover, MA: Now Publishers Inc. 4. [ABRAHAM13] Abraham, Ittai, Danny Dolev, and Joseph Y. Halpern. 2013. "Distributed Protocols for Leader Election: A Game-Theoretic Perspective." In Distributed Computing , edited by Yehuda Afek, 61-75. Berlin: Springer, Berlin, Heidelberg. 5. [AGGARWAL88] Aggarwal, Alok, and Jeffrey S. Vitter. 1988. "The input/output complexity of sorting and related problems." Communications of the ACM 31, no. 9 (September): 1116-1127. <https://doi.org/10.1145/48529.48535> . 6. [AGRAWAL09] Agrawal, Devesh, Deepak Ganesan, Ramesh Sitaraman, Yanlei Diao, Shashi Singh. 2009. "Lazy-Adaptive Tree: an optimized index structure for flash devices." Proceedings of the VLDB Endowment 2, no. 1 (January): 361-372. 7. [AGRAWAL08] Agrawal, Nitin, Vijayan Prabhakaran, Ted Wobber, John D. Davis, Mark Manasse, and Rina Panigrahy. 2008. "Design tradeoffs for SSD performance." USENIX 2008 Annual Technical Conference (ATC '08) , 57-70. USE- NIX. 8. [AGUILERA97] Aguilera, Marcos K., Wei Chen, and Sam Toueg. 1997. "Heartbeat: a Timeout-Free Failure Detector for Quiescent Reliable Communication." 317 In Distributed Algorithms , edited by M. Mavronicolas and P. Tsigas, 126-140. Berlin: Springer, Berlin, Heidelberg. 9. [AGUILERA01] Aguilera, Marcos Kawazoe, Carole Delporte-Gallet, Hugues Fauconnier, and Sam Toueg. 2001. "Stable Leader Election." In Proceedings of the 15th International Conference on Distributed Computing (DISC '01) , edited by Jennifer L. Welch, 108-122. London: Springer-Verlag. 10. [AGUILERA16] Aguilera, M. K., and D. B. Terry. 2016. "The Many Faces of Consistency." Bulletin of the Technical Committee on Data Engineering 39, no. 1 (March): 3-13. 11. [ALHOUMAILY10] Al-Houmaily, Yousef J. 2010. "Atomic commit protocols, their integration, and their optimisations in distributed database systems." International Journal of Intelligent Information and Database Systems 4, no. 4 (September): 373-412. <https://doi.org/10.1504/IJIDS.2010.035582> . 12. [ARJOMANDI83] Arjomandi, Eshrat, Michael J. Fischer, and Nancy A. Lynch. 1983. "Efficiency of Synchronous Versus Asynchronous Distributed Systems." Journal of the ACM 30, no. 3 (July): 449-456. <https://doi.org/10.1145/2402.322387> . 13. [ARULRAJ17] Arulraj, J. and A. Pavlo. 2017. "How to Build a Non-Volatile Memory Database Management System." In Proceedings of the 2017 ACM International Conference on Management of Data : 1753-1758. <https://doi.org/10.1145/3035918.3054780> . 14. [ATHANASSOULIS16] Athanassoulis, Manos,

- Michael S. Kester, Lukas M. Maas, Radu Stoica, Stratos Idreos, Anastasia Ailamaki, and Mark Callaghan. 2016. “Designing Access Methods: The RUM Conjecture.” In International Conference on Extending Database Technology (EDBT) . <https://stratos.seas.harvard.edu/files/stratos/files/rum.pdf> . 15. [ATTIYA94] Attiyaand, Hagit and Jennifer L. Welch. 1994. “Sequential consistency versus linearizability.” *ACM Transactions on Computer Systems* 12, no. 2 (May): 91-122. <https://doi.org/10.1145/176575.176576> . 16. [BABAUGLU93] Babaoglu, Ozalp and Sam Toueg. 1993. “Understanding Non- Blocking Atomic Commitment.” Technical Report. University of Bologna. 17. [BAILIS14a] Bailis, Peter. 2014. “Linearizability versus Serializability.” Highly Available, Seldom Consistent (blog). September 24, 2014. <https://www.bailis.org/blog/linearizability-versus-serializability> . 18. [BAILIS14b] Bailis, Peter, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2014. “Coordination Avoidance in Database Systems.” *Proceedings of the VLDB Endowment* 8, no. 3 (November): 185-196. <https://doi.org/10.14778/2735508.2735509> .
1984. [BAILIS14c] Bailis, Peter, Alan Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2014. “Scalable Atomic Visibility with RAMP Transactions.” *ACM Transactions on Database Systems* 41, no. 3 (July). <https://doi.org/10.1145/2909870> . 20. [BARTLETT16] Bartlett, Robert P. III, and Justin McCrary. 2016. “How Rigged Are Stock Markets?: Evidence From Microsecond Timestamps.” UC Berkeley Public Law Research Paper. <https://doi.org/10.2139/ssrn.2812123> . 21. [BAUDET19] Baudet, Mathieu, Avery Ching, Andrey Chursin, George Danezis, François Garillot, Zekun Li, Dahlia Malkhi, Oded Naor, Dmitri Perelman, and Alberto Sonnino. 2019. “State Machine Replication in the Libra Blockchain.” <https://developers.libra.org/docs/assets/papers/libra-consensus-state-machine-replication-in-the-libra-blockchain.pdf> . 22. [BAYER72] Bayer, R., and E. M. McCreight. 1972. “Organization and maintenance of large ordered indices.” *Acta Informatica* 1, no. 3 (September): 173-189. <https://doi.org/10.1007/BF00288683> . 23. [BEDALY69] Belady, L. A., R. A. Nelson, and G. S. Shedler. 1969. “An anomaly in space-time characteristics of certain programs running in a paging machine.” *Communications of the ACM* 12, no. 6 (June): 349-353. <https://doi.org/10.1145/363011.363155> . 24. [BENDER05] Bender, Michael A., Erik D. Demaine, and Martin Farach-Colton.
1985. “Cache-Oblivious B-Trees.” *SIAM Journal on Computing* 35, no. 2 (August): 341-358. <https://doi.org/10.1137/S0097> . 25. [BERENSON95] Berenson, Hal, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. 1995. “A critique of ANSI SQL isolation levels.” *ACM SIGMOD Record* 24, no. 2 (May): 1-10. <https://doi.org/10.1145/568271.223785> . 26. [BERNSTEIN87] Bernstein, Philip A., Vassco Hadzilacos, and Nathan Goodman. 1987. *Concurrency Control and Recovery in Database Systems* . Boston: Addison- Wesley Longman. 27. [BERNSTEIN09] Bernstein, Philip A. and Eric Newcomer. 2009. *Principles of Transaction Processing* . San Francisco: Morgan Kaufmann. 28. [BHATTACHARJEE17] Bhattacharjee, Abhishek, Daniel Lustig, and Margaret Martonosi. 2017. *Architectural and Operating System Support for Virtual Memory* . San Rafael, CA: Morgan & Claypool Publishers. 29. [BIRMAN07] Birman, Ken. 2007. “The promise, and limitations, of gossip protocols.” *ACM SIGOPS Operating Systems Review* 41, no. 5 (October): 8-13. <https://doi.org/10.1145/1317379.1317382> . 30. [BIRMAN10] Birman, Ken. 2010. “A History of the Virtual Synchrony Replication Model” In *Replication* , edited by Bernadette Charron-Bost, Fernando Pedone, and André Schiper, 91-120. Berlin: Springer-Verlag, Berlin, Heidelberg.
1986. [BIRMAN06] Birman, Ken, Coimbatore Chandrasekaran, Danny Dolev, Robbert vanRenesse. 2006. “How the Hidden Hand Shapes the Market for Software Reliability.” In *First Workshop on Applied Software Reliability (WASR 2006)* . IEEE. 32. [BIYIKOGLU13] Biyikoglu, Cihan. 2013. “Under the Hood: Redis CRDTs (Conflict-free Replicated Data Types).” <http://lp.redislabs.com/rs/915-NFD-128/images/WP-RedisLabs-Redis-Conflict-free-Replicated-Data-Types.pdf> . 33. [BJØRLING17] Bjørling, Matias, Javier González, and Philippe Bonnet. 2017. “LightNVM: the Linux open-channel SSD subsystem.” In *Proceedings of the 15th Usenix Conference on File and Storage*

- Technologies (FAST'17) , 359-373. USENIX. 34. [BLOOM70] Bloom, Burton H. 1970. "Space/time trade-offs in hash coding with allowable errors." *Communications of the ACM* 13, no. 7 (July): 422-426. [https:// doi.org/10.1145/362686.362692](https://doi.org/10.1145/362686.362692) . 35. [BREWER00] Brewer, Eric. 2000. "Towards robust distributed systems." *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing (PODC '00)* . New York: Association for Computing Machinery. [https:// doi.org/10.1145/343477.343502](https://doi.org/10.1145/343477.343502) . 36. [BRZEZINSKI03] Brzezinski, Jerzy, Cezary Sobaniec, and Dariusz Wawrzyniak.
1987. "Session Guarantees to Achieve PRAM Consistency of Replicated Shared Objects." In *Parallel Processing and Applied Mathematics* , 1-8. Berlin: Springer, Berlin, Heidelberg. 37. [BUCHMAN18] Buchman, Ethan, Jae Kwon, and Zarko Milosevic. 2018. "The latest gossip on BFT consensus." <https://arxiv.org/pdf/1807.04938.pdf> . 38. [CACHIN11] Cachin, Christian, Rachid Guerraoui, and Luis Rodrigues. 2011. *Introduction to Reliable and Secure Distributed Programming* (2nd Ed.) . New York: Springer. 39. [CASTRO99] Castro, Miguel. and Barbara Liskov. 1999. "Practical Byzantine Fault Tolerance." In *OSDI '99 Proceedings of the third symposium on Operating systems design and implementation* , 173-186. 40. [CESATI05] Cesati, Marco, and Daniel P. Bovet. 2005. *Understanding the Linux Kernel* . Third Edition. Sebastopol: O'Reilly Media, Inc. 41. [CHAMBERLIN81] Chamberlin, Donald D., Morton M. Astrahan, Michael W. Blasgen, James N. Gray, W. Frank King, Bruce G. Lindsay, Raymond Lorie, James W. Mehl, Thomas G. Price, Franco Putzolu, Patricia Griffiths Selinger, Mario Schkolnick, Donald R. Slutz, Irving L. Traiger, Bradford W. Wade, and Robert A. Yost. 1981. "A history and evaluation of System R." *Communications of the ACM* 24, no. 10 (October): 632-646. <https://doi.org/10.1145/358769.358784> . 42. [CHANDRA07] Chandra, Tushar D., Robert Griesemer, and Joshua Redstone.
1988. "Paxos made live: an engineering perspective." In *Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing (PODC '07)* , 398-407. New York: Association for Computing Machinery. [https://doi.org/ 10.1145/1281100.1281103](https://doi.org/10.1145/1281100.1281103) . 43. [CHANDRA96] Chandra, Tushar Deepak, and Sam Toueg. 1996. "Unreliable failure detectors for reliable distributed systems." *Journal of the ACM* 43, no. 2 (March): 225-267. <https://doi.org/10.1145/226643.226647> . 44. [CHANG79] Chang, Ernest, and Rosemary Roberts. 1979. "An improved algorithm for decentralized extrema-finding in circular configurations of processes." *Communications of the ACM* 22, no. 5 (May): 281-283. [https://doi.org/ 10.1145/359104.359108](https://doi.org/10.1145/359104.359108) . 45. [CHANG06] Chang, Fay, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. 2006. "Bigtable: A Distributed Storage System for Structured Data." In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)* . USENIX. 46. [CHAZELLE86] Chazelle, Bernard, and Leonidas J. Guibas. 1986. "Fractional Cascading, A Data Structuring Technique." *Algorithmica* 1: 133-162. [https:// doi.org/10.1007/BF01840440](https://doi.org/10.1007/BF01840440) . 47. [CHOCKLER15] Chockler, Gregory, and Dahlia Malkhi. 2015. "Active disk paxos with infinitely many processes." In *Proceedings of the twenty-first annual symposium on Principles of distributed computing (PODC '02)* , 78-87. New York: Association for Computing Machinery. <https://doi.org/10.1145/571825.571837> . 48. [COMER79] Comer, Douglas. 1979. "Ubiquitous B-Tree." *ACM Computing Survey* 11, no. 2 (June): 121-137. <https://doi.org/10.1145/356770.356776> . 49. [CORBET18] Corbet, Jonathan. 2018. "PostgreSQL's fsync() surprise." [https:// lwn.net/Articles/752063](https://lwn.net/Articles/752063) . 50. [CORBETT12] Corbett, James C., Jeffrey Dean, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. 2012. "Spanner: Google's Globally-Distributed Database." In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI '12)* , 261-264.

- USENIX. 51. [CORMODE04] Cormode, G. and S. Muthukrishnan. 2004. "An improved data stream summary: The count-min sketch and its applications." *Journal of Algorithms* 55, No. 1 (April): 58-75. <https://doi.org/10.1016/j.jalgor.2003.12.001> . 52. [CORMODE11] Cormode, Graham, and S. Muthukrishnan. 2011. "Approximating Data with the Count-Min Data Structure." <http://dimacs.rutgers.edu/~graham/pubs/papers/cmsoft.pdf> .
1989. [CORMODE12] Cormode, Graham and Senthilmurugan Muthukrishnan. 2012. "Approximating Data with the Count-Min Data Structure." 54. [CHRISTIAN91] Cristian, Flavin. 1991. "Understanding fault-tolerant distributed systems." *Communications of the ACM* 34, no. 2 (February): 56-78. <https://doi.org/10.1145/102792.102801> . 55. [DAILY13] Daily, John. 2013. "Clocks Are Bad, Or, Welcome to the Wonderful World of Distributed Systems." Riak (blog). November 12, 2013. <https://riak.com/clocks-are-bad-or-welcome-to-distributed-systems> . 56. [DECANDIA07] DeCandia, Giuseppe, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voshall, and Werner Vogels. 2007. "Dynamo: amazon's highly available key-value store." *SIGOPS Operating Systems Review* 41, no. 6 (October): 205-220. <https://doi.org/10.1145/1323293.1294281> . 57. [DECHEV10] Dechev, Damian, Peter Pirkelbauer, and Bjarne Stroustrup. 2010. "Understanding and Effectively Preventing the ABA Problem in Descriptor- Based Lock-Free Designs." *Proceedings of the 2010 13th IEEE International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing (ISORC '10)* : 185–192. <https://doi.org/10.1109/ISORC.2010.10> . 58. [DEMAINE02] Demaine, Erik D. 2002. "Cache-Oblivious Algorithms and Data Structures." In *Lecture Notes from the EEF Summer School on Massive Data Sets* . Denmark: University of Aarhus. 59. [DEMERS87] Demers, Alan, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. 1987. "Epidemic algorithms for replicated database maintenance." In *Proceedings of the sixth annual ACM Symposium on Principles of distributed computing (PODC '87)* , 1-12. New York: Association for Computing Machinery. <https://doi.org/10.1145/41840.41841> . 60. [DENNING68] Denning, Peter J. 1968. "The working set model for program behavior". *Communications of the ACM* 11, no. 5 (May): 323-333. <https://doi.org/10.1145/363095.363141> . 61. [DIACONU13] Diaconu, Cristian, Craig Freedman, Erik Ismert, Per-Åke Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. 2013. "Hekaton: SQL Server's Memory-Optimized OLTP Engine." In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD '13)* , 1243-1254. New York: Association for Computing Machinery. <https://doi.org/10.1145/2463676.2463710> . 62. [DOWNEY12] Downey, Jim. 2012. "Be Careful with Sloppy Quorums." Jim Downey (blog). March 5, 2012. <https://jimdowney.net/2012/03/05/be-careful-with-sloppy-quorums> .
1990. [DREPPER07] Drepper, Ulrich. 2007. *What Every Programmer Should Know About Memory* . Boston: Red Hat, Inc. 64. [DUNAGAN04] Dunagan, John, Nicholas J. A. Harvey, Michael B. Jones, Dejan Kostić, MarvinTheimer, and Alec Wolman. 2004. "FUSE: lightweight guaranteed distributed failure notification." In *Proceedings of the 6th conference on Symposium on Operating Systems Design & Implementation - Volume 6 (OSDI'04)* , 11-11. USENIX. 65. [DWORK88] Dwork, Cynthia, Nancy Lynch, and Larry Stockmeyer. 1988. "Consensus in the presence of partial synchrony." *Journal of the ACM* 35, no. 2 (April): 288-323. <https://doi.org/10.1145/42282.42283> . 66. [EINZIGER15] Einziger, Gil and Roy Friedman. 2015. "A formal analysis of conservative update based approximate counting." In *2015 International Conference on Computing, Networking and Communications (ICNC)* , 260-264. IEEE. 67. [EINZIGER17] Einziger, Gil, Roy Friedman, and Ben Manes. 2017. "TinyLFU: A Highly Efficient Cache Admission Policy." In *2014 22nd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing* , 146-153. IEEE. 68. [ELLIS11] Ellis, Jonathan. 2011. "Understanding Hinted Handoff." Datastax (blog). May 31, 2011. <https://www.datastax.com/dev/blog/understanding-hinted-handoff> . 69. [ELLIS13] Ellis,

- Jonathan. 2013. "Why Cassandra doesn't need vector clocks." Datastax (blog). September 3, 2013. <https://www.datastax.com/dev/blog/why-cassandra-doesnt-need-vector-clocks> . 70.
- [ELMASRI11] Elmasri, Ramez and Shamkant Navathe. 2011. *Fundamentals of Database Systems* (6th Ed.) . Boston: Pearson. 71. [FEKETE04] Fekete, Alan, Elizabeth O'Neil, and Patrick O'Neil. 2004. "A read-only transaction anomaly under snapshot isolation." *ACM SIGMOD Record* 33, no. 3 (September): 12-14. <https://doi.org/10.1145/1031570.1031573> . 72. [FISCHER85] Fischer, Michael J., Nancy A. Lynch, and Michael S. Paterson.
1991. "Impossibility of distributed consensus with one faulty process." *Journal of the ACM* 32, 2 (April): 374-382. <https://doi.org/10.1145/3149.214121> . 73. [FLAJOLET12] Flajolet, Philippe, Eric Fusy, Olivier Gandouet, and Frédéric Meunier. 2012. "HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm." In *AOFA '07: Proceedings of the 2007 International Conference on Analysis of Algorithms* . 74. [FOWLER11] Fowler, Martin. 2011. "The LMAX Architecture." Martin Fowler . July 12, 2011. <https://martinfowler.com/articles/lmax.html> .
1992. [FOX99] Fox, Armando and Eric A. Brewer. 1999. "Harvest, Yield, and Scalable Tolerant Systems." In *Proceedings of the Seventh Workshop on Hot Topics in Operating Systems* , 174-178. 76. [FREILING11] Freiling, Felix C., Rachid Guerraoui, and Petr Kuznetsov. 2011. "The failure detector abstraction." *ACM Computing Surveys* 43, no. 2 (January): Article 9. <https://doi.org/10.1145/1883612.1883616> . 77. [MOLINA82] Garcia-Molina, H. 1982. "Elections in a Distributed Computing System." *IEEE Transactions on Computers* 31, no. 1 (January): 48-59. <https://dx.doi.org/10.1109/TC.1982.1675885> . 78. [MOLINA92] Garcia-Molina, H. and K. Salem. 1992. "Main Memory Database Systems: An Overview." *IEEE Transactions on Knowledge and Data Engineering* 4, no. 6 (December): 509-516. <https://doi.org/10.1109/69.180602> . 79. [MOLINA08] Garcia-Molina, Hector, Jeffrey D. Ullman, and Jennifer Widom. 2008. *Database Systems: The Complete Book* (2nd Ed.). Boston: Pearson. 80. [GEORGOPOULOS16] Georgopoulos, Georgios. 2016. "Memory Consistency Models of Modern CPUs." <https://es.cs.uni-kl.de/publications/datarsg/Geor16.pdf>. 81. [GHOLIPOUR09] Gholipour, Majid, M. S. Kordafshari, Mohsen Jahanshahi, and Amir Masoud Rahmani. 2009. "A New Approach For Election Algorithm in Distributed Systems." In *2009 Second International Conference on Communication Theory, Reliability, and Quality of Service* , 70-74. IEEE. <https://doi.org/10.1109/CTRQ.2009.32> . 82. [GIAMPAOLO98] Giampaolo, Dominic. 1998. *Practical File System Design with the be File System* . San Francisco: Morgan Kaufmann. 83. [GILAD17] Gilad, Yossi, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. 2017. "Algorand: Scaling Byzantine Agreements for Cryptocurrencies." *Proceedings of the 26th Symposium on Operating Systems Principles* (October): 51-68. <https://doi.org/10.1145/3132747.3132757> . 84. [GILBERT02] Gilbert, Seth and Nancy Lynch. 2002. "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services." *ACM SIGACT News* 33, no. 2 (June): 51-59. <https://doi.org/10.1145/564585.564601> . 85. [GILBERT12] Gilbert, Seth and Nancy Lynch. 2012. "Perspectives on the CAP Theorem." *Computer* 45, no. 2 (February): 30-36. <https://doi.org/10.1109/MC.2011.389> . 86. [GOMES17] Gomes, Victor B. F., Martin Kleppmann, Dominic P. Mulligan, and Alastair R. Beresford. 2017. "Verifying strong eventual consistency in distributed systems." *Proceedings of the ACM on Programming Languages* 1 (October). <https://doi.org/10.1145/3133933> .
1993. [GONÇALVES15] Gonçalves, Ricardo, Paulo Sérgio Almeida, Carlos Baquero, and Victor Fonte. 2015. "Concise Server-Wide Causality Management for Eventually Consistent Data Stores." In *Distributed Applications and Interoperable Systems* , 66-79. Berlin: Springer. 88. [GOOSSAERT14] Goossaert, Emmanuel. 2014. "Coding For SSDs." *CodeCapsule* (blog). February 12, 2014. <http://codecapsule.com/2014/02/12/coding-for-ssds-part-1-introduction-and-table-of-contents> . 89. [GRAEFE04] Graefe, Goetz. 2004. "Write-Optimized B-Trees." In *Proceedings of the Thirtieth international conference on Very large data bases - Volume 30*

- (VLDB '04) , 672-683. VLDB Endowment. 90. [GRAEFE07] Graefe, Goetz. 2007. "Hierarchical locking in B-tree indexes." <https://www.semanticscholar.org/paper/Hierarchical-locking-in-B-tree-indexes-Graefe/270669b1eb0d31a99fe99bec67e47e9b11b4553f> . 91. [GRAEFE10] Graefe, Goetz. 2010. "A survey of B-tree locking techniques." *ACM Transactions on Database Systems* 35, no. 3, (July). <https://doi.org/10.1145/1806907.1806908> . 92. [GRAEFE11] Graefe, Goetz. 2011. "Modern B-Tree Techniques." *Foundations and Trends in Databases* 3, no. 4 (April): 203-402. <https://doi.org/10.1561/19000000028> . 93. [GRAY05] Gray, Jim, and Catharine van Ingen. 2005. "Empirical Measurements of Disk Failure Rates and Error Rates." Accessed March 4, 2013. <https://arxiv.org/pdf/cs/0701166.pdf> . 94. [GRAY04] Gray, Jim, and Leslie Lamport. 2004. "Consensus on Transaction Commit." *ACM Transactions on Database Systems* 31, no. 1 (March): 133-160. <https://doi.org/10.1145/1132863.1132867> . 95. [GUERRAOUI07] Guerraoui, Rachid. 2007. "Revisiting the relationship between non-blocking atomiccommitment and consensus." In *Distributed Algorithms* , 87-100. Berlin: Springer, Berlin, Heidelberg. <https://doi.org/10.1007/BFb0022140> . 96. [GUERRAOUI97] Guerraoui, Rachid, and André Schiper. 1997. "Consensus: The Big Misunderstanding." In *Proceedings of the Sixth IEEE Computer Society Workshop on Future Trends of Distributed Computing Systems* , 183-188. IEEE. 97. [GUPTA01] Gupta, Indranil, Tushar D. Chandra, and Germán S. Goldszmidt.
1994. "On scalable and efficient distributed failure detectors." In *Proceedings of the twentieth annual ACM symposium on Principles of distributed computing (PODC '01)* New York: Association for Computing Machinery. <https://doi.org/10.1145/383962.384010> . 98. [HADZILACOS05] Hadzilacos, Vassos. 2005. "On the relationship between the atomic commitment and consensus problems." In *Fault-Tolerant Distributed Computing* , 201-208. London: Springer-Verlag.
1995. [HAERDER83] Haerder, Theo, and Andreas Reuter. 1983. "Principles of transaction-oriented database recovery." *ACM Computing Surveys* 15 no. 4 (December):287–317. <https://doi.org/10.1145/289.291> . 100. [HALE10] Hale, Coda. 2010. "You Can't Sacrifice Partition Tolerance." Coda Hale (blog). <https://codahale.com/you-cant-sacrifice-partition-tolerance> . 101. [HALPERN90] Halpern, Joseph Y., and Yoram Moses. 1990. "Knowledge and common knowledge in a distributed environment." *Journal of the ACM* 37, no. 3 (July): 549-587. <https://doi.org/10.1145/79147.79161> . 102. [HARDING17] Harding, Rachael, Dana Van Aken, Andrew Pavlo, and Michael Stonebraker. 2017. "An Evaluation of Distributed Concurrency Control." *Proceedings of the VLDB Endowment* 10, no. 5 (January): 553-564. <https://doi.org/10.14778/3055540.3055548> . 103. [HAYASHIBARA04] Hayashibara, N., X. Defago, R.Yared, and T. Katayama.
1996. "The Φ Accrual Failure Detector." In *IEEE Symposium on Reliable Distributed Systems* , 66-78. <https://doi.org/10.1109/RELDIS.2004.1353004> . 104. [HELLAND15] Helland, Pat. 2015. "Immutability Changes Everything." *Queue* 13, no. 9 (November). <https://doi.org/10.1145/2857274.2884038> . 105. [HELLERSTEIN07] Hellerstein, Joseph M., Michael Stonebraker, and James Hamilton. 2007. "Architecture of a Database System." *Foundations and Trends in Databases* 1, no. 2 (February): 141-259. <https://doi.org/10.1561/19000000002> . 106. [HERLIHY94] Herlihy, Maurice. 1994. "Wait-Free Synchronization." *ACM Transactions on Programming Languages and Systems* 13, no. 1 (January): 124-149. <http://dx.doi.org/10.1145/114005.102808> . 107. [HERLIHY10] Herlihy, Maurice, Yossi Lev, Victor Luchangco, and Nir Shavit.
1997. "A Provably Correct Scalable Concurrent Skip List." <https://www.cs.tau.ac.il/~shanir/nir-pubs-web/Papers/OPODIS2006-BA.pdf> . 108. [HERLIHY90] Herlihy, Maurice P., and Jeannette M. Wing. 1990. "Linearizability: a correctness condition for concurrent object." *ACM Transactions on Programming Languages and Systems* 12, no. 3 (July): 463-492. <https://doi.org/10.1145/78969.78972> . 109. [HOWARD14] Howard, Heidi. 2014. "ARC: Analysis of Raft Consensus." Technical Report UCAM-CL-TR-857. Cambridge: University of Cambridge 110. [HOWARD16] Howard, Heidi, Dahlia Malkhi, and Alexander Spiegelman. 2016. "Flexible

- Paxos: Quorum intersection revisited.” <https://arxiv.org/abs/1608.06696> . 111. [HOWARD19] Howard, Heidi, and Richard Mortier. 2019. “A Generalised Solution to Distributed Consensus.” <https://arxiv.org/abs/1902.06776> . 112. [HUNT10] Hunt, Patrick, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. 2010. “ZooKeeper: wait-free coordination for internet-scale systems.” In Proceedings of the 2010 USENIX conference on USENIX annual technical conference (USENIXATC’10) , 11. USENIX. 113. [INTEL14] Intel Corporation. 2014. “Partition Alignment of Intel® SSDs for Achieving Maximum Performance and Endurance.” (February). <https://www.intel.com/content/dam/www/public/us/en/documents/technology-briefs/ssd-partition-alignment-tech-brief.pdf> . 114. [JELASITY04] Jelasity, Márk, Rachid Guerraoui, Anne-Marie Kermarrec, and Maarten van Steen. 2004. “The Peer Sampling Service: Experimental Evaluation of Unstructured Gossip-Based Implementations.” In Middleware ’04 Proceedings of the 5th ACM/IFIP/USENIX international conference on Middleware , 79-98. Berlin: Springer-Verlag, Berlin, Heidelberg. 115. [JELASITY07] Jelasity, Márk, Spyros Voulgaris, Rachid Guerraoui, Anne-Marie Kermarrec, and Maarten van Steen. 2007. “Gossip-based Peer Sampling.” ACM Transactions on Computer Systems 25, no. 3 (August). <http://doi.org/10.1145/1275517.1275520> . 116. [JONSON94] Johnson, Theodore, and Dennis Shasha. 1994. “2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm.” In VLDB ’94 Proceedings of the 20th International Conference on Very Large Data Bases , 439-450. San Francisco: Morgan Kaufmann. 117. [JUNQUEIRA07] Junqueira, Flavio, Yanhua Mao, and Keith Marzullo. 2007. “Classic Paxos vs. fast Paxos: caveat emptor.” In Proceedings of the 3rd workshop on Hot Topics in System Dependability (HotDep’07) . USENIX. 118. [JUNQUEIRA11] Junqueira, Flavio P., Benjamin C. Reed, and Marco Serafini.
1998. “Zab: High-performance broadcast for primary-backup systems.” 2011 IEEE/IFIP 41st International Conference on Dependable Systems & Networks (DSN) (June): 245–256. <https://doi.org/10.1109/DSN.2011.5958223> . 119. [KANNAN18] Kannan, Sudarsun, Nitish Bhat, Ada Gavrilovska, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2018. “Redesigning LSMs for Nonvolatile Memory with NoveLSM.” In USENIX ATC ’18 Proceedings of the 2018 USENIX Conference on Usenix Annual Technical Conference , 993-1005. USENIX. 120. [KARGER97] Karger, D., E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin. 1997. “Consistent hashing and random trees: distributed caching protocols for relieving hot spots on the World Wide Web.” In STOC ’97 Proceedings of the twenty-ninth annual ACM symposium on Theory of computing , 654-663. New York: Association for Computing Machinery. 121. [KEARNEY17] Kearney, Joe. 2017. “Two Phase Commit an old friend.” Joe’s Mots (blog). January 6, 2017. <https://www.joekearney.co.uk/posts/two-phase-commit> .
1999. [KEND94] Kendall, Samuel C., Jim Waldo, Ann Wollrath, and Geoff Wyant.
2000. “A Note on Distributed Computing.” Technical Report. Mountain View, CA: Sun Microsystems, Inc. 123. [KREMARREC07] Kermarrec, Anne-Marie, and Maarten van Steen. 2007. “Gossiping in distributed systems.” SIGOPS Operating Systems Review 41, no. 5 (October): 2-7. <https://doi.org/10.1145/1317379> . 124. [KERRISK10] Kerrisk, Michael. 2010. The Linux Programming Interface . San Francisco: No Starch Press. 125. [KHANCHANDANI18] Khanchandani, Pankaj, and Roger Wattenhofer. 2018. “Reducing Compare-and-Swap to Consensus Number One Primitives.” <https://arxiv.org/abs/1802.03844> . 126. [KIM12] Kim, Jaehong, Sangwon Seo, Dawoon Jung, Jin-Soo Kim, and Jaehyuk Huh. 2012. “Parameter-Aware I/O Management for Solid State Disks (SSDs).” IEEE Transactions on Computers 61, no. 5 (May): 636-649. <https://doi.org/10.1109/TC.2011.76> . 127. [KINGSBURY18a] Kingsbury, Kyle. 2018. “Sequential Consistency.” <https://jepsen.io/consistency/models/sequential> . 2018. 128. [KINGSBURY18b] Kingsbury, Kyle. 2018. “Strong consistency models.” Aphyr (blog). August 8, 2018. <https://aphyr.com/posts/313-strong-consistency-models> . 129. [KLEPPMANN15] Kleppmann, Martin. 2015. “Please stop calling databases CP or AP.” Martin Kleppmann (blog). May 11, 2015. <https://martin.kleppmann.com/2015/05/11/please-stop-calling-databases-cp>

- or-ap.html . 130. [KLEPPMANN14] Kleppmann, Martin, and Alastair R. Beresford. 2014. "A Conflict-Free Replicated JSON Datatype." <https://arxiv.org/abs/1608.03960> . 131. [KNUTH97] Knuth, Donald E. 1997. *The Art of Computer Programming, Volume 1 (3rd Ed.): Fundamental Algorithms* . Boston: Addison-Wesley Longman. 132. [KNUTH98] Knuth, Donald E. 1998. *The Art of Computer Programming, Volume 3: (2nd Ed.): Sorting and Searching* . Boston: Addison-Wesley Longman. 133. [KOOPMAN15] Koopman, Philip, Kevin R. Driscoll, and Brendan Hall. 2015. "Selection of Cyclic Redundancy Code and Checksum Algorithms to Ensure Critical Data Integrity." U.S. Department of Transportation Federal Aviation Administration . https://www.faa.gov/aircraft/air_cert/design_approvals/air_software/media/TC-14-49.pdf . 134. [KORDAFSHARI05] Kordafshari, M. S., M. Gholipour, M. Mosakhani, A. T. Haghighat, and M. Dehghan. 2005. "Modified bully election algorithm in distributed systems." *Proceedings of the 9th WSEAS International Conference on Computers (ICCOMP'05)* , edited by Nikos E. Mastorakis, Article 10. Stevens Point: World Scientific and Engineering Academy and Society.
2001. [KRASKA18] Kraska, Time, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. "The Case for Learned Index Structures." In *SIGMOD '18 Proceedings of the 2018 International Conference on Management of Data* , 489-504. New York: Association for Computing Machinery. 136. [LAMPOR77] Lamport, Leslie. 1977. "Proving the Correctness of Multiprocess Programs." *IEEE Transactions on Software Engineering* 3, no. 2 (March): 125-143. <https://doi.org/10.1109/TSE.1977.229904> . 137. [LAMPOR78] Lamport, Leslie. 1978. "Time, Clocks, and the Ordering of Events in a Distributed System." *Communications of the ACM* 21, no. 7 (July): 558-565 138. [LAMPOR79] Lamport, Leslie. 1979. "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs." *IEEE Transactions on Computers* 28, no. 9 (September): 690-691. <https://doi.org/10.1109/TC.1979.1675439> . 139. [LAMPOR98] Lamport, Leslie. 1998. "The part-time parliament." *ACM Transactions on Computer Systems* 16, no. 2 (May): 133-169. <https://doi.org/10.1145/279227.279229> . 140. [LAMPOR01] Lamport, Leslie. 2001. "Paxos Made Simple." *ACM SIGACT News (Distributed Computing Column)* 32, no. 4 (December): 51-58. <https://www.microsoft.com/en-us/research/publication/paxos-made-simple> . 141. [LAMPOR05] Lamport, Leslie. 2005. "Generalized Consensus and Paxos." <https://www.microsoft.com/en-us/research/publication/generalized-consensus-and-paxos> . 142. [LAMPOR06] Lamport, Leslie. 2006. "Fast Paxos." *Distributed Computing* 19, no. 2 (July): 79-103. <https://doi.org/10.1007/s00446-006-0005-x> . 143. [LAMPOR09] Lamport, Leslie, Dahlia Malkhi, and Lidong Zhou. 2009. "Vertical Paxos and Primary-Backup Replication." In *PODC '09 Proceedings of the 28th ACM symposium on Principles of distributed computing* , 312-313. <https://doi.org/10.1145/1582716.1582783> . 144. [LAMPSON01] Lampson, Butler. 2001. "The ABCD's of Paxos." In *PODC '01 Proceedings of the twentieth annual ACM symposium on Principles of distributed computing* , 13. <https://doi.org/10.1145/383962.383969> . 145. [LAMPSON79] Lampson, Butler W., and Howard E. 1979. "Crash Recovery in a Distributed Data Storage System." <https://www.microsoft.com/en-us/research/publication/crash-recovery-in-a-distributed-data-storage-system> . 146. [LARRIVEE15] Larrivee, Steve. 2015. "Solid State Drive Primer." Cactus Technologies (blog). February 9th, 2015. <https://www.cactus-tech.com/resources/blog/details/solid-state-drive-primer-1-the-basic-nand-flash-cell> .
2002. [LARSON81] Larson, Per-Åke, and Åbo Akedemi. 1981. "Analysis of index-sequential files with overflow chaining". *ACM Transactions on Database Systems* . 6, no. 4 (December): 671-680. <https://doi.org/10.1145/319628.319665> . 148. [LEE15] Lee, Collin, Seo Jin Park, Ankita Kejriwal, Satoshi Matsushita, and John Ousterhout. 2015. "Implementing linearizability at large scale and low latency." In *SOSP '15 Proceedings of the 25th Symposium on Operating Systems Principles* , 71-86. <https://doi.org/10.1145/2815400.2815416> . 149. [LEHMAN81] Lehman, Philip L., and s. Bing Yao. 1981. "Efficient locking for concurrent operations on B-trees." *ACM Transactions on Database Systems* 6, no. 4 (December): 650-670. <https://doi.org/10.1145/319628.319663> . 150.

- [LEITAO07] Leita, Joao, Jose Pereira, and Luis Rodrigues. 2007. "Epidemic Broadcast Trees." In SRDS '07 Proceedings of the 26th IEEE International Symposium on Reliable Distributed Systems , 301-310. IEEE. 151. [LEVANDOSKI14] Levandoski, Justin J., David B. Lomet, and Sudipta Sengupta.
2003. "The Bw-Tree: A B-tree for new hardware platforms." In Proceedings of the 2013 IEEE International Conference on Data Engineering (ICDE '13) , 302-313. IEEE. <https://doi.org/10.1109/ICDE.2013.6544> . 152. [LI10] Li, Yanan, Bingsheng He, Robin Jun Yang, Qiong Luo, and Ke Yi. 2010. "Tree Indexing on Solid State Drives." Proceedings of the VLDB Endowment 3, no. 1-2 (September): 1195-1206. <https://doi.org/10.14778/1920841.1920990> . 153. [LIPTON88] Lipton, Richard J., and Jonathan S. Sandberg. 1988. "PRAM: A scalable shared memory." Technical Report, Princeton University. <https://www.cs.princeton.edu/research/techreps/TR-180-88> . 154. [LLOYD11] Lloyd, W., M. J. Freedman, M. Kaminsky, and D. G. Andersen. 2011. "Don't settle for eventual: scalable causal consistency for wide-area storage with COPS." In Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP '11) , 401-416. New York: Association for Computing Machinery. <https://doi.org/10.1145/2043556.2043593> . 155. [LLOYD13] Lloyd, W., M. J. Freedman, M. Kaminsky, and D. G. Andersen. 2013. "Stronger semantics for low-latency geo-replicated storage." In 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI '13) , 313-328. USENIX. 156. [LU16] Lu, Lanyue, Thanumalayan Sankaranarayanan Pillai, Hariharan Gopalakrishnan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2017. "WiscKey: Separating Keys from Values in SSD-Conscious Storage." ACM Transactions on Storage (TOS) 13, no. 1 (March): Article 5. <https://doi.org/10.1145/3033273> .
2004. [MATTERN88] Mattern, Friedemann. 1988. "Virtual Time and Global States of Distributed Systems." http://courses.csail.mit.edu/6.852/01/papers/VirtTime_Glob-State.pdf . 158. [MCKENNEY05a] McKenney, Paul E. 2005. "Memory Ordering in Modern Microprocessors, Part I." Linux Journal no. 136 (August): 2. 159. [MCKENNEY05b] McKenney, Paul E. 2005. "Memory Ordering in Modern Microprocessors, Part II." Linux Journal no. 137 (September): 5. 160. [MEHTA17] Mehta, Apurva, and Jason Gustafson. 2017. "Transactions in Apache Kafka." Confluent (blog). November 17, 2017. <https://www.confluent.io/blog/transactions-apache-kafka> . 161. [MELLORCRUMMEY91] Mellor-Crummey, John M., and Michael L. Scott.
2005. "Algorithms for scalable synchronization on shared-memory multiprocessors." ACM Transactions on Computer Systems 9, no. 1 (February): 21-65. <https://doi.org/10.1145/103727.103729> . 162. [MELTON06] Melton, Jim. 2006. "Database Language SQL." In International Organization for Standardization (ISO) , 105-132. Berlin: Springer. <https://doi.org/10.1007/b137905> . 163. [MERKLE87] Merkle, Ralph C. 1987. "A Digital Signature Based on a Conventional Encryption Function." A Conference on the Theory and Applications of Cryptographic Techniques on Advances in Cryptology (CRYPTO '87) , edited by Carl Pomerance. London: Springer-Verlag, 369-378. <https://dl.acm.org/citation.cfm?id=704751> . 164. [MILLER78] Miller, R., and L. Snyder. 1978. "Multiple access to B-trees." Proceedings of the Conference on Information Sciences and Systems , Baltimore: Johns Hopkins University (March). 165. [MILOSEVIC11] Milosevic, Z., M. Hutle, and A. Schiper. 2011. "On the Reduction of Atomic Broadcast to Consensus with Byzantine Faults." In Proceedings of the 2011 IEEE 30th International Symposium on Reliable Distributed Systems (SRDS '11) , 235-244. IEEE. <https://doi.org/10.1109/SRDS.2011.36> . 166. [MOHAN92] Mohan, C., Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. "ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging." Transactions on Database Systems 17, no. 1 (March): 94-162. <https://doi.org/10.1145/128765.128770> . 167. [MORARU11] Moraru, Iulian, David G. Andersen, and Michael Kaminsky. 2013. "A Proof of Correctness for Egalitarian Paxos." <https://www.pdl.cmu.edu/PDL-FTP/associated/CMU-PDL-13-111.pdf> . 168. [MORARU13] Moraru, Iulian, David G. Andersen, and Michael Kaminsky. 2013. "There Is More Consensus

- in Egalitarian Parliaments.” In Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP ’13) , 358-372. <https://doi.org/10.1145/2517349.2517350> . 169. [MURSHED12] Murshed, Md. Golam, and Alastair R. Allen. 2012. “Enhanced Bully Algorithm for Leader Node Election in Synchronous Distributed Systems.” *Computers* 1, no. 1: 3-23. <https://doi.org/10.3390/computers1010003> . 170. [NICHOLS66] Nichols, Ann Eljenholm. 1966. “The Past Participle of ‘Overflow:’ ‘Overflowed’ or ‘Overflown.’” *American Speech* 41, no. 1 (February): 52–55. <https://doi.org/10.2307/453244> . 171. [NIEVERGELT74] Nievergelt, J. 1974. “Binary search trees and file organization.” In Proceedings of 1972 ACM-SIGFIDET workshop on Data description, access and control (SIGFIDET ’72) , 165-187. <https://doi.org/10.1145/800295.811490> . 172. [NORVIG01] Norvig, Peter. 2001. “Teach Yourself Programming in Ten Years.” <https://norvig.com/21-days.html> . 173. [ONEIL93] O’Neil, Elizabeth J., Patrick E. O’Neil, and Gerhard Weikum. 1993. “The LRU-K page replacement algorithm for database disk buffering.” In Proceedings of the 1993 ACM SIGMOD international conference on Management of data (SIGMOD ’93) , 297-306. <https://doi.org/10.1145/170035.170081> . 174. [ONEIL96] O’Neil, Patrick, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. 1996. “The log-structured merge-tree (LSM-tree).” *Acta Informatica* 33, no. 4: 351-385. <https://doi.org/10.1007/s002360050048> . 175. [ONGARO14] Ongaro, Diego and John Ousterhout. 2014. “In Search of an Understandable Consensus Algorithm.” In Proceedings of the 2014 USENIX conference on USENIX Annual Technical Conference (USENIX ATC’14) , 305-320. USENIX. 176. [OUYANG14] Ouyang, Jian, Shiding Lin, Song Jiang, Zhenyu Hou, Yong Wang, and Yuanzheng Wang. 2014. “SDF: software-defined flash for web-scale internet storage systems.” *ACM SIGARCH Computer Architecture News* 42, no. 1 (February): 471-484. <https://doi.org/10.1145/2654822.2541959> . 177. [PAPADAKIS93] Papadakis, Thomas. 1993. “Skip lists and probabilistic analysis of algorithms.” Doctoral Dissertation, University of Waterloo. <https://cs.uwaterloo.ca/research/tr/1993/28/root2side.pdf> . 178. [PUGH90a] Pugh, William. 1990. “Concurrent Maintenance of Skip Lists.” Technical Report, University of Maryland. <https://drum.lib.umd.edu/handle/1903/542> . 179. [PUGH90b] Pugh, William. 1990. “Skip lists: a probabilistic alternative to balanced trees.” *Communications of the ACM* 33, no. 6 (June): 668-676. <https://doi.org/10.1145/78973.78977> . 180. [RAMAKRISHNAN03] Ramakrishnan, Raghu, and Johannes Gehrke. 2002. *Database Management Systems* (3rd Ed.) . New York: McGraw-Hill.
2006. [RAY95] Ray, Gautam, Jayant Haritsa, and S. Seshadri. 1995. “Database Compression: A Performance Enhancement Tool.” In Proceedings of 7th International Conference on Management of Data (COMAD) . New York: McGraw Hill. 182. [RAYNAL99] Raynal, M., and F. Tronel. 1999. “Group membership failure detection: a simple protocol and its probabilistic analysis.” *Distributed Systems Engineering* 6, no. 3 (September): 95-102. <https://doi.org/10.1088/0967-1846/6/3/301> . 183. [REED78] Reed, D. P. 1978. “Naming and synchronization in a decentralized computer system.” Technical Report, MIT. <https://dspace.mit.edu/handle/1721.1/16279> . 184. [REN16] Ren, Kun, Jose M. Faleiro, and Daniel J. Abadi. 2016. “Design Principles for Scaling Multi-core OLTP Under High Contention.” In Proceedings of the 2016 International Conference on Management of Data (SIGMOD ’16) , 1583-1598. <https://doi.org/10.1145/2882903.2882908> . 185. [ROBINSON08] Robinson, Henry. 2008. “Consensus Protocols: Two-Phase Commit.” The Paper Trail (blog). November 27, 2008. <https://www.the-paper-trail.org/post/2008-11-27-consensus-protocols-two-phase-commit> . 186. [ROSENBLUM92] Rosenblum, Mendel, and John K. Ousterhout. 1992. “The Design and Implementation of a Log Structured File System.” *ACM Transactions on Computer Systems* 10, no. 1 (February): 26-52. <https://doi.org/10.1145/146941.146943> . 187. [ROY12] Roy, Arjun G., Mohammad K. Hossain, Arijit Chatterjee, and William Perrizo. 2012. “Column-oriented Database Systems: A Comparison Study.” In Proceedings of the ISCA 27th International Conference on Computers and Their Applications , 264-269. 188. [RUSSEL12] Russell, Sears. 2012. “A concurrent skiplist with hazard pointers.” <http://rsea.rs/skiplist> . 189. [RYSTSOV16] Rystsov, Denis. 2016. “Best of both worlds: Raft’s joint

- consensus + Single Decree Paxos.” Rystsov.info (blog). January 5, 2016. <http://rystsov.info/2016/01/05/raft-paxos.html> . 190. [RYSTSOV18] Rystsov, Denis. 2018. “Replicated State Machines without logs.” <https://arxiv.org/abs/1802.07000> . 191. [SATZGER07] Satzger, Benjamin, Andreas Pietzowski, Wolfgang Trumler, and Theo Ungerer. 2007. “A new adaptive accrual failure detector for dependable distributed systems.” In Proceedings of the 2007 ACM symposium on Applied computing (SAC '07) , 551-555. <https://doi.org/10.1145/1244002.1244129> . 192. [SAVARD05] Savard, John. 2005. “Floating-Point Formats.” <http://www.quadibloc.com/comp/cp0201.htm> .
2007. [SCHWARZ86] Schwarz, P., W. Chang, J. C. Freytag, G. Lohman, J. McPherson, C. Mohan, and H. Pirahesh. 1986. “Extensibility in the Starburst database system.” In OODS '86 Proceedings on the 1986 international workshop on Object-oriented database systems , 85–92. IEEE. 194. [SEGEWICK11] Sedgewick, Robert, and Kevin Wayne. 2011. Algorithms (4th Ed.) . Boston: Pearson. 195. [SHAPIRO11a] Shapiro, Marc, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. “Conflict-free Replicated Data Types.” In Stabilization, Safety, and Security of Distributed Systems , 386-400. Berlin: Springer, Berlin, Heidelberg. 196. [SHAPIRO11b] Shapiro, Marc, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. “A comprehensive study of Convergent and Commutative Replicated Data Types.” <https://hal.inria.fr/inria-00555588/document> . 197. [SHEEHY10a] Sheehy, Justin. 2010. “Why Vector Clocks Are Hard.” Riak (blog). April 5, 2010. <https://riak.com/posts/technical/why-vector-clocks-are-hard> . 198. [SHEEHY10b] Sheehy, Justin, and David Smith. 2010. “Bitcask, A Log-Structured Hash Table for Fast Key/Value Data.” 199. [SILBERSCHATZ10] Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. 2010. Database Systems Concepts (6th Ed.) . New York: McGraw-Hill. 200. [SINHA97] Sinha, Pradeep K. 1997. Distributed Operating Systems: Concepts and Design . Hoboken, NJ: Wiley. 201. [SKEEN82] Skeen, Dale. 1982. “A Quorum-Based Commit Protocol.” Technical Report, Cornell University. 202. [SKEEN83] Skeen, Dale, and M. Stonebraker. 1983. “A Formal Model of Crash Recovery in a Distributed System.” IEEE Transactions on Software Engineering 9, no. 3 (May): 219-228. <https://doi.org/10.1109/TSE.1983.236608> . 203. [SOUNDARARAJAN06] Soundararajan, Gokul. 2006. “Implementing a Better Cache Replacement Algorithm in Apache Derby Progress Report.” <https://pdfs.semanticscholar.org/220b/2fe62f13478f1ec75cf17ad085874689c604.pdf> . 204. [STONE98] Stone, J., M. Greenwald, C. Partridge and J. Hughes. 1998. “Performance of checksums and CRCs over real data.” IEEE/ACM Transactions on Networking 6, no. 5 (October): 529-543. <https://doi.org/10.1109/90.731187> . 205. [TANENBAUM14] Tanenbaum, Andrew S., and Herbert Bos. 2014. Modern Operating Systems (4th Ed.) . Upper Saddle River: Prentice Hall Press. 206. [TANENBAUM06] Tanenbaum, Andrew S., and Maarten van Steen. 2006. Distributed Systems: Principles and Paradigms . Boston: Pearson. 207. [TARIQ11] Tariq, Ovais. 2011. “Understanding InnoDB clustered indexes.” Ovais Tariq (blog). January 20, 2011. <http://www.ovaistariq.net/521/understanding-innodb-clustered-indexes/#.XTtaUpNKj5Y> .
2008. [TERRY94] Terry, Douglas B., Alan J. Demers, Karin Petersen, Mike J. Spreitzer, Marvin M. Theimer, and Brent B. Welch. 1994. “Session Guarantees for Weakly Consistent Replicated Data.” In PDIS '94 Proceedings of the Third International Conference on Parallel and Distributed Information Systems , 140–149. IEEE. 209. [THOMAS79] Thomas, Robert H. 1979. “A majority consensus approach to concurrency control for multiple copy databases.” ACM Transactions on Database Systems 4, no. 2 (June): 180–209. <https://doi.org/10.1145/320071.320076> . 210. [THOMSON12] Thomson, Alexander, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. 2012. “Calvin: Fast distributed transactions for partitioned database systems.” In Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD '12) . New York: Association for Computing Machinery. <https://doi.org/10.1145/2213836.2213838> . 211. [VANRENESSE98] van Renesse, Robbert, Yaron Minsky, and Mark Hayden.

2009. “A Gossip-Style Failure Detection Service.” In *Middleware ’98 Proceedings of the IFIP International Conference on Distributed Systems Platforms and Open Distributed Processing* , 55–70. London: Springer-Verlag. 212. [VENKATARAMAN11] Venkataraman, Shivaram, Niraj Tolia, Parthasarathy Ranganathan, and Roy H. Campbell. 2011. “Consistent and Durable Data Structures for Non-Volatile Byte-Addressable Memory.” In *Proceedings of the 9th USENIX conference on File and storage technologies (FAST’11)* , 5. USENIX. 213. [VINOSKI08] Vinoski, Steve. 2008. “Convenience Over Correctness.” *IEEE Internet Computing* 12, no. 4 (August): 89–92. <https://doi.org/10.1109/MIC.2008.75> . 214. [VIOTTI16] Viotti, Paolo, and Marko Vukolić. 2016. “Consistency in Non- Transactional Distributed Storage Systems.” *ACM Computing Surveys* 49, no. 1 (July): Article 19. <https://doi.org/10.1145/2926965> . 215. [VOGELS09] Vogels, Werner. 2009. “Eventually consistent.” *Communications of the ACM* 52, no. 1 (January): 40–44. <https://doi.org/10.1145/1435417.1435432> . 216. [WALDO96] Waldo, Jim, Geoff Wyant, Ann Wollrath, and Samuel C. Kendall.
2010. “A Note on Distributed Computing.” *Selected Presentations and Invited Papers Second International Workshop on Mobile Object Systems—Towards the Programmable Internet* (July): 49–64. <https://dl.acm.org/citation.cfm?id=747342> . 217. [WANG13] Wang, Peng, Guangyu Sun, Song Jiang, Jian Ouyang, Shiding Lin, Chen Zhang, and Jason Cong. 2014. “An Efficient Design and Implementation of LSM-tree based Key-Value Store on Open-Channel SSD.” *EuroSys ’14 Proceedings of the Ninth European Conference on Computer Systems* (April): Article 16. <https://doi.org/10.1145/2592798.2592804> . 218. [WANG18] Wang, Ziqi, Andrew Pavlo, Hyeontaek Lim, Viktor Leis, Huanchen Zhang, Michael Kaminsky, and David G. Andersen. 2018. “Building a Bw-Tree Takes More Than Just Buzz Words.” *Proceedings of the 2018 International Conference on Management of Data (SIGMOD ’18)* , 473–488. <https://doi.org/10.1145/3183713.3196895> . 219. [WEIKUM01] Weikum, Gerhard, and Gottfried Vossen. 2001. *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery* . San Francisco: Morgan Kaufmann Publishers Inc. 220. [XIA17] Xia, Fei, Dejun Jiang, Jin Xiong, and Ninghui Sun. 2017. “HiKV: A Hybrid Index Key-Value Store for DRAM-NVM Memory Systems.” *Proceedings of the 2017 USENIX Annual Technical Conference (USENIX ATC ’17)* , 349–362. USENIX. 221. [YANG14] Yang, Jingpei, Ned Plasson, Greg Gillis, Nisha Talagala, and Swaminathan Sundararaman. 2014. “Don’t stack your Log on my Log.” *INFLOW* (October). <https://www.usenix.org/system/files/conference/inflow14/inflow14-yang.pdf> . 222. [ZHAO15] Zhao, Wenbing. 2015. “Fast Paxos Made Easy: Theory and Implementation.” *International Journal of Distributed Systems and Technologies* 6, no. 1 (January): 15–33. <https://doi.org/10.4018/ijdst.2015010102> .